Stefan Hoffmann

The Set of Synchronizing Words and the Constrained Synchronization Problem

August 13, 2026

Springer Nature

Abstract

Have you ever wanted to start all over again? Has your smartphone or computer ever frozen and you had to restart it? If so, then you are familiar with the idea of resetting things, the central concept of the present work. More specifically, it is about resetting (also called synchronizing) finite state transition systems, also called finite automata. Finite automata process input sequences (also called words) and can be used to model a wide variety of systems, for example, switching circuits, networks, or simple behavior patterns in artificial intelligence. In all these situations, knowing whether we can reset the system is of interest. A circuit component might be out of sync with its environment, can we reset it? A robot might be stuck, or we are simply unsure about its current state, so can we reset its state of operation? However, while this problem is tractable for completely specified and deterministic automata, only a small deviation from this "optimal" situation, i.e., having only partial information, allowing nondeterminism, or constraining the admissible input sequences, results in intractable algorithmic problems.

Here, we are concerned with one particular problem variant: constraining the admissible input sequences (the *constrained synchronization problem*). In this setting, what computational complexity can we deduce from the kind of constraint put on the input sequences and/or types of input automata? We state various results on how to manipulate sets of admissible synchronizing input sequences so as to retain a certain complexity of the problem. We also identify various conditions that either yield tractable or intractable problems. More specifically, the following complexity classes will arise: AC^0, L, NL, P, NP, PSPACE. Except for AC^0 and P, we give sets of admissible synchronizing sequences yielding constrained problems that are complete for these classes. Moreover, we investigate the following classes of admissible sets of synchronizing sequences: finite, infinite, sparse, letter-bounded, commutative, or constraints specified by certain small-state automata; where we give a complete complexity classifications for the latter three classes. Lastly, we consider the following types of input automata to the problem: monotonic, partially ordered, commutative, solvable, SI- and ρ–automata. We obtain, for example, that for commutative input automata, the problem is always in P.

Now, the set of synchronizing sequences, i.e., our search space, of a given input automaton is acceptable by a finite automaton as well. This leads naturally to the investigation of accepting automata for sets of synchronizing sequences, specifically for

the question of when they become largest possible. Here, we identify and investigate two conditions that control the size of a minimal automaton for the set of synchronizing sequences: reachability and distinguishability. For both notions, we establish various structural results and connections with concepts from the literature such as, for example, completely reachable automata. Decision problems and algorithmic questions that naturally arise from these notions are also investigated. We state conditions yielding extremal minimal automata accepting sets of synchronizing sequences. Moreover, we identify a connection to permutation group theory, which is linked to automata theory by the group of units of transformation monoids of automata. Specifically, via graph-theoretical constructions, we obtain new characterizations of primitive permutation groups based on associated synchronizing automata. Our results link permutation groups and vertex-transitive graphs to the theory of synchronizing automata. Furthermore, we explore relations of the size of automata accepting sets of synchronizing sequences to the length of shortest synchronizing sequences.

Zusammenfassung

Wollten Sie schon einmal wieder von vorne anfangen? Hat sich Ihr Smartphone oder Computer schon einmal aufgehängt und nur ein Neustart schuf Abhilfe? Falls ja, so wissen Sie, was es heißt etwas zurückzusetzen, ein zentrales Konzept der vorliegenden Arbeit. Im Speziellen handelt sie von der Zurücksetzbarkeit (oder synonym Synchronisierbarkeit) endlicher Zustandsübergangssysteme, auch endliche Automaten genannt. Automaten interagieren über Eingabefolgen (auch Wörter genannt), welche Zustandsübergänge bewirken, mit ihrer Umwelt und dienen der Modellierung einer Vielzahl von Systemen wie Schaltkreisen, Betriebszuständen von Maschinen oder einfachen Agenten in der künstlichen Intelligenz. Ein Automat heißt synchronisierbar, sofern es eine Eingabefolge gibt, welche ihn in einen wohldefinierten Zielzustand versetzt, unabhängig von seinem aktuellen Zustand. Die Schwierigkeit auf Synchronisierbarkeit zu testen hängt von verschiedenen Parametern ab, zum Beispiel ob wir das System vollständig kennen, welches Verhalten beobachtbar ist, ob Nichtdeterminismus erlaubt ist oder ob bestimmte synchronisierende Folgen ausgeschlossen sind.

In dieser Arbeit beschäftigen wir uns mit der Einschränkung möglicher synchronisierender Eingabefolgen, das sog. *constrained synchronization problem* (dt. *Problem der eingeschränkten Synchronisierbarkeit*). Es stellt sich nun in natürlicher Weise die Frage, wie die Art der Einschränkung die algorithmische Komplexität beeinflusst. Kann man anhand einer gegebenen Einschränkung die resultierende Problemstellung in schwierig oder praktikabel lösbar einteilen? Wir untersuchen komplexitätserhaltende Operationen auf Mengen von erlaubten synchronisierenden Eingabefolgen. Weiterhin arbeiten wir verschiedene hinreichende Bedingungen für praktikabel lösbare oder schwierige Probleme heraus. Im Rahmen unserer Untersuchungen tauchen folgende Komplexitätsklassen auf: AC^0, L, NL, P, NP, PSPACE, wobei wir Bedingungen für Mitgliedschaft in einer dieser Klassen oder Schwierigkeit bezüglich NP oder PSPACE angeben sowie, außer für AC^0 und P, eine Vielzahl von vollständigen Problemen für die jeweiligen Komplexitätsklassen. Die folgenden Arten von Einschränkungen für Mengen synchronisierender Eingabefolgen untersuchen wir: endlich, unendlich, spärlich, buchstaben-beschränkt, kommutativ oder durch kleine Automaten beschrieben. Für kommutative, buchstabenbeschränkte und durch bestimmte kleine Automaton beschriebene Einschränkungen geben wir eine komplette Klassifikation der algorithmischen Komplexität des zugehö-

rigen eingeschränkten Synchronisierbarkeitsproblems. Darüber hinaus untersuchen wir die folgenden Klassen von Eingabeautomaten: monotone, partiell geordnete, kommutative, auflösbare SI- und ρ-Automaten. Unsere Ergebnisse zeigen, dass die resultierenden algorithmischen Komplexitäten stark vom betrachteten Eingabetyp abhängen. Beispielsweise ist das Problem für kommutative Eingabeautomaten stets in Polynomialzeit lösbar.

Die Menge an Eingabefolgen, die einen Automaten synchronisieren, das heißt unser Suchraum, ist durch einen weiteren Automaton akzeptierbar, was die Untersuchung von akzeptierenden Automaton für jene Mengen nahelegt. Von besonderem Interesse in Zusammenhang mit unseren Fragestellungen ist der Fall, wenn ein akzeptierender Automat zur Beschreibung synchronisierender Eingabefolgen maximal groß ist. Wie wir hier herausarbeiten, wird die Größe solcher Automaten durch zwei Faktoren beeinflusst: Erreichbarkeit und Unterscheidbarkeit. Wir untersuchen beide Begriffe, stellen Zusammenhänge her und setzen diese in Beziehung zu etablierten Konzepten aus der Literatur, wie zum Beispiel zu vollständig erreichbare Automaten (engl. completely reachable automata). Verschiedene Bedingungen für die Maximalität von Automaten für synchronisierende Eingabefolden werden herausgearbeitet. Weiterhin formulieren und untersuchen wir Entscheidungsprobleme und algorithmische Fragestellungen, welche sich aus diesen Begriffen ergeben. Über ihren Transformationsmonoid haben Automaten einen natürlichen Bezug zur Algebra. Auf dieser Grundlage werden wir Verbindungen zur Theorie der Permutationsgruppen herstellen. Im Speziellen arbeiten wir, unter Zuhilfenahme graphentheoretischer Konstruktionen, verschiedene neuartige Charakterisierungen für primitive Permutationsgruppen heraus. Unsere Ergebnisse verbinden Permutationsgruppen und ecken-transitive Graphen (engl. vertex-transitive graphs) mit der Theorie synchronisierbarer Automaten. Abschließend untersuchen wir, inwiefern die Größe eines akzeptierenden Automaten für die synchronisierenden Eingabefolgen eines gegebenen Automaten mit der Länge einer kürzesten synchronisierenden Eingabefolge in Beziehung steht.

Acknowledgement

First, I thank my supervisor Prof. Dr. Henning Fernau for giving me the freedom and opportunity to pursue my research and the topics that interest me and also for encouragement to develop (and write down) my ideas! His open-mindedness, broad knowledge, diplomatic attitude, and willingness to engage into different topics created just the right (research) environment. I learned a lot from him, both technically and humanly. Furthermore, I express my gratitude to him for having infinite patience with me; in particular with respect to this thesis which took way more time than intended. In this regard, I also have to thank Prof. Dr. Müller-Hannemann from the Martin-Luther University Halle-Wittenberg for his patience and understanding with my delays; even if things did not worked out as planned, for which I am deeply sorry.

I thank Prof. Dr. Mikhail Volkov (Ural Federal University) for introducing our working group to synchronizing automata at a common workshop in 2019 and suggesting the topics that form the basis of the present thesis. I also thank him for agreeing to be my co-supervisor, despite this was not possible in the end due to political reasons beyond our control.

I thank Prof. Igor Rystsov for agreeing to be my co-supervisor on short notice. Though I never met him in person, his research and ideas, as is certainly noticeable, influenced the content of the present work.

Furthermore, I am grateful to all the friends that made my time in Trier so enjoyable, in particular Richard Spang, Anna Andres, and Ray Kandar for the parties and moments we shared! Thanks go to the Gladiators Trier Soccer Team of the Institute of Archeology for letting me join their training despite not being an archaeologist (special thanks to Lukas Märker here!). Without the regular physical activity, my intellectual activity would certainly not be possible.

I also thank the university choir for letting me participate, and the concerts and rehearsals I enjoyed. In particular, I thank our bass leader Steffen Kramer, for many fun moments and also, together with Malte Busch, many nice movie nights and subsequent discussions. I am also thankful for all the other moments I shared with Malte traveling or spending time at a bar, or the wonderful people I meet due to him, as, for instance, Jana Thielen.

And I thank my flatmates: Andre Janipour, Florian Höller, Chemseddine Amrani, and Asek Ako-Agbor Ashu for getting along so well and a relaxed and open atmosphere, despite me not being very social for the last years due to my graduation efforts. In particular, I thank Florian Höller for sharing his ”good mood 90’s” music playlist with me that played (and put me in a good mood) during the last weeks of writing up the present thesis.

Furthermore, I thank Henning Fernau, Richard Spang, Malte Busch, Anna Andres, and (for a short glance before falling asleep on a plane) Giuseppe Romana for reading through a preliminary version of the present work and giving valuable feedback.

Lastly, I thank my parents for raising and supporting me, and making it possible for me to study and graduate.

Stefan Hoffmann, 2025

Notation Index

Standard logical and set-theoretical notation that is briefly mentioned (and fixed) at the beginnings of Sections 1.1 and 3.1 and common expressions, as, for example, $\binom{n}{2} = \frac{n(n-1)}{2}$, u^n or t^n for the n-fold concatenation product (resp. composition) of a word u or transformation t, etc., do not appear in the notation index. We assume a sufficient level of familiarity with such notions on the part of the reader to not feel it necessary to include them in this index. Furthermore, notations that are only mentioned in a single paragraph (for example, in a discussion section) and are not used beyond that are also not included.

$f\colon X \to Y$	(Total) function f from X to Y	3
Σ	A finite alphabet, i.e., set of symbols	3
Σ^*	Set of words over Σ with concatenation	3
$u \cdot v$ or uv	Concatenation of two words u, v	3
$\lvert u \rvert$	Length of word u	3
ε	The empty word	3
$\mathbb{N}_0$	$\{0, 1, 2, 3, \ldots\}$	4
$\mathbb{Z}$	Set of integers	4
$\mathbb{R}$	Set of real numbers	4
$[n]$	The set $\{0, 1, \ldots, n-1\}$	4
$\max(M)/\min(M)$	Maximum/minimum of finite $\emptyset \neq M \subseteq \mathbb{N}_0$	4
$\max(a_1, \ldots, a_n)$	Maximum of $a_1, \ldots, a_n \in N_0$	4
$\min(a_1, \ldots, a_n)$	Minimum of $a_1, \ldots, a_n \in N_0$	4
$\mathcal{A} = (Q, \Sigma, \delta)$	Automaton specification of $\mathcal{A}$	4
$\delta(S, u)$	Image of $S \subseteq Q$ in $\mathcal{A} = (Q, \Sigma, \delta)$	4
$\mathrm{Syn}(\mathcal{A})$	Set of synchronizing words of $\mathcal{A}$	4
$\mathrm{rt}(\mathcal{A})$	Reset length of $\mathcal{A}$	10
$C_n = ([n], \{a, b\}, \delta_n)$	Automata from the Černý family	10
$[p, q]$	Interval between $p, q \in Q$ in C_n	11
$\mathcal{P}(X)$	Power set of set X	27
$(f(x))_{x \in A}$	Notation to specify function $f\colon X \to Y$	28
$\pi_i\colon X_1 \times \cdots \times X_n \to X_i$	Component projection mapping	28
$a \bmod b$	Unique $0 \leq r < b$ with $a = bn + r$ for $r, n \in \mathbb{N}_0$	28

$a \equiv a' \pmod b$ or $a \equiv_b a'$	a and a' are congruent modulo b	28
$\gcd(M), \operatorname{lcm}(M)$	Greatest com. div. / least common multiple	28
$\lfloor x \rfloor$	$\lfloor x \rfloor = \min\{k \in \mathbb{Z} \mid k \leq x\}$	28
Y^X	Set of functions $f\colon X \to Y$	30
$f(S)$	Extension of $f\colon X \to Y$ to subsets $S \subseteq X$	30
$f^{-1}(S)$, $f^{-1}(x)$	Inverse image of S (resp. x) for $f\colon X \to Y$	30
$\operatorname{dupl}(f)$, $\operatorname{dupl}(u)$	Duplicate states of $f\colon X \to Y$ / word $u \in \Sigma^*$	30, 132
$\operatorname{excl}(f)$, $\operatorname{excl}(u)$	Excluded states of $f\colon X \to Y$ / word $u \in \Sigma^*$	30, 132
$g \circ f$ or gf	Function composition	31
$\operatorname{rank}(f)$, $\operatorname{rank}(u)$	Rank of $f\colon X \to Y$ / word $u \in \Sigma^*$	36, 132
$\operatorname{def}(f)$, $\operatorname{def}(u)$	Defect of $f\colon X \to Y$ / word $u \in \Sigma^*$	36, 132
$\operatorname{exc}_f(A)$	Excess of A under f	36
$(k_1, \ldots, k_l)$	Kernel-type notation	36
$\operatorname{fix}(t), \operatorname{move}(t)$	Set of fixpoints / moved points	38
$\mathrm{id}_X\colon X \to X$	Identity transformation on X	38
$(a_0\ a_1\ \cdots\ a_{m-1})_X, ()$	Cycle notation	93
$[a_1 \mapsto \cdots \mapsto a_m]_X$	Tail transformation notation	94
$[x]_\sim$	Equivalence class of x w.r.t. $\sim$	39
$X/\!\sim$	Set of equivalence classes w.r.t. $\sim$	39
$\pi_\sim\colon X \to X/\!\sim$	Canonical projection map	39
$\sim_f$	Kernel rel. of f (equiv. rel. induced by f)	39
$\uparrow T$	Smallest upper set containing T	46
$+_n, \cdot_n$	Cropped operations on $[n]$	41
AB	Product on subsets $A, B \subseteq S$, S semigroup	41
A^n, $A^{\leq n}$, $A^{<n}$, aA, Aa	Operations derived from products on subsets	42
$\leq_S, \leq_M, \leq_G$	Subsemigroup, submonoid, subgroup	41
$Z(G)$	Center of a group G	43
$\langle T \rangle_S$, $\langle T \rangle_M$, $\langle T \rangle_G$	Semigroup, monoid, group generated by T	43
$A = (a_{ij})_{i \in \{1,\ldots,n\}, j \in \{1,\ldots,m\}}$	$n \times m$ matrix	43
I_n, O_n	$n \times n$ identity and zero matrices, respectively.	44
$\mathcal{T}_X$	Full transformation semigroup on X	45
$\mathcal{T}_n$	Full transformation semigroup on $[n]$	45
$\mathcal{S}_X$	Full symmetric group on X	111
$\mathrm{Orb}_G(a)$	Orbit of a in permutation group G	112
G_a	Point stabilizer of a in permutation G	112
t^g	Conjugation of $t\colon X \to X$ by $g\colon X \to X$	112
$\mathcal{S}_n$	Full symmetric group on $[n]$	111
$\mathcal{A}_n$	Alternating group on $[n]$	112
L^*, L^+	Kleene star and plus	56
$\operatorname{Pref}(u), \operatorname{Pref}(L)$	Set of prefixes of word u (resp. $L \subseteq \Sigma^*$)	57
$\operatorname{Suff}(u), \operatorname{Suff}(L)$	Set of suffixes of word u (resp. $L \subseteq \Sigma^*$)	57
$\operatorname{Fact}(u), \operatorname{Fact}(L)$	Set of factors/infixes of word u (resp. $L \subseteq \Sigma^*$)	57
$\lvert u \rvert_a$	Number of occurrences of $a \in \Sigma$ in $u \in \Sigma^*$	57
$u^{-1}L$	Language quotient of L by u	57
$\pi_\Gamma\colon \Sigma^* \to \Gamma^*$	Projection morphism	57

$\mathcal{RE}(\Sigma)$	Set of regular expressions over Σ	58
$L(R)$	Language described by $R \in \mathcal{RE}(\Sigma)$	58
$\psi\colon \Sigma^* \to \mathbb{N}_0^\Sigma$	Parikh map/Parikh morphism	57
$\mathrm{Aut}(G)$	Automorphism group of a graph $G = (V, E)$	59
G_t / G_R	Functional graph / graph of a relation	62
$N^+(U)$	Vertices v with edge (u, v) s.t. $u \in U$, $v \notin U$	127
$O(g)$, $o(g)$	Big O and small o notation for $g\colon \mathbb{N}_0 \to \mathbb{N}_0$	64
(L-uniform) NC^k, AC^k	Circuit families or languages decidable by circuit families in (L-uniform) NC^k/AC^k	66, 71
L	Languages decidable in det. log. space	71
NL	Languages decidable in nondet. log. space	71
NC	Language class $\bigcup_{k\geq 0}$ L-uniform NC^k	71
P	Languages decidable in det. poly. time	71
NP	Languages decidable in nondet. poly. time	71
PSPACE	Languages decidable in det. poly. space	71
NPSPACE	Languages decidable in nondet. poly. space	71
$\leq_m^{\log}$ / $\equiv_m^{\log}$	log-space many-one reduction / equivalent	72
$\leq_m^{\mathsf{P}}$ / $\equiv_m^{\mathsf{P}}$	polynomial time many-one reduction / equivalent	72
$\leq_m^{\mathsf{AC}^0}$ / $\leq_m^{\mathsf{NC}^1}$	L-uniform AC^0 / NC^1 many-one reduction	72
$\equiv_m^{\mathsf{AC}^0}$ / $\equiv_m^{\mathsf{NC}^1}$	L-uniform AC^0 / NC^1 many-one equivalent	72
$\leq_T^{\mathsf{NC}^1}$ / $\equiv_T^{\mathsf{NC}^1}$	L-uniform NC^1 oracle reduction / equivalent	72
G^D	Diagonal action on G	123
$\delta^{-1}(q, u)$	Inverse transition of δ	131
$G(\mathcal{A})$	Automaton graph of automaton $\mathcal{A}$	131
$\preceq_R$, $\preceq_R^{\mathcal{A}}$	Reachability quasiorder w.r.t. $\mathcal{A}$	134
$\simeq_R$	Equivalence relation induced by $\preceq_R$	134
δ_u	$\delta_u\colon Q \to Q$ induced by $u \in \Sigma^*$ in $\mathcal{A} = (Q, \Sigma, \delta)$	132
$\mathcal{T}_S(\mathcal{A})$, $\mathcal{T}_M(\mathcal{A})$	Transformation semigroup/monoid of $\mathcal{A}$	132
$[a]_{\mathcal{A}}$, $[u]_{\mathcal{A}}$, $[a]$, $[u]$	Transition matrix for letter/word a/u w.r.t. $\mathcal{A}$	132
$\mathcal{A} \cong \mathcal{B}$	$\mathcal{A}$ and $\mathcal{B}$ are isomorphic	137
$\mathcal{A}_{\mid S}$, $\mathcal{A}_{\mid \Gamma}$, $\mathcal{A}_{\mid \Gamma, S}$	Restriction of $\mathcal{A}$ to upper set S, Γ or both.	148
$\mathcal{A}/{\sim} = (Q/{\sim}, \Sigma, \delta_\sim)$	Quotient automaton w.r.t. $\mathcal{T}_M(\mathcal{A})$-congruence $\sim$	149
$\sim_S$	Rees congruence for upper set S	151
$\mathcal{A}_1 \times \cdots \times \mathcal{A}_m$, $\mathcal{A}^n$	Product aut. of $\mathcal{A}_1, \ldots, \mathcal{A}_m$ / n-fold product	154
$\mathcal{P}(\mathcal{A})$	Power automaton of $\mathcal{A}$	154
$\mathcal{A}^{\leq 2}$	Auxiliary construction for pair automaton	157
$\mathcal{A}^{\leq 2}/{\sim_F}$	Pair automaton of $\mathcal{A}$	157
$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$	Automaton as language acceptor	159
$L(\mathcal{A})$	Language accepted by automaton $\mathcal{A}$	159
$\mathcal{A}_{q,S}$	Changing/adding start s & final set F to $\mathcal{A}$	159
$\equiv_L$	Myhill-Nerode relation	161
$\mathrm{MinAut}(L)$	Minimal automaton of $L \subseteq \Sigma^*$	161
$\pi_{\mathcal{A}}$	Morphism onto $\mathcal{A}_L$ from accessible $\mathcal{A}$	161
$\leq_{\mathcal{R}}$	$\mathcal{R}$-quasiorder on words w.r.t. automaton	172
$\sim_s$	Stability relation	183

Σ_I, Σ_G	Subalphabets w.r.t. OI-automaton	186
$\mathcal{B}_{n,m,r}$	Strongly conn. comm. automata	203
$\Gamma_k(\mathcal{A}) = (Q_k, E_k)$	Bondar-Casas-Volkov graph construction	303
$I(\mathcal{A}) = (Q, E)$	The Rystsov graph	309
$I_i(\mathcal{A}) = (Q, E_i)$	Auxiliary graph introduced by Rystsov	309
$I(S) = ([n], E)$	Rystsov graph extended to $S \leq_S \mathcal{T}_n$	310
$\Gamma_1(S) = ([n], E)$	$\Gamma_1(\mathcal{A})$-graph extended to $S \leq_S \mathcal{T}_n$	310
$\Gamma_{1,i}(\mathcal{A})$	Variant of $\Gamma_1(\mathcal{A})$-graph	306
$\mathcal{A}_{f,a}$	Adding letter a to $\mathcal{A}$ defined via $f\colon Q \to Q$	363
$a \rightsquigarrow_T b$ / $a \rightsquigarrow b$	Quasiorder used in Section 14.3.6.	379
$\mathbf{D}, \mathbf{E}, \mathbf{F}, \mathbf{G}, \mathbf{H}, \mathbf{I}, \mathbf{J}, \mathbf{K}$	Exceptional automata for Černý bound	399
$\mathcal{V}_n = \mathcal{W}_n$	Class of slowly synchronizing automata	402
$\mathcal{L}_n = \mathcal{D}'_n$	Class of slowly synchronizing automata	402
$\mathcal{F}_n$, $n > 3$ odd	Class of slowly synchronizing automata	402
$\mathcal{K}_n = \mathcal{G}_n$, $n > 5$ odd	Class of slowly synchronizing automata	402
$\overline{\mathcal{A}_n}$, $n \geq 8$ even	Class of slowly synchronizing automata	403
$\mathcal{M}_n$, $n \geq 3$	Class of slowly synchronizing automata	403
$\mathcal{B}_n$, $n \geq 5$ odd	Class of slowly synchronizing automata	403
$f\colon X \nrightarrow Y$	Partial function f from X to Y	417
$\sigma_L\colon \Sigma^* \to \Sigma^*$	Levenshtein-Romanov mapping	419
$\rho\colon \Sigma^* \to \mathbb{N}_0$	Equal-letter runs	443
$\equiv_L^{\sim,U}$	Relation derived from $\equiv_L$	480
$\equiv_L^{>n}$	Relation defined with $\equiv_L^{\sim,U}$	482
$\sim_{C,V}$	Relation used to define $\equiv_{C,V}$	482
$\equiv_{C,V}$	Relation defined with $\equiv_L^{\sim,U}$	484
$\Sigma_{i,j}$	Subalphabet w.r.t. small constraint automaton	512
$\psi_n\colon \Sigma^* \to \{0, 1, \ldots, n\}^\Sigma$	n-cropped Parikh map	518
L_i^F	Shorthand $L_i^F = L(\mathcal{B}_{i,F})$ for Table 19.1	515
$\mathcal{A}_t, \mathcal{A}_c$	Sync. binary SI-automata, see Prop. 7.31	540

List of Figures

1.1 Example automaton 5
1.2 Encoding of five symbols e, h, l, o, and r by binary sequences. 6
1.3 Introductory decoder automaton 6
1.4 Ashby's ghost taming automaton 8
1.5 A polygonal component and its possible orientations 8
1.6 Part orienter problem modeled as a finite automaton 9
1.7 Černý family C_n 11

3.1 Illustration for the identities from Theorems 3.4 and 3.6 33
3.2 Cases so that $\mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$ 34
3.3 Scheme of an $m \times n$ matrix 44
3.4 Relations among connectivity notions 61
3.5 Examples for connectedness notions 62
3.6 Example functional graph 63

4.1 Composition of cycle and simple idempotent 96
4.2 Functional graph of a one-point singular 97

5.1 Directed orbital graphs of $G = \langle\{(0\ 1\ 2\ 3)\}\rangle_G$ 128
5.2 Directed orbital graphs of $G = \langle\{(0\ 1)(2\ 3), (0\ 2)(1\ 3)\}\rangle_G$ 128

6.1 The transition matrices for the Černý automata C_n 133
6.2 Example morphisms and non-morphisms on automata 138
6.3 Automaton not admitting any non-identity endomorphism. 138
6.4 Some strongly connected component fixed and others swapped 145
6.5 Morphism between automata 148
6.6 Example power automaton 155
6.7 Automaton constructed from transformation semigroup. 166

7.1 Relations among classes of automata 176
7.2 Automata with a trap state attaining the maximal reset length 179
7.3 Unary automaton for lower bound of rank r words 186

7.4 Synchronizing binary SI-automata 188
7.5 Two synchronizing monotonic automata 192
7.6 Partially ordered automaton that is not monotonic 196
7.7 Example ρ-automata 198
7.8 Example quotient automaton of ρ-automaton 200
7.9 Example automata $\mathcal{B}_{n,m,r}$ 203
7.10 Extremal synchronizing commutative automaton 209
7.11 Non-solvable automaton with solvable pair automaton 212
7.12 Solvable automaton with sequence of upper sets 217
7.13 Inclusion relations for solvability notions 221

8.1 Haase diagram for reductions 227
8.2 Reduction of OrderBetweenVertices to SyncWords. 234
8.3 Example for reduction used in the proof of Prop. 8.10 236
8.4 Example for reduction of 3SAT to Intersection on unary automata . . . 241
8.5 Non-locally confluent synchronizing partially ordered automaton 257

9.1 C_4 and its power automaton 262
9.2 Disjoint and non-disjoint 2-sets and distinguishability 263
9.3 Distinguishability in different classes of sets 264
9.4 Tow non-synchronizing automata where all pairs of 2-sets distinguishable 274
9.5 Relations between disjoint and non-disjoint pairs of subsets w.r.t. distinguishability 278
9.6 Completely distinguishable, but not completely reachable 284
9.7 Pairs of 2-sets distinguishable but not completely distinguishable 286
9.8 Non-completely distinguishable but pairs of 2-sets distinguishable 286
9.9 Non-empty sets indistinguishable from singleton set in $\mathcal{T}_S(\mathcal{A})$ 290

10.1 Completely reachable but not sync-maximal automaton 300
10.2 Sync-maximal automata but not completely reachable automata 301
10.3 One property from Theorem 10.7 fails and so not sync-maximal 302

11.1 Automaton $\mathcal{E}_5$ to illustrate graph constructions $\Gamma_k(\mathcal{E}_5)$ 305
11.2 Example for graphs Γ_k. 305
11.3 The graphs $I(C_n)$ and $I(\mathcal{T}_S(C_n))$ 315
11.4 The graphs $I(\mathcal{T}_S(\mathcal{A}))$ and $\Gamma_1(\mathcal{A})$ for a given SI-automaton $\mathcal{A}$ 316
11.5 Pairs of 2-sets distinguishable but not completely distinguishable 319

12.1 Strongly connected and synchronizing automaton that is not completely reachable. 335
12.2 Example sync-maximal circular automata 340
12.3 Example completely reachable circular automaton 342
12.4 Not completely reachable and no non-trivial congruences 345
12.5 Ternary completely reachable automaton with non-trivial congruence . . 346

13.1 Example for reductions used in the proofs of Thm. 13.3 & 13.6 352

14.1 Equivalences for automata with primitive group in transformation semigroup 364
14.2 Unimodal transformation 382
14.3 Sequence of most complex languages 391
14.4 Sequence of most complex languages w.r.t. regular ideal languages 392

15.1 Eight exceptional automata attaining the Černý bound 399
15.2 Minimal automaton for Syn(**I**) 400
15.3 Non-sync-maximal and slowly synchronizing automata families 403
15.4 Binary sync-maximal automaton with short reset length 407

17.1 Relations between different code classes and overlap-free languages . . . 427

18.1 Construction for hardness of constraint language $\{a, b\}^* c^l L$ 458
18.2 Construction from proof of Theorem 18.19. 465
18.3 Schematic of reduction from the proof of Proposition 18.31 470
18.4 Example automaton derived from $\equiv_{C,V}$ 485
18.5 Distributions of final states that give equivalent constrained problems . . 498

20.1 Partially ordered automaton with synchronizing word ab, but $cacb$ is not a synchronizing word, see Theorem 20.11. 532
20.2 Example for the reduction from the proof of Theorem 20.16 535

List of Tables

1.1 Published upper bounds on the reset length 13

4.1 Cases for composition being a one-point singular 100
4.2 Conditions covering cases for composition being one-point singular . . . 101

8.1 Overview complexities on classes of automata 233

9.1 Distinguishing words for pairs of subsets 287

11.1 The transition table of the automaton $\mathcal{E}_5$ 304
11.2 All Defect one words of example automaton 317

13.1 Overview decision problems 349

15.1 Slowly synchronizing and sync-maximal n-state automata 402

17.1 Failure of defining conditions for various language classes 425

19.1 3-state binary constraint automaton complexities, case $\{2,3\}$ 515
19.2 3-state binary constraint PDFA pictures, case {2,3} 526

20.1 Complexities for different classes of input automata 527

Contents

Part I Introductory Part

1 Short Introduction to Synchronizing Automata 3
1.1 Notions Used in the Introduction 3
1.2 Formal Definition of Automata Synchronization 4
1.3 Introductory Examples 5
1.3.1 Error-Prone Communication Channels 5
1.3.2 The Haunted Mansion 7
1.3.3 Part Orienters 8
1.3.4 Lost in a Maze 10
1.4 Reset Length and the Černý Conjecture 10

2 Outline and Contribution 15
2.1 The Problems 15
2.1.1 The Constrained Synchronization Problem 15
2.1.2 Automata for Sets of Synchronizing Words 16
2.2 Organization of the Thesis 17
2.2.1 Structure of Part II: Preliminaries 17
2.2.2 Structure of Part III: Automata for Sets of Synchronizing Words 18
2.2.3 Structure of Part IV: The Constrained Synchronization Problem 19
2.3 Work by the Author Relevant to the Thesis 20
2.3.1 Published Work 20
2.3.2 Unpublished Work 22

Part II Preliminaries

3 General Notions 27
3.1 More on Logic, Set Theory, and Number Theory 27
3.2 More on Functions 28
3.2.1 Basic Notions 30
3.2.2 The Duplicate Set and Function Composition 32
3.2.3 Functions on a Finite Set 36

3.3 Transformations . . . 38
3.4 Relations . . . 39
3.5 Algebraic Notions . . . 40
3.5.1 Semigroups, Monoids, Groups, and Semirings . . . 40
3.5.2 Matrices . . . 43
3.5.3 Transformation Semigroups . . . 45
3.6 Order Theory . . . 45
3.6.1 Definitions and Basic Facts . . . 46
3.6.2 Equivalence Classes and Upper and Lower Sets . . . 49
3.6.3 Maximal and Minimal Elements . . . 50
3.6.4 The Set of Maximal Elements . . . 53
3.6.5 Sequences of Upper and Lower Sets . . . 54
3.7 Formal Language Theory . . . 55
3.7.1 More Basic Notions . . . 55
3.7.2 Regular Expressions . . . 58
3.8 Graph Theory . . . 59
3.8.1 Basic Notions . . . 59
3.8.2 Connectedness Notions . . . 60
3.8.3 Graphs for Transformations and Homogeneous Relations . . . 62
3.9 Computational Complexity Theory . . . 64
3.9.1 The Random Access Machine Model . . . 64
3.9.2 Turing Machines and Transducers . . . 65
3.9.3 Circuits . . . 66
3.9.4 Decision Problems . . . 69
3.9.5 Complexity Classes and Reductions . . . 71
3.9.6 Circuits and Encodings . . . 76
3.10 Remarks on Different Notions of Reduction . . . 80
3.11 Remarks on the Order of Function Composition . . . 81
3.12 Discussion . . . 84

4 Classes of Transformations . . . 91
4.1 Permutations and Quasipermutations . . . 91
4.2 Cycles and Cycles Induced by a Transformation . . . 93
4.3 Tail Transformations, Simple Idempotents, and Cycles . . . 94
4.4 One-Point Singular Transformations . . . 97
4.5 Idempotent Transformations . . . 102
4.6 Idempotent One-Point Singulars . . . 106
4.7 Discussion . . . 110

5 Permutation Groups . . . 111
5.1 Basic Notions . . . 111
5.2 Transitive Permutation Groups . . . 115
5.2.1 Separating Sets and Idempotents . . . 115
5.2.2 Blocks and Point Stabilizers . . . 116
5.2.3 Mapping 2-Sets, Orbit Sizes, and Transitivity . . . 118
5.3 Permutation Groups and Abstract Groups . . . 122

5.4 Permutation Groups and Non-Permutations 124
5.5 Graphs and Permutation Groups 125
5.5.1 Generating Sets for the Full Symmetric Group 125
5.5.2 Vertex-Transitive Graphs 126
5.5.3 Orbital Graphs 127
5.6 Discussion 129

6 Notions From Automata Theory 131
6.1 More Basic Notions 131
6.2 The Transformation Semigroup of an Automaton 132
6.3 The Reachability Order 134
6.3.1 Definition and a Characterization 134
6.3.2 Connectivity and Upper Sets 134
6.4 Morphisms 137
6.4.1 Definitions and Basic Facts 137
6.4.2 Morphisms and the Reachability Order 140
6.4.3 Endomorphisms 143
6.4.4 Cyclic Unary Automata 146
6.5 Operations on Automata 148
6.5.1 Subautomata and Restricted Automata 148
6.5.2 The Quotient Automaton 149
6.5.3 The Rees Quotient Automaton 150
6.5.4 The Product Automaton 154
6.5.5 The Power Automaton 154
6.5.6 The Pair Automaton 157
6.6 Sequences of Upper Sets and Maximal States 157
6.7 Automata as Language Acceptors 159
6.7.1 Regular Languages 159
6.7.2 The Minimal Automaton 160
6.8 Remarks on Transformation Semigroups and Automata 166
6.9 Discussion 167

7 Classes of Automata 175
7.1 Synchronizing Automata 176
7.1.1 Sets of Synchronizing Words and Ideal Languages 176
7.1.2 Subautomata, Morphic Images, and Quotient Automata 177
7.1.3 Synchronizing Automata with a Trap State 178
7.1.4 Synchronizing Automata and the Reachability Order 180
7.1.5 The Pair Criterion 182
7.1.6 Algorithmic Questions 183
7.2 Unary Automata 185
7.3 OI- and SI-Automata 186
7.4 Permutation-Reset Automata 190
7.5 Monotonic Automata 191
7.6 Automata Whose Strongly Connected Components are Blocks 192
7.7 Partially Ordered Automata 196

7.8 ρ-Automata 198
7.9 Commutative Automata 200
7.9.1 Commutativity of Rees Quotients 201
7.9.2 Strongly Connected Automata 202
7.9.3 Subset Mapping Properties 205
7.9.4 Extremal Synchronizing Automata 209
7.10 Solvable Automata 210
7.10.1 Operations 210
7.10.2 Characterizations 213
7.10.3 Synchronization and Subset Mapping Properties 218
7.10.4 Comparison with Original Definition 220
7.11 Discussion 222

8 Subset Mapping Problems 225
8.1 The Problems 225
8.2 Reductions 226
8.3 Classes of Automata 230
8.3.1 Known Results and Overview 231
8.3.2 General Automata 233
8.3.3 Solvable Automata 240
8.3.4 Commutative Automata 244
8.3.5 Partially Ordered Automata 248
8.4 Discussion 249

Part III Automata for Sets of Synchronizing Words

9 Reachable Sets and Pairs of Distinguishable Sets 261
9.1 Power Automata and Sets of Synchronizing Words 261
9.2 Reachable Subsets 265
9.2.1 Completely Reachable Automata 265
9.2.2 Reachability of k-Sets 266
9.2.3 Upper Sets and the Reachability Order 270
9.3 Pairs of Distinguishable Subsets 271
9.3.1 Distinguishability and Synchronizability 272
9.3.2 Classes of Pairwise Distinguishable Subsets 275
9.3.3 The Set of Words Distinguishing Two Sets and a Length Bound 278
9.3.4 Completely (Subset-Pair) Distinguishable Automata 279
9.3.5 Further Examples 285
9.4 From Automata to Transformation Semigroups 288
9.4.1 Transferring Notions to Transformation Semigroups 288
9.4.2 Differences Between Monoids and Semigroups 289
9.5 Discussion 291

10 Sync-Maximal Automata 293
10.1 Definition and First Properties 293
10.2 The Structure of Sync-Maximal Automata 295
10.3 Examples 299
10.4 Discussion 301

11 Graph-Theoretical Constructions 303
11.1 The Bondar-Casas-Volkov Graphs 303
11.2 Augmentative Sets of Words 307
11.3 The Rystsov Graph 309
11.4 From Automata to Transformation Semigroups 310
11.5 Relations Between the Different Graphs 310
11.6 Relations to the Group of Units 312
11.7 Examples 315
11.8 Deducing Reachability and Distinguishability 316
11.9 A Bound on the Reset Length for OI-Automata 320
11.10Discussion 323

12 Classes of Automata 325
12.1 Automata Whose Strongly Connected Components are Blocks 325
12.2 Commutative Automata 326
12.3 All Letters Have Defect at Most One 329
12.4 Automata with Simple Idempotents 331
12.4.1 Distinguishability of 2-Sets 332
12.4.2 Strongly Connected SI-Automata 333
12.5 Circular Automata 338
12.6 Binary Automata 341
12.7 Discussion 347

13 Decision Problems 349
13.1 Distinguishability 350
13.2 Complete Reachability and Sync-Maximality 351
13.3 Binary SI-Automata 353
13.4 Discussion 355

14 Permutation Groups 357
14.1 Primitive Permutation Groups 357
14.1.1 Known Characterizations 357
14.1.2 Minimal Automata 360
14.1.3 Automata for the Set of Synchronizing Words 361
14.2 Synchronizing and k-Homogeneous Groups 364
14.3 A-Primitive Permutation Groups 367
14.3.1 Introduction and First Properties 368
14.3.2 Transitivity and Strong $\{a, b\}$-Primitivity 371
14.3.3 Distinguishability, Orbit Sizes, and $\{a, b\}$-Primitivity 373
14.3.4 Orbital Graphs and $\{a, b\}$-Primitivity 376

14.3.5 Generating Sets for the Symmetric Group 378
14.3.6 Synchronization 379
14.3.7 Reachability and Distinguishability 384
14.3.8 2-Homogeneity and Transformation Semigroups 386
14.4 Congruences on Transformation Semigroups 388
14.5 Discussion 390

15 Slowly Synchronizing Automata 397
15.1 Automata Reaching the Černý Bound 397
15.2 Families of Slowly Synchronizing Automata 401
15.3 Discussion 406

Part IV The Constrained Synchronization Problem

16 Motivational Examples 413
16.1 Use in a Reduction Procedure 413
16.2 Implicit in a Reduction 414
16.3 The Matrix Orbit Problem and the Skolem Problem 414
16.4 Operating Restrictions 415
16.5 Part Orienters 416
16.6 Supervisory Control 416

17 Further Notions and Preliminaries 417
17.1 Partial Functions 417
17.2 Partial Automata (PDFAs) 418
17.3 The Levenshtein-Romanov Mapping 419
17.4 Codes and Overlap-Free Languages 423
17.4.1 Basic Notions 424
17.4.2 More on Prefix Codes 429
17.4.3 Relations to the Levenshtein-Romanov Mapping 432
17.4.4 Characterizing Solid Codes 438
17.5 Bounded Languages 441
17.6 Monoid Languages 445
17.7 The Growth of Languages 447
17.8 Assumptions on Constraint Languages and Constraint PDFAs 450
17.9 Discussion 451

18 General Results 455
18.1 Basic Complexity Results and Examples 455
18.2 Manipulating Constraint Languages 459
18.2.1 Factor Sets and Constraint Languages 459
18.2.2 The Union of Constraint Languages 460
18.2.3 The Ideal Intersection Theorem 461
18.2.4 Applying Morphisms and Solid Codes 464
18.2.5 Concatenation and Complexity Lower Bounds 467
18.3 Hardness Conditions 472

18.3.1 Auxiliary State Avoiding Words 472
18.3.2 Auxiliary State Avoiding Words and Prefix Codes 476
18.3.3 Maximal Prefix Codes 478
18.3.4 Relations Derived From the Myhill-Nerode Relation 480
18.3.5 An NL-Hardness Condition 485
18.3.6 Two PSPACE-Hardness Conditions 486
18.3.7 Two NP-Hardness Conditions 493
18.4 Remarks on Constraint PFDAs 496
18.4.1 Distributing the Start State and the Final States 497
18.4.2 Concatenation and Complexity Lower Bounds 498
18.4.3 Equivalence to Unconstrained Synchronization 499
18.5 Discussion 500

19 Classes of Constraint Languages 503
19.1 Infinite Languages 503
19.2 Sparse Languages 504
19.2.1 Containment in NP 504
19.2.2 Finite Languages 505
19.2.3 Cases Contained in L 506
19.2.4 Trichotomy Result for Letter-Bounded Languages 507
19.3 Monoid Languages 508
19.3.1 A Pair Criterion 508
19.3.2 Computing Synchronizing Words and Containment in NL 509
19.3.3 Strongly Connected Constraint Automata 510
19.4 Small Constraint Automata 512
19.4.1 Two-State Constraint PDFAs 512
19.4.2 Three-State Constraint PDFAs Over a Binary Alphabet 513
19.5 Commutative Languages 517
19.5.1 The Cropped Parikh Map 518
19.5.2 Simplifying the Constraint Language 519
19.5.3 Complexity Classification 519
19.6 Discussion 522

20 Classes of Input Automata 527
20.1 Monotonic Automata 528
20.2 Partially Ordered Automata 529
20.2.1 A Bound and Containment in NP 530
20.2.2 A Reduction Lemma 531
20.2.3 Three Special Cases 532
20.2.4 Classification for Two-State Constraint PDFAs 536
20.3 ρ-Automata With a Trap State 537
20.4 Commutative and Unary Automata 538
20.5 Solvable ρ-Automata 539
20.6 Automata with Simple Idempotents 540
20.6.1 Monoid Constraint Languages 540
20.6.2 Bounded Constraint Languages 542

20.6.3 Binary SI-Automata and Small Constraint Automata 544
20.7 Discussion 547
References 548

Part I
Introductory Part

"That fondness for science has encouraged me to compose a short work on calculating by al-jabr (completion) and al-muqabala (balancing), confining it to what is easiest and most useful in arithmetic."

—Muhammad ibn Musa Al-Khwarizmi (780–850)

Chapter 1
Short Introduction to Synchronizing Automata

This chapter gives a short introduction to synchronizing automata for readers unfamiliar with the topic. Section 1.1 fixes the notation needed here. The notation for the remainder of the thesis is given in Chapter 3. In Section 1.2 we formally introduce synchronizing automata. Then, in Section 1.3, we present several introductory examples and problems and discuss their relation to synchronizing automata. Finally, Section 1.4 defines the reset length, states Černý's conjecture, and presents the Černý automata, which serve as a running example throughout the thesis.

For readers who want to learn more about synchronizing automata, the most recent survey articles are by Kari and Volkov [360] (see also its precursor [613]) and by Volkov [617], the latter containing a good coverage of applications from different fields and types of automata considered in the literature. The surveys by Jürgensen [348], Kari and Volkov [360], and Béal and Perrin [71] discuss connections to codes (see also Chapters 3 and 10 of [87]). Sandberg [548] and Kohavi [375, Chap. 13] include relations to automata with outputs as well as applications in circuit theory and testing. The survey by Bogdanović et al. [97] presents structural results not covered in the other surveys. Another survey is by Mateescu and Salomaa [444]. Finally, we recommend [36] on synchronizing permutation groups, motivated by synchronizing automata, as results presented in that survey were inspirational for our results in Chapter 14 from Part III.

1.1 Notions Used in the Introduction

If X is a set, by $|X|$ we denote its *cardinality* and $x \in X$ denotes *set membership*, i.e., the fact that x is an element in X. We use standard *set builder notation* $\{x \mid P(x)\}$ or $\{x : P(x)\}$ for the set of all elements x satisfying a predicate $P(x)$. The *empty set* is denoted by $\emptyset$. For sets X, Y, we write $f\colon X \to Y$ for a *(total) function* (also called a *map* or *mapping*) with *domain* X and *codomain* Y, assigning to each $x \in X$ an element $f(x) \in Y$.

Let Σ denote a *finite alphabet*, i.e., a set of *symbols* or *letters* (we always use Σ to denote an alphabet in the present work). A *word (over* Σ*)*, also called a *string*, is a finite

sequence of elements from Σ. By Σ^* we denote the *set of all words* over Σ. We identify each symbol in Σ with the corresponding word of length one. In this way, we have $\Sigma \subseteq \Sigma^*$. For $u, v \in \Sigma^*$ we write $u \cdot v$, or uv for short, for the *concatenation (product)* of both words.[1] We denote the *length* of $u \in \Sigma^*$ by $|u|$. The *empty word*, i.e., the word of length zero, is denoted by ε. For $u \in \Sigma^*$, we set $u^0 = \varepsilon$ and $u^n = u \cdots u$ (n times) for $n > 0$. Subsets of Σ^* are called *(formal) languages (over Σ)*.

By $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$ we denote the *set of natural numbers*, including zero, with the usual orders $\leq$ and $<$ on $\mathbb{N}_0$. For $n \in \mathbb{N}_0$ we set $[n] = \{m \in \mathbb{N}_0 \mid m < n\}$. For a non-empty subset M of $\mathbb{N}_0$, by $\min(M)$, or $\min M$ for short, (resp. $\max(M)$, or $\max M$, for finite M) we denote the *smallest* (resp. *largest*) *number* in M w.r.t. the usual order. We also set $\min(a_1, \ldots, a_n) = \min\{a_1, \ldots, a_n\}$ (resp. $\max(a_1, \ldots, a_n) = \max\{a_1, \ldots, a_n\}$) for $a_1, \ldots, a_n \in \mathbb{N}_0$ with $n \geq 1$. We also denote the integers and the real numbers by $\mathbb{Z}$ and $\mathbb{R}$, respectively.

A *(deterministic and complete) finite semiautomaton* over Σ (or just *automaton* for short) is a tuple $\mathcal{A} = (Q, \Sigma, \delta)$ where Q is the *finite state set*, Σ the *finite input alphabet*, and $\delta\colon Q \times \Sigma \to Q$ the *(totally defined) transition function*.[2] We extend δ to a function $\hat{\delta}\colon Q \times \Sigma^* \to Q$ by setting $\hat{\delta}(q, \varepsilon) = q$ and $\hat{\delta}(q, ux) = \delta(\hat{\delta}(q, u), x)$ for $q \in Q$, $u \in \Sigma^*$, and $x \in \Sigma$. For simplicity, we denote the extension $\hat{\delta}$ by δ as well. Given $S \subseteq Q$ and $u \in \Sigma^*$ the *image (of S under u)* is $\delta(S, u) = \{\delta(q, u) \mid q \in S\}$.

Automata are often specified graphically as *state diagrams*, where states are represented as points or circles, and a transition $\delta(p, a) = q$ for $p, q \in Q$ and $a \in \Sigma$ is depicted by an arrow from p to q labeled by a. Multiple transitions between the same pair of states are typically combined into a single arrow labeled with the corresponding set of input symbols. See Theorem 1.1 for an example of an automaton together with its state diagram.

Example 1.1 Consider Figure 1.1 for the state diagram of the automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with state set $Q = \{0, 1, 2, 3\}$, two letters $\Sigma = \{a, b\}$ and where $\delta\colon Q \times \Sigma \to Q$ is given by $\delta(q, b) = q + 1$ for $q \in \{0, 1, 2\}$, $\delta(3, b) = 0$, $\delta(0, a) = 1$, and $\delta(q, a) = q$ for $q \in \{1, 2, 3\}$.

1.2 Formal Definition of Automata Synchronization

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is *synchronizing* (or *resettable*) if there exists a word, called a *reset* or *synchronizing word*, driving it into the same state regardless of its current state. Formally, a word $u \in \Sigma^*$ is said to be a *synchronizing* (or *reset*) *word* for $\mathcal{A}$ if $\delta(Q, u) = \{q\}$ for some $q \in Q$. In this case, we say that *u synchronizes $\mathcal{A}$*, and that q is a *synchronizing state* of $\mathcal{A}$. The *set of synchronizing words* of $\mathcal{A}$ is denoted by

[1] For a more formal construction of Σ^* and concatenation, see, for example, the textbook [194, p. 5].

[2] *Deterministic* means that δ is a function rather than a relation, and *complete* means that it is total. Both modifiers are redundant here, as we consider no other kind of automaton, but we state them as a reminder for readers used to the nondeterministic setting. In Part IV, we also introduce partial automata, where the transition function may be partially defined.

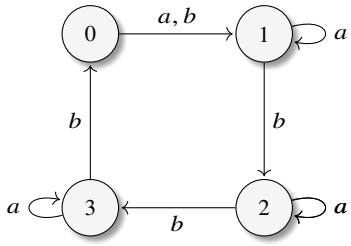


Fig. 1.1: The state diagram of the automaton from Theorem 1.1.

$\mathrm{Syn}(\mathcal{A}) = \{u \in \Sigma^* \mid \exists q \in Q\colon \delta(Q, u) = \{q\}\}$. Then $\mathcal{A}$ is synchronizing if and only if $\mathrm{Syn}(\mathcal{A})$ is non-empty.

Synchronizing words correspond to a sequence of actions (or inputs) that allow an operator (or observer) to regain control by resetting the automaton into a well-defined state. This notion arises in different contexts and applications in coding theory [87, 348, 360], robotics [198, 252, 457, 613], symbolic dynamics [605], model checking and verification [548], and group theory [36].

The introduction of synchronizing automata is often attributed to Černý and his seminal paper [140]. However, the actual history is more complicated and the concept can be traced back to earlier work of Laemmel [396] and Liu [416]. For a detailed account on the genesis of this concept, see the survey [617, Sect. 1.3].

1.3 Introductory Examples

Here, we introduce several situations and problems which can be modeled using synchronizing automata: a problem from coding theory in Section 1.3.1, a puzzle from Ashby in Section 1.3.2, a conveyor belt and part orienter design problem in Section 1.3.3, and a maze orientation problem in Section 1.3.4.

1.3.1 Error-Prone Communication Channels

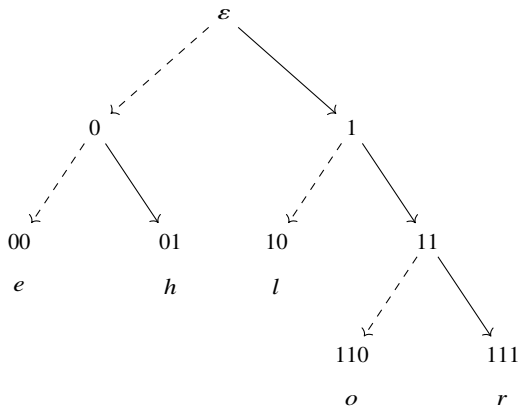
Suppose five symbols e, h, l, o, r are encoded by five binary sequences 00, 01, 10, 110, and 111, respectively, see Figure 1.2. This coding scheme is taken from Neumann [462].

When transmitting a message using this code, is it possible to recover from a loss of synchronization between the sender (encoder) and receiver (decoder) caused by channel errors? For example, suppose the sender wants to transmit "hello" to the receiver, which corresponds to the bit sequence "01 00 10 10 110". However, after sending the first three bits, the sender is uncertain whether they were received correctly.

On the receiving end, a stream of code words can be decoded with the help of the automaton shown in Figure 1.3a, which starts in the state q_ε and reflects different states

Symbol	Binary Encoding
e	00
h	01
l	10
o	110
r	111

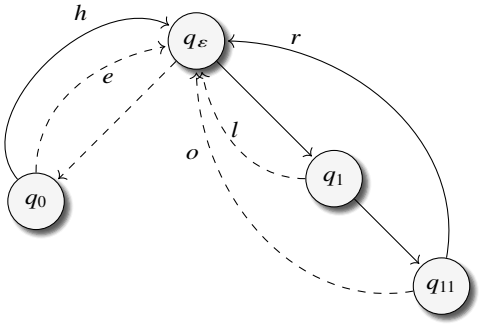
(a) Encoding of source symbols.



(b) Decision tree to decode symbols for binary inputs where dashed and solid branches correspond to 0 and 1, respectively.

Fig. 1.2: Encoding of five symbols e, h, l, o, and r by binary sequences.

of the receiver. A path starting in q_ε and going back to q_ε corresponds to a received code word, where the transition leading into q_ε indicates which code word was received. For example, when reading "01 00 10 10 110", starting from q_ε, the transitions returning to q_ε indicate the sequence "hello" in that order.



(a) Starting from q_ε, each return to q_ε corresponds to the decoding of one code word; the label on the return transition indicates the decoded source symbol.

	0	1	1	0
q_ε	q_0	q_ε	q_1	q_ε
q_0	q_ε	q_1	q_{11}	q_ε
q_1	q_ε	q_1	q_{11}	q_ε
q_{11}	q_ε	q_1	q_{11}	q_ε

(b) The state transitions for the input sequence 0110 starting from each state.

Fig. 1.3: Decoder automaton (left) for the code from Figure 1.2 where dashed (resp. solid) transitions represent input symbol 0 (resp. 1) with the synchronizing word 0110 (right).

Now suppose that the current state of the decoder automaton is unknown. This raises the following question.

Can the sender transmit a sequence of bits that guarantees the receiver is brought to the well-defined state q_ε in Figure 1.3a, from which it can correctly decode the next code word?

The answer is positive: as shown in Figure 1.3b, reading the sequence 0110 from any state of the automaton leads to q_ε. Thus, by transmitting 0110, the sender can reset the receiver in case it has lost synchronization with the stream of code words due to errors, ensuring correct decoding of subsequent bits.

Applied to the situation above, where the sender wants to transmit "hello" but is unsure whether the first three bits were transmitted correctly, the sender can send the message

$$\underbrace{010}_{\text{uncertain bits}}\ 0110\,01\,00\,10\,10\,110.$$

After reading "010 0110", the automaton is in the state q_ε, from which it can correctly decode the next code word.

1.3.2 The Haunted Mansion

Ashby [47] presented the following puzzle. A mansion is haunted by two ghostly noises, Singing and Laughter. Each of the noises can independently be either on or off, and they can be influenced by two possible actions, organ playing or burning incense, according to the following scheme described by Ashby [47, pp. 60–61].

> The Singing, in the succeeding minute, will go on as it was during the preceding minute (sounding or silent) unless there was organ-playing with no Laughter, in which case it will change to the opposite (sounding to silent, or vice versa).
>
> As for the Laughter, if there was incense burning, then it will sound or not according as the Singing was sounding or not (so that the Laughter copies the Singing a minute later). If however there was no incense burning, the Laughter will do the opposite of what the Singing did.

The puzzle asks for a sequence of actions that silences both noises when they are both active. However, more is possible: as we shall see, there exists a sequence of actions that leads to both noises becoming silent in every possible situation.

We can model the situation by an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with state set

$$Q = \{(S, L), (\overline{S}, L), (\overline{S}, \overline{L}), (S, \overline{L})\},$$

where S and $\overline{S}$ represent the presence and absence of singing, respectively, and similarly for L and $\overline{L}$ with laughter. Furthermore, the input alphabet is $\Sigma = \{a_{OB}, a_{\overline{O}B}, a_{\overline{OB}}, a_{O\overline{B}}\}$ where the symbols represent the four possible combinations of playing the organ (O), not playing organ ($\overline{O}$), burning incense (B), or not burning incense ($\overline{B}$). The transitions are given as in Figure 1.4.

The word $a_{\overline{O}\overline{B}}a_{O\overline{B}}a_{\overline{O}B}$ is a synchronizing word ending in the synchronizing state $(\overline{S}, \overline{L})$. Hence, by executing the corresponding actions, the ghosts end up silent regardless of their current behavior. As pointed out by Volkov [613], this seems to be the first

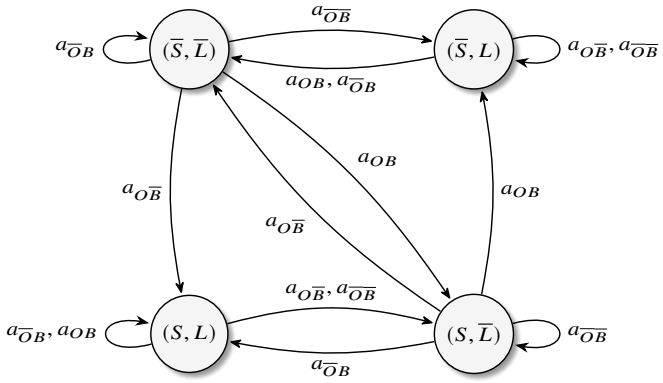


Fig. 1.4: Ashby's ghost taming automaton. The word $a_{\overline{OB}}a_{O\overline{B}}a_{\overline{OB}}$ is synchronizing.

publication in which a synchronizing automaton occurs. However, Ashby did not comment on the existence of a sequence that silences both ghosts under all circumstances.

1.3.3 Part Orienters

Suppose that, at the end of an assembly line, square-shaped components with a small square inset on one side (see Figure 1.5a) must arrive with the inset on the left side (Ananichev and Volkov [22]; see also [496, 613, 617]). For simplicity, we assume that there are only four possible orientations, shown in Figure 1.5b. As the components move along the assembly line from left to right, they encounter two kinds of obstacles: high and low. A high obstacle always rotates a component by 90-degree counterclockwise. A low obstacle rotates the component in the same way only when the inset is on the top side, since only in this orientation does the obstacle touch the component. Can one place a sequence of obstacles along the assembly line such that, regardless of the initial orientation, the final orientation always has the inset on the left side?



(a) A polygonal component.

(b) The four possible orientations.

Fig. 1.5: A polygonal component and its possible orientations on an assembly line.

Taking the four possible orientations as the states of an automaton, the effects of the two types of obstacles can be modeled by two input letters, as illustrated on the left-hand

side of Figure 1.6. As the reader might notice, this construction yields, in essence, the same automaton as the one given earlier in Figure 1.1. The original problem reduces to determining whether this automaton admits a synchronizing word. One possible synchronizing sequence is low, high, high, high, low, high, high, high, low. In the case where a component arrives with its insert oriented to the left, the effect of this sequence is shown on the right-hand side of Figure 1.6.

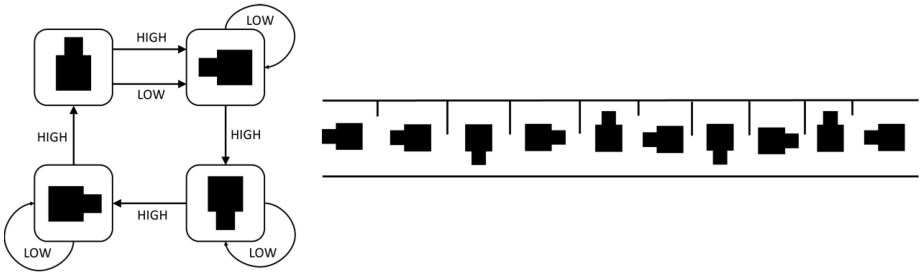


Fig. 1.6: Left: The part orienter problem modeled as a finite automaton. Right: Sequence of effects on one component moving from left to right. Image taken from Podolak et al. [496].

The above scheme works without the use of sensors and does not require any observer or external mechanism to orient the parts. Employing sensorless methods for part orientation is often preferable to using more complicated machinery because of their robustness, simplicity, and ease of maintenance. In another sensorless approach to part feeding, parts are placed on a vibrating surface and must pass certain obstacles. Such orientation schemes have apparently been employed since the mid-20th century; see [541] for a survey on vibratory part feeders. Sensorless methods for orienting parts without vibrating surfaces were first described by Erdmann and Mason [200], who simplified and mathematically analyzed a mechanical device originally introduced by Grossman and Blasgen [264].

The first author to investigate the computational complexity of designing sensorless part orienters was Natarajan [457, 458]. He studied the engineering problem of orienting polygonal objects on conveyor belts using simple, robust, and sensorless methods based on mechanical contact points installed along the belt. Mathematically, he modeled this problem both as a pebbling game and as the problem of finding a sequence of functions from a given set that realizes a constant function. He noted (without proof) that a generalization of the latter formulation is PSPACE-complete and gave polynomial-time algorithms for several variants of these problems.

Natarajan's algorithmic approach to part orienting stimulated substantial further research in this area. For example, Goldberg [252] introduced a scheme that orients parts by squeezing them with parallel-jaw grippers and gave a polynomial-time algorithm for designing a sequence of squeezes that orients any part, conjecturing that a linear number of squeezes always suffices. Chen and Ierardi [146] later confirmed this conjecture.

Eppstein [198] observed the connection to synchronizing automata previously studied in automata theory, characterized the class of automata investigated by Natarajan [457], and improved Natarajan's algorithms. In this sense, synchronizing automata emerged independently in the mechanical engineering literature. Part orienting as a motivating example for synchronizing automata was later popularized by Ananichev and Volkov [22, 23]; see also the widely cited survey [613].

1.3.4 Lost in a Maze

Imagine a maze in which every junction has exactly k outgoing corridors and all junctions look alike, so that a visitor can never tell where they currently are. The corridors leaving a junction are to be marked with k distinct colors, one color per corridor, and a visitor who is told a sequence of colors follows the corresponding corridors one after the other. The task of whoever installs the signs is the following.

> Can the corridors be colored so that some sequence of colors leads every visitor to one and the same junction, no matter where in the maze they started?

Such a sequence does not point at the exit directly; it brings all visitors to a common junction, and from there a single fixed route takes them out. Coloring the corridors turns the maze into a deterministic finite automaton: the junctions are the states, the colors form an alphabet of size k, and a corridor colored a from junction p to junction q gives the transition $\delta(p, a) = q$. The question above then asks whether the corridors admit a coloring for which the resulting automaton is synchronizing.

Not every maze does. If, for instance, the junctions can be split into two groups such that every corridor leads from one group to the other, then two visitors starting in different groups stay in different groups forever, whatever the coloring and whatever sequence they are given. Ruling out obstructions of this kind is in fact enough: the *road coloring theorem*, conjectured by Adler, Goodwyn, and Weiss [1] and proved by Trahtman [605] some thirty years later, states that a synchronizing coloring exists if and only if the maze is strongly connected and the lengths of its closed walks have greatest common divisor one.

1.4 Reset Length and the Černý Conjecture

The *reset length* of a synchronizing automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is

$$\mathrm{rt}(\mathcal{A}) = \min\{|u| : u \in \mathrm{Syn}(\mathcal{A})\}.$$

Černý [140] proved that, for $n \geq 1$, we have

$$(n-1)^2 \leq \max\{\mathrm{rt}(\mathcal{A}) \mid \mathcal{A} \text{ has } n \text{ states and } \mathrm{Syn}(\mathcal{A}) \neq \emptyset\} \leq 2^n - n - 1.$$

For the lower bound, he gave the *Černý family of automata* $C_n = ([n], \{a, b\}, \delta_n)$ where $n \geq 2$ and

$$\delta_n(i, b) = \begin{cases} i + 1 & \text{if } i < n - 1, \\ 0 & \text{if } i = n - 1, \end{cases} \qquad \delta_n(i, a) = \begin{cases} 1 & \text{if } i = 0, \\ i & \text{if } i \neq 0, \end{cases}$$

see Figure 1.7. The automaton from the part orientation problem in Figure 1.6 equals C_4. Below, we give a proof that $\mathrm{rt}(C_n) = (n-1)^2$, which also shows that $(ab^{n-1})^{n-2}a$ is the only shortest synchronizing word for C_n.

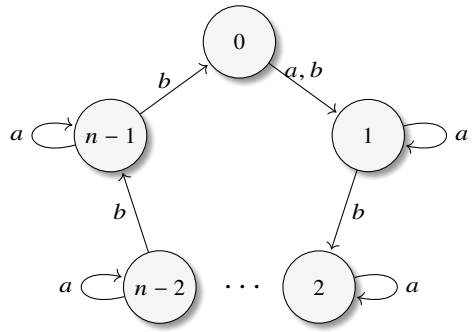


Fig. 1.7: Automata from the Černý family C_n. See Section 1.4.

Theorem 1.2 (Černý [140]) *For $n \geq 2$, we have $\mathrm{rt}(C_n) = (n-1)^2$. Moreover, $(ab^{n-1})^{n-2}a$ is the unique shortest synchronizing word of C_n.*

Proof. Identify $Q = [n]$ with $\mathbb{Z}_n$, so that $\delta_n(p, b) = p + 1$ (modulo n) for $p \in Q$, while a maps 0 to 1 and fixes all other states. An *interval* is a non-empty set $[p, q] = \{p, p + 1, \ldots, q\}$ with $p, q \in Q$. Note that $Q = [p, p - 1]$ for every $p \in Q$, and that for $I \neq Q$ the states p and q with $I = [p, q]$ are determined by I, as p is the only state in I with $p - 1 \notin I$ and q the only one with $q + 1 \notin I$. In particular, an interval containing 1 but not 0 begins at 1, and an interval containing 0 but not 1 ends at 0.

One verifies inductively that $\delta_n(Q, (ab^{n-1})^j a) = [1, n - 1 - j]$ for $0 \leq j \leq n - 2$, so $(ab^{n-1})^{n-2}a$ synchronizes C_n and a shortest synchronizing word exists. Let $w = x_1 \cdots x_m$ with $x_1, \ldots, x_m \in \Sigma$ be one with $\delta_n(Q, w) = \{q\}$ and let

$$I_j = \{\, p \in Q \mid \delta_n(p, x_{j+1} \cdots x_m) = q \,\} \qquad (0 \leq j \leq m).$$

Then $I_m = \{q\}$, $I_0 = Q$ and $I_{j-1} = \{\, p \in Q \mid \delta_n(p, x_j) \in I_j \,\}$ for $j \geq 1$. Every state p satisfies $\delta_n(p, x_1 \cdots x_j) \in I_j$; in particular, no I_j is empty. Moreover $I_j \neq Q$ for $j \geq 1$, as $I_j = Q$ means that $x_{j+1} \cdots x_m$ maps every state to q and is thus a shorter synchronizing word.

Let $1 \leq j \leq m$ and assume that I_j is an interval, say $I_j = [p_j, q_j]$, which is possible in a unique way as $I_j \neq Q$. Since $\delta_n(p, a) \in I_j$ holds for $p \neq 0$ exactly when $p \in I_j$, and for $p = 0$ exactly when $1 \in I_j$, exactly one of the following cases occurs.

1. $x_j = b$; then $I_{j-1} = [p_j - 1, q_j - 1]$, so $|I_{j-1}| = |I_j|$ and $p_{j-1} = p_j - 1$.
2. $x_j = a$ and $\{0, 1\} \subseteq I_j$ or $\{0, 1\} \cap I_j = \emptyset$; then $I_{j-1} = I_j$.
3. $x_j = a$ and $0 \in I_j$ but $1 \notin I_j$; then $I_j = [p_j, 0]$ with $p_j \neq 0$ as $I_{j-1} \neq \emptyset$, and $I_{j-1} = I_j \setminus \{0\} = [p_j, n - 1]$.
4. $x_j = a$ and $1 \in I_j$ but $0 \notin I_j$; then $p_j = 1$, and $I_{j-1} = I_j \cup \{0\} = [0, q_j]$, so $|I_{j-1}| = |I_j| + 1$ and $p_{j-1} = 0 = p_j - 1$.

In each case I_{j-1} is again an interval. So, starting from $I_m = [q, q]$ and descending, every I_j is an interval.

The cases (2) and (3) never occur: in both we have $I_{j-1} \subseteq I_j$, and, since every state p satisfies $\delta_n(p, x_1 \cdots x_{j-1}) \in I_{j-1} \subseteq I_j$ and hence $\delta_n(p, x_1 \cdots x_{j-1} x_{j+1} \cdots x_m) = q$, deleting the letter x_j from w leaves a shorter synchronizing word in these cases.

Since $|I_m| = 1$ and $|I_0| = n$, the word w contains exactly $n-1$ letters a. Let $j_1 < \cdots < j_{n-1}$ be their positions, so that j_1 is the position of the first letter a in w and j_{n-1} that of the last. Here $j_1 = 1$, as otherwise $x_1 = b$ and $Q = I_0 = [p_1 - 1, q_1 - 1]$, giving $I_1 = Q$, which we excluded; and $j_{n-1} = m$, as otherwise $x_m = b$ and $I_{m-1} = \{q - 1\}$ by case (1), so that $x_1 \cdots x_{m-1}$ maps every state to $q - 1$ and is shorter. Now let $2 \leq k \leq n - 1$. The positions j with $j_{k-1} < j < j_k$ all carry the letter b, and by case (4) we have $p_{j_k - 1} = 0$ and $p_{j_{k-1}} = 1$. Since each b decreases the first state by one and these run from $p_{j_k - 1} = 0$ down to $p_{j_{k-1}} = 1$, their number $j_k - j_{k-1} - 1$ is congruent to -1 modulo n. It equals $n - 1$, since otherwise it is at least $2n - 1$ and deleting n consecutive letters b, which changes no transition as $\delta_n(p, b^n) = p$, would again shorten w. Hence $j_k - j_{k-1} = n$ for all k and therefore

$$m = \sum_{k=2}^{n-1} (j_k - j_{k-1}) + 1 = (n - 2)n + 1 = (n - 1)^2.$$

So the letters a occupy the positions $1, n + 1, \ldots, (n - 2)n + 1 = m$, while all remaining positions carry the letter b. Thus $\mathrm{rt}(C_n) = (n - 1)^2$ and $w = (ab^{n-1})^{n-2}a$. □

Eppstein [198] introduced the notion of an interval for a more general class of automata. For a shortest synchronizing word $w = uv$ of C_n, he proved $\mathrm{rt}(C_n) = (n-1)^2$ by bounding the length of a shortest interval containing $\delta_n([n], u)$ [198, Thm. 3], whereas the original proof by Černý [140, Lem. 1] considered the length of a largest interval in $[n] \setminus \delta_n([n], u)$. The proof above instead uses the sets $\{p \in [n] \mid \delta_n(p, v) = q\}$, which are intervals, whenever non-empty, for every word v, and this also yields the uniqueness asserted in Theorem 1.2. Salomaa [547, Sec. 7] later gave a proof related to Černý's in the form of a solitaire game in which states are marked by stones (see also [546, Thm. 8], where the argument is adapted to the case in which all states are mapped to a fixed state). A graph-theoretical proof relating the reset length to the exponent of certain digraphs was given by Ananichev et al. [21, Thm. 3] (see also [360, Prop. 3.1]).

In [141, 580] the authors conjectured the following bound (the so-called *Černý bound*).

Černý conjecture: We have $\mathrm{rt}(\mathcal{A}) \leq (n-1)^2$ for every synchronizing n-state $\mathcal{A}$.

This conjecture, which has remained open for more than five decades, has stimulated extensive research since its formulation. The reader might wonder why the conjecture is not named after Starke, even though his paper [580] predates [141]. One possible reason is that older papers often referred only to the seminal work [140] when discussing the conjecture, despite the fact that, although it introduced synchronizing automata and the Černý automata, it did not explicitly state the conjecture itself. Another possible reason is that Černý discussed the conjecture at two conferences shortly after the publication of [140]. In any case, even the term "Černý–Starke conjecture" was never adopted in the literature; see [615] for further discussion on the origin of this conjecture.

Table 1.1: Published upper bounds for the reset length. Kfoury's bound was communicated to him by Paterson, see [366].

Upper Bound	Remark	Reference
$2^n - n - 1$		(1964, Černý [140])
$\frac{1}{2}n^3 - \frac{3}{2}n^2 + n + 1$		(1966, Starke [580])
$\frac{1}{2}n^3 - n^2 + \frac{1}{2}n$		(1970, Kohavi [375])
$\frac{1}{3}n^3 - n^2 - \frac{1}{3}n + 6$		(1970, Kfoury [366])
$\frac{1}{3}n^3 - \frac{3}{2}n^2 + \frac{25}{6}n - 4$		(1971, Černý et al. [141])
$\frac{7}{27}n^3 - \frac{17}{18}n^2 + \frac{17}{6}n - 3$	$n = 3k$	(1977, Pin [485])
$\left(\frac{1}{2} + \frac{\pi}{36}\right)n^3 + o(n^3)$		(1978, Pin [487, 484, 488])
$\frac{1}{6}n^3 - \frac{1}{6}n$		(1983, Pin/Frankl [227, 488], 1987, Klyachko et al. [372])
$\alpha n^3 + o(n^3)$	$\alpha \approx 0.1664$	(2018, Szykuła [593])
$\alpha n^3 + o(n^3)$	$\alpha \leq 0.1654$	(2019, Shitov [568])

Table 1.1 lists the published upper bounds in chronological order. Klyachko et al. [372] independently proved the Pin–Frankl bound from Table 1.1 behind the Iron Curtain. Pin [488] actually concluded the bound $(n^3 - n)/6 - 1$ for $n \geq 4$ with Frankl's result [227]. The bound $(n^3 - n)/6$ had previously been stated by Kohavi and Winograd [376, 377], yet, somewhat mysteriously, their work was never cited in the subsequent literature. Their argument broadly follows the same approach as that of Pin [488], analyzing traces of 2-sets in shortest synchronizing words. However, their treatment of certain maximal traces appears to lack sufficient justification, which may explain why their work was largely ignored (for this reason we also left it out in Table 1.1). Additionally, Salomaa [546, Thm. 4] proved the bound $\frac{1}{2}n^3 - \frac{3}{2}n^2 + 2n$ for the length of a shortest word mapping all states to a fixed state, rather than merely to some state as required for synchronization.

The Černý conjecture [140] has been confirmed for a variety of classes of automata, to name a few of them (without further explanation and with no claim for completeness):

1. circular automata [186, 187, 486],
2. oriented automata [198],
3. (weakly generalized) monotonic automata [22, 23, 24] and 0-monotonic automata [18],
4. automata with a trap state [52, 534],

5. commutative [54, 212, 533], partially ordered [537], solvable [534], and automata whose transition monoid belongs to the varieties **DS** [16] or **EDS** [15],
6. strongly reset automata [534],
7. nilpotent and (reverse and generalized) definite automata [54, 534],
8. automata whose set of synchronizing words is a shuffle ideal [481],
9. Eulerian automata [359],
10. automata preserving a chain of partial orders [614],
11. automata whose transition monoid contains a $\mathbb{Q}$I-group [36, 40],
12. aperiodic automata [604],
13. automata with a blocking state [52],
14. automata whose transformation semigroup contains a group fulfilling certain representation-theoretical conditions [582],
15. one-cluster automata with a cycle of prime length [583],
16. automata $\mathcal{A}$ with restrictions on generating words for $\mathrm{Syn}(\mathcal{A})$ [228, 501],
17. automata that cannot recognize $\{a,b\}^*ab\{a,b\}^*$ [15],
18. certain aperiodically 1-contracting automata [182],
19. decoder automata for finite maximal prefix codes [85],
20. automata having letters of a certain rank [85],
21. automata respecting intervals of a digraph [259],
22. strongly semisimple automata [14] (related to Item 16, see also [514]),
23. automata with two idempotent letters [616, Prop. 5],
24. 2-junction automata [260],
25. automata with simple idempotents having no permutational letter or containing a regular permutation group in the transition monoid [536],
26. automata for which their sets of synchronizing words have minimal generating words not containing a word of a certain length [228],
27. binary automata with a simple idempotent letter [618],
28. automata with strongly connected restricted Rystsov graph [135, 618],
29. media automata [199],
30. automata without involutions [601].

Furthermore, quadratic upper bounds on the reset length have been obtained for automata whose transition monoid contains every transformation [256], for one-cluster automata [70], for quasi-Eulerian and quasi-one-cluster automata [81], for regular automata [532, 531], for automata with coinciding cycles [522], for automata with simple idempotents [535], for automata having a one-point singular and satisfying additional conditions [524],[3] for automata whose transformation monoid contains a transitive group [647], and for completely reachable automata [205], the latter improving and extending the results of [256]. For several subclasses of automata, bounds tighter than $(n-1)^2$ are known. For instance, for solvable automata [534], the bound is $n-1$. A comprehensive overview of such bounds and subclasses can be found in the living list [618] of results related to the Černý conjecture maintained by Mikhail Volkov.

[3] Both [22] and [524] also contain more general formulations of the Černý conjecture for transformation monoids.

Chapter 2
Outline and Contribution

First, in Section 2.1 we introduce the two central problems with which the thesis is concerned: the constrained synchronization problem (Section 2.1.1) and the description of automata for sets of synchronizing words (Section 2.1.2). Section 2.2 gives an overview on the organization of the present work. Finally, Section 2.3 lists the papers on which this work is based, as well as the unpublished results that we have included.

2.1 The Problems

2.1.1 The Constrained Synchronization Problem

The *constrained synchronization problem* asks whether a given deterministic and complete semiautomaton has a synchronizing word contained in a given regular *constraint language*. Formally, let Σ be an alphabet and $L \subseteq \Sigma^*$ be a regular language (see Sections 3.9.4 and 6.7.1 for formal definitions of decision problems and regular languages, respectively). The problem Sync$[L]$ of *constrained synchronization* with respect to the *constraint language* L is the following decision problem.

Sync$[L]$
Input: A deterministic and complete semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$.
Question: Does there exist a word $w \in L$ synchronizing $\mathcal{A}$?

This problem, systematically studied in [215], firstly appeared in connection with a reduction procedure introduced in [271], as well as implicitly in reductions used in [438]. Chapter 16 provides further motivation and details.

2.1.2 Automata for Sets of Synchronizing Words

The set of synchronizing words of a given automaton can be accepted by another automaton of size exponential in the size of the original automaton (see Chapter 3 for the definition of automata as language acceptors). In this thesis, we also study automata for sets of synchronizing words. Our motivation comes from the constrained synchronization problem, whose computational complexity is governed by two parameters: the *structure of the constraint language* and the *structure of the set of synchronizing words* of the given input automata. If an automaton recognizing the set of synchronizing words can be constructed in polynomial time for a class of input automata, then the constrained synchronization problem (with any constraint language) is solvable in polynomial time for this class. An example is the class of commutative automata over a fixed alphabet (see Section 20.4). Consequently, NP- or PSPACE-hardness results (see Section 3.9.5 for definitions of complexity classes) imply that automata for the corresponding sets of synchronizing words cannot, in general, be constructed in polynomial time. This motivates our interest in automata needing exponentially larger accepting automata for their sets of synchronizing words.

Beyond its connection to the constrained synchronization problem, the study of automata for sets of synchronizing words is also motivated by the following observations.

1. For known families of *slowly synchronizing automata*, that is, automata whose reset length is close to the Černý bound, the minimal automata recognizing their sets of synchronizing words are exponentially larger [440, 441]. This raises the question of whether there is a deeper connection between the reset length of an automaton and the size of a minimal automaton recognizing its set of synchronizing words.
2. The operation that maps a regular language to the set of synchronizing words of its minimal automaton is regularity-preserving and can therefore be studied from the perspective of the state complexity of operations [237, 643].
3. The class of sets of synchronizing words of automata coincides with the class of regular ideal languages (Theorem 7.1). Thus, every regular ideal language can be represented by an automaton having it as its set of synchronizing words. This representation can be exponentially more succinct than a representation by an accepting automaton, as witnessed by known families of slowly synchronizing automata [440, 441]. From the perspective of the descriptional complexity of regular languages, it is therefore natural to investigate those regular ideal languages for which the gap between these two representations is maximal. In automata-theoretic terms, this leads to the study of automata for which minimal automata accepting their sets of synchronizing words are as large as possible.
   Furthermore, these observations motivate the definition of the *reset complexity* of a regular ideal language as the size of a smallest automaton having this language as its set of synchronizing words. The Černý conjecture is equivalent to the statement that the reset complexity of a regular ideal language is at least the square root of the length of a shortest word in the language, plus one [440, 441]. This yields a language-theoretic approach to the Černý conjecture through the investigation of shortest words in regular ideal languages [197, 228, 433, 442, 506, 513].

4. As we will see, our results reveal connections between subset reachability properties, the size of automata for sets of synchronizing words, and the well-known class of primitive permutation groups.

2.2 Organization of the Thesis

The thesis consists of four parts. In Part I we give a short introduction on synchronizing automata and an overview of the thesis. Part II introduces further notions, automata classes, and gives preliminary computational complexity results. A detailed overview is given in Section 2.2.1. Part III is concerned with automata accepting the set of synchronizing words of a given automaton and is outlined in Section 2.2.2 below. Part IV deals with the constrained synchronization problem and we give an overview in Section 2.2.3.

Parts III and IV are the main parts. Unless stated otherwise, all the results stated in these parts are due to the author of this thesis.

Historically, and as outlined in Section 2.1, our research on the constrained synchronization problem led us to investigate automata for sets of synchronizing words. However, in the present work, we first discuss automata for sets of synchronizing words in Part III before turning to the constrained synchronization problem in Part IV. We chose this order of presentation because Part IV relies on results from Part III, whereas Part III is independent of Part IV. Moreover, the study of automata accepting sets of synchronizing words is a natural topic in its own right, and we expand it beyond its application to the constrained synchronization problem.

2.2.1 Structure of Part II: Preliminaries

Here, we introduce further notions in Chapter 3. In Chapter 4 we investigate classes of transformations. Permutation groups are investigated in Chapter 5. More automata-theoretical notions are given in Chapter 6. Classes of automata and their properties are investigated in Chapter 7. Computational complexity results on subset mapping problems in automata with respect to certain subclasses are investigated in Chapter 8.

Most results in this part are taken from the literature. Some results on idempotent and one-point singular transformations from Chapter 4 are new. The notion of strong $\frac{3}{2}$-connectivity for digraphs introduced here is new. Further, some results in Chapter 7 are new, see also Section 2.3.2 for more details. Some structural results on solvable automata in Section 7.10 appear in [313]. Chapter 6 on automata theory contains some results on morphism and quotient automata that we have not seen elsewhere. The results in Chapter 8 were first presented in [313].

2.2.2 Structure of Part III: Automata for Sets of Synchronizing Words

Part III studies automata that accept the set of synchronizing words of a given automaton. We derive new notions and conditions related to the size of accepting automata for sets of synchronizing words, also giving conditions under which minimal automata for such sets have maximal size. We further relate our findings to classical automata-theoretic results from the field of descriptional complexity. This part is based on the papers [297, 299, 303, 304, 306, 307, 309, 315], but the presentation has been extended and unified, with additional new results included.

Except in Sections 9.2.1 and 11.1 to 11.3 and the discussion sections, which review notions and results from the literature, all results presented are novel and due to the author. Next, we describe the organization of Part III and give a broad overview of each chapter and its relation to the work on which this thesis is based. A more detailed overview of the chapters, including descriptions of their individual sections, is given at the beginning of each chapter.

In Chapter 9, we first note that the set of synchronizing words is a regular language acceptable by the power automaton. To obtain lower bounds on the size of automata accepting the set of synchronizing words, we apply minimality conditions to the power automaton. This leads us to the notions of reachability of subsets and distinguishability of pairs of subsets. The rest of the chapter investigates these two notions, giving sufficient conditions for reachability and distinguishability. We also introduce completely reachable automata, a class previously studied in the literature, and the new class of completely distinguishable automata, for which we prove several equivalent characterizations. The results in this chapter are based on [308, 315], where completely distinguishable automata were introduced, but it also contains new unpublished material, see Section 2.3.2.

Chapter 10 introduces sync-maximal automata as synchronizing automata for which an automaton accepting the set of synchronizing words is as large as possible, i.e., for which the gap between the state complexity and the reset complexity of the set of synchronizing words is maximal. We investigate sufficient conditions and derive structural properties, showing that sync-maximal automata are closely related to completely reachable automata. We also give various examples. The notion of sync-maximal automata, as well as the structural results, were first introduced in [308, 315].

Chapter 11 discusses graph-theoretical constructions that yield sufficient conditions for complete reachability and complete distinguishability. First, we review constructions and results from the literature. We then extend some of these constructions to arbitrary transformation semigroups. Finally, we give graph-theoretical conditions ensuring complete reachability or distinguishability of certain pairs of subsets. The results from this chapter are new and have not been published before.

Chapter 12 investigates the following subclasses of automata: commutative automata, automata in which every letter has defect at most one, SI-automata, and circular automata. For commutative automata, we show that an automaton recognizing the set of synchronizing words can be computed in polynomial time when the alphabet is fixed. This result was published in [299] in the context of the constrained synchronization problem for commutative constraint languages, and will be used later in Part IV. We

then consider automata in which every letter has defect at most one. In this setting, we show that if the group of units of the transformation monoid is transitive, then synchronizability implies complete distinguishability. As a consequence, a binary automaton with at least three states is completely reachable if and only if it is sync-maximal. These results are taken from [315]. Then we investigate SI-automata. For example, we show that in synchronizing SI-automata distinct pairs 2-sets are distinguishable. We derive this from more general and unpublished results, but it also follows from [306, Lem. 11]. However, the paper [306] contains a mistake stemming from a false statement [306, Prop. 2]. We discuss this in detail at the end of Section 12.7. Lastly, we investigate circular automata and state a sufficient condition for complete reachability and complete distinguishability. A special case of this condition was published in [307], with a preliminary version presented in [303].

Chapter 13 studies the decision problems for complete reachability, complete distinguishability, and sync-maximality, as well as related questions. For example, we prove that these problems are NL-hard for SI-automata, with complete reachability even being NL-complete for SI-automata, and in PSPACE for general automata. The results on SI-automata were published in [308, 315].

Chapter 14 discusses relations to permutation groups. We introduce primitive groups and characterize them in terms of complete reachability, complete distinguishability, and in terms of sync-maximality of associated automata. The relation of primitive groups to complete reachability was first published in [297], and the relation to sync-maximality in [304]. The extended journal publication [309] of [297, 304] contains both of these characterizations. We then introduce A-primitivity and study its connection to congruences, orbitals, generating sets, synchronization, reachability, and homogeneity properties. Our results generalize existing characterizations of primitivity from the literature, and a characterization by a homogeneity property also gives a new characterization. The notion of A-primitivity is introduced here for the first time, and the results of this chapter have not been published previously.

Chapter 15 discusses families of slowly synchronizing automata. We observe that, for most of these families of automata, the minimal automata accepting their sets of synchronizing words have exponential size. In particular, we use results from Chapter 12 on circular automata to deduce sync-maximality for several families of slowly synchronizing circular automata from the literature, including one family for which sync-maximality has previously only been conjectured. This confirmation of the conjecture was presented in [303, 307]. We then show, for three families of slowly synchronizing but not sync-maximal automata, that minimal automata for their sets of synchronizing words are exponentially larger. Based on our observations, we formulate various conjectures. Apart from the confirmation of the aforementioned conjecture, the results from this chapter have not been published.

2.2.3 Structure of Part IV: The Constrained Synchronization Problem

Part IV deals with the constrained synchronization problem and is based on the works [215, 294, 296, 298, 299, 301, 305, 316]. The results from [294, 298, 299,

301, 316] were revisited with an emphasis on a more streamlined, coherent, and simplified presentation. This was achieved primarily through the use of general methods introduced in [301, 316].

In Chapter 16, we give motivational examples for the constrained synchronization problem.

Chapter 17 introduces additional notions that we will need in Part IV. We also present a new characterization of solid codes that, to the best of our knowledge, has not been published before.

Chapter 18 presents general results on manipulating constraint languages while preserving the complexity or to deduce complexity lower bounds. Furthermore, we give sufficient conditions for NP- and PSPACE-hardness. Some of these results were presented in [301, 316], which generalized results from [215], and some results are new.

The central question investigated in Chapter 19 is the computational complexity of $\textsc{Sync}[L]$ for different classes of regular constraint languages $L \subseteq \Sigma^*$. While a complete classification remains an *open problem*, we present several partial results by giving complete classifications for the following classes of constraint languages: letter-bounded languages, languages accepted by partial automata (PDFAs, introduced in Chapter 17) with at most two states over an arbitrary alphabet and at most three states over a binary alphabet, and commutative languages. We also obtain results for infinite, sparse, and monoid languages, as well as for languages accepted by strongly connected PDFAs. The classification for commutative constraint languages appeared in [294, 299]. Here, we present a simplified formulation and proof. The results for sparse and letter-bounded languages appeared in [296, 298].

Lastly, motivated by the question of what features are needed in input automata to realize certain computational complexities, in Chapter 20 we investigate specific types of input automata. Some of the results from this chapter appeared in [305], and some mistakes from this paper have been corrected.

2.3 Work by the Author Relevant to the Thesis

2.3.1 Published Work

The present thesis contains and extends material from the following papers and conference contributions published since 2019, in which the author was involved.

[215] Henning Fernau et al. “Computational Complexity of Synchronization under Regular Constraints”. In: *44th International Symposium on Mathematical Foundations of Computer Science, MFCS 2019, August 26-30, 2019, Aachen, Germany*. Ed. by Peter Rossmanith, Pinar Heggernes, and Joost-Pieter Katoen. Vol. 138. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, 63:1–63:14. ISBN: 978-3-95977-117-7. DOI: 10.4230/LIPIcs.MFCS.2019.63

[296] Stefan Hoffmann. “On A Class of Constrained Synchronization Problems in NP”. In: *Proceedings of the 21st Italian Conference on Theoretical Computer Science, Ischia, Italy, September 14-16, 2020*. Ed. by Gennaro Cordasco and Luisa Gargano

and Adele A. Rescigno. Vol. 2756. CEUR Workshop Proceedings. available online: http://ceur-ws.org/Vol-2756/paper_19.pdf. CEUR-WS.org, 2020, pp. 145–157

[294] Stefan Hoffmann. "Computational Complexity of Synchronization Under Regular Commutative Constraints". In: *Computing and Combinatorics - 26th International Conference, COCOON 2020, Atlanta, GA, USA, August 29-31, 2020, Proceedings*. Ed. by Donghyun Kim and R. N. Uma and Zhipeng Cai and Dong Hoon Lee. Vol. 12273. Lecture Notes in Computer Science. Springer, 2020, pp. 460–471. DOI: 10.1007/978-3-030-58150-3_37

[303] Stefan Hoffmann. "State Complexity of the Set of Synchronizing Words for Circular Automata and Automata over Binary Alphabets". In: *Language and Automata Theory and Applications - 15th International Conference, LATA 2021, Milan, Italy, March 1-5, 2021, Proceedings*. Ed. by Alberto Leporati et al. Vol. 12638. Lecture Notes in Computer Science. Springer, 2021, pp. 318–330. DOI: 10.1007/978-3-030-68195-1_25

[297] Stefan Hoffmann. "Completely Reachable Automata, Primitive Groups and the State Complexity of the Set of Synchronizing Words". In: *Language and Automata Theory and Applications - 15th International Conference, LATA 2021, Milan, Italy, March 1-5, 2021, Proceedings*. Ed. by Alberto Leporati et al. Vol. 12638. Lecture Notes in Computer Science. Springer, 2021, pp. 305–317. DOI: 10.1007/978-3-030-68195-1_24

[301] Stefan Hoffmann. "Ideal Separation and General Theorems for Constrained Synchronization and Their Application to Small Constraint Automata". In: *Computing and Combinatorics - 27th International Conference, COCOON 2021, Tainan, Taiwan, October 24-26, 2021, Proceedings*. Ed. by Chi-Yeh Chen et al. Vol. 13025. Lecture Notes in Computer Science. Springer, 2021, pp. 176–188. DOI: 10.1007/978-3-030-89543-3_15

[300] Stefan Hoffmann. "Constrained Synchronization and Subset Synchronization Problems for Weakly Acyclic Automata". In: *Developments in Language Theory - 25th International Conference, DLT 2021, Porto, Portugal, August 16-20, 2021, Proceedings*. Ed. by Nelma Moreira and Rogério Reis. Vol. 12811. Lecture Notes in Computer Science. Springer, 2021, pp. 204–216. DOI: 10.1007/978-3-030-81508-0_17

[299] Stefan Hoffmann. "Constrained Synchronization and Commutativity". In: *Theoretical Computer Science* 890 (2021), pp. 147–170. DOI: 10.1016/j.tcs.2021.08.030 (extended version of [294])

[298] Stefan Hoffmann. "Computational Complexity of Synchronization Under Sparse Regular Constraints". In: *Fundamentals of Computation Theory - 23rd International Symposium, FCT 2021, Athens, Greece, September 12-15, 2021, Proceedings*. Ed. by Evripidis Bampis and Aris Pagourtzis. Vol. 12867. Lecture Notes in Computer Science. Springer, 2021, pp. 272–286. DOI: 10.1007/978-3-030-86593-1_19. arXiv: 2108.00081

[304] Stefan Hoffmann. "Sync-Maximal Permutation Groups Equal Primitive Permutation Groups". In: *Descriptional Complexity of Formal Systems - 23rd IFIP WG 1.02 International Conference, DCFS 2021, Virtual Event, September 5, 2021, Proceed-*

ings. Ed. by Yo-Sub Han and Sang-Ki Ko. Vol. 13037. Lecture Notes in Computer Science. Springer, 2021, pp. 38–50. DOI: 10.1007/978-3-030-93489-7_4

[306] Stefan Hoffmann. "Reset Complexity and Completely Reachable Automata with Simple Idempotents". In: *Descriptional Complexity of Formal Systems - 24rd IFIP WG 1.02 International Conference, DCFS 2022, August 29 - 31, 2022, Debrecen, Hungary, Proceedings*. Ed. by Yo-Sub Han and György Vaszil. Vol. 13439. Lecture Notes in Computer Science. Springer, 2022, pp. 85–99. DOI: 10.1007/978-3-031-13257-5_7

[305] Stefan Hoffmann. "Constrained Synchronization for Monotonic and Solvable Automata and Automata with Simple Idempotents". In: *Implementation and Application of Automata - 26th International Conference, CIAA 2022, Rouen, France, June 28 - July 1, 2022, Proceedings*. Ed. by Pascal Caron and Ludovic Mignot. Vol. 13266. Lecture Notes in Computer Science. Springer, 2022, pp. 225–237. DOI: 10.1007/978-3-031-07469-1_18

[316] Stefan Hoffmann. "General Theorems for Constrained Synchronization and their Application to Small Constraint Automata". In: *Journal of Combinatorial Optimization* (to appear) (extended version of [301])

[309] Stefan Hoffmann. "New Characterizations of Primitive Permutation Groups with Applications to Synchronizing Automata". In: *Information and Computation* 295 (2023). Special Issue: Selected papers of the 15th International Conference on Language and Automata Theory and Applications, LATA 2021, p. 105086. ISSN: 0890-5401. DOI: 10.1016/j.ic.2023.105086 (extended version of [297])

[307] Stefan Hoffmann. "Binary and Circular Automata Having Maximal State Complexity for the Set of Synchronizing Words". In: *Information and Computation* 295 (2023). Special Issue: Selected papers of the 15th International Conference on Language and Automata Theory and Applications, LATA 2021, p. 105076. ISSN: 0890-5401. DOI: 10.1016/j.ic.2023.105076 (extended version of [303])

[308] Stefan Hoffmann. "Completely Distinguishable Automata and the Set of Synchronizing Words". In: *Developments in Language Theory - 27th International Conference, DLT 2023, Umeå, Sweden, June 12-16, 2023, Proceedings*. Ed. by Frank Drewes and Mikhail Volkov. Vol. 13911. Lecture Notes in Computer Science. Springer, 2023, pp. 128–142. DOI: 10.1007/978-3-031-33264-7_11

[315] Stefan Hoffmann. "Completely Distinguishable Automata and the Set of Synchronizing Words". In: *Theory of Computing Systems* 70.20 (2 2026). DOI: 10.1007/s00224-024-10204-9 (extended version of [308])

[313] Stefan Hoffmann. "Subset Mapping Problems in Solvable Automata". In: *Journal of Automata, Languages and Combinatorics* 29.2-4 (2024), pp. 199–236. DOI: 10.25596/JALC-2024-199

2.3.2 Unpublished Work

Here, we discuss results contained in the present thesis that were not published before.

In Part II, the results on one-point singulars from Sections 4.4 and 4.6 are new. We were also unable to find the statements from Section 4.5 on compositions of idempotent

transformations in the literature (but most of them are rather technical or perhaps folklore). We also note that our treatment of automata theory in Chapter 6, tailored to our needs, differs from those found in the literature of which we are aware, in particular in its usage of the reachability order and morphisms. Furthermore, many results related to morphisms and isomorphisms between automata that we give are new or scattered throughout the literature under different terminology and notation.

In Part III, the results on reachable subsets from Section 9.2, on pairs of distinguishable subsets from Sections 9.3.2 and 9.3.3, the graph-theoretical results from Section 11.4, on automata where every letter has defect at most one from Section 12.3, and on subset mapping properties for pairs of equal-sized sets in SI-automata from Section 12.4.1 are new (but one preliminary result appeared in [306, Lem. 11]). Furthermore, the notion of A-primitivity is novel and all results from Section 14.3 appear in this thesis for the first time.

In Part IV, some characterizations of solid codes from Section 17.4.4 are (to the best of our knowledge) new. The complexity lower bounds involving concatenations from Section 18.2.5 have not been published previously. In Chapter 20 some published results have been corrected or extended. For example, the classification for partially ordered automata in Section 20.2.4 refines the results of [300]; the bound for monotonic automata claimed in Theorem 20.5 corrects that of [305]; and the claims on solvable automata in Section 20.5 have likewise been corrected. See Section 20.7 for details. Also, some material has been presented differently. Further details on new, extended, or revised material are given in the discussion section at the end of each chapter.

Part II
Preliminaries

"Es steht alles schon bei Dedekind."
—Emmy Noether (1882–1935)

Chapter 3
General Notions

Recall the definitions from Section 1.1 regarding set theory, functions, automata, and formal languages. Here, we introduce additional notions, facts, and results not covered previously. Section 3.1 gives more basic notions from logic, set and number theory. Further notions related to functions, in particular for functions on finite sets, are introduced in Section 3.2. Section 3.3 introduces transformations and gives some results for transformations on finite sets. In Section 3.4 we introduce relations. Semigroups, monoids, groups, semirings, matrices, and transformation semigroups are introduced in Section 3.5. Order-theoretical notions are introduced in Section 3.6. Section 3.7 gives more background on formal language theory and introduces regular expressions. Graphs and connectedness notions are introduced in Section 3.8. Section 3.9 introduces computational complexity theory and our theoretical framework to measure the resource requirements of algorithms. Sections 3.10 and 3.11 discusses conventions and notational choices. Lastly, Section 3.12 gives an overview of the literature related to the introduced notions.

The present chapter serves to fix notation. We assume some familiarity and prior exposure with general concepts from logic, set theory, number theory, basic algebra, formal language theory, and complexity theory.

3.1 More on Logic, Set Theory, and Number Theory

Some notions were already introduced in Section 1.1, for example $[n] = \{0, 1, \ldots, n-1\}$; see also the notation index (page xi) for a list of all notations. We use standard logical notation: $\wedge$ for *conjunction* (and), $\vee$ for *disjunction* (or), $\neg$ for *negation*, $\Rightarrow$ (*implication*), $\Leftrightarrow$ (*equivalence*), $\forall$ and $\exists$ for the *universal quantifier* and the *existential quantifier*, respectively, etc. We also assume familiarity with standard arguments on finite sets, like the *pigeonhole principle*, *proofs by induction*, and the *well-ordering principle*. Furthermore, we use standard (naive) set-theoretical notation: *union* ($\bigcup$), *disjoint union* ($\uplus$),[1] *intersection* ($\cap$), *set difference* ($\setminus$), *subset* ($\subseteq$), *proper subset* ($\subsetneq$) etc. For a set X, we

[1] Meant to be pairwise disjoint when more than two sets are involved.

denote by $\mathcal{P}(X) = \{Y \mid Y \subseteq X\}$ the *power set (of X)*. A finite set containing k elements is called a *k-set*. A 1-set is also called a *singleton set*. If $X_1, \ldots, X_n$ are sets, their *Cartesian product* is $X_1 \times \cdots \times X_n = \{(x_1, \ldots, x_n) \mid \forall i \in \{1, \ldots, n\} : x_i \in X_i\}$, i.e., the set of n-tuples where the i-th component is an element from X_i for $i \in \{1, \ldots, n\}$. For $X_1 = \cdots = X_n$, we set $X_1^n = X_1 \times \cdots \times X_n$ where X_1^0 equals the singleton set containing the empty tuple. Elements from X_1^n for $n \geq 0$ are also identified with sequences of length n and, for finite X_1, we also call them words over X_1 (see Section 1.1) where the empty tuple corresponds to the empty word. The *(component) projection mappings* are the mappings $\pi_i \colon X_1 \times \cdots \times X_n \to X_i$ given by $\pi_i((x_1, \ldots, x_i, \ldots, x_n)) = x_i$ for $i \in \{1, \ldots, n\}$. Two sets X, Y are called *disjoint* if $X \cap Y = \emptyset$. A *partition of X* is a family of non-empty sets $\{X_i \mid i \in I\}$ for some index set I such that $X = \biguplus_{i \in I} X_i$. We call $\{X\}$ and $\{\{x\} \mid x \in X\}$ the *trivial partitions*. A *section* (or *transversal*) of a partition $\{X_i \mid i \in I\}$ of X is a subset $S \subseteq X$ such that $S \cap X_i$ is a singleton for each $i \in I$.

Every finite set X, by ordering its elements, can be identified with $[n]$ for $n = |X|$. So, statements involving $[n]$ for some $n \geq 0$ hold for all finite sets X of size n by choosing some identification for X. Thus, in statements involving finite sets we can assume, without loss of generality, the finite sets to be of the form $[n]$ for some $n \geq 0$, which will be done in the present work.

For $a, a', b \in \mathbb{N}_0$ with $b > 0$, we denote by $a \bmod b$ the unique number $0 \leq r < b$ such that $a = bn + r$ for some $n \geq 0$. If $r = 0$, we say that b *divides* a (or a is a *multiple* of b). We write $a \equiv_b a'$ or $a \equiv a' \pmod{b}$ if $a \bmod b = a' \bmod b$ (equivalently b divides $(a - a')$). Suppose c divides b and $a \equiv a' \pmod{b}$. Then $a \equiv a' \pmod{c}$ and we get

$$a \equiv 0 \pmod{c} \Leftrightarrow a' \equiv 0 \pmod{c}. \tag{3.1}$$

For a finite set $M \subseteq \mathbb{N}_0$ we denote by $\gcd(M)$ (resp. $\operatorname{lcm}(M)$ if $0 \notin M$), also for short $\gcd M$ (resp. $\operatorname{lcm} M$), the *greatest common divisor* (resp. *least common multiple*) of the numbers in M where we set $\gcd(\emptyset) = 0$ (resp. $\operatorname{lcm}(\emptyset) = 1$). Also we set $\gcd(a_1, \ldots, a_n) = \gcd\{a_1, \ldots, a_n\}$ (resp. $\operatorname{lcm}(a_1, \ldots, a_n) = \operatorname{lcm}\{a_1, \ldots, a_n\}$) for $a_1, \ldots, a_n \geq 0$ (resp. $a_1, \ldots, a_n \geq 1$). Two number a, b are called *coprime* if $\gcd(a, b) = 1$. A number $p > 1$ is *prime* if it is only divisible by 1 or p. For a real number x we set $\lfloor x \rfloor = \max\{k \in \mathbb{Z} \mid k \leq x\}$.

Theorem 3.1 (Prime Number Theorem (see, for example, [404])) *Suppose we have a numbering $p_1 < p_2 < p_3 < \ldots$ of the prime numbers. Then for every $\varepsilon > 0$ there exists $N > 0$ such that $(1 - \varepsilon) \cdot n \log(n) \leq p_n \leq (1 + \varepsilon) \cdot n \log(n)$ for all $n \geq N$.*

Corollary 3.2 *There exists a number $N \geq 1$ such that $p_n < n^2$ for all $n \geq N$.*

3.2 More on Functions

We already introduced (total) functions in Section 1.1. Here, Section 3.2.1 introduces more basic notions and states some equations. We investigate the duplicate set of the composition of two functions in Section 3.2.2. We need these results when we later

investigate the composition of certain classes of functions. Section 3.2.3 investigates functions on finite sets.

3.2.1 Basic Notions

Given sets X, Y we set $Y^X = \{f \mid f\colon X \to Y \text{ is a function}\}$. For $n \geq 0$, $X^{\{1,\dots,n\}}$ can be identified with the Cartesian product X^n by identifying $f\colon \{1,\dots,n\} \to X$ for $n \geq 1$ with the tuple $(f(1),\dots,f(n))$ and the unique *empty function* $\emptyset \to X$ with the empty tuple over X. Occasionally, we use the notation $(f(x))_{x\in X}$ to denote or specify a function $f\colon X \to Y$.

Let $f\colon X \to Y$. The *extension of f to subsets* is the function $\hat{f}\colon \mathcal{P}(X) \to \mathcal{P}(Y)$ given by $\hat{f}(S) = \{f(x) : x \in S\}$ for $S \subseteq X$. In the following, we denote $\hat{f}$ also by f. We call $f(S)$ the *image of S under f* (for $S = X$ just the *image of f*). The function f also induces a function $f^{-1}\colon \mathcal{P}(Y) \to \mathcal{P}(X)$ given by $f^{-1}(T) = \{x \in X \mid f(x) \in T\}$ for $T \subseteq Y$. We call $f^{-1}(T)$ the *inverse image* (or *preimage*) *of $T \subseteq Y$ under f*. We have $f^{-1}(T) = f^{-1}(T \cap f(X))$ for $T \subseteq Y$. For $y \in Y$, we set $f^{-1}(y) = f^{-1}(\{y\})$. The *restriction of f* to a subset $S \subseteq X$ is $f_{|S}\colon S \to Y$ given by $f_{|S}(x) = f(x)$ for $x \in S$. We have $f_{|S}^{-1}(T) = f^{-1}(T) \cap S$ for $T \subseteq Y$.

The *excluded set of f* (the set of excluded elements) is

$$\operatorname{excl}(f) = Y \setminus f(X).$$

The *duplicate set of f* (the set of duplicate elements) is the subset of $f(X)$ given by

$$\operatorname{dupl}(f) = \{f(x) \mid x \in X \wedge f^{-1}(f(x)) \neq \{x\}\}.$$

For $S \subseteq X$ and $y \in Y$ we have

$$y \in \operatorname{dupl}(f_{|S}) \Leftrightarrow f^{-1}(y) \cap S \text{ contains at least two distinct elements,} \tag{3.2}$$

i.e., $y \in \operatorname{dupl}(f_{|S})$ if and only if $y = f(x_1) = f(x_2)$ for distinct $x_1, x_2 \in S$. Thus, for $|S| = 1$ we get $\operatorname{dupl}(f_{|S}) = \emptyset$. We also have

$$\operatorname{dupl}(f_{|S}) \subseteq f(S). \tag{3.3}$$

Next, for $S \subseteq T \subseteq X$ we get

$$\operatorname{dupl}(f_{|T}) = \operatorname{dupl}(f_{|S}) \cup \operatorname{dupl}(f_{|T\setminus S}) \cup (f(S) \cap f(T \setminus S)). \tag{3.4}$$

We call f *surjective* if $\operatorname{excl}(f) = \emptyset$, *injective* if $\operatorname{dupl}(f) = \emptyset$, i.e., if $f(x) = f(y)$ for $x, y \in X$ implies $x = y$, and *bijective* if it is surjective and injective. The function f is said to be *constant* if $f^{-1}(f(x)) = X$ for some $x \in X$.

The function f is bijective if and only if the preimage $f^{-1}(y)$ is a singleton for every $y \in Y$. For bijective f, we identify the map that sends each $y \in Y$ to the unique element in $f^{-1}(y)$ with f^{-1}, that is, we regard the set-valued map $f^{-1}\colon Y \to \mathcal{P}(X)$ as a function $f^{-1}\colon Y \to X$, called the *inverse function* of f. We note that, for every $S \subseteq Y$, there is no ambiguity in the notation $f^{-1}(S)$: whether interpreted as the preimage of S under f or as the image of S under the inverse function f^{-1}, i.e., the set $\{f^{-1}(x) \mid x \in S\}$, we obtain the same set $\{x \in X \mid f(x) \in S\}$.

The *identity transformation (on X)* is $\mathrm{id}_X\colon X \to X$ (or just $\mathrm{id}\colon X \to X$) given by $\mathrm{id}_X(x) = x$ for all $x \in X$.

Given $f\colon X \to Y$ and $g\colon Y \to Z$, the *(function) composition* of g and f is $g \circ f\colon X \to Z$ (or $gf\colon X \to Z$ for short) given by $(gf)(x) = g(f(x))$ for each $x \in X$. Obviously, we have $gf = g_{|f(X)}f$. For $T \subseteq Z$ we have $(gf)^{-1}(T) = f^{-1}(g^{-1}(T))$. This gives

$$(gf)^{-1} = f^{-1}g^{-1}. \tag{3.5}$$

Let I and J be index sets. For sets $T_i \subseteq Y$ and $S_j \subseteq X$ where $i \in I$, $j \in J$ we have

$$f\left(\bigcup_{j\in J} S_j\right) = \bigcup_{j\in J} f(S_j), \qquad f^{-1}\left(\bigcup_{i\in I} T_i\right) = \bigcup_{i\in I} f^{-1}(T_i),$$
$$f^{-1}\left(\bigcap_{i\in I} T_i\right) = \bigcap_{i\in I} f^{-1}(T_i), \qquad f\left(\bigcap_{j\in J} S_j\right) \subseteq \bigcap_{j\in J} f(S_j).$$

This also yields that for $S_1, S_2 \subseteq X$ with $S_1 \subseteq S_2$ we have $f(S_1) \subseteq f(S_2)$ and $f^{-1}(S_1) \subseteq f^{-1}(S_2)$.

Furthermore, for $T \subseteq Y$ and $S \subseteq X$ we have

$$f(f^{-1}(T)) = T \cap f(X) \tag{3.6}$$
$$T \setminus f(f^{-1}(T)) = T \cap \mathrm{excl}(f) \tag{3.7}$$
$$f^{-1}(f(S)) = S \cup f^{-1}(f(S) \cap \mathrm{dupl}(f)) \tag{3.8}$$
$$f^{-1}(f(S)) = (S \setminus f^{-1}(\mathrm{dupl}(f))) \uplus f^{-1}(f(S) \cap \mathrm{dupl}(f)) \tag{3.9}$$

Equations (3.6) to (3.8) are easy to verify. For Equation (3.9), split $f(S)$ into $f(S) \cap \mathrm{dupl}(f)$ and $f(S) \cap (Y \setminus \mathrm{dupl}(f))$ whose inverse images are disjoint. So it remains to note that $f^{-1}(f(S) \cap (Y \setminus \mathrm{dupl}(f))) = S \setminus f^{-1}(\mathrm{dupl}(f))$, where $\supseteq$ is clear and, conversely, $f(x) \in f(S)$ gives $f(x) = f(x')$ for some $x' \in S$, while $f(x) \notin \mathrm{dupl}(f)$ gives $f^{-1}(f(x)) = \{x\}$, so that $x = x' \in S \setminus f^{-1}(\mathrm{dupl}(f))$.

By Equation (3.6), we have $f(f^{-1}(T)) = T$ if and only if $T \subseteq f(X)$. The function f is surjective (resp. injective) if and only if $f(f^{-1}(T)) = T$ for each $T \subseteq Y$ (resp. $f^{-1}(f(S)) = S$ for each $S \subseteq X$) by Equation (3.6) (resp. Equation (3.9)). Note that, by removing S on both sides in Equation (3.9), we find $f^{-1}(f(S)) \setminus S = f^{-1}(f(S) \cap \mathrm{dupl}(f)) \setminus S$.

In the following, we will often use these relations and the above equations and inclusions without special mention.

Proposition 3.3 *A function $f\colon X \to Y$ is surjective (resp. injective for $X \neq \emptyset$) if and only if $fg = \mathrm{id}_Y$ (resp. $gf = \mathrm{id}_X$) for some $g\colon Y \to X$.*

Proof. Suppose f is surjective. Since $f^{-1}(y)$ is non-empty for $y \in Y$, we can find a function $g\colon Y \to X$ such that $fg = \mathrm{id}_Y$ by choosing $g(y) \in f^{-1}(y)$ for each $y \in Y$.[2] If

[2] Here, we need the axiom of choice.

f is not surjective, equivalently $f(X)$ is a proper subset of Y, then $f(g(Y))$ is a proper subset of Y for each $g\colon Y \to X$. This yields that $fg \neq \mathrm{id}_Y$ since $\mathrm{id}_Y(Y) = Y$.

Now, suppose f is injective. Let $g\colon Y \to X$ where $g(y)$ for $y \in Y$ equals the unique element in $f^{-1}(y)$ if $f^{-1}(y) \neq \emptyset$, and $g(y) \in X$ is arbitrary otherwise, which is possible as $X \neq \emptyset$. Then $gf = \mathrm{id}_X$. Next, suppose f is not injective. Then we have $f(x) = f(y)$ for distinct $x, y \in X$. Let $g\colon Y \to X$. We have $g(f(x)) = g(f(y))$. Since $\mathrm{id}_X(x) \neq \mathrm{id}_X(y)$ we find $gf \neq \mathrm{id}_X$. □

For bijective f, the functions from Theorem 3.3 are uniquely determined and equal to f^{-1}. In fact, the inverse function for bijective f is characterized as the unique function f^{-1} such that $f^{-1}f = \mathrm{id}_X$ and $ff^{-1} = \mathrm{id}_Y$.

3.2.2 The Duplicate Set and Function Composition

In Theorem 3.4 we present identities for the duplicate set of a composition of two functions. Theorem 3.6 gives an identity for the inverse image of the duplicate set of a composition. Figure 3.1 given an illustration for these identities. We also note some corollaries and give examples.

Proposition 3.4 *Let $f\colon X \to Y$ and $g\colon Y \to Z$. Then the following equations hold:*

$$\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|f(X)}) \cup g(\mathrm{dupl}(f)) \tag{3.10}$$

$$\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|f(X)\setminus \mathrm{dupl}(f)}) \cup g(\mathrm{dupl}(f)) \tag{3.11}$$

$$\mathrm{dupl}(gf) = \{z \in Z \mid g^{-1}(z) \cap \mathrm{dupl}(f) \neq \emptyset \text{ or } |g^{-1}(z) \setminus \mathrm{excl}(f)| \geq 2\} \tag{3.12}$$

Proof. 3.10: Clearly, we have $\mathrm{dupl}(g_{|f(X)}) \cup g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(gf)$. Next, let $y \in \mathrm{dupl}(gf)$. Then $y = g(f(x_1)) = g(f(x_2))$ for distinct $x_1, x_2 \in X$. If $f(x_1) \neq f(x_2)$ then $y \in \mathrm{dupl}(g_{|f(X)})$, and $f(x_1) = f(x_2)$ implies $y \in g(\mathrm{dupl}(f))$. So, $\mathrm{dupl}(gf) \subseteq \mathrm{dupl}(g_{|f(X)}) \cup g(\mathrm{dupl}(f))$.

3.11: Set $S = f(X) \setminus \mathrm{dupl}(f)$. By Equation (3.4) we have

$$\mathrm{dupl}(g_{|f(X)}) = \mathrm{dupl}(g_{|\,\mathrm{dupl}(f)}) \cup \mathrm{dupl}(g_{|S}) \cup (g(\mathrm{dupl}(f)) \cap g(S)).$$

Since $g(\mathrm{dupl}(f)) \cap g(S) \subseteq g(\mathrm{dupl}(f))$ and, by Equation (3.3), $\mathrm{dupl}(g_{|\,\mathrm{dupl}(f)}) \subseteq g(\mathrm{dupl}(f))$, we obtain the claim from Equation (3.10).

3.12: We have $g^{-1}(z) \cap \mathrm{dupl}(f) \neq \emptyset$ if and only if $z \in g(\mathrm{dupl}(f))$, and $|g^{-1}(z) \setminus \mathrm{excl}(f)| \geq 2$ if and only if $z \in \mathrm{dupl}(g_{|f(X)})$. Hence, the formula follows from Equation (3.10). □

Volkov and Casas [134] gave Equation (3.12) formulated for words and automata.

For finite sets, Equation (3.11) gives

$$\max\{|\,\mathrm{dupl}(g_{|f(X)})|, |g(\mathrm{dupl}(f))|\} \leq |\,\mathrm{dupl}(gf)|. \tag{3.13}$$

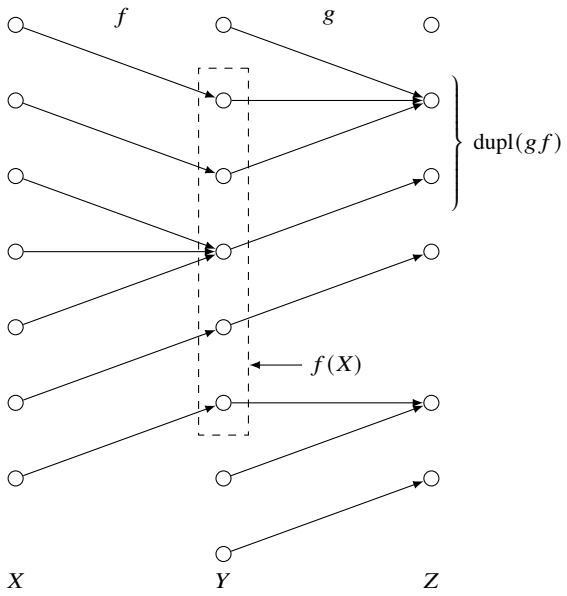


Fig. 3.1: An illustration for the identities from Theorems 3.4 and 3.6.

Corollary 3.5 *Let $f\colon X \to Y$, $g\colon Y \to Z$, and $S \subseteq X$ with $f(X)\backslash \mathrm{dupl}(f) \subseteq S \subseteq f(X)$. Then the following are equivalent.*

1. $\mathrm{dupl}(gf) \subseteq g(\mathrm{dupl}(f))$
2. $\mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$
3. $\mathrm{dupl}(g_{|S}) \subseteq g(\mathrm{dupl}(f))$
4. $g^{-1}(\mathrm{dupl}(g_{|S})) \subseteq g^{-1}(g(\mathrm{dupl}(f)))$
5. $S \cap g^{-1}(\mathrm{dupl}(g_{|S})) \subseteq g^{-1}(g(\mathrm{dupl}(f)))$
6. *For all $z \in \mathrm{dupl}(g)$, if $S \cap g^{-1}(z)$ contains two distinct elements, then $z \in g(\mathrm{dupl}(f))$.*
7. *For all $z \in \mathrm{dupl}(g)$, if $f(X) \cap g^{-1}(z)$ contains two distinct elements, then it contains an element from $\mathrm{dupl}(f)$, i.e., $y, y' \in f(X) \cap g^{-1}(z)$ for $y \neq y'$ implies $z \in g(\mathrm{dupl}(f))$.*

Proof. $1 \Rightarrow 2$: We have $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(gf)$ by Equation (3.11), giving the claim.

$2 \Rightarrow 3$: With Equation (3.10) we have $\mathrm{dupl}(g_{|S}) \subseteq \mathrm{dupl}(g_{|f(X)}) \subseteq \mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$.

$3 \Rightarrow 4$: This is clear.

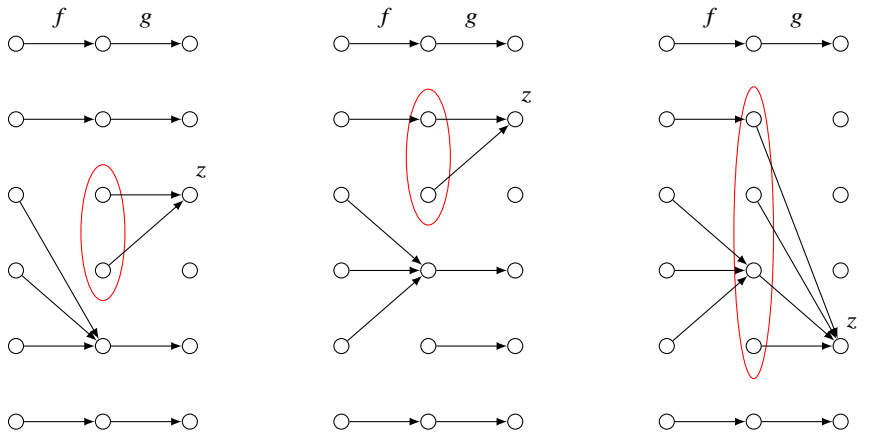
$4 \Rightarrow 5$: This is clear.

$5 \Rightarrow 6$: Let $z \in \mathrm{dupl}(g)$ such that $x, y \in S \cap g^{-1}(z)$ for distinct $x, y \in Y$. Then we have $z \in \mathrm{dupl}(g_{|S})$ and so $x, y \in S \cap g^{-1}(\mathrm{dupl}(g_{|S}))$. By assumption, we get $x, y \in g^{-1}(g(\mathrm{dupl}(f)))$. Thus we find $z = g(x) \in g(\mathrm{dupl}(f))$, which gives the claim.

$6 \Rightarrow 7$: Let $f(x), f(x') \in g^{-1}(z)$ with $f(x) \neq f(x')$ for $x, x' \in X$. If $f(x), f(x') \in S$, then, by assumption, $z = g(y)$ for some $y \in \mathrm{dupl}(f)$. We have $y \in f(X) \cap g^{-1}(z)$.

Otherwise, without loss of generality, let $f(x) \notin S$. Then, since $f(X) \setminus \mathrm{dupl}(f) \subseteq S$ by choice of S, we have $f(x) \in \mathrm{dupl}(f)$.

$7 \Rightarrow 1$: Let $z \in \mathrm{dupl}(g_{|f(X)})$. Then $f(X) \cap g^{-1}(z)$ contains two elements. Thus, by assumption, we have $z \in g(\mathrm{dupl}(f))$. This shows $\mathrm{dupl}(g_{|f(X)}) \subseteq g(\mathrm{dupl}(f))$. Then, with Equation (3.10), we find $\mathrm{dupl}(gf) \subseteq g(\mathrm{dupl}(f))$. □



(a) $f(X) \cap g^{-1}(z) = \emptyset$ (b) $f(X) \cap g^{-1}(z)$ singleton (c) $\mathrm{dupl}(f) \cap g^{-1}(z) \neq \emptyset$

Fig. 3.2: Cases that, when fulfilled for all $z \in \mathrm{dupl}(g)$, yield $\mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$ by Item 7 of Theorem 3.5. The set $g^{-1}(z)$ is indicated by a red-colored elliptical shape.

Figure 3.2 depicts the following three cases: $f(X) \cap g^{-1}(z)$ is empty, $f(X) \cap g^{-1}(z)$ is a singleton, or $f(X) \cap g^{-1}(z)$ contains two distinct elements and an element from $\mathrm{dupl}(f)$. If for all $z \in \mathrm{dupl}(g)$ one of these cases holds, then $\mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$ by Item 7 from Theorem 3.5.

Proposition 3.6 *Let $f\colon X \to Y$ and $g\colon Y \to Z$. Then*

$$(gf)^{-1}(\mathrm{dupl}(gf)) = f^{-1}(\mathrm{dupl}(f)) \cup (gf)^{-1}(\mathrm{dupl}(g_{|f(X)})).$$

Proof. By Equation (3.8) and as $g(\mathrm{dupl}(f)) = g_{|f(X)}(\mathrm{dupl}(f))$ we have

$$g^{-1}_{|f(X)}(g(\mathrm{dupl}(f))) = \mathrm{dupl}(f) \cup g^{-1}_{|f(X)}(g(\mathrm{dupl}(f)) \cap \mathrm{dupl}(g_{|f(X)})). \tag{3.14}$$

With $g^{-1}_{|f(X)}(g(\mathrm{dupl}(f))) = g^{-1}(g(\mathrm{dupl}(f))) \cap f(X)$ this then gives

$$\begin{aligned}
&f^{-1}(g^{-1}(\mathrm{dupl}(gf))) \\
&= f^{-1}(g^{-1}(g(\mathrm{dupl}(f))) \cup f^{-1}(g^{-1}(\mathrm{dupl}(g_{|f(X)})))) && \text{(Eqn. (3.10))} \\
&= f^{-1}(g^{-1}(g(\mathrm{dupl}(f)) \cap f(X)) \cup f^{-1}(g^{-1}(\mathrm{dupl}(g_{|f(X)})))) \\
&= f^{-1}(\mathrm{dupl}(f)) \cup f^{-1}(g^{-1}(\mathrm{dupl}(g_{|f(X)})))) && \text{(Eqn. (3.14))}
\end{aligned}$$

This shows the claim. □

The next example shows that we cannot replace $g_{|f(X)}$ by g in Theorem 3.6.

Example 3.7 Let $X = \{1,2,3,4\}$ and $f, g\colon X \to X$ with $f(1) = 1, f(2) = f(3) = 2, f(4) = 4$ and $g(1) = g(2) = 1, g(3) = g(4) = 3$. Then

1. $\mathrm{dupl}(gf) = \{1\}$ and $\mathrm{dupl}(g) = \{1,3\}$,
2. $f^{-1}(\mathrm{dupl}(f)) = \{2,3\}$ and $g^{-1}(\mathrm{dupl}(g)) = X$,
3. $\mathrm{dupl}(g_{|\{1,2,4\}}) = \{1\}$,
4. $(gf)^{-1}(\mathrm{dupl}(gf)) = \{1,2,3\}$.

Here $f^{-1}(g^{-1}(\mathrm{dupl}(g))) = X$ properly contains $(gf)^{-1}(\mathrm{dupl}(gf))$.

Proposition 3.8 *Let $f\colon X \to Y$, $g\colon Y \to Z$, and $S, T, R \subseteq Y$ with $f(X) \setminus \mathrm{dupl}(f) \subseteq S$, $\mathrm{dupl}(f) \subseteq T$, and $R \subseteq f(X)$. Then we have the following:*

1. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|S})$ *if and only if* $\mathrm{dupl}(gf) \subseteq \mathrm{dupl}(g_{|S})$.
2. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|R})$ *implies* $\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|f(X)})$.
3. $\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|R})$ *implies* $\mathrm{dupl}(g_{|R}) = \mathrm{dupl}(g_{|f(X)})$.
4. $\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|R})$ *if and only if* $\mathrm{dupl}(gf) \subseteq \mathrm{dupl}(g_{|R})$.
5. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|T})$ *if and only if for all $y \in \mathrm{dupl}(f)$ there exists $y' \in T$ with $y \neq y'$ and $g(y) = g(y')$.*

Proof. 1: Since $\mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)}) \subseteq \mathrm{dupl}(g_{|S})$, this follows from Equation (3.11).

2: This follows from Equation (3.10) with $\mathrm{dupl}(g_{|R}) \subseteq \mathrm{dupl}(g_{|f(X)})$.

3: Since $R \subseteq f(X)$ we have $\mathrm{dupl}(g_{|R}) \subseteq \mathrm{dupl}(g_{|f(X)})$. By assumption and with Equation (3.10), we find $\mathrm{dupl}(g_{|f(X)}) \subseteq \mathrm{dupl}(gf) = \mathrm{dupl}(g_{|R})$.

4: This follows since $\mathrm{dupl}(g_{|R}) \subseteq \mathrm{dupl}(g_{|f(X)}) \subseteq \mathrm{dupl}(gf)$ with Equation (3.10).

5: First, suppose $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|T})$. Consider $y \in \mathrm{dupl}(f)$. Then the assumption gives $g(y) \in \mathrm{dupl}(g_{|T})$. Thus, there exist distinct $y', y'' \in T$ with $g(y) = g(y') = g(y'')$ and we have $y \neq y'$ or $y \neq y''$, which gives the claim.

Conversely, suppose the other condition holds. Let $y \in \mathrm{dupl}(f)$. Then $g(y) = g(y')$ for some $y' \in T \setminus \{y\}$. Since $\mathrm{dupl}(t) \subseteq T$ we have $y, y' \in T$. So $g(y) \in \mathrm{dupl}(g_{|T})$. □

The condition $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)})$ implies $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|f(X)})$ by Equation (3.4), but the converse does not hold, as shown by the following example. By Equation (3.4) the difference between $\mathrm{dupl}(g_{|f(X)})$ and $\mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)})$ is the set $\mathrm{dupl}(g_{|\,\mathrm{dupl}(f)}) \cup S$ with $S = g(\mathrm{dupl}(f)) \cap g(f(X) \setminus \mathrm{dupl}(f))$.

Example 3.9 Let $X = \{1,2,3\}$ and $Y = \{2,3\}$. Consider $f\colon X \to Y$ with $f(1) = f(2) = 2$ and $f(3) = 3$, and $g\colon Y \to \{3\}$ with $g(2) = g(3) = 3$. Then $g(\mathrm{dupl}(f)) = \{3\}$, $\mathrm{dupl}(g_{|f(X)}) = \{3\}$, and $\mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)}) = \emptyset$. So

1. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|f(X)})$, but
2. $g(\mathrm{dupl}(f)) \not\subseteq \mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)})$.

Corollary 3.10 *Let $f\colon X \to Y$ and $g\colon Y \to Z$. Then the following are equivalent:*

1. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|f(X)})$.

2. $\operatorname{dupl}(gf) \subseteq \operatorname{dupl}(g_{|f(X)})$.
3. $\operatorname{dupl}(gf) = \operatorname{dupl}(g_{|f(X)})$.
4. *For all* $y \in \operatorname{dupl}(f)$ *exists* $y' \in f(X)$ *with* $y \neq y'$ *such that* $g(y) = g(y')$.
5. $g(\operatorname{dupl}(f)) \subseteq \operatorname{dupl}(g_{|\operatorname{dupl}(f)}) \cup (g(\operatorname{dupl}(f)) \cap g(f(X) \setminus \operatorname{dupl}(f)))$.

Proof. The first four equivalences follow from Theorem 3.8 by considering $S = T = R = f(X)$.

$1 \Leftrightarrow 5$: First, let $g(f(x))$ for some $x \in X$ such that $f(x) \in \operatorname{dupl}(f)$. By Item 1 there exists $x' \in X$ such that $f(x) \neq f(x')$ and $g(f(x)) = g(f(x'))$. If $f(x') \in \operatorname{dupl}(f)$, then $g(f(x)) \in \operatorname{dupl}(g_{|\operatorname{dupl}(f)})$. Otherwise, we have $g(f(x)) \in g(f(X) \setminus \operatorname{dupl}(f))$. This gives Item 5.

Conversely, first not that by Equation (3.4) we have

$$\operatorname{dupl}(g_{|\operatorname{dupl}(f)}) \cup (g(\operatorname{dupl}(f)) \cap g(f(X) \setminus \operatorname{dupl}(f))) \subseteq \operatorname{dupl}(g_{|f(X)}).$$

Thus Item 5 implies Item 1. □

Note that $g(\operatorname{dupl}(f)) \subseteq \operatorname{dupl}(g)$ is not equivalent to $g(\operatorname{dupl}(f)) \subseteq \operatorname{dupl}(g_{|f(X)})$ as demonstrated by the next example.

Example 3.11 Let $X = \{1, 2\}$ with $f\colon X \to X$ with $f(1) = f(2)$. Then $\operatorname{dupl}(f) = \{f(1)\}$, $f(\operatorname{dupl}(f)) = \{f(1)\}$, and $\operatorname{dupl}(f_{|f(X)}) = \emptyset$. So

1. $f(\operatorname{dupl}(f)) \subseteq \operatorname{dupl}(f)$, but
2. $f(\operatorname{dupl}(f)) \not\subseteq \operatorname{dupl}(f_{|f(X)})$.

3.2.3 Functions on a Finite Set

Let X, Y be finite sets and $f\colon X \to Y$ be a function. Then the *defect* (resp. *rank*) of f is $\operatorname{def}(f) = |\operatorname{excl}(f)|$ (resp. $\operatorname{rank}(f) = |f(X)|$). As $Y = f(X) \uplus \operatorname{excl}(f)$, we have

$$|Y| = \operatorname{rank}(f) + \operatorname{def}(f). \tag{3.15}$$

For $A \subseteq Y$ we call $\operatorname{exc}_f(A) = |f^{-1}(A)| - |A|$ the *excess of* A *under* f. As $|f^{-1}(y)| = 0$ if and only if $y \in \operatorname{excl}(f)$, and $|f^{-1}(y)| \geq 2$ if and only if $y \in \operatorname{dupl}(f)$, we have, for $y \in Y$,

$$\operatorname{exc}_f(\{y\}) \begin{cases} = -1 & \text{if } y \in \operatorname{excl}(f), \\ = 0 & \text{if } y \in f(X) \setminus \operatorname{dupl}(f), \\ \geq 1 & \text{if } y \in \operatorname{dupl}(f). \end{cases} \tag{3.16}$$

The *kernel type* [34] of f is the ordered r-tuple with $r = \operatorname{rank}(f)$ given by the sizes of the non-empty inverse images $f^{-1}(y)$ for $y \in Y$, i.e., the partition of $|X|$ into these sizes. For example, f has kernel type $(k, 1, \ldots, 1)$ for $k \geq 1$ if and only if

1. $|f^{-1}(y)| = k$ for some $y \in Y$ and
2. $|f^{-1}(z)| \leq 1$ for all $z \in Y \setminus \{y\}$.

Clearly, for $S \subseteq X$ we have

$$|f(S)| \leq |S|, \tag{3.17}$$

and $|f(S)| = |S|$ if and only if $f_{|S}$ is injective.

Proposition 3.12 *Let $f\colon X \to Y$ and $g\colon Y \to Z$ for finite sets X, Y, and Z. Then*

$$\operatorname{rank}(gf) \leq \min\{\operatorname{rank}(f), \operatorname{rank}(g)\}, \tag{3.18}$$

where

1. $\operatorname{rank}(gf) = \operatorname{rank}(f)$ *if and only if $g_{|f(X)}$ is injective, and*
2. $\operatorname{rank}(gf) = \operatorname{rank}(g)$ *if and only if $g(Y) = g(f(X))$.*

Proof. We have $\operatorname{rank}(gf) = |g(f(X))|$. By Equation (3.17), $|g(f(X))| \leq |f(X)| = \operatorname{rank}(f)$ with equality if and only if $g_{|f(X)}$ is injective. Furthermore, $g(f(X)) \subseteq g(Y)$ gives $|g(f(X))| \leq \operatorname{rank}(g)$, and, as these sets are finite, equality of the cardinalities holds if and only if $g(f(X)) = g(Y)$. □

Proposition 3.13 *Let $f\colon X \to Y$ with X, Y finite. Then the following holds:*[3]

1. *For $A \subseteq Y$ we have*

$$\operatorname{exc}_f(A) = \sum_{y \in A} (|f^{-1}(y)| - 1), \tag{3.19}$$

$$\operatorname{exc}_f(A) = \operatorname{exc}_f(A \cap \operatorname{dupl}(f)) - |A \cap \operatorname{excl}(f)|. \tag{3.20}$$

2. *For disjoint $A, B \subseteq Y$ we have* $\operatorname{exc}_f(A \uplus B) = \operatorname{exc}_f(A) + \operatorname{exc}_f(B)$.
3. $\operatorname{exc}_f(A) \geq |A|$ *for $A \subseteq \operatorname{dupl}(f)$.*
4. $\operatorname{exc}_f(A) \leq \operatorname{exc}_f(B)$ *for $A \subseteq B \subseteq f(X)$. Moreover, we have*

$$\operatorname{exc}_f(A) < \operatorname{exc}_f(B) \Leftrightarrow (B \setminus A) \cap \operatorname{dupl}(f) \neq \emptyset.$$

5. $\operatorname{exc}_f(f(X)) = \operatorname{exc}_f(\operatorname{dupl}(f)) = |X| - \operatorname{rank}(f) = \sum_{i=1}^{r}(k_i - 1)$ *where f has kernel type $(k_1, \ldots, k_r)$.*
6. *For $A \subseteq Y$ we have*

$$|A \cap \operatorname{excl}(f)| < |A \cap \operatorname{dupl}(f)| \Rightarrow \operatorname{exc}_f(A) > 0. \tag{3.21}$$

Proof. 1: As $f^{-1}(A) = \biguplus_{y \in A} f^{-1}(y)$, we have $|f^{-1}(A)| = \sum_{y \in A} |f^{-1}(y)|$, which gives Equation (3.19) with $|A| = \sum_{y \in A} 1$. Splitting the index set A in the sum from Equation (3.19) into $A \cap \operatorname{excl}(f)$, $A \cap (f(X) \setminus \operatorname{dupl}(f))$, and $A \cap \operatorname{dupl}(f)$, whose summands are -1, 0, and those of $\operatorname{exc}_f(A \cap \operatorname{dupl}(f))$ by Equation (3.16), gives Equation (3.20).

2: This holds as the index sets A and B in Equation (3.19) are disjoint.

3: This follows since for $A \subseteq \operatorname{dupl}(f)$ every summand of Equation (3.19) is at least one by Equation (3.16).

[3] By convention, the summation over an empty index set equals zero.

4: By Item 2 we have $\operatorname{exc}_f(B) = \operatorname{exc}_f(A) + \operatorname{exc}_f(B \setminus A)$. Since $B \setminus A \subseteq f(X)$, Equation (3.20) gives $\operatorname{exc}_f(B \setminus A) = \operatorname{exc}_f((B \setminus A) \cap \operatorname{dupl}(f))$, which is at least $|(B \setminus A) \cap \operatorname{dupl}(f)|$ by Item 3 and equals zero if $(B \setminus A) \cap \operatorname{dupl}(f) = \emptyset$.

5: Equation (3.20) for $A = f(X)$ gives $\operatorname{exc}_f(f(X)) = \operatorname{exc}_f(\operatorname{dupl}(f))$, as $\operatorname{dupl}(f) \subseteq f(X)$. As $f^{-1}(f(X)) = X$, we have $\operatorname{exc}_f(f(X)) = |X| - \operatorname{rank}(f)$. Since the non-empty inverse images are the sets $f^{-1}(y)$ for $y \in f(X)$, the last equality is Equation (3.19) for $A = f(X)$.

6: By Item 3 the assumption gives $|A \cap \operatorname{excl}(f)| < |A \cap \operatorname{dupl}(f)| \leq \operatorname{exc}_f(A \cap \operatorname{dupl}(f))$, so Equation (3.20) yields $\operatorname{exc}_f(A) > 0$. □

For functions between sets of equal size, the excess of the duplicate set is precisely the defect.

Proposition 3.14 *Let $f\colon X \to Y$ with X, Y finite and $|X| = |Y|$. Then the following holds:*

1. $\operatorname{def}(f) = \operatorname{exc}_f(\operatorname{dupl}(f)) = |f^{-1}(\operatorname{dupl}(f))| - |\operatorname{dupl}(f)|$,
2. $\operatorname{def}(f) \geq \operatorname{exc}_f(A)$ *for every* $A \subseteq Y$,
3. $\operatorname{def}(f) \geq |\operatorname{dupl}(f)|$.

Proof. 1: By Item 5 of Theorem 3.13, Equation (3.15), and $|X| = |Y|$, we have $\operatorname{exc}_f(\operatorname{dupl}(f)) = |X| - \operatorname{rank}(f) = |Y| - \operatorname{rank}(f) = \operatorname{def}(f)$.

2: For $A \subseteq Y$, Equation (3.20) gives $\operatorname{exc}_f(A) \leq \operatorname{exc}_f(A \cap \operatorname{dupl}(f))$, which is at most $\operatorname{exc}_f(\operatorname{dupl}(f)) = \operatorname{def}(f)$ by Item 4 of Theorem 3.13 and Item 1.

3: Combine Item 1 with Item 3 of Theorem 3.13 for $A = \operatorname{dupl}(f)$. □

Corollary 3.15 *Let $f\colon X \to Y$ with $|X| = |Y|$ and $\operatorname{def}(f) = 1$. Then $|\operatorname{dupl}(f)| = 1$ and $|f^{-1}(\operatorname{dupl}(f))| = 2$.*

Proof. By Item 3 of Theorem 3.14 we have $|\operatorname{dupl}(f)| \leq 1$, and $\operatorname{dupl}(f) \neq \emptyset$, as otherwise Item 1 of Theorem 3.14 would give $\operatorname{def}(f) = \operatorname{exc}_f(\emptyset) = 0$. So $|\operatorname{dupl}(f)| = 1$, and then the same item yields $|f^{-1}(\operatorname{dupl}(f))| = \operatorname{def}(f) + |\operatorname{dupl}(f)| = 2$. □

3.3 Transformations

A *transformation (on X)*, or *endofunction*, is a function $t\colon X \to X$ of a set X into itself, i.e., a function whose domain and codomain are the same. Let $i \geq 0$. The *i-fold composition*, or *i-th power*, is defined by $t^i = \mathrm{id}_X$ for $i = 0$ and $t^i = t^{i-1}t$ for $i > 0$.

In the context of transformations, the elements of X are also called *points*. We call $x \in X$ a *cycle point* if $t^p(x) = x$ for some $p > 0$. Clearly, cycle points are contained in the image $t(X)$. A point that is not a cycle point is called a *tail point*. A *root point* is a cycle point contained in $\operatorname{dupl}(t)$. We set $\operatorname{fix}(t) = \{x \in X \mid t(x) = x\}$ and $\operatorname{move}(t) = X \setminus \operatorname{fix}(t) = \{x \in X \mid t(x) \neq x\}$. A point $x \in X$ is said to be *fixed* (resp. *moved*) if $x \in \operatorname{fix}(t)$ (resp. $x \in \operatorname{move}(t)$). A fixed point is also called a *fixpoint*.

Let $t, h\colon X \to X$ and $A \subseteq X$. The following formulas are easy to verify.

$$\operatorname{fix}(t) \cap t(\operatorname{move}(t)) \subseteq \operatorname{dupl}(t) \subseteq t(\operatorname{move}(t)) \tag{3.22}$$

$$\operatorname{move}(t) \cap t^{-1}(\operatorname{fix}(t)) \subseteq \operatorname{dupl}(t) \tag{3.23}$$

$$\operatorname{fix}(t) \subseteq t(X) \tag{3.24}$$

$$\operatorname{excl}(t) \subseteq \operatorname{move}(t) \tag{3.25}$$

$$\operatorname{fix}(t) \cap \operatorname{fix}(h) = \operatorname{fix}(t) \cap \operatorname{fix}(ht) \tag{3.26}$$

$$t^{-1}(A) = (\operatorname{move}(t) \cap t^{-1}(A)) \cup (\operatorname{fix}(t) \cap A) \tag{3.27}$$

3.4 Relations

A *relation* R is a tuple $(R_G, X_1, \ldots, X_n)$ such that $R_G \subseteq X_1 \times \cdots \times X_n$. For $n = 2$, we call R a *binary relation* and in this case, for $(x_1, x_2) \in X_1 \times X_2$ we write $x_1 R x_2$ if and only if $(x_1, x_2) \in R_G$.

For $X = X_1 = X_2$ we say that R is a relation over X (such relations are also called *homogeneous*). If R is a relation over X, then it is called

reflexive if for all $x \in X$ we have xRx,
symmetric if for all $x, y \in X$ we have that xRy implies yRx,
antisymmetric if for all $x, y \in X$ we have that xRy and yRx imply $x = y$,
transitive if for all $x, y, z \in X$ we have that xRy and yRz imply xRz,
equivalence relation if it is reflexive, symmetric and transitive.

If $\sim$ denotes an equivalence relation[4] on X and $x \in X$, the *equivalence class of* x is

$$[x]_\sim = \{y \in X \mid y \sim x\}.$$

It is $[x]_\sim = [y]_\sim$ if and only if $x \sim y$ for $x, y \in X$. We write $X/\sim = \{[x]_\sim \mid x \in X\}$ for the set of equivalence classes. The surjective map $\pi_\sim \colon X \to X/\sim$ given by

$$\pi_\sim(x) = [x]_\sim \tag{3.28}$$

for $x \in X$ is called the *canonical projection map (for $\sim$)*.

The equivalence classes partition the set X. Conversely, given a partition $\{X_i \mid i \in I\}$ of X for some index set I, the *equivalence relation induced by it* is given by setting $x, y \in X$ to be equivalent if $\{x, y\} \subseteq X_i$ for some $i \in I$.

For a function $f \colon X \to Y$ the non-empty preimages $f^{-1}(y)$, $y \in Y$, partition X. The equivalence relation $\sim_f$ induced by this partition is called the *kernel relation (of f)*. We have $x \sim_f y$ if and only if $f(x) = f(y)$. The kernel relation of the canonical projection $\pi_\sim$ from above is $\sim$.

We say that an equivalence relation $\sim$ on X *saturates* a subset $A \subseteq X$ if and only if $x \sim y$ and $x \in A$ imply $y \in A$, or equivalently A is union of equivalence classes.

Proposition 3.16 *Let $f \colon X \to Y$ and $A \subseteq X$. Then the following are equivalent:*

[4] Using symbols like $\sim$ for equivalence relations is more common than denoting them by R.

1. *The kernel relation $\sim_f$ saturates A,*
2. $f^{-1}(f(A)) \subseteq A$,
3. $f^{-1}(f(A)) = A$.

Proof. 1 $\Rightarrow$ 2: That $\sim_f$ saturates A is equivalent to $f^{-1}(f(x)) \subseteq A$ for each $x \in A$. So $f^{-1}(f(A)) = f^{-1}(f(\bigcup_{x\in A}\{x\})) \subseteq A$.

2 $\Rightarrow$ 3: This follows from Equation (3.8).

3 $\Rightarrow$ 1: If $x \sim_f y$ for some $x \in A$ and $y \in X$, then $y \in f^{-1}(f(x)) \subseteq f^{-1}(f(A)) = A$. □

The *trivial equivalence relations* are $X \times X$ (the *universal relation*) and $\{(x, x) \mid x \in X\}$ (the *equality relation*).

For two equivalence relations $\sim$ and $\approx$ on X such that $x \sim y$ implies $x \approx y$ for all x, y, the relation $\sim$ (resp. $\approx$) is said to be *finer* (resp. *coarser*) than $\approx$ (resp. $\sim$). The *equivalence relation generated* by a given relation R over X is the finest equivalence relation $\sim$ such that xRy implies $x \sim y$ for all $x, y \in X$, i.e., the unique relation such that

1. xRy implies $x \sim y$ for all $x, y \in X$ and
2. for each equivalence relation $\equiv$ over X such that xRy implies $x \equiv y$ for all $x, y \in X$ we have that $x \sim y$ implies $x \equiv y$ for all $x, y \in X$.

Remark 3.17 In the literature, a relation R from X to Y is often just defined as a subset of $X \times Y$. However, as pointed out in [194, p. 2], under the usual identification of $(X \times Y) \times Z$ and $X \times (Y \times Z)$, a subset of $X \times Y \times Z$ can be interpreted as a relation from $X \times Y$ to Z or from X to $Y \times Z$, and both interpretations may yield relations with quite different properties. Another reason is that there might be (type- or category-theoretical) contexts when relations $(R, \emptyset, X)$ and $(R', \emptyset, Y)$ for $X \neq Y$ should be treated differently. Note that for homogeneous relations, these issues do not arise and the relation can indeed be identified with its set of tuples.

Remark 3.18 In [194] a relation from X to Y is modeled as a function $f\colon X \to \mathcal{P}(Y)$, which can be identified with functions $f\colon \mathcal{P}(X) \to \mathcal{P}(Y)$ with the property $f(\bigcup_{i\in I} S_i) = \bigcup_{i\in I} f(S_i)$ for $S_i \in \mathcal{P}(X)$ with $i \in I$. Note that, by the identities from Section 3.2.1, the extension of a function $X \to Y$ to subsets fulfills this equation.

3.5 Algebraic Notions

Here, in Section 3.5.1, we first introduce semigroups, monoids, groups, semirings and related notions. Section 3.5.2 introduced matrices.

3.5.1 Semigroups, Monoids, Groups, and Semirings

Here, we introduce the algebraic structures of relevance to us.

1. A *semigroup* is a tuple $(S, \cdot)$ where S is a set and $\cdot\colon S \times S \to S$ a binary operation, called a *product* and written as $x \cdot y$ (infix notation) for $x, y \in S$, that is *associative*, i.e., $x \cdot (y \cdot z) = (x \cdot y) \cdot z$ for $x, y, z \in S$. Often, when it is clear from context what operation $S \times S \to S$ is meant, we simply call the set S a semigroup, i.e., omit the tuple notation, and we write xy instead of $x \cdot y$, i.e., we omit the operation symbol.
2. A *monoid* is a tuple $(M, \cdot, 1)$ where $(M, \cdot)$ is a semigroup and $1 \in M$ is an *identity element*, i.e., $1x = x1 = x$ for all $x \in M$. The identity element is unique, for if $1, 1' \in M$ are identity elements, then $1 = 1 \cdot 1' = 1'$.
3. A *group* is a tuple $(G, \cdot, ^{-1}, 1)$ where $(G, \cdot, 1)$ is a monoid and $^{-1}\colon G \to G$ is a unary operation, the *inverse operation*, written as g^{-1} instead of $^{-1}(g)$ for $g \in G$, such that $gg^{-1} = g^{-1}g = 1$ for all $g \in G$. We call g^{-1} the *inverse element* of $g \in G$, which is the unique element $h \in G$ such that $hg = gh = 1$.
4. A *semiring* is a tuple $(R, +, \cdot, 0, 1)$ where $(R, +, 0)$ and $(R, \cdot, 1)$ are monoids and the following additional equations are true:

   a. $x + y = y + x$ (*commutativity*),
   b. $x \cdot 0 = 0 \cdot x = 0$ (*annihilation law* for 0 in R),
   c. $x \cdot (y + z) = x \cdot y + x \cdot z$ (*left-distributivity*) and $(y + z) \cdot x = y \cdot x + z \cdot x$ (*right-distributivity*).

   The operation $+$ is called *addition* in R and $\cdot$ is called *multiplication* in R.

Due to associativity, we omit parentheses in expressions over semigroups.

Example 3.19 1. $(\Sigma^+, \cdot)$ for some alphabet Σ is a semigroup.
2. $(\Sigma^*, \cdot, \varepsilon)$ for some alphabet Σ is a monoid.
3. $(\mathbb{N}_0, +, 0)$ and $(\mathbb{N}_0, \cdot, 1)$ are monoids with the usual addition and multiplication operations.
4. $([n], +_n, 0)$ and $([n], \cdot_n, 1)$ are monoids for $n \geq 0$ with $x +_n y = \min(x + y, n - 1)$ and $x \cdot_n y = \min(x \cdot y, n - 1)$ for $x, y \in [n]$.
5. The integers $\mathbb{Z}$ with the usual addition, inversion $x \mapsto -x$ and identity element $0 \in \mathbb{Z}$ form a group.
6. The *Boolean semiring* is the semiring $(\{0, 1\}, \wedge, \vee, 0, 1)$.
7. $(\mathbb{N}_0, +, \cdot, 0, 1)$ and $([n], +_n, \cdot_n, 0, 1)$ for $n \geq 1$ are semirings. For $n = 2$ the latter semiring equals the Boolean semiring.

Let $(S_1, \cdot_1), (S_2, \cdot_2)$ be two semigroups. A *(semigroup) morphism* is a map $\varphi\colon S_1 \to S_2$ such that $\varphi(xy) = \varphi(x)\varphi(y)$ for all $x, y \in S_1$. We call φ an *isomorphism* if there exists a semigroup morphism $\psi\colon S_2 \to S_1$ such that $\psi\varphi = \mathrm{id}_{S_1}$ and $\varphi\psi = \mathrm{id}_{S_2}$. If φ is bijective, for its inverse function $\varphi^{-1}\colon S_2 \to S_1$ we have $\varphi^{-1}(\varphi(x)\varphi(y)) = \varphi^{-1}(\varphi(xy)) = xy = \varphi^{-1}(\varphi(x))\varphi^{-1}(\varphi(y))$ for $x, y \in S_1$. This yields that φ^{-1} is a semigroup morphism. With Theorem 3.3 we find that the semigroup isomorphisms are precisely the bijective morphisms. We say that S_1 is a *subsemigroup* of S_2, written $S_1 \leq_S S_2$, if $S_1 \subseteq S_2$ and, for all $x, y \in S_1$, $x \cdot_2 y \in S_1$ and $x \cdot_1 y = x \cdot_2 y$. For $i > 0$ and $x \in S_1$ we set $x^i = x \cdots x$ (i times). Let $A, B \subseteq S_1$. The *product on subsets of* S_1 (also called *complex product* in [393]) is $A \cdot B = \{ab \mid a \in A, b \in B\}$, written AB for short. This gives a semigroup $(\mathcal{P}(S_1), \cdot)$. Also, we set $aA = \{a\}A$ and $Aa = A\{a\}$ for $a \in S_1$. We call S_1

commutative if $xy = yx$ for all $x, y \in S_1$. An element $x \in S_1$ is called *idempotent* if $x^2 = x$.

Lemma 3.20 *Let S be a finite semigroup. Then for each $s \in S$ there exists $i > 0$ such that s^i is idempotent.*

Proof. By finiteness, we have $s^{j+p} = s^j$ for some $j \geq 0$ and $p > 0$. Let $i \geq 0$ be such that $j \leq i < j + p$ and $i \equiv 0 \pmod{p}$. Then $(s^i)^2 = s^{i+i} = s^i$. □

Remark 3.21 Let $(S, \cdot)$ be a semigroup. By adding some element $1 \notin S$ and setting $1 \cdot 1 = 1$ and $1 \cdot s = s \cdot 1 = s$ for all $s \in S$, we obtain a monoid $S \cup \{1\}$.

Let M_1, M_2 be two monoids with identity elements 1_1 and 1_2, respectively. We set $x^0 = 1_1$ for $x \in M_1$. A *monoid morphism* (resp. *monoid isomorphism*) is a semigroup morphism (resp. semigroup isomorphism) $\varphi\colon M_1 \to M_2$ with $\varphi(1_1) = 1_2$. When the context is clear, we simply say morphism (resp. isomorphism) instead of semigroup or monoid morphism (resp. isomorphism). We call M_1 a *submonoid* of M_2, written $M_1 \leq_M M_2$, if M_1 is a subsemigroup of M_2 with $1_2 \in M_1$. The *group of units* of M_1 is

$$\{g \in M_1 \mid \exists h \in M_1 : gh = hg = 1\}.$$

The group of units is in fact a group. Further, for $A \subseteq M_1$, we set $A^0 = \{1\}$, $A^{\leq n} = A^0 \cup A \cup A^2 \cup \ldots \cup A^n$ for $n \geq 0$, $A^{<0} = \emptyset$ and $A^{<n} = A^{\leq n-1}$ for $n > 0$. For $a \in M_1$ we also write $a^{<n}$ and $a^{\leq n}$ for $\{a\}^{<n}$ and $\{a\}^{\leq n}$, respectively.

Remark 3.22 In monoids, there might exist subsemigroups that are monoids in itself, but not submonoids. For example, in $\{1, s, s^2\}$ with $s^2 = s^3$ and $s1 = 1s = s$, $\{s^2\}$ is a subsemigroup that is a monoid, but it is not a submonoid.

Example 3.23 Let M be a monoid with identity $1 \in M$. Then we obtain a semiring structure $(\mathcal{P}(M), \cup, \cdot, \emptyset, 1)$ from the product on subsets and set union.

Let G_1, G_2 be two groups and $1 \in G_1$ be the identity in G_1. We call G_1 *trivial* if $G_1 = \{1\}$. A *group morphism* $\varphi\colon G_1 \to G_2$ is a monoid morphism such that $\varphi(g^{-1}) = \varphi(g)^{-1}$ for all $g \in G_1$. Now, for every semigroup morphism $\psi\colon G_1 \to G_2$ we have $\psi(1) = \psi(1 \cdot 1) = \psi(1)\psi(1)$, which implies that $\psi(1)$ is the identity in G_2, and $\psi(g)\psi(g^{-1}) = \psi(gg^{-1}) = \psi(1)$, so that $\psi(g)^{-1} = \psi(g^{-1})$. Thus, the group morphisms are precisely the semigroup morphisms and, technically, the definition of group morphisms can be omitted (we included it for conceptual reasons). We say that G_1 is a *subgroup* of G_2, written $G_1 \leq_G G_2$, if G_1 is a submonoid of G_2 and the inverse operation in G_1 agrees with the inverse operation in G_2 for each element of G_1, i.e., inversion in G_1 equals the inversion in G_2 restricted to G_1.[5] Next, assume $G_1 \leq_G G_2$.

1. A subset of the form $G_1 g$ (resp. gG_1) for some $g \in G_2$ is called a *right coset* (resp. *left coset*) of G_1 in G_2. A right coset is also just called coset. The set of right cosets (resp. left cosets) of G_1 forms a partition of G_2. If G_2 is finite, then each right coset (resp. left coset) has size $|G_1|$.

[5] Here, we use the term subgroup only w.r.t. groups. In the literature, for example [449], subgroups are also defined for semigroups as subsemigroups having an identity element (that does not have to be an identity for the whole semigroup) and containing inverses w.r.t. this identity.

2. We say that G_1 is a *maximal subgroup* of G_2 if $G_1 \neq G_2$ and for every subgroup H of G_2 with $G_1 \subseteq H$ we have $H = G_1$ or $H = G_2$.
3. We say that G_1 is a *normal subgroup* of G_2 if $gG_1g^{-1} \subseteq G_1$ for all $g \in G_1$.

A group G is said to be *simple* if the only normal subgroups of G are $\{1\}$ and G. The *center* of G is

$$Z(G) = \{g \in G \mid \forall h \in G\colon hg = gh\}.$$

We call G *cyclic* if $G = \langle\{g\}\rangle_G$ for some $g \in G$. A commutative group is also often called an *abelian group* in the literature. For a finite group G, $|G|$ is also called the *order* of the group.

For a subset T of a semigroup (resp. monoid, group) we denote by $\langle T\rangle_S$ (resp. $\langle T\rangle_M$, $\langle T\rangle_G$) the *subsemigroup* (resp. *submonoid, subgroup*) *generated by* T, i.e., the smallest subsemigroup (resp. submonoid, subgroup) containing T in the semigroup (resp. monoid, group) under consideration. In this regard, the elements in T are called the *generators* of the semigroup $\langle T\rangle_S$ (resp. monoid $\langle T\rangle_M$, group $\langle T\rangle_G$). We have $\langle T\rangle_S = \bigcup_{i\geq 1} T^i$, $\langle T\rangle_M = \bigcup_{i\geq 0} T^i$, $\langle\emptyset\rangle_S = \emptyset$ and $\langle\emptyset\rangle_M = \langle\emptyset\rangle_G = \{1\}$. Suppose $(G, \cdot, ^{-1}, 1)$ is a finite group. Then for $g \in G$ there exists $n \geq 1$ such that $g^n = 1$ and so $g^{-1} = g^{n-1}$. This implies that the subsemigroups in the semigroup $(G, \cdot)$ are precisely the subgroups of $(G, \cdot, ^{-1}, 1)$ and, for non-empty $T \subseteq G$, we have $\langle T\rangle_G = \langle T\rangle_M = \langle T\rangle_S$. So, for finite groups and $T \neq \emptyset$, we can omit S, M or G in the index and just write unambiguously $\langle T\rangle$.

Let S_1, S_2 be semigroups. Then $S_1 \times S_2$ with the operation

$$(s,t)(s',t') = (ss', tt')$$

for $(s,t), (s',t') \in S_1 \times S_2$ is called the *direct product* semigroup. If S_1, S_2 are monoids (resp. groups) with identities $1 \in S_1$ and $1' \in S_2$, respectively, then $S_1 \times S_2$ is a monoid with identity $(1,1)$ (resp. group with identity $(1,1)$ and inversion $(s,t)^{-1} = (s^{-1}, t^{-1})$). Let I be some set. We obtain a semigroup $(S_1^I, \odot)$ with the *(point-wise* or *component-wise) product* $\odot$ given by $(f \odot g)(x) = f(x) \cdot g(x)$ for $f, g \in S_1^I$ and all $x \in I$. If S_1 is a monoid with identity $1 \in S_1$, then S_1^I is a monoid where the identity is the constant function $\mathbf{1} \in S_1^I$ mapping everything to 1.

Example 3.24 Consider $(\mathbb{N}_0, +, 0)$, $(\mathbb{N}_0, \cdot, 1)$, $([n], +_n, 0)$ and $([n], \cdot_n, 1)$ from Example 3.19 and an alphabet Σ. We obtain monoid structures $(\mathbb{N}_0^\Sigma, \oplus, \mathbf{0})$ and $(\mathbb{N}_0^\Sigma, \odot, \mathbf{1})$ from $(\mathbb{N}_0, +, 0)$ and $(\mathbb{N}_0, \cdot, 1)$, respectively, by the construction given above where $\oplus$ is also called the *component-wise addition* on $\mathbb{N}_0^\Sigma$. Similarly, we obtain monoid structures $([n]^\Sigma, \oplus_n, \mathbf{0})$ and $([n]^\Sigma, \odot_n, \mathbf{1})$ on $[n]^\Sigma$. In the following, unless stated otherwise, we assume these monoid structures on $\mathbb{N}_0^\Sigma$ and $[n]^\Sigma$, and we write $\oplus$ as $+$ and $\oplus_n$ as $+_n$, since there is no danger of confusion.

3.5.2 Matrices

Let R be a semiring and $m, n \geq 0$. An $m \times n$ *matrix* A (over R) is a mapping $A\colon \{1,\ldots,m\} \times \{1,\ldots,n\} \to R$, also written as $A = (a_{ij})_{i\in\{1,\ldots,m\}, j\in\{1,\ldots,n\}}$

with $a_{ij} = a(i)(j)$. When the ranges of i and j are clear from the context, we also write $A = (a_{ij})$ for short. The elements $a_{i1}, \ldots, a_{in}$ for some $i \in \{1, \ldots, m\}$ constitute the i-th *row* of A and the elements $a_{1j} \ldots, a_{mj}$ for some $j \in \{1, \ldots, n\}$ constitute the j-th *column* of A. Correspondingly, the first index (here i) is called the *row index* and the second index (here j) is called the *column index*. See Figure 3.3 for a schematic depiction of an $m \times n$ matrix $A = (a_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$.

$$A = \overbrace{\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}}^{n \text{ columns}} \Bigg\} m \text{ rows}$$

Fig. 3.3: Scheme of an $m \times n$ matrix $A = (a_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$.

Let $A = (a_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$ be an $m \times n$ matrix and $B = (b_{ij})_{i \in \{1,\ldots,n\}, j \in \{1,\ldots,l\}}$ be an $n \times l$ matrix where $l, m, n \geq 1$. The *(matrix) product*, denoted by $A \cdot B$ or simply AB, is the $m \times l$ matrix C, i.e, $C = AB$, where $C = (c_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,l\}}$ is given by

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}$$

for $i \in \{1, \ldots, m\}$ and $j \in \{1, \ldots, l\}$. The *(matrix) sum* of two $m \times n$ matrices $A = (a_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$ and $B = (b_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$ is the $m \times n$ matrix $C = (c_{ij})_{i \in \{1,\ldots,n\}, j \in \{1,\ldots,m\}}$, written $C = A + B$, given by $c_{ij} = a_{ij} + b_{ij}$ for $i \in \{1, \ldots, m\}$ and $j \in \{1, \ldots, n\}$.

Let $n \geq 1$. The $n \times n$ *identity matrix (over R)* is $I_n = (a_{ij})$ defined by $a_{ij} = 1$ if $i = j$ and $a_{ij} = 0$ if $i \neq j$. The $n \times n$ *zero matrix* (over R) is $O_n = (o_{i,j})$ defined by $o_{ij} = 0$ for all $i, j \in \{1, \ldots, n\}$.

The set of $n \times n$ matrices over a semiring forms a semiring with the product and sum operations, and the identity and zero matrices.

A *Boolean matrix* is a matrix over the Boolean semiring $(\{0, 1\}, \wedge, \vee, 0, 1)$.

Next, we show how to describe a transformation $t \colon X \to X$ on a finite set of size n by a matrix. Fix some order $X = \{x_1, \ldots, x_n\}$ (where we take $X = [n]$ with $x_i = i - 1$ for $i \in [n]$ if nothing else is said). Then, to t we associate a *(Boolean) transition matrix* $A = (a_{ij})_{i,j \in \{1,\ldots,n\}}$ by setting

$$a_{ij} = \begin{cases} 1 & \text{if } t(x_i) = x_j, \\ 0 & \text{otherwise .} \end{cases}$$

Intuitively, point x_i is mapped to x_j if and only if in the i-th row of A the unique one appears at position j, i.e., in the j-th column of A. When the specific ordering of the states is not important, we assume some order to be taken without explicitly mentioning it.

3.5.3 Transformation Semigroups

The set of all transformations on a set X with function composition as operation forms a semigroup $(X^X, \circ)$, called the *full transformation semigroup on* X. This set also forms a monoid with identity id_X. We also denote $(X^X, \circ)$ by $\mathcal{T}_X$. For $X = [n]$, we set $\mathcal{T}_n = \mathcal{T}_X$. A *transformation semigroup* (resp. *transformation monoid*) on the *domain* X is a subsemigroup (resp. submonoid) of $\mathcal{T}_X$.

Let $S \leq_S \mathcal{T}_X$ be a transformation semigroup. A *fixpoint* of S is a point fixed by all elements from S. An *S-congruence* is an equivalence relation $\sim$ on X such that, for all $x, y \in X$ and $t \in S$, we have

$$x \sim y \Rightarrow t(x) \sim t(y). \tag{3.29}$$

We also say that x and y are *congruent* if $x \sim y$. Equivalence classes of $\sim$ are also called *congruence classes* in this case. The universal and equality relations on X are S-congruences, called the *trivial S-congruences*.

Recall from Section 3.2 that the elements of X are also called *points* in the context of transformations.

Remark 3.25 Let $(M, \cdot, 1)$ be a monoid. For $m \in M$ let $\hat{m}\colon M \to M$ be given by $\hat{m}(x) = xm$ for $x \in M$. Then $\hat{M} = \{\hat{m} \mid m \in M\}$ is a transformation monoid on the domain M. The map $m \mapsto \hat{m}$ is injective since $\hat{m} = \hat{n}$ for $m, n \in M$ implies $m = \hat{m}(1) = \hat{n}(1) = n$, and it is a monoid morphism. Thus, $\hat{M}$ represents the monoid M as a transformation monoid on the domain M. The M-congruences of $\hat{M}$ are also called right-congruences of M in the literature, see, for example, [325].

Remark 3.26 Let S be a semigroup. By adjoining an identity as in Theorem 3.21 we obtain a monoid $M = S \cup \{1\}$. Consider the transformation monoid constructed from it as in Theorem 3.25 via the map $m \mapsto \hat{m}$ for $m \in M$. Then $\{\hat{s} \mid s \in S\}$ is a transformation semigroup and the map $s \mapsto \hat{s}$ for $s \in S$ is an injective semigroup morphism. Thus, the semigroup S is represented as a transformation semigroup.

The embedding of an arbitrary semigroup in a transformation semigroup as in Theorem 3.26 is also known as Cayley's theorem, for example, see [236, Thm. 2.4.3]. However, Cayley only gave it for groups [137].[6] The extension to semigroups is due to Suschkewitsch [588].

3.6 Order Theory

Here, we collect the order-theoretical notions that we need such as quasiorders, partial orders, the induced equivalence relation, upper sets, etc. Section 3.6.1 give definitions

[6] Cayley did not give an explicit statement, but only implicitly assumed it, since the notion of an abstract group was not yet invented. He investigated a formal calculus of operations with an identity element and satisfying associativity (implicitly assuming inverses) and used it to investigate groups.

and some basic equations. In Section 3.6.2, we investigate upper and lower sets and the induced equivalence relation. Maximal and minimal elements are investigated in Section 3.6.3, and the set of maximal elements in Section 3.6.4. Section 3.6.5 collects results on sequences of upper sets. We later need all these notions when we introduce a special quasiorder on automata.

3.6.1 Definitions and Basic Facts

1. A *quasiorder* (or *preorder*) on a set X is a reflexive and transitive relation. We also write $(X, \lesssim)$ for X together with the quasiorder $\lesssim$.
2. For a quasiordered set $(X, \lesssim)$ we call two elements $x, y \in X$ *incomparable* if neither $x \lesssim y$ nor $y \lesssim x$ is true.
3. Let $\lesssim$ be a quasiorder on X and $S \subseteq X$.
   a. We call $x \in X$ a *minimal element* (resp. *maximal element*) of S if $x \in S$ and for all $y \in S$, if $y \lesssim x$ (resp. $x \lesssim y$), then $x \lesssim y$ (resp. $y \lesssim x$).
   b. We call $x \in X$ a *minimum*, or *least element*, (resp. *maximum*, or *greatest element*) of S if $x \in S$ and for all $y \in S$ we have $x \lesssim y$ (resp. $y \lesssim x$).
   c. The set S is a *lower set* (resp. an *upper set*) if $x \in S$ and $y \lesssim x$ (resp. $x \lesssim y$) imply $y \in S$ for $x, y \in X$.
   d. The *smallest lower set* (resp. *upper set*) *containing S (in X)* is $\downarrow S = \{y \in X \mid \exists x \in S\colon y \lesssim x\}$ (resp. $\uparrow S = \{y \in X \mid \exists x \in S\colon x \lesssim y\}$).

   > The notions of maximal/maximum and minimal/minimum are usually only defined for partially ordered sets, but here we use them for quasiordered sets as well.

4. Given quasiordered sets $(X, \lesssim)$ and $(Y, \lessapprox)$ an *(order) morphism* (or *monotone function*) is a function $\varphi\colon X \to Y$ such that, for all $x, y \in X$, we have

   $$x \lesssim y \Rightarrow \varphi(x) \lessapprox \varphi(y).$$

   Additionally, we call $\varphi\colon X \to Y$ an *(order) isomorphism* if there exists another order morphism $\psi\colon Y \to X$ such that $\varphi\psi = \mathrm{id}_Y$ and $\psi\varphi = \mathrm{id}_X$.
5. A *partial order* on a set X is a quasiorder that is antisymmetric.
6. A *total* (or *linear*) *order* is a partial order $\leq$ on X such that, for all $x, y \in X$, we have $x \leq y$ or $y \leq x$.
7. A *topological sorting* (also *topological order* or *linear extension*) of a partial order $\leq$ on X is a total order $\leq^*$ on X such that the identity transformation $\mathrm{id}\colon X \to X$ is a morphism from $(X, \leq)$ to $(X, \leq^*)$.[7]
   For finite X, a topological sorting can be identified with an indexing $x_1, \ldots, x_n$ of all the elements from X such that $x_i \leq x_j$ implies $i \leq j$.

[7] According to Szpilrajn's extension theorem every partial order can be topologically sorted [591].

8. Given a quasiorder $\lesssim$ on a set X, the *induced equivalence relation (by $\lesssim$)* is

$$x \simeq y \Leftrightarrow x \lesssim y \wedge y \lesssim x, \tag{3.30}$$

and the well-defined *induced partial order* $\leq$ on the equivalence classes $X/\simeq$ is defined by setting

$$[x]_\simeq \leq [y]_\simeq \Leftrightarrow x \lesssim y. \tag{3.31}$$

By Equation (3.31) the function $x \mapsto [x]_\simeq$ is a surjective order morphism. When the context is clear, we also denote the induced partial order on the equivalence classes by the same symbol $\lesssim$.

A minimum (resp. maximum) is also a minimal (resp. maximal) element. If not said otherwise, we mean minimal, maximal, minimum, and maximum w.r.t. $S = X$. In a partially ordered set, a minimum (resp. maximum) equals a unique minimal (resp. maximal) element. For $p \lesssim q$, we also say that p is below q, q is above p, and that these relations hold in a strict sense if additionally $q \not\lesssim p$.

Example 3.27 Let $(X, \lesssim)$ be a quasiordered set and I be some set. Then the set of functions X^I becomes a quasiordered set $(X^I, \lesssim^*)$ by setting $f \lesssim^* g$ if and only if $f(x) \lesssim g(x)$ for all $x \in I$ and $f, g \in X^I$. If $\lesssim$ is a partial order, then $\lesssim^*$ is also a partial order, called the *component-wise order*. Often, without special mentioning, we simply denote this quasiorder on X^I by $\lesssim$ as well.

Example 3.28 For some alphabet Σ, the sets of functions $\mathbb{N}_0^\Sigma$ and $[n]^\Sigma$ are partially ordered by the component-wise order from Theorem 3.27 derived from the usual order on $\mathbb{N}_0$.

Let $(X, \lesssim)$ and $(Y, \lessapprox)$ be quasiordered sets with induced equivalence relations $\simeq$ and $\approx$, respectively, and let $\varphi\colon X \to Y$ be an order morphism.

Proposition 3.29 *For $x \in X$ we have $\varphi([x]_\simeq) \subseteq [\varphi(x)]_\approx$, i.e., the morphism φ maps an equivalence class into an equivalence class. Furthermore, if φ is an order isomorphism, we also have $\varphi([x]_\simeq) = [\varphi(x)]_\approx$.*

Proof. Let $y \in [x]_\simeq$, i.e., $x \simeq y$. Clearly, we have $\varphi(x) \approx \varphi(y)$. Thus $\varphi(y) \in [\varphi(x)]_\approx$.

Next, suppose φ is an order isomorphism. Then $\psi\varphi = \mathrm{id}_X$ for some order morphism $\psi\colon Y \to X$. Let $y \in [\varphi(x)]_\approx$. By Theorem 3.3, φ is surjective. Thus there exist $z \in X$ such that $y = \varphi(z)$. Since $\varphi(z) \approx \varphi(x)$, we find $z = \psi(\varphi(z)) \approx \psi(\varphi(x)) = x$. Thus $z \in [x]_\simeq$ and so $y \in \varphi([x]_\simeq)$. □

Proposition 3.30 *The morphism φ induces a well-defined order morphism $\overline{\varphi}\colon X/\simeq \to Y/\approx$ w.r.t. the induced partial orders given by $\overline{\varphi}([x]_\simeq) = [\varphi(x)]_\approx$.*

Proof. Since $x \simeq y$ implies $\varphi(x) \simeq \varphi(y)$, $\overline{\varphi}$ is well-defined. By Equation (3.31) it is a morphism. □

Proposition 3.31 *$\varphi(S) \cap T \neq \emptyset$ implies $\varphi(S) \subseteq T$ for equivalence classes $S \subseteq X$ and $T \subseteq Y$ of the induced equivalence relation.*

Proof. Suppose $\varphi(S) \cap T \neq \emptyset$. Since all elements in $\varphi(S)$ are equivalent w.r.t. $\cong$, and T, as an equivalence class, is a maximal set whose element are all equivalent, we find $\varphi(S) \subseteq T$. □

The following example gives a bijective (order) morphism that is not an isomorphism.

Example 3.32 Let $X = \{0, 1\}$ and $\leq$ be the partial order with $0 \leq 1$. Then id_X is a bijective morphism of X ordered by equality to X ordered by $\leq$ that is not an isomorphism.

In the following statements, $(X, \lesssim)$ is a quasiordered set with induced equivalence relation $\simeq$ and induced partial order $\leq$ on $X/\simeq$.

For $x, y \in X$, the following hold:

$$x \lesssim y \Leftrightarrow y \in \uparrow\{x\} \Leftrightarrow \uparrow\{y\} \subseteq \uparrow\{x\} \tag{3.32}$$

$$x \simeq y \Leftrightarrow \uparrow\{x\} = \uparrow\{y\} \tag{3.33}$$

$$\tag{3.34}$$

For a morphism $\varphi\colon X \to Y$ and $S \subseteq X$ we have:

$$\varphi(\uparrow S) \subseteq \uparrow\varphi(S) \tag{3.35}$$

$$S \text{ is an upper set} \Rightarrow \varphi^{-1}(S) \text{ is an upper set} \tag{3.36}$$

For $S, T \subseteq X$ we have:

$$S, T \text{ are upper sets} \Rightarrow S \cup T \text{ and } S \cap T \text{ are upper sets} \tag{3.37}$$

$$S, T \text{ are disjoint upper sets} \Rightarrow \forall x \in S\ \forall y \in T\colon x \not\leq y \wedge y \not\leq x \tag{3.38}$$

$$S \text{ is an upper set} \Leftrightarrow S = \bigcup_{x \in S} \uparrow\{x\} \tag{3.39}$$

$$S \text{ is an upper set} \Leftrightarrow X \setminus S \text{ is a lower set} \tag{3.40}$$

Similar statements to Equations (3.32), (3.33), (3.35), and (3.37) to (3.39) also hold for lower sets.

Remark 3.33 The inclusion from Equation (3.35) can be proper. For example, let $X = \{x_1, x_2\}$ where $x_1 \lesssim x_2$. Then with the morphism $\varphi\colon X \to X$ defined by $\varphi(x_1) = \varphi(x_2) = x_1$ we have $\varphi(\uparrow\{x_1\}) = \{x_1\}$ and $\uparrow\{\varphi(x_1)\} = \{x_1, x_2\}$. This also shows that a morphism does not necessarily map upper sets to upper sets. More generally, if all elements are comparable in an arbitrary set X with at least two elements, then any constant map is an order morphism such that the inclusion from Equation (3.35) is proper.

Remark 3.34 Let $x \in X$. Then $\uparrow\{x\} \setminus [x]_\simeq$ is a upper set and $\downarrow\{x\} \setminus [x]_\simeq$ is a lower set.

From Equation (3.40) we obtain the next statement.

Proposition 3.35 *$S \subseteq X$ is a maximal proper upper set w.r.t. set inclusion if and only if $X \setminus S$ is a minimal non-empty lower set w.r.t. set inclusion.*

Remark 3.36 In most of the remaining statements of the present section that make a claim on upper and lower sets, we only give the proofs for upper sets, since the corresponding statement for lower sets follows similarly.

3.6.2 Equivalence Classes and Upper and Lower Sets

Here, Theorem 3.37 relates upper and lower sets to equivalence classes, and we list consequences in Theorems 3.38 and 3.39. Theorems 3.40 to 3.42 give relations between maximal and minimal upper and lower sets and equivalence classes.

In this section, let $(X, \lesssim)$ be a quasiorder and let $\simeq$ be its induced equivalence relation.

Proposition 3.37 *For $S \subseteq X$, the following are equivalent.*

1. *S is contained in an equivalence class for $\simeq$.*
2. *For every upper set $T \subseteq X$ we have $S \subseteq T$ or $S \cap T = \emptyset$.*
3. *For every lower set $T \subseteq X$ we have $S \subseteq T$ or $S \cap T = \emptyset$.*

Proof. $1 \Rightarrow 2$: Let $T \subseteq X$ be an upper set. Suppose we have $x \in S \cap T$. Since T is an upper set, we have $\uparrow\{x\} \subseteq T$. By Equation (3.32), we find $[x]_\simeq \subseteq \uparrow\{x\}$. Thus $S \subseteq T$ since $S \subseteq [x]_\simeq$ by assumption.

$2 \Rightarrow 3$: Let $T \subseteq X$ be a lower set. Since $X \setminus T$ is an upper set by Equation (3.40), by Item 2, S is either contained in $X \setminus T$ or T.

$3 \Rightarrow 1$: Let $x, y \in S$. By assumption, we find $S \subseteq \downarrow\{y\}$ and $S \subseteq \downarrow\{x\}$. This gives $x \lesssim y$ and $y \lesssim x$, respectively. Thus $x \simeq y$ and so S is contained in an equivalence class. □

Corollary 3.38 *Every upper or lower set $S \subseteq X$ is a union of equivalence classes.*

Proof. Clearly, $S \subseteq \bigcup_{y \in S} [y]_\simeq$. By Theorem 3.37, we have $\bigcup_{y \in S} [y]_\simeq \subseteq S$. □

Corollary 3.39 *Let $S, T \subseteq X$ be both upper or lower sets. If $T \setminus S$ is non-empty and contained in an equivalence class, then it equals this equivalence class.*

Proof. Without loss of generality, suppose S and T are upper sets. Let $x \in T \setminus S$. By assumption, $T \setminus S \subseteq [x]_\simeq$. By Theorem 3.37, $[x]_\simeq \subseteq T$ and $[x]_\simeq \cap S = \emptyset$. Therefore, $[x]_\simeq \subseteq T \setminus S$. □

Lemma 3.40 *Let $S, T \subseteq X$. If $T \setminus S$ is contained in an equivalence class, then for all upper sets $R \subseteq X$ with $S \cap T \subseteq R \subseteq T$, we have $R = S \cap T$ or $R = T$.*

Proof. Let R be an upper set with $S \cap T \subseteq R \subseteq T$. By Theorem 3.37 either we have $(T \setminus S) \subseteq R$, which gives $T = (S \cap T) \cup (T \setminus S) \subseteq R$ and so $R = T$, or $(T \setminus S) \cap R = \emptyset$, which gives $R = R \cap T = R \cap ((S \cap T) \cup (T \setminus S)) = (R \cap (S \cap T)) \cup (R \cap (T \setminus S)) = R \cap (S \cap T) = S \cap T$. □

Lemma 3.41 *Let $S, T \subseteq X$. Then the following are equivalent.*

1. *For all upper sets $R \subseteq X$ with $S \cap T \subseteq R \subseteq T$, we have $R = S \cap T$ or $R = T$.*
2. *For all lower sets $R \subseteq X$ with $S \cap T \subseteq R \subseteq T$, we have $R = S \cap T$ or $R = T$.*

Proof. Let R be a lower (resp. an upper) set with $S \cap T \subseteq R \subseteq T$. Then $X \setminus T \subseteq X \setminus R \subseteq X \setminus (S \cap T) = (X \setminus S) \cup (X \setminus T)$. Since $X \setminus R$, $X \setminus S$, $X \setminus T$, and $(X \setminus T) \cup (X \setminus T)$ are upper (resp. lower) sets by Equations (3.37) and (3.40), from Item 1 we obtain $X \setminus R = X \setminus T$ or $X \setminus R = X \setminus (S \cap T)$, and so $R = T$ or $R = S \cap T$. This proves Item 2. The converse implication follows similarly. □

An analogous equivalence to Theorem 3.41, between upper and lower sets but with $S \subseteq R \subseteq S \cup T$ in place of $S \cap T \subseteq R \subseteq T$, can be established. We omit such a statement here.

Proposition 3.42 *Let $S, T \subseteq X$ be both upper sets or both lower sets. Then the following are equivalent.*

1. *$T \setminus S$ is contained in an equivalence class.*
2. *For all upper sets $R \subseteq X$ with $S \cap T \subseteq R \subseteq T$ we have $R = S \cap T$ or $R = T$.*

Proof. $1 \Rightarrow 2$: This follows from Theorem 3.40 (note that this statement applies to any sets, not just upper/lower sets).

$2 \Rightarrow 1$: First, suppose S and T are upper sets. Suppose $T \setminus S$ is not contained in an equivalence class. Then we have $x \not\lesssim y$ for some $x, y \in T \setminus S$. Let $R = \uparrow\{x\}$. We have $y \notin R$. By Equation (3.37), $(S \cap T) \cup (R \cap T)$ is an upper set. This upper set properly contains $S \cap T$ and is properly contained in T.

Next, if S and T are lower sets, we use the same reasoning but with Theorem 3.41, Item 2 and lower set instead of upper sets.[8] □

By Theorems 3.39 and 3.42 for upper sets S, T, if $T \setminus S$ is non-empty and contained in an equivalence class, then $T = (S \cap T) \cup [x]_{\simeq}$ for some $x \in T \setminus S$.

3.6.3 Maximal and Minimal Elements

Theorem 3.43 characterizes upper and lower sets that are equivalence classes. In Theorem 3.47 we characterize uniqueness of a proper non-empty upper or lower set. Theorems 3.48 and 3.49 gives quantitative statements on the number of disjoint upper and lower sets, as well as on incomparable maximal and minimal elements in finite sets. Lastly, in Theorems 3.50 to 3.52 we list results concerning on maxima and minima.

In this section, let $(X, \lesssim)$ be a quasiorder and let $\simeq$ be its induced equivalence relation.

[8] Alternatively, for a lower set $R \subseteq X$ consider the lower set $K = (S \cup T) \cap (R \cup S)$. Since $S \subseteq K \subseteq S \cup T$, we have $(X \setminus S) \cap (X \setminus T) \subseteq X \setminus K \subseteq X \setminus S$ and as these sets are upper sets by Equation (3.37), from the assumption we get $X \setminus K = X \setminus (S \cup T)$ or $X \setminus K = X \setminus S$, equivalently $K = S \cup T$ or $K = S$. In the former case, $T \setminus S \subseteq R$, and in the latter case, $(T \setminus S) \cap R = \emptyset$. So, from Theorem 3.37, we get that $T \setminus S$ is contained in an equivalence class.

Proposition 3.43 *The following are equivalent for $S \subseteq X$:*

1. *S is a minimal non-empty upper (resp. lower) set w.r.t. set inclusion.*
2. *S is an equivalence class that is an upper (resp. a lower) set.*
3. *S is non-empty such that $S = \bigcap_{x \in S} \uparrow\{x\}$ (resp. $S = \bigcap_{x \in S} \downarrow\{x\}$).*
4. *S is non-empty such that $S = \uparrow\{x\}$ (resp. $S = \downarrow\{x\}$) for each $x \in S$.*
5. *Each $x \in S$ is maximal (resp. minimal) and $S = [x]_{\simeq}$.*
6. *We have $S = [x]_{\simeq}$ for some maximal (resp. minimal) $x \in X$.*
7. *S is a maximal (resp. minimal) equivalence class in $(X/\simeq, \leq)$.*

Proof. As mentioned in Theorem 3.36, we prove the result only for upper sets.

$1 \Rightarrow 2$: By minimality, there exists no upper set $R \subseteq X$ with $R \subseteq S$ such that $R \neq \emptyset$ and $R \neq S$. Hence, by Theorem 3.42 applied with $\emptyset$ and S, S is contained in an equivalence class. Since $S \neq \emptyset$, by Theorem 3.39 (or by Theorem 3.38), it equals an equivalence class.

$2 \Rightarrow 3$ Let $x \in S$. Since S is an equivalence class we have $S \subseteq \uparrow\{x\}$ by Theorem 3.38. Thus $S \subseteq \bigcap_{x \in S} \uparrow\{x\}$. Since S is an upper set, we have $\uparrow\{x\} \subseteq S$, Thus $\bigcap_{x \in S} \uparrow\{x\} \subseteq S$. Since S is an equivalence class it is non-empty.

$3 \Rightarrow 4$ Let $x \in S$. Then $S = \bigcap_{y \in S} \uparrow\{y\} \subseteq \uparrow\{x\}$. It is easy to see that $S = \bigcap_{y \in S} \uparrow\{y\}$ is an upper set. Thus we have $\uparrow\{x\} \subseteq S$ for $x \in S$ since $\uparrow\{x\}$ is the smallest upper set generated by x.

$4 \Rightarrow 5$: Let $x \in S$ and assume $x \lesssim y$ for some $y \in X$. Then $y \in \uparrow\{x\} = S$. By assumption, $\uparrow\{y\} = S = \uparrow\{x\}$. Thus, with Equation (3.33), we get $x \simeq y$. So x is maximal. Next, let $z \in [x]_{\simeq}$. Then $z \in \uparrow\{x\} = S$. Thus we have $[x]_{\simeq} \subseteq S$. Now, suppose $z \in S$. Then $\uparrow\{z\} = S = \uparrow\{x\}$ and so $z \simeq x$ by Equation (3.33). This gives $[x]_{\simeq} \subseteq S$.

$5 \Rightarrow 6$: This is clear.

$6 \Rightarrow 7$: Let $S = [x]_{\simeq}$ with x maximal in X. Suppose $[x]_{\simeq} \leq [y]_{\simeq}$ for some $y \in X$. Then we have $x \lesssim y$. Since x is maximal, we find $x \simeq y$ and so $[x]_{\simeq} = [y]_{\simeq}$. Thus $[x]_{\simeq}$ is maximal in $(X/\simeq, \leq)$.

$7 \Rightarrow 1$: Let $x \in S$ and $y \in X$ with $x \lesssim y$. Since S is maximal, we find $x \simeq y$ and so $x \in S$. Hence S is an upper set. Next, let $R \subseteq X$ be a non-empty upper set with $R \subseteq S$. Since S is an equivalence class, Theorem 3.38 (or Theorem 3.40) implies that $S \subseteq R$. Therefore, S is minimal w.r.t. set inclusion among all non-empty upper sets. □

Corollary 3.44 *An element $x \in X$ is maximal if and only if $[x]_{\simeq}$ is an upper set.*

Corollary 3.45 *If X is finite, then every upper set contains an upper upper set forming a maximal equivalence class in $(X/\simeq, \leq)$.*

Corollary 3.46 *An upper set is a maximal proper set w.r.t. set inclusion if and only if its complement is a minimal equivalence class for the induced partial order.*

Proof. This follows from directly from Theorems 3.35 and 3.43. □

Proposition 3.47 *Let $S \subseteq X$ be a non-empty and proper upper (resp. lower) set. Then S and X are the only non-empty upper (resp. lower) sets if and only if S and $X \setminus S$ are equivalence classes and $X \setminus S$ is not an upper (resp. a lower) set.*

Proof. Suppose S and X are the only non-empty upper sets. By considering $T = X$ and S in Theorem 3.42, we find that $X \setminus S$ is contained in an equivalence class. From Theorem 3.39 we get that it equals that equivalence class. Next, for $x \in S$, since $\uparrow\{x\} \subseteq S$, we get $\uparrow\{x\} = S$ and so S is an equivalence class by Theorem 3.43. By assumption, $X \setminus S$ is not an upper set.

Conversely, suppose S and $X \setminus S$ are equivalence classes. Since all upper sets are unions of equivalence classes and X is non-empty, S, $X \setminus S$, and X are all possible non-empty upper sets. So, if $X \setminus S$ is not an upper set, only S and X remain as upper sets. □

Proposition 3.48 *Suppose X is finite. Then the following numbers are equal:*

1. *The maximal number of pairwise disjoint non-empty upper (resp. lower) sets.*
2. *The number of maximal (resp. minimal) elements in $(X/\simeq, \leq)$.*
3. *The maximal number of incomparable maximal (resp. minimal) element in X.*

Proof. Let m_1, m_2, m_3 be the maximal number of pairwise disjoint non-empty upper sets, of distinct maximal elements in $(X/\simeq, \leq)$, and the maximal number of incomparable maximal elements in X, respectively.

By Theorem 3.43 maximal elements in $X/\simeq$ are upper sets that are equivalence classes, hence pairwise disjoint. This gives $m_1 \geq m_2$.

By the definition of the induced partial order, incomparable maximal elements induce distinct maximal elements in $X/\simeq$, which gives $m_2 \geq m_3$.

By Equation (3.38), we have $m_3 \geq m_1$. □

Corollary 3.49 *For finite X, there are at most k distinct maximal (resp. minimal) elements in the partially ordered set $(X/\simeq, \leq)$ if and only if, among $k + 1$ upper (resp. lower) sets, two have a non-empty intersection.*

The following statements are easy to verify and we omit their proofs.

Proposition 3.50 *The following are equivalent for $x \in X$:*

1. *$x \in X$ is a maximum (resp. minimum) for X.*
2. *$[x]_\simeq$ is a maximum (resp. minimum) in $(X/\simeq, \leq)$.*
3. *$[x]_\simeq \subseteq S$ for every upper (resp. lower) set $S \subseteq X$.*

Proposition 3.51 *The following are equivalent:*

1. *There exists a maximum (resp. minimum) in X for $\lesssim$.*
2. *For $x, y \in X$ there exists $z \in X$ with $x \lesssim z$ and $y \lesssim z$ (resp. $z \lesssim x$ and $z \lesssim y$).*
3. *For two maximal (resp. minimal) $x, y \in X$ we have $x \simeq y$.*
4. *For every two upper (resp. lower) sets $S, T \subseteq X$ we have $S \cap T \neq \emptyset$.*

Proposition 3.52 *Let $S \subseteq X$ be an equivalence class. If S is a maximum (resp. minimum) in $X/\simeq$, then it is the only upper (resp. lower) set among the equivalence classes. Furthermore, if X is finite and S the only upper (resp. lower) set that is an equivalence class, then it is a maximum (resp. minimum) in $X/\simeq$.*

Proof. We only give the proof for upper sets and maxima. First, suppose S is an equivalence class that is a maximum in $X/\simeq$. Let $T \subseteq X$ be an equivalence class that is an upper set. By Theorem 3.50, we have $S \subseteq T$. Since S and T are equivalence classes, we find $S = T$. By Theorem 3.43, S is an upper set, and so a unique equivalence class that is an upper set.

Conversely, suppose X is finite and S is a unique upper set that is an equivalence class. By Theorem 3.43, S is maximal in $X/\simeq$ and unique with this property. By finiteness, for each $x \in X$ there exists a maximal $y \in X$ such that $x \leq y$. Since S is the unique maximal element, we find $y \in S$. This yields that S is a maximum in $X/\simeq$. □

To show that the converse implication from Theorem 3.52 might fail for infinite X, consider $X = \mathbb{N}_0 \cup \{a\}$ with the usual order on $\mathbb{N}_0$ and a being incomparable to any other element. Then $\{a\}$ is a unique upper set that is an equivalence class, but it is not a maximum w.r.t. the induced partial order on the equivalence classes (which results in the same order structure since every equivalence class is a singleton).

3.6.4 The Set of Maximal Elements

In this section, let $(X, \lesssim)$ and

$$S = \{x \in X \mid x \text{ is maximal}\}$$

be the set of maximal elements. We denote the induced equivalence with $\simeq$.

Proposition 3.53 *The set of maximal elements S is an upper set and we have*

$$S = \bigcup_{x \text{ is maximal}} [x]_{\simeq}. \tag{3.41}$$

Proof. Equation (3.41) is obvious, and together with Theorem 3.43 we obtain that S is an upper set. □

Next, we investigate S for finite X. Similar statements could be obtained for the set of minimal elements. But since we only need the set of maximal elements later, we do not formulate corresponding statements for this set.

Lemma 3.54 *If X is finite, then for each $x \in X$ there exists $y \in S$ such that $x \lesssim y$.*

Proof. Assume there exists $x \in X$ for which this condition is false. Set $x_0 = x$ and, inductively, for x_i with $i \geq 0$ there exists $x_{i+1} \in X$ such that $x_i \lesssim x_{i+1}$ but $x_{i+1} \not\lesssim x_i$, since otherwise x_i would be a maximum in $\{x_0, x_1, \ldots, x_i\}$ and so also maximal such that $x \lesssim x_i$. Now, in this way, we obtain an infinite and strictly increasing sequence, which is not possible for finite X. □

Proposition 3.55 *Suppose X is finite. Then the set of maximal elements is the minimal non-empty set w.r.t. set inclusion that has non-empty intersection with each non-empty upper set.*

Proof. Let T be a minimal non-empty upper set that has non-empty intersection with each non-empty upper set.

Consider $x \in S$. By assumption, we have $y \in T \cap \uparrow\{x\}$ for some $y \in X$. Since x is maximal, this gives $y \simeq x$. Then $x \in T$ since T is an upper set. Hence $S \subseteq T$.

Next, let R be an upper set and $x \in R$. By Lemma 3.54, there exists $y \in S$ such that $x \lesssim y$. Since R is an upper set, we have $y \in R \cap S$. Hence, by minimality of T, we find $T \subseteq S$. □

Corollary 3.56 *Let X be finite and $S \subseteq X$ be a non-empty upper set. Then S contains an upper set that is an equivalence class.*

Proof. By Theorem 3.55 we have $q \in S$ for some maximal element $q \in X$. Then $[q]_\simeq$ is an upper set by Theorem 3.43 and we have $[q]_\simeq \subseteq S$. □

3.6.5 Sequences of Upper and Lower Sets

Here, we investigate sequences of increasing upper and lower sets under certain finiteness conditions. We only give statements for upper sets, since corresponding statements for lower sets follow analogously, and we actually only need them for upper sets in later chapters.

Proposition 3.57 *Let $S \subseteq Q$ be a proper upper set and $X \setminus S$ be finite. Then there exists a sequence of upper sets*

$$S_1 \subsetneq \cdots \subsetneq S_m = X$$

with $S_1 = S$ and such that $S_i \setminus S_{i-1}$ for $i \in \{2, \dots, m\}$ are all the equivalence classes in $X \setminus S$.

Proof. By finiteness, there exists a sequence of upper sets $S_1 \subsetneq \cdots \subsetneq S_m$ with $S_1 = S$ and m maximal. Since X is an upper set, we have $S_m = X$.

First, assume there exists $i \in \{2, \dots, m\}$ such that $S_i \setminus S_{i-1}$ is not contained in an equivalence class. By Theorem 3.37 there exists an upper set $T \subseteq X$ with $(S_i \setminus S_{i-1}) \cap T \neq \emptyset$ and $(S_i \setminus S_{i-1}) \not\subseteq T$. Thus, for the upper set $S_{i-1} \cup (T \cap S_i)$ we have $S_{i-1} \subsetneq S_{i-1} \cup (T \cap S_i) \subsetneq S_i$, which contradicts the maximal choice of m.

Next, let $x \in X \setminus S$. Then there exists a minimal $i \in \{2, \dots, m\}$ such that $x \in S_i$, so that $x \in S_i \setminus S_{i-1}$. Since $S_i \setminus S_{i-1}$ is contained in an equivalence class, we have $S_i \setminus S_{i-1} \subseteq [x]_\simeq$. By Theorem 3.39 we find $S_i \setminus S_{i-1} = [x]_\simeq$. □

Remark 3.58 Theorem 3.57 can also be proved by picking a proper upper set $S \subseteq S_{m-1}$ such that $|X \setminus S_{m-1}|$ has minimal size. Then S_{m-1} is a proper maximal upper set containing S and by Theorem 3.42 and Theorem 3.39 the difference $X \setminus S_{m-1}$ equals an equivalence class. If $S \neq S_{m-1}$, inductively, there exists a sequence of upper sets $S_1 \subsetneq \cdots \subseteq S_{m-1}$ such that $S_i \setminus S_{i-1}$ for $i \in \{1, \dots, m-1\}$ are all the equivalence classes in $S_{m-1} \setminus S$. Now for each equivalence class $R \subseteq X$ we either have $R \subseteq S_{m-1}$ or $R \cap S_{m-1} = \emptyset$. Since the latter case implies $R \subseteq X \setminus S_{m-1}$, and then $R = X \setminus S_{m-1}$ since $X \setminus S_{m-1}$ is an equivalence class, with $S_m = X$ we also obtain a sequence as claimed.

For finite X, by picking for S the empty set or the set of maximal elements we obtain the following corollary to Theorem 3.57.

Corollary 3.59 *Suppose X is finite.*

1. *There exists a sequence of upper sets*

$$\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = X$$

   such that $S_i \setminus S_{i-1}$ for $i \in \{1, \ldots, m\}$ are all the equivalence classes in X.
2. *There exists a sequence of upper sets*

$$S_1 \subsetneq \cdots \subsetneq S_m = X$$

   such that S_0 equals the set of maximal elements and each equivalence class of a non-maximal element has the form $S_i \setminus S_{i-1}$ for some $i \in \{1, \ldots, m\}$.

The sequences from Theorem 3.59, Item 1 are related to topological sortings in the following way. Given a topological sorting $[x_1]_{\simeq}, \ldots, [x_m]_{\simeq}$ of the elements from $X/{\simeq}$ for finite X, we obtain a sequence as in Theorem 3.59, Item 1 by setting $S_i = \uparrow\{x_1, \ldots, x_i\}$ and $S_0 = \emptyset$. Conversely, given a sequence as in Theorem 3.59, Item 1, the equivalence classes $S_i \setminus S_{i-1}$ for $i \in \{1, \ldots, m\}$ give a topological sorting of $X/{\simeq}$ since $x_i \lesssim x_j$ for $x_i \in S_i \setminus S_{i-1}$ and $x_j \in S_j \setminus S_{j-1}$ with $i \neq j$ implies $\uparrow\{x_i\} \subsetneq \uparrow\{x_j\}$ by Equation (3.32). Thus, $\uparrow\{x_i\} \subseteq S_i$ and $\uparrow\{x_j\} \subseteq S_j$ give $S_i \subsetneq S_j$ and so $i \leq j$.

3.7 Formal Language Theory

Formal language theory is the study of Σ^* and its subsets as a means of representing and processing information. Besides the notions from formal language theory already introduced in Section 1.1, here, in Section 3.7.1, we introduce additional notions. In Section 3.7.2 we introduce regular expressions as a formalism to describe languages.

3.7.1 More Basic Notions

In Section 1.1 we introduced the $*$-operation that maps a set to the monoid of finite sequences over this set with concatenation as the binary operation. Despite being introduced only for finite sets Σ (here Σ always denotes a finite set), this operation can be applied to any set X and obeys the following property.

Proposition 3.60 (Universal Mapping Property) *Suppose X is a set. Then for every function $\varphi\colon X \to M$ into a monoid M, there exists a unique morphism $\hat{\varphi}\colon X^* \to M$*

(the evaluation morphism) such that $\varphi(x) = \hat{\varphi}(\iota(x))$ for all $x \in X$ where $\iota\colon X \to X^*$ is the embedding that maps each element from X to a sequence of length one in X^*.[9]

Proof. We set $\hat{\varphi}(x_1 \cdots x_n) = \varphi(x_1) \cdots \varphi(x_n)$ for $x_1, \ldots, x_n \in X$ and $\hat{\varphi}(\varepsilon)$ equal to the identity element of M. It is easy to verify that this gives a morphism as required and that it is in fact uniquely determined. □

The monoid X^* for some set X is called the *free monoid over* X. For a generating set X of M, $\hat{\varphi}\colon X^* \to M$ is surjective. So, every monoid is the surjective image of some free monoid.

Recall that, in Section 1.1, we have identified sequences of length one with X, i.e., x with $\iota(x)$ for $x \in X$ (which made $X \subseteq X^*$ into a meaningful statement). So, we can write $\varphi(x) = \hat{\varphi}(x)$ for $x \in X$. In the following, we denote the morphism $\hat{\varphi}$ by φ as well.

Theorem 3.60 implies that φ is fully specified by it images on the elements from X. Thus, to specify a morphism from X^* into some monoid M, it is sufficient to give the images of the element from X.

Now, let $L \subseteq X^*$. Applying Theorem 3.60 to $i\colon L \to X^*$ given by $i(u) = u$ for $u \in \Sigma^*$, we obtain a morphism $\hat{\imath}\colon L^* \to X^*$. We then have $\hat{\imath}(u_1 \cdots u_n) = u_1 \cdots u_n$ for $u_1, \ldots, u_n \in L$ where on the left hand side, concatenation in L^* is meant and on the right hand side concatenation in X^*. Thus, we find $\hat{\imath}(L^*) = \bigcup_{i \geq 0} L^i$. Then, by identifying L^* with $\hat{\imath}(L^*)$, the statement $L^* \subseteq X^*$ is meaningful and we can regard the $*$-operation as an operation of signature $\mathcal{P}(X^*) \to \mathcal{P}(X^*)$ mapping L to $\bigcup_{i \geq 0} L^i$.

Intuitively, the identification of L^* with $\hat{\imath}(L^*)$ corresponds to interpreting sequences whose entries are sequences over a set X as sequences over X. However, note that $\hat{\imath}$ need not be injective in general and in this case it is not an isomorphism. Thus, the submonoid $\hat{\imath}(L^*)$ of X^* might have different properties than L^* considered as the free monoid over L. Nevertheless, the identification of L^* and $\hat{\imath}(L^*)$ is common practice and does not pose any problems for our applications. So, we use this convention in the following without special mention.

Mostly, in our context, X equals some finite alphabet Σ. In this case, for $L \subseteq \Sigma^*$, we call L^*, with $L^* = \bigcup_{i \geq 0} L^i$ via our identification, the *Kleene star*.[10] Furthermore, the *Kleene plus* is $L^+ = \bigcup_{i \geq 1} L^i$.

Next, we state some rules for manipulating language equations. The two equations for the Kleene star follow with the identification discussed above.

Proposition 3.61 *Let $U, V, W \subseteq \Sigma^*$. Then the following holds.*

1. *$U(V \cap W) \subseteq UV \cap UW$ and $(V \cap W)U \subseteq VU \cap WU$,*
2. *$U(V \cup W) = UV \cup UW$ and $(V \cup W)U = VU \cup WU$,*

[9] This property, in fact, determines X^* up to isomorphism. First, note that $\mathrm{id}_{X^*}\colon X^* \to X^*$ fulfills $\iota(x) = \mathrm{id}_{X^*}(\iota(x))$ for all $x \in X$ and so, by the property, it is the unique morphism fulfilling this equation (and thus the unique extension of ι). Next. suppose $F(X)$ is another object fulfilling this property with embedding $\iota_{F(X)}\colon X \to F(X)$. Then there exist morphisms $\hat{\iota}_{F(X)}\colon X^* \to F(X)$ and $\hat{\iota}\colon F(X) \to X^*$ such that $\iota_{F(X)}(x) = \hat{\iota}_{F(X)}(\iota(x))$ and $\iota(x) = \hat{\iota}(\iota_{F(X)}(x))$ for $x \in X$. Thus $\iota(x) = \hat{\iota}(\hat{\iota}_{F(X)}(\iota(x)))$ for $x \in x$, and so $\hat{\iota}\hat{\iota}_{F(X)} = \mathrm{id}_{X^*}$. Similarly, we find $\hat{\iota}_{F(X)}\hat{\iota} = \mathrm{id}_{F(X)}$ and so $\hat{\iota}$ and $\hat{\iota}_{F(X)}$ are isomorphisms.

[10] In the literature, the Kleene star is often directly defined as $\bigcup_{i \geq 0} L^i$ and not as a mapping of sets into a monoid of sequences that yields this set under a suitable interpretation of sequences of sequences.

3. $(U \cup V)^* = (U^*V)^*U^*$.
4. $(U^*)^* = U^*$.

A language L is said to be *unary* (resp. *binary* or *ternary*) if there exists an alphabet Σ with $|\Sigma| = 1$ (resp. $|\Sigma| = 2$ or $|\Sigma| = 3$) such that $L \subseteq \Sigma^*$.

Let $L \subseteq \Sigma^*$. For $u \in \Sigma^*$, the *(left) quotient (of L by u)* is $u^{-1}L = \{v \in \Sigma^* \mid uv \in L\}$. Language quotients were introduced by Brzozowski [106] (called *derivatives* therein). Next, set $\mathrm{Pref}(L) = \{u \in \Sigma^* \mid \exists v \in L\ \exists x \in \Sigma^* \colon v = ux\}$, $\mathrm{Suff}(L) = \{u \in \Sigma^* \mid \exists v \in L\ \exists x \in \Sigma^* \colon v = xu\}$, and $\mathrm{Fact}(L) = \{u \in \Sigma^* \mid \exists v \in L\ \exists x, y \in \Sigma^* \colon v = xuy\}$. For $u \in \Sigma^*$, we set $\mathrm{Pref}(u) = \mathrm{Pref}(\{u\})$, $\mathrm{Suff}(u) = \mathrm{Suff}(\{u\})$, and $\mathrm{Fact}(u) = \mathrm{Fact}(\{u\})$.[11] We call $u \in \Sigma^*$ a *prefix* (resp. *suffix*, *factor*) of $v \in \Sigma^*$ if $u \in \mathrm{Pref}(v)$ (resp. $u \in \mathrm{Suff}(v)$, $u \in \mathrm{Fact}(v)$). A prefix (resp. suffix, factor) u of v is called *proper* if $u \neq v$, i.e., if $v \in u\Sigma^+$ (resp. $v \in \Sigma^+u$, $v \in \Sigma^*u\Sigma^+ \cup \Sigma^+u\Sigma^*$).

A *left ideal language* (resp. *right ideal language*) is a language $I \subseteq \Sigma^*$ such that $\Sigma^*I \subseteq I$ (resp. $I\Sigma^* \subseteq I$). Left (resp. right) ideals are precisely the languages of the form $\Sigma^* \setminus \mathrm{Suff}(L)$ (resp. $\Sigma^* \setminus \mathrm{Pref}(L)$) for some $L \subseteq \Sigma^*$. An *ideal language* is a language that is simultaneously a left and a right ideal language.

We need the following result to solve language equations.

Lemma 3.62 (Arden's rule (see [322])) *Let $U, V \subseteq \Sigma^*$ with $\varepsilon \notin U$. Then the language equation $X = XU \cup V$ (resp. $X = UX \cup V$) has the unique solution $X = VU^*$ (resp. $X = U^*V$).*

Example 3.63 Let $L \subseteq \Sigma^+$. We have $\Sigma^* = L\Sigma^* \cup (\Sigma^* \setminus L\Sigma^*)$. By Theorem 3.62 this gives $\Sigma^* = L^*(\Sigma^* \setminus L\Sigma^*)$. Note that $L^*(\Sigma^* \setminus L\Sigma^*) = \Sigma^* \setminus L\Sigma^* \uplus L^+(\Sigma^* \setminus L\Sigma^*)$.

For $L \subseteq \Sigma^*$, $u \in \Sigma^*$ and $n \geq 1$ we also need the following formulas that are easy to verify:

$$u \in \mathrm{Pref}(L) \Leftrightarrow \mathrm{Pref}(u) \subseteq \mathrm{Pref}(L) \tag{3.42}$$

$$\mathrm{Pref}(L^*) \cap \Sigma^{\leq n} = \mathrm{Pref}(L^n) \cap \Sigma^{\leq n} \tag{3.43}$$

$$\mathrm{Pref}(L^n) = L^{<n}(\mathrm{Pref}(L) \setminus L) \cup L^n \tag{3.44}$$

$$\mathrm{Pref}(L^*) = L^* \mathrm{Pref}(L) = L^+ \cup L^*(\mathrm{Pref}(L) \setminus L) \tag{3.45}$$

$$\mathrm{Pref}(L^*) = L^*(\mathrm{Pref}(L) \setminus L) \text{ if } \varepsilon \notin L \tag{3.46}$$

$$\Sigma^*v\Sigma^* \cap L \neq \emptyset \Leftrightarrow \Sigma^*v \cap \mathrm{Pref}(L) \neq \emptyset \tag{3.47}$$

Note that, if $\varepsilon \in L$, then $\varepsilon \notin L^*(\mathrm{Pref}(L) \setminus L)$, so that Equation (3.46) does not hold in this case. For a concrete example let $L = \{\varepsilon, aa\}$. Then $\mathrm{Pref}(L^*) = a^*$, $\mathrm{Pref}(L) \setminus L = \{a\}$ and $L^* = (aa)^*$ so that Equation (3.46) fails.

Let $\Gamma \subseteq \Sigma$. The *projection morphism* $\pi_\Gamma \colon \Sigma^* \to \Gamma^*$ onto Γ^* is given by $\pi_\Gamma(x) = x$ for $x \in \Gamma$ and $\pi_\Gamma(x) = \varepsilon$ for $x \notin \Gamma$. As the name suggests, π_Γ is a monoid morphism.

[11] Thus we obtain functions $\Sigma^* \to \mathcal{P}(\Sigma^*)$. Note that the extension of these functions to subsets would actually give functions $\mathcal{P}(\Sigma^*) \to \mathcal{P}(\mathcal{P}(\Sigma^*))$, which creates some ambiguity with their original definitions as function with signature $\mathcal{P}(\Sigma^*) \to \mathcal{P}(\Sigma^*)$. Our usage follows the standard convention and when applied to languages, we always mean the functions $\mathcal{P}(\Sigma^*) \to \mathcal{P}(\Sigma^*)$ as originally defined, which should be clear from context so that no confusion arises.

We set $|u|_a = |\pi_{\{a\}}(u)|$ for $u \in \Sigma^*$ and $a \in \Sigma$. Then $\pi_{\{a\}}(u) = a^{|u|_a}$ and $|u|_a$ equals the number of times the letter a occurs in the word u. The *Parikh map* [474] (or *Parikh morphism*) is $\psi\colon \Sigma^* \to \mathbb{N}_0^\Sigma$ where $\psi(u) = (|u|_a)_{a\in\Sigma}$ for $u \in \Sigma^*$. A language $L \subseteq \Sigma^*$ is *commutative* if $\psi^{-1}(\psi(L)) = L$. Equivalently, if $uabv \in L$ implies $ubav \in L$ for all $u, v \in \Sigma^*$ and $a, b \in \Sigma$.

In expressions on languages using the operations concatenation $\cdot$ (as a product on subsets), Kleene star *, Kleene plus $^+$ and the set operations, when writing a single word u we mean to interpret it as the language $\{u\}$ and we omit parentheses according to the following precedence rules: * and $^+$ have the strongest binding, then $\cdot$ and lastly the set operations due to the usual convention where set difference is evaluated before union and intersection. For example, $abc \cup b^*$ denotes $\{abc, \varepsilon, b, b^2, b^3, \ldots\}$. We also omit parenthesis for associative operations such as union and intersection. For cases not covered by these rules, as, for example, combinations of union and intersection, parenthesis are used to resolve any ambiguity.

3.7.2 Regular Expressions

Let Σ be a finite alphabet and set $\hat{\Sigma} = \{\hat{a} \mid a \in \Sigma\} \cup \{\hat{\emptyset}\} \cup \{^*, \cdot, +, (,)\}$. A *regular expression* (also known as a *rational expression*)[12] is a word over $\hat{\Sigma}$ inductively defined in the following way: $\hat{a}$ for $a \in \Sigma$ and $\hat{\emptyset}$ are regular expressions, and if R, S are regular expressions, then $(R \cdot S)$ (or (RS) for short), $(R + T)$, and (R^*) are regular expressions. By $\mathcal{RE}(\Sigma) \subseteq \hat{\Sigma}^*$ we denote the *set of regular expressions*.

Each regular expression defines a language over Σ via the (interpretation or evaluation) function $L\colon \mathcal{RE}(\Sigma) \to \mathcal{P}(\Sigma^*)$ inductively defined by $L(\hat{a}) = \{a\}$, $L(\hat{\emptyset}) = \emptyset$, $L((RS)) = L(R)L(S)$, $L((R + S)) = L(R) \cup L(S)$ and $L((R^*)) = L(R)^*$.

When writing regular expressions, we leave out the hats, i.e., identify $\hat{a}$ with a for $a \in \Sigma$ and $\hat{\emptyset}$ with $\emptyset$, while still having the distinction between formal expressions and their interpretation in mind. Furthermore, we leave out parentheses due to the usual conventions, i.e., we arbitrarily add parentheses to expressions of the form RST and, with the other operations, according to the following operation precedence: * (strongest binding), $\cdot$ and then $+$. For example, $L(abc + ab^*) = L(((a(bc)) + (a(b^*))))$. If there is no danger of confusion, we also omit the L-function and simply write, for example, $abc + ab^*$ for the language $L(abc + ab^*)$.

At the end of Section 3.7.1 we discussed expressions over languages and precedence rules. Every regular expression can be readily translated to an expression over languages, the only difference being rather superficial in the usage of $\cup$ instead of $+$. So, the reader might wonder what we have gained by this extra level of abstraction. The reason is that in computer science we are often concerned with questions of expressibility (what can be described) and succinctness (complexity of descriptions) of some formal system or

[12] Eilenberg [194] suggested the renaming to rational expressions and there are good reasons to use the term rational instead of regular when considering more general monoids than the free monoid or generalizing the theory to formal power series instead of languages. We refer to the literature for more information [194, 195]. Here we are not concerned with these generalizations, and so we prefer the term regular expression which was introduced, historically, before.

description mechanism and in proving results with regard to these two questions we need to be precise and separate the syntax from its semantic. In Section 6.7 we will characterize the class of languages describable by regular expressions using automata.

3.8 Graph Theory

Here, in Section 3.8.1, we introduce undirected and directed graphs, and related notions. Section 3.8.2 investigates different connectedness notions, paths, and cycles. In Section 3.8.3 we introduce the graph of a function and of a homogeneous relation.

3.8.1 Basic Notions

An *undirected graph*, or just *graph*, (resp. a *directed graph*, or *digraph*) is a tuple $G = (V, E)$ where V is the set of *vertices* (or *nodes*) and $E \subseteq \{\{u, v\} \mid u, v \in V\}$ (resp. $E \subseteq V \times V$) the set of *edges*.[13] For an edge $\{u, v\} \in E$ (resp. $(u, v) \in E$) we call u and v its *endpoints*. In case of digraphs, (u, v) is called an *outgoing edge w.r.t. u* (and u is called the *starting/initial/source vertex*) and an *incoming edge w.r.t. v* (and v the *ending/final/terminal vertex*). For finite V and $v \in V$ the *outdegree* (resp. *indegree*) of v is $|\{u \in V : (v, u) \in E\}|$ (resp. $|\{u \in V : (u, v) \in E\}|$).

A *subgraph* of an undirected (resp. directed) graph $G = (V, E)$ is an undirected (resp. directed) graph $G = (V', E')$ such that $V' \subseteq V$ and $E' \subseteq E$. A *(graph) morphism* of two undirected (resp. directed) graphs $G = (V, E)$ and $G' = (V', E')$ is a map $\varphi\colon V \to V'$ such that, for all $u, v \in V$,

$$\{u, v\} \in E \Rightarrow \{\varphi(u), \varphi(v)\} \in E' \quad (\text{resp. } (u, v) \in E \Rightarrow (\varphi(u), \varphi(v)) \in E').$$

A *(graph) automorphism* (for undirected and directed graphs) is a morphism $\varphi\colon V \to V$ of a graph onto itself such that there exists a morphism $\psi : V \to V$ with $\psi\varphi = \varphi\psi = \mathrm{id}_V$. These are precisely those morphisms $\varphi\colon V \to V$ such that

$$\{u, v\} \in E \Leftrightarrow \{\varphi(u), \varphi(v)\} \in E' \quad (\text{resp. } (u, v) \in E \Leftrightarrow (\varphi(u), \varphi(v)) \in E').$$

The set of all automorphisms forms a permutation group, called the *automorphism group* $\mathrm{Aut}(G)$ of the graph.

Automorphisms of finite graphs are precisely the bijective morphisms from V onto V since in this case $\varphi(E) \subseteq E$ and $|\varphi(E)| = |E|$ imply $\varphi(E) = E$. However, this fails for infinite graphs, as shown by the following example.

Example 3.64 $G = (\mathbb{Z}, E)$ with $E = \{\{u, v\} \mid \{n, n+1\} \text{ and } n > 0\}$. The map $\varphi\colon \mathbb{Z} \to \mathbb{Z}$ with $\varphi(n) = n+1$ for $n \in \mathbb{Z}$ is a morphism, but it is not an automorphism since $\{0, 1\} \notin E$ and $\varphi(\{0, 1\}) = \{1, 2\} \in E$.

[13] Here, $u = v$ (a self-loop) is permitted, which some authors exclude from the definition.

Let $G = (V, E)$ be a digraph. We call G *symmetric* (resp. *oriented*) if $(u, v) \in E$ implies $(v, u) \in E$ (resp. $(v, u) \notin E$) for $u \neq v$.

We often give a graph (resp. digraph) in graphical form by drawing the vertices as circles and the edges as connecting lines (resp. connecting arrows) between them.

Groups are also often denoted by G, but the context should make it clear whether we refer to a graph or a group.

3.8.2 Connectedness Notions

Here, we introduce various notions related to connectedness in graphs. Let $k \geq 1$.

1. A *walk* (of length $n \geq 1$)[14] in an undirected (resp. directed) graph $G = (V, E)$ from $u \in V$ (*start*) to $v \in V$ (*end*) is a sequence of edges $e_1, \ldots, e_n$ with $e_i = \{v_{i-1}, v_i\}$ (resp. $e_i = (v_{i-1}, v_i)$) for all $i \in \{1, \ldots, n\}$ such that $v_0 = u$ and $v_n = v$. We call the walk a *path* (resp. *cycle* (at u)) if $v_0, v_1, \ldots, v_n$ are distinct (resp. $v_0, v_1, \ldots, v_{n-1}$ are distinct and $v_0 = v_n$).[15] Often, when the specific edges involved do not matter, we refer to a walk/path/cycle by its sequence of vertices $v_0, v_1, \ldots, v_n \in V$.[16]
2. A directed or undirected graph G is called *acyclic* if there exists no cycle in G. Equivalently, every walk is a path. For short, a directed acyclic graph is called a *DAG*. An undirected acyclic graph is also called a *forest*.
3. An undirected (resp. directed) graph $G = (V, E)$ is *k-connected* (resp. *strongly k-connected*) if for distinct vertices $u, v \in V$ and every $(k-1)$-set $A \subseteq V$ there exists a path $u_0, u_1, \ldots, u_n$ from u to v such that $\{u_1, \ldots, u_{n-1}\} \cap A = \emptyset$. For $k = 1$ we also call G just *connected* (resp. *strongly connected*).
4. We call the digraph $G = (V, E)$ *strongly $\frac{3}{2}$-connected* if it is strongly connected and for every path $v_0, v_1, \ldots, v_n$ and $0 < i < n$ there exists a path $u_0, u_1, \ldots, u_m$ with $\{u_0, u_m\} = \{v_0, v_n\}$ and $v_i \notin \{u_1, \ldots, u_{m-1}\}$. Equivalently, G is strongly connected and for every three distinct $u, v, w \in V$ there exists a path from u to v or from v to u not traversing w, i.e., we find a path avoiding w.
5. A digraph $G = (V, E)$ is called *weakly k-connected* if the undirected graph with vertex set V and edge set $\{\{u, v\} \mid (u, v) \in E\}$ is k-connected. A weakly 1-connected graph is simply called *weakly connected*.
6. A digraph $G = (V, E)$ is called *unilaterally connected* (or *semiconnected*) if for distinct vertices $u, v \in V$ there exists a path from u to v or from v to u.
7. In an undirected (resp. directed) graph $G = (V, E)$ a *connected component* (resp. *strongly connected component*) is a subset $S \subseteq V$ maximal for subset inclusion such that there exists a path from u to v for distinct $u, v \in S$. A subset of a

[14] There exist no walks of length zero according to our definition.

[15] The terminology varies in the literature: some authors use path where we use walk, and reserve simple path for a walk whose vertices are all distinct.

[16] Paths and cycles can also be identified with graphs and digraphs such that a graph or digraph contains a path if and only if it contains these as subgraphs. However, this does not work for walks since they can have repeated vertices or edges.

digraph $G = (V, E)$ is a *weakly connected component* if it is a connected component of the undirected graph with vertex set V and edge set $\{\{u, v\} \mid (u, v) \in E\}$.

Remark 3.65 The *connectivity* of a finite graph (resp. digraph) with n vertices in which not all vertices are joined by an edge is the minimal number of vertices whose deletion results in a graph (resp. digraph) that is not connected (resp. not strongly connected). This also equals the largest k such that the graph (resp. digraph) is k-connected (resp. strongly k-connected).

If all vertices are joined by an edge, deletion of any number of vertices never destroys connectedness or strong connectedness and it is k-connected or strongly k-connected for all $k \geq 1$. In this case, the connectivity is defined to be $n - 1$.

In a DAG $G = (V, E)$, when putting $u, v \in V$ in relation if there exists a path from u to v, we obtain a partial order. When saying we *topologically sort the DAG* we refer to a topological sorting of the vertices w.r.t. this partial order.

The following classical result was proved by Menger for undirected graphs [451] and extended to digraphs by Dirac [178]. A short proof for digraphs is given in [258].

Theorem 3.66 (Menger-Dirac [178, 451]) *An undirected (resp. directed) graph $G = (V, E)$ is* k-connected *(resp.* strongly k-connected*) if for distinct $u, v \in V$ there exist at least k paths from u to v such that for two such paths $v_0, v_1, \ldots, v_n$ and $u_0, u_1, \ldots, u_m$ with $v_0 = u_0 = u$, $v_n = u_m = v$ we have $\{v_1, \ldots, v_{n-1}\} \cap \{u_1, \ldots, u_{m-1}\} = \emptyset$.*

In the context of the present work, only strong $\frac{3}{2}$-connectivity and (strong) k-connectivity with $k \leq 2$ will be of interest. The implications from Figure 3.4 are easily seen. Figure 3.5 shows that the converse implications do not hold. Furthermore, it also gives an example of a strongly connected digraph that is weakly 2-connected but not strongly $\frac{3}{2}$-connected; thus strong connectivity and weak 2-connectivity do not imply strong $\frac{3}{2}$-connectivity.

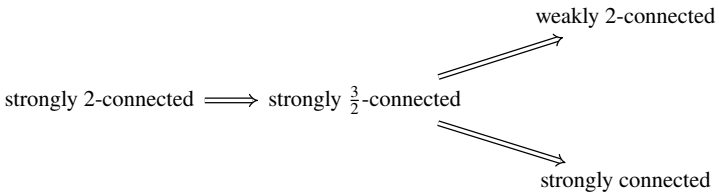
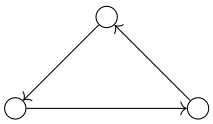


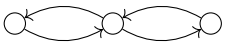
Fig. 3.4: Implications for strong 2-connectivity, strong $\frac{3}{2}$-connectivity, strong and weak connectivity. The converse implications do not hold, see Figure 3.5.

Lastly, we state a characterization of strong connectivity, which was given as a definition in [255, Def. 2].

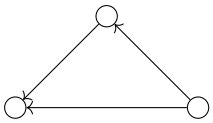
Lemma 3.67 *A digraph $G = (V, E)$ is strongly connected if and only if for every proper and non-empty $B \subsetneq V$ there exists an edge $(u, v) \in E$ such that $u \notin B$ and $v \in B$.*



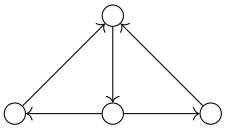
(a) Strongly $\frac{3}{2}$-connected, but not strongly 2-connected.



(b) Strongly connected, but not weakly 2-connected.



(c) Weakly 2-connected, but not strongly connected.



(d) Strongly connected and weakly 2-connected, but not strongly $\frac{3}{2}$-connected.

Fig. 3.5: Some relations among the introduced notions of connectedness.

Proof. First, suppose G is strongly connected. Let $B \subsetneq V$ with $u \in V \setminus B$ and $v \in B$. By assumption there exists a path $v_0, v_1, \ldots, v_n$ with $v_0 = u$ and $v_n = v$ in G. Let $i \in \{1, \ldots, n\}$ be a largest index such that $v_i \notin B$. We have $i < n$ since $v_n \in B$, and $v_{i+1} \in B$ by the maximal choice of i. So, the edge $(v_i, v_{i+1}) \in E$ fulfills the claim.

Conversely, suppose the condition is true. Let $u, v \in V$ be distinct vertices. Set $P = \{w \in V \mid \text{there exists a path from } u \text{ to } w\} \cup \{u\}$. If $V \neq P$, then, by applying the assumption to $B = V \setminus P$, there exists $(u', v') \in E$ with $u' \in P$ and $v' \notin P$. As there exists a path from u to u', by adjoining (u', v') we obtain a path from u to v'. However, as $v' \notin P$ this is a contradiction. So, we must have $P = V$ and there exists a path from u to v. Hence, G is strongly connected. □

Corollary 3.68 *An undirected graph $G = (V, E)$ is connected if and only if for every proper and non-empty $B \subseteq V$ there exists $e \in E$ with $|e \cap (V \setminus B)| = |e \cap B| = 1$.*

Proof. Consider the digraph $(V, \{(u, v), (v, u) \mid \{u, v\} \in E\})$. Since G is connected, it is strongly connected. By Theorem 3.67 it has an edge (u, v) such that $u \notin B$ and $v \in B$. By definition $\{u, v\} \in E$. Then the claim holds for the edge $e = \{u, v\}$. □

3.8.3 Graphs for Transformations and Homogeneous Relations

The *functional graph* of a transformation $t \colon X \to X$ is the digraph $G_t = (X, E)$ with

$$E = \{(x, t(x)) \mid x \in X\}.$$

In G_t, the vertices in $\operatorname{excl}(t)$ are precisely those with indegree zero, and those in $\operatorname{dupl}(t)$ are precisely the vertices with indegree at least two. A digraph is the functional graph of some transformation if and only if every vertex has outdegree one. These graphs are also called maximal directed pseudoforests [233], or sun-structures in [215].

Furthermore, to every homogeneous relation R over X we associate a digraph $G_R = (X, E)$ where $E = \{(a, b) \mid aRb\}$.

Remark 3.69 The cycle points of a transformation are precisely those contained in cycles of the functional graph. The cycle points can be partitioned into points on individual cycles. The tail points are connected to the cycle points in a "tree-like" fashion whose roots are precisely the root points. The edges on the tails point towards the root. See Figure 3.6 where a functional graphs with two cycles and three tails is depicted. The roots are indicated by solid black circles.

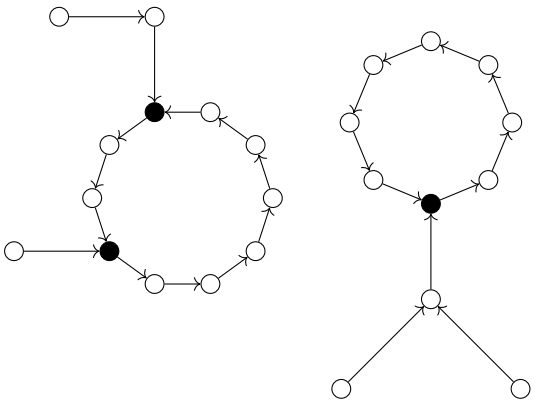


Fig. 3.6: Example of a functionl graph with root points marked, see Theorem 3.69.

Proposition 3.70 *Suppose R is a homogeneous relation over X. Then the weakly connected components of G_R are precisely the equivalence classes of the equivalence relation generated by R.*

Proof. For $a, b \in X$ we set $a \sim b$ if a and b are contained in the same weakly connected component of G_R. Then $\sim$ is an equivalence relation such that, for $a, b \in X$, we have

$$aRb \Rightarrow a \sim b. \tag{3.48}$$

Now, let E be any equivalence relation over X such that aRb implies aEb for all $a, b \in X$. Suppose $a \sim b$ for $a, b \in X$. Then there exist $v_0, v_1, \ldots, v_n \in X$ forming a path from a to b in the undirected graph resulting from G_R by forgetting about the direction of edges, i.e., $v_0 = a$, $v_n = b$ and for each $i \in \{1, \ldots, n\}$ we have $v_{i-1}Rv_i$ or v_iRv_{i-1}. Hence, by assumption and symmetry of E, we find $v_{i-1}Ev_i$ for all $i \in \{1, \ldots, n\}$. By transitivity of E, we get aEb. Thus, $\sim$ refines E.

So, by Equation (3.48) and as $\sim$ refines every equivalence relation E where aRb implies aEb, it equals the equivalence relation generated by R. □

For a more algebraic description of the equivalence relation generated by R, see [325, Prop. 1.4.9 & 1.4.10].

Remark 3.71 In the literature, the set $R_G \subseteq X \times Y$ for a relation $R = (R_G, X, Y)$ as defined in Section 3.4 is often called the graph of the relation. However, since this is not a graph as defined here, we avoid this terminology.

3.9 Computational Complexity Theory

Here, resource usage of computing processes and algorithms is measured in time and in space w.r.t. some model of computation. We introduce three models, the random access machine (RAM) model in Section 3.9.1, classical sequential Turing machines and transducers in Section 3.9.2, and circuits in Section 3.9.3. The RAM model is solely for practical purposes as a model close to existing computers in which to measure the runtime of algorithms. Turing machines and circuits serve a more theoretical purpose. Further cornerstones of our theoretical approach are decision problems, introduced in Section 3.9.4, and reductions and complexity classes, introduced in Section 3.9.5, which give a framework for the classification of problems based on their inherent computational complexity.

When giving time or space upper bounds, to abstract from some implementation specific details, only the behavior up to some constant is of interest. To express this, we use the big O notation [53] (see also [374]). For a function $g\colon \mathbb{N}_0 \to \mathbb{R}$ we set

$$O(g) = \{f\colon \mathbb{N}_0 \to \mathbb{R} \mid \exists c > 0\ \exists n_0 \in \mathbb{N}_0\ \forall n \geq n_0 : |f(n)| \leq c \cdot |g(n)|\},$$
$$o(g) = \{f\colon \mathbb{N}_0 \to \mathbb{R} \mid \forall c > 0\ \exists n_0 \in \mathbb{N}_0\ \forall n \geq n_0 : |f(n)| \leq c \cdot g(n))\}.$$

As is usually done, we identify a function with a defining expression and write, for example, $O(n^2)$ for $O(f)$ with $f\colon \mathbb{N}_0 \to \mathbb{N}_0$ given by $f(n) = n^2$. We also use $O(g)$ and $o(g)$ in equations with the meaning that there exists a function from these sets fulfilling the equation. For example, Theorem 3.1 is equivalent to $p_n = n\log(n)(1 + o(1))$.

3.9.1 The Random Access Machine Model

A *random access machine (RAM)* (or *register machine*) consists of a single generic processor having access to an (idealized) infinitely long memory indexed by natural numbers. Each memory cell is called a *register*. In this model, instructions are executed one after another, with no concurrent operations. The instruction set consists of arithmetic (such as add, subtract, multiply, divide, remainder, floor, ceiling), data movement (load, store, copy), and control operations (conditional branch, goto, subroutine call and return) each taking a constant amount of time. This model is most often used in textbooks on algorithms [163]. Hence, we do not go into more detail about the syntax and semantics of these machines, but refer to the literature [163, 566]. We assume our RAMs operate solely with non-negative integers.

When stating (asymptotic) runtime and space bounds and nothing else is said, we refer to the RAM model.

3.9.2 Turing Machines and Transducers

Here, we introduce Turing machines and transducers in an intuitive manner. For a more formal treatment of Turing machines, and also different but equivalent formalizations, we refer to the literature, for example [322].

A *nondeterministic Turing machine* (just called *Turing machine* for short) consists of a two-way read-only input tape and a two-way read/write work tape. A Turing machine is said to be *deterministic*, if in each configuration at most one configuration can follow in a single computational step.[17] A *(Turing) transducer* is a deterministic Turing machine that always halts and is additionally equipped with a write-only output tape on which the head never moves left. We say that a Turing machine *accepts* an input $u \in \Sigma^*$ if there exists a computational run starting from u that ends in a final state. We say a Turing machine accepts a language $L \subseteq \Sigma^*$ if it accepts precisely the words from L, i.e., for each input $u \in \Sigma^*$, u is accepted if and only if $u \in L$. A Turing machine *decides* a language if it accepts it and every computational run terminates.[18]

A transducer computes a function $f\colon \Sigma^* \to \Sigma^*$ if it produces, when supplied with the input u on the input tape, the value $f(u)$ on its output tape.

Let $f\colon \mathbb{N}_0 \to \mathbb{N}_0$. Following the convention of [322, p. 288], a Turing machine M is said to be $f(n)$-*time bounded* (resp. $f(n)$-*space bounded*) if all computational runs terminate and lie within the prescribed time (resp. space) bounds.[19] Similarly, a Turing transducer is $f(n)$-time (resp. $f(n)$-space) bounded if it takes at most $f(n)$ steps (resp. uses at most $f(n)$ work tape cells) to compute the output for an input of size n. As a shorthand, we also say a Turing machine or transducer is log-space bounded if it is $f(n)$-spaced bounded for some $f \in O(\log)$. This notion transfers to classes of function. For example, we say that a Turing machine is polynomial-time bounded if it is $p(n)$-time bounded for some polynomial function $p(n)$.

By the *linear speed up* and *tape compression* theorems (see [322, Sect. 12.2]) general claim on time or space requirements are only meaningful up to a multiplicative constant, which justifies using the O-notation in these contexts as well.

As a shorthand, for $f = \log$ we also say log-space bounded for $\log(n)$-space bounded machines or transducers, and we say that a machine or transducer is polynomial time/space bounded if it is $p(n)$-time/-space bounded for some polynomial p.

---

[17] This is the standard terminology, but it is somewhat unfortunate, since a deterministic Turing machine is also nondeterministic, i.e., the prefix "non" does not exclude it from being deterministic.

[18] In the literature (see, for example, [41, Sec. 2.1.2]), another convention appears that only requires that, for $u \in L$, at least one computational run terminates and accepts, and that, for $u \notin L$, all computational runs terminate and reject. For the notion of decidability, this makes no difference, since under both conventions a deterministic Turing machine deciding L can be constructed, and for deterministic Turing machines both conventions coincide. Our definition, in which all computational runs must terminate, is also used in Sipser's book [575, Def. 3.6].

[19] Other definitions of resource-bounded computations, and conditions for their equivalence, are discussed in Section 3.12.

3.9.3 Circuits

A *(Boolean) circuit* is a DAG with at least one vertex and the following additional conventions for its vertices:

1. each vertex of outdegree zero has indegree one and is called an *output*,
2. each vertex with indegree zero is called an *input*,
3. all other vertices with non-zero in- and outdegree are called *gates* and are labeled by a Boolean function $f\colon \{0,1\}^n \to \{0,1\}$ such that each input bit of f corresponds to a definite incoming edge (and so its indegree equals the arity n of f).

We also allow the constants 0 or 1 as inputs to input vertices.[20] The indegree (resp. outdegree) of a vertex is also called its *fan-in* (resp. *fan-out*). There is no constraint on the fan-out of non-output vertices. The *size* is the number of vertices in the circuit. The *depth* of the circuit is the length of a longest path.

Consider a circuit with n input and m output vertices, where we have a fixed order on the input and on the output vertices. This circuit computes a vector-valued Boolean function $\{0,1\}^n \to \{0,1\}^m$ by evaluating it for the input $(x_1,\dots,x_n) \in \{0,1\}^n$ in the following way: Assume some topological ordering of the vertices where the inputs appear first in their fixed order. The i-th input vertex is marked with the value of x_i. Then, going forward in the topological order, consider a non-input vertex g where the source vertices of the incoming edges are marked with $z_1,\dots,z_l$, $l \geq 1$. If g is a gate labeled with $f\colon \{0,1\}^l \to \{0,1\}$, where we assume $z_1,\dots,z_l$ are ordered in accordance with the input bits to f, then mark g with $f(z_1,\dots,z_l)$. Otherwise, g is an output vertex. In this case, it is $l = 1$ and we mark g with z_1. Finally, the results of the output vertices $(y_1,\dots,y_m)$ constitute the result of the evaluation of the circuit for the input $(x_1,\dots,x_n)$.

Let $k \geq 0$. We say that a family of circuits $C_1, C_2, \dots$ with strictly increasing number of input bits is in

NC^k if there exists a number $c > 0$ such that each circuit C_i with n input bits has size $O(n^c)$, depth $O(\log^k(n))$, and each gate in C_i is either a NOT gate, an AND gate of fan-in two, or an OR gate of fan-in two,

AC^k if there exists a number $c > 0$ such that each circuit C_i with n input bits has size $O(n^c)$, depth $O(\log^k(n))$, and each gate in C_i is either a NOT gate, an AND gate of unbounded fan-in, or an OR gate of unbounded fan-in.

In the following, we will use the obvious identification of Boolean formulas with circuits and often specify a circuit by only giving a formula (but note that in a circuit, the output of parts might be used several times, so a circuit might be smaller than suggested by a specifying formula).

Let $g, h\colon \{1,2,3,\dots\} \to \{1,2,3,\dots\}$ be such that g is monotone strictly increasing and $g \in O(n^c)$ for some constant c. Let $C_1, C_2, \dots$ be a family of circuits where C_n has $g(n)$ input vertices and $h(n)$ output vertices.

[20] This is also done in [102]. In [619, Def. 1.6] constants are modeled as 0-ary Boolean function with which input vertices can also be labeled.

Following the convention of [159, 160], we say the family of circuits *computes* a function $f\colon D \to \{0,1\}^+$ for $D = \bigcup_{n\geq 1}\{0,1\}^{g(n)}$ if the circuit C_n for $n \geq 1$ computes the function $f_{|\{0,1\}^{g(n)}}\colon \{0,1\}^{g(n)} \to \{0,1\}^{h(n)}$.

Suppose we have an appropriate binary encoding of the words in Σ^* where each word is encoded by a some binary word of length $g(n)$ for some $n \geq 1$. The circuit family *decides* a language $L \subseteq \Sigma^*$ if $h(n) = 1$ for all $n \geq 1$ and for $u \in \Sigma^*$ with binary encoding $x_1 \cdots x_{g(n)}$ we have $u \in L$ if and only if C_n evaluates to 1 when its i-th input vertex is initialized with the i-bit x_i of the binary encoding of u.

In our applications and problems the existence of some encoding is evident. Thus, in the following, to simplify our discussions, we often abstract away from the details of a specific encoding choice and accordingly omit g and h when giving circuit families.

With these definitions, even languages not decidable by Turing machines are decidable by circuits. For example, let $L \subseteq \{a\}^+$ be some unary language not decidable by Turing machines. Then $\{w \in \{0,1\}^+ : |w| \in L\}$ is not decidable by any Turing machine, but it is decidable by the family of circuits $C_1, C_2, \ldots$ in NC^0 [586, p. 142] given by

$$C_n = \begin{cases} x_1 \vee \neg x_1 & \text{if } a^n \in L, \\ x_1 \wedge \neg x_1 & \text{otherwise.} \end{cases}$$

To obtain only circuit families that compute functions, or decide languages, that are also computable, or decidable, by Turing machines, we incorporate a uniform computation mechanism for the construction of individual circuits into our circuit families.

Let $C_1, C_2, \ldots$ be a circuit family where C_n has $g(n)$ input bits. We call this family *L-uniform* [157] if the function $1^{g(n)} \mapsto \overline{C_n}$ w.r.t. some encoding scheme $\overline{C_n}$ for C_n is computable by a deterministic log-space Turing transducer. Thus, the family is L-uniform precisely if a deterministic log-space Turing transducer can produce each circuit of this family when supplied only with the size of the input to the circuit. With this computability condition, we obtain a computationally meaningful notion of decidability of languages and computability of functions by L-uniform circuit families.

Example 3.72 Multiplication of a fixed number of Boolean matrices can be realized by a family of L-uniform AC^0 circuits. Let $M_r = (m_{ij}^{(r)})_{i,j\in\{1,\ldots,n\}}$ for $r \in \{1,\ldots,k\}$ be k input matrices encoded with kn^2 bits. For $i, j \in \{1,\ldots,n\}$ and $r \in \{1,\ldots,k\}$ we define variables $z_{ij}^{(r)}$ by

$$z_{ij}^{(r)} = \begin{cases} m_{ij}^{(k)} & \text{if } r = 1, \\ \bigvee_{l=1}^{n} \left(m_{il}^{(k-r+1)} \wedge z_{lj}^{(r-1)} \right) & \text{if } r \geq 2. \end{cases}$$

Then $\left(z_{ij}^{(k)}\right)_{i,j\in\{1,\ldots,n\}}$ equals $M_1 \cdots M_k$. The formulas directly yield a circuit of depth $2(k-1)$ using $n^3 \cdot (k-1)$ AND gates of fan-in two and $n^2 \cdot (k-1)$ OR gates of fan-in n. We can encode the circuit (with the gates listed topologically sorted) by first listing n^2 OR gates with n inputs, each taking the result of n AND gates for $z_{ij}^{(r)} = \bigvee_{l=1}^{n} \left(m_{lj}^{(k-r+1)} \wedge z_{il}^{(r-1)} \right)$, $r = k$, and continue with this encoding for r going

from k to 2. The encoding can be generated in deterministic log-space by using counters to ensure we generate the correct number of gates with the corresponding inputs.

Remark 3.73 It is natural to ask what share of computational power of our L-uniform circuit families is due to the uniformity mechanism and what is genuinely due to circuitry. Next, we give a class of functions for which most of the computation is carried out by the log-space machine constructing the circuits and the circuitry itself is not very complex.

It is known that transformations computable by L-uniform NC^1 circuits are computable by deterministic log-space Turing transducers [102].

Next, let $f\colon \Sigma^+ \to \Sigma^+$ be such that, for some $\Gamma \subseteq \Sigma$, we have

1. $f(u) = f(v)$ for all $u, v \in \Gamma^*$ with $|u| = |v|$,
2. $f(u) = f(v)$ for all $u, v \in \Sigma^* \setminus \Gamma^*$ with $|u| = |v|$, and
3. for $n \geq 1$ we have $f(\Sigma^n) \subseteq \Sigma^m$ for some $m \geq 1$.

Suppose f is computable by a deterministic log-space Turing transducer M. We show that it can also be computed by a L-uniform AC^0 circuit family. First, we assume Γ is a proper and non-empty subset of Σ. Suppose the words from Σ are encoded by bitstrings of fixed length $r \geq 1$.

Next, we describe a L-uniform circuit family C_n in AC^0 computing f. We can assume the input has size n. Let M' be a Turing transducer operating in the following way for the input 1^n: First compute $f(a^n)$ for some $a \in \Gamma$ and $f(b^n)$ for some $b \in \Sigma \setminus \Gamma$ by simulating M. Let $y_1 \cdots y_m$ and $z_1 \cdots z_m$ be the bits of the encoding of $f(a^n)$ and $f(b^n)$, respectively. Then let M' output the description of the following circuit:

1. There exists a fixed NC^0 circuit C detecting the encoding of a word from $\Sigma \setminus \Gamma$. We can assume the input length n is a multiple of r. Consider the circuit that results from n/r copies of C where the first copy reads the first r bits, the second the next r bits and so on until the last one reads the last r bits. Then connect the outputs of these circuits to a common OR-gate and let x be the output of the resulting circuit. We have $x = 1$ if and only if the input contains a letter from $\Sigma \setminus \Gamma$.
2. Let x_1 be the first input bit to C_n. The i-th output bit of C_n equals the following formulas, from which a circuit can be derived:

$$\begin{cases} x_1 \vee \neg x_1 & \text{if } y_i = 1 \text{ and } z_i = 1, \\ x_1 \wedge \neg x_1 & \text{if } y_i = 0 \text{ and } z_i = 0, \\ x & \text{if } y_i = 0 \text{ and } z_i = 1, \\ \neg x & \text{if } y_i = 1 \text{ and } z_i = 0. \end{cases}$$

For $\Gamma = \Sigma$ or $\Gamma = \emptyset$, we do not need the circuitry to detect letters from Γ or its complement. Thus, in this case, the value of f only depends on the length of the input and we can even give a L-uniform NC^0 family computing f.

The next claims follows from the above construction:

1. Let $L \subseteq \{0,1\}^+$ be in L and suppose for $u, v \in \{0,1\}^+$ with $|u| = |v|$ we have $u \in L$ if and only if $v \in L$, i.e., membership depends only on the length of a word. Then L is also decidable by a L-uniform family of circuits $C_1, C_2, \ldots$ in NC^0

2. Let $L \subseteq 1^+$ over $\Sigma = \{0, 1\}$ be L. Then L is decidable by a L-uniform circuit family $C_1, C_2, \ldots$ in AC^0.
3. Since the binary encoding of $|u|$ from the input u can be computed by a deterministic log-space Turing transducer, it can also be computed by a L-uniform circuit family in NC^0.

Proposition 3.74 *The composition of two L-uniform AC^0 circuit families is computable by a L-uniform AC^0 circuit family.*

Proof. Let M and M' be deterministic log-space Turing transducers computing AC^0-circuits. Let M'' be a deterministic Turing transducer that parses the description of a circuit and outputs 1^m where m denotes the number of output vertices of the circuit. For an appropriate description, M'' can operate in deterministic log-space. Since the description that M produces can be assumed to have size polynomial in the input to M, the machine M'' runs in log-space w.r.t. the input to M. Next, M' can construct a circuit with input 1^m. Since m is polynomial in the input to M, M' runs in log-space w.r.t. the input of M. As shown in [322, p. 323], the composition of deterministic log-space Turing transducers can be realized by a deterministic log-space Turing transducer. Thus the composition of M, M'' and M' can be realized by a deterministic log-space Turing transducer. Thus, we can compute the above circuits in deterministic log-space from the input to M. Then connecting the output of the first circuit to the inputs of the second circuit can also be realized in deterministic log-space. This yields that all these steps can be realized by a single deterministic log-space Turing transducer that outputs a description of the circuit resulting by feeding the second circuit with the input of the first, which is a simple modification to the outputs of M and of the composition of M, M'' and M'. □

3.9.4 Decision Problems

A *decision problem* classifies a set of inputs, encoded as a language (usually over $\{0, 1\}$), into YES- and NO-instances. In Sections 3.9.2 and 3.9.3 we already defined what it means for a Turing machine or a family of circuits to decide a language. More generally, a computational mechanism (e.g., a Turing machine or a family of uniform circuits) *decides/solves/computes* a decision problem if for each input it terminates after a finite number of steps and returns YES (accepts the input) if the input encodes a YES-instance or NO (rejects the input) if it encodes a NO-instance. To every decision problem there corresponds a language encoding the YES-instances. Conversely, to every language $L \subseteq \Sigma^*$ there corresponds a decision problem (called the *membership problem*[21] *of* L) where the YES-instances are precisely those words $w \in \Sigma^*$ for which $w \in L$, and the

[21] In the literature, a more refined terminology is used, where a distinction is drawn between the *non-uniform/fixed* and the *uniform/general membership problem* for classes of languages [321]. In the former problem the language is fixed and the input is a single word, and in the latter problem a pair consisting of a language (from some class) *and* a word forms the input. In this sense, the membership problem for L as defined here is the non-uniform membership problem.

NO-instance those with $w \notin L$. So, languages realize and encode decision problems and often we identify a language with its decision problem and vice versa without special mention.

We give a decision problem by listing its name, the possible inputs, and the property we want to check on the input, where we use a notational way introduced by Karp [361] and popularized by Garey and Johnson [238]. In each case, it is obvious that an appropriate encoding as a language is possible. In proofs, a specific input is written as a tuple listing all the parameters and called an *input instance* to the (decision) problem under consideration.

Next, we list decision problems from the literature that are of relevance to the present study.

Of importance is the satisfiability problem SAT for a set $X = \{x_1, \ldots, x_n\}$ of *Boolean variables* that we formalize in the following way as in [257, p. 314]: a *literal* is an element of $X \times \{0, 1\}$, a subset of $X \times \{0, 1\}$ is a *clause* over X and an *assignment* (of values to the variables) is a function $\alpha\colon X \to \{0, 1\}$ (also written as an n-tuple $(\alpha(x_1), \ldots, \alpha(x_n))$). We say that α *satisfies* a set of clauses C if for each clause $c \in C$ there exists $x \in V$ such that $(x, \alpha(x)) \in c$. Intuitively, see also [238, 322, Sec. 2.6], literals can be seen as variables or their negations, and we often write x and $\neg x$ (or $\overline{x}$) for $(x, 1)$ and $(x, 0)$, respectively, a clause as a disjunction of literals and a set of clauses as a conjunction of clauses. Then an assignment satisfies a set of clauses if and only if if this formula evaluates to true for the assignment. For example, the formula $(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3)$ is represented by the set of clauses $\{c_1, c_2\}$ where $c_1 = \{(x_1, 1), (x_2, 1)\}$ and $c_2 = \{(x_2, 0), (x_3, 1)\}$.

SAT (Cook [156])
Input: A set X of Boolean variables and a set C clauses over X.
Question: Does there exist a satisfying assignment for the input?

DAG (Jones [345], *Directed Acyclic Graph Accessibility Problem*)
Input: DAG $G = (V, E)$ and $s, t \in V$.
Question: Does there exist a path in G from s (start) to t (target)?

StronglyConn (Jones [345, Cor. 24])
Input: Digraph $G = (V, E)$.
Question: Is G strongly connected?

SingleCycle (Cook and McKenzie [161])
Input: Permutation presented pointwise.
Question: Does the permutation consist of a single cycle,[22] i.e., cyclically permutes its domain?

[22] Note that having only a single cycle is not the same as being a cyclic permutation. A cyclic permutation has at most one cycle of length greater than one, but might have many cycles of length one; we introduced the term cyclically permuting its domain in Section 4.2 to express the fact that the permutation has a single cycle.

OrderBetweenVertices (Etessami [203, Def. 4.1])
Input: Digraph $G = (V, E)$ where $V = \{v_1, \ldots, v_n\}$ and $E = \{(v_i, v_{i+1}) \mid i \in \{1, \ldots, n-1\}\}$ (encoded as a usual graph, without additional information pertaining to the assumed total order of the vertices)[23] and two vertices $v_i, v_j \in V$.
Question: Is $i < j$?

We also need the following variants of the above problems. 3SAT is a well-known variant of SAT [238, 361]. Let $k \geq 1$.

3SAT (Karp [361])
Input: A set X of Boolean variables and a set C of clauses over X each containing at most three literals.
Question: Does there exist a satisfying assignment for the input?

k-DAG
Input: DAG $G = (V, E)$ where the outdegree of every vertex is at most k and $s, t \in V$.
Question: Does there exist a path in G from s to t?

k-DAG-ST
Input: DAG $G = (V, E)$ and $s, t \in V$ where s has no incoming edges and precisely one outgoing edge, t has no outgoing edges and the outdegree of every other vertex is at most k.
Question: Does there exist a path in G from s to t?

3.9.5 Complexity Classes and Reductions

A *complexity class* is a class of decision problems, encoded by languages, which can be decided by some computational mechanism with imposed resource constraints. We will consider the following complexity classes (we list the resource bounded computational mechanism next to each class).

L-uniform NC^k	L-uniform circuit families in NC^k for $k \geq 0$.
L-uniform AC^k	L-uniform circuit families in AC^k for $k \geq 0$.
L	Log-space bounded deterministic Turing machines.
NL	Log-space bounded nondeterministic Turing machines.
NC	$\mathsf{NC} = \bigcup_{k \geq 0}$ L-uniform NC^k (Nick's class [160]).
P	Polynomial-time bounded deterministic Turing machines.
NP	Polynomial-time bounded nondeterministic Turing machines.
PSPACE	Polynomial-space bounded deterministic Turing machines.
NPSPACE	Polynomial-space bounded nondeterministic Turing machines.

[23] For example, encoding v_i by the number i would render the problem solvable in AC^0 as we simply have to compare two positive integers in this case. Equivalently, we can think of the input as a set together with a successor relation.

In the following, we always understand that NC^k (resp. AC^k) means L-uniform NC^k (resp. AC^k) and omit this modifier.

P can equivalently be defined by polynomial time bounded RAMs, as we can simulate a RAM with a Turing machine and vice versa with a polynomial overhead [162]. So, polynomial time bounds for RAMs imply containment in P.

Intuitively, for L and NL, a machine can, on inputs of size n, use a finite number of counters storing binary encoded values that do not exceed $O(n^c)$ for some fixed $c > 0$ and memory cells with a logarithmic number of bits to store intermediate results.

By Theorem 3.72, the decision problem to decide whether the product of k given matrices equals a given matrix is in AC^0.

Let $k \geq 1$. Most of the following inclusions are well-known (see for example [56, 57, 102, 322, 575, 619]):

$$\begin{aligned}\mathsf{AC}^0 \subseteq \mathsf{NC}^1 \subseteq \mathsf{L} \subseteq \mathsf{NL} \subseteq \mathsf{AC}^k \subseteq \mathsf{NC}^{k+1} \subseteq \mathsf{AC}^{k+1} \\ \subseteq \mathsf{NC} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PSPACE} = \mathsf{NPSPACE}. \quad (3.49)\end{aligned}$$

The equality $\mathsf{PSPACE} = \mathsf{NPSPACE}$ follows from Savitch's theorem [550]. By the above inclusions we have $\mathsf{NC} = \bigcup_{k\geq 0} \mathsf{AC}^k$.

Next, we introduce several notions of reduction. An *oracle gate* for a language $L \subseteq \Sigma^*$ in a circuit is a gate that returns one if and only if the input encodes a word in L. Let $L, L' \subseteq \Sigma^*$.

1. We say L is *log-space* (resp. *polynomial-time, L-uniform* NC^1*, L-uniform* AC^0*) many-one reducible* to L', denoted by $L \leq_m^{\log} L'$ (resp. $L \leq_m^{\mathsf{P}} L'$, $L \leq_m^{\mathsf{NC}^1} L'$, $L \leq_m^{\mathsf{AC}^0} L'$), if there exists a log-space Turing transducer (resp. polynomial-time bounded Turing transducer, L-uniform circuit family in NC^1, L-uniform circuit family in AC^0) that, given input $u \in \Sigma^*$, produces an output $v \in \Sigma^*$ such that $v \in L'$ if and only if $u \in L$. For short, we call such a mapping a $\leq_m^{\log}$- (resp. $\leq_m^{\mathsf{P}}$-, $\leq_m^{\mathsf{NC}^1}$-, $\leq_m^{\mathsf{AC}^0}$-)reduction.
2. We say L is *L-uniform* NC^1 *oracle* (or *Turing*) *reducible* to L', denoted by $L \leq_T^{\mathsf{NC}^1} L'$, if there exists an L-uniform circuit family in NC^1 deciding L that can additionally use oracle gates for L' . We call such a family an $\leq_T^{\mathsf{NC}^1}$-reduction.

If we have $L \leq_m^{\log} L'$ and $L' \leq_m^{\log} L$, then we write $L \equiv_m^{\log} L'$ and call both decision problems encoded by these languages $\equiv_m^{\log}$*-equivalent*.

The language L is *hard* for a complexity class w.r.t. $\leq_m^{\log}$-reductions if for every language L'' from the complexity class we have $L'' \leq_m^{\log} L$. It is *complete* for a complexity class w.r.t. $\leq_m^{\log}$-reductions if it is hard w.r.t. $\leq_m^{\log}$-reductions and contained in the complexity class. The same definitions apply w.r.t. $\leq_m^{\mathsf{P}}$-, $\leq_m^{\mathsf{AC}^0}$-, $\leq_m^{\mathsf{NC}^1}$- and $\leq_T^{\mathsf{NC}^1}$-reductions.

Theorem 3.75 *1. $\leq_m^{\mathsf{AC}^0}$, $\leq_m^{\log}$, $\leq_m^{\mathsf{P}}$, $\leq_T^{\mathsf{NC}^1}$, and $\leq_m^{\mathsf{NC}^1}$ are quasiorders on languages.*

2. Let $\mathcal{X}$ be any of the complexity classes L, NL, P, NP, or PSPACE and L' be a language in $\mathcal{X}$. Suppose L is $\leq_m^{\mathsf{AC}^0}$-/$\leq_m^{\log}$-/$\leq_T^{\mathsf{NC}^1}$-/$\leq_m^{\mathsf{NC}^1}$-reducible to L'. Then L is also in $\mathcal{X}$. For $\leq_m^{\mathsf{P}}$ this hold for $\mathcal{X}$ equal to P, NP, or PSPACE, and for $\leq_m^{\mathsf{AC}^0}$-reductions this holds for all complexity classes introduced.

Proof. The first claim is shown in [619, Lem. 4.81] for $\leq_m^{\log}$, $\leq_T^{\mathsf{NC}^1}$, and $\leq_m^{\mathsf{NC}^1}$, and in [322, p. 322 & 323] for $\leq_m^{\log}$ and $\leq_m^{\mathsf{P}}$. For $\leq_m^{\mathsf{AC}^0}$, the first claim follows from Theorem 3.74.

The second claim follows from the first claim and the inclusions given in Equation (3.49). □

By Theorem 3.75, a complexity class equals an equivalence class for the induced equivalence relation w.r.t. some notion of reduction, represented via complete problems.

The following statement collects complexity-theoretical results from the literature for the problems introduced in Section 3.9.4.

Theorem 3.76 *1. The problems* SAT *(Cook [156])*[24] *and* 3SAT *[361] are* NP*-complete for $\leq_m^{\log}$-reductions.*

2. DAG *(Jones [345, Cor. 25]) and* StronglyConn *([345, Cor. 24]) are* NL*-complete for $\leq_m^{\log}$-reductions.*[25]
3. SingleCycle *(Cook and McKenzie [161]),* OrderBetweenVertices *(Etessami [203, Thm. 4.1]),*[26] *and* 1-DAG *(Cook [158, p. 110] and Cook and McKenzie [161, p. 390]) are* L*-complete for $\leq_T^{\mathsf{NC}^1}$-reductions.*[27]

For the variants of k-DAG, we have the following w.r.t. $\leq_m^{\mathsf{AC}^0}$-reductions.

Proposition 3.77 *Let $k \geq 1$ be fixed. Then we have k-*DAG $\equiv_m^{\mathsf{AC}^0}$ *k-*DAG-ST*. Furthermore, for $k \geq 2$, both problems are also $\equiv_m^{\mathsf{AC}^0}$-equivalent to* DAG*.*

Proof. Clearly, k-DAG-ST $\leq_m^{\mathsf{AC}^0}$ k-DAG $\leq_m^{\mathsf{AC}^0}$ DAG.

Next, we show k-DAG $\leq_m^{\mathsf{AC}^0}$ k-DAG-ST. Suppose (G, s, t) with $G = (V, E)$ is an instance of k-DAG and let $s' \notin V$. Set

$$G' = (V \cup \{s'\}, (E \setminus (\{t\} \times V)) \cup \{(s', s)\}).$$

Then G' is a valid input for k-DAG-ST that admits a path from s' to t if and only if G admits a path from s to t.

Lastly, we prove DAG $\leq_m^{\mathsf{AC}^0}$ k-DAG for $k \geq 2$, which establishes all claims. Intuitively, given an arbitrary DAG with outdegree at most l, we use k-ary branching together with auxiliary vertices to simulate the edges of the input DAG in a DAG whose outdegree is at most k.

Let $G = (V, E)$ be some input DAG and $s, t \in V$. Suppose l is the largest outdegree of some node in G. Let $m \geq 1$ be the smallest integer such that $k^m \geq l$ and let $\Sigma = \{a_1, \ldots, a_k\}$ be an alphabet.

---

[24] Originally, Cook [156] used reductions realized by deterministic polynomial-time oracle Turing machines. Later, Jones [345] and Meyer & Stockmeyer [584] introduced $\leq_m^{\log}$-reductions and noted that the reductions given by Cook in [156] can be realized to give $\leq_m^{\log}$-reductions.

[25] In [345] NL-completeness is even shown for a finer notion of rudimentary reductions.

[26] Actually, for OrderBetweenVertices completeness was established for a weak form of first order reducibility that implies AC^0-reducibility.

[27] L-completeness of 1-DAG for other notions of reduction was shown in [282, Thm. 2.1] and [345, Cor. 25].

In the following, for $v \in V$ and $x \in \mathrm{Pref}(\Sigma^{m-1})$ we write v_x for (v, x). Let $v \in V$. Set $T_v = \{u_\varepsilon \mid (v, u) \in E\}$. Since $k^m \geq l$, there exists a surjection $f_v \colon \Sigma^m \to T_v$. Let

$$E_v = \{(v_x, v_{xy}) \mid x \in \Sigma^{<m-1} \wedge y \in \Sigma\} \cup \{(v_x, f(xy)) \mid x \in \Sigma^{m-1} \wedge y \in \Sigma\}.$$

Then $G' = (V \times \mathrm{Pref}(\Sigma^{m-1}), \bigcup_{v \in V} E_v)$ gives a DAG where every vertex has outdegree at most k and there exists a path from s to t in G if and only if there exists a path from s_ε to t_ε in G'. □

Corollary 3.78 *Let $k \geq 2$. Then k-DAG and k-DAG-ST are* NL*-complete.*

Proof. This follows from Theorems 3.76 and 3.77. □

For reference, we state the following folklore fact.

Proposition 3.79 *For two numbers given in unary encoding, their greatest common divisor can be computed in deterministic log-space. As a consequence, testing coprimality of two unary numbers is in* L*.*

Proof. We use the classical Euclidean algorithm that stores two variables.

Let a, b be given in unary, so the input length is $N = a + b + 1$ (counting a separator). Maintain two variables x, y, initialized to a and b and stored in binary. Repeatedly test whether $y = 0$ and, while $y \neq 0$, replace the pair (x, y) by $(y,\ x \bmod y)$, i.e., overwrite x with the old value of y and overwrite y with the remainder of x divided by y. Comparison against 0 and division with remainder of two $O(\log N)$-bit numbers run in $O(\log N)$ space (indeed in L-uniform TC^0 [147, 289, 290]).[28] The loop needs only the registers x, y, one temporary of $O(\log N)$ bits to form the remainder, plus $O(1)$ space for control. After every two iterations the larger operand drops by at least a factor of 2, so the loop makes $O(\log N)$ steps. When $y = 0$, the remaining value x is $\gcd(a, b)$. For coprimality, accept precisely if this value is 1.

Throughout this procedure, we have $0 \leq x, y \leq \max(a, b) \leq N$, since each remainder $x \bmod y$ is strictly smaller than the current y. So each variable occupies at most $\lceil \log_2(N + 1) \rceil \in O(\log N)$ bits, and the total work space is $O(\log N)$. □

For two binary-encoded integers, we can compute their greatest common divisor in polynomial time via the Euclidean algorithm as given in the proof of Theorem 3.79. However, the precise computational complexity for binary-encoded inputs is unknown. This problem is not known to be either in NC or P-complete [578]. For testing coprimality, the preprint [415, Thm. 1] claimed this problem to be in NC, but a fatal flaw was spotted in its proof later [571]. and the precise complexity of this problem remains open [555].

We note that finding L-uniform circuit families computing the division of two integers was an open problem for quite some time, see the survey [7].

Lastly, we show that testing strong $\frac{3}{2}$-connectivity of digraphs is NL-complete.

[28] TC^0 refers to circuits of constant depth and polynomial size that use only NOT gates and majority, AND, and OR gates of unbounded fan-in.

Proposition 3.80 *Deciding whether a given digraph is strongly $\frac{3}{2}$-connected is* NL*-complete.*

Proof. The problem is in NL, as for a digraph $G = (V, E)$ and $u, v, w \in V$, we can first guess whether a path should go from u to v or from v to u and then, successively and vertex-by-vertex, guess a path (of length at most $|V|$) from the start to the end vertex not using the vertex w. If an appropriate path is found, we have a YES-instance; otherwise a NO-instance.

Next, we give a reduction from StronglyConn, which is NL-complete by Theorem 3.76. Let $G = (V, E)$ be a given digraph as input. Set $G' = (V', E')$ with $V' = V \times \{0, 1\}$ and edge set E' equal to

$$\{((u, i), (v, i)) \mid (u, v) \in E, i \in \{0, 1\}\} \cup \{((u, 0), (u, 1)), ((u, 1), (u, 0)) \mid u \in V\}.$$

Next, we show that the following properties are equivalent, where the equivalence between 1 and 3 implies that we obtain a valid reduction:

1. G is strongly connected,
2. G' is strongly 2-connected,
3. G' is strongly $\frac{3}{2}$-connected,
4. G' is strongly connected.

For the proof, suppose G is strongly connected and let $(u, i), (v, j), (w, k) \in V'$. Without loss of generality, suppose $j = 1$. There exists a path $v_0, \ldots, v_n \in V$ from u to v in G. Then, depending on the values of i and k, we obtain a path from (u, i) to (v, j) not traversing (w, k) in the following way:

$$\begin{cases} (v_0, 0), (v_0, 1), (v_1, 1), \ldots, (v_n, 1) & \text{if } i = 0, k = 0, \\ (v_0, 1), (v_1, 1), \ldots, (v_n, 1) & \text{if } i = 1, k = 0, \\ (v_0, 0), (v_1, 0), \ldots, (v_n, 0), (v, 1) & \text{if } i = 0, k = 1, \\ (v_0, 1), (v_0, 0), (v_1, 0), \ldots, (v_n, 0), (v, 1) & \text{if } i = 1, k = 1. \end{cases}$$

So, we obtain that G' is strongly 2-connected. This implies strong $\frac{3}{2}$-connectivity, which implies strong connectivity, see Figure 3.4. Finally, suppose G' is strongly connected and let $u, v \in V$. Then, let $(v_0, 0), (v_1, i_1), \ldots, (v_{n-1}, i_{n-1}), (v_n, 0)$ be a path in G' with $i_1, \ldots, i_{n-1} \in \{0, 1\}$, $v_0 = u$, and $v_n = v$. We obtain a path from u to v in G by considering the vertices $v_1, v_1, \ldots, v_{n-1}, v_n$ and replacing parts of the form $v_i, v_{i+1}, \ldots, v_j$, $i < j$, with $v_i = v_j$ by v_i (where the order in which these parts are selected does not matter) until every vertex appears precisely once.[29] □

[29] More precisely, for each $i \in \{0, 1\}$, there exists a path from u to v in G if and only if there exists a path from (u, i) to (v, i) in G' using only vertices in $V \times \{i\}$.

3.9.6 Circuits and Encodings

Here, we discuss how we encode numbers, transformations, and, lastly, automata as inputs for circuits.

3.9.6.1 Numbers

Besides the binary encoding of a number here we also consider two types of unary encodings. In the *one-hot* (resp. *thermometer*) *unary encoding* [452] using n bits a number $0 \leq m < n$ is encoded by $0^m 10^{n-m-1}$ (resp. $1^{m+1}0^{n-m-1}$). In both representations, the value of the number increases as the rightmost 1 (when read from left to right) shifts further to the right. As a technicality, note that in both encodings, zero is represented by 10^{n-1} and not by 0^n.

Proposition 3.81 *1. There exists an L-uniform circuit family $C_1, C_2, \ldots$ in AC^0 where C_n converts a number in one-hot unary encoding using n bits to the binary encoding with $k = \lfloor \log_2(n) \rfloor + 1$ bits.*

2. There exists an L-uniform circuit family $C_1, C_2, \ldots$ in AC^0 where C_n converts the thermometer unary encoding using n bits to the one-hot unary encoding using n bits and vice versa.

3. There exists an AC^0 circuit of size $O(2^n)$ to convert an n-bit binary number to a one-hot unary encoding using 2^n bits.

Proof. 1: Let $x_0 x_1 \cdots x_n$ be the one-hot encoding of a number x. Then, for $i \in \{0, 1, \ldots, n\}$, we have $x_i = 1$ if and only if i equals x. Let $y_m \cdots y_1$ be the binary encoding of x with $m = \lfloor \log_2(n) \rfloor + 1$ and where the least significant bit is the rightmost bit y_1. Then, for $j \in \{1, \ldots, m\}$, we have $y_j = 1$ if and only if

$$k2^j + 2^{j-1} \leq x < (k+1)2^j$$

for some $k \geq 0$. Thus, for $j \in \{1, \ldots, m\}$, we obtain

$$y_j = \bigvee_{\exists k \colon k2^j + 2^{j-1} \leq i < (k+1)2^j} x_i,$$

i.e., y_j is the OR over all x_i with $i \in \{0, 1, \ldots, n\}$ where the j-th bit of i is one. These formulas can be translated into AC^0 circuits that are computable by deterministic log-space Turing transducers. [30]

2: The one-hot encoding $x_0, x_1, \ldots, x_{n-1}$ and the thermometer encoding $y_0, y_1, \ldots, y_{n-1}$ of a number are related by $y_i = x_i \vee \cdots \vee x_{n-1}$ and $x_i = y_i \wedge \neg y_{i+1}$ where we set $y_n = 0$ for $i \in \{0, \ldots, n-1\}$. These formulas can be implemented by L-uniform AC^0 circuit families.

[30] This is called an n-to-k binary encoder with $k = \lfloor \log_2 n \rfloor + 1$. A different circuit commonly used for this conversion is the priority encoder described in [375, p. 117], which also allows "don't-cares".

3: Suppose $x_{n-1}, \ldots, x_1, x_0$ are the bits encoding a binary number with x_0 being the least significant bit. Let $y_0, y_1, \ldots, y_{2^n-1}$ be the bits for the output in that order. Then, for $i \in \{0, 1, \ldots, 2^n - 1\}$, we have $y_i = z_{i,n} \wedge \cdots \wedge z_{i,1}$ where, for $j \in \{1, \ldots, n\}$,

$$z_{i,j} = \begin{cases} x_{j-1} & \text{if the } j\text{-th bit of the binary encoding of } i \text{ is one,} \\ \neg x_{j-1} & \text{if the } j\text{-th bit of the binary encoding of } i \text{ is zero.} \end{cases}$$

Intuitively, the bit pattern $x_{n-1}, \ldots, x_1, x_0$ is matched against the binary encoding of i via a minterm.[31] We have

$$y_i = 1 \Leftrightarrow \text{the bit pattern } x_{n-1}, \ldots, x_1, x_0 \text{ encodes } i.$$

Now, each y_i uses a single AND gate of fan-in n and at most n NOT gates. So, this is an AC^0 circuit of size $O(2^n)$. □

By Theorem 3.74 and Item 2 from Theorem 3.81 for the unary encoding we can chose the one-hot or thermometer encoding, whatever is more convenient, up to AC^0-reductions.

The input to our problems is often some numerical parameter together with other data. An additional parameter can be encoded in the input along with other data in the following ways.

1. Most of the time the parameter can be assumed to be bounded by some monotone function $f(n)$ of the size of the data n. For example, if the parameter bounds a shortest synchronizing word of an input automaton, then it can be assumed to be bounded by a cubic polynomial, see Table 1.1. In this case, the total length of the input is the sum of n and $f(n)$ (or $\log_2(f(n))$ for a binary encoding) and n can be extracted from the input length. Thus, the parameter is deducible from the total length of the input. If this is possible in deterministic log-space, then for our L-uniform circuit families we can assume the input is split into designated bits for the parameter and for the rest of the data.
2. If the parameter can be arbitrarily large even if the remaining data is fixed, then the parameter cannot be encoded in the length of the input. In this case, it has to be encoded into the input in such a way that it can be separated from the remaining data. For example, by encoding the total input in the form $1^m 0 v$ where 1^m encodes the parameter and v the rest of the input. Then the circuits have to use additional circuitry to parse the input and separate the parameter from the remaining data.

In reductions, when we assume a unary encoding for a parameter we assume it to have the form 1^m for some $m \geq 0$.[32]

---

[31] This is a conjunction of variables and negated variables that evaluates to one for precisely one specific assignment to the variables.

[32] However, when the input is a single unary number without additional data, this assumption makes no sense since then we always just feed 1^n as input to the circuits. This is precisely the input the Turing machine constructing the circuit sees and thus for NC^1 and below, the computation can already be performed by the Turing machine and the circuitry does not give more computational power. So, in this case, the number of bits and the specific value encoded in the input bitstring must be distinguished.

3.9.6.2 Transformations

A transformation $t\colon [n] \to [n]$ can be represented by a transformation matrix consisting of n^2 binary values or by the *point-wise representation*, which lists the values $t(0), t(1), \ldots, t(n-1)$ where we assume each $t(i)$ uses the same number of bits. With this convention for the point-wise representation, the parameter n can be easily extracted solely from the length of the input, which has size n^2 if we use a unary encoding for each $t(i)$ or $n\lfloor \log_2(n) + 1 \rfloor$ if we use a binary encoding.

For the transformation $t\colon [n] \to [n]$, for $i \in [n]$, $t(i)$ in one-hot encoding equals the i-th row of the transition matrix of t w.r.t. the natural order on $[n]$. Thus, both representations are essentially the same, and we can also easily obtain the point-wise representation in terms of the thermometer unary encoding from this representation.

Next, we show that we can also convert the point-wise representation using a binary encoding and the representation by transition matrices into each other by L-uniform AC^0-circuits.

Theorem 3.82 *There exists a family of L-uniform AC^0 circuits computing the conversion of the point-wise representation of a transformation with a binary encoding into the representation by transition matrices and vice versa.*

Proof. Let $t\colon [n] \to [n]$. We can assume $n \geq 1$. First, suppose t is given by a point-wise representation $t(0), \ldots, t(n-1)$ where $t(i)$ for $i \in [n]$ is encoded in binary. By our convention, each $t(i)$ for $i \in [n]$ uses $\lfloor \log_2(n) + 1 \rfloor$ bits, so that the encoding uses $n\lfloor \log_2(n) + 1 \rfloor$ bits in total.

The AC^0 circuits for the conversion of the binary encoding of $t(i)$ to a one-hot unary encoding from Theorem 3.81, Item 3 have size $O(n)$ and they produce, for $t(i)$, the bits $y_{i+1,0}, y_{i+1,1}, \ldots, y_{i+1,2^m-1}$ storing the one-hot encoding of $t(i)$ as a word $y_{i+1,0}y_{i+1,1}\cdots y_{i+1,2^m-1} \in \{0,1\}^+$. We have $2^{m-1} \leq n \leq 2^m - 1$. Thus, since $t(i) \in [n]$, we find $y_{i+11,n+1} = \cdots = y_{i+1,2^m-1} = 0$. Then the matrix $(y_{i,j})_{i,j\in\{1,\ldots,n\}}$ equals the transition matrix of t. Inspection of the proof of Theorem 3.81, Item 3 gives that the above circuits can be generated by a deterministic log-space Turing transducer on input $1^{\lfloor \log_2(n)+1 \rfloor}$ since it can easily compute n from the input length and use this parameter in the construction of the circuit.

Conversely, suppose the transition matrix $y = (y_{ij})_{i,j\in\{1,\ldots,n\}}$ of t is given. The i-th row $y_{i1}, \ldots, y_{in}$ corresponds to a unary one-hot encoding of $t(i-1)$. So, by Theorem 3.81, Item 1, we can compute binary encodings of $t(0), t(1), \ldots, t(n-1)$ by L-uniform AC^0 circuits. □

3.9.6.3 Automata

Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton. A reasonable idea is to represent $\mathcal{A}$ as a list of transition matrices for each letter.

A list of transition matrices can be specified by $|\Sigma|n^2$ bits where the first n^2 bits correspond to the first letter, the next n^2 bits to the next letter and so on for some ordering of the alphabet.

For a fixed alphabet, a deterministic log-space Turing machine can easily separate the input into the individual transition matrices, i.e., identify the first n^2 bits, then the next n^2 bits and so on, and it just needs the total length $|\Sigma|n^2$ of the input to perform this task.

However, in our problems, the alphabet is also part of the input. In this case, the input length alone does not determine the size of the alphabet or the number of states, since multiple possibilities exist for each given length.

Hence, a machine that constructs circuits using the length of the input as parameter cannot create a circuit with fixed input gates for each transition matrix. The circuit must determine from the input which bits constitute the individual transition matrices. For the circuit to do this, we need some way to supply this information in the input, for example, by separating the individual transition matrices.

There are several schemes to achieve this. For example, we can encode every 0 by 00, every 1 by 11 and use 01 and 10 as separating sequences between transition matrices. There are also more clever schemes using so-called self-delimiting descriptions that encode the length of the individual parts (here the transition matrices) into the input, for example, see [412, Sect. 1.4] or [411, Sect. 2.2]. However, it is not immediately evident how an AC^0-circuit can parse such inputs to identify the individual transition matrices.

For this reason, we use another encoding that is easy for AC^0 circuits to process, but uses $(n + |\Sigma|)^3$ bits (and therefore, uses more space than the schemes discussed above). Let R_δ be the ternary relation over the universe $\Sigma \cup Q$ such that $R_\delta(x, y, z)$ holds if and only if x represents a letter a_x, y represents a state q_y, z represents a state q_z, and $\delta(q_y, a_x) = q_z$. Then an automaton is supplied to a circuit as the bitstring of length $(n + |\Sigma|)^3$ encoding R_δ.

It is well-known that properties that are testable by families of AC^0 circuits are precisely those checkable by first order formulas [333, Thm. 5.22]. We use this equivalence by giving first order sentences to show containment in AC^0. The only technicality left is to argue that this is also possible with our notion of L-uniformity. We argue next that, given a first order formula, a deterministic log-space Turing transducer can construct an AC^0 circuit family where each circuit evaluates to true on a given input of size $(n+|\Sigma|)^3$ if and only if the first order formula holds for the automaton encoded in the input.

Suppose a first order formula quantifying over the elements in $Q \cup \Sigma$ and using only the predicate R_δ is given. A Turing transducer, given $1^{(n+|\Sigma|)^3}$ as input, can compute the size of the universe $n + |\Sigma|$. Furthermore, for numbers $i, j, k \in \{1, \ldots, n + |\Sigma|\}$ it can also determine the bit that encodes $R_\delta(i, j, k)$. With this information, an all- or exists-quantifier can be translated into an unbounded AND or OR-gate for the input of size $(n + |\Sigma|)^3$, respectively. Using this, by induction over the structure of the formula, the transducer can construct an AC^0-circuit.

Next, we show that, when giving formulas to show containment in AC^0, we can, without loss of generality, quantify only over letters or states, and can access entries of the transition matrices of $\mathcal{A}$, thus giving us some "syntactic sugar" in comparison to our plain formulas where quantification is always over $Q \cup \Sigma$ and that only uses the relation R_δ.

Consider the two unary relations R_Q and R_Σ where $R_Q(x)$ (resp. $R_\Sigma(x)$) is true precisely if x is a state (resp. a letter). The relations R_Σ and R_Q can be specified by

the first order formulas $\bigvee_{z_1} \bigvee_{z_2} R_\delta(x, z_1, z_2)$ and $\bigwedge_{z_1} \bigwedge_{z_2} \neg R_\delta(x, z_1, z_2)$, respectively. Thus, they can be used in first order formulas. Furthermore, we can use them to quantify only over the letters or only the states. For example, the following formula, querying whether an automaton has a trap state,

$$\bigvee_{a \in \Sigma} \left(\bigwedge_{q \in Q} R_\delta(a, q, q) \right),$$

can be translated into

$$\bigvee_{x \in U} \left(R_\Sigma(x) \rightarrow \left(\bigwedge_{y \in U} R_Q(y) \rightarrow R_\delta(x, y, y) \right) \right).$$

This gives us the mean to write simpler formulas. Note that if x represents a letter a and y, z states, then $R_\delta(x, y, z)$ corresponds to an entry in the transition matrix of a. Thus, we can directly reference entries of the transition matrices of a letter in first order formulas. We will often do so, but using a simpler notation, for example denoting the transition matrices by $(m_{i,j})_{i,j \in \{1,\dots,n\}}$ for $a \in \Sigma$ and using these expressions in first order formulas instead of R_δ.

Remark 3.83 All of the representations discussed above, and the representation by giving the transformation of the letters in point-wise representation in binary with some mechanism of separation between the letters, are readily convertible into each other by deterministic log-space Turing machines. Thus, when using $\leq_m^{\log}$-reductions, we may assume either of these representations.

Remark 3.84 The representation of structures by relations and constants over a finite universe, and their relation to logic and computation, is explored in descriptive complexity, see [333]. In [333, Def. 2.1] the encoding of these structures by binary sequences is discussed. Here, we encoded letters and states in a common universe. In this respect, we also refer to the modeling of SAT in [333, Ex. 2.18] where the elements of the universe each model variables and clauses.

3.10 Remarks on Different Notions of Reduction

The notion of $\leq_{\log m}$-reduction is too strong to classify problems inside L, since the reduction itself already has the computational power of L. For this reason, we consider weaker (and finer) notions such as $\leq_{NC^1}^T$- and $\leq_{AC^0}^m$-reductions.[33]

[33] There does not seem to be a universal convention on when to call one form of reduction weaker or stronger. Our convention appears to be the prevalent one, although the opposite convention, where the roles are reversed, is also used in the literature, for example in [144].

Clearly, a $\leq_m^{\mathsf{AC}^0}$-reduction is also an $\leq_T^{\mathsf{NC}^1}$-reduction and a $\leq_m^{\log}$-reduction.[34] Most of the time, the reductions we present here are $\leq_m^{\mathsf{AC}^0}$-reductions. But often, concluding results are formulated for $\leq_T^{\mathsf{NC}^1}$- or $\leq_m^{\log}$-reductions only. The reason is that we often use results from the literature that are only stated for these stronger forms of reduction.

To make statements regarding complexity classes more concise, we omit mentioning the type of reduction and, when nothing else is said, adopt the following conventions.

1. When talking about complete or hard problems for NL, P, NP or PSPACE, we mean this with respect to $\leq_m^{\log}$-reductions.
2. When talking about complete or hard problems in L, we mean this with respect to $\leq_T^{\mathsf{NC}^1}$-reductions.
3. When talking about circuit complexity classes and families of circuits, as already said on page 72, when nothing else is said, we always mean the L-uniform variants.

Regarding the first two assumptions, when citing results from the literature we occasionally say explicitly what kind of reduction is involved. Also, complexity classification results stated in Part IV involve different reducibility notions. For example, NP- and L-completeness statements are w.r.t. $\leq_m^{\log}$- and $\leq_T^{\mathsf{NC}^1}$-reducibility, respectively. However, having a single (weak) form of reduction is preferable, as it would imply stronger separation results, see [10, Sect. 23.6.1] for a short discussion. We reserve the question whether our complexity classification results hold for a common notion of reduction for future work.

Remark 3.85 Most of our reductions can be expressed by simple operations on input automata that are computable by AC^0 circuits. For example, these include adding letters according to a fixed transition scheme, adding auxiliary states with transitions derived from the given ones, taking Cartesian products, or rewiring the transition structure of an input automaton. However, we often omit the full bit-level details of these reductions, as this would lead to lengthy and technical proofs.

3.11 Remarks on the Order of Function Composition

Our convention for function composition deviates from the one adopted in permutation group theory or in some approaches to the algebraic theory of automata. As this can be confusing at times, we feel the urge to justify and comment on that. The notation and terminology introduced in the present section will not be included in the notation index or the index of terminology, as it is only used here.

When applying a function $f\colon X \to Y$ to some $x \in X$, we can write the argument to the right, i.e., $f(x)$, and f acts on the left (left action), or we can write the argument to the left, i.e., $(x)f$ (xf for short), and f acts from the right (right action). In the present work, we use the left-action convention (introduced on page 3). Furthermore,

[34] However, $\leq_T^{\mathsf{NC}^1}$-reductions are *not* known to be $\leq_m^{\log}$-reductions. This is because of the difference between oracle and many-one reductions. For example, the former allows complementation of the results of oracle queries, see also the discussion in [10, Sect. 23.6].

when defining the composition of two functions $f\colon X \to Y$ and $g\colon Y \to Z$, either the function on the right-hand side can be applied first (right-most first, as defined here, see page 31), also called the *Leibniz order*, or the function on the left-hand side, i.e., define a composition operator by $(f;g)(x) = g(f(x))$ for $x \in X$ (left-most first), also called the *anti-Leibniz* or *diagrammatic order*.

In [468] also the notation f^*g for $g \circ f$ was mentioned as pulling back g along f to the composite. Here, the asterik operator makes $f\colon X \to Y$ into a function $f^*\colon Z^Y \to Z^X$ defined by $(f^*(g))(x) = g(f(x))$ for $x \in X$. So, conceptually, rather than regarding composition as a binary operation on functions, here we rather consider each function as an operator acting via composition (this is reminiscent of letting each element of a monoid act via multiplication from left or right, see [491, Prop. 4.25]). In the wild we have never seen anyone using this notation solely for the purpose of having a notation for function composition, but either in rather specialized contexts as operators on special function spaces as, for example, in differential geometry [579, p. 77 & p. 89], as the composition operator in functional analysis [166], or in the context of Koopman Operator Theory [180].

Now, the following combinations of conventions are conceivable.

1. Right-action and left-most first (also called "postfix convention").
   With these conventions, we have $x(f;g) = (xf)g$. This combination of conventions was in wide use in the early days of the theory of automata, for example in the books [194, 195, 247, 489]. It has the advantage of being in harmony with reading a diagram $X \xrightarrow{f} Y \xrightarrow{g} Z$ for the composition from left to right. In [194, p. 1], it is even noted that $x \in X$ can be interpreted as a function from some singleton set into X with value x and then application of f to x can be written as function composition and modeled by diagrams depicting compositions. However, these conventions seemed to be out of fashion in automata theory at least since the mid-nineties. In permutation group theory, this combination of conventions is often used [126, 179, 638], even taken a step further by using the *exponential notation* for function application, i.e., writing x^f for f applied to x, see for example [393].
2. Left-action and right-most first.
   This is the combination of conventions we adopted (see pages 3 and 31). Outside of specialized areas of mathematics such as the algebraic theory of automata or permutation group theory (or other branches of algebra as for example [217]), this seems to be the prevalent notion. For example, three famous textbooks from diverse fields: Lang's *Algebra* [405], Spivak's *Calculus on Manifolds* [579] or Königsberger's *Analysis I* [379] (the latter being a standard text for first year math students in Germany, at least it was during the author's time) all use these conventions.
3. Left-action and and left-most first.
   The textbook by Almeida [11] from 1995 uses this convention, the survey article by Pin [490] and, for example, a randomly sampled article by Diekert et al. [175]. Interestingly, function composition is not explicitly defined in these works. However, they have to use this convention, for otherwise, some cited results from this field have to be formulated differently. For example, partially ordered automata are precisely those automata having an $\mathcal{R}$-trivial transition monoid [111] (we refer to

the cited literature for definitions). But this result is only valid, due to the definition of the $\mathcal{R}$-relation via right ideals, when the left-most function is applied first in a composition (otherwise, it would correspond to $\mathcal{L}$-triviality).
A recent (work-in-progress, available online) textbook by Pin [491] solved this by using two composition operations. To cite [491, Sect. II.4.2]:

> [...] the reader should be warned that the product $\alpha\beta$ is not equal to $\alpha \circ \beta$, but to $\beta \circ \alpha$. In other word, the operation in $\mathcal{T}(P)$ is reverse composition. [...]

Another reason to adopt this convention is that it uses the common way of writing functions on the left, i.e., $f(x)$, while retaining compatibility with the *composition of relations* when defined in the following way: for a relation R from X to Y and a relation S from Y to Z, the composition is the relation RS defined, for all $x \in X$ and $z \in Z$, by

$$x(RS)z \Leftrightarrow \exists y \in Y \colon xRy \wedge ySz.$$

Now, when associating with $f \colon X \to Y$ the relation R_f from X to Y by $xR_f y$ if and only if $f(x) = y$ we have $R_{(f;g)} = R_f R_g$ for functions $f \colon X \to Y$, $g \colon Y \to Z$. This is, in particular, better suited when functions are defined as relations (which we have not done in the present work since we introduced function before as the more primitive notion as in [194]), or when representing relations by matrices, as this convention fits with matrix multiplication. For example, in teaching first-year computer science students at Trier University [208] these conventions are adopted, as relations are introduced as a more primitive concept and (partial) functions are defined as special relations.
Of course, one can define the composition of relations by reversing the order (this is done in [491, Sect. I.1.2]). However, in this case compatibility with matrix multiplication mentioned in the previous paragraph is lost (and, to the best of our knowledge, matrix multiplication is defined in the same way in all of mathematics) and a correct formula would involve additional matrix transpositions.

4. Right-action and right-most first. We have never seen this combination in use.

Furthermore, given an automaton $\mathcal{A} = (Q, \Sigma, \delta)$, some authors choose to suppress mentioning δ explicitly in the specification and application of the transition function, writing $q.u$ for $\delta(q, u)$ with $q \in Q$ and $u \in \Sigma^*$. This dot-notation actually fits well with the right-action and left-most first conventions, as then $q.u = (q)\delta_u = (q)(\delta_v; \delta_w) = ((q)\delta_v)\delta_w$ for $u = vw$, $u, v, w \in \Sigma^*$, $q \in Q$. However, we have opted against this dot-notation mainly because in Part IV often two transition functions (of a constraint automaton and of an input automaton) appear in the same context. Hence we felt the need to distinguish them notation-wise. Then, in our opinion, when applying a transition function $\delta \colon Q \times \Sigma^* \to Q$ explicitly, the left action convention seems to fit better. Also, our convention adopted for composition was motivated by what we feel is most often used in all of theoretical computer science and formal language theory (apart from specialized areas such as the algebraic theory of automata).

Note that "operators" such as $\mathrm{Syn}(\mathcal{A})$ or $Z(G)$, which are never composed with other "operators", are almost always written as acting from the left, resulting in a mix of both conventions in some works as for example in [29, 393].

Since Part III of the present thesis involves permutation group theory, we were faced with the choice what convention to adopt. In face of Part IV, which does not involve any permutation group theory at all and is a more classical formal language theory topic, we opted to the left-action convention that is more common in formal language theory. This convention has implications on how we define conjugation later. For more discussion on the two conventions of composition, see [468] (the notation $f;g$ is taken from this article).

3.12 Discussion

The primary purpose of the present chapter was to fix notation and introduce results from the literature. For more (well-known) properties of functions and compositions, the first chapter from [491] gives a nice overview.

Our algebraic notions are standard. The textbooks by Clifford and Preston [152, 153] and Ljapin [419] are two standard references from the early days of semigroup theory. More recent books are written by Howie [325] or Higgins [291]. For more information on algebra in general, we refer to the textbook [405]. The book by Lallement discusses semigroup theory while pointing out connections to formal language theory [401]. Semirings find application in generalizations of automata theory and formal language theory to weighted automata and formal power series, see [391, 88]. Idempotent elements were introduced in [478] and also appear widely throughout ring and module theory, for example, in the study of algebras [478, 402, 27], and in particular the proof of the Wedderburn-Artin classification theorem for semisimple algebras [44, 402, 630], and in semigroup theory in connection with Green's relations, see [152, 153, 325]. Idempotency is also studied in the context of analysis, see [378]. In geometrical settings or linear algebra, idempotent linear transformations are also called projections.

The textbook by Ganyushkin & Mazorchuk [236] is devoted to transformations semigroups. The first chapter of [195] recaps some folklore results from transformation semigroup theory and develops this theory further. Clifford & Preston [152, 153] also discuss transformation semigroups.

The textbooks [194, 322, 388, 422, 491, 575] contain content on formal language theory. Classes of languages with effectively computable semilinear Parikh images, as for example context-free languages [149, 474] or languages accepted by reversal-bounded multicounter machines [326], have applications in formal verification and model checking since this property yields decidability of the emptiness problem for these languages. See the introduction of [314] for details. The earliest work that investigated commutative languages that we were able to trace back is [399].

An introduction to order theory is [169]. In Theorem 3.32 we gave a bijective order morphism that is not an isomorphism. Furthermore, not every bijective endomorphism is an automorphism, for example, see [392]. (We note that this holds for finite sets, since then some power of the endomorphism yields the identity.)

Our notions and results from graph theory are standard as can be found in any textbook, for example, as [177, 278], except the notion of strong $\frac{3}{2}$-connectivity, which is new.

The systematic study of computational complexity was initiated in [284]. Turing machines were introduced in [608], where also nondeterministic machines were mentioned (called choice machines therein), but not considered further. Nondeterminism as a computational mechanism was introduced later in [502]. There are a wealth of textbooks available on computational complexity theory, for example [41, 56, 57, 322, 575, 388].

Below we discuss some more issues and literature related to computational complexity in more detail. By doing so, often new notions are mentioned in passing (for example, alternating Turing machines) without formally introducing them. In such instances, we always refer to appropriate literature where the interested reader can find definitions.

Running Time, Complexity Theory, and Machine Models.

We have introduced the following machine models: the random access machine (RAM), (sequential) Turing machines and transducers, and circuits. In theoretical computer science, Turing machines and transducers are the most traditional model, circuits serve to classify problems with low complexity or to describe problems solvable by employing parallelism. The RAM model is often used for claims about runtime and space that can be more directly translated to bounds for real-life implementations. The other two models are used for theoretical investigations into the inherent difficulty of problems and for a finer classification not possible with the RAM model by the, rather powerful, constant time assumption on the basic (arithmetic) operations. Clearly, the practical and theoretical perspectives are strongly related. We cannot expect problems inherently difficult in the theoretical sense to have exact algorithms that run practically fast in all instances. For more discussion on the practical implications of computational complexity theory, see [6]. RAMs and Turing machines can both be used to define P [162]. Furthermore, there are also theoretical investigations on the RAM model, see [26, 280, 364, 423, 479, 566, 587] to name a few, and it has been adapted to capture low-level complexities [26, 270, 362] or investigated when it is universal, i.e., capable of executing other programs, called the Random Access Stored-Program Machine (RASP) [280, 162]. Lastly, we note that there exist complexity classes that appear reasonable, but are very sensitive to the machine model. For example, the class of problems solvable in linear time, i.e., $O(n)$-bounded time, see [270]. Hence, runtime claims made for the RAM model do not necessarily translate into natural complexity classes.

In classical complexity theory, various types of reduction have been considered. To name a few more than the ones introduced in Section 3.9.5:

1. Polynomial-time Turing reductions [156].
2. Polynomial-time many-one reductions [361].
3. Log-space many-one reductions [345, 584], log-space linear many one reductions [584], and rudimentary reductions [345] (the latter notion has been used to characterize DLOGTIME-uniform AC^0 [8]).
4. One-way log-space reductions [282].
5. First-order reductions [331, 167] (these reductions equal DLOGTIME-uniform AC^0-reductions, see [333])

6. Projections (non-uniform, each bit of the output depends on at most one bit of the input) [576].
7. AC^0 truth-table reductions [144, 232].
8. NC^1 Turing reductions [160],
9. First order translations/projections [332], which equal DLOGTIME-uniform NC^0-reductions [2].

More information on notions of reduction can be found in [9].

Resource-Bounded Computations.

In Section 3.9.2 we defined resource-bounded (time or space) Turing machines by demanding that all computational runs stay within the resource bounds. This convention is also used in [322, p. 288], [345, Sec.2], [575, Def. 7.9], and [41, Sec. 2.1.2]. Following conventions for alternating Turing machines from [143, 394], in [636, p. 5], for resource-bounded computations it is only required that, for accepted inputs, at least one accepting computational run stays within the resource bounds, and no constraints are imposed on computational runs that do not accept. This convention is also used in [57, Def. 2.11]. We note that in [86] yet another convention was used, where for accepted inputs, at least one accepting computational runs is required to stay within the resource bounds, and for not accepted inputs all terminating computational runs must stay within the resource bounds.

It is well known that, for reasonable resource-bounding functions commonly considered in computational complexity theory, all of the above definitions of resource-bounded machines are equivalent. Next, we will show this for multitape Turing machines and the classes of *fully space-constructible* [322, p. 297]) and *fully time-constructible* [322, p. 299] functions, which are functions $f\colon \mathbb{N}_0 \to \mathbb{N}_0$ such that there exists a deterministic one-tape Turing machine that, for each input u, visits exactly $f(|u|)$ tape cell and makes exactly $f(|u|)$ moves, respectively. For example, the polynomial functions and the standard exponential functions are both fully time-constructible and fully space-constructible. Logarithms are not fully time-constructible since we need at least linear time to read the input, but they are fully space-constructible by using the b-adic representation of numbers.[35]

Let $s\colon \mathbb{N}_0 \to \mathbb{N}_0$ with $s(n) \geq \log_2(n)$ be fully space-constructible witnessed by a deterministic Turing machine M_s that uses exactly $s(n)$ tape cells for each input u with $n = |u|$. Let M be a Turing machine that, for an accepted input u, has at least one accepting computational run that takes up at most $s(n)$ space. Let m and t be the number of states and tape symbols, respectively, of M. If M accepts, it does so by a sequence of at most $(n+2)ms(n)t^{s(n)}$ moves (input head position, state, working head position, tape content), since otherwise some configuration repeats. We construct a new Turing machine accepting precisely the same inputs and for which all computational runs terminate and stay within the space bounds given by s in the following way:

[35] In [57, Def. 2.18], these functions are simply called time- and space-constructible, whereas in [322] the same terms refer to weaker notions.

1. For an input u, we mark out $s(n)$ tape cells by using M_s.
2. We simulate M where we halt and reject when it leaves those marked cells.
3. We maintain a counter on a separate tape that counts each step in base $4mt$. When this counter reaches $(4mt)^{s(n)}$ we halt and reject.

Since $(4mt)^{s(n)} \geq (n+2)ms(n)t^{s(n)}$, if the counter is reached, a configuration of the machine repeats. Thus, the counter prevents that the machine gets stuck in a loop. The overhead created by the counter is logarithmic and inside the space bounds demanded by $s(n)$ since $s(n) \geq \log_2(n)$.[36] The machine can also be simulated by a one-tape Turing machine without any additional space overhead [322, Proof of Thm. 7.2].

Next, let $t\colon \mathbb{N}_0 \to \mathbb{N}_0$ be fully time-constructible witnessed by a deterministic Turing machine M_t. Consider a nondeterministic Turing machine M that has, for accepted words u, at least one accepting computational run using at most $t(|u|)$ moves. Construct a new Turing machine that simulates M and also performs one step of M_t for each simulated time step of M using a separate work tape, and that rejects inputs on which M_t halts and M has not reached a final state. This gives a factor of two overhead in time. By the linear speed-up theorem [322, Thm. 12.3], we obtain a multitape Turing machine deciding the language and halting on each computational run for input u in at most $t(|u|)$ steps. From this, we can also obtain a one-tape Turing machine halting on each run in at most $t^2(|u|)$ steps [322, p. 292].

We note that in the seminal paper [284] that started the systematic study of the computational complexity of algorithms, time and space measuring functions are only required to be computable.

Circuits.

Circuits and Boolean algebra entered into computing with Shannon's master thesis [559] (see also the paper [560] based on it) where he showed how to use Boolean algebra for the synthesis and analysis of switching circuits and relays. One of the first papers investigating the relation of circuits to languages seems to be [157]. Pippenger [492] observed that size and[37] depth bounds for families of circuits correspond to resource bounds (see also [102, 523, 585] expanding these relations). He introduced the class NC in [492] to formalize this observation (the name NC is due to Cook [157, p. 338], see also [160], as an abbreviation for "Nick's class").

Uniformity is imposed on circuit families in order to define classes of languages that correspond to machine-based classes. L-uniformity (also called log-space uniformity) was introduced by Cook [157, p. 338] (see also [619, p. 47] or [56, Def. 4.2]), based on

[36] A related results is similarly proved in [322, Lem. 12.1]. However, in this book, resource-bounded machines are defined to always stay within the bounds, not just in case of acceptance. With this assumption, the proof from [322, Lem. 12.1] works for any space bounding function (not just the fully space-constructible ones). The trick is the dynamically enlarge the space used for the counter along with the number of cell i used by the simulated machine, and test whether it reaches $(4mt)^i$.

[37] Both parameters have to be considered for circuits as, by the conjunctive or disjunctive normal forms, every Boolean function can be realized by a logarithmic depth circuit using only gates of fan-in two. However, such a circuit might have exponential size.

a similar definition from [102, p. 738] (see also some related discussion in [159, 160]). We defined L-uniformity by demanding a machine that works in logarithmic space in the number of input bits. This was also done in [157, p. 338] and [7, p. 4].[38] In [619, p. 47] and [56, Def. 4.2] it was demanded that the Turing transducer constructing the circuits works in logarithmic space in the size of the circuits, but this makes no difference for our circuit families in NC. We note that in [7, 157, 619, 56], for circuit families $C_1, C_2, \ldots$ it was also assumed C_n has n input bits, where here we only demand the family to have a strictly increasing number of input bits.

In the present work, we only use L-uniformity as our form of uniformity, as it is easy to use and captures complexity classes below L. Apart from L-uniformity, other notions of uniformity have been studied in the literature, for example:

1. Ruzzo [523] introduced U_D-uniformity, U_E-uniformity, and U_{E^*}-uniformity for a circuit family $C_1, C_2, \ldots$ by requiring not that the circuits C_n itself are computable, but that a circuit connection language is decidable by, respectively, an $O(s_n)$-time deterministic Turing machine, an $O(s_n)$-time and $O(\log s_n)$-space deterministic Turing machine, and an $O(d_n)$-time and $O(\log s_n)$-space alternating Turing machine, where s_n and d_n denote the size and depth of C_n.
2. Barrington et al. [62] introduced DLOGTIME-uniformity where a random-access[39] deterministic Turing machine can decide queries about n-bit encodings of circuits in time $O(\log n)$.
3. P-uniformity [72] requiring that $1^n \mapsto C_n$ (n number of input bits here) is computable by a deterministic polynomial time Turing machine (see also [5] for a systematic study).
4. Borodin [102] introduced uniformity in depth $S(n)$ that requires that there exists a deterministic $S(n)$-space Turing transducer computing $1^n \mapsto C_n$ (n number of input bits here) and that the resulting circuit C_n has depth $O(S(n))$ (This is also called U_B-uniformity in [523, 634]).

Languages definable by first order formulas are precisely those decidable by nonuniform AC^0 circuit families [61, Thm. 4] and [332, Thm. 6.2]. DLOGTIME-uniform AC^0 corresponds to first order logic with a bit-query predicate, equality, and order [62], [61, Thm. 5], [333, Thm. 5.22]. More generally, for a class C of functions containing the DLOGTIME-computable functions and closed under composition, C-uniform AC^0 has the same expressibility as first order logic using predicates computable by functions in

[38] Originally, in [157] Cook demanded that the function $1^n \mapsto C_n$ for circuits of size $s(n)$ is computable by a transducer Turing machine in $O(\log s(n))$. (This is also called U_{BC}-uniformity in [523, 634].) For circuits in NC this equals our notion of computability by a log-space transducer measured in the number of input bits.

[39] We have introduced Turing machines as sequential devices. However, when studying sublinear-time computation, as in [62], this model is inadequate as we cannot read the whole input. A random-access Turing machine is appropriate for studying sublinear time and consists of a read-only input tape, a fixed number of sequential read-write work tapes, and a special sequential read-write *address tape*. At each computation step, the machine can read the input bit whose position is encoded (in binary) on the address tape, or determine that the encoded address exceeds the input length.

Sequential Turing machines can simulate these machines with only polynomial time overhead and logarithmic space overhead. Hence, for defining the classical complexity classes L, NL, P, NP, and PSPACE, it makes no difference whether one uses the sequential or the random-access model.

C [61, Cor. 6]. This yields that log-space uniformity is equivalent to first order theory using only predicates computable by deterministic log-space Turing machines.

The nonuniform class AC^0 was introduced in [232] and it was shown that the parity function is not computable by an AC^0 circuit family. Furthermore, the same paper introduced AC^0-Turing reductions and reduced the computation of majority, multiplication and the transitive closure to the computation of the parity function. In [3] it was shown that no first order formula with equality and a finite number of predicates evaluates to true precisely on an even subset of the universe. By the mentioned correspondence between AC^0 and first order formula this yields another proof that the parity function is not in AC^0. These results hold for non-uniform AC^0.

We remark that it is unknown whether the containment $\mathsf{NC}^1 \subseteq \mathsf{L}$ is strict, but this is generally believed. For example [168] investigates the complexity landscape between NC^1 and L. Also, it is not known whether the inclusion $\mathsf{NL} \subseteq \mathsf{NC}^2$ is strict, see also the discussion in [523].

A standard reference on circuits is the book by Wegener [634] (the "blue book"). It also features an extensive chapter on the Parallel RAM (PRAM) machine model of parallel computation. However, it contains very little material on complexity classes related to circuits and only a single short section (without proofs) mentioning uniformity notions and relations to alternating Turing machines. The book by Vollmer [619] offers more on these topics and treats uniformity thoroughly. The PRAM is also described in [225, 341]. Clote and Kranakis's book [154] covers circuit complexity classes but without much discussion on uniformity issues. But it offers additional material on randomness and proof systems. More advanced (and recent) material can be found in the book by Jukna [347]. Greenlaw et al. [262] have written a book on P-completeness that also contains much background material on NC and parallel computation. Circuit complexity classes also have strong ties to logic and descriptive complexity. We refer to the book by Immerman [333], which is a standard reference for these topics, for more information on this connection. The book by Straubing [586] focuses on the relation between logic, circuits up to NC^1 and finite automata, but does not include material beyond that, e.g., on the whole of NC. The relation between automata and circuits from NC^1 was first observed by Chandra et al. [142] and developed further by Barrington [60], Barrington and Thérien [63] and Barrington et al. [64]. The two volumes [56, 57] by Balcázar, Díaz, and Gabarró give a good coverage of circuits in complexity theory, whereas the first volume covers the non-uniform setting and the second volume the uniform setting. A selection of several contributions to a symposium on Boolean function complexity can be found in Paterson's book [475].

Chapter 4
Classes of Transformations

Here, we investigate classes of transformations. Sections 4.1, 4.2, 4.3, 4.4, and 4.5 introduce and investigate permutations, cycles, one-point singulars, and idempotent transformations, respectively. In Section 4.6, we investigate idempotent one-singulars. We will present different characterizations. Later, the transformations that arise in the study of automata theory result from compositions of transformations induced by letters. For this reason, we also investigate resulting transformations when composing different types of transformations.

4.1 Permutations and Quasipermutations

A *permutation* is a bijective transformation. Recall that, in this case, as discussed in Section 3.2.1, there exists an *inverse transformation* $t^{-1}\colon X \to X$ such that $t^{-1}t = tt^{-1} = \mathrm{id}_X$.

For an arbitrary transformation $t\colon X \to X$ and $S \subseteq X$ we say that *t permutes a subset $S \subseteq X$* if $t_{|S}$ is a permutation on S. We call $t\colon X \to X$ a *quasipermutation* [257] if it permutes its image $t(X)$.

First, in Theorem 4.1 we collect several well-known characterizations of permutations on finite sets. Then, the remaining statements investigate quasipermutations.

Proposition 4.1 *Let $t\colon [n] \to [n]$. Then the following are equivalent.*

1. *t is a permutation.*
2. $\mathrm{dupl}(t) = \emptyset$*, i.e., t is injective.*
3. $t^{-1}(\mathrm{dupl}(t)) \cap t([n])$ *is empty.*
4. *t has no root point.*
5. $\mathrm{def}(t) = 0$.
6. *Every state is a cycle state.*
7. *There exists $i > 0$ such that $t^i = \mathrm{id}_{[n]}$.*
8. $\mathrm{rank}(t) = n$*, i.e., t is surjective.*

Proof. $1 \Rightarrow 2$: This follows as a permutation is injective by definition.

$2 \Rightarrow 3$: This is obvious.

$3 \Rightarrow 4$: Let $y \in [n]$ be a root point. Then $y \in \operatorname{dupl}(t)$ and $t^p(y) = y$ for some $p > 0$. This gives $t^{p-1}(y) \in t^{-1}(\operatorname{dupl}(t)) \cap t([n])$.

$4 \Rightarrow 5$: Suppose $\operatorname{def}(t) > 0$. Then there exists $x \in \operatorname{excl}(t)$. By finiteness, there exists a minimal $i \geq 0$ such that $t^i(x) = t^{i+p}(x)$ for some $p > 0$, where, since $x \in \operatorname{excl}(t)$, we must have $i > 0$. Set $y = t^i(x)$. We have $t^p(y) = y$. By the minimal choice of i, we have $t^{i-1}(x) \neq t^{i+p-1}(x)$ and both map to the same point y. Hence we have $y \in \operatorname{dupl}(t)$. Thus y is a root point.

$5 \Rightarrow 6$: By Item 3 from Theorem 3.14, we have $|\operatorname{dupl}(t)| \leq \operatorname{def}(t) = 0$, i.e. t is injective. By finiteness there exist $i \geq 0$ and $p > 0$ with i minimal such that $t^i(x) = t^{i+p}(x)$. For $i > 0$, by injectivity, we obtain $t^{i-1}(x) = t^{i-1+p}(x)$, contradicting the minimal choice of i. So $i = 0$ and $x = t^p(x)$. Thus x is a cycle state.

$6 \Rightarrow 7$: By assumption for each $x \in [n]$ there exists $i_x > 0$ such that $t^{i_x}(x) = x$. Then $t^{\operatorname{lcm}\{i_x \mid x \in [n]\}} = \mathrm{id}_{[n]}$.

$7 \Rightarrow 8$: By assumption we have $x = t(t^{i-1}(x)) \in t([n])$ for each $x \in [n]$. So $t([n]) = [n]$, which gives $\operatorname{rank}(t) = n$.

$8 \Rightarrow 1$: By Equation (3.15) from Theorem 3.12 we have $n = \operatorname{rank}(t) = n + \operatorname{def}(t)$. Thus $\operatorname{def}(t) = 0$. By Item 3 from Theorem 3.14, we have $|\operatorname{dupl}(t)| \leq \operatorname{def}(t) = 0$, i.e. t is injective. □

Corollary 4.2 *Let $t\colon X \to X$ and $S \subseteq X$ be finite. Then t permutes S if and only if $t(S) = S$.*

Proposition 4.3 *Let $t\colon [n] \to [n]$. Then the following are equivalent.*

1. *t is a quasipermutation.*
2. *For each $x \in X$, $t(x)$ is a cycle point.*
3. *For each $x \in \operatorname{excl}(t)$, $t(x)$ is a cycle point.*
4. *The set of tail points equals* $\operatorname{excl}(t)$.
5. $\operatorname{dupl}(t_{|t([n])}) = \emptyset$.

Proof. $1 \Rightarrow 2$: By assumption, $t_{|t(X)}$ is a permtuation. Then the claim follows from Item 6 of Theorem 4.1 applied to the permutation $t_{|t(X)}$.

$2 \Rightarrow 3$: This is clear.

$3 \Rightarrow 4$: Clearly, states in $\operatorname{excl}(t)$ are tail points. For a tail point x we have $x = t^i(z)$ for an excluded point z and some $i \geq 0$. By assumption, $i = 0$ and so $x = z$. Thus $\operatorname{excl}(t)$ equals precisely the tail points.

$4 \Rightarrow 5$: Assume $t_{|t(X)}$ is not injective. Then $t(t(x)) = t(t(z))$ for $x, z \in [n]$ with $t(x) \neq t(z)$. Since a cycle point has precisely one predecessor on its cycle, one of both points $t(x)$ or $t(z)$ must be a tail point, and this point is $t(X)$.

$5 \Rightarrow 1$: We have $t(t([n])) \subseteq t([n])$. By assumption, t acts injectively on $t(X)$. This yields $|t(t([n]))| = |t([n])|$ and thus we obtain $t(t([n])) = t([n])$. □

Lemma 4.4 *Let $t\colon [n] \to [n]$ be a quasipermutation. Then*

$$|t^{-1}(y)| = |t^{-1}(y) \cap \operatorname{excl}(t)| + 1 \quad \textit{for } y \in t([n]),$$

and $t^{-1}(y) = \emptyset$ otherwise. In particular $\operatorname{dupl}(t) = t(\operatorname{excl}(t))$.

Proof. As $t(t([n])) = t([n])$ and $t([n])$ is finite, $t_{|t([n])}$ is a permutation of $t([n])$. Hence every $y \in t([n])$ has exactly one preimage in $t([n])$, and the remaining preimages lie in $\operatorname{excl}(t) = [n] \setminus t([n])$, which gives the displayed equality; points outside $t([n])$ have no preimage. Consequently $|t^{-1}(y)| \geq 2$ if and only if $t^{-1}(y)$ meets $\operatorname{excl}(t)$, that is, if and only if $y \in t(\operatorname{excl}(t))$. □

Proposition 4.5 *Let $t\colon [n] \to [n]$ be of defect at most two. If $\operatorname{dupl}(t) = t(\operatorname{excl}(t))$, then t is a quasipermutation.*

Proof. If there exists $x \in \operatorname{dupl}(t)$ not contained in a cycle, then $x = t^i(y) = t^j(z)$ for two distinct excluded points $y, z \in \operatorname{excl}(t)$ and $i, j \geq 1$. By Item 4 of Theorem 4.1 there exists another point in $\operatorname{dupl}(t)$ contained in a cycle, which must be the image of yet another excluded point by assumption. Then t would have defect at least three. Thus every duplicate point is contained in a cycle. Then, with $\operatorname{dupl}(t) = t(\operatorname{excl}(t))$, the image of every excluded point is a cycle point. Thus, with Theorem 4.3, t is a quasipermutation. □

4.2 Cycles and Cycles Induced by a Transformation

Let X be some set and $a_0, a_1, \ldots, a_{m-1} \in X$ be $m \geq 1$ distinct elements. Then the permutation $(a_0\ a_1\ \cdots\ a_{m-1})_X\colon X \to X$ given by

$$(a_0\ a_1\ \cdots\ a_{m-1})_X(x) = \begin{cases} a_{(j+1) \bmod m} & \text{if } x = a_j \text{ for some } j \in \{0, 1, \ldots, m-1\}, \\ x & \text{otherwise,} \end{cases}$$

is called an *m-cycle* or *cycle* of length m.[1]

Equivalently, $t\colon X \to X$ is an m-cycle if and only if $m \leq 1$ and $t = \mathrm{id}_X$ or $m = |\operatorname{move}(t)|$ and for $x, x' \in \operatorname{move}(t)$ there exists $i \geq 0$ such that $t^i(x) = x'$. When the context is clear, we often omit the index X and simply write $(a_0\ a_1\ \cdots\ a_{m-1})$. We do not use commas in the cycle notation between points, to distinguish it from an ordinary m-tuple. When writing a $1 \times n$ matrix as in Figure 3.3 we obtain the same notation as for a cycle, but in each case the context should make it clear whether we refer to a cycle or a $1 \times n$ matrix. For each $a \in X$, we have $(a) = () = \mathrm{id}_X$. The following equations hold:

$$(a_0\ a_1\ \cdots\ a_{m-2}\ a_{m-1}) = (a_1\ a_2\ \cdots\ a_{m-1}\ a_0) \tag{4.1}$$

$$(a_0\ a_1\ \cdots\ a_{m-1}) = (a_1\ \cdots\ a_{m-1})(a_0\ a_{m-1}) = (a_0\ a_2\ \cdots\ a_{m-1})(a_0\ a_1) \tag{4.2}$$

$$(a_0\ a_1\ \cdots\ a_{m-1}) = (a_{m-2}\ a_{m-1})(a_0\ a_1\ \cdots\ a_{m-3}\ a_{m-1}) \tag{4.3}$$

$$(a_0\ a_1\ \cdots\ a_{m-1}) = (a_0\ a_1)(a_1\ \cdots\ a_{m-1}) \tag{4.4}$$

$$(a_0\ a_1\ \cdots\ a_{m-2}\ a_{m-1}) = (a_{m-2}\ a_0)(a_{m-2}\ a_{m-1})(a_0\ \cdots\ a_{m-3}) \tag{4.5}$$

[1] Note that for $X \neq \{a_0, a_1, \ldots, a_m\}$ the cycle has fixpoints. Some authors exclude this possibility and stipulate $X = \{a_0, a_1, \ldots, a_m\}$.

And for $0 \leq i < j < m$ we have:

$$(a_0\ a_1\ \cdots\ a_{m-1})(a_i\ a_j) = (a_0\ a_1\ \cdots\ a_i\ a_{j+1}\ a_{j+1} \cdots a_{m-1})(a_{i+1}\ \cdots\ a_j)$$
$$(a_i\ a_j)(a_0\ a_1\ \cdots\ a_{m-1}) = (a_0\ a_1\ \cdots\ a_{i-1}\ a_j\ a_{j+1}\ \cdots\ a_m)(a_i\ \cdots\ a_{j-1})$$

A *transposition* is a 2-cycle. Two cycles $t, h\colon X \to X$ are *disjoint* if $\mathrm{move}(t) \cap \mathrm{move}(h) = \emptyset$, in this case we have $th = ht$. Let $t\colon X \to X$ be some transformation. For finite $S \subseteq X$, we say that t *cyclically permutes* S if t permutes S and $t_{|S}$ is an $|S|$-cycle. Then $t^{|S|}(x) = x$ for all $x \in S$. A permutation on a set of size at least two cyclically permutes its domain if and only if it is a cycle with no fixpoints. If $t_{|S} = (a_0\ a_1\ \ldots\ a_{m-1})$ we also say that t *induces the cycle*, or *contains the cycle*, $(a_0\ a_1\ \cdots\ a_{m-1})$. Equivalently, $t(a_j) = a_{(j+1) \bmod m}$ for all $j \in \{0, 1, \ldots, m-1\}$, i.e., $(x\ t^2(x)\ \cdots\ t^{m-1}(x)) = (a_0\ a_1\ \ldots\ a_{m-1})$. For $x \in X$ and $m \geq 1$, the points $x, t(x), \ldots, t^{m-1}(x)$ form a cycle induced by t if and only if m is minimal such that $t^m(x) = x$. Two distinct cycles induced by t are disjoint.

Proposition 4.6 *A permutation on a finite set equals the composition (in any order) of its induced cycles.*

Proof. Let $t\colon [n] \to [n]$ be a permutation with induced cycles $t_1, \ldots, t_m$. For $x \in [n]$, we have $t_i = (x\ t(x) \cdots t^{k-1}(x))$ for some minimal $k \geq 1$ with $t^k(x) = x$ and unique $i \in \{1, \ldots, m\}$. Furthermore, for $j \neq i$, we also have $t_j(x) = x$. Thus, the result of any composition of $t_1, \ldots, t_m$ maps x to $t_i(x) = t(x)$, and so this composition equals t. □

Corollary 4.7 *A permutation on a finite set can be written as a composition of transpositions.*

Proof. This follows from Equation (4.2) and Theorem 4.6. □

4.3 Tail Transformations, Simple Idempotents, and Cycles

Here, we introduce tail transformations and simple idempotents. We then classify the transformations obtained by composing a simple idempotent with a cycle. Theorem 4.13 bounds the length of cycles induced in the composition of a simple idempotent with a cycle.

Let $a_1, \ldots, a_m \in X$ be distinct numbers with $m \geq 2$. The *tail transformation (of* $a_1, \ldots a_m$*)* is the transformation $[a_1 \mapsto a_2 \mapsto \cdots \mapsto a_m]_X\colon X \to X$ given by

$$[a_1 \mapsto a_2 \mapsto \cdots \mapsto a_m]_X(x) = \begin{cases} a_{i+1} & \text{if } x = a_i \text{ for } i \in \{1, \ldots, m-1\}, \\ x & \text{otherwise.} \end{cases}$$

When the context is clear, we often omit X and simply write $[a_1 \mapsto a_2 \mapsto \cdots \mapsto a_m]$.

Proposition 4.8 *Let $a_1, \ldots, a_m \in X$ be pairwise distinct and $1 < i < m$. Then*

$$[a_1 \mapsto a_2 \mapsto \cdots \mapsto a_m] = [a_1 \mapsto a_2 \mapsto \cdots \mapsto a_i][a_i \mapsto a_{i+1} \mapsto \cdots \mapsto a_m]$$
$$t[a_i \mapsto a_{i-1}] = [a_i \mapsto a_{i+1}]t = [a_i \mapsto a_{i+1} \mapsto \cdots \mapsto a_m][a_1 \mapsto a_2 \mapsto \cdots \mapsto a_i]$$

where $t = [a_1 \mapsto a_2 \mapsto \cdots a_{i-1} \mapsto a_{i+1} \mapsto \cdots \mapsto a_m]$.

A *simple idempotent* is a tail transformation involving two distinct numbers, i.e., where $m = 2$.[2] So, a simple idempotent is written as $[a \mapsto b]$ for distinct $a, b \in X$. In [128] a similar notation was used. A transformation $t\colon X \to X$ is a simple idempotent if and only if $\mathrm{move}(t)$ is a singleton set.

Proposition 4.9 *For pairwise distinct* $a, b, c, d \in X$*, the following equations hold:*

$$[a \mapsto b][a \mapsto b] = [a \mapsto b] = [a \mapsto b][b \mapsto a] \tag{4.6}$$
$$[c \mapsto d][a \mapsto b] = [a \mapsto b][c \mapsto d] \tag{4.7}$$
$$[a \mapsto c][a \mapsto b] = [a \mapsto b] \tag{4.8}$$
$$[c \mapsto b][a \mapsto b] = [a \mapsto b][c \mapsto b] = [a \mapsto b][c \mapsto a] \tag{4.9}$$
$$[c \mapsto a][a \mapsto b][b \mapsto c] = [c \mapsto b][b \mapsto a][a \mapsto c][c \mapsto b] \tag{4.10}$$
$$[d \mapsto c][c \mapsto a][a \mapsto b][b \mapsto c] = [d \mapsto a][a \mapsto b][b \mapsto d][d \mapsto c][c \mapsto b] \tag{4.11}$$

Equation (4.9) also follows from Theorem 4.8 for $m = 3$, but is kept for reference. The equations from Theorem 4.9 yield a presentation in terms of generators and relations for the set of all non-permutations on a finite set [188].

Corollary 4.10 *Let* $a, b, c, d \in X$ *with* $a \neq b$ *and* $c \neq d$ *and* $[a \mapsto b] \neq [c \mapsto d]$*. Then we have* $[c \mapsto d][a \mapsto b] = [a \mapsto b][c \mapsto d]$ *if and only if* $a \neq c$ *and* $b \neq c$ *and* $d \neq a$*, i.e., both simple idempotents commute if and only if* a, b, c, d *are all distinct or* $b = d$ *and* a, b, c *are all distinct.*

Proof. For $a = c$, since $[a \mapsto b] \neq [c \mapsto d]$ we have $d \neq b$ and both simple idempotents do not commute by Equation (4.8). Next, suppose $b = c$. For $a = d$, both do not commute by Equation (4.6). For $a \neq d$, we have $[a \mapsto c][c \mapsto d] = [a \mapsto c \mapsto d] \neq [c \mapsto d][a \mapsto c]$.

Conversely, suppose $a \neq c, b \neq c, d \neq a$, i.e., $a \notin \{c, d\}$ and $b \neq c$. If $b \neq d$, i.e., all points are pairwise distinct, we have $[a \mapsto b][c \mapsto d] = [c \mapsto d][a \mapsto b]$ by Equation (4.7). For $b = d$, we have $[a \mapsto b][c \mapsto b] = [c \mapsto b][a \mapsto b]$ by Equation (4.9). □

Let $t = [a \mapsto b]$ for $a \neq b$. Then $\mathrm{dupl}(t) = \{b\}$, $\mathrm{excl}(t) = \{a\}$, and for finite $S \subseteq X$ we have

$$|t(S)| < |S| \Leftrightarrow \{a, b\} \subseteq S \Leftrightarrow S = t(S) \cup \{a\}, \tag{4.12}$$
$$|t(S)| = |S| \Leftrightarrow t(S) = S \vee (b \notin S \wedge a \in S \wedge t(S) = (S \cup \{b\}) \setminus \{a\}). \tag{4.13}$$

[2] Also called a *singular* transformation in [107].

The following equations encompass, up to a renaming of the points, all combinations to compose a simple idempotent with a cycle. Equations (4.14) and (4.19) are illustrated in Figure 4.1.

Proposition 4.11 *Let $m \geq 2$, $\{0, 1, \ldots, m\} \subseteq X$, and $0 < d \leq m$. Then we have*

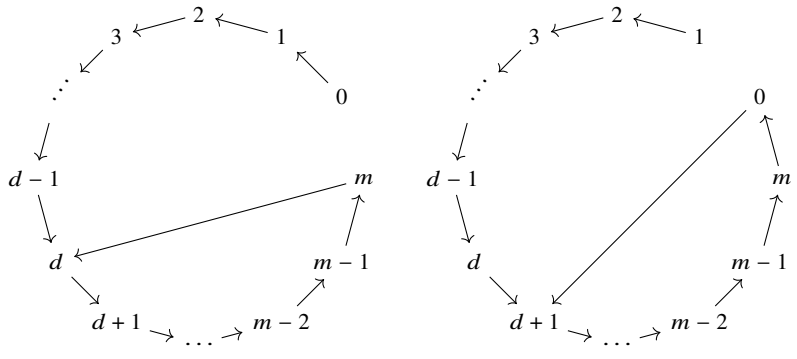
$$\begin{aligned}
[0 \mapsto d](0\ 1\ \cdots\ m) &= [0 \mapsto \cdots \mapsto d-1 \mapsto d](d\ d+1\ \cdots\ m) && (4.14)\\
&= (d\ d+1\ \cdots\ m)[0 \mapsto \cdots \mapsto d-1 \mapsto m] && (4.15)\\
&= (0\ 1\ \cdots\ m)[m \mapsto d] && (4.16)\\
&= [m \mapsto d][0 \mapsto 1 \mapsto \cdots \mapsto m] && (4.17)\\
&= [0 \mapsto 1 \mapsto \cdots \mapsto m][m \mapsto d-1] && (4.18)
\end{aligned}$$

and for distinct $a, b \in X \setminus \{0, 1, \ldots, m\}$ we have

$$\begin{aligned}
(0\ 1\ \cdots\ m)[0 \mapsto d] &= [1 \mapsto 2 \mapsto \cdots \mapsto d+1](0\ d+1\ \cdots\ m) && (4.19)\\
(0\ 1\ \cdots\ m)[0 \mapsto a] &= [1 \mapsto \cdots \mapsto m \mapsto 0 \mapsto a] && (4.20)\\
[0 \mapsto a](0\ 1\ \cdots\ m) &= [0 \mapsto 1 \mapsto \cdots \mapsto m \mapsto a] && (4.21)\\
(0\ 1\ \cdots\ m)[a \mapsto b] &= [a \mapsto b](0\ 1\ \cdots\ m) && (4.22)
\end{aligned}$$

Corollary 4.12 *For $m \geq 2$ we have the following:*

$$[0 \mapsto 1](0\,1\,\cdots\,m) = [0 \mapsto 1](1\,2\,\cdots\,m) = (1\,2\,\cdots\,m)[0 \mapsto m] = (0\,1\,\cdots\,m)[m \mapsto 0]$$



(a) $[0 \mapsto d](0\ 1\ \cdots\ m)$. (b) $(0\ 1\ \cdots\ m)[0 \mapsto d]$.

Fig. 4.1: Functional graphs resulting from the two possible compositions of $(0\ 1\ \cdots\ m)$ with $[0 \mapsto d]$ where $0 < d \leq m$, see Equations (4.14) and (4.19).

Corollary 4.13 *Let $t\colon X \to X$ be an m-cycle and $h\colon X \to X$ be a simple idempotent. Then there exist $0 \leq i, j \leq m$ and $0 < p, q < m$ such that $(ht)^i = (ht)^{i+p}$ and $(th)^j = (th)^{j+q}$.*

Proof. We can assume $m \geq 2$. This follows from the equations given in Theorem 4.11, since the results of both compositions move at most $m + 2$ points, contain at most one cycle moving more than one point, and such a cycle moves at most m points, or it equals a tail transformation moving at most $m + 1$ points. □

4.4 One-Point Singular Transformations

A transformation $t\colon X \to X$ is said to be *one-point singular* [524] if $\mathrm{dupl}(t)$ is a singleton. As a slight abuse of language, using an adjective as a noun, we call a one-point singular transformation simply a one-point singular.

To give a few examples, simple idempotents and, by Theorem 3.14, Item 3, defect-one transformations on finite sets are one-point singulars. Figure 4.2 depicts the functional graph of an example one-point singular.

Theorems 4.14 and 4.15 provide characterizations of transformations that are injective or one-point singulars, the latter investigating the case of a finite domain. Theorem 4.16 gives a characterization in the special case of quasipermutations. Theorem 4.18 characterizes when the composition of two transformations gives a one-point singular. The special case when both transformations are one-point singulars is given in Theorem 4.23.

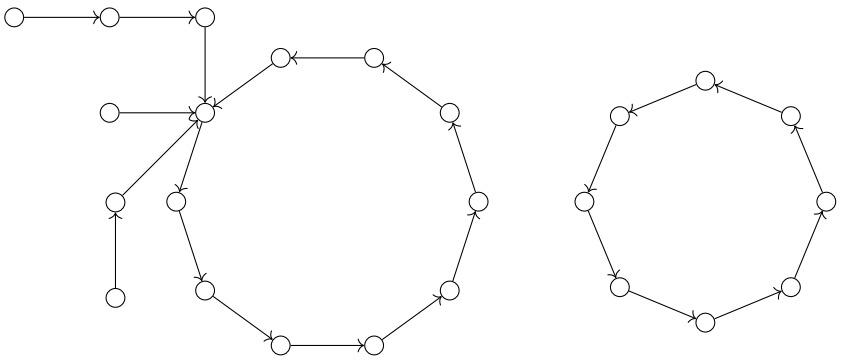


Fig. 4.2: Functional graph of a one-point singular transformation.

Proposition 4.14 *Let $t\colon X \to X$. Then the following are equivalent.*

1. *t is injective or a one-point singular.*
2. *for all $x, x', z, z' \in X$, if $x \neq x'$, $z \neq z'$, $t(x) = t(x')$, and $t(z) = t(z')$, then $t(x) = t(z)$.*

Proof. This follows directly from the definitions. □

Proposition 4.15 *Let $t\colon [n] \to [n]$. Then the following are equivalent.*

1. *t is a permutation or a one-point singular.*
2. $|t^{-1}(\mathrm{dupl}(t))| \leq \mathrm{def}(t) + 1$.
3. *t is a permutation or* $|t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t) + 1$.
4. *There exists* $y \in [n]$ *such that* $|t^{-1}(y)| = \mathrm{def}(t) + 1$.
5. *t has kernel type* $(\mathrm{def}(t) + 1, 1, \ldots, 1)$.
6. *t has kernel type* $(k, 1, \ldots, 1)$ *for some* $k \geq 1$.

Proof. $1 \Rightarrow 2$: By Theorem 3.14, Item 1 we have $|t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t) + |\mathrm{dupl}(t)|$. Since, by assumption, $|\mathrm{dupl}(t)| \leq 1$, we obtain the claim.

$2 \Rightarrow 3$: We have $|t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t) + 1$ or $|t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t)$. In the latter case, from Theorem 3.14, Item 1, we get $|\mathrm{dupl}(t)| = 0$. So t is a permutation.

$3 \Rightarrow 4$: Clearly, the implication is true for a permutation. Otherwise, by Theorem 3.14, Item 1 we have $\mathrm{def}(t) + |\mathrm{dupl}(t)| = |t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t) + 1$. So, $|\mathrm{dupl}(t)| = 1$ and $|t^{-1}(y)| = \mathrm{def}(t) + 1$ for the unique $y \in \mathrm{dupl}(t)$.

$4 \Rightarrow 5$: Let $y' \in [n] \setminus \{y\}$. Then we have

$$\begin{aligned} \mathrm{def}(t) &\geq |t^{-1}(\{y, y'\})| - 2 && \text{(Theorem 3.14, Item 2)} \\ &= |t^{-1}(y)| + |t^{-1}(y')| - 2 && (|t^{-1}(\{y, y'\})| = |t^{-1}(y)| + |t^{-1}(y')|) \\ &= \mathrm{def}(t) + |t^{-1}(y')| - 1. && (|t^{-1}(y)| = \mathrm{def}(t) + 1) \end{aligned}$$

Hence $|t^{-1}(y')| \leq 1$.

$5 \Rightarrow 6$ & $6 \Rightarrow 1$: These implications are obvious. □

Proposition 4.16 *Let* $t\colon [n] \to [n]$ *be a quasipermutation. Then t is a permutation or a one-point singular if and only if there exists* $y \in [n]$ *with* $\mathrm{excl}(t) \subseteq t^{-1}(y)$.

Proof. By Theorem 4.4 we have $\mathrm{dupl}(t) = t(\mathrm{excl}(t))$. By assumption, we have $|\mathrm{dupl}(t)| \leq 1$. Now $|t(\mathrm{excl}(t))| \leq 1$ holds if and only if $t(\mathrm{excl}(t)) \subseteq \{y\}$ for some $y \in [n]$, which is $\mathrm{excl}(t) \subseteq t^{-1}(y)$. □

Remark 4.17 Theorem 4.16 might fail if t is not a quasipermutation. For example, let $t\colon \{1, 2, 3, 4\} \to \{1, 2, 3, 4\}$ with $t(1) = t(3) = 2$ and $t(2) = t(4) = 4$. Then $\mathrm{excl}(t) = \{1, 3\} \subseteq t^{-1}(2)$, but t has kernel type $(2, 2)$.

In the remainder of this section let $t, h\colon X \to X$ and put $Z = t(X) \setminus \mathrm{dupl}(t)$. Note that a restriction $h_{|Y}$ with $Y \subseteq t(X)$ need not be a transformation, as its image need not be contained in Y. Moreover, as $\mathrm{dupl}(t) \subseteq t(X)$, injectivity of $h_{|t(X)}$ implies $|h(\mathrm{dupl}(t))| = |\mathrm{dupl}(t)|$; in particular, $h(\mathrm{dupl}(t))$ is then a singleton set if and only if t is a one-point singular.

By Equation (3.11), it is easy to see that ht is a one-point singular precisely if $h(\mathrm{dupl}(t)) \subseteq \mathrm{dupl}(h_{|Z})$ where $\mathrm{dupl}(h_{|Z})$ is a singleton set, or $\mathrm{dupl}(h_{|Z}) \subseteq h(\mathrm{dupl}(t))$ where $h(\mathrm{dupl}(t))$ is a singleton set. By Equation (3.11), a similar statement holds with $t(X)$ in place of Z. In fact, since $\mathrm{dupl}(h_{|Z}) \subseteq \mathrm{dupl}(h_{|Y}) \subseteq \mathrm{dupl}(h_{|t(X)})$ and $\mathrm{dupl}(h_{|Y \setminus Z}) \subseteq h(\mathrm{dupl}(t))$ for $Z \subseteq Y \subseteq t(X)$ as $Y \setminus Z \subseteq \mathrm{dupl}(t)$, we can substitute any set Y with $Z \subseteq Y \subseteq t(X)$ for Z and obtain a valid characterization for ht being a one-point singular. Here, we replaced Z in each disjunct by the same set and the question might arise, under what circumstances can we replace Z in each disjunct independently,

or whether we can find similar statement where this is possible. For example, it can be shown that, for each $Z \subseteq Y_1, Y_2 \subseteq t(X)$, ht is a one-point singular if and only if t is injective and $\mathrm{dupl}(h_{|Z_1})$ a singleton, or $\mathrm{dupl}(h_{|Z_2}) \subseteq h(\mathrm{dupl}(t))$ where $h(\mathrm{dupl}(t))$ is a singleton. In the following we give a more detailed analysis driven by this question.

Proposition 4.18 *Let $t, h\colon X \to X$ and $Z \subseteq Y \subseteq t(X)$. Set*

$$A = h(\mathrm{dupl}(t)), \qquad D_S = \mathrm{dupl}(h_{|S}) \text{ for } S \subseteq t(X), \qquad B_Y = h(Z) \cap h(Y \setminus Z).$$

Then ht is a one-point singular if and only if exactly one of the following cases occurs.

1. *t is injective and D_Z is a singleton set.*
2. *$h_{|t(X)}$ is injective and t is a one-point singular.*

3_Y. *$B_Y \neq \emptyset$, $h_{|Z}$ is injective, and t is a one-point singular.*[3]

$3_{\neg Y}$. *$h_{|t(X)}$ is not injective, $B_Y = \emptyset$, $h_{|Z}$ is injective, and t is a one-point singular.*

4_Y. *$B_Y \neq \emptyset$ or $D_{Y\setminus Z} \neq \emptyset$, $h_{|Z}$ is injective, $|A| = 1$, and t is not a one-point singular.*

$4_{\neg Y}$. *$B_Y = D_{Y\setminus Z} = \emptyset$, $h_{|Z}$ is injective, $|A| = 1$, and t is not a one-point singular.*

5. *$D_Z = A$ and this set is a singleton set.*

Which cases can occur depends on Y. For $Y = t(X)$, where $Y \setminus Z = \mathrm{dupl}(t)$, the cases $3_{\neg Y}$ and $4_{\neg Y}$ cannot occur; writing 3 and 4 for 3_Y and 4_Y, we are left with the five mutually exclusive cases 1, 2, 3, 4, 5. For $Y = Z$ the cases 3_Y and 4_Y cannot occur, and for $Z \neq Y \neq t(X)$ only the cases 4_Y, $4_{\neg Y}$ and 5 can occur.

See Table 4.1 for a list of the cases with the above sets.

Proof. That the listed cases are mutually exclusive is easy to see. Next, we use $\mathrm{dupl}(ht) = A \cup D_Z$ from Equation (3.11) and

$$D_Y = D_Z \cup D_{Y\setminus Z} \cup B_Y \tag{4.23}$$

from Equation (3.4). As $Y \setminus Z \subseteq \mathrm{dupl}(t)$, we have $D_{Y\setminus Z} \subseteq A$ and $B_Y \subseteq A$.

Let ht be a one-point singular, that is, $|A \cup D_Z| = 1$. If t is injective, then $A = \emptyset$, so D_Z is a singleton set: case 1. Otherwise $A \neq \emptyset$, whence $|A| = 1$ and $D_Z \subseteq A$. If $D_Z = A$, we are in case 5; so let $D_Z = \emptyset$, that is, $h_{|Z}$ be injective. If t is a one-point singular, then injectivity of $h_{|t(X)}$ gives case 2, and otherwise we are in case 3_Y or $3_{\neg Y}$ according to whether $B_Y \neq \emptyset$ or not. If t is not a one-point singular, we are in case 4_Y or $4_{\neg Y}$ according to whether $B_Y \neq \emptyset$ or $D_{Y\setminus Z} \neq \emptyset$ holds or not.

Conversely, each case gives $|A \cup D_Z| = 1$: in case 1 we have $A = \emptyset$ with D_Z a singleton set, in case 5 we have $A \cup D_Z = A$, and in the other five we have $D_Z = \emptyset$ and $|A| = 1$, the latter in case 2 since t is a one-point singular.

For $Y = t(X)$ we have $Y \setminus Z = \mathrm{dupl}(t)$. In case $3_{\neg Y}$ then Equation (4.23) reads $D_{t(X)} = B_Y$, as $D_Z = \emptyset$ and $D_{\mathrm{dupl}(t)} = \emptyset$ for a singleton set $\mathrm{dupl}(t)$; but the left-hand side is non-empty as $h_{|t(X)}$ is not injective. In case $4_{\neg Y}$ we have $|\mathrm{dupl}(t)| \geq 2$, as $A \neq \emptyset$ and t is not a one-point singular, so h identifies the points of $\mathrm{dupl}(t)$ as $h(\mathrm{dupl}(t)) = A$ is a singleton set, giving $D_{Y\setminus Z} = A$. Both contradict the respective case hypotheses. For $Y = Z$ we have $Y \setminus Z = \emptyset$ and hence $B_Y = D_{Y\setminus Z} = \emptyset$, excluding 3_Y and 4_Y. For

[3] Note that $D_{Y\setminus Z} = \emptyset$ if t is a one-point singular, as then $Y \setminus Z$ is contained in the singleton set $\mathrm{dupl}(t)$.

$Z \neq Y \neq t(X)$, such a Y exists only if $\operatorname{dupl}(t)$ has at least two points, so t is neither injective nor a one-point singular, excluding the cases 1, 2, 3_Y and $3_{\neg Y}$. □

Table 4.1: The cases from Theorem 4.18 for a fixed set Y with $Z \subseteq Y \subseteq t(X)$; an entry A means equality with A. We have $D_Z \subseteq D_Y \subseteq D_{t(X)}$ and, if t is injective, these sets are all equal, see Theorem 4.19, and contained in A if $A \neq \emptyset$. With Equations (3.10) and (3.11) $A, D_Z, D_Y, D_{t(X)}$ each can contain at most one point. For $|\operatorname{dupl}(t)| \geq 2$ we have $D_{\operatorname{dupl}(t)} = A$, as h then identifies the points of $\operatorname{dupl}(t)$ since $|A| = 1$.

Case	$\lvert\operatorname{dupl}(t)\rvert$	A	D_Z	D_Y	$D_{t(X)}$
1	0	$\emptyset$	singleton	singleton	singleton
2	1	singleton	$\emptyset$	$\emptyset$	$\emptyset$
3_Y	1	singleton	$\emptyset$	A	A
$3_{\neg Y}$	1	singleton	$\emptyset$	$\emptyset$	A
4_Y	≥ 2	singleton	$\emptyset$	A	A
$4_{\neg Y}$	≥ 2	singleton	$\emptyset$	$\emptyset$	A
5	≥ 1	singleton	A	A	A

Table 4.2 lists further conditions, each equivalent to the disjunction of a subset of the cases from Theorem 4.18. Hence the disjunction of any set of rows whose marks cover every column yields a characterization for ht being a one-point singular.

Remark 4.19 We have

$$t \text{ injective and } D_Z \text{ a singleton} \Leftrightarrow t \text{ injective and } D_Y \text{ a singleton},$$
$$D_Z \subseteq A \text{ and } A \text{ a singleton} \Leftrightarrow D_Y \subseteq A \text{ and } A \text{ a singleton},$$
$$h_{|t(X)} \text{ injective and } A \text{ a singleton} \Leftrightarrow h_{|t(X)} \text{ injective and } t \text{ a one-point singular}.$$

For the first equivalence, note that t is injective if and only if $\operatorname{dupl}(t) = \emptyset$, which by $\operatorname{dupl}(t) \subseteq t(X)$ is equivalent to $Z = t(X)$; and then $Y = t(X)$ is the only admissible choice. The second follows since, by Equation (3.4), we have $\operatorname{dupl}(h_{|Y}) \setminus \operatorname{dupl}(h_{|Z}) \subseteq h(\operatorname{dupl}(t))$. The third is the last statement of the paragraph preceding Theorem 4.18.

In Table 4.2 the marks of the rows *t injective, D_Z singleton* and *$D_Z \subseteq A$, A singleton* cover every column. Hence ht is a one-point singular if and only if t is injective and D_Z is a singleton set, or $D_Z \subseteq A$ and A is a singleton set; and by the above equivalences Z can be replaced by any set Y between Z and $t(X)$, independently in each disjunct, since each holds for some such Y if and only if it holds for every such Y.

Next, consider the two conditions

1. $A \subseteq D_{t(X)}$ and $D_{t(X)}$ a singleton, or $h_{|Y}$ injective and t a one-point singular;
2. $A \subseteq D_Z$ and D_Z a singleton, or $h_{|Y}$ injective and t a one-point singular.

The first is equivalent to ht being a one-point singular, but the second is not, as by Table 4.2 the cases 3_Y, 4_Y and $4_{\neg Y}$ are not covered by it. For a concrete example, let $X = \{1, \ldots, 5\}$ with $t(1) = t(2) = 1$, $t(3) = t(4) = 2$, $t(5) = 5$ and $h(1) =$

Table 4.2: The columns show the cases from Theorem 4.18, where "o.p.s." abbreviates "one-point singular". Each row gives a condition equivalent to the disjunction of the cases marked in it.

			Case 3		Case 4		
	1	2	3_Y	$3_{\neg Y}$	4_Y	$4_{\neg Y}$	5
t injective, D_Z singleton	x						
$h_{\mid Z}$ injective, A singleton		x	x	x	x	x	
$h_{\mid Y}$ injective, A singleton		x		x		x	
$h_{\mid t(X)}$ injective, A singleton		x					
$h_{\mid Z}$ injective, t o.p.s.		x	x	x			
$h_{\mid Y}$ injective, t o.p.s.		x		x			
$h_{\mid t(X)}$ injective, t o.p.s.		x					
$D_Y \subseteq A$, A singleton		x	x	x	x	x	x
$D_Z \subseteq A$, A singleton		x	x	x	x	x	x
$D_{t(X)} \subseteq A$, A singleton		x	x	x	x	x	x
$A \subseteq D_Z$, D_Z singleton	x						x
$A \subseteq D_Y$, D_Y singleton	x		x		x		x
$A \subseteq D_{t(X)}$, $D_{t(X)}$ singleton	x		x	x	x	x	x
$A = D_Z$, this set singleton							x
$A = D_Y$, this set singleton			x		x		x
$A = D_{t(X)}$, this set singleton			x	x	x	x	x

$h(2) = h(3) = 3$, $h(4) = 4$, $h(5) = 5$. Then $\mathrm{dupl}(t) = \{1, 2\}$, $A = \{3\}$, $D_Z = \emptyset$, $D_{t(X)} = \{3\}$, and ht maps $1, 2, 3, 4$ to 3 and fixes 5. So $\mathrm{dupl}(ht) = \{3\}$ and ht is a one-point singular. Here $Z = \{5\}$ and $\mathrm{dupl}(t)$ has two points, so we are in case 4_Y for $Y = t(X)$ and in case $4_{\neg Y}$ for every other admissible Y. Since t is not a one-point singular and $A = \{3\} \not\subseteq \emptyset = D_Z$, both disjuncts of the second condition fail for every admissible Y.

Remark 4.20 If X is finite, then ht is a one-point singular if t is injective and h a one-point singular, since then $t(X) = X$ and so, with Equation (3.10), we have $\mathrm{dupl}(ht) = \mathrm{dupl}(h_{\mid t(X)}) = \mathrm{dupl}(h)$. However, this might fail for infinite X. For $X = \mathbb{N}$ with $t(n) = n + 1$, and $h(1) = h(2) = 1$ and $h(n) = n$ for $n \geq 3$, the transformation h is one-point singular with $\mathrm{dupl}(h) = \{1\}$ and $h(\mathrm{dupl}(t)) = \emptyset$ as t is injective; but ht maps 1 to 1 and n to $n + 1$ for $n \geq 2$ and hence is injective.

Remark 4.21 By Theorem 4.20, if h is a one-point singular, for finite X, we have $\mathrm{dupl}(ht) = \mathrm{dupl}(h)$ precisely if $h(\mathrm{dupl}(t)) \subseteq \mathrm{dupl}(h)$. The example from Theorem 4.20 also shows that this equivalence might fail for infinite X.

Remark 4.22 Let $\mathrm{dupl}(h) = \{c\}$ and $S \subseteq X$. Note that then $h_{\mid S}$ is injective if and only if $|S \cap h^{-1}(\mathrm{dupl}(h))| \leq 1$, as every pair of points identified by h is mapped to c.

Finally we spell out the case in which both of the transformations are assumed to be one-point singulars.

Corollary 4.23 *Let $t, h\colon X \to X$ be one-point singulars. Then the following are equivalent.*

1. *ht is a one-point singular.*
2. *$h(\mathrm{dupl}(t)) = \mathrm{dupl}(h)$ or $h_{|t(X)}$ is injective.*
3. *Exactly one of the following holds:*
   a. *$h_{|t(X)}$ is injective.*
   b. *$h(Z) \cap h(\mathrm{dupl}(t)) \neq \emptyset$ and $h_{|Z}$ is injective.*
   c. *$\mathrm{dupl}(h_{|Z}) = h(\mathrm{dupl}(t))$.*

Proof. Here $h(\mathrm{dupl}(t))$ is a singleton set, $\mathrm{dupl}(h_{|\,\mathrm{dupl}(t)}) = \emptyset$, and t is not injective. By Equation (3.10) and $\mathrm{dupl}(h_{|t(X)}) \subseteq \mathrm{dupl}(h)$, the set $\mathrm{dupl}(ht) = h(\mathrm{dupl}(t)) \cup \mathrm{dupl}(h_{|t(X)})$ is a singleton set precisely if $\mathrm{dupl}(h_{|t(X)}) = \emptyset$ or $h(\mathrm{dupl}(t)) = \mathrm{dupl}(h)$, which gives 2. For 3 apply Theorem 4.18 for $Y = t(X)$: only case 2,3 and 5 apply and reduce to the stated conditions. □

In Theorem 4.23, Item 3 the former two conditions can be combined into the single condition that $h_{|Z}$ is injective, and the latter two conditions can be combined into $\mathrm{dupl}(h_{|t(X)}) = h(\mathrm{dupl}(t))$.

Figure 3.2 shows some cases where the composition of two one-point singulars results in a one-point singular.

4.5 Idempotent Transformations

We call $t\colon X \to X$ an *idempotent (transformation)* if it is idempotent w.r.t. functional composition, i.e., we have $t^2 = t$. For example, the idempotent transformations of defect one are precisely the simple idempotents introduced in Section 4.3.

First, in Theorem 4.24, we list several equivalent conditions to idempotency, and Theorem 4.25 collects some consequences. Then Theorem 4.26 characterizes idempotency of the composition, and Theorems 4.27, 4.29, and 4.30 give characterizations under additional assumptions on the idempotency of the given functions. Theorem 4.32 gives a formula for the inverse image of the duplicate set of the composition involving at least one idempotent, stating that it equals a union of inverse images of the individual transformations.

Proposition 4.24 *Let $t\colon X \to X$. Then the following are equivalent.*

1. *t is idempotent.*
2. *$t(x) \in t^{-1}(t(x))$ for all $x \in X$.*
3. *$t^{-1}(y) \neq \emptyset$ implies $y \in t^{-1}(y)$ for $y \in X$.*
4. *$t(X) \subseteq \operatorname{fix}(t)$.*
5. *$\operatorname{move}(t) \subseteq \operatorname{excl}(t)$.*
6. *$t(\operatorname{move}(t)) \subseteq \operatorname{fix}(t)$.*

Proof. The equivalences $1 \Leftrightarrow 2 \Leftrightarrow 3 \Leftrightarrow 4$ follow readily.

$4 \Rightarrow 5$: This follows as $\operatorname{move}(t) = X \setminus \operatorname{fix}(t) \subseteq X \setminus t(X) = \operatorname{excl}(t)$.

$5 \Rightarrow 6$: Here $t(\operatorname{move}(t)) \subseteq t(X) = X \setminus \operatorname{excl}(t) \subseteq X \setminus \operatorname{move}(t) = \operatorname{fix}(t)$.

$6 \Rightarrow 4$: As $t(\operatorname{fix}(t)) = \operatorname{fix}(t)$ and $t(\operatorname{move}(t)) \subseteq \operatorname{fix}(t)$, we find $t(X) = t(\operatorname{fix}(t)) \cup t(\operatorname{move}(t)) \subseteq \operatorname{fix}(t)$. □

Corollary 4.25 *Suppose $t\colon X \to X$ is idempotent. Then we have the following:*

1. *$t(X) = \operatorname{fix}(t)$,*
2. *$\operatorname{move}(t) = \operatorname{excl}(t)$,*
3. *$\operatorname{dupl}(t) = t(\operatorname{move}(t))$ (equivalently, $t(x) \notin \operatorname{dupl}(t)$ implies $t(x) = x$),*
4. *$\operatorname{dupl}(t) = t(\operatorname{dupl}(t))$, i.e., t fixes every element of $\operatorname{dupl}(t)$,*
5. *$t^{-1}(\operatorname{dupl}(t)) = \operatorname{move}(t) \uplus \operatorname{dupl}(t)$,*
6. *$t(A) \subseteq A$ for $A \subseteq X$ with $\operatorname{dupl}(t) \subseteq A$.*

Proof. 1: This follows from Item 4 of Theorem 4.24 and Equation (3.24).

2: This follows from Item 5 of Theorem 4.24 and Equation (3.25).

3: By Item 6 of Theorem 4.24 and Equation (3.22) we have the following inclusions, which give the claim:

$$t(\operatorname{move}(t)) \subseteq \operatorname{fix}(t) \cap t(\operatorname{move}(t)) \subseteq \operatorname{dupl}(t) \subseteq t(\operatorname{move}(t)).$$

4: This follows as $\operatorname{dupl}(t) \subseteq t(X) \subseteq \operatorname{fix}(t)$ by Item 4 of Theorem 4.24.

5: By Item 3 we have $\operatorname{move}(t) \subseteq t^{-1}(\operatorname{dupl}(t))$, and $\operatorname{dupl}(t) \subseteq t^{-1}(\operatorname{dupl}(t))$ by Item 4. Conversely, let $x \in t^{-1}(\operatorname{dupl}(t))$ with $x \notin \operatorname{move}(t)$. Then $x = t(x) \in \operatorname{dupl}(t)$. The union is disjoint as $\operatorname{dupl}(t) \subseteq \operatorname{fix}(t)$.

6: For $x \in A$ we have $t(x) = x \in A$ or, by Item 3, $t(x) \in \operatorname{dupl}(t) \subseteq A$. □

Next, we investigate when the composition of two transformations gives an idempotent transformation.

Proposition 4.26 *Let $t, h\colon X \to X$. Then the following are equivalent.*

1. *ht is idempotent.*
2. $\operatorname{move}(t) \cap h(t(X)) \subseteq \operatorname{fix}(ht)$ *and* $\operatorname{fix}(t) \cap h(t(X)) \subseteq \operatorname{fix}(h)$.
3. $t(y) \in h^{-1}(y)$ *for all* $y \in h(t(X))$.
4. $h(\operatorname{move}(h) \cap t(X)) \subseteq \operatorname{fix}(ht)$ *and* $\operatorname{fix}(h) \cap t(X) \subseteq \operatorname{fix}(ht)$.

Proof. 1 $\Rightarrow$ 2: By Theorem 4.24 we have $h(t(X)) \subseteq \operatorname{fix}(ht)$, which gives the first inclusion; the second follows with Equation (3.26).

2 $\Rightarrow$ 3: Let $y \in h(t(X))$. If $y \in \operatorname{move}(t)$, then $h(t(y)) = y$ by the first assumption. Otherwise $y \in \operatorname{fix}(t)$, so $h(y) = y$ by the second assumption and hence $h(t(y)) = h(y) = y$.

3 $\Rightarrow$ 4: Let $x \in X$. Applying 3 to $y = h(t(x))$ gives $h(t(x)) \in \operatorname{fix}(ht)$, and applying it to $y = t(x)$ in case $t(x) \in h(t(X))$ gives $t(x) \in \operatorname{fix}(ht)$. The first covers $h(\operatorname{move}(h) \cap t(X))$, the second $\operatorname{fix}(h) \cap t(X)$, as $t(x) \in \operatorname{fix}(h)$ implies $t(x) = h(t(x)) \in h(t(X))$.

4 $\Rightarrow$ 1: With $t(X) = (\operatorname{move}(h) \cap t(X)) \uplus (\operatorname{fix}(h) \cap t(X))$ we get

$$h(t(X)) = h(\operatorname{move}(h) \cap t(X)) \cup (\operatorname{fix}(h) \cap t(X)) \subseteq \operatorname{fix}(ht),$$

so ht is idempotent by Theorem 4.24. □

Corollary 4.27 *Let $t, h\colon X \to X$.*

1. *For idempotent h, ht is idempotent if and only if* $\operatorname{move}(t) \cap h(t(X)) \subseteq \operatorname{fix}(ht)$.
2. *For idempotent t, ht is idempotent if and only if* $h(\operatorname{move}(h) \cap t(X)) \subseteq \operatorname{fix}(ht)$.

Proof. 1: For idempotent h we have $h(t(X)) \subseteq h(X) \subseteq \operatorname{fix}(h)$ by Theorem 4.24, so the second condition of Item 2 of Theorem 4.26 always holds.

2: For idempotent t we have $t(X) \subseteq \operatorname{fix}(t)$ by Theorem 4.24, so $\operatorname{fix}(h) \cap t(X) \subseteq \operatorname{fix}(h) \cap \operatorname{fix}(t) \subseteq \operatorname{fix}(ht)$ by Equation (3.26), and the second condition of Item 4 of Theorem 4.26 always holds. □

Remark 4.28 By Item 1 of Theorem 4.27, for idempotent h the composition ht is idempotent if $h(t(X)) \subseteq \operatorname{fix}(t)$, as then $\operatorname{move}(t) \cap h(t(X)) = \emptyset$. This also follows directly from $h(t(X)) \subseteq \operatorname{fix}(h) \cap \operatorname{fix}(t) \subseteq \operatorname{fix}(ht)$ with Item 4 of Theorem 4.24. The condition is not necessary: if $h(x) = c$ for all $x \in X$ and $t(c) \neq c$, then h and ht are idempotent and $h(t(X)) \cap \operatorname{fix}(t) = \emptyset$.

Corollary 4.29 *Let $t, h\colon X \to X$ be idempotent. Then ht is idempotent if and only if* $\operatorname{move}(t) \cap h(\operatorname{move}(h) \cap t(X)) \subseteq \operatorname{fix}(ht)$.[4]

[4] Equivalently, for $x \in X$, $h(t(x)) \neq t(x)$ and $t(h(t(x))) \neq h(t(x))$ imply $h(t(x)) = h(t(h(t(x))))$.

Proof. If ht is idempotent, then $h(t(X)) \subseteq \operatorname{fix}(ht)$ by Theorem 4.24, and the set in question is contained in $h(t(X))$.

Conversely, as t is idempotent we have $\operatorname{fix}(h) \cap t(X) = \operatorname{fix}(h) \cap \operatorname{fix}(t) \subseteq \operatorname{fix}(ht)$ by Equation (3.26), and as h is idempotent we have $\operatorname{fix}(t) \cap h(t(X)) \subseteq \operatorname{fix}(t) \cap \operatorname{fix}(h) \subseteq \operatorname{fix}(ht)$. With $t(X) = (\operatorname{move}(h) \cap t(X)) \uplus (\operatorname{fix}(h) \cap t(X))$ and $X = \operatorname{move}(t) \uplus \operatorname{fix}(t)$ we obtain

$$h(t(X)) = (\operatorname{move}(t) \cap h(\operatorname{move}(h) \cap t(X))) \cup (\operatorname{move}(t) \cap (\operatorname{fix}(h) \cap t(X))) \cup (\operatorname{fix}(t) \cap h(t(X))),$$

where we used $h(\operatorname{fix}(h) \cap t(X)) = \operatorname{fix}(h) \cap t(X)$. All three sets are contained in $\operatorname{fix}(ht)$, the first by assumption, so ht is idempotent by Theorem 4.24. □

Corollary 4.30 *Let $t, h\colon X \to X$ be idempotent. Then ht is idempotent if one of the following is true:*

1. $\operatorname{dupl}(h) \subseteq \operatorname{fix}(ht)$*, i.e.,* $h(t(y)) = y$ *for all* $y \in \operatorname{dupl}(h)$*,*
2. $\operatorname{excl}(t) \subseteq \operatorname{fix}(ht)$*,*
3. $ht = th$*.*

Proof. 1: As h is idempotent, we have $h(\operatorname{move}(h) \cap t(X)) \subseteq h(\operatorname{move}(h)) = \operatorname{dupl}(h)$ by Item 3 of Theorem 4.25, which is contained in $\operatorname{fix}(ht)$ by assumption. Thus the claim follows from Item 2 of Theorem 4.27.

2: With Theorem 4.24 we find $\operatorname{move}(t) = \operatorname{excl}(t) \subseteq \operatorname{fix}(ht)$, so Theorem 4.29 gives the claim.

3: We have $h(t(h(t(x)))) = h(h(t(t(x)))) = h(t(x))$ for $x \in X$. Note that here only the idempotency of h and t in the last step is used, but not Theorem 4.29. □

Proposition 4.31 *Let $t, h\colon X \to X$ be idempotent. Then the following are equivalent.*

1. $ht = t$.
2. $t(X) \subseteq \operatorname{fix}(h)$.
3. $t(X) \cap h^{-1}(\operatorname{dupl}(h)) \subseteq \operatorname{dupl}(h)$.

Proof. $1 \Rightarrow 2$: By assumption we have $h(t(x)) = t(x)$ for all $x \in X$.

$2 \Rightarrow 3$: For $t(x) \in h^{-1}(\operatorname{dupl}(h))$ we have $t(x) = h(t(x)) \in \operatorname{dupl}(h)$.

$3 \Rightarrow 1$: Suppose $h(t(x)) \neq t(x)$ for some $x \in X$. Then $h(t(x)) \in \operatorname{dupl}(h)$ by Item 3 of Theorem 4.25, so $t(x) \in t(X) \cap h^{-1}(\operatorname{dupl}(h))$, but $t(x) \notin \operatorname{dupl}(h)$ as h fixes every element of $\operatorname{dupl}(h)$ by Item 4 of Theorem 4.25. Thus Item 3 fails. □

Next, we refine Theorem 3.6 for the case that one transformation is idempotent. For idempotent $t\colon X \to X$ and $A \subseteq X$ we have

$$t^{-1}(A) = (A \cap t(X)) \cup t^{-1}(A \cap \operatorname{dupl}(t)), \tag{4.24}$$

as $x \in t^{-1}(A)$ with $x \notin \operatorname{dupl}(t)$ satisfies $x = t(x) \in A \cap t(X)$ by Item 3 of Theorem 4.25, while $A \cap t(X) \subseteq t^{-1}(A)$ holds as t fixes $t(X)$.

Proposition 4.32 *Let $t\colon X \to X$ be idempotent and $h\colon X \to Y$. Then*

$$\begin{aligned}(ht)^{-1}(\mathrm{dupl}(ht)) &= t^{-1}(\mathrm{dupl}(t)) \cup (h^{-1}(\mathrm{dupl}(h_{|t(X)})) \cap t(X)) \\ &= t^{-1}(\mathrm{dupl}(t)) \cup h^{-1}(\mathrm{dupl}(ht)).\end{aligned}$$

Proof. Set $A = h^{-1}(\mathrm{dupl}(h_{|t(X)}))$ and $B = A \cap t(X) = h^{-1}_{|t(X)}(\mathrm{dupl}(h_{|t(X)}))$. For the first identity we have

$$\begin{aligned}(ht)^{-1}(\mathrm{dupl}(ht)) &= t^{-1}(\mathrm{dupl}(t)) \cup t^{-1}(A) && \text{(Theorem 3.6)} \\ &= t^{-1}(\mathrm{dupl}(t)) \cup B \cup t^{-1}(A \cap \mathrm{dupl}(t)) && \text{(Eqn. (4.24))} \\ &= t^{-1}(\mathrm{dupl}(t)) \cup B,\end{aligned}$$

the last step as $t^{-1}(A \cap \mathrm{dupl}(t)) \subseteq t^{-1}(\mathrm{dupl}(t))$.

For the second identity, note that $\mathrm{excl}(t) \cup \mathrm{dupl}(t) \subseteq t^{-1}(\mathrm{dupl}(t))$ by Items 2 and 5 of Theorem 4.25. Hence

$$\begin{aligned}t^{-1}(\mathrm{dupl}(t)) \cup h^{-1}(\mathrm{dupl}(ht)) &= t^{-1}(\mathrm{dupl}(t)) \cup h^{-1}_{|t(X)}(\mathrm{dupl}(ht)) \\ &\qquad (\mathrm{excl}(t) \subseteq t^{-1}(\mathrm{dupl}(t))) \\ &= t^{-1}(\mathrm{dupl}(t)) \cup h^{-1}_{|t(X)}(h_{|t(X)}(\mathrm{dupl}(t))) \cup B \\ &\qquad \text{(Eqn. (3.10))} \\ &= t^{-1}(\mathrm{dupl}(t)) \cup B,\end{aligned}$$

where the last step holds since, by Equation (3.8), $h^{-1}_{|t(X)}(h_{|t(X)}(\mathrm{dupl}(t))) \subseteq \mathrm{dupl}(t) \cup B$ and $\mathrm{dupl}(t) \subseteq t^{-1}(\mathrm{dupl}(t))$. Together with the first identity this shows the claim. □

Remark 4.33 In Theorem 4.32, the set $h^{-1}(\mathrm{dupl}(h_{|t(X)})) \cap t(X)$ may be replaced with either $h^{-1}(\mathrm{dupl}(h_{|t(X)}))$ or $h^{-1}(\mathrm{dupl}(ht)) \cap t(X)$, since both contain $h^{-1}(\mathrm{dupl}(h_{|t(X)})) \cap t(X)$ and are contained in $h^{-1}(\mathrm{dupl}(ht))$. However, $h^{-1}(\mathrm{dupl}(h_{|t(X)}))$ cannot be replaced with $h^{-1}(\mathrm{dupl}(h))$, see Theorem 3.7.

4.6 Idempotent One-Point Singulars

Theorem 4.34 lists several characterizations for idempotents whose duplicate set contains at most one point, and Theorem 4.35 provides an additional characterization for finite domains. Then, Theorem 4.36, which we need in Part III, gives conditions when the composition of two idempotent one-point singulars yields an idempotent one-point singular. As a consequence, we obtain that if the composition of two simple idempotents yields a simple idempotent, then it equals one of the two original simple idempotents.

A simple idempotent is an idempotent one-point singular. Rystsov [535] introduced simple idempotents and showed [535, Lem. 1] that a simple idempotent $t\colon [n] \to [n]$ on a finite set fulfills $t^{-1}(A) \subseteq A$ or $A \subseteq t^{-1}(A)$ for every $A \subseteq [n]$. In Theorem 4.34, Item 10 we extend this result by characterizing the transformations that fulfill

this equation as precisely the idempotents whose duplicate set contains at most one point.

Theorem 4.34 *Let $t\colon X \to X$ with X non-empty. Then the following are equivalent.*

1. *t is an idempotent one-point singular or equals the identity transformation.*
2. *$t = [a_{m-1} \mapsto a_m] \cdots [a_1 \mapsto a_m]$ for $\operatorname{move}(t) = \{a_1, \ldots, a_m\}$ and $\operatorname{dupl}(t) = \{a_m\}$ or $t = \mathrm{id}_X$.*
3. *$\operatorname{dupl}(t)$ contains at most one point and t is idempotent.*
4. *$\operatorname{dupl}(t)$ contains at most one point and $|t^{-1}(z)| = 1$ implies $t^{-1}(z) = \{z\}$ for all $z \in X$.*
5. *There exists $y \in X$ such that for all $z \in X$ we have $t^{-1}(z) = \{z\}$ or $z \in t^{-1}(y)$.*
6. *There exists $y \in X$ such that $x \in t^{-1}(z)$ implies $t(x) = x$ for $z, x \in X$ with $z \neq y$.*
7. *For all $x, z \in X$ with $t(x) \neq x$ and $t(z) \neq z$ we have $t(x) = t(z)$, i.e., $\operatorname{move}(t) \subseteq t^{-1}(y)$ for some $y \in X$.*
8. *There exists $y \in X$ such that for all $z \in X$ we have $t^{-1}(z) \subseteq \{z\}$ or $z \in t^{-1}(y)$.*
9. *There exists $y \in X$ such that, for all $z \in t(X)$, we have*

$$t^{-1}(z) = \begin{cases} \operatorname{excl}(t) \cup \{y\} & \text{if } z = y, \\ \{z\} & \text{otherwise.} \end{cases}$$

10. *For all $A \subseteq X$ we have $A \subseteq t^{-1}(A)$ or $t^{-1}(A) \subseteq A$.*

Proof. $1 \Rightarrow 2$ For $t \neq \mathrm{id}_X$, let $\operatorname{dupl}(t) = \{a_m\}$ and $\operatorname{move}(t) = \{a_1, \ldots, a_{m-1}\}$. Then we have $t = [a_{m-1} \mapsto a_m] \cdots [a_1 \mapsto a_m]$.

$2 \Rightarrow 3$: This is clear.

$3 \Rightarrow 4$: By assumption, $\operatorname{dupl}(t)$ contains at most one point. By idempotency, the rest of the claim follows from Theorem 4.24, Item 3.

$4 \Rightarrow 5$: For $\operatorname{dupl}(t) \neq \emptyset$, let y be the unique point in $\operatorname{dupl}(t)$, otherwise, let $y \in X$ be any point. Next, let $z \in X$ with $t(z) \neq y$. Then as $t(z) \notin \operatorname{dupl}(t)$ we have $t^{-1}(t(z)) = \{z\}$. By assumption $|t^{-1}(t(z))| = 1$ gives $t^{-1}(t(z)) = \{t(z)\}$. Thus $t(z) = z$, which gives $t^{-1}(z) = \{z\}$.

$5 \Rightarrow 6$: Let $x, z \in X$ with $z \neq y$ and suppose $x \in t^{-1}(z)$. Since $t^{-1}(z) = \{z\}$, we get $x = z$ and so $t(x) = x$.

$6 \Rightarrow 7$: Suppose $t(x) \neq x$ and $t(z) \neq z$ for $x, z \in X$. Since $x \in t^{-1}(t(x))$ and $z \in t^{-1}(t(z))$, we find $t(x) = t(z) = y$.

$7 \Rightarrow 8$: We can assume $t(x) \neq x$ for some $x \in X$, as otherwise t equals the identity transformation and the claim obviously holds. Then set $y = t(x)$. Next, consider $z \in X$ with $z \notin t^{-1}(y)$. By assumption, $t(z) = z$. Now, assume $t(z) = t(z')$ for some $z' \in X$ with $z \neq z'$. Then $t(z') \neq z'$ yields $t(z') = y$. So, this is not possible and we have $t^{-1}(z) = \{z\}$.

$6 \Rightarrow 8$: Let $z \in X$ with $z \neq y$ and assume $z \notin t^{-1}(y)$. Now, for $x \in X$ with $x \in t^{-1}(z)$, by assumption, we get $z = t(x) = x$ and so $t^{-1}(z) \subseteq \{z\}$.

$8 \Rightarrow 9$: Let $z \in t(X)$ and suppose $z \neq y$. We have $z = t(x)$ for some $x \in X$. Thus $x \in t^{-1}(z)$. Since $z \neq y$, by assumption $t^{-1}(z) \subseteq \{z\}$ and so $z = x$. This gives $t^{-1}(z) = \{z\}$. Since $t^{-1}(z) = \{z\}$ for $z \in t(X)$ with $z \neq y$, and the union of the inverse images equals X, we have that the remaining points $\operatorname{excl}(t) \cup \{y\}$ must be in the inverse

image of $t^{-1}(y)$, and since this union is pairwise disjoint, no other point is in $t^{-1}(y)$ and so $t^{-1}(y) = \operatorname{excl}(t) \cup \{y\}$.

$9 \Rightarrow 10$: Let $A \subseteq X$. Then

$$t^{-1}(A) = t^{-1}(A \cap t(X)) = \begin{cases} A \cap t(X) & \text{if } y \notin A, \\ \operatorname{excl}(t) \cup A & \text{if } y \in A. \end{cases}$$

So, $t^{-1}(A) \subseteq A$ for $y \notin A$ and $A \subseteq t^{-1}(A)$ for $y \in A$.

$10 \Rightarrow 1$: For $z \in X$, consider $A = \{t(z)\}$. Then $A \subseteq t^{-1}(A)$ or $t^{-1}(A) \subseteq A$ and as $t^{-1}(A) \neq \emptyset$, we get $t(z) \in t^{-1}(t(z))$ for both cases. Hence, by Theorem 4.24, Item 2, t is idempotent. Then, for $\operatorname{dupl}(t) = \emptyset$ we have $t = \mathrm{id}_X$. Next, for $\operatorname{dupl}(t) \neq \emptyset$, by Theorem 4.25, Item 5 there exists $y \in \operatorname{dupl}(t)$ such that $x \in t^{-1}(y) \cap \operatorname{excl}(t)$ for some $x \in X$. Let $y' \in \operatorname{dupl}(t)$ and set $A = \{x, y'\}$. Then $t^{-1}(A) = t^{-1}(y')$ and, as $t^{-1}(y')$ contains at least two points, for $A \subseteq t^{-1}(y')$ or $t^{-1}(y') \subseteq A$ we get $x \in t^{-1}(y')$ in both cases. Now $x \in t^{-1}(y) \cap t^{-1}(y')$ gives $y = y'$ and so t is a one-point singular. □

Regarding Theorem 4.34, Item 2, inductively, from Equation (4.9) (but it is also easy to verify directly) we get the following identity:

$$[a_{m-1} \mapsto a_m] \cdots [a_2 \mapsto a_3][a_1 \mapsto a_2] = [a_{m-1} \mapsto a_m] \cdots [a_2 \mapsto a_m][a_1 \mapsto a_m]. \tag{4.25}$$

Next, we state a new characterization of idempotents on finite sets whose duplicate set contains at most one point.

Proposition 4.35 *Let $t\colon [n] \to [n]$. Then $t = \mathrm{id}_{[n]}$ or t is an idempotent one-point singular if and only if for all $A \subseteq [n]$ we have that $|t^{-1}(A)| = |A|$ implies $t^{-1}(A) = A$.*

Proof. We can assume $t \neq \mathrm{id}_{[n]}$. First, suppose t is an idempotent one-point singular. Let $A \subseteq [n]$ be such that $|t^{-1}(A)| = |A|$. By Theorem 4.34, Item 10 we have $t^{-1}(A) \subseteq A$ or $A \subseteq t^{-1}(A)$. Finiteness implies $t^{-1}(A) = A$ in both cases.

Conversely, consider a set $A \subseteq [n]$. Since $\operatorname{def}(t) + |A \cap t([n])| \geq |t^{-1}(A \cap t([n]))|$ by Theorem 3.14, Item 2, we can choose $B \subseteq \operatorname{excl}(t)$ such that $|B| + |A \cap t([n])| = |t^{-1}(A \cap t([n]))|$ where we let $B \subseteq A \cap \operatorname{excl}(t)$ if $|B| \leq |A \cap \operatorname{excl}(t)|$ and $A \cap \operatorname{excl}(t) \subseteq B$ otherwise. Since $B \subseteq \operatorname{excl}(t)$ we have $|B| + |A \cap t([n])| = |B \cup (A \cap t([n]))|$ and $t^{-1}(A \cap t([n])) = t^{-1}((A \cap t([n])) \cup B)$. Thus $|t^{-1}((A \cap t([n])) \cup B)| = |(A \cap t([n])) \cup B)|$ and so $t^{-1}((A \cap t([n])) \cup B) = (A \cap t([n])) \cup B$. Since $t^{-1}(A) = t^{-1}(A \cap t([n]))$ and $A = (A \cap t([n])) \cup (A \cap \operatorname{excl}(t))$, we find $t^{-1}(A) \subseteq A$ for $B \subseteq A \cap \operatorname{excl}(t)$ and $A \subseteq t^{-1}(A)$ for $A \cap \operatorname{excl}(t) \subseteq B$. Therefore, by Theorem 4.34, Item 10, it follows that t is an idempotent one-point singular. □

Proposition 4.36 *Let $t, h\colon X \to X$ be idempotent one-point singulars. Then the following are equivalent.*

1. *ht is an idempotent one-point singular.*
2. $\operatorname{move}(t) \cap h(\operatorname{move}(h) \cap t(X)) \subseteq \operatorname{fix}(ht)$ *and* $\operatorname{dupl}(ht) = h(\operatorname{dupl}(t))$.[5]
3. $h(\operatorname{move}(h) \cap t(X)) \subseteq h(\operatorname{dupl}(t))$.

[5] Recall that $\operatorname{dupl}(h_{|t(X)}) \subseteq h(\operatorname{dupl}(t))$ is equivalent to $\operatorname{dupl}(ht) = h(\operatorname{dupl}(t))$ by Equation (3.10).

4. $t(X) \subseteq \mathrm{fix}(h)$ *or* $\mathrm{dupl}(h) = h(\mathrm{dupl}(t))$.[6]
5. *We have* $ht = t$ *or with* $\mathrm{move}(t) \cup \mathrm{move}(h) = \{a_1, \ldots, a_{m-1}\}$ *and* $\mathrm{dupl}(h) = \{a_m\}$ *for distinct* $a_1, \ldots, a_m \in X$ *we have*

$$ht = [a_1 \mapsto a_m] \cdots [a_{m-1} \mapsto a_m].$$

Proof. 1 $\Rightarrow$ 2: The first inclusion follows from Theorem 4.29. Since ht and t are one-point singulars, we must have $\mathrm{dupl}(ht) = h(\mathrm{dupl}(t))$.

2 $\Rightarrow$ 3: Let $x \in X$ be such that $h(t(x)) \neq t(x)$. If $t(h(t(x))) \neq h(t(x))$, then, by assumption, $h(t(h(t(x)))) = h(t(x))$ and thus $h(t(x)) \in \mathrm{dupl}(h_{|t(X)})$. If $t(h(t(x))) = h(t(x))$, then $h(t(x)) \in t(X)$. By idempotency of h, we find $h(h(t(x))) = h(t(x))$. Thus $h(t(x)) \in \mathrm{dupl}(h_{|t(X)})$. Since $\mathrm{dupl}(h_{|t(X)}) \subseteq \mathrm{dupl}(ht) = h(\mathrm{dupl}(t))$ we obtain the claim.

3 $\Rightarrow$ 4: Let $t(x) \in \mathrm{move}(h)$. Since h is an idempotent, we have $h(\mathrm{move}(h)) = \mathrm{dupl}(t)$. Since it is a one-point singular, we find $\mathrm{dupl}(h) = \{h(t(x))\}$. By assumption, we have $h(t(x)) \in h(\mathrm{dupl}(t))$. Since t is a one-point singular we get $h(\mathrm{dupl}(t)) = \{h(t(x))\}$. Thus $\mathrm{dupl}(h) = h(\mathrm{dupl}(t))$.

4 $\Rightarrow$ 5: Suppose $ht \neq t$, i.e., $t(X) \not\subseteq \mathrm{fix}(h)$. Then, by Item 4, we have $\mathrm{dupl}(t) \subseteq h^{-1}(a_m)$. Let $x \in \mathrm{move}(t)$. Then $t(x) \in \mathrm{dupl}(t)$ since t is idempotent and so $h(t(x)) = a_m$. Next, let $x \in \mathrm{fix}(t)$ and $x \in \mathrm{move}(h)$. Then $h(t(x)) = h(x) = a_m$ since h is idempotent. Next, for $x \in \mathrm{fix}(t) \cap \mathrm{fix}(h)$ we have $h(t(x)) = x$. Thus, precisely the points from $\mathrm{move}(t) \cup \mathrm{move}(h)$ are mapped to a_m and all other points are fixed. This is precisely the transformation described by $[a_1 \mapsto a_m] \cdots [a_{m-1} \mapsto a_m]$.[7]

5 $\Rightarrow$ 1: Clearly, in each case, ht is an idempotent one-point singular. □

Corollary 4.37 *Let* $t, h \colon X \to X$ *be idempotent one-point singulars such that* ht *is an idempotent one-point singular. Then*

$$(ht)^{-1}(\mathrm{dupl}(ht)) = \begin{cases} t^{-1}(\mathrm{dupl}(t)) & \text{if } t(X) \subseteq \mathrm{fix}(h), \\ t^{-1}(\mathrm{dupl}(t)) \cup h^{-1}(\mathrm{dupl}(h)) & \text{otherwise,} \end{cases}$$

where we also have $t^{-1}(\mathrm{dupl}(t)) \cup h^{-1}(\mathrm{dupl}(h)) = t^{-1}(\mathrm{dupl}(t)) \cup (t(X) \cap h^{-1}(\mathrm{dupl}(h)))$.

Proof. First, suppose $t(X) \subseteq \mathrm{fix}(h)$. Then $ht = t$ and $\mathrm{dupl}(ht) = \mathrm{dupl}(t)$, which gives the equality for this case.

Next, suppose $t(X) \not\subseteq \mathrm{fix}(h)$. By Theorem 4.36, we have $\mathrm{dupl}(h) = \mathrm{dupl}(ht)$. Then the claim follows from Theorem 4.32. □

Remark 4.38 Figure 3.2 shows two idempotent one-point singulars whose composition results in an idempotent one-point singular. The first two pictures on the left show the case $t(X) \subseteq \mathrm{fix}(h)$ (see also Theorem 4.31), the rightmost picture shows the case $\mathrm{dupl}(h) = h(\mathrm{dupl}(t))$.

---

[6] This is equivalent to the statement that $\mathrm{move}(h) \cap t(X) \neq \emptyset$ implies $\mathrm{dupl}(h) = h(\mathrm{dupl}(t))$.

[7] Alternatively, the equation for ht follows by deriving $\mathrm{dupl}(ht) = \mathrm{dupl}(h)$, substituting this into Theorem 4.32, and noting that all points outside of $t^{-1}(\mathrm{dupl}(t)) \cup h^{-1}(\mathrm{dupl}(h)) = \mathrm{move}(t) \cup \mathrm{move}(h) \cup \{a_m\}$ are fixed by ht.

Corollary 4.39 *Let $t, h\colon X \to X$ be two simple idempotents. If ht is a simple idempotent, then $ht \in \{t, h\}$.*

Proof. By Theorem 4.36, Item 5, if ht is a simple idempotent, we must have $ht = t$ or $\operatorname{move}(t) \cup \operatorname{move}(h) = \{a\}$ and $\operatorname{dupl}(h) = \{b\}$, which gives $h = [a \mapsto b]$, and $ht = [a \mapsto b]$. □

Remark 4.40 Several simple idempotents composed can result in new simple idempotents, a result that fails for two simple idempotents by Theorem 4.39. For example, consider $t_1 = [1 \mapsto 2]$, $t_2 = [2 \mapsto 3]$, and $t_3 = [3 \mapsto 1]$. Then $t_1t_2t_3t_1t_2t_3 = [1 \mapsto 3]$ where $t_1t_2t_3t_1t_2 = [1 \mapsto 3 \mapsto 2]$.

Remark 4.41 Let $t, h\colon X \to X$ such that t (resp. h) is a simple idempotent moving a to the distinct point b. Then a simple case yields that ht is a simple idempotent if and only if $\operatorname{move}(h) \subseteq \{a, b\}$ and $h(b) \in \{a, b\}$ (resp. $\operatorname{move}(t) \subseteq \{a, b\}$ and $t(b) \in \{a, b\}$). In this case we have that ht equals $[a \mapsto b]$ or $[b \mapsto a]$ (resp. $[a \mapsto b]$ for $t(a) \in \{a, b\}$ and $[a \mapsto t(a)]$ for $t(a) \notin \{a, b\}$). The next example illustrates some cases.

Example 4.42 Let $t, h\colon X \to X$ for $X = \{1, 2, 3\}$ with $t(1) = t(2) = 2$, $t(3) = 1$, and $h(1) = h(3) = 3$, $h(2) = 1$. Then we have

1. $[1 \mapsto 3]t = [1 \mapsto 2]$.
2. $h[1 \mapsto 2] = [2 \mapsto 1]$.
3. $[1 \mapsto 3][1 \mapsto 2] = [1 \mapsto 2]$.

4.7 Discussion

Most of the present chapter recaps known results. However, the results on one-point singulars, for example, Theorems 4.15, 4.34, and 4.35, and those on compositions of our classes of transformations, are, to the best of our knowledge, new. One-point singulars were introduced in [524], but this type of transformations has been investigated before, for example in Araújo & Cameron [34] on finite sets where they are referred to via their kernel type $(k, 1, \ldots, 1)$ with $k \geq 2$. We are also not aware of any studies that investigated when the composition of two transformations results in a one-point singular, or an idempotent. Idempotent transformations are discussed in [236, Sec. 2.7] where Theorem 4.24, Item 4 is stated and it is shown that $\mathcal{T}_n$ contains

$$\sum_{k=1}^{n} \binom{n}{k} k^{n-k}$$

idempotent transformations. In [75] algorithmic and complexity theoretical questions of deciding whether all transformations of a given transformation semigroup are idempotent were investigated.

A classical result by Howie [323] states that the set of all simple idempotents on a finite set generates the transformation semigroup consisting of all non-permutations. Gomes and Howie [254] proved that $\binom{n}{2}$ idempotents are sufficient and necessary to generate this set.

Chapter 5
Permutation Groups

"It was at this point that Wigner, Hund, Heitler, and Weyl entered the picture with their 'Gruppenpest': the pest of the group theory... The authors of the 'Gruppenpest' wrote papers which were incomprehensible to those like me who had not studied group theory, in which they applied these theoretical results to the study of the many electron problem. The practical consequences appeared to be negligible, but everyone felt that to be in the mainstream one had to learn about it. Yet there were no good texts from which one could learn group theory. It was a frustrating experience, worthy of the name of a pest."

—John Slater (1900–1976)

The set $\mathcal{S}_X$ of all permutations on a set X with function composition and inversion given by $t^{-1}\colon X \to X$ for $t \in \mathcal{S}_X$ is a group, called the *(full) symmetric group on* X. A *permutation group* G (over the *(permutation) domain* X) is a subgroup of $\mathcal{S}_X$. Here, we investigate permutation groups.

Section 5.1 introduces basic notions. In Section 5.2 we collect statements related to transitive permutation groups. The relation between permutation groups and (abstract) groups is discussed in Section 5.3. Section 5.4 investigates the group of units in transformation semigroups. Lastly, Section 5.5 points out some relations between permutation groups and graphs.

5.1 Basic Notions

Let $G \leq_G \mathcal{S}_X$ be a permutation group.

1. If X is finite, we call $|X|$ the *degree (of* G*)*. For $n \geq 0$, we set $\mathcal{S}_n = \mathcal{S}_{[n]}$.
2. A non-empty subset $\Delta \subseteq X$ is a *block*[1] if and only if, for all $g \in G$, the condition $g(\Delta) \cap \Delta \neq \emptyset$ implies $g(\Delta) = \Delta$. A collection of blocks that forms a partition of X

[1] To distinguish them from ordinary sets, blocks for G are denoted by capital Greek letters. Also, alphabets are denoted by capital Greek letters, but the context should always make it clear whether a specific capital Greek letter denotes a block or an alphabet.

is called a *system of blocks*. Blocks are precisely the congruence classes of some G-congruence on X. A block is a *trivial block* if it is a singleton or equals X.

3. A permutation group G over X is *primitive* if the only G-congruences on X are the trivial G-congruences. As the blocks are precisely the G-congruence classes for some G-congruence, G is primitive if and only if it only admits the trivial blocks. If G is transitive and not primitive, then we call it *imprimitive*.
4. The *G-orbit* (or just *orbit*) of $a \in X$ in G is $\mathrm{Orb}_G(a) = \{g(a) \mid g \in G\}$. The orbits $\mathrm{Orb}_G(a)$ for $a \in X$ form a system of blocks, and the union of several orbits is a block.
5. Let $k \geq 1$. Then G is called *k-homogeneous* (resp. *k-transitive*) if for every two k-sets $A, B \subseteq X$ (resp. k-tuples $(a_1, \ldots, a_k)$, $(b_1, \ldots, b_k) \in X^k$) there exists $g \in G$ such that $B = g(A)$ (resp. $(b_1, \ldots, b_k) = (g(a_1), \ldots, g(a_k))$). Every k-transitive group is k-homogeneous. The symmetric group $\mathcal{S}_X$ is k-transitive for each $k \geq 1$. A 1-homogeneous permutation group is also called *transitive*. Equivalently, G is transitive if and only if there exists $a \in X$ such that $X = \mathrm{Orb}_G(a)$. For $A \subseteq X$ we also say that G is *transitive on A*, or *acts transitively on A*, if $\mathrm{Orb}_G(a) = A$ for some (and so each) $a \in A$. We call G *intransitive* if it is not transitive.
6. Let $a \in X$. The *point stabilizer subgroup (of a in G)* is $G_a = \{g \in G \mid g(a) = a\}$. For $g \in G$ we have
$$gG_a = \{h \in G \mid h(a) = g(a)\}. \tag{5.1}$$
7. For any transformation $t\colon X \to X$ and permutation $g\colon X \to X$ the *conjugate of t by g* is $t^g\colon X \to X$ given by gtg^{-1}. For $a, b \in X$, we have[2]
$$t(a) = b \Leftrightarrow t^g(g(a)) = g(b). \tag{5.2}$$
This equation implies, for $a \in X$ and $g \in G$,
$$G_{g(a)} = gG_ag^{-1}. \tag{5.3}$$
By Equation (5.2) every permutation $g\colon X \to X$ is a graph morphism of the functional graphs of t and t^g for a transformation $t\colon X \to X$.
8. We call G *semiregular* if G_a is trivial for each $a \in X$. This is equivalent to the condition that $g(a) = h(a)$ for $g, h \in G$ and some $a \in X$ implies $g = h$ since $g(a) = h(a)$ yields $h^{-1}(g(a)) = a$. By semiregularity, we find $h^{-1}(g) = \mathrm{id}_X$, giving $g = h$.
9. We call G *regular* if it is semiregular and transitive. Equivalently, G is transitive and there exists $a \in X$ such that $G_a = \{\mathrm{id}_X\}$.
10. The *alternating group* (for a given n) is the unique subgroup $\mathcal{A}_n \leq_G \mathcal{S}_n$ such that $|\mathcal{A}_n| = |\mathcal{S}_n|/2$. See [126] for more information.

There should be no danger of confusion of $\mathcal{A}_n$ or $\mathcal{S}_n$ with the notation for a point stabilizer, where also n as a point might appear in the lower index, or with an automaton from an automaton family indexed by n. For the former, this should not arise since

[2] Note that by our convention of reading functional composition from right to left, the inverse has to occur on the right hand side in the defining equation of t^g to make Equation (5.2) valid. In the literature, for example in [393], conjugation of t by g is also often defined as $g^{-1}tg$ instead.

groups are not denoted by calligraphic letters in the present work, and for the latter the context should make it clear whether we refer to an automaton or the alternating group.

Proposition 5.1 *A primitive permutation group G on a domain containing at least three points is transitive.*

Proof. Suppose G is not transitive. Since X contains at least three points, there exist (not necessarily distinct) $a, b \in X$ such that the set $\Delta = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$ is a proper subset of X and contains at least two points. Now, Δ is a block, and so G is not primitive.[3] □

As defined here, every permutation group on $[2]$ is primitive, even the intransitive trivial permutation group. To exclude this special case, some authors prefer to include transitivity into the definition of primitivity. By Theorem 5.1 this makes no difference for permutation groups on domains containing at least three points.

Example 5.2
1. By Theorem 5.4, for prime n and transitive $G \leq_G \mathcal{S}_n$, each block is trivial and so G is primitive.
2. The symmetric group $\mathcal{S}_n$ is primitive. For suppose $A \subseteq [n]$ is any subset such that $a, b \in A$ and $c \notin A$ for distinct $a, b, c \in [n]$. Then, for $t = (b\ c)$, we have $a \in t(A) \cap A$ and $c \in t(A) \setminus A$, which shows that A is not a block.

If $\sim$ is a G-congruence, then we have the following equivalence for all $g \in G$ and $a, b \in X$:

$$a \sim b \Leftrightarrow g(a) \sim g(b). \tag{5.4}$$

Here, we are exclusively concerned with finite permutation domains. So, from now on, in the remaining examples and results of the present work we always take $X = [n]$.

Example 5.3 The *Klein four-group* $G = \{(), (0\ 1)(2\ 3), (0\ 2)(1\ 3), (0\ 3)(1\ 2)\}$ is a regular subgroup of $\mathcal{S}_4$ where $\{0, 1\}$ and $\{2, 3\}$ form a system of blocks.

Example 5.4 Let $G \leq_G \mathcal{S}_n$ be transitive and $\Delta \subseteq [n]$ be some block. Then the sets $g(\Delta)$ for $g \in G$ are equal-sized blocks partitioning $[n]$. So, n is a multiple of $|\Delta|$.

First, we characterize the G-congruences generated by a permutation cyclically permuting its domains. A related result appears in [287, Lem. 2].

Lemma 5.5 *Set $g = (0\ 1\ 2\ \cdots\ n-1)_{[n]}$ and $G = \langle\{g\}\rangle_G$. Then for every G-congruence $\sim$ there exists a divisor r of n such that, for all $a, b \in [n]$,*

$$a \sim b \Leftrightarrow a \equiv b \pmod{r}. \tag{5.5}$$

Conversely, each divisor r of n defines a G-congruence on $[n]$ by Equation (5.5)*.*

[3] Alternatively, for the trivial group every subset of $[n]$ is a block. So, for $n \geq 3$, a primitive G must be non-trivial. Hence, there exist $a \in [n]$ and $g \in G$ with $g(a) \neq a$. Then $|\mathrm{Orb}_G(a)| > 1$ and as $\mathrm{Orb}_G(a)$ is a block we find $\mathrm{Orb}_G(a) = [n]$, i.e., G is transitive.

Proof. First, suppose $\sim$ is a G-congruence. Let $0 < r \leq n$ be the smallest number such that $g^r(0) \sim 0$. Inductively, we find $0 \sim g^{r \cdot i}(0)$ for all $i \geq 0$. Since $g^i(0) \equiv i \pmod{n}$ for $i \geq 0$, it is $\{x \in [n] \mid x \equiv 0 \pmod{r}\} \subseteq [0]_\sim$. Assume $0 \sim g^{r \cdot i + d}(0)$ for some $i \geq 0$ and $0 \leq d < r$. Since $0 \sim g^{r \cdot i}(0)$ we find $g^d(0) \sim g^{r \cdot i + d}(0)$. So, $g^d(0) \sim 0$, which implies $d = 0$ by the minimal choice of r. Hence, $[0]_\sim \subseteq \{x \in [n] \mid x \equiv 0 \pmod{r}\}$. Now, for $0 \leq a < b < n$ we get

$$\begin{aligned} a \equiv b \pmod{r} &\Leftrightarrow 0 \equiv b - a \pmod{r} \\ &\Leftrightarrow 0 \sim b - a \Leftrightarrow g^a(0) \sim g^a(b - a) \Leftrightarrow a \sim b, \end{aligned}$$

which shows Equation (5.5). Conversely, as $g(x) = (x + 1) \bmod n$ for $x \in [n]$ the relation $a \equiv b \pmod{r}$ yields a G-congruence when n is a multiple of r. □

Next, we collect some properties of semiregular permutation groups.

Proposition 5.6 *Let $G \leq \mathcal{S}_n$ be semiregular. Then $|G| \leq n$, and we have $|G| = n$ if and only if G is regular.*

Proof. Pick any point $a \in [n]$. Consider the map $\varphi\colon G \to [n]$ given by $\varphi(g) = g(a)$. Since G is semiregular, this map is injective. Thus $|G| \leq n$. Furthermore, φ is surjective if and only if G is transitive, and φ is surjective precisely if $|G| = n$. This yields that $|G| = n$ if and only if G is regular. □

Next, we characterize semiregularity by cycle lengths.

Proposition 5.7 *Let $G \leq_G \mathcal{S}_n$. Then G is semiregular if and only if for each $g \in G$ all cycles contained in g have the same length.*

Proof. First, suppose G is not semiregular. Then there exist $a \in [n]$ and $g \in G_a \setminus \{\mathrm{id}_{[n]}\}$. As $a \in \mathrm{fix}(g)$, g contains a cycle of length one and, as $g \neq \mathrm{id}_{[n]}$, it contains a cycle of length greater than one. Conversely, suppose there exists $g \in G$ containing two cycles of different lengths. Then $g^i(a) = a$ and $g^j(b) = b$ with $g(a) \neq a$ and $g(b) \neq b$ for minimal $0 < i < j < n$ and some $a, b \in [n]$, giving $g^i \in G_a$ but $g^i \neq \mathrm{id}_{[n]}$ as $g^i(b) \neq b$. So, G is not semiregular. □

For a semiregular $G \leq_G \mathcal{S}_X$ and $a, b \in X$, since $g(a) = g'(a)$ implies $g(b) = g'(b)$ for $g \in G$, we can define a bijection $h\colon \mathrm{Orb}_G(a) \to \mathrm{Orb}_G(b)$ by setting $h(c) = g(b)$ for $g \in G$ with $g(a) = c$, i.e., the edges of the orbital digraph $\{(g(a), g(b)) \mid g \in G\}$ describe a bijection. If G is transitive, h is a permutation on X. In this case, since, by Equation (5.2), a permutation mapping a to b maps $g(a)$ to $g(b)$ if and only if $h^g = h$, the permutation described by the tuples $\{(g(a), g(b)) \mid g \in G\}$ is characterized as the permutation h with $h(a) = b$ and $hg = gh$ for all $g \in G$. Thus, if G is regular and commutative, it contains a permutation described by the tuples $\{(g(a), g(b)) \mid g \in G\}$.

Example 5.8 Let $G = \langle\{t\}\rangle_G$ for some permutation $t\colon [n] \to [n]$. Then,

1. The orbits of G are the sets $\{t^i(x) \mid i \geq 0\}$, and the non-singleton sets of this form equal the points moved by some non-identity cycle induced by t.
2. t is an cycle if and only if G has at most one non-singleton orbit,
3. G is semiregular if and only if every cycle in t has the same length,
4. G is regular $\Leftrightarrow$ G is transitive $\Leftrightarrow$ t cyclically permutes $[n]$.

5.2 Transitive Permutation Groups

The following three subsections collect results on permutation groups related to transitivity. Section 5.2.1 treats separation of subsets by a transitive group and shows which idempotents arise in the transformation semigroup generated by a transitive group together with a one-point singular. Section 5.2.2 establishes a correspondence between the subgroups containing the stabilizer of a point a and the blocks containing a, under which cosets correspond to images of blocks. Section 5.2.3 relates conditions on how a group maps 2-sets to one another to the size of orbits, to transitivity, and to 2-homogeneity.

5.2.1 Separating Sets and Idempotents

In transitive permutation groups we can separate subsets in the following way (see [36, p. 130] for related discussion).

Theorem 5.9 ([36, 91, 464]) *Let $G \leq_G \mathcal{S}_n$ be transitive and let $A, B \subseteq [n]$ with $|A| \cdot |B| < n$. Then there exists $g \in G$ such that $g(A) \cap B = \emptyset$.*

Our next results let us construct certain idempotent one-point singulars.

Proposition 5.10 *Let $t\colon [n] \to [n]$ be one-point singular with $\mathrm{dupl}(t) = \{c\}$. Then there exists an idempotent $h \in \langle\{t\}\rangle_S$ and we have the following:*

1. *If $t(c) = c$, then h is one-point singular with $\mathrm{dupl}(h) = \{c\}$ and $t^{-1}(c) \subseteq h^{-1}(c)$.*
2. *If t has defect one and the point excluded by t lies in $t^{-1}(c)$, say $\mathrm{excl}(t) = \{a\}$ and $t^{-1}(c) = \{a, b\}$, then h is the simple idempotent mapping a to b.*

Proof. By Theorem 3.20 such an h exists; write $h = t^i$ with $i > 0$.

1: We show $\mathrm{dupl}(t^j) = \{c\}$ for $j \geq 1$ by induction, the case $j = 1$ being clear. For $j \geq 2$ write $t^j = tt^{j-1}$, so that Equation (3.10) yields $\mathrm{dupl}(t^j) = \mathrm{dupl}(t_{|t^{j-1}([n])}) \cup t(\mathrm{dupl}(t^{j-1}))$, where the first set is contained in $\mathrm{dupl}(t) = \{c\}$ and the second equals $t(\{c\}) = \{c\}$. Hence $\mathrm{dupl}(h) = \{c\}$. Moreover $t(x) = c$ gives $h(x) = t^{i-1}(c) = c$, so $t^{-1}(c) \subseteq h^{-1}(c)$.

2: We have $t([n]) = [n] \setminus \{a\}$, and $t(a) = t(b) = c$ with $b \neq a$ gives $t([n] \setminus \{a\}) = t([n]) = [n] \setminus \{a\}$. So t maps the finite set $[n] \setminus \{a\}$ onto itself and thus permutes it. Hence h restricts to an idempotent permutation of $[n] \setminus \{a\}$, that is, to the identity, and $h(a) = t^{i-1}(t(a)) = t^{i-1}(t(b)) = h(b) = b$. □

Remark 5.11 The hypotheses of the two items differ in which point is required to lie in $t^{-1}(c)$: in Item 1 it is the duplicated point c itself, and in Item 2 it is the excluded point a. This accounts for the stronger conclusion in the second case. There, t maps $[n] \setminus \{a\}$ onto itself, so no point outside $t^{-1}(c)$ is ever collapsed by a power of t and we obtain $h^{-1}(c') = t^{-1}(c)$ for the point c' with $\mathrm{dupl}(h) = \{c'\}$. Under Item 1 alone, the inclusion $t^{-1}(c) \subseteq h^{-1}(c)$ can be strict, as passing to powers may collapse further points: for $n = 4$ and t given by $t(0) = t(1) = t(2) = 0$ and $t(3) = 2$ we have $c = 0$ and $t^{-1}(0) = \{0, 1, 2\}$, whereas t^2 is the constant map at 0, so that $h^{-1}(0) = [4]$.

Corollary 5.12 *Let $G \leq_G \mathcal{S}_n$ be a transitive permutation group and let $t\colon [n] \to [n]$ be one-point singular. Then for each $a \in t^{-1}(\mathrm{dupl}(t))$ there exists an idempotent one-point singular $h \in \langle G \cup \{t\}\rangle_S$ with $\mathrm{dupl}(h) = \{a\}$ and $t^{-1}(\mathrm{dupl}(t)) \subseteq h^{-1}(a)$.*

Proof. Write $\mathrm{dupl}(t) = \{c\}$ and pick $g \in G$ with $g(c) = a$, which is possible as G is transitive. As g is a bijection, gt is one-point singular with $\mathrm{dupl}(gt) = \{a\}$ and $(gt)^{-1}(a) = t^{-1}(\mathrm{dupl}(t))$. The latter contains a, so gt fixes a and Item 1 of Theorem 5.10, applied to gt, gives an idempotent $h \in \langle\{gt\}\rangle_S \subseteq \langle G \cup \{t\}\rangle_S$ as claimed. □

Corollary 5.13 *Let $G \leq_G \mathcal{S}_n$ be a transitive permutation group and let $t\colon [n] \to [n]$ be of defect one with $t(a) = t(b)$ for distinct $a, b \in [n]$. Then there exists a simple idempotent $h \in \langle G \cup \{t\}\rangle_S$ with $h(a) = h(b) = b$.*

Proof. Write $\mathrm{excl}(t) = \{e\}$ and pick $g \in G$ with $g(e) = a$, which is possible as G is transitive. As g is a bijection, gt has defect one with $\mathrm{excl}(gt) = \{a\}$ and $(gt)^{-1}(\mathrm{dupl}(gt)) = t^{-1}(\mathrm{dupl}(t)) = \{a, b\}$. Now Item 2 of Theorem 5.10, applied to gt, gives an idempotent $h \in \langle\{gt\}\rangle_S \subseteq \langle G \cup \{t\}\rangle_S$ mapping a to b. □

Applying Theorem 5.13 to the pair (a, b) and to the pair (b, a) shows that for t of defect one with $t(a) = t(b)$ the semigroup $\langle G \cup \{t\}\rangle_S$ contains *both* simple idempotents identifying a and b.

Example 5.14 Let $g = (0\ 1\ 2\ 3)$ and let $t\colon [4] \to [4]$ be given by $t(0) = 1$, $t(2) = 3$ and $t(1) = t(3) = 2$. Then gt permutes $\{0, 2, 3\}$ and g^3t permutes $\{0, 1, 2\}$. Here g^3t is already the simple idempotent mapping 3 to 1, and $(gt)^2$ is the simple idempotent mapping 1 to 3.

5.2.2 Blocks and Point Stabilizers

We next establish a correspondence between the subgroups containing the stabilizer of a point a and the blocks containing a in Item 1 of Theorem 5.15. This also appears in [179, Thm. 1.5A].

Theorem 5.15 *Let $G \leq_G \mathcal{S}_n$ be transitive and $a \in [n]$. Then the following hold.*

1. *The maps*

$$H \mapsto \mathrm{Orb}_H(a) \qquad \text{and} \qquad \Delta \mapsto \{g \in G \mid g(\Delta) = \Delta\}$$

   are mutually inverse bijections between the subgroups H with $G_a \leq_G H \leq_G G$ and the blocks Δ of G with $a \in \Delta$.
2. *For $G_a \leq_G H \leq_G G$, $\Delta = \mathrm{Orb}_H(a)$ and $g_1, g_2 \in G$ we have*

$$g_1H = g_2H \iff g_1(\Delta) = g_2(\Delta),$$

   and hence $gH = \{g' \in G \mid g'(\Delta) = g(\Delta)\}$.

Proof. 1: Let $G_a \leq_G H \leq_G G$ and $\Delta = \mathrm{Orb}_H(a)$. We show

$$\{g \in G \mid g(\Delta) \cap \Delta \neq \emptyset\} = H. \tag{5.6}$$

If $g \in H$ then $g(\Delta) = \Delta$, since Δ is an H-orbit; in particular the left-hand side contains H. Conversely, suppose $g(h_2(a)) = h_1(a)$ for some $h_1, h_2 \in H$. Then $h_1^{-1} g h_2 \in G_a \leq_G H$ and therefore $g \in h_1 H h_2^{-1} = H$. This proves (5.6), and, since $g \in H$ implies $g(\Delta) = \Delta$, it shows both that Δ is a block and that the second map returns H when applied to Δ.

Now let Δ be a block with $a \in \Delta$ and set $H = \{g \in G \mid g(\Delta) = \Delta\}$, which is a subgroup. If $g \in G_a$ then $a \in g(\Delta) \cap \Delta$, so $g(\Delta) = \Delta$ and $g \in H$; hence $G_a \leq_G H$. The inclusion $\mathrm{Orb}_H(a) \subseteq \Delta$ is immediate from the definition of H. Conversely let $b \in \Delta$. By transitivity $b = g(a)$ for some $g \in G$, so $b \in g(\Delta) \cap \Delta$ and thus $g(\Delta) = \Delta$, i.e. $g \in H$ and $b \in \mathrm{Orb}_H(a)$. Hence $\Delta = \mathrm{Orb}_H(a)$, so the first map returns Δ, and the two maps are mutually inverse.

2: By (5.6) we have $g \in H$ if and only if $g(\Delta) = \Delta$. Thus

$$g_1 H = g_2 H \iff g_2^{-1} g_1 \in H \iff g_2^{-1} g_1(\Delta) = \Delta \iff g_1(\Delta) = g_2(\Delta).$$

That $gH = \{g' \in G \mid g'(\Delta) = g(\Delta)\}$ follows since $g' \in gH$ precisely if $g'H = gH$. □

Remark 5.16 By Item 2 of Theorem 5.15 the map $\varphi(gH) = g(\Delta)$ is a well-defined bijection from the cosets of H onto the block system $\{g(\Delta) \mid g \in G\}$ generated by $\Delta = \mathrm{Orb}_H(a)$, and $\varphi(h \cdot gH) = h(\varphi(gH))$ for $h \in G$. Thus subgroups containing G_a together with their cosets correspond to blocks containing a together with their block systems, and, for $g \in G$ and $h \in G$, we have

$$h \in gH \iff h(a) \in g(\Delta). \tag{5.7}$$

As an example, let G be as in Theorem 5.5. Then G can be identified with the additive group of residues modulo n via $g^i \mapsto [i]_{\equiv_n}$. Since every subgroup of that group is the group of multiples of some divisor r of n [393, Sec. 1.4], Equation (5.7) also implies Theorem 5.5.

Remark 5.17 Theorem 5.15 is an instance of the standard correspondence between transitive G-actions and actions on the cosets of a subgroup: a system of blocks induces a G-action for which H is a point stabilizer, and the theorem states that the actions arising in this way are precisely those on the cosets of a subgroup containing a point stabilizer. We refer to [126] for the formalisation via G-actions, which we do not introduce here, and note that by our conventions permutations act on the left, whereas in [126] they act on the right; accordingly, our statements concern left cosets and multiplication from the left.

Remark 5.18 Specialising Theorem 5.15, Item 2 to $H = G_a$, the left cosets of G_a are in bijection with the points of $[n]$. As the cosets partition G and all have size $|G_a|$, we obtain $|G| = n \cdot |G_a|$, the *orbit-stabilizer property* [126, 179, 393, 638]. This implies that a transitive $G \leq_G \mathcal{S}_n$ is regular if and only if $|G| = n$.

Corollary 5.19 *Let $G \leq_G \mathcal{S}_n$ be transitive, $a \in [n]$ and $G_a \leq_G H \leq_G G$. Set $\Delta = \mathrm{Orb}_H(a)$. Then:*

1. *For $g \in G$ we have $g \notin H$ if and only if $a \notin g(\Delta)$.*
2. *If $H \neq G$ then H does not act transitively.*

Proof. 1: Since Δ is a block by Theorem 5.15, $a \in g(\Delta)$ is equivalent to $g(\Delta) = \Delta$ for $g \in G$. Since $H = \{g \in G \mid g(\Delta) = \Delta\}$ by Item 1 of Theorem 5.15, $g \in G \setminus H$ if and only if $a \notin g(\Delta)$.

2: By Theorem 5.15, Item 1 the assignment $H \mapsto \mathrm{Orb}_H(a)$ is injective, and $\mathrm{Orb}_G(a) = [n]$ by transitivity of G. Hence $\mathrm{Orb}_H(a) = [n]$ forces $H = G$. □

5.2.3 Mapping 2-Sets, Orbit Sizes, and Transitivity

We now relate mapping properties of 2-sets to orbit sizes and transitivity. Two technical statements do most of the work. Theorem 5.22 treats images of the form $\{a, y\} \mapsto \{b, y\}$, where a point of the source is retained, and Theorem 5.25 settles, for a single target 2-set, when it is the image of a 2-set containing a fixed point a under a map subject to a side condition. Quantifying the latter over all targets yields Theorem 5.26, which relates several such mapping properties to the size of $\mathrm{Orb}_G(a)$, to transitivity, and to the fixed points of the point stabiliser of a. From these we obtain characterisations of transitivity in Theorem 5.27 and sufficient conditions in Theorem 5.28. Finally, Theorem 5.31 determines which mapping properties force 2-homogeneity, i.e., that any two 2-sets can be mapped to one another. Before that we recall a result of Livingstone and Wagner [418] relating k-homogeneity to k-transitivity.

Theorem 5.20 (Livingstone and Wagner [418]) *Let $G \leq_G \mathcal{S}_n$ be k-homogeneous with $k \geq 2$ and $2k \leq n$. Then G is $(k-1)$-transitive, and for $k \geq 5$ even k-transitive.*

The argument from the following lemma is taken from [445, p. 533], where it is used to deduce that a 2-homogeneous group is transitive.

Lemma 5.21 *Let $G \leq_G \mathcal{S}_n$ and $a, b, c \in [n]$. If $g(\{a, b\}) = \{c, b\}$ for some $g \in G$, then $c \in \mathrm{Orb}_G(a)$.*

Proof. We have $g(a) = c$ or $g(a) = b$. In the former case, $c \in \mathrm{Orb}_G(a)$. In the latter case, we have $g(b) = c$ and thus $g^2(a) = c$. So also $c \in \mathrm{Orb}_G(a)$. □

Proposition 5.22 *Let $G \leq_G \mathcal{S}_n$, let $a \in [n]$ and let $A \subseteq [n]$. Suppose that for every $b \in A$ there are $y \in [n]$ and $g \in G$ with $g(\{a, y\}) = \{b, y\}$. Then*

$$A \cup \{a\} \subseteq \mathrm{Orb}_G(a).$$

Proof. By Theorem 5.21 we have $b \in \mathrm{Orb}_G(a)$ for every $b \in A$, and $a \in \mathrm{Orb}_G(a)$ holds anyway. □

Remark 5.23 The method of proof of Theorem 5.21 actually established $c \in B$ for

$$B = \{g^2(a) \mid g \in G\} \cup \{g(a) \mid g \in G \text{ with } g(b) = b \text{ for some } b \neq a\}.$$

Thus A from Theorem 5.22 is also contained in this set. For $A = [n]$ this yields that, for fixed $a \in [n]$, for each $c \in [n]$ there exist $b \in [n]$ and $g \in G$ such that $g(\{a, b\}) = \{c, b\}$ if and only if $[n] = B$. As $B \subseteq \mathrm{Orb}_G(a)$, this forces G to be transitive. For example, we have $[n] = B$ for transitive groups of odd order. Indeed, on a finite group the squaring map $g \mapsto g^2$ is surjective if and only if it is injective, and it fails to be injective precisely when there is an involution, i.e., a non-identity element whose square equals the identity. By Cauchy's theorem an involution exists whenever $|G|$ is even, and conversely an involution has order 2, so that 2 divides $|G|$. Hence for odd $|G|$ we get $\{g^2 \mid g \in G\} = G$ and therefore $\{g^2(a) \mid g \in G\} = \mathrm{Orb}_G(a) = [n]$. Note also that for transitive G the condition does not depend on the choice of a: conjugation by an element $h \in G$ with $h(a) = a'$ carries the two sets for a onto those for a'.

Remark 5.24 We do not need this fact, but it is worth noting that, for fixed $a, b \in [n]$, if, for every $c \in [n]$, we have $g(\{a, b\}) = \{a, c\}$ or $g(\{a, b\}) = \{b, c\}$ for some $g \in G$, then $[n] = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$. This follows since then $c \in \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$ for each $c \in [n]$.

Proposition 5.25 *Let $G \leq_G \mathcal{S}_n$, let $a \in [n]$ and let $\{c, d\} \subseteq [n]$ be a 2-set.*

1. *For $T \subseteq [n]$ there are $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ and $g(a) \in T$ if and only if $\{c, d\} \cap \mathrm{Orb}_G(a) \cap T \neq \emptyset$.*
2. *There are $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ and $g(b) \neq b$ if and only if there is $g \in G$ with $g(a) = c$ and $g(d) \neq d$, or with $g(a) = d$ and $g(c) \neq c$.*

Proof. 1: Given such g and b, we have $g(a) \in g(\{a, b\}) = \{c, d\}$ as well as $g(a) \in \mathrm{Orb}_G(a)$ and $g(a) \in T$. Conversely, for $x \in \{c, d\} \cap \mathrm{Orb}_G(a) \cap T$ and y the other element of $\{c, d\}$, choose $g \in G$ with $g(a) = x$ and put $b = g^{-1}(y)$; then $g(a) \in T$ and $g(\{a, b\}) = \{x, y\} = \{c, d\}$.

2: Given such g and b, we have $g(a) \in \{c, d\}$; say $g(a) = c$, so that $g(b) = d$ and $b = g^{-1}(d)$, and $g(b) \neq b$ gives $d \neq g^{-1}(d)$, i.e., $g(d) \neq d$. Conversely, for $g(a) = c$ and $g(d) \neq d$ put $b = g^{-1}(d)$; then $g(\{a, b\}) = \{c, d\}$ and $g(b) = d \neq b$, as $b = d$ would give $g(d) = d$. Interchanging c and d gives the remaining cases. □

Corollary 5.26 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 2$ and $a \in [n]$.*

1. *For every 2-set $\{c, d\} \subseteq [n]$ there are $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ if and only if $|\mathrm{Orb}_G(a)| \geq n - 1$.*
2. *For every 2-set $\{c, d\} \subseteq [n]$ there are $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ and $g(a) \neq a$ if and only if G is transitive.*
3. *For every $b \in [n] \setminus \{a\}$ there are $g \in G$ and $c \in [n]$ with $g(\{a, c\}) = \{a, b\}$ and $g(a) \neq a$ if and only if G is transitive.*
4. *For every $b \in [n] \setminus \{a\}$ there are $g \in G$ and $c \in [n]$ with $g(\{a, c\}) = \{a, b\}$ and $g(c) \neq c$ if and only if every point fixed by all of G_a lies in $\mathrm{Orb}_G(a)$.*

Proof. 1: By Item 1 of Theorem 5.25 with $T = [n]$ the condition states that every 2-set meets $\mathrm{Orb}_G(a)$, which holds if and only if $|[n] \setminus \mathrm{Orb}_G(a)| \leq 1$, as a 2-set contained in that complement exists precisely when it has at least two elements.

2 and 3: By Item 1 of Theorem 5.25 with $T = [n] \setminus \{a\}$, both conditions state that the 2-sets in question meet $\mathrm{Orb}_G(a) \cap T = \mathrm{Orb}_G(a) \setminus \{a\}$.

For a target $\{a, b\}$ with $b \neq a$ this says that $b \in \mathrm{Orb}_G(a)$, so Item 3 implies that G is transitive (For a direct argument, for the target set $\{a, b\}$ we have $g(a) \in \{a, b\}$ with $g(a) \neq a$, which gives $g(a) = b$.). If G is transitive, then $\mathrm{Orb}_G(a) \setminus \{a\} = [n] \setminus \{a\}$, which is met by every 2-set, giving Item 2. Finally, Item 2 implies Item 3, as the latter only concerns the targets $\{a, b\}$. Hence Item 2, Item 3 and transitivity of G are equivalent, which proves both stated equivalences.

4: Consider some $b \in [n] \setminus \{a\}$. By Item 2 of Theorem 5.25 applied to $\{a, b\}$, such g and c exist if and only if there is $g \in G$ with $g(a) = a$ and $g(b) \neq b$, or with $g(a) = b$, i.e., if and only if some element of G_a moves b or $b \in \mathrm{Orb}_G(a)$. This is equivalent to the condition that if the considered b is fixed by all elements from G_a, then it must be in $\mathrm{Orb}_G(a)$. Demanding this for every $b \in [n] \setminus \{a\}$ says precisely that a point fixed by all of G_a lies in $\mathrm{Orb}_G(a)$. □

Theorem 5.27 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 2$ and $a \in [n]$. Then the following are equivalent.*

1. *G is transitive.*
2. *For distinct $c, d \in [n]$ there is $g \in G$ with $g(a) = c$ and $g(d) \neq d$, or $g(a) = d$ and $g(c) \neq c$.*
3. *Every point fixed by all of G_a lies in $\mathrm{Orb}_G(a)$, and every point of $[n] \setminus \mathrm{Orb}_G(a)$ is fixed by all of G.*

Proof. $1 \Rightarrow 2$: Choose $g_0, g_1 \in G$ with $g_0(a) = c$ and $g_1(a) = d$. If $g_0(d) \neq d$ or $g_1(c) \neq c$, we are done. Otherwise $h = g_1^{-1} g_0$ satisfies $h(a) = g_1^{-1}(c) = c$ and $h(d) = g_1^{-1}(d) = a \neq d$, the latter since $a = d$ would give $c = g_0(a) = g_0(d) = d$.

$2 \Rightarrow 3$: By Item 2 of Theorem 5.25, condition 2 gives, for every 2-set $\{c, d\}$, some $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ and $g(b) \neq b$. Omitting $g(b) \neq b$ gives $|\mathrm{Orb}_G(a)| \geq n - 1$ by Item 1 of Theorem 5.26, so at most one point lies outside $\mathrm{Orb}_G(a)$, and it forms an orbit and is thus fixed by all of G. Restricting to the targets $\{a, b\}$ gives the remaining condition by Item 4 of Theorem 5.26.

$3 \Rightarrow 1$: Let $b \in [n] \setminus \mathrm{Orb}_G(a)$. Then b is fixed by all of G, hence by all of G_a, and so lies in $\mathrm{Orb}_G(a)$, a contradiction. Thus $\mathrm{Orb}_G(a) = [n]$. □

Finally, we give some sufficient conditions.

Proposition 5.28 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 2$ and $a \in [n]$. Then G is transitive if one of the following holds:*

1. *$|\mathrm{Orb}_G(a)| \geq n - 1$ and for $b \in [n] \setminus \{a\}$ we have $g(\{a, c\}) = \{a, b\}$ for some $c \in [n] \setminus \{b\}$ and $g \in G$.*
2. *For distinct $c, d \in [n]$ we have $g(\{a, b\}) = \{c, d\}$ for some $g \in G$ and $b \in [n] \setminus \{c\}$.*
3. *For every $c \in [n]$ we have $g(\{a, b\}) = \{c, b\}$ for some $b \in [n]$ and $g \in G$.*

Proof. 1: Such a g maps c to a or to b with $c \notin \{a, b\}$, so $g(c) \neq c$, and Item 4 of Theorem 5.26 gives that every point fixed by all of G_a lies in $\mathrm{Orb}_G(a)$. As $|\mathrm{Orb}_G(a)| \geq n - 1$ leaves at most one point outside $\mathrm{Orb}_G(a)$, which forms an orbit and is fixed by all of G, Theorem 5.27 applies.

2: By Item 1 of Theorem 5.26 we have $|\mathrm{Orb}_G(a)| \geq n - 1$, and the pair b, a gives $g(\{a, c\}) = \{a, b\}$ for some $c \neq b$, which is the condition from Item 1.

3: This follows from Theorem 5.22 applied with $A = [n]$. □

Remark 5.29 None of the conditions of Theorem 5.28 is necessary for transitivity: for $n = 4$ the group $G = \langle (0\ 1\ 2\ 3) \rangle_G$ is transitive, but at $a = 0$ and $b = 2$ the equality $g(\{0, c\}) = \{0, 2\}$ forces $c = 2$, so that Item 1 fails, and applying Item 2 to the pair $2, 0$ would give exactly such a $c \neq 2$. Likewise Item 3 fails for odd c: in $g(\{0, b\}) = \{c, b\}$ we have $c \neq b$, so either $g(0) = c$ and $g(b) = b$, which for a shift forces g to be the identity and hence $c = 0$, or $b = g(0)$ and $c = g(b) = g(g(0)) = 2k$, and both values are even.

Remark 5.30 The ordered reading matters in Item 2 of Theorem 5.28. Demanding instead a single witness with $b \notin \{c, d\}$ is strictly stronger and is unsatisfiable for $n = 3$: for the 2-set $[n] \setminus \{a\}$ there is no b with $b \neq a$ and $b \notin [n] \setminus \{a\}$. By contrast, $\mathcal{S}_3$ satisfies the hypothesis of Item 2 from Theorem 5.28. For $n = 4$ the transitive group $G = \langle (0\ 1\ 2\ 3) \rangle_G$ violates it at $a = 0$ and the 2-set $\{0, 2\}$: a witness would need $b \in \{1, 3\}$, whereas $g(\{0, b\}) = \{0, 2\}$ forces $b = 2$.

Proposition 5.31 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 3$. Then the following are equivalent.*

1. *G is 2-homogeneous.*
2. *There exist distinct $a, b \in [n]$ such that for all $c, d \in [n]$ with $\{c, d\} \neq \{a, b\}$ there is $g \in G$ with $g(\{a, c\}) = \{a, d\}$ or $g(\{b, c\}) = \{b, d\}$.*
3. *There exists $a \in [n]$ such that*
   a. *for all $b, c \in [n] \setminus \{a\}$ there is $g \in G$ with $g(\{a, b\}) = \{a, c\}$, and*
   b. *there are $b \in [n] \setminus \{a\}$ and $g \in G$ with $a \notin g(\{a, b\})$.*

Proof. 1⇒2: Let a, b be any two distinct points and consider $\{c, d\} \neq \{a, b\}$. Then $a \notin \{c, d\}$ or $b \notin \{c, d\}$. As the two conditions in Item 2 arise from one another by interchanging a and b, we may assume $a \notin \{c, d\}$. Then $\{a, c\}$ and $\{a, d\}$ are 2-sets and 2-homogeneity gives $g(\{a, c\}) = \{a, d\}$ for some $g \in G$.

2⇒3: Let a, b be as in Item 2 and let O be the orbit of $\{a, b\}$. Note that $g(\{x, x\})$ is a singleton and hence never a 2-set. For $c \neq a$ applying Item 2 to b, c, which is admissible as $\{b, c\} \neq \{a, b\}$, thus gives $g(\{a, b\}) = \{a, c\}$, so that $\{a, c\} \in O$. Hence any two 2-sets containing a lie in O and are therefore images of one another. Finally, picking $e \notin \{a, b\}$, possible as $n \geq 3$, and applying 2 to a, e, admissible as $\{a, e\} \neq \{a, b\}$, gives $g(\{b, a\}) = \{b, e\}$ with $a \notin \{b, e\}$.

3⇒1: By assumption, there exist $b \in [n] \setminus \{a\}$ and $g \in G$ such that $a \notin g(\{a, b\})$. Let O be the orbit of $\{a, b\}$ for the action of G on the 2-sets. By assumption, every 2-set containing a lies in O. This also implies that every 2-set containing a point from $\mathrm{Orb}_G(a)$ is in O since each such 2-set can be mapped to one containing a.

Put $d = g(a)$, so that $d \neq a$. For $z \neq a$ the 2-sets $\{a, z\}$ and $\{a, d\}$ lie in O. So $h(\{a, d\}) = \{a, z\}$ for some $h \in G$. Whence $z = h(a)$ or $z = h(d) = h(g(a))$ and thus $z \in \mathrm{Orb}_G(a)$. So $\mathrm{Orb}_G(a) = [n]$, and by the above every 2-set lies in O. □

Remark 5.32 Condition 3b cannot be omitted from Item 3 of Theorem 5.31: for $G = \langle (1\ 2\ 3) \rangle_G$ on $\{0, 1, 2, 3\}$ and $a = 0$ the three 2-sets containing 0 are images of one another, so that 3a holds, yet G is not 2-homogeneous, and not even transitive.

The following is immediate but stated for later reference.

Proposition 5.33 *Let G be a permutation group on $[n]$ with $n \geq 2$ that is not transitive. Then there exists a proper subset $A \subsetneq [n]$ with $|A| \geq \lceil \frac{n}{2} \rceil$ such that $g(A) = A$ for all $g \in G$.*

Proof. Let O be an orbit of G. Both O and $[n] \setminus O$ are G-invariant, and one of them has size at least $\lceil \frac{n}{2} \rceil$. As G is not transitive, both are proper subsets of $[n]$. □

5.3 Permutation Groups and Abstract Groups

Often groups are given by some representation. For example, by sets of permutations as permutation groups. We call a group an *abstract group*[4] if we want to emphasize that we do not assume some internal structure or a (re-) presentation beyond the axiomatic definition of a group.

Now, let $(G, \cdot, ^{-1}, 1)$ be an abstract group. Next, we describe one possibility on how to represent G as a permutation group. For each $g \in G$ let $\lambda_g \colon G \to G$ be the permutation defined by $\lambda_g(h) = gh$ for $h \in G$. The map $\Pi \colon G \to \mathcal{S}_G$ given by $\Pi(g) = \lambda_g$ is injective, and we have $\Pi(gg') = \lambda_{gg'} = \lambda_g \lambda_{g'}$, $\Pi(1) = \mathrm{id}_G$, and $\Pi(g^{-1}) = \lambda_{g^{-1}} = \lambda_g^{-1}$ for $g, g' \in G$, i.e., it is a group morphism. This yields that G can be uniquely identified with the permutation group $\{\lambda_g \mid g \in G\} \leq_G \mathcal{S}_G$, giving a permutation representation of G that is called the *left regular (Cayley) representation*. This is not the only way to represent an abstract group as a permutation group, the interested reader can find more information in the discussion (Section 5.6).

Remark 5.34 Every regular permutation group arises in the form of the left regular representation on itself in the following way. Let $G \leq \mathcal{S}_n$ be a regular permutation group and $a \in [n]$. The map $\varphi \colon [n] \to G$ given by $\varphi(b) = g$ for the unique $g \in G$ with $g(a) = b$ is a bijection. Since for $h, g \in G$ and $b \in [n]$ with $g(a) = b$ we have $(h \circ g)(a) = h(g(a)) = h(b)$, we find

$$\varphi(h(b)) = h \circ \varphi(b) = \lambda_h(\varphi(b))$$

where $\lambda_h \colon G \to G$ is the left regular representation introduced above. Thus $h = \varphi^{-1} \lambda_h \varphi$ and h maps b to c for $b, c \in [n]$ if and only if λ_h maps $\varphi(b)$ to $\varphi(c)$. Hence the map

[4] In the same vein, we can talk about abstract monoids or abstract semigroups when solely referring to their axiomatic properties.

φ identifies the permutation domains $[n]$ and G in a way so that G and $\overline{G}$ act in the same way on each. As an example, applying this construction to the right regular representation $\overline{G} = \{\rho_g \mid g \in G\}$, which is a subgroup of $\mathcal{S}_G$, and some $g \in G$, we obtain $\varphi\colon G \to \overline{G}$ such that for $h \in G$ we have $\varphi(h) = \rho_{h^{-1}g}$ since $\rho_{h^{-1}g}(g) = g(h^{-1}g)^{-1} = h$.

Next, we show how to obtain a representation of $G \times G$ as a subgroup of $\mathcal{S}_G$ for abstract groups G with trivial center, called the *diagonal action (on G)*.[5]

Proposition 5.35 *Let G be an abstract group with identity $1 \in G$ and $Z(G) = \{1\}$. For $x, y \in G$ let $\delta_{x,y}\colon G \to G$ be given by $\delta_{x,y}(g) = xgy^{-1}$.*

1. *The map $(x, y) \mapsto \delta_{x,y}$, $x, y \in G$, is an injective morphism from $G \times G$ into S_G.*
2. *$G^D = \{\delta_{x,y} \mid (x, y) \in G\}$ is a transitive permutation group (the representation of $G \times G$ by the diagonal action) on G that is primitive if and only if G is simple.*

Proof. 1 Let $x_1x_2, y_1, y_2 \in G$. As

$$\delta_{x_1x_2,y_1y_2}(g) = (x_1x_2)g(y_1y_2)^{-1} = x_1(x_2gy_2^{-1})y_1^{-1} = (\delta_{x_1,y_1}\delta_{x_2,y_2})(g),$$

for $g \in G$ we obtain a morphism. For injectivity, suppose $\delta_{x_1,y_1} = \delta_{x_2,y_2}$. Then $x_2^{-1}x_1gy_1^{-1}y_2 = g$ for all $g \in G$. For $g = 1$, we get $x_2^{-1}x_1y_1^{-1}y_2 = 1$, which yields $x_2^{-1}x_1 = y_2^{-1}y_1$. Now, $x_2^{-1}x_1g(x_2^{-1}x_1)^{-1} = g$ for all $g \in G$ if and only if $x_2^{-1}x_1 \in Z(G)$. As $Z(G) = \{1\}$, $x_1 = x_2$ and $y_1 = y_2$.

2 As $\delta_{hg^{-1},1}(g) = hg^{-1}g = h$ for $g, h \in G$, we find that G^D is transitive. Next, we show that blocks of G^D containing 1 are precisely the normal subgroups of G, which gives the claim. Let $\Delta \subseteq G$ be a block of G^D with $1 \in \Delta$ and $x, y \in \Delta$. As $x = xyy^{-1}$ we have $x \in \Delta \cap x\Delta y^{-1}$. So $x\Delta y^{-1} = \Delta$. As $1 \in \Delta$ this yields $xy^{-1} \in \Delta$. Hence Δ is a subgroup of G. Now, let $g \in G$. Then $1 \in \Delta \cap g\Delta g^{-1}$. So $\Delta = g\Delta g^{-1}$ and Δ is a normal subgroup. Conversely, let N be a normal subgroup of G. Suppose $x, y \in G$. We have $\delta_{x,y}(N) = xNy^{-1} = xy^{-1}yNy^{-1} = xy^{-1}N$, which is a left coset. Since these cosets partition G, $\delta_{x,y}(N) \cap N \neq \emptyset$ implies $\delta_{x,y}(N) = N$. So, N is a block of G^D. □

Next, more generally, suppose we have given a subgroup H of G such that [6]

$$\bigcap_{g \in G} gHg^{-1} = \{1\} \tag{5.8}$$

Then we can construct a permutation group on the left cosets $X = \{hH \mid h \in G\}$ by $\overline{G} = \{\lambda_g \mid g \in G\}$ where $\lambda_g\colon X \to X$ is a permutation on X given by $\lambda_g(hH) = (gh)H$. Equation (5.8) ensures that $\lambda_g \neq \lambda_{g'}$ if and only if $g \neq g'$. Further, we have $\lambda_{gg'} = \lambda_g\lambda_{g'}$ for $g, g' \in G$. So, $g \mapsto \lambda_g$ is an isomorphism of abstract groups between G and $\overline{G}$. In this way, we have realized G as a permutation group on X. For $H = \{1\}$ (in this case we identify X and G) we obtain the left regular representation. There is also a

[5] This is actually a more general construction related to so-called group actions, see [36, Example 2.6] or [126, Exercise 1.6].

[6] In general, $\bigcap_{g \in G} gHg^{-1}$ is called the *core* of G and H is called *core-free* if Equation (5.8) holds.

corresponding construction on the right cosets by considering the mappings $\rho_g : Y \to Y$ given by $\rho_g(Hh) = H(hg^{-1})$ for $Y = \{Hh \mid h \in H\}$. The inversion entered by our choice on the order of function composition, so that the equation $\rho_{gg'} = \rho_g\rho_{g'}$ is valid (see the related discussion in Section 3.11).[7] The permutation representation obtained here for $H = \{1\}$ is also called the *right regular (Cayley) representation*.

By considering those subgroups H that fulfill Equation (5.8) we, in fact, obtain all permutation representations of G. This follows since H precisely equals the point stabilizer of the permutation group constructed above, and that, by Theorem 5.15, every permutation group can be realized as a permutation group on the left cosets for some point stabilizer subgroup. A form of these representations without Equation (5.8) seems to have been first observed by Burnside in [118], where he also refers to Dyck (without citing a source).

5.4 Permutation Groups and Non-Permutations

Here, we collect results about transformation semigroups generated by a permutation group G and a set $T \subseteq \mathcal{T}_n$ of non-permutations.

Proposition 5.36 *Let $G \leq_G \mathcal{S}_n$ and $T \subseteq \mathcal{T}_n \setminus S_n$. Then G is the group of units of $\langle G \cup T\rangle_S$.*

Proof. By Theorem 3.12, Item 3.18, for $t \in \mathcal{T}_n \setminus \mathcal{S}_n$ or $h \in \mathcal{T}_n$ we have $ht, th \in \mathcal{T}_n \setminus S_n$, which gives the claim. □

By successively using the formula $gt = t^g g$ for a transformation t and permutation g, we obtain the next statement.

Proposition 5.37 *Let $G \leq_G \mathcal{S}_n$ be a permutation group and $T \subseteq \mathcal{T}_n$. Set $S = \langle G \cup T\rangle_S$. Let $K = \{t^g \mid t \in T, g \in G\}$ be the set of conjugates of elements from T. Then $S = \langle K\rangle_M G = G\langle K\rangle_M$.*

Proof. As $K \subseteq S$ and $G \subseteq S$, we have $\langle K\rangle_M \subseteq S$ and so also $\langle K\rangle_M G \subseteq S$. Next, suppose $s \in S$. Then we have $s = g_0t_1g_1t_2g_2\cdots t_{n-1}g_{n-1}t_ng_n$ and

$$g_0t_1g_1t_2g_2\cdots t_{n-1}g_{n-1}t_ng_n = t_1^{g_0}t_2^{g_0g_1}\cdots t_{n-1}^{g_0g_1\cdots g_{n-2}}t_n^{g_0g_1\cdots g_{n-1}}g_0g_1\cdots g_n. \tag{5.9}$$

The above formula can be seen by induction on the number of t_i's in the product. If s is a product of permutations, then the formula clearly holds. Otherwise, we have

$$g_0t_1g_1t_2g_2\cdots t_{n-1}g_{n-1}t_ng_n = t_1^{g_0}g_0g_1t_2g_2\cdots t_{n-1}g_{n-1}t_ng_n$$

and, by induction hypothesis,

$$g_0g_1t_1g_2\cdots t_{n-1}g_{n-1}t_ng_n = t_1^{g_0g_1}\cdots t_{n-1}^{g_0g_1\cdots g_{n-2}}t_n^{g_0g_1\cdots g_{n-1}}g_0g_1\cdots g_n,$$

[7] Note that when the convention is adopted that in a function composition the left-most function is applied first, we have to use an inversion for the left regular action instead, see, for example, [126].

which yields Equation (5.9). Since $t_1^{g_0} t_2^{g_0 g_1} \cdots t_{n-1}^{g_0 g_1 \cdots g_{n-2}} t_n^{g_0 g_1 \cdots g_{n-1}} \in \langle K \rangle_M$ and $g_1 g_2 \cdots g_n \in G$ we obtain $S \subseteq \langle K \rangle_M G$. So $S = \langle K \rangle_M G$. Similarly, we can show

$$g_0 t_1 g_1 t_2 g_2 \cdots t_{n-1} g_{n-1} t_n g_n = g_0 g_1 \cdots g_n t_1^{(g_1 g_2 \cdots g_n)^{-1}} t_2^{(g_2 \cdots g_n)^{-1}} \cdots t_{n-1}^{(g_{n-1} g_n)^{-1}} t_n^{g_n^{-1}},$$

which then yields $S = G\langle K \rangle_M$. □

The following corollary was noted in [29, Sec. 2]).

Corollary 5.38 *Suppose $G \leq_G \mathcal{S}_n$ is a permutation group and $t \in \mathcal{T}_n$. If $\langle G \cup \{t\} \rangle_S$ contains a constant, then some constant is a composition of conjugates of t.*

Next, we note a special case when t is idempotent.

Corollary 5.39 *Suppose $G \leq_G \mathcal{S}_n$ is a permutation group and $t \in \mathcal{T}_n$ is an idempotent. Set $S = \langle G \cup \{t\} \rangle_S$. Let E be the set of idempotents of S. Then $S = \langle E \rangle_M G$.*

Proof. Let $K = \{t^g \mid g \in G\}$. By Equation (5.2) the mappings in K are idempotents, so $K \subseteq E$. With Proposition 5.37 we find $S = \langle K \rangle_M G \subseteq \langle E \rangle_M G$. As $G \subseteq S$ and $\langle E \rangle_M \subseteq S$, and so $\langle E \rangle_M G \subseteq S$, we conclude $S = \langle E \rangle_M G$. □

As $t_1 \cdots t_n g^{-1} = g^{-1} t_1^g \cdots t_n^g$ for a permutation g and transformations $t_1, \ldots, t_n$ we have, in Theorem 5.37 and Theorem 5.39,

$$\langle K \rangle_M G = G\langle K \rangle_M \text{ and } \langle E \rangle_M G = G\langle E \rangle_M.$$

Further, as G contains the identity transformation, we have $\langle G \cup \{t\} \rangle_S = \langle G \cup \{t\} \rangle_M$. However, for a non-permutation t, $\langle K \rangle_S G \neq \langle K \rangle_M G$, as $\mathrm{id}_{[n]} \notin \langle K \rangle_S$.

5.5 Graphs and Permutation Groups

Permutation group theory enters graph theory via the automorphism group in a natural way. Here, we introduce some further notions connecting graphs and permutation groups. Section 5.5.1 collects results on generating sets for $\mathcal{S}_n$. For example, we give a graph-theoretical characterization when a set of transpositions generate $\mathcal{S}_n$. Section 5.5.2 introduces vertex-transitive graphs and investigates the connectivity of vertex-transitive graphs. Lastly, Section 5.5.3 introduces orbital graphs.

We often denote graphs and groups with the capital letter G, but the context should make it clear whether we mean a graph or a group.

5.5.1 Generating Sets for the Full Symmetric Group

It is a well-known result that permutation cyclically permuting a finite set and a transposition on this set generate the full symmetric group. Isaacs and Zieschang [338, Thm. B] gave the following generalization.

Theorem 5.40 (Isaacs and Zieschang [338, Thm. B]) *Let* $x = (0\ 1\ 2\ \cdots\ n-1)_{[n]}$ *and* $y = (0\ 1\ 2\ \cdots\ m-1)_{[n]}$ *for some* $1 < m < n$. *Then* $\langle\{x, y\}\rangle_G$ *equals the full symmetric group on* $[n]$ *unless both n and m are odd, in which case* $\langle\{x, y\}\rangle_G$ *equals* $\mathcal{A}_n$.

Isaacs and Zieschang [338, Thm. A] also proved that for $n \neq 4$ and $x \in \mathcal{S}_n$ there exists $y \in \mathcal{S}_n$ such that $\langle\{x, y\}\rangle_G = \mathcal{S}_n$ (the exception being any element x in the Klein four-group from Theorem 5.3). Actually, this result was first proved by Sophie Piccard [482, pp. 80–86] in 1946, but got lost in limbo[8] and was reproved by Isaacs & Zieschang [338] in 1996.

By Theorem 4.7 the full symmetric group on a finite set is generated by all transpositions. Next, we state a refined result, needed in Chapter 14, that characterizes the sets of transpositions generating $\mathcal{S}_n$ in terms of an associated graph. For the convenience of the reader, we give a proof (which can also be found in [249, Lem. 3.10.1]; compare also the proof of [34, Lem. 6(a)] which uses a similar argument).

Proposition 5.41 *Let* $T \subseteq \mathcal{S}_n$ *be a set of transpositions and consider the undirected graph* $G = ([n], \{\{a, b\} \mid (a\ b) \in T\})$. *Then G is connected if and only if* $\langle T\rangle_G = \mathcal{S}_n$.

Proof. Define a relation $\sim$ on $[n]$ by $a \sim b$ if and only if $a = b$ or $(a\ b) \in \langle T\rangle_G$. Clearly, $\sim$ is reflexive and symmetric. Further, for $a \sim b$ and $b \sim c$ with distinct $a, b, c \in [n]$ it is $(a\ b)^{(b\ c)} = (a\ c) \in \langle T\rangle_G$; thus $\sim$ is transitive and an equivalence relation. As the graph G results from the digraph $G_\sim$ by forgetting about the direction of the edges, from Theorem 3.70 we obtain that the connected components equal the equivalence classes of $\sim$. So, if G is connected, then all points are pairwise comparable for $\sim$, and $\langle T\rangle_G$ contains all transpositions. By Fact 4.7, we find $\langle T\rangle_G = \mathcal{S}_n$. Otherwise, suppose G is not connected. Then, we can pick a, b with $a \nsim b$, i.e., $(a\ b) \notin \langle T\rangle_G$ and so $\langle T\rangle_G \neq \mathcal{S}_n$. □

Remark 5.42 In the proof of Jordan's theorem from [338], it is also shown that for a set of 3-cycles T, $\langle T\rangle_G$ contains the alternating group if and only if the graph on $[n]$ where $a, b \in [n]$ are joined by an edge if and only if $a, b \in \mathrm{move}(t)$ for some $t \in T$ is connected.

In [373] it was shown that a k-transitive group of degree $n > k$ is generated by all elements fixing precisely i points if $i \leq k - 3$. Since the full symmetric group on a set of size n is k-transitive for each $1 \leq k \leq n$, by this result we obtain more generating sets for the full symmetric group of a finite set.

5.5.2 Vertex-Transitive Graphs

A (directed or undirected) graph is *vertex-transitive* if its automorphism group acts transitively on its vertices. If $G = (V, E)$ is vertex-transitive, then every vertex has the same outdegree and we call the common value the *outdegree of* G.

[8] We were not able to obtain a copy of Piccards original work [482] but learned this through the work of Salomaa [545].

A proof of the next statement can be found in [126, Thm 1.10], [36, Thm 1.1], [249, Chap. 2.6], and in [536] (in the latter work formulated with slightly different notions but easily seen to give the result).

Theorem 5.43 *Let $G = (V, E)$ be a finite vertex-transitive digraph. If G is weakly connected, then G is strongly connected.*

Next, we link the outdegree of a vertex-transitive digraph to its connectivity. The result appears in [275, Prop. 3.4] (compare also [48, Thm 3.7 (a)] where it is formulated in terms of the connectivity from Theorem 3.65).

Theorem 5.44 (Hamidoune [275]) *If $G = (V, E)$ is a finite strongly connected vertex-transitive digraph of outdegree d without edges of the form (v, v) for $v \in V$, then G is strongly $\left(\left\lfloor \frac{d}{2} \right\rfloor + 1\right)$-connected.*

Proof. We can assume $E \neq V \times V$. Since G is finite, there exists a largest number k such that G is strongly k-connected, but not strongly $(k+1)$-connected. For $U \subseteq V$ set $N^+(U) = \{v \in V \setminus U \mid \exists u \in U : (u, v) \in E\}$. We have $k \leq |N^+(\{u\})| = d$ for each $u \in V$. An *atom* [273] $A \subseteq V$ is a set of minimal cardinality with $|N^+(A)| = k$ and $A \cup N^+(A) \neq V$.[9] In a strongly connected digraph the atoms partition V [273] (see also [274, 275, Thm. 2.1 in both]). As an automorphism maps atoms to atoms, the atoms induce an $\mathrm{Aut}(G)$-congruence on the vertices. Hamidoune proved that $|A| \leq k$ [275, Lem. 3.3]. So, for $u \in A$, as $N^+(\{u\}) \subseteq A \setminus \{u\} \cup N^+(A)$, we get $d \leq |A \setminus \{u\}| + |N^+(A)| < |A| + k \leq 2k$. □

In the context of the present work, we actually only need the following implication.

Corollary 5.45 *If $G = (V, E)$ is a finite strongly connected vertex-transitive graph without edges of the form (v, v) for $v \in V$, then G is strongly $\frac{3}{2}$-connected.*

Proof. Let d be the outdegree of G. For $d = 1$, G forms a single cycle, i.e., a strongly connected digraph where every vertex has outdegree one, which is clearly strongly $\frac{3}{2}$-connected. For $d \geq 2$, by Theorem 5.44, G is strongly 2-connected, hence strongly $\frac{3}{2}$-connected. □

Note that, by Theorem 5.45, the conclusion of Theorem 5.44 is valid for $d = 1$ even after removal of the rounding, which somehow explains our choice of naming for strong $\frac{3}{2}$-connectivity.

5.5.3 Orbital Graphs

Suppose $G \leq_G \mathcal{S}_X$ is a permutation group. Let $a, b \in X$. Then we can associate two graphs with the tuple (a, b) in a natural way.

[9] For undirected graphs, atoms were introduced in [427, 428, 627].

1. The *directed orbital graph* of the tuple (a, b) is the graph with vertex set X and edge set $\{(g(a), g(b)) \mid g \in G\}$.
2. The *undirected orbital graph* of the tuple (a, b) (or the 2-set $\{a, b\}$) is the graph with vertex set X and edge set $\{\{g(a), g(b)\} \mid g \in G\}$.

Example 5.46 Figure 5.1 shows the directed orbital graphs of the group $G = \langle\{(0\ 1\ 2\ 3)\}\rangle_G$. The undirected orbital graphs are obtained from these by forgetting about the directions on the edges. Since G contains no element swapping two points, the orbital graphs are oriented.

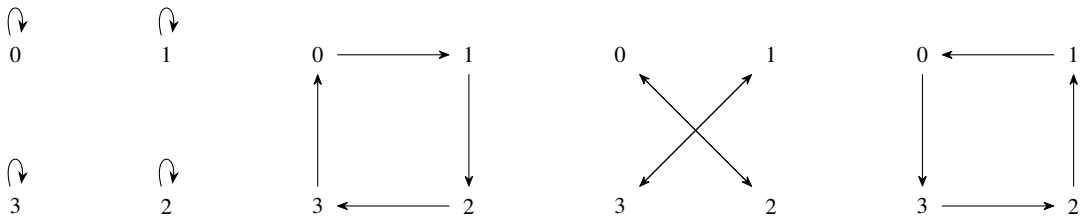


Fig. 5.1: The directed orbital graphs of $G = \langle\{(0\ 1\ 2\ 3)\}\rangle_G$.

Example 5.47 Figure 5.2 shows the directed orbital graphs of the group $G = \langle\{(0\ 1)(2\ 3), (0\ 2)(1\ 3)\}\rangle_G$. All of them except the left-most one are symmetric.

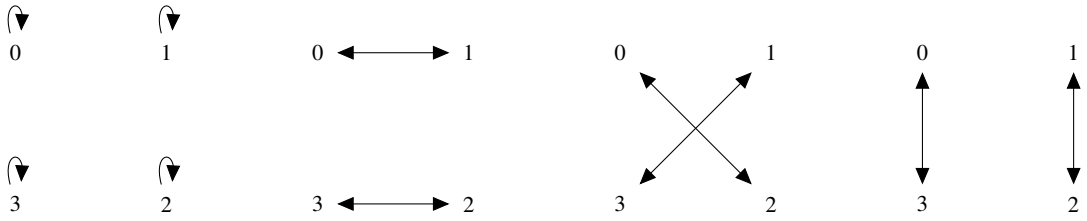


Fig. 5.2: The directed orbital graphs of $G = \langle\{(0\ 1)(2\ 3), (0\ 2)(1\ 3)\}\rangle_G$.

The edge set of the directed orbital graph of (a, b) is contained in $\mathrm{Orb}_G(a) \times \mathrm{Orb}_G(b)$. The directed orbital graph of (a, b) is either symmetric or oriented, and the former case applies precisely if G contains a permutation swapping a and b [249, Lem. 2.4.2]. Also, G is intransitive if and only if in the (directed or undirected) orbital graph exist vertices that are not part of any edge. Furthermore, G is a subgroup of the automorphism group of each (directed or undirected) orbital graph. So, by Theorem 5.43 the weakly connected components of a finite directed orbital graph are precisely the strongly connected components. Finally, a (directed or undirected) graph is vertex-transitive if and only if it is a union of orbital graphs w.r.t. its automorphism group.

Lemma 5.48 *Let $G \leq_G \mathcal{S}_n$ and $a, b \in [n]$. Then $\mathrm{Orb}_G(a) \neq \mathrm{Orb}_G(b)$ if and only if in the undirected orbital graph of $\{a, b\}$ (resp. directed orbital graph of (a, b)) there exists no edge with both endpoints in $\mathrm{Orb}_G(a)$ or both endpoints in $\mathrm{Orb}_G(b)$ (resp. the starting vertex of each edge is in $\mathrm{Orb}_G(a)$ and its ending vertex is in $\mathrm{Orb}_G(b)$).*

Proof. Since $\mathrm{Orb}_G(a)$ and $\mathrm{Orb}_G(b)$ are orbits, we have $\mathrm{Orb}_G(a) = \mathrm{Orb}_G(b)$ if and only if there exists $g \in G$ such that $g(a) \in \mathrm{Orb}_G(b)$ or $g(b) \in \mathrm{Orb}_G(a)$. So, $\{g(a), g(b)\} \subseteq \mathrm{Orb}_G(a)$ or $\{g(a), g(b)\} \subseteq \mathrm{Orb}_G(b)$ for some $g \in G$ if and only if G is transitive, which gives the claim for the undirected orbital graph. Furthermore, since the edge set of the directed orbital graph is in $\mathrm{Orb}_G(a) \times \mathrm{Orb}_G(b)$, if G is intransitive every edge of the directed orbital graph starts in $\mathrm{Orb}_G(a)$ and ends in $\mathrm{Orb}_G(b)$. □

5.6 Discussion

Jordan [346] wrote the first textbook on permutation group theory (but with an antique notation and terminology). Prior to this, permutation groups in the context of polynomial equations (as permutations of their root) were investigated by Lagrange, Galois, Abel and Ruffini, see [370, 466, 641]. At the end of the 19th century, Burnside published his classical textbook [119] (1st edition 1897) which collected and contained most results from 19th century group theory with a focus on permutation groups. Wielandt [638] wrote a standard textbook on permutation group theory which established some standard terminology. Other standard introductions are the textbooks by Dixon & Mortimer [179] and by Cameron [126]. Interestingly, permutation group theory, in its historical development, preceded semigroup theory, see [318].

Before the middle of the 19th century, groups were synonymous to permutation groups on a finite set and were simply thought of as non-empty subsets of permutations closed under function composition (called substitution groups back then). The concept of an abstract group slowly emerged. Cayley [137, 138, 139] was the first who viewed groups more abstract solely in terms of laws, and not in some permutation representation. However, his conceptualization was still far away from our modern view and still relied on implicit assumptions. Weber gave an axiom system for finite groups [629] and for general groups [628]. During the 19th century many further famous mathematician besides the ones already mentioned were involved into forming the abstract theory of groups as, for example, Cauchy, Kronecker, Dyck, Frobenius, and Hölder to name a few. The first definition of an abstract group in its modern form appears in van der Waerden's textbook [625], which is based on lectures by Artin and Noether from the beginning of the 20th century. For more information on the history and genesis of abstract group theory, see [370, 461, 641].

Not all authors use the term G-congruence (some just call them equivalence relations preserved by the group), but it is a notion that appears in the literature, for example in [117]. The generalization of it to transformation semigroups is obvious, but I have not seen in the literature. In [367], in the more general context of semigroup actions, the authors introduce the term S-act congruences, and G-congruences are also used in the more general context of group actions, also called G-sets, for example, in [467].

The books [556, 120, 319] contains information on algorithms for groups and permutation groups. Beaudry [75] investigated the computational complexity of deciding whether all transformations in a given transformation monoid are idempotent.

Vertex-transitive graphs seemed to be introduced by Sabidussi [540], at least this is the first reference that we were able to find where this term was introduced. Vertex-transitive graphs are also called *point-symmetric* in [278]. The study of symmetry in graphs was initiated by Foster [226]. More information on vertex-transitive graphs can be found in [48, 90, 127, 498].

Chapter 6
Notions From Automata Theory

Recall the definition of a (semi-)automaton and the set of synchronizing words from Sections 1.1 and 1.2. Here, we give additional background on automata. Section 6.1 collects more basic notions. The reachability quasiorder and related notions are introduced in Section 6.3. Section 6.2 introduces the transformation semigroup of an automaton and a representation by transition matrices. Section 6.4 investigates morphisms on automata. We introduce endo-, iso- and automorphisms, and collect several properties, facts, and formulas; in particular how morphisms act w.r.t. the reachability order, strongly connected components, and upper sets. Then a sequence of sections follows devoted to various constructions. Specifically, Section 6.5.1, 6.5.2, 6.5.3, 6.5.4, 6.5.5, and 6.5.6 introduce the sub- and restricted, quotient, Rees quotient, product, power, and pair automaton, respectively. Section 6.6 discusses the set of maximal states and shows ways to decompose an automaton into a sequence of upper sets. In Section 6.7 we connect automata to formal languages. We introduce regular languages as languages accepted by automata when adjoined with a start state and a set of final states (which we then call automata as language acceptors). Section 6.7.2 investigates the minimal automaton accepting a given regular language.

6.1 More Basic Notions

Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton. For $u, v \in \Sigma^*$, $S \subseteq Q$ and $q \in Q$, we set $\delta^{-1}(S, u) = \{p \in Q \mid \delta(p, u) \in S\}$ and $\delta^{-1}(q, u) = \delta^{-1}(\{q\}, u)$. We have the following formula (related to Equation (3.5)):

$$\delta^{-1}(S, uv) = \delta^{-1}(\delta^{-1}(S, v), u). \tag{6.1}$$

The *automaton (di-)graph* corresponding to $\mathcal{A}$ is the digraph $G(\mathcal{A}) = (Q, E)$ with $E = \{(s, t) \mid \exists x \in \Sigma : \delta(s, x) = t\}$.

We say $\mathcal{A}$ has a *self-loop* at $q \in Q$ w.r.t. $x \in \Sigma$ (or x induces a self-loop at q) if $\delta(q, x) = q$. The state $q \in Q$ is called a *trap* (or *sink*) *state* if $\delta(q, x) = q$ for all $x \in \Sigma$, i.e., every letter induces a self-loop at q.[1]

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called a *unary* (resp. *binary*, *ternary*) *automaton* if $|\Sigma| = 1$ (resp. $|\Sigma| = 2$, $|\Sigma| = 3$). The automata from the Černý family (Figure 1.7) are binary.

We call $\mathcal{A}$ *primitive* if the trivial $\mathcal{T}_S(\mathcal{A})$-congruences are the only $\mathcal{T}_S(\mathcal{A})$-congruences.

A subset $S \subseteq Q$ is said to be a *block* in $\mathcal{A}$ if it is non-empty and $\delta(S, x) = S$ or $\delta(S, x) \cap S = \emptyset$ for all $x \in \Sigma$. This extends the notion of a block from permutation group theory introduced in Chapter 5 (see 111) to automata. The context should make it clear whether we are referring to a block in a permutation group or in an automaton, so no confusion should arise.

6.2 The Transformation Semigroup of an Automaton

Let $\mathcal{A} = (Q, \Sigma, \delta)$. Each word $u \in \Sigma^*$ *induces a transformation* $\delta_u \colon Q \to Q$ by $\delta_u(q) = \delta(q, u)$ for all $q \in Q$. For $u, v \in \Sigma^*$ we have $\delta_{uv} = \delta_v \delta_u$ and $\delta_{uv}^{-1} = \delta_u^{-1} \delta_v^{-1}$. Compare the latter equation with Equation (6.1) wheres u and v are "swapped" in the expression on the right hand side. This follows as in the δ-notation the left-most word is applied first whereas in the composition of two functions, by our definition, the right-most function is applied first.

We call a letter $x \in \Sigma$ (resp. a word $u \in \Sigma^*$) a *permutational / idempotent / simple idempotent* letter (resp. word) if δ_x (resp. δ_u) is a permutation/idempotent/simple idempotent. The *rank* (resp. *defect*, *excluded set*, *duplicate set*) of a word $u \in \Sigma^*$, denoted by $\operatorname{rank}(u)$ (resp. $\operatorname{def}(u)$, $\operatorname{excl}(u)$, $\operatorname{dupl}(u)$) is the rank (resp. defect, excluded set, duplicate set) of δ_u. Furthermore, when using terminology from functions, transformations, and permutations, as cyclically permuting the states, the image of a set, transition matrix, being a cycle, having a fixed point etc., in the context of a word (or letter) $u \in \Sigma^*$, we refer to the induced transformation δ_u. We also say that δ_u is the *action of* u on the states or that u *acts as* δ_u. The same terminology transfers when speaking about properties of δ_u, for example, we say that u acts as a cycle if δ_u is a cycle.

For $u \in \Sigma^*$, we write $[u]_{\mathcal{A}}$ (or just $[u]$ when there is no ambiguity to which automaton we refer) for the transition matrix of δ_u. Figure 6.1 depicts transition matrices for the Černý automata C_n from Section 1.4 w.r.t. the natural order of its state set $[n]$. When the specific ordering of the states is not important, we assume some order to be taken without explicitly mentioning it.[2]

The *transformation semigroup of* $\mathcal{A}$ is

[1] A sink state is sometimes defined to be a trap state reachable from every other state, but here it is just another name for a trap state.

[2] The maps $u \mapsto \delta_u$ and $u \mapsto [u]$ are morphisms from Σ^* into $\mathcal{T}_Q$ and the set of $|Q| \times |Q|$ matrices, respectively. Thus, by Theorem 3.60, they are uniquely determined by their images on the letters Σ.

$$[b]_{C_n} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ 1 & 0 & 0 & \cdots & 0 \end{pmatrix} \qquad [a]_{C_n} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \\ 0 & 0 & 0 & \cdots & 1 & 0 \\ 0 & 0 & 0 & \cdots & 0 & 1 \end{pmatrix}$$

Fig. 6.1: The transition matrices for the Černý automata C_n from Section 1.4 (Figure 1.7) where the states are ordered w.r.t. the natural order on $[n]$.

$$\mathcal{T}_S(\mathcal{A}) = \{\delta_u \mid u \in \Sigma^+\}$$

and the *transformation monoid* is $\mathcal{T}_M(\mathcal{A}) = \{\delta_u \mid u \in \Sigma^*\}$.

Proposition 6.1 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and δ_u be a permutation for $u \in \Sigma^*$. Then $\delta_u^{-1} \in \mathcal{T}_M(\mathcal{A})$, i.e., the inverse permutation is induced by some word over Σ^*.*

Proof. By finiteness, we have $\delta_{u^i} = \mathrm{id}_Q$ for some $i > 0$. Then $\delta_u^{-1} = \delta_{u^{i-1}} \in \mathcal{T}_M(\mathcal{A})$. □

Next, we determine the group of units in $\mathcal{T}_M(\mathcal{A})$.

Proposition 6.2 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\Gamma = \{x \in \Sigma \mid x \text{ is a permutational letter}\}$. Then for the group of units G of $\mathcal{T}_M(\mathcal{A})$ we have*

$$G = \{\delta_u \mid \delta_u \text{ is a permutation and } u \in \Sigma^*\} = \{\delta_u \mid u \in \Gamma^*\},$$

and if G is non-trivial, equivalently $\Gamma \neq \emptyset$, we also have $G = \{\delta_u \mid u \in \Gamma^+\}$.

Proof. We have $G \subseteq \{\delta_u \mid \delta_u \text{ is a permutation and } u \in \Sigma^*\}$ as every transformation in G, having an inverse, is a permutation. Next, by Theorem 3.12, Item 3.18 the composition of a non-permutation with a transformation gives a non-permutation. So, for $u \in \Sigma^*$, δ_u is a permutation if and only if $u \in \Gamma^*$, i.e.,

$$\{\delta_u \mid \delta_u \text{ is a permutation and } u \in \Sigma^*\} = \{\delta_u \mid u \in \Gamma^*\}.$$

By Theorem 6.1, for $u \in \Gamma^*$ we have $\delta_u^{-1} = \delta_v$ for some $v \in \Sigma^*$, i.e., δ_u has an inverse in $\mathcal{T}_M(\mathcal{A})$ and so $\delta_u \in G$. This gives $\{\delta_u \mid u \in \Gamma^*\} \subseteq G$. Combining everything, we find $G = \{\delta_u \mid \delta_u \text{ is a permutation and } u \in \Sigma^*\} = \{\delta_u \mid u \in \Gamma^*\}$.

Lastly, if $\Gamma \neq \emptyset$, then δ_u equals the identity transformation for some $u \in \Gamma^+$. This gives $\{\delta_u \mid u \in \Gamma^*\} = \{\delta_u \mid u \in \Gamma^+\}$. □

Thus the group of units of $\mathcal{T}_M(\mathcal{A})$ is precisely the set of permutations in $\mathcal{T}_M(\mathcal{A})$ and every permutation group in $\mathcal{T}_S(\mathcal{A})$ is contained in the group of units of $\mathcal{T}_S(\mathcal{A})$ (but not every subset $S \subseteq \mathcal{T}_M(\mathcal{A})$ that forms a group in itself; for example for every non-permutational word u permuting its image $\delta(Q, u)$ the set $\{\delta_{u^i} \mid i \geq 1\}$ forms a group, but contains no permutation).

6.3 The Reachability Order

First, in Section 6.3.1 we introduce the reachability order. Then, Section 6.3.2 investigates connectivity notions and the notion of an upper set derived from the reachability order.

6.3.1 Definition and a Characterization

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $s, t \in Q$. We define the *reachability (quasi)order*

$$s \preceq_R^{\mathcal{A}} t \Leftrightarrow \exists u \in \Sigma^* \colon \delta(s, u) = t.$$

We say that t is *reachable* from s (in $\mathcal{A}$) if $s \preceq_R^{\mathcal{A}} t$. When the automaton we refer to is clear from context, we simply write $\preceq_R$. The reachability order is characterized as the finest quasiorder such that $s \preceq_R \delta(s, u)$ for all $s \in Q$ and $u \in \Sigma^*$, as we formally state next.

Proposition 6.3 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We have $s \preceq_R \delta(s, u)$ for all $s \in Q$ and $u \in \Sigma^*$. Further, if $\lesssim$ is a quasiorder on Q such that $s \lesssim \delta(s, u)$ for all $s \in Q$ and $u \in \Sigma^*$, then $s \preceq_R t$ implies $s \lesssim t$ for all $s, t \in Q$.*

Proof. Clearly, we have $s \preceq_R \delta(s, u)$ for $s \in S$ and $u \in \Sigma^*$. Next, suppose $\lesssim$ is a quasiorder such that $s \lesssim \delta(s, u)$ for all $s \in Q$ and $u \in \Sigma^*$. Now, for $s, t \in Q$, $s \preceq_R t$ is equivalent to $\delta(s, u) = t$ for some $u \in \Sigma^*$. So $s \lesssim \delta(s, u) = t$. □

By $\simeq_R$ we denote the *(reachability) equivalence relation* induced by $\preceq_R$ according to Equation (3.30).

6.3.2 Connectivity and Upper Sets

Let Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$. If nothing else is said, order-theoretical notions in the context of automata are meant w.r.t. the reachability order. For example, by calling S an *upper set*, we mean it to be an upper set w.r.t. the reachability quasiorder. This is equivalent to $\delta(S, x) \subseteq S$ for all $x \in \Sigma$. Note that the empty set $\emptyset$ is an upper set in each automaton. Since then $\delta(S, u) \subseteq \delta(Q, u) \cap S$ for all $u \in \Sigma^*$, we obtain the following equation:

$$S \text{ is an upper set} \Rightarrow \forall u \in \Sigma^* \colon \delta(Q, u) \cap S \neq \emptyset. \tag{6.2}$$

The automaton $\mathcal{A} = (\{q_1, q_2\}, \{a\}, \delta)$ with $\delta(q_1, a) = q_2$ and $\delta(q_2, a) = q_1$ shows, by considering $S = \{q_1\}$, that the condition on the right hand side can hold for sets that are not upper sets. A state $t \in Q$ is a trap state if and only if $\{t\}$ is an upper set. We also have

$$S \text{ is upper set and block} \Leftrightarrow \forall x \in \Sigma \colon \delta(S, x) = S \Leftrightarrow \forall u \in \Sigma^* \colon \delta(S, u) = S. \tag{6.3}$$

A subset $S \subseteq Q$ is called *strongly connected* (resp. *unilaterally connected*) if, for all $s, t \in S$, we have $s \preceq_R t$ (resp. $s \preceq_R t$ or $t \preceq_R s$). A maximal strongly connected subset is called a *strongly connected component* and equals an equivalence class w.r.t. $\simeq_R$. A strongly connected component that is an upper set is called a *sink (strongly connected) component*. Equivalently, it is an upper set that is strongly connected. By Theorem 3.43, a sink component is precisely a minimal non-empty upper set w.r.t. set inclusion. In the automaton graph, the sink components are the strongly connected components without transitions leading out of it.

Proposition 6.4 *In an automaton, the minimal possible rank of a word is bounded from below by the number of distinct sink components.*

Proof. This follows from Equation (6.2) and the fact that distinct sink components are disjoint. □

Proposition 6.5 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then S is a union of sink components if and only if $\delta(q, u) \simeq_R q$ for all $q \in Q$ and $u \in \Sigma^*$.*

Proof. First, suppose $\delta(q, u) \simeq_R q$ for all $q \in Q$ and $u \in \Sigma^*$. Consider some $q \in S$. By assumption $\uparrow\{q\} \subseteq [q]_{\simeq_R}$, and so $[q]_{\simeq_R}$ is a sink component by Theorem 3.43. Since S is an upper set, we get $[q]_{\simeq_R} \subseteq S$ and so S equals the union of all $[q]_{\simeq_R}$ with $q \in S$.

Conversely, suppose S is a union of sink components. Let $q \in S$. Then q is contained in some sink component, which yields that $[q]_{\simeq_R}$ is a sink component and so $\delta(q, u) \simeq_R q$. □

Corollary 6.6 *A set permuted by each letter equals a union of sink components.*

Proof. Let $S \subseteq Q$ be a set in $\mathcal{A} = (Q, \Sigma, \delta)$ such that $\delta(S, a) = S$ for each $a \in \Sigma$. Then S is an upper set. Consider $q \in S$ and $u \in \Sigma^*$. Since the word u permutes S, there exists $i > 0$ such that $\delta(p, u^i) = p$ for all $p \in Q$. Then $\delta(q, uu^{i-1}) = q$ gives $\delta(q, u) \preceq_R q$ and since $q \preceq_R \delta(q, u)$ the claim follows from Theorem 6.5. □

We call $\mathcal{A} = (Q, \Sigma, \delta)$ *strongly connected* (resp. *unilaterally connected*) if Q is strongly connected (resp. unilaterally connected). For $Q \neq \emptyset$, $\mathcal{A}$ is strongly connected if and only if Q is a strongly connected component. We call $\mathcal{A}$ *cyclic* [339] if $Q = \uparrow\{q\}$ for some $q \in Q$.

Proposition 6.7 *An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is strongly connected if and only if there exists no proper non-empty upper set, i.e., $\emptyset$ and Q are the only upper sets.*

Proof. This follows from Theorem 3.42. □

Proposition 6.8 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with a trap state $t \in Q$ and $|Q| \geq 2$. Then $Q \setminus \{t\}$ is a strongly connected component if and only if Q and $\{t\}$ are the only non-empty upper sets in $\mathcal{A}$.*

Proof. This follows from Theorem 3.42 with $S = \{t\}$. □

Proposition 6.9 *An unilaterally connected automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is cyclic.*

Proof. Let $q \in Q$ be such that $\uparrow\{q\}$ has maximal size. Assume $p \notin \uparrow\{q\}$ for some $p \in Q$. Since $\mathcal{A}$ is unilaterally connected, we have $q \in \uparrow\{p\}$ and so $\uparrow\{q\} \subsetneq \uparrow\{p\}$, contradicting the maximal choice of q. □

The converse of Theorem 6.9 does not hold. For example, for $\mathcal{A} = (\{q_1, q_2, q_3\}, \{a, b\}, \delta)$ with $\delta(q_1, a) = q_2$, $\delta(q_1, b) = q_3$, and $\delta(q_i, x) = q_i$ for $i \in \{2, 3\}$ and $x \in \{a, b\}$ we have $\uparrow\{q_1\} = \{q_1, q_2, q_3\}$, but it is not unilaterally connected.

By the results from Section 3.6, a subset of states is a strongly connected component if and only if for each upper set it is contained in this set or its complement (Theorem 3.37), every upper set is a union of strongly connected components (Theorem 3.38), and for the set difference $T \setminus S$ of two upper sets S, T the following are equivalent: strong connectivity, being a strongly connected component, and maximality relative to T and set inclusion (Theorems 3.39 and 3.42).

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is said to be *strongly trap-connected* if there exists a trap state $t \in Q$ such that for all $p \in Q \setminus \{t\}$ and $q \in Q$ we have $p \precsim_R q$. Clearly, a strongly trap-connected automaton has a unique trap-state, is synchronizing, and unilaterally connected.

Proposition 6.10 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $|Q| \geq 2$. Then the following are equivalent.*

1. *$\mathcal{A}$ is strongly trap-connected.*
2. *$Q = S \cup \{t\}$ for a trap state $t \in Q$ and a non-sink strongly connected component S.*
3. *There exists $t \in Q$ such that $\{t\}$ is the unique proper and non-empty upper set.*
4. *$\mathcal{A}$ is not strongly connected and has no proper upper set of size at least two.*

Proof. 1 $\Rightarrow$ 2: Let t be the unique trap state. Suppose R is a non-empty upper set distinct from $\{t\}$. Then there exists $q \in R \setminus \{t\}$. Since $\mathcal{A}$ is strongly trap-connected, we have $\uparrow\{q\} = Q$. As an upper set, it is $\uparrow\{q\} \subseteq R$ and so $R = Q$. Thus Q and $\{t\}$ are the only non-empty upper sets. Since $\{t\}$ is a sink component, Theorem 3.47 gives the claim.

2 $\Rightarrow$ 3: This follows from Theorem 3.47.

3 $\Rightarrow$ 4: Since $\mathcal{A}$ has at least two states and an upper set $\{t\}$ of size one, by Theorem 6.7 it is not strongly connected. By assumption $\{t\}$ is a unique proper and non-empty upper set, so that $\mathcal{A}$ has no proper upper set of size at least two.

4 $\Rightarrow$ 1: Since $\mathcal{A}$ is not strongly connected, by Theorem 6.7 there exist some proper and non-empty upper set $S \subseteq Q$. By assumption, $S = \{t\}$ for some $t \in Q$. Let $q \in Q \setminus \{t\}$. Then $\uparrow\{q\} \cup \{t\}$ is an upper set of size at least two. Hence, we have $\uparrow\{q\} \cup \{t\} = Q$. This gives $|\uparrow\{q\}| \geq 2$. Now, we have $t \in \uparrow\{q\}$ as otherwise, $\uparrow\{q\}$ is a proper upper set of size at least two, and so $\uparrow\{q\} = Q$. □

Corollary 6.11 *In automata with at least three states, there exists no proper upper set of size at least two in $\mathcal{A}$ if and only if $\mathcal{A}$ is strongly connected or strongly trap-connected.*

Proof. By Theorems 6.7 and 6.10, a strongly connected or strongly trap-connected automaton has no proper upper set of size at least two. Conversely, the automaton is either strongly connected or, by Theorem 6.10, Item 4 it has no proper upper set of size at least two. □

A two-state automaton where every letter induces a self-loop also has no proper upper set of size at least two, but is neither strongly connected nor strongly trap-connected.

6.4 Morphisms

Here, we introduce morphisms, endomorphisms, isomorphisms, and automorphisms of automata and collect basic facts about them.

6.4.1 Definitions and Basic Facts

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$ be two semiautomata.

1. A *(semiautomaton) (homo-)morphism* from $\mathcal{A}$ to $\mathcal{B}$ (or between $\mathcal{A}$ and $\mathcal{B}$) is a function $\varphi\colon Q \to S$ such that for each $q \in Q$ and $x \in \Sigma$ we have

   $$\varphi(\delta(q, x)) = \mu(\varphi(q), x).$$

   Equivalently, φ is a morphism if and only if $\varphi\delta_x = \mu_x\varphi$ for all $x \in \Sigma$. Thus, regardless of which of the two paths we follow in the diagram below, we end up at the same state.

   $$\begin{array}{ccc} Q & \xrightarrow{\delta(\cdot,x)} & Q \\ {\scriptstyle\varphi}\downarrow & & \downarrow{\scriptstyle\varphi} \\ S & \xrightarrow[\mu(\cdot,x)]{} & S \end{array}$$

   If φ is surjective, we say that it maps $\mathcal{A}$ *onto* $\mathcal{B}$.
2. A *(semiautomaton) endomorphism* is a morphism with $\mathcal{A} = \mathcal{B}$.
3. A *(semiautomaton) isomorphism* is a morphism $\varphi\colon Q \to S$ such that there exists a morphism $\psi\colon S \to Q$ with $\psi\varphi = \mathrm{id}_Q$ and $\varphi\psi = \mathrm{id}_S$. The automata $\mathcal{A}$ and $\mathcal{B}$ are called *isomorphic*, written

   $$\mathcal{A} \cong \mathcal{B},$$

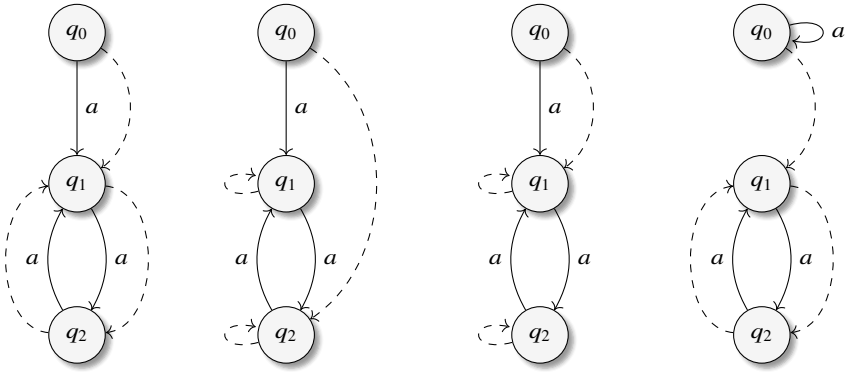
   if there exists an isomorphism between them.
4. A *(semiautomaton) automorphism* is an endomorphism that is an isomorphism.

The set of endomorphisms of a given automaton forms a transformation semigroup and the set of automorphisms forms a permutation group.

Example 6.12 Consider the automata from Figure 6.2, where the dashed lines indicate a mapping of their state sets.

In the automaton from Figure 6.2a the mapping is an endomorphism that does not map the strongly connected component $\{q_0\}$ to a strongly connected component, but to $\{q_1\}$. Another endomorphism of the same automaton is given in Figure 6.2b. Together with the identity transformation, this automaton has exactly three endomorphisms.

In Figures 6.2c and 6.2d, the mapping φ is not a morphism since in both cases we have $\varphi(\delta(q_0, a)) = q_1 \neq q_2 = \delta(\varphi(q_0), a)$.



(a) An endomorphism. (b) An endomorphism. (c) Not a morphism. (d) Not a morphism.

Fig. 6.2: Automata and mappings of their states indicated by dashed lines, see Theorem 6.12.

Example 6.13 Let $\mathcal{A} = (\{q_1, q_2\}, \{a_1, a_2\}, \delta)$ with $\delta(q_1, a_i) = \delta(q_2, a_i) = q_i$ for $i \in \{1, 2\}$, see Figure 6.3. If $\varphi\colon Q \to Q$ is an endomorphism, then $\varphi(q_i) = \varphi(\delta(q_i, a_i)) = \delta(\varphi(q_i), a_i) = q_i$ for $i \in \{1, 2\}$. Thus the identity transformation is the only endomorphism of $\mathcal{A}$.

Later, in Theorem 6.45, we will show that, in general, a strongly connected and synchronizing automaton only admits the identity transformation as an endomorphism.

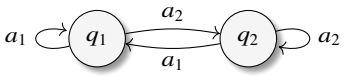


Fig. 6.3: Automaton not admitting any non-identity endomorphism, see Theorem 6.13.

Next, we give some alternative characterizations for morphisms.

Proposition 6.14 *Let $\varphi\colon Q \to S$ be a function. Then the following are equivalent.*

1. *φ is a morphism.*
2. *For all $p \in Q$ and all $u \in \Sigma^*$ we have*

$$\varphi(\delta(p, u)) = \mu(\varphi(p), u)$$

3. *There exists $T \subseteq Q$ with $Q = \bigcup_{p \in T} \uparrow\{p\}$ such that for all $p \in T$ and all $u \in \Sigma^*$ we have*

$$\varphi(\delta(p, u)) = \mu(\varphi(p), u)$$

Proof. $1 \Rightarrow 2$: This clearly holds for $u = \varepsilon$. Next, let $u = va$ for some $v \in \Sigma^*$ and $a \in \Sigma$. Inductively, we have $\varphi(\delta(p, v)) = \mu(\varphi(p), v)$. Using that φ is a morphism, this gives

$\varphi(\delta(p,u)) = \varphi(\delta(\delta(p,v),a)) = \mu(\varphi(\delta(p,v)),a) = \mu(\mu(\varphi(p),v),a) = \mu(\varphi(p),u)$. Then the claim follows by induction.

$2 \Rightarrow 3$: By assumption, this holds for $T = Q$.

$3 \Rightarrow 1$: Let $a \in \Sigma$ and $q \in Q$. By assumption, there exists $u \in \Sigma^*$ such that $q = \delta(p,u)$ for some $p \in T$. Set $s = \varphi(p)$. Then we have

$$\begin{aligned}
\varphi(\delta(q,a)) &= \varphi(\delta(p,ua)) && (q = \delta(p,u)) \\
&= \mu(s,ua) && (\varphi \text{ morphism}) \\
&= \mu(\mu(s,u),a) && (\text{Def. transition function}) \\
&= \mu(\varphi(\delta(p,u)),a) && (\varphi(\delta(p,u)) = \mu(s,u)) \\
&= \mu(\varphi(q),a). && (q = \delta(p,u))
\end{aligned}$$

Thus, φ is a morphism. □

By Item 3 of Theorem 6.14, a morphism is completely specified by the images of $\varphi(p_1), \ldots, \varphi(p_m)$ for some $p_1, \ldots, p_m \in Q$ such that $Q = \uparrow\{p_1\} \cup \cdots \cup \uparrow\{p_m\}$.

Corollary 6.15 *If $\mathcal{A}$ is cyclic, there are at most $|S|$ morphisms from $\mathcal{A}$ into $\mathcal{B}$. In particular, there are at most $|Q|$ endomorphisms of $\mathcal{A}$.*

Proof. By Theorem 6.24 two morphisms having the same image for a state p with $Q = \uparrow\{p\}$ are equal. Since we have at most $|S|$ distinct images for this state, the claim follows. □

Example 6.16 The might be strictly less than $|Q|$ endomorphisms in a cyclic automaton. For example, the automaton from Theorem 6.13 is cyclic and admits no non-identity morphism.

In the rest of this section, let $\varphi\colon Q \to S$ be a morphism. For $T \subseteq Q$, we call $\varphi(T)$ the *morphic image (of T)*.

By our next statement, isomorphisms are precisely the bijective morphisms. Note that a similar statement does not hold for order morphisms, see Theorem 3.32.

Proposition 6.17 *The morphism $\varphi\colon Q \to S$ is an isomorphism if and only if it is bijective.*

Proof. By Theorem 3.3, existence of a map $\psi\colon S \to Q$ such that $\psi\varphi = \mathrm{id}_Q$ and $\varphi\psi = \mathrm{id}_S$ is equivalent to bijectivity of φ. So, an isomorphism is bijective.

Next, assume φ is bijective. Since φ is surjective, we have $\varphi\psi = \mathrm{id}_S$ for some $\psi\colon S \to Q$ by Theorem 3.3. Let $s \in S$ and $x \in \Sigma$. Then we find

$$\begin{aligned}
\varphi(\delta(\psi(s),x)) &= \mu(\varphi(\psi(s)),x) && (\varphi \text{ morphism}) \\
&= \mu(s,x) && (\varphi\psi = \mathrm{id}_S) \\
&= \varphi(\psi(\mu(s,x))). && (\varphi\psi = \mathrm{id}_S)
\end{aligned}$$

Since φ is injective, $\varphi(\delta(\psi(s),x)) = \varphi(\psi(\mu(s,x)))$ implies $\delta(\psi(s),x) = \psi(\mu(s,x))$. Thus ψ is a morphism. □

Remark 6.18 By Theorem 3.3 a morphism $\varphi\colon Q \to S$ is injective (resp. surjective) if there exists a morphim $\psi\colon S \to Q$ such that $\psi\varphi$ (resp. $\varphi\psi$). However, unlike for functions, the converse implication does not hold in general.

To show this, for $i \in \{1,2,3\}$, let $\mathcal{A}_i = (Q_i, \{a\}, \delta_i)$ where $Q_1 = \{q_1, q_2\}$, $Q_2 = \{q_1, q_2, t\}$, $Q_3 = \{t\}$, and where the letter a swaps the states q_1 and q_2 in $\mathcal{A}_1$ and $\mathcal{A}_2$ and fixes the state t in $\mathcal{A}_2$ and $\mathcal{A}_3$. Define $\varphi\colon Q_1 \to Q_2$ by $\varphi(q_1) = q_1$, $\varphi(q_2) = q_2$, and $\psi\colon Q_1 \to Q_3$ by $\psi(q_1) = \psi(q_2) = t$. Then φ is injective and ψ is surjective and both maps are morphisms. However, there exists no morphism from $\mathcal{A}_2$ to $\mathcal{A}_1$, and no morphism from $\mathcal{A}_3$ to $\mathcal{A}_1$.

Proposition 6.19 *If $t \in Q$ is a trap state, then $\varphi(t)$ is a trap state.*

Proof. Let $x \in \Sigma$. Then $\mu(\varphi(t), x) = \varphi(\delta(t, x)) = \varphi(t)$ and so $\varphi(t)$ is a trap state. □

Proposition 6.20 *If φ is surjective, the image of a synchronizing state is a synchronizing state.*

Proof. Let $\delta(Q, u) = \{q\}$ for $u \in \Sigma^*$ and $q \in Q$. Then $\mu(S, u) = \mu(\varphi(Q), u) = \varphi(\delta(Q, u)) = \{\varphi(q)\}$ □

Next, a morphism of automata induces a morphism of the underlying digraphs.

Proposition 6.21 *$\varphi\colon Q \to S$ is also a graph morphism of the automaton digraphs of $\mathcal{A}$ and $\mathcal{B}$.*

Proof. For $p \in Q$ and $x \in \Sigma$, the edge $(p, \delta(p, x))$ is mapped to the edge $(\varphi(p), \mu(\varphi(p), x))$ by φ in the respective automaton digraphs, which gives the claim. □

The following two examples gives mappings that are graph morphisms of the automaton digraphs, but not automaton morphisms.

Example 6.22 Let $\mathcal{A} = (\{p, q\}, \{a, b\}, \delta)$ with $\delta(p, x) = q$ for $x \in \{a, b\}$, $\delta(q, a) = p$, and $\delta(q, b) = q$. Then the transformation φ that maps all states to q is a graph morphism of the automaton digraph of $\mathcal{A}$, but it is not an endomorphism since $\delta(\varphi(q), a) = p$ and $\varphi(\delta(q, a)) = q$.

Example 6.23 Let $\mathcal{A} = (\{q_1, \ldots, q_n\}, \Sigma, \delta)$ with $n \geq 2$ be such that for each pair of distinct states q_i, q_j there exists a letter a_{ij} inducing a simple idempotent mapping q_i to q_j. Every permutation of $\{q_1, \ldots, q_n\}$ is a graph automorphism of the automaton digraph of $\mathcal{A}$. However, $\mathcal{A}$ has no non-identity automaton morphism since if $\varphi(q_i) = q_j$ with $i \neq j$, then $\varphi(\delta(q_i, a_{ji})) = q_j$, but $\delta(\varphi(q_i), a_{ji}) = q_i$.

6.4.2 Morphisms and the Reachability Order

In this section, let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$ be two automata.

First, we show that two morphisms agree on an upper set of the form $\uparrow\{p\}$ if and only if they agree at p.

Proposition 6.24 *For morphisms $\varphi, \psi : Q \to S$ and $p \in Q$, we have*

$$\varphi(p) = \psi(p) \Rightarrow \forall q \in \uparrow\{p\} : \varphi(q) = \psi(q) \tag{6.4}$$

In particular, two morphism φ, ψ agree on a strongly connected component or have different images for each state of the strongly connected component.

Proof. Let $q \in \uparrow\{p\}$. Then we have $q = \delta(p, u)$ for some $u \in \Sigma^*$. By Theorem 6.14 we find $\varphi(\delta(p, u)) = \mu(\varphi(p), u) = \mu(\psi(p), u) = \psi(\delta(p, u))$. □

For the remainder of this section, suppose $\varphi : Q \to S$ is a morphism. Let $p, q \in Q$. Then φ is an order morphism with respect to $\preceq_R$. Indeed, if $p \preceq_R q$, then $q = \delta(p, u)$ for some $u \in \Sigma^*$. Consequently, $\varphi(q) = \varphi(\delta(p, u)) = \mu(\varphi(p), u)$, so $\varphi(p) \preceq_R \varphi(q)$. Thus, we have shown that[3]

$$p \preceq_R q \Rightarrow \varphi(p) \preceq_R \varphi(q). \tag{6.5}$$

Proposition 6.25 *Let $\varphi : Q \to S$ be a morphism.*

1. *Let $T \subseteq Q$. Then we have*
$$\varphi(\uparrow T) = \uparrow\varphi(T) \tag{6.6}$$
2. *$\varphi(T)$ is an upper set (resp. unilaterally connected, strongly connected, a sink component) for an upper set (resp. unilaterally connected, strongly connected, sink component) $T \subseteq Q$.*
3. *For two strongly connected components $T \subseteq Q$ and $R \subseteq S$, we have $\varphi(T) \cap R = \emptyset$ or $\varphi(T) \subseteq R$.*
4. *Let $T \subseteq Q$ be an upper set. Then $\varphi^{-1}(T)$ is also an upper set.*

Proof. 1: We have

$$\begin{aligned}\varphi(\uparrow T) &= \varphi(\{\delta(p, u) \mid p \in T \wedge u \in \Sigma^*\}) \\ &= \{\varphi(\delta(p, u)) \mid p \in T \wedge u \in \Sigma^*\} \\ &= \{\mu(\varphi(p), u) \mid p \in T \wedge u \in \Sigma^*\} \\ &= \uparrow\varphi(T).\end{aligned}$$

2: Let $T \subseteq Q$ be an upper set. Then $T = \uparrow T$ and, with Item 1, we find $\varphi(T) = \varphi(\uparrow T) = \uparrow\varphi(T)$. Since $\uparrow\varphi(T) \subseteq \varphi(T)$, $\varphi(T)$ is an upper set.

For unilaterally connected T, consider $\varphi(p), \varphi(q)$ for $p, q \in T$. Since $p \preceq_R q$ or $q \preceq_R p$, we have $\varphi(p) \preceq_R \varphi(q)$ or $\varphi(q) \preceq_R \varphi(p)$ and so $\varphi(T)$ is unilaterally connected.

By Theorem 3.29, $\varphi(T)$ is strongly connected if T is strongly connected.

Lastly, suppose T is a sink component. Then $\varphi(T)$ is strongly connected and an upper set since both properties are preserved as shown above.

3: This follows from Theorem 3.31. □

For general order morphisms, Equation (6.6) may fail and the images of upper sets need not be upper sets, see Theorem 3.33.

[3] This also follows from Theorem 6.21 since a morphism maps paths to paths in the automaton digraphs.

Corollary 6.26 *Let $T \subseteq Q$. If $\varphi(T) \subseteq R$ for a sink component R, then $\varphi(\uparrow T) = R$.*

Proof. Since $\varphi(T) \subseteq R$ we have $\uparrow\varphi(T) \subseteq \uparrow R = R$. Since R is strongly connected and $\uparrow\varphi(T) \cap R \neq \emptyset$, we have $R \subseteq \uparrow\varphi(T)$. Thus, with Theorem 6.25, we find $\varphi(\uparrow T) = \uparrow\varphi(T) = R$. □

Remark 6.27 Consider the automata in Figures 6.2a and 6.2b from Theorem 6.12. The image of $\{q_0\}$ is $\{q_1\}$, which is strictly contained in the sink component $\{q_1, q_2\}$, and the image of $\uparrow\{q_0\}$ equals $\{q_1, q_2\}$. Note that $\{q_0\}$ is a strongly connected component, but its image is not a strongly connected component in both automata.

In Figure 6.2d, $\{q_0\}$ is a sink component and its image is not a sink component. Thus, in this case, we also obtain that the mapping is not a morphism from Theorem 6.26.

In general, bijective order morphisms do not have to be order isomorphisms, see Theorem 3.32. By the next statement, with Theorem 6.17, bijective order morphisms arising from automata morphisms are in fact order isomorphisms.

Proposition 6.28 *If $\varphi\colon Q \to S$ is an isomorphism, then it is also an order isomorphism.*

Proof. By assumption, there exists a morphism $\psi\colon S \to Q$ such that $\psi\varphi = \mathrm{id}_Q$ and $\varphi\psi = \mathrm{id}_S$. By Equation (6.5), ψ is also an order morphism. Thus, φ is an order isomorphism. □

Corollary 6.29 *Suppose $\varphi\colon Q \to S$ is an isomorphism. If $T \subseteq Q$ is a strongly connected component, then $\varphi(T)$ is a strongly connected component.*

Proof. This follows from Theorems 3.29 and 6.28. □

Proposition 6.30
1. *If $Q \neq \emptyset$ and $\mathcal{B}$ is strongly connected, or strongly trap-connected where $\varphi(Q)$ has size at least two, then φ is surjective, i.e., $\varphi(Q) = S$.*
2. *If $\mathcal{A}$ is strongly connected (resp. strongly trap-connected) and φ is surjective, then $\mathcal{B}$ is strongly connected (resp. strongly trap-connected).*

Proof. 1: By Item 2 of Theorem 6.25, $\varphi(Q)$ is an upper set. First, suppose $\mathcal{B}$ is strongly connected. By Theorem 6.7, a strongly connected automaton has no proper non-empty upper set. Thus $\varphi(Q) = S$. Next, suppose $\mathcal{B}$ is strongly trap-connected and $|\varphi(Q)| \geq 2$. By Theorem 6.10, $\mathcal{B}$ has no proper upper set of size at least two. Thus $\varphi(Q) = S$.

2: We can assume $|Q|, |\varphi(Q)| \geq 2$.

If $\mathcal{A}$ is strongly connected, then Q is a sink component. Thus, by Item 2 from Theorem 6.25, $\varphi(Q)$ is also a sink component. By surjectivity, $\mathcal{B}$ is strongly connected.

Next, suppose $\mathcal{A}$ is strongly trap-connected. Let $T \subseteq S$ be an upper set in $\mathcal{B}$ of size at least two. By Equation (3.36), $\varphi^{-1}(T)$ is an upper set. Since φ is surjective, $|\varphi^{-1}(T)| \geq 2$. By Theorem 6.10 we find $Q = \varphi^{-1}(T)$. Thus $S = \varphi(Q) \subseteq T$. Since the trap state in $\mathcal{A}$ is mapped to a trap state in $\mathcal{B}$, $\mathcal{B}$ is not strongly connected. Then, Theorem 6.10 gives that $\mathcal{B}$ is strongly trap-connected. □

Corollary 6.31 *If $|Q| = |S|$ and $\mathcal{B}$ is strongly connected, or strongly trap-connected with $|\varphi(Q)| \geq 2$, then φ is an isomorphism (and so $\mathcal{A}$ is also strongly connected or strongly trap-connected).*

Proof. By Item 1 from Theorem 6.30, we have $\varphi(Q) = S$. Since $\operatorname{rank}(\varphi) = |Q|$, by Equation (3.15) from Theorem 3.12 we find $\operatorname{def}(\varphi) = \emptyset$. Thus, φ is bijective. By Theorem 6.17, φ is an isomorphism.

The inverse map of φ is an isomorphism. Since $\mathcal{B}$ is strongly connected or strongly trap-connected, by Item 2 from Theorem 6.30, we find that $\mathcal{A}$ is strongly connected or strongly trap-connected. □

6.4.3 Endomorphisms

By the first result of this section, Theorem 6.32, an endomorphism cannot map a state to one that strictly precedes it in the reachability order. This property does not hold for general quasiorders, see Theorem 3.33. We derive several consequences of this fact. For example, Theorem 6.34, showing that an endomorphism permutes a strongly connected component T if and only if it permutes the upper set generated by T, or Theorem 6.35 showing that an endomorphism either permutes a strongly connected component or maps all of its states to states outside the component. Lastly, we give several results on endomorphisms of strongly connected and synchronizing automata.

In this section, let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton and $\varphi\colon Q \to Q$ be an endomorphism.

Proposition 6.32 *Let $p \in Q$. Then $\varphi(p) \preceq_R p$ implies $p \preceq_R \varphi(p)$.*

Proof. Suppose $\varphi(p) \preceq_R p$. By Equation (3.32) we have $\uparrow\{p\} \subseteq \uparrow\{\varphi(p)\}$. By Equation (6.6) we have $\uparrow\{\varphi(p)\} = \varphi(\uparrow\{p\})$. Thus $\uparrow\{p\} \subseteq \varphi(\uparrow\{p\})$. By finiteness, this gives $\uparrow\{p\} = \varphi(\uparrow\{p\}) = \uparrow\{\varphi(p)\}$. Equation (3.33) we find $p \simeq_R \varphi(p)$. □

Corollary 6.33 *Either p and $\varphi(p)$ are incomparable or we have $p \preceq_R \varphi(p)$*

Corollary 6.34 *For a strongly connected component $T \subseteq Q$, the following are equivalent:*

1. $T \subseteq \varphi(\uparrow T)$,
2. $\varphi(T) = T$,
3. $\varphi(T) \cap T \neq \emptyset$,
4. $\varphi(\uparrow T) = \uparrow T$.

Proof. 1 ⇒ 2: Let $p \in T$. By assumption, we have $p = \varphi(q)$ for some $q \in \uparrow T$. Since T is a strongly connected component, we have $\uparrow T = \uparrow\{p\}$. Thus, there exists $u \in \Sigma^*$ such that $q = \delta(p, u)$. This gives $p = \varphi(\delta(p, u))$. By Theorem 6.3 we have $p \preceq_R \delta(p, u)$. Hence, $\varphi(\delta(p, u)) \preceq_R \delta(p, u)$. By Theorem 6.32 this gives $\varphi(\delta(p, u)) \simeq_R \delta(p, u)$. Thus $p \simeq_R \delta(p, u)$ and so $q \in T$. Hence $p \in \varphi(T)$. This gives $T \subseteq \varphi(T)$. Since $|\varphi(T)| \leq |T|$ by Equation (3.17), we find $T = \varphi(T)$.

2 ⇒ 3: This is clear.

3 ⇒ 4: Let $p \in \varphi(T) \cap T$. By Theorem 3.31 we have $\varphi(T) \subseteq T$. Thus $\varphi(p) \in T$. Since T is a strongly connected component, we have $\uparrow T = \uparrow\{p\} = \uparrow\{\varphi(p)\}$. So

$$\varphi(\uparrow T) = \varphi(\uparrow\{p\}) = \uparrow\{\varphi(p)\} = \uparrow T.$$

This gives the claim.[4]

4 $\Rightarrow$ 1: This is clear. □

Corollary 6.35 *For a strongly connected component $T \subseteq Q$, we have $\varphi(T) \cap T = \emptyset$ or $\varphi(T) = T$.*

Proof. This directly follows from Items 2 and 3 of the previous corollary. □

Remark 6.36 In general, for a sink component $T \subseteq Q$, a morphism φ need not induce an isomorphism between $\mathcal{A}_{|T}$ and $\mathcal{A}_{|\varphi(T)}$. For example, this fails when $|\varphi(T)| = 1$ and $|T| > 1$. However, by Theorem 6.35, if $\varphi(T) \cap T \neq \emptyset$, then $\varphi(T) = T$. Consequently, the restriction $\varphi_{|T}$ is an automorphism of $\mathcal{A}_{|T}$.

Corollary 6.37 *An endomorphism of a strongly connected finite automaton is an automorphism.*

Proof. By Theorem 6.35, it follows that every endomorphism is bijective, and so an automorphism by Theorem 6.17.

Alternatively, this also follows from Item 1 of Theorem 6.30. □

There exist automata that are not strongly connected and whose non-identity endomorphisms are all automorphisms. For example, a unary automaton in which the single letter induces two cycles of coprime lengths.

Corollary 6.38 *Let $S \subseteq Q$ be strongly connected. Then $\varphi(S) \subseteq S$ implies $\varphi(S) = S$.*

Proof. We have $S \subseteq T$ for some strongly connected component $T \subseteq Q$. Let $p \in S$. By assumption, we have $\varphi(p) \in T$. Thus $\varphi(p) \in T \cap \varphi(T)$. By Theorem 6.35 we find $\varphi(T) = T$. Hence φ permutes the states in T and so $\varphi_{|T}$ is injective. Then $\varphi(S) \subseteq S$ gives $\varphi(S) = S$. □

Corollary 6.39 *Let $\psi\colon Q \to Q$ be another endomorphism and $S \subseteq Q$ be strongly connected. If $\psi(\varphi(S)) \subseteq \varphi(S)$, then $\psi(\varphi(S)) = \varphi(S)$.*

Proof. By Item 3 of Theorem 6.25 the set $\varphi(S)$ is strongly connected. Then the claim follows from Theorem 6.38. □

Corollary 6.40 *For $\varphi(S) \subseteq S$ and $\delta(S, u) \subseteq T$ with strongly connected components $S, T \subseteq Q$ and $u \in \Sigma^*$ we have $\varphi(T) = T$.*

Proof. Let $q \in S$. Then $\varphi(q) \in S$. Since $\delta(S, u) \subseteq T$, we find $\delta(q, u) \in T$ and $\varphi(\delta(q, u)) = \delta(\varphi(q), u) \in T$. Thus $\varphi(\delta(q, u)) \in \varphi(T) \cap T$. By Theorem 6.34, this gives $\varphi(T) = T$. □

[4] Alternatively, since $\varphi(\uparrow T)$ is an upper set having non-empty intersection with T, we have $T \subseteq \varphi(\uparrow T)$. As $\uparrow T$ is the smallest upper set containing T, this gives $\uparrow T \subseteq \varphi(\uparrow T)$. By finiteness, we then obtain $\uparrow T = \varphi(\uparrow T)$.

By Theorem 6.34, for $\varphi(\uparrow\{p\}) = \uparrow\{p\}$, φ permutes the strongly connected component $[p]_{\simeq_R}$. However, this does not imply that φ permutes each strongly connected component in $\uparrow\{p\}$, as Theorem 6.41 shows next.

Example 6.41 Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ with $Q = \{q_1, q_2, q_3, q_4\}$ and

$$\delta(q_i, a) = \begin{cases} q_4 & \text{if } i = 1, \\ q_3 & \text{if } i = 2, \\ q_i & \text{otherwise,} \end{cases} \qquad \delta(q_i, b) = \begin{cases} q_1 & \text{if } i = 2, \\ q_2 & \text{if } i = 1, \\ q_i & \text{otherwise.} \end{cases}$$

The strongly connected components are $\{q_1, q_2\}$, $\{q_3\}$ and $\{q_4\}$. We have $\uparrow\{q_1\} = Q$. The map φ given by $\varphi(q_1) = q_2, \varphi(q_2) = q_1, \varphi(q_3) = q_4$ and $\varphi(q_4) = q_3$ is a morphism that maps $\{q_1, q_2\}$ to itself and interchanges $\{q_3\}$ and $\{q_4\}$.

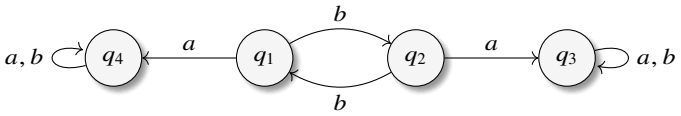


Fig. 6.4: Automaton admitting a morphism that leaves $\{q_1, q_2\}$ invariant and swaps $\{q_4\}$ and $\{q_3\}$, see Theorem 6.41.

Proposition 6.42 *For a strongly connected component $T \subseteq Q$, either we have $\varphi(p) = p$ for all $p \in T$ or $\varphi(p) \neq p$ for all $p \in T$.*

Proof. Suppose we have $\varphi(p) = p$ for some $p \in T$. Since $T \subseteq \uparrow\{p\}$, from Theorem 6.24 applied to φ and the identity transformation id_Q we get $\varphi(q) = q$ for all $q \in \uparrow\{p\}$, in particular φ fixes all states of T. □

Corollary 6.43 *The automorphism group of a strongly connected automaton is semiregular (and so has size at most $|Q|$).*

Proof. By Theorem 6.42, every non-identity isomorphism fixes no state. Thus, the automorphism group is semiregular. By Theorem 5.6, its size is at most $|Q|$. □

Proposition 6.44 *$\varphi\colon Q \to Q$ maps a synchronizing state to itself.*

Proof. Let $u \in \Sigma^*$ be such that $\delta(Q, u) = \{q\}$. Then we have $\varphi(q) = \varphi(\delta(q, u)) = \delta(\varphi(q), u) = q$. □

Corollary 6.45 *The only endomorphism of a strongly connected synchronizing automaton is the identity transformation.*

Proof. By Theorem 6.44 an endomorphism fixes the synchronizing state. Thus, by Theorem 6.42, it equals the identity transformation.[5] □

[5] Alternatively, since in this case every state is a synchronizing state as we show later in Theorem 7.14, from Theorem 6.44 we also get that the endomorphism fixes every state.

Corollary 6.46 *For a strongly connected synchronizing automaton having the same number of states as $\mathcal{A}$, there exists at most one morphism from $\mathcal{A}$ to that automaton.*

Proof. Let $\mathcal{B} = (S, \Sigma, \mu)$ be strongly connected and synchronizing with $|Q| = |S|$. Suppose $\psi, \xi \colon Q \to S$ are two morphisms. Since $\mathcal{B}$ is strongly connected, by Theorem 6.31, they are both isomorphisms. Let $\psi^{-1} \colon S \to Q$ be the inverse morphism of ψ. Then $\xi\psi^{-1}$ is an endomorphism of $\mathcal{B}$. By Theorem 6.45 it equals the identity transformation, which implies $\psi = \xi$. □

6.4.4 Cyclic Unary Automata

As a simple example, here we classify all morphisms between cyclic unary automata. First, Theorems 6.47 and 6.50 are preliminary statements, and Theorem 6.51 gives the classification.

Lemma 6.47 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ and $\mathcal{B} = (S, \{a\}, \mu)$ where $Q = \uparrow\{p\}$ for some $p \in Q$. Suppose $\varphi \colon Q \to S$ is a morphism. Let $i_2 \geq 0$ be the least number such that, with a corresponding least number $p_2 > 0$, we have $\mu(\delta(p), a^{i_2+p_2}) = \mu(\delta(p), a^{i_2})$. Then for all $i_1 \geq 0$ and $p_1 > 0$ with $\delta(p, a^{i_1+p_1}) = \delta(p, a^{i_1})$ we have $i_2 \leq i_1$ and that p_1 is a multiple of p_2.*

Proof. First, $\delta(p, a^{i_1+p_1}) = \delta(p, a^{i_1})$ implies $\mu(\varphi(p), a^{i_1+p_1}) = \varphi(\delta(p, a^{i_1+p_1})) = \varphi(\delta(p, a^{i_1})) = \mu(\varphi(p), a^{i_1})$. Hence, by the minimal choice of i_2 and p_2, we have $i_2 \leq i_1$ and $p_2 \leq p_1$. Next, write $p_1 = lp_2 + r$ for some $l \geq 0$ and $0 \leq r < p_2$. Then we have $\mu(\delta(p), a^{i_2+p_1}) = \mu(\delta(p), a^{i_2+r})$ and, by minimality, we must have $r = 0$, i.e., p_1 is a multiple of p_2. □

Example 6.48 Let $\mathcal{A} = (Q, \{a\}, \delta)$ consist of two disjoint cycles for a of lengths p and q for two distinct prime numbers. Then endomorphisms mapping some state from one cycle to the other cycle are not possible by Theorem 6.47. With Theorem 6.52, each endomorphism shifts each state on the cycle of length p and on the cycle of length q by some amount $0 \leq i < p$ and $0 \leq j < q$, respectively. So, $\mathcal{A}$ has pq endomorphisms.

Corollary 6.49 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ where $Q = \uparrow\{p\}$ for some $p \in Q$ and $i_1 \geq 0$ be the least number with a corresponding least number $p_1 > 0$ such that $\delta(p, a^{i_1+p_1}) = \delta(p, a^{i_1})$. Then $\delta(p, a^{i_2+p_2}) = \delta(p, a^{i_2})$ implies $i_1 \leq i_2$ and that p_2 is a multiple of p_1.*

Proof. Apply Theorem 6.47 with the identity transformation, which is a morphism. □

Proposition 6.50 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ with $Q = \uparrow\{q\}$ for some $q \in Q$ and $\mathcal{B} = (S, \{a\}, \mu)$. Then $\varphi \colon Q \to S$ is a morphism if and only if $\mu(\varphi(q), a^{i_2+p_2}) = \mu(\varphi(q), a^{i_2})$ for some $0 \leq i_2 \leq i_1$ with p_1 being a multiple of p_2 and $\varphi(\delta(q, a^i)) = \mu(\varphi(q), a^i)$ for $i \in \{0, 1, \ldots, n-1\}$ with $n = |Q|$.*

Proof. First, let φ be a morphism. By finiteness, we have $\mu(\varphi(q), a^{i_2+p_2}) = \mu(\varphi(q), a^{i_2})$ for some minimal $i_2 \geq 0$ and some $p_2 > 0$. By Theorem 6.47, we find $i_2 \leq i_1$ and that

p_1 is a multiple of p_2. Since φ is a morphism, we have $\varphi(\delta(q, a^i)) = \mu(\varphi(q), a^i)$ for all $i \geq 0$.

Next, suppose we have $\mu(\varphi(q), a^{i_2+p_2}) = \mu(\varphi(q), a^{i_2})$ for some $0 \leq i_2 \leq i_1$ with p_1 being a multiple of p_2 and $\varphi(\delta(q, a^i)) = \mu(\varphi(q), a^i)$ for $i \in \{0, 1, \ldots, n-1\}$. We have $n = i_1 + p_1$. By assumption, there exists $k \geq 1$ such that $p_1 = k \cdot p_2$. Since $i_2 \leq i_1$ we have $\mu(\varphi(q), a^{i_1}) = \mu(\varphi(q), a^{i_1+k\cdot p_2}) = \mu(\varphi(q), a^n)$. Then we find

$$\varphi(\delta(q, a^{n-1}), a) = \varphi(\delta(q, a^{i_1})) = \mu(\varphi(q), a^{i_1}) = \mu(\varphi(q), a^n) = \mu(\delta(q, a^{n-1}), a).$$

Since $Q = \{q, \delta(q, a), \ldots, \delta(q, a^{n-1})\}$, we have $\varphi(\delta(p, a)) = \delta(\varphi(p), a)$ for all $p \in Q$. □

Corollary 6.51 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ and $\mathcal{B} = (S, \{a\}, \mu)$ where $Q = \uparrow\{p\}$ and $S = \uparrow\{s\}$ for some $q \in Q$ and $s \in S$. Let $i_1, i_2 \geq 0$ be the least numbers with corresponding least numbers $p_1, p_2 > 0$ such that $\delta(q, a^{i_1+p_1}) = \delta(q, a^{i_1})$ and $\mu(s, a^{i_2+p_2}) = \mu(s, a^{i_2})$. Then there exist $\min\{i_1, i_2\} + p_2$ morphisms from $\mathcal{A}$ to $\mathcal{B}$ if and only if p_1 is a multiple of p_2, and there exists no morphism otherwise.*

Proof. If p_1 is not a multiple of p_2, there exists no morphism by Theorem 6.51. Next, suppose p_1 is a multiple of p_2. Then, for $i_2 - \min\{i_1, i_2\} \leq i < |S|$, setting $\varphi(\delta(q, a^j)) = \mu(s, a^{i+j})$, which gives $\varphi(\delta(q, a^j)) = \mu(\varphi(q), a^j)$, for $j \in \{0, 1, \ldots, |Q| - 1\}$ yields a morphism by Theorem 6.51 where $\mu(\varphi(q), a^{\min\{i_2-i,0\}+p_2}) = \mu(\varphi(q), a^{\min\{i_2-i,0\}})$ and $\min\{i_2 - i, 0\} \leq \min\{i_1, i_2\} \leq i_1$.

Next, let $\varphi\colon Q \to S$ be a morphism and $0 \leq i < |S|$ such that $\varphi(q) = \mu(s, a^i)$. We have $\mu(s, a^{i+j}) = \mu(s, a^{i+j+p_2})$ for $j = \min\{i_2 - i, 0\}$. By Theorem 6.51 we find $j \leq i_1$. Thus, for $i_2 > i_1$ we must have $i_2 - i_1 \leq i$ since otherwise $i + j < i_2$. This gives $i_2 - \min\{i_1, i_2\} \leq i < |S|$.

So, each morphism is uniquely given by its image $\varphi(q) = \mu(s, a^i)$ for $i_2 - \min\{i_1, i_2\} \leq i < |S|$. Thus for $i_2 < i_1$ we have $|S| = i_2 + p_2 = \min\{i_1, i_2\} + p_2$ possible morphisms. For $i_2 \geq i_1$ we have $|\{i \mid i_2 - i_1 \leq i < i_2 + p_2\}| = i_1 + p_2 = \min\{i_1, i_2\} + p_2$ possible morphisms. □

Corollary 6.52 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ with $Q = \uparrow\{q\}$ for some $q \in Q$. Then $\varphi\colon Q \to Q$ is an endomorphism if and only if there exists $0 \leq m < |Q|$ such that $\varphi(p) = \delta(p, a^m)$ for all $p \in Q$.*

Proof. Let $m \geq 0$ such that $\varphi(q) = \delta(q, a^m)$. By Theorem 6.51, φ is a morphism if and only if $\varphi(\delta(q, a^i)) = \delta(\varphi(q), a^i)$ for $0 \leq i < |Q|$. This is equivalent to $\varphi(p) = \delta(p, a^m)$ for all $p \in Q$ since $\delta(\varphi(q), a^i) = \delta(\delta(q, a^i), a^m)$. □

Since in unary automata, strongly connected components are cycles, by Theorem 6.47, the size of a strongly connected component in the image of a morphism divides the size of a strongly connected component that maps to it. This might fail for non-unary automata, as shown next.

Example 6.53 Let $\mathcal{A} = (\{q_0, q_1, q_2\}, \{a, b\}, \delta)$ and $\mathcal{B} = (\{p_0, p_1\}, \{a, b\}, \mu)$ with $\delta(q_0, a) = q_1$, $\delta(q_1, a) = q_2$, $\delta(q_1, b) = q_0$, $\delta(q_2, b) = q_0$, $\mu(p_0, a) = p_1$, $\delta(p_1, b) = p_0$ and self-loops for all the remaining transitions, see Figure 6.5. The map φ with

$\varphi(q_0) = p_0, \varphi(q_1) = p_1, \varphi(q_2) = p_1$ is a morphism since $\mu(\varphi(q), b) = p_0 = \varphi(q_0) = \varphi(\delta(q, a))$ for each $q \in \{q_0, q_1, q_2\}$ and

$$\begin{aligned}
\mu(\varphi(q_0), a) &= \mu(p_0, a) = p_1 = \varphi(q_1) = \varphi(\delta(q_0, a)),\\
\mu(\varphi(q_1), a) &= \mu(p_1, a) = p_1 = \varphi(q_2) = \varphi(\delta(q_1, a)),\\
\mu(\varphi(q_2), a) &= \mu(p_1, a) = p_1 = \varphi(q_2) = \varphi(\delta(q_2, a)).
\end{aligned}$$

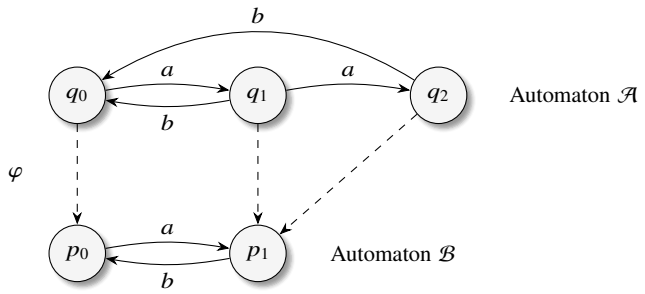


Fig. 6.5: Morphism φ between two automata $\mathcal{A}$ and $\mathcal{B}$, see Theorem 6.53.

6.5 Operations on Automata

6.5.1 Subautomata and Restricted Automata

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\Gamma \subseteq \Sigma$.

1. The *restriction of $\mathcal{A}$ to* Γ is $\mathcal{A}_{|\Gamma} = (Q, \Gamma, \delta_{|\Gamma})$ where $\delta_{|\Gamma} \colon Q \times \Gamma \to Q$ is defined by $\delta_{|\Gamma}(q, x) = \delta(q, x)$ for all $q \in Q$ and all $x \in \Gamma$.
2. Let $S \subseteq Q$ be an upper set in $\mathcal{A}$. The *restriction of $\mathcal{A}$ to the upper set S* is the deterministic and complete semiautomaton $\mathcal{A}_{|S} = (S, \Sigma, \delta_{|S})$ with $\delta_{|S} \colon S \times \Sigma \to S$ given by $\delta_{|S}(q, x) = \delta(q, x)$ for $q \in S$ and $x \in \Sigma$. We also call $\mathcal{A}_{|S}$ the *sub(semi-)automaton* corresponding to (or induced/defined by) the upper set $S \subseteq Q$. We often just denote $\delta_{|S}$ by δ in this context.
3. If $S \subseteq Q$ is an upper set in $\mathcal{A}_{|\Gamma}$, then by $\mathcal{A}_{|\Gamma,S} = (S, \Gamma, \delta_{|\Gamma,S})$ we denote the *restriction of $\mathcal{A}$ to the the alphabet Γ and the subset S*, i.e., $(\mathcal{A}_{|\Gamma})_{|S}$.

Upper sets can be identified with subautomata. Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ be upper sets. The following facts are readily seen. We have

$$(\mathcal{A}_{|S})_{|T} \cong \mathcal{A}_{|T} \text{ for } T \subseteq S \tag{6.7}$$

Clearly, $\mathcal{A}_{|S}$ is strongly connected if and only if S is a sink component.

Proposition 6.54 *The extension of the inclusion map $i\colon S \to Q$ given by $i(q) = q$ for $q \in S$ to subsets induces a bijection between the strongly connected components (resp. upper sets) in $\mathcal{A}_{|S}$ and the strongly connected components (resp. upper sets) R in $\mathcal{A}$ with $R \subseteq S$.*

6.5.2 The Quotient Automaton

Let $\mathcal{A} = (Q, \Sigma, \delta)$. Suppose $\sim$ is a $\mathcal{T}_S(\mathcal{A})$-congruence. Equivalently, for every $p, q \in Q$ and $x \in \Sigma$, we have

$$p \sim q \Rightarrow \delta(p,x) \sim \delta(q,x). \tag{6.8}$$

The *quotient automaton* is $\mathcal{A}/\!\sim = (Q/\!\sim, \Sigma, \delta_\sim)$ with the well-defined transition function $\delta_\sim([q]_\sim, x) = [\delta(q,x)]_\sim$ for all $x \in \Sigma$ and $q \in Q$.

The next two statements connect $\mathcal{T}_S(\mathcal{A})$-congruences, quotient automata, the kernel relation, and morphisms in the following way: $\mathcal{T}_S(\mathcal{A})$-congruences correspond to surjective morphisms onto quotient automata via the canonical morphism (Theorem 6.55), and surjective morphisms correspond to $\mathcal{T}_S(\mathcal{A})$-congruences via the kernel relation (Theorem 6.56).

In view of Theorem 6.55, in this context we also call $\pi_\sim$ the *canonical morphism (for $\sim$)* from $\mathcal{A}$ onto $\mathcal{A}/\!\sim$.

Theorem 6.55 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $\mathcal{T}_S(\mathcal{A})$-congruence $\sim$ on $\mathcal{A}$. Then the canonical projection map $\pi_\sim\colon Q \to Q/\!\sim$ from Equation* (3.28) *is a surjective morphism whose kernel relation equals $\sim$.*

Proof. Clearly, $\pi_\sim$ is surjective and its kernel relation equals $\sim$. It is a morphism since by definition of $\mathcal{A}/\!\sim$ we have $\delta_\sim(\pi_\sim(q), x) = \pi_\sim(\delta(q,x))$ for $q \in Q$ and $x \in \Sigma$. □

Theorem 6.56 *Let $\varphi\colon Q \to S$ be a morphism between the automata $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$. Then the following is true:*

1. *$\varphi(Q)$ is an upper set in $\mathcal{B}$,*
2. *the kernel relation $\sim_\varphi$ is a $\mathcal{T}_S(\mathcal{A})$-congruence,*
3. *$[q]_{\sim_\varphi} \mapsto \varphi(q)$, $q \in Q$, is a well-defined isomorphism between $\mathcal{A}/\!\sim_\varphi$ and $\mathcal{B}_{|\varphi(Q)}$.*

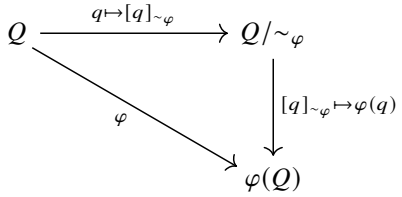
Proof. 1: This follows from Theorem 6.25, Item 2.

2: Suppose $p \sim_\varphi q$ for $p, q \in Q$. Then we have $\varphi(p) = \varphi(q)$. Since φ is a morphism, we find $\varphi(\delta(p,x)) = \mu(\varphi(p), x) = \mu(\varphi(q), x) = \varphi(\delta(q,x))$, and so $\delta(p,x) \sim_\varphi \delta(q,x)$, i.e., $\sim_\varphi$ is a $\mathcal{T}_S(\mathcal{A})$-congruence.

3: Well-definedness follows from the definition of the kernel relation. Now $\delta_{\sim_\varphi}([q]_{\sim_\varphi}, x) = [\delta(q,x)]_{\sim_\varphi}$ is mapped to $\varphi(\delta(q,x))$ and $[q]_{\sim_\varphi}$ is mapped to $\varphi(q)$. Since $\mu(\varphi(q), x) = \varphi(\delta(q,x))$, we obtain that the mapping is a morphism as we arrive at the same result when applying first a letter and then the mapping, or when first applying the mapping and then a letter. The mapping clearly maps onto $\mathcal{B}_{|\varphi(Q)}$. □

A similar statement as Theorem 6.56 in the theory of groups is often called the homomorphism theorem, for example, see [393, Sect. 1.2.5]. Theorem 6.56 states that

morphisms composed along paths in the following diagram ending at the same point are equal.



Corollary 6.57 *Let $\varphi\colon Q \to S$ be a morphism between the automata $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$ with $Q \neq \emptyset$. If $\mathcal{B}$ is strongly connected, then $\varphi(Q) = S$, i.e., φ is surjective. So $\mathcal{A}/{\sim_\varphi} \cong \mathcal{B}$. In particular, for $\mathcal{A} = \mathcal{B}$, by finiteness, φ is an isomorphism.*

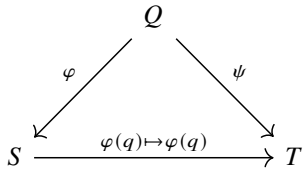
Corollary 6.58 *Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence. Then every morphism from $\mathcal{A}$ into another automaton is either injective or constant.*

Proof. For a morphism into another automaton, its kernel relation is a $\mathcal{T}_S(\mathcal{A})$-congruence by Theorem 6.56, Item 2. So it either equals the universal relation, and so the morphism is a constant, or the equality relation, and so the morphism is injective. □

Next, we show a condition to obtain a morphism from two morphisms from the same automaton. The proof is straightforward and omitted.

Theorem 6.59 *Let $\varphi\colon Q \to S$ and $\psi\colon Q \to T$ be morphisms from $\mathcal{A} = (Q, \Sigma, \delta)$ to $\mathcal{B} = (S, \Sigma, \mu)$ and $C = (T, \Sigma, \xi)$, respectively. Suppose ${\sim_\varphi}$ refines ${\sim_\psi}$. Then $\varphi(q) \mapsto \psi(q)$ is a well-defined surjective morphisms from $\mathcal{B}_{|\varphi(Q)}$ onto $C_{|\psi(Q)}$.*

By Theorem 6.56, Item 3, $\mathcal{A}/{\sim_\varphi} \cong \mathcal{B}_{|\varphi(Q)}$ and $\mathcal{A}/{\sim_\psi} \cong C_{|\psi(Q)}$ in Theorem 6.59. In the same way as for Theorem 6.56, the situation from Theorem 6.59 is illustrated in the following diagram.



Corollary 6.60 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$ with ${\sim_\mathcal{A}}$ and ${\sim_\mathcal{B}}$ being $\mathcal{T}_S(\mathcal{A})$- and $\mathcal{T}_S(\mathcal{B})$-congruences, respectively. If ${\sim_\mathcal{A}}$ refines ${\sim_\mathcal{B}}$, then $[q]_{\sim_\mathcal{A}} \mapsto [q]_{\sim_\mathcal{B}}$ is a morphism from $\mathcal{A}/{\sim_\mathcal{A}}$ into $\mathcal{B}/{\sim_\mathcal{B}}$.*

6.5.3 The Rees Quotient Automaton

Here, we introduce a special kind of quotient automata whose construction resembles a similar one from the theory of semigroups introduced by Rees [505].

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. The *Rees congruence* is the $\mathcal{T}_S(\mathcal{A})$-congruence on Q given, for $p, q \in Q$, by

$$p \sim_S q \Leftrightarrow p, q \in S \vee (p = q).$$

We call $\mathcal{A}/\sim_S$ the *Rees quotient (semi-)automaton* w.r.t. the upper set $S \subseteq Q$. For $S \neq \emptyset$, intuitively, $\mathcal{A}/\sim_S$ results from $\mathcal{A}$ by collapsing the states in S to a single trap state in $\mathcal{A}/\sim_S$.

Remark 6.61 For $S \neq \emptyset$, unraveling the definition, $\mathcal{A}/\sim_S$ has the state set $\{\{p\} \mid p \in Q \setminus S\} \cup \{S\}$, the transition function fulfills $\delta_{\sim_S}(S, x) = S$ and, for all $p \notin S$ and all $x \in \Sigma$,

$$\delta_{\sim_S}(\{p\}, x) = \begin{cases} \{\delta(p, x)\} & \text{if } \delta(p, x) \notin S, \\ S & \text{otherwise.} \end{cases} \tag{6.9}$$

So, $\mathcal{A}/\sim_S$ has $|Q| - |S| + 1$ states. By Theorem 3.37, as a strongly connected component is either contained in S or disjoint from S, for $p, q \in Q$ and $x \in \Sigma$ with $p \simeq_R q$ we have

$$p = \delta(q, x) \Leftrightarrow \begin{cases} p = \delta_{|S}(q, x) & \text{if } p \in S, \\ \pi_{\sim_S}(p) = \delta_{\sim_S}(\pi_{\sim_S}(q), x) & \text{if } p \notin S. \end{cases} \tag{6.10}$$

For $T \subseteq Q$, we have

$$\pi_{\sim_S}(T) = \begin{cases} \{S\} \cup \{\{q\} \mid q \in T \setminus S\} & \text{if } S \cap T \neq \emptyset, \\ \{\{q\} \mid q \in T\} & \text{if } S \cap T = \emptyset. \end{cases} \tag{6.11}$$

As stated next, the projection maps induce bijections between certain strongly connected components and upper sets in $\mathcal{A}$ and $\mathcal{A}/\sim_S$

Proposition 6.62 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and let $S \subseteq Q$ be an upper set.*

1. *Let P be a strongly connected component of $\mathcal{A}/\sim_S$, distinct from $\{S\}$. Then $\pi_{\sim_S}^{-1}(P)$ is a strongly connected component of $\mathcal{A}$ contained in $Q \setminus S$, and*

   $$\pi_{\sim_S}\big(\pi_{\sim_S}^{-1}(P)\big) = P.$$

   Conversely, let $R \subseteq Q \setminus S$ be a strongly connected component of $\mathcal{A}$. Then $\pi_{\sim_S}(R)$ is a strongly connected component of $\mathcal{A}/\sim_S$ distinct from $\{S\}$, and

   $$\pi_{\sim_S}^{-1}\big(\pi_{\sim_S}(R)\big) = R.$$

2. *Let P be an upper set of $\mathcal{A}/\sim_S$ containing S. Then $\pi_{\sim_S}^{-1}(P)$ is an upper set of $\mathcal{A}$ containing S, and*

   $$\pi_{\sim_S}\big(\pi_{\sim_S}^{-1}(P)\big) = P.$$

   Conversely, let R be an upper set of $\mathcal{A}$ containing S. Then $\pi_{\sim_S}(R)$ is an upper set of $\mathcal{A}/\sim_S$ containing S, and

   $$\pi_{\sim_S}^{-1}\big(\pi_{\sim_S}(R)\big) = R.$$

Proof. This follows from Theorem 6.61, in particular Equation (6.9), which yield that for $p, q \in Q \setminus S$ and $u \in \Sigma^*$ we have $\delta(p, u) = q$ if and only if $\delta_{\sim_S}(\pi_{\sim_S}(p), u) = \pi_{\sim_S}(q)$. □

Corollary 6.63 *The number of strongly connected components in $\mathcal{A}/{\sim_S}$ is precisely the number of strongly connected components not in S plus one accounting for $\{S\}$.*

Corollary 6.64 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be a proper upper set. Then $Q \setminus S$ is a strongly connected component in $\mathcal{A}$ if and only if $(Q/{\sim_S}) \setminus \{S\}$ is a strongly connected component in $\mathcal{A}/{\sim_S}$.*

Corollary 6.65 *For $S \neq \emptyset$ and $S \neq Q$, $\mathcal{A}/{\sim_S}$ is strongly trap-connected if and only if $Q \setminus S$ is strongly connected and $\delta(Q \setminus S, x) \cap S \neq \emptyset$ for some $x \in \Sigma$.*

Proof. By Theorem 6.62, $Q \setminus S$ is strongly connected if and only if $\pi_{\sim_S}(Q \setminus S)$ and $\pi_{\sim_S}(S)$ are the only strongly connected components in $\mathcal{A}/{\sim_S}$. Since $\pi_{\sim_S}(Q \setminus S)$ is not a sink component, the claim follows from Item 2 of Theorem 6.10. □

Next, we state isomorphisms between Rees quotient automata.

Proposition 6.66 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ be upper sets. Then we have:*

$$\mathcal{A} \cong \mathcal{A}/{\sim_S} \qquad \text{for } |S| \leq 1 \tag{6.12}$$

$$(\mathcal{A}_{|S})/{\sim_{S\cap T}} \cong (\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S)} \tag{6.13}$$

$$(\mathcal{A}_{|S\cup T})/{\sim_T} \cong (\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S)} \qquad \text{for } S \cap T \neq \emptyset \tag{6.14}$$

$$(\mathcal{A}/{\sim_S})/{\sim_{\pi_{\sim_S}(T)}} \cong \mathcal{A}/{\sim_{S\cup T}} \tag{6.15}$$

$$(\mathcal{A}/{\sim_S})/{\sim_{\pi_{\sim_S}(T)}} \cong (\mathcal{A}/{\sim_T})/{\sim_{\pi_{\sim_T}(S)}} \tag{6.16}$$

Proof. Equation (6.12): For $|S| \leq 1$ the Rees congruence $\sim_S$ equals the equality relation and thus $\mathcal{A} \cong \mathcal{A}/{\sim_S}$.
Equation (6.13): The map $(\pi_{\sim_T})_{|S} \colon S \to Q/{\sim_T}$ is a morphism from $\mathcal{A}_{|S}$ to $\mathcal{A}/{\sim_T}$ with kernel $\sim_{S\cap T}$. Thus the stated isomorphism follows from Theorem 6.56.[6]
Equation (6.14): Set $S' = S \cup T$. Then S' is an upper set and, by Equation (6.13), we find

$$(\mathcal{A}_{|S'})/{\sim_{S'\cap T}} \cong (\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S')}.$$

Then the claim follows since $S' \cap T = T$ and for $S \cap T \neq \emptyset$ we have $\pi_{\sim_T}(S) = \pi_{\sim_T}(S')$.
Equation (6.15): The map $\pi_{\sim_R}\pi_{\sim_S} \colon Q \to (Q/{\sim_S})/{\sim_R}$ with $R = \pi_{\sim_S}(T)$ is a morphism with kernel $\sim_{S\cup T}$. Thus the stated isomorphism follows from Theorem 6.56.[7]

[6] Alternatively, by Theorem 6.59, the map $\pi_{\sim_{S\cap T}}(q) \mapsto \pi_{\sim_T}(q)$ for $q \in S$ is a well-defined surjective morphism from $(\mathcal{A}_{|S})/{\sim_{S\cap T}}$ onto $(\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S)}$. Since

$$\pi_{\sim_{S\cap T}}(p) = \pi_{\sim_{S\cap T}}(q) \iff \pi_{\sim_T}(p) = \pi_{\sim_T}(q)$$

for all $p, q \in S$, this morphism is an isomorphism. Thus we have $(\mathcal{A}_{|S})/{\sim_{S\cap T}} \cong (\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S)}$.

[7] Alternatively, since $\sim_S$ refines $\sim_{S\cup T}$, the map $\pi_{\sim_S}(p) \mapsto \pi_{\sim_{S\cup T}}(p)$ for $q \in Q$ is a well-defined surjective morphism by Theorem 6.59. Since it has kernel $\pi_{\sim_S}(T)$ the stated isomorphism follows from Theorem 6.56.

Equation (6.16): By Equation (6.15) both automata involving two Rees quotients are isomorphic to $\mathcal{A}/{\sim_{S\cup T}}$. Thus they are isomorphic to each other. □

Note that Equation (6.13) also implies $(\mathcal{A}_{|S})/{\sim_{S\cap T}} \cong (\mathcal{A}/{\sim_{S\cap T}})_{|\pi_{\sim_{S\cap T}}(S)}$ by applying it to the upper sets S and $S \cap T$ instead of S and T.

For a morphism and an upper set, we obtain a morphism between corresponding Rees quotient automata.

Proposition 6.67 *Let $\varphi\colon Q \to S$ be a morphism from $\mathcal{A} = (Q, \Sigma, \delta)$ to $\mathcal{B} = (S, \Sigma, \mu)$ and $T \subseteq Q$ be an upper set. Then $\varphi(T)$ is an upper set and*

$$[q]_{\sim_T} \mapsto [\varphi(q)]_{\sim_{\varphi(T)}}$$

for $q \in Q$ is a well-defined surjective morphism from $\mathcal{A}/{\sim_T}$ onto $(\mathcal{B}/{\sim_{\varphi(T)}})_{|\pi_{\sim_{\varphi(T)}}(\varphi(Q))}$.

Proof. For $p, q \in Q$, we have

$$\begin{aligned} p \sim_T q &\Rightarrow p = q \vee \{p, q\} \subseteq T \\ &\Rightarrow \varphi(p) = \varphi(q) \vee \{\varphi(p), \varphi(q)\} \subseteq \varphi(T) \\ &\Rightarrow \varphi(p) \sim_{\varphi(T)} \varphi(q). \end{aligned}$$

Hence, Theorem 6.59 gives the claim. In fact, we have $\varphi([q]_{\sim_T}) = [\varphi(q)]_{\sim_{\varphi(T)}}$ and so φ induces this morphism on the equivalence classes of $\sim_T$. □

Lastly, using the Rees quotient automaton construction we can derive a necessary condition, originally due to Thierrin [599, Prop. 8], for primitivity of automata (see also [51, Cor. 3] or, in a more algebraic context, [581, Prop. 5.1]).

Proposition 6.68 (Thierrin [599, Prop. 8]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a primivite automaton with $|Q| \geq 3$. Then $\mathcal{A}$ is strongly connected or strongly trap-connected with trap state $t \in Q$ such that $\delta(p, a) = t$ and $\delta(q, a) \neq t$ for some $p, q \in Q$ and $a \in \Sigma$.*

Proof. There exists no proper upper set $S \subseteq Q$ of size at least two, as otherwise $\sim_S$ would be a non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence. Hence, by Theorem 6.11, $\mathcal{A}$ is either strongly connected or strongly trap-connected with a trap state $t \in Q$. In the latter case, since $Q \setminus \{t\}$ and $\{t\}$ form a partition that does not induce a $\mathcal{T}_S(\mathcal{A})$-congruence, there exist $p, q \in Q$ and $a \in \Sigma$ such that $\delta(p, a) = t$ and $\delta(q, a) \neq t$. □

The following corollary has been noted in [525, Sect. 5.1] and [610, Prop. 3.1].

Corollary 6.69 *A primitive automaton $\mathcal{A}$ with at least three states is either strongly connected or synchronizing.*

Proof. This follows from Theorem 6.68 since a strongly trap-connected automaton is synchronizing. □

Remark 6.70 Theorem 6.68 generalizes Theorem 5.1 (via the correspondence from Section 6.8), since a trap state in a permutation automaton is not reachable from any other state. Thus, by Theorem 6.68, a permutation automata whose transformation monoid is a primitive group G is strongly connected and so G is transitive.

6.5.4 The Product Automaton

Given $\mathcal{A}_i = (Q_i, \Sigma, \delta_i)$ for $i \in \{1, \ldots, m\}$, their *product (semi-)automaton* [322, 502] is $\mathcal{A}_1 \times \ldots \times \mathcal{A}_m = (Q_1 \times \ldots \times Q_m, \Sigma, \delta_{\mathcal{A}_1 \times \ldots \times \mathcal{A}_m})$ with

$$\delta((q_1, \ldots, q_m), x) = (\delta_1(q_1, x), \ldots, \delta_m(q_m, x))$$

for $q_1 \in Q_1, \ldots, q_m \in Q_m$ and $x \in \Sigma$. If $\mathcal{A}_1 = \cdots = \mathcal{A}_m$, then we also write $\mathcal{A}_1^m$ for $\mathcal{A}_1 \times \cdots \times \mathcal{A}_m$. For the set of synchronizing words we have

$$\mathrm{Syn}(\mathcal{A}_1 \times \cdots \times \mathcal{A}_m) = \bigcap_{i=1}^{m} \mathrm{Syn}(\mathcal{A}_i). \tag{6.17}$$

For upper sets $S_1 \subseteq Q_1, \ldots, S_m \subseteq Q_m$, the set $S = S_1 \times \cdots \times S_m$ is an upper set and since $(q_1, \ldots, q_m) \in S$ if and only if $q_1 \in S_1, \ldots, q_m \in S_m$ we have

$$(\mathcal{A}_1 \times \cdots \times \mathcal{A}_m)_{|S} \cong (\mathcal{A}_1)_{|S_1} \times \cdots \times (\mathcal{A}_m)_{|S_m} \tag{6.18}$$

$$(\mathcal{A}_1 \times \cdots \times \mathcal{A}_m)/{\sim_S} \cong (\mathcal{A}_1/{\sim_{S_1}}) \times \cdots \times (\mathcal{A}_m/{\sim_{S_m}}) \tag{6.19}$$

6.5.5 The Power Automaton

Let $\mathcal{A} = (Q, \Sigma, \delta)$. The *power (semi-)automaton* is $\mathcal{P}(\mathcal{A}) = (\mathcal{P}(Q), \Sigma, \delta)$ where $\delta\colon \mathcal{P}(Q) \times \Sigma \to \Sigma$ is the extension of $\delta\colon Q \times \Sigma \to Q$ to subsets $\delta(S, x) = \{\delta(q, x) \mid q \in Q\}$ for $S \subseteq Q$ and $x \in \Sigma$ (see also Section 1.1). Figure 6.6 shows the power automaton of a four-state automaton. Also, part of the power automaton of the Černý automaton C_4 is depicted in Figure 9.1.

In this section, we present properties of powers of a single word in the power automaton. These can also be interpreted as properties of transformations on subsets of a finite set; however, since our focus is on automata, we express them in automata-theoretic terms.

Proposition 6.71 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $n = |Q|$ and $u \in \Sigma^*$. Then for $i \geq 0$ we have $\delta(Q, u^{i+1}) \subseteq \delta(Q, u^i)$ and for $P = \delta(Q, u^{n-1})$ the following is true:*

1. *$P = \delta(Q, u^{n-|P|})$,*
2. *$\delta(P, u) = P$ (u permutes P) where P is the largest set with this property, and $P = \delta(Q, u^i)$ for all $i \geq n - |P|$,*
3. *$P \subseteq \delta(Q, u^i)$ for all $i \geq 0$.*

Proof. For the first claim, since $\delta(Q, u) \subseteq Q$ and $\delta(Q, u^{i+1}) \subseteq \delta(Q, u^i)$ implies

$$\delta(\delta(Q, u^{i+1}), u) \subseteq \delta(\delta(Q, u^i), u),$$

by induction we find that $\delta(Q, u^{i+1}) \subseteq \delta(Q, u^i)$ for all $i \geq 0$.

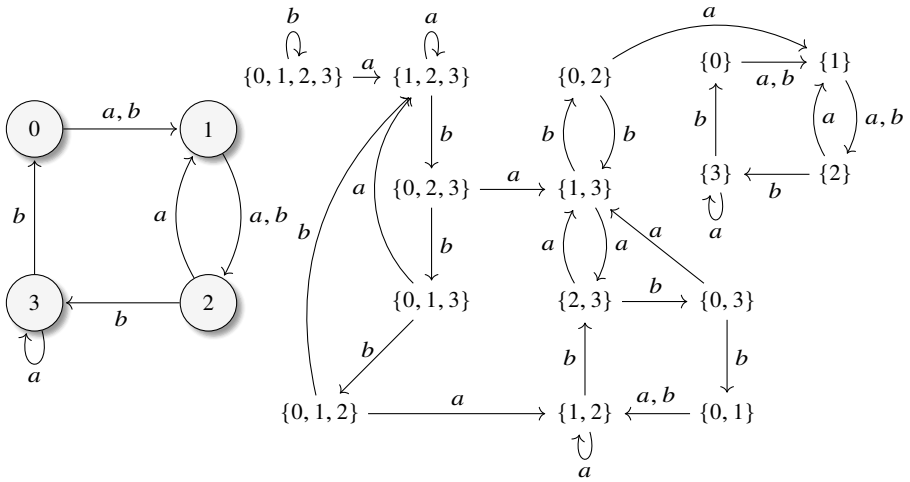


Fig. 6.6: Left: An automaton. Right: Its power automaton (empty set omitted).

Next, by finiteness, there exists a minimal $j \geq 0$ such that $\delta(Q, u^{j+1}) = \delta(Q, u^j)$. Then we have $\delta(Q, u^k) = \delta(Q, u^j)$ for all $k \geq j$ and we get the following chain of proper inclusions

$$\delta(Q, u^j) \subsetneq \delta(Q, u^{j-1}) \subsetneq \cdots \subsetneq \delta(Q, u) \subsetneq Q. \tag{6.20}$$

This yields $\delta(Q, u^j) \subseteq \delta(Q, u^i)$ for every $i \geq 0$. Since $|\delta(Q, w)| \geq 1$ for all $w \in \Sigma^*$, the above chain of proper inclusions has length at most $n - |\delta(Q, u^j)|$, i.e., $j \leq n - |\delta(Q, u^j)|$. Hence we find

$$P = \delta(Q, u^{n-1}) = \delta(Q, u^{n-|\delta(Q,u^j)|}) = \delta(Q, u^j) = \delta(Q, u^n) = \delta(P, u),$$

and so $P = \delta(Q, u^{n-|P|})$, $P \subseteq \delta(Q, u^i)$ for all $i \geq 0$, and P is permuted by u. This also gives $P = \delta(Q, u^i)$ for $i \geq n - |P|$ by induction. Furthermore, if $\delta(S, u) = S$ for $S \subseteq Q$, then $S = \delta(S, u^i) \subseteq \delta(Q, u^i)$ for all $i \geq 0$. In particular, we obtain $S \subseteq \delta(Q, u^{n-1}) = P$ and so P is the largest set permuted by u. This shows all claims. □

The set P from Theorem 6.71 is precisely the set of cycle states described in Theorem 3.69. By finiteness, after applying a word at most $n - 1$ times to any state we end up at a cycle state.

Note that since $\delta(Q, u) \subseteq Q$ for $u \in \Sigma^*$, we have $\delta(Q, uv) \subseteq \delta(Q, v)$ for each $v \in \Sigma^*$.

Corollary 6.72 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $n = |Q|$ and $u \in \Sigma^*$. Then we have $\delta(Q, u^i) = \delta(Q, u^{\min\{n-1,i\}})$ for $i \geq 0$.*

Proof. By Theorem 6.71, the word u permutes the set $\delta(Q, u^{n-1})$. Thus, for $i \geq n - 1$ we find $\delta(Q, u^i) = \delta(Q, u^{n-1})$, which gives the claim. □

In [501, Lem. 1 & 2] it was also proved that P is the largest set permuted by u in Theorem 6.71, together with the fact that we have to apply u at most $n - |P|$ times to Q to obtain P.

Note that for the properties collected in Theorem 6.71 to hold it is essential that we start from the whole state set before applying powers of u.

Proposition 6.73 *Let $\mathcal{A} = (Q, \Sigma, \delta)$, $n = |Q|$, $u \in \Sigma^*$, $S \subseteq Q$, and $i \geq 0$.*

1. *$\delta(S, u^i) = \delta(S, u^j)$ for some $0 \leq j < 2^n - 1$.*
2. *$|\delta(S, u^i)| = |\delta(S, u^j)|$ for some $0 \leq j \leq n - |\delta(S, u^i)|$.*
3. *$|\delta(S, u^i)| = |\delta(S, u^{\min\{n-1,i\}})|$.*

Proof. 1: We can assume $S \neq \emptyset$. Let $j \geq 0$ be a minimal number such that $\delta(S, u^i) = \delta(S, u^j)$. Assume $j \geq 2^n - 1$. We have $\delta(S, u^k) \neq \emptyset$ for all $k \geq 0$. Since there are at most $2^n - 1$ non-empty subsets, and the sequence $\delta(S, u^0), \delta(S, u), \ldots, \delta(S, u^{2^n-1})$ has length 2^n, there exist $0 \leq k < k' \leq 2^n - 1$ such that $\delta(S, u^k) = \delta(S, u^{k'})$. Then $\delta(S, u^{j-(k'-k)}) = \delta(S, u^j)$, contradicting the minimal choice of j. So, we have $j < 2^n - 1$.

2: We can assume $i > n - |\delta(S, u^i)|$. For the minimal $j \geq 0$ with $\delta(Q, u^{j+1}) = \delta(Q, u^j)$, we have a proper chain of inclusions as in Equation (6.20). Thus, for $k \in \{0, 1, \ldots, j\}$, we get $|\delta(Q, u^k)| \leq n-k$. Since $\delta(S, u^j) \subseteq \delta(Q, u^j)$, we find $|\delta(S, u^j)| \leq n - j$, and so $j \leq n - |\delta(S, u^j)|$. Set $P = \delta(Q, u^{n-1})$. With Theorem 6.71 we get $\delta(Q, u^j) = P$ and that u permutes P, and so u acts injectively on subsets of P. Thus, from $\delta(S, u^j) \subseteq P$ we obtain $|\delta(S, u^k)| = |\delta(S, u^j)|$ for all $k \geq j$. In particular, since $i > j$, we find $|\delta(S, u^i)| = |\delta(S, u^j)|$.

3: We can assume $i \geq n$. By Item 2 we have $|\delta(S, u^i)| = |\delta(S, u^j)|$ for some $0 \leq j < n$. With Equation (3.17) we find

1. $|\delta(S, u^{n-1})| = |\delta(\delta(S, u^j), u^{n-1-j})| \leq |\delta(S, u^j)|$ and
2. $|\delta(S, u^i)| = |\delta(\delta(S, u^{n-1}), u^{i-n+1})| \leq |\delta(S, u^{n-1})|$.

Thus we obtain $|\delta(S, u^i)| = |\delta(S, u^{n-1})|$. □

Corollary 6.74 *Let $\mathcal{A} = (Q, \Sigma, \delta)$, $u \in \Sigma^*$, and $S \subseteq Q$. Then $|\delta(S, u^i)| = 1$ for some $i \geq 0$ if and only if $|\delta(S, u^{n-1})| = 1$ for $n = |Q|$.*

Proof. First, assume $|\delta(S, u^i)| = 1$ for some $i \geq 0$. Since $|\delta(S, u^i)| = |\delta(S, u^{\min\{n-1,i\}})|$ by Theorem 6.73, Item 3 and $|\delta(S, u^{n-1})| \leq |\delta(S, u^{\min\{n-1,i\}})|$, we find $|\delta(S, u^{n-1})| = 1$. The converse implication is obvious. □

Remark 6.75 To obtain Theorem 6.73, Item 1, we have basically used the fact that in any n-state automaton we enter a cycle when we read a word more than $n-1$ times. Now, as u permutes P in Theorem 6.71, the smallest $i \geq 0$ such that $\delta(q, u^i) = q$ for all $q \in P$ is asymptotically equal to $\exp((1 + o(1))\sqrt{(|P| \ln |P|)}$ by a result from Landau [403]. This can be used to improve the bound from Theorem 6.73 asymptotically. However, as the bound given is sufficient for our applications, we do not bother to state a more precise bound.

6.5.6 The Pair Automaton

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S = \{\{p, q\} \mid p, q \in Q\}$. Then S is an upper set in $\mathcal{P}(\mathcal{A})$. We denote $\mathcal{P}(\mathcal{A})_{|S}$ also by $\mathcal{A}^{\leq 2} = (S, \Sigma, \delta^{\leq 2})$. We have $\delta^{\leq 2}(\{p, q\}, x) = \delta(\{p, q\}, x)$ for $x \in \Sigma$. The set $F = \{\{q\} \mid q \in Q\}$ is an upper set in $\mathcal{A}^{\leq 2}$ and the Rees quotient automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ is called the *pair (semi-)automaton*. The state set of the pair automaton is $\{\{\{p, q\}\} \mid p \neq q \text{ and } p, q \in Q\} \cup \{F\}$. For ease of notation, for distinct $p, q \in Q$ we identify a state $\{\{p, q\}\}$ with $\{p, q\}$. The pair automaton has $\binom{n}{2} + 1$ states.

The map $\varphi \colon Q \times Q \to \{\{p, q\} \mid p, q \in Q\}$ given by $\varphi(p, q) = \{p, q\}$ is a surjective morphism of $\mathcal{A} \times \mathcal{A}$ onto $\mathcal{A}^{\leq 2}$. Thus, by Theorem 6.55, $[(p, q)]_{\sim_\varphi} \mapsto \{p, q\}$ is an isomorphism from $(\mathcal{A} \times \mathcal{A})/{\sim_\varphi}$ to $\mathcal{A}^{\leq 2}$. Note that $(p, q) \sim_\varphi (s, t)$ if and only if $\{p, q\} = \{s, t\}$ and so $\sim_\varphi$ is the $\mathcal{T}_M(\mathcal{A} \times \mathcal{A})$-congruence that precisely identifies (p, q) with (q, p) for $p, q \in Q$.

6.6 Sequences of Upper Sets and Maximal States

In this section, we investigate sequences of upper sets as constructed in Section 3.6.5 for general quasiorders in the special case of automata and the reachability order. More specifically, we relate these sequences to subautomata and Rees quotient automata as introduced previously in Sections 6.5.1 and 6.5.3.

Proposition 6.76 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. There exists a sequence of upper sets*

$$\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$$

such that the sets $S_i \setminus S_{i-1}$ for $i \in \{1, \ldots, m\}$ are all the strongly connected components in $\mathcal{A}$, $\mathcal{A}_{|S_1}$ is strongly connected and, for $i \in \{2, \ldots, m\}$, $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is strongly trap-connected or $S_i \setminus S_{i-1}$ is a sink component in $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ (so that $S_i \setminus S_{i-1}$ and $\{S_{i-1}\}$ are the only sink components in this case).

Proof. Consider a sequence $\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Theorem 3.59, Item 1, where, according to the statement, the sets $S_i \setminus S_{i-1}$ for $i \in \{1, \ldots, m\}$ are all the strongly connected components. Since S_1 is a strongly connected component, $\mathcal{A}_{S_1}$ is strongly connected. Next, let $i \in \{2, \ldots, m\}$. Now, suppose $S_i \setminus S_{i-1}$ is not a sink component. Since the state set of $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is $S_i \setminus S_{i-1} \cup \{S_{i-1}\}$ with $S_{i-1} \neq \emptyset$, if $S_i \setminus S_{i-1}$ is not a sink component, by Theorem 6.10, Item 2 the automaton is strongly trap-connected; otherwise $S_i \setminus S_{i-1}$ is a sink component and the claim is proved. □

In a sequence as in Theorem 6.76, for $p \in S_i$ and $q \in S_j$ with $p \preceq_R q$ we have $i \leq j$.

Proposition 6.77 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. There exists a sequence of upper sets*

$$S_1 \subsetneq \cdots \subsetneq S_m = Q$$

such that S_1 equals the set of maximal states in $\mathcal{A}$, and for $i \in \{2, \dots, m\}$, $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is strongly trap-connected and $S_i \setminus S_{i-1}$ are all the strongly connected components in $\mathcal{A}$ that are not sink components.

Proof. Consider a sequence $S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Theorem 3.59, Item 2 where S_0 equals the set of maximal states and $S_i \setminus S_{i-1}$ for $i \in \{1, \dots, m\}$ are all the non-sink strongly connected components. Since the state set of $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ for $i \in \{1, \dots, m\}$ is $S_i \setminus S_{i-1} \cup \{S_{i-1}\}$ with $S_{i-1} \neq \emptyset$ and $S_i \setminus S_{i-1}$ is not a sink component, it is strongly trap-connected by Theorem 6.10, Item 2. □

Remark 6.78 The strongly trap-connected quotient automata that arise from a series as in Theorem 6.76 are always the same up to isomorphism. However, the remaining quotient automata and the subautomaton $\mathcal{A}_{S_1}$ might be different for different series, where they differ by an isolated, i.e., not reachable, trap state. However, the length of such a series is always the same. A series as in Theorem 6.77 always has the same length for a given automaton and the resulting quotient automata $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ for $i \in \{2, \dots, m\}$ are always the same up to isomorphism. However, $\mathcal{A}_{S_1}/{\sim_\emptyset}$ might admit further subautomata. For synchronizing automata, series obtained according to Theorem 6.76 or Theorem 6.77 have the same length, isomorphic quotient automata, and $\mathcal{A}_{|S_1}$ admits no subautomata. Thus, in this case, we obtain an analog of the Jordan-Hölder theorem from algebra [405].

Corollary 6.79 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then Q equals the set of maximal states or there exists a proper and non-empty upper set $S \subseteq Q$ such that $\mathcal{A}/{\sim_S}$ is strongly trap-connected.*

Proof. Consider a sequence $S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Theorem 6.77. For $m = 1$, Q equals the set of maximal states. Otherwise, the claim is fulfilled with the upper set $S = S_{m-1}$. □

Proposition 6.80 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be the set of maximal states. Then we have the following.*

1. *For all $q \in Q$ there exists $u \in \Sigma^*$ such that $\delta(Q, u) \subseteq S$.*
2. *$[q]_{\sim_S} \preceq_R^{\mathcal{A}/{\sim_S}} S$ in $\mathcal{A}/{\sim_S}$ for all $q \in Q$.*

Proof. 1 & 2: By finiteness, for each $q \in Q$ there exists a maximal $p \in S$ such that $q \preceq_R p$. By the definition of the reachability order, this yields $\delta(q, u) = p$ for some $u \in \Sigma^*$, which proves Item 1. Next for the canonical morphism $\pi_{\sim_S} \colon Q \to Q/{\sim_S}$ from Theorem 6.55 we have $\pi_{\sim_S}(q) \preceq_R^{\mathcal{A}/{\sim_S}} \pi_{\sim_S}(p)$ by Equation (6.5) where $\pi_{\sim_S}(q) = [q]_{\simeq_R}$ and $\pi_{\sim_S}(p) = S$ since $p \in S$. □

Proposition 6.81 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. The set S of maximal states is strongly connected if and only if there exists a unique sink component in $\mathcal{A}$ (which then equals S).*

Proof. First, suppose S is strongly connected. Since two sink component consist of incomparable elements, it follows that S equals a sink component, which is the only sink component in $\mathcal{A}$.

Conversely, suppose there exists a unique sink component. Since S is the union of all sink components by Equation (3.41) this unique sink component equals S and so S is strongly connected. □

6.7 Automata as Language Acceptors

6.7.1 Regular Languages

Chapter 1 introduced semiautomata as mere finite-state transition systems. Here we associate languages with automata. A *(deterministic and complete) finite automaton*[8] *(as a language acceptor)* is a 5-tuple $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ where (Q, Σ, δ) is a semiautomaton as defined before, q_0 is the *start* (or *initial) state* and $F \subseteq Q$ is the *set of final states*. The language *accepted*, or *recognized*, by $\mathcal{A}$ is

$$L(\mathcal{A}) = \{u \in \Sigma^* \mid \delta(q_0, u) \in F\},$$

i.e., the set of labels of paths going from the start state to some final state. We also say a word $u \in \Sigma^*$ is *accepted* by $\mathcal{A}$ if $u \in L(\mathcal{A})$.[9] A language $L \subseteq \Sigma^*$ is called *regular* if there exists an automaton $\mathcal{A}$ such that $L = L(\mathcal{A})$.[10]

Kleene [369] has shown that regular expressions denote precisely the set of regular languages. A proof (in contemporary notation) can be found in any textbook on formal language theory, e.g. [322, 491].

Theorem 6.82 (Kleene's theorem [369]) *The set of regular languages is precisely the set of languages describable by regular expressions, i.e., for every $\mathcal{A} = (Q, \Sigma, \delta, q, F)$ there exists a regular expression R such that $L(\mathcal{A}) = L(R)$ and vice versa.*

Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ and $q \in Q$. The state q is called *accessible* if it is reachable from the start state, i.e., there exists a word $u \in \Sigma^*$ such that $\delta(q_0, u) = q$. It is called *coaccessible* if there exists a word $u \in \Sigma^*$ such that $\delta(q, u) \in F$. We call $\mathcal{A}$ *accessible* (or *initially connected*) if all states are accessible. Obviously, for every regular language there exists an accessible automaton accepting it.

For a semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$, $q \in Q$ and $S \subseteq Q$, by

$$\mathcal{A}_{q,S} = (Q, \Sigma, \delta, q, S)$$

we denote the language acceptor automaton that results from $\mathcal{A}$ by choosing q as a start state and S as a set of final states.

There should be no confusion by calling both types of automata simply automata, as it should be clear from the context whether we mean automata as language acceptors, or whether we are talking about semiautomata. Furthermore, both types are specified by a different sets of data, a 3-tuple versus a 5-tuple, which distinguishes them notationwise. In case any ambiguity might arise, and it is necessary to differentiate between

[8] In the literature, most of the time, automata are meant to be language acceptors without a special term and automata without a start state and a final state are called semiautomata.

[9] There should be no problem with using the modifier "regular" in the context of languages and in the context of permutation groups since both domains are sufficiently different so that no confusion should arise.

[10] Eilenberg [194] called languages acceptable by automata recognizable, which makes sense when considering more general monoids than the free monoid. Since we do not consider the more general case, we do not use this terminology here. See also the related footnote on page 58.

both notions, we revert to using the proper names semiautomaton and automaton as language acceptor.

Every automaton as a language acceptor is also a semiautomaton when forgetting about the start state and the set of final states. So, all semiautomata notions transfer to automata as language acceptors and we use them also for language acceptor automata in this sense.

Given automata $\mathcal{A}_i = (Q_i, \Sigma, \delta_i, q_i, F_i)$ where $i \in \{1, \ldots, m\}$ and $m \geq 2$, for the product automaton $\mathcal{A}_1 \times \ldots \times \mathcal{A}_m$ we have the following:

$$L(\mathcal{A}_1) \setminus L(\mathcal{A}_2) = L((\mathcal{A}_1 \times \mathcal{A}_2)_{(q_1,q_2),(F_1\times Q)\setminus(Q\times F_2)}), \tag{6.21}$$

$$\bigcap_{i=1}^{m} L(\mathcal{A}_i) = L((\mathcal{A}_1 \times \cdots \times \mathcal{A}_m)_{(q_1,\ldots,q_m),F_1\times\cdots\times F_m}), \tag{6.22}$$

$$\bigcup_{i=1}^{m} L(\mathcal{A}_i) = L((\mathcal{A}_1 \times \cdots \times \mathcal{A}_m)_{(q_1,\ldots,q_m),E}), \tag{6.23}$$

where $E = \bigcup_{i=1}^{n} Q \times \cdots \times \underbrace{F_i}_{i\text{-th position}} \times \cdots \times Q$.

Next, we state two results that relate the accepted language to morphic images and congruences.

Proposition 6.83 *Let $\varphi\colon Q \to S$ be a morphism between $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$. Then $L(\mathcal{A}_{q_0,F}) \subseteq L(\mathcal{B}_{\varphi(q_0),\varphi(F)})$ for $q_0 \in Q$ and $F \subseteq Q$. Furthermore, if $\varphi^{-1}(\varphi(F)) = F$, then $L(\mathcal{B}_{\varphi(q_0),\varphi(F)}) \subseteq L(\mathcal{A}_{q_0,F})$.*

Proof. Let $u \in \Sigma^*$. For $\delta(q_0, u) \in F$, we have $\mu(\varphi(q_0), u) = \varphi(\delta(q_0, u)) \in \varphi(F)$ and so $u \in L(\mathcal{B}_{\varphi(q_0),\varphi(F)})$. Next, if $\varphi^{-1}(\varphi(F)) = F$ and $u \in L(\mathcal{B}_{\varphi(q_0),\varphi(F)})$, then we get $\varphi(\delta(q_0, u)) = \mu(\varphi(q_0), u) \in \varphi(F)$ and so $\delta(q_0, u) \in F$. □

Theorem 6.55 gives the following corollary to Theorem 6.83.

Corollary 6.84 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton and $\sim$ be a $\mathcal{T}_S(\mathcal{A})$-congruence. Then for each $q \in Q$ and $F \subseteq Q$ we have $L(\mathcal{A}_{q,F}) \subseteq L((\mathcal{A}/{\sim})_{[q]_\sim,[F]_\sim})$ and if F saturates $\sim$, then $L(\mathcal{A}_{q,F}) = L((\mathcal{A}/{\sim})_{[q]_\sim,[F]_\sim})$ (note that $[F]_\sim = \{[q]_\sim \mid q \in F\}$).*

Morphisms as introduced here are a concept defined for semiautomata. Theorem 6.83 gives a way to extend this notions to morphisms for automata as language acceptors by also stipulating that the start state is preserved and the final state set saturates the kernel relation. We do not use such an extended notion of morphisms for automata as language acceptors and thus can use the term morphism alone without any ambiguity here.

6.7.2 The Minimal Automaton

Here, we introduce the Myhill-Nerode relation, the minimal automaton of a given language, the canonical map of an accessible automaton onto the minimal automaton,

and the notion of distinguishability of states. Theorem 6.86 gives a characterization of regularity in terms of the Myhill-Nerode relation and shows that the canonical map is a surjective morphism. Then, in Theorem 6.88, we characterize those automata with the least number of states accepting a given regular language. Theorem 6.91 characterizes those automata having the least number of states for every distribution of a start state and some non-empty and proper set of final states.

Let $L \subseteq \Sigma^*$ be a language.

1. For $u, v \in \Sigma^*$ define the *Myhill-Nerode relation* w.r.t. L [456, 459] by

$$u \equiv_L v \Leftrightarrow u^{-1}L = v^{-1}L.$$

   The relation $\equiv_L$ is an equivalence relation that saturates L and such that, for all $u, v, w \in \Sigma^*$, we have[11]

$$u \equiv_L v \Rightarrow uw \equiv_L vw. \tag{6.24}$$

2. The *minimal automaton (of L)* is the (here possibly infinite) acceptor automaton $\mathrm{MinAut}(L) = (Q_L, \Sigma, \delta_L, [\varepsilon]_{\equiv_L}, F_L)$ with

$$\begin{aligned} Q_L &= \Sigma^*/\equiv_L, \\ \delta_L([u]_{\equiv_L}, v) &= [uv]_{\equiv_L}, \\ F_L &= \{[u]_{\equiv_L} \mid u \in L\} \end{aligned}$$

   where δ_L is well-defined by Equation (6.24). Clearly, $\mathrm{MinAut}(L)$ is accessible.
3. Suppose L is regular and $L = L(\mathcal{A})$ for some accessible automaton $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$. Then, for $u, v \in \Sigma^*$,

$$\delta(q_0, u) = \delta(q_0, v) \Rightarrow u \equiv_L v. \tag{6.25}$$

   Let $\mathrm{MinAut}(L) = (Q_L, \Sigma, \delta_L, [\varepsilon]_{\equiv_L}, F_L)$. By Equation (6.25), the map $\pi_{\mathcal{A}} \colon Q \to Q_L$, called the *canonical map of $\mathcal{A}$ onto* $\mathrm{MinAut}(L)$, given by

$$\pi_{\mathcal{A}}(q) = [u]_{\equiv_L}$$

   for some word $u \in \Sigma^*$ such that $\delta(q_0, u) = q$ is well-defined. So, $\pi_{\mathcal{A}}(\delta(q_0, u)) = [u]_{\equiv_L}$ and $\pi_{\mathcal{A}}$ is surjective.
4. Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be an automaton. Two states $p, q \in Q$ are *distinguishable* (in $\mathcal{A}$) if there exists $u \in \Sigma^*$ such that

$$(\delta(p, u) \in F \wedge \delta(q, u) \notin F) \vee (\delta(p, u) \notin F \wedge \delta(q, u) \in F). \tag{6.26}$$

   We call p and q *indistinguishable* if they are not distinguishable. The indistinguishable relation is a quasiorder.

[11] Every word $w \in \Sigma^*$ can be associated to a unique transformation $\sigma_w \colon \Sigma^* \to \Sigma^*$ given by $\sigma_w(u) = uw$ for $u \in \Sigma^*$. Then $\hat{\Sigma}^* = \{\sigma_w \mid w \in \Sigma^*\}$ gives a transformation monoid on Σ^* and Equation (6.24) states that $\equiv_L$ is a $\hat{\Sigma}^*$-congruence.

Suppose $\sim$ is a $\mathcal{T}_S(\mathcal{A})$-congruence that saturates $F \subseteq Q$ for $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$. Then the following implication holds for $p, q \in Q$:

$$p \sim q \Rightarrow p \text{ and } q \text{ are indistinguishable,} \tag{6.27}$$

i.e., the indistinguishability relation is the coarsest $\mathcal{T}_S(\mathcal{A})$-congruence saturating F. Note that, by Equation (6.26), two states p, q are indistinguishable in $\mathcal{A}$ if and only if

$$\forall u \in \Sigma^* \colon \delta(p, u) \in F \Leftrightarrow \delta(q, u) \in F.$$

We aim to characterize minimality in terms of $\equiv_L$, congruences, morphisms, and the distinguishability relation in Theorem 6.88. But before, we list relations, needed in the proof of Theorem 6.88, between $\equiv_L$, accessible automata $\mathcal{A}$ and $\pi_{\mathcal{A}}$. The next lemma follows readily from the definitions.

Lemma 6.85 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be accessible and $L = L(\mathcal{A})$. Then, for $u, v \in \Sigma^*$, the following are equivalent:*

1. $\pi_{\mathcal{A}}(\delta(q_0, u)) = \pi_{\mathcal{A}}(\delta(q_0, v))$,
2. $u \equiv_L v$,
3. $\delta(q_0, u)$ *and* $\delta(q_0, v)$ *are indistinguishable.*

Next, we summarize the important properties of $\mathrm{MinAut}(L)$ and $\pi_{\mathcal{A}}$.

Theorem 6.86 *Let $L \subseteq \Sigma^*$ and $\mathrm{MinAut}(L) = (Q_L, \Sigma, \delta_L, [\varepsilon]_{\equiv_L}, F_L)$. Then L is regular if and only if Q_L is finite. Further, if L is regular, then for every accessible $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ with $L = L(\mathcal{A})$, the map $\pi_{\mathcal{A}} \colon Q \to Q_L$ is a surjective morphism such that $\pi_{\mathcal{A}}(q_0) = [\varepsilon]_{\equiv_L}$, $\pi_{\mathcal{A}}^{-1}(F_L) = F$ and $\mathrm{MinAut}(L)$ accepts L.*

Proof. First, suppose Q_L is finite. As $\equiv_L$ saturates L, for $u \in \Sigma^*$, we get

$$u \in L \Leftrightarrow \exists v \in L \colon u \equiv_L v \Leftrightarrow [u]_{\equiv_L} \in F_L \Leftrightarrow \delta_L([\varepsilon]_{\equiv_L}, u) \in F_L.$$

So, $\mathrm{MinAut}(L)$ is a finite automaton accepting L. Hence, L is regular. Conversely, suppose L is regular. Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton with $L = L(\mathcal{A})$, which we can assume to be accessible. As $\pi_{\mathcal{A}}$ is surjective, we find that Q_L is finite. Next, we show the remaining claims w.r.t. $\mathcal{A}$. Let $q \in Q$ and $u \in \Sigma^*$ with $\delta(q_0, u) = q$. Then, by the definitions of $\pi_{\mathcal{A}}$ and δ_L, we have, for $v \in \Sigma^*$,

$$\pi_{\mathcal{A}}(\delta(q, v)) = \pi_{\mathcal{A}}(\delta(q_0, uv)) = [uv]_{\equiv_L} = \delta_L([u]_{\equiv_L}, v) = \delta_L(\pi_{\mathcal{A}}(q), v).$$

So, $\pi_{\mathcal{A}}$ is a morphism from $\mathcal{A}$ onto $\mathrm{MinAut}(L)$. Lastly, by the definition of F_L, $\pi_{\mathcal{A}}$ and as $\equiv_L$ saturates L, we have

$$\begin{aligned}
\pi_{\mathcal{A}}(q) \in F_L &\Leftrightarrow \exists u \in L\ \exists v \in \Sigma^* \colon \delta(q_0, v) = q \wedge v \equiv_L u && \text{(Def. } \pi_{\mathcal{A}} \text{ \& } F_L\text{)} \\
&\Leftrightarrow \exists u \in L\ \exists v \in L \colon \delta(q_0, v) = q \wedge v \equiv_L u && (\equiv_L \text{ saturates } L) \\
&\Leftrightarrow \exists v \in L \colon \delta(q_0, v) = q \\
&\Leftrightarrow q \in F.
\end{aligned}$$

Therefore, $\pi_{\mathcal{A}}^{-1}(F_L) = F$. By Theorem 6.83, we find $L(\mathcal{A}) = L(\mathrm{MinAut}(L))$. □

Brzozowski [106] has shown that a language is regular if and only if the set of its left quotients is finite, which follows from Theorem 6.86.

Corollary 6.87 (Brzozowski [106]) *A language $L \subseteq \Sigma^*$ is regular if and only if $\{u^{-1}L \mid u \in \Sigma^*\}$ is finite.*

Proof. By the defining equation of the Myhill-Nerode relation, $[u]_{\equiv} \mapsto u^{-1}L$ is a bijection between Q_L and the different left quotients. Thus, the claim follows from Theorem 6.86. □

Next, we state various characterizations for an automaton to have the least number of states for its accepted language.

Theorem 6.88 *Let $L \subseteq \Sigma^*$ be regular,* $\mathrm{MinAut}(L) = (Q_L, \Sigma, \delta_L, [\varepsilon]_{\equiv_L}, F_L)$ *and $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ with $L = L(\mathcal{A})$. Then the following are equivalent.*

1. *$\mathcal{A}$ has the least number of states such that $L = L(\mathcal{A})$.*
2. *$\mathcal{A}$ is accessible and there exists no automaton $\mathcal{B} = (S, \Sigma, \mu, s_0, E)$ and a morphism $\varphi\colon Q \to S$ with $\varphi(q_0) = s_0$ and $\varphi^{-1}(E) = F$ such that $|\varphi(Q)| < |Q|$.*
3. *$\mathcal{A}$ is accessible and $\pi_{\mathcal{A}}\colon Q \to Q_L$ is an isomorphism of $\mathcal{A}$ onto* $\mathrm{MinAut}(L)$.
4. *$\mathcal{A}$ is accessible and all pairs of distinct states are distinguishable.*
5. *$\mathcal{A}$ is accessible and the equality relation is the only $\mathcal{T}_S(\mathcal{A})$-congruence saturating F.*

Proof. 1 ⇒ 2: Set $T = \uparrow\{q_0\}$. The automaton $(\mathcal{A}_{|T})_{q_0, T\cap F}$ is accessible and we have $L((\mathcal{A}_{|T})_{q_0,T\cap F}) = L(\mathcal{A})$. By assumption, we find $T = Q$. So, $\mathcal{A}$ is accessible. Next, let $\mathcal{B}$ and φ be as in the statement. We can assume φ is surjective, i.e., $S = \varphi(Q)$. So, $\varphi(F) = \varphi(\varphi^{-1}(E)) = E$. Furthermore, we have

$$\varphi^{-1}(\varphi(F)) = \varphi^{-1}(\varphi(\varphi^{-1}(E))) = \varphi^{-1}(E) = F.$$

Then, by Theorem 6.83, we find $L(\mathcal{A}) = L(\mathcal{B})$. Since $\mathcal{A}$ has the least number of states among all automata accepting $L(\mathcal{A})$, we get $|S| = |Q|$.

2 ⇒ 3: By Theorem 6.86 we have $\pi_{\mathcal{A}}(q_0) = [\varepsilon]_{\equiv_L}$, $\pi_{\mathcal{A}}^{-1}(F_L) = F$ and $\pi_{\mathcal{A}}$ is surjective so that $|Q_L| \leq |Q|$. By assumption, we get $|Q| = |Q_L|$ and so $\pi_{\mathcal{A}}$ is bijective.

3 ⇒ 4: The automaton $\mathrm{MinAut}(L)$ is accessible and as $\mathrm{MinAut}(L) \cong \mathcal{A}$, also $\mathcal{A}$ is accessible. Let $p, q \in Q$ be two distinct states. As $\pi_{\mathcal{A}}$ is injective, we have $\pi_{\mathcal{A}}(p) \neq \pi_{\mathcal{A}}(q)$. By Theorem 6.85, it follows that p and q are distinguishable.

4 ⇒ 5: Suppose $\sim$ is some $\mathcal{T}_S(\mathcal{A})$-congruence that saturates F. As any pair of distinct states $p, q \in Q$ is distinguishable, we find, by Formula (6.27), that $p \nsim q$. So $\sim$ equals the equality relation.

5 ⇒ 1 : Suppose there exists an accessible automaton $\mathcal{B} = (S, \Sigma, \mu, s_0, E)$ with $|S| < |Q|$ and $L = L(\mathcal{B})$. By Theorem 6.86, as $\pi_{\mathcal{B}}$ is surjective, we have $|Q_L| \leq |S|$. Thus $|Q_L| < |Q|$ and so $\pi_{\mathcal{A}}$ maps to distinct states to one state in $\mathrm{MinAut}(L)$, giving that $\sim_{\pi_{\mathcal{A}}}$ does not equal the equality relation. By Theorems 6.56 and 6.86, $\sim_{\pi_{\mathcal{A}}}$ is a $\mathcal{T}_M(\mathcal{A})$-congruence, which, as $\pi_{\mathcal{A}}^{-1}(F_L) = F$, saturates F. □

For regular $L \subseteq \Sigma^*$, we call $\min\{|Q| : \mathcal{A} = (Q, \Sigma, \delta, q_0, F) \text{ with } L = L(\mathcal{A})\}$ the *state complexity* of L, which equals, by Theorem 6.88, Item 3, the number of states of $\mathrm{MinAut}(L)$. Furthermore, all automata with the least number of states accepting L are isomorphic by Theorem 6.88, Item 3, which justifies calling $\mathrm{MinAut}(L)$ the minimal automaton. In view of this, we also call $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ *minimal* if $\pi_{\mathcal{A}}$ is an isomorphism. Note that two non-coaccessible states are indistinguishable. So, a minimal automaton has at most one state that is not coaccessible and this state must be a trap state.

Remark 6.89 Since the indistinguishability relation is the coarsest $\mathcal{T}_S(\mathcal{A})$-congruence saturating F, by Theorem 6.88, Item 5 the automaton $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ is minimal if and only if it is accessible and the indistinguishability relation equals the equality relation.

Remark 6.90 Accessibility in Items 2,4 and 5 of Theorem 6.88 is crucial. For example, in an automaton $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ whose state set forms two cycles of size at least two induced by different letters a and b that induce self-loops on the other cycle and with one final state on each cycle, all pairs of distinct states are distinguishable, equality is the only $\mathcal{T}_S(\mathcal{A})$-congruence saturating F, and there exists no morphism as stated in Item 2. This follows by considering $\mathcal{A}_{|\{a\}}$ it either permutes the states on the a-cycle or maps all those states to a single state outside the a-cycle. But since this is also a morphism of $\mathcal{A}_{|\{b\}}$, it must permute the full b-cycle if it maps outside the a-cycle. The same argument for the b-cycle gives the it has to permute the a-cycle and the b-cycle. Thus it is an automorphism.

Davies [171, Lem. 3.7] (see also [172, Lem. 4.2.8]) proved that an automaton with a non-empty and proper final state set is minimal if and only if it admits no non-trivial $\mathcal{T}_M(\mathcal{A})$-congruence saturating the final state set. This readily follows from Theorem 6.88, Item 5, as the final state set is empty or equals the whole state set if and only if the accepted language is empty or equals Σ^*, respectively. Further, she also characterized strongly connected automata that are minimal for every start state and proper non-empty set of final states by the absence of non-trivial $\mathcal{T}_S(\mathcal{A})$-congruences [171, Prop. 4.2.3] (or [172, Prop. 3.2]). Next, we state this together with other characterizations as a corollary to Theorem 6.88.

Note that for strongly connected $\mathcal{A} = (Q, \Sigma, \delta)$ with at least two states, $L(\mathcal{A}_{q,F})$ is a non-empty proper subset of Σ^* for $q \in Q$ and non-empty and proper $F \subsetneq Q$ since $\delta(q, u) \in F$ and $\delta(q, v) \notin F$ for some $u, v \in \Sigma^*$ by accessibility and strong connectedness.

Corollary 6.91 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}_{q,F}$ is minimal for each $q \in Q$ and non-empty and proper $F \subsetneq Q$.*
2. *$\mathcal{A}$ is strongly connected and all pairs of distinct states are distinguishable in $\mathcal{A}_{q,F}$ for all $q \in Q$ and $\emptyset \neq F \subsetneq Q$.*
3. *$\mathcal{A}$ is strongly connected and there exists no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence.*
4. *$\mathcal{A}$ is strongly connected and there exists no $\mathcal{B} = (S, \Sigma, \mu)$ with a morphism $\varphi\colon Q \to S$ such that $1 < |\varphi(Q)| < |Q|$.*

Proof. 1 $\Rightarrow$ 2: Suppose $\mathcal{A}_{q,F}$ is minimal for each $q \in Q$ and $\emptyset \neq F \subsetneq Q$. By Theorem 6.88, Item 4, all pairs of distinct states are distinguishable in $\mathcal{A}_{q,F}$. Fix some $\emptyset \neq F \subsetneq Q$. By Theorem 6.88, Item 4, $\mathcal{A}_{q,F}$ is accessible w.r.t. each state q. Now, by Theorem 3.43, Item 4, $\mathcal{A}_{q,F}$ being accessible for any $q \in Q$ is equivalent to $\mathcal{A}$ being strongly connected. Alternatively, let $S \subseteq Q$ be a sink component and $q \in S$. As the states in $Q \setminus S$ are not accessible in $\mathcal{A}_{q,S}$, we must have $Q = S$ since minimality implies that all states are accessible by Theorem 6.88. So $\mathcal{A}$ is strongly connected.

2 $\Rightarrow$ 3: This follows from Equation (6.27).

3 $\Rightarrow$ 4: This follows from Theorem 6.56, Item 2.

4 $\Rightarrow$ 1: Let $q \in Q$ and $\emptyset \neq F \subsetneq Q$. Since $\mathcal{A}$ is strongly connected, $\mathcal{A}_{q,F}$ is accessible. As there exists no morphism onto an automaton with less states, by Theorem 6.88, Item 2, $\mathcal{A}_{q,F}$ is minimal. □

Note that the equivalences from Theorem 6.91 also hold for $|Q| = 1$ vacuously since then there exists no proper and non-empty final state set and no morphism φ with $1 < |\varphi(Q)| < |Q|$. A one-state automaton only accepts $\emptyset$ or Σ^*.

Theorem 6.68 stated that an automaton admitting no non-trivial $\mathcal{T}_M(\mathcal{A})$-congruence is either strongly connected or the state set is partitioned into a trap state and a strongly connected component. Theorem 6.91 gives more details in case the automaton is strongly connected and admits no non-trivial $\mathcal{T}_M(\mathcal{A})$-congruence.

Remark 6.92 Strong connectivity is necessary in Items 2,3, and 4 of Theorem 6.91. For example, consider $\mathcal{A} = (\{0, 1, \ldots, n\}, \{a, b\}, \delta)$ with $\delta_a = (0)(1\ 2\ \cdots n)$ and $\delta_b = [1 \mapsto 0]$ for some number $n > 1$. Then $\mathcal{A}_{0,\{0\}}$ is not minimal. However, we have the following:

1. All pairs of distinct states are distinguishable in $\mathcal{A}_{q,F}$ for each $q \in Q$ and non-empty and proper $F \subseteq Q$, for we can move one state to 0 while the other remains on the cycle, and then can be moved to a state that is final if $0 \notin F$ or not final if $0 \in F$.
2. The automaton admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence. For suppose $\sim$ is a $\mathcal{T}_S(\mathcal{A})$-congruence such that $p \sim q$ for $p, q \in Q$ with $p < q$. Then $\delta(p, b^{n-p+1}a) = 0$, $\delta(q, b^{n-p+1}a) \neq 0$ and thus $\delta(q, b^{n-p+1}a) \sim 0$. This yields $\delta(q, b^{n-p+1}ab^i) \sim 0$ for each $i \geq 0$, and since $p = \delta(q, b^{n-p+1}ab^i)$ for some $i \geq 0$ for each $p \in Q \setminus \{0\}$, we find that all pairs of states are congruent w.r.t. $\sim$.
3. There exists no morphism to a strictly smaller automaton with at least two states.

So, each condition except strong connectivity is fulfilled.

Note that any automaton $\mathcal{A}$ with at most two states, even those that are not strongly connected, admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence, no non-trivial morphic images, and has the property that distinct states are indistinguishable in $\mathcal{A}_{q,F}$ for $q \in Q$ and $\emptyset \neq F \subsetneq Q$.

Restivo & Vaglica [508, 509] also investigated automata that are minimal for all k-sets of final states for some $k \geq 1$ or for all non-empty and proper subsets of final states.

6.8 Remarks on Transformation Semigroups and Automata

To every (complete and deterministic) semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$ corresponds its transformation semigroup $\mathcal{T}_S(\mathcal{A})$ with generating set $\{\delta_a \colon Q \to Q \mid a \in \Sigma\}$. Conversely, let $S \leq_S \mathcal{T}_n$ be a transformation semigroup generated by $g_1, \ldots, g_m$. Then for $\Sigma = \{g_1, \ldots, g_n\}$ (viewing the generating sequence as a set of abstract symbols) we obtain a semiautomaton $\mathcal{A} = ([n], \Sigma, \delta)$ with $\mathcal{T}_S(\mathcal{A}) = S$ by setting $\delta(q, g_i) = g_i(q)$ for $q \in [n]$ and $i \in \{1, \ldots, m\}$.[12] Thus, a semiautomaton may be viewed as a transformation semigroup together with a distinguished generating sequence.

Example 6.93 The choice of a generating set for a given transformation semigroup $S \leq_S \mathcal{T}_n$ can lead to quite distinct automata. For example, consider the permutation group $G = \langle g \rangle_G$ with $g = (0\,1\,\cdots\,5)$. When constructing the automaton with generating set $\{g\}$ we obtain an automaton that cyclically permutes its states. Another generating set is given by $g^2 = (0\ 2\ 4)(1\ 3\ 5)$ and $g^3 = (0\ 3)(1\ 4)(2\ 5)$ since $g^2 g^2 g^3 = g$. The automaton we obtain from this generating set is depicted in Figure 6.7.

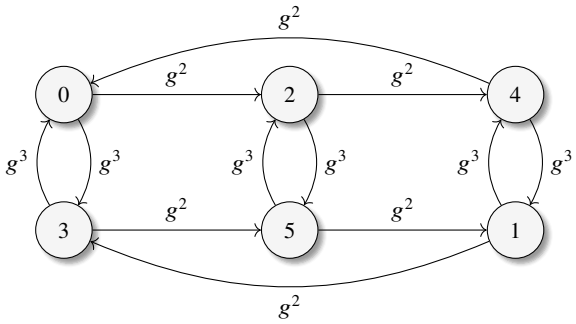


Fig. 6.7: Automaton constructed from permutation group, see Theorem 6.93.

Many of our notions and results can be formulated either for automata or for transformation semigroups. For example, $\mathcal{A}$ is synchronizing if and only if $\mathcal{T}_S(\mathcal{A})$ contains a constant transformation.

Our results are motivated by automata-theoretical problems, but properties only related to subset mapping properties do not depend on the choice of some generating set and are actually properties of transformation semigroups. So, these properties are better expressed in an algebraic setting with transformation semigroups, as done in Part III. The corresponding automata-theoretic reformulations are usually straightforward and are therefore omitted.

A semiautomaton is a machine model that processes, or reacts to, inputs. In this setting, the alphabet is crucial, as it is the means by which the machine interacts with its environment. Since inputs correspond to physical actions and an operator may want

[12] A transformation g_i is allowed to appear more than once among $g_1, \ldots, g_m$.

to minimize the effort to achieve a given goal, it is natural to ask for the minimum number of such actions required to do this. Thus, subset mapping properties with length bounds on shortest words realizing some property are naturally expressed in an automata-theoretic formulation.

Later in Part IV, we use automata-theoretic notation exclusively.

We note that there is a generalization of the Černý conjecture for transformation monoids [22, 524]. Shortest words in some generating sets have also been investigated in the context of the diameter of a Cayley graph depends [256].

6.9 Discussion

Some material on automata is presented differently here or is not covered in standard texts, such as [322, 339, 388]. Specifically, the various facts on semiautomata morphisms from Section 6.4 appear to be scattered in the literature or, as far as we know, not stated explicitly in the form presented here. Moreover, the isomorphisms in Theorem 6.66 concerning Rees quotient automata and Theorem 6.67 are, to the best of our knowledge, new. Automata can also be viewed as unary algebras with finitely many operations on a finite domain. Results from automata theory and formal language theory have been extended to unary algebras in [480]; see also [617, p. 835] and [97] for connections between unary algebras, their identities, and synchronizing automata.

Overview on Automata Theory.

The theory of finite automata as an independent field of study is generally considered to have started with the seminal book *Automata Studies* [564]. Medvedev translated this book into Russian and added his own article [450], in which he introduced semiautomata (therefore sometimes referred to as *Medvedev automata*, see [547, 617]). Early work in automata theory was primarily concerned with the processing of events, signals, and languages by automata rather than with automata viewed merely as finite transition systems. While models of computation were initially developed in the 1930s to investigate the limits of computability, the focus later shifted to modeling the operation of nervous systems using electronic components. See, for example, [460, 563] for surveys of the work and motivations of the 1940s and 1950s (where automata is also used in a broader sense than here). Synchronization is a concept tied to the transition structure of an automaton, or more generally a machine, and thus is a semiautomaton concept.[13]

Fleck [218, 219] introduced morphisms on automata and initiated a structure theory for automata centered on their automorphism groups. For example, among the results obtained in this line of research are the following.

[13] In [320] the notion of synchronization was transferred to languages via the minimal automaton by calling a language synchronizing if there exists a word always ending in the same state in the (possibly infinite) minimal automaton of the language.

1. The automorphism group of a strongly connected automaton is a semiregular permutation group [219, 631] (here stated in Theorem 6.43).[14]
2. Let $\mathcal{A} = (\{q_1, \ldots, q_n\}, \Sigma, \delta)$ be strongly connected. In [631], for $i \in \{1, \ldots, n\}$, an equivalence relation $\sim_i$ on Σ^* was introduced where, for $u, v \in \Sigma^*$,

$$u \sim_i v \quad \text{if and only if} \quad \delta(q_i, u) = \delta(q_i, v).$$

   Furthermore, in [631, Def. 5], the quotient $\Sigma^*/\sim_i$ was equipped with a semigroup structure $(\Sigma^*/\sim_i, *_i)$ by setting $[u]_{\sim_i} *_i [v]_{\sim_i} = [w]_{\sim_i}$ precisely if $\delta(q_i, u'v') = \delta(q_i, w)$ for all $u', v' \in \Sigma^*$ with $u' \sim_i u$ and $v' \sim_i v$. Barnes [58, 59] used these semigroups to characterize the automorphism group of $\mathcal{A}$. For instance, this semigroup structure is isomorphic to the automorphism group whenever the latter is regular, i.e. transitive [58, Cor. 1 and Thms. 3 & 4].

The investigation of automata via their automorphism groups was continued in the following works: [58, 59, 66, 67, 219, 220, 266, 267, 268, 443, 469, 632, 633].

A *monoid action* or *(right) M-act* [367] is a tuple (X, M, μ) where X is some set, M a monoid with identity $1 \in M$, and $\mu\colon X \times M \to X$ a function such that

1. $\mu(x, 1) = x$ and
2. $\mu(x, st) = \mu(\mu(x, s), t)$ for $x \in X$ and $s, t \in M$.

A morphism of two M-acts (X, M, μ) and (Y, M, ρ) is a map $\varphi\colon X \to Y$ such that $\varphi(\mu(x, s)) = \rho(\varphi(x), s)$ for all $x \in X$ and $s \in M$. The morphism is called an endomorphism of the M-act if $X = Y$. Via their extensions to Σ^*, semiautomata are precisely the Σ^*-acts, and morphisms of semiautomata coincide with the morphisms of these Σ^*-acts (see also [367, Prop. 4.5]). Hence, the study of semiautomata may be viewed as the study of finite Σ^*-acts. For a group G, a G-act is also called a group action, G-action, or a G-set [126, 179]. Isomorphisms of G-acts are called representation equivalences in [179, p. 21]. The notion of G-acts also arises in algebra, topology, and geometry [467].

Remark 6.94 An M-act can also be identified with a monoid morphism from M to $\mathcal{T}_X$ equipped with the left-most-first composition operation ; from Section 3.11 as its operation, in the following way. Given an M-act (X, M, μ), the map $s \mapsto \mu_s$ for $s \in M$ with $\mu_s(x) = \mu(x, s)$ for $x \in X$ gives such a morphism. Conversely, given a morphism $\varphi\colon M \to \mathcal{T}_X$ of $(M, \cdot)$ to $(T_X, ;)$, defining $\mu\colon X \times M$ by $\mu(x, s) = \varphi(s)(x)$ yields an M-act (X, M, μ). For example, every transformation monoid $M \leq_M \mathcal{T}_X$ can be considered as an M-act by applying this construction to the identity transformation on M.

In this setting, semiautomata can be seen as morphisms $\Sigma^* \to \mathcal{T}_Q$. The image of such a morphism is the transformation monoid. This viewpoint was used in [534]. Since morphisms from Σ^* are uniquely determined by their images on Σ, it suffices to specify a semiautomaton by a map $\Sigma \to \mathcal{T}_Q$ and Theorem 3.60 then yields the extension to all of Σ^*.

[14] In [219, 631], semiregularity is referred to simply as "regular". We follow the terminology that is now standard [126, 179, 638].

The notion of an $\mathcal{T}_S(\mathcal{A})$-congruence was first considered by Hartmanis [279] (therein called partitions with the substitution property) in connection with series-parallel decompositions of automata. Slightly earlier, Rabin and Scott [502] had considered the closely related concept of right-invariant congruences on Σ^*. The notion of a quotient automaton is implicit in Hartmanis's work, but was formally introduced for the first time by Bayer [68]. However, he defined it [68, Def. 9] only when $\mathcal{T}_S(\mathcal{A})$ is a group. One year later, Bayer [69] gave the general definition and related the lattice of $\mathcal{T}_S(\mathcal{A})$-congruences to endomorphisms of general automata. Thierrin [599] and Babcsányi [51] investigated primitive automata $\mathcal{A}$ (therein called *simple automata*). For example, Theorem 6.68 is due to Thierrin [599, Prop. 8]. Primitivity has been linked to minimality conditions for automata in [171, 172, 508, 509].

In the context of coding theory and code parsing, Schützenberger [551, Sect. 5 & 6] introduced the syntactic semigroup of a code (which he called the *fundamental semigroup*) and represented its elements using matrices. His construction in fact yields the transition matrices of the minimal automaton of the code. Although automata are not mentioned explicitly, this relation to the minimal automaton shows that his work can be viewed as the first investigation of the transformation semigroup of an automaton. An early work that explicitly introduced transition matrices of automata is the work by Burks and Wang [116]. Closely related stochastic transition matrices had, however, already been used decades earlier in the theory of Markov chains.

The congruences $\sim_i$ from Weeg [631, Def. 5], discussed above, are related to the transformation semigroup as follows. Let $\sim$ be the equivalence relation on Σ^* defined by

$$u \sim v \quad \Longleftrightarrow \quad u \sim_i v \text{ for all } i \in \{1, \ldots, n\}.$$

Then $u \sim v$ if and only if $\delta(q_i, u) = \delta(q_i, v)$ for all $i \in \{1, \ldots, n\}$. Moreover, if $u \sim v$, then $wu \sim wv$ and $uw \sim vw$ for every $w \in \Sigma^*$. Thus, for the product on subsets we have

$$[u]_\sim [v]_\sim = [uv]_\sim,$$

and so we obtain a well-defined operation on $\Sigma^*/\sim$, making it into a semigroup. Now the map $[u]_\sim \mapsto \delta_u$ is an isomorphism onto the transformation semigroup.

In the early days, automata were considered as *(sequential) transducers* (or *automata with output*), see [322, 548] for details. A notion related to synchronization in this setting is that of a *homing sequence* for a transducer automaton model. Such a sequence is an input word whose produced outputs uniquely determine the state reached by the automaton. For more information, we refer to the survey by Sandberg [548], which discusses homing sequences and their relation to synchronizing sequences.

The seminal paper by Rabin and Scott [502] is widely regarded as the first to systematically consider finite automata as language acceptors.[15] They also introduced nondeterministic automata and the power-set construction and established much of the standard terminology and notation. Nevertheless, it was not the first paper to relate finite automata to language acceptance. Shannon [561] investigated them as language-generating devices in the context of coding theory and Chomsky [148, 150] studied them

[15] Behind the Iron Curtain, however, automata had already been defined as language acceptors by Medvedev [450].

in the context of the processing, understanding, and production of natural languages by humans.[16] Furthermore, building on the ideas of McCulloch and Pitts [446], several authors, for example Burks and Wang [115, 116] and Kleene [369], investigated finite automata as models of nerve nets. Kleene [369] introduced regular expressions in this context.

The characterization of minimal automata and the Myhill–Nerode relation go back to Nerode [459] (input-output version) and Myhill [456] (automata/semigroup version). Regular languages have been classified by algebraic properties of the transformation semigroups of minimal accepting automata via the theory of (pseudo-)varieties of languages and semigroups [195, 196, 489]. Seminal results in this direction include the characterization of star-free languages by counter-free automata [449] and aperiodic monoids [552], see [194, 195, 489, 490, 491] for more information.

Krohn and Rhodes [389] developed the *Krohn–Rhodes decomposition theory*, a decomposition theory for automata. In this theory, automata are represented as homomorphic images of certain basic permutation automata and aperiodic automata related to the given automaton. The theory identifies basic building blocks from which every automaton can be simulated by an appropriate product of these basic automata. There are several formulations of this theory:

1. The original semigroup-theoretic formulation from [389] and [39, Chap. 5] uses the wreath product. Other formulations based on the same notions were given in [400] and [201]. Textbook expositions close to the original formulation can be found in [511, 512]. The monograph [511] also discusses broader applications to biology, physics, and psychology, together with some philosophical reflections.
2. A formulation in terms of the cascade product was developed in [247, 645, 646] and [39, Chap. 4]. In this formulation, every automaton is covered[17] by a cascade product of certain permutation-reset automata. Ginzburg [247] gives a correction to the proof in [645, 646]. We also refer to the expository article [649].
3. Meyer and Thompson's [121] formulation in terms of cascade products connects the theory to an earlier decomposition result of Hartmanis and Stearns [285].
4. Based on Zeiger's approach [645, 646], Eilenberg [195] gave an elegant formulation in the language of transformation semigroups, termed the *holonomy decomposition*, see also [317]. Egri-Nagy and Nehaniv [181, 190, 191, 192, 193] gave an automata-theoretic reformulation of the holonomy decomposition and studied its complexity.
5. Diekert, Kufleitner, and Steinberg [176] gave an algebraic formulation utilizing the notion of local divisors.

In the special case of groups, the Krohn–Rhodes theorem reduces to the *(Krasner–Kaloujnine) universal embedding theorem* [352], see [179, Sect. 2.6] for a textbook exposition of this special case.

Maler and Pnueli [431] carried out an in-depth study of the size of the Krohn–Rhodes decomposition and used their results to obtain an improved algorithm for translating counter-free automata into past temporal logic formulae. A corresponding bound for

[16] Chomsky concluded that regular languages are not sufficiently expressive to model natural languages adequately and therefore introduced more powerful grammar formalisms in [148].

[17] Cover maps are related to morphisms.

future linear temporal logic (LTL) was later established in [98]. We refer to [430, 432] for further discussion and pointers to the literature. Results on the relationship between temporal logics and automata can be found in [155].

The survey by Glushkov [248] also gives a good overview of the early developments in automata theory, and includes also work carried out behind the Iron Curtain.

More on Automata Morphisms.

In the transformation semigroup viewpoint, the endomorphism semigroup corresponds to the centralizer of a transformation semigroup, that is, the set of transformations commuting with every transformation in the semigroup. The centralizer of a single transformation has been characterized in [37]; we also refer to this work for further pointers to the literature. For permutation groups, the centralizer of a single permutation in the full symmetric group is described in [589, pp. 295], [637, Thm. 15.2], and [365], see also the generalization in [414]. It is a folklore result in permutation group theory that the centralizer in the symmetric group of a transitive (resp. semiregular) permutation group is semiregular (resp. transitive). Consequently, a regular permutation group on a finite set has the same order as its centralizer and coincides with its centralizer when it is abelian. That the centralizer of a transitive permutation group is semiregular also follows from Theorem 6.43.

Theorems 6.30 and 6.31 state that morphisms between strongly trap-connected automata are either full isomorphisms or map all states to a single trap state. This is somehow reminiscent of the well-known Schur's lemma from ring theory [405], see also a related remark from [534, p. 275].

In our definition of morphisms, the alphabet was fixed. We can generalize this by including a mapping $\alpha\colon \Sigma \to \Gamma$ of alphabets in the definition and let $\varphi\colon Q \to S$ be a morphism between $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Gamma, \mu)$ if

$$\varphi(\delta(q, x)) = \mu(\varphi(q), \alpha(x))$$

for all $q \in Q$ and $x \in \Sigma$ (a definition in this direction was given in [637]).

Given such a morphism that is surjective, let $F\colon \mathcal{T}_M(\mathcal{A}) \to \mathcal{T}_M(\mathcal{B})$ be given by $F(\delta_u) = \mu_{\alpha(u)}$ for $u \in \Sigma^*$ with α extended to a mapping (also denoted by α) from Σ^* into Γ^* via Theorem 3.60. This is well-defined since for $u, v \in \Sigma^*$ with $\delta_u = \delta_v$, by choosing for $s \in S$ some $q \in Q$ with $s = \varphi(q)$ we have

$$\mu_{\alpha(u)}(s) = \mu(\varphi(q), \alpha(u)) = \varphi(\delta(q, u)) = \varphi(\delta(q, v)) = \mu(\varphi(q), \alpha(v)) = \mu_{\alpha(v)}(s)$$

Then we have the following:

1. F is a monoid morphism since

$$F(\delta_v \delta_u) = F(\delta_{uv}) = \mu_{\alpha uv} = \mu_{\alpha(v)} \mu_{\alpha(u)} = F(\delta_v) F(\delta_u)$$

2. For $t \in \mathcal{T}_M(\mathcal{A})$ with $t = \delta_u$ for $u \in \Sigma^*$ we have

$$F(t)(\varphi(q)) = F(\delta_u)(\varphi(q)) = \mu(\varphi(q), u) = \varphi(\delta(q, u)) = \varphi(\delta_u(q)) = \varphi(t(q))$$

for $q \in Q$.

This is related to the notion of permutation isomorphism from [179, p. 17] which is defined between two permutation groups $G \leq_G \mathcal{S}_X$ and $H \leq_G \mathcal{S}_Y$ as a bijection $\lambda\colon X \to Y$ with a group isomorphism $\psi\colon G \to H$ such that $\lambda(g(p)) = \psi(g)(\lambda(p))$ for all $p \in X$ and $g \in G$.

Reachability Quasiorder.

The reachability relation appears to have been introduced independently by Meyer and Thompson [121], Brzozowski and Fich [111], and Letichevsky [408]; it was later used in [340]. Brzozowski and Fich [111] proved that a language is accepted by a partially ordered automaton if and only if Green's $\mathcal{R}$-relation [261] on its transition semigroup is the equality relation (this was also proved independently in [155, Thm. 5.1]). We obtain the $\mathcal{R}$-relation as the equivalence relation induced by the quasiorder $\leq_{\mathcal{R}}$ on $\mathcal{T}_S(\mathcal{A})$ given by

$$\delta_u \leq_{\mathcal{R}} \delta_v \Leftrightarrow \exists w \in \Sigma^*\ \forall q \in Q : \delta(q, u) = \delta(q, vw)$$

for $\mathcal{A} = (Q, \Sigma, \delta)$ and $u, v \in \Sigma^*$. Clearly, if v is a prefix of u, then $\delta_u \leq_{\mathcal{R}} \delta_v$. Furthermore, $\delta_u \leq_{\mathcal{R}} \delta_v$ implies $\delta(q, v) \preceq_R \delta(q, u)$ for every $q \in Q$. Consequently,

$$\delta_u \mathcal{R} \delta_v \Rightarrow \forall q \in Q\colon \delta(q, u) \simeq_R \delta(q, v). \tag{6.28}$$

It follows from Equation (6.28) that, for a partially ordered automaton, the $\mathcal{R}$-relation is the equality relation. Conversely, let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be such that the $\mathcal{R}$-relation on $\mathcal{T}_S(\mathcal{A})$ is the equality relation. Then the automaton $\mathcal{B} = (\mathcal{T}_S(\mathcal{A}), \Sigma, \mu, \delta_\varepsilon, E)$, where $\mu(\delta_u, a) = \delta_{ua}$ and $E = \{\delta_u \mid \delta_u(q_0) \in F\}$, is partially ordered and satisfies $L(\mathcal{A}) = L(\mathcal{B})$.[18] This gives a short proof of the characterization due to Brzozowski and Fich [111].

The notion of an upper set (for the reachability quasiorder) appears under the name *subautomaton* in [534]. It also appears in other contexts, for example in the notion of coverage of two automata introduced in [113]. As mentioned in the main text, the Rees congruence goes back to a related congruence introduced by Rees [505] for semigroups.

The notion of a sink component (and a corresponding dual notion) can actually be defined for any quasiordered set, and the results from Section 6.3 transfer to this more general setting. Furthermore, reachability is a property of the underlying automaton digraph, so the reachability order can also be defined in the context of digraphs. Similarly, the notions of strong and unilateral connectivity in digraphs correspond to these notions in automata, and a strongly connected component in an automaton is precisely a strongly connected component of its underlying automaton digraph. However, as we need these

[18] Alternatively, let $p, q \in Q$ satisfy $\delta(p, u) = q$ and $\delta(q, v) = p$ for some $u, v \in \Sigma^*$. Set $w = uv$. Since $\mathcal{A}$ is finite, $\delta_{w^i} = \delta_{w^{i+p}}$ for some $i \geq 0$ and $p > 0$. This implies $\delta_{w^i u} \mathcal{R} \delta_{w^i}$. By assumption, $\delta_{w^i u} = \delta_{w^i}$. Therefore, $q = \delta(p, (uv)^i u) = \delta(p, (uv)^i) = p$, showing that $\mathcal{A}$ is partially ordered. In fact, this proves that $\leq_{\mathcal{R}}$ is a partial order if and only if $\preceq_R$ is a partial order.

concepts only in the context of automata, we introduce them here for automata and rather than for general quasiorders and digraphs.

Chapter 7
Classes of Automata

Here, we discuss the following classes of automata that appear throughout the present work and collect results of relevance to us:

1. synchronizing automata (Section 7.1) and synchronizing automata with a trap state (Section 7.1.3),
2. unary automata (Section 7.2),
3. automata with one-point singular idempotents (OI-automata) and automata with simple idempotents (SI-automata) (Section 7.3),
4. permutation-reset automata (Section 7.4),
5. monotonic automata (Section 7.5),
6. automata whose strongly connected components are blocks (Section 7.6),
7. partially ordered automata (Section 7.7),
8. ρ-automata (Section 7.8),
9. commutative automata (Section 7.9),
10. solvable automata (Section 7.10), which generalize commutative and partially ordered automata.

See Figure 7.1 for a Hasse diagram of the inclusions that hold among these classes. The following content is new and did not appear before in the literature:

1. the novel class of ρ-automata,
2. the results on automata whose strongly connected components are blocks,
3. the structural results on solvable automata and the result that a solvable automaton admitting a word of rank r has a word of rank at most r and of length at most $n - r$ and
4. new structural results on solvable automata.

The remaining results presented are known and taken from the literature (maybe presented a little differently).

Other classes of automata, for example circular automata, completely reachable, completely distinguishable or sync-maximal automata are only used in Part III and so not introduced here but separately in Part III.

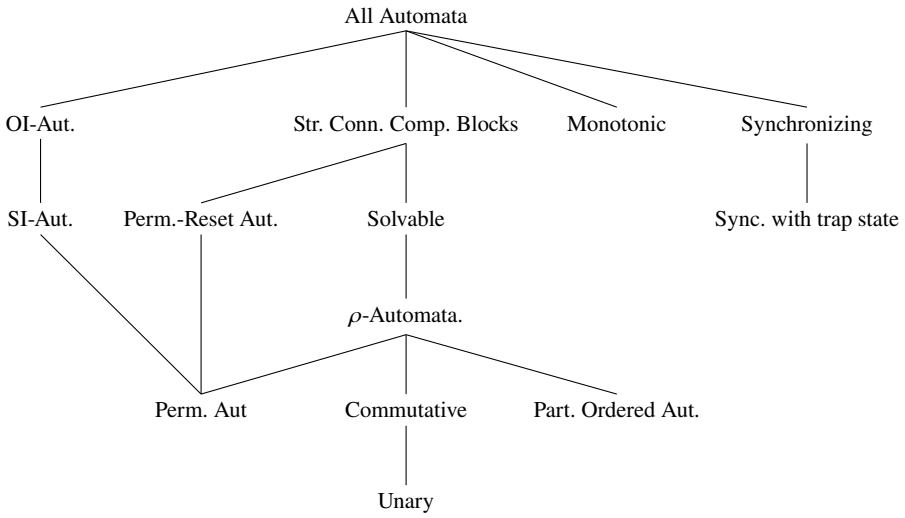


Fig. 7.1: Inclusion relations among the classes of automata from the present section.

7.1 Synchronizing Automata

Synchronizing automata were already defined in Section 1.1. Here, we collect further properties that we need in later chapters. Section 7.1.1 states that the set of synchronizing words is an ideal language. Section 7.1.3 investigates synchronizing automata with a trap state. Section 7.1.2 relates synchronizing words to subautomata, morphic images, quotient automata and vice versa. Section 7.1.4 investigates the set of maximal states in a synchronizing automaton. Section 7.1.5 expresses synchronization in terms of pairs of states. Lastly, Section 7.1.6 collects algorithmic results related to practical implementations (results from a more theoretical perspective in the framework of computational complexity theory are collected in Chapter 8 later).

7.1.1 Sets of Synchronizing Words and Ideal Languages

The set of synchronizing words of a semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$ is a regular ideal language: it is an ideal, since $\delta(Q, uwv) \subseteq \delta(\delta(Q, w), v)$ is a singleton whenever $\delta(Q, w)$ is, and it is regular, since $\mathrm{Syn}(\mathcal{A}) = L(\mathcal{P}(\mathcal{A})_{Q,\mathcal{F}})$ with $\mathcal{F} = \{\{q\} \mid q \in Q\}$.

Next, suppose $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ is minimal and accepts a non-empty ideal language. Then, as observed by Maslennikova [440, 441],

$$L(\mathcal{A}) = \mathrm{Syn}(\mathcal{A}). \tag{7.1}$$

The assumption $L(\mathcal{A}) \neq \emptyset$ cannot be dropped: the minimal automaton for $\emptyset$ is $(\{q\}, \Sigma, \delta, q, \emptyset)$ with $\delta(q, x) = q$, which is synchronizing with $\mathrm{Syn}(\mathcal{A}) = \Sigma^*$. Of course, $\emptyset$ is still the set of synchronizing words of any non-synchronizing semiautomaton, and so we obtain the following characterization.

Theorem 7.1 *Let $I \subseteq \Sigma^*$. Then I is a regular ideal language if and only if $I = \mathrm{Syn}(\mathcal{A})$ for some finite semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$.*

By Theorem 7.1, investigating acceptors for sets of synchronizing words, as we do in Part III, amounts to investigating acceptors for regular ideal languages. In the following, we use the fact that $\mathrm{Syn}(\mathcal{A})$ is an ideal language without further mention.

As a shortest word in a regular language accepted by an n-state automaton has length at most $n - 1$, Equation (7.1) yields the following bound.

Proposition 7.2 *If $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ is a minimal n-state automaton accepting an ideal language, then $\mathcal{A}$ is synchronizing with reset length at most $n - 1$.*

For $i \geq 1$, the minimal automaton for $\Sigma^* a^i \Sigma^*$ has $i + 1$ states and reset threshold i, so the bound is sharp for every $n \geq 1$.

7.1.2 Subautomata, Morphic Images, and Quotient Automata

Theorem 7.3 collects statements on how the set of synchronizing words related to subautomata, surjective morphic images, and quotient automata. Theorem 7.3, Item 4 is due to Rystsov [534, Prop. 2.3].

Proposition 7.3 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following is true.*

1. $\mathrm{Syn}(\mathcal{A}) \subseteq \mathrm{Syn}(\mathcal{B})$ *if we have a surjective morphism $\varphi\colon Q \to S$ onto $\mathcal{B} = (S, \Sigma, \mu)$.*
2. $\mathrm{Syn}(\mathcal{A}) \subseteq \mathrm{Syn}(\mathcal{A}/\!\sim)$ *for a $\mathcal{T}_S(\mathcal{A})$-congruence $\sim$ on $\mathcal{A}$.*
3. $\mathrm{Syn}(\mathcal{A}) \subseteq \mathrm{Syn}(\mathcal{A}_{|S})$ *for an upper set S. So, if $\mathcal{A}$ is synchronizing, then $\mathcal{A}_{|S}$ is synchronizing.*
4. $\mathrm{Syn}(\mathcal{A}/\!\sim_S)\,\mathrm{Syn}(\mathcal{A}_{|S}) \subseteq \mathrm{Syn}(\mathcal{A})$ *for an upper set S (Rystsov [534, Prop. 2.3]).*

Proof. 1 Suppose $|\delta(Q, u)| = 1$ for $u \in \Sigma^*$. We have $\mu(S, u) = \mu(\varphi(Q), u) = \varphi(\delta(Q, u))$. So $|\mu(S, u)| = 1$.

2 Applying 1 to the canonical morphism for $\sim$ from Theorem 6.55 gives the claim.

3 The equation $\delta_{|S}(S, u) \subseteq \delta(Q, u)$ for $u \in \Sigma^*$ gives the claim.

4 Let $u \in \mathrm{Syn}(\mathcal{A}/\!\sim_S)$. The set S is a trap state in $\mathcal{A}/\!\sim_S$. So, this state must be the synchronizing state in $\mathcal{A}/\!\sim_S$, i.e., we have $\delta_{\sim_S}([q]_{\sim_S}, u) = S$ for all $q \in Q$, or equivalently $\delta(Q, u) \subseteq S$. Then for $v \in \mathrm{Syn}(\mathcal{A}_{|S})$ we get $\delta(Q, uv) = \delta(\delta(Q, u), v) \subseteq \delta(S, v)$ where $|\delta(S, v)| = 1$. Hence $uv \in \mathrm{Syn}(\mathcal{A})$. □

Corollary 7.4 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with a trap state and a $\mathcal{T}_S(\mathcal{A})$-congruence $\sim$ on its states such that $[t]_\sim = \{t\}$ for each trap state t. Then we have $\mathrm{Syn}(\mathcal{A}) = \mathrm{Syn}(\mathcal{A}/\!\sim)$.*

Proof. By Theorem 7.3, Item 2 we only have to prove that $\mathrm{Syn}(\mathcal{A}/\sim)$ is contained in $\mathrm{Syn}(\mathcal{A})$. So, let $u \in \mathrm{Syn}(\mathcal{A}/\sim)$. Let t be some trap state in $\mathcal{A}$. By Theorem 6.19, also $[t]_\sim$ is a trap state in $\mathcal{A}/\sim$. By Theorem 7.6, we have $\delta_\sim([q]_\sim, u) = [t]_\sim$ for all $q \in Q$, i.e., $\delta(q,u) \sim t$ for all $q \in Q$. Since $[t]_\sim = \{t\}$ this yields $\delta(q,u) = t$ for all $q \in Q$ and so $u \in \mathrm{Syn}(\mathcal{A})$. □

Corollary 7.5 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a synchronizing n-state automaton all whose synchronizing words have length at least $n-1$ and $S \subseteq Q$ be an upper set. Then all synchronizing words of $\mathcal{A}_{|S}$ and $\mathcal{A}/\sim_S$ have length at least $|S|-1$ and $n-|S|$, respectively.*

Proof. If $\mathcal{A}_{|S}$ or $\mathcal{A}/\sim_S$ have a synchronizing word of shorter length, then by Theorem 7.3, Item 4 also $\mathcal{A}$ would have a synchronizing word of shorter length. □

7.1.3 Synchronizing Automata with a Trap State

Several classes of automata we investigate later have the property that a synchronizing state must be a trap state. Here, we collect statements about synchronizing automata with a trap state. First, Theorem 7.6 characterizes synchronizability for automata with a trap state. It also yields a sharp bound on the reset length of synchronizing automata with a trap state, first stated by Rystsov [526] (see also [534, Thm. 6.1]) and Babcsányi [52, Cor. 3]; we state this bound, together with a family of automata attaining it, in Theorem 7.7. Then, in Theorem 7.9, we state some consequences. For example, we characterize the synchronizing automata with a trap state as those automata possessing a unique maximal state with respect to the reachability order.

Theorem 7.6 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton with a trap state $t \in Q$. Then the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A}$ is synchronizing with unique synchronizing state t.*
3. *$\forall q \in Q\ \exists u \in \Sigma^* \colon \delta(q,u) = t$, i.e., $q \preceq_R t$ for all $q \in Q$.*
4. *$\exists u \in \Sigma^* \colon |u| \leq \frac{n(n-1)}{2} \wedge (\forall q \in Q \colon \delta(q,u) = t)$.*

Proof. 1 $\Rightarrow$ 2: Let $\delta(Q,u) = \{q\}$ for some $u \in \Sigma^*$ and $q \in Q$. As t is a trap state, $t \in \delta(Q,u)$, so $t = q$. Hence $\{t\}$ is the set of synchronizing states.

2 $\Rightarrow$ 3: By assumption there exists $u \in \Sigma^*$ such that $\delta(q,u) = t$ for all $q \in Q$, i.e., $q \preceq_R t$ for all $q \in Q$.

3 $\Rightarrow$ 4: For $n = 1$ we have $Q = \{t\}$ and ε is a synchronizing word. Thus, assume $n > 1$ in the following.

We construct words u_i and sets $Q = Q_0, Q_1, \ldots$ containing t with strictly decreasing cardinalities as follows. Set $Q_0 = Q$. Given Q_i with $t \in Q_i$, stop if $Q_i = \{t\}$. Otherwise, since t is reachable from each state by Item 3, we can pick a shortest word u_{i+1} with $\delta(q_i, u_{i+1}) = t$ for some $q_i \in Q_i \setminus \{t\}$. Then set $Q_{i+1} = \delta(Q_i, u_{i+1})$.

Since t is a trap state, we have $\delta(t, u_{i+1}) = t = \delta(q_i, u_{i+1})$. So $t \in Q_{i+1}$ and $|Q_{i+1}| < |Q_i|$. Thus there exists $m \leq n-1$ such that $Q_m = \{t\}$.

Let $i \in \{0, 1, \ldots, m-1\}$ and let $q_i = p_0, p_1, \ldots, p_\ell = t$ be the path induced by u_{i+1}, where $\ell = |u_{i+1}|$. By the minimal choice of u_{i+1}, no p_r with $r \geq 1$ lies in $Q_i \setminus \{t\}$ and no p_r with $r < \ell$ equals t, and the p_r are pairwise distinct; otherwise a proper suffix, a proper prefix, or the word obtained by deleting a factor of u_{i+1} would take a state of $Q_i \setminus \{t\}$ to t. As $p_1, \ldots, p_{\ell-1}$ are thus $\ell - 1$ distinct states outside Q_i, of which there are exactly $n - |Q_i|$, we get $|u_{i+1}| \leq n - |Q_i| + 1$.

Set $u = u_1 \cdots u_m$. We have $\delta(Q, u) = \{t\}$. As $n = |Q_0| > |Q_1| > \cdots > |Q_{m-1}| \geq 2$, the numbers $n-|Q_{i-1}|+1$ for $i \in \{1, \ldots, m\}$ are pairwise distinct and lie in $\{1, \ldots, n-1\}$. Thus, we have

$$|u| = |u_1| + \cdots + |u_m| \leq \sum_{i=1}^{m} \big(n - |Q_{i-1}| + 1\big) \leq \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}.$$

$4 \Rightarrow 1$: This is obvious. □

Remark 7.7 By Theorem 7.6, every synchronizing n-state automaton with a trap state has reset length at most $\frac{n(n-1)}{2}$, and this bound is sharp: the automata from Figure 7.2 have reset length exactly $\frac{n(n-1)}{2}$.

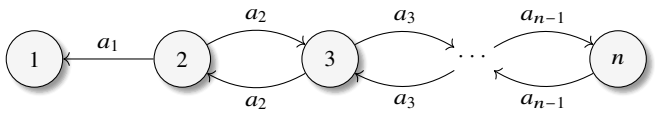


Fig. 7.2: Trap-state automata from [534] with reset length $\frac{n(n-1)}{2}$. The letter a_1 maps 2 to the trap state 1, and a_i interchanges i and $i+1$ for $i \geq 2$; self-loops are omitted. A shortest synchronizing word is $u = w_1 w_2 \cdots w_{n-1}$ where $w_i = a_i a_{i-1} \cdots a_1$ for $i \in \{1, \ldots, n-1\}$. Note that $\Sigma = \{a_1, a_2, \ldots, a_{n-1}\}$ grows with n.

Item 2 of Theorem 7.6 states that in a synchronizing automaton, a trap state is the unique synchronizing state.

Corollary 7.8 *In a synchronizing automaton, there exists at most one trap state.*

Proof. Let t, t' be trap states. By Item 2 of Theorem 7.6, each of them is the unique synchronizing state, so $t = t'$. □

Thus, by Theorem 7.6, an automaton with a trap state is synchronizing if and only if that trap state is reachable from every state. The trap state is then unique by Theorem 7.8. Note, however, that it is easy to find automata with a unique trap state that are not synchronizing.

Corollary 7.9 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing with a trap state.*
2. *There exists a trap state $t \in Q$ such that $q \preceq_R t$ for all $q \in Q$.*

3. *$Q/\simeq_R$ has a maximum, and this maximum is a singleton $\{t\}$ for some $t \in Q$.*
4. *$\mathcal{A}$ has a unique maximal state.*
5. *$\mathcal{A}$ has a unique synchronizing state.*

Proof. 1 $\Rightarrow$ 2: This follows from Item 3 of Theorem 7.6.

2 $\Rightarrow$ 3: Since t is a maximum, $[t]_{\simeq_R}$ is a maximum in $Q/\simeq_R$. Since t is a trap state, we have $[t]_{\simeq_R} = \{t\}$.

3 $\Rightarrow$ 4: Since $\{t\}$ is a maximum in $Q/\simeq_R$, t is maximal. Let $q \in Q$ be a maximal state. By assumption, $q \precsim_R t$. By maximality, $t \precsim_R q$. Thus $q \simeq_R t$, which gives $q = t$ by assumption.

4 $\Rightarrow$ 5: Let t be the unique maximal state. By Theorem 3.53, the set of maximal states $\{t\}$ is an upper set. Thus, t is a trap state. Since every state is below some maximal state by finiteness, and there is only one maximal state, we have $q \precsim_R t$ for each $q \in Q$. Thus, by Theorem 7.6, t is a unique synchronizing state.

5 $\Rightarrow$ 1: Let t be the unique synchronizing state. Then $\delta(Q, u) = \{t\}$ for some $u \in \Sigma^*$. Let $a \in \Sigma$. We have $|\delta(Q, ua)| = 1$. By assumption, $\delta(Q, ua) = \{t\}$. Thus $\delta(t, a) \in \delta(Q, ua) = \{t\}$, and so t is a trap state. □

7.1.4 Synchronizing Automata and the Reachability Order

Here, in Theorem 7.10, we first prove that the quotient automaton w.r.t. the set of maximal states is synchronizing. Then, in Theorem 7.11, we prove that an automaton is synchronizing if and only if the subautomaton on the set of maximal states is synchronizing and strongly connected. Further, in this case, the set of maximal states equals the set of synchronizing states. As an application, in Theorem 7.15, we show that if the Černý conjecture holds for strongly connected automata, then it holds for all automata.

Proposition 7.10 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be the set of maximal states for $\precsim_R$. Then S is an upper set and $\mathcal{A}/{\sim_S}$ is synchronizing with trap state S.*

Proof. By Theorem 3.53, S is an upper set forming a trap state in $\mathcal{A}/{\sim_S}$. By Theorem 6.80, this trap state is reachable from every other state, and so the claim follows with Theorem 7.9, Item 2. □

Theorem 7.11 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be the set of maximal states for $\precsim_R$. Then S is an upper set and the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A}_{|S}$ is synchronizing.*
3. *$\mathcal{A}_{|S}$ is synchronizing and strongly connected (so that S is a sink component).*
4. *$\mathcal{A}$ is synchronizing and S equals the set of synchronizing states of $\mathcal{A}$.*

Proof. By Theorem 3.53, S is an upper set. Next, we show the stated equivalences.

1 $\Rightarrow$ 2: By Theorem 7.3, Item 3, if $\mathcal{A}$ is synchronizing, then $\mathcal{A}_{|S}$ is synchronizing.

$2 \Rightarrow 3$: By assumption $\delta(S,u) = \{q\}$ for some $q \in Q$ and $u \in \Sigma^*$. So $p \preceq_R q$ for all $p \in S$, which gives, as p is a maximal state, $p \simeq_R q$ for all $p \in S$. Hence S is strongly connected and as it is an upper set it is a sink component.

$3 \Rightarrow 4$: By Theorem 7.10 the automaton $\mathcal{A}/\sim_S$ is synchronizing. So, from Theorem 7.3, Item 4 we get that $\mathcal{A}$ is synchronizing. Let $q \in Q$ be such that $\delta(Q,u) = \{q\}$ for some $u \in \Sigma^*$. Then $p \preceq_R q$ for all $p \in Q$. So, q is maximal and we have $q \in S$. Furthermore, as S is a sink component, for $p \in S$ there exists $v \in \Sigma^*$ such that $\delta(q,v) = p$. Then $\delta(Q,uv) = \{p\}$ so that p is a synchronizing state. So we obtain that S equals the set of synchronizing states.

$4 \Rightarrow 1$: This is obvious. □

Corollary 7.12 *Let $\mathcal{A} = (Q,\Sigma,\delta)$ and $q \in Q$. Then the following are equivalent.*

1. *q is synchronizing.*
2. *$\mathcal{A}$ is synchronizing and q is maximal w.r.t. $\preceq_R$.*
3. *$\mathcal{A}$ is synchronizing and q is a maximum w.r.t. $\preceq_R$.*

Theorem 7.11, Item 3 and Theorem 6.81 give the next corollary.

Corollary 7.13 *A synchronizing automaton has a unique sink component.*

Furthermore, Theorem 7.11 yields the following corollary (but this also follows from Theorem 6.4 stated earlier).

Corollary 7.14 *In an automaton every state is a synchronizing state if and only if it is synchronizing and strongly connected.*

As a nice application of Theorem 7.3 and Theorem 7.11, we prove that confirmation of the Černý conjecture for strongly connected automata implies it for all automata. This result appears in the literature in [487, Thm. 10], [130, Prop. 2], [530, Thm. 2], and in slightly more general form in [614, Prop. 2.1] and [52, Thm. 6].

Proposition 7.15 *The Černý conjecture is true for all automata if it is true for all strongly connected automata.*

Proof. Assume the Černý conjecture is true for strongly connected automata. Let $\mathcal{A} = (Q,\Sigma,\delta)$ be a synchronizing n-state automaton with set of synchronizing states $S \subseteq Q$. By Theorem 7.11 the set S equals the upper set of all maximal states and $\mathcal{A}_{|S}$ is synchronizing and strongly connected. So, by assumption, $\mathcal{A}_{|S}$ has a synchronizing word $v \in \Sigma^*$ of length at most $(|S|-1)^2$. By Theorem 7.10, $\mathcal{A}/\sim_S$ is synchronizing with trap state S. As $\mathcal{A}/\sim_S$ has $n - |S| + 1$ states, from Theorem 7.6, Item 4 we obtain that $\mathcal{A}/\sim_S$ has a synchronizing word $u \in \Sigma^*$ of length at most $\frac{(n-|S|+1)(n-|S|)}{2}$. By Theorem 7.3, Item 4, the word uv synchronizes $\mathcal{A}$ and

$$|uv| \leq \frac{(n-|S|+1)(n-|S|)}{2} + (|S|-1)^2 \leq (n-|S|)^2 + (|S|-1)^2$$
$$\leq (n-|S|+|S|-1)^2 = (n-1)^2. \square$$

Volkov [614, Prop. 2.1] (see also [21, Prop. 1]) gave the following generalization of the previous argument. Suppose we have a class of automata fulfilling the following two properties:

1. it is closed under taking subautomata and Rees quotient automata, and
2. the reset length of every n-state synchronizing automaton from this class that is strongly connected or has a trap state is bounded by $f(n)$ for some function $f\colon \mathbb{N}_0 \to \mathbb{N}_0$ with $f(n) + f(m) \leq f(n+m-1)$ for all $n, m \geq 1$.

Then, with the notation as in the proof of Theorem 7.15, $|uv| \leq f(n-|S|+1) + f(|S|) \leq f(n)$, and we obtain that every n-state synchronizing automaton in this class has a synchronizing word of length at most $f(n)$. Specifically, for the class of all automata, the function $f(n) = (n-1)^2$ fulfills both conditions.

7.1.5 The Pair Criterion

The pair criterion (Theorem 7.18), due to Černý [140, Thm. 2], states that an automaton is synchronizing if and only if all pairs of states can be mapped to a single state. Here, we deduce this important criterion from Theorem 7.6 and the observation, stated next, that the pair automaton from Section 6.5.6 is an automaton with a trap state having the same set of synchronizing states.

Proposition 7.16 (Rystsov [534, Prop. 5.2]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then we have $\mathrm{Syn}(\mathcal{A}) = \mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right)$ where $F = \{\{q\} \mid q \in Q\}$. So, $\mathcal{A}$ is synchronizing if and only if its pair automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ is synchronizing.*

Proof. Suppose $u \in \mathrm{Syn}(\mathcal{A})$. Then $\delta(Q, u) = \{q\}$ for some $q \in Q$. Hence $\delta(\{p_1, p_2\}, u) = \{q\}$ for all $p_1, p_2 \in Q$. So, u maps all states of $\mathcal{A}^{\leq 2}$ into F, giving $u \in \mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right)$.

Conversely, let $u \in \mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right)$. Pick some $q \in Q$. Consider a state $p \in Q$. Since F is a trap state, it must be the unique synchronizing state and so $\delta_{\leq 2}(\{p, q\}, u) \in F$. Thus $\delta(p, u) = \delta(q, u)$ and since p was chosen arbitrarily, we find $\delta(Q, u) = \{\delta(q, u)\}$ and so u synchronizes $\mathcal{A}$. □

We remark that it also easily follows that $\mathrm{Syn}(\mathcal{A}^{\leq 2}) = \mathrm{Syn}(\mathcal{A})$.

Remark 7.17 Since the pair automaton has $\binom{n}{2} + 1$ states, by Theorem 7.16, the length of a shortest synchronizing words is quadratically bounded in the number of states if and only if it is linearly bounded in the number of states of the pair automaton.

The pair automaton has F as a trap state. By Theorem 7.16 and by applying the quantifier-swapping for trap-state synchronizing automata stated in Theorem 7.6 3 and 4 to $\mathcal{A}^{\leq 2}/{\sim_F}$, we obtain the pair criterion, firstly stated by Černý [140, Thm. 2].

Theorem 7.18 (Pair Criterion, Černý [140, Thm. 2]) *An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with non-empty Q is synchronizing if and only if for all pairs of states $p, q \in Q$ there exists $u \in \Sigma^*$ such that $\delta(p, u) = \delta(q, u)$.*

Proof. By Theorem 7.16 we have $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$ if and only if $\mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right) \neq \emptyset$ where $F = \{\{q\} \mid q \in Q\}$. The pair automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ has F as a trap state. So, by Theorem 7.6, Item 3, we have $\mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right) \neq \emptyset$ if and only if the trap state is reachable from every other state, i.e., for $p, q \in Q$ there exists $u \in \Sigma^*$ such that $\delta(\{p, q\}, u) \in F$, or equivalently $\delta(p, u) = \delta(q, u)$. □

Remark 7.19 In the language of transformation semigroup as discussed in Section 6.8, the pair criterion states that a transformation semigroup S on a finite set contains a constant map if and only if every two points can be mapped to a common point by some transformation from S. The pair automaton has a trap state, but note that in general, we have a quadratic lower bound for trap-state automata, see Figure 7.2.

The following equivalences follow readily from Theorem 7.18.

Corollary 7.20 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A} \times \mathcal{A}$ is synchronizing.*
3. *$\mathcal{A}^{\leq 2}$ is synchronizing.*
4. *$\mathcal{A}^{\leq 2}/{\sim_F}$ with $F = \{\{q\} \mid q \in Q\}$ is synchronizing.*
5. *$(\mathcal{A} \times \mathcal{A})/{\sim_D}$ with $D = \{(q, q) \mid q \in Q\}$ is synchronizing.*

The pair criterion can be expressed in the following form using the *stability relation* $\sim_s$ introduced in [329, 358] where we set for $p, q \in Q$

$$p \sim_s q \Leftrightarrow \forall u \in \Sigma^* \; \exists v \in \Sigma^* \colon \delta(p, uv) = \delta(q, uv),$$

This is in fact a $\mathcal{T}_S(\mathcal{A})$-congruence.

Corollary 7.21 (Kari [358, 329]) *An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q \neq \emptyset$ is synchronizing if and only if the stability relation $\sim_s$ equals the universal relation.*

Later, we will also use a transformation semigroup variant of this relation. In this case, for a transformation semigroup S on a domain X and $a, b \in X$ we set $a \sim_s b$ if and only if for all $t \in S$ there exists $h \in S$ such that $h(t(a)) = h(t(b))$. Then S contains a constant transformation if and only if $\sim_s$ equals the universal relation (see also Section 6.8 for a discussion of the relation of transformation semigroups and automata).

7.1.6 Algorithmic Questions

First, in Theorems 7.22 and 7.23, we investigate the running time of the problem of deciding synchronizability for automata with a trap state. Then, Theorems 7.25 and 7.26 make algorithmic statements for general automata. Lastly, in Theorem 7.27, we prove that for synchronizing automata we can compute in linear time the set of synchronizing states. As a consequence, stated in Theorem 7.28, for general automata the set of synchronizing states can be computed in quadratic time.

Later, in Section 8.1, we will introduce a decision variant of the problem of computing a shortest synchronizing word. In general, this problem is NP-complete even for partially ordered binary automata, see Section 8.4.

Proposition 7.22 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We can test in time $O(|Q|)$ and space $O(1)$ whether $\mathcal{A}$ has a trap state, and also whether it has a unique trap state and compute this state if it exists.*

Proof. Iterating through all states and checking whether we find precisely one trap state can be done in the mentioned time and space bounds. □

By Theorem 7.6, Item 3 an automaton with a trap state is synchronizing precisely if this trap state is reachable from every other state, which gives the following time and space bounds to test synchronizability for automata with a trap state that were also noted by Martyugin [436, Prop. 3.2].

Theorem 7.23 (Martyugin [436, Prop. 3.2]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with a trap state. Then we can test in time $O(|Q| + |Q| \cdot |\Sigma|)$ and space $O(|Q|)$ whether it is synchronizing.*

Proof. By Theorem 7.22 we can test whether the automaton has a unique trap, and if so compute it, in time $O(|Q|)$ and space $O(1)$. If this is the case, then checking whether there exists a path from every other state to this trap state can be done in time $O(|Q| + |\Sigma||Q|)$ and space $O(|Q|)$ with a breadth-first search by following the edges in the automaton graph in reverse order starting from the trap state (see, for example, the textbook [163] for an explanation of this algorithm and a statement of its resource requirements). □

Corollary 7.24 *For a class of automata with the property that a synchronizing state is a trap state, we can test whether an automaton from this class is synchronizing in time $O(|Q| + |Q| \cdot |\Sigma|)$ and space $O(|Q|)$.*

Proof. First test in linear time and constant space whether an input automaton has a trap state, which can be done in the stated time and space bounds by Theorem 7.22. If this is not the case, then it is not synchronizing by assumption Otherwise, by Theorem 7.23, we can test in the given time and space bounds whether it is synchronizing. □

By Theorem 7.16, $\mathcal{A}$ is synchronizing if and only if $\mathcal{A}^{\leq 2}/{\sim_F}$ with $F = \{\{q\} \mid q \in Q\}$ is synchronizing. As the pair automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ has the trap state F, $\frac{|Q|(|Q|-1)}{2} + 1$ states, and $\left(\frac{|Q|(|Q|-1)}{2} + 1\right) \cdot |\Sigma|$ transitions, to test synchronizability of $\mathcal{A}$ we obtain the following time and space bounds from Theorem 7.23.

Theorem 7.25 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We can test in time $O\left(|Q|^2 + |Q|^2 \cdot |\Sigma|\right)$ and space $O\left(|Q|^2\right)$ whether $\mathcal{A}$ is synchronizing.*

However, the algorithm behind Theorem 7.25 does not give us a synchronizing word. Natarajan [457, p. 138] has given a greedy algorithm to compute some synchronizing word of length at most $O(|Q|^3 \cdot |\Sigma|)$ in time $O(|Q|^4|\Sigma|)$. Eppstein [198, Thm. 5, 6 &

7] analyzed this algorithm further, optimized it to run in $O(|Q|^3 + |Q|^2 \cdot |\Sigma|)$ time and $O(|Q|^2 + |Q| \cdot |\Sigma|)$ space, and proved that some synchronizing word of length at most $O(|Q|^3)$ can be computed by it (see also the surveys by Sandberg [548, Thm. 1.15], Volkov [613, p. 16] and Kari & Volkov [360, p. 531]). Later, in Section 8.3.1, we will also prove that the deciding whether there exists a synchronizing word bounded by some given length-parameter is NP-complete.

Theorem 7.26 (Eppstein [198], Natarajan [457]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We can test in time $O(|Q|^3 + |Q|^2 \cdot |\Sigma|)$ and (not counting the space for the output) working space $O(|Q|^2 + |Q| \cdot |\Sigma|)$ whether $\mathcal{A}$ is synchronizing and compute a synchronizing word of length $O(|Q|^3)$.*

Now, for a synchronizing automaton, we can compute the set of synchronizing states in linear time.

Theorem 7.27 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a synchronizing automaton. Then we can compute in time $O(|Q| + |Q||\Sigma|)$ and space $O(|Q|)$ the set of synchronizing states.*

Proof. First, compute the strongly connected components, which can be done in the mentioned time and space bounds [163, 596]. Since $\mathcal{A}$ is synchronizing, it follows from Theorem 7.11 and Theorem 6.81 there exists a unique sink component, i.e., a strongly connected component with no outgoing transitions, which can be easily identified and coincides with the set of synchronizing states.

Alternatively, we can pick some state and start a breath-first search or a depth-first search in the automaton graph from this state. The first state q we encounter with no unvisited neighbors is a maximal state. Since $\mathcal{A}$ is synchronizing, $[q]_{\simeq_R}$ equals the set of synchronizing states by Theorem 7.11. □

For a synchronizing automaton with a trap state t, the set of synchronizing states equals $\{t\}$ and this set can be computed in linear time by Theorem 7.22.

Combining Theorem 7.25 and 7.27 gives the next result.

Corollary 7.28 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We can compute in time $O\left(|Q|^2 \cdot |\Sigma|\right)$ and space $O\left(|Q|^2\right)$ the set of synchronizing states of $\mathcal{A}$.*

Proof. First, test whether $\mathcal{A}$ is synchronizing. If $\mathcal{A}$ is not synchronizing, then return the empty set. Otherwise, compute the set of synchronizing states and return this set. Combining the time and space bounds from Theorem 7.25 and 7.27 for these computations gives the claim. □

7.2 Unary Automata

The following tight bound on shortest words of rank r in unary automata is probably folklore. For completeness, we give a short proof.

Proposition 7.29 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ be an n-state unary automaton. If there exists a word of rank r in $\mathcal{A}$, then there exists a word of length at most $n - r$ with rank equal to r, and this bound is tight even in unary automata where r is the minimal rank among all words.*

Proof. Suppose we have $r = |\delta(Q, a^i)|$ for some $i \geq 0$. Then, by Theorem 6.73, there exists $0 \leq j \leq n - r$ such that $r = |\delta(Q, a^j)|$. Tightness for $1 \leq r < n$ is shown by $\mathcal{A}_n = (\{q_1, \ldots, q_n\}, \{a\}, \delta)$ with $\delta(q_i, a) = q_{\min\{i+1, n-r\}}$ for $i \in \{1, \ldots, n - r\}$ and $\delta(q_i, a) = q_i$ for $i \in \{n - r + 1, \ldots, n\}$ (see Figure 7.3). Alternatively, the automata family $\mathcal{B}_n = (\{1 \ldots, n\}, \{a\}, \delta)$ with $\delta(i, a) = \min\{i + 1, n\}$ has a^{n-r} as the shortest word of rank at most r with $\delta(\{1, \ldots, n\}, a^{n-r}) = \{n - r + 1, n - r + 2, \ldots, n\}$. □

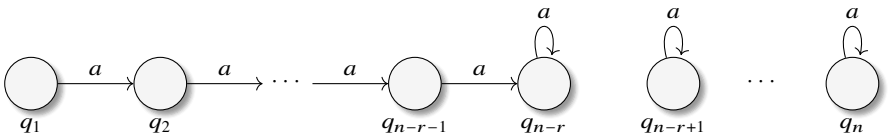


Fig. 7.3: Unary automaton realizing the lower bound from Proposition 7.29.

Remark 7.30 The unary automata $C_n = (\{1 \ldots, n\}, \{a\}, \delta)$ with $\delta(i, a) = \min\{i+1, n\}$ would also show tightness of the bound from Theorem 7.29 for general unary automata since the shortest word of rank at most r in C_n is a^{n-r} with $\delta(\{1, \ldots, n\}, a^{n-r}) = \{n - r + 1, n - r + 2, \ldots, n\}$. Here r is not the minimal rank among all words for $r > 1$.

7.3 OI- and SI-Automata

Let $\mathcal{A} = (Q, \Sigma, \delta)$. We call $\mathcal{A}$ an *automaton with one-point singular idempotents (OI-automaton)* if every letter is either permutational or induces a one-point singular idempotent. We call $\mathcal{A}$ an *automaton with simple idempotents (SI-automaton)* if every letter is either permutational or a simple idempotent. Clearly, every SI-automaton is an OI-automaton.

For example, the Černý automata from Section 1.2 (Figure 1.7) are SI-automata. Rystsov [535] introduced SI-automata and proved that every synchronizing SI-automaton with n states admits a synchronizing word of length at most $2(n - 1)^2$. In Section 11.9, we extend this bound to OI-automata.

For an OI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$, we set

$$\Sigma_I = \{a \in \Sigma \mid a \text{ induces a one-point singular idempotent }\},$$
$$\Sigma_G = \{a \in \Sigma \mid a \text{ is a permutational letter }\}.$$

Then $\Sigma = \Sigma_I \cup \Sigma_G$.

First, in Theorem 7.31, we classify synchronizing binary SI-automata, a result originally due to Martyugin [436, Prop. 6.2]. Theorem 7.32 gives synchronizing words for binary SI-automata. Theorem 7.33 establishes a lower bound on the rank of a reachable subset, and Theorem 7.34 establishes a lower bound on the length of a word of a given rank. Theorem 7.35 determines the length of shortest words reaching a given subset in SI-automata.

Proposition 7.31 (Martyugin [436, Prop. 6.2]) *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be an n-state binary synchronizing SI-automaton. Then we have $n \leq 2$ or one of the following holds:*

1. *$n = 3$ and $\mathcal{A}$ is, up to renaming of the letters, one of the automata in Figures 7.4a and 7.4b, or*
2. *one letter, say a, is a simple idempotent with $\delta(s, a) = t$ for distinct $s, t \in Q$, the letter b permutes the states with $Q = \{\delta(s, b^i) \mid i \geq 0\} \cup \{t\}$, and either*
   a. *t is a trap state (see Figure 7.4c), or*
   b. *$\delta(s, b^d) = t$ for some $0 < d < n$ with $\gcd(d, n) = 1$ (see Figure 7.4d).*[1]

Proof. Clearly, one letter, say a, must be a simple idempotent.

First, suppose a and b are both simple idempotents. If $n \geq 4$, then, since each letter moves only one state, there exist two distinct states $p, q \in Q$ that are fixed by both letters. Consequently, every word fixes them, and so no word synchronizes the automaton. Thus, we have $n \leq 3$. It is clear that for $n \leq 2$, $\mathcal{A}$ is always synchronizing. For $n = 3$, a simple case analysis yields that $\mathcal{A}$ is synchronizing if and only if it equals, up to renaming of the letters, one of the automata from Figures 7.4a and 7.4b.

Next, let b be a permutational letter. Let $s, t \in Q$ be distinct with $\delta(s, a) = t$. Set $S = \{\delta(s, b^i) \mid i \geq 0\}$ and $T = \{\delta(t, b^i) \mid i \geq 0\}$. As $\delta(S \cup T, x) \subseteq S \cup T$ and $\delta(Q \setminus (S \cup T), x) \subseteq Q \setminus (S \cup T)$ for $x \in \{a, b\}$, we have $Q = S \cup T$. If $s \notin T$, then $\delta(T, x) = T$ for all $x \in \Sigma$, which gives $T = \{t\}$ and that t is a trap state. Otherwise, $s \in T$ implies $S = T$. Then $\delta(s, b^d) = t$ for some $0 < d < n$. Set $r = \gcd(d, n)$ and, for $p, q \in Q$, write $p \sim q$ if there exists $i \geq 0$ such that $\delta(p, b^{ir}) = q$. For all $x \in \{a, b\}$ and $p, q \in Q$ we have

$$p \sim q \Leftrightarrow \delta(p, x) \sim \delta(q, x).$$

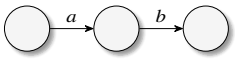
So $|\delta(\{p, q\}, u)| = 2$ for all $p, q \in Q$ with $p \nsim q$. Hence, as $\mathcal{A}$ is synchronizing, all states must be equivalent w.r.t. $\sim$, which implies $r = 1$. □

Next, we give synchronizing words for each of the two types of automata from Theorem 7.31.

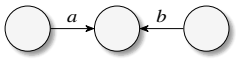
Proposition 7.32 *Let $n \geq 2$.*

1. *The word $(ab)^{n-2}a$ synchronizes the n-state SI-automaton from Figure 7.4c.*
2. *The word $(ab^{n-d})^{n-2}a$ synchronizes the n-state SI-automaton from Figure 7.4d.*

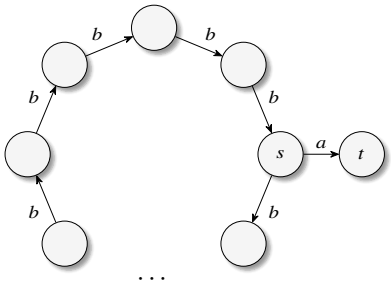
[1] In [436, Prop. 6.2] a different naming of the states is used, with a parameter $2 \leq p \leq n$ such that $p = n - d + 1$. There appears to be a minor mistake in [436, Prop. 6.2]. It is claimed that the automaton is not synchronizing whenever $\gcd(n, p) > 1$. However, if $p = n$ (which results in the Černý automaton), then $\gcd(n, p) = n$. The correct condition is $\gcd(n, n - p + 1) > 1$.



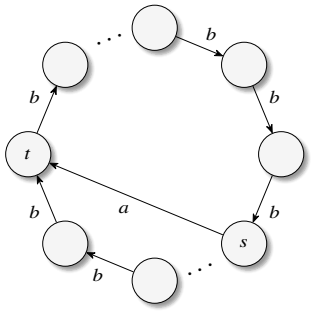
(a) Three states, with a trap state.



(b) Three states, both letters merging.



(c) Binary synchronizing SI-automaton with a trap state.



(d) Strongly connected binary synchronizing SI-automaton where $\delta(s, b^d) = t$.

Fig. 7.4: Synchronizing binary SI-automata (self-loops omitted) as listed in Theorem 7.31. Note that, in Figure 7.4d, for $d = 1$ we obtain the Černý family.

Proof. We use the notation from Theorem 7.31.

1: For $i \in \{1, \ldots, n-1\}$ we have

$$t = \delta(s, b^i(ab)^{(n-1)-i}a) = \delta(s, b^i(ab)^{n-2}a).$$

Since the state set is $\{s, \delta(s,b), \ldots, \delta(s, b^{n-2})\} \cup \{t\}$ and t is a trap state, we find $\delta(Q, (ab)^{n-2}a) = \{t\}$.

2: We can assume $n \geq 3$. For $i \in \{0, 1, \ldots, n-1\}$ set $Q_i = \{ s, \delta(s, b^d), \ldots, \delta(s, b^{id}) \}$. Since d and n are coprime, the smallest $x > 0$ with $xd \equiv 0 \pmod{n}$ is $x = n$. Hence the states $\delta(s, b^{jd})$ for $j \in \{0, 1, \ldots, n-1\}$ are pairwise distinct, so $|Q_i| = i + 1$ and $Q_{n-1} = Q$.

We show by induction on i that $Q_i \subseteq \delta^{-1}(s, (ab^{n-d})^i)$. This is easy to see for $i = 1$. Next, suppose $i > 1$. By induction hypothesis, we have $Q_{i-1} \subseteq \delta^{-1}(s, (ab^{n-d})^{i-1})$. For $j \in \{0, 1, \ldots, i-1\}$, since d and n are coprime, we have $(j+1)d \not\equiv 0 \pmod{n}$ and thus $\delta(s, b^{(j+1)d}) \neq s$. This gives $\delta(s, b^{(j+1)d}ab^{n-d}) = \delta(s, b^{jd})$. With $\delta(s, ab^{n-d}) = s$ we find $\delta(Q_i, ab^{n-d}) \subseteq Q_{i-1}$. Thus $Q_i \subseteq \delta^{-1}(s, (ab^{n-d})^i)$. This concludes our induction argument.

Now, set $q = \delta(s, b^{(n-1)d})$. We have $\delta(q, (ab^{n-d})^{n-2}) = \delta(s, b^d)$ and so

$$\delta(Q, (ab^{n-d})^{n-2}) = \delta(Q_{n-2} \cup \{q\}, (ab^{n-d})^{n-2}) = \{s, \delta(s, b^d)\}.$$

Thus, $\delta(Q, (ab^{n-d})^{n-2}a) = \{\delta(s, b^d)\}$. □

Further synchronizing words can be constructed. For example, for the automata from Figure 7.4c, all words in

$$\Sigma^* a^+ (b((ba^*)^{n-1})^*)a^+)^{n-2}\Sigma^*$$

synchronize it, or $abbabbabbabba$ synchronizes this automaton for $n = 6$. For the automata from Figure 7.4d all words in $\Sigma^* a^+((ba^*)^{n-p}(ba^*)^n a^+)^{n-2}\Sigma^*$ synchronize it.

Lemma 7.33 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton. Then for $S \subseteq Q$ and $u \in \Sigma^*$ we have $|S| - \sum_{a\in\Sigma_I} |u|_a \leq |\delta(S, u)|$.*

Proof. Clearly, the claim holds for $u = \varepsilon$. Next, let $u = wx$ with $x \in \Sigma$ and $w \in \Sigma^*$. Set $T = \delta(S, w)$. Inductively, we have

$$|S| - \sum_{a\in\Sigma_I} |w|_a \leq |T|. \tag{7.2}$$

For $x \in \Sigma_I$, since x maps only two states to one state, we have $|T| - 1 \leq |\delta(T, x)|$. Thus, with $\delta(S, u) = \delta(T, x)$ and Equation (7.2), we find

$$|S| - \sum_{a\in\Sigma_I} |u|_a = |S| - \left(\sum_{a\in\Sigma_I} |w|_a\right) - 1 \leq |T| - 1 \leq |\delta(S, u)|.$$

For $x \in \Sigma_G$, we have $|\delta(T, x)| = |T|$ and $\sum_{a\in\Sigma_I} |u|_a = \sum_{a\in\Sigma_I} |w|_a$ and substituting these values into Equation (7.2) we get the claim. □

Corollary 7.34 *In an n-state SI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$ for a word u of rank m we have $n - m \leq |u|$.*

Proof. By Theorem 7.33 we have $n - \sum_{a\in\Sigma_I} |u|_a \leq m$. This gives $n - m \leq \sum_{a\in\Sigma_I} |u|_a \leq |u|$. □

Lastly, we note a bound on shortest words reaching a given subset of states in SI-automata without permutational letters.

Proposition 7.35 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton without permutational letters. Then for a shortest word $u \in \Sigma^*$ such that $\delta(Q, u) = S$ we have $|u| = |Q| - |S|$.*

Proof. We can assume $S \neq Q$. Let $u_1, \ldots, u_m \in \Sigma$ and $p_1, \ldots, p_m \in Q$ be such that $u = u_1 \cdots u_m$ and $\mathrm{excl}(u_i) = \{p_i\}$ for $i \in \{1, \ldots, m\}$. Next, by induction, we show that for $i \in \{1, \ldots, m\}$ we have

1. $\delta(Q, u_1 \cdots u_i) = Q \setminus \{p_1, \ldots, p_i\}$ and
2. the states $p_1, \ldots, p_i$ are pairwise distinct.

The claim is clear for $i = 1$. Now let $i > 1$. By the minimal choice of u, we have $p_i \in \delta(Q, u_1 \cdots u_{i-1})$ since otherwise the letter u_i fixes every state in $\delta(Q, u_1 \cdots u_{i-1})$ and thus can be omitted. Since, inductively, $\delta(Q, u_1 \cdots u_{i-1}) = Q \setminus \{p_1, \ldots, p_{i-1}\}$, we find $\delta(Q, u_1 \cdots u_i) = Q \setminus \{p_1, \ldots, p_i\}$ and $p_i \notin \{p_1, \ldots, p_{i-1}\}$. Since $p_1, \ldots, p_{i-1}$ are pairwise distinct by induction hypothesis, the states $p_1, \ldots, p_i$ are pairwise distinct.

Finally, for $i = m$ we find $|S| = |\delta(Q, u)| = |Q| - m$. Thus $m = |Q| - |S|$. □

7.4 Permutation-Reset Automata

Let $\mathcal{A} = (Q, \Sigma, \delta)$. We call $\mathcal{A}$ a *permutation-reset automaton* if $\delta(Q, a) \neq Q$ implies $|\delta(Q, a)| = 1$ for every $a \in \Sigma$, a *permutation automaton* if every letter is a permutational letter, and a *reset automaton* if every letter synchronizes the automaton. The automaton $\mathcal{A}$ is a permutation automaton if and only if $\mathcal{T}_S(\mathcal{A})$ is a permutation group on Q. Permutation automata are SI-automata without idempotent letters. Clearly, if $\mathcal{A}$ is a permutation-reset automaton, then we have

$$\mathrm{Syn}(\mathcal{A}) = \Sigma^* \cdot \{a \in \Sigma : |\delta(Q, a)| = 1\} \cdot \Sigma^* \tag{7.3}$$

Proposition 7.36 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then the following are equivalent.*

1. *$\mathcal{A}/{\sim_S}$ is a permutation-reset automaton*
2. *$\delta(Q \setminus S, a) \neq Q \setminus S$ implies $\delta(Q, a) \subseteq S$ for each $a \in \Sigma$.*
3. *$Q \setminus S$ is a block.*
4. *$\mathcal{A}_{|Q\setminus S,\Gamma}$ for $\Gamma = \{a \in \Sigma \mid \delta(Q \setminus S, a) \subseteq Q \setminus S\}$ is a permutation automaton and $a \in \Sigma \setminus \Gamma$ implies $\delta(Q \setminus S, a) \subseteq S$.*

Proof. $1 \Rightarrow 2$: By Theorem 6.61, the state set $Q/{\sim_S}$ is permuted by a letter $a \in \Sigma$ if and only if $\delta(Q \setminus S, a) = Q \setminus S$ and $\delta_{\sim_S}(S, a) = S$. So, $\delta(Q \setminus S, a) \neq Q \setminus S$ implies $|\delta_{\sim_S}(Q/{\sim_S}, a)| = 1$. Since S is a trap state in $\mathcal{A}/{\sim_S}$, this yields $\delta_{\sim_S}(Q/{\sim_S}, a) = \{S\}$, which is equivalent to $\delta(Q, a) \subseteq S$.

$2 \Rightarrow 3$: Since S is an upper set, $\delta(Q, a) \subseteq S$ is equivalent to $\delta(Q \setminus S, a) \subseteq S$. So, by assumption, we have $\delta(Q \setminus S, a) = Q \setminus S$ or $\delta(Q \setminus S, a) \subseteq S$, and the latter condition is equivalent to $\delta(Q \setminus S, a) \cap (Q \setminus S) = \emptyset$. Thus, $Q \setminus S$ is a block.

$3 \Rightarrow 4$: Since $Q \setminus S$ is a block, $\delta(Q \setminus S, a) \subseteq Q \setminus S$ implies $\delta(Q \setminus S, a) = Q \setminus S$, and so $\mathcal{A}_{|Q\setminus S,\Gamma}$ is a permutation automaton. For $a \in \Sigma \setminus \Gamma$, we have $\delta(Q \setminus S, a) \cap (Q \setminus S) = \emptyset$, which implies $\delta(Q \setminus S, a) \subseteq S$.

$4 \Rightarrow 1$: The assumption yields that $\delta(Q \setminus S, a) = Q \setminus S$ or $\delta(Q \setminus S, a) \subseteq S$ for each $a \in \Sigma$. Since S is an upper set, the latter condition is equivalent to $\delta(Q, a) \subseteq S$. From Theorem 6.61 we get that $\mathcal{A}/{\sim_S}$ is a permutation-reset automaton. □

Remark 7.37 By Theorem 7.36, a strongly connected component $S \subseteq Q$ in $\mathcal{A} = (Q, \Sigma, \delta)$ is a block if and only if $(\mathcal{A}_{|T})/{\sim_{T\setminus S}}$ for $T = \uparrow S$ is a permutation-reset automaton.

By Theorem 7.36 in a permutation-reset automaton with a trap state t, by considering $S = \{t\}$, every non-permutational letter maps everything to the trap state.

The equivalences from Theorem 7.36 fail for $S = \emptyset$ in case of permutation-reset automata $\mathcal{A} = (Q, \Sigma, \delta)$ with $|Q| \geq 2$ and a letter $a \in \Sigma$ such that $|\delta(Q, a)| = 1$.

Proposition 7.38 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a permutation automaton. Then $\delta(q, u) \simeq_R q$ for all $q \in Q$ and $u \in \Sigma^*$.*

Proof. This follows from Theorem 6.6 since the whole state set is permuted by each letter. □

Corollary 7.39 *In a permutation automaton each strongly connected component is a sink component.*

By Theorem 7.38, the strongly connected components in a permutation automaton are precisely the orbits of the group $\mathcal{T}_S(\mathcal{A})$. Furthermore, for a permutation automaton the following are equivalent: strong connectivity, transitivity of the group of units of the transformation monoid, univalent connectedness, and the property of being cyclic.

Next, using Theorem 6.88, we prove the following minimality condition for permutation automata (the special cases $|F| = 1$ or $|F| = |Q| - 1$ were mentioned in [171, Cor. 3.8]).

Proposition 7.40 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a permutation automaton with $1 \leq |F| < |Q|$ such that $\gcd(|F|, |Q|) = 1$. Then $\mathcal{A}$ is minimal if and only if $\mathcal{A}$ is accessible.*

Proof. A non-accessible automaton is not minimal by Theorem 6.88. Next, assume $\mathcal{A}$ is accessible. Set $G = \mathcal{T}_S(\mathcal{A})$. We show that $\mathcal{A}$ admits no non-trivial G-congruence that saturates F, which gives minimality of $\mathcal{A}$ by Theorem 6.88, Item 5. Assume $\sim$ is some G-congruence that saturates F. Then F is a union of blocks. Let $\Delta \subseteq F$ be a block. As $\mathcal{A}$ is accessible, G is transitive. Then, by Theorem 5.4, F and Q are unions of equal-sized sets of the form $g(\Delta)$ for $g \in G$. Thus $|\Delta|$ divides $|F|$ and $|Q|$, and so it divides $\gcd(|F|, |Q|)$. By coprimality, we find $|\Delta| = 1$. So Δ is a trivial block and the only G-congruence saturating F is the equality relation. □

Corollary 7.41 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a permutation automaton with $1 \leq |F| < |Q|$ such that $\gcd(|F|, |Q|) = 1$. If $\mathcal{T}_S(\mathcal{A})$ contains a transitive permutation group G, then $\mathcal{A}$ is minimal.*

Proof. Let $\Gamma = \{x \in \Sigma \mid x \text{ is a permutational letter}\}$. Since $G \subseteq \mathcal{T}_S(\mathcal{A}_{|\Gamma})$, $\mathcal{T}_S(\mathcal{A}_{|\Gamma})$ is a transitive permutation group, which yields that $\mathcal{A}_{|\Gamma}$ is an accessible permutation automaton. So, by Theorem 7.40, $\mathcal{A}_{|\Gamma}$ is minimal. Since accessibility and distinguishability of pairs of states extend from $\mathcal{A}_{|\Gamma}$ to $\mathcal{A}$, also $\mathcal{A}$ is minimal by Theorem 6.88, Item 4. □

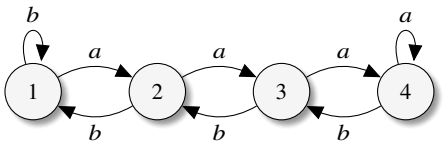
7.5 Monotonic Automata

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called *monotonic* if there exists a linear order $\leq$ on the states such that $p \leq q$ implies $\delta(p, x) \leq \delta(q, x)$ for all $p, q \in Q$ and $x \in \Sigma$. See Figure 7.5 for two example monotonic automata.

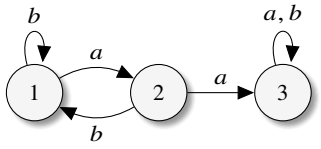
Such an order is generally not unique. For example, if all transitions are self-loops in an n-state automaton, $n!$ orderings are possible, and for each state, there exists an ordering in which it is either the minimal or the maximal element

Suppose $q = \delta(q, u^i)$ for $q \in Q$, $i \geq 1$ and $u \in \Sigma^*$ in a monotonic $\mathcal{A} = (Q, \Sigma, \delta)$. We have $q \leq \delta(q, u)$ or $\delta(q, u) \leq q$. In the first case, we get $q \leq \delta(q, u) \leq \delta(q, u^2) \leq \ldots \leq \delta(q, u^{i-1}) \leq q$, so that $q = \delta(q, u)$. A similar argument yields $q = \delta(q, u)$ for

$\delta(q, u) \leq q$. So, no word can permute a subset of states non-trivially.[2] In particular, no permutation automata is monotonic.



(a) Linear order on states is $1 < 2 < 3 < 4$



(b) Linear order on states is $1 < 2 < 3$

Fig. 7.5: Two synchronizing monotonic automata.

With respect to synchronizability Ananichev and Volkov [23] have established the following tight bound (tightness is shown by the automata from Figure 7.3, which are monotonic).

Proposition 7.42 (Ananichev and Volkov [23]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state monotonic automaton. If there exists a word of rank r in $\mathcal{A}$, then there exists a word of length at most $n - r$ whose rank is at most r and this bound is tight.*

7.6 Automata Whose Strongly Connected Components are Blocks

In the present section, we investigate automata all of whose strongly connected components are blocks. These automata arose from the study of solvable automata, which we introduce later in Section 7.10. We observed that several properties of solvable automata extend to this more general class.

First, Theorem 7.43 gives a bound on the length of the shortest words of rank r. Theorem 7.45 gives bounds on the length of the shortest words mapping the entire state set into an upper set. Theorem 7.46 gives words ending in certain maximal states. We derive several consequences in Theorems 7.47 to 7.49. Theorem 7.50 characterizes maximal states. Lastly, in Theorem 7.51, we show that every synchronizing state is a trap state.

Theorem 7.43 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton all of whose strongly connected components are blocks that has a word of rank r. Then there exists a word of length at most $n - r$ whose rank is at most r, and this bound is tight.*

Furthermore, let $u \in \bigcap_{a \in \Sigma} \Sigma^* a \Sigma^*$. Then u^{n-r} has rank at most r, and $\delta(Q, u^{n-1})$ is the union of all sink components of $\mathcal{A}$.

Proof. Suppose $\mathcal{A}$ admits a word of rank r. We can assume $n > r$. By Proposition 6.77 there exists a sequence of upper sets

[2] Automata with this property are called counter-free or aperiodic [449].

$$S_1 \subsetneq \cdots \subsetneq S_m = Q \tag{7.4}$$

where S_1 equals the union of all sink components and $S_i \setminus S_{i-1}$ for $i \in \{2, \ldots, m\}$ are all the strongly connected components that are not sink components. Let $i \in \{2, \ldots, m\}$ and set $S = S_i \setminus S_{i-1}$. As S is not a sink component, we have $\delta(S, x_i) \setminus S \neq \emptyset$ for some $x_i \in \Sigma$, which gives $\delta(S, x_i) \cap S = \emptyset$ as S is a block. As S_i is an upper set, we have $\delta(S, x_i) \subseteq S_i = S_{i-1} \cup S$ and so $\delta(S, x_i) \subseteq S_{i-1}$. Hence, we find

$$\delta(S_i, x_i) = \delta(S_{i-1} \cup S, x_i) = \delta(S_{i-1}, x_i) \cup \delta(S, x_i) \subseteq S_{i-1}.$$

This also gives $\delta(S_i, u) \subseteq S_{i-1}$ for all $i \in \{2, \ldots, m\}$ since $\delta(S_i, a) \subseteq S_i$ and $\delta(S_{i-1}, a) \subseteq S_{i-1}$ for all $a \in \Sigma$. Now, inductively, from $\delta(S_i, x_i) \subseteq S_{i-1}$ and $\delta(S_i, u) \subseteq S_{i-1}$ for all $i \in \{2, \ldots, m\}$, we obtain that for all $i \in \{2, \ldots, m\}$ we have

$$\delta(Q, x_m \cdots x_i) \subseteq S_{i-1} \quad \text{and} \quad \delta(Q, u^{m-(i-1)}) \subseteq S_{i-1}.$$

Since the sequence from Equation (7.4) strictly decreases, we get $|S_i| \leq n - (m - i)$ for $i \in \{1, \ldots, m\}$. As S_1 is a union of sink components that are blocks, from Equation (6.3) we get $S_1 = \delta(S_1, u) \subseteq \delta(Q, u)$ for each $u \in \Sigma^*$, which gives $S_1 \subseteq \delta(Q, u^{n-1})$ and $|S_1| \leq r$. As $|S_1| \leq r$ there exists a maximal $j \in \{1, \ldots, m\}$ such that $|S_j| \leq r$. We have $j \neq m$ and $r < |S_{j+1}| \leq n - (m - (j + 1))$. So we get $m - j - 1 < n - r$. Now, as $\delta(Q, x_m \cdots x_{j+1}) \subseteq S_j$ and $\delta(Q, u^{m-j}) \subseteq S_j$ the words $x_m \cdots x_{j+1}$ and u^{m-j} of length $m - j \leq n - r$ and $|\Sigma|(m - j)$, respectively, have rank at most r, and the same follows for u^{n-r}. Lastly, $\delta(Q, u^{m-1}) \subseteq S_1$ gives $\delta(Q, u^{n-1}) \subseteq S_1$ as $m - 1 \leq n - 1$ and so $\delta(Q, u^{n-1}) = S_1$.

For tightness, consider the automata from the proof of Theorem 7.29. □

Remark 7.44 Theorem 6.79 can be used to give an inductive proof of Theorem 7.43. We can assume $\mathcal{A}$ admits a word $u \in \Sigma^*$ of rank r with $r < n$ for $n = |Q|$. If Q equals the set of maximal states, then Q equals a union of sink components. By assumption, these sink components are blocks. By Equation (6.3) each letter permutes each sink component. Thus, $\mathcal{A}$ is a permutation automaton. Now, for $r < |Q|$, this cannot be the case. Then Theorem 6.79 yields existence of a proper and non-empty upper set $S \subseteq Q$ such that $\mathcal{A}/{\sim_S}$ is strongly trap-connected. Since $|\delta(S, u)| \leq r$, inductively, there exists $v \in \Sigma^*$ with $|v| \leq \max\{|S| - r, 0\}$ such that $|\delta(S, v)| \leq r$. Furthermore, we have $\delta(Q \setminus S, x) \cap S \neq \emptyset$ for some $x \in \Sigma$ since $Q \setminus S$ is not a sink component by Theorem 6.65. Thus, as $Q \setminus S$ is a block, we find $\delta(Q \setminus S, x) \subseteq S$. Hence $\delta(Q, x) \subseteq S$. Then $|\delta(Q, xv)| \leq r$ and $|xv| \leq 1 + \max\{|S| - r, 0\} \leq n - r$. For $w \in \bigcap_{a \in \Sigma} \Sigma^* a \Sigma^*$, we can also inductively suppose $|\delta(S, w^{\max\{|S|-r,0\}})| \leq r$ and, similarly as above, conclude $|\delta(Q, w^{n-r})| \leq r$.

Proposition 7.45 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton all of whose strongly connected components are blocks and $S \subseteq Q$ be an upper set. Suppose $\delta(Q, u) \subseteq S$ for some $u \in \Sigma^*$. Then there exists $v \in \Sigma^*$ with $\delta(Q, v) \subseteq S$ and $|v| \leq |Q| - |S|$, and this bound is tight.*

Proof. We may assume $Q \neq \emptyset$; then $\emptyset \neq \delta(Q, u) \subseteq S$, so $S \neq \emptyset$. We induct on $|Q|$.

If $S = Q$, then $v = \varepsilon$ works. So assume $S \subsetneq Q$. Then S is a non-empty proper upper set, so we may choose an inclusion-maximal proper non-empty upper set $T \subseteq Q$ and put $C := Q \setminus T$. By Theorems 3.39 and 3.42, C is a strongly connected component, hence a block by assumption.

As T is an upper set, $\mathcal{A}_{|T}$ is well defined and its strongly connected components are strongly connected components of $\mathcal{A}$, hence blocks. Moreover $T \cap S$ is an upper set of $\mathcal{A}_{|T}$ and $\delta(T, u) \subseteq T \cap \delta(Q, u) \subseteq T \cap S$. Since $|T| < |Q|$, induction gives $w \in \Sigma^*$ with $\delta(T, w) \subseteq T \cap S$ and $|w| \leq |T| - |T \cap S| = |T \setminus S|$.

Case 1: $C \subseteq S$. Then $Q \setminus S \subseteq T$, so $|w| \leq |T \setminus S| = |Q \setminus S| = |Q| - |S|$. As S is an upper set, $\delta(C, w) \subseteq S$, whence $\delta(Q, w) = \delta(C, w) \cup \delta(T, w) \subseteq S$. Take $v = w$.

Case 2: $C \not\subseteq S$. First, $\delta(C, a) \cap T \neq \emptyset$ for some $a \in \Sigma$: otherwise $\delta(C, a) \subseteq C$ for all $a \in \Sigma$, so C and hence $C \cap S$ are upper sets; the latter is non-empty because $\emptyset \neq \delta(C, u) \subseteq C \cap S$, and it is a proper subset of C, so $T \subsetneq T \cup (C \cap S) \subsetneq Q$ contradicts the maximality of T. Fix such an a. Since C is a block, $\delta(C, a) \cap C = \emptyset$, i.e. $\delta(C, a) \subseteq T$, and $\delta(T, a) \subseteq T$; thus $\delta(Q, a) \subseteq T$ and $\delta(Q, aw) \subseteq \delta(T, w) \subseteq S$. Finally $Q \setminus (S \cup T) = C \setminus S \neq \emptyset$, so

$$|Q| - |S| = |Q \setminus (S \cup T)| + |T \setminus S| \geq 1 + |T \setminus S| \geq 1 + |w| = |aw|.$$

Take $v = aw$.

Tightness follows from the automata in Figure 7.3 with $S = \{q_{n-r}, \ldots, q_n\}$. □

Theorem 7.45 might fail if S is not an upper set. For example, in the automaton from Theorem 7.30, the shortest word mapping all states into $\{1, n\}$ is a^{n-1}.

Proposition 7.46 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton all of whose strongly connected components are blocks and $S \subseteq Q$. Then, for $u \in \bigcap_{a \in \Sigma} \Sigma^* a \Sigma^*$ and $n \geq |Q| - 1$ we have*

$$\delta(S, u^n) \subseteq \{q \in \uparrow S \mid q \text{ is a maximal state } \}.$$

Proof. Let $q \in S$. Clearly, $\delta(q, u^n) \in \uparrow S$. By finiteness, we have $\delta(q, u^{i+p}) = \delta(q, u^i)$ for some $0 \leq i < |Q|$ and $p > 0$. Let T be the strongly connected component T with $\delta(q, u^i) \in T$. We have

$$\delta(q, u^{i+p}) \simeq_R \delta(q, u^{i+p-1}) \simeq_R \cdots \simeq_R \delta(q, u^i)$$

Thus $\delta(T, u) \cap T \neq \emptyset$. Since T is a block, we find $\delta(T, u) = T$. Because u contains each letter, this yields $\delta(T, a) = T$ for each $a \in \Sigma$ and so T is a sink component. Hence, $\delta(q, u^i)$ is maximal, and so $\delta(q, u^n)$ is maximal. □

The containment in Theorem 7.46 can be proper. For example, this is the case for $S = \{q_1\}$ and $u = ab$ in the automaton whose strongly connected components are blocks from Figure 7.7a

Corollary 7.47 *Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is an automaton all of whose strongly connected components are blocks. Let $S \subseteq Q$ and $R = \{q \in \uparrow S \mid q \text{ is a maximal state}\}$. Suppose $R \subseteq S$.*

1. *For $T \subseteq Q$ we have $\delta(S, u) \subseteq T$ for some $u \in \Sigma^*$ if and only if $R \subseteq T$.*

2. *For* $u \in \bigcap_{a\in\Sigma} \Sigma^* a \Sigma^*$ *and* $n \geq |Q| - 1$ *we have* $\delta(S, u^n) = R$.

Proof. 1: First, let $\delta(S, u) \subseteq T$ for some $u \in \Sigma^*$. For $R \subseteq S$, the set R is the union of the sink components contained in S. By Equation (6.3), $\delta(S, u)$ also contains these sink components. Thus R is contained in $\delta(S, u)$ and therefore in T.

Conversely, if $R \subseteq T$, by Theorem 7.46 there exists a word mapping Q into R, and thus into T.

2: Set $T = \delta(S, u^n)$. We have $\delta(S, u^n) \subseteq T$. Thus, by Item 1, we get $R \subseteq T$. By Theorem 7.46 we have $T \subseteq R$. □

Note that $R \subseteq S$ holds in Theorem 7.47 if S is an upper set.

Theorem 7.47, Item 1 generalizes the statement from Theorem 7.43 that words which contain each letter applied $|Q| - 1$ times map the whole state set onto the set of maximal states.

Corollary 7.48 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *be an automaton all of whose strongly connected components are blocks. For* $S \subseteq Q$ *there exists* $u \in \Sigma^*$ *such that* $\delta(Q, u) \subseteq S$ *if and only if* $T \subseteq S$ *for each sink component* $T \subseteq Q$ *of* $\mathcal{A}$.

Proof. This follows from Theorem 7.47 applied to Q and S (since in this case, R equals the union of all sink components). □

Corollary 7.49 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *be an automaton all of whose strongly connected components are blocks and* $S \subseteq Q$. *Suppose* $\delta(Q, u) \subseteq S$ *for some* $u \in \Sigma^*$. *Then there exists* $v \in \Sigma^*$ *with* $\delta(Q, v) \subseteq S$ *and* $|v| \leq |Q| - 1$.

Proof. Let R be the union of all sink components in $\mathcal{A}$. By Theorem 7.46 there exists a word mapping Q into R. Thus, by Theorem 7.45, we have $\delta(Q, u) \subseteq R$ for some $u \in \Sigma^*$ with $|u| \leq |Q| - |R| \leq |Q| - 1$. By Theorem 7.48, we have $R \subseteq S$ and so u maps Q into S. □

Next, we characterize the maximal states.

Proposition 7.50 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *be an automaton all of whose strongly connected components are blocks. A state* $q \in Q$ *is contained in a sink component if and only if for all* $x \in \Sigma$ *we have* $\delta(q, x^i) = q$ *for some* $i \in \{1, \ldots, |Q|\}$.

Proof. Let $S \subseteq Q$ be the strongly connected component with $q \in S$. First, assume S is a sink component. By Equation (6.3), every letter $x \in \Sigma$ permutes S and so $\delta(q, x^i) = q$ for some $i \in \{1, \ldots, |S|\}$. Next, assume S is not a sink component. Then, as S is a block, we have $\delta(S, x) \cap S = \emptyset$ for some $x \in \Sigma$. As S is a strongly connected component, $\delta(q, x) \notin S$ yields $\delta(q, x^i) \notin S$, and so $\delta(q, x^i) \neq q$, for all $i \geq 1$. □

Proposition 7.51 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *be an automaton all of whose sink components are blocks. Then each synchronizing state is a trap state.*

Proof. By Theorem 7.11, the set of synchronizing states S is a sink component. Thus $\mathcal{A}_{|S}$ is a synchronizing permutation automaton. This implies $|S| = 1$. □

So, by Theorem 7.51 an automaton in which all sink components are blocks is synchronizable if and only if it has a unique trap state that is reachable from every other state.

7.7 Partially Ordered Automata

The automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called *partially ordered* if $q \simeq_R p$ implies $p = q$, i.e., the reachability quasiorder is a partial order. This is equivalent to the fact that all cycles are induced by self-loops, i.e., $\delta(q, u) = q$ implies $\delta(q, v) = q$ for every factor v of u, in particular for the letters $v \in \Sigma$. Equivalently, the strongly connected components are singletons (and so also blocks). Hence, the subset mapping properties from Section 7.6 hold for partially ordered automata.

Here, we collect results for partially ordered automata that we need later and, in Theorem 7.58, characterizes synchronizable partially ordered automata as those with a unique trap state.

Example 7.52 Figure 7.6 depicts a partially ordered automaton with four states. This automaton is not monotonic. For assume $\leq$ is a linear order on Q. If $q_3 \leq q_4$ (resp. $q_4 \leq q_3$), then $q_2 = \delta(q_3, aab) \leq \delta(q_4, aab) = q_1$ (resp. $q_1 \leq q_2$) and $q_1 = \delta(q_3, bba) \leq \delta(q_4, bba) = q_2$ (resp. $q_2 \leq q_1$), so $q_1 = q_2$. However, since $q_1 \neq q_2$, $\mathcal{A}$ is not monotonic.

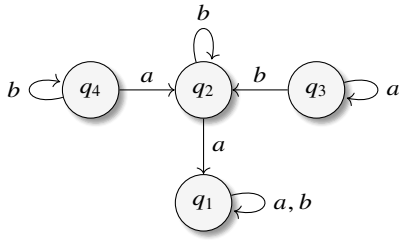


Fig. 7.6: Partially ordered automaton

Proposition 7.53 *In a partially ordered automaton $\mathcal{A} = (Q, \Sigma, \delta)$ a state is maximal for $\precsim_R$ if and only if it is a trap state.*[3] *In particular, there exists at least one trap state, and every upper set contains at least one trap state.*

Proof. Clearly, a trap state is maximal. Furthermore, for a maximal state $q \in Q$ and $a \in \Sigma$ we have $\delta(q, a) \simeq_R q$, so that $\delta(q, a) = q$. Hence, q is a trap state. □

Since trap states equal maximal states, the subset mapping conditions in terms of maximal states from Section 7.6 involve sets of trap states. For example, the following conditions follows from Theorem 7.48.

Proposition 7.54 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton and $S \subseteq Q$. Then $\delta(Q, u) \subseteq S$ for some $u \in \Sigma^*$ if and only if S contains all trap states of $\mathcal{A}$.*

[3] We note that this is only true for the model of complete automata considered in the first half of the present work, but not for the more general model of PDFAs introduced later in Section 17.2.

Proposition 7.55 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be partially ordered. Then for $q \in Q$, $i \geq 0$, and $u \in \Sigma^*$ we have*

$$\delta(q, u^i) = \delta(q, u^{\min\{|Q|-1, i\}}). \tag{7.5}$$

Proof. By finiteness, we have $\delta(q, u^{j+p}) = \delta(q, u^j)$ for some $0 \leq j < |Q|$ and some minimal $p > 0$. Since $\mathcal{A}$ is partially ordered, it is $p = 1$. Hence, for $i > |Q| - 1 \geq j$, we find $\delta(q, u^i) = \delta(q, u^{|Q|-1})$, which gives Equation (7.5). □

Proposition 7.56 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be partially ordered and $u \in \bigcap_{a \in \Sigma} \Sigma^* a \Sigma^*$. Then $\delta(q, u^{|Q|-1})$ is a trap state.*

Proof. This follows from Theorems 7.46 and 7.53. □

Proposition 7.57 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton and $q \in Q$. Then the following are equivalent.*

1. *The state q is a synchronizing state.*
2. *The state q is a unique trap state.*
3. *The state q is a maximum state in the partial order.*

Proof. $1 \Rightarrow 2$: By Theorem 7.9, q is a trap state. By Theorem 7.8, q is a unique trap state in $\mathcal{A}$.

$2 \Rightarrow 3$: By Theorem 7.53 the state q is the unique maximal state, i.e., is a maximum.

$3 \Rightarrow 1$: This follows from Theorem 7.9, Item 4. □

By Theorem 7.57, a state is a synchronizing state if and only if it is a unique trap state, giving the following corollary.

Corollary 7.58 *A partially ordered automaton is synchronizing if and only if it has a unique trap state.*

Comparing Theorem 7.58 with Theorem 7.9, we see that we do not have to state that the unique trap state is reachable from every other state, since this is automatically fulfilled.

By Theorem 7.53 the set of maximal states S equals the set of trap states in partially ordered $\mathcal{A}$. So, $\mathcal{A}_{|S}$ is synchronizing if and only if $|S| = 1$. As, by Theorem 7.11, Item 2, $\mathcal{A}$ is synchronizing if and only if $\mathcal{A}_{|S}$ is synchronizing, we obtain another proof for Theorem 7.58.

Next, by Theorem 7.58 together with Theorem 7.22 we obtain the following time and space bounds on checking synchronizability, since we simply have to check existence of a unique trap state.

Proposition 7.59 *For n-state partially ordered automata we can test in time $O(n)$ and space $O(1)$ whether it is synchronizing.*

Later, in Theorem 20.9, we will prove a statement implying that for a shortest $u \in \Sigma^*$ such that $\delta(S, u) \subseteq T$ we have $|u| \leq |S|(|Q| - 1)$.

Lemma 7.60 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton. Then, for $q \in Q$, $i \geq 0$, and $u \in \Sigma^*$ we have $\delta(q, u^i) = \delta(q, u^{\min\{|Q|-1, i\}})$.*

Proof. By finiteness, there exists such that $\delta(q, u^j) = \delta(q, u^l)$ for some $0 \leq j < |Q|$. We have $l \leq |Q|$. Now, set $p = l - j$. Then $\{\delta(q, u^j), \delta(q, u^{j+1}), \ldots, \delta(q, u^{j+p-1})\}$ is strongly connected, and so must be a singleton as $\mathcal{A}$ is partially ordered. Hence, by the minimal choice of l, we have $p = 1$. This gives $\delta(q, u^r) = \delta(q, u^{|Q|-1})$ for all $r \geq |Q| - 1$. So, we find $\delta(q, u^i) = \delta(q, u^{\min\{|Q|-1, i\}})$. □

Corollary 7.61 *Let $\Sigma = \{a_1, \ldots, a_k\}$ with $k = |\Sigma|$. Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is a partially ordered commutative automaton. Then, for $n \geq |Q| - 1$, $q \in Q$, and $u \in \Sigma^*$ we have*

$$\delta(q, u) = \delta(q, a_1^{\min\{n, |u|_{a_1}\}} \cdots a_k^{\min\{n, |u|_{a_k}\}}). \tag{7.6}$$

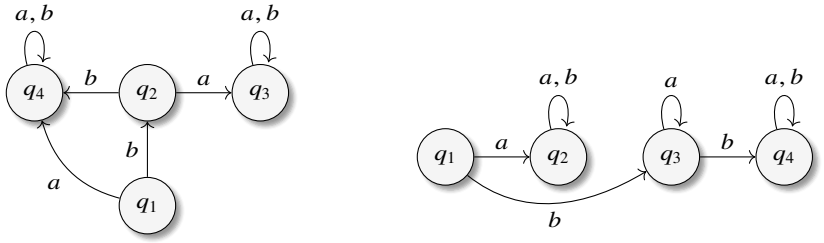
Proof. By commutativity, we have $\delta(q, u) = \delta(q, a_1^{|u|_{a_1}} \cdots a_k^{|u|_{a_k}})$. Then, for $i \in \{1, \ldots, k\}$, with Theorem 7.60, we find

$$\begin{aligned}
\delta(q, a_1^{|u|_{a_1}} \cdots a_k^{|u|_{a_k}}) &= \delta(q, a_1^{|u|_{a_1}} \cdots a_{i-1}^{|u|_{a_{i-1}}} a_{i+1}^{|u|_{a_{i+1}}} \cdots a_k^{|u|_{a_k}} a_i^{|u|_{a_i}}) \\
&= \delta(q, a_1^{|u|_{a_1}} \cdots a_{i-1}^{|u|_{a_{i-1}}} a_{i+1}^{|u|_{a_{i+1}}} \cdots a_k^{|u|_{a_k}} a_i^{\min\{n, |u|_{a_i}\}}) \\
&= \delta(q, a_1^{|u|_{a_1}} \cdots a_{i-1}^{|u|_{a_{i-1}}} a_i^{\min\{n, |u|_{a_i}\}} a_{i+1}^{|u|_{a_{i+1}}} \cdots a_k^{|u|_{a_k}}).
\end{aligned}$$

By applying the above reasoning successively to each $i \in \{1, \ldots, k\}$ we obtain the claim. □

7.8 ρ-Automata

In general, the equivalence relation $\simeq_R$ induced by the reachability quasiorder is not a $\mathcal{T}_S(\mathcal{A})$-congruence. For example, consider the automaton from Figure 7.5b where $1 \simeq_R 2$, but $\delta(1, a) \not\simeq_R \delta(2, a)$. An automaton $\mathcal{A}$ is said to be a *ρ-automaton* if $\simeq_R$ is a $\mathcal{T}_S(\mathcal{A})$-congruence. In this case, the strongly connected components are congruence classes. Trivially, every strongly connected automaton is a ρ-automaton.



(a) Here $q_1 \precsim_R q_2$, but $\delta(q_1, a) \not\precsim_R \delta(q_2, a)$. (b) Here $q_1 \precsim_R q_2$, but $\delta(q_1, b) \not\precsim_R \delta(q_2, b)$.

Fig. 7.7: Two ρ-automata where Equation (7.7) is not true and $\delta(q_1, a) \not\precsim_R q_1$.

First, we give two sufficient conditions to identify ρ-automata.

Proposition 7.62 *Let $\mathcal{A} = (Q, \Sigma, \delta)$.*

1. *Suppose, for all $p, q \in Q$ and $a \in \Sigma$, we have*

$$p \preceq_R q \Rightarrow \delta(p, a) \preceq_R \delta(q, a). \tag{7.7}$$

   Then $\mathcal{A}$ is a ρ-automaton.
2. *Suppose, for all $q \in Q$ and $a \in \Sigma$, we have $\delta(q, a) \preceq_R q$. Then $\mathcal{A}$ fulfills Equation* (7.7) *and so it is a ρ-automaton.*

Proof. 1: Let $a \in \Sigma$ and $p, q \in Q$. If $p \simeq_R q$, then $\delta(p, a) \simeq_R \delta(q, a)$. Hence Equation (6.8) holds and so $\simeq_R$ is a $\mathcal{T}_S(\mathcal{A})$-congruence.

2: Let $p, q \in Q$ with $p \preceq_R q$ and $a \in \Sigma$. Then, by assumption and Theorem 6.3, we have $\delta(p, a) \preceq_R p \preceq_R q \preceq_R \delta(q, a)$. Whence Equation (7.7) is true and the claim follows from Item 1. □

Figure 7.7 shows two ρ-automata where both conditions from Theorem 7.62 do not hold, so that they give proper subclasses of ρ-automata. Also, the classes of partially ordered and permutation automata form proper subclasses of ρ-automata.

Proposition 7.63 *Every partially ordered automaton is a ρ-automaton and every permutation automaton is a ρ-automaton.*

Proof. For a partially ordered automaton $\mathcal{A}$, the relation $\simeq_R$ equals the equality relation, which is a trivial $\mathcal{T}_S(\mathcal{A})$-congruence. By Theorem 7.38 and Item 2 of Theorem 7.62 we find that permutation automata are ρ-automata. □

The reachability order on $\mathcal{A}/\simeq_R$ equals the order induced on the $\simeq_R$-classes by the reachability order on $\mathcal{A}$. Since this is a partial order, we find that $\mathcal{A}/\simeq_R$ is a partially ordered automaton. See Figure 7.8 for an example.

Moreover, constructing $\mathcal{A}/\simeq_R$ amounts to contracting the strongly connected components of $\mathcal{A}$.[4] With Tarjan's algorithm [596] this can be done in time $O(|Q| + |\Sigma||Q|)$. We thus obtain the following statement.

Proposition 7.64 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a ρ-automaton. Then $\mathcal{A}/\simeq_R$ is a partially ordered automaton computable in $O(|Q| + |\Sigma||Q|)$.*

Next, we show that for ρ-automata with a trap state, the set of synchronizing words is preserved when considering the quotient automaton $\mathcal{A}/\simeq_R$.

Proposition 7.65 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a ρ-automaton with a trap state. Then* $\mathrm{Syn}(\mathcal{A}) = \mathrm{Syn}(\mathcal{A}/\simeq_R)$.

Proof. For the reachability order, we have $[t]_{\simeq_R} = \{t\}$ for each trap state t. Thus the claim follows from Theorem 7.4. □

The assumption that $\mathcal{A}$ has a trap state is crucial. For example, for the Černý family C_n (Figure 1.7) we have $\mathrm{Syn}(C_n) \neq \Sigma^*$ but $\mathrm{Syn}(C_n/\simeq_R) = \Sigma^*$.

[4] In graph theory, the graph obtained by contracting each strongly connected component to a single vertex is called the *condensation* of the graph.

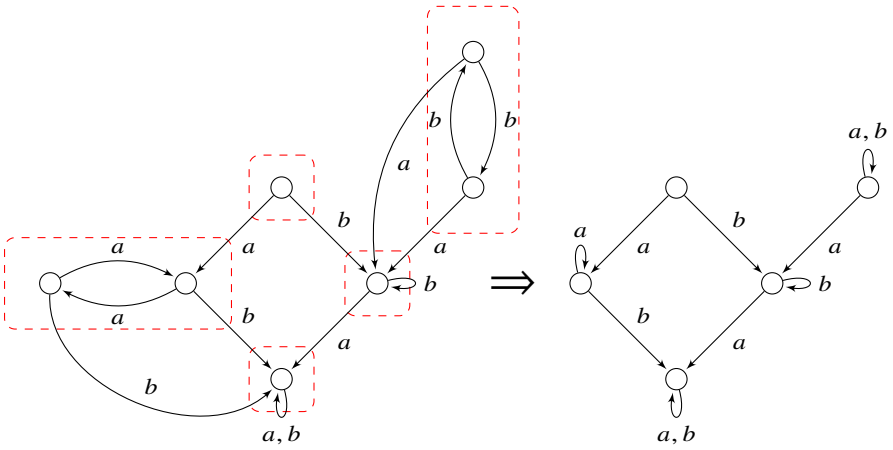


Fig. 7.8: Left: A ρ-automaton $\mathcal{A}$ (that is commutative and synchronizing). The strongly connected components of $\mathcal{A}$ are surrounded by boxes. Right: The automaton $\mathcal{A}/\simeq_R$.

Corollary 7.66 *Every synchronizing ρ-automaton $\mathcal{A}$ with n states and a trap state has a synchronizing word of length at most $n-1$, and this bound is tight.*

Proof. Since $\mathcal{A}/\simeq_R$ is partially ordered, its strongly connected components form blocks. Hence, by Theorem 7.43 with $r = 1$, there exists a synchronizing word of length at most $n-1$ in $\mathrm{Syn}(\mathcal{A}/\simeq_R)$. The result now follows from $\mathrm{Syn}(\mathcal{A}/\simeq_R) = \mathrm{Syn}(\mathcal{A})$, which holds by Theorem 7.65. □

Corollary 7.67 *A synchronizing n-state ρ-automaton $\mathcal{A}$ with a trap state all of whose synchronizing words have length at least $n-1$ is partially ordered.*

Proof. If it is not partially ordered, then $\mathcal{A}/\simeq_R$ has strictly less states. Since it has a trap state, it has a synchronizing word of length less than $n-1$ by Theorem 7.66, which also synchronizes $\mathcal{A}$ by Theorem 7.65. Thus, $\mathcal{A}$ must be partially ordered. □

7.9 Commutative Automata

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is *commutative* if $\delta(q, ab) = \delta(q, ba)$ for all $q \in Q$ and $a, b \in \Sigma$. Said differently, the mapping $q \mapsto \delta(q, x)$ is a morphism for $x \in \Sigma$ and so an automaton is commutative precisely if its transformation semigroup is contained in its endomorphism semigroup. Thus, the letters fulfill the properties from Section 6.4.3. For example, by Equation (6.5), we find that Equation (7.7) holds, and so commutative automata are ρ-automata, $q = \delta(q, u)$ for some $q \in Q$ and $u \in \Sigma^*$ implies $\delta(q, v) = \delta(\delta(q, v), u)$ for all $v \in \Sigma^*$, and, by Theorem 6.35, every strongly connected component is a block. Every unary automaton is commutative. See Figure 7.8 for two commutative

example automata. Clearly, for commutative $\mathcal{A}$, $L(\mathcal{A}_{q,F})$ is commutative for each $q \in Q$ and $F \subseteq Q$.

First, Section 7.9.1 we characterize the commutativity of Rees quotient automata. Then in Section 7.9.2 we investigate strongly connected commutative automata and give structural results on them and their endomorphisms. Section 7.9.3 gives some results on subset mapping and synchronization properties in commutative automata. Lastly, Section 7.9.4 characterizes those commutative automata with longest possible synchronizing words.

7.9.1 Commutativity of Rees Quotients

Proposition 7.68 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then the following are equivalent.*

1. *$\mathcal{A}/{\sim_S}$ is commutative.*
2. *$\delta(q, ab) = \delta(q, ba)$ for each $q \in Q \setminus S$ and $a, b \in \Sigma$ with $\delta(q, ab) \in Q \setminus S$.*
3. *$\delta(q, ab) \neq \delta(q, ba)$ implies $\{\delta(q, ab), \delta(q, ba)\} \subseteq S$ for all $q \in Q$ and $a, b \in \Sigma$.*

Proof. $1 \Rightarrow 2$: This directly follows from Theorem 6.61.

$2 \Rightarrow 3$: By Item 1, for $\{\delta(q, ab), \delta(q, ba)\} \not\subseteq S$, we have $\delta(q, ab) = \delta(q, ba)$, which gives the claim.

$3 \Rightarrow 1$: Since S is a trap state in $\mathcal{A}/{\sim_S}$, from Item 3 and Theorem 6.61 we obtain $\delta_{\sim_S}(\pi_{\sim_S}(p), ab) = \delta_{\sim_S}(\pi_{\sim_S}(q), ba)$ for all $q \in Q$ and $a, b \in \Sigma$, i.e., $\mathcal{A}/{\sim_S}$ is commutative. □

Corollary 7.69 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then $\mathcal{A}/{\sim_S}$ is a commutative permutation-reset automaton if and only if $\mathcal{A}_{|\Gamma, Q\setminus S}$ for $\Gamma = \{a \in \Sigma \mid \delta(Q \setminus S, a) = Q \setminus S\}$ is commutative and $\delta(Q \setminus S, a) \subseteq S$ for $a \in \Sigma \setminus \Gamma$.*

Proof. First, suppose $\mathcal{A}/{\sim_S}$ is a commutative permutation-reset automaton. Then clearly $\mathcal{A}_{|\Gamma, Q\setminus S}$ is commutative. By Theorem 7.36 we have $\delta(Q \setminus S, a) \subseteq S$ for $a \in \Sigma \setminus \Gamma$. Conversely, since each letter in Γ permutes $Q \setminus S$, $\mathcal{A}_{|\Gamma, Q\setminus S}$ is a permutation automaton. Thus, with Theorem 7.36, we find that $\mathcal{A}/{\sim_S}$ is a permutation-reset automaton. Next, let $q \in Q \setminus S$ and $a, b \in \Sigma$. For $a, b \in \Gamma$, we have $\delta(q, ab) = \delta(q, ba)$ since $\mathcal{A}_{|\Gamma, Q\setminus S}$ is commutative. Next, without loss of generality, suppose $a \notin \Gamma$. Then $\delta(Q, a) \subseteq S$ and this gives $\{\delta(q, ab), \delta(q, ba)\} \subseteq S$. Thus, by Theorem 7.68, $\mathcal{A}/{\sim_S}$ is commutative. □

Corollary 7.70 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set such that $\mathcal{A}/{\sim_S}$ is commutative and $Q \setminus S$ is strongly connected. Then $\mathcal{A}/{\sim_S}$ is a permutation-reset automaton (resp. permutation automaton for $S = \emptyset$).*

Proof. Since S is an upper set, $Q \setminus S$ is a strongly connected component. By Theorem 6.62, $\pi_{\sim_S}(Q \setminus S)$ is a strongly connected component in $\mathcal{A}/{\sim_S}$. Thus, $\pi_{\sim_S}(Q \setminus S)$ is a block by Theorem 6.35. Now, a set $R \subseteq Q \setminus S$ is a block in $\mathcal{A}$ if and only if $\pi_{\sim_S}(R)$ is a block in $\mathcal{A}/{\sim_S}$. Thus, $Q \setminus S$ is also a block in $\mathcal{A}$. For $S = \emptyset$, this yields that $\mathcal{A}/{\sim_S}$ is a permutation automaton and, by Theorem 7.36, we find that $\mathcal{A}/{\sim_S}$ is a permutation-reset automaton. □

7.9.2 Strongly Connected Automata

Proposition 7.71 *A commutative automaton that is strongly connected (resp. strongly trap-connected) is a permutation automaton (resp. permutation-reset automaton with a trap state).*

Proof. This follows from Theorem 7.70 applied with $S = \emptyset$ or with $S = \{t\}$ for a trap state. □

Corollary 7.72 *A strongly connected commutative automaton with at least two states is not synchronizing.*

Proof. This follows from Theorem 7.71 since a synchronizing permutation automaton has a single state. □

Proposition 7.73 *In a cyclic commutative automaton $\mathcal{A} = (Q, \Sigma, \delta)$, the endomorphism semigroup of $\mathcal{A}$ equals $\mathcal{T}_M(\mathcal{A})$.*

Proof. In a commutative automaton, every element of its transformation monoid is an endomorphism. By assumption, we have $Q = \uparrow\{q\}$ for some $q \in Q$. Now, let $\varphi\colon Q \to Q$ be an endomorphism. Then $\varphi(q) = \delta(q, u)$ for some $u \in \Sigma^*$. Consider some state $p \in Q$. We have $p = \delta(q, v)$ for some $v \in \Sigma^*$. This gives $\varphi(p) = \varphi(\delta(q, v)) = \delta(\varphi(q), v) = \delta(q, uv) = \delta(\delta(q, v), u) = \delta(p, u)$ and so $\varphi = \delta_u$. □

Corollary 7.74 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a strongly connected commutative automaton. Then $\mathcal{T}_M(\mathcal{A})$ is a regular permutation group and thus has size $|Q|$.*

Proof. Since $\mathcal{A}$ is strongly connected, its endomorphism semigroup equals it automorphism group by Theorem 6.37. By Theorem 6.43 the automorphism group is semiregular. By Theorem 7.73 it equals $\mathcal{T}_M(\mathcal{A})$. Since $\mathcal{A}$ is strongly connected, $\mathcal{T}_M(\mathcal{A})$ is transitive. Thus, $\mathcal{T}_M(\mathcal{A})$ is regular. □

Next, we give examples of commutative automata where the transformation monoid is properly contained in its endomorphism monoid.

Example 7.75 Let $\mathcal{A} = (Q, \{a_1, a_2\}, \delta)$ with $Q = \{q_{1,1}, q_{1,2}, q_{2,1}, q_{2,2}\}$ where, for $i \in \{1, 2\}$, we set $\delta(q_{i,1}, a_i) = \delta(q_{i,2}, a_i) = q_{i,2}$ and the remaining transitions are self-loops. Then $\mathcal{A}$ is commutative and $\varphi\colon Q \to Q$ with $\varphi(q_{i,1}) = \varphi(q_{i,2}) = q_{i,2}$ for $i \in \{1, 2\}$ is a morphism that is not contained in $\mathcal{T}_M(\mathcal{A})$.

Example 7.76 Let $\mathcal{A} = (\{q_{i,j} \mid i \in \{0, 1\},\ j \in [n]\}, \{a\}, \delta)$ be the semiautomaton with

$$\delta(q_{i,j}, a) = q_{i,(j+1) \bmod n}$$

for $i \in \{0, 1\}$ and $j \in [n]$. For every $0 \le s, t < n$, the map φ defined by

$$\varphi(q_{0,j}) = q_{1,(j+s) \bmod n}, \qquad \varphi(q_{1,j}) = q_{0,(j+t) \bmod n}$$

is an endomorphism since

$$\delta(\varphi(q_{0,j}), a) = q_{1,(j+s+1) \bmod n} = \varphi(q_{0,(j+1) \bmod n}) = \varphi(\delta(q_{0,j}, a)),$$

and similarly for the states $q_{1,j}$. None of these endomorphisms is induced by a word and thus none belongs to the transformation monoid.

Next, we determine the structure of binary commutative strongly connected automata. Let $n, m \geq 1$ and $r \in [n]$. Define automata $\mathcal{B}_{n,m,r} = ([n] \times [m], \{a, b\}, \mu_{n,m,r})$ where $\mu_{n,m,r}((i, j), a) = ((i + 1) \bmod n, j)$ and

$$\mu_{n,m,r}((i, j), b) = \begin{cases} (i, j + 1) & \text{if } 0 \leq j \leq m - 2, \\ ((i + r) \bmod n, 0) & \text{if } j = m - 1. \end{cases}$$

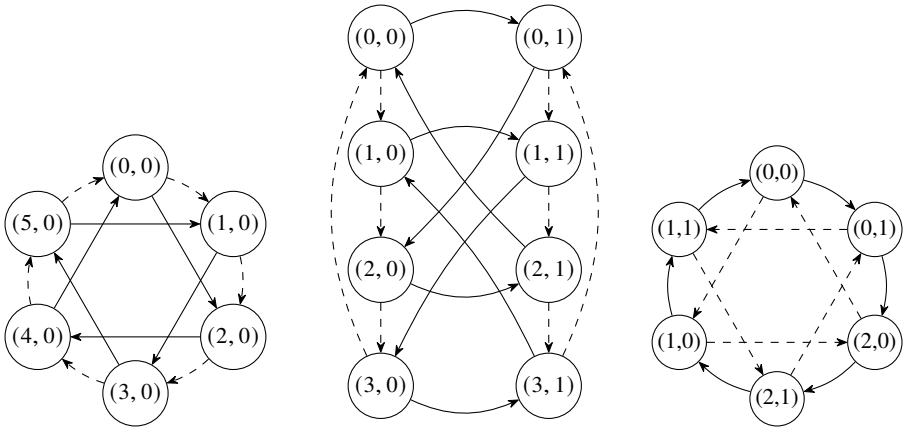
for $(i, j) \in [n] \times [m]$. These automata are commutative. See Figure 7.9 for some examples.

Remark 7.77 For $n, m \geq 2$, the automaton $\mathcal{B}_{n,m,1}$ looks like a grid where each component counts modulo n and m the number of a's and b's, respectively, and each state is uniquely determined by these numbers.

Remark 7.78 In $\mathcal{B}_{n,1,r}$ the letter a induces a single n-cycle and we have

$$\mu_{n,1,r}((i, 1), b) = \mu_{n,1,r}((i, 1), a^r), \quad i \in [n],$$

i.e., the letter b "wraps" around this cycle by skipping r steps on the cycle.



$\mathcal{B}_{6,1,2}$ $\mathcal{B}_{4,2,2}$ $\mathcal{B}_{3,2,2}$

Fig. 7.9: Automata $\mathcal{B}_{n,m,r}$ with a-transitions drawn dashed and b-transitions solid.

Remark 7.79 Suppose r is coprime to n, and let $q = (0, 0)$. Then there exists an integer l with $1 \leq l < n$ such that $rl \equiv 1 \pmod{n}$, i.e., l is the multiplicative inverse of r modulo n. We have the following properties:

1. The states
$$q, \delta(q, b), \ldots, \delta(q, b^{nm})$$
are pairwise distinct and form a single cycle.
2. Since $\delta(q, b^m) = \delta(q, a^r)$, we obtain
$$\delta(q, a) = \delta(q, a^{rl}) = \delta(q, b^{ml}).$$
Consequently, we have
$$\delta(q, b^i a) = \delta(q, ab^i) = \delta(q, b^{i+ml})$$
for all $i \geq 0$. In other words, the letter a acts on the b-cycle as a shift by ml positions.

By these properties, the automaton has the same structure as $\mathcal{B}_{nm,1,ml}$, with the roles of a and b interchanged.

Proposition 7.80 *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be a commutative and strongly connected. Then $\mathcal{A}$ is isomorphic to $\mathcal{B}_{n,m,r}$ for some $n, m \geq 1$ and $r \in [n]$.*

Proof. By Theorem 7.71, $\mathcal{A}$ is a permutation automaton. Let $q \in Q$ and $n \geq 0$ be minimal such that $\delta(q, a^n) = q$. Then the set
$$S = \{q, \delta(q, a), \ldots, \delta(q, a^{n-1})\} \tag{7.8}$$
forms a strongly connected component in $\mathcal{A}_{\{a\}}$. The letter b acts as an isomorphism on this automaton. By Theorem 6.29 from Theorem 6.28 it permutes the strongly connected components of $\mathcal{A}_{\{a\}}$. Thus, we have $\delta(S, b^m) = S$ for some minimal $m \geq 1$.

Next, we show that
$$Q = S \cup \delta(S, b) \cup \cdots \cup \delta(S, b^{m-1}). \tag{7.9}$$
Let $p \in Q$. Since $\mathcal{A}$ is commutative and strongly connected, every state is reachable from q by some word from a^*b^*. Thus $p = \delta(q, a^i b^j)$ for some $i, j \geq 0$ where these numbers are picked such that j is minimal. Assume $j > m$. We have $\delta(q, a^i b^m) \in S$. By Equation (7.8), this yields $\delta(q, a^i b^m) = \delta(q, a^l)$ for some $0 \leq l < n$. Hence $p = \delta(q, a^l b^{j-m})$, contradicting the minimal choice of j. Thus, we have $j < m$. So, $p \in \delta(S, b^j)$ with $0 \leq j < m$.

Now, consider $\varphi\colon [n] \times [m] \to Q$ given by $\varphi(i, j) = \delta(q, a^i b^j)$ for $(i, j) \in [n] \times [m]$. By Equations (7.8) and (7.9) this map is surjective. Next, let $\varphi(i, j) = \varphi(i', j')$ for $(i, j), (i', j') \in [n] \times [m]$. Then $\varphi(i', j') \in \delta(S, b^j)$. Since the sets $S, \delta(S, b), \ldots, \delta(S, b^{m-1})$ are pairwise disjoint, we must have $j' = j$. Since b^j induces a permutation on the state set, we get $\delta(q, a^i) = \delta(q, a^{i'})$. Since $i, i' \in [n]$ and $q, \delta(q, a), \ldots, \delta(q, a^{n-1})$ are pairwise distinct, we find $i = i'$. Thus, φ is also injective.

Since $\delta(S, b^m) = S$ and $q \in S$, by Equation (7.8) there exists $r \in [n]$ such that
$$\delta(q, b^m) = \delta(q, a^r). \tag{7.10}$$
Next, we show that φ is a morphism of $\mathcal{A}$ and $\mathcal{B}_{n,m,r} = ([n] \times [m], \{a, b\}, \mu_{n,m,r})$. Let $(i, j) \in [n] \times [m]$. We have

$$\begin{aligned}
\delta(\varphi(i,j),a) &= \delta(\delta(q,a^i b^j),a) = \delta(q,a^{i+1}b) && \text{(Def. } \varphi \text{ and } \delta\text{)}\\
&= \varphi((i+1) \bmod n, j) && \text{(Eqn. (7.8) and Def. } \varphi\text{)}\\
&= \varphi(\mu_{n,m,r}((i,j),a)). && \text{(Def. } \mu_{n,m,r}\text{)}
\end{aligned}$$

For $j \leq m-2$, we similarly get $\delta(\varphi(i,j),b) = \varphi(\mu_{n,m,r}((i,j),b))$. Lastly, we have

$$\begin{aligned}
\delta(\varphi(i,m-1),b) &= \delta(q,a^i b^m) = \delta(q,b^m a^i) && \text{(Def. } \varphi \text{ and } \mathcal{A} \text{ comm.)}\\
&= \delta(q,a^{i+r}) && \text{(Eqn. (7.10))}\\
&= \varphi((i+r) \bmod n, 0) && \text{(Eqn. (7.8) and Def. } \varphi\text{)}\\
&= \varphi(\mu_{n,m,r}((i,m-1),b)). && \text{(Def. } \mu_{n,m,r}\text{)}
\end{aligned}$$

Thus φ is an isomorphism between $\mathcal{A}$ and $\mathcal{B}_{n,m,r}$. □

Corollary 7.81 *Let $\mathcal{A} = (Q, \{a,b\}, \delta)$ be a commutative permutation automaton and $S \subseteq Q$ be a strongly connected component. Then S is a sink component and $\mathcal{A}_{|S} \cong \mathcal{B}_{n,m,r}$ for some $n, m \geq 1$ and $r \in [n]$.*

Proof. By Theorem 7.39, S is a sink component. By Theorem 7.80, $\mathcal{A}_{|S}$ isomorphic to $\mathcal{B}_{n,m,r}$ for some $n, m \geq 1$ and $r \in [n]$. □

Remark 7.82 Let $\varphi\colon [n] \times [m] \to [n] \times [m]$ be an automorphism of $\mathcal{B}_{n,m,r}$. Then $\varphi((0,0)) = (s,t) = \mu_{n,m,r}((0,0), a^s b^t)$ for some $(s,t) \in [n] \times [m]$ and this value determines φ uniquely by Theorem 6.14. Thus, the automorphism group of $\mathcal{B}_{n,m,r}$ consists of all mappings $(i,j) \mapsto \mu((i,j), a^s b^t)$ for $(s,t) \in [n] \times [m]$. We see that this group equals $\mathcal{T}_M(\mathcal{B}_{n,m,r})$ and is regular. With Theorem 7.80 this gives another proof of Theorem 7.74

7.9.3 Subset Mapping Properties

We begin with some statements on synchronizing commutative automata.

Proposition 7.83 (Fernau and Hoffmann [212, Lem. 20]) *In a commutative automaton, a synchronizing state is a trap state.*

Proof. This follows from Theorem 7.51 since the sink components are blocks. □

Corollary 7.84 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative permutation-reset automaton that is not a permutation automaton. Then there exists a trap state $t \in Q$ such that for each $a \in \Sigma$ that is not a permutational letter we have $\delta(Q,a) = \{t\}$.*

Proof. By assumption, $\mathcal{A}$ has a letter mapping all states to the same state t. By Theorem 7.83, t is a trap state. Let $a \in \Sigma$ be some state that is not a permutational letter. Then $|\delta(Q,a)| = 1$ and, since t is a trap state, $t \in \delta(Q,a)$. This gives $\delta(Q,a) = \{t\}$. □

Proposition 7.85 *$\mathcal{A} = (Q, \Sigma, \delta)$ with $|Q| \geq 2$ is a synchronizing commutative permutation-reset automaton if and only if there exists a non-permutational letter, a trap state $t \in Q$, and $S = Q \setminus \{t\}$ is a block such that $\delta(q, ab) = \delta(q, ba)$ for all $q \in S$ and $a, b \in \Sigma$ with $\delta(q, ab) \in S$.*

Proof. First, suppose $\mathcal{A}$ is a synchronizing commutative permutation-reset automaton. By Theorem 7.84 we have a trap state t, a non-permutational letter, and for the upper set $S = \{t\}$ we have $\delta(Q, a) = Q$ or $\delta(Q, a) = \{t\}$ for each $a \in \Sigma$. Thus, S is a block. Since $\mathcal{A}$ is commutative, we have $\delta(q, ab) = \delta(q, ba)$ for all $q \in S$ and $a, b \in \Sigma$.

Conversely, by considering $\mathcal{A}/{\sim_{\{t\}}}$, which is isomorphic to $\mathcal{A}$, from Theorems 7.36 and 7.68 we find that it is a commutative permutation-reset automaton, which is synchronizing since it has a non-permutational letter mapping everything to t. □

Proposition 7.86 *A two-state automaton is commutative and synchronizing if and only if it has a trap state t, some non-identity letter, and every letter acts as the identity or maps both states to t.*

Proof. A synchronizing and commutative automaton has a trap state t by Theorem 7.83. Thus, each letter either fixes both states or maps both states to t. Conversely, an automaton of this form is easily seen to be synchronizing and commutative. □

Proposition 7.87 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative n-state automaton and $u \in \Sigma^*$.*

1. *We have $\delta(Q, u) = \delta(Q, v)$ for all $v \in \Sigma^*$ with $|v|_a = \min\{n-1, |u|_a\}$ for $a \in \Sigma$.*
2. *For $S \subseteq Q$, $|\delta(S, u)| = |\delta(S, v)|$ for all $v \in \Sigma^*$ with $|v|_a = \min\{n-1, |u|_a\}$ for $a \in \Sigma$.*

Proof. 1 Let $\Gamma \subseteq \Sigma$ be a largest alphabet such that, for all $v \in \Sigma^*$ with $|v|_a = \min\{n-1, |u|_a\}$ for $a \in \Gamma$ and $|v|_a = |u|_a$ for $a \in \Sigma \setminus \Gamma$, we have $\delta(Q, u) = \delta(Q, v)$. Since, by commutativity, $\Gamma = \emptyset$ fulfills this condition, such a set Γ exists.

Assume there exist $a \in \Sigma \setminus \Gamma$. Let $w \in \Sigma^*$ with $|w|_b = \min\{n-1, |u|_b\}$ for $b \in \Gamma$, $|w|_b = |u|_b$ for $b \in \Sigma \setminus \Gamma$ with $b \neq a$, and $|w|_a = 0$. By choice of Γ, we have $\delta(Q, u) = \delta(Q, a^{|u|_a} w)$. By Theorem 6.72 we find $\delta(Q, a^{|u|_a} w) = \delta(Q, a^{\min\{n-1, |u|_a\}} w)$. Set $w' = a^{\min\{n-1, |u|_a\}} w$. By commutativity, we have $\delta(Q, w') = \delta(Q, v)$ for all $v \in \Sigma^*$ with $|v|_b = |w'|_b$ for $b \in \Sigma$. Since $|w|_a = \min\{n-1, |u|_a\}$, $|w|_b = \min\{n-1, |u|_b\}$ for $b \in \Gamma$, and $|w|_b = |u|_b$ otherwise, and $\delta(Q, u) = \delta(Q, v)$, this contradicts the maximal choice of Γ since the condition by which we selected it is also fulfilled for $\Gamma \cup \{a\}$. Thus, we must have $\Sigma = \Gamma$, which gives the claim.

2: Let $S \subseteq Q$. We show, by induction, that for each $\Gamma \subseteq \Sigma$ we have $|\delta(S, u)| = |\delta(S, v)|$ for all $v \in \Sigma^*$ with $|v|_a = \min\{n-1, |u|_a\}$ for $a \in \Gamma$ and $|v|_a = |u|_a$ for $a \in \Sigma \setminus \Gamma$. Then picking $\Gamma = \Sigma$ gives the statement we want to prove. The claim clearly holds for $\Gamma = \emptyset$.

Next, consider $a \in \Sigma \setminus \Gamma$. Let $v \in \Sigma^*$ with $|v|_b = \min\{n-1, |u|_b\}$ for $b \in \Gamma \cup \{a\}$ and $|v|_b = |u|_b$ for $b \in \Sigma \setminus \Gamma$ with $b \neq a$. Choose $w \in \Sigma^*$ such that $|w|_b = \min\{n-1, |u|_b\}$ for $b \in \Gamma$, $|w|_b = |u|_b$ for $b \in \Sigma \setminus \Gamma$ with $b \neq a$ and $|w|_a = 0$. Then, by induction hypothesis, we have $|\delta(S, u)| = |\delta(S, wa^{|u|_a})|$. From Item 3 of Theorem 6.73, for $v' = wa^{\min\{n-1, |u|_a\}}$ we get

$$|\delta(S, wa^{|u|_a})| = |\delta(S, v')|.$$

We have $|v|_b = |v'|_b$ for all $b \in \Sigma$. Thus, by commutativity, $\delta(S, v) = \delta(S, v')$ and so $|\delta(S, u)| = |\delta(S, v)|$. This concludes the induction step. □

Corollary 7.88 *In a commutative n-state automaton, a shortest word reaching a given subset has length at most $|\Sigma|(n-1)$. The same bound holds for shortest words synchronizing a given subset in commutative automata.*

Note that Theorem 7.45 gives a better bound than $|\Sigma|(n-1)$ in commutative automata for words mapping the whole state set into a given subset.

Corollary 7.89 *Let $\Sigma = \{a_1, \ldots, a_k\}$. Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is a commutative n-state automaton and $u \in \Sigma^*$. Then we have*

1. $\delta(Q, a_1^{n-1} \cdots a_k^{n-1}) \subseteq \delta(Q, u)$ *and*
2. $|\delta(S, a_1^{n-1} \cdots a_k^{n-1})| \leq |\delta(S, u)|$ *for $S \subseteq Q$.*

Proof. Set $v = a_1^{\min\{n-1, |u|_{a_1}\}} \cdots a_k^{\min\{n-1, |u|_{a_k}\}}$ and $w = a_1^{n-1-|v|_{a_1}} \cdots a_k^{n-1-|v|_{a_k}}$. By Theorem 7.87 we have $\delta(Q, u) = \delta(Q, v)$ and $|\delta(S, u)| = |\delta(S, v)|$. Since $\delta(Q, w) \subseteq Q$ we have $\delta(Q, wv) \subseteq \delta(Q, v)$ and $|\delta(S, vw)| \leq |\delta(S, v)|$. By commutativity, we find $\delta(Q, wv) = \delta(Q, a_1^{n-1} \cdots a_k^{n-1})$ and $\delta(S, vw) = \delta(S, a_1^{n-1} \cdots a_k^{n-1})$, and this yields the claim. □

By Theorem 7.89, in a commutative n-state automaton we have $\delta(Q, u) \subseteq S$ (resp. $|\delta(S, u)| = 1$) for some $u \in \Sigma^*$ if and only if $\delta(Q, a_1^{n-1} \cdots a_k^{n-1}) \subseteq S$ (resp. $|\delta(S, a_1^{n-1} \cdots a_k^{n-1})| = 1$).

Because it easily follows from other results, we state a tight bound for shortest synchronizing words that was first obtained by Rystsov [533, Thm. 3.5] and [534, Thm. 3.2] (see also [212, Cor. 19] for another proof).

Proposition 7.90 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be commutative and $n = |Q|$.*

1. $\mathrm{Syn}(\mathcal{A})$ *is commutative.*
2. $uwv \in \mathrm{Syn}(\mathcal{A})$ *for all $u, v, w \in \Sigma^*$ with $uv \in \mathrm{Syn}(\mathcal{A})$.*
3. *(Rystsov [533, 534]) A shortest word in* $\mathrm{Syn}(\mathcal{A})$ *has length at most $n-1$ and this bound is tight.*

Proof. 1: By commutativity, $\delta(Q, uabv) = \{q\} = \delta(Q, ubav)$ for all $u, v \in \Sigma^*$ and $a, b \in \Sigma$, which yields that $\mathrm{Syn}(\mathcal{A})$ is commutative.

2: Since $\mathrm{Syn}(\mathcal{A})$ is an ideal by Theorem 7.1, we have $uvw \in \mathrm{Syn}(\mathcal{A})$. By Item 1, we get $uwv \in \mathrm{Syn}(\mathcal{A})$.

3: Since in a commutative automaton the strongly connected components are blocks, this follows from Theorem 7.43 with $r = 1$, or from Theorem 7.45 applied $S = \{t\}$ where t is the synchronizing state, which must be a trap state so that S is an upper set. □

Next, in Theorem 7.94, we show that weak connectivity and existence of a trap state is sufficient for synchronizability in commutative automata. This follows from a couple of statements we give next.

Lemma 7.91 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative automaton with sink component $S \subseteq Q$ and $\Gamma \subseteq \Sigma$. Then the set $T = \{q \in Q \mid \exists u \in \Gamma^* \colon \delta(q, u) \in S\}$ is an upper set such that S is the only sink component having non-empty intersection with T.*

Proof. Let $q \in T$ and $b \in \Sigma$. We have $\delta(q, u) \in S$ for some $u \in \Gamma^*$. By commutativity, we find $\delta(q, bu) = \delta(q, ub)$. Since $\delta(q, u) \in S$ and S is a sink component we have $\delta(q, ub) \in S$. Thus, $\delta(q, bu) \in S$ and so $\delta(q, b) \in T$. Hence, T is an upper set.

Next, let $R \subseteq Q$ be a sink component with $R \cap T \neq \emptyset$. By the definition of T, we have $\delta(R, u) \cap S \neq \emptyset$ for some $u \in \Gamma^*$. Since R is a sink component, we have $\delta(R, u) \subseteq R$. Thus $R \cap S \neq \emptyset$. Since distinct sink components are disjoint, this gives $R = S$. □

Corollary 7.92 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be commutative with sink component $S \subseteq Q$. Set $T = \{q \in Q \mid \exists u \in \Sigma^* \colon \delta(q, u) \in S\}$. Then T is a weakly connected component of the automaton graph of $\mathcal{A}$.*

Proof. Since the states in T are precisely those having a path leading into S, and S is strongly connected, T is weakly connected and thus contained in some weakly connected component. By the definition of T, there are no transitions leading into T since $\delta(q, a) \in T$ implies $q \in T$. Since T is an upper set by Theorem 7.91 there are also no transitions leading out of T since $\delta(q, a)$ with $q \in T$ implies $\delta(q, a) \in T$. Thus T equals the weakly connected component containing it, since otherwise there would either exist a transition going out of T or into T. □

Corollary 7.93 *A commutative automaton $\mathcal{A} = (Q, \Sigma, \delta)$ whose automaton graph is weakly connected has a unique sink component.*

Proof. Let $S \subseteq Q$ be a sink component and consider $T = \{q \in Q \mid \exists u \in \Sigma^* \colon \delta(q, u) \in S\}$. Since $\mathcal{A}$ is weakly connected, by Theorem 7.92, we find $Q = T$. From Theorem 7.91 we get that S is the only sink component in $\mathcal{A}$. □

Proposition 7.94 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative automaton. Then the following are equivalent.*

1. *$\mathcal{A}$ has a trap state and its automaton graph is weakly connected.*
2. *$\mathcal{A}$ has a trap state reachable from every other state.*
3. *$\mathcal{A}$ is synchronizing.*

Proof. 1 ⇒ 2: By Theorem 7.93 the trap state forms a unique sink component. Thus, it is reachable from every other state.

2 ⇒ 3: By Theorem 7.6, $\mathcal{A}$ is synchronizing.

3 ⇒ 1: By Theorem 7.112, a synchronizing state is a trap state. Clearly, a synchronizing automaton has an automaton graph that is weakly connected. □

Theorem 7.94 was stated in [210, Lem. 5.1].

7.9.4 Extremal Synchronizing Automata

Since commutative automata are ρ-automata, and every synchronizing commutative automaton has a trap state, Theorem 7.67 implies that every synchronizing commutative n-state automaton with reset length $n-1$ is partially ordered. We now determine the structure of those commutative automata that are extremal w.r.t. the reset length. Such automata were first classified in [212, Thm. 23]. Figure 7.10 shows a commutative automaton with the largest possible reset length.

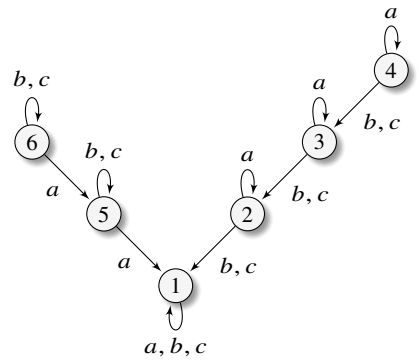


Fig. 7.10: Commutative six-state automaton with shortest synchronizing word $aabbb$.

Theorem 7.95 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a synchronizing commutative automaton with trap state t. Then the reset length of $\mathcal{A}$ equals $|Q| - 1$ if and only if we have a partition $\Gamma_1, \ldots, \Gamma_m$ of Σ and sets $T_{\Gamma_i} \subseteq Q$ such that*

1. *$T_{\Gamma_i} \cap T_{\Gamma_j} = \{t\}$ for $i \neq j$ and $Q = T_1 \cup \cdots \cup T_m$,*
2. *$T_{\Gamma_i} = \{q_{i,0}, \ldots, q_{i,m_i}\}$ for some $m_i \geq 0$,*
3. *$\delta(q_{i,j}, x) = q_{i,\min\{j+1,m_i\}}$ for $x \in \Gamma_i$, and*
4. *$\delta(q_{i,j}, x) = q_{i,j}$ for $x \in \Sigma \setminus \Gamma_i$.*

Proof. That automata of this form have reset length $|Q| - 1$ is clear.

Conversely, we do the proof only for a binary alphabet $\Sigma = \{a_1, a_2\}$, since for larger alphabets the arguments are essentially the same, but involve more indices and more complicated notation.

By Theorem 7.67, $\mathcal{A}$ is partially ordered. By Theorem 3.59 there exists a sequence of upper sets

$$S_1 \subsetneq S_2 \subsetneq \cdots \subsetneq S_m$$

such that $S_1 = \{t\}$ and $S_i \setminus S_{i-1}$ for $i \in \{2, \ldots, m\}$ is a singleton set since $\mathcal{A}$ is partially ordered. We show that the claim holds for each $\mathcal{A}_{|S_i}$ by induction.

For $i = 1$, the claim clearly holds for the single-state automaton $\mathcal{A}_{|S_1}$.

Next, let $i \geq 2$. We have $S_i \setminus S_{i-1} = \{q\}$ for some $q \in Q$. Without loss of generality, let $\delta(q, a_1) \neq q$. By Theorem 7.5, $\mathcal{A}_{|S_{i-1}}$ and $\mathcal{A}_{|S_i}$ have reset lengths $|S_{i-1}| - 1$ and

$|S_i| - 1$, respectively. Thus, by induction hypothesis, we either have $S_{i-1} = \{q_0, \ldots, q_l\}$ with $q_l = t$ and $\delta(q_j, x) = q_{\min\{j+1,l\}}$ for $j \in \{0, 1, \ldots, l\}$ for $x \in \Sigma$, or $S_i = \{q_{1,0}, \ldots, q_{1,l_1}, q_{2,0}, \ldots, q_{2,l_2}\}$ such that

1. $q_{l_1} = q_{l_2} = t$,
2. $\delta(q_{j,r}, a_j) = q_{j,\min\{r+1,l_j\}}$ for $j \in \{1, 2\}$ and $r \in \{0, 1, \ldots, l_j\}$,
3. $\delta(q_{j,r}, x) = q_{j,r}$ for $x \in \Sigma \setminus \{a_j\}$, $j \in \{1, 2\}$ and $r \in \{0, 1, \ldots, l_j\}$.

First, suppose the case $S_{i-1} = \{q_0, \ldots, q_l\}$ applies. If $\delta(q, a_1) \in \{q_1, \ldots, q_l\}$, then a_1^l synchronizes $\mathcal{A}_{|S_i}$, which is not possible since $l = |S_{i-1}| - 1 < |S_i| - 1$. Thus, $\delta(q, a_1) = q_0$. We also have $\delta(q, a_2) \notin \{q_1, \ldots, q_l\}$ since otherwise a_2^l synchronizes $\mathcal{A}_{|S_i}$. Next, $\delta(q, a_2) = q$ is excluded, since then $a_2 a_1^l$ ends at the state q_{l-1} and $a_1^l a_2$ ends at q_l, which contradicts commutativity of $\mathcal{A}$. Thus also $\delta(q, a_2) = q_0$. Then $\mathcal{A}_{|S_i}$ has the form as claimed.

Now, suppose the case $S_i = \{q_{1,0}, \ldots, q_{1,l_1}, q_{2,0}, \ldots, q_{2,l_2}\}$ applies. Assume $\delta(q, a_1) \in \{q_{2,0}, \ldots, q_{2,l_2}\}$ or $\delta(q, a_1) \in \{q_{1,1}, \ldots, q_{1,l_1}\}$. Then the word $a_1^{l_1} a_2^{l_2}$ also synchronizes $\mathcal{A}_{|S_i}$. Thus, we must have $\delta(q, a_1) = q_{1,0}$. Assume $\delta(q, a_2) \neq q$. Then $\delta(q, a_2 a_1) \neq \delta(q, a_1 a_2) = \delta(q, a_1)$, contradicting the commutativity of $\mathcal{A}$. Thus $\delta(q, a_2) = q$ and $\mathcal{A}_{|S_i}$ has the form as claimed.

Thus, we obtain the statement by induction. □

7.10 Solvable Automata

We call $\mathcal{A}$ *solvable* if $\delta(q, ab) \preceq_R q$ implies $\delta(q, ab) = \delta(q, ba)$ for all $q \in Q$ and $a, b \in \Sigma$. Clearly, every commutative automaton is solvable. Since for partially ordered automata, $\delta(q, ab) \preceq_R q$ implies $\delta(q, a) = \delta(q, b) = q$, and so $\delta(q, ab) = \delta(q, ba) = q$, partially ordered automata are also solvable. Clearly, commutative automata are solvable as well.

Here, we investigate solvable automata. In Section 7.10.1, we show that the class of solvable automata is closed under various operations. Section 7.10.2 gives other characterizations of solvability. In Section 7.10.3, we prove bounds on shortest words whose rank is bounded, and we give a technical result on words mapping one subset into another subset. Solvable automata were introduced by Rystsov [534] with a slightly different, more restricted, definition, we discuss this in Section 7.10.4.

7.10.1 Operations

Here, we prove that solvability is preserved in subautomata and morphic images, Rees quotient, product, and pair automata in Theorem 7.96, Theorem 7.97, Theorem 7.98, Theorem 7.99, and Theorem 7.101, respectively. Theorem 7.100 proves that solvability of $\mathcal{A}$, $\mathcal{A} \times \mathcal{A}$, and $\mathcal{A}^{\leq 2}$ are equivalent properties, and in Theorem 7.101 that solvability of $\mathcal{A}$ implies solvability of $\mathcal{A}^{\leq 2}/\sim_F$ and $(\mathcal{A} \times \mathcal{A})/\sim_D$ with $F = \{\{q\} \mid q \in Q\}$ and

$D = \{(q,q) \mid q \in Q\}$. In Theorem 7.102 we show that for permutation automata, solvability of $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ implies it for $\mathcal{A}$.

Proposition 7.96 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a solvable automaton and $S \subseteq Q$ be an upper set. Then $\mathcal{A}_{|S}$ is solvable.*

Proof. Suppose $\delta_{|S}(q, ab) \preceq_R q$ for some $q \in S$ and $a, b \in \Sigma$. Then $\delta(q, ab) \preceq_R q$ and, by solvability of $\mathcal{A}$, we have $\delta_{|S}(q, ab) = \delta(q, ab) = \delta(q, ba) = \delta_{|S}(q, ba)$. Hence, $\mathcal{A}_{|S}$ is solvable. □

Proposition 7.97 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$. Suppose there exists a surjective morphism $\varphi\colon Q \to S$. Then solvability of $\mathcal{A}$ implies solvability of $\mathcal{B}$.*

Proof. Suppose we have $\mu(\varphi(q), abu) = \varphi(q)$ for some $u \in \Sigma^*$. Then it is $\varphi(\delta(q, (abu)^i)) = \varphi(q)$ for all $i \geq 0$. By finiteness, there exist $i \geq 0$ and $p > 0$ such that $\delta(q, (abu)^{i+p}) = \delta(q, (abu)^i)$. Hence $\delta(q, (abu)^i ab) \preceq_R \delta(q, (abu)^i)$. Since $\mathcal{A}$ is solvable, we find $\delta(q, (abu)^i ab) = \delta(q, (abu)^i ba)$. Then

$$\begin{aligned}\mu(\varphi(q), ab) &= \mu(\varphi(\delta(q, (abu)^i)), ab) = \varphi(\delta(q, (abu)^i ab)) \\ &= \varphi(\delta(q, (abu)^i ba)) = \mu(\varphi(\delta(q, (abu)^i)), ba) = \mu(\varphi(q), ba).\end{aligned}$$

Hence, $\mathcal{B}$ is solvable. □

By Theorem 7.97 the class of solvable automata is closed under isomorphisms.

Corollary 7.98 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a solvable automaton and $S \subseteq Q$ be an upper set. Then $\mathcal{A}/{\sim_S}$ is solvable.*

Proof. The canonical morphism for $\sim_S$ from Theorem 6.55 is surjective and maps $\mathcal{A}$ onto $\mathcal{A}/{\sim_S}$. So the claim follows from Theorem 7.97. □

Proposition 7.99 *Let $\mathcal{A}_1 = (Q_1, \Sigma, \delta_1)$ and $\mathcal{A}_2 = (Q_2, \Sigma, \delta_2)$. Then $\mathcal{A}_1$ and $\mathcal{A}_2$ are solvable if and only if $\mathcal{A}_1 \times \mathcal{A}_2$ is solvable.*

Proof. First, assume $\mathcal{A}_1 \times \mathcal{A}_2$ is solvable. Since the maps $(p, q) \mapsto p$ and $(p, q) \mapsto q$ are surjective morphisms from $\mathcal{A}_1 \times \mathcal{A}_2$ to $\mathcal{A}_1$ and $\mathcal{A}_2$, respectively, we find that $\mathcal{A}_1$ and $\mathcal{A}_2$ are solvable by Theorem 7.97.

Now, assume $\mathcal{A}_1$ and $\mathcal{A}_2$ are solvable. Let $(p, q) \in Q_1 \times Q_2$ and $a, b \in \Sigma$ be such that $\delta_{\mathcal{A}_1 \times \mathcal{A}_2}((p, q), ab) \preceq_R (p, q)$. This gives $\delta_1(p, ab) \preceq_R p$ and $\delta_2(q, ab) \preceq_R q$.[5] Since $\mathcal{A}_1$ and $\mathcal{A}_2$ are solvable, we find $\delta_1(q, ab) = \delta_1(q, ba)$ and $\delta_2(p, ab) = \delta_2(q, ba)$. Thus, we get $\delta_{\mathcal{A}_1 \times \mathcal{A}_2}((p, q), ab) = \delta_{\mathcal{A}_1 \times \mathcal{A}_2}((p, q), ba)$ from the definition of $\mathcal{A}_1 \times \mathcal{A}_2$. □

Corollary 7.100 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $i \geq 2$. Then the following are equivalent.*

1. *$\mathcal{A}$ is solvable.*

[5] Note that $\delta_1(p, ab) \preceq_R p$ and $\delta_2(q, ab) \preceq_R q$ not necessarily imply $\delta_{\mathcal{A}_1 \times \mathcal{A}_2}((p, q), ab) \preceq_R (p, q)$.

2. $\mathcal{A} \times \cdots \times \mathcal{A}$ *(i times) is solvable.*
3. $\mathcal{A} \times \mathcal{A}$ *is solvable.*
4. $\mathcal{A}^{\leq 2}$ *is solvable.*

Proof. $1 \Rightarrow 2$: This follows from Theorem 7.99 applied i times.

$2 \Rightarrow 3$: We can assume $i > 2$. Then

$$\mathcal{A} \times \cdots \times \mathcal{A} = \underbrace{(\mathcal{A} \times \cdots \times \mathcal{A})}_{i-2 \text{ times}} \times (\mathcal{A} \times \mathcal{A}).$$

By Theorem 7.99 we obtain that $\mathcal{A} \times \mathcal{A}$ is solvable.

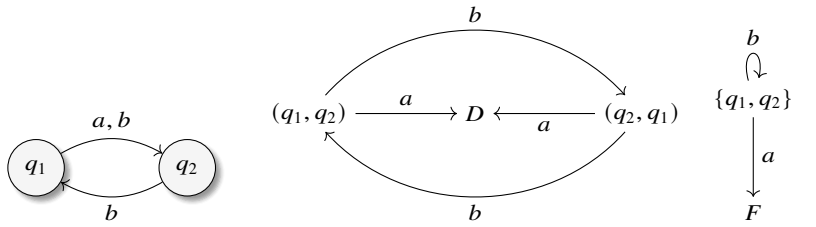
$3 \Rightarrow 4$: We have $\mathcal{A}^{\leq 2} \cong (\mathcal{A} \times \mathcal{A})/{\sim}$ for the $\mathcal{T}_S(\mathcal{A} \times \mathcal{A})$-congruence $\sim$ given by $(p_1, p_2) \sim (q_1, q_2)$ if and only if $\{p_1, p_2\} = \{q_1, q_2\}$. So, the claim follows from Theorem 7.98.

$4 \Rightarrow 1$: As $(\mathcal{A}^{\leq 2})_{|F} \cong \mathcal{A}$ with $F = \{\{q\} \mid q \in F\}$, the claim follows from Theorem 7.98. □

Corollary 7.101 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be solvable. Then the automata $\mathcal{A}^{\leq 2}/{\sim_F}$ and $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ with $F = \{\{q\} \mid q \in Q\}$ and $D = \{(q, q) \mid q \in Q\}$ are solvable.*

Proof. For solvable $\mathcal{A} = (Q, \Sigma, \delta)$, $\mathcal{A}^{\leq 2}$ and $\mathcal{A} \times \mathcal{A}$ are solvable by Theorem 7.100. So, Theorem 7.98 gives the claim. □

The converse statement to Theorem 7.101 does not hold in general, as the example from Figure 7.11 shows.



(a) The automaton $\mathcal{A}$. (b) $(\mathcal{A} \times \mathcal{A})/{\sim_D}$, which is solvable. (c) $\mathcal{A}^{\leq 2}/{\sim_F}$

Fig. 7.11: The automaton $\mathcal{A}$ is not solvable, but $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ and $\mathcal{A}^{\leq 2}/{\sim_F}$ with $D = \{(q, q) \mid q \in Q\}$ and $F = \{\{q\} \mid q \in Q\}$ are solvable. Self-loops omitted.

However, for permutation automata, a converse of Theorem 7.101 holds for $(\mathcal{A} \times \mathcal{A})/{\sim_D}$.

Proposition 7.102 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a permutation automaton. Then solvability of $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ with $D = \{(q, q) \mid q \in Q\}$ implies that $\mathcal{A}$ is solvable.*

Proof. We can assume $|Q| \geq 2$. Clearly, $\mathcal{A}$ is solvable if each subautomaton of the form $\mathcal{A}_{|\uparrow\{q\}}$ for each $q \in Q$ is solvable. Thus, we can assume $Q = \uparrow\{q\}$ for some $q \in Q$ in

the following. Let $p \in Q$ be a state with $p \neq q$. Consider the upper set $S = \uparrow\{(p,q)\}$ in $\mathcal{A} \times \mathcal{A}$. Since $p \neq q$ and $\mathcal{A}$ is a permutation automaton, we have

$$(\mathcal{A} \times \mathcal{A})_{|S} \cong ((\mathcal{A} \times \mathcal{A})/{\sim_D})_{|\pi_{\sim_D}(S)} .$$

and the projection mapping restricted to S, i.e., $(\pi_2)_{|S} S \to Q$, is a surjective morphism from $(\mathcal{A}\times\mathcal{A})_{|S}$ onto $\mathcal{A}$. Thus, by Theorems 7.96 and 7.97 we find that $\mathcal{A}$ is solvable. □

The partition $\{\{(p,q),(q,p)\}\} \cup \{\{D\}\}$ of the state set of $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ induces a congruence whose quotient automaton is isomorphic to $\mathcal{A}^{\leq 2}/{\sim_F}$. Then by Theorems 6.55 and 7.97 it follows that $\mathcal{A}^{\leq 2}/{\sim_F}$ is solvable if $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ is solvable. However, we do not know whether the converse implication holds.

7.10.2 Characterizations

Solvable automata enjoy a wealth of other characterizations as we show next. Theorem 7.103 collects various characterizations in terms of strongly connected components. Theorem 7.107 characterizes solvability by increasing sequences of upper sets, resembling one definition originally given by Rystsov [534] (see the discussion in Section 7.10.4). Theorem 7.108 gives definition in terms of a single upper sets, some of them are suitable for an inductive reasoning and construction of solvable automata, starting from commutative automata.

Theorem 7.103 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is solvable.*
2. *For each $q \in Q$, the automaton*

   $$(\mathcal{A}_{|S})/{\sim_T}$$

   with $S = \uparrow\{q\}$ and $T = S \setminus [q]_{\simeq_R}$ is commutative.
3. *For each $q \in Q$, the automaton*

   $$(\mathcal{A}_{|S})/{\sim_T}$$

   with $S = \uparrow\{q\}$ and $T = S \setminus [q]_{\simeq_R}$ is a strongly trap-connected commutative permutation-reset automaton or a commutative permutation automaton (where the latter case includes $T = \emptyset$, in which case $\mathcal{A}$ is strongly connected).
4. *For all upper sets $S, T \subseteq Q$ such that $T \subseteq S$ and $S \setminus T$ is a strongly connected component, the automaton $(\mathcal{A}_{|S})/{\sim_T}$ is a commutative permutation-reset automaton.*
5. *For each strongly connected component $S \subseteq Q$ the automaton $\mathcal{A}_{|\Gamma_1,S}$ with $\Gamma_1 = \{a \in \Sigma \mid \delta(S,a) = S\}$ is a commutative permutation automaton and $\Sigma = \Gamma_1 \cup \Gamma_2$ for $\Gamma_2 = \{a \in \Sigma \mid \delta(S,a) \cap S = \emptyset\}$.*
6. *Each strongly connected component $S \subseteq Q$ is a block such that $\delta(q,ab) = \delta(q,ba)$ for all $q \in S$ and $a, b \in \Sigma$ with $\delta(S,a) = \delta(S,b) = S$.*

7. *For each strongly connected component* $S \subseteq Q$ *and* $a, b \in \Sigma$ *with* $\delta(S, a) \cap S \neq \emptyset$ *and* $\delta(S, b) \cap S \neq \emptyset$ *we have* $\delta(q, ab) = \delta(q, ba)$ *for all* $q \in S$.
8. *For each strongly connected* $S \subseteq Q$ *and* $a, b \in \Sigma$ *with* $\delta(S, a) \cap S \neq \emptyset$ *and* $\delta(S, b) \cap S \neq \emptyset$ *we have* $\delta(q, ab) = \delta(q, ba)$ *for all* $q \in S$.
9. *For each strongly connected* $S \subseteq Q$ *we have* $\delta(q, ab) = \delta(q, ba)$ *for all* $q \in S$ *and all* $a, b \in \Sigma$ *with* $\delta(q, ab) \in S$.
10. *For each* $q \in Q$ *and* $u \in \Sigma^*$, $\delta(q, u) = q$ *implies* $\delta(q, v) = \delta(q, u)$ *for all* $v \in \Sigma^*$ *such that* $|v|_x = |u|_x$ *for all* $x \in \Sigma$.
11. *For each* $q \in Q$ *and* $u \in \Sigma^*$, $\delta(q, u) = q$ *implies*

$$\delta(q, v) = \delta(q, v_{\pi(1)} \cdots v_{\pi(m)})$$

for each permutation $\pi \colon \{1, \ldots, m\} \to \{1, \ldots, m\}$ *and* $v \in \Sigma^*$ *with* $u \in v\Sigma^*$ *and* $v = v_1 \cdots v_m$ *for* $v_1, \ldots, v_m \in \Sigma$.

Proof. 1 $\Rightarrow$ 2: Let $p \in S$ with $p \notin T$. Assume $\delta_{|S}(p, ab) \neq \delta_{|S}(p, ab)$. Then $\delta(p, ab) \neq \delta(p, ba)$ and since $\mathcal{A}$ is solvable, we have $\delta(p, ab) \not\simeq_R p$ and $\delta(p, ab) \not\simeq_R p$. Since $p \simeq_R q$, we find $\delta(p, ab) \not\simeq_R q$ and $\delta(p, ba) \not\simeq_R q$ and so $\delta(p, ab), \delta(p, ba) \in T$ by Theorem 6.3 and Equation (3.32). Hence, in $(\mathcal{A}_{|S})/{\sim_T}$, the state p is mapped to T by both words ab and ba. This already establishes commutativity of $(\mathcal{A}_{|S})/{\sim_T}$ since if $\delta_{|S}(p, ab) = \delta_{|S}(p, ab)$, then also in $(\mathcal{A}_{|S})/{\sim_T}$ the state p is mapped to the same state by ab and ba.

2 $\Rightarrow$ 3: By assumption $(\mathcal{A}_{|S})/{\sim_T}$ is commutative. Hence, each letter acts as a morphism on its state set, which is $[q]_{\simeq_R} \cup \{T\}$. By Theorem 6.35, each letter either permutes $[q]_{\simeq_R}$ or maps all states in it to T. So, $(\mathcal{A}_{|S})/{\sim_T}$ is a permutation-reset automaton. If there exists a letter mapping $[q]_{\simeq}$ to T, then it is not a sink component and by Theorem 6.10, Item 2 it is strongly trap-connected. Otherwise, since T is a trap state for $T \neq \emptyset$, each letter permutes all states.

Alternatively, this follows with Theorem 7.70.

3 $\Rightarrow$ 4: First, assume S is a sink component. Then, by Item 3, $\mathcal{A}_{|S}$ is a commutative permutation automaton. So, $(\mathcal{A}_{|S})/{\sim_T}$ is a commutative permutation automaton, since for $T \neq \emptyset$ merely a trap state is added. Next, assume S is not a sink component. By assumption, we have $S \setminus T = [q]_{\simeq_R}$ for some $q \in Q$. Since S is an upper set, we have $\uparrow\{q\} \subseteq S \subseteq T$. Set $K = \uparrow\{q\}$ and $R = K \setminus [q]_{\simeq_R}$, where $R \neq \emptyset$ and $T \neq \emptyset$ as S is a sink component. Then, it is

$$(\mathcal{A}_{|S})/{\sim_T} \cong (\mathcal{A}_{|K})/{\sim_R}$$

since $\sim_T$ on S equals $[q]_{\simeq_R} \times [q]_{\simeq_R} \cup \{T\}$ and $\sim_R$ on K equals $[q]_{\simeq_R} \times [q]_{\simeq_R} \cup \{R\}$ and so both induce isomorphic Rees quotient automata. By Item 3, $(\mathcal{A}_{|K})/{\sim_R}$ is a commutative permutation-reset automaton.

4 $\Rightarrow$ 5: Let $S \subseteq Q$ be a strongly connected component. By assumption, $(\mathcal{A}_{|R})/{\sim_T}$ with $R = \uparrow S$ and $T = R \setminus S$ is a permutation-reset automaton, which is equivalent to S being a block by Theorem 7.36, which gives $\Sigma = \Gamma_1 \cup \Gamma_2$, and by Theorem 7.68 it is commutative so that $\mathcal{A}_{|\Gamma_1, S}$ is a commutative permutation automaton.

5 $\Rightarrow$ 6: This is clear.

$6 \Rightarrow 7$: Since a strongly connected component S is a block by assumption, $\delta(S,a) \cap S \neq \emptyset$ and $\delta(S,b) \cap S \neq \emptyset$ imply $\delta(S,a) = \delta(S,b) = S$ and then $\delta(q,ab) = \delta(q,ba)$ for all $q \in S$.

$7 \Rightarrow 8$: Let $S \subseteq Q$ be strongly connected, $q \in \delta(S,a) \cap S$, and $p \in \delta(S,b) \cap S$. Set $T = [q]_{\simeq}$. Then $S \subseteq T$, $q \in \delta(T,a) \cap T$, $p \in \delta(T,b) \cap T$, and so $\delta(r,ab) = \delta(r,ba)$ for all $r \in T$, in particular for those states from S.

$8 \Rightarrow 9$: Let S, $q \in S$ and $a, b \in \Sigma$ as in the statement. Since $\delta(q,ab) \in S$ and $q \in S$, the set $T = S \cup \{\delta(q,a)\}$ is also strongly connected. we have $\delta(q,a) \in T$ and $\delta(\delta(q,a),b) \in T$ so that, by Item 8, we find $\delta(q,ab) = \delta(q,ba)$.

$9 \Rightarrow 10$: Write $u = u_1 \cdots u_n$ with $u_i \in \Sigma$ for $i \in \{1,\ldots,n\}$. Let $i \in \{1,\ldots,n\}$. Since $\delta(q,u) = q$, we have

$$q \simeq_R \delta(q, u_1 \cdots u_{i-1}) \simeq_R \delta(q, u_1 \cdots u_{i-1}u_i) \simeq_R \delta(q, u_1 \cdots u_{i-1}u_iu_{i+1}).$$

So, the set $S = \{q, \delta(q,u_1 \cdots u_{i-1}), \delta(q,u_1 \cdots u_{i-1}u_i), \delta(q,u_1 \cdots u_{i-1}u_iu_{i+1}), q\}$ is strongly connected. By assumption, we get $\delta(p,u_iu_{i+1}) = \delta(p,u_{i+1}u_i)$ for all $p \in S$, in particular we have

$$\delta(q,u_1 \cdots u_{i-1}u_iu_{i+1}) = \delta(q,u_1 \cdots u_{i-1}u_{i+1}u_i).$$

Since every permutation of the letters $u_1,\ldots,u_n$ can be realized by swapping adjacent letters, we obtain the claim.

$10 \Rightarrow 11$: Let $q \in Q$ and $u, v, w \in \Sigma^*$ be such that $\delta(q,u) = q$ and $u = vw$. Write $v = v_1 \cdots v_m$ for $v_1,\ldots,v_m \in \Sigma$ and consider a permutation $\pi\colon \{1,\ldots,m\} \to \{1,\ldots,m\}$. Since $\delta(q,vw) = q$ we have $\delta(q,v) = \delta(q,vwv)$ and $\delta(q, v_{\pi(1)} \cdots v_{\pi(m)}) = \delta(q, vwv_{\pi(1)} \cdots v_{\pi(m)})$. By assumption, the former equality gives $\delta(q, vwv_{\pi(1)} \cdots v_{\pi(m)}) = \delta(q,v)$. Hence, we find $\delta(q,v) = \delta(q, v_{\pi(1)} \cdots v_{\pi(m)})$. This gives the claim.

$11 \Rightarrow 1$: Let $p \in Q$ and $a, b \in \Sigma$ such that $\delta(q,ab) \precsim_R q$. Then $\delta(q,abu) = q$ for some $u \in \Sigma^*$ and, by assumption, we have $\delta(q,ab) = \delta(q,ba)$. □

Our first corollary extends Theorem 7.71.

Corollary 7.104 *A strongly trap-connected (resp. strongly connected) solvable automaton $\mathcal{A} = (Q,\Sigma,\delta)$ is a commutative permutation-reset (resp. commutative permutation) automaton.*

Proof. By Theorem 6.10, Item 2 we have $Q = [q]_{\simeq_R} \cup \{t\}$ for a non-maximal $q \in Q$ and a trap state $t \in Q$ (resp. $Q = [q]_{\simeq_R}$ for some $q \in Q$). Set $S = \uparrow\{q\}$ and $T = S \setminus [q]_{\simeq_R}$. Since $Q = S$ and $T = \{t\}$ (resp. $T = \emptyset$) from Theorem 7.103, Item 3 and Equation (6.12) we get that $\mathcal{A}$ is a commutative permutation-reset (resp. commutative permutation) automaton. □

Corollary 7.105 *Let $\mathcal{A} = (Q,\Sigma,\delta)$ be solvable and $S \subseteq Q$ a non-empty union of sink components (as, for example, the set of maximal states). Then $\mathcal{A}_{|S}$ is a commutative permutation automaton.*

Proof. For each sink component $T \subseteq S$, $\mathcal{A}_{|T}$ is solvable and strongly connected, and so a commutative permutation automaton by Theorem 7.104, which gives the claim. □

Corollary 7.106 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then $\mathcal{A}/{\sim_S}$ is solvable if and only if for each strongly connected component $T \subseteq Q$ we have $\delta(q, ab) = \delta(q, ba)$ for all $a, b \in \Sigma$ and $q \in T \setminus S$ with $\delta(q, ab) \in T$.*

Proof. This follows directly from Theorem 7.103, Item 9, the definition of the Rees quotient automaton, and the fact that a strongly connected set is either completely outside of S or completely inside of S. □

Proposition 7.107 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is solvable.*
2. *There exists a sequence of upper sets $S_1 \subsetneq \cdots \subsetneq S_m = Q$ where S_1 equals the union of all sink components, $\mathcal{A}_{|S_1}$ is a commutative permutation automaton, and, for $i \in \{2, \ldots, m\}$, $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is a strongly trap-connected commutative permutation-reset automaton and $S_i \setminus S_{i-1}$ are all the strongly connected components that are not sink components.*
3. *There exists a sequence of upper sets $\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$ such that $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is a commutative permutation-reset automaton and $S_i \setminus S_{i-1}$ are all the strongly connected components for $i \in \{1, \ldots, m\}$.*
4. *There exists a sequence of upper sets $\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$ such that $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is a solvable automaton for each $i \in \{1, \ldots, m\}$.*

Proof. 1 $\Rightarrow$ 2: Consider a sequence of upper sets as in Theorem 3.59, Item 2. By Theorem 7.105, $\mathcal{A}_{|S_0}$ is a commutative permutation automaton. Next, each set S_i for $i \in \{1, \ldots, m\}$ has the form $S_i = [q]_{\simeq} \cup S_{i-1}$ for some $q \in S_i \setminus S_{i-1}$, where q is not maximal since S_1 equals the set of maximal states. By Theorem 7.103, Item 3, $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is a commutative permutation-reset automaton.[6] Since q is not a maximal state, $[q]_{\simeq}$ is not a sink component and so $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is strongly trap-connected by Theorem 6.10.

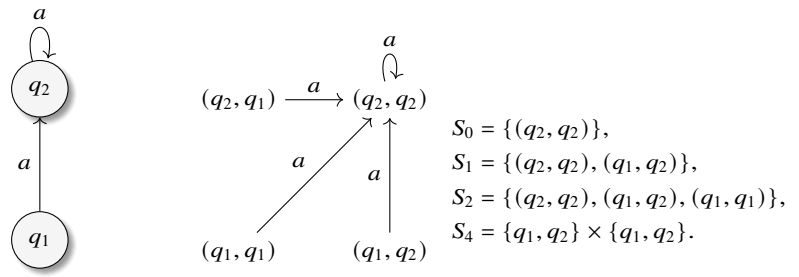
2 $\Rightarrow$ 3: Consider a sequence $S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Item 2. Applying Theorem 3.59, Item 1 to $\mathcal{A}_{|S_1}$ there exists a sequence $\emptyset = T_0 \subsetneq T_1 \subsetneq \cdots \subsetneq T_l$ such that $T_i \setminus T_{i-1}$ for $i \in \{1, \ldots, l-1\}$ are all the sink components. Since $\mathcal{A}_{|S_1}$ is a commutative permutation automaton, each $\mathcal{A}_{|T_i}$ is a commutative permutation automaton. Then each $(\mathcal{A}_{|T_i})/{\sim_{T_{i-1}}}$ is a commutative permutation automaton. Now, $T_0 \subsetneq T_1 \subsetneq \cdots \subsetneq T_l \subsetneq S_2 \subsetneq \cdots \subsetneq S_m$ yields a sequence as claimed.

3 $\Rightarrow$ 4: As a commutative automaton is solvable, we obtain the claim.

4 $\Rightarrow$ 1: Let $q \in Q$ and $a, b \in \Sigma$ be such that $\delta(q, ab) \precsim_R q$. Since, by Theorem 6.3, $\delta(q, ab)$ and q are contained in the same strongly connected component, by Theorem 3.37, we have $\{\delta(q, ab), q\} \subseteq S_i \setminus S_{i-1}$ for some $i \in \{1, \ldots, m\}$. Since $(\mathcal{A}_{|S_i})/{\sim_{S_i}}$ is solvable, by Theorem 6.61 we have $\{\delta_{|S_i}(q, ab)\} = \delta_{\sim_{S_i}}([q]_{\sim_{S_i}}, ab) = \delta_{\sim_{S_i}}([q]_{\sim_{S_i}}, ba) = \{\delta_{|S_i}(q, ba)\}$ where $[q]_{\sim_{S_i}} = \{q\}$, and so $\delta(q, ab) = \delta(q, ba)$. Hence, $\mathcal{A}$ is solvable. □

Figure 7.12 gives a commutative automaton with a sequence as in Theorem 7.107, Item 2.

[6] Alternatively, since the quotient automata $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ for $i \in \{2, \ldots, m\}$ are strongly trap-connected by Theorem 6.10 and solvable by Theorem 7.98, from Theorem 7.104 we obtain that they are commutative permutation-reset automata.



(a) An automaton $\mathcal{A}$. (b) The automaton $\mathcal{A} \times \mathcal{A}$. (c) Sequence of upper sets.

Fig. 7.12: A sequence of subsets $S_0 \subseteq S_1 \subseteq S_2 \subseteq S_3$ as stated in Theorem 7.107, Item 2 for a unary (and so solvable) automaton of the form $\mathcal{A} \times \mathcal{A}$.

Proposition 7.108 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is solvable.*
2. *$\mathcal{A}$ is a commutative permutation-reset automaton or there exists a proper upper set $S \subseteq Q$ with $|S| \geq 2$ such that $\mathcal{A}_{|S}$ is solvable and $\mathcal{A}/{\sim_S}$ is a commutative and strongly trap-connected permutation-reset automaton.*
3. *$\mathcal{A}$ is a commutative permutation-reset automaton or there exists a proper upper set $S \subseteq Q$ with $|S| \geq 2$ such that $\mathcal{A}_{|S}$ is a commutative permutation-reset automaton and $\mathcal{A}/{\sim_S}$ is solvable.*
4. *$\mathcal{A}$ is commutative or there exists a proper upper set $S \subseteq Q$ with $|S| \geq 2$ such that $\mathcal{A}_{|S}$ and $\mathcal{A}/{\sim_S}$ are solvable.*
5. *There exists an upper set $S \subseteq Q$ such that $\mathcal{A}_{|S}$ and $\mathcal{A}/{\sim_S}$ are solvable.*

Proof. $1 \Rightarrow 2$ & $1 \Rightarrow 3$: Consider a sequence $S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Theorem 7.107, Item 2. For $m = 1$, Q equals the set of maximal states and $\mathcal{A}$ is a commutative permutation automaton and so Items 2 and 3 hold. Next, suppose $m \geq 2$. Then $\mathcal{A}/{\sim_{S_{m-1}}}$ is a commutative and strongly trap-connected permutation-reset automaton. For $|S_{m-1}| = 1$, we have $\mathcal{A} \cong \mathcal{A}/{\sim_{S_{m-1}}}$ and this shows Item 2. Otherwise, since $\mathcal{A}_{|S_{m-1}}$ is solvable by Theorem 7.96, Item 2 holds with $S = S_{m-1}$.

Next, if $|S_1| \geq 2$, then Item 3 is established since $\mathcal{A}_{|S_1}$ is a commutative permutation automaton and $\mathcal{A}/{\sim_{S_1}}$ is solvable by Theorem 7.98. For $|S_1| = 1$, we have $\mathcal{A}_{|S_2} \cong (\mathcal{A}_{|S_2})/{\sim_{S_1}}$, and the latter automaton is a commutative permutation-reset automaton by choice of the sequence. Thus for $m = 2$, $\mathcal{A}$ is a commutative permutation-reset automaton and Item 3 holds or Item 3 holds with the proper upper set of size at least two $S = S_2$.

$2 \Rightarrow 4$ & $3 \Rightarrow 4$ & $4 \Rightarrow 5$: These implications are clear.

$5 \Rightarrow 1$: Let $S \subseteq Q$ be an upper set such that $\mathcal{A}_{|S}$ and $\mathcal{A}/{\sim_S}$ are solvable. We can assume $0 < |S| < |Q|$. Then $\emptyset \subsetneq S \subsetneq Q$ gives a sequence as in Theorem 7.107, Item 4. So $\mathcal{A}$ is solvable. □

Theorem 7.108, Item 2 is similar to Theorem 6.79, but gives more information on automata-theoretical properties.

Remark 7.109 Theorem 7.108, Item 4 can be used to give an alternative inductive proof of the statement from Theorem 7.99 that $\mathcal{A}_1 \times \mathcal{A}_2$ is solvable for solvable $\mathcal{A}_1, \mathcal{A}_2$. Since, if $\mathcal{A}_1$ and $\mathcal{A}_2$ are commutative, then it is readily seen that $\mathcal{A}_1 \times \mathcal{A}_2$ is commutative. Next, if there exists, without loss of generality, an upper set $S_1 \subseteq Q_1$ with $2 \leq |S_1| < |Q_1|$ such that $(\mathcal{A}_1)_{|S_1}$ and $\mathcal{A}_1/{\sim_{S_1}}$ are solvable, then the set $T = S_1 \times Q_2$ is an upper set in $\mathcal{A}_1 \times \mathcal{A}_2$ and by Equations (6.18) and (6.19) we have

$$(\mathcal{A}_1 \times \mathcal{A}_2)_{|T} \cong (\mathcal{A}_1)_{|S_1} \times \mathcal{A}_2 \quad \text{and} \quad (\mathcal{A}_1 \times \mathcal{A}_2)/{\sim_T} \cong \mathcal{A}_1/{\sim_{S_1}}.$$

Inductively, $(\mathcal{A}_1)_{|S_1} \times \mathcal{A}_2$ is solvable. So we obtain solvability of $\mathcal{A}_1 \times \mathcal{A}_2$.

Remark 7.110 Theorem 7.108, Item 4 can be used to give an alternative proof for Theorem 7.96 using the various isomorphisms established in Section 6.5.3 as we shown next with the same notation as in Theorem 7.96. First, commutativity of $\mathcal{A}$ transfers to $\mathcal{A}_{|S}$. Next, suppose there exists an upper set $R \subseteq Q$ with $2 \leq |R| < |Q|$ such that $\mathcal{A}_{|R}$ and $\mathcal{A}/{\sim_R}$ are solvable. By Equation (6.7) we have

$$(\mathcal{A}_{|S})_{|S\cap R} \cong \mathcal{A}_{|S\cap R} \cong (\mathcal{A}_{|R})_{|S\cap R},$$

and with the canonical projection map $\pi_{\sim_R} : Q \to Q/{\sim_R}$ introduced in Section 3.4, by Equation (6.13),

$$(\mathcal{A}_{|S})/{\sim_{S\cap R}} \cong (\mathcal{A}/{\sim_R})_{|\pi_{\sim_R}(S)}.$$

Inductively, $(\mathcal{A}_{|R})_{|S\cap R}$ and $(\mathcal{A}/{\sim_R})_{|\pi_{\sim_R}(S)}$ are solvable. So, by Theorem 7.108, Item 4 or by Equation (6.12), $\mathcal{A}_{|S}$ is solvable.

Remark 7.111 Theorem 7.108, Item 4 can be used to give an alternative proof for Theorem 7.97 using the various isomorphisms established in Section 6.5.3 as we shown next with the same notation as in Theorem 7.97. First, if $\mathcal{A}$ is commutative, then clearly $\mathcal{B}$ is commutative. Now, suppose there exists an upper set $R \subseteq Q$ with $2 \leq |R| < |Q|$ such that $\mathcal{A}_{|R}$ and $\mathcal{A}/{\sim_R}$ are solvable. The set $\varphi(R)$ is an upper set in $\mathcal{B}$ by Theorem 6.25. Now, $\varphi_{|R}$ considered as a surjective function from R onto $\varphi(R)$ is a surjective morphism from $\mathcal{A}_{|R}$ onto $\mathcal{B}_{|\varphi(R)}$. By Theorem 6.67 there exists a surjective morphism from $\mathcal{A}/{\sim_R}$ onto $(\mathcal{B}/{\sim_{\varphi(R)}})_{|\pi_{\sim_{\varphi(R)}}(\varphi(Q))}$, where $(\mathcal{B}/{\sim_{\varphi(R)}})_{|\pi_{\sim_{\varphi(R)}}(\varphi(Q))} = \mathcal{B}/{\sim_{\varphi(R)}}$ as $\varphi(Q) = S$. By induction hypothesis, as $\mathcal{A}_{|R}$ and $\mathcal{A}/{\sim_R}$ are solvable, $\mathcal{B}_{|\varphi(R)}$ and $\mathcal{B}/{\sim_{\varphi(R)}}$ are solvable. So, by Theorem 7.108, Item 4, $\mathcal{B}$ is solvable.

7.10.3 Synchronization and Subset Mapping Properties

First, in Theorem 7.112, we state that in solvable automata, a synchronizing state is a trap state (and hence unique). Theorem 7.113 states the bound $n - r$ for shortest words of rank at most r in solvable automata that admit a word of rank r, thus extending Rystsov's bound for $r = 1$ [534, Cor. 4.2]. Proposition 7.114 gives words of a specific form to map one subset onto another subset in solvable automata. This result is rather technical, and we need it in Section 8.3.3 (Theorem 8.18) to prove that SetTrans is in NP for solvable automata.

Proposition 7.112 *A synchronizing state in a solvable automaton is a trap state.*

Proof. By Theorem 7.103, Item 6 in solvable automata the sink components are blocks. So, the claim follows from Theorem 7.51. □

Theorem 7.113 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state solvable automaton. If there exists a word of rank r in $\mathcal{A}$, then there exists a word of length at most $n - r$ whose rank is at most r and these bounds are tight even for unary automata where r is the minimal rank among all words. Furthermore, for $\Sigma = \{a_1, \ldots, a_k\}$, the word $(a_1 \cdots a_k)^{n-r}$ has rank at most r.*

Proof. By Theorem 7.103, Item 6 the strongly connected components in $\mathcal{A}$ are blocks. So, Theorem 7.43 gives the upper bound $n-r$ and that $(a_1 \cdots a_k)^{n-r}$ has rank at most r. Tightness as claimed follows from the automata given in the proof of Theorem 7.29. □

Proposition 7.114 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be solvable with n states, $S \subseteq Q$, and $w \in \Sigma^*$. Then for $\Sigma = \{a_1, \ldots, a_k\}$ there exists $v \in \Sigma^*$ with $|v| < 2^n$ and $\delta(S, v) = \delta(S, w)$ of the form*

$$v = v_0 x_1 \cdots v_{r-1} x_r v_r$$

for some $r \leq n^2$ where $x_1, \ldots, x_r \in \Sigma$ and $v_i = a_1^{r_{i,1}} \cdots a_k^{r_{i,k}}$ for numbers $0 \leq r_{i,j} < 2^n$ with $i \in \{0, 1, \ldots, r\}$ and $j \in \{1, \ldots, k\}$.

Proof. We can assume $S \neq \emptyset$. By Theorem 7.107, Item 2 there exist upper sets

$$S_1 \subsetneq \cdots \subsetneq S_m = Q \tag{7.11}$$

where S_1 equals the union of all sink components, $\mathcal{A}_{|S_1}$ is commutative, and $S_i \setminus S_{i-1}$ for $i \in \{2, \ldots, m\}$ are all the strongly connected components that are not sink components. Consider a word $u \in \Sigma^*$ of shortest possible length with $\delta(S, u) = \delta(S, w)$. Let $S = \{s_1, \ldots, s_l\}$ for $l = |S| \geq 1$ and set $P = \mathrm{Pref}(\{u\})$. Next, we define a function $f \colon P \to \{1, \ldots, m\}^l$ by setting $f(u') = (i_1, \ldots, i_l)$ for $u' \in P$ where $i_j \in \{1, \ldots, m\}$ equals the minimal number with $\delta(s_j, u') \in S_{i_j}$ for all $j \in \{1, \ldots, l\}$. Now, there exists a unique factorization $u = u_0 x_1 u_1 \cdots u_{r-1} x_r u_r$ with $u_0, u_1, \ldots, u_r \in \Sigma^*$ and $x_1, \ldots, x_r \in \Sigma$ for some $r \geq 0$ such that $f(u_0 x_1 \cdots u_{t-1}) \neq f(u_0 x_1 \cdots u_{t-1} x_t)$ for $t \in \{1, \ldots, r\}$ and

$$\begin{aligned} f(\varepsilon) &= f(u_0), \\ f(u_0 x_1) &= f(u_0 x_1 u_1), \\ &\vdots \\ f(u_0 x_1 \cdots u_{r-1} x_r) &= f(u_0 x_1 \cdots u_{r-1} x_r u_r). \end{aligned}$$

Since the S_i are upper sets, for $t \in \{1, \ldots, r\}$ with $f(u_0 x_1 \cdots u_{t-1}) = (i_1, \ldots, i_l)$ and

$$f(u_0 x_1 \cdots u_{t-1} x_t) = (i'_1, \ldots, i'_l)$$

we have $i_j \geq i'_j$ for all $j \in \{1, \ldots, l\}$ where $i_j > i'_j$ for at least one $j \in \{1, \ldots, l\}$. So, $i'_1 + \ldots + i'_l < i_1 + \ldots + i_l$. As these sums are between l and lm, they can decrease

at most $l(m-1)$ times and we find $r \leq l(m-1)$. As $l(m-1) \leq n^2$ we get $r \leq n^2$. Now, for $j \in \{1,\dots,l\}$ and $t \in \{0,1,\dots,r\}$ the states $\delta(s_j, u_0x_1\cdots u_{t-1}x_t)$ and $\delta(s_j, u_0x_1\cdots u_{t-1}x_tu_t)$ are both contained in the strongly connected component $S_{i_j} \setminus S_{i_j-1}$ for $i_j \geq 2$ or both states are contained in S_1.[7] By Theorem 7.103, Item 10 and as $\mathcal{A}_{|S_1}$ is commutative we can reorder the letters in u_t while still ending up at the same state and so we find

$$\delta(s_j, u_0x_1\cdots u_{t-1}x_tu_t) = \delta(s_j, u_0x_1\cdots u_{t-1}x_ta_1^{|u_t|_{a_1}}\cdots a_k^{|u_t|_{a_k}}).$$

Now, as we have chosen u of minimal length, for distinct $u', u'' \in P$ we have $\delta(S,u') \neq \delta(S,u'')$ as otherwise, for, without loss of generality, $|u'| < |u''|$ we have $\delta(S,u'u''') = \delta(S,u''u''')$ for $u''' \in \Sigma^*$ with $u = u''u'''$ where $|u'u'''| < |u|$, contradicting the minimal choice of u. So, each $u' \in P$ corresponds to a distinct nonempty subset of states $\delta(S,u')$ and we conclude $|u| \leq 2^n - 1$, which gives $|u_t|_{a_h} \leq 2^n - 1$ for each $h \in \{1,\dots,k\}$. Then for the word $v = v_0x_1\cdots v_{r-1}x_rv_r$ where $v_i = a_1^{|u_i|_{a_1}}\cdots a_k^{|u_i|_{a_k}}$ for $i \in \{0,1,\dots,r\}$ we have $\delta(S,u) = \delta(S,v)$ and the claim is proved with v. □

7.10.4 Comparison with Original Definition

Rystsov [533] originally defined an automaton to be solvable, if it is synchronizing and admits a sequence $S_1 \subsetneq \cdots \subsetneq S_m = Q$ of upper sets such that S_1 is a singleton set containing a trap state and $\mathcal{A}_{|S_i}/{\sim_{S_{i-1}}}$ for $i \in \{2,\dots,m\}$ are commutative. This is equivalent to being synchronizing and solvable since, with Theorem 7.112, it follows that S_1 is always a singleton for synchronizing and solvable automata. We call a synchronizing and solvable automaton also an R_1-*solvable* automaton in the following. Furthermore, we say that $\mathcal{A} = (Q,\Sigma,\delta)$ is R_2-*solvable* if $(\mathcal{A}\times\mathcal{A})/{\sim_D}$ is R_1-solvable where $D = \{(q,q) \mid q \in Q\}$. This was another notion of solvability considered in [533]. Since synchronizability of $(\mathcal{A}\times\mathcal{A})/{\sim_D}$ is equivalent to synchronizability of $\mathcal{A}$ by Theorem 7.16, an R_2-solvable automaton is synchronizing.[8]

Proposition 7.115 *Let $\mathcal{A} = (Q,\Sigma,\delta)$ and $S \subseteq Q$ be the set of maximal states in $\mathcal{A}$ w.r.t. $\preceq_R$. Then $\mathcal{A}$ is solvable if and only if $\mathcal{A}/{\sim_S}$ is R_1-solvable and $\mathcal{A}_{|S}$ is a commutative permutation automaton.*

Proof. First, suppose $\mathcal{A}$ is solvable. Then, by Theorem 7.107, Item 2, $\mathcal{A}_{|S}$ is a commutative permutation automaton. By Theorem 7.98, $\mathcal{A}/{\sim_S}$ is solvable and, by Theorem 7.10, it is synchronizing; and so R_1-solvable.

Conversely, suppose $\mathcal{A}_{|S}$ is a commutative permutation automaton and $\mathcal{A}/{\sim_S}$ is R_1-solvable, and so also solvable. Thus, by Theorem 7.108, Item 5, since also $\mathcal{A}_{|S}$ is solvable, $\mathcal{A}$ is solvable. □

Proposition 7.116 *An automaton $\mathcal{A} = (Q,\Sigma,\delta)$ is R_2-solvable if and only if it is synchronizing and $(\mathcal{A}\times\mathcal{A})/{\sim_D}$ with $D = \{(q,q) \mid q \in Q\}$ is solvable.*

[7] We set $u_0x_1\cdots u_{-1}x_0 = \varepsilon$.

[8] In the original definition from [533, p. 277] synchronizability of $\mathcal{A}$ was explicitly included.

Proof. If $\mathcal{A}$ is R_2-solvable, then $(\mathcal{A} \times \mathcal{A})/\sim_D$ is R_1-solvable, and so solvable and synchronizing. Synchronizability of $(\mathcal{A} \times \mathcal{A})/\sim_D$ implies synchronizability of $\mathcal{A}$ by Theorem 7.16.

Conversely, Theorem 7.16 implies synchronizability of $(\mathcal{A} \times \mathcal{A})/\sim_D$ and so it is R_1-solvable. Thus $\mathcal{A}$ is R_2-solvable. □

Proposition 7.117 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. If $\mathcal{A}$ is R_1-solvable, then $\mathcal{A}$ is R_2-solvable.*

Proof. Since $\mathcal{A}$ is solvable, Theorem 7.101 gives solvability of $(\mathcal{A} \times \mathcal{A})/\sim_D$. Then, synchronizability of $\mathcal{A}$ yields that it is R_2-solvable. □

By the following corollary, the class of R_1-solvable automata also equals the intersection of the class of R_2-solvable with the class of solvable automata.

Corollary 7.118 *The automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is R_1-solvable if and only if it is R_2-solvable and solvable.*

Proof. An R_1-solvable automaton is solvable and, by Theorem 7.117, R_2-solvable. Conversely, since an R_2-solvable automaton is synchronizing, if it is solvable, it is also R_1-solvable. □

Figure 7.11 depicts an R_2-solvable automaton that is not solvable, and so also not R_1-solvable. By definition, every non-synchronizing solvable automaton is neither R_1- nor R_2-solvable. Figure 7.13 shows the inclusions relations between our notions of solvability that follow from the above considerations.

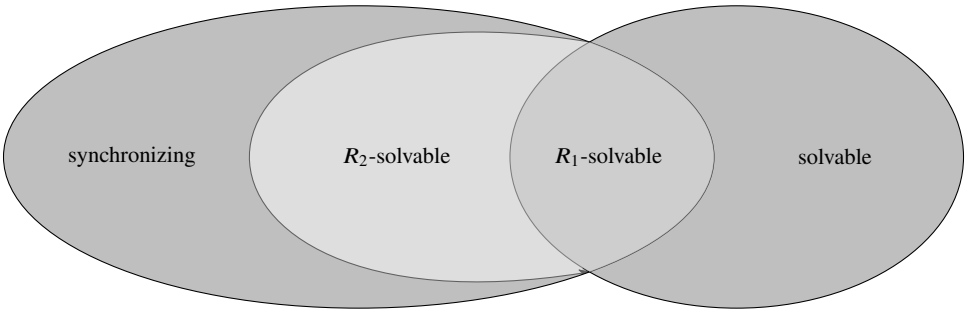


Fig. 7.13: Inclusion relations between the different solvability notions.

Remark 7.119 Yet another, more general notion, is obtained by considering those automata where $(\mathcal{A} \times \mathcal{A})/\sim_D$ is solvable. This class contains the R_2-solvable automata and, by Theorem 7.101, all solvable automata. By adding a trap state to the automaton $\mathcal{A}$ from Figure 7.11, we obtain a non-solvable automaton such that $(\mathcal{A} \times \mathcal{A})/\sim_D$ is solvable but not synchronizing. Thus this class properly contains the R_2-solvable automata.

7.11 Discussion

Here, we discuss the classes, and related literature, introduced in this chapter. Later, in Chapters 8, 12, and 20, we investigate these subclasses w.r.t. to decision problems, automata for sets of synchronizing words, and the constrained synchronization problem, respectively. A discussion of the results from the these chapters can be found in Sections 8.4, 12.7, and 20.7. Here, we leave out a discussion of synchronizable automata, because they are the topic of the whole work and some related literature was already discussed in the introductory part (Chapter 1). The new notion of ρ-automata is introduced here for the first time.

SI-Automata.

Rystsov introduced SI-automata in [535] and proved the bound $2(n-1)^2$ for the reset length of an n-state SI-automata $\mathcal{A}$. In [536] the author showed the bound $n-1$ in case the group of units of $\mathcal{T}_S(\mathcal{A})$ is trivial and the bound $(n-1)^2$ in case the group of units of $\mathcal{T}_S(\mathcal{A})$ is a regular permutation group [536]. We deduce both bounds in Theorems 11.7 and 11.8 in the case of OI-automata.

Permutation-Reset Automata.

Permutation-reset automata were firstly mentioned by Zeiger in the conference contribution [644] (see [645] for the extended version) in connection with the Krohn-Rhodes decomposition theory [389] (see Section 3.12 for some background on this theory). McNaughton [448] and Thierrin [598] investigated permutation automata as language acceptors, see also [195, 489, 490] for information on the class of languages accepted by permutation automata, the so-called *variety of group languages*.

Commutative Automata.

Commutative automata were introduced, to the best of our knowledge, by Laing and Wright [399]. They focused on commutative automata over a binary alphabet, gave normal form theorems for regular expressions modulo commutativity, and also considered infinite-state commutative automata and regularity conditions. Rystsov [533] gave (by an algebraic proof) the tight bound $n-1$ for a shortest synchronizing word of an n-state commutative automaton. Fernau and Hoffmann [212] gave a different (combinatorial) proof and a structure theorem for commutative automata that reach this bound. Fleck [219] determined the automorphism group of strongly connected and commutative automata. Cascade product decomposition results (see the discussion in Section 3.12 on automata theory for this notion) for commutative automata were given in [241, 242]. See also [55, 202, 334, 477] for further algebraic investigations. Question on automata sizes for commutative regular languages were investigated in [312, 311].

The languages accepted by different types of commutative automata were investigated in [13, 293, 302, 310, 312, 489, 490]. Algorithmic problems on commutative automata were investigated in [213], and the membership problem on commutative transformation semigroups in [73].

Partially Ordered Automata.

This type of automata has been considered in the literature under different names: *weakly acyclic* [537], *acyclic* [344], *extensive* [491] or *1-weak/very weak/linear* [173] (linear stems from the existence of a topological sorting of the states). Ryzhikov [537] investigated several synchronization and rank problems on partially ordered automata and obtained the tight bound $n - r$ for a shortest word of rank at most r. Bruchertseifer and Fernau [105] investigated the subclass of *series-parallel automata with loops* with respect to questions of synchronizability in the setting of parameterized complexity (see the part on parameterized complexity in Section 8.4 for more information). Brzozowski and Fich [111] obtained that partially ordered automata accept precisely those languages with $\mathcal{R}$-trivial syntactic monoids. Jirásková and Masopust [344] investigated the state complexity of the reverse operation for partially ordered automata, Dax and Klaedtke [173] investigated the translation of alternating partially ordered automata to nondeterministic automata in the setting of infinite words.

Monotonic Automata.

Ananichev and Volkov [23] introduced monotonic automata and proved Theorem 7.42.[9] However, transformation semigroups whose elements preserve some linear order were investigated before, for example, in [324]. The bound $n - 1$ for shortest synchronizing of monotonic automata was stated in [22] in the language of transformation semigroups. In [539] problems related to (subset) synchronization problems on monotonic partial automata were investigated. Checking whether a given complete automaton is monotonic is NP-complete [592]. Anachichev and Volkov [24] extended the tight bound $n - r$ on shortest words of rank at most r (Theorem 7.42) to so-called *generalized monotonic automata* (these automata where defined in levels that yield an infinite hierarchy even in the case of strongly strongly connected automata [12]) Apart from its rather simple definition, the behavior of monotonic automata as language acceptors remains rather elusive. Shcherbak [565] posed the *open problem* to give a combinatorial description (for example, by regular expressions) of the languages accepted by monotonic automata. In [382, 383] monotonic automata were used to describe certain winning conditions in infinite games. In [19] shortest words that are undefined on each state in

[9] The monotonic automata as introduced earlier by Eppstein [198] are more general, as they are only required to respect a cyclic order and were renamed *oriented* automata by Ananichev and Volkov [23], and *cyclically monotonic* by Martyugin [436]). Linearly and circularly monotonic might be another possible renaming for both concepts.

partially ordered monotonic automata were investigated, which correspond to shortest synchronizing words in the complete automaton resulting from adding a trap state.

Solvable Automata.

Rystsov [533] gave the tight bound $n-1$ for shortest synchronizing words of n-state R_1- and R_2-solvable automata. Here, in Theorem 7.113, for solvable automata, we gave the bound $n-r$ for shortest words of rank at most r. In personal communication, Mikhail V. Volkov noted that the transition monoid of a solvable automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is in the pseudovariety **DS** (in fact, of any automaton whose strongly connected components are blocks). Automata with this property admit a word of rank r and length at most $n-r$ if the minimal rank among its words is r [15, Thm. 2.6]. Theorem 7.113 is slightly more general since it applies to values of r that do not equal the minimal rank among all the ranks of the words. The containment of their transition monoids in **DS** follows from a characterization of **DS** as the set of finite semigroups that are semilattices over semigroups nilpotent over their kernel (see the equivalence (4c) $\Leftrightarrow$ (1b) in [567, Thm. 3] and [216, 567] for definitions of the mentioned notions). For let $P_1, \ldots, P_m$ be a topological sorting of the strongly connected components w.r.t. $\preceq_R$. Then set

$$Y = \{(x_1, \ldots, x_m) \in \{0,1\}^m \mid \exists w \in \Sigma^* \colon x_i = 1 \Leftrightarrow \delta(P_i, w) \subseteq P_i\}$$

and, for $(x_1, \ldots, x_m) \in Y$, set

$$S_{(x_1,\ldots,x_m)} = \{\delta_w \colon Q \to Q \mid \forall i \in \{1, \ldots, m\} \colon x_i = 1 \Leftrightarrow \delta(P_i, w) \subseteq P_i\}.$$

Then Y is a semilattice with the operation

$$(x_1, \ldots, x_m)(y_1, \ldots, y_m) = (\min\{x_1, y_1\}, \ldots, \min\{x_m, y_m\})$$

and $\{S_y\}_{y \in Y}$ forms a semilattice of semigroups where the S_y are groups, and so nilpotent over their kernel.

Chapter 8
Subset Mapping Problems

In this chapter, we investigate various subset mapping and synchronization problems on automata, which we introduce in Section 8.1. Section 8.2 explores reductions and relations between them. In Section 8.3 we investigate the computational complexity of these problems on general automata and the subclasses of solvable, commutative, and partially ordered automata. We give a classification of the (classical) complexity landscape and close some gaps from the literature.

8.1 The Problems

Here, we list the automata-theoretical decision problems of interest to us.

Recall that in Section 3.9.6 we discussed how we encode automata as inputs, and that here, automata have a totally defined transition function.

Sync (Černý [140])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$.
Question: Is $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$?

SyncTrap
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with trap state.
Question: Is $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$?

SyncState
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and state $q \in Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(Q, u) = \{q\}$?

SyncWords
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and finite list of words $u_1, \ldots, u_k \in \Sigma^*$.
Question: $\exists i \in \{1, \ldots, k\} : |\delta(Q, u_i)| = 1$?

SyncShort (Rystsov [528], Eppstein [198])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $l \geq 0$.
Question: Is $\mathrm{Syn}(\mathcal{A}) \cap \Sigma^{\leq l} \neq \emptyset$?

SyncFrom (Rystsov [529])[1]
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $|\delta(S, u)| = 1$?

SyncInto (Rystsov [529])[2]
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(Q, u) \subseteq S$?

SetTrans (Rystsov [529], Luks and McKenzie [425], Blondin et al. [96])[3]
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(S, u) \subseteq T$?

DisjSetTrans
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ with $S \cap T = \emptyset$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(S, u) \subseteq T$?

Intersection (Kozen [387])
Input: List of automata $\mathcal{A}_1, \ldots, \mathcal{A}_k$ as language acceptors.
Question: Do we have $\bigcap_{i=1}^{k} L(\mathcal{A}_i) \neq \emptyset$?

Reach (Rystsov [529], Bondar and Volkov [99])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(Q, u) = S$?

ReachFrom (Rystsov [529])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(S, u) = T$?

8.2 Reductions

Here, in Theorem 8.2, we state various $\leq_m^{\mathsf{AC}^0}$-reductions that work for any fixed alphabet. Omitting the AC^0-equivalent problems, the problems and the reductions from Theorem 8.2 are depicted in the form of a Haase diagram in Figure 8.1. Theorems 8.3, 8.4, and 8.6 give some further reductions for classes of automata. Theorem 8.5 shows that

[1] Called *(global) inclusion problem for weakly-initial automata with a single final state* in [529].
[2] Called *(global) inclusion problem for non-initial automata* in [529].
[3] Called *(global) inclusion problem for weakly initial automata* in [529].

in SyncShort, it does not matter whether we encode the length parameter in unary or binary up to $\leq_m^{\mathsf{AC}^0}$-equivalence.

Remark 8.1 The reductions in Theorems 8.2 to 8.4 involving automata do not change the alphabet. So, they can be used when we investigate problem variants where the alphabet is fixed (as in some statements in Section 8.3). Furthermore, because they do not require that the alphabet is held fixed, they can also be used for complexity upper bounds when the alphabet is part of the input.

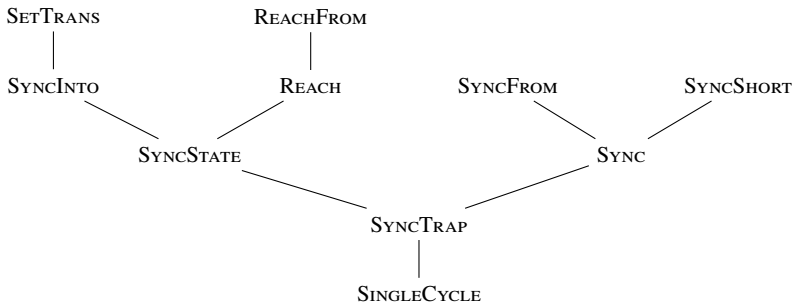


Fig. 8.1: Haase diagram of the reductions from Theorem 8.2; those equivalent to SetTrans omitted.

Proposition 8.2 *Fix a non-empty alphabet Σ. Then the following statements hold:*

1. SingleCycle $\leq_m^{\mathsf{AC}^0}$ SyncTrap,
2. SyncTrap $\leq_m^{\mathsf{AC}^0}$ Sync,
3. SyncTrap $\leq_m^{\mathsf{AC}^0}$ SyncState,
4. Sync $\leq_m^{\mathsf{AC}^0}$ SyncShort,
5. Sync $\leq_m^{\mathsf{AC}^0}$ SyncFrom,
6. SyncState $\leq_m^{\mathsf{AC}^0}$ SyncInto *and* SyncState $\leq_m^{\mathsf{AC}^0}$ Reach,
7. Reach $\leq_m^{\mathsf{AC}^0}$ ReachFrom,
8. SyncInto $\leq_m^{\mathsf{AC}^0}$ SetTrans,
9. Intersection $\equiv_m^{\mathsf{AC}^0}$ SetTrans,
10. DisjSetTrans $\equiv_m^{\mathsf{AC}^0}$ SetTrans.

Furthermore, besides preserving the alphabet, the reductions used in Items 2 to 10 transform a commutative (resp. partially ordered, solvable, permutation) automaton into a commutative (resp. partially ordered, solvable, permutation) automaton.

Proof. The following reductions given in Items 1 to 10 below can be easily implemented by L-uniform circuit families in AC^0. Further, for each reduction involving automata, the alphabet is not altered and it is readily seen that when it is applied to a commutative

(resp. partially ordered) automaton it yields a commutative (resp. partially ordered) automaton.

1: Let $g\colon [n] \to [n]$ be a input permutation to SingleCycle. Set $\mathcal{A} = ([n], \Sigma, \delta)$ where, for $x \in \Sigma$, $\delta(0, x) = 0$ and $\delta(i, x) = g(i)$ for $i \in [n] \setminus \{0\}$. Then $\mathcal{A}$ has 0 as a unique synchronizing trap state if and only if g has a single cycle.

Note that $\mathcal{A}$ has the form of a single tail with a trap state at the end if it is synchronizing. But this is not the case if g has multiple cycles, since then the automaton also has at least one cycle.

2: This is obvious.

3: We assume the n-state input $\mathcal{A}$ to SyncTrap is given as a list of transition matrices $(m_{ij}^{(a)})_{i,j\in\{1,\ldots,n\}}$ for each $a \in \Sigma$, and the input to SyncState is supplied as a list of transition matrices and a vector with a one indicating the state to which we want to synchronize and zero at all other positions. The state i is a trap state if and only if $m_{ii}^{(a)} = 1$ for all $a \in \Sigma$. Let

$$b_i = \left(\bigwedge_{a\in\Sigma} m_{ii}^{(a)}\right) \wedge \left(\bigwedge_{j=1}^{i-1} \bigvee_{a\in\Sigma} \neg m_{jj}^{(a)}\right).$$

Each b_i can be computed by a circuit family in AC^0 by directly implementing the defining formula. We have $b_i = 1$ if and only if i is a trap state and the other states are not trap states. For $b_1 = \cdots = b_n = 0$ we feed a fixed non-synchronizing two-state automaton with some state to SyncState. Otherwise, $(b_1, \ldots, b_n)$ contains the entry "1" precisely once at a position corresponding to a unique trap state. Then, the transition matrices together with $(b_1, \ldots, b_n)$ are an encoding of an input instance for SyncState that is an YES-instance if and only if the original automaton is synchronizing.

4: Let $\mathcal{A}$ be an n-state input automaton to Sync. A shortest synchronizing word has length at most $(n^3 - n)/6$, see Table 1.1 in Section 1.4. So, $\mathcal{A}$ has a synchronizing word if and only if $(\mathcal{A}, n^3)$ is a YES-instance of SyncShort. The number of states n can be readily computed from the input $\mathcal{A}$ and n^3 can be easily computed by circuits constructible by deterministic log-space Turing transducers. Thus the input $(\mathcal{A}, n^3)$ can be computed by a L-uniform family of circuits in AC^0 (where the encoding of n^3 does not matter by Theorem 8.5).

5: This follows as an automaton $\mathcal{A}$ is synchronizing if and only if $(\mathcal{A}, Q)$ is a YES-instance of SyncFrom.

6: An input instance $(\mathcal{A}, q)$ of SyncState can be converted readily to an equivalent input $(\mathcal{A}, \{q\})$ to SyncInto, which is also an equivalent input for Reach.

8: The SyncInto-instance $(\mathcal{A}, S)$ is equivalent to the SetTrans-instance $(\mathcal{A}, Q, S)$, giving a reduction doable by an L-uniform circuit family in AC^0.

9: Let $\mathcal{A}_i = (Q_i, \Sigma, \delta_i)$, $q_i \in Q_i$, and $F_i \subseteq Q_i$ for $i \in \{1, \ldots, k\}$ be an input instance for Intersection. We can assume the state sets Q_i are pairwise disjoint. Set $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q = Q_1 \cup \cdots \cup Q_k$ and $\delta(q, x) = \delta_i(q, x)$ if $q \in Q_i$. Then, for $u \in \Sigma^*$, we have $\delta(\{q_1, \ldots, q_k\}, u) \subseteq F_1 \cup \ldots \cup F_k$ if and only if $\delta_i(q_i, u) \in F_i$ for all $i \in \{1, \ldots, k\}$, giving a reduction preserving the alphabet and the mentioned automata types.

Next, let $(\mathcal{A}, S, T)$ with $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ be a SetTrans-instance. Write $S = \{s_1, \ldots, s_m\}$. Then, for $i \in \{1, \ldots, m\}$, with $\mathcal{A}_i = \mathcal{A}$, $q_i = s_i$, and $F_i = T$ we obtain an instance of Intersection that has a solution if and only if there exists a word in $\mathcal{A}$ mapping S into T.

10: It is sufficient to show SetTrans $\leq_m^{\mathsf{AC}^0}$ DisjSetTrans. Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$. We can assume $S \not\subseteq T$. Set $\mathcal{A} = (Q \cup \{0\} \times S, \Sigma, \overline{\delta})$ where, for $x \in \Sigma$, we let $\overline{\delta}((0, s), x) = \delta(s, x)$ for $s \in S$ and $\overline{\delta}(q, x) = \delta(q, x)$ for $q \in Q$. This construction can be implemented by a L-uniform circuit family in AC^0 as we only add states and transitions according to a fixed scheme. For a non-empty word $u \in \Sigma^+$ we have $\delta(S, u) \subseteq T$ if and only if $\overline{\delta}(\{0\} \times S, u) \subseteq T$ and as $\{0\} \times S$ and T are disjoint this gives a valid reduction. □

Proposition 8.3 *For automata whose synchronizing states are trap states as, for example, solvable automata, we have* Sync $\equiv_m^{\mathsf{AC}^0}$ SyncState *and* SyncState $\equiv_m^{\mathsf{AC}^0}$ SyncTrap.

Proof. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an input automaton and $n = |Q|$. We assume $\mathcal{A}$ is supplied as a list of transition matrices $(m_{ij}^{(a)})_{i,j \in \{1,\ldots,n\}}$, $a \in \Sigma$, w.r.t. to an identification of the states with $\{1, \ldots, n\}$. By Theorem 8.2, Item 3, SyncTrap $\leq_m^{\mathsf{AC}^0}$ SyncState. Thus, it is sufficient to show SyncState $\leq_m^{\mathsf{AC}^0}$ Sync $\leq_m^{\mathsf{AC}^0}$ SyncTrap.

SyncState $\leq_m^{\mathsf{AC}^0}$ Sync: Since a trap state must be a unique synchronizing state, with the assumption we obtain that a state q in $\mathcal{A}$ is synchronizing if and only if $\mathcal{A}$ is synchronizing and q a trap state. So, we first check whether q is a trap state by implementing the formula $\bigwedge_{a \in \Sigma} m_{qq}^{(a)}$ and, if q is a trap state, pass $\mathcal{A}$ to Sync. Otherwise, we pass some fixed NO-instance to Sync.

Sync $\leq_m^{\mathsf{AC}^0}$ SyncTrap: Since by assumption a synchronizing state is a trap state, $\mathcal{A}$ is synchronizing if and only if it has a trap state and is synchronizing. Existence of a trap state can be checked by the formula $\bigvee_{i=1}^{n} \bigwedge_{a \in \Sigma} m_{ii}^{(a)}$. If this check succeeds, we pass $\mathcal{A}$ to SyncTrap. Otherwise, we pass some fixed NO-instance to SyncTrap. □

Proposition 8.4 *For commutative input automata we have* Sync $\leq_m^{\mathsf{AC}^0}$ SyncWords.

Proof. Let $\Sigma = \{a_1, \ldots, a_k\}$ and $\mathcal{A} = (Q, \Sigma, \delta)$ be commutative. By Theorem 7.89, $\mathcal{A}$ is synchronizing if and only if $(\mathcal{A}, a_1^{n-1} \cdots a_k^{n-1})$ is a YES-instance of SyncWords. This yields an $\leq_m^{\mathsf{AC}^0}$-reduction. □

The length of a shortest synchronizing word is polynomially bounded in the number of states of the automaton, see Table 1.1 in Section 1.4. Hence, for an input instance $(\mathcal{A}, l)$ to SyncShort we can assume that l is polynomially bounded in the number of states of $\mathcal{A}$. As a consequence, as we show next, it does not matter, up to $\equiv_m^{\mathsf{AC}^0}$-equivalence, whether the length parameter is encoded in unary or binary.

Proposition 8.5 SyncShort *with length parameter encoded in thermometer or one-hot unary encoding is* $\equiv_m^{\mathsf{AC}^0}$*-equivalent to* SyncShort *with length parameter encoded in binary.*

Proof. Let $(\mathcal{A}, l)$ be an input to SyncShort with an n-state automaton $\mathcal{A}$. Suppose l is encoded in unary together with an encoding of $\mathcal{A}$. Whether it is encoded as 1^l, in one-hot, or thermometer representation with some delimiting bits to the rest of the input, similarly as done in in the proof of Theorem 3.81 we can convert it to a binary encoding of l by L-uniform AC^0-circuit families.

Next, suppose l is given in binary encoding. If $\mathcal{A}$ is synchronizing, then a shortest synchronizing word has length at most $\frac{n^3-n}{6}$, see Table 1.1 in Section 1.4. Hence, it is sufficient to consider only $O(\log(n))$ least-significant bits of the binary encoding of l. Then, by Theorem 3.81, Item 3, there exists an AC^0 circuit of size $O(n)$ computing the one-hot unary encoding of the number encoded by these bits. Inspection of the proof of Theorem 3.81, Item 3 yields that these circuits are constructible in deterministic log-space, so that we obtain a L-uniform circuit family in AC^0 converting the binary encoding to the one-hot unary encoding of l. □

By Theorem 8.5, we can use the most convenient encoding of the length parameter. In the following, we will often do this without special mentioning.

Lastly, we state an AC^0-equivalence that is valid for a fixed unary alphabet.

Proposition 8.6 *For unary input automata we have* SyncShort $\equiv_m^{\mathsf{AC}^0}$ SyncWords *which is true for the unary and the binary encoding of the length parameter in* SyncShort.

Proof. Let $\mathcal{A} = (Q, \{a\}, \delta)$ be a unary input automaton and $n = |Q|$. By Theorem 8.5, we can assume the length parameter is supplied in unary encoding.

SyncWords $\leq_m^{\mathsf{AC}^0}$ SyncShort: By Theorem 7.1, the unary automaton $\mathcal{A}$ has a synchronizing word of length at most L where $L = \max\{l_1, \ldots, l_m\}$ for $l_1, \ldots, l_m \geq 0$ if and only if it has a synchronizing word among the words $a^{l_1}, \ldots, a^{l_m}$. Thus, an input $(\mathcal{A}, a^{l_1}, \ldots, a^{l_m})$ to SyncWords is a YES-instance if and only if $(\mathcal{A}, L)$ is a YES-instance of the problem SyncShort for $L = \max\{l_1, \ldots, l_m\}$, giving an $\leq_m^{\mathsf{AC}^0}$-reduction.

SyncShort $\leq_m^{\mathsf{AC}^0}$ SyncWords: An input $(\mathcal{A}, l)$ for $l \geq 0$ is a YES-instance of SyncShort if and only if $(\mathcal{A}, a^l)$ is a YES-instance of SyncWords. If l is given in unary, we can easily transform l into the word a^l by an AC^0 circuit, which gives a reduction. □

8.3 Classes of Automata

Here, we collect results on the automata-theoretical problems introduced in Section 8.1 w.r.t. classes of automata. First, in Section 8.3.1, we recap known results from the literature and give an overview of those results from the present section that are new. Then, in Section 8.3.2, 8.3.5, and 8.3.4, we investigate general, partially ordered, and commutative automata, respectively.

We distinguish the case when Σ is part of the input and can be arbitrarily large (the case of an unbounded alphabet) from the case when the size of Σ is bounded, where in the latter case we can equivalently assume Σ is fixed and not part of the input.

8.3.1 Known Results and Overview

The following problems were proved to be PSPACE-complete for $\leq_m^{\log}$-reductions in the literature:

1. Intersection (Kozen [387]),
2. SetTrans (Rystsov [529]),
3. SyncFrom (Rystsov [529]),
4. SyncInto (Rystsov [529], Berlinkov et al. [84]),[4]
5. Reach (Rystsov [529], Bondar and Volkov [99]),
6. ReachFrom (Rystsov [529]).

Recall that, by Theorem 8.2, Intersection and SetTrans are equivalent w.r.t. our notion of AC^0-reduction.

Besides these results, the following results on classes of input automata investigated in the present thesis were obtained in the literature:

Intersection: Kozen [387, p. 261] introduced this problem and proved that it is PSPACE-complete for an unbounded alphabet; Fernau and Krebs [214, p. 2] mentioned PSPACE-completeness for a fixed binary alphabet. NP-completeness was proved for the following classes:

1. partially ordered automata even for a binary alphabet by Ryzhikov [537, Prop. 9],[5]
2. ternary partially ordered monotonic automata by Ryzhikov and Shemyakov [539, Thm. 4.1],
3. unary automata by Martens and Neven [434, Lem. 27] (using arguments that are implicitly used in [234, Thm. 2] and [584, Thm. 6.1]), Beaudry [74], and Fernau et al. [213, Thm. 4.1], where in the latter work also containment in NP was shown for commutative automata [213, Thm 4.3],
4. partially ordered commutative automata over an unbounded alphabet by Arrighi et al. [42, Lem. 17] (by a similar reduction as used by Martyugin [436, Prop. 5.3.] for SyncShort).[6]

[4] Berlinkov et al. [84] investigated the problem of totally extending a given subset and showed this problem to be PSPACE-complete. In fact, this problem is equivalent to SyncInto, see the discussion section (Section 8.4).

[5] He stated the result only for a fixed binary alphabets, but the given proof for containment in NP also works when the alphabet is part of the input.

[6] NP-hardness can be seen by the following reduction from SAT. A SAT-instance with clauses $C_1, \ldots, C_m$ over the variables $\{x_1, \ldots, x_n\}$ admits a satisfying assignment if and only if the following intersection of languages acceptable by partially ordered commutative automata is non-empty:

SyncFrom: Vorel [620, Thm. 12] proved PSPACE-completeness for binary strongly connected automata. Ryzhikov [537, Thm. 4] proved the problem to be NP-complete for binary partially ordered automata, where the proof for NP-membership also works for an unbounded alphabet.

SyncInto: Berlinkov et al. [84, Thm. 3 & 4] proved PSPACE-completeness for binary strongly connected automata.[7]

SyncShort: Eppstein [198, Thm. 8] and Rystsov [528] proved NP-completeness on binary partially ordered automata, Martyugin [436, Prop. 5.3] for partially ordered commutative automata over an unbounded alphabet by a reduction from SAT. Fernau, Heggernes and Villanger [211, Thm. 3] gave a reduction from HittingSet to SyncState that results in a partially ordered commutative automaton to show W[2]-hardness w.r.t. the length-bound parameter, but the reduction also yields NP-hardness for this class. Salomaa [546] and Goralčýk and Koubek [257] investigated more general problems and obtained NP-completeness for SyncShort as a special case. For related problems and more information, see the discussion in Section 8.4.

Sync: Holzer & Jacobi [320, Lem. 14] proved NL-completeness even for a ternary alphabet and trap-state automata. Sreejith (see Volkov [617, Prop. 2.2])[8] and Hoffmann [299, Thm. 5] gave proofs yielding NL-hardness even for a binary alphabet and trap-state automata (Theorems 8.10 and 8.13 present a simplified version of the proof from [299, Thm. 5], which is similar to the proof from [617, Prop. 2.2]); Fernau, Haase and Hoffmann [210, Thm. 5.4] stated (without proof) that Sync is L-complete for commutative automata (we will prove in Theorem 8.20 that it remains L-complete for solvable automata); L-membership was also proved in [299, Thm. 5] for unary automata.

Černý [140, Thm. 2] remarked that the pair automaton, via Theorem 7.18, can be used to decide Sync. Natarajan [457] and Eppstein [198] explicitly gave polynomial time algorithms running in cubic time, see Theorem 7.26.

We prove further results for solvable automata and some of their subclasses in Sections 8.3.2, 8.3.4, and 8.3.5. These results were first presented by the author in [299, 210, 313]. Recall that, by Items 9 and 10 from Theorem 8.2, the problems Intersection, SetTrans, and DisjSetTrans are equivalent under $\leq_m^{\mathsf{AC}^0}$-reductions. Moreover, these reductions preserve commutativity, the property of being partially ordered, and the alphabet.

Table 8.1 summarizes our results and those from the literature. We say that the alphabet is unbounded (or arbitrary) if it is part of the input. Note that every unary automaton is commutative. Moreover, hardness for a unary alphabet implies hardness

$$\left(\bigcap_{j=1}^{m} \bigcup_{y \in C_j} \Sigma^* y \Sigma^*\right) \cap \left(\bigcap_{i=1}^{n} \Sigma^* \setminus (\Sigma^* x_i \Sigma^* \overline{x}_i \Sigma^* \cup \Sigma^* x_i \Sigma^* \overline{x}_i \Sigma^*)\right)$$

[7] They formulated the problem differently in terms of totally extending a subset, see the discussion section, Section 8.4.

[8] As written by Volkov [617] the proof presented by him is originally due to the Indian computer scientist Sreejith, but was not published before and presented courtesy of the author.

for any non-empty alphabet, since additional letters can either be added as self-loops or made to act identically to the single letter.

Table 8.1: Complexity landscape for general, partially ordered (PO), commutative, and partially ordered commutative automata (POC). It is indicated whether the result holds for an unbounded (U), any fixed (F), including a unary, or any fixed alphabet containing at least two letters ($F^{\geq 2}$). Gray cells indicate known results, see Section 8.3.1.

	Automata Class						
	General	Solvable	PO	Commutative		POC	
Alphabet Σ	$F^{\geq 2}$ & U	$F^{\geq 2}$ & U	$F^{\geq 2}$ & U	F	U	F	U
Sync	NL-c [A]	L-c [B]	AC^0 [C]	L-c [B]	L-c [B]	AC^0 [C]	AC^0 [C]
SyncState	NL-c [A]	L-c [B]	AC^0 [C]	L-c [B]	L-c [B]	AC^0 [C]	AC^0 [C]
SyncTrap	NL-c [A]	L-c [B]	AC^0 [C]	L-c [B]	L-c [B]	AC^0 [C]	AC^0 [C]
SyncWords	L-c [D]	L-c [D]	L-c [D]	L-c [D]	L-c [D]	L-c [D]	L-c [D]
SyncShort	NP-c [E]	NP-c [E]	NP-c	L-c [E]	NP-c [E]	L-c [E]	NP-c [E]
SyncFrom	PSPACE-c	NP-c [F]	NP-c [F]	L-c [G]	L-c [G]	L-c [G]	L-c [G]
SyncInto	PSPACE-c	L-c [B]	AC^0 [H]	L-c [B]	L-c [B]	AC^0 [H]	AC^0 [H]
SetTrans	PSPACE-c	NP-c [I]	NP-c [I]	NP-c [I]	NP-c [I]	L-c [G]	NP-c [I]
Reach	PSPACE-c	NP-c [J]	NP-c [J]	L-c [K]	NP-c [K]	L-c [K]	L-c [K]
ReachFrom	PSPACE-c	NP-c [L]	NP-c [L]	P [M]	NP-c [L]	L-c [N]	NP-c [L]

References. [A] Thm. 8.13; [B] Thm. 8.20; [C] Thm. 8.29; [D] Thm. 8.9; [E] Thm. 8.24; [F] Thm. 8.19; [G] Thm. 8.23; [H] Thm. 8.28; [I] Thm. 8.18; [J] Thm. 8.21; [K] Thm. 8.25; [L] Thm. 8.22; [M] Thm. 8.27; [N] Thm. 8.26.

8.3.2 General Automata

Here, we state and prove the following.

1. SyncWords is L-complete even for partially ordered unary input automata and a single input word (Theorems 8.7 and 8.9).
2. SyncTrap is NL-hard even for a fixed binary alphabet (Theorem 8.10). The proof is a simplified version of a construction given by Hoffmann [299, Thm. 5], which is similar to a construction from Holzer & Jacobi [320, Lem. 14] but uses a binary instead of a ternary alphabet. The proof from Volkov [617, Prop. 2.2, due to Sreejith] uses only a binary alphabet as well, and is also similar to the proof given here. In Theorems 8.11 and 8.12 we also give the related results that checking whether a state is a maximum or maximal for $\preceq_R$ in an automaton is NL-complete.
3. Sync, SyncTrap, and SyncState are all NL-complete, even for a fixed binary alphabet that is not part of the input (Theorem 8.13).
4. PSPACE-completeness of SyncFrom, SyncInto, Intersection, DisjSetTrans, SetTrans, Reach, and ReachFrom (Theorems 8.14 and 8.16).

Proposition 8.7 SyncWords *is L-hard even for a single word as parameter and unary partially ordered automata.*

Proof. We give a reduction from OrderBetweenVertices, which is L-complete by Theorem 3.76, Item 3. Let (G, v_i, v_j) be an input instance with $G = (V, E)$ and $v_i, v_j \in V$ where $V = \{v_1, \ldots, v_n\}$ and $E = \{(v_i, v_{i+1}) \mid i \in \{1, \ldots, n-1\}\}$. We can assume $i \neq j$ and $i, j \notin \{1, n\}$. Note that v_1 (resp. v_n) is easily computable as the unique vertex without any ingoing (resp. outgoing) vertices.

Construction: Set $\mathcal{A} = (V \cup \{1, \ldots, n\} \times \{v_1, v_i, v_j\}, \{a\}, \delta)$ where, for $v_r \in V$, $v_s \in \{v_1, v_i, v_j\}$, $m \in \{1, \ldots, n\}$,

$$\delta(v_r, a) = \begin{cases} v_{r+1} & \text{if } r \notin \{i, j, n\}, \\ (1, v_r) & \text{if } r \in \{i, j\}, \\ v_r & \text{if } r = n, \end{cases}$$

$$\delta((m, v_s), a) = \begin{cases} (m+1, v_s) & \text{if } m < n, \\ v_1 & \text{if } m = n \text{ and } s = 1, \\ v_i & \text{if } m = n \text{ and } s = j, \\ v_n & \text{if } m = n \text{ and } s = i. \end{cases}$$

See Figure 8.2 for a graphical depiction of $\mathcal{A}$ for both cases $i < j$ and $i > j$. Now, $\mathcal{A}$ is a unary partially ordered automaton with trap state v_n.

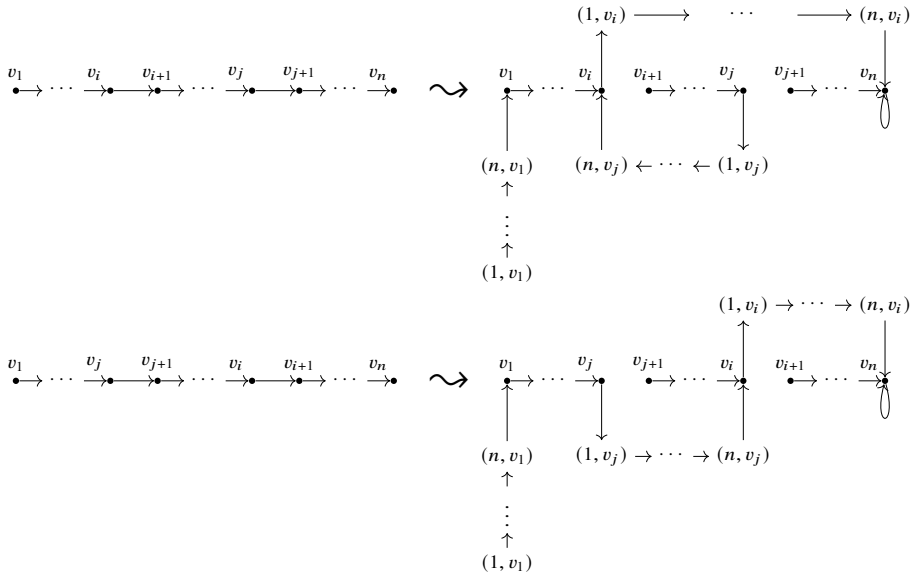


Fig. 8.2: Reduction of OrderBetweenVertices to SyncWords from the proof of Theorem 8.7 for the two cases $i < j$ (upper part), i.e., v_i appears before v_j, and $i > j$ (lower part), i.e., v_j appears before v_i, where the right hand side depicts the resulting unary automata over a single letter (which is omitted at the transitions).

Verification: We show that a^{3n-1} synchronizes $\mathcal{A}$ if and only if $i < j$. The state v_n is a trap state and so must be the synchronizing state. First, assume $j < i$. Then a shortest path in the automaton graph of $\mathcal{A}$ to v_n traverses the following vertices

$$(1, v_1), \ldots, (n, v_1), v_1, \ldots, v_j, (1, v_j), \ldots, (n, v_j), v_i, (1, v_i), \ldots, (n, v_i), v_n,$$

which has length $3n + j + 1 > 3n$. So a^{3n-1} does not synchronize $\mathcal{A}$. Next, assume $i < j$. Then the following three sequences of vertices trace out three paths ending at v_n and contain all states:

$$\begin{aligned}
&(1, v_1), \ldots, (n, v_1), v_1, \ldots, v_i, (1, v_i), \ldots, (n, v_i), v_n,\\
&v_{i+1}, \ldots, v_j, (1, v_j), \ldots, (n, v_j), v_i, (1, v_i), \ldots, (n, v_i), v_n,\\
&v_{j+1}, \ldots, v_n.
\end{aligned}$$

As each path has length strictly less than $3n$ the word a^{3n-1} synchronizes $\mathcal{A}$. □

Remark 8.8 We easily obtain L-hardness of SyncWords for unary automata since, by Theorems 8.2 and 8.4, we have SingleCycle $\leq_m^{\mathsf{AC}^0}$ SyncTrap $\leq_m^{\mathsf{AC}^0}$ Sync $\leq_m^{\mathsf{AC}^0}$ SyncWords, and SingleCycle is L-hard by Theorem 3.76. However, the reduction used to establish SingleCycle $\leq_m^{\mathsf{AC}^0}$ SyncTrap might result in an automaton that is not partially ordered.

It was proved that deciding whether a single word synchronizes a given automaton is in L in [299, Thm. 4]. By iterating through the given words, this also yields that SyncWords is in L. We give an independent proof of containment in L next for self-containment.

Theorem 8.9 SyncWords *is L-complete for general automata, where L-hardness holds even for a single word as parameter and commutative partially ordered automata as input over any fixed non-empty, even a unary, alphabet.*

Proof. Proposition 8.7 gives L-hardness even for a unary alphabet. Next, we describe a deterministic log-space bounded Turing machine that solves the problem for the inputs $\mathcal{A} = (Q, \Sigma, \delta)$ and $u_1, \ldots, u_r \in \Sigma^*$. Set $Q = \{q_1, \ldots, q_n\}$. The machine iterates through the words $u_1, \ldots, u_r$ and through the states, and checks whether $\delta(q, u_i)$ equals the same state for all $q \in Q$ and some $i \in \{1, \ldots, r\}$. This can be done using three counters and two memory cells in the following way. First, we iterate through $i \in \{1, \ldots, r\}$ and $j \in \{1, \ldots, n\}$ using two counters. While doing this, we compute $\delta(q_j, u_i)$ by using

1. a third counter to iterate letter-by-letter through the prefixes $u_{i,1} \cdots u_{i,l}$ for $l \in \{1, \ldots, m_i\}$ of $u_i = u_{i,1} \cdots u_{i,m_i}$ where $u_{i,1}, \ldots, u_{i,m_i} \in \Sigma$ and
2. one memory cell to store $\delta(q_j, u_{i,1}, \ldots, u_{i,l-1})$ for $l \geq 1$ that is initialized with q_j and updated for each letter $u_{i,l}$ to $\delta(q_j, u_{i,1}, \ldots, u_{i,l-1}u_{i,l})$.

Then the memory cell stores $\delta(q_j, u_i)$ when $l = m_i$. Furthermore, another memory cell stores $\delta(q_{j-1}, u_i)$ for $j \geq 2$ to check equality of $\delta(q_{j-1}, u_i)$ and $\delta(q_j, u_i)$ for each $j \in \{2, \ldots, n\}$. The word u_i synchronizes $\mathcal{A}$ if and only if all of the equality tests for $\delta(q_{j-1}, u_i)$ and $\delta(q_j, u_i)$ pass. The machine operates in logarithmic space by using a binary encoding for the counters. □

Proposition 8.10 *Let $\Sigma = \{a, b\}$ be a fixed binary alphabet. Then* SyncTrap *is* NL*-hard.*

Proof. We give a reduction from 2-DAG-ST.

Construction: Let (G, s, t) with $G = (V, E)$ be an instance of 2-DAG-ST. For each $q \in V \setminus \{s, t\}$, denote by $q^R, q^L \in V$ vertices such that, if $(\{q\} \times V) \cap E$ is non-empty, we have $(\{q\} \times V) \cap E = \{(q, q^R), (q, q^L)\}$, and $q^R = q^L = s$ for $(\{q\} \times V) \cap E = \emptyset$. Next, set $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q = (V \setminus \{s, t\}) \times \{\varepsilon, b\} \cup \{(s, \varepsilon), (t, \varepsilon)\}$ where, for $(q, u) \in Q \setminus \{(s, \varepsilon), (t, \varepsilon)\}$ and $x \in \Sigma$,

$$\delta((q, u), x) = \begin{cases} (q^R, \varepsilon) & \text{if } ux = a, \\ (q^L, \varepsilon) & \text{if } ux = ba, \\ (q, b) & \text{if } ux = b, \\ (s, \varepsilon) & \text{otherwise,} \end{cases}$$

and $\delta((t, \varepsilon), x) = (t, \varepsilon)$ for all $x \in \Sigma$, $\delta((s, \varepsilon), a) = (q, \varepsilon)$ for the unique $q \in V$ with $(s, q) \in E$ and $\delta((s, \varepsilon), b) = (s, \varepsilon)$. See Figure 8.3 for an example.

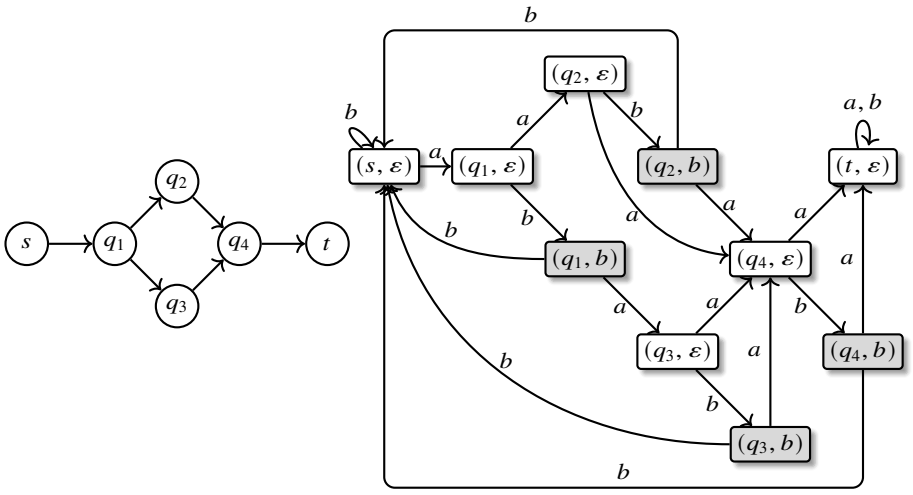


Fig. 8.3: Example for the reduction used in Theorem 8.10. Left: Input graph. Right: Transformation into an automaton having a synchronizing word if and only if there exists a path from s to t in the input graph (states from $V \times \{b\}$ are colored grey).

Properties: The following two properties are readily seen.

1. $\delta(Q \setminus \{(t, \varepsilon)\}, bb) = \{(s, \varepsilon)\}$ and $\delta((t, \varepsilon), bb) = (t, \varepsilon)$.
2. For $q \in V \setminus \{s, t\}$ we have $\delta((q, \varepsilon), a) = (q^R, \varepsilon)$ and $\delta((q, \varepsilon), ba) = (q^L, \varepsilon)$.

Verification: First, suppose there exists a path $q_0, q_1, \ldots, q_n$ with $q_0 = s$ and $q_n = t$ in G. Define a word $u = u_1 u_2 \cdots u_n$ where $u_1 = a$ and, for $i \in \{2, \ldots, n\}$,

$$u_i = \begin{cases} a & \text{if } q_i = q_{i-1}^R, \\ ba & \text{if } q_{i-1}^R \neq q_{i-1}^L \text{ and } q_i = q_{i-1}^L. \end{cases}$$

By Properties 1 and 2, $\delta(Q, bbu_1 \cdots u_i) = \{(q_i, \varepsilon), (t, \varepsilon)\}$. So $\delta(Q, bbu) = \{(t, \varepsilon)\}$.

Conversely, suppose the word $u \in \Sigma^*$ synchronizes $\mathcal{A}$. As (t, ε) is a trap state, we have $\delta(Q, u) = \{(t, \varepsilon)\}$. Let w be a shortest factor of u such that $\delta((s, \varepsilon), w) = (t, \varepsilon)$. By Property 1 we have $w \notin \Sigma^* bb\Sigma^*$, i.e., $w \in \{a, ba\}^* \cup \{a, ba\}^* b$. As we can enter the state (t, ε) from another state only via the letter a, we have $w \in \{a, ba\}^*$. Let $w = w_1 \cdots w_n$ with $w_1, \ldots, w_n \in \{a, ba\}^*$. Then, by Property 2, the vertices

$$(s, \varepsilon), \delta(s, w_1), \delta(s, w_1 w_2) \ldots \delta(s, w_1 w_2 \cdots w_{n-1}), (t, \varepsilon)$$

trace out a path in G from s to t. □

Proposition 8.11 *Given an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $q \in Q$ it is* NL*-complete to test whether q is a maximum w.r.t. the reachability quasiorder $\preceq_R$.*

Proof. First, we give an $\leq_m^{\mathsf{AC}^0}$-reduction from SyncTrap, which yields NL-hardness by Proposition 8.10. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an input instance to SyncTrap. Suppose $t \in Q$ is the trap state, which can be computed by an AC^0-circuit as in the proof of Theorem 8.2, Item 3. So, the reduction produces $\mathcal{A}$ together with the trap state t and $(\mathcal{A}, t)$ is a Yes-instance of our problem if and only if $\mathcal{A}$ is synchronizing, since the state t is a maximum in $\mathcal{A}$ if and only if $\mathcal{A}$ is synchronizing by Theorem 7.6.

For containment in NL, suppose $\mathcal{A} = (Q, \Sigma, \delta)$ and $q \in Q$ are given. To test whether q is a maximum for $\preceq_R$, we guess, for each state $p \in Q$, a word $u \in \Sigma^{<|Q|}$ letter by letter and check whether $\delta(p, u) = q$; a property that characterizes that q is a maximum by Theorem 7.12. This can be done in nondeterministic log-space by storing a state and using two counters, one to iterate through all states $p \in Q$ and the other to make at most $|Q|$ guessing steps for each $p \in Q$. □

Proposition 8.12 *Given an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $q \in Q$ it is* NL*-complete to test whether q is maximal w.r.t. the reachability quasiorder $\preceq_R$.*

Proof. We give a reduction from the NL-complete problem 2-DAG-ST. Let $G = (V, E)$ and $s, t \in V$ be an input instance. Construct an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ as in the proof of Theorem 8.10, but instead of making (t, ε) a trap state, let both letters a and b map back to the state (s, ε). Then t is a maximal state (and $\mathcal{A}$ is strongly connected) if and only if there exists a path from s to t in G.

To test maximality, we have to check that for each state $p \in Q$ with $\delta(q, u) = p$ for some $u \in \Sigma^*$ there exists $v \in \Sigma^*$ with $\delta(p, v) = q$. To verify this condition, we can pick words u, v of length at most $|Q|$ and proceed similarly as in the proof of Theorem 8.11 to obtain NL-membership. □

Similar procedures as in Theorems 8.11 and 8.12 can be used to check whether a state is minimal or a minimum, but we do not need to check these properties in the following and hence give no formal statement. We also note that we obtain L-completeness of these problems in case the input automata are unary automata since we only needed nondeterminism when guessing different letters in the above proofs.

Theorem 8.13 *The problems* Sync, SyncTrap *and* SyncState *are* NL*-complete, even for a fixed binary alphabet.*

Proof. By Proposition 8.2 we have SyncTrap $\leq_m^{\mathsf{AC}^0}$ SyncState and SyncTrap $\leq_m^{\mathsf{AC}^0}$ Sync. By Theorem 8.10, SyncTrap is NL-hard for an alphabet containing at least two letters. Sync is in NL, see Section 8.3.1 (page 232). For $\mathcal{A} = (Q, \Sigma, \delta)$, by Theorem 7.11, a state $q \in Q$ is a synchronizing state if and only if $\mathcal{A}$ is synchronizing and q is a maximum for the reachability order $\preceq_R$. The latter two properties can be checked in NL since Sync is in NL and by Theorem 8.11, respectively. So, SyncState is in NL. □

Proposition 8.14 SetTrans *is in* PSPACE.

Proof. Let $(\mathcal{A}, S, T)$ with $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ where $S = \{s_1, \ldots, s_m\}$ and $n = |Q|$. Store the values $s_1, \ldots, s_m$ in m memory cells and then repeat the following steps at most n^m times: If the memory cells store a subset of T, then abort and return YES, i.e., classify this input instance as a YES-instance of the decision problem. Otherwise guess a letter $a \in \Sigma$ and update the memory cells with $\delta(s_1, a), \ldots, \delta(s_m, a)$. If this procedure finishes without ever writing a subset of T into the memory cells, then return NO. The upper bound for the number of steps can be ensured by utilizing an $(m \cdot \log n)$-bit counter counting the number of steps in binary encoding. After at most n^m steps we have seen at least one memory cell configuration twice. So, if there exists a word mapping S into T, then there exists such a word of length at most n^m, which implies correctness of the algorithm. As it runs in nondeterministic polynomial space, and as PSPACE = NPSPACE by Savitch's theorem [550], the claim follows. □

The following lemma is a technical result that we need to reduce an arbitrary alphabet to a binary alphabet in our subset mapping problems. Another reduction to reduce the alphabet for synchronization problems has been given in [439, Sec. 5] and [438, Prop. 2].

Lemma 8.15 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *with* $\Sigma = \{a_1, \ldots, a_k\}$. *Set* $C = \{0^k 10^i 1 \mid i \in \{0, 1, \ldots, k-1\}\}$ *and* $\mathcal{B} = (Q \times \mathrm{Pref}(C) \setminus C, \{a, b\}, \mu)$ *where, for* $q \in Q$, $u \in \mathrm{Pref}(C) \setminus C$, *and* $x \in \{a, b\}$, *we have*

$$\mu((q,u),x) = \begin{cases} (\delta(q,a_i),\varepsilon) & \text{if } ux = 0^k 10^{i-1}1 \text{ for some } i \in \{1,\ldots,k\}, \\ (q,ux) & \text{if } ux \in \mathrm{Pref}(C) \setminus C, \\ (q,ux) & \text{if } u \in \mathrm{Pref}(0^{k-1}) \text{ and } x = 0, \\ (q,0^k) & \text{if } u = 0^k \text{ and } x = 0, \\ (q,0^k) & \text{if } u = 0^k 10^{k-1} \text{ and } x = 0, \\ (q,\varepsilon) & \text{otherwise.} \end{cases}$$

Then $\mathcal{A}$ *is strongly connected if and only if* $\mathcal{B}$ *is strongly connected and, for* $i \in \{1, \ldots, k\}$, $q \in Q$, $u \in \mathrm{Pref}(C) \setminus C$, *we have*

$$\mu((q,u), 0^k 10^{i-1}1) = (\delta(q,a_i),\varepsilon), \tag{8.1}$$

and, for distinct $S, T \subseteq Q$, *we have* $\delta(S,u) = T$ *for some* $u \in \Sigma^*$ *if and only if* $\mu(S \times \mathrm{Pref}(C) \setminus C, v) = T \times \{\varepsilon\}$ *for some* $u \in \{a,b\}^*$.

Proof. That $\mathcal{A}$ is strongly connected if and only if $\mathcal{B}$ is strongly connected is easy to see. Equation (8.1) is also easy to verify.

Next, suppose $\delta(S, u) = T$ for some $u \in \Sigma^*$. Since $S \neq T$ we have $|u| > 0$. Let $v_j = 0^k 10^{i-1}1$ if the j-th symbol in u equals a_i for $i \in \{1, \ldots, k\}$. By Equation (8.1) we have $\mu(S \times \mathrm{Pref}(C) \setminus C, v_1 \cdots v_{|u|}) = (T, \varepsilon)$.

Conversely, suppose $\mu(S \times \mathrm{Pref}(C) \setminus C, v) = T \times \{\varepsilon\}$ for some $u \in \{a, b\}^*$. Let $q \in S$ and $u \in \mathrm{Pref}(C) \setminus C$. The language C has the property that there exist unique words $u_1, \ldots, u_m \in C$ and $v_0, \ldots, v_m \in \Sigma^*$, each v_j containing no factor from C, such that

$$uv = v_0 u_1 v_1 \cdots u_m v_m.$$

For each $j \in \{1, \ldots, m\}$, set $x_j = a_i$ if $u_j = 0^k 10^{i-1}1$ for some $i \in \{1, \ldots, k\}$. By induction, using Equation (8.1), we obtain

$$\mu((q, u), v) = (\delta(q, x_1 \cdots x_m), \varepsilon).$$

Hence $\delta(S, x_1 \cdots x_m) = T$. □

Theorem 8.16 *The following problems are* PSPACE*-complete w.r.t.* $\leq_m^{\log}$*-reductions:*

1. SyncFrom *and* SyncInto *even on binary strongly connected automata,*
2. Intersection, SetTrans, *and* DisjSetTrans *even on binary strongly connected automata,*
3. Reach *and* ReachFrom *even for binary automata.*

Proof. Membership of these problems in PSPACE was proved in the literature, see Section 8.3.1 (page 232). As an example, we included a proof for SetTrans in Theorem 8.14.

PSPACE-hardness of SyncFrom and SyncInto for binary strongly connected automata was also mentioned in Section 8.3.1 (page 232).

By Theorem 8.2 we have

$$\textsc{SyncInto} \leq_m^{\mathsf{AC}^0} \textsc{SetTrans} \leq_m^{\mathsf{AC}^0} \textsc{Intersection}$$

and the reductions from Theorem 8.2 preserve strong connectedness and the alphabet. Thus, the claim for SetTrans and Intersection follow.

The reduction showing $\textsc{SetTrans} \leq_m^{\mathsf{AC}^0} \textsc{Intersection}$ from Item 9 in Theorem 8.2 does not preserve strong connectedness and the alphabet. However, it can easily be adapted by adding a new letter that swaps two states q and $(q, 0)$ with $q \in S$ and induces a self-loop at every other state not in S. Then the resulting automaton is strongly connected and this gives a valid reduction. With Theorem 8.15 we then obtain PSPACE-hardness even for binary strongly connected automata. This proves the claim for Intersection, SetTrans, and DisjSetTrans.

Reach was shown to be PSPACE-hard in [529, Thm. 3.2] and [99, Thm. 3] by reductions from SyncInto and Intersection, respectively, that add a single new letter. Neither of these reductions preserves strong connectedness. By Theorem 8.15 we obtain a reduction to a binary alphabet. Since $\textsc{Reach} \leq_m^{\mathsf{AC}^0} \textsc{ReachFrom}$ by Theorem 8.2 where

the reduction preserves the alphabet, we obtain the hardness claims for Reach and ReachFrom. □

8.3.3 Solvable Automata

Here, we show the following:

1. SetTrans, DisjSetTrans, and Intersection are NP-complete for solvable input automata (Theorem 8.18).
2. SyncFrom is NP-complete for solvable automata (Theorem 8.19).
3. SyncTrap, SyncState, Sync, and SyncInto are L-complete for solvable automata even for a unary alphabet (Theorem 8.20).
4. ReachFrom and Reach are NP-complete for solvable automata in (Theorems 8.21 and 8.22).

For self-containment, we give a proof of NP-hardness of Intersection for unary permutation automata. The proof is similar to the one given in [434, Lem. 27], but uses automata instead of regular expressions.

Proposition 8.17 Intersection *is* NP*-hard for unary permutation automata.*

Proof. We give a reduction from 3SAT, which is NP-complete by Theorem 3.76. Let $C_1, \dots, C_m \subseteq \{x_1, \overline{x}_1, \dots, x_n, \overline{x}_n\}$ be the clauses of an 3SAT-instance over the variables $x_1, \dots, x_n$. We can assume every variable appears, as a positive or negative literal, in some clause.

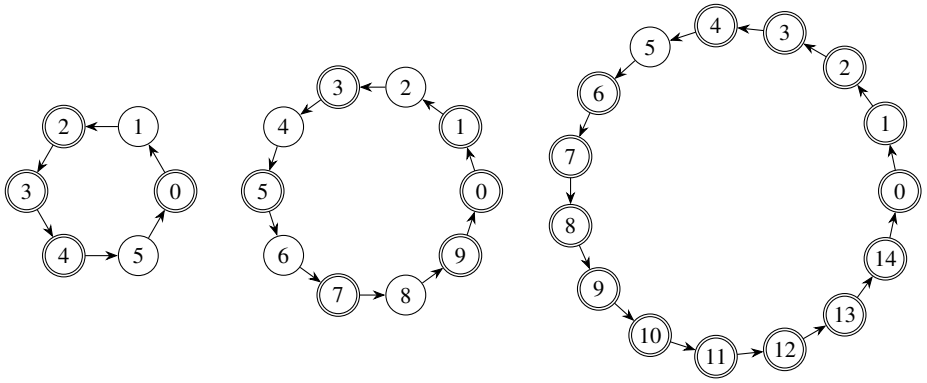
Construction: Let $p_1 < p_2 < \cdots < p_n$ be the first n prime numbers. For each clause C_j, $j \in \{1, \dots, m\}$, we construct $\mathcal{A}_j = (Q_j, \{a\}, \delta_j, q_j, F_j)$ with

$$\begin{aligned} Q_j &= \{0, 1, \dots, N_j - 1\}, \\ q_j &= 0, \\ F_j &= \{q \in Q_j \mid \exists i \in \{1, \dots, n\} : (x_i \in C_j \wedge q \equiv_{p_i} 0) \vee (\overline{x}_i \in C_j \wedge q \not\equiv_{p_i} 0)\}, \\ \delta_j(q, a) &= (q + 1) \bmod N_j \end{aligned}$$

where $N_j = \left(\prod_{\substack{i \in \{1,\dots,n\} \\ x_i \in C_j \vee \overline{x}_i \in C_j}} p_i \right)$, i.e., we multiply those primes that correspond to the variables in C_j.

By Theorem 3.2 there exists $N \geq 1$ such that $p_n < n^2$ for all $n \geq N$. We can assume $p_n < n^2$ in the following. Since $|C_j| \leq 3$ we get $|Q_j| \leq n^6$. Then, as detecting primality for unary numbers is in L [281, Thm. 7], the automata $\mathcal{A}_j$ can be constructed in deterministic log-space.

Figure 8.4 shows the construction for the clauses $C_1 = \{x_1, x_2\}$, $C_2 = \{\overline{x_1}, x_3\}$, and $C_3 = \{x_2, \overline{x_3}\}$ (shown in their logical interpretation in the figure). If $x_1 = 1$, then $x_3 = 1$ to fulfill C_2 and thus $x_2 = 1$ to fulfill C_3. For $x_1 = 0$, we must have $x_2 = 1$ to fulfill C_1



$\mathcal{A}_1$ for $C_1 = x_1 \vee x_2$ $\mathcal{A}_2$ for $C_2 = \overline{x_1} \vee x_3$ $\mathcal{A}_3$ for $C_3 = x_2 \vee \overline{x_3}$

Fig. 8.4: Example for the proof of Theorem 8.17. States with label 0 are start states. Final states are indicated by double circles. The label a at the transitions is omitted.

and x_3 can be arbitrary. Thus we find three satisfying assignments that correspond to the words a^{30}, a^3, and a^{15}.

Properties: We have the following two properties:

1. $\delta_j(q_j, a^r) \equiv_{N_j} r$ for each $r \geq 0$.
2. $(b_1, \ldots, b_n)$ is a satisfying assignment if and only if $a^{\prod_{i=1}^n p_i^{b_i}} \in \bigcap_{j=1}^m L(\mathcal{A}_j)$.

Property 1 follows from the construction of $\mathcal{A}_j$ whose automaton graph forms a single cycle. Property 2 follows since for $(b_1, \ldots, b_n) \in \{0,1\}^n$, $l = \prod_{i=1}^n p_i^{b_i}$, $j \in \{1, \ldots, m\}$, and $s_j = \delta_j(q_j, a^l)$ we have $s_j \equiv l \pmod{N_j}$ and then

$$
\begin{aligned}
&\exists i \in I\colon ((x_i \in C_j \wedge b_i = 1) \vee (\overline{x}_i \in C_j \wedge b_i = 0)) \\
&\Leftrightarrow \exists i \in I\colon ((x_i \in C_j \wedge l \equiv_{p_i} 0) \vee (\overline{x}_i \in C_j \wedge l \not\equiv_{p_i} 0)) && \text{(Def. } l\text{)} \\
&\Leftrightarrow \exists i \in I\colon ((x_i \in C_j \wedge s_j \equiv_{p_i} 0) \vee (\overline{x}_i \in C_j \wedge s_j \not\equiv_{p_i} 0)) && \text{(Eqn. (3.1), Property (1))} \\
&\Leftrightarrow s_j \in F_j && \text{(Def. } F_j\text{)}
\end{aligned}
$$

where $I = \{1, \ldots, n\}$.

Verification: By Property 2, if there exists a satisfying assignment, then we have $\bigcap_{j=1}^m L(\mathcal{A}_j) \neq \emptyset$. Conversely, suppose $a^l \in \bigcap_{j=1}^m L(\mathcal{A}_j)$ for some $l \geq 0$. Define an assignment $(b_1, \ldots, b_n)$ to the variables by setting, for $i \in \{1, \ldots, n\}$,

$$
b_i = \begin{cases} 1 & \text{if } p_i \text{ divides } l, \\ 0 & \text{if } p_i \text{ does not divide } l. \end{cases}
$$

Let $l' = \prod_{i=1}^n p_i^{b_i}$. Then $l' \equiv_{p_i} 0$ if and only if $l \equiv_{p_i} 0$ for $i \in \{1, \ldots, n\}$. So $a^{l'} \in L(\mathcal{A}_j)$ if and only if $a^l \in L(\mathcal{A}_j)$ for $j \in \{1, \ldots, m\}$. Hence, by Property 2, we obtain that $(b_1, \ldots, b_n)$ is a satisfying assignment. □

Theorem 8.18 SetTrans, DisjSetTrans, *and* Intersection *are* NP*-complete for solvable input automata, where* NP*-hardness holds even for the following classes of input automata:*

1. *binary partially ordered input automata,*
2. *ternary partially ordered monotonic automata,*
3. *commutative partially ordered automata over an unbounded alphabet,*
4. *unary permutation automata.*

Proof. That Intersection is NP-complete for the mentioned classes follows from results in the literature mentioned in Section 8.3.1, page 231. By Items 9 and 10 from Theorem 8.2 we obtain NP-hardness for all three problems, and by these statements it is sufficient to establish NP-membership for SetTrans, which we do in the following.

Let $\Sigma = \{a_1, \ldots, a_k\}$ and $\mathcal{A} = (Q, \Sigma, \delta)$ be a solvable input automaton to SetTrans with n states and input sets $S, T \subseteq Q$. If there exists a word mapping S into T, by Proposition 7.114 there exists a word of the form

$$v = v_0 x_1 \cdots v_{r-1} x_r v_r$$

for some $r \leq n^2$ where $x_1, \ldots, x_r \in \Sigma$ and $v_i = a_1^{r_{i,1}} \cdots a_k^{r_{i,k}}$ for numbers $0 \leq r_{i,j} < 2^n$ with $i \in \{0, 1, \ldots, r\}$ and $j \in \{1, \ldots, k\}$ mapping S into T. By using a binary encoding for the exponents $0 \leq r_{i,j} < 2^n$, a word of the above form can be stored using $O(|\Sigma|n \cdot (n^2 + 1) + \log(|\Sigma|)n^2)$ bits where each of the $n^2 + 1$ words v_i, $i \in \{0, 1, \ldots, r\}$, takes up $|\Sigma|n$ bits and each letter x_i, $i \in \{1, \ldots, r\}$, takes $\log(|\Sigma|)$ bits. So, using this encoding, it can be guessed in polynomial time. Next, the transition matrix of $a_j^{r_{i,j}}$ for $0 \leq r_{i,j} < 2^n$ can be computed in polynomial time by a square-and-multiply algorithm (also known as exponentiation by squaring) using the transition matrix of a_j. Hence, the transition matrix of v can be computed in polynomial time by multiplying the transition matrices of $v_0, x_1, \ldots, v_{r-1}, x_r$, and v_r in this order. Then v maps S into T if and only if each row in the transition matrix of v corresponding to a state in S has precisely one 1 at a column corresponding to a state in T. So, the above algorithm shows containment in NP. □

Theorem 8.19 SyncFrom *is* NP*-complete for solvable input automata even for binary partially ordered automata.*

Proof. Ryzhikov [537, Thm. 4] proved the problem to be NP-complete for partially ordered binary automata, see also page 232 in Section 8.3.1.

For NP-membership, we can guess a state $q \in Q$ in an n-state solvable input automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and then test whether there exists a word mapping Q to $\{q\}$. As the latter problem is in NP by Theorem 8.18, we obtain that SyncFrom is in NP. NP-hardness follows as SyncFrom is NP-complete for binary partially ordered automata as proved in [537, Thm. 4], see also Table 8.1. □

Theorem 8.20 SyncTrap, SyncState, Sync, *and* SyncInto *are* L*-complete for solvable input automata, even for a fixed unary alphabet.*

Proof. By Theorem 8.2 we have

$$\textsc{SingleCycle} \leq_m^{\mathsf{AC}^0} \textsc{SyncTrap} \leq_m^{\mathsf{AC}^0} \textsc{SyncState} \leq_m^{\mathsf{AC}^0} \textsc{SyncInto},$$

and $\textsc{SyncTrap} \leq_m^{\mathsf{AC}^0} \textsc{Sync}$, where the reductions of an automata-theoretical problem to another automata-theoretical problem given in the proof of Theorem 8.2 preserve the alphabet. Since SingleCycle is L-complete by Theorem 3.76, and the reduction to SyncTrap works even for unary automata, we obtain the L-hardness claims even for a fixed unary alphabet.

Since the reductions from Theorem 8.2 transforming automata preserve the property of being solvable, by the reductions from above, it is sufficient to prove L-membership for Sync and SyncInto to establish it for the other problems as well.

For Sync, to test synchronizability, by Theorem 7.113 we can simply test whether the word $(a_1 \cdots a_k)^{n-1}$ for $\Sigma = \{a_1, \ldots, a_k\}$ synchronizes the n-state solvable input automaton, which can be done in deterministic log-space by Theorem 8.9.

However, this is not sufficient since we cannot simply construct the word as this would take up more than logarithmic space, i.e., having a deterministic log-space Turing machine deciding whether a word synchronizes a given automaton, we cannot simply write $(a_1 \cdots a_k)^{n-1}$ and an encoding of the automaton on its input tape and then simulate it, as this would take up more than logarithmic space. But we can simulate the machine by letting it read the input automaton from the original input tape and feeding $(a_1 \cdots a_k)^{n-1}$ one symbol at a time to it in log-space without storing the full word similarly as closure of L is proved for log-space transducers, see [322, Lem. 13.2]; in fact, a log-space transducer that additionally produces $(a_1 \cdots a_k)^{n-1}$ can also be given to obtain the claim.

To solve SyncInto for $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$, by Theorem 7.103, Item 6 and Theorem 7.48 we have to test whether S contains all sink components of $\mathcal{A}$. We can do this by iterating through all states $q \in Q$, test whether it is contained in a sink component and if this is the case, test whether $q \in S$. By Theorem 7.50, a state $q \in Q$ is in a sink component if and only if for all $x \in \Sigma$ we have $\delta(q, x^i) = q$ for some $i \in \{1, \ldots, |Q|\}$. This can be checked in deterministic log-space by successively applying $|Q|$ times — using a counter that counts up to $|Q|$ and so takes up at most logarithmic space via a binary encoding — the letter x to a state stored in a memory cell initialized with q. Then it is easy to give an algorithm running in deterministic log-space. □

Theorem 8.21 Reach *is* NP*-complete for solvable automata even for partially ordered binary automata.*

Proof. The problem is NP-complete for partially ordered binary automata [537, Thm. 5]. Furthermore, as in the proof of Theorem 8.18 by using Proposition 7.114 we obtain NP-membership for solvable automata. □

Theorem 8.22 ReachFrom *is* NP*-complete for solvable automata even for commutative partially ordered automata, and also even for partially ordered binary automata.*

Proof. NP-hardness for partially ordered binary automata follows from Theorem 8.21 and Item 7 from Theorem 8.2. Next, in [73, Thm. 9] it has been shown that deciding for commutative partially ordered $\mathcal{A} = (Q, \Sigma, \delta)$ and $t \colon Q \to Q$ whether there exists

$u \in \Sigma^*$ such that $\delta(q, u) = t(q)$ for all $q \in Q$ is NP-complete.[9] This problem can be reduced to Intersection by letting $Q = \{q_1, \ldots, q_n\}$ and $\mathcal{A}_i = \mathcal{A}$ and $F_i = \{t(q_i)\}$ for $i \in \{1, \ldots, n\}$. Then when applying the reduction from Theorem 8.2, Item 9 to obtain an instance of SetTrans from this Intersection-instance the resulting automaton is a commutative partially ordered automaton $\mathcal{A}' = (Q', \Sigma, \delta')$ where $\delta'(\{q_1, \ldots, q_n\}, u) \subseteq F_1 \cup \ldots \cup F_n$ if and only if $\delta'(\{q_1, \ldots, q_n\}, u) = F_1 \cup \ldots \cup F_n$ for $u \in \Sigma^*$, i.e., we actually have a reduction to ReachFrom.

As in the proof of Theorem 8.18, by using Proposition 7.114, we obtain that ReachFrom is in NP for solvable automata. □

8.3.4 Commutative Automata

Here, we show the following:

1. Intersection, DisjSetTrans, and SetTrans are L-completeness for partially ordered commutative automata over a fixed non-empty alphabet (Theorem 8.23). Recall that in Section 8.3.3 we have shown that these problems are NP-complete for partially ordered commutative automata over an unbounded alphabet.
2. SyncFrom is L-complete for commutative automata (Theorem 8.23) even for unary partially ordered automata. Recall that this problem is NP-complete for solvable automata by Theorem 8.19.
3. SyncShort is L-complete for commutative automata over a fixed alphabet and NP-complete for an unbounded alphabet (Theorem 8.24). Hardness remains in both cases even for partially ordered commutative automata.

In this section, we assume an ordering $\Sigma = \{a_1, \ldots, a_k\}$ of each input alphabet Σ with $k \geq 1$ (the case $\Sigma = \emptyset$ is trivial and most problems are solvable in constant time).

Theorem 8.23 *1.* SyncFrom *is L-complete for commutative automata over an unbounded alphabet, where L-hardness holds even for unary partially ordered automata.*

2. SetTrans, DisjSetTrans, Intersection *are L-complete for partially ordered commutative automata over a fixed non-empty alphabet even for an alphabet of size one.*

Proof. By Theorem 8.2, where the reductions preserve the properties of commutativity, partial orderness, and the alphabet, it is sufficient to show the second claim for SetTrans.

For hardness, we give a reduction from 1-DAG-ST that works for SyncFrom and SetTrans, and also even if we fix k to be equal to one. For an 1-DAG-instance (G, s, t) with $G = (V, E)$, let $\mathcal{A} = (V, \Sigma, \delta)$ where, for $p \in V$ and $x \in \Sigma$,

[9] In [73] this problem was formulated as a problem on transformation monoids, where the commutative aperiodic transformation monoids used correspond precisely to the commutative partially ordered automata.

$$\delta(p,x) = \begin{cases} q & \text{if } (p,q) \in E \text{ for some } q \in V, \\ p & \text{otherwise.} \end{cases}$$

We have the following:

1. $\mathcal{A}$ is a well-defined deterministic and complete automaton as for every $p \in V$ there exists at most one $q \in Q$ such that $(p,q) \in E$.
2. $\mathcal{A}$ is a partially ordered commutative automaton.
3. t is a trap state in $\mathcal{A}$ since it has no outgoing transitions in G.

Now, the following are equivalent:

1. there exists a path from s to t in G,
2. $\delta(\{s\}, u) = \{t\}$ for some $u \in \Sigma^*$,
3. $|\delta(\{s,t\}, u)| = 1$ for some $u \in \Sigma^*$,

where the latter two properties are equivalent as t is a trap state and so must be the synchronizing state. By these equivalences, the reduction works for the problems SetTrans and SyncFrom. Thus these problems are L-hard even for $|\Sigma| = 1$ by Item 3 from Theorem 3.76.

Next, we show membership in L. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered commutative automaton with n states.

SyncFrom: Let $v = a_1^{n-1} \cdots a_k^{n-1}$. By Theorem 7.89, there exists $u \in \Sigma^*$ such that $|\delta(S,u)| = 1$ if and only if $|\delta(S,v)| = 1$.

To test whether $|\delta(S,v)| = 1$, we iterate through the states $S = \{s_1, \ldots, s_m\}$. For each state s_i, we compute $\delta(s_i, v)$ by processing the letters of v one by one and, for each letter a_j, applying it repeatedly $n-1$ times. We then check whether all computed values $\delta(s_i, v)$ are equal.

This procedure requires three counters: one to iterate over the states of S, one to iterate over the letters $a_1, \ldots, a_k$, and one to count the repetitions $0, \ldots, n-1$ for each letter. In addition, two memory cells suffice: one for the current state reached during the computation of $\delta(s_i, v)$, and one for storing the result of the previous iteration in order to compare $\delta(s_i, v)$ with $\delta(s_{i-1}, v)$ for $i > 1$.

Using binary encodings for the counters and for the stored states, the total space needed is logarithmic.

SetTrans: Let $S, T \subseteq Q$ and suppose, unlike in the proof of SyncFrom, that Σ is fixed. We iterate through all tuples $(n_1, \ldots, n_k)$ satisfying $0 \leq n_j < n$ for all $j \in \{1, \ldots, k\}$. Each tuple can be stored using k counters, which require $k \cdot \log n$ space when encoded in binary.

For each tuple $(n_1, \ldots, n_k)$, let $w = a_1^{n_1} \cdots a_k^{n_k}$. We then iterate through all states $q \in S$ and compute $\delta(q, w)$. If $\delta(q, w) \in T$ for every $q \in S$, we return YES. Otherwise, we continue with the next tuple. If the tuple $(n-1, \ldots, n-1)$ is reached without finding a word w such that $\delta(q, w) \in T$, we return NO.

The computation of $\delta(q, w)$ can be done in logarithmic space. Besides the k counters storing $(n_1, \ldots, n_k)$, we use a counter to iterate through the states in S, a memory cell storing the current state, a counter j going from 1 to k to keep track fo the current letter a_j, and a counter r to feed in the factor $a_j^{n_j}$ of $a_1^{n_1} \cdots a_k^{n_k}$. All of these require only $O(\log n)$ space.

Correctness follows from Equation (7.6) of Corollary 7.61, which implies that whenever there exists a word mapping S into T, there also exists a word of the form $a_1^{n_1} \cdots a_k^{n_k}$ with $0 \leq n_j < n$ for all $j \in \{1, \ldots, k\}$. □

Theorem 8.24 SyncShort *is* NP*-complete even for partially ordered commutative input automata. For a fixed non-empty alphabet, the problem is* L*-complete for commutative input automata even for unary partially ordered automata.*

Proof. NP-completeness: Martyugin [436, Prop. 5.3] proved NP-hardness for commutative automata over an unbounded alphabet, where the reduction resulted in a partially ordered commutative automaton. SyncShort is in NP for general input automata. See also the discussion in Section 8.3.1 (page 232).

L-hardness for fixed unary alphabet: By Theorem 8.6, we have SyncShort $\equiv_m^{\mathsf{AC}^0}$ SyncWords where the reductions preserve the automaton. Then, the claim follows as, by Theorem 8.9, SyncWords is L-complete for unary partially ordered automata.

L-membership for fixed alphabet: Let $(\mathcal{A}, l)$ with $\mathcal{A} = (Q, \Sigma, \delta)$ be an input instance. By Item 3 from Theorem 7.90, we can assume $l < |Q|$. Using k counters, iterate through all tuples $(n_1, \ldots, n_k)$ with $n_1 + \cdots + n_k \leq l$. By using a binary encoding, this can be done in $O(k \log l)$ space. Then, for each tuple, compute $\delta(q, a_1^{n_1} \cdots a_k^{n_k})$ for all $q \in Q$ and check whether always the same state results. If this is the case, then return YES. Otherwise, continue with the next tuple. If the iteration through all tuples ends and we have not found a synchronizing word, return NO.

This algorithm can be made to run in deterministic log-space similarly as in the proof of Theorem 8.23. □

Theorem 8.25 Reach *is* NP*-complete even for partially ordered commutative automata. For a fixed non-empty alphabet, the problem is* L*-complete even for unary partially ordered automata.*

Proof. Since commutative automata are solvable, by Theorem 8.21, the problem is in NP for commutative automata.

For hardness, we give a reduction from SyncShort for partially ordered commutative automata.

Let $(\mathcal{A}, l)$ with $\mathcal{A} = (Q, \Sigma, \delta)$ be an input instance for SyncShort where $\mathcal{A}$ is partially ordered and commutative. By Item 3 from Theorem 7.90, we can assume $l < |Q|$. we can assume $l < |Q| - 1$. By Theorem 7.83 if $\mathcal{A}$ is synchronizing, the synchronizing state is a unique trap state t. Existence of a unique trap state and its computation if it exists can be done by an AC^0-circuit as in the proof of Item 3 of Theorem 8.2. By Theorem 7.1, $\mathcal{A}$ has a synchronizing word of length at most l if and only if it has one of length exactly l. Let $\mathcal{A}' = (Q', \Sigma, \delta')$ with $Q' = Q \cup \{q_1, \ldots, q_l, q_{l+1}\}$ and, for $x \in \Sigma$, we set $\delta'(q_i, x) = q_{\min\{l+1, i+1\}}$ and $\delta'(q, x) = \delta(q, x)$ for $q \in Q$. Then $\mathcal{A}$ has a synchronizing word of length l if and only if $\delta'(Q', u) = \{t, q_l, q_{l+1}\}$ for some $u \in \Sigma^*$. Thus, we obtain a valid reduction to Reach.

Note that the alphabet is preserved and that $\mathcal{A}'$ is partially ordered and commutative. Thus, by Theorem 8.24, for an unbounded alphabet we obtain NP-hardness and for a

fixed non-empty alphabet we obtain L-hardness even for partially ordered commutative automata and unary partially ordered automata, respectively.[10]

Next, let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative n-state automaton. By Theorem 7.87, we obtain that all the sets of the form $\delta(Q, u)$ for $u \in \Sigma^*$ can be written as $\delta(Q, a_1^{i_1} \cdots a_k^{i_k})$ for $0 \leq i_1, \ldots, i_k < n$.

Now, the subsets $\delta(Q, a_1^{i_1} \cdots a_k^{i_k})$ for $0 \leq i_1, \ldots, i_k < n$ can be iterated through and tested for equality with a given target set S in deterministic log-space similarly as in the proof of Theorem 8.23 for SetTrans by iterating through the states, the letters, the numbers i_j, and using two memory cells. We also use, additionally, $|S|$ Boolean flags that indicate for each state in S whether it has been hit by some state when applying $a_1^{i_1} \cdots a_k^{i_k}$ to the state set. Then the algorithm returns YES precisely if all these flags are set to true for some $0 \leq i_1, \ldots, i_k < n$. For a fixed alphabet, this gives a deterministic algorithm using only logarithmic space. □

Theorem 8.26 ReachFrom *is* L*-complete for partially ordered commutative automata over a fixed non-empty alphabet, even for an alphabet of size one.*

Proof. By Theorem 8.2, Item 7, we have Reach $\leq_m^{\mathsf{AC}^0}$ ReachFrom and the reduction preserved the automaton type and the alphabet. This yields L-hardness by Theorem 8.25.

Next, the problem on commutative partially ordered automata can be solved with the same algorithm given in the proof of Theorem 8.23 for SetTrans, but where we also have to check for each tuple $(n_1, \ldots, n_k)$ not only that $\delta(q, a_1^{n_1} \cdots a_k^{n_k})$ is in T, but that each state from T appears at least once for some $q \in S$, i.e., the image equals precisely T. □

Theorem 8.27 ReachFrom *is solvable in polynomial time for commutative automata when the alphabet is fixed.*

Proof. In [50, Thm. 1.4], for fixed h and r, commuting $n \times n$ matrices $A_1, \ldots, A_h$, $B_1, \ldots, B_r$, and $n \times l$ matrices A and B, it was proved that it is decidable in polynomial time whether there exist non-negative integers x_i and y_j such that

$$\left(\prod_{i=1}^{h} A_i^{x_i}\right) A = \left(\prod_{j=1}^{r} B_j^{y_j}\right) B. \tag{8.2}$$

Suppose a commutative automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with $\Sigma = \{a_1, \ldots, a_h\}$ and $S, T \subseteq Q$ is given. Let A_i be the transition matrix of a_i, let $r = 1$ and B_1 be the identity matrix, and let A and B be the $n \times 1$ matrices resulting from the characteristic vectors of S and T, respectively. Then Equation (8.2) with these matrices has a solution $x_1, \ldots, x_h \geq 0$ if and only if $\delta(S, a_1^{x_1} \cdots a_h^{x_h}) = T$. Hence, we find that Reach is solvable in polynomial time for fixed h and commutative $\mathcal{A}$. □

[10] For L-hardness the reduction from the proof of Theorem 8.7 can also be used. In this construction, the set $S = \{(1, v_j), \ldots, (n, v_j), v_i, (1, v_i), \ldots, (n, v_i)\}$ is only reachable (by the word a^{n+j} in this case) from the whole state set if $j < i$ since $(n, v_1) \notin S$ and so each word reaching S has to have length at least n. But for $i < j$ this implies $(1, v_j) \notin S$ since the path v_{i+1} to $(1, v_j)$ has length $j - i < n$.

8.3.5 Partially Ordered Automata

Here, supplementing Ryzhikov's [537, Prop. 9 & Thm. 4] NP-completeness results for Intersection and SyncFrom on partially ordered automata, we show the following results for partially ordered input automata in Theorems 8.28 and 8.29: SyncInto, Sync, SyncTrap, and SyncState are in AC^0. Later, in Theorem 8.18, we will also prove that SetTrans is NP-complete for solvable automata, where NP-hardness remains even for partially ordered automata.

Theorem 8.28 SyncInto *is in* AC^0 *for partially ordered input automata.*

Proof. Let $(\mathcal{A}, S)$ be an n-state input instance where $\mathcal{A} = (Q, \Sigma, \delta)$ is partially ordered and $S \subseteq Q$. For an ordering from 1 to n of the states, we assume $\mathcal{A}$ is given by a bit-wise encoding of its transition matrices $[a]_{\mathcal{A}} = (m_{ij}^{(a)})_{i,j\in\{1,\ldots,n\}}$, $a \in \Sigma$, and S by a bit-vector $(s_1, \ldots, s_n)$ encoding the characteristic vector of S. By Theorem 7.54, we have to check whether S contains every trap state of $\mathcal{A}$. The state $i \in \{1, \ldots, n\}$ is a trap state if and only if $\bigwedge_{a\in\Sigma} m_{ii}^{(a)}$ is true. So, an AC^0 circuit implementing the following formula checks that S contains every trap state:

$$\bigwedge_{i=1}^{n} \left[\neg \left(\bigwedge_{a\in\Sigma} m_{ii}^{(a)} \right) \vee s_i \right] \qquad \left(\text{equivalent to } \bigwedge_{i=1}^{n} \left[\left(\bigwedge_{a\in\Sigma} m_{ii}^{(a)} \right) \rightarrow s_i \right] \right). \qquad \square$$

Theorem 8.29 Sync, SyncTrap *and* SyncState *are in* AC^0 *for partially ordered input automata.*

Proof. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state partially ordered input automaton encoded by its set of transition matrices $[a]_{\mathcal{A}} = (m_{ij}^{(a)})_{i,j\in\{1,\ldots,n\}}$ for each letter $a \in \Sigma$. By Corollary 7.58, $\mathcal{A}$ is synchronizing if and only if it has a unique trap state. This condition can be checked by an AC^0 circuit implementing the following formula:[11]

$$\bigvee_{i=1}^{n} \left[\left(\bigwedge_{a\in\Sigma} m_{ii}^{(a)} \right) \wedge \bigwedge_{\substack{j=1 \\ j\neq i}}^{n} \left(\bigvee_{a\in\Sigma} \neg m_{jj}^{(a)} \right) \right].$$

This shows the claim for Sync. As SyncTrap $\leq_m^{\mathsf{AC}^0}$ Sync and SyncState $\leq_m^{\mathsf{AC}^0}$ SyncInto by Theorem 8.2, Item 3 & 6 and these reductions preserve the property of being partially ordered, the claim follows for SyncTrap, and for SyncState by Theorem 8.28. $\square$

[11] Alternatively, the partially ordered automaton $\mathcal{A}$ has a synchronizing word if and only if there exists a trap state t such that $(\mathcal{A}, \{t\})$ is a YES-instance of SyncInto. This can be used to give an $\leq_m^{\mathsf{AC}^0}$-reduction from Sync to SyncInto. However, this reduction has to compute a trap state, and so the proof would not be simpler than the direct proof given here.

8.4 Discussion

Here, we discuss some other complexity results on problems related to synchronization from the literature. In view of the correspondence between automata and transformation semigroups from Section 6.8, we also discuss related problems on transformation semigroups. For notions from (parameterized) complexity theory or automata theory not defined before, we refer to the cited papers or other literature as, for example, [41] for classical computational complexity theory, [224] for parameterized complexity theory, or [490] for automata theory.

In this section, we do not discuss complexity issues related to the constrained synchronization problem introduced in Section 2.1.1 as this is done in Part IV of the present work.

Further Complexity Theoretical Results.

NP-completeness of SyncShort was established for the following classes of input automata in the literature:

1. Binary partially ordered automata with a unique trap state [198, 528];
2. SI-automata over an unbounded alphabet [210, 436];
3. Partially ordered commutative automata [210, 211, 436];
4. Eulerian[12] automata over a binary alphabet [621];
5. Circular automata over a binary alphabet [437];
6. Automata whose automaton multigraph is a two-terminal series-parallel multigraph with loops (TTSPL automata) over an alphabet with at least four letters [105].

Rystsov [528] and Eppstein [198], independently, gave the same reduction to show NP-hardness by constructing a binary partially ordered automaton. Essentially the same reduction was also given by Salomaa [546] to show NP-hardness of the problem whether a specific constant function can be composed out of a given set of functions. The reduction also appears as a special case in a paper by Goralčýk and Koubek [257] where the question of realizing idempotent transformations of a certain rank was investigated. Martyugin [436, Prop. 4.2] noted that the automaton resulting from this reduction has a trap state, is aperiodic, has a $\mathcal{J}$-trivial transition monoid, and belongs to the class of automata preserving a linear order on the states except at a single trap state, thus giving NP-hardness for these classes as well.[13] An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called *locally confluent* if, for any $q \in Q$ and $a, b \in \Sigma$, there exists $u \in \{a, b\}^*$ such that $\delta(q, au) = \delta(q, bu)$. Clearly, every synchronizing automaton over a binary alphabet is locally confluent. So, by the Rystsov-Eppstein-Salomaa reduction, SyncShort is NP-complete for locally confluent partially ordered automata. The NP-hardness result

[12] An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is *Eulerian* if $\sum_{x \in \Sigma} |\{p \in Q \mid \delta(p, x) = q\}| = |\Sigma|$ for each $q \in Q$.

[13] He called the latter automata partially monotonic automata, but since these automata have a totally defined transition function we avoid this term here to not cause confusion with partial automata where the transition function can be a partial function introduced later in Part IV.

on TTSPL automata was shown in [105] by adding two additional letters and additional states to obtain a TTSPL automaton from the Rystsov-Eppstein-Salomaa construction.

Eppstein [198] showed that for oriented n-state automata[14] over the alphabet Σ the problem SyncShort is solvable in time $O(|\Sigma|n^2)$. Every monotonic automaton is oriented [23], yielding polynomial time solvability for monotonic automata as well. Martyugin [436] showed that SyncShort is solvable in $O(n^{|\Sigma|}\log(n))$ for commutative automata and in $O(n)$ for binary SI-automata with n states (by checking whether it equals one of the two SI-automata from Theorem 7.31).

Note that, while SyncShort remains NP-complete even for partially ordered commutative input automata (Theorem 8.24), as we have shown in Section 8.3, the complexity of Sync drops from NL-completeness in the general case to AC^0 on partially ordered input automata (Theorem 8.29) and to L-completeness for solvable input automata (Theorem 8.20).

Olschweski and Ummels [472] proved that deciding whether a given automaton has a shortest synchronizing word of length *exactly* k is DP-complete. Furthermore, they proved that computing the length of a shortest synchronizing word, a synchronizing word of length k, and a shortest synchronizing word is $\mathsf{FP}^{\mathsf{NP}[\log]}$-complete, FNP-complete, and $\mathsf{FP}^{\mathsf{NP}[\log]}$-hard and in $\mathsf{FP}^{\mathsf{NP}}$, respectively. With respect to approximability, for n-state automata, Gawrychowski and Straszak [239] proved that for every $\epsilon > 0$ it is NP-hard to approximate the length of a shortest synchronizing word within a factor of $n^{1-\epsilon}$ even for a fixed binary alphabet. This result is optimal, as the Eppstein-Natarajan algorithm [198, 457] mentioned in Section 7.1.6 gives an $(n-1)$-approximation algorithm [243]. More generally, Gerbush and Heeringa [243] gave a $O(k|\Sigma|n^k + n^4/k)$-time $\lceil \frac{n-1}{k-1} \rceil$-approximation algorithm, where $k \geq 2$ denotes the size of the intermediate sets used for compression. Historically, before the optimal result by Gawrychowski and Straszak [239], under the assumption $\mathsf{P} \neq \mathsf{NP}$, Berlinkov firstly proved that there exists no polynomial time $O(1)$-approximation [79, 82] over a fixed binary alphabet. Gerbush and Heeringa [243] then proved that there is no polynomial time $O(\log n)$-approximation over an unbounded alphabet, which was improved by Berlinkov [83] to a fixed binary alphabet.

Fernau and Hoffmann [212] introduced, for a given partial order $\prec$ on Σ^*, the following *extension variant* of the synchronization problem.

ExtDFA-SW-$\prec$ (Fernau and Hoffmann [212])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$, $u \in \Sigma^*$.
Question: Is there $w \in \mathrm{Syn}(\mathcal{A})$ minimal w.r.t. $\prec$ in $\mathrm{Syn}(\mathcal{A})$ such that $u \prec w$?

For the prefix/suffix/lexicographic orders the problem is in P, for the scattered subsequence order even on commutative automata (and even for TTSPL automata) it is NP-hard. For the parameter $|u|$ and the scattered subsequence order it is W[3]-hard even on commutative automata (and even for TTSPL automata). The complementary problem that switches YES and NO answers for the length-lexicographic order is NP-complete even for TTSPL automata and in $\mathsf{WNL} \cap \mathsf{W}[P] \cap \mathsf{A}[2]$ w.r.t. $|u|$. For more information, we refer to [105, 209, 212].

[14] This is the terminology due to Ananichev and Volkov [23] which we use here as well, Eppstein himself called these automata monotonic; see also the discussion on page 223.

Maslennikova [441] showed that it is PSPACE-complete to decide whether a given regular ideal language is the set of synchronizing words of an automaton with at most four states over a binary alphabet.

Türker and Yenigün [609] introduced and investigated variants of the synchronization problem, the following problems being their main problems.

MinimumSynchronizableSubautomaton (Türker and Yenigün [609])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $k \geq 0$.
Question: Does there exist $\Gamma \subseteq \Sigma$, $|\Gamma| \leq k$, such that $\mathcal{A}_{|\Gamma}$ is synchronizing?

ExclusiveSyncWord (Türker and Yenigün [609])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$, $S, T \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta^{-1}(T, u) = S$?

MaxF-ReducingWord (Türker and Yenigün [609])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$, $F \subseteq Q$, $k \geq 0$.
Question: Does there exist $S \subseteq Q$, $|S| \geq k$, $u \in \Sigma^*$ such that $\delta(S, u) \subseteq F$?

They proved that MinimumSynchronizableSubautomaton is NP-complete[15] (which holds even for TTSPL automata as shown in [105, Cor. 5.9]), MaxF-ReducingWord is NP-hard for monotonic automata and ExclusiveSyncWord is PSPACE-complete. The reduction for MaxF-ReducingWord was challenged by Ryzhikov [537, p. 80] as "they reduce the N-Queens Puzzle [78] to this problem to prove its NP-hardness. However, their poof is unclear, since the input has size $O(\log N)$, and the output size is polynomial in N." Ryzhikov [537, Prop. 3] showed that MaxF-ReducingWord is PSPACE-complete for binary automata. In the context of parameterized complexity, MinimumSynchronizableSubautomaton was shown to be W[2]-hard even for TTSPL automata [105, Cor. 5.7 & 5.9] and in WNL ∩ W[P] [209, Thm. 12].

Section 8.3.1 collected known results on the subset mapping problems. Next, we give some further background. SyncInto, SyncFrom, and SetTrans were introduced, and shown to be PSPACE-complete, by Rystsov [529] under the names *(global) inclusion problem for non-initial automata* (*NIA*) [529, Thm. 2.1], *(global) inclusion problem for weakly-initial automata with a single final state* (WIA_1) [529, Cor. 3.3], and *(global) inclusion problem for weakly-initial automata* (*WIA*) [529, Thm. 2.1], respectively. Independently, Luks and McKenzie [425] later introduced SetTrans for permutation groups that asks, for a permutation group G on $[n]$ given by generators and subsets $A, B \subseteq [n]$, whether $g(A) = B$ for some $g \in G$. They proved that the problem is in NC for solvable permutation groups. For groups, the sets A and B must have the same size and $g(A) = B$ is equivalent to $g(A) \subseteq B$. We note that the automata-theoretical analogous problem with equality instead of set inclusion was investigated by Rystsov under the name *reachability problem for weakly-initial automata* and proved to be PSPACE-complete [529, Thm. 3.2], even when the starting set is the whole state set

[15] We omitted a cost function on the letters mentioned in [609], which does not change NP-completeness. Also, in [609] only NP-hardness is stated, but, as noted in [105], containment in NP follows as we can guess Γ and check synchronizability of $\mathcal{A}_{|\Gamma}$ in polynomial time.

and the target has size two.[16] In [251], reachability problems in certain discrete event systems were investigated that translate, in our language, to the problem Intersection for unary permutation automata and with different types of final state sets, giving polynomial-time solvable and NP-complete problems.

Blondin et al. [96] investigated different variants and generalizations of Intersection. They named SetTrans the problem whether there exists, in a transformation monoid given by a list of generators, a transformation mapping a given set into a set included in another given set. SetTrans as defined here is the automata-theoretical version of this problem, i.e., we opted to use the naming suggested by the work of Blondin et al. [96]. For a fixed number $k \geq 1$ of automata, Intersection is NL-complete [321, p. 464].

Berlinkov, Ferens and Szykuła [84] classified and investigated various further subset mapping problems. For $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$, their problems fall into the following broad main categories: *extending problems*, *totally extending problems*, and *resizing problems* that ask for $w \in \Sigma^*$ such that $|\delta^{-1}(S, w)| > |S|$, $\delta^{-1}(S, w) = Q$, and $|\delta^{-1}(S, w)| \neq |S|$, respectively. They investigated those problems with and without a binary encoded bound on $|w|$ and, for the former two categories, with and without a fixed parameter k such that $|S| \leq k$ or $|S| \geq n - k$, which is used, for example, in runtime bounds of algorithms. A word $w \in \Sigma^*$ *avoids* $S \subseteq Q$ if $\delta(Q, w) \cap S = \emptyset$. As this is equivalent to $\delta^{-1}(Q \setminus S, w) = Q$, totally extending problems can be rephrased as problems that ask for avoiding words of the complement. Furthermore, the authors considered *compression*, i.e., $|\delta(S, w)| < |S|$, which they showed to be polynomial-time solvable, and *shrinking*, i.e., $|\delta^{-1}(S, w)| < |S|$, which is equivalent to extending $Q \setminus S$. As $\delta^{-1}(S, w) = Q$ is equivalent to $\delta(Q, w) \subseteq S$, the problem of total extension without parameters is equivalent to SyncInto, which was investigated earlier by Rystsov [529]. For a full overview of the results we refer to their work [84].

A recent trend in fixed parameter tractability [65] and the field of artificial intelligence [288, 336] asks for *diverse solution sets*, giving a user more freedom of selection of a particular solution to fit his/her needs. Let $\mathcal{A}$ be an automaton. Arrighi et al. [43] extended automata synchronization to the diversity setting by saying a finite set $W \subseteq \mathrm{Syn}(\mathcal{A})$ has *synchronization diversity* k if two distinct words from W have edit distance at least k and each word in W is minimal with respect to the scattered subsequence order in $\mathrm{Syn}(\mathcal{A})$. Then, they proved that deciding whether an automaton admits a sets of synchronization diversity at least k is co-NP-hard and that computing the diversity of the set of words minimal w.r.t. scattered subsequence order in $\mathrm{Syn}(\mathcal{A})$ is hard for the counting complexity class #P.

Parameterized Complexity.

The following results on the parameterized complexity of SyncShort are known (the paramterized complexity class WNL is a lesser known class defined via the number of tape cells used by a nondeterministic Turing machine [269]):

[16] The case that the starting set is the whole state set was also shown to be PSPACE-complete independently by Bondar and Volkov [99, Thm. 3].

1. SyncShort is in FPT w.r.t. $(|\Sigma|, l)$ [211, Thm. 7], as we can try all $|\Sigma|^l$ words of length l, and the number of states n [211, Thm. 10], as we can search in the power automaton with 2^n states and $|\Sigma|2^n$ transitions. Furthermore, unless the polynomial-time hierarchy collapses on the third level, SyncShort admits no polynomial-size kernel when parameterized by $(|\Sigma|, l)$ [105, Cor. 4.6] or by n [623].
2. Bruchertseifer and Fernau [209] introduced the parameterized complexity class W[Sync] as the class of problems fpt-reducible to SyncShort with parameter being the length-bound l. They showed that W[Sync] sits between W[2] and WNL ∩ W[P] ∩ A[2], see [211, Thm. 3], [105, Thm. 4.2 & Prop. 4.4], and [209, Thm. 3, 4 & 5].

Montoya and Nolasco [453, 454] proved that SyncFrom with a length bound on the solution word is WNL-complete when parameterized by the size of the input set.

Wareham [626] investigated *bounded automata intersection* with a parameter l asking for a solution word of length exactly l accepted by m automata each with at most n states. He obtained containment in FPT for the following parameter combinations:

1. $(|\Sigma|, l)$ by trying all words in Σ^l,
2. (m, n) via the product automaton of size n^m of all input automata,
3. $(n, |\Sigma|)$ by noting that there are at most $n^{|\Sigma|n}$ different automata, so that we can assume $m \leq n^{|\Sigma|n}$, and use the reasoning as for (m, n).

In fact, Wareham stated the above results under the assumption $l \leq n$. Clearly, this restriction is not necessary for 1 to hold. But it is also not necessary for 2, and so 3, since the l-th power of the transition matrix of the automaton graph of the product automaton mentioned in 2 can be computed with $\log(l)$ multiplications by a square-and-multiply algorithm, and this number is polynomial even for a binary encoding of l. For hardness, he obtained the following results: $W[1]$-hard for (m, l), $W[2]$-hard for (n, l), $W[t]$-hard for all $t \geq 1$ for the sole parameter m, and not in XP for the sole parameter $|\Sigma|$ (under the assumption P ≠ NP); the construction also implies this for the sole parameter n, as noted in [213]. Further, Bruchertseifer and Fernau [209, Thm. 9], Fernau et al. [213, Cor. 2.2], and Guillemot [269, Prop. 10] proved W[Sync]-completeness w.r.t. l, $W[1]$-completeness w.r.t. (m, k), and WNL-completeness w.r.t. m, respectively. Moreover, Fernau et al. [213, Thm. 5.8] proved $W[1]$-completeness w.r.t. m for automata accepting sparse languages even for unary automata (which accept sparse languages).[17] We remark that the co-$W[2]$-hardness proof in [213, Prop. 6.3] is wrong. For further information on synchronization problems, parameterized complexity, and also runtime lower bounds derived from ETH and SETH we refer to [105, 211].

Later, in Theorems 12.4 and 12.6, we will show that Intersection and SyncShort are solvable in time $O(m|\Sigma|n^{|\Sigma|+1})$ on m commutative partially ordered input automata with at most n states and in time $O(n^{|\Sigma|+2})$ on commutative n-state automata, respectively.[18] So, both problems are in the parameterized complexity class XP w.r.t. $|\Sigma|$ for the mentioned types of input automata. The XP-result for Intersection on commutative partially ordered automata has been extended in [42] to *totally star-free nondeterministic automata* accepting commutative languages.

[17] See Section 17.7 for a definition of sparse languages.

[18] Martyugin [436] obtained an algorithm running in time $O(|\Sigma|n^{|\Sigma|}\log(n))$.

Lastly, testing whether m binary automata accept a common word is equivalent to the simulation of a nondeterministic Turing machine that uses $m \log(n)$ bits of space [363] (see also [234, 356, 406, 635, 636]). In [356, 635], for each fixed m the resulting families of automata intersection problems have been used to obtain conditional separation results between NL and P.

Practical Implementations to Compute Shortest Synchronizing Words.

As SyncShort is NP-complete there exists no polynomial time algorithm computing a shortest synchronizing word under the assumption NP $\neq$ P. However, people have tried to come up with various heuristics to obtain algorithms that run efficiently on most instances in practice. The earliest implementations of exact algorithms using several approaches were given by Rho et al. [510], Pixley et al. [493], and Deshmukh and Hawat [174]. Kisielewicz et al. [342] have given an algorithm that combines multiple techniques and performs well on most instances. This algorithm has been recently improved by Szykuła and Zyzik [594] (see also the related forward and backward scheme by Szykuła and Roman [519]). Algorithms based on machine learning were proposed by Podolak et al. [495, 496], and an evolutionary algorithm by Kowalksi and Roman [386]. Tas et al. [597] studied the performance of some algorithms exploiting GPUs and parallelism on automata with up to a billion states. Furthermore, in [151, 355, 549] the authors optimize existing algorithms for real-life applications, for example in model-based testing.

The Natarajan-Eppstein algorithm [198, 457] mentioned in Section 7.1.6 runs in polynomial time and outputs some synchronizing word that is guaranteed to have at most cubic length. Two other algorithms with these properties were suggested by Trahtman [602][19] and Roman [517]. Kudłacik et al [390] compare different exact and non-exact algorithms.

For more information, we refer to the introductory section of [342] that contains further information and additional references to implementations and algorithms.

Related Problems on Transformation Semigroups.

Goralčík and Koubek [257] investigated the problem TransformationRank (resp. IdempotentRank) which asks, for a set $T \subseteq \mathcal{T}_n$ and $r > 0$, whether $\langle T \rangle_S$ contains a transformation (resp. idempotent transformation) of rank r. For the fixed parameter $r = 1$, both problems reduce to Sync. Further, for each fixed $r > 0$ they proved that TransformationRank is NP-hard and IdempotentRank is solvable in time $O((r+1)n^{(2r+1)}|T|)$. In general, IdempotentRank was proved to be PSPACE-complete. The precise complexity of TransformationRank is an *open problem*. A variant of IdempotentRank with a length-bound for a product of transformations from T was proved to be NP-complete

[19] In other works spelled as Trakhtman.

for each fixed r.[20] In the realm of parameterized complexity theory, Bruchertseifer and Fernau [209, Thm. 8] proved TransformationRank complete for W[Sync].

Kozen [387] introduced Membership that asks, for $T \subseteq \mathcal{T}_n$ and $t \in \mathcal{T}_n$, whether $t \in \langle T \rangle_S$. Now, we obtain various variants of Membership by restricting T to generate monoids from some given class. Similarly, for the problem Intersection by focusing the attention to automata whose transformation monoids come from some common class of monoids, we obtain variants of this problem With respect to these variants, Blondin et al. [96] observed the following relation.

> "[DFA-Intersection] is equivalent to [Membership] when both are intractable, but appears harder than [Membership] when [Membership] is efficiently solvable." [96]

In fact, the complexity gap can be quite large. For example, by results of Arvind and Vijayaraghavan [45] it follows that Membership on abelian permutation groups is in NC^2, whereas Intersection even on unary permutation automata is NP-complete [74, 213, 234, 434, 584] (see also Theorem 8.18). More specifically, we have Membership $\leq_m^{\mathsf{AC}^0}$ Intersection as for $t, t_1, \ldots, t_k \colon [n] \to [n]$ with $\mathcal{A} = ([n], \Sigma, \delta)$ given by $\Sigma = \{t, t_1, \ldots, t_k\}$ and $\delta(i, x) = x(i)$ for $x \in \Sigma$ we have

$$t \in \langle \{t_1, \ldots, t_k\} \rangle_S \Leftrightarrow \bigcap_{i=0}^{n-1} L(\mathcal{A}_{i,\{t(i)\}}) \neq \emptyset.$$

Now, suppose $\mathcal{A}_i = (Q_i, \Sigma, \delta_i, q_i, F_i)$ with $i \in \{1, \ldots, k\}$ form an input to Intersection. We can assume the state sets Q_i are pairwise disjoint. Let $t_a \colon Q_1 \cup \cdots \cup Q_k \to Q_1 \cup \cdots \cup Q_k$ for $a \in \Sigma$ be the transformations with $t_a(q) = \delta_i(q, a)$ for the unique $i \in \{1, \ldots, k\}$ such that $q \in Q_i$. Then

$$\bigcap_{i=1}^{k} L(\mathcal{A}_i) \neq \emptyset \Leftrightarrow \exists t \in \langle \{t_a \mid a \in \Sigma\} \rangle_S \; \forall i \in \{1, \ldots, k\} \colon t(q_i) \in F_i.$$

So, we can solve $\bigcap_{i=0}^{n-1} L(\mathcal{A}_i) \neq \emptyset$ by guessing a transformation t and then checking checking whether $t \in \langle \{t_a \mid a \in \Sigma\} \rangle_S$ and $t(q_i) \in F_i$ for all $i \in \{1, \ldots, k\}$. This reduction up to some "nondeterminism" makes the "equivalent when intractable"-statement more precise.[21] Note that, for classes generated by the t_a such that Membership is solvable in polynomial time, this procedure gives an FPT-algorithm for the parameter $\max\{|F_i| \mid i \in \{1, \ldots, k\}\}$.

Closer inspection of Kozen's proof [387, Thm. 3.2.6] reveals that Membership is PSPACE-complete even for aperiodic transformation monoids as input. For aperiodic transformation monoids, the Membership problem was classified into five classes giving problems that are in AC^0, P- , NP-, or PSPACE-complete, or NP-hard, by Beaudry et al. [77], where NP-hardness holds for aperiodic transformation monoids of threshold at least two. Partial results for general threshold-one monoids were obtained by

[20] Their proof [257, Thm. 5] reduces the problem to the special case $r = 1$ and so, basically, SyncShort. They give a reduction from 3SAT which is a little different than the Rystsov-Eppstein construction [198, 528] but similar to constructions from [210, 211, 212, 436].

[21] The construction is due to Lukas Fleischer (personal communication).

Beaudry [76]. For commutative transformation monoids, Beaudry [73] proved NP-completeness.

For permutation groups, MEMBERSHIP was shown to belong to P by Furst et al. [231] (analyzing an algorithm given by Sims [574]), then to NC^3 for abelian groups by McKenzie and Cook [447] and Mulmuley [455] (improved by Arvind and Vijayaraghavan [45] to containment in $\mathsf{ZPL}^{\mathsf{ModL}}$, which is in NC^2). Luks and McKenzie [483] proved membership in NC for solvable groups, Luks [424] extended this to groups with bounded non-abelian composition factors, and Babai et al. [49] finally established membership in NC for all groups.

Now consider the variant of MEMBERSHIP parameterized by an integer l, where one asks whether a given transformation admits a factorization of length at most l. This problem is PSPACE-complete for l in binary encoding and NP-complete for l in unary encoding, even for permutation groups as shown by Jerrum [343, Thm. 2.2] and Even and Goldreich [204, Thm. 1], respectively. Furthermore, Fernau and Bruchertseifer [209] proved W[Sync]-completeness w.r.t. l and general monoids. When asking for a product of length exactly l, Cai et al. [123, Thm. 6 & 7] proved W[2]-hardness in general and W[1]-hardness for permutation groups w.r.t. l. In case of permutation groups, asking for a product of length exactly l was proved DP-complete for a unary encoding of l and PSPACE-complete for a binary encoding by Lohrey and Rosowski [421, Thm. 4.2 & 4.3].

For further information on INTERSECTION and MEMBERSHIP, the paper by Blondin et al. [96] and the master thesis by Blondin [95] give a good overview. For more algorithmic investigations on INTERSECTION, we refer to further work by Wehar and coauthors [213, 470, 471, 635, 636] and others [42, 211, 214, 356, 406, 626]. Intersection problems on transformation semigroups were investigated by Fleischer and Kufleitner [223] and Fleischer [222], see also the thesis by Fleischer [221]. The membership problem for finite monoids and groups has been generalized to the *rational (subset) membership problem* [353, 420].

A Remark on Local Confluence and Synchronization

Simon [572, 573] introduced piecewise testable languages and characterized them as those languages acceptable by automata with $\mathcal{J}$-trivial transition monoids. Klíma [371, Thm. 1] proved that a language is piecewise testable if and only if it is acceptable by a locally confluent partially ordered automaton. Since we mentioned locally confluent automata above, we point out a connection of synchronizing automata to piecewise testable languages that has not been noted before and is based on the following statement.

Proposition 8.30 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be partially ordered and accessible. If $\mathcal{A}$ is locally confluent, then it is synchronizing.*

Proof. We do induction on $|Q|$. Clearly, the claim holds for $1 \leq |Q| \leq 2$. Next, suppose $|Q| \geq 3$.

Since $\mathcal{A}$ is accessible, there either exists $a \in \Sigma$ such that $\delta(q_0, a) \neq q_0$ and $\delta(q_0, a) = \delta(q_0, b)$ for each $b \in \Sigma$ with $\delta(q_0, b) \neq q_0$, or there exist $a, b \in \Sigma$ such that for

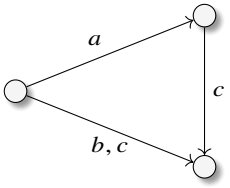


Fig. 8.5: Synchronizing partially ordered automaton that is not locally confluent. Self-loops omitted.

$p = \delta(q_0, a)$ and $q = \delta(q_0, b)$ we have $q_0 \neq p$ and $q_0 \neq q$ and $p \neq q$. In the first case, the set $\uparrow\{\delta(q_0, a)\}$ is a proper upper set of size at least two. In the second case, by assumption, there exists $u \in \{a, b\}^*$ with $\delta(p, u) = \delta(q, u)$. Since $p \neq q$, we have $p \neq \delta(p, u)$ or $q \neq \delta(q, u)$. Thus either $\uparrow\{p\}$ or $\uparrow\{q\}$ has size at least two and neither contains q_0.

Thus, in each case we find $q \in Q$ such that for $S = \uparrow\{q\}$ we have $2 \leq |S| < |Q|$.[22] Then $\mathcal{A}_{|S}$ and $\mathcal{A}/{\sim_S}$ are accessible, partially ordered, and locally confluent. So, inductively, both automata are synchronizing. By Theorem 7.3, Item 4 it follows that $\mathcal{A}$ is synchronizing. □

Corollary 8.31 *An accessible partially ordered automaton over an alphabet of size at most two is synchronizing if and only if it is locally confluent.*

Proof. First, let $\mathcal{A} = (Q, \Sigma, \delta)$ be synchronizing with $|\Sigma| \leq 2$ and synchronizing word $u \in \Sigma^*$. Let $q \in Q$ and $a, b \in \Sigma$. We can assume $a \neq b$, so that $\Sigma = \{a, b\}$. Thus $u \in \{a, b\}^*$ and since u is synchronizing we have $\delta(q, au) = \delta(q, bu)$. Hence, $\mathcal{A}$ is locally confluent.

Since by Theorem 8.30 local confluence implies synchronizability, we obtain the claim. □

Thus, with the result by Klíma [371, Thm. 1] mentioned above, we obtain that a binary language is piecewise testable if and only if its minimal automaton is a synchronizing partially ordered automaton.

Theorem 8.31 does not hold for alphabets with more than two letters, for example, see Figure 8.5.

[22] Alternatively, either there exists $a \in \Sigma$ such that $q = \delta(q_0, a)$ is not a trap state and $q \neq q_0$, in which case $\uparrow\{q\}$ is proper and has size at least two, or $\delta(q_0, a)$ is a trap state for each $a \in \Sigma$. Since for each $a, b \in \Sigma$, $\delta(q_0, au) = \delta(q_0, bu)$ for some $u \in \{a, b\}^*$, in the latter case we must have $|Q| \leq 2$.

Part III
Automata for Sets of Synchronizing Words

"My best tool for efficiency and performance is abstraction."
—Bjarne Stroustrup (1950–present)

Chapter 9
Reachable Sets and Pairs of Distinguishable Sets

In this chapter, we investigate the subset mapping properties in automata and transformation semigroups, motivated by minimality conditions for automata accepting sets of synchronizing words. The results presented here can be combined to derive lower bounds for the size of these automata.

In Section 9.1, we introduce and motivate the notions of reachability and distinguishability. We do this by first observing that the power automaton accepts the set of synchronizing, and then naturally obtain both notions via minimality conditions for accepting automata from Section 6.7.2 applied to the power automaton. Thus, these notions derive from necessary conditions characterizing the size of minimal automata for sets of synchronizing words.

Section 9.2 investigates the reachability of subsets of states in automata. We derive synchronizability conditions and structural limitations in terms of the reachability of families of subsets of states.

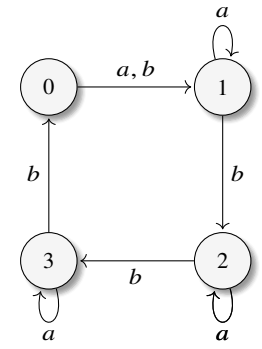
Section 9.3 is concerned with distinguishability, where we investigate its relation to synchronizability and consider different families of pairwise distinguishable subsets and their relations. We also introduce the novel notion of completely distinguishable automaton.

Reachability and distinguishability transfer readily from automata to transformation semigroups. Section 9.4 shortly discusses this and some differences that arise as a transformation semigroup might not contain the identity transformation.

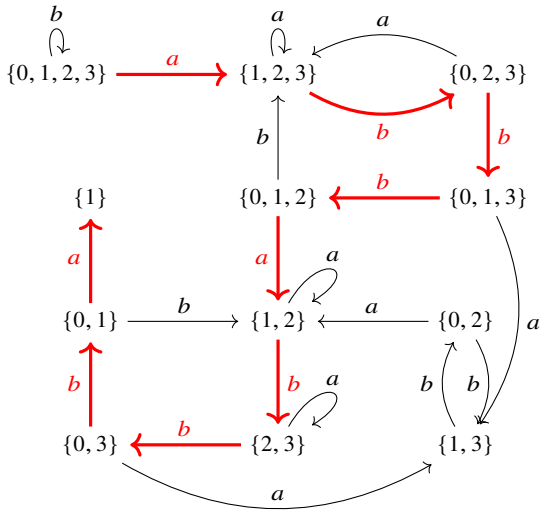
9.1 Power Automata and Sets of Synchronizing Words

Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton. Set $F = \{\{q\} \mid q \in Q\}$. We have $\mathrm{Syn}(\mathcal{A}) = \{u \in \Sigma^* \mid \delta(Q, u) \in F\} = L(\mathcal{P}(\mathcal{A})_{Q,F})$, i.e., the power automaton $\mathcal{P}(\mathcal{A})_{Q,F}$ accepts $\mathrm{Syn}(\mathcal{A})$. Figure 9.1 depicts $\mathcal{P}(C_4)$ for the Černý automaton C_4, excluding the empty set and the singleton subsets except $\{1\}$, with the path traced out by the synchronizing word $abbbabbba$ marked in red.

We get an automaton with at most $2^n - 1 - (n-1) = 2^n - n$ states accepting $\mathrm{Syn}(\mathcal{A})$ by only considering the states reachable from Q in $\mathcal{P}(\mathcal{A})$ and by merging all states in F to a single state. Formally, it is $\mathrm{Syn}(\mathcal{A}) = L(((\mathcal{P}(\mathcal{A})_{|S})/{\sim_F})_{\pi_{\sim_F}(S),\{F\}})$ for $S = \{A \mid \exists u \in \Sigma\colon \delta(Q,u) = A\}$, giving the next theorem.[1]



(a) The Černý automaton C_4.



(b) Part of the power automaton of C_4.

Fig. 9.1: C_4 and part of its power automaton. The synchronizing word *abbbabbba* of C_4 is marked by thick red transitions in the power automaton.

Theorem 9.1 *For every n-state automaton $\mathcal{A}$, the set of synchronizing words* $\mathrm{Syn}(\mathcal{A})$ *is acceptable by an automaton with $2^n - n$ states.*

Theorem 6.88, Item 4 stated two conditions for the minimality of a given automaton as a language acceptor: reachability (or accessibility) of states and distinguishability of pairs of states. Motivated by these two requirements translated to $\mathcal{P}(\mathcal{A})_{Q,F}$ as a language acceptor for $\mathrm{Syn}(\mathcal{A})$, we introduce the following two notions for subsets of states of the automaton $\mathcal{A} = (Q, \Sigma, \delta)$.

1. A subset $A \subseteq Q$ is *reachable (in $\mathcal{A}$)* if there exists a word $u \in \Sigma^*$ such that $\delta(Q,u) = A$, i.e., the set is reachable from Q in the power automaton.
2. We call two subsets of states $A, B \subseteq Q$ *distinguishable (in $\mathcal{A}$)* if they are distinguishable in $\mathcal{P}(\mathcal{A})_{Q,F}$ according to Equation (6.26) where $F = \{\{q\} \mid q \in Q\}$. Equivalently, there exists a word $u \in \Sigma^*$ (called a *distinguishing word*) such that

$$(|\delta(A,u)| = 1 \wedge |\delta(B,u)| \neq 1) \vee (|\delta(A,u)| \neq 1 \wedge |\delta(B,u)| = 1). \tag{9.1}$$

[1] Note that $S \subseteq \{A \mid A \neq \emptyset, A \subseteq Q\}$ for $Q \neq \emptyset$. Thus, in this case, we also obtain an automaton by merging all states in F and retaining all subsets of states except the empty set.

We say the word u *distinguishes* the two subsets A and B, or that u *distinguishes the pair of subsets* formed by A and B. Note that two singleton subsets are never distinguishable.

Remark 9.2 Let $p, q \in Q$ be distinct and $u \in \Sigma^*$. Then u distinguishes all non-empty sets $A \subseteq \delta^{-1}(p, u)$ and $B \subseteq Q$ with $\delta^{-1}(p, u) \cap B \neq \emptyset$ and $\delta^{-1}(q, u) \cap B \neq \emptyset$.

Theorem 9.3 gives an automaton where all pairs of non-disjoint 2-sets are distinguishable and pairs of disjoint 2-sets are indistinguishable.

Example 9.3 Consider the automaton from Figure 9.2. Let $\{p_1, p_2\}, \{p_1, p_3\}$ be two distinct 2-sets of states with one common state. Then there exists a letter mapping p_1 to p_2, and p_3 to another state, so that both subsets are distinguishable. This also implies that every 2-set is distinguishable from every set of size at least three. Next, we have the following pairs of disjoint 2-sets:

$$\{1, 2\}, \{3, 4\}, \quad \{1, 3\}, \{2, 4\}, \quad \{1, 4\}, \{2, 3\},$$

and it can be easily seen that none of these pairs is distinguishable. Furthermore, also all pairs of 3-sets are indistinguishable. This can be seen by first noting that all pairs of distinct 3-sets have two states in common. So, let $\{p_1, p_2, p_3\}$ and $\{p_1, p_2, p_4\}$ be two distinct 3-sets. Consider some letter. First, suppose it moves one point from $\{p_1, p_2\}$, say p_1. If it maps p_1 to p_2, then it maps, say p_3, to p_4 and we end up at the same 2-sets. If it maps p_1 to, say p_3, then it moves p_2 to p_4 or vice versa and we also end up at the same 2-set. Otherwise, if it fixes both points p_1 and p_2, then, without loss of generality, it moves p_3 to p_1 and p_4 to p_2 and starting from both sets, we end up at $\{p_1, p_2\}$.

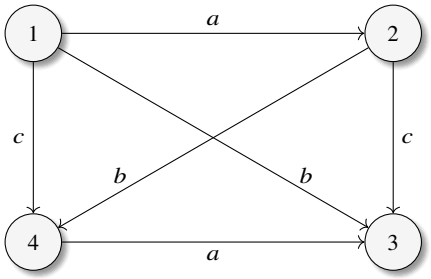


Fig. 9.2: Automaton where non-disjoint 2-sets are distinguishable and pairs of disjoint 2-sets are indistinguishable. Self-loops omitted.

Example 9.4 In the automaton from Figure 9.3, for a subset of states A of size at least two, A and $A \cup \{1\}$ are indistinguishable. Pairs of sets A, B with $|A|, |B| \geq 2$ that do not have this form are distinguishable, for example, all pairs of subsets of states of size at three are distinguishable.

Example 9.5 In the synchronizing SI-automaton from Figure 7.4c, sets of the form A and $A \cup \{t\}$ for $|A| \geq 2$ are indistinguishable and pairs of non-empty and non-singleton subsets that do not have this form are distinguishable.

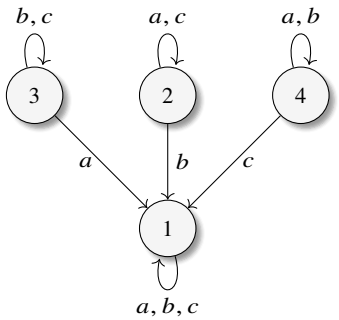


Fig. 9.3: Sets of the form A and $A \cup \{1\}$ with $|A| \geq 2$ are indistinguishable. Pairs of sets A, B with $|A|, |B| \geq 2$ that do not have this form are distinguishable.

Our next examples show that completely distinguishable automata can have a trap state. Theorem 9.6 is a modification of Theorem 9.5 by adding a single letter.

Example 9.6 Consider the SI-automaton $\mathcal{A} = (Q, \{a, b\}, \delta)$ from Figure 7.4c and add another simple idempotent c letter mapping some $q \in Q \setminus \{t\}$ to the state $\delta(q, b)$. Then distinct subsets $A, B \subseteq Q \setminus \{t\}$ are distinguishable since the automaton with letters b and c acts in the same way as the Černý automaton on these states. For $A \subseteq Q$ and $B \subseteq Q \setminus \{t\}$ with $t \in A$, we can map B to $\{p\}$ for some $p \in Q \setminus \{t\}$ by some $u \in \{b, c\}^*$ for the same reason as before and where $|\delta(A, u)| \geq 2$. For distinct $A, B \subseteq Q$ with $t \in A \cap B$, if, without loss of generality, $q \in A \setminus B$ for some $q \in Q$, then we can successively map all states in B to t while the state q is mapped to a state distinct from t to obtain a distinguishing word for both sets. Thus, all pairs of distinct non-empty and non-singleton subsets are distinguishable.

Example 9.7 Let $\mathcal{A} = (Q, \Sigma, \delta)$ with a state $t \in Q$ and $\Sigma = \{a_{pq} \mid p, q \in Q \setminus \{t\}, p \neq q\} \cup \{b_{pt} \mid p \in Q \setminus \{t\}\}$ where a_{pq} is a simple idempotent letter mapping p to q for distinct p, q not equal to t and b_{pt} is a simple idempotent letter mapping p to t for a state p unequal to t. Then t is a trap state. Next, we show that $\mathcal{A}$ is completely distinguishable. Consider two distinct non-empty and non-singleton subsets A, B with $|A| \leq |B|$. First, suppose $t \notin A$ and write $A = \{q_1, \ldots, q_m\}$ with $m = |A|$. Then $\delta(A, u) = \{q_1\}$ and $|\delta(B, u)| = 2$ for $u = a_{q_2 q_1} \cdots a_{q_m q_1}$ (note that B cannot be a subset of A since $|A| \leq |B|$ and $A \neq B$). Lastly, suppose $t \in A \cap B$, or $t \in A$ and $t \notin B$. Write $A = \{q_1, \ldots, q_m, t\}$ with $m = |A| - 1$. Then $\delta(A, u) = \{t\}$ and $|\delta(B, u)| \geq 2$ for $u = b_{q_1 t} \cdots b_{q_m t}$. Note that since $|A| \leq |B|$ and $A \neq B$, we have $B \neq \{q_1, \ldots, q_m\}$.

Example 9.8 An automaton is synchronizing if and only if $\emptyset$ is distinguishable from every other non-empty set.

Remark 9.9 Let $q \in Q$ and $u \in \Sigma^*$. Then u distinguishes every non-empty $A \subseteq \delta^{-1}(q, u)$ from every $B \subseteq Q$ such that $B \cap \delta^{-1}(p, u) \neq \emptyset$ and $B \cap \delta^{-1}(r, u) \neq \emptyset$ for distinct $p, r \in Q$. For example, if $a \in \Sigma$ has defect one with $\delta(p, a) = \delta(q, a)$ for distinct $p, q \in Q$, then a distinguishes $\{p, q\}$ from every other set of size at least two.

Theorem 6.88, Item 4 gives the following lower bound for the size of every automaton accepting $\mathrm{Syn}(\mathcal{A})$.

Theorem 9.10 *Let $\mathcal{A}$ be a synchronizing automaton. Suppose there exist m reachable and pairwise distinguishable subsets of states. Then every automaton for $\mathrm{Syn}(\mathcal{A})$ has at least m states. In fact, the size of a minimal automaton for $\mathrm{Syn}(\mathcal{A})$ equals the size of a largest family of pairwise distinguishable and reachable subsets of states.*

Note that $\mathcal{A}$ is synchronizing if and only if at least one singleton subset of states is reachable.

Remark 9.11 Clearly, the set of synchronizing words equals $\bigcup_{q\in Q}\bigcap_{p\in Q} L(\mathcal{A}_{p,\{q\}})$. As shown above, this set can be accepted by the power automaton.

Other possibilities to describe this set are, for example, with an "inverse power automaton" with state set $\{\delta^{-1}(q,u) \mid q \in Q, u \in \Sigma^*\}\cup\{Q\}$ and transition function that maps $\delta^{-1}(q,u)$ to $\delta^{-1}(q,ua)$ for $a \in \Sigma$. Then it equals the union of all languages that go from some state of the form $\{q\}$ to Q. Another possibility derives from Theorem 7.16. By this statement, it also equals the set of synchronizing words of the pair automaton $\mathcal{A}^{\leq 2}/{\sim_F}$, and since it has F as a trap state, this language equals the intersection of all languages accepted with some start state of the form $\{p,q\}$, $p \neq q$, and ending at F in the pair automaton. Both constructions can be seen as "multiple-entry acceptor automata" where we allow a set of starting states rather than a single starting state.

However, for our purposes, the automaton obtained from the power automaton by merging singleton state sets is the most direct approach, as the notions of reachability and distinguishability directly arise from considering this automaton. Additionally, as we will see later, this automaton sometimes even leads to a minimal automaton.

9.2 Reachable Subsets

Here, in Section 9.2.1, we introduce completely reachable automata. This section contains only results from the literature and no new results. Section 9.2.2 investigates automata in which all k-sets are reachable for some $k \geq 1$. We give a sufficient condition for synchronizability in terms of reachability of k-sets and state structural limitations in n-state automata where all $(n-1)$-sets are reachable or where all $(n-2)$-set are reachable. Lastly, in Section 9.2.3 we investigate how the reachability of all k-sets for some $k \geq 1$ affects the possible upper sets, sink components, and the number of states.

9.2.1 Completely Reachable Automata

Bondar and Volkov [99, 101] called $\mathcal{A} = (Q, \Sigma, \delta)$ *completely reachable* if every nonempty subset $S \subseteq Q$ is reachable, i.e., there exists a word $u \in \Sigma^*$ such that $\delta(Q,u) = S$. Equivalently, in the power automaton with the empty set removed, Q is the unique minimum element for the reachability quasiorder.

Completely reachable automata are strongly connected and synchronizing.

Lemma 9.12 (Bondar & Volkov [99, 101]) *If $\mathcal{A} = (Q, \Sigma, \delta)$ is completely reachable, then $\mathcal{A}$ is synchronizing and strongly connected.*

Proof. Let $p, q \in Q$. Then $\delta(Q, u) = \{q\}$ for some word $u \in \Sigma^*$. Thus $u \in \mathrm{Syn}(\mathcal{A})$ and since $\delta(p, u) = q$ we also conclude that $\mathcal{A}$ is strongly connected. □

In a completely reachable automaton there exists for each non-empty subset a word mapping the whole state set to this subset. So, its transformation monoid has size at least $2^n - 1$.

Next we show that the automata from the Černý family (Section 1.4 and Figure 1.7) are completely reachable for each $n \geq 1$. This fact also follows from Theorem 12.31, a more general result stated later, but it might be instructive to have a concrete example together with a self-contained proof.

Theorem 9.13 *The automata C_n from the Černý family (Section 1.4 & Figure 1.7) are completely reachable.*

Proof. Let $C_n = ([n], \{a, b\}, \delta_n)$. The whole state set is reachable by the empty word. Next, consider a non-empty and proper set $S \subseteq Q$. We have $0 \notin \delta_n(S, b^i)$ and $1 \in \delta_n(S, b^i)$ for some $0 \leq i < n$. Inductively, there exists $u \in \{a, b\}^*$ such that $\delta_n(S, b^i) \cup \{0\} = \delta_n(Q, u)$. Then we have $\delta(Q, uab^{n-i}) = S$. □

9.2.2 Reachability of k-Sets

Here, we investigate structural limitations and implications that result from the reachability of k-sets for some fixed $k \geq 1$. Theorem 9.14, gives a sufficient condition of synchronizability in terms of the reachability of k-sets. Then, Theorem 9.17 relates reachability to the group of units. From this lemma we derive results relating the reachability of classes of subsets to properties of the group of units. Specifically, Theorem 9.18 relates the orbits of the group of units to letters of defect one, Theorem 9.22 and Theorem 9.23 give conditions for transitivity and k-homogeneity, respectively. Lastly, in Theorem 9.24 we make a statement on the orbits of 2-sets for the group of units in SI-automata with a single simple idempotent letter and where all 2-sets are reachable.

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is synchronizing if and only if there exist $q \in Q$ and $u, v \in \Sigma^*$ such that

$$\delta(Q, v) \subseteq \delta^{-1}(q, u),$$

i.e., we can reach a subset in $\delta^{-1}(q, u)$ for some $q \in Q$ and $u \in \Sigma^*$. Next, using this observation, we derive a sufficient condition for synchronizability in terms of reachability of k-sets.

Proposition 9.14 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton and $k \geq 2$. If $n > k(k-1)$ and every k-subset is reachable in $\mathcal{A}$, then $\mathcal{A}$ is synchronizing.*

Proof. Let $A \subseteq Q$ be some k-set. Then there exists $u \in \Sigma^*$ such that $\delta(Q, u) = A$. The k sets $\delta^{-1}(q, u)$ for $q \in A$ partition Q. As $n > k(k-1)$ and $k = |A|$, by the pigeonhole principle, we have $|\delta^{-1}(q, u)| \geq k$ for some $q \in A$. Then let $B \subseteq \delta^{-1}(q, u)$ be some k-set. By assumption there exists $v \in \Sigma^*$ with $\delta(Q, v) = B$. So $\delta(Q, vu) = \delta(B, u) = \{q\}$. □

Let $k \geq 2$. We do not know whether the bound from Theorem 9.14 is optimal, i.e., whether there exist non-synchronizing automata with $k^2 - k$ states where every k-set is reachable. Next, we give a non-synchronizing automaton with $2k - 2$ states where every k-set is reachable.

Example 9.15 Suppose $k \geq 2$ and $m = k - 1$. Let $C_m = ([m], \{a, b\}, \delta_m)$ and $C'_m = (\{0', 1', \ldots, (m-1)'\}, \{a', b'\}, \delta'_m\})$ be two disjoint copies of the Černý automata from Figure 1.7. Consider them as one automaton over $\{a, b, a', b'\}$ with the $2m$ states $[m] \cup \{0', 1', \ldots, m'\}$ and where the letters a, b and a', b' induce self-loops in $\{0', 1,', \ldots, (m-1)'\}$ and $[m]$, respectively. If A is a k-set, then $A \cap [m] \neq \emptyset$ and $A \cap \{0', 1', \ldots, (m-1)'\} \neq \emptyset$. So, as the Černý automata are completely reachable by Theorem 9.13, there exist $u \in \{a, b\}^*$ and $v \in \{a', b'\}^*$ such that $A \cap [m] = \delta_m([m], u)$ and $A \cap \{0', 1', \ldots, (m-1)'\} = \delta_m(\{0', 1', \ldots, (m-1)'\}, v)$. Then uv maps the whole states set onto A. Hence, every k-set is reachable, but this automaton is not synchronizing.

In the following, we state a few corollaries of Proposition 9.14.

Corollary 9.16 *Let $\mathcal{A} = (Q, \Sigma, \delta)$.*

1. *If $|Q| \geq 3$ and every 2-set of states is reachable in $\mathcal{A}$, then $\mathcal{A}$ is synchronizing.*[2]
2. *If $|Q| \geq 7$ and every 3-set of states is reachable, then $\mathcal{A}$ is synchronizing.*
3. *If $|Q| \geq 13$ and every 4-set of states is reachable, then $\mathcal{A}$ is synchronizing.*

The next observation will be used to derive results on the group of units under the assumption of reachability of all k-sets for some k.

Lemma 9.17 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and G be the group of units of $\mathcal{T}_M(\mathcal{A})$. Suppose $A \subseteq Q$ is a proper non-empty subset of states. Then the following are equivalent.*

1. *A is reachable in $\mathcal{A}$.*
2. *There exist $g \in G$, $u \in \Sigma^*$, and a non-permutational letter $a \in \Sigma$ such that*

$$\delta(Q, ua) = g(A). \tag{9.2}$$

3. *There exist $g \in G$, $u \in \Sigma^*$, and a non-permutational letter $a \in \Sigma$ such that $g(Q \setminus A) = \mathrm{excl}(ua)$.*

[2] This case also follows easily in the following way: Since $|Q| \geq 3$ there exists $a \in \Sigma$ such that $\delta(p, a) = \delta(q, a)$ for distinct $p, q \in Q$. By assumption, $\delta(Q, u) = \{p, q\}$ for some $u \in \Sigma^*$. Then we find $|\delta(Q, ua)| = 1$.

Proof. Set $\Gamma = \{a \in \Sigma \mid \operatorname{rank}(a) = |Q|\}$, i.e., the set of permutational letters.

$1 \Rightarrow 2$: As $1 \leq |A| < |Q|$, a word w mapping Q onto A must contain a letter from $\Sigma \setminus \Gamma$. By considering the right-most occurrence of a letter from $\Sigma \setminus \Gamma$, we obtain the following equivalences:

$$\begin{aligned}\exists w \in \Sigma^* \colon \delta(Q, w) = A &\Leftrightarrow \exists u \in \Sigma^* \; \exists a \in \Sigma \setminus \Gamma \; \exists v \in \Gamma^* \colon \delta(Q, uav) = A \\ &\Leftrightarrow \exists u \in \Sigma^* \; \exists a \in \Sigma \setminus \Gamma \; \exists v \in \Gamma^* \colon \delta(Q, ua) = \delta_v^{-1}(A).\end{aligned}$$

By Theorem 6.2 we have $G = \{\delta_v \mid v \in \Gamma^*\}$, which gives 2.

$2 \Leftrightarrow 3$: As g is a permutation, we have $Q \setminus g(A) = g(Q \setminus A)$. So, by complementation we find that Equation (9.2) is equivalent to $g(Q \setminus A) = \operatorname{excl}(ua)$.

$2 \Rightarrow 1$: By Theorem 6.2 there exists $v \in \Gamma^*$ such that $g^{-1} = \delta_v$. Then Equation (9.2) yields $\delta(Q, uav) = A$. □

The next result relates the reachability of sets missing a single state to orbits of the group of units.

Proposition 9.18 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton and G be the group of units of $\mathcal{T}_M(\mathcal{A})$. Then every subset of size $|Q| - 1$ is reachable in $\mathcal{A}$ if and only if*

$$Q = \bigcup_{\substack{p \in Q \\ \exists a \in \Sigma \colon \operatorname{excl}(a) = \{p\}}} \operatorname{Orb}_G(p).$$

Proof. By Theorem 9.17, Item 3, every subset of the form $Q \setminus \{q\}$ for $q \in Q$ is reachable in $\mathcal{A}$ if and only if for each $q \in Q$ there exists $g \in G$ such that $\{q\} = g(\operatorname{excl}(ua))$ for some $u \in \Sigma^*$ and non-permutational letter $a \in \Sigma$. Now, as $\operatorname{excl}(a) \subseteq \operatorname{excl}(ua)$, $|\operatorname{excl}(ua)| = 1$ and $|\operatorname{excl}(a)| \geq 1$, we find $\operatorname{excl}(ua) = \operatorname{excl}(a)$. Hence, reachability of every subset of the form $Q \setminus \{q\}$, $q \in Q$, is equivalent to the fact that, for each $q \in Q$, there exist $g \in G$ and $a \in \Sigma$ of defect one such that $\{q\} = g(\operatorname{excl}(a))$. □

Corollary 9.19 *In an n-state automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with a single non-permutational letter, every $(n-1)$-set of states is reachable if and only if the non-permutational letter has defect one and its group of units is transitive (and so $|\Sigma| \geq 2$).*

Proof. Let $a \in \Sigma$ be the single non-permutational letter. By Theorem 9.18, every $(n-1)$-set is reachable if and only if $Q = \operatorname{Orb}_G(p)$ for some $p \in Q$ with $\operatorname{excl}(a) = \{p\}$, giving the claim. □

Corollary 9.20 *In an n-state automaton over Σ with $n \geq 3$ in which every $(n-1)$-set is reachable, we have $|\Sigma| \geq 2$, one letter has defect one, and for $|\Sigma| = 2$ the other letter cyclically permutes the states.*

Proof. At least one letter has to have defect one. Then $|\Sigma| \geq 2$, as otherwise only a single $(n-1)$-set would be reachable. For $|\Sigma| = 2$, by Theorem 9.18, as $n \geq 3$, the group of units has to be non-trivial. So, the there exists a permutational letter. As a single permutational letter generates a transitive group if and only if it cyclically permutes its domain, the claim follows from Theorem 9.19. □

In the two-state automaton $\mathcal{A} = (\{q_1, q_2\}, \{a, b\}, \delta)$ with $\delta(\{q_1, q_2\}, a) = \{q_1\}$ and $\delta(\{q_1, q_2\}, b) = \{q_2\}$ all 1-sets are reachable, but $\mathcal{A}$ has no permutational letter. Hence, Theorem 9.20 does not hold for two-state automata.

Corollary 9.21 *A binary completely reachable automaton with at least three states has a letter cyclically permuting the states and a letter of defect one.*

Proof. This directly follows from Theorem 9.20. □

Next, we state a consequence for n-state automata having only a single non-permutational letter and in which all $(n-2)$-sets are reachable.

Proposition 9.22 *Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is an automaton having a single non-permutational letter $a \in \Sigma$ in which every $(|Q|-2)$-set is reachable and where G is the group of units of $\mathcal{T}_M(\mathcal{A})$. Then a has defect one or two, where if a has defect two, then G is 2-homogeneous, and if a has defect one and G has no fixpoint, then G is transitive.*

Proof. The letter a must have defect one or two, since otherwise no set of size greater than $|Q| - 3$ would be reachable. Let $p \in \mathrm{excl}(a)$ and $\{s, t\} \subseteq Q$ be a 2-set. By Theorem 9.17, Item 3, applied to $Q \setminus \{s, t\}$, there exists $g \in G$ and $u \in \Sigma^*$ such that $g(\{s, t\}) = \mathrm{excl}(ua)$. We have $\mathrm{excl}(a) \subseteq \mathrm{excl}(ua)$. So, $\mathrm{excl}(ua) = \{p, q\}$ for some $q \in Q$. Then $\{s, t\} = g^{-1}(\{p, q\})$. If a has defect two, then we must have $\mathrm{excl}(a) = \{p, q\}$, i.e., q is independent of u. So, as $\{s, t\}$ was chosen arbitrarily, G is 2-homogeneous. Otherwise, if a has defect one, then $\mathrm{excl}(a) = \{p\}$. By Item 1 the orbit of p in G has size at least $|Q| - 1$. If it has precisely size $|Q| - 1$, then the single state outside of it is a fixpoint of G, which cannot be the case by assumption. So, it has size $|Q|$, which means that G is transitive. □

Next, we give a characterization for k-homogeneity of the group of units that generalizes Theorem 9.19.

Proposition 9.23 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $1 \leq k < |Q|$ be such that for all non-permutational letters $a, b \in \Sigma$ we have $\delta(Q, a) = \delta(Q, b)$ and $\mathrm{rank}(a) = k$. Then all k-sets are reachable in $\mathcal{A}$ if and only if the group of units $\mathcal{T}_M(\mathcal{A})$ is k-homogeneous.*

Proof. Clearly, if the group of units is k-homogeneous, then all k-sets are reachable from $\delta(Q, a)$. Conversely, let $A \subseteq Q$ be some k-set. By assumption and Theorem 9.17, Item 2, there exist $u \in \Sigma^*$, a non-permutational letter $a \in \Sigma$ and an element g from the group of units of $\mathcal{T}_M(\mathcal{A})$ such that $g^{-1}(\delta(Q, ua)) = A$. As $|\delta(Q, ua)| = |\delta(Q, a)|$ and $\delta(Q, ua) \subseteq \delta(Q, a)$ we get $\delta(Q, ua) = \delta(Q, a)$. Now, the set $\delta(Q, a)$ is the same for each letter a of rank k. As this set can be mapped to any other k-set by an element from the group of units, G is k-homogeneous. □

Proposition 9.24 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton with a single simple idempotent letter $a \in \Sigma$ where $\delta(q, a) \neq q$ for $q \in Q$. Suppose all 2-sets are reachable in $\mathcal{A}$. Then for every 2-set $\{q_1, q_2\}$ there exist $p \in Q \setminus \{q, \delta(p, a)\}$ and a permutational word $u \in \Sigma^*$ such that*

$$\delta(\{p, \delta(q, a)\}, u) = \{q_1, q_2\}.$$

Thus the group of units G of $\mathcal{T}_S(\mathcal{A})$ has at most $|Q| - 2$ orbits on the 2-sets.

Proof. By Theorem 9.17 we have $\delta(Q, va) = g(\{q_1, q_2\})$ for some $g \in G$ and $v \in \Sigma^*$. This implies $|\delta(Q, va)| = 2$. When choosing va of minimal length, we can assume $\delta(Q, v) \neq \delta(Q, va)$. Since q is the only state moved by a, this implies $q \in \delta(Q, v)$ and so $\delta(q, a) \in \delta(Q, va)$. Furthermore, since $q \notin \delta(Q, a)$, we have $q \notin \delta(Q, va)$. Thus we obtain the first part of the claim by picking u such that $g = \delta_u$. Since there are at most $|Q| - 2$ sets of size two and of the form $\{p, \delta(q, a)\}$ with $p \neq q$, we get the rest of the claim. □

9.2.3 Upper Sets and the Reachability Order

Here, we investigate how the reachability of subsets relates to the number of sink components. Recall that, as stated in Theorem 3.48, the number of different sink components equals the maximal number of pairwise incomparable states and the maximal number of pairwise disjoint upper sets.

Theorem 9.25, is a consequence of Equation (6.2). Theorem 9.26 gives various bounds in terms of the number of sink components for automata where every k-set of states is reachable.

Proposition 9.25 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Suppose $S \subseteq Q$ is an upper set containing m different sink components. Then $|\delta(Q, u) \cap S| \geq m$ for all $u \in \Sigma^*$.*

Proof. Let $u \in \Sigma^*$. By Equation (6.2), for a sink component $R \subseteq Q$ we have $\delta(Q, u) \cap R \neq \emptyset$. Let T be the union of the sink components in S. As distinct sink components are disjoint, we have $|\delta(Q, u) \cap T| \geq m$. Hence $|\delta(Q, u) \cap S| \geq |\delta(Q, u) \cap T| \geq m$. □

Proposition 9.26 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with m sink components. Suppose every k-set of states is reachable for some $k \geq 1$. Then we have the following:*

1. *$k \geq m$.*
2. *Every upper set of size at least k contains all sink components.*
3. *$|Q \setminus S| \leq k - r$ for an upper set $S \subseteq Q$ that contains r sink components.*
4. *For $m \geq 2$ we have $|R| \leq k - m + 1$ for each upper set R containing precisely one sink component and for the whole state set we have $|Q| \leq 2k - m$.*[3]

Proof. 1: Let $\delta(Q, u)$ be a k-set for some $u \in \Sigma^*$. By Equation (6.2), $\delta(Q, u)$ contains a state from each sink component. Thus $k = |\delta(Q, u)| \geq m$.

2: By assumption, we can pick a k-set $A \subseteq S$. We have $A = \delta(Q, u)$ for some $u \in \Sigma^*$. By Equation (6.2), each sink component has non-empty intersection with A, and thus with S. Since S is an upper set, this yields that each sink component is contained in S.

3: Assume $|Q \setminus S| > k - r$. Then we find a k-set $A \subseteq Q$ such that A contains at least $k - r + 1$ states not in S. This gives $|A \cap S| < r$. By assumption, $A = \delta(Q, u)$ for some $u \in \Sigma^*$. By Theorem 9.25, we get $|A \cap S| \geq r$, contradicting $|A \cap S| < r$.

4: First, suppose R contains precisely one sink component. Let K be the union of the $m - 1$ sink components outside of R. By Item 3 we have $|Q \setminus K| \leq k - (m - 1)$. Then

[3] Observe that $1 \leq |R| \leq k - m + 1$ also yields $m \leq k$.

the first claim follows since $R \subseteq Q \setminus K$. Next, let S be a sink component. By Item 3 we have $|Q \setminus S| \leq k-1$. Applying the already proven part with S in the role of R, we find $|S| \leq k-m+1$. Thus $|Q| \leq 2k-m$. □

By Theorem 9.14 an automaton in which every k-sets is reachable with more than $k(k-1)$ states is synchronizing, and so, by Theorem 7.13, has a unique sink component. This gives the bound $k(k-1)$ for the number of states of an automaton in which every k-set is reachable and with two distinct sink components. The bound $2(k-1)$ given above is better for $k > 2$.

Corollary 9.27 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton in which every k-set is reachable for some $k \geq 1$.*

1. *If $|Q| > 2k-2$, then $\mathcal{A}$ has a unique sink component.*
2. *If $|S| \geq k$ for some sink component $S \subseteq Q$, then $\mathcal{A}$ has a unique sink component.*

Proof. 1: This follows readily from Theorem 9.26, Item 4.

2: If $\mathcal{A}$ has more than one sink component, then $|S| \leq k-1$ by Theorem 9.26, Item 4. This gives the claim. □

The next example shows that the bounds from Theorem 9.26, Item 4 are optimal (note that such an example cannot be a synchronizing automaton, since synchronizing automata have a unique sink component).

Example 9.28 Consider the automaton from Example 9.15. It has $2k-2$ states and every k-set is reachable. Furthermore, the sink components are the state sets $[m]$ and $\{0', 1', \ldots, (m-1)'\}$ of size $k-1$. By changing the sizes of the two copies of the Černý automata, i.e., having them different sizes, we can construct automata with any number of states less than $2k-2$ where every k-set of states is reachable but which have two sink components.

9.3 Pairs of Distinguishable Subsets

In Section 9.3.1 we give sufficient conditions for synchronizability in terms of different families of pairwise distinguishable subsets of states.

Section 9.3.2 investigates families of subsets and their relations w.r.t. distinguishability.

We show in Section 9.3.3 that the set of distinguishing words of two subsets A, B of states of a given n-state automaton is a regular language acceptable by an automaton of size $O(n^{|A|+|B|})$, which also yields a bound on the length of shortest distinguishing words.

Section 9.3.4 introduces completely distinguishable automata. We show that completely distinguishable automata with at least three states are synchronizing. Then, we give several equivalent characterizations of this new class of automata.

Lastly, Section 9.3.5 gives examples of automata where all pairs of distinct 2-sets are distinguishable, but that are not completely distinguishable.

9.3.1 Distinguishability and Synchronizability

In Theorem 9.30 we prove, using Theorem 9.29, that distinguishability of pairs of k-sets in automata with at least $k+1$ states implies synchronizability for $k \geq 3$, and for $k = 2$ existence of a word of rank at most two. Then we list several consequences, for example, that a strongly connected automaton in which pairs of 2-sets are distinguishable is synchronizing. Theorems 9.33 and 9.34 give non-synchronizing automata where all pairs of 2-sets are distinguishable. Lastly, in Theorem 9.35, for automata whose transformation monoid contains a transitive group, we give a sufficient condition for synchronizability in terms of the distinguishability of certain disjoint sets.

Lemma 9.29 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Suppose for all distinct states $p, q \in Q$ there exist distinguishable subsets $A, B \subseteq Q$ with $\{p, q\} \subseteq A \cap B$. Then $\mathcal{A}$ is synchronizing.*

Proof. A word $u \in \Sigma^*$ that distinguishes A from B maps at least one subset to a singleton subset, which implies $\delta(p, u) = \delta(q, u)$. As this holds for any pair of distinct states $p, q \in Q$, by Theorem 7.18 the automaton $\mathcal{A}$ is synchronizing. □

Proposition 9.30 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $2 \leq k, l \leq |Q|$ with $|Q| \geq k+1$ or $|Q| \geq l+1$. Suppose each k-set is distinguishable from each distinct l-set of states in $\mathcal{A}$ and let u be a word of minimal rank r. For $k \geq 3$ or $l \geq 3$, we have $r = 1$, i.e., $\mathcal{A}$ is synchronizing. For $k = l = 2$, we have $r \leq 2$ where for $r = 2$ the set $\delta(Q, u)$ is permuted by each letter, and so a union of sink components by Theorem 6.6.*

Proof. Without loss of generality, let $k \leq l$. First, suppose $l \geq 3$. Let $p, q \in Q$ be distinct. By assumption, there exist distinct $p_3, \ldots, p_k, p_{k+1} \in Q \setminus \{p, q\}$. Set $A = \{p, q, p_3, \ldots, p_k\}$. Next, there also exists an l-subset $B \subseteq Q$ with $\{p, q, p_{k+1}\} \subseteq B$. Since $p_{k+1} \notin A$ we have $A \neq B$. By assumption, A and B are distinguishable. The, from Theorem 9.29 we conclude that $r = 1$.

Next, suppose $k = l = 2$. Assume $r \geq 3$. Then we can pick three distinct states $p, q, s \in \delta(Q, u)$. By assumption, the sets $\{p, q\}$ and $\{p, s\}$ are distinguishable by some word $v \in \Sigma^*$. Since one of both sets is mapped to a singleton, we find $|\delta(Q, uv)| < |\delta(Q, u)|$, contradicting the choice of u as a word of minimal rank. So, we have $r \leq 2$. Now, suppose $r = 2$. Assume there exists $a \in \Sigma$ such that $\delta(\delta(Q, u), a) \neq \delta(Q, u)$. Since r is the minimal possible rank, $\delta(\delta(Q, u), a)$ is a 2-set. Then, by assumption, there exists a word v distinguishing $\delta(\delta(Q, u), a)$ and $\delta(Q, u)$. However, then one of both subsets is mapped to a singleton, which is not possible since the minimal possible rank of a word is 2. Hence, $\delta(\delta(Q, u), a) = \delta(Q, u)$ for each $a \in \Sigma$. □

Next, we list a few consequences of Theorems 9.29 and 9.30.

Corollary 9.31 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton and $2 \leq k < n$.*

1. *If $n \geq 4$ and all pairs of distinct 3-sets are distinguishable in $\mathcal{A}$, then $\mathcal{A}$ is synchronizing.*
2. *If for every k-set $A \subseteq Q$ there exists $B \subseteq Q$ with $A \subseteq B$ such that A and B are distinguishable, then $\mathcal{A}$ is synchronizing.*

3. *Let* $0 < l \leq n - k$. *If every* k*-set is distinguishable from every* $(k + l)$*-set, then* $\mathcal{A}$ *is synchronizing.*
4. *If every* k*-set is distinguishable from the whole state set, then the automaton is synchronizing.*

Proof. 1: This follows from Theorem 9.30 with $k = 3$.

2: For distinct $p, q \in Q$ choose some k-set with $p, q \in A$. By assumption, there exists $B \subseteq Q$ containing A so that A and B are distinguishable. Then Theorem 9.29 gives the claim.

3 and 4 clearly follow from 2. □

Corollary 9.32 *Suppose* $\mathcal{A} = (Q, \Sigma, \delta)$ *is an automaton with at least three states in which all pairs of distinct* 2*-sets are distinguishable.*

1. *If for distinct* $p, q \in Q$ *we have* $\delta(\{p, q\}, a) \neq \{p, q\}$ *for some* $a \in \Sigma$, *then* $\mathcal{A}$ *is synchronizing.*
2. *If* $\mathcal{A}$ *is strongly connected, then it is synchronizing.*
3. *Either* $\mathcal{A}$ *has a unique sink component and is synchronizing, or it has precisely two sink components each of size one.*

Proof. 1: Let $u \in \Sigma^*$ be a word of minimal possible rank. By Theorem 9.30 we have $|\delta(Q, u)| \leq 2$ where $|\delta(Q, u)| = 2$ is not possible since otherwise every letter would permute this set by Theorem 9.30.

2: Let u be a word of minimal rank and assume $|\delta(Q, u)| \geq 2$. By Theorem 9.30, $\delta(Q, u)$ has size two and equals a union of sink components. Since $\mathcal{A}$ is strongly connected, Q is its only sink component and we find $\delta(Q, u) = Q$, which is not possible since $|Q| \geq 3$.

3: By Theorem 6.4, $\mathcal{A}$ has at most two sink components. First, if it has a unique sink component $S \subseteq Q$, then $\mathcal{A}_{|S}$ is strongly connected and all pairs of distinct 2-sets from S are distinguishable. Thus, by Item 2, $\mathcal{A}_{|S}$ is synchronizing. Since the set of maximal states equals the union of all sink components by Theorem 3.53, S equals the set of maximal states and so $\mathcal{A}$ is synchronizing by Theorem 7.11. Next, suppose $\mathcal{A}$ has two sink components. Then it is not synchronizing by Theorem 6.4. By Theorem 9.30, $\delta(Q, u)$ for a word of minimal rank u equals a union of sink components and has size two. Since this set has non-empty intersection with both sink components by Equation (6.2), both sink components must have size one. □

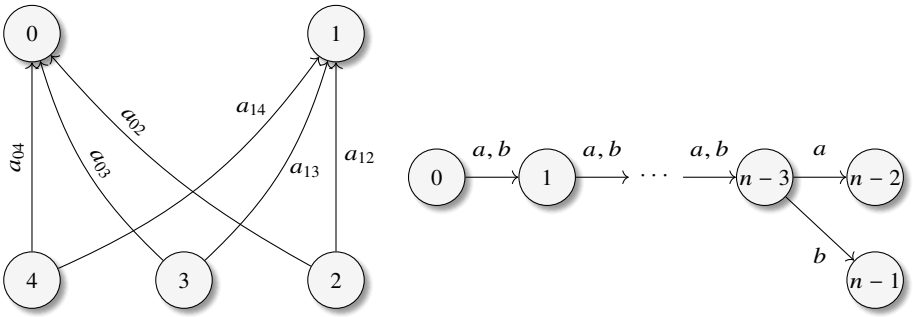
Next, Theorems 9.33 and 9.34 give infinite families of non-synchronizing automata where all pairs of 2-sets are distinguishable, demonstrating that the case $k = 2$ in Theorem 9.30 is special in the sense that it does not imply synchronizability similar to the case $k > 2$. By adding in these examples another letter that swaps the two maximal states and fixes every other state, we obtain non-synchronizing automata with a unique sink component where all pairs of 2-sets are distinguishable.

Example 9.33 Set $\mathcal{A}_n = ([n], \Sigma, \delta)$ for $n \geq 3$ where $\Sigma = \{a_{ij} \mid i \in \{0, 1\}, j \in \{2, \ldots, n-1\}\}$ and, for $i \in \{0, 1\}$, $j \in \{2, \ldots, n-1\}$, we have

$$\delta(k, a_{ij}) = \begin{cases} i & \text{if } k = j, \\ k & \text{otherwise,} \end{cases}$$

for $k \in [n]$. See Figure 9.4a for $\mathcal{A}_3$. Now, consider two distinct 2-sets $\{p, q\}$ and $\{s, t\}$ of states. First, without loss of generality, suppose $\{p, q\} \subseteq \{2, \ldots, n-1\}$. Then $\delta(p, u) = \delta(q, u) = 0$ and $|\delta(\{s, t\}, u)| = 2$ for $u = a_{0p}a_{0q}$. Otherwise, $\{0, 1\}$ has non-empty intersection with both 2-sets where we assume, without loss of generality, $p = 0$ and $q \neq 1$. Then, $\delta(p, a_{0q}) = \delta(q, a_{0q}) = 0$ and $|\delta(\{s, t\}, a_{0q})| = 2$.

Example 9.34 Let $\mathcal{A}_n = (Q_n, \{a, b\}, \delta_n)$ be the automaton given by the state diagram in Figure 9.4b where $n \geq 3$. Consider two distinct sets of states $\{p_1, p_2\}, \{q_1, q_2\}$ with $p_1 < p_2$ and $q_1 < q_2$. First, without loss of generality, suppose $p_1 < q_1$. In this case, for $q_2 \leq n-2$, we have $\delta(\{q_1, q_2\}, a^{n-2-q_1}) = \{n-2\}$ and $|\delta(\{p_1, p_2\}, a^{n-2-q_1})| = 2$. For $q_2 = n-1$ and $q_1 \neq n-2$, we have $\delta(\{q_1, q_2\}, b^{n-2-q_1}) = \{n-1\}$ and $|\delta(\{p_1, p_2\}, b^{n-2-q_1})| = 2$. For $q_2 = n-1$ and $q_1 = n-2$, if $p_2 = n-1$ the word b^n distinguishes both subsets, and if $p_2 \leq n-2$ the word a^n distinguishes both subsets. Next, suppose $p_1 = q_1$. In this case, without loss of generality, assume $p_2 < q_2$. For $q_2 = n-1$, we have $\delta(\{p_1, p_2\}, a^n) = \{n-2\}$ and $\delta(\{q_1, q_2\}, a^n) = \{n-1, n-2\}$. Otherwise, for $q_2 < n-1$, we have $\delta(\{p_1, p_2\}, u) = \{n-2\}$ and $\delta(\{q_1, q_2\}, u) = \{n-1, n-2\}$ for $u = a^{n-3-q_2}ba^n$.



(a) Automaton as in Theorem 9.33. (b) Binary automaton as in Theorem 9.34.

Fig. 9.4: Two non-synchronizing automata where all pairs of distinct 2-sets are distinguishable, self-loops omitted.

Lastly, we consider the case that the transformation monoid contains a transitive group. In this case, we state a sufficient condition for synchronizability in terms of the distinguishability of certain disjoint subsets. We need this result later in a more algebraic context. For this reason, we give a formulation in terms of transformation monoids without reference to automata.

Proposition 9.35 *Let $M \leq_M \mathcal{T}_n$ be a transformation monoid with a transitive group of units and $2 \leq k, l < n$ such that $kl < n$. If every k-set is distinguishable from every disjoint l-set, then M contains a constant map.*

Proof. Without loss of generality, assume $k \leq l$. Let $p, q \in [n]$ be two distinct states. Pick $A, B \subseteq [n]$ with $k = |A|, l = |B|$ and $p, q \in A \cap B$ (note that the following argument includes the case $k = l = 2$ and $A = B = \{p, q\}$). As G is transitive, by Theorem 5.9 we have $g(A) \cap B = \emptyset$ for some $g \in G$. By assumption there exists $h \in M$ mapping precisely one of both subsets $g(A)$ or B to a singleton subset. If $|h(g(A))| = 1$, then, as $\{p, q\} \subseteq A$, the transformation hg maps $\{p, q\}$ to a singleton. If $|h(B)| = 1$, then, as $\{p, q\} \subseteq B$, the transformation h maps $\{p, q\}$ to a singleton. Using the correspondence between automata and transformation semigroups (see Section 6.8), the pair criterion (Theorem 7.18) implies that M contains a constant map as all pairs of states p, q can be mapped to the same state. □

9.3.2 Classes of Pairwise Distinguishable Subsets

Theorem 9.38 shows that distinguishability of k-sets for $k \geq 3$ in automata with at least $k+1$ states implies distinguishability of all $(k-1)$-sets. Then Theorem 9.39 investigates conditions equivalent to the distinguishability of $(k + 1)$-sets whose intersection has size k. Theorem 9.40 investigates conditions when we can map, from two distinct subsets, one specific set to a singleton, but not the other; a property that implies distinguishability, but, in general, is stronger since we can choose which subset is mapped to a singleton. Theorem 9.3 gave an automaton having indistinguishable pairs of disjoint 2-sets, but where all 2-sets are distinguishable from sets of size at least three (this also implies distinguishability of non-disjoint pairs of distinct 2-sets by Theorem 9.40). In Theorem 9.41 we give two automata in which all disjoint 2-sets are distinguishable, but not all non-disjoint 2-sets, and vice versa. Thus, both properties are logically independent. Our results use Theorems 9.36 and 9.37 stated first.

Lemma 9.36 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton and $A, B, C \subseteq Q$ with $A \cap B \neq \emptyset$. Then, for a word u with $\delta(A \cup C, u) = \{q\}$ for some $q \in Q$ and $|\delta(B \cup C, u)| \geq 2$ we have $|\delta(B, u)| \geq 2$.*

Proof. Since $A \cap B \neq \emptyset$ we find $q \in \delta(B \cup C, u)$. We have $q \neq \delta(p, u)$ for some $p \in B \cup C$. Since $\delta(C, u) \subseteq \{q\}$ we must have $p \in B$ and so $\{q, \delta(p, u)\} \subseteq \delta(B, u)$. □

Lemma 9.37 *Let $A, B \subseteq X$ and $p, q \in X$ such that $p \in A \setminus B$ and $q \in B \setminus A$. Then the following are equivalent.*

1. $A \setminus \{p\} = B \setminus \{q\}$.
2. $A \cup \{q\} = B \cup \{p\}$.
3. $A \setminus B = \{p\}$ *and* $B \setminus A = \{q\}$.
4. $A \cup B = A \cup \{q\}$ *and* $A \cap B = A \setminus \{p\}$.
5. $A = (A \cap B) \cup \{p\}$ *and* $B = (A \cap B) \cup \{q\}$.
6. $A = (B \setminus \{q\}) \cup \{p\}$.

Proof. 1 $\Rightarrow$ 2: With $A \setminus \{p\} = B \setminus \{q\}$ we find $A \cup \{q\} = (A \setminus \{p\}) \cup \{p, q\} = (B \setminus \{q\}) \cup \{p, q\} = B \cup \{p\}$.

$2 \Rightarrow 3$: Since $q \in B$ and $p \notin B$ we have $A \setminus B = (A \cup q) \setminus B = (B \cup \{p\}) \setminus B = \{p\}$. Similarly, we find $B \setminus A = \{q\}$.

$3 \Rightarrow 4$: This follows since $A \cup B = A \cup (B \setminus A) = A \cup \{q\}$ and $A \cap B = A \setminus (A \setminus B) = A \setminus \{p\}$.

$4 \Rightarrow 5$: Since $A \cap B = A \setminus \{p\}$ we have $A = (A \setminus \{p\}) \cup \{p\} = (A \cap B) \cup \{p\}$. Since $B \subseteq A \cup \{q\}$ and $q \in B \setminus A$, we have $B \setminus A = \{q\}$. This gives $B = (A \cap B) \cup (B \setminus A) = (A \cap B) \cup \{q\}$.

$5 \Rightarrow 6$: Since $B = (A \cap B) \cup \{q\}$ and $q \notin A$ we have $B \setminus \{q\} = A \cap B$. Thus $A = (A \cap B) \cup \{p\} = (B \setminus \{q\}) \cup \{p\}$.

$6 \Rightarrow 1$: Since $p \notin B$, $A = (B \setminus \{q\}) \cup \{p\}$ implies $A \setminus \{p\} = B \setminus \{q\}$. □

Proposition 9.38 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with at least $k + 1$ states for $k \geq 3$ in which all pairs of distinct k-sets are distinguishable. Then pairs of distinct $(k-1)$-sets are distinguishable.*

Proof. Let $A, B \subseteq Q$ be two distinct $(k-1)$-sets. Then there exist $p \in A \setminus B$ and $q \in B \setminus A$.

First, suppose $A \cup \{q\} = B \cup \{p\}$. In this case, by Theorem 9.37, we find $A \cap B \neq \emptyset$ and $|A \cup B| = k$. Since $|Q| \geq k + 1$ there exists $s \in Q \setminus (A \cup B)$. Then $A \neq B$ gives $A \cup \{s\} \neq B \cup \{s\}$ and so, by assumption, there exists $u \in \Sigma^*$ distinguishing these two k-sets. By Theorem 9.36, u distinguishes also A and B.

Next, suppose $A \cup \{q\} \neq B \cup \{p\}$. By assumption, there exists a word u distinguishing $A \cup \{q\}$ and $B \cup \{p\}$. Without loss of generality, suppose $\delta(A \cup \{q\}, u) = \{s\}$ for some $s \in Q$. Thus $\delta(q, u) = \delta(p, u) = s$ and since $|\delta(B \cup \{p\}, u)| \geq 2$ we have $\delta(r, u) \neq s$ for some $r \in B \setminus \{q\}$. Now, $q \in B$ and $r \in B$ give $\{\delta(q, u), \delta(r, u)\} \subseteq \delta(B, u)$ and so $|\delta(B, u)| \geq 2$. □

In automata with at most k states, it is vacuously true that all pairs of distinct k-sets are distinguishable. Next, there exist automata (for example permutation automata) with k states for $k \geq 3$ where pairs of distinct $(k-1)$-sets are indistinguishable. Thus, we have to assume $|Q| \geq k + 1$ in Theorem 9.38. Also, since two singleton sets are indistinguishable, a similar statement as Theorem 9.38 is false for $k = 2$.

Proposition 9.39 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $|Q| \geq k + 2$ for $k \geq 2$. Then the following are equivalent.*

1. *All pairs of $(k+1)$-sets whose intersection has size k are distinguishable.*
2. *All pairs of $(l+1)$-sets whose intersection has size $1 \leq l \leq k$ are distinguishable.*
3. *For distinct $A, B \subseteq Q$ with $2 \leq |A| \leq k$ and $|B \setminus A| \geq 2$ we have $|\delta(A, u)| = 1$ and $|\delta(B, u)| \geq 2$ for some $u \in \Sigma^*$, and pairs of distinct 2-sets with non-empty intersection are distinguishable.*
4. *For a k-set A and 2-set B with $A \cap B = \emptyset$ there exists a word mapping A to a singleton and B to a 2-set, and all pairs of distinct 2-sets with non-empty intersection are distinguishable.*
5. *For distinct $p_1, \ldots, p_{k+2} \in Q$ there exists $u \in \Sigma^*$ such that $\delta(p_1, u) \neq \delta(p_{k+2}, u)$, and $\delta(p_1, u) = \cdots = \delta(p_{k+1})$ or $\delta(p_2, u) = \cdots = \delta(p_{k+2}, u)$.*

Proof. $1 \Rightarrow 2$: Let $A, B \subseteq Q$ be two $(l+1)$-sets with $|A \cap B| = l$ for some $1 \leq l \leq k$. Then $A = (A \cap B) \cup \{p\}$ and $B = (A \cap B) \cup \{q\}$ for some $p \in A \setminus B$ and some $q \in B \setminus A$. Thus we have $A \cup B = (A \cap B) \cup \{p, q\}$, which gives $|A \cup B| = l + 2$. Since $|Q| \geq k + 2$ we can pick a $(k - l)$-set $C \subseteq Q \setminus (A \cup B)$. Then $A \cup C$ and $B \cup C$ are two distinct $(k+1)$-sets whose intersection has size k. Since these sets are distinguishable by assumption, by Theorem 9.36 we find that A and B are distinguishable.

$2 \Rightarrow 3$: For $l = 1$ we find distinguishability of distinct 2-sets with non-empty intersection. Next, let $A, B \subseteq Q$ be as in the statement and $p, q \in B \setminus A$ be distinct. The sets $A \cup \{p\}$ and $A \cup \{q\}$ are distinct with intersection A of size at most k. So, by assumption, distinguishable by some word $u \in \Sigma^*$. This yields $|\delta(A, u)| = 1$ and $\delta(p, u) \neq \delta(q, u)$ and so $|\delta(B, u)| \geq 2$.

$3 \Rightarrow 4$: This is clear.

$4 \Rightarrow 5$: Let $p_1, \ldots, p_{k+2} \in Q$ be distinct. By considering $A = \{p_2, \ldots, p_{k+1}\}$ and $B = \{p_1, p_{k+2}\}$ we find that there exists $u \in \Sigma^*$ such that $\delta(p_2, u) = \cdots = \delta(p_{k+1}, u)$ and $\delta(p_1, u) \neq \delta(p_{k+2}, u)$. For $\delta(p_2, u) \in \delta(B, u)$ we obtain the claim. Otherwise, for $\delta(p_2, u) \notin \delta(B, u)$ we have distinct 2-sets $\{\delta(p_1, u), \delta(p_2, u)\}$ and $\{\delta(p_{k+2}, u), \delta(p_2, u)\}$, which are distinguishable by some word $v \in \Sigma^*$. In this case the word uv fulfills the claim.

$5 \Rightarrow 1$: Given two distinct $(k+1)$-sets $A, B \subseteq Q$ with $|A \cap B| = k$, we can apply the assumption to $A \cap B = \{p_2, \ldots, p_{k+1}\}$, $p_1 \in A \setminus (A \cap B)$, and $p_{k+2} \in B \setminus (A \cap B)$ and obtain that A and B are distinguishable. □

Proposition 9.40 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $|Q| > k \geq 2$. Then the following are equivalent.*

1. *Two distinct $A, B \subseteq Q$ with $2 \leq |A| \leq k$ and $A \cap B \neq \emptyset$ are distinguishable.*
2. *Two distinct $A, B \subseteq Q$ with $|A| = k$ and $A \cap B \neq \emptyset$ are distinguishable.*
3. *For a k-set $A \subseteq Q$ and $p \notin Q \setminus A$ the sets A and $A \cup \{p\}$ are distinguishable.*
4. *For distinct k-sets $A, B \subseteq Q$ with $A \cap B \neq \emptyset$ we have $|\delta(A, u)| = 1$ and $|\delta(B, u)| \geq 2$ for some $u \in \Sigma^*$.*
5. *For $A \subseteq Q$ with $2 \leq |A| \leq k$ and $p \notin Q \setminus A$ the sets A and $A \cup \{p\}$ are distinguishable.*
6. *For $A, B \subseteq Q$ with $|A|, |B| \geq 2$ and $B \not\subseteq A$ where $|A| \leq k$ and $|A| = k$ implies $A \cap B \neq \emptyset$ we have $|\delta(A, u)| = 1$ and $|\delta(B, u)| \geq 2$ for some $u \in \Sigma^*$.*

Proof. $1 \Rightarrow 2$: This is clear.

$2 \Rightarrow 3$: This follows from the assumption with $A \cup \{p\}$ in the role of B.

$3 \Rightarrow 4$: Since $|A| = |B|$ there exists $p \in B \setminus A$. Then a word distinguishing A and $A \cup \{p\}$ maps A to a singleton and B to a set of size at least two since $A \cap B \neq \emptyset$ and $p \in B$.

$4 \Rightarrow 5$: Since $|Q| > k$ there exists $C \subseteq Q \setminus \{p\}$ such that $A \cup C$ is a k-set. Set $D = ((A \cup C) \setminus \{q\}) \cup \{p\}$ for some $q \in A \cup C$. By assumption, there exists u such that $\delta(A \cup C, u) = \{s\}$ and $|\delta(D, u)| \geq 2$. Then we must have $\delta(D, u) = \{s, \delta(p, u)\}$. Since $A \cap D \neq \emptyset$, this gives $\delta(A \cup \{p\}, u) = \{s, \delta(p, u)\}$.

$5 \Rightarrow 6$: By assumption, there exist $p \in B \setminus A$ and $q \in B$ with $p \neq q$ such that $|A \cup \{q\}| \leq k$. We find u distinguishing $A \cup \{q\}$ and $A \cup \{q, p\}$. This word must

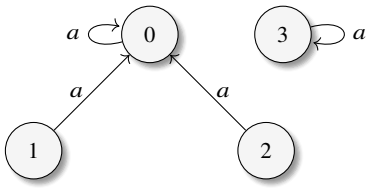
map $A \cup \{q\}$ to a singleton and p to a different state than those in $A \cup \{q\}$. Since $p, q \in B$, it maps B to a set of size at least two.

$6 \Rightarrow 1$: By assumption, there exists u mapping A to a singleton and $A \cup B$ to a set of size at least two. Since $A \cap B \neq \emptyset$, this word also maps B to a set of size at least two. □

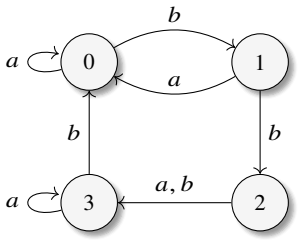
Note that Theorem 9.40, Item 3 implies that for non-disjoint 2-sets A, B there exist two distinguishing words, one mapping A to a singleton and another mapping B to a singleton, and for $k \geq 3$ by Theorem 9.40, Item 2, that for every pair of distinct 2-sets A, B there exist two distinguishing words, each mapping a different set to a singleton.

For the automaton from Theorem 9.3, the conditions from Theorem 9.40 hold for $k = 2$.

Example 9.41 Figure 9.5 gives two examples showing that neither distinguishability of disjoint pairs of 2-subsets nor distinguishable of distinct non-disjoints pairs of 2-subsets is sufficient for distinguishability of all pairs of 2-subsets. Figure 9.2 gives another example where non-disjoint pairs of 2-sets are distinguishable but disjoint pairs of 2-sets are indistinguishable.



(a) All disjoint pairs of 2-sets are distinguishable, but $\{0, 1\}$ is not distinguishable from $\{0, 2\}$.



(b) Pairs of distinct non-disjoint 2-sets are distinguishable and pairs of disjoint 2-sets are indistinguishable.

Fig. 9.5: Two automata showing that the properties of distinguishability of disjoint and non-disjoint 2-sets do not imply each other, see Theorem 9.41.

9.3.3 The Set of Words Distinguishing Two Sets and a Length Bound

Let A and B be two subsets of states in an n-state automaton. Here, we show that the set of words distinguishing A from B is acceptable by an automaton with $O(n^{|A|+|B|})$ states. As a consequence, a shortest distinguishing word has length in $O(n^{|A|+|B|})$.

Theorem 9.42 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton and $A, B \subseteq Q$ with $|A|, |B| \geq 2$. Then the set of words distinguishing A from B is regular and acceptable by an automaton with*

$$\left(n + \binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{|A|}\right) \cdot \left(n + \binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{|B|}\right) - n + 1$$

states. Furthermore, for a shortest distinguishing word $u \in \Sigma^*$ we have

$$|u| \leq \left(\binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{|A|}\right) \cdot \left(\binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{|B|}\right)$$

Proof. Let $\mathcal{B} = (Q', \Sigma, \mu, (A, B), E)$ where

$$\begin{aligned} Q' &= \{R \subseteq Q : 0 < |R| \leq |A|\} \times \{P \subseteq Q : 0 < |P| \leq |B|\}, \\ E &= \{(R, P) \in Q' \times Q' : |R| = 1 \text{ if and only if } |P| > 1\}, \\ \mu((R, P), x) &= (\delta(R, x), \delta(P, x)) \text{ for } (R, P) \in Q' \times Q' \text{ and } x \in \Sigma. \end{aligned}$$

Set $F = \{\{q\} \mid q \in Q\}$. Then $\mathcal{B}/{\sim_{F\times F}}$ with start state $\pi_{\sim_{F\times F}}((A, B))$ and final state set $\pi_{\sim_{F\times F}}(E)$ accepts the set of words distinguishing A from B. This automaton has the claimed number of states.

Next, let $u \in \Sigma^*$ be a shortest distinguishing word. We have $|u| > 0$ and thus we can write $u = wx$ for some $w \in \Sigma^*$ and $x \in \Sigma$. By the minimal choice and since u is a distinguishing word, for each proper prefix $v \in \Sigma^*$ of u, for $(R, P) = \mu(A, B, v)$ we have $(R, P) \notin E$ with $|R| \geq 2$ and $|P| \geq 2$. Thus, since we have

$$|\{(R, P) \in Q' \times Q' : |R| \geq 2, |P| \geq 2\}| = \left(\binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{|A|}\right) \cdot \left(\binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{|B|}\right)$$

and all states traverses by prefixes of w are distinct, we find

$$|w| \leq \left(\binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{|A|}\right) \cdot \left(\binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{|B|}\right) - 1$$

This gives the claimed bound for u. □

9.3.4 Completely (Subset-Pair) Distinguishable Automata

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called *completely subset-pair distinguishable* (or just *completely distinguishable* for short) if every two distinct non-empty and non-singleton subsets $A, B \subseteq Q$ are distinguishable in $\mathcal{A}$.

Recall that $L(\mathcal{P}(\mathcal{A})_{Q,F}) = \mathrm{Syn}(\mathcal{A})$ with $F = \{\{q\} \mid q \in Q\}$, see Section 9.1. By Theorem 6.88, Item 4, distinguishability of pairs of distinct states is a necessary condition for minimality. So, applying this to $\mathcal{P}(\mathcal{A})_{Q,F}$, complete distinguishability is a necessary condition for the minimal automaton of $\mathrm{Syn}(\mathcal{A})$ to have $2^{|Q|} - |Q|$ states.

First, in Theorem 9.43, as an example, we prove that the Černý automata from Figure 1.7 are completely distinguishable (this also follows from Theorem 12.31 given later, but as an example we give a self-contained proof here). Then, in Theorem 9.44, we show that completely distinguishable automata with at least three states are synchro-

nizing. Theorem 9.46 collects various equivalent characterizations, and Theorems 9.49 and 9.50 are two consequences of these characterizations. Theorems 9.51 to 9.53 give completely distinguishable automata that are not completely reachable. Theorem 9.55 proves that in binary completely distinguishable automata where one letter has rank two, the other letter cyclically permutes the states. Theorem 9.56 lists an equivalence for a set missing one state being distinguishable from the whole state set (a condition that appears in Theorem 9.46 for each set missing one state).

Theorem 9.43 *The Černý automata $C_n = ([n], \{a, b\}, \delta_n)$ from Figure 1.7 are completely distinguishable.*

Proof. For $i \in [n]$ set $u_i = b^{n-i}ab^i$. Then, for $j \in [n]$,

$$\delta_n(j, u_i) = \begin{cases} (j+1) \bmod n & \text{if } i = j, \\ j & \text{if } i \neq j. \end{cases} \tag{9.3}$$

Now, let $A, B \subseteq [n]$ be distinct with $|A|, |B| \geq 2$. Without loss of generality, suppose $i \in A \setminus B$. Set $u = u_{i+1} \cdots u_0 u_1 \cdots u_{i-1}$. Then $\delta(i, u) = i$, $\delta(B, u) \subseteq \delta([n] \setminus \{i\}, u) = \{i-1\}$ and $\delta(A, u) = \{i, i-1\}$. So, u distinguishes A from B. □

Theorem 9.31, Item 3 yields that completely distinguishable automata with at least three states are synchronizing.

Proposition 9.44 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a completely distinguishable automaton with $|Q| \geq 3$. Then $\mathcal{A}$ is synchronizing.*

Proof. This follows from Theorem 9.31, Item 3, since every 2-set is distinguishable from every 3-set. □

Since the empty set is distinguishable from every other set if and only if the automaton is synchronizing, by Theorem 9.44 an automaton with at least three states is completely distinguishable if every pair of non-singleton subsets is distinguishable.

Remark 9.45 An automaton with at most two states admits no pair of distinct non-empty and non-singleton subsets of states. Thus, it is completely distinguishable, even if it is non-synchronizing.

Next, we formulate various equivalent characterizations.

Theorem 9.46 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton with $n \geq 3$. The following are equivalent:*

1. *$\mathcal{A}$ is completely distinguishable.*
2. *For every $A, B \subseteq Q$ with $A \subsetneq B$ and A non-empty there exists $u \in \Sigma^*$ such that $1 = |\delta(A, u)| < |\delta(B, u)|$.*
3. *For every non-empty $A \subseteq Q$ and $q \in Q \setminus A$ there exists $u \in \Sigma^*$ such that $\delta(q, u) \notin \delta(A, u)$ and $|\delta(A, u)| = 1$.*
4. *For every $q \in Q$ there exists $u \in \Sigma^*$ such that $1 = |\delta(Q \setminus \{q\}, u)|$ and $\delta(q, u) \notin \delta(Q \setminus \{q\}, u)$, i.e., Q is distinguishable from every $(n-1)$-set.*

5. *For two distinct non-empty and non-singleton subsets there exists a word mapping one to a singleton and the other to a* 2*-set.*
6. *For distinct* $p, q, r \in Q$ *there exists* $u \in \Sigma^*$ *such that* $\delta(p,u) = \delta(q,u)$ *and*

$$\delta^{-1}(\{\delta(r,u)\},u) = \{r\},$$

   i.e., $\delta(s,u) \neq \delta(r,u)$ *for all* $s \in Q \setminus \{r\}$.
7. *For every* $A, B \subseteq Q$ *with* $\min\{|A|,|B|\} \geq 2$ *and* $A \subsetneq B$ *there exists* $u \in \Sigma^*$ *such that* $|\delta(A,u)| < |\delta(B,u)| < |B|$.
8. *For every* $A \subseteq Q$ *with* $|A| \geq 2$ *and* $p \in Q \setminus A$ *there exists* $u \in \Sigma^*$ *such that* $\delta(p,u) \notin \delta(A,u)$ *and* $|\delta(A,u)| < |A|$.
9. *For every distinct non-empty and non-singleton* $A, B \subseteq Q$ *with* $A \cap B \neq \emptyset$ *there exists* $u \in \Sigma^*$ *with* $3 \leq |\delta(A,u)| + |\delta(B,u)| < |A| + |B|$ *and* $\delta(A,u) \neq \delta(B,u)$.
10. *For every distinct non-empty and non-singleton* $A, B \subseteq Q$ *there exists* $u \in \Sigma^*$ *with* $3 \leq |\delta(A,u)| + |\delta(B,u)| < |A| + |B|$ *and* $\delta(A,u) \neq \delta(B,u)$.

Proof. $1 \Rightarrow 2$: By assumption there exists $u \in \Sigma^*$ distinguishing A from B. As $A \subsetneq B$ we must have $|\delta(A,u)| = 1$ and $|\delta(B,u)| > 1$.

$2 \Rightarrow 3$: For $A \subseteq Q$ and $q \in Q \setminus A$ the claim follows by applying Item 2 to A and $B = A \cup \{q\}$.

$3 \Rightarrow 4$: For $q \in Q$ the claim follows by applying Item 3 to $A = Q \setminus \{q\}$ and $q \in Q \setminus A$.

$4 \Rightarrow 5$: Let $A, B \subseteq Q$ be two distinct subsets of size at least two. Without loss of generality, suppose $q \in A \setminus B$. By assumption, there exists $u \in \Sigma^*$ such that $|\delta(Q \setminus \{q\},u)| = 1$ and $\delta(q,u) \notin \delta(Q \setminus \{q\})$. Since $B \subseteq Q \setminus \{q\}$ and A contains some state distinct from q, we have $|\delta(B,u)| = 1$ and $|\delta(A,u)| = 2$.

$5 \Rightarrow 6$: Let $p, q, r \in Q$ be distinct. Consider $Q \setminus \{r\}$ and Q. By assumption, there exists a word $u \in \Sigma^*$ mapping one to a singleton and the other to a 2-set, where we must have $|\delta(Q \setminus \{r\},u)| = 1$ and $|\delta(Q,u)| = 2$, which gives $\delta(r,u) \notin \delta(Q \setminus \{r\},u)$. Then as $p, q \in Q \setminus \{r\}$ we have $\delta(p,u) = \delta(q,u)$ and, as for $s \in Q \setminus \{r\}$ we have $\delta(s,u) = \delta(p,u)$, we have $\delta^{-1}(\{\delta(r,u)\},u) = \{s \in Q \mid \delta(s,u) = \delta(r,u)\} = \{r\}$, i.e., r is the only state mapped to $\delta(r,u)$.

$6 \Rightarrow 7$: Pick distinct states $p, q \in A$ and $r \in B \setminus A$. By assumption there exists $u \in \Sigma^*$ such that $\delta(p,u) = \delta(q,u)$ and $\delta(r,u) \neq \delta(s,u)$ for all $s \in Q \setminus \{r\}$. So $\delta(r,u) \notin \delta(A,u)$ and thus $|\delta(A,u)| < |\delta(B,u)|$. Furthermore, since $\{p,q\} \subseteq B$ we have $|\delta(B,u)| < |B|$.

$7 \Rightarrow 8$: Consider A and $B = A \cup \{p\}$. By assumption, we have $|\delta(A,u)| < |\delta(B,u)| < |B|$ for some $u \in \Sigma^*$. Since $|\delta(A,u)| < |\delta(B,u)|$, we find $\delta(p,u) \notin \delta(A,u)$. We have $|B| = |A| + 1$ and $\delta(B,u) = \delta(A,u) \cup \delta(\{p\},u)$ so that $|\delta(B,u)| = |\delta(A,u)| + 1$. Thus, $|\delta(B,u)| < |B|$ yields $|\delta(A,u)| < |A|$.

$8 \Rightarrow 9$: Let $A, B \subseteq Q$ be distinct with $|A|, |B| \geq 2$ and $A \cap B \neq \emptyset$. Without loss of generality, suppose $p \in B \setminus A$ for some $p \in Q$. By assumption we have $|\delta(A,u)| < |A|$ and $\delta(p,u) \notin \delta(A,u)$ for some $u \in \Sigma^*$. Then we get $|\delta(A,u)| + |\delta(B,u)| < |A| + |B|$ and, as $\delta(p,u) \in \delta(B,u)$, we get $\delta(A,u) \neq \delta(B,u)$. Furthermore, $\delta(A \cap B,u) \cup \{\delta(p,u)\} \subseteq \delta(B,u)$ with $A \cap B \neq \emptyset$ and $\delta(p,u) \notin \delta(A \cap B,u)$ gives $|\delta(B,u)| \geq 2$ so that $|\delta(A,u)| + |\delta(B,u)| \geq 3$.

$9 \Rightarrow 10$: Let $A, B \subseteq Q$ be distinct with $|A|, |B| \geq 2$. Without loss of generality, let $p \in B \setminus A$ and $q \in B \setminus \{p\}$. By the assumption applied to the distinct non-empty and non-singleton sets $A \cup \{q\}$ and $A \cup \{p,q\}$ there exists $u \in \Sigma^*$ with $3 \leq$

$|\delta(A \cup \{q\}, u)| + |\delta(A \cup \{p, q\}, u)| < 2|A| + 3$ (where we even have $|\delta(A \cup \{q\}, u)| + |\delta(A \cup \{p, q\}, u)| < 2|A| + 1$ for $q \in A$) and $\delta(A \cup \{q\}, u) \neq \delta(A \cup \{p, q\}, u)$. Since $\delta(A \cup \{p, q\}, u) = \delta(A \cup \{q\}, u) \cup \delta(\{p\}, u)$ we find $\delta(p, u) \notin \delta(A \cup \{q\}, u)$. Hence, as $\delta(p, u) \in \delta(B, u)$, we have $\delta(A, u) \neq \delta(B, u)$ and $|\delta(B, u)| \geq 2$ as $\{\delta(p, u), \delta(q, u)\} \subseteq \delta(B, u)$ where $\delta(p, u) \neq \delta(q, u)$, which gives $3 \leq |\delta(A, u)| + |\delta(B, u)|$. Next, for $\delta(\{p, q\}, u) \cap \delta(A, u) = \emptyset$ we have

$$\begin{aligned} 2|\delta(A, u)| + 3 &= |\delta(A, u)| + |\delta(\{q\}, u)| + |\delta(A, u)| + |\delta(\{p, q\}, u)| \\ &= |\delta(A \cup \{q\}, u)| + |\delta(A \cup \{p, q\}, u)| \\ &< 2|A| + 3. \end{aligned}$$

Now, $2|\delta(A, u)| + 3 < 2|A| + 3$ gives $|\delta(A, u)| < |A|$. So, we find $|\delta(A, u)| + |\delta(B, u)| < |A| + |B|$. Otherwise, suppose $\delta(\{p, q\}, u) \cap \delta(A, u) \neq \emptyset$, which gives $\delta(A, u) \cap \delta(B, u) \neq \emptyset$. In this case, either we have $|\delta(A, u)| + |\delta(B, u)| < |A| + |B|$ and the claim holds with u, or $|\delta(A, u)| = |A| \geq 2$ and $|\delta(B, u)| = |B| \geq 2$, and by the assumption applied to $\delta(A, u)$ and $\delta(B, u)$ there exists $v \in \Sigma^*$ with $3 \leq |\delta(A, uv)| + |\delta(B, uv)| < |\delta(A, u)| + |\delta(B, u)|$ and $\delta(A, uv) \neq \delta(B, uv)$. Then the claim holds with uv as $|\delta(A, u)| + |\delta(B, u)| \leq |A| + |B|$.

10 $\Rightarrow$ 1: Let $A, B \subseteq Q$ be distinct with $|A|, |B| \geq 2$. We do induction on $|A| + |B|$. First, suppose $|A| + |B| = 4$, which is the base case. By assumption, there exists $u \in \Sigma^*$ such that $3 \leq |\delta(A, u)| + |\delta(B, u)| < 4$, which implies that one of both sets must be a singleton and the other a 2-set. Hence, $u \in \Sigma^*$ distinguishes both subsets. Next, suppose $|A| + |B| > 4$. Assume all pairs of distinct subsets $A', B' \subseteq Q$ with $4 \leq |A'| + |B'| < |A| + |B|$ and $|A'|, |B'| \geq 2$ are distinguishable. By the assumption from Item 10 there exists $u \in \Sigma^*$ such that $3 \leq |\delta(A, u)| + |\delta(B, u)| < |A| + |B|$ and $\delta(A, u) \neq \delta(B, u)$. If one of both subsets is a singleton, the other subset has size at least two. So, u distinguishes A and B. Otherwise, as $\delta(A, u) \neq \delta(B, u)$, by induction hypothesis, there exists $v \in \Sigma^*$ distinguishing $\delta(A, u)$ from $\delta(B, u)$. Then uv distinguishes A from B. □

Theorem 9.46, Item 10 is useful for an inductive reasoning because of the fact that it only demands to compress the subsets (while retaining their distinctiveness) instead of instantly mapping precisely one to a singleton subset.

Remark 9.47 By Theorem 9.4 distinguishability of all pairs of distinct $(n - 1)$-sets is not sufficient for complete distinguishability. With Theorem 9.38, this also implies that there exists no k such that distinguishability of all pairs of distinct k-sets implies complete distinguishability.

Remark 9.48 Having for each $q \in Q$ a word $u_q \in \Sigma^*$ that distinguishes Q form $Q \setminus \{q\}$ is necessary, and, by Theorem 9.46, Item 4 also sufficient, for complete distinguishability. Let $p, q, r \in Q$ be distinct. Assume $\delta(r, u_q) = \delta(r, u_p)$. Then we have $\delta(p, u_q) = \delta(r, u_q) = \delta(r, u_p) = \delta(q, u_p)$. Since $\delta(q, u_q) \neq \delta(p, u_q)$, we find $\delta(q, u_q) \neq \delta(q, u_p)$. Thus we conclude that each word u_q induces a different transformation on the states. Hence, the transformation monoid of completely distinguishable automata with at least three states contains at least $|Q|$ different elements.

Corollary 9.49 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. If for all $r \in Q$ and $p, q \in Q \setminus \{q\}$ there exists $u \in \Sigma^*$ such that $\delta(p, u) = \delta(q, u)$, $\delta(r, u) = r$, and $\delta(s, u) \neq r$ for all $s \in Q \setminus \{r\}$, then $\mathcal{A}$ is completely distinguishable.*

Proof. This follows directly from Theorem 9.46, Item 6 since it equals this statement in the special case that the word fixes the state r. □

Corollary 9.50 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton such that the group of units of $\mathcal{T}_M(\mathcal{A})$ is transitive. Then $\mathcal{A}$ is completely distinguishable if and only if Q is distinguishable from $Q \setminus \{q\}$ for some $q \in Q$.*

Proof. First, suppose Q and $Q \setminus \{q\}$ are distinguishable for $q \in Q$. Then $|\delta(Q \setminus \{q\}, u)| = 1$ with $\delta(q, u) \notin \delta(Q \setminus \{q\}, u)$ for some $u \in \Sigma^*$. Let $p \in Q$. Since $\mathcal{T}_M(\mathcal{A})$ contains a transitive permutation group, there exists $v \in \Sigma^*$ inducing a permutation on Q such that $\delta(p, v) = q$. Then we have $\delta(Q \setminus \{p\}, vu) = \delta(Q \setminus \{q\}, u)$ and $\delta(p, vu) = \delta(q, u)$. So $\delta(p, vu) \notin \delta(Q \setminus \{p\}, vu)$ and $|\delta(Q \setminus \{p\}, vu)| = 1$. By Theorem 9.46, Item 4, it follows that $\mathcal{A}$ is completely distinguishable. The converse implication is clear. □

Example 9.51 Consider the automaton in Figure 9.6a. The letter a distinguishes $\{0, 1, 2, 3\}$ from the whole state set. Since the letter b generates a transitive permutation group, the automaton is completely distinguishable by Theorem 9.50. The automaton is not completely reachable since it does not have a letter of defect one.

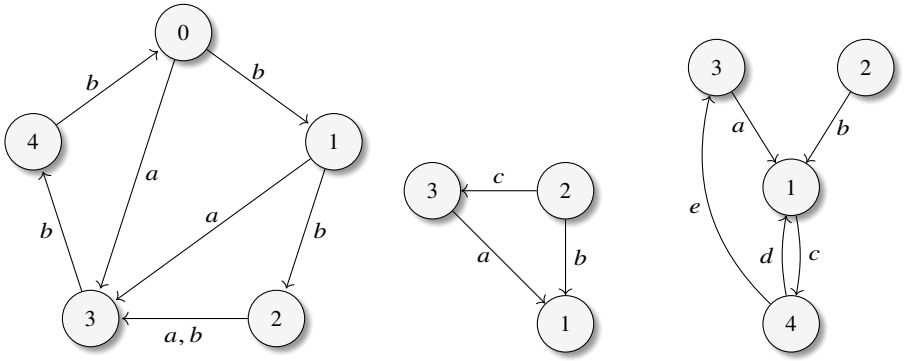
Example 9.52 Figure 9.6b depicts an automaton that is easily seen to be completely distinguishable and that is not completely reachable since it is not strongly connected.

Example 9.53 The word ab distinguishes $\{1, 2, 3\}$ and $\{4\}$. The word $ecab$ distinguishes $\{2, 3, 4\}$ and $\{1\}$. The word bc distinguishes $\{1, 2, 4\}$ and $\{3\}$. The word ac distinguishes $\{1, 3, 4\}$ and $\{2\}$. So, by Theorem 9.46, Item 4, the automaton from Figure 9.6c is completely distinguishable, but it is not completely reachable since it is not strongly connected. Note that by adding a simple idempotent mapping 1 to 2, we obtain a strongly connected SI-automaton that is completely distinguishable, but whose automaton graph is not strongly $\frac{3}{2}$-connected since all paths from 2 to 4 have to pass through 1.

It is easily seen that quotient automata of completely reachable automata are completely reachable. However, complete distinguishability is not preserved under taking quotient automata as shown in Figure 9.6 next.

Example 9.54 Automata $\mathcal{A} = (Q, \Sigma, \delta)$ as in Theorem 9.51 also show that complete distinguishability is not preserved when taking quotient automata. For consider the obvious generalization where the letter b cyclically permutes n states and suppose we have some non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence $\sim$. Let $s \in Q$ be the state fixed by a. Since the letter b cyclically permutes the whole state set, all congruence classes for $\sim$ have the same size and we find $t \in Q$ with $s \neq t$ such that $s \sim t$.

First, we show that for every $u \in \Sigma^*$ not permuting the set of states we have $\delta(p, u) \sim \delta(q, u)$ for all $p, q \in Q$, i.e., the letter a induces a constant in $\mathcal{A}/\sim$). Let $p, q \in Q$. We can assume $p \nsim q$. For $u \in \Sigma^*$ not permuting Q we can write $u = b^i av$



(a) The Strongly connected automaton from Example 9.51.

(b) The automaton from Example 9.52.

(c) The automaton from Theorem 9.53

Fig. 9.6: Completely distinguishable automata that are not completely reachable. Self-loops omitted.

for some $v \in \Sigma^*$ and $i \geq 0$. If $\delta(p, b^i) \neq s$ and $\delta(q, b^i) \neq s$, then $\delta(p, b^i a) = \delta(q, b^i a)$ and so $\delta(p, u) = \delta(q, u)$. Otherwise, without loss of generality, assume $\delta(p, b^i) = s$. Then $\delta(q, b^i) \neq s$. Now, we have $\delta(p, b^i a) = s$ and $\delta(q, b^i a) = \delta(t, a)$. Furthermore, $s \sim t$ implies $\delta(p, b^i a) = s = \delta(s, a) \sim \delta(t, a) = \delta(q, b^i a)$ and thus $\delta(p, u) \sim \delta(q, u)$.

Next, assume we want to distinguish $Q/\!\sim$ and $Q/\!\sim \setminus \{[p]_\sim\}$ in $\mathcal{A}/\!\sim$ for some $p \in Q$. Then a distinguishing word u maps all $q \in Q$ with $q \nsim p$ into an equivalence class and for all $r \in Q$ with $r \sim p$ we have $\delta(r, u) \nsim \delta(q, u)$ for $q \in Q$ with $q \nsim p$. However, since a distinguishing word cannot permute the whole state set, as shown above this is not possible.

Next, we show that a binary completely distinguishable automaton with a letter of rank two, such as the one from Figure 9.6a, must contain a letter cyclically permuting the states.

Proposition 9.55 *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be a binary automaton with at least three states where a has rank two. If $\mathcal{A}$ is completely distinguishable, then b cyclically permutes the states.*

Proof. Assume $\delta(p_1, b) = \delta(p_2, b)$ for two distinct states $p_1, p_2 \in Q$. Then a word distinguishing $\{p_1\}$ from $Q \setminus \{p_1\}$ must start with a. This implies $\delta(p_1, a) \notin \delta(Q \setminus \{p_1\}, a)$. Since a has rank two, we get $|\delta(Q \setminus \{p_1\}, a)| = 1$ and so a distinguishes both subsets. By a similar reasoning with $\{p_2\}$ and $Q \setminus \{p_2\}$, we find $|\delta(Q \setminus \{p_2\}, a)| = 1$. Since $|Q| \geq 3$, this yields that $Q \setminus \{p_1\}$ and $Q \setminus \{p_2\}$ are mapped to the same state and so $\delta(p_1, a) = \delta(p_2, a)$, giving a contradiction to $\delta(p_1, a) \notin \delta(Q \setminus \{p_1\}, a)$. Thus, no two distinct states are mapped to the same state by b, and by finiteness we find that b permutes the states.

Next, by assumption we have $Q = A \cup B$ where $A = \delta^{-1}(p, a)$ and $B = \delta(q, a)$ for some $p, q \in Q$. Assume there exist two distinct $|A|, |B| \geq 2$. Each word distinguishing two subsets must have the form $b^i a$ for some $i \geq 0$, since a has rank two. We have

$\delta^{-1}(p, b^i a) = \delta^{-1}(A, b^i)$ and $\delta^{-1}(q, b^i a) = \delta^{-1}(B, b^i)$ and the two subsets have size at least two and partition Q since b permutes the states. However, a word distinguishes a state s from $Q \setminus \{s\}$ if and only if $\{s\}$ and $Q \setminus \{s\}$ are the δ^{-1}-images of two states. Thus, one of both subsets A or B must be a singleton. Without loss of generality, let $A = \{p_1\}$ and consider some state $p_2 \in Q$. A word $b^i a$ distinguishes p_2 and $Q \setminus \{p_2\}$ if and only if p_2 is mapped to p_1 by b^i. Thus we find that b cyclically permutes the states. □

Lastly, we also note a readily seen equivalence when the whole state set is distinguishable from a set of the form $Q \setminus \{q\}$, since these sets appear in Theorem 9.46, Item 4

Proposition 9.56 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$Q \setminus \{q\}$ is distinguishable from Q.*
2. *There exist $u_1, \ldots, u_m \in \Sigma$ such that for $A_i = \delta(Q \setminus \{q\}, u_1 \cdots u_i)$ and $q_i = \delta(q, u_1 \cdots u_i)$ where $i \in \{1, \ldots, m\}$ we have $|A_m| = 1$ and $q_i \notin A_i$ for all $i \in \{1, \ldots, m\}$.*

Example 9.57 Let $\mathcal{A} = (\{1, 2, 3, 4\}, \Sigma, \delta)$ with

$$\Sigma = \{a \mid \exists y_1, y_2 \in Q \colon y_1 \neq y_2 \wedge |\delta^{-1}(y_1, a)| = |\delta^{-1}(y_1, a)| = 2\}$$

Then disjoint pairs of 2-sets are indistinguishable and non-disjoint pairs of 2-sets are distinguishable. Furthermore, two sets of size at least three are indistinguishable. A set of size two is distinguishable from any set of size at least three.

9.3.5 Further Examples

Theorems 9.33 and 9.34 gave infinite families of automata that are not completely distinguishable (as they are not synchronizing and have more than three states), but in which all 2-sets are pairwise distinguishable. Furthermore, in the synchronizing binary SI-automaton with trap state t from Figure 7.4c, every pair of distinct 2-sets is distinguishable, whereas any two subsets of the form A and $A \cup \{t\}$, where $t \notin A$ and $|A| \geq 2$, are indistinguishable. Here, in Theorems 9.58 and 9.59, we give further automata that are not completely distinguishable, but are synchronizing and have the property that all distinct pairs of 2-sets are distinguishable. The automaton from Theorem 9.58 is even a completely reachable SI-automaton.

Example 9.58 Figure 9.7 gives a synchronizing and strongly connected SI-automaton with the following properties.

1. A set $\{q_1, \ldots, q_m\}$ with $q_1 < \cdots < q_m$ is indistinguishable from $\{q_1, q_m\}$.
2. All distinct pairs of 2-subsets of states are distinguishable.
3. For $A, B \subseteq Q$ with $|A|, |B| \geq 2$ there exists $u \in \Sigma^*$ such that $|\delta(A, u)| + |\delta(B, u)| < |A| + |B|$.
4. For $A, B \subseteq Q$ with $|A|, |B| \geq 2$ and $A \neq B$ there exists $u \in \Sigma^*$ such that $|\delta(A, u)| \neq |\delta(B, u)|$.

The automaton shows that the latter two properties, which are similar to Theorem 9.46, Item 10, are not sufficient for complete distinguishability. It is interesting to note that adding a simple idempotent letter mapping 1 to n to the automaton in Figure 9.7 yields a completely distinguishable automaton, which follows from Theorem 11.36 since the resulting automaton graph is strongly $\frac{3}{2}$-connected. We note that the automaton from Theorem 9.33 also has the property that two distinct subsets of size at least two can be mapped to subsets of different sizes, but it is not even synchronizing.

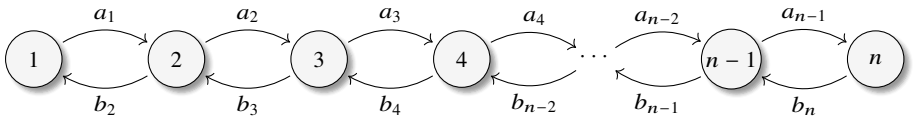
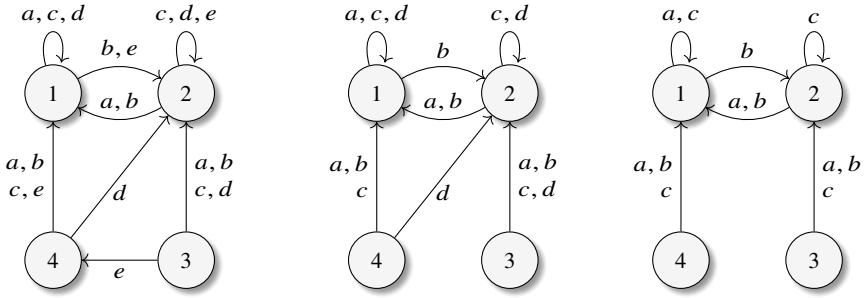


Fig. 9.7: Strongly connected and synchronizing SI-automata (self-loops omitted) where all pairs of 2-sets are distinguishable and a set $\{q_1, \ldots, q_m\}$ with $q_1 < \cdots < q_m$ is indistinguishable from $\{q_1, q_m\}$. See Theorem 9.58.

Example 9.59 Let $\mathcal{A} = (\{1,2,3,4\}, \{a,b,c,d,e\}, \delta)$ be the automaton from Figure 9.8a over the alphabet $\{a,b,c,d,e\}$. Figure 9.8b and Figure 9.8c show $\mathcal{A}_{|\{a,b,c,d\}}$ and $\mathcal{A}_{|\{a,b,c\}}$, respectively.



(a) $\{1,2,3,4\}$ and $\{1,3,4\}$ are indistinguishable.

(b) $\{1,2,3,4\}$ and $\{1,2,3\}$ are indistinguishable.

(c) $\{1,2,3\}$ and $\{3,4\}$ are indistinguishable.

Fig. 9.8: Automata that are not completely distinguishable, but in which all pairs of distinct 2-sets are distinguishable; see Theorem 9.59.

These automata are not completely distinguishable.[4] Table 9.1 shows distinguishing words for all distinguishable pairs in $\mathcal{A}$. We have the following maximal classes of pairwise indistinguishable subsets of states:

[4] However, similarly as in Theorem 9.58, for each pair of distinct A, B with $|A|, |B| \geq 2$ there exists $u \in \Sigma^*$ such that $|\delta(A,u)| \neq |\delta(B,u)|$.

Table 9.1: Pairs of subsets in the automaton from Figure 9.8a that are distinguishable by the indicated word, or indistinguishable as indicated by a hyphen.

	$\{1,2\}$	$\{1,3\}$	$\{1,4\}$	$\{2,3\}$	$\{2,4\}$	$\{3,4\}$	$\{1,2,3\}$	$\{1,2,4\}$	$\{1,3,4\}$	$\{2,3,4\}$
$\{1,3\}$	a	–								
$\{1,4\}$	c	a	–							
$\{2,3\}$	a	b	a	–						
$\{2,4\}$	b	a	c	a	–					
$\{3,4\}$	a	b	a	c	a	–				
$\{1,2,3\}$	a	b	a	c	b	d	–			
$\{1,2,4\}$	e	b	c	c	b	d	a	–		
$\{1,3,4\}$	a	b	a	c	b	d	ee	a	–	
$\{2,3,4\}$	a	b	a	c	b	ec	d	a,d	d	–
$\{1,2,3,4\}$	a	b	a	c	b	d	eee	a	–	d

$$\begin{aligned}\mathcal{A} &: \{\{1,3,4\},\{1,2,3,4\}\},\\ \mathcal{A}_{|\{a,b,c,d\}} &: \{\{1,2\},\{1,2,4\}\},\{\{3,4\},\{2,3,4\}\},\{\{1,2,3\},\{1,3,4\},\{1,2,3,4\}\},\\ \mathcal{A}_{|\{a,b,c\}} &: \{\{1,2\},\{1,2,4\}\},\{\{3,4\},\{2,3,4\},\{1,2,3\},\{1,3,4\},\{1,2,3,4\}\}.\end{aligned}$$

For $\mathcal{A}_{|\{a,b,c\}}$ and $\mathcal{A}_{\{a,b,c,d\}}$ this follows from the following arguments:

1. We have $\delta^{-1}(1,a) = \{1,2,4\}$. So, for two distinct 2-sets A, B of states such that $A \subseteq \{1,2,4\}$ and $3 \in B$ the letter a yields a distinguishing word. As $\delta^{-1}(1,c) = \{1,4\}$ and $\delta^{-1}(1,b) = \{2,4\}$ every pair of distinct 2-sets from $\{1,2,4\}$ is distinguishable and as $\delta^{-1}(2,c) = \{2,3\}$ and $\delta^{-1}(2,b) = \{1,3\}$ every pair of distinct 2-sets A, B with $3 \in A \cap B$ is distinguishable. So, we find that all pairs of distinct 2-sets are distinguishable in $\mathcal{A}_{\{a,b,c\}}$.
   We also obtain that every 2-set not equal to $\{3,4\}$ is distinguishable from $\{1,2,3,4\}$, and that $\{1,2,4\}$ is distinguishable from every other 3-set.
2. Let $x \in \{a,b,c\}$. For each A with $|A| \geq 3$ and $A \neq \{1,2,4\}$ we have

   $$\delta(A,x) = \delta(\{3,4\},x) = \{1,2\}.$$

   Therefore, any two sets A, B with $|A|, |B| \geq 3$ and $A, B \neq \{1,2,3\}$ are indistinguishable in $\mathcal{A}_{|\{a,b,c\}}$. Moreover, $\{3,4\}$ is indistinguishable from every subsets A with $|A| \geq 3$ and $A \neq \{1,2,4\}$. Since $\delta(\{1,2\},x) = \delta(\{1,2,4\},x)$, the sets $\{1,2\}$ and $\{1,2,4\}$ are also indistinguishable in $\mathcal{A}_{|\{a,b,c\}}$.
   Items 1 and 2 yield that

   $$\{\{1,2\},\{1,2,4\}\} \text{ and } \{\{3,4\},\{2,3,4\},\{1,2,3\},\{1,3,4\},\{1,2,3,4\}\}$$

   are two maximal classes of pairwise indistinguishable subsets in $\mathcal{A}_{|\{a,b,c\}}$.
3. We have $\delta^{-1}(2,d) = \{2,3,4\}$. Hence, every subset of $\{2,3,4\}$ is distinguishable in $\mathcal{A}_{|\{a,b,c,d\}}$ from every subset A with $|A| \geq 2$ and $1 \in A$. Since $\delta(\{3,4\},d) = \delta(\{2,3,4\},d) = \{2\}$,

   $$\delta(\{1,2,3\},d) = \delta(\{1,2,4\},d) = \delta(\{1,3,4\},d) = \delta(\{1,2,3,4\},d) = \{1,2\},$$

and $\{\{3,4\},\{2,3,4\},\{1,2,3\},\{1,3,4\},\{1,2,3,4\}\}$ is a maximal set of pairwise indistinguishable subsets in $\mathcal{A}_{|\{a,b,c\}}$, we find that

$$\{\{3,4\},\{2,3,4\}\} \text{ and } \{\{1,2,3\},\{1,3,4\},\{1,2,3,4\}\}$$

are maximal sets of pairwise indistinguishable subsets in $\mathcal{A}_{|\{a,b,c,d\}}$.

Recall that distinguishability of pairs of distinct 2-sets does not imply synchronizability, see Figure 9.4. However, this implication holds for strongly connected automata by Theorem 9.32.

9.4 From Automata to Transformation Semigroups

Here, in Section 9.4.1, we transfer our notions to transformation semigroups. Technicalities that arise due to the possible absence of an identity transformation in transformation semigroups are discussed in Section 9.4.2 where we investigate reachability and distinguishability w.r.t. to differences in semigroup and monoids.

9.4.1 Transferring Notions to Transformation Semigroups

Let $S \leq_S \mathcal{T}_X$ be a transformation semigroup. Similarly as for automata, a subset $A \subseteq X$ is said to be *reachable (in S)* if there exists $t \in S$ such that $t(X) = A$. We call two subsets $A, B \subseteq X$ *distinguishable (in S)* if there exists $t \in S$ (called a *distinguishing transformation*) such that precisely one of the two subsets $t(A)$ and $t(B)$ is a singleton subset, i.e.,

$$(|t(A)| = 1 \wedge |t(B)| \neq 1) \vee (|t(A)| \neq 1 \wedge |t(B)| = 1).$$

The following fact is readily seen.

Proposition 9.60 *Every non-empty set is distinguishable from the empty set in an automaton (resp. transformation semigroup) if and only if the automaton is synchronizing (resp. the transformation semigroup contains a constant map).*

Furthermore, in analogy to automata, S is called *completely reachable* (resp. *completely distinguishable*) if every non-empty subset of X is reachable in S (resp. if every pair of non-empty and non-singleton subsets is distinguishable).

In Section 6.8, we associated automata with transformation semigroups. By the given construction, automata-theoretical results transfer to transformation semigroups. For example, our results about distinguishable pairs of subsets from Section 9.3 can be reformulated for transformation semigroups. For reference, we state a reformulation of Theorem 9.44 and Theorem 9.46 for finite X (where we only list, as an example, a selection of equivalences from Theorem 9.46).

Proposition 9.61 *Let $S \leq_S \mathcal{T}_n$ be a completely distinguishable semigroup with $n \geq 3$. Then S contains a constant map.*

Proof. Construct an automaton $\mathcal{A}$ as in Section 6.8 such that $\mathcal{T}_S(\mathcal{A}) = S$. Then two sets are distinguishable in $\mathcal{A}$ if and only if they are distinguishable in S. So, $\mathcal{A}$ is completely distinguishable and has at least three states. Thus, by Theorem 9.44, it contains a synchronizing word u, which induces a constant map in $\mathcal{T}_S(\mathcal{A})$. □

Theorem 9.62 *Let $S \leq_S \mathcal{T}_n$ with $n \geq 3$. Then the following are equivalent:*

1. *S is completely distinguishable.*
2. *For every $A, B \subseteq [n]$ with $A \subsetneq B$ and A non-empty there exists $t \in S$ such that $1 = |t(A)| < |t(B)|$.*
3. *For every $p \in [n]$ there exists $t \in S$ such that $t(p) \notin t([n] \setminus \{p\})$ and $|t([n] \setminus \{p\})| = 1$.*
4. *For every $A \subseteq [n]$ with $|A| \geq 2$ and $p \in [n] \setminus A$ there exists $t \in S$ such that $t(p) \notin t(A)$ and $|t(A)| < |A|$.*
5. *For every two distinct $A, B \subseteq [n]$ with $\min\{|A|, |B|\} \geq 2$ there exists $t \in S$ such that $3 \leq |t(A)| + |t(B)| < |A| + |B|$ and $t(A) \neq t(B)$.*

Proof. This follows similarly as in the proof of Theorem 9.61 by considering an automaton $\mathcal{A}$ with $\mathcal{T}_S(\mathcal{A}) = S$. □

9.4.2 Differences Between Monoids and Semigroups

Reachability.

In a transformation semigroup $S \leq_S \mathcal{T}_X$ every proper subset is reachable in S if and only if it is reachable in $S \cup \{\mathrm{id}_X\}$. So, for proper subsets, absence of an identity element makes no difference. For finite X, X is reachable if and only if S contains the identity transformation.

Proposition 9.63 *Let $S \leq_S \mathcal{T}_n$. Then $[n]$ is reachable in S if and only if $\mathrm{id}_{[n]} \in S$.*

Proof. If $\mathrm{id}_{[n]} \in S$, then clearly $[n]$ is reachable. Conversely, suppose $t([n]) = [n]$ for some $t \in S$. Then t is a permutation, and so $\mathrm{id}_{[n]} = t^i \in S$ for some $i > 0$. □

As a subset is reachable in an automaton $\mathcal{A}$ if and only if it is reachable in $\mathcal{T}_M(\mathcal{A})$, we obtain the following equivalences.

Proposition 9.64 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

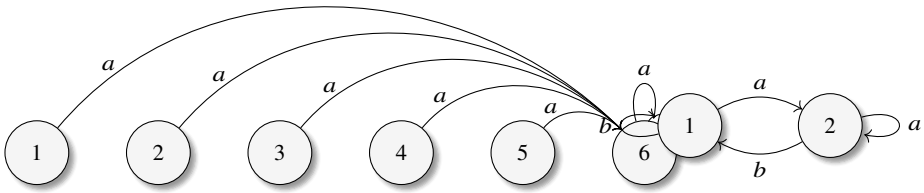
1. *$\mathcal{A}$ is completely reachable.*
2. *$\mathcal{T}_M(\mathcal{A})$ is completely reachable.*
3. *$\mathcal{T}_S(\mathcal{A}) \cup \{\mathrm{id}_Q\}$ is completely reachable.*

Most of our results about reachable subsets in automata from Section 9.2 do not use the empty word. Hence they carry over to transformation semigroups via the correspondence discussed in Section 6.8.

Distinguishability.

For automata (resp. transformation monoids), the empty word (resp. identity transformation) distinguishes any singleton subset from any non-singleton subset. For transformation semigroups, as the following example shows, distinguishability of non-singleton subsets from singleton subsets is not automatically fulfilled.

Example 9.65 Figure 9.9 depicts two automata where all non-empty subsets are indistinguishable from any singleton set in $\mathcal{T}_S(\mathcal{A})$. Note that transformation semigroups over sets of size at most two are always completely distinguishable.



(a) Every non-empty set is indistinguishable from any singleton set in $\mathcal{T}_S(\mathcal{A})$.

(b) $\{1, 2\}$ is indistinguishable from any singleton set in $\mathcal{T}_S(\mathcal{A})$.

Fig. 9.9: Two automata $\mathcal{A}$ where every non-empty set is indistinguishable from any singleton set in $\mathcal{T}_S(\mathcal{A})$. Note that in $\mathcal{T}_M(\mathcal{A})$ every non-singleton subset is distinguishable from a singleton subset by the identity transformation.

Distinguishability of sets of size at least two from any singleton subset is equivalent to the property that for any two distinct points there exists a transformation mapping those two points to distinct points, which is also equivalent to distinguishability of singletons from 2-sets. This is fulfilled when the transformation semigroup contains the identity transformation. Next, we list other sufficient conditions implying that for any two distinct points there exists a transformation mapping them to distinct points.

Lemma 9.66 *Let $S \leq_S \mathcal{T}_n$. Suppose for distinct $p, q \in [n]$ there exists $r \in [n]$ such that $\{p, r\}$ and $\{q, r\}$ are distinguishable. Then for distinct $p, q \in [n]$ there exists $t \in S$ such that $t(p) \neq t(q)$.*

Proof. Let $p, q \in [n]$. Then there exists $t \in S$ distinguishing $\{p, r\}$ from $\{q, r\}$ for some $r \in Q$. Without loss of generality, suppose $t(p) = t(r)$, i.e., $\{p, r\}$ is mapped to a singleton set. Then $t(q) \neq t(p)$ as $\{q, r\}$ is mapped to a 2-set. □

For example, if there exists $r \in [n]$ such that, for all distinct $p, q \in [n]$, some word $u \in \Sigma^*$ distinguishes $\{r, p\}$ from $\{r, p, q\}$, then u distinguishes $\{r, p\}$ from $\{r, q\}$. So, Theorem 9.66 yields that every singleton subset is distinguishable from every subset of size at least two.

Proposition 9.67 *Let $S \leq_S \mathcal{T}_n$ and $n \geq 3$. Suppose pairs of distinct 2-sets are distinguishable in S. Then every singleton subset is distinguishable from every subset containing at least two elements in S.*

Proof. By assumption and as $n \geq 3$, the condition from Theorem 9.66 is fulfilled, which gives the claim. □

Proposition 9.68 *A transformation semigroup $S \leq_S \mathcal{T}_X$ is completely distinguishable if and only if the semigroup $S \cup \{\mathrm{id}_X\}$ is completely distinguishable.*

Proof. Complete distinguishability does not require the distinguishability of singleton subsets from non-singleton subsets. Since the identity transformation distinguishes two subsets if and only if precisely one equals a singleton subset, we obtain the claim. □

Corollary 9.69 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is completely distinguishable.*
2. *$\mathcal{T}_M(\mathcal{A})$ is completely distinguishable.*
3. *$\mathcal{T}_S(\mathcal{A})$ is completely distinguishable.*

Besides pairs A, B with $|A| = 1$ and $|B| \geq 2$, complete distinguishability does not require distinguishability of a non-empty set from the empty set. Next, we show that for a transformation semigroup on $[n]$ with $n \neq 2$, complete distinguishability implies that all pairs of sets where at least one is not a singleton set are distinguishable.

Proposition 9.70 *A transformation semigroup $S \leq_S \mathcal{T}_n$ with $n \neq 2$ is completely distinguishable if and only if all pairs of distinct $A, B \subseteq [n]$ with $|A| \neq 1$ or $|B| \neq 1$ are distinguishable.*

Proof. For $n \leq 1$ the claim is obvious, so assume $n \geq 3$ in the following. First, suppose $S \leq_S \mathcal{T}_n$ is completely distinguishable. By Theorem 9.67, every singleton subset is distinguishable from every subset containing at least two points. By Theorem 9.61, the semigroup S contains a constant map. So, by Theorem 9.60, the empty set is distinguishable from every other set.

Conversely, if all pairs of distinct subsets where at least one subset is not a singleton set are distinguishable in S, then S is clearly completely distinguishable. □

9.5 Discussion

The notions of distinguishability of pairs of subsets and of complete distinguishability, and some accompanying results, were introduced in [308]. Bondar & Volkov [99, 101] introduced completely reachable automata. Theorems 9.12 and 9.19 were stated by them, see also [101, Sect. 6], [134] or [307, Prop. 3] for other statements. A preliminary version of Theorem 9.18 has been given in [303, Prop. 3]. Section 11.1 will present additional results on completely reachable automata by Bondar & Volkov [99, 100] and Bondar et al. [101]. The remaining results on reachable subsets from Section 9.2 have not

been mentioned in the literature before. In Theorem 9.15, we gave non-synchronizing n-state automata where every k-set is reachable with $n = 2k - 2$. We do not know whether there exist such automata where we have $n = k^2 - k - 1$ and $k \geq 2$, i.e., whether the bound from Theorem 9.14 is optimal. Complete reachability has been used in [309] to characterize primitive permutation groups (these results are also presented in Section 14.1.3 here).

The distinguishability relation was introduced in [308, 315]. In [197, Def. 5] the first definition equals the indistinguishably relation derived from our notion of distinguishability, but this has not been noted in [197].

An Erroneous Result.

In [304, Thm. 7] it was wrongly claimed that a transformation monoid is completely distinguishable if and only if this is true for all pairs of distinct 2-sets (a related false statement appears in [297, Lem. 3.1]). Section 9.3.5 lists various counterexamples. Now, the main result of [304, Thm. 12] (which is still true) characterizing primitive permutation groups in terms of the state complexity of associated automata used this false statement in its proof, as well as a special case of it stated before in [297, Prop. 4.8]. In Section 14.1.3, Theorem 14.9, we present a correct proof, another correct proof has also been given in the extended version [309, Thm. 6.3] of the conference contributions [297, 304]. The false claim has also been used in [306], and one main result from this paper, stating that a completely reachable SI-automaton is completely distinguishable, is in fact false, see Figure 9.7 for a counterexample. In the paper [306], it was shown that in a completely reachable SI-automaton all pairs of distinct 2-sets are distinguishable, and then the wrong claim has been used to conclude complete distinguishability. The inductive "proof" in [303, Thm. 7] (not contained in the proceedings version) missed a case (we note that this is related to the condition $\delta(A, u) \neq \delta(B, u)$ in Theorem 9.46, Item 10), which was reported to us by an anonymous reviewer of a paper that was (of course) rejected. The counterexample in Figure 9.8c is due to the reviewer. We genuinely thank the reviewer for spotting the mistake! Our attempts at fixing it led to more insight, the notion of strong $\frac{3}{2}$-connectivity, and involvement of the theory of vertex-transitive graphs.

Chapter 10
Sync-Maximal Automata

Here, in Section 10.1, we introduce sync-maximal automata as automata whose sets of synchronizing words have maximal state complexity. Then, in Section 10.2, we give structural results. The main result is Theorem 10.7, which states that these automata contain a completely reachable and completely distinguishable subautomaton that differs from the whole automaton by at most one "dangling" state. Lastly, Section 10.3 provides various examples illustrating relations between sync-maximal, completely distinguishable, and completely reachable automata. For instance, we present sync-maximal automata that are not completely reachable, as well as completely reachable automata that are not sync-maximal, along with other examples illustrating our results.

10.1 Definition and First Properties

Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton. Recall that by Theorem 9.1 a minimal automaton for $\mathrm{Syn}(\mathcal{A})$ has at most $2^n - n$ states. We call $\mathcal{A}$ *sync-maximal* if the minimal automaton of $\mathrm{Syn}(\mathcal{A})$ has $2^n - n$ states.

A sync-maximal automaton $\mathcal{A}$ with non-empty state set must be synchronizing. For assume $\mathrm{Syn}(\mathcal{A})$ is empty. Then it is acceptable by a one-state automaton with empty final state set. Now $2^n - n$ equals one only for $n \in \{0, 1\}$. So, $\mathcal{A}$ has a single state, but this implies $\mathrm{Syn}(\mathcal{A}) = \Sigma^*$, contradicting the assumption. Next, in Theorem 10.1 we prove that in a sync-maximal automaton also all subsets of size at least two must be reachable.

Theorem 10.1 *An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with non-empty Q is sync-maximal if and only if the following two conditions hold:*

1. *$\mathcal{A}$ is synchronizing (i.e., at least one singleton subset is reachable) and every non-empty subset with at least two states is reachable.*
2. *$\mathcal{A}$ is completely distinguishable.*

Proof. We can assume $|Q| \geq 2$. Let $\mathcal{F}$ be a maximal family of pairwise distinguishable and reachable subsets. The family contains at most one singleton subset, as two singleton

subsets are indistinguishable, and only non-empty subsets, as the empty set is not reachable. So, we have $\mathcal{F} \subseteq \{A \subseteq Q : |A| \geq 2\} \cup \{\{q\}\}$ for some $q \in Q$. As $|\{A \subseteq Q : |A| \geq 2\} \cup \{\{q\}\}| = 2^n - n$, by Theorem 9.10 $\mathcal{A}$ is sync-maximal if and only if $\{A \subseteq Q : |A| \geq 2\} \cup \{\{q\}\} = \mathcal{F}$, which is equivalent to complete distinguishability, synchronizability, and reachability of all subsets of size at least two. □

Next, we give two corollaries to Theorem 10.1.

Corollary 10.2 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then $\mathcal{A}$ is sync-maximal if and only if one of the following is true:*

1. $|Q| = 1$*;*
2. $|Q| = 2$ *and $\mathcal{A}$ is synchronizing;*
3. $|Q| \geq 3$*, all subsets $A \subseteq Q$ with $|A| \geq 2$ are reachable, and $\mathcal{A}$ is completely distinguishable.*

Proof. 1: The set of synchronizing words of a one-state automaton is Σ^* and a minimal automaton accepting all words has a single state.

2: By Theorem 9.45, every two-state automaton is completely distinguishable and the whole state set is reachable by the empty word. So, by Theorem 10.1, sync-maximality reduces to the reachability of a single state, i.e., synchronizability. Thus, $\mathcal{A}$ is sync-maximal if and only if it is synchronizable.[1]

3: If $\mathcal{A}$ is completely distinguishable, then $\mathcal{A}$ is synchronizing by Theorem 9.44 as $|Q| \geq 3$. So, when also all subsets of size at least two are reachable, then Theorem 10.1 yields sync-maximality.[2] Conversely, if $\mathcal{A}$ is sync-maximal, then, by Theorem 10.1, all subsets of size at least two are reachable and $\mathcal{A}$ is completely distinguishable. □

Corollary 10.3 *An automaton is strongly connected and sync-maximal if and only if it is completely reachable and completely distinguishable.*

Proof. By Theorem 10.1 a sync-maximal automaton is completely distinguishable. Furthermore, every subset of size at least two is reachable and some singleton subset is reachable. Since in a strongly connected automaton, some singleton subset is reachable if and only if all singleton subsets are reachable, we obtain complete reachability. Conversely, by Theorem 9.12 and Theorem 10.1, a completely reachable and completely distinguishable automaton is strongly connected and sync-maximal. □

Theorem 10.3 together with Theorems 9.13 and 9.43 yield that the Černý automata are sync-maximal (this also follows from Theorem 12.31 or from Theorem 12.10 given later).

Theorem 10.4 *The Černý automata C_n from Figure 1.7 are sync-maximal.*

---

[1] For an alternative argument, observe that for a two-state automaton $\mathcal{A}$ we have $\mathrm{Syn}(\mathcal{A}) = \Sigma^*\Gamma\Sigma^*$ for $\Gamma = \{a \in \Sigma \mid \delta(Q, a) \neq Q\}$. So, the minimal automaton of $\mathrm{Syn}(\mathcal{A})$ has two states if and only if $\Gamma \neq \emptyset$, which is equivalent to $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$ in this case.

[2] By Theorem 9.16, Item 1, reachability of all 2-sets also implies synchronizability for $|Q| \geq 3$.

10.2 The Structure of Sync-Maximal Automata

Here, in Theorem 10.7, we show that an automaton is sync-maximal if and only if it contains a completely reachable and completely distinguishable subautomaton that misses at most one "dangling" state. Before stating this result, we give a lemma needed in its proof.

Lemma 10.5 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Suppose $S \subseteq Q$ is an upper set, $|S| = |Q| - 1$, and $u \in \Sigma^*$ is a word of shortest length such that $\delta(Q, u) = S$. Then $u \in \Sigma$, i.e., $|u| = 1$.*

Proof. We can write $u = av$ for some $a \in \Sigma$ and $v \in \Sigma^*$ where we must have $\delta(Q, a) \neq Q$ by the choice of u as a shortest word. Since $|\delta(Q, u)| = |Q| - 1$ we also have $|\delta(Q, a)| = |Q| - 1$. Write $\delta(Q, a) = Q \setminus \{p\}$ for some $p \in Q$. Assume $p \in S$. Then $\delta(p, v) \in S$. With $\delta(Q \setminus \{p\}, v) = S$ this gives $\delta(Q, v) = S$, contradicting the minimal choice of u. Thus, we have $Q = S \cup \{p\}$ and so $\delta(Q, a) = S$. □

Remark 10.6 Theorem 10.5 does not generalize in the sense that upper sets of size $|Q| - k$ for some $1 \leq k < |Q|$ are reachable by a word of length k. For example, let $\mathcal{A} = (Q, \{a, b, c\}, \delta)$ with $Q = \{0, 1, 2, 3\}$, a and b being simple idempotents mapping 0 to 1 and 2 to 3, and c being a permutational letter that swaps 1 and 2 and fixes every other state. Then a shortest word mapping Q to the upper set $\{3\}$ is, for example, $abcb$.

By Theorem 10.3, complete reachability and complete distinguishability imply sync-maximality. By Theorem 10.1, every sync-maximal automaton is completely distinguishable. However, there exists sync-maximal automata that are not completely reachable, see Section 10.3.

The next result also characterizes sync-maximal automata that are not completely reachable. It turns out that in every such automaton, exactly one singleton subset is unreachable.

Theorem 10.7 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with at least three states and let*

$$S = \{q \in Q \mid \exists u \in \Sigma^* : \delta(Q, u) = \{q\}\}$$

be the set of synchronizing states (which forms a sink component by Theorem 7.11). Then $\mathcal{A}$ is sync-maximal if and only if one of the following is true:

1. *$\mathcal{A}$ is completely reachable and completely distinguishable or*
2. *$Q = S \cup \{q\}$ for some $q \notin S$ and, with $\Gamma = \{x \in \Sigma \mid \delta(q, x) = q\}$, we have*
   a. *$\delta(Q, a) = S$ for some $a \in \Sigma \setminus \Gamma$,*
   b. *for each $p \in Q$, all pairs of non-empty, non-singleton subsets of $\delta^{-1}(\mathrm{dupl}(a), a) \cup \{p\}$ that are both distinct from $\delta^{-1}(\mathrm{dupl}(a), a)$ and contain q are distinguishable,*[3]

[3] Let $\{s, t\} = \delta^{-1}(\mathrm{dupl}(a), a)$. This condition is equivalent to distinguishability of $\{s, t, p\}$ and $\{p, q\}$ if $q \in \{s, t\}$; and to pairwise distinguishability of $\{s, t, q\}$, $\{s, q\}$, and $\{t, q\}$ if $q \notin \delta^{-1}(\mathrm{dupl}(a), a)$. Note that for $q \notin \{s, t\}$ we can assume $p = q$ since otherwise no subset of $\{s, t, p\}$ contains q.

c. $\mathcal{A}_{|\Gamma,S}$ *is completely reachable,*
d. $\mathcal{A}_{|S}$ *is completely distinguishable.*

Proof. First, suppose $\mathcal{A}$ is sync-maximal. By Theorem 7.11, S is a sink component. By Theorem 10.2, Item 3, every 2-set is reachable in $\mathcal{A}$. So, we have $|Q \setminus S| \leq 1$ by Theorem 9.26. If $Q = S$, then $\mathcal{A}$ is strongly connected. So, by Theorem 10.3, it is completely reachable and completely distinguishable. Otherwise, let $q \in Q \setminus S$ and set $\Gamma = \{x \in \Sigma \mid \delta(q,x) = q\}$. We have $Q = S \cup \{q\}$. As S is reachable in $\mathcal{A}$, by Theorem 10.5 there exists $a \in \Sigma$ such that $\delta(Q,a) = S$ and as $q \notin S$ we have $a \notin \Gamma$. Let $A \subseteq S$ be a non-empty subset. By Theorem 10.2, Item 3 there exists $u \in \Sigma^*$ such that $\delta(Q,u) = A \cup \{q\}$. We have $\delta(S,u) \subseteq S$. Thus $\delta(q,u) \neq q$ would give $\delta(q,u) \in S$ and then $\delta(Q,u) \subseteq S$ so that $q \notin \delta(Q,u)$. Hence, since $q \in \delta(Q,u)$, we must have $\delta(q,u) = q$ and thus $u \in \Gamma^*$. This gives $\delta(S,u) = A$, and so $\mathcal{A}_{|S,\Gamma}$ is completely reachable. Complete distinguishability transfers from $\mathcal{A}$ to $\mathcal{A}_{|S}$. Furthermore, all pairs of distinct subsets sharing the common state q are distinguishable, in particular those mentioned in Item 2b.

Conversely, if $\mathcal{A}$ is completely reachable and completely distinguishable, then it is sync-maximal by Theorem 10.3. Next, suppose $Q = S \cup \{q\}$ with $q \notin S$ and that, for $\Gamma = \{x \in \Sigma \mid \delta(q,x) = q\}$, we have

1. $\delta(Q,a) = S$ for some $a \in \Sigma \setminus \Gamma$,
2. for each $p \in Q$, all pairs of non-empty, non-singleton subsets of $\delta^{-1}(\mathrm{dupl}(a),a) \cup \{p\}$ that are both distinct from $\delta^{-1}(\mathrm{dupl}(a),a)$ and contain q are distinguishable,
3. $\mathcal{A}_{|S}$ is completely distinguishable and $\mathcal{A}_{|\Gamma,S}$ is completely reachable.

Let $A \subseteq Q$ with $|A| \geq 2$. As $\mathcal{A}_{|\Gamma,S}$ is completely reachable, there exists $u \in \Gamma^*$ such that $\delta(S,u) = A \setminus \{q\}$. Then we have

$$A = \begin{cases} \delta(Q,au) & \text{if } q \notin A, \\ \delta(Q,u) & \text{if } q \in A. \end{cases}$$

So, every subset of size at least two is reachable in $\mathcal{A}$.

Now, let $A, B \subseteq Q$ be distinct with $|A|, |B| \geq 2$. We can suppose $|A \cap S| \leq 1$ or $|B \cap S| \leq 1$, for if this is not the case, then, since $\mathcal{A}_{|S}$ is completely distinguishable, there exists a word u mapping precisely one of $A \cap S$ or $B \cap S$ to a singleton set. In this case, if u does not distinguish A and B, we may argue instead with $\delta(A,u)$ and $\delta(B,u)$ in what follows. So, without loss of generality, suppose $A = \{p,q\}$ for some $p \in S$. Then we have the following two cases.

$q \notin B$: Then $B \subseteq S$. Since $\mathcal{A}_{|\Gamma,S}$ is synchronizing, there exists $u \in \Sigma^*$ such that $|\delta(B,u)| = 1$. Then $\delta(A,u) = \{\delta(p,u), q\}$ with $\delta(p,u) \in S$ since $p \in S$. We have $q \notin S$ and so u distinguishes A and B.

$q \in B$: Suppose $\delta(A,a) = \delta(B,a)$. Since $A \neq B$ and $|B| \geq 2$, we have $\delta(s,a) \in \{\delta(p,a), \delta(q,a)\}$ for some $s \in B \setminus A$. The letter a has defect one and so there exists a unique pair of states that is mapped to the same state. This pair must be s and p, or s and q, and with $A = \{p,q\}$ we conclude that $B \subseteq \{s,p,q\}$. Since $q \in B$ and $B \neq \{p,q\}$, we have $B = \{s,p,q\}$ or $B = \{s,q\}$. In the latter case, since a

does not map B to a singleton, we have $\delta(s,a) \neq \delta(q,a)$. In all cases, neither A nor B equal $\delta^{-1}(\mathrm{dupl}(a),a)$ and both sets contain q. By assumption, A and B are distinguishable.
Next, suppose $\delta(A,a) \neq \delta(B,a)$. Since a has defect one, it cannot map both subsets to a singleton. Since $\mathcal{A}_{|S}$ is completely distinguishable, there exists a word v distinguishing $\delta(A,a)$ and $\delta(B,a)$. Then av distinguishes A and B.

Thus, $\mathcal{A}$ is completely distinguishable. By Theorem 10.2, Item 3 $\mathcal{A}$ is sync-maximal. □

The condition Theorem 10.7, Item 2b is rather technical. Next, Theorems 10.8 and 10.9 give conditions that imply it.

Corollary 10.8 *Let $\mathcal{A} = (Q,\Sigma,\delta)$ and $S \subseteq Q$ be a proper upper set such that $Q = S \cup \{q\}$ for some $q \in Q$. Set $\Gamma = \{x \in \Sigma \mid \delta(q,x) = q\}$. Suppose the following holds:*

1. *We have $\delta(Q,a) = S$ for some $a \in \Sigma$, and*
2. *for distinct $A,B \subseteq Q$ with $2 \leq |A| \leq 3$, $|B| = 2$ and $q \in A \cap B$ there exists $u \in \Sigma^*$ such that $\delta(A,u) \neq \delta(B,u)$, $|\delta(A,u)| \geq 2$ or $|\delta(B,u)| \geq 2$, and $\delta(A,u) \subseteq S$ or $\delta(B,u) \subseteq S$.*

Then $\mathcal{A}$ is sync-maximal if and only if $\mathcal{A}_{|S}$ is completely distinguishable and $\mathcal{A}_{|\Gamma,S}$ is completely reachable.

Proof. By Theorem 10.2 we can assume $|Q| \geq 3$. The set S is the unique sink component in $\mathcal{A}$. Thus, it equals the set of maximal states.

First, suppose $\mathcal{A}$ is sync-maximal. By Theorem 7.11, S equals the set of synchronizing states. By Theorem 10.7, $\mathcal{A}_{|S}$ is completely distinguishable and $\mathcal{A}_{|\Gamma,S}$ is completely reachable.

Conversely, suppose $\mathcal{A}_{|S}$ is completely distinguishable and $\mathcal{A}_{|\Gamma,S}$ is completely reachable. Then $\mathcal{A}$ is synchronizing and, by Theorem 7.11, S is the set of synchronizing states. Let $p \in S$ and $A,B \subseteq \delta^{-1}(\mathrm{dupl}(a),a) \cup \{p\}$ be distinct with $|A|,|B| \geq 2$ and $q \in A \cap B$. By assumption, we have $\delta(A,u) \neq \delta(B,u)$ such that, without loss of generality, $\delta(A,u) \subseteq S$ for some $u \in \Sigma^*$. If $|\delta(A,u)| = 1$ or $|\delta(B,u)| = 1$, then u distinguishes A and B. Otherwise, for $\delta(B,u) \subseteq S$ there exists a distinguishing word $v \in \Sigma^*$ for $\delta(A,u)$ and $\delta(B,u)$. Then uv distinguishes A and B. For $q \in \delta(B,u)$, we find $w \in \Gamma^*$ such that $|\delta(A,uw)| = 1$ since $\mathcal{A}_{|\Gamma,S}$ is completely reachable. We have $\delta(q,w) = q$ and $\delta(\delta(B,u) \cap S,w) \neq \emptyset$. This yields $|\delta(B,uw)| \geq 2$. Then, uw distinguishes A and B. Thus, Theorem 10.7, Item 2b holds and so we conclude that $\mathcal{A}$ is sync-maximal. □

Next, we list conditions that imply Item 2 from Theorem 10.8 for specific sets.

Proposition 10.9 *Let $\mathcal{A} = (Q,\Sigma,\delta)$. Suppose $S \subseteq Q$ is a proper upper set such that $\delta(Q,a) = S$ and $Q = S \cup \{q\}$ for some $a \in \Sigma$ and some $q \in Q$. Set $\Gamma = \{x \in \Sigma \mid \delta(q,x) = q\}$.*

1. *Let $A,B \subseteq Q$ be distinct with $|A|,|B| \geq 2$ and $q \in A \cap B$. If $\delta(A,ua) \neq \delta(B,ua)$ for some $u \in \Gamma^*$, then $|\delta(A,ua)| \geq 2$ or $|\delta(B,ua)| \geq 2$.*

2. *If $\delta(S,a) = S$, then $\delta(A,a) \neq \delta(B,a)$ for all distinct $A, B \subseteq Q$ with $|A|, |B| \geq 2$ and $q \in A \cap B$.*

Proof. 1: The set S is an upper set and the sets $A \cap S$ and $B \cap S$ are non-empty. Thus, we have that $\delta(A,u) \cap S$ and $\delta(B,u) \cap S$ are non-empty. We also have $q \in \delta(A,u) \cap \delta(B,u)$ since $\delta(q,u) = q$. Thus $|\delta(A,u)| \geq 2$ and $|\delta(B,u)| \geq 2$. We have $\delta(A,u) \neq \delta(B,u)$ since $\delta(A,ua) \neq \delta(B,ua)$. Let $p \in \delta(A,u) \cap S$ and $r \in \delta(B,u) \cap S$ with $p \neq r$. If $|\delta(A,ua)| = |\delta(B,ua)| = 1$, then $\delta(p,a) = \delta(q,a)$ and $\delta(r,a) = \delta(q,a)$, which is not possible as a has defect one. Thus, a can map at most one of both subsets $\delta(A,u)$ or $\delta(B,u)$ to a singleton set, i.e., we have $|\delta(A,ua)| \geq 2$ or $|\delta(B,ua)| \geq 2$.

2: The letter a acts injectively on S. Thus, we must have $q \in \delta^{-1}(\mathrm{dupl}(a), a)$ and $\delta(A \cap S, a) \neq \delta(B \cap S, a)$ where $A \cap S$ and $B \cap S$ are non-empty. In either case, whether $\delta(q,a) \in \delta(A \cap S, a) \cup \delta(B \cap S, a)$ or not, we find $\delta(A,a) \neq \delta(B,a)$. □

Corollary 10.10 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton. Then $\mathcal{A}$ is sync-maximal if and only if it is completely reachable and completely distinguishable or we have $Q = S \cup \{q\}$ for some upper set $S \subseteq Q$ and $q \in Q \setminus S$ such that $\delta(Q,a) = S$ for some $a \in \Sigma$ and $\mathcal{A}_{|\Gamma,S}$ is completely reachable and completely distinguishable for $\Gamma = \{x \in \Sigma \mid \delta(q,x) = q\}$.*

Proof. First, note that $\mathcal{A}_{|S}$ is completely distinguishable if and only if $\mathcal{A}_{|\Gamma,S}$ is completely distinguishable since those letters in $\Sigma \setminus \Gamma$ only move q and fix all other states.

The letter a is a simple idempotent with $\delta(q,a) \neq q$. Thus $\delta(S,a) = S$. Then the claim follows from Theorems 10.7 to 10.9. □

Corollary 10.11 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with at least five states and $S \subseteq Q$ be a proper upper set such that $\delta(Q,a) = S$ and $Q = S \cup \{q\}$ for some $a \in \Sigma$ and some $q \in Q$. Suppose the set S is cyclically permuted by some letter $b \in \Gamma$. Set $\Gamma = \{x \in \Sigma \mid \delta(q,x) = q\}$. Then $\mathcal{A}$ is sync-maximal if and only if $\mathcal{A}_{|S}$ is completely distinguishable and $\mathcal{A}_{|\Gamma,S}$ is completely reachable.*

Proof. Let $A, B \subseteq Q$ be distinct with $2 \leq |A| \leq 3$, $|B| = 2$, and $q \in A \cap B$. Set $T = \delta^{-1}(\mathrm{dupl}(a), a)$. Suppose $\delta(A,a) = \delta(B,a)$. Then $A \setminus T = B \setminus T$ and, since $A \neq B$, we also have $A \cap T \neq \emptyset$ and $B \cap T \neq \emptyset$ where $A \cap T \neq B \cap T$. Set $n = |Q|$. Suppose $q \in T$. Let $T = \{p, q\}$. Then $A = \{s, p, q\}$ for some $s \in Q \setminus T$ and $B = \{s, q\}$. Let $\delta(s, b^i) = p$ for some $0 < i < n$. If $i > 1$, set $u = b$ and, otherwise, set $u = bb$. We have $\delta(A,u) \cap T = \{q\}$ and $\delta(A,u) \setminus T \neq \delta(B,u) \setminus T$. Thus $\delta(A,ua) \neq \delta(B,ua)$. Next, suppose $q \notin T$. Then $B = \{p, q\}$ for some $p \in T$. Let $u = b$ if $\delta(B,b) \cap T = \emptyset$ or $u = bb$ otherwise. Then $\delta(B,u) \cap T = \emptyset$. We have $\delta(A,u) \setminus T \neq \delta(B,u) \setminus T$. Thus $\delta(A,ua) \neq \delta(B,ua)$. Since the image of a is S, by Theorems 10.8 and 10.9 we obtain the claim. □

Next, we list three corollaries to Theorem 10.7. The first three follow readily, the last one uses Theorem 9.19.

Corollary 10.12 *In a sync-maximal automaton $\mathcal{A}$ with set of synchronizing states S, we have $|Q \setminus S| \leq 1$ and $\mathcal{A}_{|S}$ is completely reachable and completely distinguishable.*

Corollary 10.13 *A sync-maximal automaton with at least three states has no trap state.*

Corollary 10.14 *A sync-maximal automaton is a ρ-automaton.*

Corollary 10.15 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be sync-maximal with $|Q| \geq 3$. If $\mathcal{A}$ is strongly connected, then $|\Sigma| \geq 2$ where Σ contains a letter of defect one and for $|\Sigma| = 2$ the other letter cyclically permutes the states. If $\mathcal{A}$ is not strongly connected, then there exists a non-synchronizing state $q \in Q$ such that $Q = S \cup \{q\}$ for the set of synchronizing states S and with $\Gamma = \{x \in \Sigma \mid \delta(q, x) = q\}$ we have:*

1. *each of the sets $\Sigma \setminus \Gamma$ and Γ contains a letter of defect one,*
2. *$|\Gamma| \geq 2$, $|\Sigma| \geq 3$,*
3. *if $|\Gamma| = 2$ and $|Q| \geq 4$, then Γ also contains a permutational letter that cyclically permutes the states in S (this might not hold for $|Q| \leq 3$, see Figure 10.2c).*

Proof. First, assume $\mathcal{A}$ is strongly connected. By Theorem 10.3, $\mathcal{A}$ is completely reachable. So, Theorem 9.20, we find that $|\Sigma| \geq 2$, Σ contains a letter of defect one, and for $|\Sigma| = 2$ another letter cyclically permuting the states. Now, assume $\mathcal{A}$ is not strongly connected. By Theorem 10.7, Item 2, we have $Q = S \cup \{q\}$ for some state $q \in Q \setminus S$ where S is the set of synchronizing states, that $\Sigma \setminus \Gamma$ contains a letter of defect one, and that for $\Gamma = \{x \in \Sigma \mid \delta(q, x) = q\}$ the automaton $\mathcal{A}_{|\Gamma,S}$ is completely reachable. Applying Theorem 9.20 to the completely reachable automaton $\mathcal{A}_{|\Gamma,S}$ yields the existence of a letter in Γ of defect one, $|\Gamma| \geq 2$, and so $|\Sigma| \geq 3$, and that for $|\Gamma| = 2$ and $|S| \geq 3$ the other letter in Γ cyclically permutes S. □

10.3 Examples

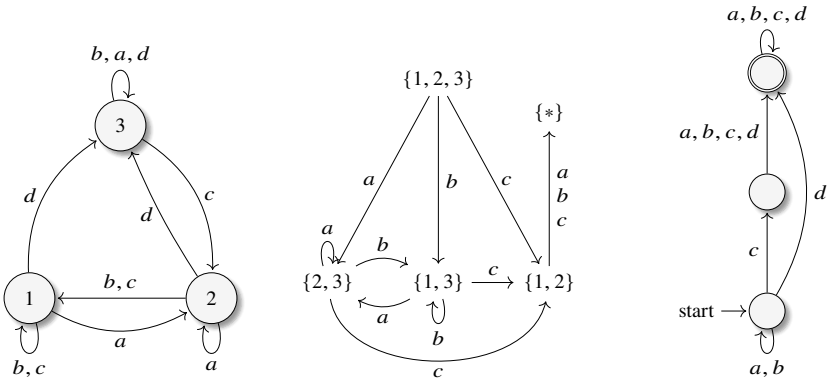
Theorems 9.51 to 9.53 gave completely distinguishable automata that are not sync-maximal (which is easily seen with Theorem 10.7 in each case). The automaton from Theorem 9.58 is completely reachable, but not sync-maximal. Here, Theorem 10.16 gives another completely reachable automaton that is not sync-maximal. Then, Theorem 10.17 gives two sync-maximal automata, derived from the Černý automata, that are not completely reachable. Theorem 10.18 gives a three-state sync-maximal ternary automaton without permutational letters, which is not possible for more states by Theorem 10.15. Theorem 10.19 gives an automaton that is not sync-maximal but where all conditions from Theorem 10.7, Item 2 except Theorem 10.7, Item 2b are fulfilled.

Example 10.16 The automaton (see Figure 10.1) with the state set $Q = \{1, 2, 3\}$, input letters a, b, c, d and transition function given by

$$\delta(i, a) = \begin{cases} 2 & \text{if } i = 1, 2, \\ 3 & \text{if } i = 3; \end{cases} \qquad \delta(i, b) = \begin{cases} 1 & \text{if } i = 1, 2, \\ 3 & \text{if } i = 3; \end{cases}$$

$$\delta(i, c) = \begin{cases} 1 & \text{if } i = 1, 2, \\ 2 & \text{if } i = 3; \end{cases} \qquad \delta(i, d) = 3 \text{ for all } i = 1, 2, 3.$$

taken from [99, Ex. 2] is completely reachable. However, it is not sync-maximal as, for example, the two 2-sets $\{1, 3\}$ and $\{2, 3\}$ are not distinguishable.



(a) Non-sync-maximal completely reachable automaton from [99, Ex. 2].

(b) Power automaton, where singleton sets are merged, the letter d, as it always ends in $\{3\}$, and the empty set are omitted.

(c) Minimal automaton accepting set of synchronizing words, obtained from Figure 10.1b by merging $\{2, 3\}$,$\{1, 3\}$, and $\{1, 2, 3\}$.

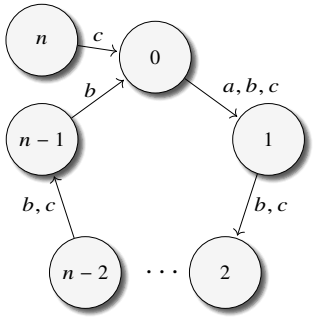
Fig. 10.1: Completely reachable automaton that is not sync-maximal [99, Ex. 2].

Example 10.17 Let $\mathcal{A} = (Q, \{a, b, c\}, \delta)$ be one of the automata from Figures 10.2a and 10.2b with $n \geq 5$. The set $S = \{0, 1, \ldots, n-1\}$ is an upper set and $\mathcal{A}_{|\{a,b\},S}$ equals the Černý automaton, which is completely reachable and completely distinguishable by Theorem 10.4. Next, consider distinct $A, B \subseteq Q$ with $n \in A \cap B$. Without loss of generality, $q \in A \setminus B$ for some $q \in Q$. Then there exists $i \geq 0$ such that $1 \in \delta(A, b^i) \setminus \delta(B, b^i)$ and so $\delta(A, b^i c) \neq \delta(B, b^i c)$. Also, we have $\delta(Q, c) = S$. So, by Theorem 10.8 we obtain that $\mathcal{A}$ is sync-maximal.

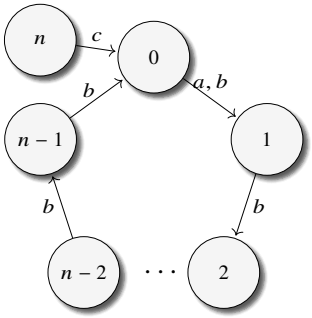
Example 10.18 By Theorem 10.15, Item 3 a ternary sync-maximal automaton with at least four states has a permutational letter. Figure 10.2c gives a three-state sync-maximal automaton over a ternary alphabet with no permutational letter.

Example 10.19 Here, we construct automata that are not strongly connected and where all conditions from Theorem 10.7, Item 2b except Theorem 10.7, Item 2b are fulfilled; Figure 10.3 illustrates the construction in the case of four states. Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q = \{0, 1, \ldots, n\}$ for $n \geq 3$ and where, for $S = Q \setminus \{0\}$, we have $\Sigma = \{a\} \cup \Gamma$ with $\Gamma = \{b_A \mid \emptyset \neq A \subsetneq S\} \cup \{c_q \mid q \in S \setminus \{1\}\}$ and

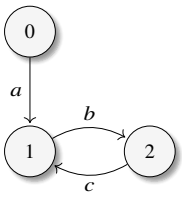
1. $\delta(q, a) = \min(q + 1, n)$ for $q \in Q$,
2. for $A \subseteq S$ with $A = \{p_1, \ldots, p_m\}$ where $p_1 < \ldots < p_m$ and $1 \leq m < n$ we have $\delta(q, b_A) = p_{\min(q,m)}$ for $q \in S$ and $\delta(0, b_A) = 0$,



(a) Sync-maximal, but not completely reachable.



(b) Sync-maximal, but not completely reachable.



(c) Sync-maximal automaton with no permutational letters.

Fig. 10.2: Different sync-maximal automata that are not completely reachable, self-loops omitted. Figures 10.2a to 10.2b are derived from the Černý automata by adding a "dangling" state, see Theorem 10.17.

3. for $q \in S \setminus \{1\}$ and $p \in Q$ we have

$$\delta(p, c_q) = \begin{cases} 0 & \text{if } p = 0, \\ n & \text{if } p \notin \{0, q\}, \\ n-1 & \text{if } p = q. \end{cases}$$

Now, the letters in Γ are precisely those fixing 0, we have $\delta(Q, a) = S$, S equals precisely the set of synchronizing states and $\mathcal{A}_{|\Gamma,S}$ is completely reachable as for each $A \subseteq Q$ with $1 \leq |A| < n$ we have $\delta(S, b_A) = A$. Furthermore, as $\delta(S \setminus \{1\}, b_{\{1,2\}}) = \{2\}$ and $\delta(S, b_{\{1,2\}}) = \{1, 2\}$, and $\delta(S \setminus \{q\}, c_q) = \{n\}$ and $\delta(S, c_q) = \{n, n-1\}$ for $q \in \{2, \ldots, n\}$, $\mathcal{A}_{|\Gamma,S}$ is completely distinguishable by Theorem 9.46 4. Next, we have $\delta(\{0, n-1\}, c_n) = \{0, n\}$ and $\delta(\{0, n\}, c_n) = \{0, n-1\}$, $\delta(\{0, n-1\}, c_{n-1}) = \{0, n-1\}$ and $\delta(\{0, n\}, c_{n-1}) = \{0, n\}$, for $q \notin \{n-1, n\}$ we have $\delta(\{0, n-1\}, c_q) = \delta(\{0, n\}, c_q) = \{0, n\}$, $\delta(\{0, n-1\}, a) = \delta(\{0, n\}, a) = \{1, n\}$ and, for $\emptyset \neq A \subsetneq S$, we have

$$\delta(\{0, n-1\}, b_A) = \delta(\{0, n\}, b_A) = \{0, \max(A)\}.$$

So, $\{0, n-1\}$ and $\{0, n\}$ are not distinguishable in $\mathcal{A}$ and Theorem 10.7, Item 2b is the only condition from Theorem 10.7 that fails.

10.4 Discussion

Theorem 10.7 and Theorem 10.15 gave structural results linking the class of sync-maximal automata to completely reachable and completely distinguishable automata. Later, in Section 12.4, we will give more precise structural results for classes of automata. For example, an automaton whose transformation semigroup contains a transi-

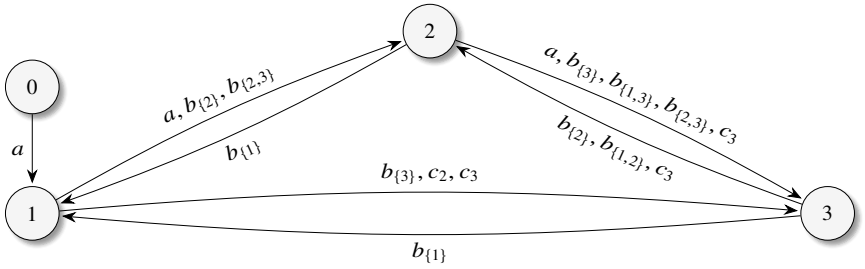


Fig. 10.3: Automaton (self-loops omitted) according to Theorem 10.19.

tive group of units and where every letter has defect at most one is synchronizing if and only if it is sync-maximal, see Section 12.3. This statement applies to the the Černý automata from Figure 1.7.

We note that deciding sync-maximality is PSPACE-complete, see the discussion in Section 13.4.

By Theorem 7.1, languages of the form $\mathrm{Syn}(\mathcal{A})$ are precisely the regular ideal languages. Thus, shortest words in regular ideal languages corresponds to shortest paths in accepting automata for sets of synchronizing words. Now, investigating the length of the shortest words in regular ideal languages as a language-theoretic approach to the Černý conjecture has been pursued and extended by several authors [14, 197, 228, 442, 501, 506, 513], based on a method introduced by Rodero et al. [442, 506, 513] that even associates a strongly connected synchronizing automaton with each regular ideal language. Maslennikova [440, 441] introduced the *reset complexity* of a regular ideal language $I \subseteq \Sigma^*$ as the number of states of a smallest automaton $\mathcal{A}$ such that $I = \mathrm{Syn}(\mathcal{A})$. For a sync-maximal automaton $\mathcal{A}$, the gap between the size of an accepting automaton for $\mathrm{Syn}(\mathcal{A})$, i.e., the state complexity of this set, and the size of an automaton having $\mathrm{Syn}(\mathcal{A})$ as its set of synchronizing words, i.e., its reset complexity, is maximal possible (note that in this case, no automaton with fewer states than $\mathcal{A}$ can have $\mathrm{Syn}(\mathcal{A})$ as its set of synchronizing words). In this sense, for sync-maximal automata the complexity of the set of synchronizing words is maximal.

Chapter 11
Graph-Theoretical Constructions

Here, we introduce digraphs associated to automata and transformation semigroups that we use to deduce complete reachability and complete distinguishability. Section 11.1, 11.2, and 11.3 review existing constructions from the literature. We introduce the Bondar-Casas-Volkov graphs, which apply to general automata, and the Rystsov graphs, which only apply to SI-automata, from the literature. Section 11.4 extends these graphs to arbitrary transformation semigroups. Then we investigate the influence of the group of units on the introduced digraphs (Section 11.6), relations between them (Section 11.5), give examples that highlight their differences (Section 11.7), and finally formulate sufficient conditions in terms of connectedness notions to deduce complete reachability, complete distinguishability, or lower bounds for the size of accepting automata of sets of synchronizing words (Section 11.8). In Section 11.8, we also point out some relations to congruences and minimal automata.

11.1 The Bondar-Casas-Volkov Graphs

Bondar et al. [101] introduced a graph construction to characterize complete reachability that we shortly present here (but in a condensed and slightly modified form). Another slightly different construction appeared in [100] for the same purpose. For a more detailed discussion we refer to [101].

Definition 11.1 (Bondar et al. [100, 101]) Let $\mathcal{A} = (Q, \Sigma, \delta)$ and set $n = |Q|$. Define the digraph $\Gamma_0(\mathcal{A})$ with vertex set $Q_0 = \{\{q\} \mid q \in Q\}$ and no edges. Then, for $0 < k < n$ define, inductively, digraphs $\Gamma_k(\mathcal{A}) = (Q_k, E_k)$ with vertices $Q_k \subseteq \mathcal{P}(Q)$ and edges $E_{\leq k} \subseteq Q_k \times Q_k$ by setting[1]

$$Q_k = \left\{ \bigcup_{A \in S} A \mid S \text{ is a strongly connected component in } \Gamma_{k-1}(\mathcal{A}) \right\},$$

[1] The union over all sets inside a set S is sometimes also written as $\bigcup S$ instead of $\bigcup_{A \in S} A$, especially in axiomatic treatments of set theory w.r.t. the axiom of union from Zermelo–Fraenkel set theory.

and $E_{\leq k} = E_k \cup I$ where

$$E_k = \{(C, C') \in Q_k \times Q_k \mid C \neq C',\ C \supseteq \mathrm{excl}(w),\\ C' \cap \mathrm{dupl}(w) \neq \emptyset \text{ for some } w \in \Sigma^* \text{ with } \mathrm{def}(w) = k\},$$
$$I = \{(C, C') \in Q_k \times Q_k \mid \exists (D, D') \in E_{k-1} : D \cap C \neq \emptyset \wedge D' \cap C' \neq \emptyset\}.$$

We call the edges in E_k the edges *induced*, or *forced, by words of defect k* and the edges in I the edges *inherited from* Γ_{k-1}.

Unraveling the definition, the elements in Q_1 are singleton sets and so can be identified with the elements in Q. With respect to this identification, which we will use in the following without special mention, we have $Q_1 = Q$ and

$$E_1 = \{(p, q) \mid \exists u \in \Sigma^* \text{ with } \mathrm{excl}(u) = \{p\} \text{ and } \mathrm{dupl}(u) = \{q\}\}. \tag{11.1}$$

As an example for the construction, taken from [101], consider the automaton $\mathcal{E}_5$ with the state set $\{1, 2, 3, 4, 5\}$ and eight input letters, denoted $a_{[1]}$, $a_{[2]}$, $a_{[3]}$, $a_{[4]}$, $a_{[5]}$, $a_{[1,2]}$, $a_{[4,5]}$, and $a_{[1,3]}$ from Figure 11.1. The transition function of $\mathcal{E}_5$ is given in Table 11.1.

Table 11.1: The transition table of the automaton $\mathcal{E}_5$

q	$\delta(q, a_{[1]})$	$\delta(q, a_{[2]})$	$\delta(q, a_{[3]})$	$\delta(q, a_{[4]})$	$\delta(q, a_{[5]})$	$\delta(q, a_{[1,2]})$	$\delta(q, a_{[4,5]})$	$\delta(q, a_{[1,3]})$
1	2	1	1	1	1	3	1	4
2	2	1	1	2	2	3	1	4
3	3	3	2	3	3	3	2	4
4	4	4	4	5	4	4	3	5
5	5	5	5	5	4	5	3	5

The stages of the construction of $\Gamma_k(\mathcal{E}_5)$ are shown in Figure 11.2. For example, to construct $\Gamma_1(\mathcal{E}_r)$, one has to compute the transformations caused by words of defect one with respect to $\mathcal{E}_5$. A word of defect one in $\mathscr{E}_5$ must be a product of the defect-one letters $a_{[1]}$, $a_{[2]}$, $a_{[3]}$, $a_{[4]}$, $a_{[5]}$. The 'multiplication table' for these words is given next, in which $*$ means that the defect of the product is larger than 1:

	$a_{[1]}$	$a_{[2]}$	$a_{[3]}$	$a_{[4]}$	$a_{[5]}$
$a_{[1]}$	$a_{[1]}$	$a_{[2]}$	$a_{[3]}$	$*$	$*$
$a_{[2]}$	$a_{[1]}$	$a_{[2]}$	$a_{[3]}$	$*$	$*$
$a_{[3]}$	$*$	$*$	$*$	$*$	$*$
$a_{[4]}$	$*$	$*$	$*$	$a_{[4]}$	$a_{[5]}$
$a_{[5]}$	$*$	$*$	$*$	$a_{[4]}$	$a_{[5]}$

The graph $\Gamma_1(\mathcal{A})$ was introduced in [99] to give sufficient condition for complete reachability. Later, all the graphs $\Gamma_k(\mathcal{A})$ were introduced in [100, 101] to characterize complete reachability as stated in the following theorem.

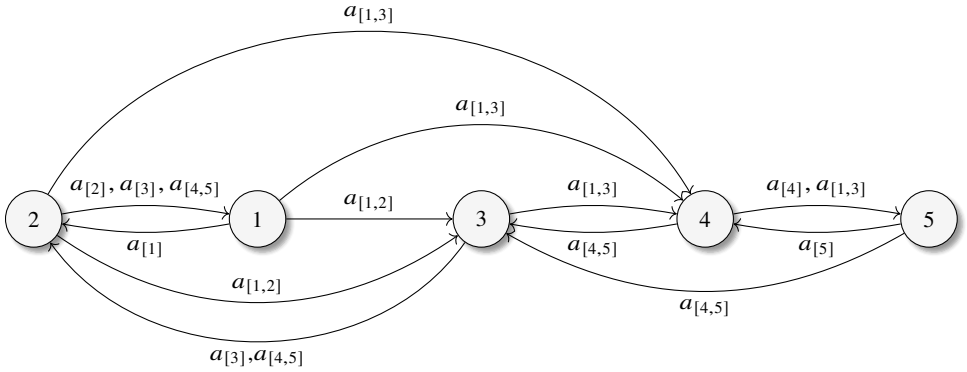
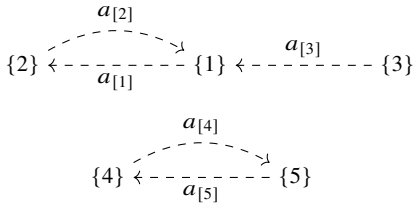
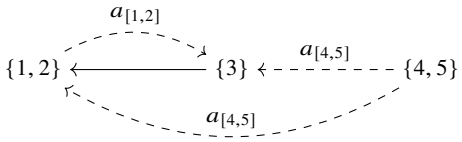


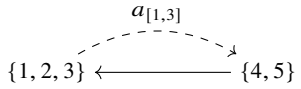
Fig. 11.1: The automaton $\mathcal{E}_5$ from [101] with self-loops omitted. Here $\mathrm{excl}(a_{[i]}) = \{i\}$ and $\mathrm{excl}(a_{[i,j]}) = \{i, i+1, \ldots, j\}$ for $1 \le i < j \le 5$.



(a) The graph $\Gamma_1(\mathcal{A})$.



(b) The graph $\Gamma_2(\mathcal{A})$.



(c) The graph $\Gamma_3(\mathcal{A})$.

Fig. 11.2: The graphs $\Gamma_k(\mathcal{A})$ for $k \in \{1,2,3\}$. Dashed edges are induced by words of defect 1, 2, or 3 written above the edge. Solid edges are inherited from the previous graph.

Theorem 11.2 (Bondar et al. [100, 101]) *The automaton $\mathcal{A}$ is completely reachable if and only if there exists $0 < k < n$ such that $\Gamma_k(\mathcal{A})$ is strongly connected (and in this case every non-empty subset is reachable by a product of words of defect at most k).*

The following proposition states that the vertices of each $\Gamma_k(\mathcal{A})$ partition Q and the partition given by Q_{k-1} is coarser than the partition given by Q_k for $k \ge 1$.

Proposition 11.3 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\Gamma_k(\mathcal{A}) = (Q_k, E_k)$ for $k \ge 0$. Suppose $D, D' \in Q_k$ and $C \in Q_{k+1}$. Then the following is true.*

1. *$D = D'$ or $D \cap D' = \emptyset$.*
2. *$D \cap C \ne \emptyset$ if and only if $D \subseteq C$.*

Proof. 1: For $k = 0$ this is obvious. So, assume $k \geq 1$. Suppose there exists $p \in D \cap D'$ and let $q \in D$. By definition, we have $p \in B$, $p \in B'$ and $q \in B''$ for $B, B', B'' \in Q_{k-1}$ with $B \subseteq D$, $B' \subseteq D'$, $B'' \subseteq D$ and where B and B'' are in the same strongly connected component in $\Gamma_{k-1}(\mathcal{A})$. Inductively, as $p \in B \cap B'$, we find $B = B'$. So, B' and B'' are in the same strongly connected component in $\Gamma_{k-1}(\mathcal{A})$ and as $B' \subseteq D'$ we get $B'' \subseteq D'$ so that $q \in D'$. Similarly, we find $D' \subseteq D$.

2: Suppose $q \in D \cap C$ for some $q \in Q$. As $q \in C$ there exists $D'' \in Q_{k-1}$ with $q \in D''$ and $D'' \subseteq C$. By Item 1 we get $D = D''$, which gives the claim. □

By Item 2 of Theorem 11.3 the set I in Definition 11.1 can also be defined by the conditions $D \subseteq C$ and $D' \subseteq C'$.

Lastly, we introduce a variant of $\Gamma_1(\mathcal{A})$ that we need later to establish bounds on shortest words reaching non-empty subsets in classes of automata.

For $i \geq 1$, let $\Gamma_{1,i}(\mathcal{A})$ be the subgraph of $\Gamma_1(\mathcal{A})$ whose edges are all induced be words of defect one of the form au with $a \in \Sigma$ being a letter of defect one and u being a permutational word with $|u| < i$, i.e., its edge set has the form

$$\{(p, q) \mid \operatorname{excl}(au) = \{p\} \text{ and } \operatorname{dupl}(au) = \{q\} \text{ for some } a \in \Sigma \text{ of defect one and a permutational word } u \in \Sigma^{<i}\}$$

Theorem 11.4 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton. Suppose $\Gamma_{1,i}(\mathcal{A})$ is strongly connected for $i \geq 1$. Then $\mathcal{A}$ is completely reachable, every non-empty subset of size r is reachable by a word of length at most $i \cdot (n - r)$ and a shortest synchronizing word has length at most $1 + i \cdot (n - 2)$.*

Proof. We can assume $n \geq 2$. Let $S \subseteq Q$ be a non-empty r-subset. First, we prove that S is reachable by a word of length at most $i \cdot (n - r)$ by induction on $k = |Q \setminus S|$. If $k = 0$, then $S = Q$ and so S is reachable via the empty word and the claim holds. Now let $k > 0$ so that S is a proper subset of Q. Since $\Gamma_{1,i}(\mathcal{A})$ is strongly connected, by Theorem 3.67 it has an edge (p, q) with $p \in Q \setminus S$ and $q \in S$. By the definition of $\Gamma_{1,i}(\mathcal{A})$, there exists a word of defect one of the form au for some letter of defect one $a \in \Sigma$ and a permutational word u with $|u| < i$ such that $\operatorname{excl}(au) = \{p\}$ and $\operatorname{dupl}(au) = \{q\}$. Since the excluded p of au does not belong to S we have $|\delta^{-1}(s, au)| \geq 1$ for each $s \in S$ where $|\delta^{-1}(q, au)| = 2$. Thus for $T = \delta^{-1}(S, w)$ we have $|T| = |S| + 1$ and $\delta(T, au) = S$. Inductively, T is reachable by a word w of length at most $i \cdot (n - |T|)$. Then $\delta(Q, wau) = S$ and $|wau| \leq i \cdot (n - |T|) + i = i \cdot (n - |S|)$. This gives the first part of the claim by induction.

Lastly, since $\mathcal{A}$ is synchronizing, there exist distinct states $p, q \in Q$ and a letter $a \in \Sigma$ with $\delta(p, a) = \delta(q, a)$. Now the set $\{p, q\}$ is reachable by a word w of length at most $i \cdot (n - 2)$. Then the word wa synchronizes $\mathcal{A}$ and has length at most $1 + i \cdot (n - 2)$. □

11.2 Augmentative Sets of Words

Let $\mathcal{A} = (Q, \Sigma, \delta)$. A subset $W \subseteq \Sigma^*$ is called *augmentative (for $\mathcal{A}$)* if for every non-empty proper subset $S \subseteq Q$ there exists $u \in W$ such that $|\delta^{-1}(S, u)| > |S|$. Let $W \subseteq \Sigma^*$ be augmentative. Since the number of subsets of states is finite, every augmentative subset contains a finite augmentative subset. Hence, we can assume W is finite. Clearly, if $\mathcal{A}$ admits an augmentative subset, then it is synchronizing and there exist $q \in Q$ and $a \in \Sigma$ so that $|\delta^{-1}(\{q\}, a)| \geq 2$. Then, by using W to successively enlarge this subset, we obtain the following bound (see also [531, Thm. 2]):

$$\begin{aligned} \operatorname{rt}(\mathcal{A}) &\leq \max\{|u| : u \in W\} \cdot (n - |\delta^{-1}(\{q\}, a)|) + 1 \\ &\leq \max\{|u| : u \in W\} \cdot (n - 2) + 1. \end{aligned} \tag{11.2}$$

Obtaining bounds for the reset length by using an augmentative set with Equation (11.2) is called the *bottom-up* (or *extension*) *method* [80].

The next statement from Rystsov [531, 535] characterizes automata admitting an augmentative subset as the strongly connected synchronizing automata.

Theorem 11.5 (Rystsov [531, 535]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then $\mathcal{A}$ is strongly connected and synchronizing if and only if $\mathcal{A}$ admits an augmentative subset $W \subseteq \Sigma^*$.*

Proof. First, assume $\mathcal{A}$ is strongly connected and synchronizing. Then for every $q \in Q$ there exists $u_q \in \Sigma^*$ such that $\delta(Q, u_q) = \{q\}$. In this case, since for a proper and non-empty $S \subseteq Q$ and $q \in S$ we have $|\delta^{-1}(S, u_q)| = |Q| > |S|$, the set $W = \{u_q \mid q \in Q\}$ is augmentative. Conversely, assume $W \subseteq \Sigma^*$ is augmentative. Then, every state $q \in Q$ is synchronizing as we can enlarge $\{q\}$ successively to obtain the whole state set. So, by Corollary 7.14, $\mathcal{A}$ is strongly connected. □

Recall that, for an OI-automaton $\mathcal{A}$, Σ_I denotes the set of idempotent one-point singular letters and Σ_G the set of permutational letters. Rystsov [535, Lem. 2] has shown the following for SI-automata. The proof works for OI-automata without modification. We give it for the convenience of the reader.

Proposition 11.6 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an OI-automaton and $S \subseteq Q$. If $u \in \Sigma^*$ is a shortest word such that $|S| < |\delta^{-1}(S, u)|$, then $u \in \Sigma_I \Sigma_G^*$.*

Proof. We can write $u = vaw$ with $v \in \Sigma^*$, $a \in \Sigma_I$ and $w \in \Sigma_G^*$. Assume $|v| \geq 1$. Then, by the minimal choice of u, we have $|\delta^{-1}(S, aw)| \leq |S|$. Set $T = \delta^{-1}(S, w)$. As w permutes Q, we have $|T| = |S|$. By Equation (6.1) we have $\delta^{-1}(S, aw) = \delta^{-1}(T, a)$. Thus we find $|\delta^{-1}(T, a)| \leq |T|$. Hence T cannot be properly contained in $\delta^{-1}(T, a)$. By Theorem 4.34, Item 10, this gives $\delta^{-1}(T, a) \subseteq T$. Then, with Equation (6.1), we find

$$\delta^{-1}(S, vaw) = \delta^{-1}(\delta^{-1}(T, a), v) \subseteq \delta^{-1}(T, v) = \delta^{-1}(S, vw),$$

which gives $|S| < |\delta^{-1}(S, vw)|$, contradicting the minimal choice of u. So $v = \varepsilon$ and this gives $u \in \Sigma_I \Sigma_G^*$. □

The next corollaries we give by Rystsov[535, 536] for SI-automata, where Theorem 11.9 was only stated for the case that the group of units is regular.

Corollary 11.7 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an OI-automaton. If there exists an augmentative subset, then there exists an augmentative subset in $\Sigma_I \Sigma_G^*$.*

Proof. For each non-empty and proper subset $S \subseteq Q$ let $u_S \in \Sigma^*$ be a shortest words such that $|\delta^{-1}(S, u_S)| > |S|$. The set of all these words u_S for some non-empty and proper $S \subseteq Q$ is augmentative and, by Theorem 11.6, a subset of $\Sigma_I \Sigma_G^*$. □

The next bound also follows from Theorem 7.35 for SI-automata and is tight even for SI-automata.

Corollary 11.8 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state synchronizing OI-automaton without permutational letters. Then a shortest synchronizing word has length at most $n - 1$.*

Proof. Since $\mathcal{A}$ is synchronizing, there exist $q \in Q$ and $u \in \Sigma^*$ such that $\delta^{-1}(q, u) = Q$. Set $S_1 = \{q\}$ and, inductively, construct a sequence $S_1, \ldots, S_m$ with $m < |Q|$ such that $S_1 \subseteq \cdots \subseteq S_m$ with $S_m = Q$ and where, for each $i \in \{2, \ldots, m\}$,

1. $S_i = \delta^{-1}(S_{i-1}, a_i)$ for some $a_i \in \Sigma_I$ and
2. $|S_i| > |S_{i-1}|$

in the following way: Let $i \in \{2, \ldots, m\}$. Let $i \in \{2, \ldots, m\}$. Since $q \in S_{i-1}$ and $S_{i-1} \neq Q$ we have $|\delta^{-1}(S_{i-1}, u)| > |S_{i-1}|$. Thus there exists a shortest $v \in \Sigma^*$ such that $|\delta^{-1}(S_{i-1}, v)| > |S_{i-1}|$. By Theorem 11.6 we have $v \in \Sigma_I \Sigma_G^*$. Since Σ_G is empty, this yields $v = a_i$ for some $a_i \in \Sigma_I$. Set $S_i = \delta^{-1}(S_{i-1}, a_i)$. By Theorem 4.34, Item 10, we find $S_{i-1} \subseteq S_i$. By finiteness the sets S_i cannot grow indefinitely. Thus, this process must come to a halt after at most m steps with $m < |Q|$ and $S_m = Q$. The word $a_m \cdots a_1$ synchronizes $\mathcal{A}$. □

Corollary 11.9 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a strongly connected and synchronizing OI-automaton. If the group of units of $\mathcal{A}$ is a semiregular permutation group, then* $\mathrm{rt}(\mathcal{A}) \leq (n-1)^2$ *and this bound is tight.*

Proof. By Theorem 6.2 for an OI-automaton the group of units is $G = \{\delta_u \mid u \in \Sigma_G^*\}$. For $g \in G$ let $u \in \Sigma_G^*$ be a shortest permutational word such that $g = \delta_u$. Assume $|u| \geq n$. Then $\delta(q, u_1) = \delta(q, u_1 u_2) = \delta(\delta(q, u_1), u_2)$ for some $u_1, u_2, u_3 \in \Sigma_G^*$ with $u = u_1 u_2 u_3$, $|u_2| > 0$, and $|u_1| < n$. So δ_{u_2} has the fixpoint $\delta(q, u_2)$, which gives $\delta_{u_1} = \mathrm{id}_Q$ as G is semiregular. Thus $g = \delta_{u_1 u_2}$, contradicting the minimal choice of u. Thus $|u| < n$, which gives $G = \{\delta_u \mid u \in \Sigma_G^{<n}\}$. This yields that every transformation induced by a word in $\Sigma_I \Sigma_G^*$ is also induced by a word in $\Sigma_I \Sigma_G^{<n}$, and the claim follows from Theorem 11.7 and Equation (11.2). Tightness of this bound follows from the Černý automata (Figure 1.7) whose group of units is regular. □

11.3 The Rystsov Graph

Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton. For $i \geq 1$, Rystsov [535] introduced the following digraphs $I_i(\mathcal{A}) = (Q, E_i)$ with vertex set Q and edge sets

$$E_i = \begin{cases} \{(p,q) \mid \exists a \in \Sigma_I : p \neq q \wedge \delta(p,a) = q\} & \text{if } i = 1, \\ E_{i-1} \cup \{(\delta(p,a), \delta(q,a)) \mid a \in \Sigma_G \wedge (p,q) \in E_{i-1}\} & \text{if } i \geq 2. \end{cases}$$

The *Rystsov graph* is $I(\mathcal{A}) = (Q, E)$ with $E = \bigcup_{i \geq 1} E_i$. Directly from the definition we get

$$E = \{(\delta(p,u), \delta(q,u)) \mid (p,q) \in E_1 \wedge u \in \Sigma_G^*\}. \tag{11.3}$$

So, for every $u \in \Sigma_G^*$ and $p, q \in Q$ we have

$$(p,q) \in E \Rightarrow (\delta(p,u), \delta(q,u)) \in E. \tag{11.4}$$

Furthermore, we have $E_1 = \bigcup_{a \in \Sigma_I} \operatorname{excl}(a) \times \operatorname{dupl}(a)$ and as, for $a \in \Sigma_I$ and $u \in \Sigma_G^*$, we have $\operatorname{excl}(au) = \delta(\operatorname{excl}(a), u)$ and $\operatorname{dupl}(au) = \delta(\operatorname{dupl}(a), u)$, Equation (11.3) gives

$$E = \bigcup_{a \in \Sigma_I} \bigcup_{u \in \Sigma_G^*} \operatorname{excl}(au) \times \operatorname{dupl}(au). \tag{11.5}$$

Rystsov [535] proved the following statement (we include a proof as well).

Theorem 11.10 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton. Then $I(\mathcal{A})$ is strongly connected if and only if $\Sigma_I \Sigma_G^*$ is augmentative.*

Proof. For a proper subset $S \subsetneq Q$ with $S \neq \emptyset$ we have

$$\begin{aligned} &\exists a \in \Sigma_I \; \exists u \in \Sigma_G^* \colon |\delta^{-1}(S, au)| > |S| \\ &\Leftrightarrow \exists a \in \Sigma_I \; \exists u \in \Sigma_G^* \colon \operatorname{excl}(au) \cap S = \emptyset \wedge \operatorname{dupl}(au) \subseteq S && (au \text{ has defect one}) \\ &\Leftrightarrow \exists (p,q) \in E \colon p \notin S \wedge q \in S. && (\text{Eqn. (11.5)}) \end{aligned}$$

Then the claim follows from these equivalences and Theorem 3.67. □

Theorem 11.11 (Rystsov [535, 536]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton. Then $\mathcal{A}$ is strongly connected and synchronizing if and only if $I(\mathcal{A})$ is strongly connected.*

Proof. First, suppose $\mathcal{A}$ is strongly connected and synchronizing. By Theorem 11.5, $\mathcal{A}$ admits an augmentative set. From Theorem 11.7 we get that some subset of $\Sigma_I \Sigma_G^*$ is augmentative. This implies that $\Sigma_I \Sigma_G^*$ is augmentative. By Theorem 11.10, $I(\mathcal{A})$ is strongly connected.

Conversely, if $I(\mathcal{A})$ is strongly connected, by Theorem 11.10, we find that $\Sigma_I \Sigma_G^*$ is augmentative. Thus, by Theorem 11.5, $\mathcal{A}$ is strongly connected and synchronizing. □

Theorem 11.12 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton with $I(\mathcal{A}) = (Q, E)$. Then $(q_1, q_2) \in E$ if and only if there exists a simple idempotent $t \in \mathcal{T}_S(\mathcal{A})$ such that $t(q_1) = q_2$ and $t = \delta_x^g$ for some $x \in \Sigma_I$ where g is a permutation from the group of units of $\mathcal{T}_S(\mathcal{A})$.*

Proof. First, suppose $(q_1, q_2) \in E$. By the definition of the edge set of $I(\mathcal{A})$, there exist $x \in \Sigma_I$, $u \in \Sigma_G^*$, and $p_1, p_2 \in Q$ such that $\delta(p_1, x) = p_2$, $\delta(p_1, u) = q_1$, and $\delta(p_2, u) = q_2$. By Theorem 6.2, $g = \delta_u$ is contained in the group of units of $\mathcal{T}_S(\mathcal{A})$. By Equation (5.2), $t = \delta_x^g$ is a simple idempotent mapping $q_1 = g(p_1)$ to $q_2 = g(p_2)$. Let $i > 0$ be such that $g^i = \mathrm{id}_Q$. Then we have $t = \delta_{u^{i-1}xu}$ and therefore $t \in \mathcal{T}_S(\mathcal{A})$.

Conversely, suppose $t \in \mathcal{T}_S(\mathcal{A})$ is a simple idempotent of the form $t = \delta_x^g$ for some $x \in \Sigma_I$ and g a permutation from the group of units of $\mathcal{T}_S(\mathcal{A})$ with $t(q_1) = q_2$. By Equation (5.2), δ_x maps $g^{-1}(q_1)$ to $g^{-1}(q_2)$. Thus, by the definition of E, $(g^{-1}(q_1), g^{-1}(q_2)) \in E$. By Proposition 6.2 we have $g = \delta_u$ for some $u \in \Sigma_G^*$. By Equation (11.4), we have $(\delta(g^{-1}(q_1), u), \delta(g^{-1}(q_2), u)) \in E$. Since $q_1 = \delta(g^{-1}(q_1), u)$ and $q_2 = \delta(g^{-1}(q_2), u)$ we obtain the claim. □

By Theorem 11.12, the edges in $I(\mathcal{A})$ correspond to certain simple idempotents in the transformation semigroup. However, as we will see later, not every simple idempotent of the transformation semigroup corresponds to an edge of $I(\mathcal{A})$. Note, however, that by Theorem 5.37, every element of the transformation semigroup is a product of conjugates of simple idempotents together with an element of the group of units.

Later, in Section 13, we will need the following property that follows readily.

Proposition 11.13 *If $\mathcal{A}$ is an SI-automaton without permutational letters, i.e., $\Sigma_G = \emptyset$, then the automaton graph $G(\mathcal{A})$ with self-loops omitted equals the Rystsov graph $I(\mathcal{A})$.*

11.4 From Automata to Transformation Semigroups

For transformation semigroups $S \leq_S \mathcal{T}_n$ we define the digraph $I(S) = (V, E)$ with vertex set $V = [n]$ and edge set E equal to

$$\{(p, q) \mid \ p \neq q \text{ and there exists a simple idempotent } t \in S \text{ with } t(p) = q\}.$$

Note that the Rystsov graph is defined only for SI-automata, whereas the above construction applies to arbitrary transformation semigroups.

We also extend $\Gamma_1(\mathcal{A})$ to transformation semigroups in the obvious way by letting $\Gamma_1(S)$ be the digraph with vertex set $[n]$ and edge set

$$\{(p, q) \mid \exists t \in S : \{p\} = \mathrm{excl}(t) \text{ and } \mathrm{dupl}(t) = \{q\}\}.$$

11.5 Relations Between the Different Graphs

Here, we investigate relations between $I(\mathcal{A})$, $I(S)$, $\Gamma_1(\mathcal{A})$, and $\Gamma_1(S)$. Recall the identification we made for $\Gamma_1(\mathcal{A})$ (see also Equation (11.1)) so that its vertex set equals the state set of $\mathcal{A}$.

First, in Theorem 11.14, we state general relations among these graphs. Then, in Theorem 11.16, we prove that strong connectivity of $I(S)$ for $S \leq_S \mathcal{T}_n$ implies that every ordered pair of distinct vertices forms an edge in $\Gamma_1(S)$.

Proposition 11.14 *Let $S \leq_S \mathcal{T}_n$ and $\mathcal{A} = (Q, \Sigma, \delta)$.*

1. *$\Gamma_1(\mathcal{T}_M(\mathcal{A})) = \Gamma_1(\mathcal{A})$ and $\Gamma_{1,i}(\mathcal{A})$ is a subgraph of $\Gamma_1(\mathcal{A})$ for each $i \geq 1$.*
2. *$I(S) = I(S \cup \{\mathrm{id}_{[n]}\})$ and $\Gamma_1(S) = \Gamma_1(S \cup \{\mathrm{id}_{[n]}\})$. So $I(\mathcal{T}_S(\mathcal{A})) = I(\mathcal{T}_M(\mathcal{A}))$ and $\Gamma_1(\mathcal{T}_S(\mathcal{A})) = \Gamma_1(\mathcal{T}_M(\mathcal{A}))$.*
3. *$I(S)$ is a subgraph of $\Gamma_1(S)$.*
4. *If $\mathcal{A}$ is an SI-automaton, then $I(\mathcal{A})$ is a subgraph of $I(\mathcal{T}_S(\mathcal{A}))$.*

Proof. 1: This follows as every mapping in $\mathcal{T}_M(\mathcal{A})$ is induced by a word and every word induces a mapping in $\mathcal{T}_M(\mathcal{A})$. That $\Gamma_{1,i}(\mathcal{A})$ is a subgraph of $I(\mathcal{A})$ follows directly from its definition.

2: As the identity transformation is not relevant for the set of defect one mappings in a transformation semigroup the claim follows.

3: Both graphs have the same vertex set. Now, suppose (p, q) is an edge in $I(S)$. Then there exists a simple idempotent $t \in S$ with $t(p) = t(q) = q$. As $\mathrm{excl}(t) = \{p\}$ and $\mathrm{dupl}(t) = \{q\}$, it follows that (p, q) is an edge in $\Gamma_1(S)$. So, $I(S)$ is a subgraph of $\Gamma_1(S)$.

4: Suppose $\mathcal{A}$ is an SI-automaton. Both graphs $I(\mathcal{A})$ and $\mathcal{T}_S(\mathcal{A})$ have the same vertex set Q. Next, by Theorem 11.12, for each edge (p, q) in $I(\mathcal{A})$ there exists a simple idempotent in $\mathcal{T}_S(\mathcal{A})$ mapping p to q. This yields that (p, q) is also an edge in $\mathcal{T}_S(\mathcal{A})$. □

By Item 2 of Theorem 11.14, it makes no difference whether we use monoids or semigroups in most of our statements.

Remark 11.15 Item 4 of Theorem 11.14 implies that the edge (p, q) of $I(\mathcal{A})$ is also an edge of $I(\mathcal{T}_S(\mathcal{A}))$. Hence, by the definition of $I(\mathcal{T}_S(\mathcal{A}))$, there exists a simple idempotent $\delta_u \in \mathcal{T}_S(\mathcal{A})$, where $u \in \Sigma^*$, such that $\delta_u(p) = q$, or equivalently, $\delta(p, u) = q$. In fact, Theorem 11.12 gives the stronger statement that u can be chosen in the form $u = vaw$, where $v, w \in \Sigma_G^*$, $a \in \Sigma_I$, $p = \delta(s, w)$, and $q = \delta(t, w)$ for distinct states $s, t \in Q$ such that $\delta(s, a) = t$ and $\delta(r, vw) = r$ for every $r \in Q$.

Next, we show that if $I(S)$ is strongly connected, then in $\Gamma_1(S)$ all distinct pairs of vertices are connected by an edge.

Proposition 11.16 *Let $S \leq_S \mathcal{T}_n$. If $I(S)$ is strongly connected, then the edge set of $\Gamma_1(S)$ equals $\{(p, q) \mid p, q \in [n], p \neq q\}$.*

Proof. Let $p, q \in [n]$. Consider a path $q_0, q_1, \ldots, q_m$ in $I(S)$ such that $q_0 = p$ and $q_m = q$. For each edge (q_{i-1}, q_i) of $I(S)$, there exists a simple idempotent $t_i \in S$ mapping q_{i-1} to q_i. Set $t = t_1 t_2 \cdots t_m$. Then, for every $x \in [n]$, we have

$$t(x) = \begin{cases} x, & \text{if } x \notin \{q_0, q_1, \ldots, q_m\}, \\ q_{\min\{i+1, m\}}, & \text{if } x = q_i \text{ for some } i \in \{0, 1, \ldots, m\}. \end{cases}$$

Hence t has defect one, with $\mathrm{excl}(t) = \{p\}$ and $\mathrm{dupl}(t) = \{q\}$. Therefore, t induces the edge (p, q) in $\Gamma_1(S)$. □

11.6 Relations to the Group of Units

Let M be a transformation monoid. Here, we show the following:

1. in Theorem 11.17 that the group of units of M (resp. an automaton $\mathcal{A}$) is contained in the automorphism groups of $I(M)$ and $\Gamma_1(M)$ (resp. $\mathcal{T}_S(\mathcal{A})$, and $I(\mathcal{A})$ if $\mathcal{A}$ is an SI-automaton),
2. in Theorem 11.22 that for a transitive group of units, for each edge in $I(M)$ the reversed edge is also in $I(M)$,
3. in Theorem 11.26 that for a 2-homogeneous group of units all pairs of distinct states are connected by an edge.

Proposition 11.17 *1. Let $M \leq_M \mathcal{T}_n$ be a transformation monoid with group of units G. Then we have $G \leq_G \mathrm{Aut}(I(M))$ and $G \leq_G \mathrm{Aut}(\Gamma_1(M))$.*
2. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a semiautomaton, and let G denote the group of units of $\mathcal{T}_M(\mathcal{A})$. Then we have $G \leq_G \mathrm{Aut}(\Gamma_1(\mathcal{A}))$. Moreover, if $\mathcal{A}$ is an SI-automaton, then $G \leq_G \mathrm{Aut}(I(\mathcal{A}))$.

Proof. 1: By Equation (5.2), for any $t\colon [n] \to [n]$ and permutation $g\colon [n] \to [n]$ we have $\mathrm{excl}(t^g) = g(\mathrm{excl}(t))$, $\mathrm{dupl}(t^g) = g(\mathrm{dupl}(t))$, and t is a simple idempotent mapping a to b ($a \neq b$) if and only if t^g is a simple idempotent mapping $g(a)$ to $g(b)$. These equations yield that if (a, b) is an edge in $I(M)$ (resp. $\Gamma_1(M)$), then $(g(a), g(b))$ is an edge in $I(M)$ (resp. $\Gamma_1(M)$).

2: By Item 1 of Theorem 11.14, we have $\Gamma_1(\mathcal{A}) = \Gamma_1(\mathcal{T}_M(\mathcal{A}))$. Then the claim $G \leq_G \mathrm{Aut}(\Gamma_1(\mathcal{A}))$ follows from Item 1. Next, by Theorem 6.2, $G = \{\delta_u \mid u \in \Sigma_G^*\}$. So Equation (11.4) yields $G \leq_G \mathrm{Aut}(I(\mathcal{A}))$. □

Remark 11.18 For $g \in G$ and a defect-one transformation $t \in M$ with $\mathrm{excl}(t) = \{p\}$ and $\mathrm{dupl}(t) = \{q\}$, the transformation gt also has defect one, with $\mathrm{excl}(gt) = \{g(p)\}$ and $\mathrm{dupl}(gt) = \{g(q)\}$. Hence $G \leq_G \mathrm{Aut}(\Gamma_1(M))$. The same argument does not apply to $I(M)$, since the composition gt need not be a simple idempotent.

Remark 11.19 With similar arguments as in Theorem 11.17, it can also be shown that the group of units G of $\mathcal{T}_S(\mathcal{A})$ is contained in the automorphism group of each $\Gamma_k(\mathcal{A})$ for $k \geq 1$. Thus, by Theorem 11.3, the strongly connected components of each $\Gamma_k(\mathcal{A})$ yield a sequence of G-congruences that refine each other.

Remark 11.20 By Theorem 11.17 and as, by Theorem 6.2, for every automaton $\mathcal{A}$, the set $\{u \in \Sigma^* \mid u \text{ induces a permutation }\}$ equals the group of units, we find that Equation (11.4) is true for words $u \in \Sigma^*$ that permute the states and each of the graphs $I(\mathcal{A})$ (if $\mathcal{A}$ is an SI-automaton), $I(\mathcal{T}_M(\mathcal{A}))$, and $\Gamma_1(\mathcal{T}_M(\mathcal{A})) = \Gamma_1(\mathcal{A})$.

Remark 11.21 Item 4 of Theorem 11.14 also follows from Theorem 11.17 in the following way. Let (q_1, q_2) be an edge in $I(\mathcal{A})$. Then there exist $x \in \Sigma_I$ and $u \in \Sigma_G^*$ such that $\delta(p_1, x) = p_2$ for distinct $p_1, p_2 \in Q$ and $\delta(p_1, u) = q_1$ and $\delta(p_2, u) = q_2$. By Proposition 6.2, δ_u is contained in the group of units of $\mathcal{T}_S(\mathcal{A})$. As δ_x is a simple idempotent, the edge (p_1, p_2) is in $I(\mathcal{T}_S(\mathcal{A}))$. By Proposition 11.17, δ_u acts as an automorphism on $I(\mathcal{T}_S(\mathcal{A}))$. So the edge $(q_1, q_2) = (\delta_u(p_1), \delta_u(p_2))$ is also in $I(\mathcal{T}_S(\mathcal{A}))$.

Proposition 11.22 *Let $M \leq \mathcal{T}_n$ be a transformation monoid whose group of units acts transitively on $[n]$. Then the graph $I(M) = ([n], E)$ is symmetric.*

Proof. Let $(p, q) \in E$ with $p \neq q$. Then there exists a simple idempotent $t \in M$ such that $t(q) = t(p)$. By Theorem 5.13 there exists another simple idempotent $h \in M$ such that $h(p) = h(q) = p$. So, $(q, p) \in E$. □

Remark 11.23 A transitive group of units does not force $\Gamma_1(\mathcal{A})$ to be symmetric. Consider $\mathcal{A} = (Q, \{a, b\}, \delta)$ with $Q = \{0, 1, 2, 3, 4, 5\}$, where a acts as the shift $\delta(i, a) = i + 1 \pmod 6$ and b fixes $0, 1, 4$, swaps $2 \leftrightarrow 3$, and sends $5 \mapsto 3$. Thus b has rank 5: its image is $Q \setminus \{5\}$, so $\mathrm{excl}(b) = 5$, and the pair $\{2, 5\}$ is merged onto 3, so $\mathrm{dupl}(b) = 3$. The group of units G is the cyclic group generated by the 6-cycle (0 1 2 3 4 5), which acts transitively on the states.

Claim. For every word w of defect one, the difference $\Delta(w) := \mathrm{dupl}(w) - \mathrm{excl}(w) \pmod 6$ lies in $\{1, 3, 4\}$; in particular it is never 2 or 5.

Since a is a bijection, the rank can drop only at an application of b, and b drops it precisely when the current image contains both merged states 2 and 5. A defect-one word therefore has a unique rank-dropping b (applied to all of Q as the first b in the word), after which every further b must preserve the rank. The permutational word before the first b has no excluded and no duplicate state, so it suffices to follow the pair (E, D) of the excluded and duplicate state through the remaining letters, starting from $(E, D) = (5, 3)$ (the value right after the first b).

Two things happen as we read on (addition modulo 6):

- *Reading a.* The image $Q \setminus \{E\}$ becomes $Q \setminus \{E + 1\}$ and the duplicate value moves from D to $D+1$, so $(E, D) \mapsto (E+1, D+1)$. Hence a leaves $\Delta = D - E$ unchanged.
- *Reading a rank-preserving b.* This requires that $Q \setminus \{E\}$ omit one of $2, 5$, i.e. $E \in \{2, 5\}$. In both cases ($E = 5$: image $\{0, 1, 2, 3, 4\}$; $E = 2$: image $\{0, 1, 3, 4, 5\}$) the map b is injective on the image, the new image is again $\mathbb{Q} \setminus \{5\}$, and the merged value is carried to $\delta(D, b)$. Thus $(E, D) \mapsto (5, \delta(D, b))$: after *any* b the excluded state is 5 again, and the duplicate state is replaced by its b-value.

Now record the duplicate states $q_0, q_1, q_2, \ldots$ at the successive moments right after each b; at all these moments $E = 5$. We have $q_0 = 3$. Between the j-th and $(j+1)$-th b the word contains a block a^{k_j}, which shifts the state q_j (and $E = 5$) by k_j; for the next b to be rank-preserving we need $E = 5 + k_j \in \{2, 5\}$, that is $k_j \equiv 0$ or $k_j \equiv 3 \pmod 6$ (note $5 + 3 \equiv 2$). Applying the rule above then gives

$$q_{j+1} = \delta\big(q_j + k_j, b\big), \qquad k_j \in \{0, 3\}.$$

So the set of duplicate states reachable at an "$E = 5$" moment is the smallest subset of Q containing $q_0 = 3$ and closed under the *two* maps

$$q \longmapsto \delta(q, b) \qquad \text{and} \qquad q \longmapsto \delta(q, b).$$

(These two maps already absorb the effect of the intervening a-block through the offset +3; we do *not* claim closure under a itself.) Both maps send $\{0, 2, 3\}$ into itself,

$$\begin{aligned} \delta(0, b) = 0, \quad \delta(2, b) = 3, \quad \delta(3, b) = 2, \\ \delta(0{+}3, b) = 2, \quad \delta(2{+}3, b) = 3, \quad \delta(3{+}3, b) = 0. \end{aligned}$$

and $\{0, 2, 3\}$ contains the seed 3, so every duplicate value stays in $\{0, 2, 3\}$. Since $E = 5$ at each such moment, this gives that for the duplicate state $q \in \{0, 2, 3\}$ we have $\Delta = q - 5 \in \{-5, -3, -2\} \equiv \{1, 3, 4\}$; a possible trailing block a^m only shifts (E, D) by m and preserves Δ. This proves the claim.

Consequently the edge set of $\Gamma_1(\mathcal{A})$ is the union of the G-orbitals of differences $1, 3, 4$. It is strongly connected, so $\mathcal{A}$ is completely reachable, but it is *not* symmetric: the letter b forces the edge $(5, 3)$ (with $\Delta \equiv 4$), whereas its reverse $(3, 5)$ would need $\Delta = 2$, which never occurs. Hence $(5, 3) \in E_1$ while $(3, 5) \notin E_1$; likewise $(5, 0) \in E_1$ (difference 1) but $(0, 5) \notin E_1$ (difference 5).

Remark 11.24 Later, we will investigate transformation semigroups of the form $\langle G \cup \{t\}\rangle_S$ for some group G and non-permutation t. If t is a simple idempotent, then not every simple idempotent is of the form t^g for some $g \in G$. For example, let

$$G = \langle (0\ 1\ 2\ 3)\rangle_G.$$

The simple idempotents of the form t^g are t itself and the three simple idempotents mapping 1 to 2, 2 to 3, and 3 to 0, respectively. By Theorem 11.22, $\langle G \cup \{t\}\rangle_S$ also contains the simple idempotent mapping 1 to 0, which is not of this form.

Remark 11.25 Let $a, b \in [n]$ be distinct. We have

$$[a \mapsto b][b \mapsto a][a \mapsto b] = [a \mapsto b]$$

An element s in a semigroup is called *regular* [325] if there exists t such that $tst = t$. Thus, if $I(M)$ is symmetric, then each simple idempotent is regular.

Proposition 11.26 *Let $M \leq \mathcal{T}_n$ be a transformation monoid whose group of units is 2-homogeneous, and set $I(M) = ([n], E)$. If M contains a transformation of defect one, then $E = \{(p, q) \mid p, q \in [n],\ p \neq q\}$; otherwise $E = \emptyset$.*

Proof. Let G be the group of units of M. By definition of $I(M)$, if M contains no transformation of defect one then $E = \emptyset$. So assume there is a transformation $t \in M$ of defect one, and let $p, p' \in [n]$ be the distinct points with $t(p) = t(p')$.

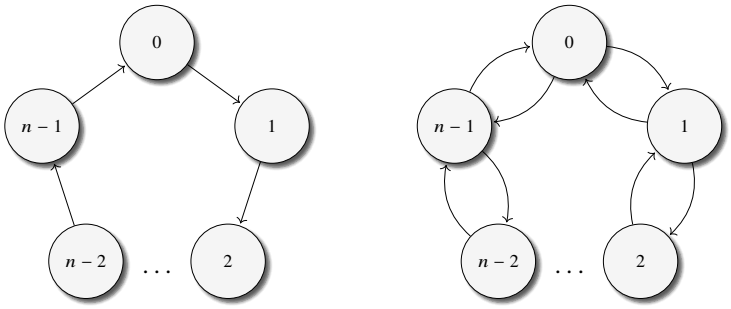
Let $q, q' \in [n]$ be distinct. Since G is 2-homogeneous, there exists $g \in G$ with $g(\{q, q'\}) = \{p, p'\}$, and hence $(tg)(q) = (tg)(q')$. By Theorem 5.28, G is transitive, so Theorem 5.13 applied to tg yields simple idempotents $e, f \in M$ with $e(q) = q'$ and $f(q') = q$. Thus $(q, q'), (q', q) \in E$. As q, q' were arbitrary distinct points, $E = \{(p, q) \mid p, q \in [n],\ p \neq q\}$. □

11.7 Examples

Here, we give examples showing that all these graphs can be different. Note that $I(\mathcal{A})$ is only defined for SI-automata, whereas $I(\mathcal{T}_S(\mathcal{A}))$ and $\Gamma_1(\mathcal{A})$ are defined for any automaton $\mathcal{A}$.

Example 11.27 computes $I(C_n)$, $I(\mathcal{T}_S(C_n))$ and $\Gamma_1(\mathcal{A})$ for the Černý automata C_n. Example 11.29 gives an SI-automaton $\mathcal{A}$ such that $I(\mathcal{A}) = I(\mathcal{T}_S(\mathcal{A}))$ and $I(\mathcal{T}_S(\mathcal{A})) \neq \Gamma_1(\mathcal{A})$. Example 11.30 gives an automaton $\mathcal{A}$ with $I(\mathcal{T}_S(\mathcal{A})) \neq \Gamma_1(\mathcal{A})$ and such that in $\Gamma_1(\mathcal{A})$ all pairs of distinct vertices are connected by an edge, but where this is not implied by Theorem 11.16 as $I(\mathcal{T}_S(\mathcal{A}))$ is not strongly connected.

Example 11.27 Figure 11.3 shows $I(C_n)$ and $I(\mathcal{T}_S(C_n))$, where $I(C_n)$ equals the orbital digraph of $\langle \delta_b \rangle_G$ on $(0, 1)$. By Theorem 11.22, $I(\mathcal{T}_S(C_n))$ also includes the edges from $I(C_n)$ going in the other direction. Next, we show that these are the only edges in $I(\mathcal{T}_S(C_n))$. Let (p, q) be an edge in $I(\mathcal{T}_S(C_n))$. Then $t(p) = t(q) = q$ for some simple idempotent $t \in \mathcal{T}_S(C_n)$. Recall the definition of an interval from the proof of Theorem 1.2. We have $\{p, q\} = t^{-1}(q)$ and as $\{q\}$ is an interval, $\{p, q\}$ must be an interval. This implies $\delta_n(p, b) = q$ or $\delta_n(q, b) = p$, and so (p, q) is one of the edges shown in Figure 11.3 for the graph $I(\mathcal{T}_S(C_n))$. Lastly, as $I(\mathcal{T}_S(C_n))$ is strongly connected, by Theorem 11.16, in $\Gamma_1(C_n)$ all pairs of distinct vertices are connected by an edge. Thus for each distinct pair p, q of states, there exists a word of defect one with excluded state p and duplicate state q, but there exists no simple idempotent mapping p to q. Since the group of units is transitive, by Theorem 5.28, there even exists no word mapping p and q to a common state.



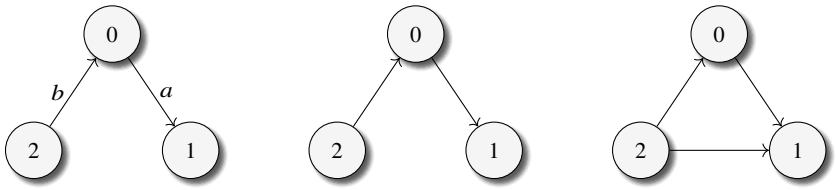
(a) The graph $I(C_n)$. (b) The graph $I(\mathcal{T}_S(C_n))$

Fig. 11.3: The graphs $I(C_n)$ and $I(\mathcal{T}_S(C_n))$ for the Černý automata C_n. By Proposition 11.16 in $\Gamma_1(C_n)$ all pairs of distinct vertices are connected by an edge. Compare Example 11.27.

Remark 11.28 By Theorem 11.12 words in $I(C_n)$ correspond to simple idempotent words of a form as stated in Theorem 11.15. However, not every simple idempotent in $\mathcal{T}_S(\mathcal{A})$ for an SI-automaton $\mathcal{A}$ might be representable by such a word since there exist

SI-automata where $I(\mathcal{A})$ is a proper subgraph of $I(\mathcal{T}_S(\mathcal{A}))$, as, for example, the Černý automata, see Theorem 11.27.

Example 11.29 Figure 11.4 shows an SI-automaton $\mathcal{A}$ (without permutational letters) with $I(\mathcal{A})$, $I(\mathcal{T}_S(\mathcal{A}))$, and $\Gamma_1(\mathcal{A})$ where $I(\mathcal{A}) = I(\mathcal{T}_S(\mathcal{A}))$ and $I(\mathcal{T}_S(\mathcal{A})) \neq \Gamma_1(\mathcal{A})$.



(a) Automaton $\mathcal{A}$. Self-loops omitted.

(b) The graph $I(\mathcal{T}_S(\mathcal{A}))$ (which equals $I(\mathcal{A})$ here).

(c) The graph $\Gamma_1(\mathcal{A})$.

Fig. 11.4: The graphs $I(\mathcal{T}_S(\mathcal{A}))$ and $\Gamma_1(\mathcal{A})$ for a given SI-automaton $\mathcal{A}$. The additional edge in $\Gamma_1(\mathcal{A})$ is induced by the word ab of defect one.

Example 11.30 Let $\mathcal{A} = ([4], \{a, b\}, \delta)$ be the automaton from Figure 6.6, formally given by $\delta(i, b) = (i + 1) \bmod 4$ and

$$\delta(0, a) = 1, \quad \delta(1, a) = 2, \quad \delta(2, a) = 1, \quad \delta(3, a) = 3.$$

The graph $I(\mathcal{T}_S(\mathcal{A}))$ has only the edges $\{(0, 2), (2, 0), (1, 3), (3, 1)\}$ induced by four simple idempotent words, see Table 11.2.[2] In $\Gamma_1(\mathcal{A})$ all pairs of distinct states are connected by an edge, the additional edges $(0, 1)$, $(3, 0)$, $(2, 3)$, and $(1, 2)$ are induced by the words a, bab^3, ab^2, and b^2ab, respectively. The edges $(1, 0)$, $(0, 3)$, $(3, 2)$, and $(2, 1)$ in $\Gamma_1(\mathcal{A})$ are induced by the words ab^2ab, ab^2a, bab^2ab^3, and $ab^2a^2b^2$, respectively. Later, in Theorem 12.33, we will also show that $\mathcal{A}$ is completely reachable and completely distinguishable.

11.8 Deducing Reachability and Distinguishability

ere, in Theorem 11.31 (resp. Theorem 11.31) we show that strong connectivity (resp. strong $\frac{3}{2}$-connectivity) of $I(S)$ implies complete reachability and distinguishability of pairs of 2-sets (resp. complete distinguishability) in a transformation semigroup S. Furthermore, we conclude in Theorem 11.39 that strong connectivity implies absence of any non-trivial congruence, which gives minimality in terms of automata as stated in Theorem 11.40.

Theorem 11.31 *Let $M \leq_M \mathcal{T}_n$ be a transformation monoid. If $I(M)$ is strongly connected, then M is completely reachable and every 2-set $\{a, b\}$ is distinguishable from*

[2] Taken from [303] but with some mistakes fixed.

Table 11.2: Defect one words of the automaton from Theorem 11.30 (see Figure 6.6). To the right of each word w the induced transformation on the states is written, where $j \in [4]$ appears at position i iff $\delta(i-1, w) = j$. For words u, v in the same row it is $\delta(i, u) = \delta(j, u)$ iff $\delta(i, v) = \delta(j, v)$ for $i, j \in [4]$

Word	Mapping	Word	Mapping	Word	Mapping	Word	Mapping
$b^2a^2b^2$	$[0,1,0,3]$	a	$[1,2,1,3]$	ab^3	$[0,1,0,2]$	ab^2ab	$[0,2,0,3]$
a^2b^2	$[0,3,0,1]$	b^2a	$[1,3,1,2]$	b^2ab^3	$[0,2,0,1]$	ab^2a^2b	$[0,3,0,2]$
$ab^2a^2b^2$	$[1,0,1,3]$	a^2	$[2,1,2,3]$	a^2b^3	$[1,0,1,2]$	b^2ab	$[2,0,2,3]$
ab^2ab^2	$[1,3,1,0]$	b^2a^2	$[2,3,2,1]$	$b^2a^2b^3$	$[1,2,1,0]$	ab	$[2,3,2,0]$
ab^2	$[3,0,3,1]$	ab^2a	$[3,1,3,2]$	ab^2ab^2	$[2,0,2,1]$	b^2a^2b	$[3,0,3,2]$
b^2ab^2	$[3,1,3,0]$	ab^2a^2	$[3,2,3,1]$	$ab^2a^2b^3$	$[2,1,2,0]$	a^2b	$[3,2,3,0]$
$bab^2a^2b^2$	$[0,1,3,1]$	ba^2	$[1,2,3,2]$	ba^2b^3	$[0,1,2,1]$	b^3ab	$[0,2,3,2]$
bab^2	$[0,3,1,3]$	bab^2a	$[1,3,2,3]$	bab^2ab^3	$[0,2,1,2]$	b^3a^2b	$[0,3,2,3]$
$b^3a^2b^2$	$[1,0,3,0]$	ba	$[2,1,3,1]$	bab^3	$[1,0,2,0]$	bab^2ab	$[2,0,3,0]$
b^3ab^2	$[1,3,0,3]$	bab^2a^2	$[2,3,1,3]$	$bab^2a^2b^3$	$[1,2,0,2]$	ba^2b	$[2,3,0,3]$
ba^2b^2	$[3,0,1,0]$	b^3a	$[3,1,2,1]$	b^3ab^3	$[2,0,1,0]$	bab^2a^2b	$[3,0,2,0]$
bab^2ab^2	$[3,1,0,1]$	b^3a^2	$[3,2,1,2]$	$b^3a^2b^3$	$[2,1,0,1]$	bab	$[3,2,0,2]$

every $B \subseteq [n]$ with $\{a, b\} \not\subseteq B$. In particular, two distinct 2-subsets in $[n]$ are distinguishable.

Proof. Suppose $I(M)$ is strongly connected. Let $\mathcal{A}$ be an automaton with $\mathcal{T}_M(\mathcal{A}) = M$ constructed as in Section 6.8 by selecting a generating set of M as a set of letters. Then, as $\Gamma_1(\mathcal{A}) = \Gamma_1(M)$ and $I(M)$ is a subgraph of $\Gamma_1(M)$ by Theorem 11.14 , by Theorem 11.2, M is completely reachable.

Next, we can assume $|B| \geq 2$ and, without loss of generality, suppose $a \notin B$. Let $x_0, x_1, x_2, \ldots, x_k$ be a shortest path in $I(M)$ with $x_0 = a$ and $x_k = b$. By definition, there exist simple idempotent mappings $t_i \in M$ with $t_i(x_{i-1}) = x_i$ for $i \in \{1, \ldots, k\}$. We distinguish two cases.

1. $B \not\subseteq \{x_0, x_1, \ldots, x_k\}$.
   Let $b_1, b_2 \in B$ be distinct with $b_1 \notin \{x_0, x_1, \ldots, x_k\}$ and $t = t_k \cdots t_2 t_1$. By Equation (4.25) we find $t(\{a, b\}) = \{b\}$ and
   $$t(\{b_1, b_2\}) = \begin{cases} \{b_1, b\} & \text{if } b_2 \in \{x_0, x_1, \ldots, x_k\}, \\ \{b_1, b_2\} & \text{if } b_2 \notin \{x_0, x_1, \ldots, x_k\}. \end{cases}$$
2. $B \subseteq \{x_0, x_1, \ldots, x_k\}$.
   As $x_0 \notin B$, we have $B \subseteq \{x_1, \ldots, x_k\}$. Set $\bar{t} = t_k \cdots t_3 t_2$. By Equation (4.25) we find $\bar{t}(B) = \{x_k\}$ and $\{x_0, x_k\} = \bar{t}(\{x_0, x_k\})$ where $\{x_0, x_k\} = \{a, b\}$.

In both cases, there exists a transformation distinguishing $\{a, b\}$ from B. □

Remark 11.32 To deduce complete reachability in Theorem 11.31 we must start from a transformation monoid and not merely from a transformation semigroup since otherwise

the whole domain might not be reachable. For example, consider the semigroup of all transformations of defect at least one on the set $[n]$. In this semigroup $[n]$ is not reachable, but since it contains all simple idempotents, $I(S)$ equals the digraph where all pairs of vertices are connected by an edge (but which implies that every proper and non-empty set is reachable).

Example 11.33 Let $t_i : [3] \to [3]$ be three distinct constant transformations with $t_i(0) = t_i(1) = t_i(2) = i$ tor $i \in [3]$. The set $S = \{t_1, t_2, t_3\}$ forms a transformation semigroup. The digraph on $[3]$ obtained by connecting point a to b if and only if there exists an idempotent mapping a to b is strongly $\frac{3}{2}$-connected. However, no pair of distinct non-empty and non-singleton subsets of states is distinguishable in S and S is not completely reachable. In this sense, restricting ourselves to simple idempotents in the definition of $I(S)$ is necessary.

Theorem 11.31 with Theorem 9.10 and the fact that $I(\mathcal{A})$ is a subgraph of $I(\mathcal{T}_M(\mathcal{A}))$ for SI-automata $\mathcal{A}$ yields the following corollary.

Corollary 11.34 *Let $\mathcal{A}$ be an n-state automaton. If $I(\mathcal{T}_M(\mathcal{A}))$ (or $I(\mathcal{A})$ if $\mathcal{A}$ is an SI-automaton) is strongly connected, then $\mathcal{A}$ is completely reachable and a minimal automaton for* $\mathrm{Syn}(\mathcal{A})$ *has at least* $\binom{n}{2} + 1$ *states.*

Example 11.35 Consider the strongly connected SI-automaton from Figure 9.7. Here, $G(\mathcal{A}) = I(\mathcal{A}) = I(\mathcal{T}_M(\mathcal{A}))$. So $I(\mathcal{A})$ is strongly connected, but $\mathcal{A}$ is not completely distinguishable.

Next, we state a sufficient condition for complete reachability and complete distinguishability.

Theorem 11.36 *Let $M \leq_S \mathcal{T}_n$ be a transformation monoid. If $I(M)$ is strongly $\frac{3}{2}$-connected and $n \geq 3$, then M is completely reachable and completely distinguishable.*

Proof. As strong $\frac{3}{2}$-connectivity implies strong connectivity, by Theorem 11.31 complete reachability follows. Next, let $A \subseteq [n]$ be a set containing two distinct states $p, q \in A$ and $r \in [n] \setminus A$. By assumption there exists a path $q_0, q_1, \ldots, q_m$ in $I(M)$ such that $r \notin \{q_0, q_1, \ldots, q_m\}$ and $\{q_0, q_m\} = \{p, q\}$. Without loss of generality, suppose $q_0 = p$ and $q_m = q$. By the definition of $I(M)$ there exist simple idempotents $t_i \colon [n] \to [n]$ for each $i \in \{1, \ldots, m\}$ such that $t_i(q_{i-1}) = q_i$. Set $t = t_k \cdots t_2 t_1$. By Equation (4.25) we find $t(\{p, q\}) = \{q\}$, $t(A) \subseteq A$, and $t(r) = r$. Hence $|t(A)| < |A|$ and $t(r) \notin t(A)$. From Theorem 9.46, Item 8 it follows that M is completely distinguishable. □

However, strong $\frac{3}{2}$-connectivity of $I(M)$ is not a necessary condition for complete distinguishability, as shown by the next example.

Example 11.37 Consider the SI-automaton $\mathcal{A} = (\{1, 2, 3, 4\}, \Sigma, \delta)$ without permutational letters with automaton graph given in Figure 11.5. This automaton graph equals $I(\mathcal{T}_S(\mathcal{A}))$, and it is not strongly $\frac{3}{2}$-connected since the state 2 is part of every path between 3 and any other state. However, the automaton is completely distinguishable.

To see this, first, suppose $q \in A \setminus B$ for two non-empty and non-singleton subsets of states A, B for some state $q \neq 2$. Let u be a concatenation of the letters $a_{p,2}$ with $p \in \{1,4,3\} \setminus \{q\}$ in any order. Then $\delta(B,u) = \{2\}$ and $\delta(A,u) = \{q,2\}$. Next, we have $A = \delta(Q, a_{4,1}) = Q \setminus \{4\}$ and $B = \delta(Q \setminus \{2\}, a_{4,1}) = Q \setminus \{2,4\}$, and since the latter sets are distinguishable as shown before, we obtain that also Q and $Q \setminus \{2\}$ are distinguishable.

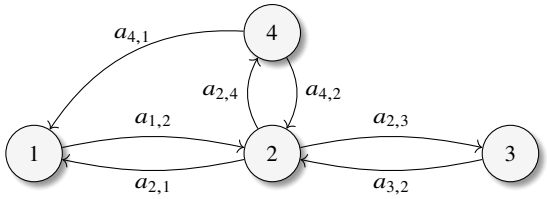


Fig. 11.5: A strongly connected and completely distinguishable SI-automaton $\mathcal{A}$ (self-loops omitted) where $I(\mathcal{T}_S(\mathcal{A}))$ is not strongly $\frac{3}{2}$-connected. See Theorem 11.37.

Note that for $n \leq 2$ every strongly connected graph is $\frac{3}{2}$-connected and every transformation semigroup on $[n]$ is completely distinguishable (but possibly not completely reachable).

Example 11.38 Strong $\frac{3}{2}$-connectivity of the Γ_1-graph is not sufficient for complete distinguishability. For example, consider the SI-automaton from Figure 9.7, in its Γ_1-graph all pairs of states are connected by an edge by Theorem 11.16. So, it is strongly $\frac{3}{2}$-connected. However, the automaton is not completely distinguishable.

Next, we show that strong connectivity of $I(S)$ also implies that S admits no non-trivial S-congruences.

Theorem 11.39 *Let $S \leq_S \mathcal{T}_n$ be a transformation semigroup. If $I(S)$ is strongly connected, then S admits no non-trivial S-congruence on $[n]$.*

Proof. Suppose $\sim$ is a non-trivial S-congruence on $[n]$. Pick some congruence class $A \subseteq [n]$, for which $A \neq [n]$, with $|A| \geq 2$. As $I(S)$ is strongly connected, by Lemma 3.67, there exists an edge (p,q) in $I(S)$ with $p \in A$ and $q \notin A$. So, we find a simple idempotent $t \in S$ with $t(p) = q$. Let $r \in A \setminus \{p\}$. Then $r \sim p$ but $r = t(r) \not\sim t(p)$, contradicting the assumption that $\sim$ is an S-congruence. So there exists no non-trivial S-congruence on $[n]$. □

From Theorem 11.39 with Theorem 6.88 we get the following corollary.

Corollary 11.40 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. If $I(\mathcal{T}_S(\mathcal{A}))$ is strongly connected, then $\mathcal{A}_{q,F}$ is minimal for any $q \in Q$ and non-empty and proper $F \subseteq Q$.*

Proof. First, if $I(\mathcal{T}_S(\mathcal{A}))$ is strongly connected, then $\mathcal{A}$ is completely reachable by Theorem 11.34 and so also strongly connected. Hence, $\mathcal{A}_{q,F}$ is accessible for any $q \in Q$ and proper and non-empty $F \subseteq Q$. By Theorem 11.39 it admits no $\mathcal{T}_S(\mathcal{A})$-congruence so Theorem 6.88, Item 5 yields that $\mathcal{A}_{q,F}$ is minimal. □

11.9 A Bound on the Reset Length for OI-Automata

Here, we extend the bound $2(n-1)^2$ on a shortest synchronizing word from [535] (also mentioned in Section 7.3) from SI-automata to OI-automata. We do this by adapting the methods of proof from [535]. Rystsov already noted in the last paragraph of [535] that this is possible.

For an OI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $i \geq 1$ we define digraphs $I_i^{OI}(\mathcal{A}) = (Q, E_i)$ with vertex set Q and edge sets

$$E_i = \begin{cases} \{(p,q) \mid \exists a \in \Sigma_I : p \neq q \wedge \delta(p,a) = q\} & \text{if } i = 1, \\ E_{i-1} \cup \{(\delta(p,a), \delta(q,a)) \mid a \in \Sigma_G \wedge (p,q) \in E_{i-1}\} & \text{if } i \geq 2. \end{cases}$$

We then let $I^{OI}(\mathcal{A}) = (Q, E)$ with $E = \bigcup_{i \geq 1} E_i$. It is clear that for an SI-automaton $\mathcal{A}$ we have $I_i^{OI}(\mathcal{A}) = I_i(\mathcal{A})$ and $I^{OI}(\mathcal{A}) = I(\mathcal{A})$. We do not add these and other new notions from this section to the notation index since they are only used in this section and nowhere else.

First, in Theorem 11.41, we show that an edge from the orbital digraph of two states, where one is mapped to the other by some idempotent one-point singular, is contained in a cycle of $I_n^{OI}(\mathcal{A})$. As a consequence, stated in Theorems 11.42 and 11.43, this gives a method to construct the graph $I^{OI}(\mathcal{A})$ by sequences of graphs where every edges is contained in a cycle. Theorems 11.44 and 11.45 relate strong connectedness, synchronizability, and augmentative sets. Lastly, in Theorem 11.46 we give a bound on the length of shortest synchronizing words in OI-automata.

Lemma 11.41 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a strongly connected n-state OI-automaton and G be the group of units of $\mathcal{T}_M(\mathcal{A})$. Suppose for distinct $p_0, q_0 \in Q$ we have $\delta(p_0, a_0) = q_0$ for $a_0 \in \Sigma_I$. Then the orbital digraph of (p_0, q_0) w.r.t. G contains an edge from $I_n^{OI}(\mathcal{A})$ inside a strongly connected component of $I_n^{OI}(\mathcal{A})$.*

Proof. Let $a_0 \in \Sigma$ be an idempotent one-point singular with $\delta(p_0, a_0) = q_0$ for distinct $p_0, q_0 \in Q$. Since $\mathcal{A}$ is strongly connected, we have $\delta(q_0, u) = p_0$ for some $u \in \Sigma^*$. Write $u = u_0 a_1 u_1 \cdots u_{m-1} a_m u_m$ with idempotent one-point singulars $a_1, \ldots, a_m \in \Sigma_I$ and permutational words $u_0, u_1, \ldots, u_m$ where we choose m minimal.

Let $i \in \{0, 1, \ldots, m\}$. Set $p_i = \delta(q_{i-1}, u_{i-1})$ and $q_i = \delta(p_i, a_i)$ if $i \geq 1$. We have

1. $\delta(q_m, u_m) = p_0$,
2. $\mathrm{Orb}_G(q_i) = \mathrm{Orb}_G(p_{(i+1) \bmod (m+1)})$,
3. $\mathrm{Orb}_G(p_0), \ldots, \mathrm{Orb}_G(p_m)$ are all distinct, and thus disjoint, by the minimal choice of m.

Next, define a digraph (V, K) with $V = \mathrm{Orb}_G(p_0) \cup \cdots \cup \mathrm{Orb}_G(p_m)$ and $K = \bigcup_{i=0}^{m} K_i$ where

$$K_i = \{(\delta(p_i, w), \delta(q_i, w)) \mid w \text{ a permutational word with } |w| < n\}.$$

For $i \in \{0, 1, \ldots, m\}$ we have $K_i \subseteq \mathrm{Orb}_G(p_i) \times \mathrm{Orb}_G(p_{(i+1) \bmod (m+1)})$ and

$$(p,q) \in K_i \wedge (q,s) \in K \Rightarrow (q,s) \in K_{(i+1) \bmod (m+1)} \tag{11.6}$$

Now, for each $q \in V$ there exists a unique $i \in \{0, 1, \ldots, m\}$ such that $q \in \mathrm{Orb}_G(p_i)$. Thus $q = \delta(p_i, w)$ for some permutational word w. By finiteness, for a shortest such word w, we have $|w| < n$. This yields that q has an outgoing edge $(q, \delta(q_i, w)) \in K_i$. So every vertex has an outgoing edges. This yields that there exists a cycle

$$(s_0, s_1), (s_1, s_2), \ldots, (s_{l-1}, s_l)$$

in (V, K). Suppose $(s_0, s_1) \in K_i$ for some $i \in \{0, 1, \ldots, m\}$. By Equation (11.6) we find

$$(s_1, s_2) \in K_{(i+1) \bmod (m+1)}, \ldots, (s_{m-1}, s_m) \in K_{(i+m) \bmod (m+1)}.$$

Since all orbits are disjoint, this implies $l \geq m$ and that there exists $j \in \{1, \ldots, m\}$ such that $(s_{j-1}, s_j) \in K_0$.

Clearly, (V, K) is a subgraph of $I_n^{OI}(\mathcal{A})$ and K_0 is contained in the edge set of the orbital subgraph of (p_0, q_0). Thus the edge (s_{j-1}, s_j) is contained in the orbital subgraph of (p_0, q_0) and is also in a strongly connected component of $I_n^{OI}(\mathcal{A})$. □

Corollary 11.42 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a strongly connected n-state OI-automaton. Denote by $C_1(\mathcal{A}) = (Q, K_1)$ the digraph that results from $I_n^{OI}(\mathcal{A})$ by only retaining those edges that are contained in some strongly connected component. For $i > 1$, set $C_i(\mathcal{A}) = (Q, K_i)$ where*

$$K_i = K_{i-1} \cup \{(\delta(p, x), \delta(q, x)) \mid x \text{ is a permutational letter and } (p, q) \in K_{i-1}\}.$$

Then $\bigcup_{i \geq 1} K_i$ equals the edge set of $I^{OI}(\mathcal{A})$.

Proof. The edge set of $I^{OI}(\mathcal{A})$ equals the union of all orbital digraphs of pairs (p, q) such that $\delta(p, a) = q$ for some idempotent one-point singular $a \in \Sigma$ and $p \neq q$. By Theorem 11.41, for each such pair (p, q), its orbital digraph contains an edge from $I_n^{OI}(\mathcal{A})$ in a strongly connected component. Thus, such an orbital digraph contains an edge from K_1. Hence, by definition $\bigcup_{i \geq 1} K_i$ also equals the union of all such orbital digraphs, which gives the claim. □

Lemma 11.43 *With the notation from Theorem 11.42, we have $C_{n-1}(\mathcal{A}) = I^{OI}(\mathcal{A})$.*

Proof. Since $K_i \subseteq K_{i+1}$ for all $i \geq 1$, the sequence $(K_i)_{i \geq 1}$ stabilizes by finiteness. Hence, by Theorem 11.42, there exists a minimal $m \geq 1$ such that

$$C_m(\mathcal{A}) = I^{OI}(\mathcal{A}).$$

Since each permutational letter acts as an automorphism of $I^{OI}(\mathcal{A})$, it permutes the strongly connected components of $I^{OI}(\mathcal{A})$ and maps cycles to cycles. It follows that every edge of $C_i(\mathcal{A})$ for $i \geq 1$ lies on a cycle, since this holds for $C_1(\mathcal{A})$.

Suppose $K_i \neq K_{i+1}$ for some $i \geq 1$. Then some permutational letter fails to permute the strongly connected components of $C_i(\mathcal{A})$, since otherwise K_i would be preserved by each permutational letter, which would yield $K_i = K_{i+1}$. So, there exists an edge in K_i whose endpoints are mapped to different strongly connected components in $C_i(\mathcal{A})$. Since this edge lies on a cycle in K_i, it also lies on a cycle in K_{i+1}. Hence, the two

different strongly connected components from $C_i(\mathcal{A})$ are merged into a single strongly connected component in $C_i(\mathcal{A})$.

Let $p, q \in Q$ be distinct with $\delta(q, a) = q$ for some $a \in \Sigma_I$. By Theorem 11.41, there exists a permutational letter $u \in \Sigma_G^*$ such that $(\delta(p, u), \delta(q, u))$ is contained in a strongly connected component of $I_n^{OI}(\mathcal{A})$. Thus, $(\delta(p, u), \delta(q, u))$ is an edge in $C_1(\mathcal{A})$. By the construction of $C_1(\mathcal{A})$, every edge belongs to a strongly connected component, so $C_1(\mathcal{A})$ has a strongly connected component containing at least two vertices. Hence, $C_1(\mathcal{A})$ has at most $n - 1$ strongly connected components.

Each strict inclusion $K_i \subsetneq K_{i+1}$ merges at least two strongly connected components. Since $C_1(\mathcal{A})$ has at most $n - 1$ strongly connected components, there can be at most $n - 2$ such strict inclusions. Therefore $m \leq n - 1$, and

$$C_j(\mathcal{A}) = C_m(\mathcal{A}) = I^{OI}(\mathcal{A})$$

for all $j \geq m$. In particular, $C_{n-1}(\mathcal{A}) = I^{OI}(\mathcal{A})$. □

Theorem 11.44 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an OI-automaton and $i \geq 1$. Then the graph $I_i^{OI}(\mathcal{A}) = (Q, E_i)$ is strongly connected if and only if $\Sigma_I \Sigma_G^{\leq i}$ is augmentative for $\mathcal{A}$.*

Proof. First, assume $I_i^{OI}(\mathcal{A})$ is strongly connected for $i \geq 1$. Let $S \subseteq Q$ be some non-empty and proper subset of Q. By Theorem 3.67 there exists an edge $(\delta(p, u), \delta(q, u)) \in E_i$ such that

$$\delta(p, u) \notin S \text{ and } \delta(q, u) \in S.$$

for a word $u \in \Sigma_G^{\leq i}$ and distinct $p, q \in Q$ where $\delta(p, x) = q$ for some $x \in \Sigma_I$. Then we have $p \notin \delta^{-1}(S, u)$ and $q \in \delta^{-1}(S, u)$. Since x is an idempotent one-point singular with duplicate state p, this gives

$$\delta^{-1}(S, u) \cup \{p\} \subseteq \delta^{-1}(S, xu)$$

and $|\delta^{-1}(S, xu)| > |S|$. As S was chosen arbitrarily, we find that $\Sigma_I \Sigma_G^{\leq i}$ is augmentative for $\mathcal{A}$.

Conversely, suppose $\Sigma_I \Sigma_G^{\leq i}$ is augmentative for $\mathcal{A}$. Let $S \subseteq Q$ be a proper and non-empty subset of Q. Then there exist $x \in \Sigma_I$ and $u \in \Sigma_G^*$ with $|u| < i$ such that $|\delta^{-1}(S, xu)| > |S|$. As the letters in Σ_G permute Q, this is equivalent to $|\delta^{-1}(T, x)| > |T|$ where $T = \delta^{-1}(S, u)$. Let $q \in Q$ be the unique state with $|\delta^{-1}(q, x)| \geq 2$. Since $|\delta^{-1}(T, x)| > |T|$ and x induces a one-point singular on Q we have $q \in T$ and there must exist $p \in \delta^{-1}(q, x) \setminus T$. The edge set E_1 contains precisely those edges consisting of two distinct states where the start is mapped to the end by some idempotent one-point singular. Therefore, we have $(p, q) \in E_1$. Then $(\delta(p, u), \delta(q, u)) \in E_i$. Since $\delta(p, u) \notin \delta(T, u) = S$ and $\delta(q, u) \in S$, by Theorem 3.67 it follows that $I_i^{OI}(\mathcal{A})$ is strongly connected. □

The previous theorem yields a generalization of Theorem 11.11.

Corollary 11.45 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an OI-automaton. Then $\mathcal{A}$ is strongly connected and synchronizing if and only if $I^{OI}(\mathcal{A})$ is strongly connected.*

Proof. First, suppose $\mathcal{A}$ is strongly connected and synchronizing. By Theorem 11.5, $\mathcal{A}$ admits an augmentative set of words. By Theorem 11.7 there exists an augmentative set in $\Sigma_I\Sigma_G^*$. By finiteness, there exists $m \geq 1$ such that $\Sigma_I\Sigma_G^{<m}$ is augmentative. By Theorem 11.44, $I_m^{OI}(\mathcal{A})$ is strongly connected. This yields that $I^{OI}(\mathcal{A})$ is strongly connected.

Conversely, suppose $I^{OI}(\mathcal{A})$ is strongly connected. By finiteness, $I_m^{OI}(\mathcal{A})$ is strongly connected for some $m \geq 1$. By Theorem 11.44, the set $\Sigma_I\Sigma_G^{<m}$ is augmentative. By Theorem 11.5, $\mathcal{A}$ is strongly connected and synchronizing. □

Theorem 11.46 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a strongly connected and synchronizing n-state OI-automaton. Then $\mathcal{A}$ has a synchronizing word of length at most $2n^2 - 6n + 5$.*

Proof. We can assume $n \geq 2$. The digraphs $C_i(\mathcal{A})$ from Theorem 11.42 are subgraphs of $I_{n+i-1}^{OI}(\mathcal{A})$ for $i \geq 1$. Since $\mathcal{A}$ is synchronizing, it has an idempotent one-point singular letter. By Theorem 11.43 we have $C_{n-1}(\mathcal{A}) = I^{OI}(\mathcal{A})$. The graph $I_{n+i-1}^{OI}(\mathcal{A})$ is a subgraph of $I^{OI}(\mathcal{A})$. Therefore, we find $I_{2n-2}^{OI}(\mathcal{A}) = I^{OI}(\mathcal{A})$. Since $\mathcal{A}$ is strongly connected and synchronizing, by Theorem 11.45, $I^{OI}(\mathcal{A})$ is strongly connected. Hence, by Theorem 11.44, the set $\Sigma_I\Sigma_G^{<2n-2}$ is augmentative. Then, by Equation (11.2), a shortest synchronizing word has length at most $(2n-2)(n-2)+1$. □

11.10 Discussion

The Rystsov graph and $\Gamma_1(\mathcal{A})$ were introduced in [535] and [99], respectively. The graphs $\Gamma_k(\mathcal{A})$ for $k \geq 1$ are from [100, 101]. The graph $\Gamma_1(\mathcal{A})$ was referred to as the Rystsov graph in [134]. Here, however, we reserve the term Rystsov graph for the graph $I(\mathcal{A})$ originally introduced by Rystsov in [535]. In [256] an approach utilizing a sequence of increasing subgraphs of $\Gamma_1(\mathcal{A})$ was used to prove a quadratic upper bound on the reset length of automata whose transformation semigroup is the full transformation semigroup.

Gonze and Jungers [255] gave an algorithm running in time $O(|\Sigma||Q|^2)$ to compute $\Gamma_1(\mathcal{A})$. They also refuted the conjecture of Bondar and Volkov [99] that $\Gamma_1(\mathcal{A})$ is strongly connected if and only if every subset is reachable by a concatenation of words of defect one. Bondar et al. [101] gave an algorithm to compute $\Gamma_k(\mathcal{A})$ with running time $O(|\Sigma||Q|^{2k}\log(|Q|))$.

Associating to an edge (p, q) in a given digraph the simple idempotent $[p \mapsto q]$ and investigating the semigroup generated by them has been done in [128, 189]. In these works, the authors start with a digraph and construct a semigroup, whereas our digraphs $I(S)$ arise from a given semigroup S.

Rystsov [531, 532] called a set $W \subseteq \Sigma^*$ of words *k-regular* w.r.t. an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ if it contains the empty word, $W \subseteq \Sigma^{<|Q|}$ and for each $p, q \in Q$ there are exactly k words in W that map p to q. He has shown (using Equation (11.2) and other techniques) that the reset length of a synchronizing automaton admitting a k-regular set of words for some $k \geq 1$ is bounded by $2(n-1)^2$. Rystsov [531, 532] conjectured that every strongly connected automaton admits a k-regular set of words for some $k \geq 1$.

A similar notion was introduced by Carpi and D'Alessandro [130] who called an n-state automaton $\mathcal{A} = (Q, \Sigma, \delta)$ *strongly transitive* if it admits words $w_1, \ldots, w_n$ such that for every pair of states $p, q \in Q$ we have $\delta(p, w_i) = q$ for some $i \in \{1, \ldots, n\}$. They proved that if such an automaton is synchronizing, then it has a synchronizing word of length at most $(n-2)(n+L-1)+1$ where $L = \max\{|w_1|, \ldots, |w_n|\}$. This result has been extended in [131]. Carpi and D'Alessandro wrote the survey [132] on their findings. They conjectured [130] that every strongly connected synchronizing automaton admits such a sequence with $L < kn$ for some $k \geq 1$. Both conjectures have been refuted by Berlinkov [80]. He gave the modified conjecture that in a strongly connected synchronizing automaton there exists an augmentative set whose words have length at most $2n$.

Chapter 12
Classes of Automata

Here, we investigate the following subclasses of automata with respect to the notions of complete distinguishability, complete reachability, and sync-maximality: solvable automata (Section 12.1) commutative automata (Section 12.2), automata where every letter has defect at most one (Section 12.3), SI-automata (Section 12.4), circular automata (Section 12.5), where Section 12.5 also introduces circular automata, and binary automata (Section 12.6).

12.1 Automata Whose Strongly Connected Components are Blocks

In an automaton whose strongly connected components are blocks, a synchronizing state is a trap state by Theorem 7.51. Thus, by Theorem 7.24 for these automata we can decide synchronizability in linear time and constant space. Furthermore, from Theorems 9.12 and 10.1 we obtain that such an automaton is completely reachable if and only if it has a single state, and sync-maximal if and only if it has at most two states and is synchronizing with a trap state. Recall that this class includes partially ordered, commutative, and solvable automata (for partially ordered automata the time bound on deciding synchronizability was given in [300, Cor. 6]).

Theorem 12.1 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton in which every strongly connected component is a block. If $\mathcal{A}$ is completely distinguishable, then $|Q| \leq 2$.*

Proof. By Theorem 7.112, $\mathcal{A}$ is synchronizing and by Theorem 7.51 the synchronizing state must be a trap state $t \in Q$.

Since t is not the only state of $\mathcal{A}$, we can choose some maximal $p \in Q$ w.r.t. $\preceq_R$ that is not equal to t. This is equivalent to the condition that, for each $x \in \Sigma$, we have $\delta(p, x) = t$ or $\delta(p, x) \simeq_R p$. Note that in the latter case, $\delta(p, x)$ is also maximal and distinct from t.

Now, assume there exists another $q \in Q \setminus \{p, t\}$ with this property. Then the sets $\{p, q\}$ and $\{p, q, t\}$ are indistinguishable. For assume $u = u_1 \cdots u_m$ with $u_1, \ldots, u_m \in \Sigma$ distinguishes both sets. First, there must exist a minimal $i \in \{1, \ldots, m\}$ such that,

without loss of generality, $\delta(p, v_i) = t$ for $v_i = u_1 \cdots u_i$. For assume this is not the case. Then, inductively, $\delta(p, v_i) \simeq_R p$ and $\delta(q, v_i) \simeq_R q$, which also implies that none of these states equals t. Since each strongly connected component is a block, we also have $\delta(p, v_i) \neq \delta(q, v_i)$. Thus u cannot distinguish both sets. However, $\delta(p, v_i) = t$ implies $\delta(\{p, q\}, v_i) = \delta(\{p, q, t\}, v_i)$, and so u cannot distinguish both sets in this case as well. Thus, there exists no state q as above, which implies that $\{p\}$ is a strongly connected component and that every path from every state distinct from t to t has to pass through the state p.

Now, let $q \in Q$ be a state distinct from p and t that is minimal w.r.t. $\preceq_R$. A word distinguishing both sets has to map $\{q, t\}$ to a singleton, since it is a subset of the other set, which must be $\{t\}$, since t is a trap state. Now, since a prefix of a word mapping q to t has to map q to p, this prefix maps both sets to the same set and thus cannot distinguish both sets. This yields that both sets are indistinguishable. □

Recall that, by Theorem 9.45, automata with at most two states are always completely distinguishable.

12.2 Commutative Automata

As follows Section 12.1, a commutative automaton that is completely reachable has a single state, and if it completely distinguishable it has at most two states.[1]

Next, for a fixed alphabet, we give polynomial-time algorithms to compute (i) a partially ordered automaton accepting the intersection of partially ordered commutative automata (Theorem 12.2) and (ii) an automaton accepting the set of synchronizing words of a commutative automaton (Theorem 12.5). The latter result implies that there is no exponential blow-up in the size of automata accepting this set. As consequences, stated in Theorems 12.4 and 12.6, the problems Intersection and SyncShort are solvable in polynomial time for a fixed alphabet. Lastly, in Theorem 12.7 we give an expression for the set of synchronizing words of a unary automaton.

Theorems 12.2 and 12.5 were first presented in [299, Prop. 41 & Lem. 43] by the author.

Proposition 12.2 *Let $\mathcal{A}_i = (Q_i, \Sigma, \delta_i, q_i, F_i)$, $i \in \{1, \ldots, m\}$, be partially ordered commutative automata with at most n states. Then, $\bigcap_{i=1}^{m} L(\mathcal{A}_i)$ is recognizable by a partially ordered commutative automaton of size $n^{|\Sigma|}$ computable in time $O(m|\Sigma|n^{|\Sigma|+1})$.*

Proof. Fix an ordering $\Sigma = \{a_1, \ldots, a_k\}$ with $k \geq 1$. By Theorem 7.61 we obtain

$$u \in \bigcap_{i=1}^{m} L(\mathcal{A}_i) \Leftrightarrow a_1^{\min\{n-1, |u|_{a_1}\}} \cdots a_k^{\min\{n-1, |u|_{a_k}\}} \in \bigcap_{i=1}^{m} L(\mathcal{A}_i) \tag{12.1}$$

[1] By Theorem 7.74, the transformation monoid of a strongly connected commutative n-state automaton has size n. But as noted in Section 9.2.1 the transformation monoid of a completely reachable automaton has size at least $2^{|Q|} - |Q|$. This also immediately implies $|Q| \leq 2$ for completely reachable commutative automata.

as $n \geq \max\{|Q_i| : i \in \{1, \ldots, m\}\} - 1$. Next, set $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ where $Q = [n]^k$, $q_0 = (0, \ldots, 0)$,

$$\delta((s_1, \ldots, s_k), a_j) = (s_1, \ldots, s_{j-1}, \min\{n-1, s_j + 1\}, s_{j+1}, \ldots, s_k),$$

and

$$F = \left\{ (s_1, \ldots, s_k) \in Q \mid a_1^{s_1} \cdots a_k^{s_k} \in \bigcap_{i=1}^{m} L(\mathcal{A}_i) \right\}.$$

Then we have

$$\begin{aligned}
u \in \bigcap_{i=1}^{m} L(\mathcal{A}_i) &\Leftrightarrow a_1^{\min\{n-1, |u|_{a_1}\}} \cdots a_k^{\min\{n-1, |u|_{a_k}\}} \in \bigcap_{i=1}^{m} L(\mathcal{A}_i) && \text{(Eqn. (12.1))} \\
&\Leftrightarrow (\min\{n-1, |u|_{a_1}\}, \ldots, \min\{n-1, |u|_{a_k}\}) \in F && \text{(Def. } F\text{)} \\
&\Leftrightarrow \delta(q_0, a_1^{|u|_{a_1}} \cdots a_k^{|u|_{a_k}}) \in F && \text{(Def. } \delta\text{)} \\
&\Leftrightarrow a_1^{|u|_{a_1}} \cdots a_k^{|u|_{a_k}} \in L(\mathcal{A}) && \text{(Def. } L(\mathcal{A})\text{)} \\
&\Leftrightarrow u \in L(\mathcal{A}). && (\mathcal{A} \text{ commutative})
\end{aligned}$$

So $\bigcap_{i=1}^{m} L(\mathcal{A}_i) = L(\mathcal{A})$.

The automaton, without F, can clearly be computed in time $O(n^k)$. To compute F, iterate through all n^k words $a_1^{s_1} \cdots a_k^{s_k}$ with $(s_1, \ldots, s_k) \in [n]^k$ and test for each word whether $a_1^{s_1} \cdots a_k^{s_k} \in L(\mathcal{A}_i)$ for all automata $\mathcal{A}_i$, $i \in \{1, \ldots, m\}$. For $i \in \{1, \ldots, m\}$, testing a single word $a_1^{s_1} \cdots a_k^{s_k}$ of length in $O(kn)$ whether it is accepted by $\mathcal{A}_i$ can be done in time $O(kn)$. So testing whether it is accepted by all automata $\mathcal{A}_i, i \in \{1, \ldots, m\}$, can be done in time $O(mkn)$. In total, the algorithm runs in time $O(mkn^{k+1})$. □

Remark 12.3 The bound from Theorem 12.2 is tight. For example, set $\Sigma = \{a_1, \ldots, a_m\}$ and for $i \in \{1, \ldots, m\}$ consider $\mathcal{A}_i = ([n], \Sigma, \delta_i, 0, \{n-1\})$ with $\delta(j, a_i) = \min\{j + 1, n-1\}$ and $\delta(j, a_k) = j$ for $k \neq i$ and $j \in [n]$. We have $L(\mathcal{A}_i) = \{u \in \Sigma^* : |u|_{a_i} \geq n\}$ and $L = \bigcap_{i=1}^{m} L(\mathcal{A}_i) = \{u \in \Sigma^* : \forall i \in \{1, \ldots, k\} \colon |u|_{a_i} \geq n\}$. Let $0 \leq i_1, \ldots, i_k, j_1, \ldots, j_k < n$ where $i_r < j_r$ for some $r \in \{1, \ldots, k\}$. Set $u = a_1^{i_1} \cdots a_k^{i_k}$, $v = a_1^{j_1} \cdots a_k^{j_k}$, and $w = a_1^{n-j_1} \cdots a_k^{n-j_k}$. Then

$$uw \notin L \text{ and } vw \in L.$$

Thus $u \not\equiv_L v$. This yields that the words $a_1^{i_1} \cdots a_k^{i_k}$ for $0 \leq i_1, \ldots, i_k < n$ induce distinct Myhill-Nerode equivalence classes. Then every automaton accepting L has at least n^k states and thus the bound from Theorem 12.2 is reached. This also follows from results in [312] where the state complexity on commutative languages is investigated. The above languages were used in the proof of [112, Thm. 1], so that this proof established the above in the special case $m = 2$.

Corollary 12.4 INTERSECTION *is solvable in time* $O(m|\Sigma|n^{|\Sigma|+1})$ *for m partially ordered input automata each having at most n states.*

Proof. The input automata accept a common word if and only if the final state set in the automaton constructed in the proof of Theorem 12.2 is non-empty. So, we can use the same algorithm as in the proof of Theorem 12.2 and simply test whether $F \neq \emptyset$ at the end. □

Proposition 12.5 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative automaton with n states. Then, the set of synchronizing words is recognizable by a partially ordered commutative automaton of size $n^{|\Sigma|}$ computable in time $O(|\Sigma| n^{|\Sigma|+2})$.*

Proof. By Theorem 7.83, for $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$, $\mathcal{A}$ must have a unique trap state. So, first check whether $\mathcal{A}$ has a unique trap state $t \in Q$, and compute it if present, which can be done in time $O(n)$ by Theorem 7.22. In case $\mathcal{A}$ has no trap state, then $\mathrm{Syn}(\mathcal{A}) = \emptyset$ so that a one-state automaton with empty final state set accepts $\mathrm{Syn}(\mathcal{A})$. Otherwise, as, by Theorem 7.63, $\mathcal{A}$ is a ρ-automaton with a trap state, we find $\mathrm{Syn}(\mathcal{A}) = \mathrm{Syn}(\mathcal{A}/\simeq_R)$ by Theorem 7.65. As a quotient automaton of a commutative automaton, $\mathcal{A}/\simeq_R = (Q/\simeq_R, \Sigma, \delta_{\simeq_R})$ is commutative, and has the trap state $[t]_{\simeq_R} = \{t\}$. By Theorem 7.64, $\mathcal{A}/\simeq_R$ is a partially ordered automaton computable in linear time. As, by Theorem 7.6, Item 2, $[t]_{\simeq_R}$ must be the unique synchronizing state, we have

$$\mathrm{Syn}(\mathcal{A}/\simeq_R) = \bigcap_{[q]_{\simeq_R} \in Q/\simeq_R} L((\mathcal{A}/\simeq_R)_{[q]_{\simeq_R}, \{[t]_{\simeq_R}\}}).$$

By Theorem 12.2 we can compute a partially ordered commutative automaton of size $n^{|\Sigma|}$ accepting this intersection involving n automata, each having n states, in time $O(|\Sigma| n^{|\Sigma|+2})$.

The above intersection for $\mathrm{Syn}(\mathcal{A}/\simeq_R)$ equals the same intersection but only over the minimal states $Q/\simeq_R$, which might give a slightly better running time for the algorithm. □

We note a direct consequence of the previous result.

Corollary 12.6 SyncShort *is solvable in time $O(|\Sigma| n^{|\Sigma|+2})$ for n-state commutative input automata $\mathcal{A} = (Q, \Sigma, \delta)$.*

Proof. By Theorem 12.5, we can compute an automaton of size $n^{|\Sigma|}$ for $\mathrm{Syn}(\mathcal{A})$ in time $O(|\Sigma| n^{|\Sigma|+2})$. Then a shortest word accepted by this automaton can be computed in time $O(|\Sigma| n^{|\Sigma|})$ as a shortest path from the start state to some final state [163]. □

Martyugin [436, Prop. 5.2] provided a direct algorithm to decide SyncShort running in time $O(|\Sigma| n^{|\Sigma|} \log n)$ for commutative input automata (he actually omits the factor $|\Sigma|$ as he assumed a fixed alphabet; however, the proof shows that this is the running time when the alphabet size is taken into account).

Lastly, we note the following improvement for unary languages.

Proposition 12.7 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ be a synchronizing unary automaton. Then $\mathrm{Syn}(\mathcal{A}) = a^l a^*$ where l is the length of a longest path traversing distinct states in $\mathcal{A}$. In particular, $\mathrm{Syn}(\mathcal{A}) = a^l a^*$ is acceptable by an automaton with at most $l + 1 \leq |Q|$ states.*

Proof. A synchronizing unary automaton consists of tails all ending at a common state at which the single letter induces a self-loop. Now, it is easy to see that the set of synchronizing words has the form as claimed. Clearly, $l < |Q|$ and $a^l a^*$ is acceptable by an automaton with $l + 1$ states. □

The smallest automaton whose set of synchronizing words is $a^l a^*$ equals the minimal automaton accepting this language over $\{a\}$. Thus the reset complexity and the state complexity of $a^l a^*$ both equal $l + 1$. This was also stated in [440, Prop. 1].

12.3 All Letters Have Defect at Most One

Here, Theorem 12.8 shows that for automata where every letter has defect at most one and whose transformation monoid contains a transitive permutation group, synchronizability implies complete distinguishability. Theorem 12.10 lists some equivalences that follow from Theorem 12.8. Lastly, in Theorems 12.11 to 12.13, we conclude that in automata with a single non-permutational letter and for binary automata, complete reachability and sync-maximality are equivalent properties, and that every binary completely reachable is sync-maximal. The latter statement answers the open question from [307] whether there exist binary completely reachable automata that are not sync-maximal.

Theorem 12.8 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton such that the group of units of $\mathcal{T}_M(\mathcal{A})$ is transitive and every letter has defect at most one. If $\mathcal{A}$ is synchronizing, then it is completely distinguishable.*

Proof. Since all automata with at most two states are completely distinguishable by Theorem 9.45, we can assume $|Q| \geq 3$. Let $u \in \Sigma^*$ be a shortest word such that $\delta(Q, u) = \{q\}$ for some $q \in Q$. Then we can write $u = xv$ with $x \in \Sigma$ and $v \in \Sigma^*$. By the minimal choice of u, we have $|\delta(Q, v)| \geq 2$ and $\delta(Q, x) \neq Q$. By assumption, x has defect one, so that $\delta(Q, x) = Q \setminus \{p\}$ for some state $p \in Q$. Hence, as $\delta(Q \setminus \{p\}, v) = \{q\}$, we get

$$\delta(Q, v) = \{\delta(p, v)\} \cup \delta(Q \setminus \{p\}, v) = \{\delta(p, v), q\}$$

where $\delta(p, v) \neq q$. Now, let $r \in Q$. As $\mathcal{T}_M(\mathcal{A})$ contains a transitive permutation group, there exists $w \in \Sigma^*$ inducing a permutation on Q such that $\delta(r, w) = p$. Then $\delta(Q \setminus \{r\}, wv) = \delta(Q \setminus \{p\}, v) = \{q\}$ and $\delta(r, wv) = \delta(p, v)$. As $\delta(r, wv) \notin \delta(Q \setminus \{r\}, wv)$ and $|\delta(Q \setminus \{r\}, wv)| = 1$, by Theorem 9.46, Item 4, it follows that $\mathcal{A}$ is completely distinguishable. □

Remark 12.9 The automaton from Theorem 9.58, and those from Figure 9.7 given later, only have simple idempotent letters (and so of defect one), are completely reachable, but they are not completely distinguishable. Thus, strong connectivity and synchronizability do not imply complete distinguishability for these automata.

Corollary 12.10 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton such that the group of units of $\mathcal{T}_M(\mathcal{A})$ is transitive (equivalently $\mathcal{A}_{|\Gamma,Q}$ for $\Gamma = \{x \in \Sigma \mid x \text{ is a permutational letter}\}$ is strongly connected) and every letter has defect at most one with $|Q| \geq 5$. Then the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A}$ is completely distinguishable.*
3. *Every k-sets of states is distinguishable from every distinct l-set of states for some $2 \leq k, l \leq |Q|$ with $|Q| > k$ or $|Q| > l$.*
4. *For two distinct non-empty and non-singleton subsets $A, B \subseteq Q$ we have $|\delta(A, u)| \neq |\delta(B, u)|$ for some $u \in \Sigma^*$.*
5. *Every k-set of states is distinguishable from every disjoint l-set of states for some $k, l \geq 2$ with $kl < |Q|$.*

Proof. $1 \Rightarrow 2$: This follows from Theorem 12.8.[2]

$2 \Rightarrow 3$: This is obvious.

$3 \Rightarrow 1$: For $k \geq 3$ or $l \geq 3$, by Theorem 9.30 the automaton $\mathcal{A}$ is synchronizing. For $k = l = 2$, since by assumption $\mathcal{A}$ is strongly connected it follows from Theorem 9.32 that $\mathcal{A}$ is synchronizing.

$2 \Leftrightarrow 4$: If $\mathcal{A}$ is completely distinguishable, then precisely one subset from A or B is mapped to a singleton while the other is mapped to a set of size at least two. Thus, Item 4 holds. Conversely, if Item 4, then distinct pairs of 2-sets are distinguishable and since the equivalence of Item 3 and Item 2 has already been established, we obtain that the automaton is completely distinguishable.

$5 \Leftrightarrow 1$: First, suppose Item 5 holds. Then, Theorem 9.35 applied to $\mathcal{T}_M(\mathcal{A})$ yields that $\mathcal{A}$ is synchronizing. Conversely, if $\mathcal{A}$ is synchronizing, then since the other equivalences have already been established it is completely distinguishable, in particular the pairs of sets as stated are distinguishable. □

In general, distinguishability of pairs of distinct 2-sets does not imply synchronizability even for SI-automata, see Theorem 9.33. For strongly connected automata, distinguishability of pairs of distinct 2-sets does imply synchronizability by Theorem 9.32, but it does not imply complete distinguishability even for strongly connected SI-automata, see the examples from Section 9.3.5. Theorems 9.58 and 9.59 showed that, in general, the ability to map two distinct non-singleton subsets to subsets of different cardinality is not sufficient for complete distinguishability.

Corollary 12.11 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with at least three states and a single non-permutational letter. Then $\mathcal{A}$ is completely reachable if and only if it is sync-maximal.*

---

[2] For SI-automata, this also follows from the graph-theoretical methods from Chapter 11. For consider the Rystsov graph $I(\mathcal{T}_M(\mathcal{A}))$. By the assumption the group of units of $\mathcal{T}_M(\mathcal{A})$ is transitive. As the group of units is part of the automorphism group of $I(\mathcal{T}_M(\mathcal{A}))$ (Proposition 11.17) the graph $I(\mathcal{T}_M(\mathcal{A}))$ is vertex-transitive. As the completely reachable automaton $\mathcal{A}$ is synchronizing (Lemma 9.12), by Theorem 11.11 and Theorem 11.14, Item 4 the graph $I(\mathcal{T}_M(\mathcal{A}))$ is strongly connected. By Theorem 5.45 it is strongly $\frac{3}{2}$-connected. So Theorem 11.36 yields that $\mathcal{A}$ is completely distinguishable.

Proof. First, suppose $\mathcal{A}$ is completely reachable. By Theorem 9.19 the group of units of $\mathcal{T}_M(\mathcal{A})$ is transitive and the single non-permutational letter has defect one. So, by Theorem 12.8, since $\mathcal{A}$ is synchronizing we find that it is completely distinguishable. From Theorem 10.7, Item 1 we get that $\mathcal{A}$ is sync-maximal.

Conversely, suppose $\mathcal{A}$ is sync-maximal. If $\mathcal{A}$ is not completely reachable, then, by Theorem 10.7, it is not strongly connected, and so, by Theorem 10.15, Item 1, has two distinct non-permutational letters. Hence, as $\mathcal{A}$ has only one non-permutational letter, it must be completely reachable. □

Corollary 12.12 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a binary automaton with at least three states. Then $\mathcal{A}$ is completely reachable if and only if it is sync-maximal.*

Proof. First, suppose $\mathcal{A}$ is completely reachable. By Theorem 9.21, $\mathcal{A}$ has precisely one non-permutational letter. So, by Theorem 12.11cor:single-perm:1, it is sync-maximal. Conversely, by Theorem 10.7 and Theorem 10.15, a binary sync-maximal automaton with at least three states is completely reachable. □

Corollary 12.13 *Every binary completely reachable automaton is sync-maximal.*

Proof. This follows from Theorem 12.12 since by Theorem 10.2 synchronizing automata with at most two states are sync-maximal. □

Theorems 12.11 and 12.12 do not hold for two-state automata, as by Theorem 10.2, Item 2 every two-state synchronizing automaton is sync-maximal, but there exist two-state synchronizing automata that are not strongly connected, and so not completely reachable.

As complete reachability is decidable in polynomial time [205], it follows from Theorem 12.12 that, for binary automata, sync-maximality is also decidable in polynomial time. We note that Casas and Volkov [134] gave a characterization of binary completely reachable automata (see also the discussion in Section 12.7), as well as a faster polynomial-time algorithm than the general one from [205] for deciding complete reachability in the binary case.

12.4 Automata with Simple Idempotents

First, in Section 12.4.1, we prove that in a synchronizing SI-automaton every pair of distinct 2-sets is distinguishable. Lastly, in Section 12.4.2 we investigate strongly connected SI-automata and derive conditions connecting synchronizability, complete reachability, and complete distinguishability. We also derive these connections for binary SI-automata in this section.

12.4.1 Distinguishability of 2-Sets

Here, in Theorem 12.20, we prove that in a synchronizing SI-automaton all pairs of distinct 2-sets are distinguishable. This follows from a series of preliminary results regarding mapping properties of equal-sized sets.

Lemma 12.14 *Let $t\colon X \to X$ be an one-point singular and $A, B \subseteq X$ be finite with $|A| = |B|$ and $t(A) = t(B)$. Then* $\mathrm{move}(t) \cap A = \mathrm{move}(t) \cap B$ *implies $A = B$.*

Proof. Let $x \in X$ be such that $\mathrm{dupl}(t) = \{x\}$. Consider $z \in A$ with $z \neq x$. If $t(z) \neq z$, then $z \in B$ by assumption. Next, suppose $t(z) = z$. Since $t(A) = t(B)$, we have $t(z) = t(y)$ for some $y \in B$. As t is a one-point singular and $t(z) \neq x$, we find $t^{-1}(t(z)) = \{z\}$. Therefore $y = z$ and so $z \in B$. This gives $A \setminus \{x\} \subseteq B$. Similarly, we obtain the other inclusion and thus $A \setminus \{x\} = B \setminus \{x\}$. Since $|A| = |B|$, either $x \in A \cap B$ or $x \notin A \cup B$. This gives $A = B$. □

Corollary 12.15 *Let $t\colon X \to X$ be a simple idempotent and $A, B \subseteq X$ be finite. Suppose $|A| = |B|$, $t(A) = t(B)$, $t(A) \neq A$, and $t(B) \neq B$. Then $A = B$.*

Proof. Let $\mathrm{move}(t) = \{a\}$. Since $t(A) \neq A$ and $t(B) \neq B$, we have $a \in A \cap B$. Thus, the claim follows from Theorem 12.14. □

By Theorem 12.14, if $A \neq B$ with $|A| = |B|$ and $t(A) = t(B)$, then, without loss of generality, $t(B) \neq B$ and $t(A) = A$ and so $t(x) = x$ for all $x \in A$.

Lemma 12.16 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton, $A, B, C \subseteq Q$ be k-sets, and let $u \in \Sigma^*$ be a shortest word such that $\delta(A, u) = C$ or $\delta(B, u) = C$. If $A \neq B$, then $\delta(A, u) \neq \delta(B, u)$.*

Proof. Suppose $\delta(A, u) = \delta(B, u)$. Next, we prove $A = B$ by induction on $m = \sum_{a \in \Sigma_I} |u|_a$. If $m = 0$, then $u \in \Sigma_G^*$. So, as u permutes the states, $\delta(A, u) = \delta(B, u)$ implies $A = B$. Otherwise, write $u = vaw$ with $v \in \Sigma^*$, $a \in \Sigma_I$, and $w \in \Sigma_G^*$. Then $\delta(A, va) = \delta(B, va)$ since w permutes the states. As u has shortest length, we must have $\delta(\delta(A, v), a) \neq \delta(A, v)$ and $\delta(\delta(B, v), a) \neq \delta(B, v)$, as otherwise vu would give a shorter word mapping at least one of A or B onto C. Since $|C| \leq |\delta(A, v)| \leq |A|$ and $|C| \leq |\delta(B, v)| \leq |B|$ by Equation (3.17), and $|A| = |B| = |C|$ by assumption, we have $|\delta(A, v)| = |\delta(B, v)|$. So, by Theorem 12.14 applied to δ_a, we get $\delta(A, v) = \delta(B, v)$. Since u was chosen minimal such that $\delta(A, u) = C$ or $\delta(B, u) = C$, also v has shortest length such that $\delta(A, v) = C'$ or $\delta(B, v) = C'$ for $C' = \delta(A, v) = \delta(B, v)$. By induction hypothesis, we conclude that $A = B$. □

Remark 12.17 The assumption that the sets A, B, C have the same size in Theorem 12.16 can not be dropped. For example, consider the SI-automaton with states $\{1, 2, 3\}$ and two simple idempotent letters a and b that map 1 to 2 and 2 to 3, respectively. Then the word ab maps the two distinct sets $\{1, 2\}$ and $\{2, 3\}$ to $\{3\}$.

Corollary 12.18 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton and $p, q \in Q$ be distinct states. Then for a shortest word $u \in \Sigma^*$ with $\delta(p, u) = r$ or $\delta(q, u) = r$ for some $r \in Q$ we have $\delta(p, u) \neq \delta(q, u)$.*

Proof. This is Theorem 12.16 for the special case $k = 1$. □

Proposition 12.19 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton and $A, B \subseteq Q$ be two distinct 2-sets. Then for a shortest word $u \in \Sigma^*$ with $|\delta(A, u)| = 1$ or $|\delta(A, u)| = 1$ we have $|\delta(A, u)| \geq 2$ or $|\delta(A, u)| \geq 2$.*

Proof. Let $q \in Q$ be the unique state with $\delta(q, a) \neq q$. As u has shortest length mapping at least one to a singleton subset, we can write $u = va$ with $v \in \Sigma^*$ and $a \in \Sigma_I$ where v has minimal length such that $\delta(A, v) = \{q, \delta(q, a)\}$ or $\delta(B, v) = \{q, \delta(q, a)\}$. Now, suppose $|\delta(A, u)| = |\delta(B, u)| = 1$. By the minimal choice of u, we must have $|\delta(A, v)| = |\delta(B, v)| = 2$. Since both sets are mapped to a singleton, we find $\delta(A, v) = \delta(B, v) = \{q, \delta(q, a)\}$. Then we have $|A| = |B| = |\delta(A, v)|$. Therefore, by Theorem 12.16, since v has shortest length mapping at least one to $\{q, \delta(q, a)\}$, we conclude $A = B$. So, with the assumption $A \neq B$, we find that at least one of both subsets is mapped to a non-singleton set. □

Corollary 12.20 *In a synchronizing SI-automaton every two distinct 2-sets are distinguishable.*

Proof. By Proposition 12.19, a shortest word mapping at least one of two distinct 2-sets to a singleton set must map the other set to a non-singleton set. Hence, it distinguishes both 2-sets. □

Recall that Theorem 9.58 gave synchronizing SI-automata that are not completely distinguishable.

12.4.2 Strongly Connected SI-Automata

Here, in Theorem 12.21, we show that for strongly connected SI-automata, synchronizability, complete reachability, and distinguishability of pairs of 2-sets are equivalent properties. As a consequence, stated in Theorem 12.22, completely reachable SI-automata are precisely those that are synchronizing and strongly connected. Theorem 12.24 and Theorem 12.26 link strong connectivity and strong trap-connectivity to complete reachability and absence of non-trivial congruences in SI-automata without permutational letters. Theorem 12.28 states various equivalences for SI-automata with a single non-permutational letter, and Theorem 12.29 for binary SI-automata.

Theorem 12.21 *For a strongly connected n-state SI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with $n \geq 3$ the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A}$ is completely reachable.*
3. *Every automaton for $\mathrm{Syn}(\mathcal{A})$ needs $\binom{n}{2} + 1$ states.*
4. *All pairs of distinct 2-sets of states are distinguishable.*

Proof. $1 \Rightarrow 2$: Suppose $\mathcal{A}$ is synchronizing. Let $I(\mathcal{A}) = (Q, E)$ be the Rystsov graph of $\mathcal{A}$. By Theorem 11.11 this graph is strongly connected. By Theorem 11.14 it is a subgraph of $I(\mathcal{T}_M(\mathcal{A}))$. From Theorem 11.31 it follows that $\mathcal{T}_M(\mathcal{A})$, and so $\mathcal{A}$, is completely reachable.[3]

$2 \Rightarrow 3$: Since a completely reachable automaton is synchronizing, from Theorem 12.20 we get that distinct pairs of 2-sets are distinguishable (alternatively, it also follows from Theorem 11.31 as in the proof of the previous implication). Furthermore, all 2-sets are reachable and so $\{\{p, q\} \mid p, q \in Q, p \neq q\} \cup \{\{s\}\}$ for some $s \in Q$ yields a family of pairwise distinguishable and reachable subsets. By Theorem 9.10, it follows that $\mathrm{Syn}(\mathcal{A})$ has at least $\binom{n}{2} + 1$ states.

$3 \Rightarrow 4$: As $n \geq 3$ the minimal automaton for $\mathrm{Syn}(\mathcal{A})$ has at least five states. So, $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$, i.e., $\mathcal{A}$ is synchronizing, and from Theorem 12.20 we get that distinct pairs of 2-sets are distinguishable.

$4 \Rightarrow 1$: As $\mathcal{A}$ is strongly connected and $n \geq 3$, by Proposition 9.32 the automaton $\mathcal{A}$ is synchronizing. □

Figure 9.7 gives examples of strongly connected and completely reachable SI-automata that are not completely distinguishable (and so not sync-maximal).

Theorem 12.21 fails for synchronizing SI-automata that are not strongly connected since they are not completely reachable, as, for example, the SI-automaton from Figure 7.4c.

Corollary 12.22 *An SI-automaton is strongly connected and synchronizing if and only if it is completely reachable.*

Proof. By Theorem 12.21, Item 2, a strongly connected and synchronizing SI-automaton is completely reachable. Theorem 9.12 yields the other implication. □

Strong connectivity and synchronizability do not imply complete reachability even for automata where every letter has defect at most one, see Figure 12.1.

Corollary 12.23 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton such that the group of units of $\mathcal{T}_M(\mathcal{A})$ is transitive. Then the following are equivalent.*

---

[3] For an alternative proof: Let $a_1, \ldots, a_k$ be the simple idempotent letters where we have $k \geq 1$ since $\mathcal{A}$ is synchronizing. By assumption, for each $q \in Q$ there exists $t \in \mathcal{T}_M(\mathcal{A})$ such that $t(Q) = \{q\}$. We can write $t = g_0 a_{i_1} g_1 a_{i_2} \cdots g_{m-1} a_{i_m} g_m$ for permutations $g_0, g_1, \ldots, g_m \in \mathcal{T}_M(\mathcal{A})$ and numbers $1 \leq i_1, i_2, \ldots, i_m \leq k$ where we write a_i for δ_{a_i} here as a shorthand. Since

$$t = a_{i_1}^{g_0} a_{i_2}^{g_0 g_1} \cdots a_{i_m}^{g_0 g_1 \cdots g_m} g_0 g_1 \cdots g_m,$$

we conclude that $a_{i_1}^{g_0} a_{i_2}^{g_0 g_1} \cdots a_{i_m}^{g_0 g_1 \cdots g_m}$ already maps Q to $\{q\}$. By considering the transformations a_i^g for a permutation $g \in \mathcal{T}_M(\mathcal{A})$ as letters in a new automaton $\mathcal{B}$ (Automata constructed by introducing new symbols for existing words or transformations were also considered in [21, p. 270]. We will also define automata in this way in Chapter 15, for example, in the proof of Theorem 15.5.), we find that $\mathcal{B}$ must be synchronizing and, since each state is synchronizing, strongly connected. Each a_i^g induces a letter of simple idempotent, and so it induces an edge in $\Gamma_1(\mathcal{A})$. Now, the automaton graph of $\mathcal{B}$ with its self-loops omitted is a subgraph of $\Gamma_1(\mathcal{A})$ since the unique transition of a simple idempotent that is not a self-loop correspond to an edge of $\Gamma_1(\mathcal{A})$ (this does not hold in general, since some transitions for a letter might not induce edges in $\Gamma_1(\mathcal{A})$). So $\Gamma_1(\mathcal{A})$ is strongly connected. By Theorem 11.2 we find that $\mathcal{B}$, and so $\mathcal{A}$, is completely reachable.

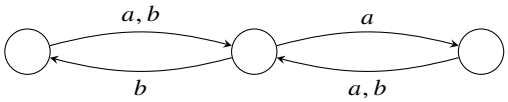


Fig. 12.1: Strongly connected and synchronizing automaton that is not completely reachable.

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A}$ is completely reachable.*
3. *$\mathcal{A}$ is completely distinguishable.*

Note that we also have the equivalences from Theorem 12.10 for SI-automata where the group of units of its transformation monoid is transitive.

Corollary 12.24 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton without permutational letters. Then $\mathcal{A}$ is completely reachable if and only if it is strongly connected.*

Proof. Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is a strongly connected SI-automaton without permutational letters. Let $A \subseteq Q$ be a maximal subset such that $|\delta(A, u)| = 1$ for some $u \in \Sigma^*$. Assume $|A| < |Q|$. As $\mathcal{A}$ is strongly connected, there exists a transition labeled by some letter $a \in \Sigma$ starting from some state in $Q \setminus A$ and ending in A, i.e. $\delta(p, a) \in A$ for some $p \in Q \setminus A$. As a is a simple idempotent, we have $\delta(A \cup \{p\}, a) = A$. Then $|\delta(A \cup \{p\}, au)| = 1$, contradicting the maximal choice of A. So, we must have $A = Q$ and $\mathcal{A}$ is synchronizing. Then the claim follows from Theorem 12.22.[4] □

Proposition 12.25 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton with at least three states. Then $\mathcal{A}$ does not admit some non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence and has some simple idempotent letter if and only if it is strongly trap-connected, or it is strongly connected and synchronizing.*

Proof. First, assume $\mathcal{A}$ does not admit some non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence and has some simple idempotent letter. By [610, Cor. 2.3] an automaton not admitting non-trivial congruences and having a letter inducing a one-point singular is synchronizing. Thus, $\mathcal{A}$ is synchronizing. By Theorem 6.68, it is strongly connected or strongly trap-connected.

Conversely, first suppose $\mathcal{A}$ is strongly trap-connected with trap state t. Let $\sim$ be some $\mathcal{T}_S(\mathcal{A})$-congruence and suppose $p \sim q$ for distinct states $p, q \in Q$. Consider a shortest word u mapping p or q to t. By Theorem 12.18 it maps only one of both states to t and the other state remains in $Q \setminus \{t\}$. Without loss of generality, let $\delta(p, u) = t$ and $\delta(q, u) \in Q \setminus \{t\}$. Let $r \in Q$ be some state with $r \neq t$. Since $Q \setminus \{t\}$ is strongly connected, we have $\delta(q, uv) = r$ for some $v \in \Sigma^*$. With $\delta(p, u) \sim \delta(q, u)$, this implies $t \sim r$. Since r was chosen arbitrarily in $Q \setminus \{t\}$, we find that every state is congruent to t and thus $\sim$ is the universal relation.

[4] For an alternative argument, for such an automaton, the automaton graph with its self-loops omitted equals its Rystsov graph $I(\mathcal{A})$. Thus it is strongly connected and so $\mathcal{A}$ is completely reachable by Theorem 11.31.

Next, suppose $\mathcal{A}$ is strongly connected and synchronizing. By Theorem 11.11, $I(\mathcal{A})$ is strongly connected. Since $I(\mathcal{A})$ is a subgraph of $I(\mathcal{T}_S(\mathcal{A}))$ by Theorem 11.14, Item 4, from Theorem 11.39 we get that $\mathcal{A}$ admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence. □

SI-automata without simple idempotent letters whose group of units is primitive do not admit non-trivial congruences and are not synchronizing (and so also not strongly trap-connected).

Corollary 12.26 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton with at least three states without permutational letters. Then $\mathcal{A}$ does not admit some non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence if and only if it is strongly trap-connected or strongly connected.*

Proof. If $\mathcal{A}$ does not admit any non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence, then, by Theorem 6.68, it is strongly connected or strongly trap-connected. The converse implication follows from Theorem 12.25 and the fact that if $\mathcal{A}$ is strongly connected, then it is synchronizing by Theorem 12.24. □

Proposition 12.27 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton with at least three states and a single simple idempotent letter. Then the following implies that the group of units G of $\mathcal{T}_M(\mathcal{A})$ is transitive:*

1. *$\mathcal{A}$ is strongly connected.*
2. *Every k-set is distinguishable from every distinct l-set of states for some $2 \leq k, l \leq |Q|$ with $\min\{k, l\} < |Q|$ and $\max\{k, l\} > 2$.*

Proof. 1: Suppose G is not transitive and let $\mathrm{Orb}_G(p)$ and $\mathrm{Orb}_G(q)$ be two distinct orbits for $p, q \in Q$. If the single idempotent letter connects two states in one of these orbits, then no state from one orbit is reachable from a state from the other orbit and so $\mathcal{A}$ is not strongly connected. Otherwise, if, without loss of generality, this letter connects a state from $\mathrm{Orb}_G(p)$ with a state from $\mathrm{Orb}_G(q)$, then no state from $\mathrm{Orb}_G(p)$ is reachable from a state from $\mathrm{Orb}_G(q)$ and so $\mathcal{A}$ is not strongly connected.

2: Without loss of generality, suppose $k \leq l$. Let $a \in \Sigma$ be the simple idempotent letter and $\delta(p, a) = q$ with $p \neq q$. From Theorem 9.30 we get that $\mathcal{A}$ is synchronizing. The orbits $\mathrm{Orb}_G(p)$ and $\mathrm{Orb}_G(q)$ are the only orbits of G, for if there exists another orbit, then it is mapped to itself by every letter and no letter maps some state into it, which implies that $\mathcal{A}$ cannot be synchronizing. For $\mathrm{Orb}_G(p) \neq \mathrm{Orb}_G(q)$, there is no letter mapping two distinct states in $\mathrm{Orb}_G(q)$ to one state and so $\mathrm{Orb}_G(q) = \{q\}$ and q is a trap state. We can pick distinct $s, t \in A$ as $|Q| \geq 3$. Since $l > 2$ and $2 \leq k < |Q|$ there exists a k-set A with $s, t \in A$ and an l-set B with $q, s, t \in B$ such that $A \neq B$. The only way to map two states to one state is to use the letter a, which yields that a word mapping two states to one state maps them both to q. Thus, $|\delta(A, u)| = 1$ if and only if $\delta(A, u) = \{q\}$ if and only if $\delta(B, u) = \{q\}$ if and only if $|\delta(B, u)| = 1$, which yields that A and B are indistinguishable. So, G must be transitive. □

Corollary 12.28 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton with at least three states and a single simple idempotent letter. Then the following are equivalent.*

1. *$\mathcal{A}$ is strongly connected and synchronizing.*

2. *$\mathcal{A}$ is completely reachable.*
3. *$\mathcal{A}$ is sync-maximal.*
4. *$\mathcal{A}$ is completely distinguishable.*
5. *Every k-set is distinguishable from every distinct l-set of states for some $2 \leq k, l \leq |Q|$ with $\min\{k,l\} < |Q|$ and $\max\{k,l\} > 2$.*
6. *$\mathcal{A}$ is strongly connected and all pairs of distinct 2-sets are distinguishable.*

Proof. $1 \Rightarrow 2$: This follows from Theorem 12.22,

$2 \Rightarrow 3$: As it has only one non-permutational letter, by Theorem 9.19, we find that the group of units of of $\mathcal{T}_M(\mathcal{A})$ is transitive.[5] So, since $|Q| \geq 3$, $\mathcal{A}$ is completely distinguishable by Theorem 12.10 and thus it is sync-maximal by Theorem 10.3.[6]

$3 \Rightarrow 4$: By Theorem 10.1, $\mathcal{A}$ is completely distinguishable.

$4 \Rightarrow 5$: This is obvious.

$5 \Rightarrow 6$: By Theorem 12.27, the group of units of $\mathcal{T}_M(\mathcal{A})$ is transitive. Since every state can be mapped to any other state by a permutational word, $\mathcal{A}$ is strongly connected. By Theorem 12.21 we find that all pairs of distinct 2-sets are distinguishable.

$6 \Rightarrow 1$: This follows from Theorem 9.32. □

Next, we state equivalences for binary SI-automata derived from Theorem 12.28.

Corollary 12.29 *Let $\mathcal{A} = (Q, \{a,b\}, \delta)$ be a binary SI-automaton with at least three states. Then the following are equivalent.*

1. *$\mathcal{A}$ is strongly connected and synchronizing.*
2. *$\mathcal{A}$ is completely reachable.*
3. *$\mathcal{A}$ is sync-maximal.*
4. *$\mathcal{A}$ is completely distinguishable.*
5. *Every k-set is distinguishable from every distinct l-set of states for some $2 \leq k, l \leq |Q|$ with $\min\{k,l\} < |Q|$ and $\max\{k,l\} > 2$.*
6. *$\mathcal{A}$ is strongly connected and all pairs of distinct 2-sets are distinguishable.*
7. *In $\mathcal{A}$, one letter, say $b \in \Sigma$, cyclically permutes the states and the other letter $a \in \Sigma$ is a simple idempotent such that $\delta(q,a) = \delta(q, b^d a)$ for some $q \in Q$ and $0 < d < n$ coprime to n (see Figure 7.4d).*

[5] Alternatively, this also follows from Theorem 12.27, or with the following argument: for a completely reachable SI-automaton $\mathcal{A}$, by Theorem 9.12 and Theorem 11.11, $I(\mathcal{A})$ is strongly connected. In case $\mathcal{A}$ has only a single simple idempotent mapping s to t, $I(\mathcal{A})$ equals the directed orbital graph of (s,t) w.r.t. the group of units G of $\mathcal{T}_M(\mathcal{A})$. Then, strong connectivity implies that each $q \in Q$ has an outgoing edge $(g(s), g(t))$, $g \in G$. So, $q = g(s)$ for some $g \in G$ and so G is transitive.

[6] For an alternative proof of complete distinguishability: Let $p, q \in Q$ with $\mathrm{excl}(a) = \{p\}$ and $\mathrm{dupl}(a) = \{q\}$ for the unique simple idempotent letter $a \in \Sigma$. Consider the automaton $\mathcal{B}$ with letters a^g where a^g acts as the conjugate of the action of the letter a for some g from the group of units of $\mathcal{T}_M(\mathcal{A})$. Since $\mathcal{A}$ is completely reachable, as in the proof of Theorem 12.21 presented in the footnote, we find that this automaton is strongly connected. Since a^g maps $g(p)$ to $g(q)$ by Equation (5.2), the automaton graph of $\mathcal{B}$ equals the orbital digraph of (p,q) of the group of units of $\mathcal{T}_M(\mathcal{A})$ and it also equals $I(\mathcal{A})$. In this orbital digraph, by strong connectivity, every state has an outgoing transition, which implies that the group of units must be transitive since then there must exist a permutation mapping p to every other state. Thus it is vertex-transitive and since $p \neq q$ it has no self-loops, by Theorem 5.45, we find that the orbital digraph is strongly $\frac{3}{2}$-connected. Since $I(\mathcal{A})$ is a subgraph of $I(\mathcal{T}_M(\mathcal{A}))$ by Theorem 11.14, from Theorem 11.36, we find that $\mathcal{A}$ is completely distinguishable.

Proof. It is easily seen that the only synchronizing binary SI-automata without permutational letters and $|Q| \geq 3$ have three states and fulfill neither of the mentioned properties. Thus, we can assume $\mathcal{A}$ has a single simple idempotent letter. Then, by Theorem 12.28, we obtain the equivalences between Items 1 to 6.

A synchronizing binary SI-automaton has only the two forms from Theorem 7.31, where only the type of automaton as claimed is strongly connected and synchronizing. This gives the equivalence between Items 1 and 7. □

In the binary automaton from Figure 7.4c, pairs of distinct 2-set are distinguishable, but it is neither strongly connected nor completely distinguishable.

12.5 Circular Automata

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is *circular* if there exists a letter $b \in \Sigma$ that cyclically permutes the states, i.e., $Q = \{\delta(q, b^i) \mid i \geq 0\}$ and $\delta(q, b^{|Q|}) = q$ for some state $q \in Q$ (and hence all states $q \in Q$). The automata from the Černý family (see Figure 1.7) are circular. Dubuc [187] confirmed the Černý conjecture for circular automata.

In Theorem 12.30, we first give a condition implying complete reachability, complete distinguishability, and bounds on the lengths of shortest words reaching non-empty subsets. This result was stated by Don [182, Prop. 15] for the special case of a single letter of defect one and without concluding complete distinguishability. We then derive several consequences and give examples.

Theorem 12.30 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton with $\Sigma = \{a_1, \ldots, a_k, b\}$ where $b \in \Sigma$ cyclically permutes the states and $a_1, \ldots, a_k \in \Sigma$ are letters of defect one with $0 < d_1, \ldots, d_k < n$ such that*

1. *for all $i \in \{1, \ldots, k\}$ we have $\mathrm{dupl}(a_i) = \delta(\mathrm{excl}(a_i), b^{d_i})$ and*
2. *the greatest common divisor of $d_1, \ldots, d_k$ and n is one.*

Then $\mathcal{A}$ is completely distinguishable, every non-empty subset of size r is reachable by a word of length $n(n-r)$, and a shortest synchronizing word has length at most $(n-1)^2$.

Proof. Consider the digraph $\Gamma_{1,n}(\mathcal{A})$ defined on page 306. First, we show that the group of units G of $\mathcal{T}_M(\mathcal{A})$ is contained in the automorphism group of $\Gamma_{1,n}(\mathcal{A})$. Since G is generated by the action of the letter b, this follows if for every edge (p, q) in $\Gamma_{1,n}(\mathcal{A})$ also the edge $(\delta(p, b), \delta(q, b))$ is in $\Gamma_{1,n}(\mathcal{A})$. Let (p, q) be an edge in $\Gamma_{1,n}(\mathcal{A})$. Then it is induced by a word of the form $a_i b^j$ for some $i \in \{1, \ldots, k\}$ and $j \in \{0, 1, \ldots, n-1\}$. For the word $u = a_i b^{(j+1) \bmod n}$ we have $\mathrm{excl}(u) = \{\delta(p, b)\}$ and $\mathrm{dupl}(u) = \{\delta(q, b)\}$. Thus, u induces the edge $(\delta(p, b), \delta(q, b))$ in $\Gamma_{1,n}(\mathcal{A})$.

Next, assume $\Gamma_{1,n}(\mathcal{A})$ is not strongly connected. An automorphism maps a path to another path. Thus, since the elements from G act as automorphisms, the weakly connected components form the congruence classes of a G-congruence on Q. Then, by Theorem 5.5 there exists a divisor d of n such that two states $p, q \in Q$ are in a common weakly connected component if and only if $\delta(p, b^j) = q$ for some multiple $0 \leq j < n$ of d. Note that since d divides n, the numbers j and $n - j$ are either both divisible by d

or neither is divisible by d. For $i \in \{1, \ldots, k\}$, let $\mathrm{excl}(a_i) = \{p_i\}$ and $\mathrm{dupl}(a_i) = \{q_i\}$. Since (p_i, q_i) is an edge in $\Gamma_{1,n}(\mathcal{A})$, the states p_i and q_i are in the same weakly connected component. Thus $\delta(p_i, b^j) = q_i$ for some multiple $0 \leq j < n$ of d. Since $\delta(p_i, b^{d_i}) = q_i$, we find $j = d_i$. This yields that d is a common divisor of $d_1, \ldots, d_k$ and n. By assumption, we find $d = 1$. This implies that all states are congruent, i.e., $\Gamma_{1,n}(\mathcal{A})$ is weakly connected. Since the group G is transitive, $\Gamma_{1,n}(\mathcal{A})$ is vertex transitive. Thus, by Theorem 5.43, it is strongly connected. Then, by Theorem 11.4, $\mathcal{A}$ is completely reachable, each non-empty subset of size r is reachable by a word of length $n(n-r)$, and a shortest synchronizing word has length at most $1 + n(n-2) = (n-1)^2$. By Theorem 12.8, we find that $\mathcal{A}$ is completely distinguishable. □

The following corollary was stated in [307, Thm. 13] in the special case $k = 1$.

Corollary 12.31 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton where $b \in \Sigma$ cyclically permutes the states. Suppose $a_1, \ldots, a_k \in \Sigma$ are letters of defect one and $0 < d_1, \ldots, d_k < n$ are such that for all $i \in \{1, \ldots, k\}$ we have $\delta(q_i, a_i) = \delta(q_i, b^{d_i} a_i)$ for some $q_i \in Q$. If the greatest common divisor of $d_1, \ldots, d_k$ and n is one, then $\mathcal{A}$ is completely reachable and completely distinguishable.*

Proof. Since $\mathcal{T}_M(\mathcal{A})$ contains the transitive permutation group $G = \langle \{\delta_u\} \rangle_G$, by Theorem 5.13, for each $i \in \{1, \ldots, k\}$ there exist simple idempotent words u_i mapping q_i to $\delta(q_i, b^{d_i})$. We have $\mathrm{excl}(u_i) = \{q_i\}$ and $\mathrm{dupl}(u_i) = \{\delta(q_i, b^{d_i})\}$. Considering a new automaton with state set Q with the letter b acting in the same way and where we consider the words $u_1, \ldots, u_k$ as letters, from Theorem 12.30 we get that this automaton is completely reachable and completely distinguishable. This also yields that $\mathcal{A}$ is completely reachable and completely distinguishable. □

We will apply Theorem 12.31 in Section 15.2 (see Theorem 15.3) to show sync-maximality for a variety of automata families (among them one for which this was previously only conjectured).

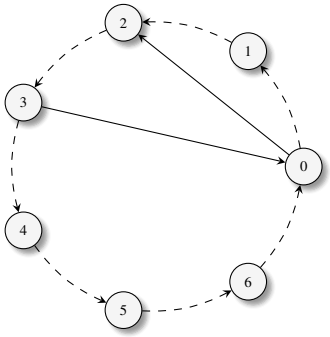
Since in the proof of Theorem 12.31 we have applied Theorem 12.30 when considering certain words as letters, the bounds from Theorem 12.30 do not hold (but they transfer when multiplied by the maximum of the lengths of these words).

Recall that, by Theorem 12.12, for binary automata with at least three states, complete reachability is equivalent to sync-maximality. However, Complete distinguishability in itself does not imply complete reachability for binary circular automata. For example, Figure 9.6a gave a binary circular automaton that is completely distinguishable, but not completely reachable.

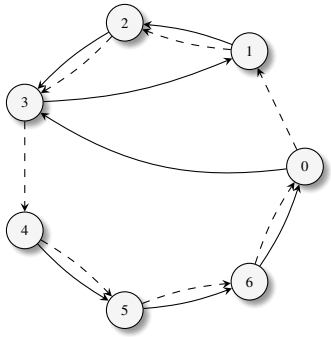
Theorem 12.31 can be used to construct sync-maximal circular automata. We give a few examples in the following statement. See also Figure 12.2 which depicts some cases from this statement.

Proposition 12.32 *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ with $Q = [n]$ and $\delta(i, b) = (i+1) \bmod n$. Then $\mathcal{A}$ is sync-maximal if a is defined in the following way:*

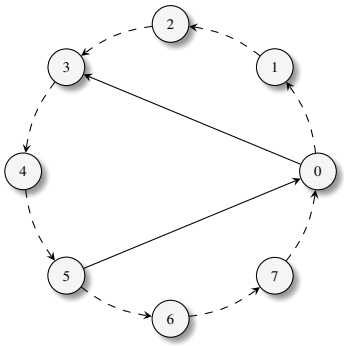
1. *$\delta(0, a) = 2$, precisely one odd $i \in \{1, 3, \ldots, n-2\}$ such that $\delta(i, a) = 0$ and $\delta(j, a) = j$ for $j \in [n] \setminus \{0, i\}$ with $n > 2$ being odd,*



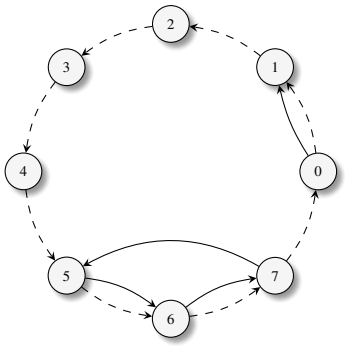
Automaton as in Theorem 12.32, Item 1



Automaton as in Theorem 12.32, Item 2



Automaton as in Theorem 12.32, Item 3



Automaton as in Theorem 12.32, Item 4

Fig. 12.2: Solid and dashed edges show the actions of a and, resp., b; self-loop omitted.

2. $\delta(0,a) = 3$, *precisely one odd* $i \in \{1,3,\ldots,n-2\}$ *such that* $\delta(i,a) = 1$ *and* $\delta(j,a) = (j+1) \bmod n$ *for* $j \in [n] \setminus \{0,i\}$ *with* $n > 2$ *being odd,*
3. $\delta(0,a) = 3$, *precisely one* $i \in \{1,3,\ldots,n-2\}$ *not divisible by* 3 *such that* $\delta(i,a) = 0$ *and* $\delta(j,a) = j$ *for* $j \in [n] \setminus \{0,i\}$ *where* $n \not\equiv 0 \pmod 3$ *with* $n > 3$,
4. $\delta(0,a) = 1$, $\delta(i,a) = i$ *for* $i \in \{1,\ldots,\lfloor n/2 \rfloor\}$, *and* a *acts injectively on* $\{\lfloor n/2 \rfloor + 1,\ldots,n-1\}$ *i.e.,* $\delta(j,a) = \delta(j',a)$ *implies* $j = j'$ *for* $j, j' \in \{\lfloor n/2 \rfloor + 1,\ldots,n-1\}$.

Proof. All these automata are circular with a having defect one. Then the claims follow from Theorem 12.31 due to the following reasons:
1: We have $\delta(0,a) = \delta(0,b^2a)$ and n is odd.
2: We have $\delta(0,a) = \delta(0,b^2a)$ and n is odd.
3: We have $\delta(0,a) = \delta(0,b^3a)$ and n is not a multiple of 3.
4: We have $\delta(0,a) = \delta(0,ba)$. □

As the next example shows, Theorem 12.30 applies to a wider class of automata than Theorem 12.31.

Example 12.33 Consider the automaton from Theorem 11.30 (see Figure 6.6). Let w be a word of defect one. As can be read off from Table 11.2, for $d > 0$ and $q \in [4]$,

if $\delta(q, w) = \delta(\delta(q, b^d), w)$, then d is a multiple of two. So Theorem 12.31 does not apply here. We have $\mathrm{excl}(a) = \{0\}$, $\mathrm{dupl}(a) = \{1\}$, and $\delta(0, a) = 1 = \delta(0, b)$. Thus Theorem 12.30 yields that the automaton is completely reachable (which also can be seen in Figure 6.6).

Next, we give examples of completely reachable binary automata for which complete reachability does not follow directly from Theorem 12.30. In Theorem 12.34, by considering certain words as letters, it becomes possible to apply Theorem 12.30. However, it is unclear whether such an argument can also be applied in Theorem 12.35.

Example 12.34 Let $\mathcal{A} = (Q, \Sigma, \delta)$ where $Q = [n]$ for some $n \geq 3$ and $\delta(q, b) = (q + 1) \bmod n$ for $q \in Q$, $\delta(0, a) = 2$, and $\delta(q, a) = q + 1$ for $q \in \{1, 2, \ldots, n - 2\}$ and $\delta(n - 1, a) = 1$. For even n, Theorem 12.30 is not applicable. However, we have $\delta(0, a^n) = 1$, a^n has defect one, and 0 and 1 are the excluded and duplicate state of a^n, respectively. Thus, regarding a^n as an alphabet letter, from Theorem 12.30 we get that $\mathcal{A}$ is completely reachable.

Example 12.35 Let $\mathcal{A} = (Q, \Sigma, \delta)$ where $Q = [9]$, $\delta(q, b) = (q + 1) \bmod 9$ for $q \in Q$ and $\delta(0, a) = 3$, $\delta(a, 6) = 7$, $\delta(7, a) = 6$, and $\delta(q, a) = q$ for $q \in Q \setminus \{0, 6, 7\}$, see Figure 12.3. Next, we show that $\mathcal{A}$ is completely reachable. Since $\mathrm{excl}(a) = \delta(\mathrm{dupl}(a), b^3)$, this does not follow from Theorem 12.30, and it is also not clear to us how to find words acting in a way to apply Theorem 12.30. Next, let $S \subseteq Q$ be a non-empty subset of Q. We show, by induction on $|Q \setminus S|$, that S is reachable in $\mathcal{A}$. The claim holds for $S = Q$. Next, let $S \neq Q$.

1. Suppose we have $\delta(p, b^3) \in S$ for some $p \in Q \setminus S$.
   In this case, there exists $i \geq 0$ such that $0 \notin \delta^{-1}(S, b^i)$ and $3 \in \delta^{-1}(S, b^i)$. This gives $|\delta^{-1}(S, ab^i)| = |S| + 1$ and $\delta(\delta^{-1}(S, ab^i), ab^i) = S$. Inductively, we have $\delta(Q, u) = \delta^{-1}(S, ab^i)$ for some $u \in \Sigma^*$. Then $\delta(Q, uab^i) = S$.
2. Otherwise, each residue class modulo three is either contained in S or in its complement.
   Let $q \in Q \setminus S$ be such that $\delta(q, b) \in S$. Then we have $6 \notin \delta^{-1}(S, b^i)$ and $7 \in \delta^{-1}(S, b^i)$ for some $i \geq 0$. Since $6 \notin \delta^{-1}(S, b^i)$ we also find that 0 and 3 are not contained in $\delta^{-1}(S, b^i)$. For $T = \delta^{-1}(S, ab^i)$, this gives $T = (\delta^{-1}(S, b^i) \setminus \{7\}) \cup \{6\}$ and $\delta(T, ab^i) = S$. We have $0 \notin \delta^{-1}(T, b^3)$ since $3 \notin T$ and $3 \in \delta^{-1}(T, b^3)$ since $6 \in T$. We also have $7 \in \delta^{-1}(T, b^3)$ since $1 \in T$ and $6 \notin \delta^{-1}(T, b^3)$ since $0 \notin T$. Thus we find
   $$\delta^{-1}(T, ab^3) = (\delta^{-1}(T, b^3) \setminus \{7\}) \cup \{0, 6\}$$
   and $\delta(\delta^{-1}(T, ab^3), ab^3) = T$. Since $|\delta^{-1}(T, ab^3)| = |T| + 1 = |S| + 1$, inductively, we have $\delta^{-1}(T, ab^3) = \delta(Q, u)$ for some $u \in \Sigma^*$. Then we find $\delta(Q, uab^3ab^i) = S$.

12.6 Binary Automata

In Theorem 12.12 we have shown that for binary automaton with at least three states, sync-maximality and complete reachability are equivalent properties. Here, we show

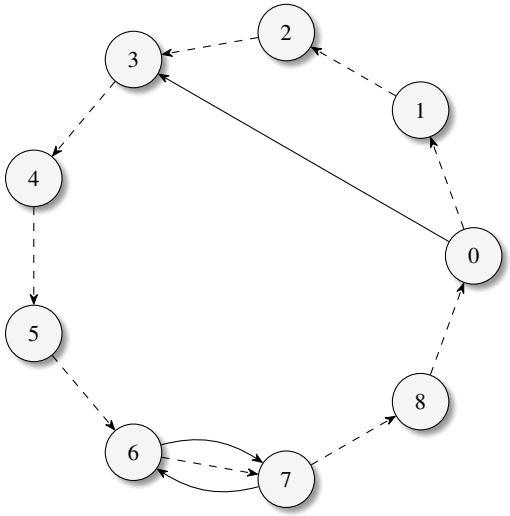


Fig. 12.3: Automaton from Theorem 12.35. Solid and dashed edges show the actions of a and, resp., b; self-loop omitted.

that a binary completely reachable automaton does not admit any non-trivial congruences. We also show that a binary circular automaton with a letter of defect one does not admit a non-trivial congruence if and only if it is synchronizing.

Casas and Volkov [134, Thm. 2] gave the following characterization of binary completely reachable automata as certain circular automata (the authors formulated it differently, assuming a standardized form and identifying the states with residue classes, that we briefly explain in Theorem 12.39 below).

Theorem 12.36 (Casas and Volkov [134, Thm. 2]) *An automaton $\mathcal{A} = (Q, \{a, b\}, \delta)$ is completely reachable if and only if it equals the automaton from Figure 6.3 or*

1. *one letter, say a, has defect one with $\operatorname{excl}(a) = \{q_e\}$,*
2. *the other letter b cyclically permutes the states, and*
3. *there does not exist $s \in \delta^{-1}(\operatorname{dupl}(a), a)$ such that $\delta([s]_\sim, a) \subseteq [q_e]_\sim$ for some non-trivial $\langle \delta_b \rangle_G$-congruence $\sim$ on Q.*

Remark 12.37 Theorem 12.36 easily implies that the automaton from Theorem 12.35 is completely reachable since, by Theorem 5.5, its only non-trivial $\langle \delta_b \rangle_G$-congruence $\sim$ is the congruence modulo three. The excluded state of a is 0 and we have $[0]_\sim = \{0, 3, 6\}$ and $\delta^{-1}(\operatorname{dupl}(a), a) = \{0, 3\}$. Now, $\delta([0]_\sim, a) = \{0, 7\}$, which is not contained in $[0]_\sim$. Thus, the automaton admits no non-trivial congruences as in Theorem 12.36 and so it is completely reachable.

Remark 12.38 We shortly sketch how complete reachability follows from the assumptions in Theorem 12.30 for binary automata. By Theorem 5.5, there exists a divisor r of n such that $q_1 \sim q_2$ if and only if $\delta(q_1, b^i) = q_2$ for some multiple $i > 0$ of r. Thus, if $\operatorname{dupl}(a) = \delta(\operatorname{excl}(a), b^d)$ for some d coprime to n, assuming the duplicate and the

excluded state are congruent implies $r = 1$. Hence, the duplicate state and the excluded state of a are never congruent for a non-trivial congruence. In Theorem 12.36 we have $\delta(s, a) \sim q_e$ and so the duplicate state and the excluded state are congruent. Thus, Theorem 12.36 yields that $\mathcal{A}$ is completely reachable.

Originally, Casas and Volkov formulated Theorem 12.36 differently, as we explain next.

Remark 12.39 Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be an automaton with n states where a has defect one, b cyclically permutes the states, $s, t \in Q$ are distinct with $\delta(s, a) = \delta(t, a)$, and $\mathrm{excl}(a) = \{q_e\}$. Let $0 \leq i, j < n$ be such that $\delta(q_e, b^i) = s$ and $\delta(s, b^j) = q_e$. The words $u = b^i a$ and $v = a b^j$ each permute their images and have defect one. Treating them as letters in two new automata over the alphabets $\{u, b\}$ and $\{v, b\}$, we obtain automata whose transformation monoids equal $\mathcal{T}_S(\mathcal{A})$; so each of them is completely reachable precisely if $\mathcal{A}$ is completely reachable. Thus, we can assume a permutes its image. Casas and Volkov [134] called $\mathcal{A}$ *standardized* if it has this form. (Actually, they also numbered the states from 0 to $n - 1$, starting with $q_e = 0$, in such a way that applying b adds one modulo n to each state. In this setting, congruences correspond to subgroups and their cosets of the cyclic group generated by the permutation induced by b.) They showed that $\mathcal{A}$ in standardized form is not completely reachable if and only if a preserves the congruence class of q_e for some non-trivial congruence. Translating this condition to the general (not necessarily standardized) case, we obtain Theorem 12.36 since $\delta([s]_\sim, a) \subseteq [q_e]_\sim$ is equivalent to each of the conditions that u preserves $[q_e]_\sim$ and that v preserves $[s]_\sim$.

Since $G = \langle \delta_b \rangle_G$ in Theorem 12.36 is transitive, the congruence classes for $\sim$ all have the same size. The next statement investigates the condition $\delta([s]_\sim, a) \subseteq [q_e]_\sim$ for general equivalence relation (not necessarily congruences) all of whose classes have the same size. One of its consequences is that, for the specified condition, it does not matter which state from $\delta^{-1}(\mathrm{dupl}(a), a)$ we pick. In fact, it implies that it can be reformulated to exclude existence of any $q \in Q$ such that $\delta([q]_\sim, a) \subseteq [q_e]_\sim$ since then both states from $\delta^{-1}(\mathrm{dupl}(a), a)$ are mapped into $[q_e]_\sim$ by a.

Proposition 12.40 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with a non-trivial equivalence relation $\sim$ all of whose equivalence classes have the same size and a letter $a \in \Sigma$ of defect one and $s, t, q_e, q \in Q$ such that $\{s, t\} = \delta^{-1}(\mathrm{dupl}(a), a)$ and $\mathrm{excl}(a) = \{q_e\}$. Suppose $\delta([q]_\sim, a) \subseteq [q_e]_\sim$. Then we have the following:*

1. *$\delta([q]_\sim, a) = [q_e]_\sim \setminus \{q_e\}$.*
2. *$s \sim q$ and $s \sim t$ (and so $\mathrm{dupl}(a) \subseteq [q_e]_\sim$).*
3. *For all $p \in Q$ with $p \not\sim s$ we have $\delta(p, a) \not\sim q_e$.*

Proof. 1: Since q_e is the excluded state of a, we have $\delta([q]_\sim, a) \subseteq [q_e]_\sim \setminus \{q_e\}$. All equivalence classes of $\sim$ have the same size. Assume that $\delta([q]_\sim, a)$ is a proper subset of $[q_e]_\sim \setminus \{q_e\}$. Then we either have three distinct states from $[q]_\sim$ mapped by a to the same state, or there are two disjoint pairs of states from $[q]_\sim$ where each pair is mapped to a common state. Both of these cases imply that a has defect at least two, contradicting the assumption that a has defect one. Thus, we must have $\delta([q]_\sim, a) = [q_e]_\sim \setminus \{q_e\}$.

2: Since $\delta([q]_\sim, a) = [q_e]_\sim \setminus \{q_e\}$ we have $|\delta([q]_\sim, a)| = |[q]_\sim| - 1$. Thus two states from $[q]_\sim$ are mapped to one state by a. Since a has defect one, these two states from $[q]_\sim$ must be s and t and so $q \sim s$ and $s \sim t$.

3: Let $p \in Q$ with $p \nsim s$. Assume $\delta(p, a) = p'$ for some $p' \in Q$ with $p' \sim q_e$. Since p' is in the image of a, we have $p' \neq q_e$ and thus $\delta(r, a) = p'$ for some $r \in Q$ with $r \sim q$ by Item 1. Thus we also have $r \sim s$ by Item 2 and so $r \neq p$. However, since a has defect one, only s and t can be mapped to the same state. So, this is not possible. □

Next, we list some properties for equivalence class that are congruences.

Proposition 12.41 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with a letter $a \in \Sigma$ of defect one and $s, t \in Q$ such that $\delta^{-1}(\mathrm{dupl}(a), a) = \{s, t\}$. Suppose $\sim$ is a non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence all of whose congruence classes have the same size. Then the following holds:*

1. *We have $\delta([p]_\sim, a) = [\delta(p, a)]_\sim$ for each $p \in Q$ with $p \nsim s$.*
2. *If $\delta(p, a) \sim \delta(q, a)$ for $p, q \in Q$, then $p \sim q$.*
3. *$\delta([s]_\sim, a) \subseteq [q_e]_\sim$ for the state $q_e \in Q$ with $\mathrm{excl}(a) = \{q_e\}$.*

Proof. 1: Since $\sim$ is a congruence, we have $\delta([p]_\sim, a) \subseteq [\delta(p, a)]_\sim$. The letter a acts injectively on $Q \setminus \{s\}$. We have $[p]_\sim \subseteq Q \setminus \{s\}$. Thus, since $[p]_\sim$ and $[\delta(p, a)]_\sim$ have the same size, we conclude that $\delta([p]_\sim, a) = [\delta(p, a)]_\sim$.

2: Let $\delta(p, a) \sim \delta(q, a)$ for $p, q \in Q$. Assume $p \nsim q$. Without loss of generality, let $p \nsim s$. By Item 1 we have $\delta([p]_\sim, a) = [\delta(p, a)]_\sim$. Consider a state $q_1 \neq q$ with $q_1 \sim q$. We have $\delta(q_1, a) \sim \delta(q, a)$. Since $\delta([p]_\sim, a) = [\delta(p, a)]_\sim$, there exist $p_1, p_2 \in [p]_\sim$ such that $\delta(p_1, a) = \delta(q_1, a)$ and $\delta(p_1, a) = \delta(q, a)$. A letter of defect one maps precisely two states to the same state. Thus, since p_1, q_1, q are pairwise distinct, we find that a has not defect one, contradicting the choice of a.[7] Thus, we must have $p \sim q$.

3: By Item 2, the letter a acts injectively on the congruence classes. Thus, it permutes these classes and for every class there exists some other class mapped to it by a. By Item 1, we find $\delta(p, a) \nsim q_e$ for $p \nsim s$ since otherwise not every state in the target congruence class has an inverse image in $[p]_\sim$. So, the only class that can be mapped to $[q_e]_\sim$ is $[s]_\sim$.

□

Corollary 12.42 *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be a binary circular automaton with a letter of defect one. If $\mathcal{A}$ is synchronizing, then there exists no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence.*

Proof. Let a be the letter of defect one and b be the letter cyclically permuting the states. Assume there exists a non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence $\sim$. Then it is also a G-congruence for the group generated by the permutation induced by b. Since b cyclically permutes the states, and so induces a full cycle on the states, G is transitive. Thus, all congruence classes have the same size. Let $p, q \in Q$ be such that $p \nsim q$. Since b permutes the states, we have $\delta(p, b) \nsim \delta(q, b)$. By Theorem 12.41, we also have $\delta(p, a) \nsim \delta(q, a)$. This yields that $\delta(p, u) \nsim \delta(q, u)$ for all $u \in \{a, b\}^*$ and so $\mathcal{A}$ is not synchronizing. □

[7] Alternatively, $\delta([p]_\sim, a) = [\delta(p, a)]_\sim$ and $\delta([q]_\sim, a) = [\delta(q, a)]_\sim$ imply $\delta([q]_\sim, a) \subseteq \mathrm{dupl}(a)$. This yields $\mathrm{dupl}(a) \subseteq [\delta(p, a)]_\sim$ and since all states from $[q]_\sim$ are mapped to $\mathrm{dupl}(a)$ and at least one state from $[p]_\sim$, the letter has not defect one.

In [610, Cor. 4] it was shown that an automaton as in Theorem 12.42 that does not admit any non-trivial congruences is also synchronizing.

Corollary 12.43 *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be a binary completely reachable automaton. Then there exists no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence.*

Proof. We can assume $\mathcal{A}$ has at least three states. Suppose $\mathcal{A}$ is completely reachable. By Theorem 9.20 a binary completely reachable automaton with more than three states is circular with a letter of defect one. Since a completely reachable automaton is synchronizing, by Theorem 12.42, $\mathcal{A}$ admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruences.[8] □

Next, in Theorem 12.44, we give an example of a binary circular automaton with a letter of defect one that does not admit any non-trivial congruence and which is not completely reachable.

Example 12.44 Consider the binary automaton $\mathcal{A} = (Q, \{a, b\}, \delta)$ in Figure 12.4. By Theorem 5.5 the only non-trivial G-congruence for the group G generated by the full cycle induced by b is the congruence modulo three.

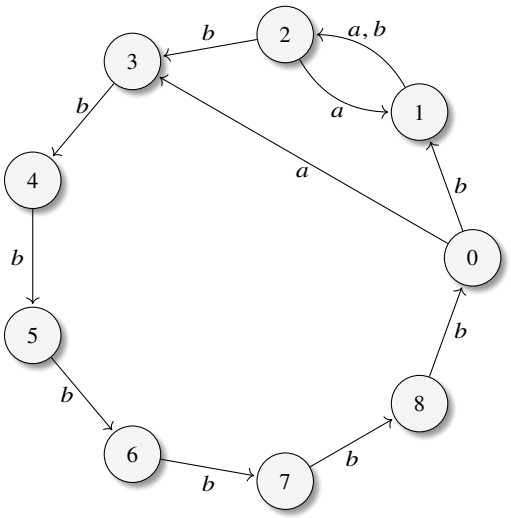


Fig. 12.4: An automaton $\mathcal{A}$ (self-loops omitted) that is not completely reachable and that admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence, see Theorem 12.44.

1. $\mathcal{A}$ admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence.
   We have $1 \equiv 4 \pmod 3$, $\delta(1, a) = 2$, and $\delta(4, a) = 4$. Since $2 \not\equiv 4 \pmod 3$, congruence classes modulo three do not form a $\mathcal{T}_S(\mathcal{A})$-congruence. Thus there exists no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence.

[8] Since for a non-trivial congruence $\sim$ we would have $\delta([s]_\sim, a) \subseteq [q_e]_\sim$ for the state $q_e \in Q$ with $\mathrm{excl}(a) = \{q_e\}$ and each $s \in \delta^{-1}(\mathrm{dupl}(a), a)$ by Theorem 12.41, this would also yield that $\mathcal{A}$ does not admit any non-trivial congruences if it is completely reachable.

2. $\mathcal{A}$ is not completely reachable.
   The excluded state of a is 0, 0 and 3 are mapped to the same state, and the letter a preserves the residue class modulo three of zero. Thus, by Theorem 12.36, $\mathcal{A}$ is completely reachable.

By the above results, it follows that $\mathcal{A}$ is synchronizing. However, this can also be shown directly in the following way. Set $n = 9$. Let $p, q \in Q$ be two distinct states with $\delta(p, b^i) = q$ for some $0 < i \leq 4$. Then we have the following cases:

1. $i = 2$: Let $u = b^{n-q+1}$. We have $\delta(p, u) = 8$ and $\delta(q, u) = 1$. Then $\delta(p, ua) = 8$ and $\delta(q, ua) = 2$. We have $\delta(8, ba) = \delta(2, ba) = 3$. Thus $uaba$ maps both states to 3.
2. $i = 1$: Let $u = b^{n-p+2}$. We have $\delta(p, u) = 2$ and $\delta(q, u) = 3$. Thus $\delta(p, ua) = 1$ and $\delta(q, ua) = 3$. We have $\delta(1, b^2) = 3$ and we have already shown how to map two such states to the same state.
3. $i = 3$: Let $u = b^{n-p}$. Then $\delta(p, u) = 0$ and $\delta(q, u) = 3$. Thus $\delta(p, ua) = \delta(q, ua) = 3$.
4. $i = 4$: Let $u = b^{n-p}$. We have $\delta(p, u) = 0$ and $\delta(q, u) = 4$. Then $\delta(p, ua) = 3$ and $\delta(q, ua) = 4$. We have $\delta(3, b) = 4$ and we have already shown how to map two such states to the same state.

For a ternary alphabet, complete reachability does not exclude the possibility of non-trivial $\mathcal{T}_S(\mathcal{A})$-congruences, see Theorem 12.45.

Example 12.45 Let $\mathcal{A} = (\{0, 1, 2, 3\}, \{a, b, c\}, \delta)$ where a is a simple idempotent mapping 0 to 2, b induces the cycle (0 1 2 3) on the states, $\delta(0, c) = \delta(2, c) = 0$, and $\delta(1, c) = \delta(3, c) = 2$.

1. The partition $\{0, 2\}, \{1, 3\}$ of the states induces a $\mathcal{T}_S(\mathcal{A})$-congruence.
2. $\mathcal{A}$ is completely reachable.
3. $\{0, 1\}$ and $\{2, 3\}$ are indistinguishable.

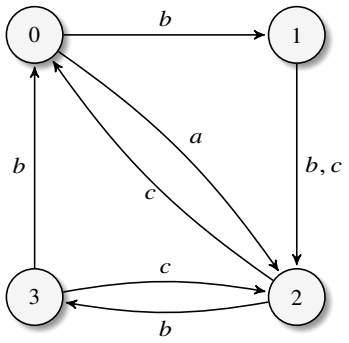


Fig. 12.5: Automaton (self-loops omitted) that is completely reachable and admits a non-trivial congruence, see Theorem 12.45.

12.7 Discussion

Shortest Words Reaching Certain Subsets.

Considering Theorem 12.30, Don [182, Conj. 18] conjectured that if a subset of size r is reachable, then it is reachable by a word of length at most $n(n-r)$. Gonze and Jungers [255, Prop. 7] refuted this conjecture by giving a series of n-state automata for $n > 6$ having a subset of size $\lfloor \frac{n}{2} \rfloor$ where the shortest word reaching this subset has length greater than $\frac{2^n}{n}$. They also noted [255, Sect. 2.3] that truth of this conjecture would have implied the collapse of the polynomial time hierarchy, which, while still an open problem, is considered to be highly unlikely. The authors asked whether the conjecture on the length of shortest words reaching certain subsets might still hold for completely reachable automata. This was refuted by Zhu [648] even for binary automata. He also proved the bound $n(n-r) + (n-1)$ for binary automata where one letter permutes its image, and for a subclass of these automata the conjecture was confirmed, which expanded a result of Casas & Volkov [135] who investigated this conjecture for binary automata. For circular binary automata having a letter of defect one that permutes its image the conjecture still remains open [135]. In general, Ferens & Szykuła [206] proved the bound $2n(n-r) - n\sum_{i=1}^{n-r} \frac{1}{i}$ for completely reachable automata.

An Erroneous Statement on SI-Automata.

In [306, Thm. 12] it was wrongly claimed that a completely reachable SI-automaton is sync-maximal. Figure 9.7 gives a counterexample. This wrong conclusion was based on the false statement that distinguishability of all pairs of distinct 2-sets implies complete distinguishability (see the related discussion in Section 9.5). Here, for automata whose permutational letters generate a transitive group, we proved in Theorem 12.10 that distinguishability of pairs of distinct 2-sets, complete distinguishability, complete reachability, and sync-maximality are equivalent properties. Theorem 12.20 was also given in [306, Lem. 11], but with a different (less general) proof.

Chapter 13
Decision Problems

In Sections 13.1 and 13.2 we investigate the complexity of deciding complete distinguishability, complete reachability, and sync-maximality. Also, we prove that the problem to decide whether two subsets are distinguishable is PSPACE-complete. In Section 13.3, we investigate these problems, and the problem to decide synchronizablity, for binary SI-automata as input, which appears in a separate section as complete reachability, complete distinguishability, and sync-maximality are equivalent properties for binary SI-automata by Theorem 12.29. An overview of our results and those from the literature on complete reachability, complete distinguishability, and sync-maximality, is given in Table 13.1.

Table 13.1: Overview of results on deciding complete reachability, complete distinguishability, and sync-maximality. NL-hardness holds even for SI-automata without permutational letters.

Problem	Subclass of Automata			
	SI-Automata	Binary Automata	Binary SI-Automata	All Automata
Complete Reach.	NL-c (Th. 13.7)	P [134]	L-c (Th. 13.10)	P [205]
Complete Dist.	NL-hard (Th. 13.3)	Open	L-c (Th. 13.10)	PSPACE-c [595]
Sync-Maximality	NL-hard (Th. 13.6)	P (by Cor. 12.12)	L-c (Th. 13.10)	PSPACE-c [595]

With the correspondence between automata and transformation semigroups (Section 6.8), the same complexities for the respective problems for transformation semigroups given as a list of generators follow.

13.1 Distinguishability

First, Theorem 13.1 shows that it is PSPACE-complete to decide whether two given subsets are distinguishable in a given automaton. Then, in Theorem 13.2, we show that deciding complete distinguishability is in PSPACE, and in Theorem 13.3 that it is NL-hard even for SI-automata without permutational letters.

Theorem 13.1 *The problem to decide, for a given automaton and two subsets of states as input, whether both subsets of states are distinguishable in the automaton is* PSPACE*-complete.*

Proof. Let $(\mathcal{A}, S)$ be an instance of SyncFrom. We can assume $\emptyset \neq S \neq Q$. Then there exists a word mapping S to a singleton if and only if S and $\emptyset$ are distinguishable in $\mathcal{A}$, which gives an $\leq_m^{\mathsf{AC}^0}$-reduction. Hence, the problem is PSPACE-hard.

Next, we show that the problem is in PSPACE. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state input automaton and $A = \{q_1, \ldots, q_s\}$, $B = \{p_1, \ldots, p_r\}$ for $s, r \geq 0$ two subsets of states. Store A and B into $s+r$ memory cells. Then, using a counter, iterate through the numbers $i \in \{1, \ldots, 2^{n+1}\}$ and, in the i-step, first check whether precisely one of both subsets stored in the memory cells is a singleton. If this is the case, then return YES (in this case, the word $x_1 \cdots x_{i-1}$ [1] distinguishes A from B where the $x_1, \ldots, x_{i-1}$ are letters guessed in the previous steps). Otherwise, guess a letter $x_i \in \Sigma$, update the memory cells storing A and B with $\{\delta(q_1, x_i), \ldots, \delta(q_s, x_i)\}$ and $\{\delta(p_1, x_i), \ldots, \delta(p_r, x_i)\}$, respectively, and continue with the iteration. If the iteration ends after going through all values for i without discovering a distinguishing word, then return NO. By Theorem 9.42 a shortest synchronizing word has length at most 2^{n+1}. So, by nondeterminism, the above procedure returns YES if and only if A and B are distinguishable. Using a binary encoding for the counter for i, the procedure runs in polynomial space. As by Savitch's theorem [550] we have NPSPACE = PSPACE, the claim follows. □

Theorem 13.2 *Deciding whether a given automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is completely distinguishable is in* PSPACE.

Proof. The complement of this problem is in NPSPACE as witnessed by the following algorithm: nondeterministically guess two distinct subsets of states $A, B \subseteq Q$ with $|A|, |B| \geq 2$ and check whether they are indistinguishable, where the latter step can be done in polynomial space by Theorem 13.1 and as PSPACE is closed under complement. As PSPACE = NPSPACE by Savitch's theorem [550] and by the complement-closure of PSPACE, we obtain the claim. □

Theorem 13.3 *Deciding whether a given automaton is completely distinguishable is* NL*-hard, even for SI-automata without permutational letters as input.*

Proof. We give a reduction from 2-DAG-ST, which is NL-complete [345, Cor. 25], see also Theorem 3.78. Let (G, s, t) with $G = (V, E)$ and $s, t \in V$ be an input instance. We can assume $s \neq t$ and that s has no incoming transitions and t no outgoing transitions.

[1] Setting $x_1 \cdots x_{-1} = \varepsilon$.

Construction: Let $\hat{s} \notin V$ be a new element and $\hat{E} = E \cup \{(\hat{s}, q) \mid (s, q) \in E\}$. Then set $\mathcal{A} = (Q, \Sigma, \delta)$ where $Q = V \cup \{\hat{s}\}$ and $\Sigma = \Sigma_1 \cup \Sigma_2$ with the subalphabets $\Sigma_1 = \{a_e \mid e \in \hat{E}\}$ and $\Sigma_2 = \{c_v, d_v \mid v \in V \setminus \{s, t\}\} \cup \{c_{\hat{s}}, c_s, d_s, d_{\hat{s}}\}$ where c_s and $d_{\hat{s}}$ induce self-loops at each state and the remaining letters are simple idempotents given by $\delta(v_1, a_e) = v_2$ for $e = (v_1, v_2) \in \hat{E}$, $\delta(v, c_v) = s$ for $v \in V \setminus \{t, s\}$ and $\delta(v, d_v) = \hat{s}$ for $v \in V \setminus \{t, \hat{s}\}$. See Figure 13.1b for an example construction.

Properties:

1. Let $p, q \in Q$ and $u \in \Sigma_1^*$. As the letters in Σ_1 directly encode edges in G we have $\delta(p, u) = q$ if and only if there exists a path from p to q in G for $p \neq \hat{s}$ and a path from s to q in G for $p = \hat{s}$.
2. As every letter induces a self-loop at t, for all $p \in Q$ and $u \in \Sigma^*$, we have $\delta(t, u) = t$ and so
$$\delta(p, u) = \delta(t, u) \Leftrightarrow \delta(p, u) = t. \tag{13.1}$$

Verification: First, suppose there exists no path from s to t in G. Then, by Property 1 and Equation (13.1), we have $|\delta(\{s, t\}, u)| = |\delta(\{\hat{s}, t\}, u)| = 2$ for all $u \in \Sigma^*$. So $\{s, t\}$ and $\{\hat{s}, t\}$ are not distinguishable.

Next, suppose there exists a path $p_0, p_1, \ldots, p_n$ ($n \geq 1$) from s to t in G with edges $e_i = (p_{i-1}, p_i)$ for $i \in \{1, \ldots, n\}$. Set $\hat{e}_1 = (\hat{s}, p_1) \in \hat{E}$. Let $A \subseteq Q$ with $|A| \geq 2$ and $p \in Q \setminus A$ where $A \setminus \{t\} = \{q_1, \ldots, q_m\}$ for some $m \geq 1$. Set

$$w = \begin{cases} c_{q_1} \cdots c_{q_m} & \text{if } p = t, \\ d_{q_1} \cdots d_{q_m} a_{\hat{e}_1} a_{e_2} \cdots a_{e_n} & \text{if } p = s, \\ c_{q_1} \cdots c_{q_m} d_p a_{e_1} a_{e_2} \cdots a_{e_n} & \text{if } p \notin \{s, t\}. \end{cases}$$

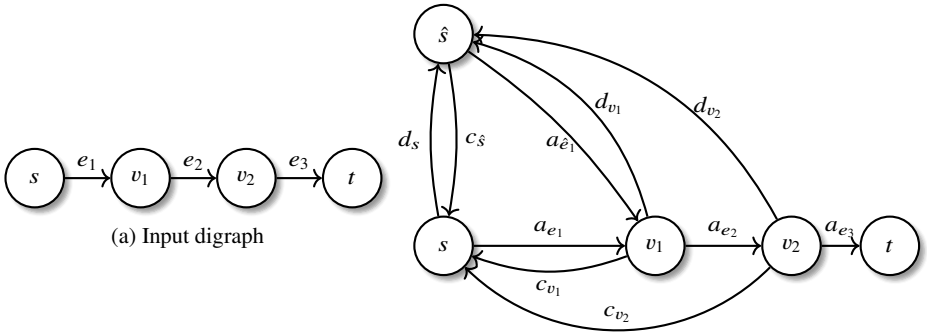
Then $\delta(p, w) \in \{p, \hat{s}\}$ and

$$\delta(A, w) = \begin{cases} \{s\} & \text{if } p = t, \\ \{t\} & \text{if } p \neq t. \end{cases}$$

By Theorem 9.46, Item 3, it follows that $\mathcal{A}$ is completely distinguishable. □

13.2 Complete Reachability and Sync-Maximality

In Theorem 13.5 we show that deciding sync-maximality is in PSPACE. Then, in Theorem 13.6, we show that each of the problems to decide complete reachability and to decide sync-maximality are NL-hard, even for those without permutational letters (the proof uses one construction to establish NL-hardness for both problems). Then in Theorem 13.7, we derive that deciding complete reachability is NL-complete for SI-automata.



(a) Input digraph

(b) Construction from the proof of Theorem 13.3.

Fig. 13.1: Example for the reductions used in the proof of Theorem 13.3 with $\hat{e}_1 = (\hat{s}, v_1)$. The automaton in Figure 13.1b is completely distinguishable as there exists a path from s to t in the input graph. Self-loops are omitted.

In [99] the question was posed whether complete reachability can be decided in polynomial time. This problem has been later solved by Ferens & Szykuła [205]. Still, the precise computational complexity is unknown.

Theorem 13.4 (Ferens & Szykuła [205]) *Complete reachabilty can be decided in polynomial time.*

Theorem 13.5 *Given $\mathcal{A} = (Q, \Sigma, \delta)$ it can be decided in* PSPACE *whether $\mathcal{A}$ is sync-maximal.*

Proof. We can assume $|Q| \geq 3$. First, guessing some $A \subseteq Q$ with $|A| \geq 2$ and testing whether it is reachable, which can be done in PSPACE, see page 231. So, we can decide in NPSPACE whether some subset of size at least two is not reachable. Since NPSPACE = PSPACE by Savitch's theorem [550] the complement of this problem, i.e., whether all subsets of size at least two are reachable, is in PSPACE. By Theorem 13.2, testing whether $\mathcal{A}$ is completely distinguishable is in PSPACE. Since both properties characterize sync-maximality for automata with at least three states by Theorem 10.2, Item 3, we obtain the claim.

Alternatively, we can also test whether the automaton has the structure as in Theorem 10.7 in PSPACE by Theorems 13.2 and 13.4 since S (and q, Γ, and $a \in \Sigma$ in Theorem 10.7, Item 2) can be computed in polynomial time as is easily seen using Theorem 7.28. □

Theorem 13.6 *The two decision problems to decide whether a given automaton is completely reachable and whether a given automaton is sync-maximal are each* NL*-hard even for SI-automata without permutational letters as input.*

Proof. We give a reduction from the problem to decide whether a given digraph is strongly connected, which is NL-complete [345, Cor. 24], see also Theorem 3.76.

So, let $G = (V, E)$ be a digraph. We can assume $|V| \geq 2$. Set $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q = V \times \{0, 1\}$ and an alphabet of simple idempotent letters

$$\Sigma = \{a_{(p,i),(q,i)} \mid (p, q) \in E, i \in \{0, 1\}\} \cup \{a_{(p,0),(p,1)}, a_{(p,1),(p,0)} \mid p \in V\}$$

where $a_{(p,i),(q,j)}$ maps (p, i) to (q, j) for distinct $(p, i), (q, j) \in Q$, $p, q \in V$, $i, j \in \{0, 1\}$. Intuitively, we have two copies of G inside $\mathcal{A}$, where an edge (p, q) in G corresponds to the letters $a_{(p,i),(q,i)}$, $i \in \{0, 1\}$, and for every two states from (p, i) and (p, j) with $i \neq j$ there exist two simple idempotent letters mapping them onto each other. Next, we prove that the following properties are equivalent, which shows that the construction gives a valid reduction for both decision problems:

1. G is strongly connected,
2. $\mathcal{A}$ is strongly connected,
3. $\mathcal{A}$ is completely reachable,
4. $\mathcal{A}$ is sync-maximal.

$1 \Rightarrow 2$: This is implied by construction.

$2 \Rightarrow 3$: As $\mathcal{A}$ is an SI-automaton without permutational letters, this follows from Theorem 12.24.

$3 \Rightarrow 4$: Let $A \subseteq Q$ with distinct $(p_1, i), (p_2, j) \in A$ for $i, j \in \{0, 1\}$ and, without loss of generality, $(q, 0) \in Q \setminus A$. There exist $u_1, u_2 \in \Sigma \cup \{\varepsilon\}$ fixing $(q, 0)$ such that $\delta((p_1, i), u_1) = (p_1, 1)$ and $\delta((p_2, j), u_2) = (p_2, 1)$. As $\mathcal{A}$ is strongly connected by Theorem 9.12, we can pick a shortest word $u \in \Sigma^*$ mapping $(p_1, 1)$ to $(p_2, 1)$. By construction, this word traces a path using only states in $V \times \{1\}$, which gives $\delta((q, 0), u) = (q, 0)$, $(q, 0) \notin \delta(A, u)$, and $\delta((p_1, 1), u) = \delta((p_2, 1), u) = (p_2, 1)$. Then for $v = u_1 u_2 u$ we have $|\delta(A, v)| < |A|$ and $\delta((q, 0), v) \notin \delta(A, v)$. So, by Theorem 9.46, Item 8, $\mathcal{A}$ is completely distinguishable. Hence, by Theorem 10.7, Item 1 the automaton is sync-maximal.

$4 \Rightarrow 1$: Assume G is not strongly connected. Then there exist distinct $p, q \in V$ such that we have no path from p to q. By construction, there exists no path from $(p, 0)$ to $(q, 0)$ and no path from $(p, 1)$ to $(q, 1)$. By Theorem 10.7 a sync-maximal automaton has at most one state that is not reachable from some other state. So, $\mathcal{A}$ is not sync-maximal. □

Theorem 13.7 *Deciding complete reachability is* NL*-complete for SI-automata.*

Proof. By Theorem 12.22, for SI-automata, complete reachability is equivalent to synchronizability and strong connectedness, and each property can both be decided in NL by Theorems 3.76 and 8.13. By Theorem 13.3, deciding complete reachability is NL-hard even for SI-automata as input. □

13.3 Binary SI-Automata

Here, in Theorems 13.9 and 13.10, we prove L-completeness of deciding each of the properties of synchronizability, complete reachability, complete distinguishability, or

sync-maximality for binary SI-automata as input. Theorem 13.9 is an easy consequence of the following lemma (we note that it was essentially proved by Martyugin [436, Prop. 6.2] that the problem from Theorem 13.8 can be solved in $O(n)$).

Lemma 13.8 *For $n \geq 4$, which case from Theorem 7.31 applies and the computation of the respective parameters $s, t \in Q$, $a, b \in \Sigma$, i.e., which letter is the simple idempotent a and which cyclically permutes the states as b, and $0 < d < n$ can be done in deterministic log-space.*

Proof. We denote by $\mathcal{A}_t$ the synchronizing binary SI-automaton from Figure 7.4c with a trap state and by $\mathcal{A}_c$ the other automaton from Figure 7.4d where b permutes the states, a is a simple idempotent, and $s, t \in Q$ and $0 < d < n$ are the parameters as in Theorem 7.31 w.r.t. $\mathcal{A}_t$ and $\mathcal{A}_c$ for $n \geq 4$. We show a scheme to test whether $\mathcal{A}$ equals $\mathcal{A}_c$ or $\mathcal{A}_t$, which gives the claim by Theorem 7.31. By iterating through the states, we search for $t \in Q$ with $\delta^{-1}(t, a) = \{s, t\}$ for some $a \in \Sigma$ and $s \in Q$. Let $b \in \Sigma \setminus \{a\}$. Then, $\mathcal{A}$ equals $\mathcal{A}_c$ or $\mathcal{A}_t$ precisely in the following two cases.

$$\mathcal{A} = \begin{cases} \mathcal{A}_c & \text{if } \delta(s, b^n) = s, \delta(s, b^d) = t, \forall i \in \{1, \dots, n-1\} \colon \delta(s, b^i) \neq s, \\ \mathcal{A}_t & \text{if } \delta(s, b^{n-1}) = s, \delta(t, b) = t, \forall i \in \{1, \dots, n-1\} \colon \delta(s, b^i) \neq s, \end{cases}$$

for some $d \in \{1, \dots, n-1\}$ coprime to n. By iterating through the states $\delta(s, b^i)$ and by Theorem 3.79 each case can be tested in deterministic log-space. □

Proposition 13.9 *Deciding whether a given binary n-state SI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is synchronizing is in L.*

Proof. A binary SI-automaton is synchronizing if and only if it has one of the two forms from Theorem 7.31, and this can be tested in deterministic log-space by Theorem 13.8. □

Theorem 13.10 *For a binary SI-automaton it is L-complete to decide each of the following properties individually: synchronizability, complete reachability, complete distinguishability, sync-maximality.*

Proof. We can assume the input automaton has at least three states. Containment in L follows from Theorem 13.8 for synchronizability. By Theorem 12.29, for a binary SI-automaton the properties of complete reachability, complete distinguishability, and sync-maximality are each equivalent to the fact that the automaton equals the one from Figure 7.4d, which can be decided in L by Theorem 13.8. For L-hardness, we give a reduction from SingleCycle. Let $f \colon [n] \to [n]$ be a permutation with $n \geq 3$. We can assume $f \neq \mathrm{id}_{[n]}$ with $f(p) \neq p$ for $p \in [n]$, which can be determined by an AC^0 circuit. Set $\mathcal{A} = ([n], \{a, b\}, \delta)$ with $\delta(q, b) = f(q)$ and

$$\delta(q, a) = \begin{cases} f(p) & \text{if } q = p, \\ q & \text{if } q \neq p. \end{cases}$$

for $q \in [n]$. Observe that $\mathcal{A}$ cannot equal the automaton from Figure 7.4c, as a moves a point in a cycle of f to another point in the same cycle. Now, the following are equivalent:

1. f cyclically permutes $[n]$,
2. $\mathcal{A}$ is synchronizing,
3. $\mathcal{A}$ has the form from Theorem 12.29, Item 7, where $d = 1$ in this case.

So, by Theorem 12.29, f cyclically permuting $[n]$ is also equivalent to complete reachability and to sync-maximality. Hence, the construction gives a valid reduction for all these properties. □

13.4 Discussion

By the result of Ferens & Szykuła [205] (see Theorem 13.4) complete reachability can be decided in polynomial time. With regard to the NL-hardness results, whose proofs are from [308] with some minor corrections, we pose the *open problem* whether deciding complete reachability is in NL. Szykuła & Zyzik [595] (personal communication) informed me that they found proofs showing that deciding complete distinguishability and deciding sync-maximality are both PSPACE-complete. As noted in Table 13.1 the precise computational complexity of deciding complete distinguishability for binary automata is unknown, as well as the precise complexity for complete distinguishability/sync-maximality for binary automata and for SI-automata as input.

Chapter 14
Permutation Groups

Here, we investigate the properties of synchronizability, reachability, and distinguishability in transformation monoids generated by a permutation group and some non-permutation. Section 14.1 recaps some known characterizations for primitive groups, and gives new characterizations in terms of minimality, synchronizability, complete reachability, and complete distinguishability of associated automata. The subclass of synchronizing groups is introduced in Section 14.2. This section also discusses k-homogeneous groups. Section 14.3 introduces the new notion of A-primitivity for a subset A of the permutation domain as a generalization of primitivity. We show that several properties of primitive groups carry over to A-primitive groups.

14.1 Primitive Permutation Groups

In Sections 14.1.1 and 14.1.2 we recap known characterizations. Lastly, in Section 14.1.3, we state novel characterizations in terms of reachability and distinguishability properties of associated automata and transformation semigroups.

14.1.1 Known Characterizations

We say $G \leq_G \mathcal{S}_n$ *synchronizes* a non-permutation $t\colon [n] \to [n]$ if $\langle G \cup \{t\}\rangle_S$ contains a constant map.

The following characterization of primitivity appears implicitly in [535]. Explicitly, it was stated for general transformations in [34, Thm. 1] and for general and idempotent transformations in [36, Thm. 8.1].[1]

[1] In [36, Thm. 8.1] the permutation group is assumed to be transitive, but with Theorem 5.1 and as a defect one map on two points is constant this assumption can be omitted w.r.t. our definition of primitivity.

Remark 14.1 The square brackets (with "idempotent" written inside) indicate that Theorem 14.2, Item 2 entails two statements, the first statement universally quantifies over all transformations of defect one and the second statement universally quantifies only over all idempotent transformations of defect one.

Theorem 14.2 (Rystsov [535], see also [36, Thm. 8.1] or [34, Thm. 1]) *Let G be a permutation group on $[n]$. Then the following are equivalent:*

1. *G is primitive.*
2. *G synchronizes every [idempotent] transformation $t\colon [n] \to [n]$ of defect one.*

Next, we list four closely related characterizations (with proofs).

Theorem 14.3 *Let $G \leq_G \mathcal{S}_n$ be a permutation group and $n \geq 3$. The following are equivalent.*

1. *G is primitive.*
2. *For every distinct $a, b \in [n]$ the undirected orbital graph of $\{a, b\}$ is connected.*
3. *(Higman [292, (1.12)]) For every distinct $a, b \in [n]$ the directed orbital graph of (a, b) is strongly connected.*
4. *(Araújo, Bentz & Cameron [31, Prop. 2.2]) For any disjoint and non-empty $A, B \subseteq [n]$ with $A \cup B = [n]$ and distinct $a, b \in [n]$ there exists $g \in G$ such that $g(a) \in A$ and $g(b) \in B$.*
5. *(Araújo, Bentz & Cameron [31, Thm. 1.1]) For every $t\colon [n] \to [n]$ of rank two and $S = \langle G \cup \{t\}\rangle_S$ we have $S = EG$ where E is the set of idempotents of S.*[2]

Proof. $1 \Rightarrow 2$: The group G is contained in the automorphism group of the undirected orbital graph of $\{a, b\}$; so, its connected components form blocks. The points a and b are in the same connected component. As $a \neq b$, by primitivity, it follows that $[n]$ is a connected component.

$2 \Rightarrow 3$: Let $a, b \in [n]$ be distinct. By Item 2, the directed orbital graph of (a, b) is weakly connected. So, as $n \geq 3$, the group G must be non-trivial and we have $g(c) \neq c$ for some $c \in [n]$. By assumption, the undirected orbital graph of $\{c, g(c)\}$ is connected. Since all edges of it have their endpoints in $\mathrm{Orb}_G(c)$, we find $\mathrm{Orb}_G(c) = [n]$. So, the directed orbital graph of (a, b) is vertex-transitive. By Theorem 5.43 it is strongly connected.

$3 \Rightarrow 4$: By Theorem 3.67, Item 4 is equivalent to strong connectivity of the directed orbital graph of (a, b).

$4 \Rightarrow 5$: By Theorem 5.37 we have $S = \langle K\rangle_M G$ for $K = \{t^g \mid g \in G\}$. First, we show $\langle K\rangle_M \subseteq EG$. Let $h \in \langle K\rangle_M$ be a non-identity transformation. By Theorem 3.12, Item 3.18 we have $\mathrm{rank}(h) \leq 2$. Transformations of rank one are idempotent and so in EG. Thus, suppose $h([n]) = \{a, b\}$ for distinct $a, b \in [n]$. As $h^{-1}(a)$ and $h^{-1}(b)$ partition $[n]$, by Item 4 there exists $g \in G$ such that $g(a) \in h^{-1}(a)$ and $g(b) \in h^{-1}(b)$. Then $(hg)(a) = a$ and $(hg)(b) = b$, which gives that hg is idempotent of rank two. Hence $h = (hg)g^{-1} \in EG$. So $S = \langle K\rangle_M G \subseteq EGG = EG \subseteq S$, which gives $S = EG$.

[2] As $ge = e^g g$ for $e \in E$ and $g \in G$ we have $EG = GE$.

$5 \Rightarrow 1$: Suppose G is not primitive and let $\Delta \subseteq [n]$ be a non-trivial block. Pick distinct $a, b \in \Delta$ and let $t\colon [n] \to [n]$ be given by

$$t(x) = \begin{cases} a & x \in \Delta, \\ b & x \notin \Delta. \end{cases}$$

For $g \in G$ we have $t(g([n])) = t([n]) = \{a, b\}$. As Δ is a block, it is $g(\{a, b\}) \cap \Delta = \emptyset$ or $g(\{a, b\}) \subseteq \Delta$. So $t(g(a)) = t(g(b))$. As tg moves a point of its image it is not idempotent, i.e., $tg \notin E$. As $t \in EG$ is equivalent to $tg \in E$ for some $g \in G$ we find $t \notin EG$. □

Camille Jordan [346] (see also [34, Lem. 6] or [337, Thm. 8.7]) gave the following sufficient condition for a group to be the whole symmetric group.

Theorem 14.4 (Jordan [346]) *Let $G \leq_G \mathcal{S}_n$ be primitive. Then for every transposition $t \in \mathcal{S}_n$ we have $\langle G \cup \{t\}\rangle_G = \mathcal{S}_n$.*

Next, we reformulate Jordan's theorem as a characterization of primitivity.[3] Actually, Jordan proved, more generally, that every primitive permutation group $G \leq \mathcal{S}_n$ containing a p-cycle for some prime $p < n - 2$ equals $\mathcal{S}_n$ or $\mathcal{A}_n$. Later, Neumann [463] proved that containment of a p^r-cycle with $p \neq 3$, $p^r < n - 2$ and p prime is sufficient as well for this conclusion.

Corollary 14.5 *A group $G \leq_G \mathcal{S}_n$ is primitive if and only if for every transposition $t \in \mathcal{S}_n$ we have $\langle G \cup \{t\}\rangle_G = \mathcal{S}_n$.*

Proof. First, let $\sim$ be a non-trivial G-congruence with $a \sim b$ for $a \neq b$. Consider the transposition $t = (a\ b)$. For all $x, y \in [n]$ we have $x \sim y$ if and only if $t(x) \sim t(y)$. This yields that $\sim$ is also a $\langle G \cup \{t\}\rangle_G$-congruence. As $\mathcal{S}_n$ is primitive and $\sim$ is a non-trivial $\langle G \cup \{t\}\rangle_G$-congruence, we have $\langle G \cup \{t\}\rangle_G \neq \mathcal{S}_n$. Conversely, for primitive G, the claim follows from Theorem 14.4. □

Lastly, we present a well-known characterization in terms of the maximality of point stabilizers. For the convenience of the reader, we include the proofs.

Theorem 14.6 *A permutation group $G \leq_G \mathcal{S}_n$ with $n \geq 3$ is primitive if and only if G is transitive and for each $a \in [n]$ (equivalently for some $a \in [n]$ by Equation* (5.3)*) the point stabilizer subgroup G_a is a maximal subgroup in G.*

Proof. Suppose G is primitive. As $n \geq 3$, by Theorem 5.1, G is transitive. Let $a \in [n]$ and $H \leq_G G$ with $G_a \leq_G H \leq_G G$. By Theorem 5.15, Item 1, $\mathrm{Orb}_H(a)$ is a block. So, it equals $\{a\}$ or $[n]$, where in the former case we have $H = G_a$ and in the latter case $H = G$ by Theorem 5.19, Item 2.

Conversely, let $\Delta \subseteq [n]$ be a block and $a \in \Delta$. By Theorem 5.15, Item 1, $H = \{g \in G \mid g(\Delta) = \Delta\}$ is a subgroup containing G_a. So, by assumption, $H = G_a$ or $H = G$.

[3] The is also another statement in permutation group theory known as Jordan's theorem in the literature, namely the fact that every transitive permutation group contains a permutation with no fixpoint [557].

Suppose $H = G_a$. Assume $b \in \Delta$ with $a \neq b$. We have $g(a) = b$ for some $g \in G$ by transitivity. Since Δ is a block, this gives $g(\Delta) = \Delta$ and thus $g \in H$, contradicting the assumption that all element from H fix a. Thus, in this case, we have $\Delta = \{a\}$. Next, suppose $H = G$. By transitivity, the only only set permuted by all elements from G is $[n]$. Thus, in this case, we have $\Delta = [n]$.[4] □

We cannot omit transitivity in Theorem 14.6. For example in $G = \langle\{(1\,2\,3), (4\,5\,6)\}\rangle_G$ all point stabilizers are maximal, but G is not primitive.

14.1.2 Minimal Automata

Davies [171, Cor. 3.3] observed (see also [172, Cor. 4.2.4]) that for a permutation automaton $\mathcal{A} = (Q, \Sigma, \delta)$, $\mathcal{A}_{q,F}$ is minimal for each $q \in Q$ and $\emptyset \neq F \subsetneq Q$ if and only if $\mathcal{T}_M(\mathcal{A})$ is a primitive permutation group. Here, in Theorem 14.7, we state and prove her result, also with a corresponding statement related to morphisms. Similarly as in Theorem 14.1, we use square brackets to combine two related statements, one making a statement on all automata with $G \subseteq \mathcal{T}_S(A)$ and the other on all automata with $G = \mathcal{T}_S(A)$.

Theorem 14.7 (Davies [171, 172]) *Let G be a permutation group on the finite set Q with $|Q| \geq 3$. Then the following are equivalent.*

1. *G is a primitive permutation group.*
2. *For every $\mathcal{A} = (Q, \Sigma, \delta)$ with $G \subseteq \mathcal{T}_S(\mathcal{A})$ [with $G = \mathcal{T}_S(\mathcal{A})$] there exists no morphism $\varphi\colon Q \to S$ into an automaton $\mathcal{B} = (S, \Sigma, \mu)$ such that $1 < |\varphi(Q)| < |Q|$.*
3. *For every $\mathcal{A} = (Q, \Sigma, \delta)$ with $G \subseteq \mathcal{T}_S(\mathcal{A})$ [with $G = \mathcal{T}_S(\mathcal{A})$] the automaton $\mathcal{A}_{q,F}$ is minimal for all $F \subseteq Q$ with $F \notin \{\emptyset, Q\}$ and all $q \in Q$.*

Proof. 1 $\Rightarrow$ 2: As every $\mathcal{T}_S(\mathcal{A})$-congruence is a G-congruence for automata $\mathcal{A} = (Q, \Sigma, \delta)$ with $G \subseteq \mathcal{T}_S(\mathcal{A})$ (resp. $G = \mathcal{T}_S(\mathcal{A})$), primitivity of G yields the absence of any non-trivial $\mathcal{T}_S(\mathcal{A})$-congruences for all $\mathcal{A} = (Q, \Sigma, \delta)$ with $G \subseteq \mathcal{T}_S(\mathcal{A})$ (resp. $G = \mathcal{T}_S(\mathcal{A})$). By Theorem 6.56, Item 2, the kernel relation of every morphism is trivial, which gives Item 2.

2 $\Rightarrow$ 3: By Theorem 6.56, Item 2, all $\mathcal{A} = (Q, \Sigma, \delta)$ with $G \subseteq \mathcal{T}_S(\mathcal{A})$ (resp. $G = \mathcal{T}_S(\mathcal{A})$) do not admit any non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence. Consider $\mathcal{A}$ with $G = \mathcal{T}_S(\mathcal{A})$. Then $\mathcal{A}$ is a permutation automaton not admitting any non-trivial $\mathcal{T}_M(\mathcal{A})$-congruences. Since in a permutation automaton a trap state is not reachable from any other state, from Theorem 6.68 we obtain that $\mathcal{A}$ must be strongly connected, which yields that G is transitive. Then, every $\mathcal{A}$ with $G \subseteq \mathcal{T}_S(\mathcal{A})$ is strongly connected. So, by Theorem 6.91, from Item 2 we obtain Item 3.

[4] Alternatively, Suppose G is imprimitive. Let $\Delta \subseteq [n]$ be a non-trivial block. Set $H = \{g \in G \mid g(\Delta) = \Delta\}$ and let $a, b \in \Delta$ be distinct. As G is transitive and $\Delta \neq [n]$, we have $H \neq G$ and there exists $g \in G$ such that $g(a) = b$. The fact that Δ is a block yields $G_a \subseteq H$ and $g(\Delta) = \Delta$ so that $g \in H \setminus G_a$. Hence G_a is not a maximal subgroup.

$3 \Rightarrow 1$: By assumption and Theorem 6.91 every automaton $\mathcal{A}$ with $G = \mathcal{T}_S(\mathcal{A})$ does not admit any non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence, which is equivalent to primitivity of G. □

Theorem 14.7, Item 3 fails for $|Q| = 2$, as witnessed by the two-state automaton in which all letters induce self-loops. This automaton is not minimal for any choice of start state and subset of final states and its transformation monoid is a primitive group. Note that Theorem 14.7, Item 2 does not exclude the possibility of a non-trivial isomorphism.

14.1.3 Automata for the Set of Synchronizing Words

Here, we present new characterizations of primitive permutation groups by properties related to synchronization and subset mapping properties of associated automata having a defect-one letter and containing the group in their transformation semigroups. Specifically, Theorem 14.9 characterizes primitivity by complete reachability, complete distinguishability, sync-maximality, and distinguishability of pairs of disjoint k-sets for some $4 \leq k^2 < n$ in associated automata with n states. Theorem 14.11 formulates the results from the present section in a purely algebraic language, without reference to automata. The results of the present section are taken from [309]. The following lemma is used in the proof of Theorem 14.9.

Lemma 14.8 *Let $G \leq_G \mathcal{S}_n$. Then G is transitive if one of the following holds:*

1. *We have $n \geq 5$ and for each simple idempotent $t\colon [n] \to [n]$ every k-set of states is distinguishable from every disjoint l-set of states $\langle G \cup \{t\}\rangle_S$ for some $k, l \geq 2$ with $k + l \leq n$.*[5]
2. *We have $n \geq 3$ and for each simple idempotent $t\colon [n] \to [n]$ every k-set of states is distinguishable from every distinct l-set of states in $\langle G \cup \{t\}\rangle_S$ for some $2 \leq k, l \leq n$ where $n > k$ or $n > l$.*

Proof. 1: Suppose G is not transitive. By Theorem 5.33 there exists a subset $A \subseteq [n]$ permuted by each element from G containing three distinct $a_1, a_2, a_3 \in A$ and such that there exists $b \in [n] \setminus A$. First, suppose the complement of A contains two distinct points. Consider the simple idempotent $t\colon [n] \to [n]$ mapping one point from $[n] \setminus A$ to another point in $[n] \setminus A$. Then we have $h(a_1) \neq h(a_2)$ and $h(a_3) \neq h(b)$ for all $h \in \langle G \cup \{t\}\rangle_S$. Since $k + l \leq n$, we can pick $A, B \subseteq [n]$ where $|A| = l$ and $|B| = k$ such that $A \cap B = \emptyset$ with $a_1, a_2 \in A$ and $a_3, b \in A$. Then A and B are not distinguishable.[6] Next, suppose $[n] \setminus A = \{b\}$. Since $n \geq 5$, there exists a fourth point $a_4 \in A \setminus \{a_1, a_2, a_3\}$. Consider the simple idempotent $t\colon [n] \to [n]$ mapping b to a_1. We have $h(a_1) \neq h(a_2)$ and $h(a_3) \neq h(a_4)$ for all $h \in \langle G \cup \{t\}\rangle_S$ since $t(A) = A$ and $g(A) = A$ for $g \in G$. Pick

[5] In [309] this was only proved for $k = l = 2$.

[6] If the complement of A contains two distinct points b_1, b_2, the same conclusion can be drawn by considering $\{a_1, b_1\}$ and $\{a_2, b_2\}$ instead of $\{a_1, a_2\}$ and $\{a_3, b\}$.

$A, B \subseteq [n]$ where $|A| = l$ and $|B| = k$ such that $A \cap B = \emptyset$ with $a_1, a_2 \in A$ and $a_3, a_4 \in A$. Then A and B are not distinguishable.

1: Suppose G is not transitive. By Theorem 5.33 there exists a subset $A \subseteq [n]$ permuted by each element from G and three distinct $a, b, c \in [n]$ with $a, b \in A$ and $c \in [n] \setminus A$. Let $t\colon [n] \to [n]$ be the simple idempotent mapping a to b. We have $h(a) \neq h(c)$ and $h(b) \neq h(c)$ for all $h \in \langle G \cup \{t\}\rangle_S$ since $t(A) \subseteq A, t([n] \setminus A) = [n] \setminus A$, $g(A) = A$ and $g([n] \setminus A) = [n] \setminus A$ for $g \in G$. Since $2 \leq k, l \leq n$ with $n > l$ or $n > k$ there exist two distinct $A, B \subseteq [n]$ such that $|A| = k$ and $|B| = l$ with $a, c \in A$ and $b, c \in B$, and these two subsets are indistinguishable since neither can be mapped to a singleton.[7] □

Theorem 14.9 (Hoffmann [309]) *Let G be a permutation group on the finite set Q of size at least five. Then the following are equivalent.*

1. *G is primitive.*
2. *Every SI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with a single simple idempotent letter, i.e., $|\Sigma_I| = 1$, and $G = \{\delta_u \mid u \in \Sigma_G^*\}$ is synchronizing.*
3. *Every $\mathcal{A} = (Q, \Sigma, \delta)$ having a letter of defect one with $G \subseteq \mathcal{T}_S(\mathcal{A})$ is synchronizing.*
4. *Every $\mathcal{A} = (Q, \Sigma, \delta)$ having a letter of defect one with $G \subseteq \mathcal{T}_S(\mathcal{A})$ is completely reachable.*
5. *Every $\mathcal{A} = (Q, \Sigma, \delta)$ having a letter of defect one with $G \subseteq \mathcal{T}_S(\mathcal{A})$ is completely distinguishable.*
6. *Every $\mathcal{A} = (Q, \Sigma, \delta)$ having a letter of defect one with $G \subseteq \mathcal{T}_S(\mathcal{A})$ is sync-maximal.*
7. *For $\mathcal{A} = (Q, \Sigma, \delta)$ having a letter of defect one with $G \subseteq \mathcal{T}_S(\mathcal{A})$ every k-sets of states is distinguishable from every distinct l-set of states for some $2 \leq k, l \leq |Q|$ with $|Q| > k$ or $|Q| > l$.*
8. *For $\mathcal{A} = (Q, \Sigma, \delta)$ having a letter of defect one with $G \subseteq \mathcal{T}_S(\mathcal{A})$ every k-sets of states is distinguishable from every disjoint l-set of states for some $k, l \geq 2$ with $kl < |Q|$.*

Proof. The equivalences of Items 1, 2, and 3 are basically a reformulation of Rystsov's theorem (Theorem 14.2) in automata-theoretical language (see Section 6.8 for a discussion of the relation between transformation semigroups and semiautomata).

Next, if Items 4, 5, or 6 hold, then the automaton $\mathcal{A}$ is synchronizing by Theorem 9.12,Theorem 9.44, Theorem 10.1, Item 1, If Items 7 and 8 holds, then, by Theorem 14.8, G is transitive. By restricting the alphabet, we can assume $\mathcal{A}$ has a single non-permutational letter of defect one. Then, with Theorem 12.10 we get that $\mathcal{A}$ is synchronizing. Thus, each of these assumptions implies Item 3.[8]

Lastly, suppose G is primitive. Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $G \subseteq \mathcal{T}_S(\mathcal{A})$ having a letter of defect one. By Theorem 5.1, G is transitive. By Theorem 5.13 we find that there

[7] If $k > 2$ or $l > 2$, then we can also consider distinct $A, B \subseteq [n]$ with $a, c \in A \cap B$; however, the point b is needed for the case $k = l = 2$.

[8] We can also give an alternative argument that shows Item 1. From assuming Item 7: Suppose there exists a non-trivial G-congruence $\sim$. Then there exist distinct $a, b, c \in X$ such that $a \sim b$ and $a \nsim c$. Consider the simple idempotent $t\colon X \to X$ mapping a to b. Then $\sim$ is a $\langle G \cup \{t\}\rangle_S$-congruence and so $h(a) \nsim h(c)$ and $h(b) \nsim h(c)$ for all $h \in \langle G \cup \{t\}\rangle_S$. Thus, no two subsets $A, B \subseteq X$ with $\{a, c\} \subseteq A$ and $\{b, c\} \subseteq B$ are distinguishable.

exists a word u inducing a simple idempotent. Now consider a new automaton on the same state set but where u and all permutational letters are regarded as letters. This gives an SI-automaton that also has G in the group of units of its transformation monoid. Since we have already established the equivalence of Items 1 and 3, we find that this SI-automaton is synchronizing. By Theorem 12.23 it is also completely reachable and completely distinguishable. The transformation monoid of this SI-automaton is a submonoid of $\mathcal{T}_M(\mathcal{A})$. This yields that $\mathcal{A}$ is completely reachable and completely distinguishable. Thus we obtain Items 4,5,6, 7, and 8. This concludes the proof and establishes all the equivalences. □

Corollary 14.10 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with at least five states such that the group of units of $\mathcal{T}_S(\mathcal{A})$ is primitive. Then $\mathcal{A}$ is completely reachable if and only if it is sync-maximal.*

Proof. If $\mathcal{A}$ is completely reachable, it must contain a letter a of defect one. By Theorem 14.9, the automaton restricted to the alphabet $\{a\} \cup \{x \in \Sigma \mid x \text{ is a permutational letter}\}$ is completely distinguishable. Thus, $\mathcal{A}$ is completely distinguishable and so sync-maximal. Conversely, since a primitive group is transitive by Theorem 5.1, $\mathcal{A}$ is strongly connected and so sync-maximality implies complete reachability. □

By Theorem 14.10, if the transformation monoid contains a primitive permutation group, complete reachability implies complete distinguishability. But the converse implication does not hold in general, see, for example, Theorem 9.51.

The equivalences from Theorem 14.9 also hold true when restricted to the following subclasses of automata: SI-automata with $\Sigma_I \neq \emptyset$, SI-automata with $|\Sigma_I| = 1$, automata with a group of units equal to G having a letter of defect one, and automata having only a single non-permutational letter of defect one.

For $\mathcal{A} = (Q, \Sigma, \delta)$, $t \colon Q \to Q$, and a new symbol $a \notin \Sigma$ we define a semiautomaton $\mathcal{A}_{t,a} = (Q, \Sigma \cup \{a\}, \mu)$ with, for $q \in Q$ and $x \in \Sigma \cup \{a\}$,

$$\mu(q, x) = \begin{cases} \delta(q, x) & \text{if } x \in \Sigma, \\ t(q) & \text{if } x = a. \end{cases}$$

Figure 14.1 shows the main equivalences from Theorems 14.7 and 14.9 when adjoining a single letter of defect one.

Next, as an example, we give a formulation of some properties from Theorem 14.9 without any reference to automata. In the sections to follow all results will be given without reference to automata, but they all can be formulated as statements about automata similarly as in Theorem 14.9. We combine two statements (related via Theorem 5.13) into one by putting the word "idempotent" in square brackets, see Theorem 14.1.

Corollary 14.11 (Hoffmann [309]) *Let $G \leq_G \mathcal{S}_n$ be a permutation group with $n \geq 5$. Then the following are equivalent.*

A similar argument works when assuming Item 8. If we can pick distinct a, b, c, d with $a \sim b$, $c \nsim a$, and $d \nsim a$. then we can conclude as before that disjoint sets A, B with $\{a, b\} \subseteq A$ and $\{c, d\} \subseteq B$ are not distinguishable. Otherwise, if four distinct a, b, c, d as stated cannot be picked, then G has a fixpoint. In this case, consider the simple idempotent that maps this fixpoint to some other point. Then pairs of subsets where neither contains the fixpoint are not distinguishable since it is not possible to map them to smaller sets.

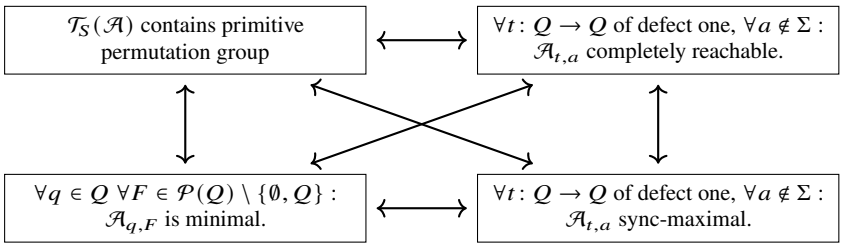


Fig. 14.1: Equivalences from Theorems 14.7 and 14.9 for $\mathcal{A} = (Q, \Sigma, \delta)$.

1. *G is primitive.*
2. *For every [idempotent] transformation $t\colon [n] \to [n]$ of defect one, $\langle G \cup \{t\}\rangle_S$ is completely reachable.*
3. *For every [idempotent] transformation $t\colon [n] \to [n]$ of defect one, $\langle G \cup \{t\}\rangle_S$ is completely distinguishable.*
4. *For every [idempotent] transformation $t\colon [n] \to [n]$ of defect one, in $\langle G \cup \{t\}\rangle_S$ every k-set is distinguishable from every distinct l-set for some $2 \leq k, l < n$.*
5. *For every [idempotent] transformation $t\colon [n] \to [n]$ of defect one, in $\langle G \cup \{t\}\rangle_S$ every k-set is distinguishable from every disjoint l-set for some $k, l \geq 2$ with $kl < n$.*

We cannot omit the assumption $kl < n$ in Item 5 from Theorem 14.11 as demonstrated by the following example with $k = l = 2$.

Example 14.12 Let $g = (0\ 1\ 2)(3)$. Then $G = \langle\{g\}\rangle_G$ is imprimitive as it is not transitive. Set

$$S = \{\{0,3\}^h \mid h \in G\} = \{\{0,3\},\{1,3\},\{2,3\}\},$$
$$T = \{\{0,1\}^h \mid h \in G\} = \{\{0,1\},\{1,2\},\{2,0\}\}.$$

Let $\{a,b\}$, $\{c,d\}$ be two disjoint 2-sets. Then $\{a,b\} \in S$ and $\{c,d\} \in T$, or $\{a,b\} \in T$ and $\{c,d\} \in S$. Now, for $t\colon [4] \to [4]$ of defect one, there exists $h \in G$ mapping precisely one of $\{a,b\}$ or $\{c,d\}$ to the unique 2-set mapped by t to a singleton. Then th distinguishes $\{a,b\}$ and $\{c,d\}$.

14.2 Synchronizing and k-Homogeneous Groups

Let $G \leq_G \mathcal{S}_n$. We call G *synchronizing* [40] if it synchronizes every non-permutation. A partition $P_1, \ldots, P_m$ of $[n]$ is called *section-regular* [465] for G if there exists a subset $S \subseteq [n]$ that is a section for each partition of the form $g(P_1), \ldots, g(P_m)$ where $g \in G$. Equivalently for each $g \in G$ the set $g(S)$ is a section of $P_1, \ldots, P_m$.

Theorem 14.13 (Neumann [465, Note 6.1]) *A permutation group $G \leq_G \mathcal{S}_n$ is synchronizing if and only if there exists no non-trivial section-regular partition for G.*

Next, we show that synchronizing groups lie between the 2-homogeneous and primitive groups.

Proposition 14.14 *1. (Arnold and Steinberg [40]) A synchronizing group is primitive.*
2. Every 2-homogeneous group is synchronizing.

Proof. 1: This follows since a system of blocks is a section-regular partition (for every section).

2: Let $t\colon [n] \to [n]$ be a transformation such that $t(a) = t(b)$ for distinct $a, b \in [n]$. As G is 2-homogeneous, for distinct $c, d \in [n]$ there exists $g \in G$ such that $g(\{c, d\}) = \{a, b\}$. So, tg maps $\{c, d\}$ to a singleton set. Thus, every pair of states can be mapped to a singleton set. By the transformation semigroup version of the pair criterion (Theorem 7.18 and Section 6.8) we find that every 2-homogeneous group is synchronizing. □

Example 14.15 Let $n \geq 4$ and $G = \langle (0\ 1\ \ldots\ n-1) \rangle_G$. Then G is synchronizing if and only if n is prime. For assume n is prime and suppose $P_1, \ldots, P_m$ is a section-regular partition witnessed by $S \subseteq [n]$. Let $i \in \{1, \ldots, m\}$. For $y \in P_i$ set

$$R_y = \{g \in G \mid \exists s \in S\colon g(s) = y\}.$$

Then $G = \bigcup_{y \in P_i} R_y$ and $R_y \cap R_{y'} = \emptyset$ for distinct $y, y' \in P_i$. As G is a regular permutation group, for $s \in S$ and $y \in P_i$ there exists precisely one $g \in G$ with $g(s) = y$. So, $|R_y| = |S|$, which gives $n = |G| = |S||P_i|$. As n is prime, $|P_i| \in \{1, n\}$. Hence, the partition is trivial. By Theorem 14.13, G is synchronizing. Conversely, assume $n = rm$ for $0 < r, m < n$. Then $\Delta = \{a \in [n] \mid a \equiv 0 \pmod{r}\}$ is a non-trivial block. So, by Item 1 of Theorem 14.14, G is not synchronizing.

Araújo asked whether the synchronizing groups are properly contained inside the primitive groups (see the introduction of [465]). This was answered affirmatively by Neumann [465]. The next example giving a non-synchronizing primitive group is due to Neumann [465, Ex. 3.3].

Example 14.16 (Neumann [465, Ex. 3.3]) Set $G = \mathcal{A}_5 \times \mathcal{A}_5$. It is well-known [126, 179] that $\mathcal{A}_5$ is simple and $Z(\mathcal{A}_5) = \{\mathrm{id}_{[5]}\}$ (recall that $Z(G) = \{g \in G \mid \forall h \in G\colon hg = gh\}$ for a group G, see page 43). By Theorem 5.35, the representation of G by the diagonal action G^D as a permutation group on $\mathcal{A}_5$ is primitive. Let U be the point stabilizer subgroup of 5 in $\mathcal{A}_5$, which is isomorphic to $\mathcal{A}_4$. Set $g = (0\ 1\ 2\ 3\ 4)$.

Let $y \in G$. By Equation (5.3), U^y is the point stabilizer of $y(5)$. By Equation (5.1), the sets $g^i U^y$ for $i \in \{0, 1, 2, 3, 4\}$ are all the left cosets of U^y. Thus, the set $S = \{\mathrm{id}_{[5]}, g, g^2, g^3, g^4\}$ is a section for the partition induced by all these cosets. Since for $i \in \{0, 1, 2, 3, 4\}$ and $(x, y) \in G$ we have $xg^iUy^{-1} = xy^{-1}(g^y)^iU^y$, the sets

$$xUy^{-1}, xgUy^{-1}, xg^2Uy^{-1}, xg^3Uy^{-1}, xg^4Uy^{-1}$$

are all the left cosets of U^y. So, the partition into left cosets of U is section-regular for G^D. Thus, G^D is not synchronizing.

McAlister [445] (see also [36, Thm. 8.2]) proved that a permutation group G is 2-homogenous if and only if $\langle G \cup \{t\}\rangle_S$ contains every non-permutation for any $t\colon [n] \to [n]$ of defect one. Next, we give a different characterization for k-homogeneity in Theorem 14.17. This characterization was given in [309, Lem. 4.1]. Then, in Theorem 14.19, we characterize those groups on domains of size n in which all subsets of size $n - i \cdot k$ are reachable when adjoining a transformation of defect k. As a sufficient condition for subset reachability in the special case $k = 2$ this was proved in [309, Prop. 4.2].

Proposition 14.17 *Let $G \leq_G \mathcal{S}_n$ and $1 \leq k \leq n-1$. Then the following are equivalent.*

1. *G is k-homogeneous.*
2. *Every $(n-k)$-set is reachable in $\langle G \cup \{t\}\rangle_S$ for every $t\colon [n] \to [n]$ of defect k.*
3. *Every $(n-k)$-set is reachable in $\langle G \cup \{t\}\rangle_S$ for some $t\colon [n] \to [n]$ of defect k.*
4. *Every k-set is reachable in $\langle G \cup \{t\}\rangle_S$ for every $t\colon [n] \to [n]$ of rank k.*
5. *Every k-set is reachable in $\langle G \cup \{t\}\rangle_S$ for some $t\colon [n] \to [n]$ of rank k.*

Proof. We first show that Items 1 to 3 are equivalent. Since k-homogeneity is equivalent to $(n-k)$-homogeneity, and t has defect $n-k$ if and only if it has rank k, it then follows that Items 1, 4, and 5 are also equivalent.

$1 \Rightarrow 2$: Let $A \subseteq [n]$ be of size $n-k$. Since G is k-homogeneous, there exists $g \in G$ such that $[n] \setminus A = g\big([n] \setminus t([n])\big)$. Consequently, $A = g(t([n]))$.

$2 \Rightarrow 3$: This is clear.

$3 \Rightarrow 1$: Let $A \subseteq [n]$ be of size $n-k$. By assumption, and since $|A| < n$, there exist $h \in \langle G \cup \{t\}\rangle_S$ and $g \in G$ such that

$$A = g(t(h([n]))).$$

Hence, $|t(h([n]))| = n-k$. Since $t(h([n])) \subseteq t([n])$ and $|t([n])| = n-k$, we must have $t(h([n])) = t([n])$. Therefore, every $(n-k)$-subset of $[n]$ is the image of $t([n])$ under an element of G. Thus G is $(n-k)$-homogeneous, and hence k-homogeneous. □

Corollary 14.18 *Let $G \leq_G \mathcal{S}_n$ be a permutation group and $2 \leq k < n$. Suppose for each $t\colon [n] \to [n]$ of defect k, all subsets of size at most $n-k$ are reachable in $\langle G \cup \{t\}\rangle_S$. Then G is k-homogeneous.*

Theorem 14.19 *Let $G \leq_G \mathcal{S}_n$ be a permutation group and let $1 \leq k < n$. Then the following are equivalent:*

1. *For every [idempotent] one-point singular $t\colon [n] \to [n]$ of defect k, all subsets of size $n - ik$ for $1 \leq i < \frac{n}{k}$ are reachable in $\langle G \cup \{t\}\rangle_S$.*
2. *For distinct $a_1, \ldots, a_{k+1} \in [n]$ and every subset $A \subseteq [n]$ of size $n-ik$ for $1 \leq i < \frac{n}{k}$, there exists $g \in G$ such that $a_1 \in g(A)$ and $\{a_2, \ldots, a_{k+1}\} \cap g(A) = \emptyset$.*

Proof. Let $t\colon [n] \to [n]$ be a one-point singular of defect k. Consider $A \subseteq [n]$ with $|A| = n - ik$ for some $1 \leq i < \frac{n}{k}$. We proceed by induction on i.

Suppose first that $i = 1$. Since $\operatorname{excl}(t)$ is a k-set, by assumption, we have $\operatorname{excl}(t) \cap g(A) = \emptyset$ for some $g \in G$. As $|A| = n - k$, this gives $g(A) = t([n])$. Thus $A = g^{-1}(t([n]))$.

Next, suppose $i > 1$. Let $\mathrm{dupl}(t) = \{c\}$. By assumption, there exists $g \in G$ such that

$$c \in g(A) \quad \text{and} \quad \mathrm{excl}(t) \cap g(A) = \emptyset.$$

Observe the following.

1. By Theorem 4.15, since t is a one-point singular of defect k, we have $c = t(a_1) = \cdots = t(a_k)$ for k distinct points $a_1, \ldots, a_k \in [n]$.
2. Since $g(A)$ contains no point of $\mathrm{excl}(t)$, every point of $g(A)$ lies in $t([n])$. Hence, for every $a \in A$ with $g(a) \neq c$, there exists $b \in [n]$ such that $g(a) = t(b)$.

Both conditions yield that $B = t^{-1}(g(A))$ has size $|A| + k$ and satisfies $t(B) = g(A)$. By the induction hypothesis, we have $B = h([n])$ for some $h \in \langle G \cup \{t\}\rangle_S$. Hence, we find $A = g^{-1}(t(h([n])))$.

Conversely, let $a_1, \ldots, a_{k+1} \in [n]$ be distinct and $A \subseteq [n]$ with $|A| = n - ik$ for some $1 \leq i < \frac{n}{k}$. Consider the idempotent one-point singular of defect k such that

$$t(a_1) = t(a_2) = \cdots = t(a_{k+1}) = a_1.$$

Since A is reachable in $\langle G \cup \{t\}\rangle_S$ and $A \neq [n]$, we have $A = g(t(h([n])))$ for some $g \in G$ and $h \in \langle G \cup \{t\}\rangle_S$. Since a permutation maps $[n]$ to itself, we may assume $h = g_1 t \cdots t g_m t$ for some $g_1, \ldots, g_m \in G$, where $m \geq 0$ is minimal. We have $a_1 \in t(h([n]))$, since otherwise, none of the points $a_1, \ldots, a_{k+1}$ is in $h([n])$ and so t acts as the identity on the remaining points. In this case, $A = g(h([n]))$, contradicting the minimal choice of m. Since $\mathrm{excl}(t) = \{a_2, \ldots, a_{k+1}\}$, none of these points belongs to $t(h([n]))$. Therefore,

$$a_1 \in g^{-1}(A) \quad \text{and} \quad \{a_2, \ldots, a_{k+1}\} \cap g^{-1}(A) = \emptyset,$$

as required. □

Note that for $k = 1$, Theorem 14.19 reduces to Item 4 of Theorem 14.3.

14.3 A-Primitive Permutation Groups

Section 14.3.1 introduces A-primitive groups and states some basic properties. In Section 14.3.2 we investigate orbits of A-primitive groups and introduce strongly $\{a, b\}$-primitive groups. In Section 14.3.3, we collect some technical results relating distinguishability, synchronizability, and orbit sizes to $\{a, b\}$-primitivity. Then we show that the following known characterizations of primitivity admit natural generalizations to (strong) $\{a, b\}$-primitivity: Higman's orbital graph characterization (Section 14.3.4), Jordan's characterization via generating sets for the symmetric group (Section 14.3.5), and Rystsov's characterizations related to synchronizability properties (Section 14.3.6). Then, we give characterizations in terms of reachability and distinguishability properties (Section 14.3.7). Lastly, we note a characterizing property that resembles 2-homogeneity (Section 14.3.8) and yields a new characterization of primitivity.

14.3.1 Introduction and First Properties

Let $A \subseteq [n]$. A permutation group $G \leq_G \mathcal{S}_n$ is called *A-primitive* if there does not exist a non-trivial G-congruence on $[n]$ such that all elements in A are congruent. Equivalently, if $\Delta \subseteq [n]$ is a block with $A \subseteq \Delta$, then $|\Delta| = 1$ or $\Delta = [n]$. Every group is $[n]$-primitive. So, only the case $|A| < n$ is of interest. For $|A| \geq 2$ and A-primitive $G \leq_G \mathcal{S}_n$ we have that $A \subseteq \Delta$ implies $\Delta = [n]$.

Proposition 14.20 *Let $G \leq_G \mathcal{S}_n$ and $A \subseteq [n]$. If G is A-primitive, then G is B-primitive for every $B \subseteq [n]$ with $A \subseteq B$. Furthermore, if $G \leq_G H$ for some $H \leq_G \mathcal{S}_n$ and G is A-primitive, then H is A-primitive.*

Proof. First, suppose G is A-primitive. If $B \subseteq \Delta$ for some block, then $A \subseteq \Delta$. So, $|\Delta| = 1$ or $\Delta = [n]$. Next, assume $G \leq_G H$ for some $H \leq_G \mathcal{S}_n$. If Δ is a block for H and $A \subseteq \Delta$, then, as Δ is also a block for G, we get $|\Delta| = 1$ or $\Delta = [n]$. □

Obviously, a permutation group is A-primitive for all k-subsets $A \subseteq [n]$ if and only if it has no non-trivial block of size at least k.

Example 14.21 Let $G = \langle\{(0\ 1\ 2\ 3\ 4\ 5\ 6\ 7)\}\rangle_G$. The only non-trivial blocks are $\{0, 2, 4, 6\}$ and $\{1, 3, 5, 7\}$. So G is A-primitive for all subsets A that contain at least one even and at least one odd number.

Example 14.22 Generalizing the previous example, let $G = \langle\{(0\ 1\ \cdots\ n-1)\}\rangle_G$ and $A = \{a_1 < \cdots < a_m\}$ be some subset of $[n]$ with $m \geq 2$. Then G is A-primitive if and only if the greatest common divisor d of the numbers $a_{i+1} - a_i$ for $i \in \{1, \ldots, m-1\}$ and n is one. For suppose $d > 1$. Then for $a_i, a_j \in A$ with $i < j$ we have $a_j - a_i = \sum_{k=i}^{j-1}(a_{k+1} - a_k)$. Thus, the difference of any two numbers from A is divisible by d and so A is contained in the residue class modulo d, which forms a block by Theorem 5.5. Next, suppose $d = 1$. Suppose Δ is a block with $A \subseteq \Delta$. By Theorem 5.5, Δ is a residue class modulo some divisor r of n. Now $A \subseteq \Delta$ implies that all differences of numbers from A are divisible by r. Thus, we find $r = 1$, which gives $\Delta = [n]$.

A set A of minimal size such that G is A-primitive is not necessarily unique. For example, in Theorems 14.21 and 14.22, there are different sets of minimal size for which the groups are A-primitive.

Next, we give two other examples. Theorem 14.23 is rather straightforward, Theorem 14.25 constructs an imprimitive group on $[n]$ whose only non-trivial blocks have size two; and so the group is A-primitive for $A \subseteq [n]$ with $|A| \geq 3$.

Example 14.23 Let $G \leq_G \mathcal{S}_n$ be transitive and $\Delta \subseteq [n]$ be a block. By Theorem 5.4, $|\Delta|$ divides n. So, transitive permutation groups are A-primitive for each set $A \subseteq [n]$ with $\frac{n}{2} < |A| \leq n$.

Example 14.24 Consider $G = \langle\{(0\ 1)(2\ 3), (0\ 2)(1\ 3)\}\rangle_G$. Then the partitions $\{0, 1\}, \{2, 3\}$, $\{0, 2\}, \{1, 3\}$, and $\{0, 3\}, \{1, 2\}$ each induce G-congruences. Thus, with Theorem 14.23, G is only A-primitive for A with $|A| \geq 3$.

Example 14.25 Let $n \geq 2$. Here, we construct[9] a transitive imprimitive group on $[n]$ that is A-primitive for each $|A| \geq 3$ (so, by imprimitivity, there must exist a block of size two). Set $W = \{(g_1, \ldots, g_n, h) \mid g_1, \ldots, g_n \in \mathcal{S}_2, h \in \mathcal{S}_n\}$ and define an operation on W by

$$(g'_1, \ldots, g'_n, h')(g_1, \ldots, g_n, h) = (g'_{h(1)} g_1, \ldots, g'_{h(n)} g_n, h'h)$$

for $(g_1, \ldots, g_n, h), (g'_1, \ldots, g'_n, h') \in W$. Then W together with this operation forms an abstract group with identity element $(\mathrm{id}_{[2]}, \ldots, \mathrm{id}_{[2]}, \mathrm{id}_{[n]})$ and inversion given by

$$(g_1, \ldots, g_n, h)^{-1} = (g^{-1}_{h^{-1}(1)}, \ldots, g^{-1}_{h^{-1}(n)}, h^{-1}).$$

The map that sends $(g_1, \ldots, g_n, h) \in W$ to the permutation $(i, j) \mapsto (g_j(i), h(j))$ for $(i, j) \in [2] \times [n]$ is an injective morphism from W into $\mathcal{S}_{[2]\times[n]}$, which yields that W can be seen as a permutation group on $[2]\times[n]$. This permutation group is transitive and the sets $\{1, 2\} \times \{j\}$ for $j \in [n]$ form a system of blocks. Next, suppose $\Delta \subseteq [2] \times [n]$ is a block with $(i, j), (i', j') \in \Delta$ for $j \neq j'$. Consider $(i'', j'') \in [2] \times [n]$. Without loss of generality, assume $j' \neq j''$. Let $h \in \mathcal{S}_n$ and $g \in \mathcal{S}_2$ be such that $h(j) = j''$, $h(j') = j'$ and $g(i) = i''$. Then $(\mathrm{id}_{[n]}, \ldots, \mathrm{id}_{[n]}, g, \mathrm{id}_{[n]}, \ldots, \mathrm{id}_{[n]}, h) \in W$, where g is put into the j-th position, maps (i, j) to (i'', j'') and (i', j') to itself. As Δ is a block, the element permutes Δ and we get $(i'', j'') \in \Delta$. So, as (i'', j'') was chosen arbitrarily, we have $\Delta = [2] \times [n]$. This implies that there exist no non-trivial blocks besides those of the form $\{1, 2\} \times \{j\}$ for some $j \in [n]$.

Next, we express primitivity with the notion of A-primitivity.

Proposition 14.26 *Let $G \leq_G \mathcal{S}_n$. Then the following are equivalent.*

1. *G is primitive.*
2. *G is A-primitive for all subsets $A \subseteq [n]$.*
3. *G is $\{a, b\}$-primitive for all distinct $a, b \in [n]$.*
4. *G is $\{a\}$-primitive for all $a \in [n]$.*
5. *G is $\emptyset$-primitive.*

Proof. We can assume $n \geq 2$.

$1 \Rightarrow 2$: By primitivity, G only admits the trivial blocks, giving the implication.

$2 \Rightarrow 3$: This is obvious.

$3 \Rightarrow 4$: Suppose $a \in [n]$ and that $\Delta \subseteq [n]$ is a block with $|\Delta| > 1$ and $a \in \Delta$. Pick $b \in \Delta \setminus \{a\}$. As, by assumption, G is $\{a, b\}$-primitive, we find $\Delta = [n]$.

$4 \Rightarrow 5$: Let $\Delta \subseteq [n]$ be a block. Pick some $a \in \Delta$. By assumption G is $\{a\}$-primitive so that $|\Delta| = 1$ or $\Delta = [n]$. This yields that G is $\emptyset$-primitive as Δ contains the empty set.

$5 \Rightarrow 1$: Let $\Delta \subseteq [n]$ be a block. As $\emptyset \subseteq \Delta$, we have $|\Delta| = 1$ or $\Delta = [n]$. So G has no non-trivial blocks and is primitive. □

Primitivity, A-primitivity for all 2-sets A, and A-primitivity for all singletons A are equivalent properties by Theorem 14.26. This equivalence cannot be extended to k-sets

[9] The construction is actually a special case of a variant of the so-called *wreath product construction* with the *imprimitive action*. We refer to [126, 179] for details.

for some $k \geq 3$ as shown by the imprimitive groups from Theorem 14.25, which are A-primitive for each k-set A with $k \geq 3$. Further examples are given by the Klein four-group from Example 5.3 or the permutation group $\langle\{(0\ 1\ 2\ 3)\}\rangle_G$, which are both imprimitive and, as they are transitive, A-primitive for each 3-set by Theorem 14.23.

Next, we characterize groups that are $\{a\}$-primitive for some $a \in [n]$.

Proposition 14.27 *Let $G \leq_G \mathcal{S}_n$ and $a \in [n]$. Then G is $\{a\}$-primitive if and only if G is primitive, or a is a fixpoint of G, i.e., $G = G_a$, and $\mathrm{Orb}_G(b) = [n] \setminus \{a\}$ for some $b \in [n] \setminus \{a\}$.*

Proof. We can assume $n \geq 2$. Suppose G is $\{a\}$-primitive. First, assume a is not a fixpoint of G. Then we have $|\mathrm{Orb}_G(a)| > 1$. As $\mathrm{Orb}_G(a)$ is a block, this implies $\mathrm{Orb}_G(a) = [n]$ and so G is transitive. Now, let $\Delta \subseteq [n]$ be a block with $|\Delta| > 1$. There exists $g \in G$ such that $a \in g(\Delta)$. As $g(\Delta)$ is also a block, we get $g(\Delta) = [n]$ and so $\Delta = [n]$. Hence G is primitive. Next, assume a is a fixpoint of G. Consider $\Delta = \{a\} \cup \mathrm{Orb}_G(b)$ for some $b \in [n] \setminus \{a\}$. Then Δ is a block containing a with $|\Delta| \geq 2$. So $\Delta = [n]$. As $\mathrm{Orb}_G(a) = \{a\}$ we get $\mathrm{Orb}_G(b) = [n] \setminus \{a\}$.

Conversely, if G is primitive, then G is $\{a\}$-primitive by Theorem 14.26. Next, suppose a is a fixpoint of G and $\mathrm{Orb}_G(b) = [n] \setminus \{a\}$ for some $b \in [n] \setminus \{a\}$. Let $\Delta \subseteq [n]$ be a block with $|\Delta| \geq 2$ and $a \in \Delta$. Then there exists $c \in \mathrm{Orb}_G(b) \cap \Delta$. Let $g \in G$. As a is a fixpoint, we have $a \in g(\Delta)$, which gives $g(\Delta) = \Delta$ as Δ is a block. Further, $g(c) \in g(\Delta) = \Delta$. As g was chosen arbitrarily, we find $\mathrm{Orb}_G(c) \subseteq \Delta$. Hence, $[n] \setminus \{a\} = \mathrm{Orb}_G(b) = \mathrm{Orb}_G(c) \subseteq \Delta$ and so $\Delta = [n]$. □

The group $\langle(1\ 2\ 3)(4)\rangle_G$ is $\{4\}$-primitive and intransitive. Theorems 5.1 and 14.27 imply, as stated next, that transitivity and $\{a\}$-primitivity for a single point a are equivalent to primitivity for $n \geq 3$. This was noted as a lemma (without the notion of A-primitivity) in [338, p. 735].

Corollary 14.28 *Let $G \leq \mathcal{S}_n$ with $n \geq 3$. Then G is primitive if and only if it is transitive and $\{a\}$-primitive for some $a \in [n]$.*

Proof. If G is primitive and $n \geq 3$, then it follows that G is transitive (Theorem 5.1) and $\{a\}$-primitive for all $a \in [n]$ (Theorem 14.26).

Conversely, suppose G is transitive and $\{a\}$-primitive. Then by transitivity a is not a fixpoint. So, by Theorem 14.27, G is primitive. □

Corollary 14.29 *Let $G \leq \mathcal{S}_n$ with $n \geq 2$. Then G is primitive if and only if there exist two distinct $a, b \in [n]$ such that G is $\{a\}$-primitive and $\{b\}$-primitive.*

Proof. We can assume $n \geq 3$. First, suppose G is $\{a\}$- and $\{b\}$-primitive for distinct $a, b \in [n]$. By Theorem 14.27, if G were imprimitive, then a and b would be fixed points and $[n] = \{a, b\}$. So, as $n \geq 3$, G must be primitive. Conversely, if G is primitive, then G is $\{a\}$- and $\{b\}$-primitive for all distinct $a, b \in [n]$ by Theorem 14.26. □

Remark 14.30 Let $G \leq_G \mathcal{S}_n$ be A-primitive and transitive for some $A \subseteq [n]$. For $a \in A$, by Theorem 5.15, either G_a is maximal or, if there exists a proper subgroup $H \leq_G G$ that properly contains G_a, then A is not a subset of $\mathrm{Orb}_H(a)$ and so $b \notin \mathrm{Orb}_H(a)$ for some $b \in A$. Furthemore, if G is $\{a, b\}$-primitive for $a \neq b$, then $H = \langle G_a \cup G_b \rangle_G$ equals the trivial group or it equals G, or $\mathrm{Orb}_H(a)$ and $\mathrm{Orb}_H(b)$ are disjoint H-orbits.

In an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ whose transformation monoid contains an A-primitive permutation group for $A \subseteq Q$, by Theorem 6.56, for every morphism φ the map $\varphi_{|A}$ is injective.

Lastly, we characterize the $\{a, b\}$-primitive cyclic permutation groups.

Proposition 14.31 *Let $g \in \mathcal{S}_n$ with distinct $a, b \in [n]$. Then $G = \langle\{g\}\rangle_G$ is $\{a, b\}$-primitive if and only if (1) g induces two cycles of coprime length, one cycle moving a and the other b, (2) or g cyclically permutes $[n]$ and $g^i(a) = b$ for some $i \geq 0$ coprime to n.*[10]

Proof. We can assume $n \geq 3$. First, suppose G is $\{a, b\}$-primitive. By Theorem 14.32, we have $[n] = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$. Now, assume G is intransitive. Then we can write

$$g = (a_0\ a_1\ \cdots\ a_{r-1})(b_0\ b_1\ \cdots\ b_{s-1})$$

for $\mathrm{Orb}_G(a) = \{a_0, a_1, \ldots, a_r\}$ and $\mathrm{Orb}_G(b) = \{b_0, b_1, \ldots, b_s\}$ with $n = r + s$ where $a = a_0$ and $b = b_0$. Let k be a common divisor of r and s. Then $\Delta = \{a_i \mid i \equiv 0 \pmod{k}, i \in \{0, 1, \ldots, r-1\}\} \cup \{b_i \mid i \equiv 0 \pmod{k}, i \in \{0, 1, \ldots, s-1\}\}$ is a block with $a, b \in \Delta$. So, $\Delta = [n]$. As $r \geq 2$ or $s \geq 2$, we obtain $1 \equiv 0 \pmod{k}$, which gives $k = 1$, i.e., the cycle lengths are coprime. Next, assume G is transitive. Then g must cyclically permute $[n]$. Without loss of generality, assume $g = (0\ 1\ \cdots\ n-1)$ and $a < b$. By Theorem 5.5 there exists a block containing a and b if and only if $a \equiv b \pmod{r}$ for some divisor r of n. This is equivalent to $b - a \equiv 0 \pmod{r}$, or that r is a common divisor of $b - a$ and n. So, as there does not exist a non-trivial block containing a and b, $b - a$ and n must be coprime.

Conversely, assume $g = (a_0\ a_1\ \cdots\ a_{r-1})(b_0\ b_1\ \cdots\ b_{s-1})$ for r and s being coprime and $a = a_0, b = b_0$. Then $kr \equiv 1 \pmod{s}$ for some $k \geq 1$. For a block Δ with $a, b \in \Delta$, we then obtain $g^{kr}(b_i) \equiv b_i + 1 \pmod{s}$ for $i \in \{0, 1, \ldots, s-1\}$. As $b \in \Delta$, inductively, we obtain $\{b_0, b_1, \ldots, b_{s-1}\} \subseteq \Delta$. Similarly, we find $\{a_0, a_1, \ldots, a_{r-1}\} \subseteq \Delta$. So, $\Delta = [n]$. Hence, G is $\{a, b\}$-primitive. Next, if g cyclically permutes the states and $g^i(a) = b$ for some i coprime to n, by Theorem 5.5 we obtain, as before, that G is $\{a, b\}$-primitive. □

14.3.2 Transitivity and Strong $\{a, b\}$-Primitivity

Next, we investigate how A-primitivity relates to transitivity and, more generally, possible decompositions of the permutation domain into orbits. Lastly, we introduce strongly $\{a, b\}$-primitive permutation groups.

Proposition 14.32 *Let $G \leq_G \mathcal{S}_n$ and $A \subseteq [n]$ with $|A| \geq 2$. If G is A-primitive, then $[n] = \bigcup_{a \in A} \mathrm{Orb}_G(a)$.*

Proof. The set $\bigcup_{a \in A} \mathrm{Orb}_G(a)$ forms a block containing A. Hence it equals $[n]$. □

[10] When assuming $a < b$, coprimality of i and n is equivalent to the fact that $b - a$ is coprime to n.

The next example illustrates Theorem 14.32 with an intransitive group that is A-primitive for a 2-set A.

Example 14.33 Let $G = \langle\{g, h\}\rangle_G$ where $g = (0\ 1\ 2\ 3)$ and $h = (4\ 5\ 6\ 7)$. This group is intransitive. Suppose $\Delta \subseteq [n]$ is a block with $\{0, 4\} \subseteq \Delta$. As 4 is fixed by g, we have $4 \in g(\Delta)$ and so we get $g(\Delta) = \Delta$. Furthermore, we have $i \in g^i(\Delta) = \Delta$ for each $i \in \{1, 2, 3\}$ so that $\{0, 1, 2, 3\} \subseteq \Delta$. Similarly, we find $\{4, 5, 6, 7\} \subseteq \Delta$. Hence $\Delta = [n]$ and G is $\{0, 4\}$-primitive.

The condition $[n] = \bigcup_{a\in A} \mathrm{Orb}_G(a)$ does not imply A-primitivity. For example, if $G \leq_G \mathcal{S}_n$ is transitive and imprimitive, then for each $a \in [n]$ we have $[n] = \mathrm{Orb}_G(a)$, but, by Theorem 14.28, it is not $\{a\}$-primitive. The next example gives an intransitive group showing that the converse of Theorem 14.32 does not hold in general.

Example 14.34 Let $G = \langle\{(0\ 1\ 2\ 3)(4\ 5\ 6\ 7)\}\rangle_G$. Then $[8] = \mathrm{Orb}_G(0) \cup \mathrm{Orb}_G(4)$, $\mathrm{Orb}_G(0) \neq \mathrm{Orb}_G(4)$ and $\{0, 4\}$ is a non-trivial block so that G is not $\{0, 4\}$-primitive.

Next, we investigate sufficient conditions for transitivity in terms of A-primitivity.

Proposition 14.35 *Let $G \leq \mathcal{S}_n$ with $n \geq 3$ and $k \geq 2$. Suppose G is B-primitive for each k-subset $B \subseteq A$ for some $A \subseteq [n]$ with $|A| \geq k(k-1)+1$. Then G is transitive.*

Proof. Let $B \subseteq A$ be some k-subset. As G is B-primitive, by Theorem 14.32, we have $[n] = \bigcup_{b\in B} \mathrm{Orb}_G(b)$. By the pigeonhole principle, considering the points from A distributed among the orbits $\mathrm{Orb}_G(b)$ for $b \in B$, there exists $b \in B$ such that $|\mathrm{Orb}_G(b) \cap A| \geq k$. Thus we find a k-set C in $\mathrm{Orb}_G(b) \cap A$. By assumption, G is C-primitive. Since the orbit $\mathrm{Orb}_G(b)$ is a block containing C, we get $\mathrm{Orb}_G(b) = [n]$. □

Corollary 14.36 *Let $a, b, c \in [n]$ be distinct and $G \leq_G \mathcal{S}_n$ be $\{a, b\}$-, $\{a, c\}$- and $\{b, c\}$-primitive. Then G is transitive.*

Proof. This follows from Theorem 14.35 with $A = \{a, b, c\}$ and $k = 2$.[11] □

The next statements connect A-primitivity to transitivity.

Proposition 14.37 *Let $G \leq_G \mathcal{S}_n$ and $A \subseteq [n]$ with $|A| \geq 2$. Suppose G is A-primitive. Then G is transitive if and only if $A \subseteq \mathrm{Orb}_G(b)$ for some $b \in [n]$.*

Proof. First, suppose G is transitive. Pick some $b \in [n]$. Then $\mathrm{Orb}_G(b) = [n]$ and so $A \subseteq \mathrm{Orb}_G(b)$.

Next, suppose $A \subseteq \mathrm{Orb}_G(b)$. Since G is A-primitive, by Theorem 14.32, we have $[n] = \bigcup_{a\in A} \mathrm{Orb}_G(a)$. Now, $a \in \mathrm{Orb}_G(b)$ implies $\mathrm{Orb}_G(a) = \mathrm{Orb}_G(b)$. Thus, we obtain $[n] = \mathrm{Orb}_G(b)$ and so G is transitive. □

Corollary 14.38 *Let $G \leq_G \mathcal{S}_n$ be $\{a, b\}$-primitive for distinct $a, b \in [n]$. Then there exists $g \in G$ such that $g(a) = b$ if and only if G is transitive.*

[11] Alternatively, from Theorem 14.32 we get $[n] = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b) = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(c) = \mathrm{Orb}_G(b) \cup \mathrm{Orb}_G(c)$. Thus, we have $a \in \mathrm{Orb}_G(b)$ or $a \in \mathrm{Orb}_G(c)$, and each case yields $[n] = \mathrm{Orb}_G(a)$.

Proof. This follows from Theorem 14.38 since $\{a, b\} \subseteq \mathrm{Orb}_G(c)$ for some $c \in [n]$ is equivalent to the existence of some $g \in G$ with $g(a) = b$. □

For a 2-set $\{a, b\}$, we call $G \leq_G \mathcal{S}_n$ *strongly* $\{a, b\}$*-primitive* if G is $\{a, b\}$-primitive and there exists $g \in G$ such that $g(a) = b$ (or equivalently G is $\{a, b\}$-primitive and transitive). As $g^{-1}(b) = a$, G is strongly $\{a, b\}$-primitive if and only if it is strongly $\{b, a\}$-primitive, so the set notation "$\{a, b\}$" is justified here.

Example 14.39 The permutation group from Example 14.21 is strongly $\{a, b\}$-primitive for all distinct $a, b \in [8]$ with $a \not\equiv b \pmod 2$. The permutation group from Example 14.25 is strongly $\{(i, j), (i, j')\}$-primitive for $(i, j), (i', j') \in [2] \times [n]$ with $j \neq j'$.

As a primitive permutation group over a permutation domain with at least three points is transitive (Proposition 5.1), by Proposition 14.26 we get the following statement.

Proposition 14.40 *Let $G \leq \mathcal{S}_n$ with $n \geq 3$. Then G is primitive if and only if G is strongly $\{a, b\}$-primitive for all distinct $a, b \in [n]$.*

14.3.3 Distinguishability, Orbit Sizes, and $\{a, b\}$-Primitivity

Here, we collect several mostly technical statements relating $\{a, b\}$-primitivity to the distinguishability of pairs of subsets and to synchronizability. These statements will be used throughout the following sections.

Theorem 14.42 gives sufficient conditions for $\{a, b\}$-primitivity. Theorem 14.46 and Theorem 14.48 give bounds on orbit sizes under distinguishability and synchronizability assumptions. Finally, Theorem 14.50 combines these results to derive a sufficient condition for strong $\{a, b\}$-primitivity.

Lemma 14.41 *Let $G \leq \mathcal{S}_n$, let $\sim$ be a G-congruence, and let $t \colon [n] \to [n]$ be such that, for all $a, b \in [n]$,*

$$t(a) \sim t(b) \iff a \sim b.$$

Then, for every $h \in \langle G \cup \{t\}\rangle_S$ and all $a, b \in [n]$,

$$h(a) \sim h(b) \iff a \sim b.$$

Proof. The claim follows by induction on the length of a word representing h in terms of t and generators of G. □

Corollary 14.42 *Let $G \leq \mathcal{S}_n$ and $A \subseteq [n]$. Then G is A-primitive if one of the following cases applies:*

1. *G synchronizes some idempotent $t \colon [n] \to [n]$ with $t^{-1}(\mathrm{dupl}(t)) \subseteq A$.*
2. *There exists an idempotent $t \colon [n] \to [n]$ with $t^{-1}(\mathrm{dupl}(t)) \subseteq A$ and two distinguishable sets $B, C \subseteq [n]$ each containing a point from A and from $[n] \setminus A$.*

Proof. Suppose G is not A-primitive and let $\sim$ be a non-trivial G-congruence such that all points in A are congruent. Let $a \in A$. Pick some point $c \in [n] \setminus A$ such that $c \nsim a$.

Let $t\colon [n] \to [n]$ be some idempotent with $t^{-1}(\mathrm{dupl}(t)) \subseteq A$. Then $t(x) = x$ for all $x \in [n] \setminus A$ and $t(A) \subseteq A$ since $t^{-1}(\mathrm{dupl}(t)) = t(\mathrm{move}(t)) \cup \mathrm{move}(t)$ by Theorem 4.25. This yields

$$t(d) \sim t(e) \Leftrightarrow d \sim e$$

for $d, e \in [n]$. By Theorem 14.41 we obtain that $h(a) \nsim h(c)$ for all $h \in \langle G \cup \{t\}\rangle_S$. In particular, we have $h(a) \neq h(c)$. Thus $\langle G \cup \{t\}\rangle_S$ does not contain a constant map. Furthermore, we can conclude that for $B \subseteq [n]$ with $A \cap B \neq \emptyset$ and $B \cap ([n] \setminus A) \neq \emptyset$ we have $|h(B)| \geq 2$ for all $h \in \langle G \cup \{t\}\rangle_S$. This yields that two such sets are indistinguishable in $\langle G \cup \{t\}\rangle_S$. □

Example 14.43 Let $G = \langle (0)(1\,2\,\ldots\,n{-}1)\rangle_G$. Then $\langle G \cup \{t\}\rangle_S$ contains a constant map for the simple idempotent $t\colon [n] \to [n]$ that maps 1 to 0. So, by Theorem 14.42, G is $\{0,1\}$-primitive. Observe that in $\langle G \cup \{t\}\rangle_S$ all pairs of distinct 2-sets are distinguishable, but $\{0,1,2\}$ is not distinguishable from $\{1,2\}$.

Example 14.44 The group $G = \langle (0\ 1)(2\ 3\ 4)\rangle_G \leq_G \mathcal{S}_5$ is $\{0,2\}$-primitive. This group synchronizes neither of the two simple idempotents $t\colon [5] \to [5]$ with $t(0) = t(2)$. So, the converse statement to Theorem 14.42 is not true. Theorem 14.33 gives another example where the given permutation group is $\{0,4\}$-primitive but does not synchronize a simple idempotent mapping 0 and 4 to the same state.

Next, we investigate orbits under distinguishability and synchronizability assumptions.

Lemma 14.45 *Let $G \leq_G \mathcal{S}_n$ and $t\colon [n] \to [n]$. Suppose $A \subseteq [n]$ is a union of orbits of G such that $t(A) = A$. Then for each $h \in \langle G \cup \{t\}\rangle_S$ we have $h(A) = A$.*

Proof. As A is a union of orbits, we have $g(A) = A$ for $g \in G$. So, with $t(A) = A$, the claim follows by induction on the length of a word representing h in terms of t and generators of G. □

Lemma 14.46 *Let $G \leq_G \mathcal{S}_n$ and $t\colon [n] \to [n]$. Suppose $A \subseteq [n]$ is a non-empty union of orbits of G such that $t(A) = A$. If all pairs of distinct 2-sets are distinguishable in $\langle G \cup \{t\}\rangle_S$ (resp. $\langle G \cup \{t\}\rangle_S$ contains a constant map), then $|A| \leq 2$ (resp. $|A| = 1$) and $A \cap \mathrm{dupl}(t) \neq \emptyset$.*

Proof. We can assume $n \geq 3$. By Theorem 14.45, pairs of distinct 2-sets from A are not distinguishable (resp. no two distinct points from A can be mapped to a common point) in $\langle G \cup \{t\}\rangle_S$, which gives $|A| \leq 2$ (resp. $|A| = 1$). Next, let $a \in A$ and $b, c \in [n] \setminus \{a\}$ be distinct. Pick $h \in \langle G \cup \{t\}$ distinguishing $\{a,b\}$ and $\{a,c\}$ (resp. mapping $\{a,b\}$ to a singleton) such that a product in the generators $G \cup \{t\}$ has shortest possible length. Without loss of generality, assume $|h(\{a,b\})| = 1$. By the minimal choice, we have $h = tg_mt\cdots tg_1tg_0$ for some $g_0, g_1, \ldots, g_m \in G$ with $|(g_mt\cdots tg_1tg_0)(\{a,b\})| = 2$. So, $h(a) \in \mathrm{dupl}(t)$. As $t(A) = A$ and A is a union of G-orbits, so that $g(A) = A$ for $g \in G$, from $a \in A$ we get $h(a) \in A$. Hence, $A \cap \mathrm{dupl}(t) \neq \emptyset$. □

Example 14.47 Consider G from Theorem 14.44 with a simple idempotent t mapping 0 to 2 and $A = \mathrm{Orb}_G(2) = \{2, 3, 4\}$. Then $t(A) = A$. By Theorem 14.46, G does not synchronize t and there exist pairs of distinct indistinguishable 2-sets (for example, $\{2, 3\}$ and $\{2, 4\}$).

Lemma 14.48 *Let $G \leq_G \mathcal{S}_n$ be intransitive and $t\colon [n] \to [n]$ be a simple idempotent mapping a to b with $a \neq b$ and $a, b \in [n]$. Suppose in $\langle G \cup \{t\}\rangle_S$ all pairs of distinct 2-sets are distinguishable (resp. $\langle G \cup \{t\}\rangle_S$ contains a constant map). Then we have $|\mathrm{Orb}_G(b)| \leq 2$ (resp. $|\mathrm{Orb}_G(b)| = 1$).*

Proof. By Theorem 14.42, G is $\{a, b\}$-primitive. So, by Theorem 14.32, we have $[n] = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$. Since G is intransitive, both orbits are disjoint. Thus, since t only moves a, we have $t(\mathrm{Orb}_G(b)) = \mathrm{Orb}_G(b)$. By Theorem 14.46, we obtain $|\mathrm{Orb}_G(b)| \leq 2$ (resp. $|\mathrm{Orb}_G(b)| = 1$). □

Next, illustrating Theorem 14.48, we give an example of an intransitive group in which all pairs of distinct 2-sets are distinguishable when adjoining a simple idempotent mapping a to b and where $|\mathrm{Orb}_G(b)| = 2$.

Example 14.49 Let $n \geq 3$ and U be the intersection of the points stabilizers of $n - 2$ and $n - 1$ in $\mathcal{S}_n$ (U is isomorphic to $\mathcal{S}_{n-2}$). Set $G = \langle U \cup \{(n-2\ n-1)\}\rangle_G$ and let $t\colon [n] \to [n]$ be the simple idempotent that maps $n - 3$ to $n - 2$. Then in $\langle G \cup \{t\}\rangle_S$ all distinct pairs of 2-sets are distinguishable and $\mathrm{Orb}_G(n-1) = \{n-1, n-2\}$.

Proposition 14.50 *Let $G \leq \mathcal{S}_n$, and let $a, b \in [n]$ be distinct, with $n \geq 5$ (resp. $n \geq 3$). Suppose that for both simple idempotents $t\colon [n] \to [n]$ satisfying $t(a) = t(b)$, all pairs of distinct 2-sets are distinguishable in $\langle G \cup t\rangle_S$ (resp. G synchronizes t). Then G is strongly $\{a, b\}$-primitive.*

Proof. By Theorem 14.48, if G is intransitive, we get and $|\mathrm{Orb}_G(a)| \leq 2, |\mathrm{Orb}_G(b)| \leq 2$ (resp. $|\mathrm{Orb}_G(a)| = 1$, $|\mathrm{Orb}_G(b)| = 1$). With Theorem 14.32 we have $[n] = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$, which gives $n \leq 4$ (resp. $n = 2$). Thus, for $n \geq 5$ (resp. $n \geq 3$), G must be transitive. By Theorem 14.42, G is $\{a, b\}$-primitive. □

Theorem 14.50 fails for $n \in \{3, 4\}$ as shown by the next example.

Example 14.51 Let $G_1 = \langle\{(0\ 1)(2)\}\rangle_G \leq_G \mathcal{S}_3$, $G_2 = \langle\{(0\ 1), (2\ 3)\}\rangle_G \leq_G \mathcal{S}_4$ and $t_1\colon [3] \to [3]$, $t_2\colon [4] \to [4]$ be simple idempotents with $t_1(1) = t_1(2)$ and $t_2(1) = t_2(2)$. Then in $\langle G_1 \cup \{t_1\}\rangle_S$ and $\langle G_2 \cup \{t_2\}\rangle_S$ all pairs of distinct 2-sets of elements are distinguishable.

Lemma 14.52 *Let $G \leq \mathcal{S}_n$ and let $t\colon [n] \to [n]$ be an idempotent one-point singular with $\mathrm{dupl}(t) = \{a_m\}$ and $t^{-1}(a_m) = \{a_1, \ldots, a_m\}$, where $m \geq 2$. Then G synchronizes t if and only if there exists $h \in \langle G \cup \{t\}\rangle_S$ such that*

$$|h(\mathrm{Orb}_G(a_m))| = 1$$

and $[n] = \mathrm{Orb}_G(a_1) \cup \cdots \cup \mathrm{Orb}_G(a_m)$.

Proof. Set $B = \mathrm{Orb}_G(a_1) \cup \cdots \cup \mathrm{Orb}_G(a_m)$. For every $g \in G$ we have

$$g(\mathrm{Orb}_G(a_m)) = \mathrm{Orb}_G(a_m). \tag{14.1}$$

Next, we also have $t(\mathrm{Orb}_G(a_m)) \subseteq \mathrm{Orb}_G(a_m)$. Together with Equation (14.1), this implies that

$$h(\mathrm{Orb}_G(a_m)) \subseteq \mathrm{Orb}_G(a_m) \tag{14.2}$$

for every $h \in \langle G \cup \{t\}\rangle_S$.

Now, first, suppose G synchronizes t. Then $\langle G \cup \{t\}\rangle_S$ contains a constant map h. Hence $|h(\mathrm{Orb}_G(a_m))| = 1$. Next, assume, for contradiction, that there exists $b \in [n] \setminus B$. We have $t(b) = b$. Moreover, by Equation (14.2), every element of $\langle G \cup \{t\}\rangle_S$ maps $\mathrm{Orb}_G(a_m)$ into itself. Hence $|h(\{b, a_m\})| = 2$ for every $h \in \langle G \cup \{t\}\rangle_S$, contradicting the assumption that G synchronizes t. Therefore, $[n] = B$.

Conversely, suppose there exists $h \in \langle G \cup \{t\}\rangle_S$ such that $|h(\mathrm{Orb}_G(a_m))| = 1$ and $[n] = B$. Let $i \in \{1, \ldots, m-1\}$. Consider some $b \in \mathrm{Orb}_G(a_i)$. There exists $g \in G$ such that $g(b) = a_i$. Then $t(g(b)) = a_m$. With Equation (14.2) we can successively select such an element $g \in G$ to map a point from $\mathrm{Orb}_G(a_i)$ into $\mathrm{Orb}_G(a_m)$ until there are no more points left in $\mathrm{Orb}_G(a_i)$. By this procedure, we can construct $h_i \in \langle G \cup \{t\}\rangle_S$ such that $h_i(\mathrm{Orb}_G(a_i)) \subseteq \mathrm{Orb}_G(a_m)$. By Equation (14.2), $h_{m-1} \cdots h_1$ maps all points into $\mathrm{Orb}_G(a_m)$. Then $h h_{m-1} \cdots h_1$ maps all points to a single point. □

14.3.4 Orbital Graphs and $\{a, b\}$-Primitivity

Here, we relate the notion of $\{a, b\}$-primitivity to undirected and directed orbital graphs, thereby generalizing Higman's characterization of primitivity (Theorem 14.3, Item 3). More specifically, in Theorem 14.53 (resp. Theorem 14.54), we prove that $\{a, b\}$-primitivity (resp. strong $\{a, b\}$-primitivity) is equivalent to connectedness of the undirected orbital graph of $\{a, b\}$ (resp. strong connectedness of the directed orbital graph of (a, b)). Theorem 14.55 characterizes strong $\{a, b\}$-connectivity in terms of partitions of the domain into two sets. Lastly, in Theorem 14.56, we show that a group is $\{a, b\}$-primitive, but not strongly $\{a, b\}$-primitive, if and only if the orbits of a and b are disjoint and partition the domain, in the undirected orbital graph of $\{a, b\}$ no edge is completely contained in one orbit, and in the directed orbital graph of (a, b) all edges start in the orbit of a and end in the orbit of b.

Proposition 14.53 *Let $G \leq_G \mathcal{S}_n$ and $a, b \in [n]$ be distinct. Then G is $\{a, b\}$-primitive if and only if the undirected orbital graph of $\{a, b\}$ is connected.*

Proof. Let H be the undirected orbital graph of $\{a, b\}$. First, assume that G is $\{a, b\}$-primitive. For $c, d \in [n]$, write $c \sim d$ if there exists a path from c to d in H. Since G is a subgroup of the automorphism group of H, the relation $\sim$ is a G-congruence. So, the equivalence classes, i.e., the connected components in this case, are blocks. As $\{a, b\}$ is an edge in H we have $a \sim b$. Therefore, by $\{a, b\}$-primitivity, all pairs of distinct points are congruent, i.e., the connected component containing a and b equals $[n]$.

Thus, H is connected. Conversely, assume H is connected. Let $\Delta \subseteq [n]$ be a block with $\{a, b\} \subseteq \Delta$. No edge can leave Δ, i.e., no edge can have precisely one endpoint in Δ; for if $\{g(a), g(b)\}$ with $g \in G$ is an edge such that, without loss of generality, $g(a) \in \Delta$, then, as Δ is a block containing a and b, we have $g(b) \in \Delta$. Hence Δ is a union of connected components of H. As H is connected, this implies $\Delta = [n]$. □

Proposition 14.54 *Let $G \leq_G \mathcal{S}_n$ and $a, b \in [n]$ be distinct. Then G is strongly $\{a, b\}$-primitive if and only if the directed orbital graph of (a, b) is strongly connected.*

Proof. Let $H = ([n], E)$ be the directed orbital graph of (a, b). First, suppose G is strongly $\{a, b\}$-primitive. As G is $\{a, b\}$-primitive the graph H is weakly connected by Theorem 14.53. Further, as G is transitive, H is vertex-transitive. By Theorem 5.43, it follows that H is strongly connected. Conversely, suppose H is strongly connected. Then, as the undirected orbital graph of $\{a, b\}$ is connected, G is $\{a, b\}$-primitive by Theorem 14.53. Further, every vertex of H must have an outgoing edge. Since we have $E \subseteq \mathrm{Orb}_G(a) \times \mathrm{Orb}_G(b)$, an edge outgoing at b has the form $(g(a), g(b))$, which gives $b = g(a)$.[12] So, G is strongly $\{a, b\}$-primitive. □

Combining Theorem 14.54 with Theorem 3.67 gives the next corollary.[13]

Corollary 14.55 *A group $G \leq_G \mathcal{S}_n$ is strongly $\{a, b\}$-primitive for distinct $a, b \in [n]$ if and only if for non-empty and disjoint $A, B \subseteq [n]$ with $A \cup B = [n]$ we have $g(a) \in A$ and $g(b) \in B$ for some $g \in G$.*

Finally, we investigate the orbital graphs of groups that are $\{a, b\}$-primitive but not strongly $\{a, b\}$-primitive.

Proposition 14.56 *Let $G \leq_G \mathcal{S}_n$ be $\{a, b\}$-primitive for distinct $a, b \in [n]$. Then G is intransitive if and only if in the undirected orbital graph of $\{a, b\}$ (resp. directed orbital graph of (a, b)) the orbits $\mathrm{Orb}_G(a)$ and $\mathrm{Orb}_G(b)$ are disjoint and partition $[n]$ such that there exists no edge with both endpoints in $\mathrm{Orb}_G(a)$ or both endpoints in $\mathrm{Orb}_G(b)$ (resp. the starting vertex of each edge is in $\mathrm{Orb}_G(a)$ and its ending vertex is in $\mathrm{Orb}_G(b)$).*[14]

Proof. By $\{a, b\}$-primitivity, from Theorem 14.32 we get $[n] = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$. So, intransitivity is equivalent with $\mathrm{Orb}_G(a) \neq \mathrm{Orb}_G(b)$ and so the claim follows from Theorem 5.48. □

---

[12] Alternatively, we obtain transitivity of G from Theorem 3.67 applied to $\mathrm{Orb}_G(a)$, which yields $g(b) \in \mathrm{Orb}_G(a)$ for some $g \in G$.

[13] A different proof was given in [309, Prop. 2.2] by considering, for a subset A of the domain, the G-congruence that relates x and y if and only if

$$\forall g \in G\colon g(x) \in A \Leftrightarrow g(y) \in A.$$

[14] Undirected graphs with these properties are known as bipartite graphs w.r.t. bipartition $\mathrm{Orb}_G(a)$ and $\mathrm{Orb}_G(b)$.

Example 14.57 The intransitive group from Theorem 14.33 is $\{0,4\}$-primitive. Its undirected orbital graph for $\{0,4\}$ has the edge set $\{\{a,b\} \mid a \in \{0,1,2,3\}, b \in \{4,5,6,7\}\}$.

Example 14.58 Let $G = \langle g \rangle_G$ with $g = (0\ 1\ 2)(0'\ 1'\ 2'\ 3')$. This group is $\{0,1'\}$-primitive. For if Δ is a block with $0, 1' \in \Delta$, then since $0, 0' \in g^3(\Delta)$, $0, 3' \in g^6(\Delta)$, and $0, 2' \in g^9(\Delta)$ we find $\{0', 1', 2', 3'\} \subseteq \Delta$. We have $g^i(\Delta) \cap \{0', 1', 2', 4'\} \neq \emptyset$ for all $i \geq 0$, and thus $\Delta = \{0, 1, 2, 0', 1', 2', 3'\}$. Now, the edges of the undirected orbital graph of $\{0, 1'\}$ are

$$\{\{0,1'\},\{1,2'\},\{2,3'\},\{0,0'\},\{1,1'\},\{2,2'\},\\ \{0,3'\},\{1,0'\},\{2,1'\},\{0,2'\},\{1,3'\},\{2,0'\}\}.$$

Example 14.59 Consider the $\{0,2\}$-primitive group G from Theorem 14.44. Its undirected orbital graph for $\{0,2\}$ has the edge set $\{\{a,b\} \mid a \in \{0,1\}, b \in \{2,3,4\}\}$.

14.3.5 Generating Sets for the Symmetric Group

Here, in Theorem 14.61, we characterize $\{a,b\}$-primitivity in terms of generating sets for the full symmetric group, thereby generalizing Jordan's theorem (Theorem 14.4). As a consequence, in Theorem 14.62, we characterize those pairs consisting of a permutation and a transposition that generate the full symmetric group. Clearly, adding permutations preserves A-primitivity. Theorem 14.60 gives a condition for the preservation of A-primitivity when removing permutations.

Lemma 14.60 *Let $G \leq_G \mathcal{S}_n$ and $t \in \mathcal{S}_n$ be a permutation. If $\langle G \cup \{t\} \rangle_G$ is A-primitive, then G is also A-primitive for $A = \mathrm{move}(t)$.*

Proof. Suppose G is not A-primitive. Then there exists a non-trivial block $\Delta \subseteq [n]$ such that $A \subseteq \Delta$. Let $x \in A$. Then $t(x) \neq x$. Since t is a permutation, we find $t(t(x)) \neq t(x)$ and so $t(x) \in A$. Thus $t(A) \subseteq A$. We also have $t(t^{-1}(x)) = x \neq t^{-1}(x)$, giving $t^{-1}(x) \in A$ and then $x \in t(A)$. Thus $A \subseteq t(A)$. As all points outside of A are fixed by t, we find $t(\Delta) = \Delta$. This yields that Δ is also a block for $\langle G \cup \{t\} \rangle_G$ and so this group is not A-primitive. □

By Theorem 14.60, if g and h generate a primitive group, then $\langle g \rangle_G$ is $\mathrm{move}(h)$-primitive.

Theorem 14.61 *Let $a, b \in [n]$ be distinct. Then $G \leq_G \mathcal{S}_n$ is $\{a,b\}$-primitive if and only if $\langle G \cup \{(a\ b)\} \rangle_G = \mathcal{S}_n$.*

Proof. First, suppose $\langle G \cup \{(a\ b)\} \rangle_G = \mathcal{S}_n$. As the full symmetric group $\mathcal{S}_n$ is $\{a,b\}$-primitive, by Theorem 14.60 we find that G is $\{a,b\}$-primitive.

Conversely, suppose G is $\{a,b\}$-primitive. Set $T = \{(g(a)\ g(b)) \mid g \in G\}$. The graph with vertex set $[n]$ and edge set $\{\{c,d\} \mid (c\ d) \in T\}$ equals the undirected orbital graph on $\{a,b\}$. By Theorem 14.53 it is connected. So, by Theorem 5.41,

we get $\langle T\rangle_G = \mathcal{S}_n$. By Equation (5.2), we have $(g(a)\ g(b)) = (a\ b)^g$, which gives $T \subseteq \langle G \cup \{(a\ b)\}\rangle_G$. Hence $\mathcal{S}_n = \langle G \cup \{(a\ b)\}\rangle_G$. □

The fact that $(0\ 1)$ and $(0\ 1\ \ldots\ n-1)$ generate $\mathcal{S}_n$ is well-known, for example, see [126, 179]. As a consequence of Theorems 14.31 and 14.61, we obtain the following characterization of those permutations that, together with a transposition, generate the whole symmetric group.

Corollary 14.62 *Let $g \in \mathcal{S}_n$ and $a, b \in [n]$ be distinct. Then $\langle\{g, (a\ b)\}\rangle_G = \mathcal{S}_n$ if and only if g induces two cycles of coprime length, one cycle moving a and the other b, or g cyclically permutes its domain and $g^i(a) = b$ for some $i \geq 0$ coprime to n.*

14.3.6 Synchronization

Here, in Theorem 14.64 we characterize A-primitivity in transitive groups by synchronization of all one-point singulars. We then note a few consequences. At the end, we also introduce the more general class of unimodular transformations and characterize strongly $\{a, b\}$-primitiveness by synchronization of transformations from this class. This is in fact the most general class of transformations synchronized by strongly $\{a, b\}$-primitive groups, but we cannot generalize this to transitive A-primitive group for sets A with more than three points.

We use the following quasiorder in the present section: for a transformation monoid $T \leq_M \mathcal{T}_n$, write $a \rightsquigarrow_T b$ if $a = b$ or there exists an idempotent one-point singular $t \in T$ such that $t(a) = b$ (note that for $a \neq b$ this implies $\mathrm{dupl}(t) = \{b\}$). [15] If the transformation monoid T is clear from context, we shall simply write $\rightsquigarrow$ instead.

Lemma 14.63 *Let $T \leq_M \mathcal{T}_n$ be a transformation monoid with group of units G. Then $\rightsquigarrow$ is a quasiorder such that, for $a, b \in [n]$ and $g \in G$, $a \rightsquigarrow b$ implies $g(a) \rightsquigarrow g(b)$.*

Proof. By definition, $\rightsquigarrow$ is reflexive. Next, let $a, b, c \in [n]$ be distinct such that $a \rightsquigarrow b$ and $b \rightsquigarrow c$, witnessed by idempotent one-point singulars $h_1, h_2 \in T$ such that $h_1(a) = b$ and $h_2(b) = c$. The points b and c are the unique duplicate points of h_1 and h_2, respectively. Thus $\mathrm{dupl}(h_1) \subseteq h_1([n]) \cap h_2^{-1}(\mathrm{dupl}(h_2))$ and so, by Theorem 4.36 the

[15] Two distinct points $a, b \in [n]$ are also in relation for the following types of transformations $t\colon [n] \to [n]$:

1. if t is a one-point singular that permutes its image with $t(a) = t(b)$ and $t^p(b) = b$ for some $p > 0$, or
2. t is a one-point singular with $t^j(a) = b$ for some $j \geq 1$ and $t(b) = b$.

This follows since then we can find $i > 0$ such that t^i is an idempotent one-point singular with $t^i(a) = b$. By Theorem 3.20 we find $i > 0$ such that t^i is idempotent. Then, for (1), we have $\mathrm{dupl}(t_{|t([n])}) = \emptyset$ and, with Equation (3.10), this yields that t^i is a one-point singular with image $t([n])$. Then $t^i(a) = t^i(b)$ and since $b \in t([n])$, we have $t^i(b) = b$. For (2), we have $\mathrm{dupl}(t) = \{b\}$ and as in the proof of Theorem 5.12 we can conclude that t^i is an idempotent one-point singular. We then must have $t^i(a) = b$ since otherwise $i < j$ and $t^i(a)$ is not contained in a cycle, contradicting $t^i(t^i(a)) = t^i(a)$.

composition h_2h_1 is an idempotent one-point singular. We have $h_2(h_1(a)) = h_2(b) = c$. We also have $h_1(c) = c$ or $h_1(c) = b$ and both cases give $h_2(h_1(c)) = c$. Thus we have $(h_2h_1)(a) = c$ and so $a \rightsquigarrow c$. Next, let $g \in G$. By Equation (5.2) we have $h_1^g(g(a)) = g(b)$ and h_1^g is an idempotent one-point singular. So we find $g(a) \rightsquigarrow g(b)$. □

Theorem 14.64 *Suppose $G \leq_G \mathcal{S}_n$ is transitive and $A \subseteq [n]$ is a subset with $|A| \geq 2$. Then the following are equivalent.*

1. *G is A-primitive.*
2. *G synchronizes every one-point singular t on $[n]$ with $A \subseteq t^{-1}(\mathrm{dupl}(t))$.*
3. *G synchronizes some idempotent one-point singular t on $[n]$ with $t^{-1}(\mathrm{dupl}(t)) = A$.*

Proof. $1 \Rightarrow 2$: Since G is transitive, by Theorem 5.12, there exists an idempotent one-point singular $h \in \langle G \cup \{t\}\rangle_S$ with $A \subseteq h^{-1}(\mathrm{dupl}(h))$. Let $A = \{a_1, \ldots, a_m\}$ for $m \geq 2$ and $h(a_i) = a_m$ for $i \in \{1, \ldots, m\}$. Pick $i \in \{1, \ldots, m\}$. We have $a_i \rightsquigarrow a_m$ since $h(a_i) = a_m$. Denote by $\leftrightsquigarrow$ the equivalence relation induced by the quasiorder $\rightsquigarrow$, which is a G-congruence by Theorem 14.63. Since G is transitive, there exists $g \in G$ with $g(a_1) = a_m$. Pick some $k > 0$ such that $g^k(a_i) = a_i$. By successively applying g to $a_i \rightsquigarrow a_m$ we find $a_i \rightsquigarrow g(a_i), g(a_i) \rightsquigarrow g^2(a_i), \ldots, g^{k-1}(a_i) \rightsquigarrow a_i$. So $a_i \leftrightsquigarrow g(a_i)$ where $g(a_i) = a_m$. Thus, we have $a_i \leftrightsquigarrow a_m$ for all $i \in \{1, \ldots, m\}$. Since G is A-primitive, this implies that all pairs of states are equivalent. Hence, for all pairs $c, d \in [n]$ there exists an idempotent $f \in \langle G \cup \{t\}\rangle_S$ such that $f(c) = f(d) = d$. So, with the transformation semigroup version of the pair criterion (Section 6.8 and Theorem 7.18) it follows that $\langle G \cup \{t\}\rangle_S$ contains a constant map.

$2 \Rightarrow 3$: This is clear.

$3 \Rightarrow 1$: If G is not A-primitive, then, by Theorem 14.42, it synchronizes no idempotent $t\colon [n] \to [n]$ with $t^{-1}(\mathrm{dupl}(t)) \subseteq A$. This gives the claim. □

Remark 14.65 We note that instead of quasiorders in the proof of the implication "$1 \Rightarrow 2$" in Theorem 14.64, we can also construct a digraph on $[n]$ that has an edge (c, d) for each idempotent one-point singular mapping c to d. By Equation (5.2), G is a subgroup of the automorphism group of this graph. Furthermore, as G is transitive, this graph is strongly connected by Theorem 5.43, and paths correspond to idempotent one-point singulars mapping the starting vertex to the end vertex. Then, all pairs of points can be mapped to a common point by following a path, which yields claim.

Corollary 14.66 *Let $G \leq_G \mathcal{S}_n$, $n \geq 3$ and $a, b \in [n]$ be distinct. Then the following are equivalent.*

1. *G is strongly $\{a, b\}$-primitive.*
2. *G synchronizes every one-point singular $t\colon [n] \to [n]$ with $t(a) = t(b)$.*
3. *G synchronizes every simple idempotent $t\colon [n] \to [n]$ with $t(a) = t(b)$.*

Proof. $1 \Rightarrow 2$: This follows from Theorem 14.64.

$2 \Rightarrow 3$: This is clear.

$3 \Rightarrow 1$: This follows from Theorem 14.50. □

Remark 14.67 Theorem 14.25 gives a transitive permutation group that is not $\{a, b\}$-primitive for all pairs of points $a, b \in [n]$, and so does not synchronize some simple idempotent, but which is A-primitive for all $A \subseteq [n]$ with $|A| \geq 3$. Thus, synchronization of all idempotent one-point singulars with $|t^{-1}(\mathrm{dupl}(t))| = 3$ is not sufficient for $\{a, b\}$-primitivity for some $\{a, b\}$.

As the trivial subgroup on $[2]$, which is not strongly $\{0, 1\}$-primitive, synchronizes every non-permutation, Theorem 14.66 does not hold for $n = 2$.

By Theorem 14.42, if G synchronizes one simple idempotent t with $t(a) = t(b)$, then G is $\{a, b\}$-primitive. However, as Theorem 14.44 shows, $\{a, b\}$-primitivity does not imply that G synchronizes some simple idempotent t with $t(a) = t(b)$. Theorem 14.66 states that synchronization of both simple idempotents mapping a and b to the same point is equivalent to strong $\{a, b\}$-primitivity.

With our results we obtain a characterization of primitivity first given by Araújo and Cameron [34, Thm. 2].

Corollary 14.68 (Araújo and Cameron [34, Thm. 2]) *Let $G \leq_G \mathcal{S}_n$ be transitive and $1 < k < n$. Then G admits a non-trivial block of size at least k if and only if there exists a one-point singular of defect $k - 1$ not synchronized by G.*

Proof. First, suppose G has a non-trivial block Δ of size at least k. Thus, G is not Δ-primitive. By Theorem 14.64, Item 3, no idempotent one-point singular $t\colon [n] \to [n]$ with $t^{-1}(\mathrm{dupl}(t)) = \Delta$ is synchronized by G and such a t has defect $k - 1$. Conversely, suppose G does not synchronize some one-point singular $t\colon [n] \to [n]$ of defect $k - 1$. By Theorem 14.64, G is not A-primitive for $A = t^{-1}(\mathrm{dupl}(t))$ and so there exists a non-trivial block of size at least $|A|$. By Theorem 4.15, we have $|A| = k$. □

With Theorems 14.40 and 14.66 we also easily obtain that a permutation group on $[n]$ where $n \geq 3$ is primitive if and only if it synchronizes every idempotent one-point singular.

Since in a transitive group $G \leq_G \mathcal{S}_n$ a block Δ and its equal-sized translates $g(\Delta)$ partition the domain, n is a multiple of the size of every block in a transitive permutation group. Thus Theorem 14.68 implies that a transitive permutation group $G \leq_G \mathcal{S}_n$ synchronizes every transformation of kernel type $(k, 1, \ldots, 1)$ with $\frac{n}{2} < k < n$, or that an imprimitive permutation group with non-trivial blocks of size at most l synchronizes every transformation of kernel type $(k, 1, \ldots, 1)$ for $l < k < n$. Example 14.25 gave imprimitive groups of arbitrarily large size that synchronize every transformation of kernel type $(k, 1, \ldots, 1)$ with $k \geq 3$ as these groups only have non-trivial blocks of size two.

Lastly, we give a more general synchronization property for strongly $\{a, b\}$-primitive groups and so-called unimodular transformations introduced in [610]. This is based on two lemmas from [360, Lem. 4.3 & 4.4] originally used for the solution of the road coloring problem in [360]. For the convenience of the reader we give full proofs (our lemmas are actually a little less general than the original ones from [360] and adapted to our setting of groups and non-permutations). We will use the stability relation $\sim_s$ introduced on page 183.

Let $t\colon [n] \to [n]$. For $a \in [n]$, set $l(a) = \min\{i \mid \exists p > 0\colon t^i(a) = t^{i+p}(a)\}$, i.e., the shortest distance to a cycle state for the transformation induced by a (which is the shortest distance to a root state for a tail state). Let $L = \max\{l(a) \mid a \in [n]\}$. We call t *unimodal* [610] if $L \geq 1$ and for $a, b \in [n]$ with $l(a) = l(b) = L$ we have $t^L(a) = t^L(b)$. In this case, two points $a, b \in [n]$ such that

1. $t(b) = t^L(c)$ and $a = t^{L-1}(c)$ for some $c \in [n]$ with $l(c) = L$, and
2. $t^p(b) = b$ for some $p > 0$, i.e., b lies in a cycle of the functional graph of t,

are called a *unimodal pair* for t.

Intuitively, a unimodular transformation is a non-permutation with the property that all paths of maximal length that traverse a single cycle state (which must be an end state) share the same cycle state as their end state. See Figure 14.2 for an example (taken from [610]). One-point singulars are unimodal.

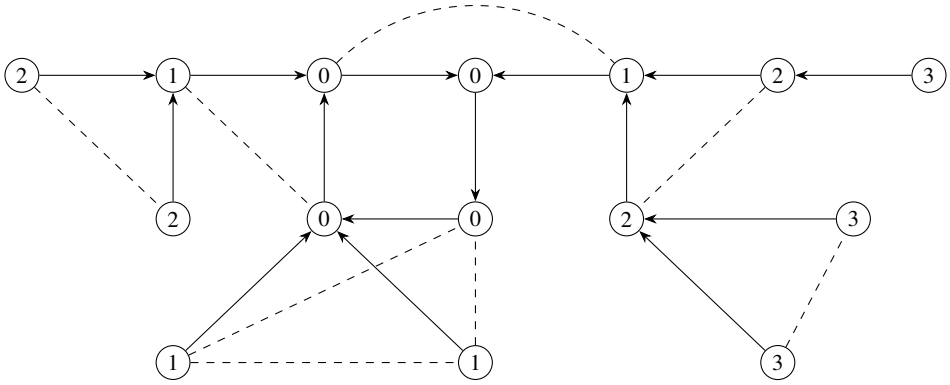


Fig. 14.2: Functional graph of a unimodal transformation. Marks indicate $l(a)$; we have $L = 3$. Dashed lines show the kernel relation.

For $S \leq_S \mathcal{T}_X$ we call $A \subseteq X$ *incompressible* if there exists no $t \in S$ such that $|A| < |t(A)|$. Clearly, if A is incompressible, then $t(A)$ is incompressible for each $t \in S$ and $|A| = |t(A)|$.

Lemma 14.69 *Let $S \leq_S \mathcal{T}_n$ be a semigroup and $A \subseteq [n]$ be a maximal incompressible set. Suppose that there exists $t \in S$ that fixes every point in A except one point $a \in A$. Then $a \sim_s t(a)$.*

Proof. Let $h \in S$. To show $a \sim_s t(a)$, we have to show that $f(h(a)) = f(h(t(a)))$ for some $f \in S$. We may assume $h(a) \neq h(t(a))$. Set $B = A \setminus \{a\}$. Then we have the following:

1. The set $h(t(A))$ is incompressible and has the same size as A
2. The set $h(A)$ is incompressible and has the same size as A.
3. $h(t(B)) = h(B)$.

This yields that the set $C = h(B) \cup \{h(a), h(t(a))\}$ has size $|A| + 1$. Then, by the maximality of A, there exists a pair of points in C that can be mapped to the same point. Since $h(t(A))$ and $h(A)$ are incompressible, this pair must be $h(a)$ and $h(t(a))$. Thus, we have $f(h(a)) = f(h(t(a)))$ for some $f \in S$. This yields $a \sim_s t(a)$. □

Lemma 14.70 *Let $G \leq_G \mathcal{S}_n$ be transitive and $t\colon [n] \to [n]$ be unimodal with unimodal pair $a, b \in [n]$. Then $a \sim_s b$ w.r.t. the transformation semigroup $\langle G \cup \{t\}\rangle_S$.*

Proof. Let $A = \{c \in [n] \mid l(c) = L\}$. Since $a, b \in [n]$ is a unimodal pair, without loss of generality, we have $t(a) = t(b) = t^L(c)$ and $a = t^{L-1}(c)$ for some $c \in A$ and b is contained in a cycle of the functional graph of t.

Next, consider a maximal incompressible $B \subseteq [n]$ w.r.t. $\langle G \cup \{t\}\rangle$. Let $d \in B$. By transitivity, we have $g(d) = c$ for some $g \in G$. The set $g(B)$ is also incompressbile. Thus, since $t^L(d) = t^L(d')$ for all $d, d' \in A$, there can only be one state from A in $g(B)$ and so we have $g(B) \cap A = \{c\}$. Let $C = t^{L-1}(g(B))$. The set C is incompressible and, by choice of L, all of its points except $a = t^{L-1}(c)$ are contained in a cycle of the functional graph of t. Let m be the least common multiple of the lenghts of the cycles in the functional graph of t. Then t^m fixes all states in the cycles but, since $t^{m+1} = t^m(t(a)) = t^m(t(b)) = t(b)$, we have $t^m(a) = b$. By Theorem 14.69 (applied with t^m, C, and $\langle G \cup \{t\}\rangle_S$ in the roles of t, A, and S, respectively) we find $a \sim_s b$. □

Theorem 14.71 *Let $G \leq_G \mathcal{S}_n$. Then G is strongly $\{a, b\}$-primitive if and only if it G synchronizes every unimodal $t\colon [n] \to [n]$ with unimodal pair a, b.*

Proof. First, suppose G is strongly $\{a, b\}$-primitive. Let $t\colon [n] \to [n]$ be unimodal with unimodal pair a, b. By Theorem 14.70, we have $a \sim_s b$ in $\langle G \cup \{t\}\rangle_S$. Since G is $\{a, b\}$-primitive, this yield that $\sim_s$ equals the universal relation. By Theorem 7.21 it follows that $\langle G \cup \{t\}\rangle_S$ contains a constant map, i.e., G synchronizes t.

Conversely, each simple idempotent mapping a to b or vice versa is a unimodal with unimodal pair a, b. Thus, by Theorem 14.50, G is strongly $\{a, b\}$-primitive. □

Corollary 14.72 *A group on a finite domain of size at least three is primitive if and only if it synchronizes every unimodal transformation.*

Proof. This follows from Theorems 14.26 and 14.71. □

One-point singulars are unimodal and with Theorem 5.12 for two points mapped to a common state we find an idempotent one-point singular having having these two points as an unimodal pair. Thus Theorem 14.71 also implies Theorems 14.66 and 14.68.

Remark 14.73 Comparing Theorem 14.71 and Theorem 14.64 the question arises whether we can characterize transitive A-primite groups for sets A with more than three points by the property that such a group synchronizes every unimodal transformation having a unimodal pair in A. We do not know how to obtain such a characterization.

14.3.7 Reachability and Distinguishability

Here, we give generalizations of Theorem 14.9 to strong $\{a, b\}$-primitivity, but formulated in a purely group-theoretical language without any reference to automata. An automata-theoretical formulation in the spirit of Theorem 14.9 can be easily given via the constructions from Section 6.8. Specifically, we characterize strong $\{a, b\}$-primitivity by

1. complete reachability and complete distinguishability,
2. the property of having a transformation mapping distinct subsets to subsets of different cardinality,
3. distinguishability of pairs of disjoint subsets of fixed cardinalities in case of a transitive permutation group,
4. distinguishability of pairs of subsets of fixed cardinalities.

So, we give further characterizations than merely those suggested by Theorem 14.9 for primitivity. From these, we also obtain, via Theorem 14.26, characterizations of primitivity; but as written in the introductory paragraph to Section 14.3 we do not state them explicitly as they follow readily.

As in Theorem 14.1, here, we combine two statements into one using square brackets.

Theorem 14.74 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 5$ and $a, b \in [n]$ be distinct. Then the following are equivalent.*

1. *G is strongly $\{a, b\}$-primitive.*
2. *For every [idempotent] $t\colon [n] \to [n]$ of defect one with $t(a) = t(b)$ the transformation semigroup $\langle G \cup \{t\}\rangle_S$ is completely reachable.*
3. *For every [idempotent] $t\colon [n] \to [n]$ of defect one with $t(a) = t(b)$ the transformation semigroup $\langle G \cup \{t\}\rangle_S$ is completely distinguishable.*
4. *For every [idempotent] $t\colon [n] \to [n]$ of defect one with $t(a) = t(b)$ we have $|h(A)| \neq |h(B)|$ for all distinct non-singletons $A, B \subseteq [n]$ for some $h \in \langle G \cup \{t\}\rangle_S$.*
5. *The group G is transitive and for some simple idempotent $t\colon [n] \to [n]$ with $t(a) = t(b)$ every k-set is distinguishable from every disjoint l-set for some $2 \leq k, l < n$ with $kl < n$ by some element from $\langle G \cup \{t\}\rangle_S$.*
6. *For every [idempotent] $t\colon [n] \to [n]$ of defect one with $t(a) = t(b)$ every k-set is distinguishable from every distinct l-set for some $2 \leq k, l < n$ by some element from $\langle G \cup \{t\}\rangle_S$.*
7. *For every [idempotent] $t\colon [n] \to [n]$ of defect one with $t(a) = t(b)$ every k-set is reachable in $\langle G \cup \{t\}\rangle_S$ for some $k \geq 2$ with $n > k(k-1)$.*

Proof. $1 \Rightarrow 2$: First, assume G is strongly $\{a, b\}$-primitive. Suppose $t\colon [n] \to [n]$ has defect one with $t(a) = t(b)$. By Lemma 5.13 we can assume t is a simple idempotent mapping (thus the same arguments works when t is assumed to be a simple idempotent). Set $M = \langle G \cup \{t\}\rangle_S$, which is a monoid since G contains the identity transformation. The mapping t induces the edge (a, b) in the digraph $I(M)$. By Proposition 5.36 the group G is precisely the group of units of M. As G is a subgroup of the automorphism group of $I(M)$ (Proposition 11.17) the edge set of $I(M)$ is a union of edge sets of orbital digraphs of G. Hence, having (a, b) as an edge in $I(M)$ implies that the orbital digraph

of (a, b) is a subgraph of $I(M)$. As G is strongly $\{a, b\}$-primitive, by Theorem 14.54, the orbital digraph of (a, b), and so $I(M)$, is strongly connected. Then the claim follows from Theorem 11.31.

$2 \Rightarrow 3$: This follows from Theorem 12.11 (reformulated for transformation semigroups, see Section 6.8) and since sync-maximality implies complete distinguishability by Theorem 10.1.[16]

$3 \Rightarrow 4$: This follows since an element distinguishing two subsets maps them to sets of distinct cardinalities.

$4 \Rightarrow 5$: Let $A = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$. Assume there exists $c \in [n] \setminus A$. Then $\{a, c\}$ and $\{b, c\}$ are not distinguishable in $\langle G \cup \{t\}\rangle_S$ for any $t\colon [n] \to [n]$ of defect one with $t(a) = t(b)$. Thus, we have $[n] = A$. Next, assume $\mathrm{Orb}_G(a) \neq \mathrm{Orb}_G(b)$ and, without loss of generality, $|\mathrm{Orb}_G(a)| \geq 3$. Then for distinct $c, d \in \mathrm{Orb}_G(a)$ with $c \neq a$ and $d \neq a$, the sets $\{a, c\}$ and $\{a, d\}$ are not distinguishable in $\langle G \cup \{t\}\rangle_S$ for the simple idempotent t with $t(b) = a$. Thus, we must have $|\mathrm{Orb}_G(a)| \leq 2$ and $|\mathrm{Orb}_G(b)| \leq 2$, which is not possible since $n \geq 5$ by assumption. Hence G must be transitive.

Next, we choose some simple idempotent $t\colon [n] \to [n]$. Then, the assumption yields that pairs of distinct 2-sets are distinguishable. So, the claim holds with $k = l = 2$.

$5 \Rightarrow 1$: Let $t\colon [n] \to [n]$ be some simple idempotent with $t(a) = t(b)$ such that in $\langle G \cup \{t\}\rangle_S$ all pairs of disjoint k- and l-sets are distinguishable for $2 \leq k, l < n$ with $kl < n$. Then, by Theorem 9.35 $\langle G \cup \{t\}\rangle_S$ contains a constant map. Theorem 14.42 yields that G is $\{a, b\}$-primitive. So, as G is transitive, it is strongly $\{a, b\}$-primitive.

$6 \Leftrightarrow 1$: First, if G is strongly $\{a, b\}$-primitive, then, as shown in Item 3, $\langle G \cup \{t\}\rangle_S$ is completely distinguishable. In particular, every k-set is distinguishable from every distinct l-set for all $k, l \geq 2$. This, we can choose, for example, $k = l = 2$.

Conversely, without loss of generality, suppose $l \leq k$. For $l = k = 2$, as $n \geq 5$, we get strong $\{a, b\}$-primitivity from Theorem 14.50. For $l = k$ with $k \geq 3$, as $n \geq k + 1$, from Theorem 9.30 we get that G synchronizes t. For $l < k$, from Theorem 9.31, Item 3 we get that G synchronizes t. In the latter two cases, by Theorem 14.66, G is strongly $\{a, b\}$-primitive.

Both implications hold whether we consider only simple idempotents t or any transformation t of defect one mapping a and b to a common point.

$7 \Leftrightarrow 1$: This follows from combining Theorem 9.14, Item 3, and Theorem 14.66. □

Corollary 14.75 *A strongly $\{a, b\}$-primitive permutation group on $[n]$ has at most $n - 2$ orbits on the 2-sets.*

Proof. By adding a simple idempotent, we obtain that the generated transformation semigroup is completely reachable by Theorem 14.74. Then, by Theorem 9.24, the number of orbits on the 2-sets is bounded by $n - 2$. □

Remark 14.76 The first three equivalences in Theorem 14.74 also hold for $n \geq 3$.

[16] This also follows similarly as Item 2 from Item 1. With the notation from the proof of Item 2, since G is strongly $\{a, b\}$-primitive, by Theorem 14.38, it is transitive. So the orbital digraph of (a, b) (which is strongly connected as shown in the proof of Item 2) is a vertex-transitive digraph. By Proposition 5.45 it is strongly $\frac{3}{2}$-connected. So $I(M)$ is strongly $\frac{3}{2}$-connected. Then Item 3 follows from Theorem 11.36.

Remark 14.77 Combining Theorem 14.22 and Theorem 14.74 immediately gives an alternative proof of Theorem 12.31 for $k = 1$.

Remark 14.78 Note that in Theorem 14.74, Item 5 we only demand existence of some simple idempotent t, and not that the written claim holds for all simple idempotents t (or mappings of defect one). However, in this case, the assumption of transitive cannot be omitted to obtain a characterization of $\{a, b\}$-primitivity. For example, consider $G = \langle (0\ 1\ \cdots\ n-1)(n)(n+1)\rangle_G$ and the simple idempotent $t\colon [n+2] \to [n+2]$ that maps $n-1$ to n. Then in $\langle G \cup \{t\}\rangle_S$ all disjoint pairs of 2-sets are distinguishable, but G is not $\{n-1, n\}$-primitive as $\{0, 1, \ldots, n-1, n\}$ is a non-trivial block containing $\{n-1, n\}$.

Remark 14.79 Note that for transitive $G \leq_G \mathcal{S}_n$, by Theorem 5.13, for both simple idempotents $t, h\colon [n] \to [n]$ we have $\langle G \cup \{t\}\rangle_S = \langle G \cup \{h\}\rangle_S$. Since all conditions from Theorem 14.74 imply transitivity of G for $n \geq 5$ and it is sufficient to consider simple idempotents, all of them are actually equivalent to G being transitive and the condition only holding for a single simple idempotent.

Remark 14.80 Theorem 14.74, Item 6 is also equivalent to the same condition but where t is only some transformation t of defect one with $t(a) = t(b)$. This follows since by Theorem 5.13, $\langle G \cup \{t\}\rangle_S$ contains both simple idempotents mapping a and b to a common point and so the claim from Theorem 14.74, Item 6 if this is true and t only has defect one.

Remark 14.81 The conditions from Theorem 14.74 cannot be weakened to give equivalences for $\{a, b\}$-primitivity only, for example, by omitting transitivity in Theorem 14.74, Item 6. For instance, the transformation monoids $\mathcal{T}_S(\mathcal{A})$ of the automata $\mathcal{A} = (Q, \Sigma, \delta)$ from Figure 7.4c are $\{s, t\}$-primitive since if $s \sim t$, then $\delta(s, b^i) \sim \delta(t, b^i) = t$ for all $i \geq 0$ and $\{\delta(s, b^i) \mid i \geq 0\} = Q \setminus \{t\}$, which yields that all states are congruent to t. However, in this automata pairs of sets of the form A and $A \cup \{t\}$ for $|A| \geq 2$ are indistinguishable.

14.3.8 2-Homogeneity and Transformation Semigroups

Here, we characterize strong $\{a, b\}$-primitivity through a property of the associated generated transformation monoid, reminiscent of 2-homogeneity. Via Theorem 14.26, this yields a novel characterization of primitivity that appears to have gone unnoticed so far. As before (see Theorem 14.1), we combine two statements using square brackets.

Theorem 14.82 *Let $n \geq 5$ and $a, b \in [n]$ be distinct. A permutation group $G \leq_G \mathcal{S}_n$ is strongly $\{a, b\}$-primitive if and only if for every [idempotent] transformation $t\colon [n] \to [n]$ of defect one with $t(a) = t(b)$ and distinct $c, d \in [n]$ there exists $h \in \langle G \cup \{t\}\rangle_S$ such that $h(\{c, d\}) = \{a, b\}$.*

Proof. We show the first conclusion for every defect-one transformation, and the converse implication under the weaker assumption that the condition holds for simple idempotents, which gives the claim for both types of mappings.

Now, suppose G is strongly $\{a, b\}$-primitive. Let $t\colon [n] \to [n]$ be of defect one with $t(a) = t(b)$. By Theorem 14.38, G is transitive. So, by Theorem 5.9 and as $n \geq 5$, $g(\{c, d\}) \cap \{c, d\} = \emptyset$ for some $g \in G$. By Theorem 14.74 there exists $h \in \langle G \cup \{t\}\rangle_S$ such that precisely one of $\{c, d\}$ or $g(\{c, d\})$ is mapped to a singleton set by h. We can write $h = g_m t g_{m-1} t \cdots g_2 t g_1 t g_0$ for some $m > 0$ and $g_i \in G$ for $i \in \{0, 1, \ldots, m\}$. Let $i \in \{1, \ldots, m\}$ be minimal such that

$$|(tg_{i-1}t \cdots g_2 t g_1 t g_0)(\{c, d\})| \neq |(tg_{i-1}t \cdots g_2 t g_1 t g_0)(g(\{c, d\}))|,$$

which is equivalent to the fact that precisely one of $\{c, d\}$ or $g(\{c, d\})$ is mapped to a singleton. Without loss of generality, assume $|(tg_{i-1}t \cdots g_2 t g_1 t g_0)(\{c, d\})| = 1$. By the minimal choice of i, we have $|(g_{i-1}t \cdots g_2 t g_1 t g_0)(\{c, d\})| = 2$. As t has defect one, we find $(g_{i-1}t \cdots g_2 t g_1 t g_0)(\{c, d\}) = \{a, b\}$.

Conversely, suppose for each simple idempotent $t\colon [n] \to [n]$ with $t(a) = t(b)$ and distinct $c, d \in [n]$ we have $h(\{a, b\}) = \{c, d\}$ for some $h \in \langle G \cup \{t\}\rangle_S$. Now, $h(\{a, b\}) = \{c, d\}$ implies $(th)(\{c, d\}) = \{t(a)\}$. So, by the transformation semigroup version of the pair criterion (Section 6.8 & Theorem 7.18) we can conclude that $\langle G \cup \{t\}\rangle_S$ contains a constant map. By Theorem 14.66 it follows that G is strongly $\{a, b\}$-primitive. □

Remark 14.83 By Theorem 12.16, we can even chose an element $h \in \langle G \cup \{t\}\rangle_S$ such that h maps distinct 2-sets to distinct sets, i.e., it induces an injective mapping on the 2-sets.

By Theorem 14.26 we obtain the following characterization of primitivity, which, to the best of our knowledge, is new.

Corollary 14.84 *A permutation group $G \leq_G \mathcal{S}_n$ with $n \geq 5$ is primitive if and only if for every [idempotent] transformation $t\colon [n] \to [n]$ of defect one with $t(a) = t(b)$ and distinct $c, d \in [n]$ there exists $h \in \langle G \cup \{t\}\rangle_S$ such that $h(\{c, d\}) = \{a, b\}$.*

Example 14.85 The group $G = \langle\{(0\ 1\ \cdots n-1)\}\rangle$ is $\{0, 1\}$-primitive. Thus in the transformation semigroup $\mathcal{T}_{C_n}$ of the Černý automaton we can map any 2-set onto $\{0, 1\}$. We note that even more is true for $\mathcal{T}_{C_n}$, namely that for any two 2-sets $\{i_1, i_2\}$ and $\{j_1, j_2\}$ we have $h(\{i_1, i_2\}) = \{j_1, j_2\}$ for some $h \in \mathcal{T}_{C_n}$. To show this, it is sufficient to show that we can map $\{0, 1\}$ to any other 2-set. So, consider some 2-set $\{i_1, i_2\}$ with $i_1 < i_2$. We have $\delta(\{0, 1\}, b^{i_1}) = \{i_1, i_2\}$ if $\delta(i_1, b) = i_2$ and $\delta(\{0, 1\}, b^{i_2}) = \{i_1, i_2\}$ if $\delta(i_2, b) = i_1$. Otherwise, $\delta(i_1, b^m) = i_2$ for some $m \in \{2, \ldots, n-1\}$. Then consider the words $u_i = b^{n-i}ab^i$ for $i \in [n]$ from the proof of Theorem 9.43. We have $\delta(\{0, 1\}, u) = \{0, i_2 - i_1\}$ for $u = u_1 \cdots u_{i_2 - i_1 - 1}$. Now $\delta(\{0, i_2 - i_1\}, b^{i_1}) = \{i_1, i_2\}$ and so the word ub^{i_1} maps $\{0, 1\}$ to $\{i_1, i_2\}$.

Remark 14.86 Let $t\colon X \to X$ be a simple idempotent mapping a to b with $a \neq b$ and $A \subseteq X$ be such that and $a \notin A$ (which holds for $|t(A)| = |A|$ and $t(A) \neq A$). Then $g(A) = t(A)$ for the transposition $g = (a\ b)$. This follows since, if $b \notin A$, then $g(A) = A = t(A)$, and otherwise, $g(A) = (A \setminus \{b\}) \cup \{a\} = t(A)$.

Now, if we choose in Theorem 14.82 a product $h = g_m t g_{m-1} t \cdots g_2 t g_1 t g_0$ of minimal length, then, by the previous paragraph, h and the permutation we obtain from the above

product by changing each t to $(a\ b)$, both map $\{c, d\}$ to $\{a, b\}$. Thus, $\langle G \cup \{(a\ b)\}\rangle_G$ is 2-homogeneous, and since it contains a transposition and, by Equation (5.2), also the transpositions $(g(a)\ g(b))$ for $g \in G$, and so all transpositions, it equals $\mathcal{S}_n$ by Theorem 4.7. Thus, adding to a strongly $\{a, b\}$-primitive permutation group the transposition $(a\ b)$ yields the full symmetric group. In this way, applying Theorem 14.82 to all pairs, we obtain a new proof of Jordan's theorem (Theorem 14.4).

14.4 Congruences on Transformation Semigroups

The preceding sections have linked the absence of certain congruences to synchronizability, reachability, and distinguishability properties. Here, we collect some sufficient conditions for the absence of non-trivial S-congruences in a transformation semigroups S.

For $g = (0\ 1\ \cdots n-1)$ and $t\colon [n] \to [n]$ let r be the greatest common divisor of the numbers $t(i) - i$ for $i \in [n]$ and n. If $r > 1$, then the mod-r residue classes form a non-trivial $\langle G \cup \{t\}\rangle_S$-congruence. Next, we show that under certain assumptions, $r = 1$ also implies that there exist no non-trivial congruences. This statement extends a statement from [287, Lem. 3].

Proposition 14.87 *Let $g = (0\ 1\ \cdots\ n-1)$ and $t_1, \ldots, t_k\colon [n] \to [n]$ be transformations moving at least one point such that, for each $i \in \{1, \ldots, k\}$, we have $n \geq 2M_i - 1$ or $|\operatorname{move}(t_i)| < d$ for all proper divisors d of n with $d > 1$ where $M_i = \max(\operatorname{move}(t_i))$. If the greatest common divisor of the numbers $t_i(j) - j$ for $j \in \operatorname{move}(t_i)$ and $i \in \{1, \ldots, k\}$ is coprime to n, then $S = \langle\{g, t_1, \ldots, t_k\}\rangle_S$ admits no non-trivial S-congruence.*

Proof. Assume there exists an S-congruence that does not equal the universal relation. Since this is also a $\langle g\rangle_S$-congruence, by Theorem 5.5, each class is a mod-r residue class where r divides n and we have $r < n$. For a contradiction, assume $r > 1$.

Next, we show that for each $i \in \{1, \ldots, k\}$, each congruence class contains an element fixed by t_i. To see this, note that for $|\operatorname{move}(t_i)| \geq n/r$ we have $n - M_i \geq r$ since $r \leq n/2$ as it is a proper divisor and, with the assumption $n \geq 2M_i - 1$, we find

$$n - M_i \geq n - \frac{n+1}{2} = \frac{n-1}{2} \geq r - \frac{1}{2}.$$

Thus, if $|\operatorname{move}(t_i)| \geq n/r$, then since $\{M_i, M_i + 1, \ldots, n\} \subseteq \operatorname{fix}(t_i)$ and $n - M_i \geq r$ each congruence class contains a point from $\operatorname{fix}(t_i)$, and if $|\operatorname{move}(t_i)| < n/r$, since each congruence class has size n/r, it also contains a point from $\operatorname{fix}(t_i)$.

Next, consider $j \in \operatorname{move}(t_i)$. As written above, there exists $l \in \operatorname{fix}(t_i)$ with $j \equiv l \pmod r$. Thus we have $t_i(l) \equiv l \pmod r$ and since $\equiv$ is an S-congruence and $j \equiv l \pmod r$ we get $t_i(j) \equiv j \pmod r$, i.e., r divides $t_i(j) - j$. Hence, r divides the numbers $t_i(j) - j$ for $i \in \{1, \ldots, k\}$ and $j \in \operatorname{move}(t_i)$. But r also divides n, and by the assumption, it follows that $r = 1$, giving a contradiction. □

Corollary 14.88 *Let $g = (0\ 1\ \cdots\ n-1)$ and $t_1, \ldots, t_k\colon [n] \to [n]$ be simple idempotents. If the greatest common divisor of the numbers $t_i(j) - j$ for $j \in [n]$ with $t_i(j) \neq j$*

where $i \in \{1, \ldots, k\}$ is coprime to n, then $S = \langle \{g, t_1, \ldots, t_k\} \rangle_S$ admits no non-trivial S-congruence.

Proof. This follows from Theorem 14.87 since for simple idempotents we have $|\operatorname{move}(t_i)| = 1$. □

Example 14.89 Let $G = \langle \{(0\ 1\ \cdots\ n-1)\} \rangle_G$. Then for non-empty $A \subseteq [n]$ where there exists no divisor r of d with $r > 1$ such that $i \equiv j \pmod{r}$ for all $i, j \in A$ we have that G is A-primitive by Theorem 5.5. In this case, by Theorem 14.87, when adding simple idempotents mapping every two distinct states in A onto each other, the generated transformation semigroup S does not admit any non-trivial S-congruences. For example, if n is even and $n \geq 8$, then G is $\{0, 2, 5, 7\}$-primitive. Adding a simple idempotent mapping 0 to 5 results in a transformation semigroup admitting no non-trivial congruence, but adding the two simple idempotents mapping 0 to 2 and 5 to 7 results in a transformation semigroup that admits a congruence whose classes are the mod-2 residue classes. By Theorem 14.87 we have to add simple idempotents t such that the numbers $t(i) - i$ and n are coprime. In fact, for simple idempotents, this condition characterizes those sets of simple idempotents resulting in transformation semigroups admitting a non-trivial congruence.

Lemma 14.90 *Let G be a transitive permutation group on X and t be a simple idempotent with $t(a) \neq a$ and $S = \langle G \cup \{t\} \rangle_S$. Then $t(a) \sim a$ for every non-trivial S-congruence $\sim$ on X.*

Proof. Let Δ be an equivalence class for some non-trivial S-congruence. Since this is also a G-congruence, Δ is a block of G of size at least two. By transitivity, we have $a \in \Delta^g$ for some $g \in G$. Let $b \in \Delta^g$ with $b \neq a$. We have $b \sim a$. Since $\sim$ is a S-congruence, we find $b = t(b) \sim t(a)$ and so $t(a) \sim a$. □

Corollary 14.91 *Let G be a transitive and A-primitive permutation group on X for some $A \subseteq X$. For a set $T \subseteq \mathcal{T}_X$ of simple idempotents such that (X, E) with $E = \{\{a, t(a)\} \mid a \in X, t(a) \neq a\}$ is a connected graph, the semigroup $S = \langle G \cup T \rangle_S$ admits no non-trivial S-congruences.*

Proof. By Theorem 14.90 and the assumption, the points in A are all congruent for each S-congruence not equal to the equality relation, and thus by A-primitivity the universal relation is the only S-congruence not equal to the equality relation. □

Lemma 14.92 *Let G be a permutation group on X having a non-empty set of fixpoints F and where $X \setminus F$ is an orbit. Then the only blocks containing F are F and X.*

Proof. Clearly, F is a block. Next, let Δ be a block properly containing F. Pick $a \in \Delta \setminus F$. Assume there exists $b \in X \setminus \Delta$. By assumption, we have $g(a) = b$ for some $g \in G$. Now, $F \subseteq g(\Delta)$. Thus, since Δ is a block, we have $\Delta = g(\Delta)$. However, since $b \in g(\Delta)$, this gives $b \in \Delta$, which is a contradiction. □

Let $\sim$ be an S-congruence. Clearly, $t([a]_\sim) \subseteq [a]_\sim$ for $a \in \operatorname{fix}(t)$. Thus, if for a transformation semigroup S on X with S-congruence $\sim$ and $A \subseteq X$, for each pair of distinct $a, b \in X$ there exist $t \in S$ and $c \in \operatorname{fix}(t)$ with $t(a) = b$ and $c \sim a$, we have $b = t(a) \subseteq [a]_\sim$ and this yields that all points in A are congruent w.r.t. $\sim$.

Proposition 14.93 *Let G be $\{a, b\}$-primitive for distinct a, b on a set X of size at least three and $S = \langle G \cup \{t\}\rangle_S$ for a simple idempotent $t\colon X \to X$ mapping a to b. Then G is transitive and S admits no non-trivial S-congruence, or G is intransitive with two distinct orbits $\mathrm{Orb}_G(a)$ and $\mathrm{Orb}_G(b)$ and the non-trivial S-congruences are precisely those induced by sets of the form $\{c\}$ for $c \in \mathrm{Orb}_G(a)$ and the equivalence classes of some H-congruence on $\mathrm{Orb}_G(b)$ for $H = \{g_{|\,\mathrm{Orb}_G(b)} \mid g \in G\}$ that does not equal the equality relation.*

Proof. Let $\sim$ be some S-congruence not equal to the equality relation. First, assume there exists $c \in X \setminus \{a\}$ with $c \sim a$. Then $c = t(c) \sim t(a) = b$ and so $a \sim b$, which yields that $\sim$ is the universal relation. Otherwise, $\{a\}$ is a congruence class. If G is transitive, then all blocks have size one and so $\sim$ equals the equality relation. If G is intransitive, then by Theorem 14.32 we have two disjoint orbits $\mathrm{Orb}_G(a)$ and $\mathrm{Orb}_G(b)$. Since $\{a\}$ is an equivalence class, and so a block for G, we can conclude that all equivalence classes in $\mathrm{Orb}_G(a)$ are singletons. Thus, for $c \in \mathrm{Orb}_G(b)$ we have $[c]_\sim \subseteq \mathrm{Orb}_G(b)$ and since $\mathrm{Orb}_G(b) \subseteq \mathrm{fix}(t)$, $[c]_\sim$ is a block. Next, it is easy to verify that the given sets induce S-congruences on X. So, we obtain the claim. □

Example 14.94 The group $\langle (0)(1\ 2\ 3)\rangle_G$ is $\{0, 1\}$-primitive, intransitive and the partition $\{0\}, \{1, 2, 3\}$ induces a congruence for the transformation semigroup generated when adding the simple idempotent mapping 0 to 1.

14.5 Discussion

In Sections 14.1.2 and 14.1.3 we characterized primitivity by extremal state complexity properties of associated automata. Here, we discuss some more relations to the fields of state and descriptional complexity; see [237, 321] for surveys of both fields. In particular, we discuss so-called most complex families of languages. Then we discuss the relation of primitivity to A-primitivity and give an overview on some research concerned with transformation semigroups generated by transformations having certain properties. Lastly, we give some historical remarks on primitive permutation groups.

Synchronizing groups were introduced in [40]. Since then, they have been studied extensively, including generalizations such as almost synchronizing groups [29] and connections with graph theory [38, 36]. For a survey, see [36].

In [610], it was shown that automata admitting no non-trivial congruences and containing a unimodal letter are synchronizing. This result is more general than Theorem 14.72, since such an automaton need not have any permutational letters at all, see also the discussion in [610].

In Section 14.1.3 we characterized primitive groups by the complete reachability of associated automata and transformation semigroups generated by adjoining a single transformation of defect one. Conditions that yield the complete reachability of transformation semigroups generated by an imprimitive group and a single transformation of defect one have been investigated by Casas [600].

Most Complex Languages.

Section 14.1.2 and 14.1.3 linked the absence of non-trivial G-congruences to extremal properties of the state complexity of $\mathrm{Syn}(\mathcal{A})$, or of languages accepted by $\mathcal{A}$, for automata $\mathcal{A}$ with $G \subseteq \mathcal{T}_S(\mathcal{A})$. Restivo & Vaglica [508, Prop. 1] (see also Davies [171, Prop. 3.2] or [172, Prop. 4.3.2]) firstly noted that minimality of a strongly connected automaton for each distribution of a start state and a proper non-empty set of final states is equivalent to absence of non-trivial $\mathcal{T}_S(\mathcal{A})$-congruences. Can we relate other extremal properties investigated in the field of state complexity to $\mathcal{T}_S(\mathcal{A})$-congruences? Brzozowski [107, 108, 110] called a sequence of regular languages $L_n \in C$ *most complex in a class C of regular languages* with $n \geq k$ for some $k \geq 1$ if the following conditions hold (for definitions of atoms, reversal, etc., we refer to [107]):

1. The minimal automaton for L_n has n states.
2. The transformation monoid of the minimal automaton has maximal cardinality for all languages in the class C.
3. The minimal automata of the left quotients have n states.
4. The number of distinct atoms (intersections of complemented and uncomplemented left quotients) of L_n is maximal possible and each atom has maximal state complexity for languages in the class C.
5. The resulting minimal automata for the following operations are maximal possible w.r.t. the class C: reversal, star of L_n, and product (concatenation), union, intersection, set difference and symmetric difference of L_m and L_n.

For the class of all languages, in [107] the sequence of languages accepted by the automata $\mathcal{U}_n = ([n], \{a, b, c\}, \delta_n, \{0\}, \{n-1\})$ shown in Figure 14.3 was given where a and b induce $(0\ 1 \cdots n-1)$, and $(0\ 1)$, respectively, on the states and c is a simple idempotent letter mapping $n-1$ to 0. Furthermore, the following subclasses C have been investigated: union-free regular languages [108], ideal languages [110], and non-returning regular languages [109]. The most complex sequences of languages from [107, 108, 110] are all over fixed alphabets, whereas the sequence from [109] is over a growing alphabet.

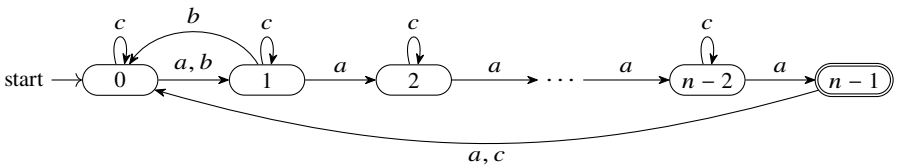


Fig. 14.3: Automata from [107] yielding a sequence of most complex languages in the class of all languages. The sole final state is marked by a double line.

As the full transformation semigroup is completely reachable, completely distinguishable, and contains the full symmetric group, which is primitive (see Theorem 5.2, Item 2), we obtain the following for the class of all regular languages.

Proposition 14.95 *Let $\mathcal{A}_n$ be a sequence of n-state minimal automata such that $L_n = L(\mathcal{A}_n)$ is most complex in the class of all regular languages. Then $\mathcal{T}_S(\mathcal{A}_n)$ equals the full transformation semigroup on n states and so $\mathcal{A}_n$ admits no non-trivial $\mathcal{T}_S(\mathcal{A}_n)$-congruence and is completely reachable, completely distinguishable, and sync-maximal.*

So, for the class of all regular languages, we also obtain sync-maximal automata not admitting any non-trivial $\mathcal{T}_S(\mathcal{A})$-congruences. In general, how do both properties, sync-maximality (or more generally, the size of accepting automata for sets of synchronizing words, i.e., the operation $\mathcal{A} \mapsto \mathrm{Syn}(\mathcal{A})$ that associates a regular ideal language with every automaton or regular language by using the minimal automaton) and absence of non-trivial $\mathcal{T}_S(\mathcal{A})$-congruences of the minimal automata, relate to the other extremal properties of most complex languages?

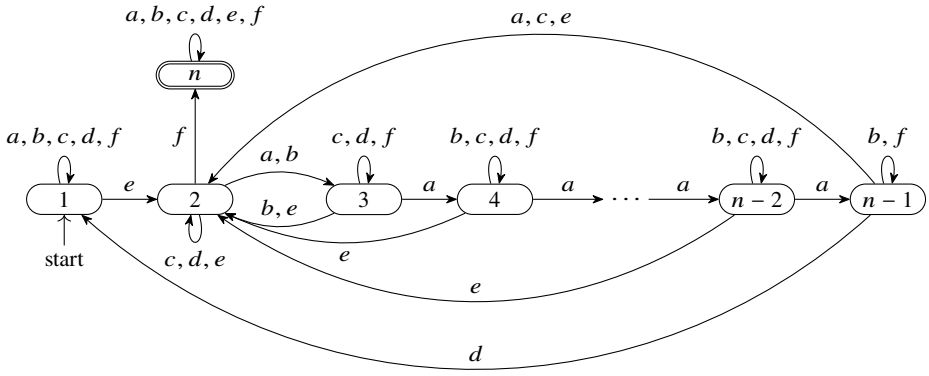


Fig. 14.4: Automata $\mathcal{D}_n$ from [110] yielding a sequence of most complex languages in the class of regular ideal languages. The sole final state is marked by a double line.

For regular ideal languages, by Equation (7.1), there is no blow-up of minimal automata for the set of synchronizing words. Thus, for a most complex sequence in the class of ideal languages accepting automata are not sync-maximal. For this class, Brzozowski [110] gave, for $n \geq 5$, the sequence

$$\mathcal{D}_n = (\{1, 2, \ldots, n\}, \{a, b, c, d, e, f\}, \delta_n, 1, \{n\})$$

of minimal automata accepting most complex regular ideal languages where

1. a induces the cyclic permutation $(2\ 3\ \cdots\ n-1)$,
2. b induces the transposition $(2\ 3)$,
3. c is a simple idempotent letter mapping $n-1$ to 2,
4. d is a simple idempotent letter mapping $n-1$ to 1,
5. e maps everything in $\{1, \ldots, n-1\}$ to 2 and fixes n,
6. f is a simple idempotent letter mapping 2 to n.

See also Figure 14.4 for a state diagram. Here, $|\mathcal{T}_S(\mathcal{D}_n)| = n^{n-2} + (n-2)2^{n-2} + 1$ (the proof of this result appears in [114]) and a minimal automaton for reversal of $L((\mathcal{D}_n)_{\{a,d,e,f\}})$ has size $2^{n-2} + 1$ [110]. So, w.r.t. other properties than the state complexity of $\mathrm{Syn}(\mathcal{D}_n)$ we obtain an exponential blow-up. Next, in Theorem 14.96, we show that these automata do not admit any non-trivial $\mathcal{T}_S(\mathcal{D}_n)$-congruence and that an automaton constructed from the subsets with a property similar to distinguishability indeed is minimal and exponentially larger.

Proposition 14.96 *1. The automata $\mathcal{D}_n$ admit no non-trivial $\mathcal{T}_S(\mathcal{D}_n)$-congruence.*
2. Every subset $A \subseteq \{1, 2, \ldots, n\}$ with $\{1, n\} \subseteq A$ is reachable.
3. For distinct $A, B \subseteq \{1, 2, \ldots, n\}$ with $\{1, n\} \subseteq A \cap B$ there exists $u \in \{a, b, c\}^* f$ mapping precisely one subset to $\{1, n\}$ and the other to a set containing at least three states.
4. The minimal automaton for $L(\mathcal{P}(\mathcal{D}_n)_{\{1,2,\ldots,n\},\{\{1,n\}\}})$ has at least 2^{n-2} states.

Proof. Set $S = \{2, 3, \ldots, n-1\}$. Since $(2\ 3\ \cdots\ n-1)$ and $(2\ 3)$ restricted to the domain S generate the full symmetric group on S, $\mathcal{T}_M((\mathcal{D}_n)_{|\{a,b\},S})$ contains every permutation of S that fixes 1. As c is a simple idempotent when restricted to S and the symmetric group is primitive, by Theorem 14.9, $(\mathcal{D}_n)_{|\{a,b,c\},S}$ is completely reachable and completely distinguishable (note that $(\mathcal{D}_n)_{|\{a,b,c\},S}$ is isomorphic to the automaton from Figure 14.3.)

1: Let $\sim$ be a $\mathcal{T}_M(\mathcal{D}_n)$-congruence with $p \sim q$ for states p, q with $p < q$. There exists $u \in a^* f$ such that $\delta_n(q, u) = n$ and $\delta_n(p, u) \in \{1, 2, \ldots, n-1\}$. Now, for any $r \in \{1, \ldots, n-1\}$ we find $v \in \{a, d, e\}^*$ such that $\delta_n(p, uv) = r$. Then $\delta_n(q, uv) = n$ and so $r \sim n$. Hence, $\sim$ equals the universal relation.

2: We have $\delta_n(\{1, 2, \ldots, n\}, ea^{n-3}d) = \{1, n\}$. Next, let $A \subseteq \{1, 2, \ldots, n\}$ be a subset that properly contains $\{1, n\}$. As $(\mathcal{D}_n)_{|\{a,b,c\},S}$ is completely reachable there exists $u \in \{a, b, c\}^*$ such that $\delta_n(S, u) = A \setminus \{1, n\}$. Since u fixes 1 and n, we find $\delta_n(\{1, 2, \ldots, n\}, u) = A \cup \{1, n\} = A$.

3: Consider $A \cap S$ and $B \cap S$, which can be assumed to be non-empty, since otherwise exactly one of them equals $\{1, n\}$. As $(\mathcal{D}_n)_{|\{a,b,c\},S}$ is completely distinguishable, there exists $u \in \{a, b, c\}^*$ mapping precisely one subset, say $A \cap S$, to a singleton. As a cyclically permutes S, we can assume $\delta_n(A \cap S) = \{2\}$. Then $\delta_n(A, uf) = \delta_n(\{1, n, 2\}, f) = \{1, n\}$ and $|\delta_n(B, uf)| \geq 3$ since $\delta_n(B, uf) = \delta_n(\{1, n\}, uf) \cup \delta_n(B \cap S, uf)$ where $|\delta_n(B \cap S, u)| \geq 2$ and $\delta_n(\{1, n\}, u) = \{1, n\}$.

4: There are 2^{n-2} sets of the form $\{1, n\} \cup A$ with $A \subseteq S$, each reachable by 2 and, by 3, every pair of distinct sets among these sets is distinguishable in $\mathcal{P}(\mathcal{D}_n)_{\{1,2,\ldots,n\},\{\{1,n\}\}}$. So, Theorem 6.88, Item 4 gives the claim. □

Note that $ef \in \mathrm{Syn}(\mathcal{D}_n)$. While, by Equation (7.1), the automata $\mathcal{D}_n$ themselves accept $\mathrm{Syn}(\mathcal{D}_n)$, Theorem 14.96 shows that we obtain exponentially larger minimal automata w.r.t. different final state sets in the power automata whose start states are the whole state set. So, in view of the observations made in Theorem 14.96, we ask the following questions.

How do families of most complex regular languages $L_n = L(\mathcal{A}_n)$ in some class and with minimal automata $\mathcal{A}_n$ relate to $\mathcal{T}_S(\mathcal{A}_n)$-congruences? Furthermore, in a family of most complex regular languages w.r.t. some class, can we choose final state sets in the power automata with start state the whole state set to obtain exponentially larger minimal automata; and conversely how does existence of those final state sets relate to extremal properties w.r.t. the complexity measures of most complex families of regular languages?

We mention in passing that it has been observed that state-wise maximal automata for the results of a wide class of regularity-preserving operations can often be realized by so-called *monsters* [129, 164]. These are automata that contain a letter for each transformation of the underlying state set (and so preserve no congruence, are completely reachable, and are completely distinguishable). However, there is no notion of a monster relative to some class of languages.

Primitivity and A-Primitivity.

Most characterizations of primitivity from Section 14.1 transfer easily to the setting of strongly $\{a, b\}$-primitive groups. In this sense, the properties behind these characterizations are actually "local" properties applying to pairs of distinct points, yielding primitivity when using them "globally", i.e., applying them to all points or small subsets of points, via Theorem 14.26. However, the characterizations Item 1, Item 5 and Theorem 14.6 do not seem to admit a (at least straightforward) generalization to strongly $\{a, b\}$-primitive groups. Theorem 14.21 gave a strongly $\{0, 1\}$-primitive group G that is imprimitive, and so where no point stabilizer is maximal. Furthermore, for $t\colon [8] \to [8]$ with $t(0) = t(2) = t(4) = t(6) = 0$ and $t(1) = t(3) = t(5) = t(7) = 1$ there is no $g \in G$ such that tg is idempotent. This implies that $\langle G \cup \{t\}\rangle_S$ cannot be written in the form EG for its set of idempotents E. In this sense, the characterizations by maximal point stabilizers and the one on adjoining transformations of rank two are genuinely "global" properties characterizing primitivity involving all points, or their point stabilizers, and it breaks down when only considering a subset A of points (or the point stabilizers of points from A) in imprimitive permutation groups that are A-primitive.

Permutation Groups and Non-Permutations.

Pin [486] proved that a transitive permutation group of prime degree synchronizes any non-permutation (which follows from [40, Lem. 3.1 & Cor. 3.3] as well). This result was also stated by Salomaa [545, Thm. 5] (without reference to Pin). In [544, 545, 546, 547], besides Pin's result, he gave further sufficient conditions on sets of functions to generate a given function, or a function of a given rank (where synchronizability corresponds to functions of rank one). For example, he also stated that a 2-transitive group synchronizes every non-permutation [545, Thm. 4] (this follows from Theorem 14.14, Item 2). He also investigated the length of generating sequences, i.e., words over the generators, that

realize functions from some class. The focus of these studies was not on algorithmic questions, but the problems investigated also overlap with the algorithmic problems mentioned in Section 8.4, in particular those related to transformation semigroups discussed at the end. In a broader context, Salomaa was motivated by questions in logic, and under what conditions a set of basic logical functions generate all logical functions. In [542, 543] he investigated sets of functions $f\colon [n]^i \to [n]$, $i \geq 0$, generating all m-ary functions for every $m \geq 0$ under substitutions when adding a surjective function $[n]^j \to [n]$, $j \geq 2$, depending on at least two input variables. At the intersection between group theory and semigroup theory, one of the driving forces is the paradigm of "detecting properties [of permutation groups] with functions" [36], or, as formulated in [31, 35], the following related question:

> Classify the pairs (G, a), where a is a map on a finite set Ω and G is a group of permutations on Ω, such that the semigroup $\langle G, a\rangle$ generated by a and G has a given property P.

The results of the present chapter fit into this paradigm. Other results are given in [36, Sec. 8], as for example, that 2-homogeneity is characterized by the property P of containing every non-permutation [445, Thm. 3.18] (see also [36, Thm. 8.2]).

For further connections, or similar properties, we refer to [28, 30, 32, 33, 34, 36, 38, 103]. Göhring [250] has investigated so-called *Černý-like automata* by various properties of permutation groups contained in the transformation semigroup of an automaton.

For the result from Chapter 14 it does not matter whether we formulate them in terms of the generated semigroup or monoid, as for non-empty $P \subseteq \mathcal{S}_n$ and $T \subseteq \mathcal{T}_n \setminus \mathcal{S}_n$ we have $\langle P \cup T\rangle_S = \langle P \cup T\rangle_M$ as $\mathrm{id}_{[n]} \in \langle P\rangle_S$.

Historical Remarks on Primitive Permutation Groups.

Galois [466] introduced primitive permutation groups in his seminal approach to the characterization of the solvability of polynomial equations, based on an observation made by Lagrange [398] relating permutation groups to polynomial equations.[17] The first systematic treatment of permutation group theory was written by Jordan [346], where he proved many, now classical, results about primitive permutation groups as for example Theorem 14.4. An intransitive permutation group can be decomposed as a subcartesian product of its transitive constituents [126, Thm. 1.2]. Now, a transitive but imprimitive permutation group can be decomposed into a wreath product where the action of the group equals the imprimitive action of the wreath product for an identification of the permutation domain with a Cartesian product, see [126, Thm. 1.8], [385], [179, Sect. 2, 6], or [393, Sect. 4.4] for a very detailed explanation.

Scott [554, p. 329] classified the subgroups of the symmetric group up to conjugation (with a footnote mentioning that O'Nan obtained the same result independently). The cases fall into categories covering the primitive and imprimitive groups separately (which can be seen from the proof). So, the result yields a classification of the

[17] See also `https://hsm.stackexchange.com/questions/15489/where-did-lagrange-write-his-technique-of-resolvents-for-solving-polynomials?noredirect=1&lq=1` for links to the original sources.

primitive permutation groups, called the *O'Nan-Scott theorem* and explicitly given by Cameron [125, Thm. 4.1] via the action on the *socle*, i.e., the subgroup generated by the minimal normal subgroups (see [125, Sect. 8] for a discussion in a broader context). The cases from the statement involving 2-transitive groups, which were either almost simple or affine, were given previously by Burnside [119]. However, the result was even given earlier by Jordan [346] as pointed out by Peter Neumann, see the blog post [124]. However, a case was missing in the statement of the theorem, which was discovered and corrected by Aschbacher & Scott [46] and, independently, Kovács [384]. Liebeck et al. [413] gave a fully self-contained and corrected statement of the O'Nan-Scott theorem, where the previously missing case was described using the *twisted wreath product action*. For further information, we refer to the surveys [125, 497] and the textbooks [126, 179].

Chapter 15
Slowly Synchronizing Automata

In [20, 368] an automaton was called *slowly synchronizing* if "[...][its] reset length [is] close to the Černý bound [...]" [20]. We adhere to this intuitive "non-quantitative" definition. In [21] the authors focused on slowly synchronizing n-state automaton with reset lengths between $(n-2)^2+2$ and $(n-1)^2$. In [435, 616] slowly synchronizing automata whose reset lengths do not fall within these bounds have been given. As we are dealing mainly with families of automata and to formulate two conjectures, we introduce *slowly synchronizing families of automata* as infinite families of distinct automata $\mathcal{A}_n = (Q_n, \Sigma, \delta_n)$ such that $\mathrm{rt}(\mathcal{A}_n) \geq C \cdot |Q_n|^2$ for some $C > 0$ and all $n \geq 1$ (most of the time we have $|Q_n| = n$).[1]

For example, the Černý family C_n from Section 1.4 (Figure 1.7) is slowly synchronizing. For any reasonable interpretation of slowly synchronizing automata, we would expect every family of slowly synchronizing automata to give a slowly synchronizing family of automata.

In Section 15.1, we discuss automata whose reset lengths reach the Černý bound. In Section 15.2 we investigate certain known families of slowly synchronizing automata and observe that for them minimal automata for their sets of synchronizing words are exponentially larger (we also confirm sync-maximality for the automata $\mathcal{K}_n$ for which this was previously only conjectured). Section 15.3 concludes by discussing and formulating conjectures based on our observations.

15.1 Automata Reaching the Černý Bound

Besides the Černý family from Figure 1.7, for $n \geq 3$, only the eight exceptional synchronizing n-state automata from Figure 15.1 are known to have reset length $(n-1)^2$ and the property of becoming non-synchronizing after the removal of any letter [184, 613]. These exceptional automata are given in Figure 15.1. The automaton **K** was discovered

[1] We denote the alternating group also with $\mathcal{A}_n$, see 112. But there should no confusion arise since the context makes it clear whether we denote an automaton family or the alternating group by $\mathcal{A}_n$.

by Kari [357], the automaton **I** by Roman [516], and the remaining six automata by Trahtman via a computer search [603].

By extensive compute analysis, Don et al [184] found 19 additional automata with at least three and at most six states reaching the Černý bounds, but they do not have the property of becoming non-synchronizing after removal of any letter. In this sense, with these 8 exceptional automata and the 4 automata from the Černý family we have 12 automata with at least two and at most six states that reach the Černý bound and are minimal w.r.t. the alphabet.

Here, we investigate the eight automata from Figure 15.1 w.r.t. sync-maximality.

Theorem 15.1 *Consider the exceptional automata for the Černý bound from Figure 15.2. The automata **D**, **E**, **I** and **F** are sync-maximal. **G**, **H**, **J** and **K** are not sync-maximal.*

Proof. We handle each automaton individually.

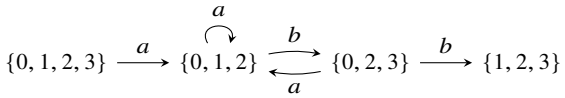
D, **E**, **F**: In **D** the letter a generates a primitive permutation group; in **E** and **F** the letters b and c generate the full symmetric group on $\{0,1,2\}$ in the transformation semigroup, which is primitive. So, in all cases, sync-maximality follows from Theorem 14.9.

For **E**, we also obtain sync-maximality directly by observing that a has image $\{1,2\}$, ac has image $\{0,1\}$, acb has image $\{0,2\}$, $acba$ has image $\{1\}$, and since we have the following distinguishing words for all pairs:

Pair of Subsets	Dist. Word	Pair of Subsets	Dist. Word
$\{0,2\}, \{1,2\}$	a	$\{0,1\}, \{1,2\}$	ba
$\{0,2\}, \{0,1\}$	a	$\{0,1,2\}, \{0,1\}$	ba
$\{0,2\}, \{0,1,2\}$	a	$\{0,1,2\}, \{1,2\}$	cba

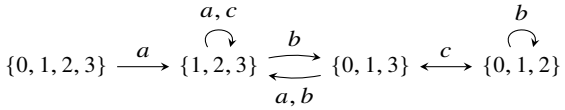
For **E**, we have that a, ab, abc, $abca$ map $\{0,1,2\}$ to $\{1,2\}$, $\{0,2\}$, $\{0,1\}$ and $\{1\}$ respectively. Furthermore, a distinguishes $\{0,1\}$ form every other 2-set, and ca distinguishes $\{1,2\}$ from $\{0,2\}$.

G: The following diagram shows the reachable 3-sets:

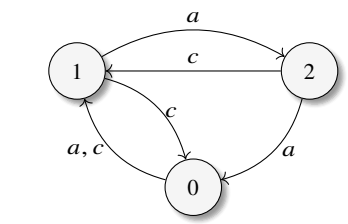


Since $\{0,1,3\}$ is not reachable, the automaton is not sync-maximal by Theorem 10.7.

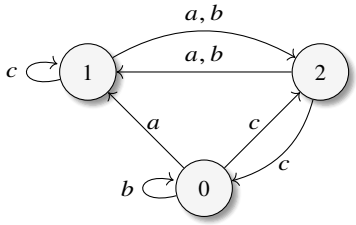
H: The following diagram shows all reachable 3-sets:



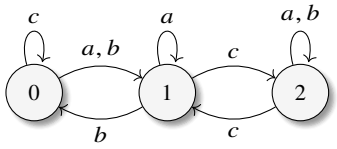
We see that not all 3-sets are reachable. So, the strongly connected automaton H is not sync-maximal by Theorem 10.7.



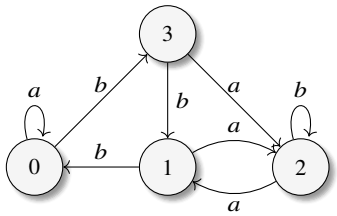
The automaton **D**



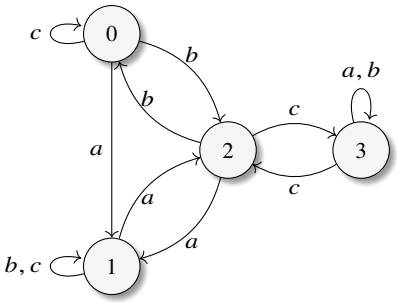
The automaton **E**



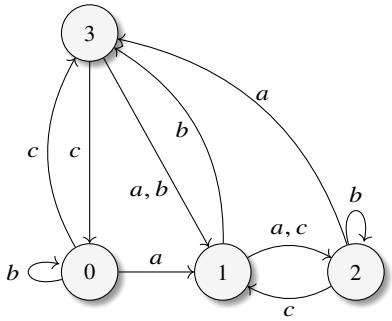
The automaton **F**



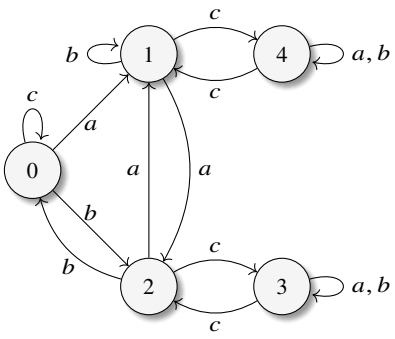
The automaton **G**



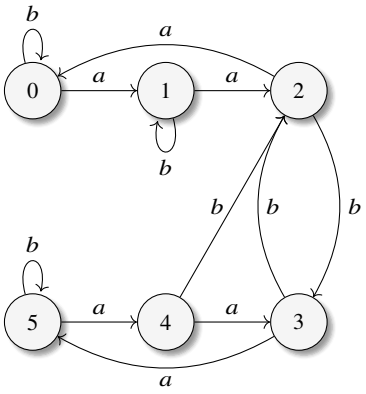
The automaton **H**



The automaton **I**



Roman's automaton **J**



Kari's automaton **K**.

Fig. 15.1: The eight exceptional automata whose reset lengths equal the Černý bound $(n-1)^2$. The naming is taken from [286] (a mistake for **J** in [286] has been fixed here, the correct automaton is written in [613]).

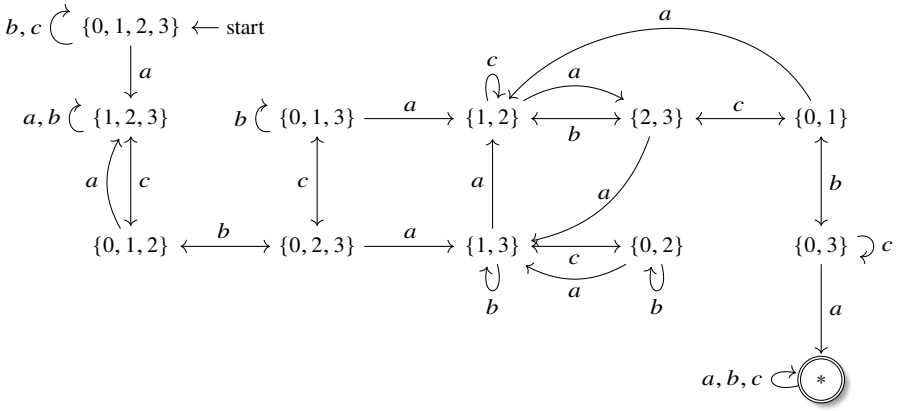


Fig. 15.2: A minimal automaton for the set of synchronizing word of the automaton **I** from Figure 15.1. See the proof of Theorem 15.1 for details.

I: The word b induces a transposition and the word cb induces the cycle (0 1 2 3). Thus by Theorem 5.40 the transformation semigroup contains the full symmetric group. Since the full symmetric group is primitive, from Theorem 14.9 we find that the automaton is sync-maximal.
For an alternative proof, consider the power automaton with the singleton sets merged into a single state and the empty set omitted shown in Figure 15.2. We see that **I** is completely reachable. Now, the words $u_1 = aacba$, $u_2 = au_1$, $u_3 = bu_2$, and $u_4 = cu_3$ map $\{0,1,3\}$, $\{0,2,3\}$, $\{0,1,2\}$, and $\{1,2,3\}$, respectively, to a singleton while mapping the state not in the respective set to another distinct state. So, by Theorem 9.46, Item 4, the automaton **I** is completely distinguishable. By Theorem 10.7, **I** is sync-maximal.

J: We have the following transitions from the whole state set (omitting transitions into sets with less than four elements):

$$\{0,1,2,3,4\} \xrightarrow{a} \{1,2,3,4\} \overset{a}{\circlearrowright} \xleftrightarrow{b} \{0,1,3,4\} \xleftrightarrow{c} \{0,1,2,4\} \overset{b}{\circlearrowright}$$

Since $\{0,1,2,3\}$ is not reachable, **J** is not sync-maximal by Theorem 10.7.

K: All the reachable 5-sets are $\{0,1,2,3,5\}$,$\{0,1,2,4,5\}$, and $\{0,1,2,3,4\}$. Thus, not all 5-sets are reachalbe. Since, by Theorem 10.7, for strongly connected automata, complete reachability is a necessary condition for sync-maximality, **K** is not sync-maximal. □

Remark 15.2 In **J**, the letters b and c restricted to $\{0,2,3\}$ generate the full symmetric group on $\{0,2,3\}$. Since a maps $\{0,1,2\}$ to $\{1,2\}$, we can conclude that the subsets of the form $A \cup \{1,4\}$ with $A \subseteq \{0,2,3\}$ form a family of reachable and pairwise distinguishable subsets.

In **K**, the letters a and bb induce a subautomaton on $\{3,4,5\}$ that is isomorphic to the Černý automaton on three states and since a and b permute $\{0,1,2\}$ the sets

of the form $A \cup \{0, 1, 2\}$ with $A \subseteq \{3, 4, 5\}$ form a family of reachable and pairwise distinguishable subsets.

15.2 Families of Slowly Synchronizing Automata

Here, we present a couple of slowly synchronizing families of automata from the literature and investigate the size of their minimal automata for the set of synchronizing words.

Table 15.1 shows slowly synchronizing automata which we show to be sync-maximal in Theorem 15.3. We only give the state diagrams, the formal definitions should be clear from these diagrams. For example, the automata family $\mathcal{K}_n = ([n], \Sigma, \delta_n)$ ($n > 5$ odd) is defined[2] by

$$\begin{aligned} \delta_n(i, b) &= (i + 1) \bmod n \text{ for } i \in \{0, \ldots, n-1\}, \\ \delta_n(i, a) &= (i + 1) \bmod n \text{ for } i \in \{1, \ldots, n-3, n-1\}, \\ \delta_n(n-2, a) &= 1 \text{ and } \delta_n(0, a) = 3. \end{aligned}$$

For the families from Table 15.1, except for $\mathcal{K}_n$ (for which this was an open problem), sync-maximality has been shown in [441] by rather involved arguments.[3] By our general results this result follows readily.

Theorem 15.3 *The automata from the families listed in Figure 15.1 are sync-maximal.*

Proof. Let $\mathcal{A} = (Q, \{a, b\}, \delta)$. For $\mathcal{A} \in \{\mathcal{C}_n, \mathcal{V}_n, \mathcal{L}_n\}$ there exists a state q (marked in Table 15.1) such that $\delta(q, a) = \delta(q, ba)$. As the letter b cyclically permutes the states and a has defect one, these automata are sync-maximal by Theorem 12.31 (for $\mathcal{C}_n$, this was also established in Theorem 10.4 before). For $\mathcal{A} \in \{\mathcal{F}_n, \mathcal{K}_n\}$ there exists a state q such that $\delta(q, a) = \delta(q, b^2a)$. As in this case n is odd, a has defect one, and the letter b cyclically permutes the states, sync-maximality follows as well from Theorem 12.31. □

Theorem 15.4 *The automata $\overline{\mathcal{M}}_n$ (n even and $n \geq 8$) from Figure 15.3a introduced in [435][4] are not sync-maximal but a minimal automaton for* $\mathrm{Syn}(\mathcal{M}_n)$ *has at least* $2^{n/2}$ *states.*

Proof. Let $\overline{\mathcal{M}}_n = (Q_n, \{a, b\}, \delta_n)$ with $Q_n = [2m]$ where $m \geq 4$. Since 0 is a trap state, $\overline{\mathcal{M}}_n$ is not sync-maximal by Theorem 10.13. Set $P = \{m, m+1, \ldots, 2m-1\}$. In the following, we write δ for δ_n for simplicity. Then

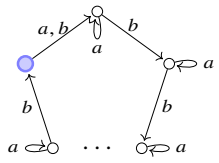
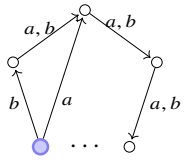
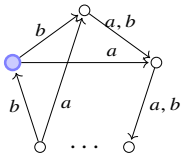
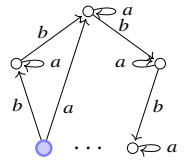
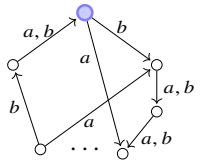
---

[2] The numbering of the states w.r.t. the letter a has been changed compared to [441].

[3] For $\mathcal{V}_n$ sync-maximality follows readily by considering a^{n-1} and b as letters, as the resulting automaton is isomorphic to the Černý automaton.

[4] In [612] the notation $\mathcal{M}_n$ was used for this family, that we already used for the family from Figure 15.3b. In [435] the notation $\mathcal{A}_n$ has been used, but we prefer to use $\mathcal{A}_n$ to denote a general family of automata.

Table 15.1: Slowly synchronizing and sync-maximal automata. Names from the literature and reset lengths are indicated. The marked state is used to deduce sync-maximality via Theorem 12.31 in the proof of Theorem 15.3.

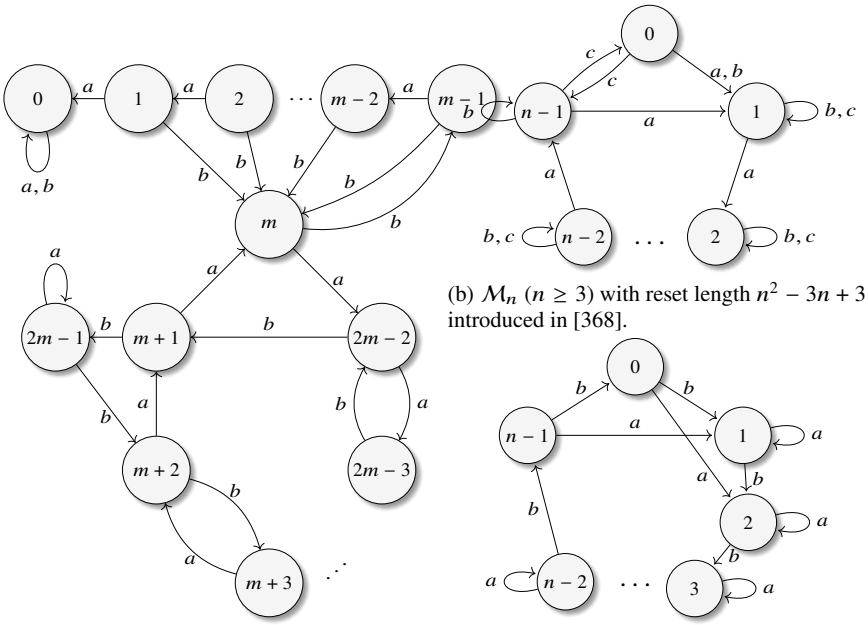
Name	Automata	Reset Length	Referenz
C_n		$(n-1)^2$	[21, 140, 441]
$\mathcal{V}_n = \mathcal{W}_n$		$n^2 - 3n + 3$	[21, 441]
$\mathcal{L}_n = \mathcal{D}'_n$		$n^2 - 3n + 4$	[21, 441]
$\mathcal{F}_n$ $n > 3$ odd		$n^2 - 3n + 3$	[21, 441]
$\mathcal{K}_n = \mathcal{G}_n$ $n > 5$ odd		$n^2 - 4n + 7$	Thm. 15.3 & [21, 441]

$(\delta_{babab})_{|P} = (m\ m+1\ 2m-1\ m+2\ \cdots\ 2m-2)$ and $(\delta_a)_{|P} = (2m-2\ \cdots\ m+1\ m)$.

Set $u = ba^{m-1}$. Then we have $\delta(m,u) = 0$ and, since $\delta(q,a^{m-1}) = q$ for each $q \in P$, we have $\delta(q,u) = \delta(q,b)$ for $q \in P \setminus \{m\}$.

Claim 1: Every subset of the form $S \cup \{0\}$ with $S \subseteq P$ is reachable.

Proof of Claim 1: We do induction on $|P \setminus S|$. For $|P \setminus S| = 0$, we have $S = P$ and $\delta(Q_n,u) = \{0\} \cup P$. Next let $S \subsetneq P$ be a proper subset and $q \in P \setminus S$. Set $T = \{m+1,\ldots,m+|S|\}$. Inductively, we find $\delta(Q_n,v) = T \cup \{m\} \cup \{0\}$ for some $v \in \{a,b\}^*$. This gives $\delta(Q_n,vu) = \delta(T,u) \cup \{0\}$. Since $T \subseteq \{m+1,\ldots,2m-1\}$ and b permutes the latter set and a permutes P, we have $|\delta(T,u)| = |T| = |S|$. By Theorem 5.40, there exists a word $w \in \{a, babab\}^*$ inducing a permutation on P mapping $\delta(T,u)$ onto S. Then $\delta(Q_n,vuw) = S \cup \{0\}$. *[End, Proof of Claim 1]*



(a) $\overline{\mathcal{M}_n}$ (n even and $n \geq 8$) from [435] with reset length $\frac{n^2+6n-16}{4}$.

(b) $\mathcal{M}_n$ ($n \geq 3$) with reset length $n^2 - 3n + 3$ introduced in [368].

(c) $\mathcal{B}_n$ ($n \geq 5$ odd) from [21, 25] with reset length $n^2 - 3n + 2$.

Fig. 15.3: Slowly synchronizing families whose automata are not sync-maximal, but minimal automata for their sets of synchronizing words are exponentially larger in the number of states. See the main text for details.

Claim 2: Distinct subsets $S \cup \{0\}, T \cup \{0\}$ with $S, T \subseteq P$ are distinguishable.

Proof of Claim 2: We do induction on $\min\{|S|, |T|\}$. If one set is empty, the other is non-empty and so the empty word distinguishes $S \cup \{0\}$ and $T \cup \{0\}$. Otherwise, suppose both sets are non-empty. Without loss of generality, let $|S| \leq |T|$. Pick some $q \in S$. Since $|S| \leq |T|$, in both cases $q \in T$ or $q \notin T$, we have $S \setminus \{q\} \neq T \setminus \{q\}$. Note that also $T \setminus \{q\} \neq \emptyset$. Let $v \in \{babab\}^*$ be such that $\delta(q, v) = m$. Set $S' = \delta(S, v)$ and $T' = \delta(T, v)$. We have $m \in S'$, $\delta(S', u) = \delta(S' \setminus \{m\}, u) \cup \{0\}$, and $S' \setminus \{m\} \neq T' \setminus \{m\}$. Now, since u acts on $P \setminus \{m\}$ in the same way as b, we find

$$\delta(S' \setminus \{m\}, u) \neq \delta(T' \setminus \{m\}, u)$$

and both subsets are contained in $P \setminus \{m\}$. Since $|\delta(S' \setminus \{m\}, u)| < |S|$ as $m \in S'$ and $|S'| = |S|$, by induction hypothesis, there exists $w \in \{a, b\}^*$ distinguishing $\delta(S' \setminus \{m\}, u) \cup \{0\}$ and $\delta(T' \setminus \{m\}, u) \cup \{0\}$. We have $\delta(T' \setminus \{m\}, u) \cup \{0\} = \delta(T', u) \cup \{0\}$ since $\delta(m, u) = 0$. Now vu maps $S \cup \{0\}$ and $T \cup \{0\}$ to $\delta(S' \setminus \{m\}, u) \cup \{0\}$ and $\delta(T', u) \cup \{0\}$, respectively, and so we find that vuw distinguishes S and T. *[End, Proof of Claim 2]*

By Claims 1 & 2, the $2^{n/2}$ sets of the form $S \cup \{0\}$ for $S \subseteq P$ are reachable and pairwise distinguishable. Then Theorem 9.10 gives the statement. □

Theorem 15.5 *Consider the automata $\mathcal{M}_n$ from Figure 15.3b introduced in [368]. The minimal automaton for* $\mathrm{Syn}(\mathcal{M}_n)$ *has precisely* $2^{n-1} + 2^{n-2} - n + 1$ *states.*

Proof. Let $\mathcal{M}_n = (Q_n, \{a, b, c\}, \delta_n)$ as in Figure 15.3 where $Q_n = [n]$ and let $S = \{1, \ldots, n-1\}, T = \{0, \ldots, n-2\}$, and $u = cb$ and $v = ac$. When considering the words $\{a, u\}$ and $\{v, b\}$ as letters on the state sets S and T, respectively, we obtain automata isomorphic to the Černý automaton C_{n-1}. Since $\delta_n(Q_n, b) = S$ and $\delta_n(Q_n, v) = T$, and the Černý automaton is completely reachable and completely distinguishable by Theorems 9.13 and 9.43, we obtain that all non-empty subsets of S and T are reachable and pairs of distinct non-empty and non-singleton subsets of these sets are distinguishable.

Q_n is distinguishable from every other set: Let $A \subseteq Q_n$. For $A \subseteq T$, we have $\delta_n(A, b) \subseteq S \setminus \{n-1\}$ and $\delta_n(Q_n, b) = S$, and so both subsets are distinguishable by a word $w \in \{a, u\}^*$. Then bw distinguishes A and Q_n. For $A \subseteq S$, we have $\delta_n(A, c) \subseteq T$ and $\delta_n(Q_n, c) = Q_n$ and as shown before there exists a word w distinguishing both subsets and so cw distinguishes A and Q_n.

Distinct $A \subseteq S$ and $B \subseteq T$ with $|A|, |B| \geq 2$ are distinguishable: We have $\delta_n(B, b) \subseteq S, n-1 \notin \delta_n(B, b)$, and $\delta_n(A, b) = A$. Thus, for $n-1 \in A$, we have $\delta_n(A, b) \neq \delta_n(B, b)$ and so since $|\delta_n(A, b)| \geq 2$ there exists a word in $\{a, u\}^*$ distinguishing $\delta_n(A, b)$ and $\delta_n(B, b)$. For $n - 1 \notin A$, we have $\delta_n(B, c) = (B \setminus \{0\}) \cup \{n-1\}$ and $\delta_n(A, c) = A$ so that $\delta_n(B, c)$ and $\delta_n(A, c)$ are distinct subsets of S of size at least two. So some word in $\{a, u\}^*$ distinguishes both subsets.

In conclusion, we obtain that two non-empty and distinct subsets of states of size at least two which contain at most one state from $\{0, n-1\}$ or equal the whole state set are distinguishable and each such set is reachable.

Now, for $A \subseteq Q_n$ with $0 \notin A$ or $n - 1 \notin A$, inductively, we obtain $0 \notin \delta_n(A, w)$ or $n-1 \notin \delta_n(A, w)$ for all $w \in \Sigma^*$. Since $\delta_n(Q_n, a) = \delta_n(Q_n, b) = S$ and $\delta_n(Q_n, c) = Q_n$, we find that for each $w \in \Sigma^*$ with $\delta_n(Q_n, w) \neq Q_n$ we have $0 \notin \delta_n(Q_n, w)$ or $n - 1 \notin \delta_n(Q_n, w)$. Thus, no proper subset that contains $\{0, n-1\}$ is reachable.

Summarizing, we conclude that $\mathcal{F} = \{A \subseteq Q : |A| \geq 2, |\{0, n-1\} \cap A| \leq 2\} \cup \{\{0\}\} \cup \{Q_n\}$ is a maximal family of pairwise distinguishable and reachable subsets of states. Since there are $2^{n-1} - (n-1) - 1$ subsets of size at least two of S and 2^{n-2} subsets of T that contain 0, we find that $\mathcal{F}$ has size $2^{n-1} + 2^{n-2} - n + 1$. Then we get the claim from Theorem 9.10. □

Theorem 15.6 *Consider the family of automata given by $\mathcal{B}_n$ with $n \geq 5$ being odd from Figure 15.3c introduced in [21, 25]. The automata $\mathcal{B}_n$ are not sync-maximal and for $n \equiv 0 \pmod 5$, the minimal automaton for* $\mathrm{Syn}(\mathcal{B}_n)$ *has at least* $2^{n/5}$ *states.*

Proof. Let $\mathcal{B}_n = (Q_n, \{a, b\}, \delta_n)$ with $Q_n = [n]$. By Theorem 10.7, $\mathcal{B}$ is not sync-maximal since it has no letter of defect one. Suppose n is odd and $n \equiv 0 \pmod 5$. Set $R_r = \{q \in Q \mid q \equiv r \pmod 5\}$ for $r \in \{0, 1, 2, 3, 4\}$. Then $bbbbb$ cyclically permutes R_r for $r \in \{0, 1, 2, 3, 4\}, \delta_n(R_1, a) = R_1, \delta_n(R_2, a) = R_2$, and $\delta_n(R_3, a) = R_3$.

Claim 1: A subset of the form $S \cup \delta_n(S, b) \cup R_1 \cup R_2 \cup R_3$ with $S \subseteq R_4$ is reachable.

Proof of Claim 1: We do induction on $|R_4 \setminus S|$. For $S = R_4$, we have $Q_n = S \cup \delta_n(S, b) \cup R_1 \cup R_2 \cup R_3$, which is reachable by the empty word. Next, suppose there exists $q \in R_4 \setminus S$. Then for $S' = \delta_n(S, b^{n-1-q})$ we have $n-1 \notin S'$ and $0 \notin \delta_n(S', b)$. Set $T = S' \cup \delta_n(S', b) \cup R_1 \cup R_2 \cup R_3$. Since $\{1, 2\} \subseteq T$ we get

$$\delta_n^{-1}(T, a) = \{n-1, 0\} \cup T = (S' \cup \{n-1\}) \cup \delta_n(S' \cup \{n-1\}, b) \cup R_1 \cup R_2 \cup R_3.$$

Inductively, since $S' \cup \{n-1\} \subseteq R_4$, there exists $w \in \Sigma^*$ with $\delta_n(Q_n, w) = \{n-1, 0\} \cup T$. Then $\delta_n(Q_n, wa) = T$ and $\delta_n(T, b^{q+1}) = S \cup \delta_n(S, b) \cup R_1 \cup R_2 \cup R_3$. *[End, Proof of Claim 1]*

Claim 2: Every pair of distinct subsets of the form $S \cup \delta_n(S, b) \cup R_1 \cup R_2 \cup R_3$ with $S \subseteq R_4$ is distinguishable.

Proof of Claim 2: Let $u \in \Sigma^*$ be a shortest synchronizing word. Then we can write $u = av$ with $v \in \Sigma^*$ where $\delta_n(Q_n \setminus \{n-1, 0\}, v) = \{p\}$ and $|\delta_n(Q_n, v)| \geq 2$. Let $S, T \subseteq R_4$ be distinct and $S \neq \emptyset$. Pick some $i \geq 0$ such that $n-1 \in \delta_n(S, u^i)$. Set

$$R = \delta_n(S, u^i) \cup \delta_n(S, u^i b) \cup R_1 \cup R_2 \cup R_3,$$
$$K = \delta_n(T, u^i) \cup \delta_n(T, u^i b) \cup R_1 \cup R_2 \cup R_3.$$

As $0 \in \delta_n(S, u^i b)$, we have $\{n-1, 0\} \subseteq R$, $R \setminus \{n-1, 0\} \neq \emptyset$, and $\{n-1, 0\} \cap K = \emptyset$. Hence $\delta_n(K, v) = \{p\}$ and $\delta_n(R, v) = \{\delta_n(n-1, v), \delta_n(0, v), p\} = \delta_n(Q_n, v)$. Then, $u^i v$ distinguishes $S \cup \delta_n(S, b) \cup R_1 \cup R_2 \cup R_3$ from $T \cup \delta_n(T, b) \cup R_1 \cup R_2 \cup R_3$. *[End, Proof of Claim 2]*

Finally, by Claim 1 & 2 and as $|R_4| = n/5$ we find that $2^{n/5}$ subsets are reachable and pairwise distinguishable. So, Theorem 9.10 gives the claim. □

Remark 15.7 We remark that two distinct subsets S, T which are unions of sets from

$$\{\{0, 1\}, \{2, 3\}, \ldots, \{n-3, n-2\}\}$$

with $|S|, |T| \geq 3$ are distinguishable in $\mathcal{B}_n$. To show this, consider a shortest synchronizing word, which must have the form av for some $v \in \Sigma^*$. By the minimal choice of av, with the notation from the proof of Theorem 15.6, we have $\delta_n(Q_n \setminus \{n-1, 0\}, v) = \{p\}$ for some state p and $|\delta_n(Q_n, v)| \geq 2$, and so at least one of both states 0 or $n-1$ is not mapped to p. Now suppose, without loss of generality, $\{q, q+1\} \subseteq S$ and $\{q, q+1\} \cap T = \emptyset$ for some state q with $q \equiv 0 \pmod{n}$. Then for $i = (n-1) - q$ we have

$$\{n-1, 0\} \subseteq \delta_n(S, b^i) \text{ and } \{n-1, 0\} \cap \delta_n(T, b^i) = \emptyset.$$

Thus $\delta_n(T, b^i v) = \{p\}$ and, since $|\delta_n(S, b^i)| \geq 3$ we have $\delta_n(S, b^i) \cap \{1, \ldots, n-2\} \neq \emptyset$ and so $|\delta_n(S, b^i v)| \geq 2$.

Remark 15.8 Every non-empty set that is a union of sets from the family

$$\{\{i, j\} \subseteq [n] \mid i \equiv 2x \pmod{n} \text{ and } j \equiv 2x-1 \pmod{n} \text{ for some } x \geq 0\}$$

is reachable in $\mathcal{B}_n$ (the given family equals the orbit of $\langle \delta_{bb} \rangle$ on $\{0, n-1\}$). Next, we show this inductively using the notation from the proof of Theorem 15.6. It is clearly true for the whole state set. Consider a proper and non-empty subset A of states which is a union of these sets. Then there exist states i_1, i_2, j_1, j_2 with $i_2 \equiv i_1 - 1 \pmod{n}$ and $j_2 \equiv j_1 - 1 \pmod{n}$ such that $\{i_1, i_2\} \cap A = \emptyset$ and $\{j_1, j_2\} \subseteq A$ with $j_1 \equiv i_1 + 2 \pmod{n}$ and $j_2 \equiv i_2 + 2 \pmod{n}$. Since n is odd, the word bb cyclically permutes the states. Consider the word $u = (bb)^{n-i_1} a (bb)^{i_1}$. We have $\delta_n(\{i_1, i_2\}, u) = \{j_1, j_2\}$ and $\delta_n(i, u) = i$ for all $i \in Q_n \setminus \{i_1, i_2\}$. Thus, $\delta_n^{-1}(A, u) = A \cup \{i_1, i_2\}$ and $\delta_n^(A \cup \{i_1, i_2\}, u) = A$. Inductively, we have $\delta_n(Q_n, v) = A \cup \{i_1, i_2\}$ for some $v \in \Sigma^*$. Hence, vu maps the whole states set onto A.

15.3 Discussion

Černý [140] introduced the Černý family from Figure 1.7, which forms a family of slowly synchronizing automata. Only the eight automata from Section 15.1 are known to have the same reset length, see also [613]. A little more than forty years later, in [25] the authors gave two slowly synchronizing families of automata have with a letter of defect two, one of them was the family $\mathcal{B}_n$ given in Figure 15.3c. In [435] the slowly synchronizing family of binary automata with a sink state $\overline{\mathcal{M}}_n$ from Figure 15.3a was introduced. For this class of automata, an improved lower bound was given in [612] (based on a construction from [622]). A construction for an arbitrary alphabet was given in [500]. A general method to obtain slowly synchronizing families of automata based on labeling edges of primitive digraphs with alphabet symbols was investigated in [20, 21]. Slowly synchronizing automata families having three, four, and five letters were given in [184]. Volkov [616] gave a slowly synchronizing family of n-state automata all of whose letters are idempotents of rank $n/2$.

The relation of slowly synchronizing automata and the size of automata for their sets of synchronizing words was first systematically studied by Maslennikova [440, 441]. She proved Theorem 15.3, except for $\mathcal{K}_n$, which was left open as a conjecture in [441] (which we confirmed here, a result published in [307]). We note that the proofs given in [440, 441] were all quite involved, whereas here this result followed rather easily from more general statements we gave.

We proved that many families of slowly synchronizing automata have exponentially larger automata for their sets of synchronizing words. We call an infinite family of distinct automata $\mathcal{A}_n = (Q_n, \Sigma, \delta_n)$ *almost sync-maximal*, if there exists $C > 0$ such that the minimal automata for $\mathrm{Syn}(\mathcal{A}_n)$ have at least $2^{C \cdot |Q_n|}$ states. Similarly, we call an individual n-state automaton *almost sync-maximal* if the minimal automaton has size close to $2^n - n$ (this being an informal definition similarly to the definition for individual slowly synchronizing automata).

Note that since the individual automata are over a fixed alphabet, an almost sync-maximal infinite family of distinct automata never consists solely of commutative automata by Theorem 12.5.

The evidence suggests that being slowly synchronizing is related to being almost sync-maximal. As a shortest synchronizing word is a path in an accepting automaton for the set of synchronizing words, when we need long synchronizing words, we might expect accepting automata for the sets of synchronizing words to be "complex" or "large". These consideration lead us to our first conjecture.

Conjecture: Every slowly synchronizing family of automata is almost sync-maximal.

However, the converse does not hold. Theorems 15.9 and 15.10 show how to obtain sync-maximal automata with short synchronizing words.

Example 15.9 Adding letters to a sync-maximal or almost sync-maximal automaton retains these properties, but may alter the reset length. For example, adding a letter that maps everything to a single state yields a reset length of one.

Example 15.10 Consider the family of binary n-state automata from Figure 15.4. The automata from this family are sync-maximal since by considering the word $u = b^{n-1}$ as a new letter and $\{u, a\}$ as an alphabet, the resulting automata over $\{u, a\}$ are isomorphic to the Černý automata $\mathscr{C}_n$ and induce the same transformation monoids as b is a permutational letter. However, this family is not slowly synchronizing since $(ab)^n$ is a synchronizing word. Note that removal of any letter results in a non-synchronizing automaton for $n \geq 3$.

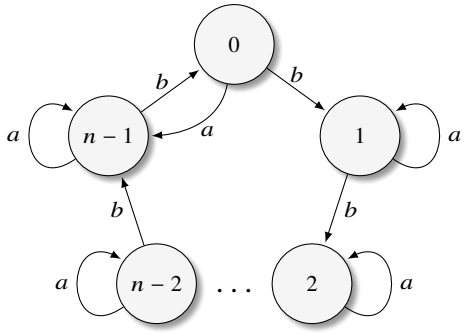


Fig. 15.4: Sync-maximal n-state automata with synchronizing words $(ab)^n$, see Theorem 15.10.

Now, for two automata $\mathcal{A}$ and $\mathcal{B}$ with $\mathcal{T}_M(\mathcal{A}) = \mathcal{T}_M(\mathcal{B})$ the state complexities of $\mathrm{Syn}(\mathcal{A})$ and $\mathrm{Syn}(\mathcal{B})$ are equal as the properties of reachability of subsets of states and distinguishability of pairs of subsets of states only depend on the transformation monoid. For example, adding a letter synchronizing the automaton in Theorem 15.9 does not change the transformation monoid, or considering $\{u, a\}$ as an alphabet in Theorem 15.9. In view of these observations, we conjecture that a slowly synchronizing family "hides inside" an almost sync-maximal family.

Conjecture: For every almost sync-maximal n-state family of automata $\mathcal{A}_n$ there exists a slowly synchronizing family of automata $\mathcal{B}_n$ with state sets of size at most n such that $\mathcal{T}_M(\mathcal{B}_n)$ is isomorphic to a subsemigroup of $\mathcal{T}_M(\mathcal{A}_n)$ for all $n \geq 1$?

By Theorem 10.15, sync-maximal automata have at least two letters and binary sync-maximal automata are circular and have a letter of defect one. So, as the simplest class of automata containing sync-maximal automata, it is natural to first try to confirm both conjectures from the present section for binary circular automata having a letter of defect one.

The following statement confirms the above conjecture for families of binary sync-maximal automata, where we even have an isomorphism to the whole transformation monoid of $\mathcal{A}_n$ if one letter is a simple idempotent.

Proposition 15.11 *Let $\mathcal{A}_n$ be a binary sync-maximal automaton $\mathcal{A}_n$ with state set $Q = [n]$ where $n \geq 3$.*

1. *If one letter is a simple idempotent, then $\mathcal{T}_M(\mathcal{A}_n)$ is isomorphic to $\mathcal{T}_M(C_n)$ where C_n denotes the n-state automaton from the Černý family.*
2. *$\mathcal{T}_M(\mathcal{A}_n)$ contains a subsemigroup isomorphic to $\mathcal{T}_M(C_m)$ for some divisor m of n with $m > 1$.*
3. *If n is prime, then $\mathcal{T}_M(C_n)$ is isomorphic to a subsemigroup of $\mathcal{T}_M(\mathcal{A}_n)$.*

Proof. Since $\mathcal{A}_n$ is sync-maximal, by Theorem 10.15 we have $\Sigma = \{a, b\}$ where a has defect one and b cyclically permutes the states. First, assume a is a simple idempotent mapping q to a distinct state $\delta(q, b^i)$ for some $i \in \{1, \ldots, n-1\}$. Let $u = b^i$. The word u cyclically permutes the states and the automaton we obtain when considering $\{a, u\}$ as letters equals, up to renaming of the states and letters, C_n (and so has a transformation monoid isomorphic to $\mathcal{T}_M(C_n)$) and, since b permutes the states, has the same transformation monoid as $\mathcal{A}_n$.

Next, consider the general case. By Theorem 5.13 there exists a word v inducing a simple idempotent on the states that maps a state q to a distinct state $\delta(q, b^j)$ for some $j \in \{1, \ldots, n-1\}$. Set $w = b^j$. The word w cyclically permutes the states $S \subseteq Q$ that are multiples of j. So, $|S|$ divides $|Q|$ and we have $|S| > 1$. The automaton resulting when considering the words $\{w, v\}$ as letters on the state set S equals, up to renaming of the states and letters, C_m for $m = |S|$. Thus the transformation semigroup of C_m can be identified with the subsemigroup of $\mathcal{T}_M(\mathcal{A}_n)$ generated by w and v.

Now, if n is prime, we have $S = Q$ in the previous argument, so that $m = n$ and C_n also acts on the same state set Q. Note that $\mathcal{T}_M(C_n)$ does not has to be a subset of $\mathcal{T}_M(\mathcal{A}_n)$ since the states in the automaton over $\{w, v\}$ are numbered according to w and not b, but is only isomorphic to a subsemigroup. □

The second statement from Theorem 15.11 for non-prime n can be seen in the automaton from Theorem 11.30 where we find a subsemigroup in its transformation monoid isomorphic to $\mathcal{T}_M(C_2)$.

We obtain a stronger variant of the last conjecture if we demand that the automata $\mathcal{B}_n$ also have n states.

Next, we give a family of binary automata where the state complexities of the sets of synchronizing words have superpolynomial growth and the reset lengths of all automata having the same transformation monoids are linear. So, the above conjecture is false when we only demand superpolynomial growth.

Example 15.12 Starting from the Černý automata $C_n = ([n], \{a, b\}, \delta_n)$ for $n \geq 3$, here, we construct a family of m-state automata $\mathcal{A}_m = ([m], \{a, b\}, \mu_m)$ with $m = (n-1)^2$ where the state complexity of $\mathrm{Syn}(\mathcal{A}_m)$ is $2^{\sqrt{m}+1} - \sqrt{m} - 1$ and such that for every family of automata $\mathcal{B}_m$ with $\mathcal{T}_M(\mathcal{B}_m) = \mathcal{T}_M(\mathcal{A}_m)$ we have $\mathrm{rt}(\mathcal{B}_m) \leq m$. For $q \in [m]$ and $x \in \{a, b\}$ we set

$$\mu_m(q, x) = \begin{cases} \delta_n(q, x) & \text{if } q \in [n], \\ 1 & \text{otherwise.} \end{cases}$$

We have $\mathrm{Syn}(\mathcal{A}_m) = \mathrm{Syn}(C_n)$. So, a minimal automaton for $\mathrm{Syn}(\mathcal{A}_m)$ has size $2^{\sqrt{m}+1} - (\sqrt{m} + 1)$ and the reset length of $\mathcal{A}_m$ is m. Now, let $\mathcal{B} = ([m], \Gamma, \rho)$ be with $\mathcal{T}_M(\mathcal{B}) = \mathcal{T}_M(\mathcal{A}_m)$. Set $S = [n]$. Then

1. S is an upper set in $\mathcal{A}_m$ and in $\mathcal{B}$,
2. $t(q) \in S$ for every $t \in \mathcal{T}_S(\mathcal{A}_m)$,
3. $\mathcal{A}_{|S} = C_n$,
4. $\mathcal{T}_M(\mathcal{B}_{|S}) = \mathcal{T}_M((\mathcal{A}_m)_{|S})$.

The group of units of $\mathcal{T}_M(C_n)$ is generated by a permutation cyclically permuting S. As $\mathcal{T}_M(\mathcal{B}_{|S}) = \mathcal{T}_M((\mathcal{A}_m)_{|S})$, we find that $\mathcal{B}_{|S}$ must be a circular automaton. So, the reset length of $\mathcal{B}_{|S}$ is at most $(n-1)^2$ as the Černý conjecture has been confirmed for circular automata [187]. Hence, the reset length of $\mathcal{B}$ is at most $m + 1$.

Part IV
The Constrained Synchronization Problem

"Du glaubst nicht, wie müde ich bin, wie satt ich die Chemie habe, wie namentlich gewisse Teile mich ordentlich anekeln, mir wenigstens so langweilig sind, dass ich gähnen muss, wenn ich daran denke. Sind wir denn schon so alt, oder wie ist das? Diese nervenschwächende Wirkung muss wirklich der Chemie eigentümlich sein. Ich glaube, die materiellen Influenzen, die Dämpfe, Gerüche und all die Teufelsstinkereien haben großen Anteil daran. Besonders ist es das Praktikum, was einen so herunterbringt."

–Friedrich Wöhler (1800–1882)
Aus einem Brief an Justus von Liebig (22.07.1847)

Chapter 16
Motivational Examples

Vladimir Gusev [271] introduced the constrained synchronization problem in connection with a reduction procedure on automata that preserves the reset length up to plus/minus one w.r.t. a given constraint language. We discuss this in Section 16.1. The problem of constrained synchronization also appears implicitly in a reduction used by Martyugin [438]. Here, we give details on the procedure from [271] and on the reduction used in [438] in Section 16.2. Then we outline a relation to the *Skolem problem* [94, 473] and the *matrix orbit problem* [354] in Section 16.3. Finally, we sketch possible applications that naturally extend areas where synchronizing automata have been used previously in Sections 16.4 to 16.6.

16.1 Use in a Reduction Procedure

Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a semiautomaton with n states over a binary alphabet $\Sigma = \{a, b\}$. A set of matrices X_1, X_2, Y_1, Y_2 over $\{0, 1\}$ is an *r-decomposition* [271] of $\mathcal{A}$ if

1. X_1 and Y_1 are $n \times r$ matrices,
2. X_2 and Y_2 are $r \times n$ matrices,
3. $[a] = X_1X_2$, $[b] = Y_1Y_2$,
4. ev ery row of X_1, X_2, Y_1, Y_2 has only one occurrence of 1.

The automaton $\mathcal{A}$ admits an r-decomposition if and only if $r \geq \max\{\mathrm{rank}(a), \mathrm{rank}(b)\}$ [271]. By the first three conditions X_1, Y_1 and X_2, Y_2 constitute *rank factorizations* of $[a]$ and $[b]$, respectively, as discussed in [494]. Since the matrices X_1, X_2, Y_1, Y_1 have one 1 in each row, the same holds for their products. Hence, we obtain a complete and deterministic r-state automaton $\mathcal{A}_{X_1,X_2,Y_1,Y_2} = (Q_{X_1,X_2,Y_1,Y_2}, \Gamma, \delta_{X_1,X_2,Y_1,Y_2})$ by setting $Q_{X_1,X_2,Y_1,Y_2} = \{1, \ldots, r\}$ and $\Gamma = \{a_{X_2Y_1}, a_{X_2X_1}, a_{Y_2X_1}, a_{Y_2Y_1}\}$ where the action of the letters is given by $[a_{X_2Y_1}] = X_2Y_1$, $[a_{X_2X_1}] = X_2X_1$, $[a_{Y_2X_1}] = Y_2X_1$ and $[a_{Y_2Y_1}] = Y_2Y_1$. The following was shown by Gusev [271, Prop. 5].

Proposition 16.1 *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be an automaton with r-decomposition X_1, X_2, Y_1, Y_2 and $\mathcal{A}_{X_1,X_2,Y_1,Y_2} = (Q_{X_1,X_2,Y_1,Y_2}, \Gamma, \delta_{X_1,X_2,Y_1,Y_2})$. Then $\mathcal{A}$ is synchronizing*

if and only if

$$Syn(\mathcal{A}_{X_1,X_2,Y_1,Y_2}) \cap R \neq \emptyset$$

with

$$\begin{aligned} R = \Gamma^* \setminus \big(&\Gamma^*(a_{X_2X_1}(a_{Y_2X_1} + a_{Y_2Y_1}))\Gamma^* + \Gamma^*(a_{X_2Y_1}(a_{X_2Y_1} + a_{X_2X_1}))\Gamma^* + \\ &\Gamma^*(a_{Y_2Y_1}(a_{X_2Y_1} + a_{X_2X_1}))\Gamma^* + \Gamma^*(a_{Y_2X_1}(a_{Y_2X_1} + a_{Y_2Y_1}))\Gamma^*\big), \end{aligned}$$

where, if this is the case, $m - 1 \leq \mathrm{rt}(\mathcal{A}) \leq m + 1$ *for* $m = \min\{|u| : u \in \mathrm{Syn}(\mathcal{A}_{X_1,X_2,Y_1,Y_2}) \cap R\}$.

So, the reset length of an automaton $\mathcal{A}$ without permutational letters can be bounded by the length of a shortest synchronizing word of a smaller automaton $\mathcal{A}_{X_1,X_2,Y_1,Y_2}$ contained in a constraint language independent of $\mathcal{A}$.

Gusev [271] used r-decompositions to construct n-state automata with reset length at least $r^2 - r - 1$. This was the sole purpose of this work and while the above statement used constrained synchronization, it was not introduced as a decision problem and questions of computational complexity were not investigated.

16.2 Implicit in a Reduction

Martyugin [438] proved that it is PSPACE-complete to decide whether a given partial automaton has a carefully synchronizing word (for the definition of these notions, see Section 17.2). His reduction constructs a partial automaton over Σ where a shortest carefully synchronizing word has the form $x\Gamma^*y$ for $x, y \in \Sigma$ and $\Gamma = \Sigma \setminus \{x, y\}$ [438, Lem. 1]. By adding self-loops to obtain a complete automaton in the construction, one can easily see that it has a synchronizing word in $x\Gamma^*y$ if and only if the original partial automaton has a carefully synchronizing word. In this sense, Sync$[x\Gamma^*y]$ is implicit in the construction.

16.3 The Matrix Orbit Problem and the Skolem Problem

The *(matrix) orbit problem* is the problem which asks for an $n \times n$ matrix A over $\mathbb{Q}$ and $x, y \in \mathbb{Q}^n$ whether $xA^i = y$ for some $i \geq 0$. This problem was shown to be solvable in polynomial time by Kannan & Lipton [354]. In [94, Cor. 1.3] the problem which asks for an $n \times n$ matrix A over $\mathbb{Z}$ and $1 \times n$ and $n \times 1$ vectors x and y over $\mathbb{Z}$, respectively, whether $xA^iy = 0$ for some $i \geq 0$ was shown to be NP-hard. As noted in [94], this implies NP-hardness of the *Skolem problem* (also called *Pisot's problem*) which ask whether an integer linear recurrent sequence has zero as a value, see [94], and also [473] for more information on the Skolem problem. Decidability of the Skolem problem is unknown [473].

Next, we point out a connection to the two problems above.[1] Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ with $Q = \{q_1, \ldots, q_n\}$ and $q \in Q$. Set

$$V_q = \{(y_1, \ldots, y_n) \mid y_j > 0 \text{ implies } q_j \in \delta^{-1}(q, a) \text{ for } j \in \{1, \ldots, n\}\},$$

and define numbers $x_1, \ldots, x_n$ and $z_1, \ldots, z_n$ by

$$x_i = \begin{cases} 1 & \text{if } q_i \in \delta(Q, a), \\ 0 & \text{otherwise,} \end{cases} \quad \text{and} \quad z_i = \begin{cases} 1 & \text{if } \delta(q_i, a) \neq q, \\ 0 & \text{otherwise,} \end{cases}$$

for $i \in \{1, \ldots, n\}$. Then for $i \geq 0$ we have

$$\delta(Q, ab^i a) = \{q\} \Leftrightarrow x \cdot [b]^i_{\mathcal{A}} \in V_q \Leftrightarrow x \cdot [b]^i_{\mathcal{A}} \cdot \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix} = 0$$

We will show later in Theorem 18.2, Item 4 that Sync$[ab^*a]$ is NP-complete. The above construction then also yields NP-hardness of the problem introduced above that asks whether $xA^i y = 0$ for a matrix A and vectors x, y over $\mathbb{Q}$. The construction also yields another proof that Sync$[ab^*a]$ is in NP since checking $x \cdot [b]^i_{\mathcal{A}} \in V_q$ for some $q \in Q$ is in NP by guessing $q \in Q$ and then checking $y \in V_q$ and $x[b]^i_{\mathcal{A}} = y$, which can be done in polynomial time since the matrix orbit problem can be solved in polynomial time.

Remark 16.2 In Theorem 8.27 we proved that ReachFrom is solvable in polynomial time for commutative automata over a fixed alphabet. The result of Kannan & Lipton [354] stating polynomial-time solvability of existence of some $i \geq 0$ such that $xA^i = y$ for a matrix A and vectors x, y over $\mathbb{Q}$ also implies polynomial-time solvability of ReachFrom for unary automata similarly as above by considering the transition matrix and encoding state sets S into vectors $(x_1, \ldots, x_n)$ by setting $x_i = 1$ if $q_i \in S$ and $x_i = 0$ otherwise. Note that the related problem SetTrans is NP-complete for unary automata by Theorem 8.18.

16.4 Operating Restrictions

Fernau et al. [215] gave the following motivating example. Suppose that a system, modeled by an automaton, has a "normal" and a "directing" operating mode, and only in the latter mode synchronizing instructions are allowed. The switching between both modes can be modeled, in its simplest form, by the constraint ab^*a, where a switches into the "directing" mode. As written in the previous paragraph, Sync$[ab^*a]$ is NP-complete, see Theorem 18.2, Item 4. Even more generally, it might be possible that a system, for example, a remotely controlled rover on a distant planet or a satellite in

[1] These connections were originally pointed out by Vladimir V. Gusev (personal communication).

orbit, is not allowed to execute all commands in every possible order. For example, some commands are only allowed in a certain order or after other commands have been executed. All of this imposes constraints on the possible reset sequences.

16.5 Part Orienters

Generalizing the part orienters example from Section 1.3.3, suppose some parts arrive at a manufacturing site and they need to be sorted and oriented before assembly. Practical considerations favor methods which require little or no sensing, employ simple devices, and are as robust as possible. As in Section 1.3.3, the parts are put on a conveyor belt so that the stream of the parts encounter a series of passive obstacles placed along the belt before reaching the assembly point. Now, furthermore, assume, due to the availability of components, requirements how to lay things out or physical restrictions, the passive components can not be placed at random along the belt, but have to obey some restrictions, or restrictions in what order they are allowed to happen. These constraints on the permissible sequences of applicable operations can be modeled by the constrained synchronization problem.

16.6 Supervisory Control

Ramadge and Wonham [503] modelled a *discrete event system* (DES) by a partial automaton generating an event sequence (see also [136, 640]). Then, a supervisor, depending on the event sequence that occurred so far, is able to restrict the set of events that are possible in the next step, where, however, he can only limit the use of so-called controllable events. So, if we want to (globally) reset a finite state DES [17] under supervisory control, this can be seen as an instance of a constrained synchronization problem in case the permissible event sequences form a regular language.

Chapter 17
Further Notions and Preliminaries

Here, we introduce additional concepts and notations needed in Part IV, and give two new results in coding theory. Specifically, in Section 17.1, we introduce partial functions. Section 17.2 introduces partial automata (PDFAs), which generalize our automata model by allowing the transition relation to be partial function. These automata give a convenient specification mechanism for constraint languages (see the discussion in Section 17.9). Section 17.3 introduces the Levenshtein-Romanov mapping. In Chapter 19 we use prefix codes in some hardness conditions and solid codes for a statement on the preservation of the computational complexity. Section 17.4 introduces the relevant notions related to codes, various subclasses of codes as, for example, solid codes, and overlap-free languages. We also present two new characterizations of solid codes. Later, in Theorem 18.19, we will prove, using the Levenshtein-Romanov mapping, that the computational complexity of the constrained synchronization is preserved for reencodings of the constraint language by solid codes. Sections 17.5 and 17.6 introduce bounded and monoid languages, respectively. Lastly, in Section 17.7, we investigate the growth of languages, and state connections to bounded and monoid languages. In particular, we classify the growth of monoid languages.

17.1 Partial Functions

A *partial function* $f\colon X \nrightarrow Y$ either associates with $x \in X$ a unique value, written $f(x)$, or is undefined for $x \in X$, and $f(x) = y$ for $x \in X$ and $y \in Y$ means that $f(x)$ is defined and equals y. Correspondingly, $f(x) \neq y$ means that either $f(x)$ is defined but it does not equal y or that $f(x)$ is undefined. Setting $D = \{x \in X \mid f(x) \text{ is defined }\}$, we obtain a (total) function $f\colon D \to Y$. Every function can be seen as a partial function defined everywhere. Given $f\colon X \nrightarrow Y$ and $g\colon Y \nrightarrow Z$, their *composition* is $gf\colon X \nrightarrow Z$ where $(gf)(x) = z$ for $x \in X$ and $z \in Z$ if and only if $f(x)$ is defined with $g(f(x)) = z$, and otherwise $(gf)(x)$ is undefined. Other notions also carry over from functions without difficulty, for example the image of a subset $S \subseteq X$ by setting $f(S) = \{f(x) \mid x \in X \text{ and } f(x) \text{ is defined }\}$.

17.2 Partial Automata (PDFAs)

Here, we generalize the notion of a complete and deterministic automaton. A *partial deterministic and finite automaton (PDFA)* is a 5-tuple $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ where, as for complete and deterministic automata, Q is the *finite state set*, Σ the *finite input alphabet*, q_0 a *start state*, $F \subseteq Q$ a set of *final states*, but where the *transition function* is allowed to be a partial function $\delta\colon Q \times \Sigma \nrightarrow Q$.[1] So, every complete and deterministic automaton is a partial automaton as well.[2] The modifier *complete* refers to the fact that the transition function is a total function, i.e., defined for each argument.

> When we just say "automaton" we always mean a complete and deterministic semiautomaton, i.e., the automaton model previously considered with totally defined transition functions, and we explicitly say PDFA to denote a partial automaton with a start state and set of final states. In the context of this work, we only use PDFAs to describe constraint languages. Input automata to our problems are always assumed to be complete and deterministic.

The PDFA $\mathcal{A}$ can be made complete by adding a non-final trap state and by adding transitions to this new state for each $q \in Q$ and $x \in \Sigma$ where $\delta(q, x)$ is undefined. The notions accessible states, coaccessible states, accessible PDFAs, the reachability order on the states $\preceq_R$, accepted words, the accepted language $L(\mathcal{A})$, the notation $\mathcal{A}_{q,S}$ for $q \in Q$ and $S \subseteq Q$, the extension of the partial transition function to words, i.e., $\delta\colon Q \times \Sigma^* \nrightarrow Q$, and the extension of δ to subsets of states $\delta(S, u) = \{\delta(q, u) \mid \delta(q, u) \text{ is defined and } q \in S\}$ all carry over to PDFAs without difficulty.[3] We call $S \subseteq Q$ an upper set if $\delta(S, x) \subseteq S$ for each $x \in \Sigma$. We define $\mathcal{A}_{|S}$, $\mathcal{A}_{|\Gamma}$, and $\mathcal{A}_{|\Gamma,S}$ by restricting the partial transition function to inputs from S,Γ, and $S \times \Gamma$, respectively, analogously to the corresponding constructions for the previous automaton model. The product, power, and pair automata constructions also carry over and will be used in the following for PDFAs. A PDFA is called *trim* if every state is accessible and coaccessible. Every non-empty regular language is accepted by some trim PDFA; the empty language $\emptyset$ is the only language that is not accepted by a trim PDFA.

Proposition 17.1 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a PDFA. Then $\mathcal{A}$ is trim if and only if $[q_0]_{\simeq_R}$ is a minimum in the order induced by $\preceq_R$ on the strongly connected components and $[q]_{\simeq_R} \cap F \neq \emptyset$ for every maximal strongly connected component $[q]_{\simeq_R}$.*

[1] We only consider PDFAs as language acceptors and hence always endow them with a start state and a set of final states.

[2] This is actually a rather unfortunate naming. Usually, one would expect that when adding modifiers to a concept this would narrow down the possibilities, which also allows the description of subtypes by combining modifiers, but here adding the modifier "partial" gives a more general concept. For this reason, some authors, for example [122], deviate from the standard terminology by calling the most general model simply finite automaton.

[3] Martyugin [438] defines the extension of the transition function to subsets differently, requiring that $\delta(q, u)$ be defined for every $q \in S$, and leaving $\delta(S, u)$ undefined otherwise. A similar approach for nondeterministic automata was used in [43, Sec. 8] in the context of *conformant planning*. However, apart from these cases, this convention appears to be uncommon in the literature.

Proof. Every state is accessible if and only if $q_0 \precsim_R q$ for all $q \in Q$, i.e., $[q_0]_{\precsim_R}$ is a minimum. Further, every state is coaccessible if and only if for each $q \in Q$ we have $q \precsim_R p$ for some $p \in F$, which is fulfilled if every maximal strongly connected component contains a state from F. □

Minimal PDFAs are characterized by adding the additional requirement that every state is coaccessible to Theorem 6.88. The minimal PDFA differs from a minimal complete automaton if and only if the minimal complete automaton has a non-final trap state, which can be discarded in this case to obtain the minimal PDFA. Note that in a minimal complete automaton for the set of synchronizing words of a given complete and deterministic automaton every state is coaccessible, so that it makes no difference whether we consider complete or partial automata when investigating minimal deterministic automata for the for the set of synchronizing words.

Next, we introduce two generalizations of synchronization to PDFAs. Let $u \in \Sigma^*$. We call u a *carefully synchronizing word* [438, 527] for $\mathcal{A}$ if there exists $p \in Q$ such that $\delta(q, u) = p$ for all $q \in Q$, i.e., $|\delta(Q, u)| = 1$ and $\delta(q, u)$ is defined for each $q \in Q$. We call u an *exactly synchronizing word* [558, 606] if $|\delta(Q, u)| = 1$. Further, $\mathcal{A}$ is said to be *carefully synchronizing* (resp. *exactly synchronizing*) if there exists a carefully (resp. exactly) synchronizing word for $\mathcal{A}$.

17.3 The Levenshtein-Romanov Mapping

Levenshtein [409] and Romanov [520] introduced the following mapping.

Definition 17.2 Let $L \subseteq \Sigma^*$ with $L \neq \emptyset$. The *Levenshtein-Romanov mapping* is $\sigma_L \colon \Sigma^* \to \Sigma^*$ where $\sigma_L(u)$ for $u \in \Sigma^*$ equals the longest suffix of u that is also a prefix of a word in L.

For example $\sigma_{\{a,bba\}}(ababb) = bb$. Note that σ_L is only defined for non-empty languages L.

Let $L \subseteq \Sigma^*$ be non-empty and $u, v \in \Sigma^*$. The following relations are readily verified:

$$\sigma_L(u) \in \mathrm{Suff}\,(u) \cap \mathrm{Pref}(L) \tag{17.1}$$

$$\sigma_L(uv) \in \Sigma^* v \text{ for } v \in \mathrm{Pref}(L) \tag{17.2}$$

$$\sigma_L(u) = \sigma_{\mathrm{Pref}(L)}(u) \tag{17.3}$$

$$\sigma_L(v) = v \Leftrightarrow v \in \mathrm{Pref}(L) \tag{17.4}$$

$$\sigma_L(v) = v \Leftrightarrow v \in \mathrm{Pref}(L \setminus \{v\}) \text{ for } v \notin L \tag{17.5}$$

$$\sigma_{\{u\}}(v) = u \Leftrightarrow u \in \mathrm{Suff}\,(v) \tag{17.6}$$

$$\sigma_{\{u\}}(v) = \sigma_{\mathrm{Pref}(u)\setminus\{u\}}(v) \text{ for } u \notin \mathrm{Suff}\,(v) \tag{17.7}$$

By Equations (17.4) and (17.6) we have $\sigma_{\{u\}}(v) \in \{u, v\}$ if and only if $v \in \mathrm{Pref}(u)$ or $u \in \mathrm{Suff}\,(v)$. Note that $v \in \mathrm{Pref}(L \setminus \{v\})$ if and only if v is a proper prefix of some word from L.

We need the following two properties from [350, Prop. 2]. For the convenience of the reader, we include their proofs.

Proposition 17.3 ([350, Prop. 2]) *Let $L \subseteq \Sigma^*$ be non-empty and $u, v \in \Sigma^*$.*

1. *$\sigma_L(v)$ is a suffix of $\sigma_L(uv)$. So, σ_L preserves the suffix relation, i.e., if u is a suffix of v, then $\sigma_L(u)$ is a suffix of $\sigma_L(v)$.*
2. *$\sigma_L(uv) = \sigma_L(\sigma_L(u)v)$.*

Proof. 1: Since $\sigma_L(v)$ is a suffix of uv in $\mathrm{Pref}(L)$ and $\sigma_L(uv)$ is the longest suffix of uv in $\mathrm{Pref}(L)$, the suffix $\sigma_L(uv)$ is at least as long as the suffix $\sigma_L(v)$ of uv. Thus, $\sigma_L(v)$ is a suffix of $\sigma_L(uv)$.

2: In general, we have

$$\sigma_L(uv) = \begin{cases} \sigma_L(v) & \text{if } \mathrm{Suff}\,(u)\, v \cap \mathrm{Pref}(L) = \emptyset, \\ wv & \text{otherwise, where } w \in \mathrm{Suff}\,(u) \text{ is maximal with } wv \in \mathrm{Pref}(L). \end{cases} \tag{17.8}$$

(this equation also gives Item 1 since $v = x\sigma_L(v)$ for some $x \in \Sigma^*$.) Now, first suppose $\mathrm{Suff}\,(u)\, v \cap \mathrm{Pref}(L) = \emptyset$. Then $\mathrm{Suff}\,(\sigma_L(u))\, v \cap \mathrm{Pref}(L) = \emptyset$, and so we obtain $\sigma_L(uv) = \sigma_L(v) = \sigma_L(\sigma_L(u)v)$. Next, suppose $\mathrm{Suff}\,(u)\, v \cap \mathrm{Pref}(L) \neq \emptyset$. Then $\sigma_L(uv) = wv$ for the longest $w \in \mathrm{Suff}\,(u)$ such that $wv \in \mathrm{Pref}(L)$. As $w \in \mathrm{Pref}(L) \cap \mathrm{Suff}\,(u)$, we find that w is a suffix of $\sigma_L(u)$. So, wv is also the longest suffix of $\sigma_L(\sigma_L(u)v)$ in $\mathrm{Pref}(L)$, i.e., $\sigma_L(\sigma_L(u)v) = wv$.[4] □

Proposition 17.4 *Let $L, L' \subseteq \Sigma^*$ be non-empty and $u \in \Sigma^*$.*

1. *If $\mathrm{Pref}(L) \subseteq \mathrm{Pref}(L')$, then $\sigma_L(u)$ is a suffix of $\sigma_{L'}(u)$.*
2. *$\sigma_L(u)$ is a suffix of $\sigma_{LL'}(u)$.*
3. *We have*
$$\sigma_{L\cup L'}(u) = \begin{cases} \sigma_L(u) & \text{if } \sigma_{L\cup L'}(u) \in \mathrm{Pref}(L), \\ \sigma_{L'}(u) & \text{otherwise.} \end{cases} \tag{17.9}$$
4. *We have*
$$\sigma_{L\cup L'}(u) \in \mathrm{Pref}(L) \Leftrightarrow \sigma_{L'}(u) \in \mathrm{Suff}\,(\sigma_L(u)) \Leftrightarrow |\sigma_L(u)| \geq |\sigma_{L'}(u)| \tag{17.10}$$
5. *We have $\sigma_L(u) \in \mathrm{Suff}\,(L')$ if and only if $\sigma_{\{v\}}(u) \in \mathrm{Suff}\,(L')$ for all $v \in L$.*
6. *We have $\mathrm{Suff}\,(\sigma_L(u)) = \bigcup_{v\in L} \mathrm{Suff}\,\left(\sigma_{\{v\}}(u)\right)$.*
7. *We have $\sigma_L(u) = \varepsilon$ if and only if $\sigma_{\{v\}}(u) = \varepsilon$ for each $v \in L$.*

Proof. 1: Every suffix of u that is a prefix of a word from L is also a suffix that is a prefix of a word from L'. Thus, $\sigma_{L'}(u)$ is longer than $\sigma_L(u)$, which yields that $\sigma_L(u)$ is a suffix of $\sigma_{L'}(u)$.

2: We have $\mathrm{Pref}(L) \subseteq \mathrm{Pref}(LL')$. So this follows from Item 1.

[4] For an alternative argument: Note that, by Item 1, we have $\mathrm{Suff}\,(\sigma_L(v)) \subseteq \mathrm{Suff}\,(\sigma_L(\sigma_L(u)v)) \subseteq \mathrm{Suff}\,(\sigma_L(uv))$. So, for $\sigma_L(uv) = \sigma_L(v)$ we get $\sigma_L(\sigma_L(u)v) = \sigma_L(uv)$. Otherwise, we have $\sigma_L(uv) = wv$ for some $w \in \Sigma^*$. As $w \in \mathrm{Pref}(L)$ this yields that w is a suffix of $\sigma_L(u)$ and so $\sigma_L(\sigma_L(u)v) = wv$.

3: Suppose $\sigma_{L\cup L'}(u) \in \mathrm{Pref}(L)$. By Item 1, $\sigma_L(u)$ is a suffix of $\sigma_{L\cup L'}(u)$. Since $\sigma_{L\cup L'}(u) \in \mathrm{Pref}(L)$, $\sigma_{L\cup L'}(u)$ is a suffix of $\sigma_L(u)$ by the definition of σ_L. Thus $\sigma_{L\cup L'}(u) = \sigma_L(u)$. Otherwise, suppose $\sigma_{L\cup L'}(u) \notin \mathrm{Pref}(L)$. Since $\sigma_{L\cup L'}(u) \in \mathrm{Pref}(L \cup L')$, this yields $\sigma_{L\cup L'}(u) \in \mathrm{Pref}(L')$. With the same argument as before, we find $\sigma_{L\cup L'}(u) = \sigma_{L'}(u)$.

4: First, suppose $\sigma_{L\cup L'}(u) \in \mathrm{Pref}(L)$. Then $\sigma_{L\cup L'}(u) = \sigma_L(u)$ by Item 3. Thus, by Item 1, we have $\sigma_{L'}(u) \in \mathrm{Suff}(\sigma_L(u))$.

Next, suppose $\sigma_{L'}(u) \in \mathrm{Suff}(\sigma_L(u))$. Then the longest suffix of u in L is longer than the longest suffix of u in L'. This gives $\sigma_{L\cup L'}(u) = \sigma_L(u) \in \mathrm{Pref}(L)$.

The last equivalence follows easily since $\sigma_L(u)$ and $\sigma_{L'}(u)$ are both suffixes of u. This shows Equation (17.10).

5: First, suppose $\sigma_L(u) \in \mathrm{Suff}(L')$. By Item 1, $\sigma_{\{v\}}(u)$ is a suffix of $\sigma_L(u)$ for $v \in L$. Thus we find $\sigma_{\{v\}}(u) \in \mathrm{Suff}(L')$.

Conversely, suppose $\sigma_{\{v\}}(u) \in \mathrm{Suff}(L')$ for all $v \in L$. We have $\sigma_L(u) = \sigma_{\{v\}}(u)$ for some $v \in L$. This yields $\sigma_L(u) \in \mathrm{Suff}(L')$.

6: By Item 1, we have $\sigma_{\{v\}}(u) \in \mathrm{Suff}(\sigma_L(u))$ for $v \in L$. Thus $\bigcup_{v\in L} \mathrm{Suff}\left(\sigma_{\{v\}}(u)\right) \subseteq \mathrm{Suff}(\sigma_L(u))$.

Since $\sigma_L(u) \in \mathrm{Suff}(\sigma_L(u))$, by Item 6 we find $\sigma_{\{v\}}(u) \in \mathrm{Suff}(\sigma_L(u))$ for all $v \in L$. This gives $\bigcup_{v\in L} \mathrm{Suff}\left(\sigma_{\{v\}}(u)\right) \subseteq \mathrm{Suff}(\sigma_L(u))$.

7: This follows from Item 5 by considering $L' = \{\varepsilon\}$. □

Proposition 17.5 *Let $L \subseteq \Sigma^*$ and $u \in \Sigma^*$ such that $L \setminus \{u\} \neq \emptyset$. Then we have*

$$\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) = \begin{cases} \sigma_{L\setminus\{u\}}(u) & \text{if } \sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) \in \mathrm{Pref}(L \setminus \{u\}), \\ \sigma_{\mathrm{Pref}(u)\setminus\{u\}}(u) & \text{otherwise.} \end{cases} \tag{17.11}$$

Proof. We have

$$\mathrm{Pref}(\mathrm{Pref}(L) \setminus \{u\}) = \begin{cases} \mathrm{Pref}(L \setminus \{u\}) \cup (\mathrm{Pref}(u) \setminus \{u\}) & \text{if } u \in \mathrm{Pref}(L), \\ \mathrm{Pref}(L \setminus \{u\}) & \text{otherwise.} \end{cases}$$

First, suppose $u \notin \mathrm{Pref}(L)$. Then $u \notin L$ and $\mathrm{Pref}(L) = \mathrm{Pref}(L \setminus \{u\})$. By Equation (17.1), we have $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) \in \mathrm{Pref}(L)$. Thus, in this case, the first case from Equation (17.11) applies. Since we have $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) = \sigma_{\mathrm{Pref}(L)}(u) = \sigma_L(u) = \sigma_{L\setminus\{u\}}(u)$, we find that Equation (17.11) holds.

Next, suppose $u \in \mathrm{Pref}(L)$. Then we find

$$\begin{aligned} &\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) \\ &= \sigma_{\mathrm{Pref}(\mathrm{Pref}(L)\setminus\{u\})}(u) && \text{(Eqn. (17.3))} \\ &= \sigma_{\mathrm{Pref}(L\setminus\{u\})\cup(\mathrm{Pref}(u)\setminus\{u\})}(u) \\ &= \begin{cases} \sigma_{\mathrm{Pref}(L\setminus\{u\})}(u) & \text{if } \sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) \in \mathrm{Pref}(L \setminus \{u\}) \\ \sigma_{\mathrm{Pref}(u)\setminus\{u\}}(u) & \text{otherwise} \end{cases} && \text{(Eqn. (17.9))} \end{aligned}$$

Since, by Equation (17.3), we have $\sigma_{\mathrm{Pref}(L\setminus\{u\})}(u) = \sigma_{L\setminus\{u\}}(u)$, the claim follows. □

The Levenshtein-Romanov mapping can be used to describe the Myhill-Nerode equivalence classes for a left ideal as shown next.

Note that for a non-empty left ideal I, we always have $\sigma_I(u) = u$ since $\mathrm{Pref}(I) = \Sigma^*$.

Proposition 17.6 *Let $L \subseteq \Sigma^*$ be non-empty and consider the left ideal $I = \Sigma^* L$. Then for $u \in \Sigma^*$ we have $u \equiv_I \sigma_L(u)$.*

Proof. Let $u \in \Sigma^*$. Next, we show that $uv \in I$ if and only if $\sigma_L(u)v \in L$ for all $v \in \Sigma^*$, which is equivalent to $u \equiv_L \sigma_L(u)$.

First, suppose we have $uv \in I$ for some $v \in \Sigma^*$. If $v \in I$, then $\sigma_L(u)v \in I$ since I is a left ideal. Next, let $v \notin I$. In this case, since $uv \in I$, there must exist $x, y \in \Sigma^*$ with $y \neq \varepsilon$ such that $u = xy$ and $yv \in L$. Since $y \in \mathrm{Pref}(L)$, y is a suffix of $\sigma_L(u)$. Thus we also have $\sigma_L(u)v \in I$.

Conversely, suppose $\sigma_L(u)v \in I$. Since $\sigma_L(u)$ is a suffix of u, this clearly gives $uv \in I$. □

Corollary 17.7 *Let $I = \Sigma^* L$ for some $L \subseteq \Sigma^*$. Then $\{[w]_{\equiv_I} \mid w \in \Sigma^*\} = \{[w]_{\equiv_I} \mid w \in \mathrm{Pref}(L)\}$.*

Proof. This follows from Theorem 17.6 and since $\sigma_L(u) \in \mathrm{Pref}(L)$ for all $u \in \Sigma^*$. □

Next, we give some equalities involving suffix sets.

Proposition 17.8 *Let $L \subseteq \Sigma^*$ and $u \in \Sigma^+$. Then the following holds:*

1. $\mathrm{Suff}\,(\sigma_L(u)) \cap \mathrm{Pref}(L) = \mathrm{Suff}\,(u) \cap \mathrm{Pref}(L)$.
2. $\mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap \mathrm{Pref}(\mathrm{Pref}(L) \setminus \{u\}) = \mathrm{Suff}\,(u) \cap \mathrm{Pref}(\mathrm{Pref}(L) \setminus \{u\})$.
3. $\mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L = \mathrm{Suff}\left(\sigma_{L\setminus\{u\}}(u)\right) \cap L$ *if* $L \setminus \{u\}$ *is non-empty.*
4. $\mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L = (\mathrm{Suff}\,(u) \cap (L \setminus \{u\})) \cup (\{u\} \cap L \cap \mathrm{Pref}(L \setminus \{u\}))$.

Proof. 1: Since $\sigma_L(u)$ is a suffix of u, we have $\mathrm{Suff}\,(\sigma_L(u)) \subseteq \mathrm{Suff}\,(u)$ and so $\mathrm{Suff}\,(\sigma_L(u)) \cap \mathrm{Pref}(L) \subseteq \mathrm{Suff}\,(u) \cap \mathrm{Pref}(L)$. Next, let $v \in \mathrm{Suff}\,(u) \cap \mathrm{Pref}(L)$. Since $\sigma_L(u)$ equals the longest suffix of u that is in $\mathrm{Pref}(L)$, this suffix has length at least $|v|$. Thus v is a suffix of $\sigma_L(u)$, which gives $v \in \mathrm{Suff}\,(\sigma_L(u)) \cap \mathrm{Pref}(L)$.

2: This follows from Item 1 applied to $\mathrm{Pref}(L) \setminus \{u\}$.

3: Since $L \setminus \{u\} \subseteq \mathrm{Pref}(L) \setminus \{u\}$, by Item 1 of Theorem 17.4 we have

$$\mathrm{Suff}\left(\sigma_{L\setminus\{u\}}(u)\right) \cap L \subseteq \mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L.$$

Next, let $v \in \mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L$. Suppose $v = u$. Then $u = \sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)$. This yields that u is a proper prefix of a word from L and thus $u = \sigma_{L\setminus\{u\}}(u)$. If $v \neq u$, then v is a suffix of u in $L \setminus \{u\}$ and thus it is a suffix of $\sigma_{L\setminus\{u\}}(u)$. In both cases, we find $v \in \mathrm{Suff}\left(\sigma_{L\setminus\{u\}}(u)\right) \cap L$.

4: By Item 2 we have $\mathrm{Suff}\,(u) \cap (L \setminus \{u\}) = \mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap (L \setminus \{u\})$. By Equation (17.5), $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) = u$ if and only if u is a proper prefix of a word from $\mathrm{Pref}(L) \setminus \{u\}$. This is equivalent to u being a proper prefix of a word from L, i.e, $u \in \mathrm{Pref}(L\setminus\{u\})$. Thus $u \in \mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L$ if and only if $u \in L \cap \mathrm{Pref}(L\setminus\{u\})$, which gives the claim. □

Proposition 17.9 *Let $L, U, V \subseteq \Sigma^*$ and $v \in \mathrm{Pref}(L)$. Then the following are equivalent.*

1. $\sigma_L(uv) \in U$ *for some* $u \in \mathrm{Pref}(L) \cap \mathrm{Suff}(V)$.
2. $\sigma_L(uv) \in U$ *for some* $u \in \mathrm{Suff}(V)$.
3. $(\mathrm{Pref}(L) \cap U) \cap (\mathrm{Pref}(L) \cap \mathrm{Suff}(V))v \neq \emptyset$.
4. $(\mathrm{Pref}(L) \cap U) \cap \mathrm{Suff}(V)\, v \neq \emptyset$.
5. $\sigma_L(uv) = uv$ *and* $uv \in U$ *for some* $u \in \mathrm{Suff}(V)$.

Proof. $1 \Rightarrow 2$: This is clear.

$2 \Rightarrow 3$: Since $v \in \mathrm{Pref}(L)$, we have $\sigma_L(uv) = wv \in \mathrm{Pref}(L)$ for some $w \in \Sigma^*$. We have $w \in \mathrm{Pref}(L)$ and the word w is a suffix of u, and so $w \in \mathrm{Suff}(V)$. Thus we have $wv \in \mathrm{Pref}(L) \cap U$ and $wv \in \mathrm{Suff}(V)\, v$.

$3 \Rightarrow 4$: This is clear.

$4 \Rightarrow 5$: Let $u \in \mathrm{Suff}(V)$ with $uv \in \mathrm{Pref}(L) \cap U$. By Equation (17.4) we have $\sigma_L(uv) = uv$.

$5 \Rightarrow 1$: Since $\sigma_L(uv) \in \mathrm{Pref}(L)$, we get $uv \in \mathrm{Pref}(L) \cap U$ and $u \in \mathrm{Pref}(L) \cap \mathrm{Suff}(V)$. □

Proposition 17.10 *Let $L, U, V \subseteq \Sigma^*$ with $v \in \mathrm{Pref}(L)$ and $U \subseteq \mathrm{Pref}(L)$. Then the following are equivalent.*

1. $\sigma_L(uv) \in U$ *for some* $u \in \mathrm{Suff}(V)$.
2. $uv \in U$ *for some* $u \in \mathrm{Suff}(V)$.
3. $uv \in U$ *for some* $u \in \mathrm{Pref}(L) \cap \mathrm{Suff}(V)$.

Proof. $1 \Rightarrow 2$: Since $v \in \mathrm{Pref}(L)$ we have $\sigma_L(uv) = wv$ for some $w \in \mathrm{Suff}(V)$. This gives the claim.

$2 \Rightarrow 3$: Since $uv \in \mathrm{Pref}(L)$ we have $u \in \mathrm{Pref}(L)$, which gives the claim.

$3 \Rightarrow 1$: Since $uv \in \mathrm{Pref}(L)$ we have $\sigma_L(uv) = uv$. □

Remark 17.11 The following are equivalent: $L = \{\varepsilon\}$, $\sigma_L(y) = \varepsilon$ for all $y \in \Sigma^*$, and $\sigma_L(xy) = \sigma_L(y)$ for all $x, y \in \Sigma^*$.

Remark 17.12 For $v \in \Sigma^+$, the following are equivalent: $\sigma_L(uv) = \varepsilon$ for all $u \in \Sigma^*$, $\sigma_L(uv) = \varepsilon$ for all $u \in \mathrm{Pref}(L)$, $\sigma_L(v) = \varepsilon$ and $wv \notin \mathrm{Pref}(L)$ for all $w \in \Sigma^+$, $\sigma_L(v) = \varepsilon$ and $wv \notin \mathrm{Pref}(L)$ for all $w \in \mathrm{Pref}(L)$.

17.4 Codes and Overlap-Free Languages

Here, in Section 17.4.1 we introduce codes and overlap-free languages and state some basic relations. Some of our later results will use prefix codes and solid codes. A few conditions using prefix codes use constructions related to maximal prefix codes. This class of codes is discussed in Section 18.3.3. In Section 17.4.2 we collect properties of prefix codes that we need for those results. Section 17.4.3 states some further characterizations of our classes of codes and overlap-free languages and relations to the Levenshtein-Romanov mapping. Section 17.4.4 collects characterizations of solid codes (some of them have not been observed before).

17.4.1 Basic Notions

Here, in Theorem 17.13, we introduce overlap-free languages, codes, and various subclasses of codes as, for example, solid codes, comma-free codes, and infix codes. Theorem 17.16 proves that every solid code is a comma-free code, and that every comma-free code is an infix code. In Theorem 17.14 we also show that overlap-free languages are not codes in general. The remaining inclusions among infix, bifix, prefix, and suffix codes are obvious. Figure 17.1 gives an overview of the inclusion relations among the introduces code classes and overlap-free languages. As formally stated in Theorem 17.18, codes are used to encode symbols from some source alphabet. Theorems 17.20 and 17.22 give two combinatorial results on word equations.

Definition 17.13 Let $L \subseteq \Sigma^+$ be non-empty and $u, v, w \in \Sigma^*$.

1. L is a *code* if, for $u_1, \ldots, u_n, v_1, \ldots, v_m \in L$, the equality
$$u_1 \cdots u_n = v_1 \cdots v_m$$
implies $n = m$ and $u_i = v_i$ for all $i \in \{1, \ldots, n\}$ (the *unique factorization property* in L^* w.r.t. L).
2. L is *overlap-free* if $uv, vw \in L$ with $u \neq \varepsilon$ and $w \neq \varepsilon$ implies $v = \varepsilon$, i.e., no proper non-empty suffix of a word from L is also a proper non-empty prefix of some word in L.[5]
3. L is an *infix code* (or *infix-free*) if $u, vuw \in L$ implies $v = w = \varepsilon$.
4. L is a *prefix code* (or *prefix-free*) if $uv, u \in L$ implies $v = \varepsilon$.
5. L is a *suffix code* (or *suffix-free*) if $vu, u \in L$ implies $v = \varepsilon$.
6. L is a *bifix code* if it is a prefix and a suffix code.
7. L is a *solid code* if it is an overlap-free infix code.
8. L is a *comma-free code* if $L^2 \cap \Sigma^+ L \Sigma^+ = \emptyset$.[6]

Table 17.1 gives graphical depictions when some of the defining conditions from Theorem 17.13 fail.

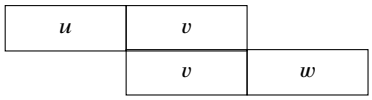
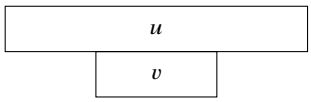
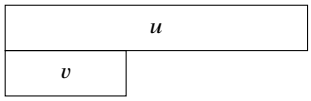
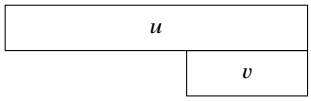
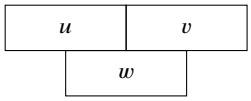
By definition a code is a subset of Σ^+, but as $\varepsilon = \varepsilon \cdot \varepsilon$ a language fulfilling Theorem 17.13, Item 1 never contains the empty word and we could replace Σ^+ by Σ^* in the definition. However, the defining conditions for overlap-free languages, infix, prefix, suffix, bifix, solid, and comma-free codes are also fulfilled for $L = \{\varepsilon\}$ and the exclusion of the empty word makes a difference in these cases.

Example 17.14
1. The set Σ is a code, as Σ^* fulfills the unique factorization property w.r.t. Σ.
2. For $n \geq 1$, the set Σ^n is a code. A subset of Σ^n is a solid code if and only if it is overlap-free. Thus, Σ is a solid code.

[5] The condition that u and v are non-empty is missing in [350], but it is essential since without it, for $v \in L$, we have $\varepsilon \cdot v, v \cdot \varepsilon \in L$ and thus $v = \varepsilon$, which contradicts the fact that L is a subset of Σ^+.

[6] The name comma-free refers to the fact that, when facing a stream of code words, we can uniquely decipher the next code words when starting to read from somewhere in the middle of the stream without the need of a special separation symbol.

Table 17.1: Situations where the defining conditions for overlap-, infix-, prefix-, suffix-, and comma-freeness fail. Boxes labeled u, v, w represent words or factors, and their relative positions indicate the overlaps.

not overlap-free	
not infix-free	
not prefix-free	
not suffix-free	
not comma-free	

3. $\{w\}$ is a code for $w \in \Sigma^+$. Here, we have that $\{w\}$ is not a comma-free code if and only if $w = uv$ for $u, v \in \Sigma^*$ such that $uv = vu$. This follows since $uv = vu$ implies $uvuv \in \Sigma^+vu\Sigma^+$ and, conversely, $ww = xwy$ for $x, y \in \Sigma^+$ implies that we can write $w = xu = vy$ for $u, v \in \Sigma^+$ with $uv = w$. Now $xu = uv$ implies $|x| = |v|$ and similarly we obtain $|u| = |y|$. Thus, since x and v are equal-length prefixes of w we have $x = v$ and similarly $u = y$ and thus $vu = w = uv$.
4. $\{a, ab\}$ is an overlap-free code that is not a prefix code.
5. $\{a, ab, aab\}$ is overlap-free, but not a code since $(aab) = (a)(ab)$.
6. $\{abc, b\}$ is an overlap-free bifix code that is not an infix code.
7. a^*b is a prefix code that is overlap-free and not a suffix code.
8. $\{aba^ibb \mid i \geq 1\}$ is a solid code.
9. The language C from Theorem 8.15 forms a solid code.
10. $\{abb, aba\}$ is a comma-free code, but not a solid code since it is not overlap-free.
11. $\{ab, ba\}$ is an infix code, but not a comma-free code.
12. Let $C \subseteq \Sigma^+$ be a prefix code. Then $C\Sigma^n$ is a prefix code for each $n \geq 1$.
13. For $L \subseteq \Sigma^*$ with $L \cap \Sigma^+ \neq \emptyset$, the language $L \setminus (L\Sigma^+ \cup \{\varepsilon\})$ is a prefix code.
14. Suppose $C \subseteq \Sigma^+$ is a finite prefix code such that $c\Sigma^* \cap C = C \cap \Sigma^*c = \emptyset$ for some $c \in \Sigma$. Set $k = \max\{l \geq 0 \mid \Sigma^*c^l\Sigma^* \cap C \neq \emptyset\} + 1$. For example, when applying the construction to the prefix code $C = \{bcb, bccb\}$ we obtain $\{cccbcb, cccbccb\}$. Then, c^kC is a solid code as we show next.

1. c^kC is an infix code: Assume $xc^kuy = c^kv$ with $u, v \in C$ and $x, y \in \Sigma^*$. As c^k is not a factor of v, we have $|x| < k$, which gives $x \in c^*$. Since $v \notin c\Sigma^*$ we conclude $|x| = 0$. Hence we have $uy = v$ and as C is a prefix code we get $y = \varepsilon$. So c^kC is an infix code.
2. c^kC is overlap-free: Assume $xy = c^ku$ and $yz = c^kv$ for $u, v \in C$ and $x, y, z \in \Sigma^*$. For $|y| \leq k$, as $yz = c^kv$ we have $y \in c^*$ and, as $u \notin \Sigma^*c$, from $xy = c^ku$ we get $|y| = 0$. For $|y| > k$, from $yz = c^kv$ we find $y = c^kw$ for some $w \in \mathrm{Pref}(v)$. Then, from $xy = c^ku$, as $c^k \notin \mathrm{Fact}(u)$, we must have $|x| < k$. Further, as $u \notin c\Sigma^*$, $xy = c^ku$ then gives $|x| = 0$, which implies that c^ku is a prefix of c^kv. As c^kC is an infix code, we find $|z| = 0$ and so $y = c^ku = c^kv$. Hence c^kC is overlap-free.

15. By the previous item, for a finite prefix code $C \subseteq \Sigma^+$ and distinct $c, a \in \Sigma$ with $k = \max\{l \geq 0 \mid \Sigma^*c^l\Sigma^* \cap C \neq \emptyset\} + 1$ we have that c^kaCa is a solid code and c^kaC and c^kCa are prefix codes if $C \cap \Sigma^*c = \emptyset$ and $c\Sigma^* \cap C = \emptyset$, respectively, since aC, Ca and aCa are prefix code as well.

Remark 17.15 We actually came up with the construction from Item 14 in Theorem 17.14 by analysing a result from [215, Thm. 29] and extending it in [298, Thm. 17] (see also the discussion on page 501). However, a closely related variant of this construction is much older and was introduced by Gilbert [244] in the context of synchronization between sender and receiver during message transmission and by Levenshtein [409, 410] to construct maximal solid codes of equal-length words.[7] Levenshtein considers codes of the form c^kC, where $C = \{u \in \Sigma^n \mid u \notin \Sigma^*c^k\Sigma^* \text{ and } u \notin c\Sigma^* \text{ and } u \notin \Sigma^*c\}$ for $1 \leq k < n$ and $c \in \Sigma$. Further generalizations appear in [93, Construction 2]. It was also independently rediscovered in [145].

A prefix code is also characterized by the property that for code words u, v and any words w, w' the equality $uw = vw$ implies $u = v$ since $uv = vw$ implies that either u is a prefix of v or vice versa by the unique factorization property of Σ^* w.r.t. Σ and so both must be equal. Now, as the naming anticipates, this yields that prefix codes (and similarly suffix codes) are indeed codes: The equation $u_1 \cdots u_n = v_1 \cdots v_m$ implies $u_1 = v_1$ and $u_2 \cdots u_n = v_2 \cdots v_m$ for prefix codes (resp. $u_m = v_m$ and $u_1 \cdots u_{n-1} = v_1 \cdots v_{m-1}$ for suffix codes). Continuing, we ultimately find $n = m$ and $u_i = v_i$ for all $i \in \{1, \ldots, n\}$. Thus, as subclasses, bifix and infix codes are also codes. Furthermore, as we show next, solid and comma-free codes are infix codes, yielding that all the mentioned classes except overlap-free languages are codes. See Figure 17.1 for an overview of the inclusion relations.

Proposition 17.16 ([349, 350])

1. *The class of solid codes is properly contained in the class of comma-free codes.*
2. *The class of comma-free codes is properly contained in the class of infix codes.*

Proof. A comma-free code that is not a solid code and an infix code that is not comma-free are given in Item 10 and Item 11 of Theorem 17.14, respectively. Next, we show the claimed inclusions.

[7] Levenshtein referred to them as non-overlapping codes. In fact, codes of equal-length words are infix codes. Thus, such a code forms a solid code if and only if it is overlap-free.

1: Let $L \subseteq \Sigma^+$ be non-empty. Suppose L is not comma-free. Then there exist $u, v, w \in L$ such that $uv = xwy$ for some $x, y \in \Sigma^+$. Then $|x| \geq |u|$ or $|y| \geq |v|$ implies $w \in \mathrm{Fact}(v)$ or $w \in \mathrm{Fact}(v)$, respectively, so that L is not an infix code. For $|x| < |u|$ and $|y| < |v|$, we have $w = u'v'$ for a proper non-empty suffix u' of u and a proper non-empty prefix v' of v, so that L is not overlap-free. Hence, if L is a solid code, we must have $L^2 \cap \Sigma^+ L \Sigma^+ = \emptyset$.

2: Let $C \subseteq \Sigma^+$ be a comma-free code. Suppose $u, vuw \in C$ for $u, v, w \in \Sigma^*$. For $w \neq \varepsilon$, we have $uvuw \in C^2 \cap \Sigma^+ u \Sigma^+$, and for $v \neq \varepsilon$, we have $vuwu \in C^2 \cap \Sigma^+ u \Sigma^+$. Whence, by comma-freeness, we find $v = w = \varepsilon$. So C is an infix code. □

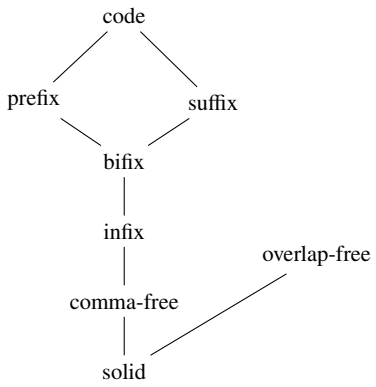


Fig. 17.1: Inclusion relations between different code classes and overlap-free languages.

Remark 17.17 Prefix codes can be visualized as rooted trees. A *rooted tree* is an undirected connected and acyclic graph with the selection of a distinguished node, called the *root*. A prefix code $C \subseteq \Sigma^+$ induces the rooted tree $(C, \{\{u, ux\} \mid u, ux \in \mathrm{Pref}(C)\})$ with root ε and where each node has outdegree at most $|\Sigma|$. For example, $\{00, 01, 10, 110, 111\}$ from Figure 1.2 is a prefix code with its corresponding rooted tree shown (with added directions). Conversely, by labeling the outgoing edges of each node of a rooted tree where each node has outdegree at most $|\Sigma|$ by a distinct letter from Σ we uniquely obtain a prefix code. In this sense, up to a reordering of the alphabet, prefix codes over Σ correspond uniquely to rooted trees with outdegree at most $|\Sigma|$.

Comma-free codes are also known as *intercodes of index one* [348, Thm. 3].[8] Originally, comma-free codes were defined to be subsets of Σ^n for some $n \geq 1$ [253], but this restriction was subsequently dropped [87, 348, 350].

[8] There appear to be two typographical errors in [348]. The definition of intercodes of index one omits the exponent 2, i.e., it states $L \cap \Sigma^+ L \Sigma^+ = \emptyset$ instead of $L^2 \cap \Sigma^+ L \Sigma^+ = \emptyset$. Furthermore, the condition $u, w \neq \varepsilon$ appears to be missing from the definition of an overlap-free code. Without this condition, only $L = \emptyset$ satisfies the definition, for if $v \in L$, we have $\varepsilon \cdot v = v \cdot \varepsilon$, and so $v = \varepsilon$. But this is impossible because $L \subseteq \Sigma^+$.

The (algebraic) approach regarding codes as certain subsets of Σ^* goes back to Schützenberger [551]. Prior to this, the theory of codes had its origin in Shannon's theory of information [561, 562]. Both viewpoints are linked via *coding morphisms*, i.e., injective morphisms $\varphi\colon \Gamma^* \to \Sigma^*$ such that $\varphi(\Gamma)$ is a code. Here, Γ is called the *source alphabet* and Σ the *channel alphabet*. Intuitively, this expresses the fact that codes may be used as communication tools via a correspondence (encoding) between the letters $a \in \Gamma$ of the source alphabet by code words $\varphi(a)$ over the channel alphabet Σ, where injectivity guarantees that a message $\varphi(u)$, $u \in \Gamma^*$, can be uniquely deciphered as u by the receiver. For instance, in the introductory example on error-prone communication from Section 1.3.1, the source alphabet is $\{a, h, l, o, r\}$ and the channel alphabet is $\{0, 1\}$. The following statement [87, Prop 2.1.1] summarizes the relation of codes to coding morphisms.

Proposition 17.18 ([87, Prop. 2.1.1]) *To every code $C \subseteq \Sigma^+$ there exists a coding morphism $\varphi\colon \Gamma^* \to \Sigma^*$ with $\varphi(\Gamma) = C$. Moreover, any bijection from some alphabet Γ (here possibly infinite) onto a code $C \subseteq \Sigma^+$ extends (as in Theorem 3.60) to a unique injective morphism from Γ^* to Σ^* with image C^*. Conversely, for injective morphisms $\varphi\colon \Gamma^* \to \Sigma^*$, $\varphi(\Gamma)$ is a code.*

Throughout this work, our alphabet is always finite, the only exception being Theorems 3.60 and 17.18 where the (source) alphabets can be infinite.

Remark 17.19 Combining these notions, we find that $L \subseteq \Sigma^+$ is a code if and only if the map $\hat{\imath}\colon L^* \to \Sigma^*$ from Section 3.7.1 is injective, i.e., a coding morphism where L is considered as a (possibly infinite) alphabet. Thus we can identify words formed by concatenating words from a code C with tuples over C in a unique way. This yields that Theorem 3.60 transfers to codes, i.e., every function from C into a monoid M extends uniquely into a monoid morphism from C^* into M.

The next two results are well-known in combinatorics on words. Theorem 17.20 characterizes codes of size two and Theorem 17.22 gives a factorization result for triples of words u, v, w such that $uv = wu$. For a proof of the former result see [422, Cor. 1.2.6, Prop. 1.3.1 & 1.3.2] or [245, Lem. 5.1] or [426, Lem. 3 & Cor. 4.1] for the latter three equivalences, and for the latter result see [422, Prop. 1.3.4] or [426, Lem. 2].

Proposition 17.20 *Let $u, v \in \Sigma^*$ be distinct. Then the following are equivalent.*

1. *$\{u, v\}$ is not a code.*
2. *$u, v \in w^*$ for some w^*.*
3. *$u^i = v^j$ for some $i, j \geq 0$.*
4. *$uv = vu$.*

Remark 17.21 By Theorem 17.20, with Item 3 from Theorem 17.14, $\{w\}$ for $w \in \Sigma^+$ is comma-free if and only if $w = v^i$ for $i \geq 1$ implies $i = 1$,

Proposition 17.22 *Let $u, v, w \in \Sigma^*$ with $w \neq \varepsilon$ such that $uv = wu$. Then $w = xy$, $u = (xy)^k x$, and $v = yx$ for some $x, y \in \Sigma^*$ and $k \geq 0$.*

17.4.2 More on Prefix Codes

Here we give further properties of prefix codes. We begin with another characterization in Theorem 17.23. Theorem 17.24 gives additional conditions equivalent to part of this characterization. As pointed out in Theorem 17.25, these statements characterize prefix codes in a way that generalizes the defining property of codes by including certain suffixes in addition to code words. We then derive several consequences that will be needed later. Finally, in Theorem 17.30, we characterize regular prefix codes via PDFAs.

Proposition 17.23 *A language $C \subseteq \Sigma^+$ is a prefix code if and only if it is a code such that $C^* \cap C^*u \neq \emptyset$ for $u \in \Sigma^*$ implies $u \in C^*$.*

Proof. First, suppose C is a prefix code. Then it is a code. The remaining claim is equivalent to the fact that $vu \in C^*$ for $v \in C^*$ implies $u \in C^*$. We show this by induction on the length of a product of v into words from C. For $v = \varepsilon$, the claim clearly holds. Otherwise, we have $v = xx'$ and $vu = yy'$ with $x, y \in C$ and $x', y' \in C^*$. Since C is a prefix code and $yy' = vu = xx'u$, we find $y = x$ and $y' = x'u$. Since $y', x' \in C^*$, inductively, we find $u \in C^*$. Thus, by induction, we obtain the claim.

Next, suppose that C is not a prefix code. Then there exist $u \in C$ and a non-empty word $v \in \Sigma^+$ such that $uv \in C$. Thus either C is not a code, or, if C is a code, necessarily $v \notin C^+$. Indeed, otherwise the word $uv \in C$ would have the alternative factorization $uv = u \cdot v$ with $u \in C$ and $v \in C^+$, contradicting the unique factorization property of a code. □

The latter condition in Theorem 17.23 is equivalent to two other conditions concerning the uniqueness of certain suffixes, as stated below. Notably, these equivalences hold for arbitrary languages in Σ^+, not just codes. For example, they apply to $(\Sigma^i)^+$ for $i \geq 1$ and $\{ab, b\}$, although neither of these languages is a code.

Proposition 17.24 *Let $L \subseteq \Sigma^+$ be any language not containing the empty word. Then the following are equivalent.*

1. *For $u, v \in \Sigma^* \setminus L\Sigma^*$ with $L^*u \cap L^*v \neq \emptyset$ we have $u = v$.*
2. *$L^* \cap L^*v \neq \emptyset$ for $v \in \Sigma^*$ implies $v \in L^*$.*
3. *For $u, v \in \Sigma^+ \setminus L\Sigma^+$ with $L^*u \cap L^*v \neq \emptyset$ we have $u = v$.*

Proof. 1 $\Rightarrow$ 2: Let $u, uv \in L^*$. Since $\varepsilon \notin L$, we can write $v = xy$ with $x \in L^*$ and $y \notin L\Sigma^*$. Then we have $uv \in L^* \cap L^*y$. By Item 1, we find $y = \varepsilon$ and thus $v \in L^*$.

2 $\Rightarrow$ 3: Let $w, w' \in L^*$ such that $wu = w'v$ with $u, v \in \Sigma^+ \setminus L\Sigma^+$. Assume $u \neq v$. Then, without loss of generality, we have $u = xv$ for some non-empty $x \in \Sigma^+$. This gives $w' = wx$. By assumption, we find $x \in L^+$ and thus $u \in L\Sigma^+$, but $u \notin L\Sigma^+$ by choice of u. Thus, we must have $u = v$.

3 $\Rightarrow$ 1: Suppose Item 3 holds true. Let $L^*u \cap L^*v \neq \emptyset$ for some $u, v \notin L\Sigma^*$. Then there exists $w \in L^*$ such that $wu \in L^*v$. Assume $u \neq v$. Then, by Item 3, we have $u, v \notin \Sigma^+ \setminus L\Sigma^+$. Since $u, v \notin L\Sigma^+$, this yields $u = \varepsilon$ or $v = \varepsilon$. Without loss of generality, let $u = \varepsilon$. Write $w = xx'$ and $x'x' = yy'$ with x and y being the longest proper prefixes of w and $x'x'$, respectively, in L^*. Then we have

$$x', y' \in \Sigma^+ \setminus L\Sigma^+ \text{ and } wx' \in L^*x' \cap L^*y'.$$

By assumption, this gives $x' = y'$. Since $x'x' = yy'$ we find $x' = y$. Thus $x' \in L^*$ since $y \in L^*$. With $x' \notin L\Sigma^+$ and $|x| > 0$, we get $x' \in L$. We have $w \in L^*x' \cap L^*v$ where $v \in \Sigma^+$ since $u \neq v$. By assumption, we find $x' = v$, contradicting $v \notin L\Sigma^*$ since $x' \in L$. Thus, we must have $v = \varepsilon$. □

Note that u and v must be non-empty in Item 3 of Theorem 17.24. For example, consider any non-empty language $L \subseteq \Sigma^+$. Then for $w \in L$ we have $w \in L^*\varepsilon \cap L^*w$, but $w \neq \varepsilon$ since L is a subset of Σ^+.

Combining Theorems 17.23 and 17.24, we obtain that prefix codes extend the unique factorization property of codes in the following sense.

Corollary 17.25 *A language $C \subseteq \Sigma^+$ is a prefix code if and only if for $u_1, \ldots, u_{n-1} \in C$, $v_1, \ldots, v_{m-1} \in C$, and $u_n, v_m \in \Sigma^+ \setminus C\Sigma^+$ such that*

$$u_1 \cdots u_{n-1}u_n = v_1 \cdots v_{m-1}v_m$$

we have $n = m$ and $u_i = v_i$ for all $i \in \{1, \ldots, n\}$.

Proof. Suppose C is a prefix code and let $u_1, \ldots, u_n$ and $v_1, \ldots, v_m$ as in the statement with $u_1 \cdots u_{n-1}u_n = v_1 \cdots v_{m-1}v_m$. Thus $C^*u_n \cap C^*v_m \neq \emptyset$. By Theorems 17.23 and 17.24 this gives $u_n = v_m$. Thus $u_1 \cdots u_{n-1} = v_1 \cdots v_{m-1}$. Since C is a code, we find $n = m$ and $u_1 = v_1, \ldots, u_{n-1} = v_{n-1}$.

Conversely, this condition yields that C is a code since it generalizes the unique factorization property. Next, suppose $C^*u \cap C^*v$ for some $u, v \in \Sigma^+ \setminus C\Sigma^+$. Then there exist $u_1, \ldots, u_n \in C$ and $v_1, \ldots, v_m \in C$ such that $u_1 \cdots u_n u = v_1 \cdots v_m v$. By assumption, we find $n = m$, $u_i = v_i$ for $i \in \{1, \ldots, n\}$, and $u = v$. Thus, by Theorems 17.23 and 17.24, we conclude that C is a prefix code. □

Note that for a prefix code, we have $C \subseteq \Sigma^+ \setminus C\Sigma^+$, i.e., the condition from Theorem 17.25 also allows u_n and v_m to belong to C.

With our result, a variant of Theorem 17.25 can also be formulated with $u_n, v_m \in \Sigma^* \setminus C\Sigma^*$ instead of $u_n, v_m \in \Sigma^+ \setminus C\Sigma^+$.

Corollary 17.26 *Let $C \subseteq \Sigma^+$ be a prefix code. Then the factorization witnessing a word to be in* $\mathrm{Pref}(C^*) = C^*(\mathrm{Pref}(C) \setminus C)$ *is unique, i.e., for $u \in \mathrm{Pref}(C^*)$ there exists precisely one factorization of the form $u = u_1 \cdots u_{n-1}u_n$ with $u_1, \ldots, u_{n-1} \in C$ and $u_n \in \mathrm{Pref}(C) \setminus C$.*

Proof. This follows directly from Theorem 17.25 since $\mathrm{Pref}(C) \cap C\Sigma^+ = \emptyset$ for a prefix code C. □

Corollary 17.27 *Let $C \subseteq \Sigma^+$ be a prefix code. Then* $\mathrm{Pref}(C^*) \cap C^*u \neq \emptyset$ *for $u \in \Sigma^*$ implies $u \in \mathrm{Pref}(C^*)$.*

Proof. Let $vu \in \mathrm{Pref}(C^*)$ with $v \in C^*$. Write $vu = xx'$ and $u = yy'$ with $x \in C^*$ and x' being a proper prefix of some word in C and y being a longest proper prefix of u in C^*. Since C is a prefix code, we have $x' \notin C\Sigma^+$ and, by choice of y, we

have $y' \in \Sigma^+ \setminus C\Sigma^+$. For $x' = \varepsilon$, we have $vu \in C^*$ and by Theorem 17.23 we find $u \in C^*$. Next, suppose $x' \neq \varepsilon$. We have $xx' = vu = vyy'$ and so $vu \in C^*x' \cap C^*y'$ and by Theorems 17.23 and 17.24 we find $x' = y'$. Since $x' \in \mathrm{Pref}(C^*)$, this gives $u = yx' \in C^* \mathrm{Pref}(C^*) = \mathrm{Pref}(C^*)$. □

Theorem 17.27 fails for general codes. For example, for $C = \{a, ab\}$ we have $ab \in \mathrm{Pref}(C^*)$, $a \in C$, but $b \notin \mathrm{Pref}(C^*)$.

Corollary 17.28 *Let $C \subseteq \Sigma^+$ be a prefix code and $u \in \Sigma^*$. Suppose $C^*u \cap C^*(\mathrm{Pref}(C) \setminus C) \neq \emptyset$ and that u is not a proper prefix of some word from C and $u \notin C\Sigma^+$. Then we have $u \in C$.*

Proof. Since $C^*w \subseteq \mathrm{Pref}(C^*)$, we have $C^*u \cap \mathrm{Pref}(C^*) \neq \emptyset$. Thus, by Theorem 17.27, we find $u \in \mathrm{Pref}(C^*)$. Since $u \notin C\Sigma^+$, this gives $u \in \mathrm{Pref}(C)$. Then, since $u \notin \mathrm{Pref}(C) \setminus C$, we must have $u \in C$. □

Corollary 17.29 *Let $C \subseteq \Sigma^+$ be a prefix code and $L \subseteq \Sigma^*$ such that $C^*L \subseteq L$. Then we have the following:*

$$C^*(L \setminus \mathrm{Pref}(C^*)) = L \setminus \mathrm{Pref}(C^*) \tag{17.12}$$

$$(\mathrm{Pref}(C) \setminus C)\Sigma \cap \mathrm{Pref}(C^*) = (\mathrm{Pref}(C) \setminus C)\Sigma \cap \mathrm{Pref}(C), \tag{17.13}$$

Proof. Equation (17.12): Since $\varepsilon \in C^*$ we have $L \setminus \mathrm{Pref}(C^*) \subseteq C^*(L \setminus \mathrm{Pref}(C^*))$. Next, consider $u \in C^*$ and $v \in L \setminus \mathrm{Pref}(C^*)$. Now, Theorem 17.27 would yield $v \in \mathrm{Pref}(C^*)$ for $uv \in \mathrm{Pref}(C^*)$. Thus, we must have $uv \in L \setminus \mathrm{Pref}(C^*)$.

Equation (17.13): Let $ux = v_1 \cdots v_n$ for $u \in \mathrm{Pref}(C) \setminus C$, $x \in \Sigma$, and $v_1, \ldots, v_{n-1} \in C$ and $v_n \in \mathrm{Pref}(C)$. We have $ux \notin C\Sigma^+$. Thus $n = 1$ and so $ux \in \mathrm{Pref}(C)$. □

Lastly, we collect some facts on PDFAs accepting prefix codes. Related statements to Item 1 and 2 from 17.30 have been given in [194, Prop. 2.2].

Proposition 17.30 *Let $L \subseteq \Sigma^+$ be a regular language.*

1. *L is a prefix code if and only if for every trim PDFA $\mathcal{B} = (P, \Sigma, \mu, p_0, E)$ with $L = L(\mathcal{B})$ the states in E have no outgoing transitions, i.e., $\mu(p, x)$ is undefined for each $p \in E$ and $x \in \Sigma$.*
2. *If L is a prefix code, then the complete and deterministic minimal automaton $\mathrm{MinAut}(L)$ and a PDFA with the least number of states accepting L both have a single final state.*
3. *If L is a prefix code and $\mathcal{B} = (P, \Sigma, \mu, p_0, \{e\})$ a trim PDFA with $L = L(\mathcal{B})$, then there exists an exactly synchronizing word $w \in L(\mathcal{B})$ with $\mu(P, w) = \{e\}$.*

Proof. 1: Let $\mathcal{B} = (P, \Sigma, \mu, p_0, E)$ be a trim PDFA with $L = L(\mathcal{B})$. First, suppose $\mu(p, x)$ is defined for some $p \in E$ and $x \in \Sigma$. As $\mathcal{B}$ is trim, there exist $u, v \in \Sigma^*$ with $\mu(\mu(p, x), u) \in E$ and $\mu(p_0, v) = p$. As $v \in L$ is a proper prefix of $vxu \in L$, L is not a prefix code. Conversely, suppose L is not a prefix code and $L = L(\mathcal{B})$. Then there exist $u, v \in L$ such that $v = uxu'$ for some $x \in \Sigma$ and $u' \in \Sigma^*$. Then $\mu(p_0, u) \in E$ and $\mu(p_0, ux)$ is defined.

2: For $u, v \in L$ and $w \in \Sigma^+$ we have $uw, vw \notin L$ since L is a prefix code, which gives $u \equiv_L v$. Therefore we obtain the claim for the complete and deterministic minimal automaton. The PDFA with the least number of states (up to renaming of the states) is obtained from the minimal automaton by omitting all non-coaccessible states (which form a set of size at most one), i.e., by omitting the state $[u]_{\equiv_L}$ for $u \notin \mathrm{Pref}(L)$ if such a word u exists. Formally, let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ where $P = \{[u]_{\equiv_L} \mid u \in \mathrm{Pref}(L)\}$, $p_0 = [\varepsilon]_{\equiv_L}$, $F = \{[u]_{\equiv_L} \mid u \in L\}$, and $\mu([u]_{\equiv_L}, x) = [ux]_{\equiv_L}$ for $x \in \Sigma$ and $u \in \mathrm{Pref}(L)$. Since L is regular, P is finite. We have $L = L(\mathcal{B})$ and F is a singleton, which gives the claim.

3: Set $n = |P|$. By Item 1 the state e has no outgoing transitions. Inductively, define a sequence of subsets of states $P_0, P_1, P_2, \ldots$, all containing the state e, by setting $P_0 = P$ and, for $i > 0$, $P_i = \mu(P_{i-1}, u_i)$ where u_i is some word with $\mu(p, u_i) = e$ if there exists $p \in P_{i-1} \setminus \{e\}$ and $u_i = \varepsilon$ otherwise. Then, for $|P_i| > 1$, as $u_i \neq \varepsilon$ yields that $\mu(e, u_i)$ is undefined, it is $|P_{i+1}| < |P_i|$. So $\mu(P, u_0 u_1 \cdots u_{n-1}) = \{e\}$ and there exist $p \in P$ and $v \in \Sigma^*$ such that $\mu(p, u_0 u_1 \cdots u_{n-1}) = e$ and $\mu(p_0, v) = p$. Then for $w = v u_0 u_1 \cdots u_{n-1}$ we have $w \in L(\mathcal{B})$ and $\mu(P, w) = \{e\}$. □

Example 17.31 Consider the prefix code $C = \{b, ab, aab\}$. Then we obtain a trim PDFA $\mathcal{B}$ accepting C by setting $\mathcal{B} = (P, \{a, b\}, \mu, \varepsilon, \{e\})$ with $P = \mathrm{Pref}(C) \setminus C \cup \{e\}$, i.e., the proper prefixes of C and e are considered as states of $\mathcal{B}$, and

$$\mu(u, x) = \begin{cases} e & \text{if } ux \in C, \\ ux & \text{if } ux \in \mathrm{Pref}(C) \setminus C. \end{cases}$$

We have $\mu(P, b) = \{e\}$ (we also obtain this word by following the procedure from the proof of Theorem 17.30, Item 3). Other exactly synchronizing words are aa and aab with $\mu(P, aa) = \{aa\}$ and $\mu(P, aab) = \{e\}$.

17.4.3 Relations to the Levenshtein-Romanov Mapping

Here, we state characterizations and conditions for prefix, suffix, bifix, and infix codes and overlap-free languages and combinations of these properties. We collect several conditions here, but our focus is on those involving the Levenshtein-Romanov mapping.

Proposition 17.32 *Let $L \subseteq \Sigma^+$ be non-empty. Then the following are equivalent.*

1. *L is a prefix code.*
2. *$\mathrm{Pref}(L) \setminus L$ equals the set of proper prefixes $\bigcup_{u \in L} (\mathrm{Pref}(u) \setminus \{u\})$ of words from L.*
3. *We have $\sigma_{\mathrm{Pref}(L) \setminus \{u\}}(u) \neq u$ for all $u \in L$.*[9]
4. *For all $u \in L$ and $v \in \Sigma^+$ we have $\sigma_L(uv) \neq uv$.*

Proof. 1 ⇒ 2: The set $\mathrm{Pref}(L) \setminus L$ is always a subset of the set of proper prefixes of words from L. Conversely, if $u \in \mathrm{Pref}(L)$ such that $uv \in L$ for some non-empty $v \in \Sigma^+$, then $u \notin L$ by the defining condition of a prefix code.

[9] With Equation (17.11), this is equivalent to L being a singleton or, otherwise, to $\sigma_{L \setminus \{u\}}(u) \neq u$ for all $u \in L$.

$2 \Rightarrow 3$: Let $u \in L$ with $\sigma_{\mathrm{Pref}(L)\setminus u}(u) = u$. Then u is a proper prefix of a word $v \in L$ with $u \neq v$. We have $u \notin \mathrm{Pref}(L) \setminus L$ and thus $\mathrm{Pref}(L) \setminus L$ does not equal the set of proper prefixes.

$3 \Rightarrow 4$: Suppose $\sigma_L(uv) = uv$ for $u \in L$ and $v \in \Sigma^+$. Then we have $uv \in \mathrm{Pref}(L)$ where $uv \neq u$. Thus L is not a singleton set and we find $\sigma_{L\setminus\{u\}}(uv) = uv$.

$4 \Rightarrow 1$: If $\sigma_L(uv) = uv$ for $u \in L$ and $v \in \Sigma^+$, then $uv \in \mathrm{Pref}(L)$ and thus $uvw \in L$ for some $w \in \Sigma^*$. Since $vw \neq \varepsilon$, L is not a prefix code in this case. □

Proposition 17.33 *Let $L \subseteq \Sigma^+$ be non-empty. Then the following are equivalent.*

1. *L is a suffix code.*[10]
2. *$\mathrm{Suff}(L) \setminus L$ equals the set of proper suffixes $\bigcup_{u\in L}(\mathrm{Suff}(u) \setminus \{u\})$ of words from L.*
3. *We have $\sigma_L(uv) \notin L \setminus \{v\}$ for all $v \in L$ and $u \in \Sigma^+$.*

Proof. $1 \Rightarrow 2$: The set $\mathrm{Suff}(L) \setminus L$ is contained in the set of proper suffixes for any language. Conversely, if u is a proper suffix of a word from L, then it is a suffix not contained in L if L is a suffix code, i.e., it is contained in $\mathrm{Suff}(L) \setminus L$.

$2 \Rightarrow 3$: Suppose $\sigma_L(uv) \in L \setminus \{v\}$ for some $v \in L$ and $u \in \Sigma^+$. Then $\sigma_L(uv)$ is a word in L having v as a proper suffix, but we have $v \notin \mathrm{Suff}(L) \setminus L$ since $v \in L$.

$3 \Rightarrow 1$: For $uv \in L$ with $v \in L$ and $u \in \Sigma^+$ we have $\sigma_L(uv) = uv \in L \setminus \{v\}$. □

Example 17.34 $L = \{aaab, ab, aabb\}$ is not a suffix code and $\sigma_{L\setminus\{u\}}(u) \notin L$ for all $u \in L$.

Proposition 17.35 *Let $L \subseteq \Sigma^+$ be non-empty. Then the following are equivalent.*

1. *L is a bifix code.*
2. *We have $u, v \notin \mathrm{Suff}(u) \cap \mathrm{Pref}(v)$ for all $u, v \in L$ with $u \neq v$.*
3. *We have $\mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L = \emptyset$ for all $u \in L$.*

Proof. $1 \Leftrightarrow 2$: For distinct $u, v \in L$ we have $u \in \mathrm{Suff}(u) \cap \mathrm{Pref}(v)$ (resp. $v \in \mathrm{Suff}(u) \cap \mathrm{Pref}(v)$) if and only if u is a proper prefix of v (resp. v is a proper suffix of v) in L, which gives the claim.

$2 \Leftrightarrow 3$: By Theorem 17.8, Item 4, we have $v \in \mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L$ if and only if v is a proper suffix of u or $v = u$ and u is a proper prefix of some word from L, which gives the claim. □

Clearly, $L \subseteq \Sigma^+$ is a prefix (resp. suffix) code if and only if $u \notin \mathrm{Suff}(u) \cap \mathrm{Pref}(v)$ (resp. $u \notin \mathrm{Suff}(v) \cap \mathrm{Pref}(u)$) for distinct $u, v \in L$, giving similar characterizations as Item 2 of Theorem 17.35.

Proposition 17.36 *Let $L \subseteq \Sigma^+$ be non-empty. Then the following are equivalent.*

1. *L is an infix code.*
2. *$L\Sigma^+ \cap \mathrm{Suff}(L) = \Sigma^+L \cap L = \emptyset$.*
3. *$\Sigma^*L\Sigma^+ \cap L = \Sigma^+L\Sigma^* \cap L = \emptyset$.*
4. *$\Sigma^*L\Sigma^+ \cap L = \Sigma^+L \cap L = \emptyset$.*

[10] By Theorem 17.8, Item 4, L being a suffix code is also equivalent to $\mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}\right) \cap (L \setminus \{u\}) = \emptyset$ for all $u \in L$.

5. $\Sigma^+ L \cap \mathrm{Pref}(L) = L\Sigma^+ \cap L = \emptyset$.
6. *L is a prefix code such that $\sigma_L(uv) = v$ for all $u \in \Sigma^*$ and $v \in L$.*
7. *We have $uv \notin \mathrm{Pref}(L)$ and $vu \notin L$ for all $u \in \Sigma^+$ and $v \in L$.*
8. *We have* $\mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L = \emptyset$ *for all* $u \in \mathrm{Pref}(L)$.

Proof. The equivalences from Items 1 to 5 are clear.

5 $\Rightarrow$ 6: The condition $\Sigma^+ L \cap L = \emptyset$ implies that L is a prefix code. Since $v \in L$ we have $\sigma_L(uv) \in \Sigma^* v \cap \mathrm{Pref}(L)$. By assumption, $\Sigma^+ v \cap \mathrm{Pref}(L)$ is empty. Thus $\sigma_L(uv) = v$.

6 $\Rightarrow$ 7: Let $v \in L$ and $u \in \Sigma^+$. For $uv \in \mathrm{Pref}(L)$, by Equation (17.4) we have $\sigma_L(uv) = uv \neq v$. For $vu \in L$, L is not a prefix code.

7 $\Rightarrow$ 8: Let $v \in \mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)\right) \cap L$. By Item 4 of Theorem 17.8, v is a proper suffix of u or we have $u = v$ and u is a proper prefix of some word from L. In the first case, we have $wv \in L$ for some $w \in \Sigma^+$ and in the latter case $vw \in L$ for some $w \in \Sigma^+$. Thus, Item 7 does not hold.

8 $\Rightarrow$ 1: Let $v \in L$ and $uvw \in L$ with $u \neq \varepsilon$ or $w \neq \varepsilon$. For $u = \varepsilon$, v is a proper prefix of some word from L. Thus we have $v = \sigma_{\mathrm{Pref}(L)\setminus\{v\}}(v)$ and $v \in \mathrm{Pref}(L)$. So, Item 8 does not hold. Next, suppose $u \neq \varepsilon$. Since $v \in L$ and $uv \neq v$ we have $\sigma_{\mathrm{Pref}(L)\setminus\{uv\}}(uv) = w'v$ for some $w' \in \Sigma^*$. Thus $v \in \mathrm{Suff}\left(\sigma_{\mathrm{Pref}(L)\setminus\{uv\}}(uv)\right) \cap L$. Since $uv \in \mathrm{Pref}(L)$, Item 8 does not hold. □

Example 17.37 For $L = \{a, ab\}$, we have $\sigma_L(uv) = v$ for all $u \in \Sigma^*$ and $v \in L$, but L is not a prefix code. Hence, the assumption that L is a prefix code in Theorem 17.36, Item 6 is necessary.

Similar characterizations as in Items 2 to 5 from Theorem 17.36 can also be easily given for prefix and suffix codes.

For $u, v \in \Sigma^*$, $\sigma_{\{v\}}(u)$ is the longest word w such that $u \in w\Sigma^*$ and $v \in \Sigma^* w$. This suggests that we can relate overlap-freeness to the Levenshtein-Romanov mapping. The next statements give some such relations.

Proposition 17.38 *Let $L \subseteq \Sigma^+$ be non-empty. Then the following are equivalent.*

1. *L is overlap-free.*
2. *For all $u \in \Sigma^+$ we have $u\Sigma^+ \cap L = \emptyset$ or $\Sigma^+ u \cap L = \emptyset$.*
3. *For all $u, v \in L$ we have* $\mathrm{Suff}(u) \cap \mathrm{Pref}(v) \subseteq \{\varepsilon, u, v\}$.
4. *For all $u, v \in L$ we have $\sigma_{\{v\}}(u) \in \{\varepsilon, u, v\}$ and* $\mathrm{Suff}(u) \cap \mathrm{Pref}(u) \subseteq \{\varepsilon, u\}$.
5. *For all $u, v \in L$ we have $\sigma_{\mathrm{Pref}(v)\setminus\{v\}}(u) \in \{\varepsilon, u\}$.*
6. *We have $uv \notin L$ for all non-empty and proper prefixes u, v of words from L.*

Proof. 1 $\Rightarrow$ 2: Suppose we have $u\Sigma^+ \cap L \neq \emptyset$ and $\Sigma^+ u \cap L \neq \emptyset$ for some $u \in \Sigma^+$. Then $uv \in L$ and $wu \in L$ for some $v, w \in \Sigma^+$ and so L is not overlap-free.

2 $\Rightarrow$ 3: Let $w \in \mathrm{Suff}(u) \cap \mathrm{Pref}(v)$ with $w \notin \{\varepsilon, u, w\}$ for $u, v \in L$. Then $u = xw$ and $v = wy$ for some $x, y \in \Sigma^+$ and so $u \in \Sigma^+ w$ and $v \in w\Sigma^+$.

3 $\Rightarrow$ 4: By Equation (17.1) and the assumption we have $\sigma_{\{v\}}(u) \in \mathrm{Suff}(u) \cap \mathrm{Pref}(v) \subseteq \{\varepsilon, u, v\}$. From Item 3 we also get $\mathrm{Suff}(u) \cap \mathrm{Pref}(u) \subseteq \{\varepsilon, u\}$.

4 $\Rightarrow$ 5: Suppose $\sigma_{\mathrm{Pref}(v)\setminus\{v\}}(u) = w$ for some $w \notin \{\varepsilon, u\}$. Then we have

$$\sigma_{\{v\}}(u) = \begin{cases} u & \text{if } u \text{ is a prefix of } v, \\ w & \text{otherwise.} \end{cases}$$

In the first case, we have $w \in \mathrm{Suff}(u) \cap \mathrm{Pref}(u)$, in the second case we have $\sigma_{\{v\}}(u) \notin \{\varepsilon, u, v\}$.

5 $\Rightarrow$ 6: Suppose we have $uv \in L$ for non-empty and proper prefixes u, v of words from L. Then there exists $w \in \Sigma^+$ such that $vw \in L$. We have $\sigma_{\mathrm{Pref}(vw)\setminus\{vw\}}(uv) = xv$ for some $x \in \Sigma^*$. By assumption, $x = u$. By Equation (17.4), in this case, uv is a prefix of vw. Thus, uv has v as a proper prefix and we find that $\sigma_{\mathrm{Pref}(uv)\setminus\{uv\}}(uv)$ has v as a suffix. Since uv is not a prefix of a word from $\mathrm{Pref}(uv) \setminus \{uv\}$, we also have $\sigma_{\mathrm{Pref}(uv)\setminus\{uv\}}(uv) \neq uv$. Thus Item 5 does not hold.

6 $\Rightarrow$ 1: If L is not overlap-free, then $uv, vw \in L$ for some non-empty $u, v, w \in \Sigma^+$ where u, v are non-empty and proper prefixes of words from L. Thus, Item 6 does not hold. □

Remark 17.39 Note that $\sigma_{\mathrm{Pref}(v)\setminus\{v\}}(u) = \varepsilon$ implies $\sigma_{\{v\}}(u) \in \{\varepsilon, v\}$, but the converse implication does not hold. For example, we have $\sigma_{\{aa\}}(baa) = aa$, but $\sigma_{\{a\}}(baa) = a$.

Corollary 17.40 *Let $u \in \Sigma^*$. Then the following are equivalent.*

1. *$\{u\}$ is overlap-free.*
2. *$\sigma_{\mathrm{Pref}(u)\setminus\{u\}}(u) = \varepsilon$.*
3. *$\mathrm{Suff}(u) \cap \mathrm{Pref}(u) \subseteq \{\varepsilon, u\}$.*

Proposition 17.41 *Let $L \subseteq \Sigma^+$ and $u \in L$ where L contains at least two words. Then we have $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) = \varepsilon$ if and only if $\{u\}$ is overlap-free and $\sigma_{L\setminus\{u\}}(u) = \varepsilon$.*

Proof. First, suppose $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) = \varepsilon$. Since $u \in L$ we have $\mathrm{Pref}(L) \setminus \{u\} = (L \setminus \{u\}) \cup (\mathrm{Pref}(u) \setminus \{u\})$. Thus, by Theorem 17.4, Item 7, we find $\sigma_{L\setminus\{u\}}(u) = \varepsilon$ and $\sigma_{\mathrm{Pref}(u)\setminus\{u\}}(u) = \varepsilon$, and thus u is overlap-free by Theorem 17.40. Conversely, if $\{u\}$ is overlap-free and $\sigma_{L\setminus\{u\}}(u) = \varepsilon$, then by Theorem 17.40 and Equation (17.11) we obtain $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) = \varepsilon$. □

Example 17.42 Let $L = \{a, ab, ba\}$. Then L is not overlap-free. Note that here we have $uv \notin L$ for all $u, v \in \mathrm{Pref}(L) \setminus L = \{\varepsilon, b\}$.

Suppose $uv, vw \in L$ for non-empty $u, v, w \in \Sigma^+$. Then $uvw \in (\bigcup_{x\in L} \mathrm{Pref}(x) \setminus \{x\})\, L \cap L\Sigma^+$. Thus, if the latter set is empty, then L is overlap-free. The following statement also shows that when L is a prefix, this condition characterizes overlap-freeness of L.

Proposition 17.43 *Let $L \subseteq \Sigma^+$ be non-empty. Then the following are equivalent.*

1. *L is an overlap-free prefix code.*
2. *For all $u, v \in L$ we have $\mathrm{Suff}(u) \cap \mathrm{Pref}(v) \subseteq \{\varepsilon, v\}$.*
3. *For all $u, v \in L$ we have $\sigma_{\mathrm{Pref}(u)\setminus\{u\}}(v) = \varepsilon$.*
4. *For all $u \in L$, the language $\{u\}$ is overlap-free and $\sigma_{\{v\}}(u) \in \{\varepsilon, v\}$ for distinct $u, v \in L$.*
5. *For all $u \in L$, $\{u\}$ is overlap-free and $\sigma_{L\setminus\{u\}}(u) \in (L \setminus \{u\}) \cup \{\varepsilon\}$ if $L \setminus \{u\} \neq \emptyset$.*
6. *For all $u \in L$ we have $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) \in (L \setminus \{u\}) \cup \{\varepsilon\}$.*

7. *We have $uv \notin L$ for all $u, v \in \Sigma^+$ such that $u \in L$ or $v \in \operatorname{Pref}(L) \setminus L$.*
8. *L is a prefix code such that $(\bigcup_{u \in L} \operatorname{Pref}(u) \setminus \{u\})\, L \cap L\Sigma^+ = \emptyset$.*

Proof. 1 $\Rightarrow$ 2: This follows from Theorem 17.38 and the fact that $u \notin \operatorname{Pref}(v)$ for distinct $u, v \in L$ when L is a prefix code.

2 $\Rightarrow$ 3: We have $\sigma_{\operatorname{Pref}(u) \setminus \{u\}}(v) \in \operatorname{Suff}(v) \cap (\operatorname{Pref}(u) \setminus \{u\}) \subseteq \{\varepsilon\}$ by assumption and the definition of σ_L.

3 $\Rightarrow$ 4: By Theorem 17.40, we find that $\{u\}$ is overlap-free for each $u \in L$. Next, for $u, v \in \Sigma^*$ we have

$$\sigma_{\{v\}}(u) = \begin{cases} \sigma_{\operatorname{Pref}(v) \setminus \{v\}}(u) & \text{if } v \notin \operatorname{Suff}(u), \\ v & \text{otherwise.} \end{cases}$$

Thus, we get $\sigma_{\{v\}}(u) \in \{\varepsilon, v\}$ if $u, v \in L$ by Item 3.

4 $\Rightarrow$ 5: We have $\sigma_{L \setminus \{u\}}(u) = \sigma_{\{v\}}(u)$ for some $v \in L$ with $v \neq u$. By assumption, $\sigma_{\{v\}}(u) \in \{\varepsilon, v\}$ and this gives the claim.

5 $\Rightarrow$ 6: This follows from Equation (17.11) and Theorem 17.40.

6 $\Rightarrow$ 7: Let u be a shortest non-empty word such that $uv \in L$ for some $v \in \Sigma^+$ with $u \in L$ or $v \in \operatorname{Pref}(L) \setminus L$. If $u \in L$, then $\sigma_{\operatorname{Pref}(L) \setminus \{u\}}(u) = u$. Next, suppose $v \in \operatorname{Pref}(L) \setminus L$. Then $\sigma_{\operatorname{Pref}(L) \setminus \{uv\}}(uv) = xv$ for some $x \in \Sigma^*$. Assume $\sigma_{\operatorname{Pref}(L) \setminus \{uv\}}(uv) \in L$. Since $v \notin L$, this gives $x \neq \varepsilon$. By the minimal choice of u, we find $x = u$. Thus we have $\sigma_{\operatorname{Pref}(L) \setminus \{uv\}}(uv) = uv$. This gives the claim.

7 $\Rightarrow$ 8: If L is not a prefix code, then $uv \in L$ for some $u \in L$ and $v \in \Sigma^*$. Next, suppose L is a prefix code and $uv = wx$ for u being a proper prefix of some word from L and where $v, w \in L$ and $x \in \Sigma^+$. Since L is a prefix code, we have $w = uy$ and $v = yx$ for some $y \in \Sigma^+$ where $y \notin L$. Then we have $uy \in L$ with $y \in \operatorname{Pref}(L) \setminus L$.

8 $\Rightarrow$ 1: Suppose $uv \in L$ and $vw \in L$ for some $u, v, w \in \Sigma^+$. Since L is a prefix code, we have $u \notin L$. Thus $uvw \in (\bigcup_{u \in L} \operatorname{Pref}(u) \setminus \{u\})\, L \cap L\Sigma^+$. □

Remark 17.44 Let $L = \{aabb, bba, abb\}$ and $u = aabb$. Then we have $\sigma_{\operatorname{Pref}(L) \setminus \{u\}}(u) = abb$ where $abb \in L \setminus \{u\}$, but $\sigma_{\{bba\}}(u) = bb \notin \{\varepsilon, bba\}$. Thus the equivalences of Item 4 and Item 6 in Theorem 17.43 do not hold for a fixed $u \in L$, but only when quantified over all $u \in L$.

Corollary 17.45 *Let $C \subseteq \Sigma^+$ be a prefix code. Then C is overlap-free if and only if $(\operatorname{Pref}(C) \setminus C)C \cap C\Sigma^+ = \emptyset$.*

Proof. Let C be a prefix code. Then its set of proper prefixes $\bigcup_{u \in L} \operatorname{Pref}(u) \setminus \{u\}$ equals $\operatorname{Pref}(C) \setminus C$. Thus the claim follows from Theorem 17.43. □

Remark 17.46 Let $L \subseteq \Sigma^*$ be non-empty and $u \in \Sigma^*$. We have

$$\sigma_{\operatorname{Pref}(L) \setminus \{u\}}(u) \notin \bigcup_{v \in L} (\operatorname{Pref}(v) \setminus \{\varepsilon, v\}) \Rightarrow \sigma_{\operatorname{Pref}(L) \setminus \{u\}}(u) \in (L \setminus \{u\}) \cup \{\varepsilon\}.$$

The converse implication holds if no proper suffix of u contained in L is a proper prefix of some other word from L (which holds if L is a prefix code). This yields that Item 6 from Theorem 17.43 can also be stated by saying the image of the Levenshtein-Romanov

mapping is not in $\bigcup_{v\in L}(\mathrm{Pref}(v) \setminus \{\varepsilon, v\})$ since either condition, when quantified over all words $u \in L$, yields that L is a prefix code.

Proposition 17.47 *Let $L \subseteq \Sigma^+$. Then the following are equivalent.*

1. *L is an overlap-free suffix code.*
2. *For all $u, v \in L$ we have $\mathrm{Suff}(u) \cap \mathrm{Pref}(v) \subseteq \{\varepsilon, u\}$.*
3. *$\{u\}$ is overlap-free for $u \in L$ and $\sigma_{\{v\}}(u) \in \{\varepsilon, u\}$ for $u, v \in L$.*
4. *$uv \notin L$ for all $u, v \in \mathrm{Pref}(L) \setminus \{\varepsilon\}$.*

Proof. 1 $\Leftrightarrow$ 2: This follows from Item 3 of Theorem 17.38 and the fact that for distinct $u, v \in L$ we have $v \notin \mathrm{Suff}(u)$ if and only if L is a suffix code.

1 $\Leftrightarrow$ 3: This follows from Item 4 of Theorem 17.38 and Equation (17.6).

1 $\Leftrightarrow$ 4: This follows from Item 6 of Theorem 17.38 and the definition of a suffix code. □

Proposition 17.48 *Let $L \subseteq \Sigma^+$ be non-empty. Then the following are equivalent.*

1. *L is an overlap-free bifix code.*
2. *For $u, v \in L$ we have*

$$\mathrm{Suff}(u) \cap \mathrm{Pref}(v) = \begin{cases} \{\varepsilon\} & \text{if } u \neq v, \\ \{\varepsilon, u\} & \text{if } u = v. \end{cases}$$

3. *For $u \in L$ we have $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u) = \varepsilon$.*[11]
4. *We have $uv \notin L$ for all $u, v \in \Sigma^+$ with $u \in L$ or $v \in \mathrm{Pref}(L)$.*

Proof. 1 $\Leftrightarrow$ 2: This follows from Theorems 17.35 and 17.38.

1 $\Leftrightarrow$ 3: First, suppose L is an overlap-free bifix code. By Theorem 17.43, Item 6 we have that $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)$ is in $(L \setminus \{u\}) \cup \{\varepsilon\}$. Since $\sigma_{\mathrm{Pref}(L)\setminus\{u\}}(u)$ is a suffix of u, it cannot be contained in $L \setminus \{u\}$ since L is a suffix code. Conversely, by Theorem 17.43, Item 6 this condition implies that L is an overlap-free prefix code. Next, if $uv \in L$ for $u \in \Sigma^+$ and $v \in L$, then $\sigma_{\mathrm{Pref}(L)\setminus\{uv\}}(uv)$ has the non-empty word v as a suffix. Thus, L must be a suffix code.

1 $\Leftrightarrow$ 4: This follows from Theorem 17.43, Item 7 and the definition of a suffix code. □

Note that $L \subseteq \Sigma^+$ is a prefix (resp. suffix, bifix, infix, overlap-free) code/language if and only if $\{u, v\}$ is a prefix (resp. suffix, bifix, infix, overlap-free) code/language for all $u, v \in L$. So, the conditions from the above statements characterizing those language classes can be written as conditions that must be true for all pairs (of possibly equal) words in L.

The following further characterizations were given in [350] for non-empty $L \subseteq \Sigma^+$:

1. $\mathrm{Pref}(L) \cap \Sigma^* L = L$ if and only if $\sigma_L(u) \in L$ for all $u \in \Sigma^* L$,

[11] As stated in Theorem 17.41, this is equivalent to $\{u\}$ being overlap-free and $\sigma_{L\setminus\{u\}}(u) = \varepsilon$ if $L \setminus \{u\} \neq \emptyset$.

2. L is an overlap-free prefix code if and only if $\sigma_L(uv) = \sigma_L(v)$ for all $u \in L$ and $v \in \Sigma^+$,
3. L is a suffix code such that $\mathrm{Pref}(L) \cap \Sigma^* L = L$ if and only if $\sigma_L(uv) = v$ for all $u \in \Sigma^+$ and $v \in L$ (this condition also appears in Theorem 17.36, Item 6).

In the next section we will investigate overlap-free infix codes, i.e., solid codes, in more detail.

We always have $L \cup \{\varepsilon\} \subseteq \mathrm{Pref}(L) \cap \mathrm{Suff}(L)$ for any language $L \subseteq \Sigma^*$. Lastly, we connect the converse inclusion to overlap-freeness.

Proposition 17.49 *Let $C \subseteq \Sigma^+$ be a prefix or a suffix code. If $\mathrm{Pref}(C) \cap \mathrm{Suff}(C) \subseteq C \cup \{\varepsilon\}$, then C is overlap-free.*

Proof. Let $uv, vw \in C$ for non-empty u, v, w. Then we have $v \in \mathrm{Pref}(C) \cap \mathrm{Suff}(C)$ and since C is a prefix or suffix code, we have $v \notin C$, i.e., $\mathrm{Pref}(C) \cap \mathrm{Suff}(C)$ is not a subset of C. □

Proposition 17.50 *Let $L \subseteq \Sigma^*$. If L is overlap-free, then $\mathrm{Pref}(L) \cap \mathrm{Suff}(L) \subseteq L \cup \{\varepsilon\}$.*

Proof. Let $u \in \mathrm{Pref}(L) \cap \mathrm{Suff}(L)$ with $u \neq \varepsilon$. Then $uv, wu \in L$ for some $v, w \in \Sigma^*$. If L is overlap-free, then $v = \varepsilon$ or $w = \varepsilon$ and each case gives $u \in L$, which shows the claim. □

Thus, we find that a prefix or suffix code C is overlap-free if and only if $\mathrm{Pref}(C) \cap \mathrm{Suff}(C) \subseteq C \cup \{\varepsilon\}$.

17.4.4 Characterizing Solid Codes

Solid codes provide outstanding fault-tolerance when used for information transmission through a noisy channel w.r.t. symbol substitutions and synchronization errors. In this context, they were introduced by Levenshtein [409] under the name *strongly regular codes*, and further investigated by Romanov [520]. Shyr and Yu [570] introduced solid codes to investigate disjunctive domains, but did not recognize the connection to the earlier work by Levenshtein and Romanov. Later, Jürgensen and Yu [351] observed that strongly regular codes describe precisely the solid codes. For further information, we refer to the surveys [348, 349], or the introductory section of [350]. Recall that Items 8 and 14 from Theorem 17.14 gave an infinite binary solid code and a method to construct a solid codes, respectively.

In our investigation on the constrained synchronization problem and synchronization conditions for automata obtained in reductions, we discovered (to the best of our knowledge) new characterizations of solid codes, which we present in the following (see the discussion section, Section 18.5, page 501, for some background on how this characterization arose in the context of the constrained synchronization problem).

Here, first, in Theorem 17.51, we recap known characterizations. Theorem 17.53 collects further characterizations of solid codes which are new and connect solid codes to various language equations. Lastly, Theorems 17.56, 17.60, and 17.61 give two other new characterizations.

Theorem 17.51 *Let $L \subseteq \Sigma^+$ with $L \neq \emptyset$. Then the following are equivalent.*

1. *L is a solid code.*
2. *(Jürgensen & Yu [351]) For every word $w \in \Sigma^*$ we have $w = u_0 w_1 u_1 w_2 u_2 \cdots w_n u_n$ for some $n \geq 0$ and unique words $u_0, u_1, \ldots, u_n \in \Sigma^* \setminus \Sigma^* L \Sigma^*$ and $w_1, w_2, \ldots, w_n \in L$.*
3. *(Staiger [350], Romanov [520], Levenshtein [409]) There exists a state-invariant decoder without look-ahead for L (see [350, Def. 9] for a definition of these decoders for finite solid codes and extensions to infinite solid codes).*
4. *(Liu, Guo & Shum [417, Thm. 2.4]) L is an infix code and $\{u_1 \cdots u_n\}$ is overlap-free for all sequences of distinct words $u_1, \ldots, u_n \in L$ with $n \geq 1$.*[12]
5. *(Liu, Guo & Shum [417, Thm. 2.4]) L is an infix code such that $\{u\}$ is overlap-free for all $u \in L$ and $\{uv\}$ is overlap-free for $u, v \in L$ with $u \neq v$.*

Remark 17.52 Since for distinct $u_1, \ldots, u_n \in L$ we have $u_i \cdots u_n \neq u_1 \cdots u_{n-i+1}$ for all $i \in \{1, \ldots, n\}$. Thus L is overlap-free if $\{u_1 \cdots u_n\}$ is overlap-free for distinct $u_1, \ldots, u_n \in L$. Next, if for distinct $u, v \in L$ we have $u = xy$ and $v = yz$ for non-empty $x, y, z \in \Sigma^+$, then $vu = yzxy$ and so $\{vu\}$ is not overlap-free. Thus, if $\{u\}$ for $u \in L$ and $\{uv\}$ for distinct $u, v \in L$ are overlap-free, then L is overlap-free. This shows that the additional conditions in Items 4 and 5 of Theorem 17.51, beyond being an infix code, are equivalent to overlap-freeness (without any additional assumptions on L).

Theorem 17.53 *Let $L \subseteq \Sigma^+$. Then the following are equivalent:*

1. *L is a solid code.*
2. *L is a bifix code and $(\mathrm{Pref}(L) \setminus L)L \cap \Sigma^* L \Sigma^+ = \emptyset$.*
3. *L is a suffix code and $(\bigcup_{u \in L} (\mathrm{Pref}(u) \setminus \{u\}))\, L \cap \Sigma^* L \Sigma^+ = \emptyset$.*
4. *L is an infix code and $(\mathrm{Pref}(L) \setminus L)L \cap L\Sigma^+ = \emptyset$.*
5. *L is an infix code and $\mathrm{Pref}(L) \cap \mathrm{Suff}(L) = L \cup \{\varepsilon\}$.*

Proof. $1 \Rightarrow 2$: A solid code is an infix code, and so a bifix code, and overlap-free. Suppose $vw = xuy$ with $x \in \Sigma^*$, $y \in \Sigma^+$, $u, w \in L$, and $v \in \mathrm{Pref}(L) \setminus L$. By Theorem 17.45 we have $|x| > 0$. For $|x| \geq |v|$ (resp. $|y| \geq |w|$), we have $u \in \mathrm{Fact}(w)$ (resp. $u \in \mathrm{Fact}(v)$). For $0 < |x| < |v|$ and $0 < |y| < |w|$, we have $u = v'w'$ for a non-empty and proper suffix v' (resp. prefix w') of v (resp. w), so that L is not overlap-free. Hence, for $(\mathrm{Pref}(L) \setminus L)L \cap \Sigma^+ L \Sigma^+ \neq \emptyset$, we get that L is not a solid code, which gives the claim.

$2 \Rightarrow 3$: Since L is a prefix code, its set of proper prefixes equals $\mathrm{Pref}(L) \setminus L$, which gives the claim.

$3 \Rightarrow 4$: Suppose L is a suffix code as stated. Assume we have $uvw \in L$ with $u \in \Sigma^*$, $v \in L$, and $w \in \Sigma^+$. Let x be a proper prefix of some word in L. Then $xuvw \in \Sigma^* L \Sigma^+$ and this is excluded by assumption. Thus, L is an infix code. The assumption yields that $(\mathrm{Pref}(L) \setminus L)L \cap L\Sigma^+ = \emptyset$.

$4 \Rightarrow 5$: By Theorem 17.45, L is overlap-free. Thus, Theorem 17.50 gives $\mathrm{Pref}(L) \cap \mathrm{Suff}(L) \subseteq L \cup \{\varepsilon\}$. The inclusion $L \cup \{\varepsilon\} \subseteq \mathrm{Pref}(L) \cap \mathrm{Suff}(L)$ holds for any language.

[12] In [417, Thm. 2.4], if L is finite, then overlap-freeness is only stated to hold for $\{u_1 \cdots u_n\}$ if $2 \leq n \leq |L|$. However, this seems to be a typo, as for example $L = \{aba, bcb\}$ is an infix code and $\{ababcb\}$ and $\{bcbaba\}$ are each overlap-free, while L is not overlap-free and so not a solid code.

$5 \Rightarrow 1$: By Theorem 17.49, L is overlap-free and thus a solid code. □

Remark 17.54 As far as we are aware of, the condition $(\mathrm{Pref}(L) \setminus L)L \cap \Sigma^* L\Sigma^+ = \emptyset$ was never investigated in the literature before (apart from our own work [298] whose results are presented and extended in the present thesis). Note that $\mathrm{Pref}(L)L \subseteq L\Sigma^+$ holds for every language $L \subseteq \Sigma^*$.

We mention an implication for languages $L \subseteq \Sigma^*$ such that $(\mathrm{Pref}(L)\setminus L)L\cap\Sigma^* L\Sigma^+ = \emptyset$.

Proposition 17.55 *Let $L \subseteq \Sigma^*$ such that $(\mathrm{Pref}(L) \setminus L)L \cap \Sigma^* L\Sigma^+ = \emptyset$. Then $\mathrm{Pref}(L) \subseteq L$ or $uvw \in L$ with $u, v \in \Sigma^*$ and $v \in L$ implies $w = \varepsilon$.*

Proof. Suppose we have $x \in \mathrm{Pref}(L) \setminus L$ for some $x \in \Sigma^*$. Let $v \in L$ with $uvw \in L$ for some $u, w \in \Sigma^*$. Assume $|w| > 0$. Then we have $xuvw \in (\mathrm{Pref}(L) \setminus L)L \cap \Sigma^* L\Sigma^+$. Thus, we must have $|w| = 0$. □

Theorem 17.56 *Let $L \subseteq \Sigma^+$. Then L is a solid code if and only if for all $u \in \mathrm{Pref}(L)$ and $v \in L$ such that $uv = xwy$ for $x \in \Sigma^*$ and $y \in \Sigma^+$ and $w \in L$ we have $x = \varepsilon$ and $u = w$.*

Proof. First, suppose L is a solid code. Let $u \in \mathrm{Pref}(L)$, $v, w \in L$, and $uv = xwy$ for $x \in \Sigma^*$ and $y \in \Sigma^+$. By Theorem 17.53, Item 2, we must have $u \in L$. By Theorem 17.16, Item 1, L is comma-free. So $u, v \in L$ imply $x = \varepsilon$ and, as L is a prefix code, $w = u$. Conversely, suppose the stated condition holds. Then, clearly, L is comma-free. Next, suppose $u \in \mathrm{Pref}(L) \setminus L$ and $v, w \in L$ with $uv = xwy$ for some $x \in \Sigma^*$ and $y \in \Sigma^+$. By assumption, $x = \varepsilon$ and $u = w$. However, as $u \notin L$ we have $u \neq w$. So we must have $(\mathrm{Pref}(L) \setminus L)L \cap L\Sigma^+ = \emptyset$. □

Corollary 17.57 *Let $C \subseteq \Sigma^+$ be a solid code. Then $\mathrm{Pref}(C)C \cap \Sigma^+ C\Sigma^+ = \emptyset$.*

The condition $\mathrm{Pref}(C)C \cap \Sigma^+ C\Sigma^+ = \emptyset$ clearly implies that C is comma-free, but not every comma-free code fulfills it. For example, for the comma-free code $C = \{aba, cd, bc\}$ we have $abcd \in (\mathrm{Pref}(C) \setminus C)C \cap \Sigma bc\Sigma$.

The next examples show that $\mathrm{Pref}(C)C \cap \Sigma^+ C\Sigma^+ = \emptyset$ does not characterize solid codes.

Example 17.58
1. For the comma-free code $C = \{abb, aba\}$ from Theorem 17.14, Item 10 that is not a solid code we have $\mathrm{Pref}(C)C \cap \Sigma^+ C\Sigma^+ = \emptyset$.
2. Let $C = \{ab, bc\}$. Then C is not a solid code, but we have $\mathrm{Pref}(C)C \cap \Sigma^+ C\Sigma^+ = \emptyset$.

Proposition 17.59 *Let $L \subseteq \Sigma^+$. Then $\mathrm{Pref}(L)L \cap \Sigma^+ L\Sigma^+ = \emptyset$ if and only if for all $u \in \mathrm{Pref}(L)$ and $v \in L$ such that $uv = xwy$ for $x \in \Sigma^+$ and $y \in \Sigma^*$ and $w \in L$ we have $y = \varepsilon$ and $w = v$.*

Proof. First, suppose $\mathrm{Pref}(L)L \cap \Sigma^+ L\Sigma^+ = \emptyset$. In this case, we must have $y = \varepsilon$ since otherwise $uv \in \Sigma^+ L\Sigma^+$. Since L is a suffix code, this implies $w = v$.

Conversely, the condition implies $\mathrm{Pref}(L)L \cap \Sigma^+ L\Sigma^+ = \emptyset$. □

Corollary 17.60 *Let $L \subseteq \Sigma^+$. Then L is a solid code if and only if for all $u \in \mathrm{Pref}(L)$ and $v \in L$, if $uv = xwy$ for $w \in L$ and $x, y \in \Sigma^*$, then $x = \varepsilon$ and $w = u$ or $y = \varepsilon$ and $w = v$.*

Proof. Theorems 17.56, 17.57, and 17.59 show that a solid code fulfills the condition. Conversely, it implies the condition from Theorem 17.56 if $|y| > 0$ and thus that L is a solid code. □

Lastly, we give a characterization using the Levenshtein-Romanov mapping. It basically combines results from Section 17.4.3.

Proposition 17.61 *Let $L \subseteq \Sigma^+$ be non-empty. Then L is a solid code if and only if for all $u, v \in \Sigma^*$ the following is true:*

1. *$\sigma_L(uv) = uv$ implies $u \notin L$ or $v = \varepsilon$,*
2. *$\sigma_L(uv) \in L$ implies $v \notin \mathrm{Pref}(L) \setminus L$ or $v = \varepsilon$,*
3. *$\sigma_L(uv) \neq v$ implies $v \notin L$.*

Proof. First, suppose L is a solid code. Assume $\sigma_L(uv) = uv$. Since L is a prefix code, by Item 4 of Theorem 17.32 we find that $u \notin L$ or $v = \varepsilon$. Next, let $\sigma_L(uv) \in L$ and $v \in \mathrm{Pref}(L) \setminus L$. Then $\sigma_L(uv) = wv$ for some $w \in \Sigma^*$ with $|w| > 0$ since $wv \in L$. By overlap-freeness, this yields $v = \varepsilon$. Item 3 follows from Item 6 of Theorem 17.36 since L is an infix code.

Conversely, Item 1 yields that L is a prefix code by Theorem 17.32, Item 4. Then, Item 3 yields that L is an infix code by Item 6 of Theorem 17.36. Next, assume $uv, vw \in L$ for non-empty $u, v, w \in \Sigma^+$. We have $v \in \mathrm{Pref}(L) \setminus L$ since L is a prefix code and $\sigma_L(uv) = uv$. Thus, Item 2 does not hold. So, if all these conditions hold, we find that L is overlap-free. □

Remark 17.62 By Item 3 from Theorem 17.61, for a solid code C and $v \in C$, we have $\sigma_C(uv) = v$ for $u \in \Sigma^*$. By Item 2, we also have $\sigma_C(uw) \notin L$ for every proper and non-empty prefix w of a word from C. However, note that we do not have $\sigma_C(uw) = w$ in this case. For example, for the solid code $C = \{abb, aac\}$ we have $\sigma_C(uw) = uw$ for $u = a$ and $w = a$.

17.5 Bounded Languages

A language $L \subseteq \Sigma^*$ is *bounded* [245] if there exist words $u_1, \ldots, u_r \in \Sigma^*$ such that $L \subseteq u_1^* \cdots u_r^*$. The language L is called *letter-bounded* if each u_i is a letter, i.e., $u_i \in \Sigma$ for all $i \in \{1, \ldots, r\}$. It is called *strictly bounded* if it is letter-bounded and the letters $u_1, \ldots, u_r$ are pairwise distinct.

For example, finite languages are letter-bounded. The bounded and letter-bounded languages are closed under finite unions and concatenation since for $L \subseteq u_1^* \cdots u_r^*$ and $L' \subseteq v_1^* \cdots v_l^*$ both $L \cup L'$ and LL' are contained in $u_1^* \cdots u_r^* v_1^* \cdots v_l^*$.

Our first result characterizes subset of w^*. It was also stated in [245, Lem. 5.2].

Proposition 17.63 *Let $L \subseteq \Sigma^*$. Then there exists $w \in \Sigma^*$ such that $L \subseteq w^*$ if and only if, for all $u, v \in L$, there exists $w \in \Sigma^*$ such that $\{u, v\} \subseteq w^*$.*

Proof. If $L \subseteq w^*$ for some $w \in \Sigma^*$, then in particular $\{u, v\} \subseteq w^*$ for all $u, v \in L$. Conversely, suppose for all $u, v \in L$ there exists $w \in \Sigma^*$ with $\{u, v\} \subseteq w^*$. Assume $L \not\subseteq w^*$ for all $w \in \Sigma^*$. Then for each $w \in \Sigma^*$ with $v \in w^*$ for some $v \in L$ we have $u \notin w^*$ for some $u \in L$. Now, pick a shortest word $w_1 \in \Sigma^*$ such that $u \in L \setminus w_1^*$ and $v \in L \cap w_1^*$ for some $u, v \in \Sigma^+$. By assumption there exists $w_2 \in \Sigma^*$ such that $\{u, v\} \subseteq w_2^*$. So $w_1^i = v = w_2^j$ for some $i, j \geq 0$. We have $i, j \geq 1$ and $w_1, w_2 \neq \varepsilon$ as $|v| > 0$. By Theorem 17.20 there exists $w_3 \in \Sigma^*$ such that $w_1, w_2 \in w_3^*$. Let $k \geq 1$ be such that $w_1 = w_3^k$. As $u \in w_2^* \subseteq w_3^*$ and $u \notin w_1^*$ we have $k \geq 2$ so that $0 < |w_3| < |w_1|$. By the choice of w_1 and as $v \in L \cap w_3^*$ we find $L \subseteq w_3^*$, contradicting the assumption.[13] □

For regular subsets of w^* we have the following representation.

Proposition 17.64 *Let $L \subseteq w^*$ for some $w \in \Sigma^*$. Then L is regular if and only if $L = I \cdot (w^p)^* \cup F$ for finite sets $I, F \subseteq w^*$ and some $p > 0$.*

Proof. First, suppose $L \subseteq w^*$ is regular. By Theorem 6.87 there exist $0 \leq i < j$ such that $(w^i)^{-1}L = (w^j)^{-1}L$. Set $p = j - i$. Then, for all $k \geq i$ this gives

$$w^k \in L \Leftrightarrow w^{k+p} \in L.$$

This yields that for $F = \{w^k \mid 0 \leq k < i \wedge w^k \in L\}$ and $I = \{w^k \mid i \leq k < j \wedge w^k \in L\}$ we have $L = I \cdot (w^p)^* \cup F$. Conversely, a language of the form $L = I \cdot (w^p)^* \cup F$ for finite sets $I, F \subseteq w^*$ and some $p \geq 0$ is clearly regular and contained in w^*. □

The next representation theorem for bounded regular sets is from [246, Thm. 1.2].

Theorem 17.65 (Ginsburg and Spanier [246, Thm. 1.2]) *Let $w_1, \ldots, w_r \in \Sigma^*$. A language $L \subseteq w_1^* \cdots w_r^*$ is regular if and only if it is a finite union of sets of the form $L_1 \cdots L_r$, where each L_i is a regular language in w_i^* for $i \in \{1, \ldots, r\}$.*

Corollary 17.66 *$L \subseteq w_1^* \cdots w_r^*$ for $w_1, \ldots, w_r \in \Sigma^*$ is regular if and only if it is a finite union of sets of the form $w_1^{i_1}(w_1^{p_1})^* \cdots w_r^{i_r}(w_r^{p_r})^*$ for numbers $i_1, \ldots, i_r, p_1, \ldots, p_r \geq 0$.*

Proof. By Theorem 3.61, Item 2 we can write $I \cdot (w^p)^* \cup F$ for finite I and F as a finite union of sets of the form $w^i(w^q)^*$ with $i, q \geq 0$ (note that $q = 0$ is allowed). Thus, the claim follows from Theorems 17.64 and 17.65. □

[13] For a similar inductive argument: Let $L = \{u_0, u_1, u_2, \ldots\}$. Inductively, for $i \geq 1$ construct words $w_i \in \Sigma^*$ with $w_i \in w_{i+1}^*$ and $\{u_{i-1}, u_i\} \subseteq w_i^*$ in the following way: Let $w_1 \in \Sigma^*$ be a word such that $\{u_0, u_1\} \subseteq w_1^*$. For $i \geq 2$ we have $\{u_i, u_{i+1}\} \subseteq w^*$ for some $w \in \Sigma^*$. Thus, $u_i \in w_i^*$ (by induction hypothesis) and $u_i \in w^*$. By Theorem 17.20 we have $w_i, w \in w_{i+1}^*$ for some $w_{i+1} \in \Sigma^*$. Now, having such a sequence, $w_i \in w_{i+1}^*$ implies $|w_{i+1}| \leq |w_i|$. Thus there exists $N \geq 0$ such that $|w_N| = |w_{N+i}|$ for all $i \geq 0$. But then $w_i \in w_{i+1}^*$, gives $w_N = w_{N+i}$ for all $i \geq 0$. This yields $L \subseteq w_N^*$.

The function $\rho\colon \Sigma^* \to \mathbb{N}_0$ given, for $w \in \Sigma^*$ and $a, b \in \Sigma$, by

$$\rho(wab) = \begin{cases} \rho(wa) + 1 & \text{if } a \neq b, \\ \rho(wa) & \text{if } a = b, \end{cases}$$

and $\rho(\varepsilon) = \rho(a) = 0$ counts the number of *symbol switches*, or equivalently the number of *equal-letter runs* minus one.

Example 17.67 1. $\rho(abab) = \rho(aaabbab) = 3$.
2. For $L = (ab)^+$ we have $\{\rho(u) \mid u \in L\} = \{n \mid n \text{ is odd}\}$.

We list a few properties of ρ.

Proposition 17.68 *1. $\rho(a^i) = 0$ for $a \in \Sigma$ and $i \geq 0$.*
2. For $w \in \Sigma^+$ we have $\rho(w) = |\{i \in \{1, \ldots, n-1\} : w_i \neq w_{i+1}\}|$ where $w = w_1 \cdots w_n$ with $w_i \in \Sigma$ for $i \in \{1, \ldots, n\}$ and $n \geq 1$.
3. For $w \in \Sigma^+$ we have $\rho(w) = k - 1$ if and only if there exist letters $a_1, \ldots, a_k \in \Sigma$, $k \geq 1$, such that $w \subseteq a_1^+ \cdots a_k^+$ and $a_i \neq a_{i+1}$ for $i \in \{1, \ldots, k-1\}$.
4. $\rho(w) = \min\{k \geq 1 \mid \exists a_1, \ldots, a_k \in \Sigma\colon w \subseteq a_1^* \cdots a_k^*\} - 1$ for $w \in \Sigma^*$.
5. For $u, v \in \Sigma^*$ we have $\rho(u) + \rho(v) \leq \rho(uv) \leq \rho(u) + \rho(v) + 1$. In particular, $\rho(u) \leq |u|$.

Proof. The first four equations follow readily. For Item 5, let $u, v \in \Sigma^*$ be two words. We can assume $u, v \neq \varepsilon$. By Item 3 there exist $a_1, \ldots, a_n, b_1, \ldots, b_m \in \Sigma$ for $n, m \geq 1$ such that $\rho(u) = n - 1$ and $\rho(v) = m - 1$ where $u \subseteq a_1^+ \cdots a_n^+$ with $a_i \neq a_{i+1}$ for $i \in \{1, \ldots, n-1\}$ and $v \subseteq b_1^+ \cdots b_m^+$ with $b_j \neq b_{j+1}$ for $j \in \{1, \ldots, m-1\}$. Then $uv \subseteq a_1^+ \cdots a_n^+ b_1^+ \cdots b_m^+$ and we get

$$\rho(uv) = \begin{cases} n + m - 1 & \text{if } a_n \neq b_1, \\ n + m - 2 & \text{if } a_n = b_1. \end{cases}$$

For an alternative argument, write $u = u_1 \cdots u_n$ and $v = v_1 \cdots v_m$ for $n, m \geq 0$ and $u_i \in \Sigma$, $v_j \in \Sigma$ for $i \in \{1, \ldots, n\}$ and $j \in \{1, \ldots, m\}$. Then, by Item 2, we find $\rho(uv) = |\{i \in \{1, \ldots, n-1\} \mid u_i \neq u_{i+1}\}| + |\{n \mid u_n \neq v_1\}| + |\{i \in \{1, \ldots, m\} \mid v_i \neq v_{i+1}\}| = \rho(u) + \rho(v) + |\{n \mid u_n \neq v_1\}|$. □

Next, we characterize letter-bounded languages with the mapping ρ.

Proposition 17.69 *A language $L \subseteq \Sigma^*$ is letter-bounded if and only if there exists $n \geq 0$ such that $\rho(u) \leq n$ for all $u \in L$.*

Proof. First, suppose $\rho(u) \leq n$ for all $u \in L$. Let $f_1, \ldots, f_m$ be some enumeration of all functions $\{1, \ldots, n\} \to \Sigma$ where $m = |\Sigma|^n$. Let $u \in L$. We have $u \in a_1^* \cdots a_n^*$ for some $a_1, \ldots, a_n \in \Sigma$. Thus $u \in \prod_{i=1}^m f_i(1)^* \cdots f_i(n)^*$. Hence L is letter-bounded. Conversely, suppose $L \subseteq \Sigma^*$ is letter-bounded. Then there exist $a_1, \ldots, a_n \in \Sigma$ such that $L \subseteq a_1^* \cdots a_n^*$. So, we have $\rho(u) \leq n-1$ for all $u \in L$ by Theorem 17.68, Item 4. □

Corollary 17.70 *Let $L \subseteq \Sigma^*$. Suppose there exists $u \in \Sigma^* a \Sigma^* b \Sigma^*$ for distinct $a, b \in \Sigma$ such that $\Sigma^* u^n \Sigma^* \cap L \neq \emptyset$ for infinitely many n. Then L is not letter-bounded.*

Proof. We have $\rho(u) \geq 1$. Thus for $vu^n w \in L$ with $n \geq 1$ we have $\rho(vu^n w) \geq \rho(u^n) \geq n$ by Theorem 17.68, Item 5. Whence, by Theorem 17.69, the language L is not letter-bounded. □

For bounded languages, the condition from Theorem 17.70 also characterizes letter-boundedness, as we show next.

Proposition 17.71 *Let $L \subseteq \Sigma^*$ be bounded. Then L is letter-bounded if and only if for all $u \in \Sigma^*$ such that $\Sigma^* u^n \Sigma^* \cap L \neq \emptyset$ for infinitely many n we have $u \subseteq a^*$ for some $a \in \Sigma$.*

Proof. First, suppose L is letter-bounded. Then, by Theorem 17.70, for all $u \in \Sigma^*$ such that $\Sigma^* u^n \Sigma^* \cap L \neq \emptyset$ for infinitely many n we must have $u \subseteq a^*$ for some $a \in \Sigma$.

Conversely, suppose the stated condition holds. Let $L \subseteq u_1^* \cdots u_k^*$ for some $u_1, \ldots, u_k \in \Sigma^*$. Suppose we have $u_j \in \Sigma^* a \Sigma^* b \Sigma^*$ for some $j \in \{1, \ldots, k\}$ with distinct $a, b \in \Sigma$ and such that for each $N \geq 0$ there exist $i_1, \ldots, i_k \geq 0$ with $i_j \geq N$ such that $u_1^{i_1} \cdots u_k^{i_k} \in L$. Then the the assumption is violated with $u = u_j$. Thus, there exists $N \geq 0$ such that $u_1^{i_1} \cdots u_k^{i_k} \in L$ with $i_j > N$ for some $j \in \{1, \ldots, k\}$ implies $u_j \subseteq a^*$ for some $a \in \Sigma^*$. Then pick all those indices $1 \leq j_1 < \cdots < j_l \leq k$ for some $l \in \{1, \ldots, k\}$ where for $i \in \{1, \ldots, l\}$ we have $u_{j_i} \in \Sigma^* a \Sigma^* b \Sigma^*$ for distinct $a, b \in \Sigma$. Then for $u_1^{i_1} \cdots u_k^{i_k} \in L$, by assumption, we have $i_{j_r} \leq N$ for $r \in \{1, \ldots, l\}$ and $\rho(u_i) = 0$ for $i \notin \{j_1, \ldots, j_l\}$. With Theorem 17.68, for $M = i_1 + \cdots + i_k + k$ this gives

$$\rho(u_1^{i_1} \cdots u_k^{i_k}) - M \leq i_1 \rho(u_1) + \cdots + i_k \rho(u_k) \leq \sum_{r=1}^{l} i_{j_r} \rho(u_{j_r}) \leq N \cdot \sum_{r=1}^{l} \rho(u_{j_r}).$$

Thus, by Theorem 17.69, the language L is letter-bounded. □

Next, we characterize regular letter-bounded languages.

Proposition 17.72 *Let $L \subseteq \Sigma^*$ be regular. Then the following are equivalent.*

1. *L is letter-bounded.*
2. *If $uv^*w \subseteq L$ with $u, v, w \in \Sigma^*$, then $v^* \subseteq a^*$ for some $a \in \Sigma$.*
3. *For every PDFA $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ with $L = L(\mathcal{B})$ and state $p \in P$ that is accessible and coaccessible, we have $L(\mathcal{B}_{p,\{p\}}) \subseteq a^*$ for some $a \in \Sigma$.*

Proof. 1 $\Rightarrow$ 2: Suppose there exist $u, v, w \in \Sigma^*$ with $uv^*w \subseteq L$ and $|v|_a, |v|_b > 0$ for two distinct $a, b \in \Sigma$. Then, by Theorem 17.70, L is not letter-bounded.

2 $\Rightarrow$ 3: Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA. Assume $v \in L(\mathcal{B}_{p,\{p\}})$ for some accessible and coaccessible $p \in P$ and $v \in \Sigma^*$ with $|v|_a > 0$ and $|v|_b > 0$ for distinct $a, b \in \Sigma$. Then $\mu(p_0, u) = p$ and $\mu(p, w) \in F$ for some $u, w \in \Sigma^*$. Hence $uv^*w \subseteq L(\mathcal{B})$ and Item 2 is not true, giving the claim.

3 $\Rightarrow$ 1: Suppose L is not letter-bounded. Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA with $L = L(\mathcal{B})$ and $n = |P|$. By Theorem 17.69 there exists $u \in L(\mathcal{B})$ with $\rho(u) \geq 2n - 1$. By Theorem 17.68, Item 3, we have $u = a_1^{i_1} \cdots a_{2n}^{i_{2n}}$ for some $a_1, \ldots, a_{2n} \in \Sigma$ with $a_j \neq$

a_{j+1} for $j \in \{1, \ldots, 2n-1\}$ and $i_j \geq 1$ for $j \in \{1, \ldots, 2n\}$. Set $p_j = \mu(p_0, a_1^{i_1} \cdots a_{i_{2j}}^{i_{2j}})$ for $j \in \{0, 1, \ldots, n\}$.[14] By finiteness there exists $0 \leq j < j' \leq n$ such that $p_j = p_{j'}$. Set $w = a_{2j}^{i_{2j}} \cdots a_{2j'}^{i_{2j'}}$. Then we have $w^* \subseteq L(\mathcal{B}_{p_j,\{p_j\}})$. Since $a_{2j} \neq a_{2j+1}$ the language w^* is not contained in a^* for each $a \in \Sigma$. The PDFA $\mathcal{B}$ accepts u, and so the state p_j is accessible and coaccessible. Thus Item 3 does not hold. □

17.6 Monoid Languages

A submonoid $L \subseteq \Sigma^*$ is also called a *monoid (language)* for short. The language $L \subseteq \Sigma^*$ is a monoid language if and only if $L^* \subseteq L$. For any $L \subseteq \Sigma^*$, we have $\langle L \rangle_M = L^*$.

For a monoid language $L \subseteq \Sigma^*$ there exists a unique minimal generating set X, i.e., a set X minimal for set-inclusion with $L = X^*$. It is given as the set of non-empty words in L that cannot be written as a product of two non-empty words from L, namely (see [422, p. 4]):

$$X = (L \setminus \{\varepsilon\}) \setminus (L \setminus \{\varepsilon\})^2. \tag{17.14}$$

For every set $Y \subseteq \Sigma^*$ such that $Y^* = L$ we have $X \subseteq Y$ (more precisely, we have $X = (Y \setminus \{\varepsilon\}) \setminus (Y \setminus \{\varepsilon\})^2$). This implies that L is generated by a prefix code if and only if X is a prefix code. The next examples give languages where X is not a code, and where is it a code, but not a prefix code.

Example 17.73 For $L = \{a, ab, ba\}^*$ we have $X = \{a, ab, ba\}$, which is not a code since $a(ba) = (ab)a$.

Example 17.74 For $L = \{a, ab\}^*$ we have $X = \{a, ab\}$.

Equation (17.14) implies that L is regular if and only if X is regular.

We always have $L \setminus (L\Sigma^+ \cup \{\varepsilon\}) \subseteq X$ since $(L \setminus \{\varepsilon\})^2 \subseteq L\Sigma^+$. In case X is a prefix code, the following equalities hold:[15]

$$X = L \setminus (L\Sigma^+ \cup \{\varepsilon\}) = \{w \in L \setminus \{\varepsilon\} \mid \mathrm{Pref}(w) \cap L = \{w\}\}. \tag{17.15}$$

Regular languages that are monoids generated by prefix codes admit an automata-theoretical characterization as languages accepted by PDFAs whose start state is the only final state (a related statement is given in [194, Prop. 3.2]).

Theorem 17.75 ([87, Prop. 3.2.5],[507, Thm. 3.1]) *Let $L \subseteq \Sigma^*$. Then we have $L = L(\mathcal{B})$ for some PDFA $\mathcal{B} = (P, \Sigma, \mu, p_0, \{p_0\})$ if and only if $L = \{\varepsilon\}$ or L is a regular language that is a monoid generated by a prefix code that is regular.*

Proof. We can assume $L \neq \{\varepsilon\}$. Let $L = L(\mathcal{B})$ for a PDFA $\mathcal{B} = (P, \Sigma, \mu, p_0, \{p_0\})$. Set $X = \{u \in L \mid \forall v \in \mathrm{Pref}(u) \setminus \{u\} \colon \mu(p_0, v) \neq p_0\}$. Then X is a regular prefix code

[14] We set $a_1^{i_1} \cdots a_0^{i_0} = \varepsilon$.

[15] Since codes are defined here as non-empty subsets of Σ^+, for a prefix code X we have $X^* \neq \{\varepsilon\}$.

(we obtain a PDFA accepting it by removing all outgoing transitions from final states in $\mathcal{B}$) such that $L = X^*$.

Conversely, let $L = X^*$ where X is a regular prefix code. By Theorem 17.30 we obtain a trim PDFA $\mathcal{B} = (P, \Sigma, \mu, p_0, \{q\})$ such that q has no outgoing transitions. Thus we can merge the states q_0 and q to a single state p to obtain a new PDFA where all transitions going out of p_0 go out of this new state and all transitions ending in some state from $\{q, p_0\}$ now end at p. This new PDFA with start state p and final state set $\{p\}$ accepts L. □

Theorem 17.75 has been extended in [507, Thm. 3.2] to characterize submonoids that are regular languages as precisely those languages $L \subseteq \Sigma^*$ such that $L = \{u \in \Sigma^* \mid \mu(S, u) = S\}$ for a PDFA $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ and $S \subseteq P$.

Remark 17.76 The construction from the proof of Theorem 17.30, Item 2 (used in the proof of Theorem 17.75) is closely related to the rooted tree from Theorem 17.17. In fact, this rooted tree can be considered as a (possibly infinite) PDFA accepting the prefix code. This PDFA is called the *literal automaton* of the prefix code in [87, Sect. 3.2]. See also [538, Sect. 1.3 & 4.3.8], where this construction is used for finite maximal prefix codes. In the construction from Theorem 17.30, Item 2, we additionally incorporate the Myhill–Nerode relation to obtain a finite automaton whenever the prefix code is regular.

Next, we show that monoid languages in u^* for some $u \in \Sigma^*$ are always regular.

Proposition 17.77 *Le $L \subseteq u^*$ be a monoid language for some $u \in \Sigma^*$. Then there exist $d \geq 1$ and a multiple of n of d such that $L = u^n(u^d)^* \cup F$ for a finite set $F \subseteq \{u\}^{<n} \cap (u^d)^*$. In particular, L is regular.*

Proof. We can assume L is infinite. Let $d = \min\{\gcd(i_1, \ldots, i_r) \mid \{u^{i_1}, \ldots, u^{i_r}\} \subseteq L\}$. Then there exist numbers $i_1, \ldots, i_r \geq 0$ with $\{u^{i_1}, \ldots, u^{i_r}\} \subseteq L$ such that $d = \gcd(i_1, \ldots, i_r)$. Let $i \geq 0$ with $u^i \in L$. As $d = \gcd(a_1, \ldots, a_r, i)$ the number d divides i. This yields $L \subseteq (u^d)^*$. By Bézout's identity there exists a number $n \geq 0$ divisible by d such that every number m divisible by d with $m \geq n$ can be written as $m = a_1 i_1 + \cdots + a_r i_r$ for numbers $a_1, \ldots, a_r \geq 0$.[16] So for $u^m \in u^n(u^d)^*$ we have $u^m \in (u^{i_1})^* \cdots (u^{i_r})^* \subseteq L$ and we get $u^n(u^d)^* \subseteq L$. As $L \subseteq (u^d)^*$ and n is a multiple of d, the language $F = L \setminus u^n(u^d)^*$ is finite and $F \subseteq \{u\}^{<n}$. Hence, $L = a^n(u^d)^* \cup F$. □

The next corollary generalizes the well-known folklore fact that the Kleene star of any unary language is always regular. Via the mapping $n \mapsto u^n$ for a fixed $u \in \Sigma^*$ it is in fact a reformulation of the statement that a submonoid of $\mathbb{N}_0$ is ultimately periodic [521, Prop. 2.2].

Corollary 17.78 *Let $L \subseteq u^*$. Then L^* is a regular language.*

Proof. As L^* is a monoid and since $L^* \subseteq (u^*)^* = u^*$ by Item 4, the claim follows from Theorem 17.84. □

[16] We note that the *Frobenius coin problem* asks for the smallest n for given $i_1, \ldots, i_r$ when $d = 1$, see [4].

Remark 17.79 A *semigroup language* is a language $L \subseteq \Sigma^*$ such that $L^+ \subseteq L$. Equivalently, L forms a subsemigroup of Σ^*. Every monoid language is also a semigroup language, but not vice versa. For example, $aaaa^*$ is a semigroup language that is not a monoid language. However, a language $L \subseteq \Sigma^*$ is a semigroup language if and only if $L \cup \{\varepsilon\}$ is a monoid language. Similar statements can be given for semigroup languages and our later results on the constrained synchronization problem w.r.t. monoid languages are easily adaptable and also hold for semigroup languages.

17.7 The Growth of Languages

A language $L \subseteq \Sigma^*$ is *sparse* (resp. has *exponential growth*) if $|L \cap \Sigma^{\leq n}|$ is bounded by a polynomial in n (resp. there exists a real number $c > 1$ such that $|L \cap \Sigma^{\leq n}| \geq c^n$ for infinitely many n).[17]

Example 17.80
1. The code ab^* is sparse whereas the code $c\{a, b\}^*c$ has exponential growth.
2. Let $L \subseteq u^*$ with $u \in \Sigma^*$. Then we have $L \cap \Sigma^{\leq n} \subseteq \{\varepsilon, u, \ldots, u^n\}$. Thus $|L \cap \Sigma^{\leq n}| \leq n + 1$, and so L is sparse.
3. Let $L \subseteq \Sigma^*$ and $a \in \Sigma$. If there exists $N \geq 0$ such that $w \in L$ implies $|w|_b \leq N$ for all $b \in \Sigma \setminus \{a\}$, then the language L is a sparse language.
4. Let $u, v \in \Sigma^*$ and $L \subseteq \Sigma^*$. Then L is sparse (resp. has exponential growth) if and only if uLv is sparse (resp. has exponential growth) since $|uLv \cap \Sigma^n| = |L \cap \Sigma^{n-|u|-|v|}|$ for $n \geq |u| + |v|$. (in fact, the map $w \mapsto uwv$ is a bijection between Σ^* and $u\Sigma^*v$.)

Next, generalizing Theorem 17.80, Item 2, we show that every bounded language is sparse, a proof is also given in [504, Prop. 1].

Proposition 17.81 ([504, Prop. 1]) *A bounded language is sparse.*

Proof. Let $L \subseteq u_1^* \cdots u_k^*$ for $u_1, \ldots, u_k \in \Sigma^*$. Then $L \cap \Sigma^{\leq n}$ is contained in $\{u_1^{i_1} \cdots u_k^{i_k} \mid i_1|u_1| + \cdots + i_k|u_k| \leq n\}$, which is contained in the set $\{u_1^{i_1} \cdots u_k^{i_k} \mid i_1|u_1| + \cdots + i_k|u_k| \wedge \forall j \in \{1, \ldots, k\}\colon 0 \leq i_j \leq n\}$ of size at most $(n + 1)^k$. □

Next, we give some examples of sparse languages that are not bounded.

Example 17.82 1. Let $L = \{(a^n b^n)^n \mid n \geq 1\}$. This language is sparse since

$$L \cap \Sigma^i = \begin{cases} \{(a^n b^n)^n\} & \text{if } i = 2n^2 \text{ for some } n \geq 1, \\ \emptyset & \text{otherwise.} \end{cases}$$

Assume L is bounded. We have $\rho((a^n b^n)^n) = 2n - 1$ and thus L is not letter-bounded by Theorem 17.69. By Theorem 17.71 there exists u with $|u|_a > 0$ and

[17] $|L \cap \Sigma^{\leq n}|$ is bounded by a polynomial in n (resp. has exponential growth) if and only if $|L \cap \Sigma^n|$ is bounded by a polynomial in n (resp. has exponential growth). So, we can equivalently use $|L \cap \Sigma^n|$ in both definitions.

$|u|_b > 0$ such that $\Sigma^* u^i \Sigma^* \cap L \neq \emptyset$ for infinitely many i. Without loss of generality, we have $ab^m a \in \mathrm{Fact}(uu)$ for some $m \geq 0$. Let $vu^i w \in L$ for $i \geq 2$ and $v, w \in \Sigma^*$; and then $\Sigma^* u^j \Sigma^* \cap L \neq \emptyset$ for $j \geq |vu^i w| + 1$. Thus L contains two distinct words with the factor $ab^m a$, but this contradicts the definition of L since $(a^m b^m)^m$ is the only word with this factor.

2. Let $u_n = ba^{i_1} ba^{i_2} \cdots ba^{i_n} b$ for some strictly increasing sequence $1 \leq i_1 < i_2 < \cdots$. Set $L = \{u_1, u_2, \ldots\}$. Then L contains at most one word of each length since the lengths of the u_n strictly increase. Thus L is sparse. Assume L is bounded. Since $\rho(u_n) = 2n$, L is not letter-bounded. By Theorem 17.71 there exists u with $|u|_a > 0$ and $|u|_b > 0$ such that $\Sigma^* u^i \Sigma^* \cap L \neq \emptyset$ for infinitely many i. No word in L contains the factor bb. Thus a factor of the form $ba^j b$ for some $j \geq 0$ appears at least twice in uuu. We have $\Sigma^* uuu \Sigma^* \cap L \neq \emptyset$. However each factor of the form $ba^j b$ for some $j \geq 1$ appears at most once in each word from L, so this is not possible.
3. (Pin [491, Sect. XII.4.2]) Consider a bijection $f\colon \{a, b\}^* \to \mathbb{N}_0$ such that $f(u) \geq |u|$ for all $u \in \{a, b\}^*$. One can construct such a bijection by letting $f(u)$ be the position of u in the shortlex order: ε, a, b, aa, ab, ba, bb, aaa, aab, aba, ... Now, let
$$L = \{uc^{f(u)-|u|} \mid u \in \{a, b\}^*\}.$$
Then for $n \geq 0$ we have $L \cap \Sigma^n = \{uc^{f(u)-|u|}\}$ for the unique $u \in \{a, b\}^*$ with $f(u) = n$ and thus L is sparse. If $L \subseteq u_1^* \cdots u_k^*$, then $\mathrm{Pref}(L) \subseteq \bigcup_{i=1}^{k} \bigcup_{v \in \mathrm{Pref}(u_i)} u_1^* \cdots u_{i-1}^* u_i^* v$; and thus the set of prefixes of L is also bounded if L is bounded. However, $\{a, b\}^* \subseteq \mathrm{Pref}(L)$ and this language has exponential growth. So $\mathrm{Pref}(L)$ has exponential growth and thus cannot be bounded by Theorem 17.81, which implies that L is not bounded.

Next, in Theorem 17.85, we show that a regular language is either bounded, and thus sparse, or has exponential growth. But first, in Theorem 17.83, we give a sufficient condition for exponential growth.

Proposition 17.83 *Let $L \subseteq \Sigma^*$ be a language containing two words $u, v \in L$ that are not powers of a common word and such that $\{u, v\}^* \subseteq L$. Then L has exponential growth.*

Proof. By Theorem 17.20, $C = \{u, v\}$ is a code. By Proposition 17.18 there exists a coding morphism $\varphi\colon \Gamma^* \to \Sigma^*$ such that $\varphi(\Gamma^*) = C^*$. We can suppose $\Gamma = \{a, b\}$ where $a \neq b$. Since φ is injective, we have $|\varphi(\{a, b\}^i)| = 2^i$ for all $i \geq 0$. As $\varphi(\{a, b\}^{\lfloor n/m \rfloor}) \subseteq C^* \cap \Sigma^{\leq n}$ for $m = \max\{|\varphi(a)|, |\varphi(b)|\}$ and $n \geq 0$, C^* has exponential growth. Since $C^* \subseteq L$, also L has exponential growth. □

As Theorem 17.82, Item 3 shows, $\{u, v\}^* \subseteq \mathrm{Fact}(L)$ for a code $\{u, v\}$ does not imply exponential growth. However, it implies that L cannot be bounded, as stated in [245, Lem. 5.3].

Corollary 17.84 *A monoid language $L \subseteq \Sigma^*$ is either a regular subset of u^* for some $u \in \Sigma^*$ or has exponential growth.*

Proof. First, suppose $L \subseteq u^*$ for some $u \in \Sigma^*$. Then, L is regular by Theorem 17.77. Otherwise, by Theorem 17.63, there exist $v, w \in L$ such that $\{v, w\} \not\subseteq u^*$ for each $u \in \Sigma^*$. Since L is a monoid, we have $\{v, w\}^* \subseteq L$. By Theorem 17.83, L has exponential growth. □

By Theorems 17.20 and 17.84 a code either contains at most one word or its Kleene star has exponential growth.

Theorem 17.85 *A regular language is either bounded or has exponential growth.*

Proof. Let $L \subseteq \Sigma^*$ be regular and $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a trim PDFA accepting L. As a shorthand, for $q \in Q$, set $L_q = L(\mathcal{A}_{q,\{q\}})$. By Theorem 17.75 these are monoid languages. Suppose L_q has exponential growth for some $q \in Q$. Since $\mathcal{A}$ is trim, we have $\delta(q_0, u) = q$ and $\delta(q, v) \in F$ for some $u, v \in \Sigma^*$. This yields $uL_qv \subseteq L$. By Theorem 17.80, Item 4 the language uL_qv has exponential growth. Thus, also L has exponential growth. Otherwise, by Theorem 17.84 for each $q \in Q$ we have $L_q \subseteq v_q^*$ for some $v_q \in \Sigma^*$. Let

$$E = \{u_1 \cdots u_m \in L \mid u_1, \ldots, u_m \in \Sigma \wedge \delta(q_0, u_1 \cdots u_i) \notin \{q_0, \delta(q_0, u_1), \ldots, \delta(q_0, u_1 \cdots u_{i-1})\}\}.$$

The words in E are precisely those words $u \in L$ for which there do not exist $u_1, u_2, u_3 \in \Sigma^*$ with $u = u_1u_2u_3$, $|u_2| > 0$, and $\mu(p_0, u_1) = \mu(p_0, u_1u_2)$. Each word in E has length at most $|Q|$ and so E is finite. We have[18]

$$L = \bigcup_{u_1 \cdots u_m \in E} L_{q_0} u_1 L_{\delta(q_0,u_1)} u_2 \cdots u_m L_{\delta(q_0,u_1 \cdots u_m)}. \tag{17.16}$$

Thus L is contained in a finite union of languages of the form $v_{q_0}^* u_1 v_{q_1}^* u_2 \cdots u_m v_{q_m}^*$ for $u_1 \cdots u_m \in E$ and $q_i = \delta(q_0, u_1 \cdots u_i)$ and so L is bounded.[19] □

Corollary 17.86 *A regular language is bounded if and only if it is sparse.*

Proof. By Theorem 17.81, every bounded language is sparse. Conversely, a sparse regular language cannot have exponential growth and is therefore bounded by Theorem 17.85. □

The following corollary collects some conditions characterizing languages in u^* that follow from previous results.

Corollary 17.87 *Let $L \subseteq \Sigma^*$. Then the following are equivalent.*

1. $L \subseteq u^*$ for some $u \in \Sigma^*$.
2. $L^* \subseteq u^*$ for some $u \in \Sigma^*$.
3. $L^* = u^n(u^d)^* \cup F$ for some $u \in \Sigma^*$, $n \geq 0$, $d \geq 0$ and finite $F \subseteq \{u\}^{<n}$.[20]

[18] Note that $\delta(q_0, u_1 \cdots u_m) \in F$ in Equation (17.16) since $E \subseteq L$.

[19] Equation (17.16) can also be used to prove "3 $\Rightarrow$ 1" in Theorem 17.72 since with the assumption that, for each $q \in Q$, we have $L_q \subseteq a^*$ for some $a \in \Sigma^*$ we can then show $\rho(u) \leq \max\{\rho(v) \mid v \in E\} + 2|Q|$ for $u \in L$.

[20] Note that in case $L = \{\varepsilon\}$ we have $d = 0$ where the first three equivalences hold for any $u \in \Sigma^*$.

4. L^* is sparse.
5. L^* has not exponential growth.
6. For all $u, v \in L$ there exists $w \in \Sigma^*$ such that $\{u, v\} \subseteq w^*$.

Proof. $1 \Rightarrow 2$: This follows since $L^* \subseteq (u^*)^* = u^*$ by Theorem 3.61, Item 4.
$2 \Rightarrow 3$: This follows from Theorem 17.77 since L^* is a monoid language.
$3 \Rightarrow 4$: In this case, $L^* \subseteq u^*$ and so it is sparse by Theorem 17.81.
$4 \Rightarrow 5$: This is clear.
$5 \Rightarrow 6$: Since L^* is a monoid language, if it does not has exponential growth, then $L^* \subseteq u^*$ for some $u \in \Sigma^*$ by Theorem 17.84.
$6 \Rightarrow 1$: This follows from Theorem 17.63. □

17.8 Assumptions on Constraint Languages and Constraint PDFAs

Consider a constrained synchronization problem $\textsc{Sync}[L]$ with some regular constraint language $L \subseteq \Sigma^*$.

Remark 17.88 Every constraint language also specifies an alphabet. As we show next, up to $\equiv_m^{\mathsf{AC}^0}$-reductions, the specific alphabet does not matter and we can always chose a smallest possible alphabet over which to consider the constraint language. For let $\Gamma \subseteq \Sigma$ and $L \subseteq \Gamma^*$ be regular. Then $L \subseteq \Sigma^*$ and $\textsc{Sync}[L]$ with L over Σ and $\textsc{Sync}[L]$ with L over Γ are equivalent for $\leq_m^{\mathsf{AC}^0}$-reductions: for $\mathcal{A}$ over Σ we can consider $\mathcal{A}_{|\Gamma}$ and, in the other direction for $\mathcal{A}$ over Γ, we can add self-loops for all letters in $\Sigma \setminus \Gamma$ to $\mathcal{A}$ to obtain an automaton over Σ. Both reductions can be implemented by AC^0-circuits by either ignoring certain input bits or adding the identity matrix as a transition matrix.

In this context, we call a PDFA $\mathcal{B}$ accepting L a *constraint PDFA*. Since the constraint languages is assumed to be fixed for a given constrained synchronization problem, we can assume $\mathcal{B}$ is trim. Furthermore, with respect to Remark 17.88, we can choose the alphabet Σ as small as possible. So, when nothing else is said, we make the following assumptions.

1. The constraint PDFA $\mathcal{B}$ is trim.
2. For a single regular constraint language $L \subseteq \Sigma^*$, every letter is used in L, i.e., $\Sigma^* a \Sigma^* \cap L \neq \emptyset$ for each $a \in \Sigma$.

Here, PDFAs are only used to describe constraint languages.

Due to Theorem 17.88, we do not specify an alphabet in the problem specification of $\textsc{Sync}[L]$. Changing the alphabet is actually a (very) special case of a more general way to encode the input by so-called solid codes, see Remark 18.20.

Later, in Theorem 18.106, we justify the extension of our model to PDFAs rather than using only the automaton model with a completely defined transition function introduced earlier.

17.9 Discussion

Partial Functions.

In principle, everything that can be formulated with partial functions can be recast using only (total) functions, and every PDFA can be converted into an automaton as defined before by adding a trap state at which all previously undefined transitions end. The reader might therefore ask what is gained from this generalization. In theoretical computer science, especially in computability theory (see [322, 388]), where a notion of effectiveness (or computability) is associated with functions via input-output machines, it is often natural to allow the "result" to be undefined. In this setting, a function may be undefined for inputs on which a machine is not required to produce a result, or on which it may fail to terminate by entering an infinite loop. In our context, we introduced PDFAs based on partial functions. As we will also see in the next paragraph, it is convenient to specify specific instances of the constrained synchronization problem by PDFAs.

Partial Automata.

We will not be concerned with (carefully or exactly) synchronizing PDFAs in this work. The reason for introducing partial automata is that they give a convenient specification for constraint languages. More specifically:

1. If $\mathcal{A}$ is a complete automaton having only coaccessible states, then $\textsc{Sync}[L(\mathcal{A})]$ is equivalent to $\textsc{Sync}[\Sigma^*]$ see Section 18.4.3, Theorem 18.104, Item 2, and Theorem 18.106 for details. But non-coaccessible states can be discarded to give a PDFA accepting the same language. So, instead of carrying around these useless states it is more economical to use trim PDFAs, in particular in our discussions of small constraint automata later in Sections 19.4.1 and 19.4.2.
2. There is some freedom on how to distribute the start state and the final states without altering the computational complexity of the corresponding constrained synchronization problem (see Theorem 18.99). This might give rise to many different regular expressions for the resulting languages while the underlying automaton, apart from its start state and set of final states, remains the same. Because of this, we regard PDFAs as a better specification mechanism compared to regular expressions.

The fact mentioned in Theorem 17.88 that we can ignore letters that are not used in any transitions up to AC^0-reductions was stated [215, Lem. 11] for $\leq_m^{\log}$-reductions.

Codes and Overlap-Free Languages.

Codes, and a connection to synchronizing automata, appeared in our introductory example on error-prone communication from Section 1.3.1. In this example, a decoder modeled by a synchronizing automaton allows recovery from errors by sending a synchronizing word that realigns the decoder with the encoder. This connection to codes

has been noted since the early days of synchronizing automata, for example, by Liu [416] and Laemmel [395, 396, 397]; see also the surveys [360, 617], the textbook [87] or the theses [92, 538].

Without reference to automata or regular languages, Schützenberger [553] called a prefix code $C \subseteq \Sigma^+$ *synchronizing* if there exists $u \in C^*$ such that $\Sigma^* u \subseteq C^*$.[21] Almost every prefix code that is maximal for set-inclusion is synchronizing [185].

We can obtain a decoding automaton for a regular prefix code C via the constructions outlined in Theorems 17.17, 17.30, and 17.76. Each time the decoding automaton enters a final state, this indicates the detection of a code word in the input stream. According to Theorem 17.30, this automaton admits exactly synchronizing words, and each such word can be extended to an exactly synchronizing word ending in the unique final state. In case the automaton is complete, each exactly synchronizing word is a synchronizing word in the usual sense as defined in Part I, Moreover, those exactly synchronizing words ending in the final state are precisely the synchronizing words of C. For example, consider the complete automaton obtained from Figure 1.3a, whose start state and unique final state coincide at q_ε. The word 0110, which synchronizes every state to q_ε, is also a synchronizing word of the code. Apart from the literature already mentioned above, for more information on the relation of codes to synchronizing automata, see also [70, 131, 272, 538].

Overlap-free languages were introduced in [277], but with a different definition by defining $L \subseteq \Sigma^*$ overlap-free if for all distinct $u, v \in L$ we have that $\mathrm{Pref}(u) \cap \mathrm{Suff}(v)$ and $\mathrm{Suff}(u) \cap \mathrm{Pref}(v)$ both equal $\{\varepsilon\}$ and $\mathrm{Pref}(u) \cap \mathrm{Suff}(u) = \{\varepsilon, u\}$ for all $u \in \Sigma^*$.[22] This implies overlap-freeness in our sense, but also that L is a bifix code since if, without loss of generality, $v \in \mathrm{Pref}(u)$ for distinct $u, v \in \Sigma^*$, then $v \in \mathrm{Pref}(u) \cap \mathrm{Suff}(v)$ since $v \in \mathrm{Suff}(v)$. For example, $L = \{aba, b\}$ fulfills this definition of overlap-freeness, but $\{a, ab\}$, which is overlap-free according to our definition, is excluded with this definition. In terms of Theorem 17.13 this wider definition corresponds to replacing the condition "$u \neq \varepsilon$ and $w \neq \varepsilon$" from our definition by "$u \neq \varepsilon$ or $w \neq \varepsilon$". We have not seen this definition of overlap-freeness elsewhere and the one we adopted seems to be the standard definition used, for example, also in [276, 348, 349, 350, 381]. In fact, the authors of [277], a conference contribution, later adopted the definition we gave here in their extended journal version [276] of the conference contribution.

As noted in Section 17.4.4, the characterization of solid codes from Theorem 17.53, Item 4 appears to be new.

---

[21] This definition implies maximality of the code C w.r.t. set inclusion. For consider $v \in \Sigma^*$. By assumption, $vu \in C^*$. Thus this word has a factorization in words from C, which excludes that $C \cup \{v\}$ can be a code since $u \in C^*$ implies that it also has a factorization in words from $C \cup \{v\}$. For a more general definition of synchronization applying to a wider class of prefix codes (and even general codes), see [87].

[22] In fact, the original definition demands equality to the empty set of $\mathrm{Pref}(u) \cap \mathrm{Suff}(v)$, which does not seem to be the intended definition since the empty word is always a common prefix and suffix of any two words, and also the condition $\mathrm{Pref}(u) \cap \mathrm{Suff}(u) = \{\varepsilon, u\}$ is missing that excludes a non-empty proper overlap of a word with itself only a condition on distinct words is imposed. See also the footnote in [350, p. 33] which remarks on the problematic definition from [277].

We note that for codes that are regular languages, often interesting relations hold among the classes considered in the present work. For example, in [380] it was proved that a regular bifix code is a finite union of regular infix codes.

Monoid Languages.

The free monoid Σ^* has a central role in theoretical computer science as our primary means of representing and processing information. As Σ^* is a monoid, languages that are submonoids of Σ^* form a natural object of study. A submonoid having a generating set C that is a code is said to be a *free (sub-)monoid* of Σ^*. By Theorem 17.18 a bijection between some set Γ and C extends to a coding morphism. Conversely, the image of a coding morphism yields a submonoid generated by a code. So, free submonoids of Σ^* are precisely the images of coding morphisms, i.e., isomorphic images of free monoids over some set Γ as introduced in Section 3.7 as sets of finite sequences, which explains why we call them free monoids as well. The relevance of these submonoids lies in the transmission, translation and encoding of information w.r.t. different representations. Every language over Σ is contained in a smallest free submonoid of Σ^* [422]. The *defect theorem* [422, Thm. 1.2.5] states that in case $L \subseteq \Sigma^*$ is a finite language that is not a code, there exists a finite code $C \subseteq \Sigma^+$ with $|C| \leq |L| - 1$ such that $L \subseteq C^*$.

The Growth Of Languages and Bounded Languages.

Berman and Hartmanis [86] introduced the notion of sparse languages to study encodings of NP-complete problems. They conjectured that, under the assumption $\mathsf{P} \neq \mathsf{NP}$, sparse languages cannot encode NP-complete problems with respect to $\leq_m^{\mathsf{P}}$-reductions. This conjecture was later proved by Mahaney [429] (see also the survey [283]). Surveys of sparse regular languages can be found in [642, Chap. 2, Sect. 3.5] and [491, Sect. XII.4.2].

Păun and Salomaa [476] introduced *thin* and *slender* languages where $|L \cap \Sigma^n| \leq 1$ and $|L \cap \Sigma^n| \leq c$, respectively, for sufficiently large n. Slender context-free languages were investigated in [330, 504]. Bertoni et al. [89] investigated the computational complexity of the function $n \mapsto |L \cap \Sigma^n|$ on context-free languages. D'Alessandro et al. [165] studied this function for sparse context-free languages.

Ginsburg and Spanier [245] introduced bounded languages and investigated bounded context-free languages. They later studied bounded regular languages in [246]. Several authors extended Theorem 17.85 to context-free languages [104, 327, 335, 407, 504, 607]. Indexed languages do not satisfy this dichotomy: there exist indexed languages of intermediate growth [263]. Ginsburg and Spanier [245, Thm. 5.2] proved that boundedness is decidable for context-free languages. Consequently, the growth type of a context-free language is decidable. Gawrychowski et al. [240] gave a polynomial-time algorithm for deciding the growth type of a context-free language.

Ilie et al. [265] characterized sparse context-free languages via special Dyck languages. Szilard et al. [590, Cor. 1] characterized sparse regular languages by regular

expressions of star-height one. Shur [569] related the growth of a regular language to structural properties of its accepting automata.

A language $L \subseteq \Sigma^*$ is called *dense* if $\Sigma^* w \Sigma^* \cap L \neq \emptyset$ for every $w \in \Sigma^*$ [491, Sect. XII.4.1]. Regular dense languages over alphabets of size at least two are never sparse [491, Cor. 4.28].

Our terminology for letter-bounded languages is not standard. For example, [328] uses the term letter-bounded for the strict variant in which all letters are distinct, whereas [133] refers to this variant as distinct-letter-bounded.

The function ρ via equal-letter runs occurs naturally in text compression algorithms, see [229, 499], and combinatorics on words [170] since it is related to run-length encodings. In the theory of synchronizing automata, number of equal-letter runs in synchronizing words have also been investigated in [183].

Chapter 18
General Results

In Section 18.1 we show that Sync[L] is decidable and in PSPACE. Furthermore, we give example constraint languages yielding constrained synchronization problems with computational complexities in AC^0, or L-, NL-, NP-, or PSPACE-complete.

In Section 18.2 we present results on altering and simplifying the constraint language and how certain operations as forming factor sets, unions, intersections, morphisms, and concatenation affect the computational complexity. This also yields general methods to deduce complexity lower bounds between different instances of constrained synchronization problems.

General hardness conditions for NL, NP, and PSPACE are given in Section 18.3. This section also develops a general framework to deduce these hardness results introducing the notion of auxiliary state avoiding words and the introduction of relations derived from the Myhill-Nerode relation.

In Section 18.4 we discuss the representation of the constraint by PDFAs. We show that we have some freedom in the choice of a start state and a set of final states.

18.1 Basic Complexity Results and Examples

Here, Theorem 18.1 shows that Sync[L] is decidable and solvable in polynomial space for regular $L \subseteq \Sigma^*$. Then, Theorem 18.2 presents various example problems and their complexities.

Theorem 18.1 *Let $L \subseteq \Sigma^*$ be regular. Then* Sync[L] *is decidable and in* PSPACE.

Proof. An input semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$ admits a synchronizing word in L, i.e. $L \cap \mathrm{Syn}(\mathcal{A}) \neq \emptyset$, if and only if there exists $t \in Q$ such that

$$L \cap \bigcap_{q \in Q} L(\mathcal{A}_{q,\{t\}}) \neq \emptyset.$$

Let $\mathcal{B}$ be a complete automaton with $L = L(\mathcal{B})$, which can be chosen in advance since L is fixed by the definition of the problem.

An algorithm solving the problem can guess the state t and then call a procedure to solve Intersection for $\mathcal{B}$ and the automata $\mathcal{A}_{q,\{t\}}$ for $q \in Q$ as input (so that it is called with $|Q| + 1$ automata as arguments). By Theorem 3.76, Intersection is in PSPACE. So this algorithm can run in nondeterministic polynomial space. Since PSPACE = NPSPACE by Equation (3.49) we obtain the claim. □

The following introductory examples (one for each complexity class that will come up in this part) are presented here to motivate the later results and to have some easy and concrete cases with self-contained and independent proofs. Each example will be generalized in the coming sections.

Theorem 18.2 *Let $a, b, c \in \Sigma$ be three distinct letters.*

1. Sync$[\{a\}]$ *is in* AC^0.
2. Sync$[b^*]$ *and* Sync$[ab^*]$ *are* L*-complete even for trap-state input automata.*
3. Sync$[(a + b)^*]$ *and* Sync$[\Sigma^*]$ *are* NL*-complete.*
4. Sync$[ab^*a]$ *is* NP*-complete even for input automata with a trap state [215].*
5. Sync$[(a + b)^*c^l L]$ *is* PSPACE*-complete for non-empty and regular $L \subseteq \Sigma^*$ such that $|\operatorname{Fact}(L) \cap \{a^n, b^m, c^l\}| \leq 2$ for some $n, m, l \geq 1$ even for trap-state automata as input.*
6. Sync$[c(a + b)^*]$ *is* PSPACE*-complete even for strongly connected automata as input [215].*

Proof. 1: The letter a synchronizes a given input automaton if and only if the transition matrix $A = (a_{ij})_{1 \leq i,j \leq n}$ of a contains a column of ones. Equivalently, there exists $J \in \{1, \ldots, n\}$ such that for all $i \in \{1, \ldots, n\}$ we have $a_{iJ} = 1$. This can be checked by an AC^0 circuit implementing the formula

$$\bigvee_{J \in \{1,\ldots,n\}} \left(\bigwedge_{i \in \{1,\ldots,n\}} a_{iJ} \right).$$

In Section 19.2.2 (Theorem 19.3) this is generalized to finite constraint languages.

2: In Theorem 8.20 we have shown that SyncTrap for unary automata is L-complete. This problem is equivalent to Sync$[b^*]$. By adding self-loops to each state for the letter a in an input automaton $\mathcal{A} = (Q, \{b\}, \delta)$ over $\{b\}$, we obtain an $\leq_m^{\mathsf{AC}^0}$-reduction from this problem. So, Sync$[b^*] \leq_m^{\mathsf{AC}^0}$ Sync$[ab^*]$. Next, we get an $\leq_m^{\mathsf{AC}^0}$-reduction from Sync$[ab^*]$ to SyncFrom over the unary alphabet $\{b\}$ in the following way: for $\mathcal{A} = (Q, \{a, b\}, \delta)$ construct the SyncFrom-instance $(\mathcal{A}_{|\{b\}}, S)$ with $S = \delta(Q, a)$ where w.r.t. $Q = \{q_1, \ldots, q_n\}$ a characteristic bit-vector of S can be computed by an AC^0 circuit as, for $[a] = (m_{ij}^{(a)})_{i,j \in \{1,\ldots,n\}}$, we have $q_j \in S$ if and only if $m_{ij}^{(a)} = 1$ for some $i \in \{1, \ldots, n\}$. Then the claim follows as SyncFrom over $\{b\}$ is in L by Theorem 8.23.

3: Sync$[(a + b)^*]$ and Sync$[\Sigma^*]$ are equivalent to the unconstrained synchronization problem over $\{a, b\}$ and Σ, which are NL-complete by Theorem 8.13 since $|\Sigma| \geq 2$.

4: We give a reduction from DisjSetTrans over a unary alphabet, which is NP-complete by Theorem 8.18. Let $(\mathcal{A}, S, T)$ be an instance with $\mathcal{A} = (Q, \{b\}, \delta)$ and

$S, T \subseteq Q$ such that $S \cap T = \emptyset$. Pick some $\hat{s} \in S$ and let $\overline{S}\colon (Q \setminus T) \to S$ be given by $\overline{S}(q) = \hat{s}$ if $q \in Q \setminus (S \cup T)$ and $\overline{S}(q) = q$ otherwise. Construct $\mathcal{A}' = (Q \cup \{t\}, \{a, b\}, \delta')$ with a new state $t \notin Q$ where, for $q \in Q$ and $x \in \{a, b\}$,

$$\delta'(q, x) = \begin{cases} \delta(q, b) & \text{if } x = b, \\ \overline{S}(q) & \text{if } x = a \text{ and } q \in Q \setminus T, \\ t & \text{if } x = a \text{ and } q \in T \end{cases}$$

and $\delta'(t, x) = t$ for all $x \in \{a, b\}$. The state t must be the unique synchronizing state since it is a trap state. Furthermore, for $m \geq 0$, we have $\delta(S, b^m) \subseteq T$ if and only if $\delta'(Q \cup \{t\}, ab^m a) = \{t\}$, which yields that this is a valid reduction.

In Chapter 16 we have already established that Sync$[ab^*a]$ is in NP by using the matrix orbit problem. For a different argument, given $\mathcal{A} = (Q, \Sigma, \delta)$, by Theorem 6.73 applied to $S = \delta(Q, a)$ there exists a word in ab^*a synchronizing $\mathcal{A}$ if and only if there exists $0 \leq i < 2^{|Q|} - 1$ such that $|\delta(Q, ab^i a)| = 1$. Using a binary encoding, i can be guessed in polynomial time. By a square-and-multiply algorithm the transition matrix of b^i can be computed in polynomial time. Then, the condition $|\delta(Q, ab^i a)| = 1$ can be checked in polynomial time.

5: By Theorem 18.1 we only have to establish PSPACE-hardness. We give a reduction from SyncInto over a binary alphabet, which is PSPACE-complete by Theorem 8.16. Let $(\mathcal{A}, S)$ be an instance of SyncInto where $\mathcal{A} = (Q, \{a_1, a_2\}, \delta)$ and $S \subseteq Q$.

Let $u_1 = a^n b^m$, $u_2 = a^{n+1} b^m$. Intuitively, we re-encode the letters a_1 and a_2 into words u_1 and u_2, respectively, and build a new automaton $\mathcal{A}'$ containing a copy of Q in its state set that branches along these two words in the same way as $\mathcal{A}$ branches on a_1 and a_2. Additionally, the new automaton uses c^l to map states from S into a trap state. A word from the constraint language $(a + b)^* c L$ synchronizes $\mathcal{A}'$ if and only if, before reading c^l, we land inside S. This follows since the suffix of this word following c^l only uses letters from $\{u_1, u_2\}$, $\{u_1, c^l\}$, or $\{u_2, c^l\}$, but the construction is done in a way such that we need all three words to make any changes to the current state set yielding a new synchronizing word.

Set $P = \mathrm{Pref}(\{a^n b^{m-1}, a^{n+1} b^{m-1}, c^{l-1}\})$. Let $\mathcal{A}' = ((Q \times P) \cup \{t\}, \{a, b, c\}, \delta')$ where, for $q \in Q$, $u \in P$, and $x \in \{a, b, c\}$, we set

$$\delta'((q, u), x) = \begin{cases} (q, ux) & \text{if } ux \in P, \\ (\delta(q, a_i), \varepsilon) & \text{if } ux = u_i \text{ for some } i \in \{1, 2\}, \\ (q, \varepsilon) & \text{if } ux = c^l \text{ and } q \notin S, \\ t & \text{if } ux = c^l \text{ and } q \in S, \\ (q, \sigma_P(ux)) & \text{otherwise, i.e., if } ux \notin P \cup \{u_1, u_2, c^l\}, \end{cases}$$

and $\delta'(t, x) = t$ for all $x \in \{a, b, c\}$.[1]

Figure 18.1 shows a part of the reduction for a single state $q \in Q$ of the input automaton and $n = m = 2$. The additional "auxiliary" states, used to track the current

[1] Note that for $u \in P$ and $x \in \{a, b, c\}$ with $ux \notin P$ we have $\sigma_P(ux) = x$ expect in the case $u = a^{n+1}$ and $x = a$.

prefix and to branch when abb or $aabb$ has been read, are indicated by rectangles. Transitions labeled c are omitted for clarity: they either return to (q, ε) if $q \notin S$ or lead to the trap state t. Along such a c-path (q, c^i) with $1 \leq i \leq l-2$, reading a or b leads to (q, ε).

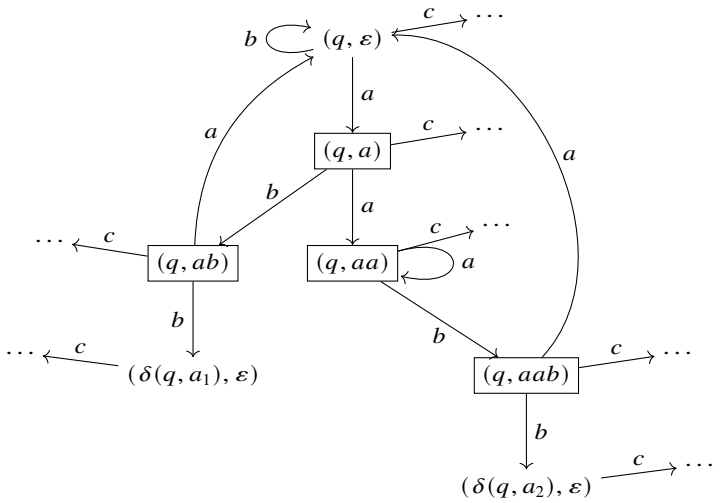


Fig. 18.1: Some states for an example of the construction in Item 5 with $n = m = 2$, see the proof of Item 5 for an explanation.

Next, we show that $(\mathcal{A}, S)$ is a YES-instance of SyncInto if and only if $\mathrm{Syn}(\mathcal{A}')$ contains a word from the constraint language.

First, suppose $\delta(Q, w) \subseteq S$ for some $w \in \{a_1, a_2\}^*$. We have

$$\delta'((Q \times P) \cup \{t\}, b^{m+1}) = Q \times \{\varepsilon\} \cup \{t\}.$$

Let $\varphi\colon \{a_1, a_2\}^* \to \{a, b, c\}^*$ be the morphism given by $\varphi(a_i) = u_i$ for $i \in \{1, 2\}$. Since $\delta'((q, \varepsilon), u_i) = (\delta(q, a_i), \varepsilon)$ for $i \in \{1, 2\}$ and $q \in Q$, we find $\delta'((q, \varepsilon), \varphi(w)) = (\delta(q, w), \varepsilon)$ for all $q \in Q$ and thus $\delta'(Q \times \{\varepsilon\}, \varphi(w)) \subseteq S \times \{\varepsilon\}$. For $q \in S$ we have $\delta'((q, \varepsilon), c^l) = t$. Since L is non-empty, we can pick some word $v \in L$. Combining everything and since t is a trap state, we find that $\delta'((Q \times P) \cup \{t\}, b^{m+1}\varphi(w)c^l v) = \{t\}$.

Conversely, suppose we have $|\delta'(Q \times P \cup \{t\}, wc^l v)| = 1$ for some $w \in \{a, b\}^*$ and $v \in L$. Since t is a trap state, we find $\delta'(Q \times P \cup \{t\}, wc^l v) = \{t\}$. Let $q \in Q$. We have $\delta'((q, \varepsilon), w) = (p, u)$ for some $u \in \mathrm{Pref}(\{u_1, u_2\})$ and $p \in Q$. Assume $p \notin S$. Then $\delta'((p, u), c^l) = (p, \varepsilon)$ (this holds for $u = \varepsilon$ and for $u \neq \varepsilon$). Next, to map (p, ε) to t, we must first map it to a state in $S \times P$ and then apply c^l. Now, for $w' \in \{a, b, c\}^*$ to have $\delta'((p, \varepsilon), w') \in S \times P$ it must have u_1 or u_2 as a factor, and to map a state from $S \times P$ to t it must have c^l as a factor. Since by assumption $|\mathrm{Fact}(L) \cap \{a^n, b^m, c^l\}| \leq 2$, we have $c^l \notin \mathrm{Fact}(L)$ or $|\mathrm{Fact}(L) \cap \{a^n, b^m\}| \leq 1$. where the latter case yields $u_1, u_1 \notin \mathrm{Fact}(L)$. Thus there is no way to map (p, ε) to t with a word from L and so we must have $p \in S$.

Now, we have $\delta'((q,\varepsilon),w) = (p,u)$ if and only if there exist words $v_0, v_1, \cdots, v_r$ and $w_1, w_2, \cdots w_r \in \{u_1, u_2\}$ such that

1. $w = v_0 w_1 v_1 w_2 \cdots w_r v_r$,
2. v_r has u as a suffix, and
3. $\delta'((q,\varepsilon), w_1 \cdots w_r) = (p,\varepsilon)$.

Let $x_i \in \{a_1, a_2\}$ for $i \in \{1,\ldots,r\}$ with $x_i = a_1$ if $w_i = u_1$ and $x_i = a_2$ otherwise. Then $\delta(q, x_1 \cdots x_r) = p$ and thus $\delta(Q, x_1 \cdots x_r) \subseteq S$.

6: By Theorem 18.1 we only have to establish PSPACE-hardness. We give a reduction from SyncFrom over a binary alphabet, which is PSPACE-complete by Theorem 8.16 even for strongly connected automata. Let $(\mathcal{A}, S)$ be an SyncFrom-instance with $\mathcal{A} = (Q, \{a,b\}, \delta)$ and $S \subseteq Q$. Pick some $\hat{s} \in S$. By extending δ with $\delta(q,c) = q$ for each $q \in S$ and $\delta(q,c) = \hat{s}$ for $q \notin S$ we obtain an automaton over $\{a,b,c\}$ with $\delta(Q,c) = S$ that has a synchronizing word of the form cu for $u \in \{a,b\}^*$ if and only if $|\delta(S,u)| = 1$. □

18.2 Manipulating Constraint Languages

Here, we investigate how applying operations to constraint languages affects the computational complexities of resulting constrained problems. Specifically, we investigate the following operations: the prefix/suffix/factor operations (Section 18.2.1), set union (Section 18.2.2), intersection with a regular ideal language (Section 18.2.3), applying morphism (Section 18.2.4), and concatenating specific languages to an arbitrary constraint language (Section 18.2.5). The operations from Section 18.2.1 preserve the complexity, those from Sections 18.2.2 to 18.2.4 mostly decrease the complexity, and those from Section 18.2.5 increase the complexity.

18.2.1 Factor Sets and Constraint Languages

Our first result is a consequence of Theorem 7.1. Although it has a simple proof, it is quite important for many of the following results.

Theorem 18.3 *Let $L, L' \subseteq \Sigma^*$ with $L \subseteq \mathrm{Fact}(L')$. If $\mathcal{A} = (Q, \Sigma, \delta)$ has a synchronizing word in L, then it has one in L'.*

Proof. Suppose $u \in L$ synchronizes $\mathcal{A}$. By assumption there exist $v, w \in \Sigma^*$ such that $vuw \in L'$. By Theorem 7.1 the latter word synchronizes $\mathcal{A}$ as well. □

Note that $L \subseteq \mathrm{Fact}(L')$ is equivalent to $\mathrm{Fact}(L) \subseteq \mathrm{Fact}(L')$. This gives the following consequence of Theorem 18.3.

Corollary 18.4 *Let $L, L' \subseteq \Sigma^*$ with $\mathrm{Fact}(L) = \mathrm{Fact}(L')$. Then every automaton $\mathcal{A}$ has a synchronizing word in L if and only if it has one in L'.*

For any $L \subseteq \Sigma^*$ we have $\mathrm{Fact}(L) = \mathrm{Fact}(\mathrm{Pref}(L)) = \mathrm{Fact}(\mathrm{Suff}(L))$. So with Corollary 18.4, we conclude that a given automaton has a synchronizing word in one of the languages L, $\mathrm{Fact}(L)$, $\mathrm{Pref}(L)$, or $\mathrm{Suff}(L)$ if and only if it has one in all of these languages. As a consequence, we obtain that the corresponding constrained synchronization problems are equivalent for $\leq_m^{\mathsf{AC}^0}$-reductions (in fact, the input can be passed through without any modifications and so these problems are also equivalent for reductions by projections as considered in [611]).

Corollary 18.5 *Let $L \subseteq \Sigma^*$ be a regular language. The following problems are equivalent for $\leq_m^{\mathsf{AC}^0}$-reductions.*

1. $\textsc{Sync}[L]$
2. $\textsc{Sync}[\mathrm{Fact}(L)]$
3. $\textsc{Sync}[\mathrm{Pref}(L)]$
4. $\textsc{Sync}[\mathrm{Suff}(L)]$

Furthermore, if for two languages $L, L' \subseteq \Sigma^*$ the factor, prefix, or suffix sets are equal, then $\textsc{Sync}[L] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L']$.

Corollary 18.6 *Let $L, L' \subseteq \Sigma^*$ be regular languages. Then we have $\textsc{Sync}[L \cup L'] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\mathrm{Fact}(L) \cup L']$*

Proof. This follows from Theorem 18.5 since $\mathrm{Fact}(L \cup L') = \mathrm{Fact}(\mathrm{Fact}(L) \cup L')$. □

Corollary 18.7 *Let $L \subseteq \Sigma^*$ be a regular language containing a non-empty left or right ideal language. Then we have $\textsc{Sync}[L] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\Sigma^*]$.*

Proof. Without loss of generality, we have $I \subseteq L$ for some non-empty left ideal language $I \subseteq \Sigma^*$ (which might be non-regular). Pick some $u \in I$ and consider a word $v \in \Sigma^*$. Then $vu \in I$ so that $v \in \mathrm{Fact}(I)$. This gives $\Sigma^* \subseteq \mathrm{Fact}(I) \subseteq \mathrm{Fact}(L)$. Thus $\mathrm{Fact}(L) = \Sigma^*$ and the claim follows from Theorem 18.5. □

Corollary 18.8 *Let $I, J \subseteq \Sigma^*$ be a left and a right ideal, respectively, which are both non-empty. Then $\textsc{Sync}[I \cap J] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\Sigma^*]$.*

Proof. By assumption, we have $u \in I$ and $v \in J$ for some $u, v \in \Sigma^*$. Let $w \in \Sigma^*$. Then we have $vwu \in I$ since I is a left ideal and $vwu \in J$ since J is a right ideal. Thus $vwu \in I \cap J$. This gives $\Sigma^* \subseteq \mathrm{Fact}(I \cap J)$ and the claim follows from Theorem 18.5. □

18.2.2 The Union of Constraint Languages

Here, in Theorem 18.9, we state that certain complexity classes are preserved when considering the union of two constraint languages. Theorem 18.10 gives further conditions for complexity preservation w.r.t. union and concatenation.

Theorem 18.9 *Let C be one of the complexity classes AC^0, NC^1, L, NL, P or NP. Suppose $\textsc{Sync}[L]$ and $\textsc{Sync}[L']$ are both in C for regular languages $L, L' \subseteq \Sigma^*$. Then the problem $\textsc{Sync}[L \cup L']$ is in C.*

Proof. For L, NL, P, and NP suppose M_1 and M_2 are Turing machines deciding $\textsc{Sync}[L]$ and $\textsc{Sync}[L']$, respectively, for an input automaton by halting and writing 1 for a YES-instance or 0 for a NO-instance. Let M be a Turing machine that, for an input automaton $\mathcal{A}$, first simulates M_1 with input $\mathcal{A}$ yielding output r_1, then M_2 with input $\mathcal{A}$ yielding output r_2, and finally outputs $\max\{r_1, r_2\}$, which equals one if and only if $\mathcal{A}$ has a synchronizing word in $L \cup L'$. We can combine the results r_1 and r_2 (repositioning reading head, erasing the working tape while storing the output of M_1, compute $\max\{r_1, r_2\}$) by operating deterministically and using only constant space; and if M_1 and M_2 obey the convention to position the working tape head next to the output, erase everything else and move the reading head back to the start, the overhead needed to simulate both machines and combine their results is actually constant in time and space. Hence, if M_1 and M_2 both work in deterministic logspace, nondeterministic logspace, deterministic polynomial time, or nondeterministic polynomial time, also M works in deterministic logspace, nondeterministic logspace, deterministic polynomial time, or nondeterministic polynomial time, respectively.

For AC^0 or NC^1, we can combine the outputs of two circuits from respective circuit families in AC^0 or NC^1 for $\textsc{Sync}[L]$ and $\textsc{Sync}[L']$ by an OR-gate to obtain an L-uniform circuit family in AC^0 or NC^1, respectively, for $\textsc{Sync}[L \cup L']$. □

Next, we state a condition such that the complexity is preserved when taking the union.

Corollary 18.10 *Let $L, L' \subseteq \Sigma^*$ be regular languages such that $L' \subseteq \mathrm{Fact}(L)$. Then we have $\textsc{Sync}[L] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L \cup L']$.*

Proof. By Theorem 18.6 we have $\textsc{Sync}[L \cup L'] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\mathrm{Fact}(L) \cup L']$ and the assumption gives $\mathrm{Fact}(L) \cup L' = \mathrm{Fact}(L)$. By Theorem 18.5 we have $\textsc{Sync}[\mathrm{Fact}(L) \cup L'] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L]$ (the claim also follows directly from Theorem 18.5 by noting that $\mathrm{Fact}(L \cup L') = \mathrm{Fact}(L) \cup \mathrm{Fact}(L') = \mathrm{Fact}(L)$). □

By Theorem 18.10, for a right or left ideal I that is regular and any regular $L \subseteq \Sigma^*$ we have $\textsc{Sync}[I \cup L] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\Sigma^*]$ since $\mathrm{Fact}(I) = \Sigma^*$.

Remark 18.11 Let $L = \{a, b\}^* c \{a, b\}^*$ for distinct $a, b, c \in \Sigma$. By Theorem 18.2, Item 5, $\textsc{Sync}[L]$ is PSPACE-complete. We have $\Sigma^* \setminus L = \{u \in \Sigma^* : |u|_c = 0 \text{ or } |u|_c \geq 0\}$ and $\mathrm{Fact}(\Sigma^* \setminus L) = \{a, b, c\}^*$. By Theorem 18.2, Item 3, the problem $\textsc{Sync}[\Sigma^* \setminus L]$ is NL-complete. Thus, in general, the computational complexity of the constrained synchronizing problem is not preserved when taking the complement.

18.2.3 The Ideal Intersection Theorem

Here, we show the ideal intersection theorem (Theorem 18.12). Later, we mostly use this result to deduce PSPACE-hardness from known PSPACE-hard problems, especially for constraint languages written as unions (see Remark 18.15 for details).

Theorem 18.12 (Ideal Intersection Theorem) *Let $I \subseteq \Sigma^*$ be a fixed ideal language that is regular. Suppose $L \subseteq \Sigma^*$ is any regular language, then*

$$\textsc{Sync}[L \cap I] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L].$$

Proof. By Theorem 7.1 there exists an automaton $\mathcal{A}_I$ such that $I = \mathrm{Syn}(\mathcal{A}_I)$. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an input to $\textsc{Sync}[L \cap I]$. By Equation (6.17), it is $\mathrm{Syn}(\mathcal{A}) \cap I = \mathrm{Syn}(\mathcal{A} \times \mathcal{A}_I)$, which gives

$$\mathrm{Syn}(\mathcal{A}) \cap (L \cap I) \neq \emptyset \Leftrightarrow \mathrm{Syn}(\mathcal{A} \times \mathcal{A}_I) \cap L \neq \emptyset, \tag{18.1}$$

i.e., $\mathcal{A} \times \mathcal{A}_I$ has a synchronizing word in L if and only if $\mathcal{A}$ has a synchronizing word in $L \cap I$. As I is fixed, we can construct the product automaton $\mathcal{A} \times \mathcal{A}_I$ by a circuit family in AC^0 as we only have to add transitions and states to $\mathcal{A}$ determined by a fixed automaton $\mathcal{A}_I$. So, by Equation (18.1) constructing $\mathcal{A} \times \mathcal{A}_I$ gives a valid reduction to $\textsc{Sync}[L]$. □

Example 18.13 Let $\Sigma = \{a, b, c\}$ and consider the ideal language $I = \Sigma^* c^n \Sigma^*$ that is regular. Let $L_1 = \{u \in \Sigma^* : |u|_c \leq n\}$, $L_2 = \{u \in \Sigma^* : |u|_c = n\}$, and $L_3 = \{u \in \Sigma^* : m \leq |u|_c \leq n\}$ for $n, m \geq 1$ with $m \leq n$. Then we have $L_1 \cap I = L_2 \cap I = L_3 \cap I = \{a, b\}^* c^n \{a, b\}^*$. From Theorem 18.2, Item 5 and Theorem 18.12 we get that $\textsc{Sync}[L_i]$ for $i \in \{1, 2, 3\}$ are all PSPACE-complete problems.

Corollary 18.14 *Let $L \subseteq \Sigma^*$ be regular and $u_1, \ldots, u_n \in \Sigma$. Then we have*

$$\textsc{Sync}[L \cap \Sigma^* u_1 \cdots u_n \Sigma^*] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L],$$
$$\textsc{Sync}[L \cap \Sigma^* u_1 \Sigma^* \cdots \Sigma^* u_n \Sigma^*] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L].$$

Proof. The languages $\Sigma^* u_1 \cdots u_n \Sigma^*$ and $\Sigma^* u_1 \Sigma^* \cdots \Sigma^* u_n \Sigma^*$ are regular and ideals. So, the claim follows from Theorem 18.12. □

Mostly, we apply Theorem 18.12 with *principal ideals* $\Sigma^* u \Sigma^*$ for $u \in \Sigma^*$.

As pointed out in the next remark, for a union the problem either has the same complexity as each part, or its complexity is bounded from below by the intersection of one part with a principal ideal.

Remark 18.15 Consider two regular languages $L_1, L_2 \subseteq \Sigma^*$. There exists $u \in L_2$ such that $u \notin \mathrm{Fact}(L_1)$ or $L_2 \subseteq \mathrm{Fact}(L_1)$. By Theorem 18.10, in the former case, we have

$$\textsc{Sync}[L_1] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L_1 \cup L_2]$$

and in the latter case, since then

$$(L_1 \cup L_2) \cap \Sigma^* u \Sigma^* = L_2 \cap \Sigma^* u \Sigma^*,$$

from Theorem 18.12 we get $\textsc{Sync}[L_2 \cap \Sigma^* u \Sigma^*] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L_1 \cup L_2]$.

Because we often use Theorem 18.12 to "separate a hard part out" of a constraint language we called it the Ideal Separation Theorem in [316], but we changed the name here because it involves an intersection and we thought the new name to be more appropriate.

Lastly, we show how the Ideal Intersection Theorem can be of help in a heuristic way by employing it to exclude problematic cases from a construction. This usage might not always be possible (in fact, in most cases it is not possible), but we give the following example to also build some intuition on various ways in which the Ideal Intersection Theorem can be applied (more examples of its application will follow in the rest of this work).

Remark 18.16 Consider $L = \{bb, ba\}^* a\Gamma^*$ for $|\Gamma| \leq 1$. We use a reduction from SyncInto to show PSPACE-hardness of Sync$[L]$. However, the construction we give does not work straightforward way for L, but we then argue that it can be used for a modified problem obtained by intersection with an appropriate ideal. We then obtain PSPACE-hardness of Sync$[L]$ from Theorem 18.12.

Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ and $S \subseteq Q$. Set $\mathcal{A}' = (Q \times \{\varepsilon, b\} \cup \{t\}, \{a, b\}, \delta')$ where $\delta'(t, x) = t$ and

$$\delta'((q,u),x) = \begin{cases} t & \text{if } u = \varepsilon \text{ and } x = a, \\ (q,b) & \text{if } u = \varepsilon \text{ and } x = b, \\ (\delta(q,a),\varepsilon) & \text{if } ux = bb, \\ (\delta(q,b),\varepsilon) & \text{if } ux = ba, \\ (q,\varepsilon) & \text{if } ux \notin \{bb, ba\} \text{ and } q \notin S, \\ t & \text{if } ux \notin \{bb, ba\} \text{ and } q \in S, \end{cases}$$

for $u \in \{\varepsilon, b\}$, $q \in Q$ and $x \in \{a, b\}$.

First, suppose $u_1 \cdots u_n a v$ with $u_1, \ldots, u_n \in \{bb, ba\}$ and $v \in \Gamma^*$ synchronizes $\mathcal{A}'$. Then it must map all states to t. Consider the state $\delta'((q, \varepsilon), u_1 \cdots u_n)$ for some $q \in Q$. Then it must have the form (p, ε) for some $p \in Q$. Assume $p \notin S$. Then $\delta'((p, \varepsilon), a) = (p, \varepsilon)$ and since $|\Gamma| \leq 1$ the word v cannot map (p, ε) to t. Thus we must have $p \in S$ and then the word $v_1 \cdots v_n$ with $v_i = a$ if $u_i = bb$ and $v_i = b$ if $u_i = ba$ maps Q into S.

If $u_1 \cdots u_n$ for $u_1, \ldots, u_n \in \{a, b\}$ maps Q into S, then $v_1 \ldots v_n a$ with $v_i = bb$ if $u_i = a$ and $v_i = ba$ if $u_i = b$ for all $i \in \{1, \ldots, n\}$ maps all states of $Q \times \{\varepsilon\}$ to t. However, if $u_1 = \cdots = u_n = bb$, then $\delta'(Q \times \{b\}, u_1 \cdots u_n) \subseteq Q \times \{b\}$ and thus $\delta'(Q \times \{b\}, u_1 \cdots u_n a) \cap \delta(Q, a) \times \{\varepsilon\} \neq \emptyset$. Now no word from Γ^* can map this state to t. But this problem does not arise if we read ba at least once, since this word maps all states in $Q \times \{b\}$ to t. Now, note that $L \cap \Sigma^* baa\Sigma^* = \{bb, ba\}baa\Gamma^*$. Thus, by Theorem 18.12 we obtain PSPACE-hardness for Sync$[L]$ if we can show PSPACE-hardness for Sync$[\{bb, ba\}baa\Gamma^*]$ and to establish PSPACE-hardness of this problem we can use the above construction since the problematic case discussed above does not arise for $\{bb, ba\}baa\Gamma^*$.

18.2.4 Applying Morphisms and Solid Codes

Here, in Theorem 18.17, we show that transforming a constraint language yields a problem whose complexity is at most that of the original problem w.r.t. $\leq_m^{\mathsf{AC}^0}$-reductions. In Theorem 18.19 we show that for coding morphisms of solid codes the complexity is preserved.

Theorem 18.17 *Let $\varphi\colon \Sigma^* \to \Gamma^*$ be a fixed morphism and $L \subseteq \Sigma^*$. Then*

$$\textsc{Sync}[\varphi(L)] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L].$$

Proof. For the $\leq_m^{\mathsf{AC}^0}$-reduction, let $\mathcal{A} = (Q, \Gamma, \delta)$ be an input automaton. We want to know if $\mathcal{A}$ is synchronizing with respect to $\varphi(L)$.

Construction: Let $\mathcal{A}' = (Q, \Sigma, \delta')$ with $\delta'(q, x) = \delta(q, \varphi(x))$ for $q \in Q$ and $x \in \Sigma$. We obtain a representation of $\mathcal{A}'$ by transition matrices by multiplying for each $x \in \Sigma$ the respective transition matrices of the letters in $\varphi(x)$, which is doable by a L-uniform circuit family in AC^0 as φ is fixed and matrix multiplication for a fixed number of matrices is in AC^0 by Theorem 3.72.

Verification: We have

$$\begin{aligned} \exists u \in \varphi(L)\colon |\delta(Q, u)| = 1 &\Leftrightarrow \exists v \in L\colon |\delta(Q, \varphi(v))| = 1 && (\varphi(v) = u) \\ &\Leftrightarrow \exists v \in L\colon |\delta'(Q, v)| = 1. && (\delta'(Q, v) = \delta(Q, \varphi(v))) \end{aligned}$$

So $\mathcal{A}'$ has a synchronizing word in L if and only if $\mathcal{A}$ has a synchronizing word in $\varphi(L)$. □

Example 18.18 Let $L = \{a, b\}^* \cup \{a, c\}^* \cup \{b, c\}^*$. Then $\pi_{\{a,b\}}(L) = \{a, b\}^*$, so that $\textsc{Sync}[L]$ is NL-hard with Theorem 18.17 and Theorem 18.2, Item 3. With Theorem 18.9 we also get membership in NL.

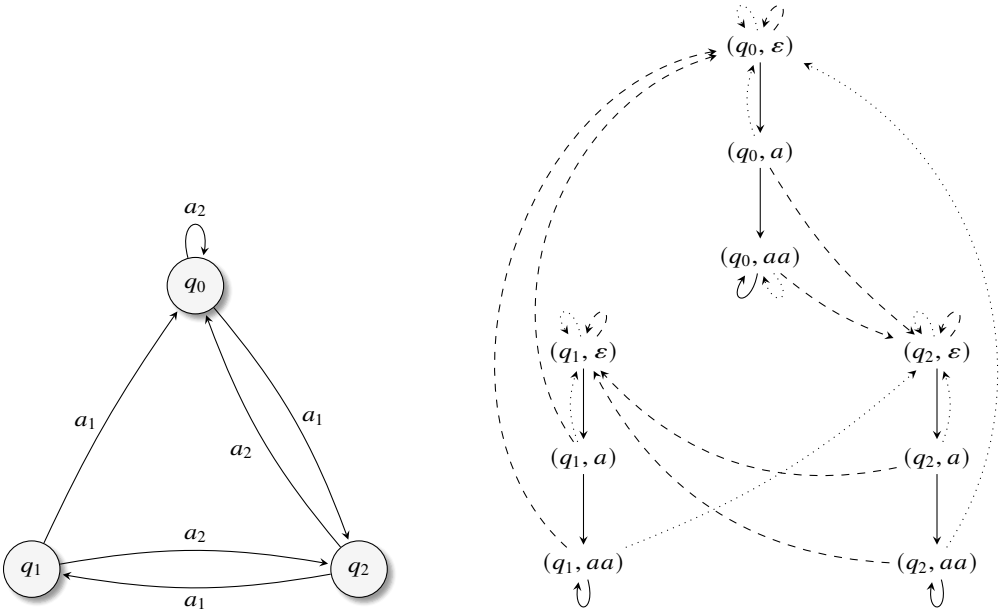
For coding morphisms of solid codes, we have a stronger statement than Theorem 18.17.

Theorem 18.19 *Let $\varphi\colon \Sigma^* \to \Gamma^*$ be a coding morphism such that[2] $C = \varphi(\Sigma)$ is a solid code. Then, for each regular $L \subseteq \Sigma^*$ we have $\textsc{Sync}[L] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\varphi(L)]$.*

Proof. By Theorem 18.17, we have $\textsc{Sync}[\varphi(L)] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L]$. Next, we show $\textsc{Sync}[L] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[\varphi(L)]$ by utilizing the Levenshtein-Romanov mappping from Section 17.3 w.r.t. the language C. Now, let $\mathcal{A} = (Q, \Sigma, \delta)$ be an input automaton. Suppose $\Sigma = \{a_1, \ldots, a_k\}$ and set $u_i = \varphi(a_i)$ for $i \in \{1, \ldots, k\}$. Then $C = \{u_1, \ldots, u_k\}$.

Construction: Define $\mathcal{A}' = (Q \times (\mathrm{Pref}(C) \setminus C), \Gamma, \delta')$ where, for $(q, u) \in Q \times \mathrm{Pref}(C) \setminus C$ and $x \in \Sigma$,

[2] Recall that Σ is finite, and so C is finite. Also coding morphism are injective by definition.



(a) A base automaton $\mathcal{A}$

(b) Constructed automaton as in the proof.

Fig. 18.2: Construction from the proof of Theorem 18.19 for the solid code $\{ab, aac\}$ and base automaton shown on the left side. Solid, dashed, and dotted lines represent transitions labeled with a, b, and c, respectively.

$$\delta'((q,u),x) = \begin{cases} (q, \sigma_C(ux)) & \text{if } \sigma_C(ux) \notin C, \\ (\delta(q,a_i),\varepsilon) & \text{if } \exists i \in \{1,\ldots,k\} : \sigma_C(ux) = u_i. \end{cases} \tag{18.2}$$

By encoding the proper prefixes in the second component we keep track of the prefix scanned in the input so far. See Figure 18.2 for an example construction for the solid code $C = \{ab, aac\}$. Note that in the example $\delta'((q,a),b) = \delta'((q,aa),b) = (\delta(q,a_1),\varepsilon)$ since $\sigma_C(ab) = \sigma_C(aab) = ab$.

Properties: We give the proofs for Properties 1 and 2, Property 3 follows from Property 2 by induction as $\varphi(v) \in C^+$ for $v \in \Sigma^+$.

1. For $(q,u) \in Q \times (\mathrm{Pref}(C) \setminus C)$ and $v \in \mathrm{Pref}(C) \setminus C$ we have $\sigma_C(uv) \in \mathrm{Pref}(C) \setminus C$ and $\delta'((q,u),v) = (q, \sigma_C(uv))$.
   Proof of 1: We show the claim by induction on v. As $\sigma_C(u) = u$, the claim is true for $v = \varepsilon$. Now, suppose $v = wx$ for $w \in \Sigma^*$ and $x \in \Sigma$. As C is overlap-free, $wx \in \mathrm{Pref}(C) \setminus C$ implies $\sigma_C(uwx) \notin C$ and so $\sigma_C(uwx) \in \mathrm{Pref}(C) \setminus C$; this implication is noted in Theorem 17.61, Item 2. By Theorem 17.3, Item 2 we have $\sigma_C(\sigma_C(uw)x) = \sigma_C(uwx)$. So, for $q \in Q$, we obtain

$$
\begin{aligned}
&\delta'((q,u),wx)\\
&= \delta'(\delta'((q,u),w),x) && \text{(Def. of } \delta'\text{)}\\
&= \delta'((q,\sigma_C(uw)),x) && \text{(Induction hypothesis)}\\
&= (q,\sigma_C(\sigma_C(uw)x)). && \text{(Eqn. (18.2) as } \sigma_C(\sigma_C(uw)x) \notin C\text{)}
\end{aligned}
$$

Hence, we get $\delta((q,u),wx) = (q,\sigma_C(uwx))$ and the claim follows by induction. *[End, Proof of 1]*

2. For $(q,u) \in Q \times (\mathrm{Pref}(C) \setminus C)$ and $i \in \{1,\ldots,k\}$ we have

$$\delta'((q,u),u_i) = (\delta(q,a_i),\varepsilon)$$

Proof of 2: Let $u_i = vx$ for $v \in \Sigma^*$ and $x \in \Sigma$. As C is a prefix code, we have $v \in \mathrm{Pref}(C) \setminus C$. By Theorem 17.3, Item 2 we have $\sigma_C(\sigma_C(uv)x) = \sigma_C(uvx)$. By Theorem 17.61, Item 3 as $vx \in C$, we find $\sigma_C(uvx) = vx = u_i$. Then

$$
\begin{aligned}
\delta'((q,u),vx) &= \delta'(\delta'((q,u),v),x) && \text{(Def. of } \delta'\text{)}\\
&= \delta'((q,\sigma_C(uv)),x) && \text{(Property 1)}\\
&= (\delta(q,a_i),\varepsilon). && \text{(Eqn. (18.2) as } \sigma_C(\sigma_C(uv)x) = u_i\text{)}
\end{aligned}
$$

So, $\delta'((q,u),u_i) = (\delta(q,a_i),\varepsilon)$. *[End, Proof of 2]*

3. For every $v \in \Sigma^+$ and $(q,u) \in Q \times (\mathrm{Pref}(C) \setminus C)$,

$$\delta'((q,u),\varphi(v)) = (\delta(q,v),\varepsilon). \tag{18.3}$$

Verification: That the reduction is correct follows by the following equivalences:

$$
\begin{aligned}
&\exists w \in \varphi(L)\colon |\delta'(Q \times (\mathrm{Pref}(C) \setminus C),w)| = 1\\
&\Leftrightarrow \exists v \in L\colon |\delta'(Q \times (\mathrm{Pref}(C) \setminus C),\varphi(v))| = 1\\
&\Leftrightarrow \exists v \in L\colon |\delta(Q,v) \times \{\varepsilon\}| = 1 && \text{(Eqn. (18.3))}\\
&\Leftrightarrow \exists v \in L\colon |\delta(Q,v)| = 1.\square
\end{aligned}
$$

Remark 18.20 The construction from Theorem 17.88 is a special case of Theorem 18.19 as for Γ, Σ with $\Gamma \subseteq \Sigma$ the inclusion map $i\colon \Gamma \to \Sigma$ given by $i(x) = x$ for $x \in \Gamma$ is a coding morphism where $i(\Gamma) \subseteq \Sigma$ is a solid code.

Remark 18.21 The construction from the proof of Theorem 18.19 with the transition function given by Equation (18.2) can be applied to any prefix code C and yields a well-defined automaton $\mathcal{A}' = (Q \times \mathrm{Pref}(C) \setminus C, \Sigma, \delta')$. Can the assumption for C to be a solid code be weakened? The crucial property for the reduction to work is that $\delta'(Q \times \mathrm{Pref}(C) \setminus C, u_i) = \delta(Q,a_i) \times \{\varepsilon\}$ for $i \in \{1,\ldots,k\}$ and $C = \{u_1,\ldots,u_k\}$. However, this is not guaranteed to hold if C is merely a comma-free code. For example, consider $C = \{u_1,u_2\}$ with $u_1 = aab$ and $u_2 = bba$, which is a comma-free code, but not a solid code since u_1 and u_2 have a non-empty proper overlap. Then we have $\delta'(Q \times \{aa\},u_2) = \delta(Q,a_1) \times \{a\}$ and so $\delta(Q,a_1) \times \{a\} \subseteq \delta'(Q \times \mathrm{Pref}(C) \setminus C,u_2)$.

However, there are codes that are not solid codes for which it does hold. For example, for $C = \{a_1, \ldots, a_{k-1}, a_k^m a_{k-1}\}$ for distinct $a_1, \ldots, a_k \in \Sigma$ and $m \geq 1$.

Remark 18.22 Solid codes and the construction from the proof of Theorem 18.19 can also be used to give an alternative proof of Theorem 8.10 by first giving an automaton derived from the digraph by labeling the edges with letters and using an additional letter to realize the transitions going back to the start state. Then using a binary solid code and the construction from the proof of Theorem 18.19 we can reduce the alphabet to a binary alphabet. However, note that the words bb, a, ba actually used in the proof of Theorem 8.10 do not form a solid code.

Remark 18.23 We actually have $\mathrm{Syn}(\mathcal{A}') \cap \varphi(\Sigma^*) = \varphi(\mathrm{Syn}(\mathcal{A}))$ in the construction from the proof of Theorem 18.19.

Coding morphism of solid codes are not the only morphisms that preserve the complexity, as shown in the following remark.

Remark 18.24 Consider the coding morphism $\varphi\colon \{a, b\}^* \to \{c, d, e\}^*$ given by $\varphi(a) = cd$ and $\varphi(b) = ec$. Now $\varphi(\{a, b\}) = \{cd, ec\}$ is not a solid code. For $\mathcal{A} = (Q, \{a, b\}, \delta)$ let $\mathcal{A}' = (Q', \{c, d\}, \delta')$ with $Q' = Q \times \{\varepsilon, c, e\}$ where, for $q \in Q$, $u \in \{\varepsilon, c, d\}$, and $x \in \Sigma$,

$$\delta'((q,u),x) = \begin{cases} (\delta(q,a),\varepsilon) & \text{if } u \in \{\varepsilon, c\} \text{ and } x = d, \\ (\delta(q,b),\varepsilon) & \text{if } u = e \text{ and } x = c, \\ (q,x) & \text{if } u \in \{\varepsilon, c, e\} \text{ and } x \in \{c, e\} \text{ and } ux \neq ec \\ (q,u) & \text{if } u = e \text{ and } x = d. \end{cases}$$

Then we have $\delta'((q,u),\varphi(v)) = (\delta(q,v),\varepsilon)$ for $v \in \{a,b\}^+$ with $u \in \{\varepsilon, c, e\}$ and $q \in Q$. This yields

$$\delta'(Q', \varphi(a)) = \delta(Q, a) \times \{\varepsilon\},$$
$$\delta'(Q', \varphi(b)) = \delta(Q, b) \times \{\varepsilon\},$$

and that for $|Q| \geq 2$ and regular $L \subseteq \{a, b\}^*$, the automaton $\mathcal{A}$ has a synchronizing word in L if and only if $\mathcal{A}'$ has a synchronizing word in $\varphi(L)$.

18.2.5 Concatenation and Complexity Lower Bounds

In general, the computational complexity might increase or decrease when concatenating languages. For example, by Theorem 18.2, Theorem 18.5 and as $\mathrm{Fact}((a + b + c)^*c(a+b)^*) = \mathrm{Fact}((a+b)^*)$, $\textsc{Sync}[(a + b)^*]$, $\textsc{Sync}[c(a + b)^*]$, and $\textsc{Sync}[(a + b + c)^*c(a + b)^*]$ are complete for NL, PSPACE, and NL, respectively. Here, we identify conditions on concatenating languages to obtain complexity lower bounds. First, we give two results for concatenating a language to the front of a constraint language such that hardness is preserved, where Theorem 18.25 is related to languages contained in a subalphabet and

Theorem 18.28 to prefix codes. Then, Theorem 18.31 is concerned with concatenation to the back. The conditions presented here are used to deduce hardness results in later sections.

The reductions of the present section involve operations as mentioned in Theorem 3.85 and can be realized by AC^0 circuits.

Proposition 18.25 *Let $L \subseteq \Sigma^*$ and $U \subseteq \Gamma^*$ for some $\Gamma \subseteq \Sigma$ be regular languages and $w \in \Sigma^*$. Suppose one of the following holds:*

1. *$L \cap \Gamma\Sigma^* = \emptyset$ with $w \in \Gamma^*$, or*
2. *$w \notin \Gamma^*$.*

Then $\textsc{Sync}[wL] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[UwL]$.

Proof. First, assume $L \cap \Gamma\Sigma^* = \emptyset$ and $w \in \Gamma^*$. Let $\mathcal{A} = (Q, \Sigma, \delta)$. We construct a semiautomaton $\mathcal{A}'$ that has a synchronizing word in UwL if and only if $\mathcal{A}$ has a synchronizing word in wL.

Construction: Let $\mathcal{A}' = (Q \times \{0,1\}, \Sigma, \delta')$ where, for $i \in \{0,1\}$, $q \in Q$ and $x \in \Sigma$,

$$\delta'((q,i),x) = \begin{cases} (q,0) & \text{if } i = 0 \text{ and } x \in \Gamma, \\ (\delta(q,wx),1) & \text{if } i = 0 \text{ and } x \notin \Gamma, \\ (\delta(q,x),1) & \text{if } i = 1. \end{cases}$$

Properties: Let $q \in Q$, $u \in U$ and $v \in L$. Then, as $v \notin \Gamma\Sigma^*$ and $uw \in \Gamma^*$, we have

$$\delta'((q,0),uwv) = \delta'((q,0),v), \tag{18.4}$$
$$\delta'((q,0),v) = (\delta(q,wv),1), \tag{18.5}$$
$$\delta'((q,1),uwv) = (\delta(q,uwv),1). \tag{18.6}$$

Verification: Let $u \in U$ and $v \in L$. Then we have

$$\begin{aligned} \delta'(Q \times \{0,1\}, uwv) &= \delta'(Q \times \{0\}, uwv) \cup \delta'(Q \times \{1\}, uwv) \\ &= \delta'(Q \times \{0\}, v) \cup \delta'(Q \times \{1\}, uwv) && \text{(Eqn. (18.4))} \\ &= \delta(Q, wv) \times \{1\} \cup \delta'(Q \times \{1\}, uwv) && \text{(Eqn. (18.5))} \\ &= \delta(Q, wv) \times \{1\} \cup \delta(Q, uwv) \times \{1\} && \text{(Eqn. (18.6))} \\ &= \delta(Q, wv) \times \{1\}. && (\delta(\delta(Q,u),wv) \subseteq \delta(Q,wv)) \end{aligned}$$

So $|\delta'(Q \times \{0,1\}, uwv)| = 1$ is equivalent to $|\delta(Q, wv)| = 1$ and $\mathcal{A}'$ has a synchronizing word in UwL if and only if $\mathcal{A}$ has a synchronizing word in wL.

Lastly, for the remaining case, assume $w \notin \Gamma^*$. Then $w = ubv$ for some $u \in \Gamma^*$, $b \notin \Gamma$, and $v \in \Sigma^*$. Set $L' = bvL$. We have $L' \cap \Gamma\Sigma^* = \emptyset$. So, applying the previous case to L', u and U, we get $\textsc{Sync}[uL'] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[UuL']$ where $uL' = wL$. □

Example 18.26 By Theorem 18.2, Item 5, $\textsc{Sync}[c(a+b)^*]$ is PSPACE-complete. By Theorem 18.25, the problems $\textsc{Sync}[aabbc(a+b)^*]$ and $\textsc{Sync}[a^*c(a+b)^*]$ are PSPACE-complete as well.

For reference, we list two special cases of Theorem 18.25.

Corollary 18.27 *Let $L \subseteq \Sigma^*$ be regular.*

1. *If $L \cap \Gamma\Sigma^* = \emptyset$, $\Gamma \subseteq \Sigma$ and $U \subseteq \Gamma^*$ is regular, then* $\textsc{Sync}[L] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[UL]$.
2. $\textsc{Sync}[wL] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[a^*wL]$ *for $w \notin a^*$ and $a \in \Sigma$.*

Next, we present a condition related to prefix codes.

Theorem 18.28 *Let $L \subseteq \Sigma^*$ be regular and $P \subseteq \Sigma^*$ be a prefix code that is a regular language. Then* $\textsc{Sync}[L] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[PL]$.

Proof. By Theorem 17.30, Item 1 there exists a trim PDFA $C = (R, \Sigma, \rho, r_0, \{e\})$ such that $P = L(C)$ and where the state e has no outgoing transitions. Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is an input semiautomaton.

Construction: Let $\mathcal{A}' = (R \times Q, \Sigma, \delta')$ where, for $(r, q) \in R \times Q$ and $x \in \Sigma$,

$$\delta'((r,q),x) = \begin{cases} (\rho(r,x), q) & \text{if } \rho(p,x) \text{ is defined,} \\ (e, \delta(q,x)) & \text{if } \rho(p,x) \text{ is undefined.} \end{cases}$$

Properties: Property 1 follows readily by induction. We give the proof for Property 2.

1. For $r \in R$, $q \in Q$ and $u \in \Sigma^*$ we have

$$\delta'((r,q),u) = \begin{cases} (e, \delta(q,xu_2)) & \text{if } \exists u_1, u_2 \in \Sigma^* \; \exists x \in \Sigma \colon u = u_1 x u_2 \wedge \\ & \qquad \rho(r,u_1) \text{ is defined} \wedge \\ & \qquad \rho(r,u_1x) \text{ is undefined,} \\ (\rho(r,u), q) & \text{if } \rho(r,u) \text{ is defined.} \end{cases} \tag{18.7}$$

2. Let $w \in \Sigma^*$ be such that $\rho(R, w) = \{e\}$. Then $\delta'(R \times Q, w) = \{e\} \times Q$.
   Equation (18.7) gives $\delta'(R \times Q, w) \subseteq \{e\} \times Q$. Furthermore, there exists $r \in R$ such that $\rho(r, w) = e$. Then, for each $q \in Q$, we have $\delta'((r, q), w) = (e, q)$ and this gives $\{e\} \times Q = \delta'(\{r\} \times Q, w) \subseteq \delta'(R \times Q, w)$.

Verification: By Property 1 we have $\delta'(\{r_0\} \times Q, uv) = \{e\} \times \delta(Q, v)$ for $u \in P$ and $v \in \Sigma^*$. (note that, since P is a prefix code, uv can only be factored into a word from P and an arbitrary word in one way) So, $|\delta'(R \times Q, uv)| = 1$ for $u \in P$ and $v \in L$ implies $|\delta(Q, v)| = 1$.

Conversely, suppose $|\delta(Q, v)| = 1$ for $v \in L$. By Property 1 and as $\rho(e, x)$ is undefined for each $x \in \Sigma$ we have $\delta'(\{e\} \times Q, v) = \{e\} \times \delta(Q, v)$ (here, we used that P is a prefix code). By Theorem 17.30, Item 3 there exists $w \in P$ with $\rho(R, w) = \{e\}$. Then, by Property 2 we have $\delta'(R \times Q, w) \subseteq \{e\} \times Q$ (note that we might have that $\rho(r, w)$ is undefined for some $r \in R$ and that, in fact, we have equality since $\delta'(\{r_0\} \times Q), w) = \{e\} \times Q$). So $|\delta'(R \times Q, wv)| = 1$. □

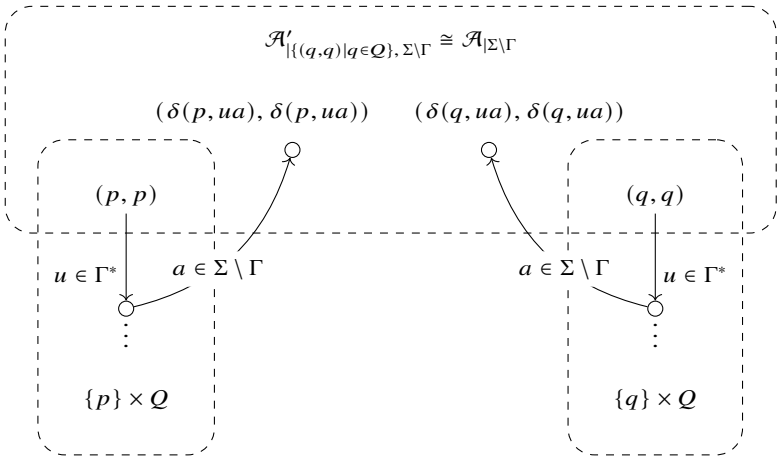


Fig. 18.3: Schematic of the reduction from the proof of Proposition 18.31. For a state q and word $u \in \Gamma^*$ we "branch out" into a copy $\{q\} \times Q$. For a letter from $a \in \Sigma \setminus \Gamma$, we return to the state $(\delta(q,ua),\delta(q,ua))$. This is indicated for two states p and q.

Remark 18.29 For $L = \{c\}$ and $P = \{a,b\}^*c$ we have that $\textsc{Sync}[L]$ and $\textsc{Sync}[PL]$ are in AC^0 and PSPACE-complete, respectively, by Theorem 18.2. However, the complexity does not have to increase when concatenating a prefix code that is regular to the front of a constraint language as can be seen with $L = \Sigma^*$. Then $\textsc{Sync}[L] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[PL]$ by Theorem 18.7.

As $\{u\}$ for $u \in \Sigma^*$ is a prefix code, we obtain the following corollary.

Corollary 18.30 *Let $L \subseteq \Sigma^*$ be regular and $u \in \Sigma^*$. Then $\textsc{Sync}[L] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[uL]$.*

Next, we state a result on concatenating languages to the back of the constraint language.

Proposition 18.31 *Let $L \subseteq \Sigma^*$ be regular and $\Gamma \subseteq \Sigma$ such that $L \cap \Sigma^*\Gamma = \emptyset$. If $U \subseteq \Gamma^*$ is a regular non-empty language, then $\textsc{Sync}[L] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[LU]$.*

Proof. We can assume $L \neq \emptyset$. Let $\mathcal{A} = (Q,\Sigma,\delta)$. We construct a semiautomaton $\mathcal{A}'$ that has a synchronizing word in LU if and only if $\mathcal{A}$ has a synchronizing word in L. Figure 18.3 shows the basic idea of the construction graphically.

Construction: Let $\mathcal{A}' = (Q',\Sigma,\delta')$ where[3] $Q' = Q \times Q$ and, for $(q,p) \in \{q\} \times Q$, $x \in \Sigma$,

[3] Actually, setting

$$Q' = \bigcup_{q\in Q} \{q\} \times S_q$$

where $S_q = \{\delta(q,u) \mid u \in \Gamma^*\}$ for $q \in Q$ would be sufficient. However, it is not clear whether the sets S_q can be computed by an L-uniform AC^0 circuit family whereas $Q \times Q$ can be computed by such a circuit family.

$$\delta'((q,p),x) = \begin{cases} (q,\delta(p,x)) & \text{if } x \in \Gamma, \\ (\delta(p,x),\delta(p,x)) & \text{if } x \notin \Gamma. \end{cases}$$

Properties: Next, we collect some properties of the given construction.

1. For $w \in \Gamma^*$ and $q, p \in Q$, we have $\delta'((q,p),w) = (q,\delta(p,w))$.
   Proof of 1: This is clear for $w = \varepsilon$. Next, let $w = ux$ for some $x \in \Sigma$ and $u \in \Sigma^*$. Inductively, we have $\delta'((q,p),ux) = \delta'((q,\delta(p,u)),x)$. Since $x \in \Gamma$, by the definition of the transition function we find $\delta'((q,\delta(p,u)),x) = (q,\delta(p,w))$. *[End, Proof of 1]*
2. If $w \in \Sigma^*(\Sigma \setminus \Gamma)$ and $q, p \in Q$, then $\delta'((q,p),w) = (\delta(p,w),\delta(p,w))$.
   Proof of 2: By assumption, we can write $w = w_0u_1w_1 \cdots u_nw_n$ for some $n \geq 0$ with $w_0, w_1, \ldots, w_n \in (\Sigma \setminus \Gamma)^+$ and $u_1, \ldots, u_n \in \Gamma^*$. For $n = 0$, the claim follows by an easy induction from the defining equation of δ'. Next, suppose $n \geq 1$. Since $|w_{n-1}| \geq 1$, inductively, we obtain

   $$\delta'((q,p),w') = (\delta(p,w'),\delta(p,w'))$$

   for $w' = w_0u_1w_1 \cdots w_{n-1}$. By Property 1 we have

   $$\delta'((\delta(p,w'),\delta(p,w')),u_n) = (\delta(p,w'),\delta(p,w'u_n)).$$

   Then applying the claim for $n = 0$ again, we find

   $$\delta'((\delta(p,w'),\delta(p,w'u_n)),w_n) = (\delta(p,w'u_nw_n),\delta(p,w'u_nw_n)).$$

   Thus we have $\delta'((q,p),w) = (\delta(p,w),\delta(p,w))$. *[End, Proof of 2]*
3. For $v \in L \cap \Sigma^+$, we have $\delta'(Q',v) = \{(q,q) \mid q \in \delta(Q,v)\}$.
   Proof of 3: By assumption on L, we have $L \cap \Sigma^+ \subseteq \Sigma^*(\Sigma \setminus \Gamma)$ and so $v \in \Sigma^*(\Sigma \setminus \Gamma)$. Thus, the claim follows from Property 2. *[End, Proof of 3]*

Verification: We can assume $|Q| \geq 2$. First, suppose $|\delta(Q,v)| = 1$ for some $v \in L$. Then, by Property 3, $|\delta'(Q \times Q, v)| = 1$. So, $|\delta'(Q',vu)| = 1$ for $u \in U$.

Conversely, assume $|\delta'(Q',vu)| = 1$ for $v \in L$ and $u \in U$. By Property 1, for $p, q \in Q$ and $u \in \Gamma^*$, we have $\delta'((q,q),u) = (q,\delta(q,u))$ and $\delta'((p,p),u) = (p,\delta(p,u))$. So,

$$\delta'((q,q),u) = \delta'((p,p),u) \Leftrightarrow q = p.$$

Hence, with Property 3, $|\delta'(Q',vu)| = 1$ implies $|\delta'(Q',v)| = 1$, which, again by Property 3, then implies $|\delta(Q,v)| = 1$. □

Lastly, we give a condition for preservation of the complexity w.r.t. concatenation.

Proposition 18.32 *Let $L_1, L_2, L_3 \subseteq \Sigma^*$ be regular languages. If $L_1L_2L_3 \subseteq \mathrm{Fact}(L_2)$, then* $\textsc{Sync}[L_1L_2L_3] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L_2]$.

Proof. In general, it is $\mathrm{Fact}(L_2) \subseteq \mathrm{Fact}(L_1L_2L_3)$. By assumption, $\mathrm{Fact}(L_1L_2L_3) \subseteq \mathrm{Fact}(L_2)$. So, the claim follows from Theorem 18.5. □

Corollary 18.33 *For $u, v \in \Sigma^*$ and $L \subseteq \Sigma^*$ with $uLv \subseteq \mathrm{Fact}(L)$ we have* $\textsc{Sync}[uLv] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L]$.

18.3 Hardness Conditions

Here we present NL-, PSPACE-, and NP-hardness conditions in Sections 18.3.5 to 18.3.7. Some of these conditions use the notion of auxiliary state avoiding words, which we introduce in Section 18.3.1. Section 18.3.2 investigates the relation of auxiliary state avoiding words to codes. The proofs also uses equivalence relations presented in Section 18.3.4 that are derived from the Myhill-Nerode relation from Section 6.7.2.

18.3.1 Auxiliary State Avoiding Words

In our PSPACE-hardness reductions in Sections 18.3.5 and 18.3.6 we construct, from a given automaton, a new automaton having additional "auxiliary" states. For this new automaton to be synchronizable, those additional states must be mapped to a synchronizing state as well. Existence of certain (weakly/strongly) auxiliary state avoiding words guarantees that this is possible, which also explains why we call them auxiliary state avoiding.

Here, we introduce (weakly/strongly) auxiliary state avoiding words and collect various statements related to closure under concatenation (Theorem 18.35), weak auxiliary state avoidability w.r.t. different languages (Theorem 18.36), the relation between strong and weak auxiliary state avoidability (Theorems 18.38 and 18.42), partitions (Theorem 18.44), and a sufficient condition for weak auxiliary state avoidability (Theorem 18.45).

Let $L \subseteq \Sigma^*$. We call a word $w \in \Sigma^*$

1. *weakly auxiliary state avoiding (w.r.t. L)* if $w \in (\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*))^+$,
2. *strongly auxiliary state avoiding (w.r.t. L)* if $w \in ((\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L))^+$.

When the context is clear, we omit mentioning the language with respect to which we consider a word weakly or strongly auxiliary state avoiding.

Example 18.34
1. Let $\Gamma \subseteq \Sigma$ be a proper subset. Then $(\Sigma \setminus \Gamma)^+$ is the set of all strongly auxiliary state avoiding words w.r.t. Γ and $(\Gamma^*(\Sigma \setminus \Gamma))^+$ the set of weakly auxiliary state avoiding words w.r.t. Γ.
2. Every non-empty word is weakly auxiliary state avoiding w.r.t. $\{\varepsilon\}$ and, as $\emptyset^* = \{\varepsilon\}$, also w.r.t. $\emptyset$. However, no word is strongly auxiliary state avoiding w.r.t. $\{\varepsilon\}$ nor w.r.t. $\emptyset$.
3. If $L = \mathrm{Pref}(L)$, then no word is strongly auxiliary state avoiding w.r.t. L.
4. By Theorem 18.58 no word is weakly or strongly auxiliary state avoiding w.r.t. a maximal prefix code. (in fact, for prefix codes this property characterizes maximality.)

The empty word is never weakly nor strongly auxiliary state avoiding for any language.

Weak (resp. strong) auxiliary state avoidability w.r.t. to a common language is preserved under concatenation.

Proposition 18.35 *Let $L \subseteq \Sigma^*$ and $u, v \in \Sigma^*$ be two weakly (resp. strongly) auxiliary state avoiding words w.r.t. L. Then uv is weakly (resp. strongly) auxiliary state avoiding w.r.t. L.*

Proof. This follows as the set of weakly auxiliary state avoiding words $(\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*))^+$ (resp. the set of strongly auxiliary state avoiding words $((\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L))^+$) forms a semigroup, i.e., is closed for concatenation. □

Next, we show that being weakly auxiliary state avoiding w.r.t. L, L^n for $n \geq 1$, or L^* are all equivalent properties.

Proposition 18.36 *Let $L \subseteq \Sigma^*$, $u \in \Sigma^*$, and $n \geq 1$. Then the following are equivalent:*

1. *u is weakly auxiliary state avoiding w.r.t. L,*
2. *u is weakly auxiliary state avoiding w.r.t. L^n,*
3. *u is weakly auxiliary state avoiding w.r.t. L^*.*

Proof. All equivalences follow as $\mathrm{Pref}(L^*) = \mathrm{Pref}((L^n)^*)$ and $(L^*)^* = L^*$. □

Next, we give a statement that yields, in Theorem 18.38 a relation between weak and strong auxiliary state avoidability w.r.t. L and L^n, respectively, for some $n \geq 1$.

Proposition 18.37 *Let $L \subseteq \Sigma^+$ be some language and $n \geq 1$. Then we have* $(\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*)) \cap \Sigma^{\leq n} = ((\mathrm{Pref}(L^n) \setminus L^n)\Sigma \setminus \mathrm{Pref}(L^n)) \cap \Sigma^{\leq n}$.

Proof. Let $u \in \Sigma^{\leq n}$. By Equation (3.43) we have $v \in \mathrm{Pref}(L^*)$ if and only if $v \in \mathrm{Pref}(L^n)$ for $v \in \Sigma^{\leq n}$. Thus, $u \in \mathrm{Pref}(L^*)\Sigma$ and $u \notin \mathrm{Pref}(L^*)$ are equivalent to $u \in \mathrm{Pref}(L^n)\Sigma$ and $u \notin \mathrm{Pref}(L^n)$. Since $\varepsilon \notin L$, we have that $v \in L^m$ implies $|v| \geq m$ for $m \geq 1$. So from $u \in \Sigma^{\leq n}$ we get that for $v \in \Sigma^*$ with $u \in v\Sigma$ we have $|v| < n$ and so $v \notin L^n$. Thus we find $u \in \mathrm{Pref}(L^n)\Sigma$ if and only if $u \in (\mathrm{Pref}(L^n) \setminus L^n)\Sigma$. This gives the claim. □

Corollary 18.38 *Let $L \subseteq \Sigma^+$ be some language and $n \geq 1$. A word $u \in \Sigma^{\leq n}$ is weakly auxiliary state avoiding w.r.t. L if and only if it is strongly auxiliary state avoiding w.r.t. L^n.*

Proof. By 18.37 we obtain $(\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*))^+ \cap \Sigma^{\leq n} = ((\mathrm{Pref}(L^n)\Sigma \setminus L^n) \setminus \mathrm{Pref}(L^n))^+ \cap \Sigma^{\leq n}$, which gives the claim. □

As a consequence of the next statement, we obtain that for languages with $\varepsilon \notin L$ every weakly auxiliary state avoiding word w.r.t L has a suffix that is strongly auxiliary state avoiding w.r.t. L.

Proposition 18.39 *Let $L \subseteq \Sigma^+$ be a language not containing the empty word. Then we have*

$$\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*) \subseteq L^*((\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L)).$$

Proof. Let $u \in \operatorname{Pref}(L^*)\Sigma \setminus \operatorname{Pref}(L^*)$. Write $u = u_1 \cdots u_n x$ with

1. $u_1, \ldots, u_{n-1} \in L$,
2. $x \in \Sigma$, and
3. $u_n = \varepsilon$ or $u_n \in \operatorname{Pref}(L) \setminus L$.

We have $\varepsilon \in \operatorname{Pref}(L) \setminus L$ since $\varepsilon \notin L$. Thus, even for $u_n = \varepsilon$, we have $u_n \in \operatorname{Pref}(L) \setminus L$. Since $u \notin \operatorname{Pref}(L^*)$ we have $u_n x \notin \operatorname{Pref}(L)$. Hence, $u \in L^*((\operatorname{Pref}(L) \setminus L)\Sigma \setminus \operatorname{Pref}(L))$. □

Theorem 18.39 might fail for languages containing the empty word as shown next.

Example 18.40 Let $L = \{\varepsilon, ac\}$. Then $\operatorname{Pref}(L) \setminus L = \{a\}$ and $acb \in \operatorname{Pref}(L^*)\Sigma \setminus \operatorname{Pref}(L^*)$ but $acb \notin L^*((\operatorname{Pref}(L) \setminus L)\Sigma \setminus \operatorname{Pref}(L))$.

The inclusion from Theorem 18.39 can be proper, as the next example shows.

Example 18.41 Let $L = \{b, babb, aba\}$. Then $babb \in L^*((\operatorname{Pref}(L) \setminus L)\Sigma \setminus \operatorname{Pref}(L)) \setminus (\operatorname{Pref}(L^*)\Sigma \setminus \operatorname{Pref}(L^*))$.

Corollary 18.42 *Let $L \subseteq \Sigma^+$. Every weakly auxiliary state avoiding word w.r.t. L has a suffix that is strongly auxiliary state avoiding w.r.t. L.*

Proof. By Theorem 18.39, we have $(\operatorname{Pref}(L^*)\Sigma \setminus \operatorname{Pref}(L^*))^+ \subseteq (L^*((\operatorname{Pref}(L) \setminus L)\Sigma \setminus \operatorname{Pref}(L)))^+$. □

In Theorem 18.51 we will show that we have equality in Theorem 18.39 for prefix codes.

Next, in Theorem 18.44, we give various partitions of Σ^*. Equations (18.10) and (18.11) involve weakly and strongly auxiliary state avoiding words, respectively. The proof of Equation (18.11) is based on the following lemma.

Lemma 18.43 *Let $L \subseteq \Sigma^*$. Set $L_1 = (\operatorname{Pref}(L) \setminus L)\Sigma \setminus \operatorname{Pref}(L)$ and $L_2 = L\Sigma \setminus \operatorname{Pref}(L)$. Then we have $\operatorname{Pref}(L)\Sigma \setminus \operatorname{Pref}(L) = L_1 \cup L_2$ and the right ideals $L_1\Sigma^*$ and $L_2\Sigma^*$ are disjoint.*

Proof. Clearly, we have $\operatorname{Pref}(L)\Sigma = (\operatorname{Pref}(L) \setminus L)\Sigma \cup L\Sigma$, which gives the first equation.

Next, let $u_1 x v = u_2 y w$ with $u_1 x \in L_1$, $u_2 y \in L_2$, $x, y \in \Sigma$, and $v, w \in \Sigma^*$. Since $u_1, u_2 \in \operatorname{Pref}(L)$ while $u_1 x, u_2 y \notin \operatorname{Pref}(L)$, neither u_1 nor u_2 can be a proper prefix of the other. Indeed, if $|u_1| < |u_2|$, then $u_1 x$ would be a prefix of u_2, and hence $u_1 x \in \operatorname{Pref}(L)$, a contradiction. Similarly, $|u_2| < |u_1|$ leads to a contradiction. Therefore $|u_1| = |u_2|$, which gives $u_1 = u_2$ since $u_1 x v = u_2 y w$. However, by the definitions of L_1 and L_2, we have $u_1 \notin L$ and $u_2 \in L$, contradicting $u_1 = u_2$. □

Proposition 18.44 *Let $L \subseteq \Sigma^*$. Then the following holds:*

$$\Sigma^* = \operatorname{Pref}(L) \uplus (\operatorname{Pref}(L)\Sigma \setminus \operatorname{Pref}(L))\Sigma^* \tag{18.8}$$

$$\Sigma^* = \operatorname{Pref}(L) \uplus (\operatorname{Pref}(L)\Sigma \setminus \operatorname{Pref}(L))^+ \operatorname{Pref}(L) \tag{18.9}$$

$$\Sigma^* = \operatorname{Pref}(L^*) \uplus (\operatorname{Pref}(L^*)\Sigma \setminus \operatorname{Pref}(L^*))^+ \operatorname{Pref}(L^*) \tag{18.10}$$

$$\Sigma^* = U \uplus ((\operatorname{Pref}(L) \setminus L)\Sigma \setminus \operatorname{Pref}(L)))^+ U \tag{18.11}$$

where $U = \operatorname{Pref}(L) \uplus (L\Sigma \setminus \operatorname{Pref}(L))\Sigma^*$.

Proof. Equation (18.8): Let $u \in \Sigma^*$ and let $v \in \Sigma^*$ be a longest prefix of u in $\mathrm{Pref}(L)$. Then we either have $u = v$, and so $u \in \mathrm{Pref}(L)$, or $u = vxw$ with $x \in \Sigma$ and $w \in \Sigma^*$ such that $vx \notin \mathrm{Pref}(L)$, i.e., $u \in (\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L))\Sigma^*$.

Since the prefix of every word in $\mathrm{Pref}(L)$ is in $\mathrm{Pref}(L)$, and each word in $(\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L))\Sigma^*$ has a prefix not in $\mathrm{Pref}(L)$, both sets are disjoint.

Equation (18.9): The language $(\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L))\Sigma^*$ does not contain the empty word. Then Arden's lemma (Theorem 3.62) applied to Equation (18.8) gives

$$\Sigma^* = (\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L))^* \mathrm{Pref}(L) = \mathrm{Pref}(L) \cup (\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L))^+ \mathrm{Pref}(L),$$

where disjointness of the union also follows since the union from Equation (18.8) is disjoint.[4]

Equation (18.10): This follows from Equation (18.9) applied to L^*.

Equation (18.11): By Theorem 18.43 we find

$$\Sigma^* = U \uplus ((\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L))\Sigma^*.$$

Then Theorem 3.62 gives $\Sigma^* = ((\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L)))^* U$. The sets $\mathrm{Pref}(L)$ and $((\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L)))\Sigma^*$ are disjoint. By Theorem 18.43, $(L\Sigma \setminus \mathrm{Pref}(L))\Sigma^*$ and $(\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L))\Sigma^*$ are disjoint. These sets are also pairwise disjoint. This gives the claim.[5] □

Lastly, we give a sufficient condition to identify weakly auxiliary state avoiding words in terms of the Levenshtein-Romanov mapping.

Proposition 18.45 *Let $L \subseteq \Sigma^*$ and $u \in \Sigma^+$. If $\sigma_L(u) = \varepsilon$, then u is weakly auxiliary state avoiding w.r.t. L.*

Proof. As $\sigma_L(u) = \varepsilon$, u has no non-empty suffix in $\mathrm{Pref}(L^*)$. So, by Equation (18.10) we find that $u \in (\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*))^+$. □

The next example gives $L \subseteq \Sigma^*$ and weakly auxiliary state avoiding $u \in \Sigma^*$ with $\sigma_L(u) \neq \varepsilon$.

Example 18.46 Let $L = \{bc, a\}$. Then ba is strongly and weakly auxiliary state avoiding w.r.t. L, but $\sigma_L(ba) = a$. For another example, the word aa is strongly and weakly auxiliary state avoiding w.r.t. $\{ab, b\}$ and we have $\sigma_{\{ab,b\}}(aa) = a$.

---

[4] The claim also follows from Theorem 3.63 applied to $\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L)$ since $\mathrm{Pref}(L) = \Sigma^* \setminus (\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L))\Sigma^*$.

[5] The claim also follows by applying Theorem 3.63 to $(\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L)$, which gives

$$\Sigma^* = U \cup (\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L))^+ U$$

with $U = \Sigma^* \setminus ((\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L))\Sigma^*) = \mathrm{Pref}(L) \cup (L\Sigma \setminus \mathrm{Pref}(L))\Sigma^*$.

18.3.2 Auxiliary State Avoiding Words and Prefix Codes

In our applications, the notions of strongly or weakly auxiliary state avoiding words occur in conjunction with prefix codes. Here, we investigate relations to prefix codes.

Theorem 18.48 states that the sets of weakly and strongly auxiliary state avoiding words w.r.t. some language are generated by prefix codes. This follows from Theorem 18.47. In Theorem 18.49 we prove that w.r.t. a prefix code, every strongly auxiliary state avoiding word is weakly auxiliary state avoiding. The property of weak auxiliary state avoidability is preserved w.r.t. codes when concatenating code words to the front, as proved in Theorem 18.53. Theorem 18.53 follows from Theorem 18.51. Theorem 18.57 collects equivalent conditions related to concatenating words to the front while preserving weak auxiliary state avoidability.

Proposition 18.47 *Let $L \subseteq \Sigma^*$. Then* $\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L)$ *is a prefix code if it is non-empty.*

Proof. This follows since every proper prefix of a word from $\mathrm{Pref}(L)\Sigma$ is contained in $\mathrm{Pref}(L)$ and therefore cannot belong to $\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L)$. □

Corollary 18.48 *Let $L \subseteq \Sigma^*$. If the language* $\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*)$ *(resp.* $(\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L)$*) is non-empty, then it is a prefix codes.*

Proof. For the first language the claim follows from Theorem 18.47 applied to the language L^*. For the second language the claim follows since it is a subset of $\mathrm{Pref}(L)\Sigma \setminus \mathrm{Pref}(L)$ and thus inherits the property of being a prefix code. □

By Theorem 18.48 a factorization of weakly (resp. strongly) auxiliary state avoiding words into words from $\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*)$ (resp. $(\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L)$) is unique.

Let $C \subseteq \Sigma^+$ be a prefix code. In this case, we have $C\Sigma \cap \mathrm{Pref}(C) = \emptyset$. Thus $(C\Sigma \setminus \mathrm{Pref}(C))\Sigma^* = C\Sigma^+$. Using this in Equation (18.11) then gives the following partition involving strongly state avoiding words:

$$\Sigma^* = \mathrm{Pref}(C) \uplus C\Sigma^+ \uplus ((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C))^+(\mathrm{Pref}(C) \uplus C\Sigma^+). \quad (18.12)$$

Next, we show that every strong auxiliary state avoiding word w.r.t. some prefix code is also weakly auxiliary state avoiding w.r.t. the same prefix code.

Proposition 18.49 *Suppose $C \subseteq \Sigma^*$ is a prefix code. Then every strongly auxiliary state avoiding word w.r.t. C is also weakly auxiliary state avoiding w.r.t. C.*

Proof. By Equation (17.13) we have $((\mathrm{Pref}(C) \setminus C)\Sigma) \setminus \mathrm{Pref}(C) = ((\mathrm{Pref}(C) \setminus C)\Sigma) \setminus \mathrm{Pref}(C^*)$. As $\mathrm{Pref}(C) \setminus C)\Sigma \subseteq \mathrm{Pref}(C^*)\Sigma$ we find that $(\mathrm{Pref}(C) \setminus C)\Sigma) \setminus \mathrm{Pref}(C))$ is contained in $\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$, which gives the claim. □

The following example shows that Theorem 18.49 does not hold for arbitrary codes.

Example 18.50 Consider the code $C = \{a, aab, c\}$. Then $aac \in (\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C)$ but $aac \in C^*$. By Equation (18.10) we have $aac \notin (\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*))^+$

and so aac is not weakly auxiliary state avoiding w.r.t. C. For another example (that is not a code), consider $L = \{a, b, aa\}$. Then $ab \in (\mathrm{Pref}(L) \setminus L)\Sigma \setminus \mathrm{Pref}(L)$ but $ab \in \mathrm{Pref}(L^*)$.

Theorem 18.86 involved weakly auxiliary state avoiding words of the form wu for $w \in \Sigma^*$ and a weakly auxiliary state avoiding $u \in \Sigma^*$. Next, we investigate conditions when weak auxiliary state avoidability is preserved by concatenating words to the front.

Proposition 18.51 *Let $C \subseteq \Sigma^+$ be a prefix code. Then the following holds:*

$$\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*) = C^*((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C))$$

Proof. By Equation (17.13) we have $(\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C) = (\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C^*)$. Since $\mathrm{Pref}(C) \setminus C \subseteq \mathrm{Pref}(C^*)$, we have

$$(\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C^*) \subseteq \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*).$$

Thus $C^*((\mathrm{Pref}(C)\setminus C)\Sigma\setminus\mathrm{Pref}(C^*)) \subseteq C^*(\mathrm{Pref}(C^*)\Sigma\setminus\mathrm{Pref}(C^*))$. Since $C^*\,\mathrm{Pref}(C^*)\Sigma \subseteq \mathrm{Pref}(C^*)\Sigma$, by Equation (17.12) we have $C^*(\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)) = \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$. The other inclusion follows from Theorem 18.39. □

Corollary 18.52 *Let $C \subseteq \Sigma^+$ be a prefix code. Then we have*

$$(\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C) \subseteq \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$$
$$\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*) = C^*(\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*))$$

Proof. The first equation clearly follows from Theorem 18.51. We have $C^*C^*((\mathrm{Pref}(C)\setminus C)\Sigma \setminus \mathrm{Pref}(C)) = C^*((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C))$, which gives the second equation with Theorem 18.51. □

Corollary 18.53 *Let $C \subseteq \Sigma^+$ be a prefix code and u weakly or strongly state avoiding w.r.t. C. Then vu is weakly state avoiding w.r.t. C for each $v \in C^*$.*

Proof. We can write $u = u_1 \cdots u_n$ where, for $i \in \{1, \ldots, n\}$, we have $u_i \in \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$ if u is weakly auxiliary state avoiding and $u_i \in (\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C)$ if u is strongly auxiliary state avoiding. By Theorems 18.51 and 18.52 we have

$$wu_1 \in \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$$

and

$$u_i \in \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$$

for $i \in \{2, \ldots, n\}$ for both cases that u is strongly or weakly auxiliary state avoiding. Thus $wu_1 \cdots u_n$ is weakly auxiliary state avoiding. □

Example 18.54 Theorem 18.51 is not valid for arbitrary languages. For example consider $L = \{aa, bc, aabd\}$. Then bd is weakly auxiliary state avoiding w.r.t. L, but $aabd \in L$ is not weakly auxiliary state avoiding w.r.t. L.

Corollary 18.55 *Let $C \subseteq \Sigma^+$ be a prefix code. Then* $(C \cup \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*))^* = (\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*))^* C^*$.

Proof. By Theorem 3.61, Item 3 we have $(C \cup \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*))^* = (C^*(\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)))^* C^*$. By Equation (17.12) the last language equals $(\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*))^* C^*$. □

The following examples show that concatenating a word from the set of prefixes to a weakly auxiliary state avoiding word does not, in general, yield a weakly auxiliary state avoiding word even w.r.t. prefix codes.

Example 18.56
1. Consider the prefix code $C = \{aa, bac\}$. Then ac is weakly (and strongly) auxiliary state avoiding w.r.t. C, but $bac \in C$ is not weakly auxiliary state avoiding w.r.t. C.
2. Consider the prefix code $C = aaa^*b$. Here, $\mathrm{Pref}(C^*)$ is a submonoid of Σ^* and ab is strongly and weakly auxiliary state avoiding w.r.t. C. However, for $a \in \mathrm{Pref}(C^*)$ we have $aab \in C$ and so aab is neither weakly nor strongly auxiliary state avoiding w.r.t. C.

Next, we show that the property of preservation of weak auxiliary state avoidability (for a fixed word) is equivalent for certain types of prefixes.

Proposition 18.57 *Let $C \subseteq \Sigma^*$ be a prefix code, $u \in \Sigma^*$ be weakly auxiliary state avoiding w.r.t. C, and $n \geq 1$ be some fixed number. Then the following are equivalent.*

1. *vu is weakly auxiliary state avoiding w.r.t. C for each $v \in \mathrm{Pref}(C) \setminus C$.*
2. *vu is weakly auxiliary state avoiding w.r.t. C for each $v \in \mathrm{Pref}(C^*)$.*
3. *vu is weakly auxiliary state avoiding w.r.t. C for each $v \in \mathrm{Pref}(C^n)$.*

Proof. $1 \Rightarrow 2$: Set $L = \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$ so that L^+ is the set of weakly auxiliary state avoiding words w.r.t. C. By Equation (3.46) we have $\mathrm{Pref}(C^*) = C^*(\mathrm{Pref}(C) \setminus C)$. By Item 1 we have $(\mathrm{Pref}(C) \setminus C)L^+ \subseteq L^+$. By Theorem 18.53 we have $C^*L^+ \subseteq L^+$. Thus we obtain $\mathrm{Pref}(C^*)L^+ \subseteq L^+$, which gives the claim.

$2 \Rightarrow 3$: This is clear since $\mathrm{Pref}(C^n) \subseteq \mathrm{Pref}(C^*)$.

$3 \Rightarrow 1$: This is clear since $\mathrm{Pref}(C) \setminus C \subseteq \mathrm{Pref}(C^n)$. □

18.3.3 Maximal Prefix Codes

We call $C \subseteq \Sigma^+$ *maximal* if $C \cup \{u\}$ is not a prefix code for each $u \in \Sigma^* \setminus C$.

Auxiliary state avoiding words are related to maximal codes. Here, we demonstrate this by deriving characterizations of maximal codes from the results of Sections 18.3.1 and 18.3.2.

Proposition 18.58 *Let $C \subseteq \Sigma^+$ be a prefix code. Then the following are equivalent:*

1. *C is maximal.*
2. *$\Sigma^* = \mathrm{Pref}(C) \uplus C\Sigma^+$.*

3. $(\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C) = \emptyset$.
4. $\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*) = \emptyset$.
5. $\mathrm{Pref}(C^*) = \Sigma^*$.

Proof. $1 \Rightarrow 2$: Let $u \in \Sigma^*$. For $u \in C$, we have $u \in \mathrm{Pref}(C)$. For $u \notin C$, since C is maximal, $C \cup \{u\}$ is not a prefix code. This yields that some word in C has u as a prefix, i.e., $u \in \mathrm{Pref}(C) \setminus C$, or that u has some word from C as a prefix, i.e., $u \in C\Sigma^+$. This yields $\Sigma^* = \mathrm{Pref}(C) \uplus C\Sigma^+$.

$2 \Rightarrow 3$: Since $\Sigma^* = \mathrm{Pref}(C) \uplus C\Sigma^+$ and the union from Equation (18.12) is disjoint, we must have $((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C))^+(\mathrm{Pref}(C) \uplus C\Sigma^+) = \emptyset$, which gives $(\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C) = \emptyset$.

$3 \Rightarrow 4$: By Theorem 18.39 every word in $\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$ has a suffix in $(\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C)$. However, this latter set is empty by assumption. Thus, there can be no word in $\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$, i.e., this set is empty as well.

$4 \Rightarrow 5$: This follows from the disjoint union given in Equation (18.10) since the set $(\mathrm{Pref}(L^*)\Sigma \setminus \mathrm{Pref}(L^*))^+ \mathrm{Pref}(L^*)$ is empty by assumption.

$5 \Rightarrow 1$: Suppose C is not maximal. Then $C \cup \{u\}$ is a prefix code for some $u \in \Sigma^*$ with $u \notin C$. Since u has no prefix in C it is not contained in $C\,\mathrm{Pref}(C^*)$ and since no word from C has it as a prefix, it is not contained in $\mathrm{Pref}(C)$. We have $\mathrm{Pref}(C^*) = \mathrm{Pref}(C) \cup C\,\mathrm{Pref}(C^*)$ and thus $u \notin \mathrm{Pref}(C^*)$. □

Remark 18.59 Let $C \subseteq \Sigma^+$ be a prefix code. The set $((\mathrm{Pref}(C) \setminus C)\Sigma) \setminus \mathrm{Pref}(C)$ is precisely the set of minimal words that are not prefixes of words from C.

Proposition 18.60 *Let $C \subseteq \Sigma^+$ be a prefix code. Then $D = C \cup ((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C))$ is a maximal prefix code.*

Proof. Since C and $(\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C)$ are prefix codes and no word from C can be a prefix of a word from $(\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C)$ and vice versa, we find that D is a prefix code.

Next, by Equation (18.12), we have

$$\begin{aligned}\Sigma^* &\subseteq \mathrm{Pref}(C) \cup C\Sigma^+ \cup ((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C))\Sigma^* \\ &= \mathrm{Pref}(C) \cup ((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C)) \cup C\Sigma^+ \cup ((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C))\Sigma^+ \\ &\subseteq \mathrm{Pref}(D) \cup D\Sigma^+.\end{aligned}$$

Thus $\Sigma^* = \mathrm{Pref}(D) \cup D\Sigma^+$. Since D is a prefix code, this union is disjoint. So D is maximal by Item 2 of Theorem 18.58. □

Example 18.61 Let $C = a^i a^* b$ for some $i \geq 0$. Then $\mathrm{Pref}(C) \setminus C = a^*$, $(\mathrm{Pref}(C) \setminus C)\{a, b\} = \{b, ab, aab, \ldots, a^{i-1}b\} \cup C \cup aa^*$, and $(\mathrm{Pref}(C) \setminus C)\{a, b\} \setminus \mathrm{Pref}(C) = \{b, ab, aab, \ldots, a^{i-1}b\}$.

Example 18.62 Let $C = \{ab, aab, aaab\}$. Then $\mathrm{Pref}(C) \setminus C = \{\varepsilon, a, aa, aaa\}$, $(\mathrm{Pref}(C) \setminus C)\{a, b\} = \{a, b, aa, ab, aaa, aab, aaaa, aaab\}$, and $(\mathrm{Pref}(C) \setminus C)\{a, b\} \setminus \mathrm{Pref}(C) = \{b, aaaa\}$.

Remark 18.63 Note that, in general, there might be several possibilities to obtain a maximal prefix code by extending a given prefix code C. For example, for $C = \{b, ab, aab\}$, $D_i = \{aaa^j b \mid 1 \leq j \leq i-1\} \cup \{a^{i+2}\}$ for $i \geq 1$, and $D = \{aaab, aaaab, aaaaab, \ldots\}$ the sets $C \cup D_i$ for $i \geq 1$ and $C \cup D$ all give maximal prefix codes. Note that here $D_1 = (\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C)$.

18.3.4 Relations Derived From the Myhill-Nerode Relation

Here, we introduce relations derived from the Myhill-Nerode relation $\equiv_L$, which was introduced in Section 6.7.2. We first abstractly describe a method for constructing a new relation that refines the Myhill-Nerode relation from a given relation $\sim$ when certain assumptions are satisfied. We then derive various properties of this new relation. Via this abstract construction we obtain a relation $\equiv_{C,V}$, which will be central to most of our hardness proofs given in the remaining sections since it allows to distinguish, or "keep track", of certain words in automata that we construct in our reductions.

Let $L \subseteq \Sigma^*$. Suppose we have an equivalence relation $\sim$ on Σ^* and a language $U \subseteq \Sigma^*$ such that, for all $u, v \in \Sigma^*$ and $x \in \Sigma$, the following properties hold:

1. If $u \sim v$, then $u \equiv_L v$, i.e., $\sim$ refines $\equiv_L$.
2. The relation $\sim$ saturates U.
3. If $u \equiv_L v$ and $u, v \in U$, then $ux, vx \in U$ or $ux \sim vx$.
4. If $u \sim v$, then $ux \sim vx$ or $ux, vx \in U$.

Then for $u, v \in \Sigma^*$ define a relation $\equiv_L^{\sim,U}$ by

$$u \equiv_L^{\sim,U} v \Leftrightarrow (u \sim v) \vee (u \equiv_L v \wedge u, v \in U).$$

Before establishing some properties of this relation, we present two examples to build intuition.

Example 18.64
1. Properties 1, 2,3, and 4 are obviously fulfilled when $\sim$ equals the equality relation and U is a right ideal.
2. For $U = \Sigma^*$, the above construction always yields the Myhill-Nerode relation $\equiv_L$, regardless of which relation $\sim$ (that refines $\equiv_L$) we choose.

Remark 18.65 Since $\sim$ refines $\equiv_L$, for $u, v \in \Sigma^*$, we have

$$u \equiv_L^{\sim,U} v \Leftrightarrow (u \sim v \wedge |\{u, v\} \cap U| \leq 1) \vee (u \equiv_L v \wedge u, v \in U).$$

In the special case that $\sim$ is the equality relation, we also have

$$u \equiv_L^{\sim,U} v \Leftrightarrow (u = v \wedge u, v \notin U) \vee (u \equiv_L v \wedge u, v \in U).$$

The relation $\equiv_L^{\sim,U}$ is an equivalence relation that is invariant under right concatenation as stated next.

Theorem 18.66 *The relation $\equiv_L^{\sim,U}$ is an equivalence relation such that, for all $u, v \in \Sigma^*$, if $u \equiv_L^{\sim,U} v$, then $ux \equiv_L^{\sim,U} vx$ for all $x \in \Sigma$.*

Proof. Symmetry and reflexivity are clear. For transitivity, consider $u, v, w \in \Sigma^*$ such that $u \equiv_L^{\sim,U} v$ and $v \equiv_L^{\sim,U} w$. If $u \sim v$ and $v \sim w$, then $u \sim w$ and so $u \equiv_L^{\sim,U} w$. For $u \equiv_L v$ and $v \equiv_L w$ with $u, v, w \in U$ we have $u \equiv_L w$ since $\equiv_L$ is transitive and so $u \equiv_L^{\sim,U} w$. Now, suppose $u \sim v$ and $v \equiv_L w$ with $v, w \in U$. By Property 1, $u \sim v$ implies $u \equiv_L v$. Since $\sim$ saturates U by Property 2, we also have $u \in U$. Thus we find $u \equiv_L w$ and $u, w \in U$, i.e., $u \equiv_L^{\sim,U} w$. Next, suppose $u \equiv_L v$ with $u, v \in U$ and $v \sim w$. By Property 1 we have $v \equiv_L w$. Since $\sim$ saturates U by Property 2, we also have $w \in U$. Thus we find $u \equiv_L w$ and $u, w \in U$, i.e., $u \equiv_L^{\sim,U} w$.

Next, suppose $u \equiv_L^{\sim,U} v$ for $u, v \in \Sigma^*$ and let $x \in \Sigma$. We show that then we have $ux \equiv_L^{\sim,U} vx$. By combining Property 1 with Equation (6.24) and using Property 4, we get

$$u \sim v \Rightarrow (ux \sim vx) \vee (ux \equiv_L vx \wedge ux, vx \in U)$$

Thus, if $u \sim v$, we have $ux \equiv_L^{\sim,U} vx$. Next, let $u \equiv_L v$ with $u, v \in U$. If $ux \sim vx$, then we have $ux \equiv_L^{\sim,U} vx$ by the definition of $\equiv_L^{\sim,U}$. Otherwise, by Property 3, we have $ux, vx \in U$. Since, by Equation (6.24), from $u \equiv_L v$ we get $ux \equiv_L vx$, we also have $ux \equiv_L^{\sim,U} vx$. This shows the claim. □

Proposition 18.67 *For $u \in \Sigma^*$ we have*

$$[u]_{\equiv_L^{\sim,U}} = \begin{cases} [u]_\sim & \text{if } u \notin U, \\ [u]_\sim \cup ([u]_{\equiv_L} \cap U) & \text{if } u \in U. \end{cases}$$

Proof. Let $v \in [u]_{\equiv_L^{\sim,U}}$. First, suppose $u \notin U$. Then, by definition, we must have $u \sim v$, i.e., $v \in [u]_\sim$. Next, suppose $u \in U$ and $v \notin [u]_\sim$. In this case, by definition, we have $u \equiv_L v$ and $v \in U$, i.e., $v \in [u]_{\equiv_L} \cap U$.

Conversely, by definition, we always have $[u]_\sim \subseteq [u]_{\equiv_L^{\sim,U}}$. If also $u \in U$ and $v \in [u]_{\equiv_L} \cap U$, then $u \equiv_L v$ and $u, v \in U$ so that $v \in [u]_{\equiv_L^{\sim,U}}$. □

Proposition 18.68 *Let $V \subseteq \Sigma^*$ be such that $U \subseteq V$ and $\approx$ be an equivalence relation that is coarser than $\sim$ and that fulfills Properties 1, 2,3, and 4 w.r.t. L and V. Then for $u, v \in \Sigma^*$ we have that $u \equiv_L^{\sim,U} v$ implies $u \equiv_L^{\approx,V} v$, i.e., $\equiv_L^{\sim,U}$ refines $\equiv_L^{\approx,V}$.*

Proof. Suppose $u \equiv_L^{\sim,U} v$ for $u, v \in \Sigma^*$. Then we have $u \sim v$ or $u \equiv_L v$ with $u, v \in U$. In the former case, since $u \sim v$ implies $u \approx v$, we also have $u \equiv_L^{\approx,V} v$. In the latter case, since $U \subseteq V$, we find $u, v \in V$ and with $u \equiv_L v$ we also have $u \equiv_L^{\approx,V} v$. □

Corollary 18.69 *The relation $\equiv_L^{\sim,U}$ refines $\equiv_L$.*

Proof. The equivalence relation $\equiv_L$ and the set Σ^* fulfill all Properties 1, 2,3, and 4. Now $\equiv_L^{\equiv_L,\Sigma^*}$ equals $\equiv_L$ and so the claim follows from Theorem 18.68. □

Corollary 18.70 *The relation $\equiv_L^{\sim,U}$ saturates L and* $\mathrm{Pref}(L)$.

Proof. This follows from Theorem 18.69 since $\equiv_L$ saturates L and $\operatorname{Pref}(L)$. That $\equiv_L$ saturates $\operatorname{Pref}(L)$ follows since if $u \equiv_L v$ and $u \in \operatorname{Pref}(L)$, then $uw \in L$ for some $w \in \Sigma^*$. Now we have $uw \equiv_L vw$. Thus, we also get $vw \in L$ and so $v \in \operatorname{Pref}(L)$. □

Let $n \geq 0$. For $L \subseteq \Sigma^*$ and $U = \Sigma^n\Sigma^*$ we now consider the equivalence relation $\equiv_L^{=,U}$, which we also denote by $\equiv_L^{\geq n}$ in the following. With Theorem 18.65, for $u, v \in \Sigma^*$, we have

$$\begin{aligned} u \equiv_L^{\geq n} v &\Leftrightarrow (u = v) \vee (\min\{|u|, |v|\} \geq n \wedge u \equiv_L v) \\ &\Leftrightarrow (\max\{|u|, |v|\} < n \wedge u = v) \vee (\min\{|u|, |v|\} \geq n \wedge u \equiv_L v) \\ &\Leftrightarrow (\min\{|u|, |v|\} < n \wedge u = v) \vee (\min\{|u|, |v|\} \geq n \wedge u \equiv_L v) \end{aligned}$$

Clearly, for $m \geq n$ the relation $\equiv_L^{\geq m}$ refines $\equiv_L^{\geq n}$ and $\equiv_L^{\geq 0}$ equals $\equiv_L$.

Proposition 18.71 *Let $u \in \Sigma^*$. Then we have*

$$[u]_{\equiv_L^{\geq n}} = \begin{cases} \{u\} & \text{if } |u| < n, \\ \{v \in \Sigma^n\Sigma^* \mid v \equiv_L u\} & \text{otherwise.} \end{cases}$$

Thus, we have at least $|\Sigma^{<n}| = (|\Sigma|^n - 1)/(|\Sigma| - 1)$ many equivalence classes.

Proof. This follows directly from Theorem 18.67 with $U = \Sigma^n\Sigma^*$ and the fact that $\sim$ equals the equality relation in the definition of $\equiv_L^{\geq n}$. □

Corollary 18.72 *There are finitely many equivalence classes for $\equiv_L^{\geq n}$ if and only if L is regular.*

Proof. First, suppose L is regular. Then, by Theorem 6.86, we have finitely many equivalence classes for $\equiv_L$. Thus there are only finitely many distinct sets of the form $\{v \in \Sigma^n\Sigma^* \mid v \equiv_L u\}$ for $u \in \Sigma^*$ with $|u| \geq n$. Since there are only finitely many words of length at most n, with Theorem 18.71 we find that there are only finitely many equivalence classes for $\equiv_L^{\geq n}$.

Conversely, since $\equiv_L^{\geq n}$ refines $\equiv_L$, the $\equiv_L$-classes are unions of $\equiv_L^{\geq n}$-classes. Thus, if there are only finitely many $\equiv_L^{\geq n}$-classes, then there are only finitely many $\equiv_L$-classes and so L is regular by Theorem 6.86. □

Next, let $C \subseteq \Sigma^+$ be a prefix code and $V \subseteq C$. Consider the relation $\sim_{C,V}$ on Σ^* which is, for $u, v \in \Sigma^*$, given by

$$u \sim_{C,V} v \Leftrightarrow (u = v) \vee (\exists w \in \operatorname{Pref}(V) \setminus V \colon u, v \in C^*w).$$

Proposition 18.73 *The relation $\sim_{C,V}$ given above is an equivalence relation.*

Proof. Clearly, the relation is reflexive and symmetric. It is also transitive, for assume we have $u_1, u_2, u_3 \in \Sigma^*$ with $u_1 \sim_{C,V} u_2$ and $u_2 \sim_{C,V} u_3$, where we can suppose all three words are distinct. Then we find $u_1, u_2 \in C^*w_1$ and $u_2, u_3 \in C^*w_2$ for some $w_1, w_2 \in \operatorname{Pref}(V) \setminus V$. This gives $u_2 \in C^*w_1 \cap C^*w_2$. We have $w_1, w_2 \notin C\Sigma^*$ as otherwise some word from V would have a word from C as a proper prefix, which is not possible

since C is a prefix code. As C is a prefix code, by Theorems 17.23 and 17.24 we find $w_1 = w_1$. Thus $u_1, u_3 \in C^*w_1$, giving $u_1 \sim_{C,V} u_3$. □

Remark 18.74 Transitivity might fail if C is not a prefix code. For example, consider $C = \{aa, ba, baa\}$, which is only a code, but not a prefix code. Then $\mathrm{Pref}(C) \setminus C = \{\varepsilon, a, b\}$. We have $aaa, baa \in C^*a$ and $baa, ba \in C^*$, but there exists no $w \in \{\varepsilon, a, b\}$ such that C^*w contains aaa and ba. Thus, the relation defined above would not yield a transitive relation. However, there exist codes that are not prefix codes such that $u_1, u_2 \in C^*w_1$ and $u_2, u_3 \in C^*w_2$ for some $w_1, w_2 \in (\mathrm{Pref}(C) \setminus C)$ would imply $u_1, u_3 \in C^*w_3$ for some $w_3 \in \mathrm{Pref}(C) \setminus C$, and so they would yield transitivity. For example, this holds for the code $C = \{a, ab\}$, because $\mathrm{Pref}(C) \setminus C = \{\varepsilon\}$. However, we only need this construction in the context of prefix codes.

Proposition 18.75 *Let $u, v \in \mathrm{Pref}(V) \setminus V$. Then $u \sim_{C,V} v$ implies $u = v$.*

Proof. By the definition of $\sim_{C,V}$, from $u \sim_{C,V} v$ we get $u, v \in C^*w$ for some $w \in \mathrm{Pref}(V) \setminus V$. Since C is a prefix code, which gives $u, v \notin C\Sigma^*$, by Theorems 17.23 and 17.24, from $u \in C^*u \cap C^*w$ and $v \in C^*v \cap C^*w$ we get $u = w = v$. □

Proposition 18.76 *Let $u \in \Sigma^*$. Then we have*

$$[u]_{\sim_{C,V}} = \begin{cases} C^*w & \text{if } u \in C^*w \text{ with } w \in \mathrm{Pref}(V) \setminus V, \\ \{u\} & \text{otherwise.} \end{cases}$$

Proof. First, suppose $u \notin C^*(\mathrm{Pref}(V) \setminus V)$. By reflexivity, we have $\{u\} \subseteq [u]_{\sim_{C,V}}$. Next, let $v \in [u]_{\sim_{C,V}}$. By the definition of $\sim_{C,V}$ we have $u = v$. This gives $[u]_{\sim_{C,V}} = \{u\}$. Now, let $u \in C^*w$ for some $w \in \mathrm{Pref}(V) \setminus V$. In this case, by the definition of $\sim_{C,V}$, we have $C^*w \subseteq [u]_{\sim_{C,V}}$. Next, let $v \in [u]_{\sim_{C,V}}$. We can assume $u \neq v$. Then $u \sim_{C,V} v$ implies $u, v \in C^*w'$ for some $w' \in \mathrm{Pref}(V) \setminus V$. Since $u \in C^*w \cap C^*w'$ and $w' \notin C\Sigma^*$, by Theorems 17.23 and 17.24, we get $w = w'$ and so $v \in C^*w$. This gives $[u]_{\sim_{C,V}} \subseteq C^*w$. □

Proposition 18.77 *The relation $\sim_{C,V}$ fulfills Properties 1, 2, 3, and 4 for $L = C^*$ and $U = \Sigma^* \setminus (C^*(\mathrm{Pref}(V) \setminus V))$.*

Proof. Observe that $u \equiv_{C^*} \varepsilon$ if and only if $u \in C^*$ (this in fact holds for the Kleene star of any language). We will use this equivalence in the following arguments.

Property 1: Suppose $u \sim_{C,V} v$ for $u, v \in \Sigma^*$. For $u = v$ we have $u \equiv_L v$. So, now let $u \neq v$. Then $u = u'w$ and $v = v'w$ for some $u', v' \in C^*$ and $w \in \mathrm{Pref}(V) \setminus V$. We have $u' \equiv_{C^*} \varepsilon$ and $v' \equiv_{C^*} \varepsilon$. With Equation (6.24) this yields $u \equiv_{C^*} w \equiv_{C^*} v$.

Property 2: By Theorem 18.76, for $u \in U$, we have $[u]_{\sim_{C,V}} = \{u\}$. Thus $\sim_{C,V}$ saturates U.

Property 3: Suppose $u \equiv_{C^*} v$ for $u, v \in U$ and let $x \in \Sigma$. We have $ux, vx \in U$ or, without loss of generality, $ux \notin U$. Then $ux \in C^*w$ for some $w \in \mathrm{Pref}(V) \setminus V$. Assume $w \neq \varepsilon$. Then we can write $w = w'x$ for some $w' \in \mathrm{Pref}(V) \setminus V$. This gives $u \in C^*w'$, which is not possible since $u \in U$. Thus, we must have $w = \varepsilon$ and so $ux \in C^*$. By Equation (6.24), we find $ux \equiv_{C^*} vx$. This then gives $vx \in C^*$ and so $ux \sim_{C,V} vx$.

Property 4: Suppose $u \sim_{C,V} v$ for $u, v \in \Sigma^*$ and let $x \in \Sigma$. We can assume $u \neq v$. Then we have $u, v \in C^*w$ for some $w \in \mathrm{Pref}(V) \setminus V$. The claim holds for $ux, vx \in U$. Next, without loss of generality, let $ux \notin U$. Then $ux \in C^*w'$ for some $w' \in \mathrm{Pref}(V) \setminus V$. Suppose $w' = \varepsilon$. By Property 1 we have $u \equiv_{C^*} v$. This gives $ux \equiv_{C^*} vx$. With $ux \in C^*$ we have $ux \equiv_{C^*} \varepsilon$ and so $vx \equiv_{C^*} \varepsilon$ and thus $vx \in C^*$. This then gives $ux \sim_{C,V} vx$. Next, suppose $w' \neq \varepsilon$. We have $ux \in C^*w' \cap C^*wx$. Since C is a prefix code, we have $w', wx \notin C\Sigma^+$ (note that the possibility $wx \in C$ has not been excluded yet). By Theorems 17.23 and 17.24 and as $|w'|, |wx| > 0$ we then get $w' = wx$. This gives $ux, vx \in C^*w'$ and so $ux \sim_{C,V} vx$. □

Thus, by Theorem 18.77, we can define an equivalence relation in the following way.

Definition 18.78 Let $C \subseteq \Sigma^+$ be a prefix code and $V \subseteq C$. The relation $\equiv_{C,V}$ is defined to be $\equiv_{C^*}^{\sim_{C,V},U}$ with $U = \Sigma^* \setminus (C^*(\mathrm{Pref}(V) \setminus V))$.

This relation fulfills Theorems 18.66 to 18.68. Unraveling the definition, for $u, v \in \Sigma^*$ we have the following:

$$\begin{aligned} u \equiv_{C,V} v &\Leftrightarrow (u = v) \vee (\exists w \in \mathrm{Pref}(V) \setminus V \colon u, v \in C^*w) \vee (u \equiv_{C^*} v \wedge u, v \notin C^*(\mathrm{Pref}(V) \setminus V)) \\ &\Leftrightarrow (\exists w \in \mathrm{Pref}(V) \setminus V \colon u, v \in C^*w) \vee (u \equiv_{C^*} v \wedge u, v \notin C^*(\mathrm{Pref}(V) \setminus V)) \end{aligned}$$

where $u = v$ can be omitted since each word is either in $C^*(\mathrm{Pref}(V) \setminus V)$ or its complement. Then with Theorems 18.67 and 18.76 we have

$$u \equiv_{C,V} v \Leftrightarrow \begin{cases} v \in C^*w & \text{if } u \in C^*w \text{ with } w \in \mathrm{Pref}(V) \setminus V, \\ v \in [u]_{\equiv_{C^*}} \setminus (C^*(\mathrm{Pref}(V) \setminus V)) & \text{if } u \notin C^*(\mathrm{Pref}(V) \setminus V). \end{cases} \tag{18.13}$$

With Theorem 18.75 we also obtain that for $u, v \in \mathrm{Pref}(V) \setminus V$ we have

$$u \equiv_{C,V} v \Leftrightarrow u = v. \tag{18.14}$$

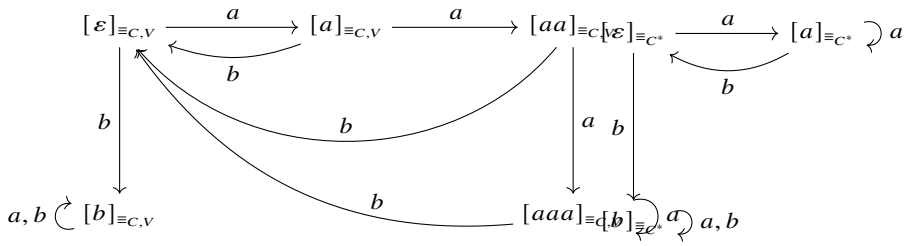
Example 18.79 For $C = aa^*b$ and $V = \{ab, aab\}$ we have the distinct equivalence classes $[\varepsilon]_{\equiv_{C,V}} = C^*$, $[a]_{\equiv_{C,V}} = C^*a$, $[aa]_{\equiv_{C,V}} = C^*aa$, $[aaa]_{\equiv_{C,V}} = C^*aaaa^*$, and $[b]_{\equiv_{C,V}} = \Sigma^* \setminus \mathrm{Pref}(C^*)$. In fact, we can construct an automaton $\mathcal{A}_{\equiv_{C,V}} = (\Sigma^*/{\equiv_{C,V}}, \Sigma, \delta_{\equiv_{C,V}}, [\varepsilon]_{\equiv_{C,V}}, \{[\varepsilon]_{\equiv_{C,V}}\})$ with transition function given by $\delta_{\equiv_{C,V}}([w]_{\equiv_{C,V}}, x) = [wx]_{\equiv_{C,V}}$ for $w \in \Sigma^*$ and $x \in \Sigma$, which is well-defined by Theorem 18.66. This automaton accepts C^* since $[\varepsilon]_{\equiv_{C,V}} = C^*$. See Figure 18.4 for a depiction of $\mathcal{A}_{\equiv_{C,V}}$ with the minimal automaton $\mathrm{MinAut}(C^*)$ for comparison.

Example 18.80 Consider $C = \{aba, bba\}$ and $V = \{aba\}$. Then we have $ab \not\equiv_{C,V} bb$, but $ab \equiv_{C^*} bb$.

Remark 18.81 Setting two words $u, v \in \Sigma^*$ in relation precisely if

$$(\exists w \in \mathrm{Pref}(V) \setminus V \colon u, v \in C^*w) \vee (u \equiv_{C^*} v \wedge \forall w \in \mathrm{Pref}(V) \setminus V \colon |\{u, v\} \cap C^*w| \leq 1)$$

For the code and language V from Theorem 18.79 this relation would give the same classes. However, for example, for $C = \{ac, bc\}$ and $V = \{ac\}$ we have that a and b are in relation whereas $a \not\equiv_{C,V} b$.



(a) The automaton $\mathcal{A}_{\equiv_{C,V}}$ (b) The automaton $\mathrm{MinAut}(C^*)$

Fig. 18.4: Figure 18.4a shows the automaton $\mathcal{A}_{\equiv_{C,V}}$ from Theorem 18.79 for $C = aa^*b$ and $V = \{ab, aab\}$; Figure 18.4b shows $\mathrm{MinAut}(C^*)$.

In our constructions we need this relation to have finitely many classes, a property characterized in the next statement.

Proposition 18.82 *There are finitely many equivalence classes for $\equiv_{C,V}$ if and only if C^* is regular and V is finite.*

Proof. First, suppose C^* is regular and V is finite. By Theorem 6.86 there are only finitely many equivalence classes for $\equiv_{C^*}$. By Equation (18.13) each equivalence class has the form C^*w for some $w \in \mathrm{Pref}(V) \setminus V$ or $[u]_{\equiv_{C^*}} \cap U$. Since $\mathrm{Pref}(V) \setminus V$ is finite and there are only finitely many $\equiv_{C^*}$-equivalence classes, we obtain that there are only finitely many $\equiv_{C,V}$-equivalence classes.

Conversely, suppose there are finitely many equivalence classes for $\equiv_{C,V}$. By Theorem 18.69, $\equiv_{C,V}$ refines $\equiv_{C^*}$. Thus the $\equiv_{C^*}$-equivalence classes are unions of $\equiv_{C,V}$-equivalence classes and we find that there are only finitely many $\equiv_{C^*}$-equivalence classes. So C^* is regular by Theorem 6.86. By Theorems 18.67, 18.75, and 18.76, distinct words in $\mathrm{Pref}(V) \setminus V$ belong to distinct $\equiv_{C,V}$-equivalence classes. Thus $\mathrm{Pref}(V) \setminus V$ is finite. Since V is a prefix code, this set equals the set of proper prefixes of words from V. Now, finiteness of its set of proper prefixes yields that V is finite. □

18.3.5 An NL-Hardness Condition

Theorem 18.83 *Let $C \subseteq \Sigma^+$ be a prefix code that is a regular language and $u_1, u_2 \in C$ be distinct. Suppose there exists $u \in C^*$ such that*

$$vu \in ((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C) \cup C)^*$$

for all $v \in \mathrm{Pref}(C) \setminus C$. Then Sync$[C^*]$ *is* NL*-hard.*

Proof. We give a reduction from Sync for automata over a binary alphabet, which is NL-hard by Theorem 8.13. Let $\mathcal{A} = (Q, \{a_1, a_2\}, \delta)$ be an input automaton over the binary alphabet $\{a_1, a_2\}$. Let $v_1, v_2 \in \Sigma^*$ such that $u_1 \in v_1\Sigma$ and $u_2 \in v_2\Sigma$. Set

$D = (\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C) \cup C$. For short, denote $[v]_{\equiv_D^{\geq\{u_1,u_2\}}}$ simply by $[v]$ for $v \in \Sigma^*$ in the following. Let $\mathcal{A}' = (Q', \Sigma, \delta)$ with $Q' = Q \times \{[v] \mid v \in \Sigma^*\}$ and where, for $q \in Q$, $v \in \Sigma^*$, and $x\Sigma$, we set

$$\delta'((q,[v]),x) = \begin{cases} (\delta(q,a_i),[vx]) & \text{if } [v] = [v_i] \text{ and } v_i x = u_i, \\ (q,[vx]) & \text{otherwise.} \end{cases}$$

Let $\varphi\colon D^* \to \{a_1,a_2\}^*$ be given by $\varphi(u_i) = a_i$ for $i \in \{1,2\}$ and $\varphi(v) = \varepsilon$ for $v \in D \setminus \{u_1,u_2\}$ with $\varphi(w_1 \dots w_m) = \varphi(w_1)\cdots\varphi(w_m)$, which is well-defined since D is a code. Inductively, with $[\varepsilon] = [w]$ if and only if $w \in D^*$, we obtain that for $w_1,\dots,w_m \in D$ we have

$$\delta'((q,[\varepsilon],w_1\cdots w_m) = (\delta(q,\varphi(w_1\cdots w_m)),[\varepsilon]). \tag{18.15}$$

Suppose for the word $w_1\cdots w_m$ with $w_1,\dots,w_m \in C$ we have $|\delta'(Q\times\{[\varepsilon]\},w_1\cdots w_m)| = 1$. By Equation (18.15) we find $\delta'(Q\times\{[\varepsilon]\},w_1\cdots w_m) = \delta(Q,\varphi(w_1\cdots w_m))\times\{[\varepsilon]\})\}$. Thus we have $|\delta(Q,\varphi(w_1\cdots w_m))| = 1$.

Conversely, suppose the word $x_1\cdots x_m$ with $x_1,\dots,x_m \in \{a_1,a_2\}$ synchronizes $\mathcal{A}$. For each $i \in \{1,\dots,m\}$ let $w_i = u_1$ if $x_i = a_1$ and $w_i = u_2$ if $x_i = a_2$. Then for $q \in Q$ we have

$$\delta'((q,[\varepsilon]),w_1\cdots w_m) = (\delta(q,x_1\cdots x_m),[\varepsilon]).$$

By assumption, for $v \in \mathrm{Pref}(C) \setminus C$, we have $vu \in D^*$ and thus

$$\delta'((q,[v]),u) = \delta'((q,[\varepsilon]),vu) \in Q \times \{[\varepsilon]\}.$$

Since $[\varepsilon] = [w]$ if and only if $w \in D^*$ and with $\mathrm{Pref}(C) \setminus C = \mathrm{Pref}(D) \setminus D$ this also gives $\{[v] \mid v \in \Sigma^*\} = \{[v] \mid v \in \mathrm{Pref}(C) \setminus C\}$, we find $\delta'(Q',vu) \subseteq Q \times \{[\varepsilon]\}$. Thus, we find $\delta'(Q',vuw_1\cdots w_m) = \delta(Q,x_1\cdots x_m)\times\{[\varepsilon]\}$, and this set is a singleton set. □

Example 18.84 Let Let $C \subseteq \Sigma^+$ be a finite prefix code with $a \in C$ for some $a \in \Sigma$. Set $n = \max\{i : a^i \in \mathrm{Fact}(L)\} + 1$. Then for $u = a^{n+1}$ we have

$$vu \in ((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C) \cup C)^*$$

for all $v \in \mathrm{Pref}(C) \setminus C$.

Corollary 18.85 *Let $C \subseteq \Sigma^+$ be a finite prefix code with $a \in C$ for some $a \in \Sigma$. Then* Sync$[C^*]$ *is* NL*-hard.*

Proof. The claim follows from Theorems 18.83 and 18.84. □

18.3.6 Two PSPACE-Hardness Conditions

Here, we present two PSPACE-hardness conditions. Theorem 18.86 is concerned with constraint languages of the form UuV, $U \subseteq u^*$ non-empty, and Theorem 18.89 with

constraint languages of the form C^*uL for some prefix code C, word u and $L \subseteq \Sigma^*$, where U, V, L, and C are regular languages. The proof of Theorem 18.89 uses Theorem 18.88 from the present section. In Theorem 18.93, utilizing Theorems 18.86 and 18.89 in its proof, we give a list of binary constraint languages giving PSPACE-hard problems which will be used later in Section 19.4.2.

Theorem 18.86 *Let $C \subseteq \Sigma^+$ be a regular language that is a prefix code with $u_1, u_2 \in C^+$ where $\{u_1, u_2\}$ is a code and $U, V \subseteq \Sigma^*$ be non-empty and regular such that*

$$\{u_1, u_2\}^* \subseteq V \subseteq \mathrm{Pref}(C^*)$$

and $U \subseteq u^*$ for some $u \in \Sigma^+$. Suppose vu is weakly auxiliary state avoiding w.r.t. C for each $v \in \mathrm{Pref}(C) \setminus C$. Then Sync$[UuV]$ *is* PSPACE*-hard.*

Proof. We give a reduction from the problem SyncFrom over a binary alphabet which is PSPACE-complete by Theorem 8.16. Let $(\mathcal{A}, S)$ with $\mathcal{A} = (Q, \{a_1, a_2\}, \delta)$ and $S \subseteq Q$ be an SyncFrom-instance. We can assume S is non-empty.

Set $n = r \cdot s \cdot |u|$ for $r, s \geq 1$ such that $u_1 \in C^r$ and $u_2 \in C^s$. Let $\overline{u}_1 = u_1^{s|u|}$ and $\overline{u}_2 = u_2^{r|u|}$. We have $\overline{u}_1, \overline{u}_2 \in C^n$ and since $\{u_1, u_2\}$ is a code, we have $\overline{u}_1 \neq \overline{u}_2$ by Theorem 17.20. For readability, in this proof, for $v \in \Sigma^*$, we denote $[v]_{\equiv_{C^n, \{\overline{u}_1, \overline{u}_2\}}}$ simply by $[v]$ (confusion with the notation for transition matrices should not arise as these matrices are not used in the present proof). Since C is regular, by Theorem 18.82, $\{[v] \mid v \in \Sigma^*\}$ is finite. Note that $\mathrm{Pref}(C^*) = \mathrm{Pref}((C^n)^*)$, and that $[v] = [\varepsilon]$ if and only if $v \in (C^n)^*$ and so $\{[v] \mid v \in \mathrm{Pref}(C^*)\} = \{[v] \mid v \in \mathrm{Pref}(C^n)\}$.

Construction: Pick some $s \in S$ and let $\overline{S} \colon Q \to S$ be given by $\overline{S}(q) = s$ for $q \in Q \setminus S$ and $\overline{S}(q) = q$ for $q \in S$. Then set $\mathcal{A}' = (Q', \Sigma, \delta)$ where $Q' = Q \times \{[v] \mid v \in \mathrm{Pref}(C^*)\}$ and, for $v \in \mathrm{Pref}(C^*)$ and $x \in \Sigma$, we set

$$\delta'((q, [v]), x) = \begin{cases} (\delta(q, a_i), [\varepsilon]) & \text{if } vx \in (C^n)^*\overline{u}_i \text{ for some } i \in \{1, 2\}, \\ (q, [vx]) & \text{if } vx \in \mathrm{Pref}(C^*) \text{ and } vx \notin (C^n)^*\overline{u}_1 \cup (C^n)^*\overline{u}_2, \\ (\overline{S}(q), [\varepsilon]) & \text{otherwise.} \end{cases}$$

Consider $v, v' \in \mathrm{Pref}(C^*)$ with $[v] = [v']$ and $x \in \Sigma$. Suppose $vx \in (C^n)^*\overline{u}_i$ for some $i \in \{1, 2\}$. Then we have $v \in (C^n)^*w$ for $w \in \mathrm{Pref}(\overline{u}_i) \setminus \{u_i\}$ with $|w| + 1 = |\overline{u}_i|$. By Equation (18.13) we find $v' \in (C^n)^*w$. Thus $v'x \in (C^n)^*\overline{u}_i$. We have $[vx] = [v'x]$ by Theorem 18.66. Thus, by Theorem 18.70, either both words $vx, v'x$ are contained in $\mathrm{Pref}(C^*)$ or both are not contained in this set. Combining these facts, we obtain that if one of the defining cases of δ' applies to v it also applies to v' and vice versa, and so the function δ' is well-defined.

Let $\varphi \colon (C^n)^* \to \{a_1, a_2\}^*$ be given by $\varphi(w_1 \cdots w_m) = \varphi(w_1) \cdots \varphi(w_m)$ for $w_1, \ldots, w_m \in C^n$ where $\varphi(\overline{u}_i) = a_i$ for $i \in \{1, 2\}$ and $\varphi(v) = \varepsilon$ for $v \in C^n \setminus \{\overline{u}_1, \overline{u}_2\}$. Since the factorization of w into words from C^n is unique, φ is well-defined. Note that φ is surjective and for each $w \in C^*$ there exists $\hat{w} \in \{\overline{u}_1, \overline{u}_2\}^*$ such that $\varphi(w) = \varphi(\hat{w})$.

Properties: By an easy induction, for $w \in (C^n)^*$, we find that we have

$$\delta'((q, [\varepsilon]), w) = (\delta(q, \varphi(w)), [\varepsilon]). \tag{18.16}$$

Next, consider a word $v \in \mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$. We can write $v = v_1 \cdots v_{m-1} v_m x$ for $m \geq 1$ with $v_1, \ldots, v_{m-1} \in C^n$, $v_m \in \mathrm{Pref}(C^n) \setminus C^n$, and $x \in \Sigma$ where we set $v_1 \cdots v_0$ equal to ε. Then, by Equation (18.16) and the definition of δ', for $q \in Q$ with $p = \delta(q, \varphi(v_1 \cdots v_{m-1}))$ we have

$$\delta'((q, [\varepsilon]), v) = \delta'((p, [\varepsilon]), v_m x) = \delta'((p, [v_m]), x) = (\overline{S}(p), [\varepsilon]). \tag{18.17}$$

Let $v \in \mathrm{Pref}(C^*)$. We have $\mathrm{Pref}(C^*) = C^*(\mathrm{Pref}(C) \setminus C)$. By assumption, each word in $(\mathrm{Pref}(C) \setminus C)u$ is weakly auxiliary state avoiding w.r.t. C. By Theorem 18.53 we obtain that each word in $C^*(\mathrm{Pref}(C) \setminus Cu$ is weakly auxiliary state avoiding w.r.t. C, i.e., a concatenation of words from $\mathrm{Pref}(C^*)\Sigma \setminus \mathrm{Pref}(C^*)$. By induction, with Equation (18.17) we then obtain

$$\delta'((q, [v]), u) = \delta'((q, [\varepsilon]), vu) \in S \times \{[\varepsilon]\}. \tag{18.18}$$

More precisely, since $|u| \leq n$ (and so it does not contain $\overline{u}_1$ or $\overline{u}_2$ as a factor), we also get[6]

$$\delta'((q, [\varepsilon]), u) = (\overline{S}(q), [\varepsilon]). \tag{18.19}$$

This then gives $\delta'(Q \times [\varepsilon], u^i) = S \times \{[\varepsilon]\}$ for all $i \geq 1$ since $\overline{S}^j = \overline{S}$ for all $j \geq 1$. Thus, with Equation (18.18), we find $\delta'(Q', u^i) = S \times \{[\varepsilon]\}$ for all $i \geq 1$.

Verification: Suppose $u^i v$ with $i \geq 1$ where $u^i \in U$ and $v \in V$ synchronizes $\mathcal{A}'$. By Equations (18.18) and (18.19) we have $\delta'(Q', u^i) = S \times \{\varepsilon\}$. Since $v \in \mathrm{Pref}(C^*)$, we can write $v = v_1 \cdots v_{m-1} v_m$ with $v_1, \ldots, v_{m-1} \in C^n$ and $v_m \in \mathrm{Pref}(C^n) \setminus C^n$. Then, for each $q \in S$, we have

$$\delta'((q, [\varepsilon]), v_1 \cdots v_{m-1} v_m) = (\delta(q, \varphi(v_1 \cdots v_{m-1})), [v_m]).$$

By assumption, this equals the same state for each $q \in S$ and so $|\delta(S, \varphi(v_1 \cdots v_{m-1}))| = 1$.

Conversely, let $v \in \{a_1, a_2\}^*$ be such that $|\delta(S, v)| = 1$. Pick a word $u^i \in Uu$ with $i \geq 1$. Write $v = v_1 \cdots v_m$ with $v_1, \ldots, v_m \in \{a_1, a_2\}$ and let w_j equal $\overline{u}_1$ if $v_j = a_1$ and $\overline{u}_2$ if $v_j = a_2$ for each $j \in \{1, \ldots, m\}$. We have $v = \varphi(w_1 \cdots w_m)$ and so, from Equation (18.16), for $q \in S$, we get

$$\delta'((q, [\varepsilon]), w_1 \cdots w_m) = (\delta(q, v), [\varepsilon]),$$

where $\delta(q, v)$ equals the same state for each $q \in S$ by assumption. Thus, since $\delta'(Q', u^i) = S \times \{\varepsilon\}$ by Equations (18.18) and (18.19), the word $u^i w_1 \cdots w_m$ synchronizes $\mathcal{A}'$. Since $w_1 \cdots w_m \in \{u_1, u_2\}^* \subseteq V$, this word is a synchronizing word for $\mathcal{A}'$ from UuV. □

Remark 18.87 By Theorem 18.64, Item 1, we obtain the relation $\equiv_{C^n}^{=, \Sigma^* \setminus \mathrm{Pref}(\{\overline{u}_1, \overline{u}_2\})}$. Let us also denote $[w]_{\equiv_{C^n}^{=, \Sigma^* \setminus \mathrm{Pref}(\{\overline{u}_1, \overline{u}_2\})}}$ by $[w]$ for $w \in \Sigma^*$ in the following. This relation can also be used to give an alternative construction in the proof of Theorem 18.86 by setting

[6] Note that $|u| \leq n$ implies, since u is also weakly auxiliary state avoiding w.r.t. C^n by Theorem 18.36, that u is strongly auxiliary state avoiding w.r.t. C^n by Theorem 18.39, Item 18.42.

$\mathcal{A}' = (Q \times \{[w] \mid w \in \mathrm{Pref}(C^n) \setminus C^n\}, \Sigma, \delta')$ and, for $q \in Q$, $w \in \mathrm{Pref}(C^n) \setminus C^n$ and $x \in \Sigma$,

$$\delta'((q,[w]),x) = \begin{cases} (q,[\varepsilon]) & \text{if } wx \in C^n \setminus \{\overline{u}_1, \overline{u}_2\}, \\ (\delta(q,a_i),[\varepsilon]) & \text{if } wx = \overline{u}_i \text{ for some } i \in \{1,2\}, \\ (q,[wx]) & \text{if } wx \in \mathrm{Pref}(C^n) \setminus C^n, \\ (\overline{S}(q),[\varepsilon]) & \text{if } wx \notin \mathrm{Pref}(C). \end{cases}$$

Then the remaining argument can be continued in a similar way as the original proof. Yet another similar proof can be given by using the classes $[w]_{\equiv_C^{\geq |\overline{u}_1|}}$ for $w \in \Sigma^*$, in the following simply denoted by $[w]$. Then let $\mathcal{A}' = (Q', \Sigma, \delta')$ with $Q' = Q \times \{[w] : w \in \mathrm{Pref}(C^n) \setminus C^n\}$ and, for $q \in Q$, $w \in \mathrm{Pref}(C^n) \setminus C^n$ and $x \in \Sigma$, set

$$\delta'((q,[w]),x) = \begin{cases} (\delta(q,a_1),[\varepsilon]) & \text{if } wx = \overline{u}_1, \\ (\delta(q,a_2),[\varepsilon]) & \text{if } wx \in C^n \setminus \{\overline{u}_1\}, \\ (q,[wx]) & \text{if } wx \in \mathrm{Pref}(C^n) \setminus C^n \\ (\overline{S}(q),[\varepsilon]) & \text{if } wx \notin \mathrm{Pref}(C^n). \end{cases}$$

Lemma 18.88 *Let $C \subseteq \Sigma^+$ be a prefix code and $w \in C^+$ be such that we have*

$$(\mathrm{Pref}(C) \setminus C)w \subseteq ((\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C) \cup C)^*.$$

Then $D = (C \setminus \{u\}) \cup u(C \setminus \{u\}) \cup uuC$ for $u \in C$ is a prefix code such that for $\hat{u} \in C \setminus \{u\}$ we have

$$(\mathrm{Pref}(D) \setminus D)w\hat{u} \in ((\mathrm{Pref}(D \setminus D)\Sigma \setminus \mathrm{Pref}(D) \cup D)^*.$$

Proof. Set $U = (\mathrm{Pref}(C) \setminus C)\Sigma \setminus \mathrm{Pref}(C)$ and $V = (\mathrm{Pref}(D) \setminus D)\Sigma \setminus \mathrm{Pref}(D)$. The following equations are easy to verify:

$$\mathrm{Pref}(D) \setminus D = \{\varepsilon, u, uu\}(\mathrm{Pref}(C) \setminus C) \tag{18.20}$$

$$\{\varepsilon, u, uu\}(\{\varepsilon\} \cup C \cup C^2 \cup C^3) = (\{\varepsilon\} \cup D)\{\varepsilon, u, uu\} \tag{18.21}$$

$$V = \{\varepsilon, u, uu\}U \tag{18.22}$$

Note that, from Equation (18.21), by induction, we obtain $\{\varepsilon, u, uu\}C^* = D^*\{\varepsilon, u, uu\}$. Since $C^* \subseteq \{\varepsilon, u, uu\}C^*$ and $D^*\{\varepsilon, u, uu\} \subseteq C^*$ this gives $C^* = D^*\{\varepsilon, u, uu\}$. Next, we show that

$$(U \cup C)^* \subseteq (V \cup D)^*\{\varepsilon, u, uu\} \tag{18.23}$$

Let $w \in (U \cup C)^*$. For $w = \varepsilon$, this is clear. Otherwise, we have $w = vx$ with $v \in (U \cup C)^*$ and $x \in U \cup C$. Inductively, we find $v \in (V \cup D)^*\{\varepsilon, u, uu\}$. By Equations (18.21) and (18.22), we have

$$\{\varepsilon, u, uu\}x \subseteq \begin{cases} V & \text{if } x \in U, \\ D^*\{\varepsilon, u, uu\} & \text{if } x \in C. \end{cases}$$

Thus $\{\varepsilon, u, uu\}x \in (V \cup D)^*\{\varepsilon, u, uu\}$, which gives $w \in (V \cup D)^*\{\varepsilon, u, uu\}$.

Next, since $\hat{u} \in D$ and $\{\varepsilon, u, uu\}\hat{u} \in D$, with Equation (18.23) we find $(U \cup C)^*\hat{u} \subseteq (V \cup D)^*D$. Then, from Equation (18.20) and the assumption $(\mathrm{Pref}(C) \setminus C)w \subseteq (U \cup C)^*$, we get

$$(\mathrm{Pref}(D) \setminus D)w \subseteq \{\varepsilon, u, uu\}(\mathrm{Pref}(C) \setminus C)w \subseteq \{\varepsilon, u, uu\}(U \cup C)^* \subseteq (U \cup C)^*$$

Thus we have $(\mathrm{Pref}(D) \setminus D)w\hat{u} \subseteq (V \cup D)^*D$ since $(U \cup C)^*\hat{u} \subseteq (V \cup D)^*D$. □

Theorem 18.89 *Let $C \subseteq \Sigma^+$ be a prefix code that is a regular language and contains three distinct words $u_1, u_2, u_3 \in C$. Suppose we have a non-empty regular language $L \subseteq \Sigma^*$ with $|\mathrm{Fact}(L) \cap \{u_1, u_2, u_3\}| \leq 2$ and there exists $w \in (C \setminus \{u_3\})^*$ such that*

$$(\mathrm{Pref}(C) \setminus C)w \subseteq ((((\mathrm{Pref}(C) \setminus C)\Sigma) \setminus \mathrm{Pref}(C)) \cup C)^*.$$

Then Sync$[(C \setminus \{u_3\})^*u_3L]$ *is* PSPACE*-hard.*

Proof. Set $D = (C\setminus\{u_1\})\cup u_1(C\setminus\{u_1\})\cup u_1u_1C$ and $E = ((\mathrm{Pref}(D)\setminus D)\Sigma\setminus\mathrm{Pref}(D))\cup D$. Then E is a maximal prefix code by Theorem 18.60 and, by Theorem 18.88, we have

$$vwu_2 \in E^* \tag{18.24}$$

for all $v \in \mathrm{Pref}(D) \setminus D$. Observe that we have $\mathrm{Pref}(D) \setminus D = \mathrm{Pref}(E) \setminus E$.

Let $\mathcal{A} = (Q, \{a_1, a_2, a_3\}, \delta)$ be an automaton. Set $v_1 = u_1u_2$, $v_2 = u_1u_1u_2$, $v_3 = u_3$, $v_4 = u_1u_3$, and $v_5 = u_1u_1u_3$. Define morphisms $\varphi\colon E^* \to \{a_1, a_2, a_3\}$ and $\psi\colon \{a_1, a_2, a_3\}^* \to E^*$ by $\varphi(v_i) = a_{\min\{i,3\}}$ for $i \in \{1, 2, 3, 4, 5\}$ and $\varphi(v) = \varepsilon$ for $v \in E \setminus \{v_1, v_2, v_3, v_4, v_5\}$, and $\psi(a_i) = v_i$ for $i \in \{1, 2, 3\}$ and then $\varphi(w_1 \cdots w_n) = \varphi(w_1) \cdots \varphi(w_n)$ for $w_1, \ldots, w_n \in E$ and $\psi(x_1 \cdots x_m) = \psi(x_1) \cdots \psi(x_m)$ for $x_1, \ldots, x_m \in \{a_1, a_2, a_3\}$. These morphisms are well-defined since E and $\{a_1, a_2, a_3\}$ are codes. Let $L' = \{u \in E^* \mid uv \in L \text{ for some } v \in \Sigma^*\}$. Since $|\mathrm{Fact}(L') \cap \{u_1, u_2, u_3\}| \leq 2$, at most one of the two sets $\mathrm{Fact}(L') \cap \{v_1, v_2\}$ or $\mathrm{Fact}(L') \cap \{v_3, v_4, v_5\}$ is non-empty. This yields that at most one of the two sets $\mathrm{Fact}(\varphi(L')) \cap \{a_1, a_2\}$ or $\mathrm{Fact}(\varphi(L')) \cap \{a_3\}$ is non-empty. By Theorem 18.2, Item 5, Sync$[\{a_1, a_2\}^*a_3\varphi(L')]$ is PSPACE-complete. Next, we give a reduction from this problem. Consider $\mathcal{A}$ to be an input automaton for this problem. For simplicity, in this proof, for $v \in \Sigma^*$, we denote $[v]_{\equiv_{E,\{v_1,\ldots,v_5\}}}$ simply by $[v]$ (as in the proof of Theorem 18.86, confusion with the notation for transition matrices should not arise). By Theorem 18.82, we have finitely many equivalence classes w.r.t. $\equiv_{E,\{v_1,\ldots,v_5\}}$.

Construction: Let $w_i \in \Sigma^*$ be such that $v_i \in w_i\Sigma$ for $i \in \{1, \ldots, 5\}$. Define $\mathcal{A}' = (Q \times \{[v] : v \in \Sigma^*\}, \Sigma, \delta')$ where, for $q \in Q$, $v \in \Sigma^*$ and $x \in \Sigma$,

$$\delta'((q, [v]), x) = \begin{cases} (\delta(q, a_{\min\{3,i\}}, [vx]) & \text{if } [v] = [w_i] \text{ for } i \in \{1, \ldots, 5\} \text{ with } w_ix = v_i, \\ (q, [vx])) & \text{otherwise.} \end{cases}$$

Consider $v, v' \in \Sigma^*$ with $[v] = [v']$. Suppose $[w_i] = [w_j]$ for $i, j \in \{1, \ldots, 5\}$. By Equation (18.13) this gives $w_i, w_j \in C^*w$ for some $w \in \mathrm{Pref}(\{v_1, \ldots, v_5\}) \setminus \{v_1, \ldots, v_5\}$.

By Theorem 17.25 we find $w_i = w_j = w$. Thus $[v] = [w_i]$ for some $i \in \{1, \ldots, j\}$ implies $[v] \neq [w_j]$ for $j \neq i$, and this yields that δ' is well-defined.

Note that for $v \in \Sigma^*$ we have $[v] = [\varepsilon]$ if and only if $v \in E^*$, which also gives $\{[v] : v \in \mathrm{Pref}(E) \setminus E\} = \{[v] : v \in \Sigma^*\}$ since $\mathrm{Pref}(E^*) = \Sigma^*$ by Theorem 18.58.[7]

Properties: For $w_1, \ldots, w_m \in E$, $x_1, \ldots, x_l \in \{a_1, a_2, a_3\}$, and $q \in Q$, we have

$$\delta'((q, [\varepsilon]), w_1 \cdots w_m) = (\delta(q, \varphi(w_1 \cdots w_m)), [\varepsilon]) \tag{18.25}$$

$$\delta'((q, [\varepsilon]), \psi(x_1 \cdots x_l)) = (\delta(q, x_1 \cdots x_l), [\varepsilon]) \tag{18.26}$$

$$\varphi(\psi(x_1 \cdots x_l)) = x_1 \cdots x_l \tag{18.27}$$

Verification: Suppose $\mathcal{A}'$ has a synchronizing word of the form $w_1 \cdots w_m u_3 v$ with $w_1, \ldots, w_m \in C \setminus \{u_3\}$ and $v \in L$. We have $w_1 \cdots w_m \in (D \setminus \{v_3, v_4, v_5\})^* \{\varepsilon, u_1, u_1 u_1\}$. Thus, we find $w_1 \cdots w_m u_3 \in (D \setminus \{v_3, v_4, v_5\})^* \{v_3, v_4, v_5\} \subseteq E^*$. Since E is a maximal prefix code, by Theorem 18.58 we have $v \in \mathrm{Pref}(E^*)$ and thus we can write $v = v'v''$ with $v' \in E^*$ and $v'' \in \mathrm{Pref}(E) \setminus E$. For $q \in Q$, we have

$$\begin{aligned} \delta'((q, [\varepsilon]), w_1 \cdots w_m u_3 v) &= \delta'((\delta(q, \varphi(w_1 \cdots w_m u_3 v')), [\varepsilon]), v'') && \text{(Eqn. (18.25))} \\ &= (\delta(q, \varphi(w_1 \cdots w_m u_3 v')), [v'']). && \text{(Def. } \delta') \end{aligned}$$

Since $\delta'((q, [\varepsilon]), w_1 \cdots w_m u_3 v)$ equals the same state for each $q \in Q$, also $\delta(q, \varphi(w_1 \cdots w_m u_3 v'))$ equals the same state for each $q \in Q$ and so $\varphi(w_1 \cdots w_m u_3 v')$ synchronizes $\mathcal{A}$. We have $\varphi(w_1 \cdots w_m) \in \{a_1, a_2\}^*$, $\varphi(u_3) = a_3$ and $\varphi(v') \in \varphi(L')$ since $v' \in L'$.

Conversely, suppose the word $x_1 \cdots x_m a_3 v$ synchronizes $\mathcal{A}$ where $x_1, \ldots, x_m \in \{a_1, a_2\}$ and $v = \varphi(v')$ for some $v' \in L'$. For $q \in Q$ and $w' \in \mathrm{Pref}(D) \setminus D$ we have $w'wu_2 \in E^*$ by Equation (18.24). This gives

$$\delta'((q, [w']), wu_2) = (q, [w'wu_2]) \in Q \times \{[\varepsilon]\}.$$

By Equations (18.25) and (18.27), for $q \in Q$, we have

$$\delta'((q, [\varepsilon]), \psi(x_1 \cdots x_m a_3)v')) = (\delta(q, \varphi(\psi(x_1 \cdots x_m a_3)v')), [\varepsilon]) = (\delta(q, x_1 \cdots x_m a_3 v), [\varepsilon]).$$

Since on the right hand side always the same state results, the word $wu_2\psi(x_1 \cdots x_m a_3)v'$ synchronizes $\mathcal{A}'$.[8] By the definition of L' we have $v'v'' \in L$ for some $v'' \in \Sigma^*$. Then $wu_2\psi(x_1 \cdots x_m a_3)v'v''$ synchronizes $\mathcal{A}'$ where $\psi(x_1, \ldots, x_m) \in (D \setminus \{v_3, v_4, v_5\})^* \subseteq (C \setminus \{u_3\})^*$ and $\psi(a_3) = u_3$. □

Remark 18.90 Using $[v]_{\equiv_{D,\{v_1,\ldots,v_5\}}}$ instead of $[v]_{\equiv_{E,\{v_1,\ldots,v_5\}}}$ in the proof of Theorem 18.89 (denoting it by $[v]$ in the following) for $v \in \Sigma^*$, we can also, alternatively,

[7] We note that $[v] = [w_i]$ and $w_i x = v_i$ is equivalent to $vx \in E^* v_i$. Thus, the transition function can also be defined as in the proof of Theorem 18.86 by having a case for $vx \in E^* v_i$, and also setting the second component equal to $[\varepsilon]$ in this case (but $vx \in E^* v_i$ implies $[vx] = [\varepsilon]$ anyway). However, we defined it as above since then it is easier to see that $\delta'((q, [v]), u) \in Q \times \{[vu]\}$ for $u, v \in \Sigma^*$.

[8] Note that with $\varphi(\psi(v)) = v = \varphi(v')$ we have $\varphi(\psi(x_1 \cdots x_m a_3 v)) = \varphi(\psi(x_1) \cdots \psi(x_m) u_3 \psi(v)) = \varphi(\psi(x_1 \cdots x_m a_3)v')$. Then Equation (18.26) yields that also $wu_2\psi(x_1 \cdots x_m a_3 v)$ synchronizes $\mathcal{A}'$.

use the following construction in its proof: $\mathcal{A}' = (Q \times \{[v] : v \in \mathrm{Pref}(D) \setminus D\}, \Sigma, \delta')$ where, for $v \in \mathrm{Pref}(D) \setminus D$ and $x \in \Sigma$,

$$\delta'((q,[v]),x) = \begin{cases} ((\delta(q, a_{\min\{i,3\}}), [\varepsilon]) & \text{if } vx = v_i \text{ for some } i \in \{1,2,3,4,5\}, \\ (q, [vx]) & \text{if } vx \in \mathrm{Pref}(D) \setminus \{v_1, \ldots, v_5\}, \\ (q, [\varepsilon]) & \text{otherwise.} \end{cases}$$

Let $v, v' \in \mathrm{Pref}(D) \setminus D$ with $[v] = [v']$ and $x \in \Sigma$. Then either both v, v' are contained in $\mathrm{Pref}(\{v_1, \ldots, v_5\})$ or none is contained in this set by Equation (18.13) and since D is a prefix code. If $v, v' \in \mathrm{Pref}(\{v_1, \ldots, v_5\})$, then $v = v'$ by Equation (18.14) and so $vx = v'x$. Otherwise, we have $[vx] = [v'x]$. By Theorem 18.70 we have that $vx \in D$ if and only if $v'x \in D$ and $vx \in \mathrm{Pref}(D) \setminus D$ if and only if $v'x \in \mathrm{Pref}(D) \setminus D$. Combining these relations, we find that one of the defining cases of δ' applies to vx if and only if it applies to $v'x$ and with $[vx] = [v'x]$ we get that δ' is well-defined. Since we pick $v \in \mathrm{Pref}(D) \setminus D$ and not $v \in \mathrm{Pref}(D^*)$ we can test for equality with one of the v_i in the first case, and do not have to check for containment in $D^* v_i$, or equivalently whether $[v] = [w_i]$ with $w_i x = v_i$. Since $[\varepsilon] = [u]$ for $u \in \Sigma^*$ if and only if $u \in D^*$, this construction works in the same way as the original construction and it can be verified similarly that it yields a reduction.

Recall that, by Theorems 17.20 and 17.83, a code contains at least two words if and only if C^* has exponential growth.

Example 18.91 In the proof of Theorem 18.2, Item 6 it was directly shown that $\textsc{Sync}[c(a+b)^*]$ is PSPACE-hard. By considering the prefix code $C = \{a, b\}$ and $u = c$, this also follows easily from Theorem 18.86, since we have $\mathrm{Pref}(C) \setminus C = \{\varepsilon\}$ and the word $\varepsilon \cdot u$ is weakly auxiliary state avoiding w.r.t. C by Theorem 18.34.

Example 18.92 Let $m, n \geq 1$. By Theorem 18.89 with $C = \{a^m, b, c\}$ it follows that the problem $\textsc{Sync}[\{b,c\}^* a^m]$ is PSPACE-hard. Further, as $\{b,c\}^* a^m \cap \Sigma^* \{b,c\} = \emptyset$, by Theorem 18.31 we find that $\textsc{Sync}[\{b,c\}^* a^m b^n c^n \{b,c\}^*]$ is PSPACE-hard.

Next, we list a few example constraint languages that yield PSPACE-complete problems. We will refer to most of them later in Section 19.4.2. PSPACE-hardness for the constraint language from Theorem 18.93, Item 8 has already been established in Theorem 18.16 as an example; nevertheless, we include another proof using Theorem 18.89, since we believe it gives a good illustration of its application.

Proposition 18.93 *Suppose* $\Sigma = \{a, b\}$ *and* $\Gamma \subseteq \Sigma$ *with* $|\Gamma| \leq 1$. *The problem* $\textsc{Sync}[L]$ *is* PSPACE*-hard for the following constraint languages* L*:*

1. $aa(bb^*a)^*$ *and* $bb(aa^*b)^*$,
2. $aa(bb + ba)^*$ *and* $bb(aa + ab)^*$,
3. $aba(a + bb)^*$ *and* $bab(b + aa)^*$,
4. $bba(aa^*b)^*$ *and* $aab(bb^*a)^*$,
5. $aab(b + ab)^*$ *and* $bba(a + ba)^*$,
6. $aab(bb + ab)^*$ *and* $bba(aa + ba)^*$,
7. $baab(aa + ab)^*$ *and* $baab(bb + ba)^*$,

8. $(bb + ba)^*a\Gamma^*$,
9. $(a + bb)^*ba\Gamma^*$,
10. $(b + ab)^*aa\Gamma^*$.

Proof. Two languages listed side by side are the same up to renaming of the letters. Hence, it is sufficient to show the claim for one language in each case.

1: All words of the form b^iaa with $i \geq 0$ are weakly auxiliary state avoiding w.r.t. $C = bb^*a$. So, Sync$[aaC^*]$ is PSPACE-hard by Theorem 18.86.

2: The words aa and baa are weakly auxiliary state avoiding w.r.t. $\{bb, ba\}$. So, the problem Sync$[aa\{bb, ba\}^*]$ is PSPACE-hard by Theorem 18.86.

3: The words aba and $baba$ are weakly auxiliary state avoiding w.r.t. $\{a, bb\}$. So, the problem Sync$[aba(a + bb)^*]$ is PSPACE-hard by Theorem 18.86.

4: The word a^ibb is weakly auxiliary state avoiding w.r.t. aa^*b for $i \geq 0$. Set $C = (aa^*b)^*$ and $V = aC$. Then $\{aab, aaab\}^* \subseteq V \subseteq C^*$. So, by Theorem 18.86, Sync$[bba(aa^*b)^*]$ is PSPACE-hard.

5: Follows by 1 and Theorem 18.5 as $\mathrm{Fact}(aab(b + ab)^*) = \mathrm{Fact}(aa(bb^*a)^*)$.[9]

6: Follows by 2 and Theorem 18.5 as $\mathrm{Fact}(aab(bb + ab)^*) = \mathrm{Fact}(aa(bb + ba)^*)$.

7: The two words $baab$ and $abaab$ are weakly auxiliary state avoiding w.r.t $\{aa, ab\}$. So, Sync$[baab\{aa, ab\}^*]$ is PSPACE-hard by Theorem 18.86.

8: Set $C = \{bb, ba, a\}$. We have $(\mathrm{Pref}(C) \setminus C)ba \subseteq C$ with $ba \in C$ and since $|\Gamma| \leq 1$ we have $|\mathrm{Fact}(\Gamma^*) \cap \{bb, ba, a\}| \leq 1$. By Theorem 18.89, Sync$[\{bb, ba\}^*a\Gamma^*]$ is PSPACE-hard.

9: Set $C = \{a, bb, ba\}$. We have $(\mathrm{Pref}(C) \setminus C)a \subseteq C$ with $a \in C$ and since $|\Gamma| \leq 1$ we have $|\mathrm{Fact}(\Gamma^*) \cap \{a, bb, ba\}| \leq 1$. By Theorem 18.89, Sync$[\{a, bb\}^*ba\Gamma^*]$ is PSPACE-hard.

10: Set $C = \{b, ab, aa\}$. We have $(\mathrm{Pref}(C) \setminus C)b \subseteq C$ with $b \in C$ and since $|\Gamma| \leq 1$ we have $|\mathrm{Fact}(\Gamma^*) \cap \{a, ab, aa\}| \leq 1$. By Theorem 18.89, Sync$[\{b, ab\}^*aa\Gamma^*]$ is PSPACE-hard. □

18.3.7 Two NP-Hardness Conditions

Here, we give two sufficient condition to deduce NP-hardness in Theorems 18.94 and 18.98. The latter uses two lemmas: Theorem 18.96 on non-equivalent words w.r.t. the Myhill-Nerode relation for special left ideal languages, Theorem 18.97 on the Levenshtein-Romanov mapping for a language.

Theorem 18.94 *Let $u, v \in \Sigma^*$ and $L \subseteq \Sigma^+$ be non-empty and regular with $|v| > 0$ such that $u \notin \mathrm{Fact}(v^*)$ and $v \notin \mathrm{Fact}(L)$. Then* Sync$[uv^+L]$ *and* Sync$[uv^*L]$ *are* NP*-hard.*

[9] Observe that $b(b + ab)^* = L(\mathcal{B}_{1,\{2\}})$ and $(bb^*a)^* = L(\mathcal{B})$ for $\mathcal{B} = (\{1, 2\}, \{a, b\}, \mu, 1, \{1\})$ with $\mu(1, b) = \mu(2, b) = 2$ and $\mu(2, a) = 1$. Now we have $L(\mathcal{B}) = \{\varepsilon\} \cup L(\mathcal{B}_{1,\{2\}})a$ and $L(\mathcal{B}_{1,\{2\}}) = L(\mathcal{B})b$. This gives the stronger claims $aab(b+ab)^*a \subseteq aa(bb^*a)$ and $aa(bb^*a)b \subseteq aab(b+ab)^*$ than mere equality of the factor sets. Thus, both languages arise from the same constraint PDFA by merely redistributing the final states inside a sink component. For Item 6 a similar PDFA can be found with this property.

Proof. Since $v \notin \mathrm{Fact}(L)$ we have $\Sigma^* v \Sigma^* \cap uv^* L = uv^+ L$ and thus, by Theorem 18.12, it is sufficient to show the claim for $\textsc{Sync}[uv^+L]$.

We give a reduction from DisjSetTrans over a unary alphabet, which is NP-complete by Theorem 8.18. Let $(\mathcal{A}, S, T)$ be an instance with $\mathcal{A} = (Q, \{b\}, \delta)$ and $S, T \subseteq Q$ such that $S \cap T = \emptyset$. Pick some $\hat{s} \in S$ and let $\overline{S}\colon (Q \setminus T) \to S$ be given by $\overline{S}(q) = \hat{s}$ if $q \in Q \setminus (S \cup T)$ and $\overline{S}(q) = q$ otherwise. Let $\mathcal{A}' = (\Sigma, (Q \times \mathrm{Pref}(uv')) \cup \{t\}, \delta')$ where, for $w \in \mathrm{Pref}(uv')$ and $x \in \Sigma$ we set

$$\delta'((q,w),x) = \begin{cases} (\delta(q,b),u) & \text{if } wx = uv, \\ t & \text{if } wx \notin \mathrm{Pref}(uv') \text{ and } q \in T, \\ (\overline{S}(q),u) & \text{if } \sigma_{\{u\}}(wx) = u, \\ (q, \sigma_{\{uv'\}}(wx)) & \text{otherwise,} \end{cases}$$

for $q \in Q$ and $\delta'(t,x) = t$. Next, we show that there exists $i \geq 0$ such that $\delta(q, b^i) \in T$ for all $q \in S$ if and only if $\mathcal{A}'$ admits a synchronizing word in uv^+L.

First, suppose we have $\delta(q, b^i) \in T$ for all $q \in S$ and for some $i \geq 0$. Since $S \cap T = \emptyset$ we must have $i > 0$. Let (q, w) with $q \in Q$ and $w \in \mathrm{Pref}(uv')$. Then we have $\delta'((q,w),u) \in S \times \{u\} \cup \{t\}$. This follows by the definition of the third case, since if we still read a prefix of uv', but have encountered the u as a suffix along this prefix, we move into a state with u as it second component. This also works if we read u but encounter the suffix u before having finished with reading u as input by Theorem 17.3, Item 2. Next, let $q \in S$. Then $\delta'((q,u), v^i) = (\delta(q, b^i), u)$. Consider $w' \in L$. Since $v \notin \mathrm{Fact}(w')$, there is a smallest prefix w'' of w' and $x \in \Sigma$ such that $uw'' \in \mathrm{Pref}(uv')$ and $uw''x \notin \mathrm{Pref}(uv')$. Since $\delta(q, b^i) \in T$ we then find $\delta'((\delta(q, b^i), u), w'x) = t$. Combining everything, the word $uv^i w'$ maps all states to t.

Conversely, let $uv^i w$ with $i \geq 1$ and $w' \in L$ be a synchronizing word for $\mathcal{A}'$. Since t is a trap state, it must be the synchronizing state. Consider a state $q \in S$. We have $\delta'((q, \varepsilon), uv^i) = (\delta(q, b^i), uv^i)$. Assume $\delta(q, b^i) \notin T$. Write $w = w_1 \cdots w_m$ with $w_1, \ldots, w_m \in \Sigma$. We have $\delta'((\delta(q, b^i), u), w_1 \cdots w_i) \in (Q \setminus T) \times \{\sigma_{\{uv'\}}(w_1 \cdots w_i)\}$ for $i \in \{0, 1, \ldots, m\}$ (we set $w_1 \cdots w_0$ equal to ε). This follows inductively. For $i = 0$ the claim clearly holds. Next, let $i \in \{1, \ldots, m\}$. For $\sigma_{\{uv'\}}(w_1 \cdots w_{i-1}) = uv'$ we have $uv'w_i \neq uv$ since $v \notin \mathrm{Fact}(w)$, and thus only the last two cases of the transition function can apply, which gives the claim. For $\sigma_{\{uv'\}}(w_1 \cdots w_{i-1}) \neq uv'$, we also have $uv'w_i \neq uv$ and also only the last two cases of the transition function can apply. Thus, we do not end up in the state t and so we must have $\delta(q, b^i) \in T$ for all $q \in S$. □

Remark 18.95 Theorem 18.94 was given in [296, Prop. 8] but with the additional assumption $\mathrm{Pref}(v^*) \cap L = \emptyset$ (and the assumption $|v| > 0$ was missing, which was a mistake).

Lemma 18.96 *Let $I = \Sigma^* uv^* w$ for $u, v, w \in \Sigma^*$ and $x, y \in \mathrm{Pref}(uw)$ with $x \neq y$. Then we have $x \not\equiv_I y$.*

Proof. Without loss of generality, let $|x| < |y|$. There exists $z \in \mathrm{Suff}(u)$ such that $yz = uw$. Thus we have $yz \in I$. Since $|xz| < |uw|$ and uw is the shortest word in I, we have $xz \notin I$. This gives $x \not\equiv_I y$. □

Lemma 18.97 *Suppose that $a, b, c \in \Sigma$ with $a \neq b$ and $c \neq b$ (while $a = c$ is allowed). Let $u, v \in \Sigma^*$, $i > |u| + |v|$, and $p \geq 0$. Set $L = uab^i(b^p)^*cv$. Then for $w \in \Sigma^*$ we have $\sigma_L(wuab^i) = uab^i$.*

Proof. Since $uab^i \in \mathrm{Pref}(L)$ for $w \in \Sigma^*$ we have $\sigma_L(wuab^i) = yuab^i$ for some $y \in \Sigma^*$ such that $yuab^i \in \mathrm{Pref}(L)$. The factor ab^i appears in a word from L precisely once after the prefix u as $i > |u|+|v|$. This yields that the only prefix of a word from L ending with ab^i is uab^i. Thus $yuab^i \in \mathrm{Pref}(L)$ gives $y = \varepsilon$. □

Proposition 18.98 *Suppose that $a, b, c \in \Sigma$ with $a \neq b$ and $c \neq b$ (while $a = c$ is allowed). Let $L \subseteq \Sigma^*$ be regular such that*

$$uab^i(b^p)^*cv \subseteq L \subseteq x^*uab^i(b^p)^*cvy^*$$

for $x, y \in \Sigma$ with $i > |u| + |v|$ and $p > 0$. Then Sync$[L]$ *is* NP*-hard.*

Proof. We give a reduction from SetTrans on unary automata, which is NP-complete by Theorem 8.18. Set $I = \Sigma^*U$ for $U = uab^i(b^p)^*cv$. For short, in this proof we write $[w]$ for $[w]_{\equiv_I}$ and $w \in \Sigma^*$.

Construction: Let $(\mathcal{A}, S, T)$ be an instance of SetTrans where $\mathcal{A} = (Q, \{b\}, \delta)$. Our reduction first checks whether $S \subseteq T$, and if this is the case, $(\mathcal{A}, S, T)$ is a YES-instance. Next, assume $S \not\subseteq T$. Then pick some state $\hat{q} \in S$ and let $\overline{S}\colon Q \to S$ be given by $\overline{S}(q) = q$ if $q \in Q$ and $\overline{S}(q) = \hat{q}$ if $q \notin S$. Set $\mathcal{A}' = (Q', \Sigma, \delta')$ with $Q' = (Q \cup \{t\}) \times P$ for $P = \{[w] \mid w \in \Sigma^*\}$ and

$$\delta'((q,[w]),x) = \begin{cases} (\delta(q,b),[wx]) & \text{if } wx \equiv_I uab^i, \\ (t,[wx]) & \text{if } w \equiv_I uab^i \text{ and } x = c \text{ and } q \in T, \\ (\overline{S}(q),[wx]) & \text{otherwise.} \end{cases}$$

By Equation (6.24), δ' is well-defined. Note that $P = \{[w] \mid w \in \mathrm{Pref}(uab^icv)\}$ by Theorem 17.7 and since $w \in \mathrm{Pref}(U)$ with $w = uab^iw'$ for some $w' \in \Sigma^*$ is $\equiv_I$-equivalent to each word from $uab^i(b^p)^*w'$.

Verification: Next, we show that $\mathcal{A}'$ has a synchronizing word in L if and only if there exists $j \geq 0$ such that $\delta(S, b^j) \subseteq T$ in $\mathcal{A}$.

First, suppose $x^ruab^{i+l\cdot p}cvy^s \in L$ for $l, r, s \geq 0$ is a word synchronizing $\mathcal{A}'$. Since the states in $\{t\} \times P$ form an upper set, the synchronizing state must belong to this set. Let $q \in Q$. Consider a prefix $u' \in \mathrm{Pref}(x^ruab^{i-1})$. Assume $u'cv \in I$. Then u' must have a word from $uab^i(b^p)^*$ as a suffix, in particular, it must have ab^i as a factor. This implies that b^i must be a factor of u, which is not possible since $i > |u|$. Thus we have $u'cv \notin I$. As $uab^icv \in I$, we find $u' \not\equiv_I uab^i$. Next, write $x^ruab^{i-1} = zu'$ for some $z \in \Sigma$ and $u' \in \Sigma^*$. By the definition of δ', we then find $\delta'((q,[\varepsilon]),z) = (\overline{S}(q),[z])$. Since $\overline{S}^2 = \overline{S}$, and no prefix of x^ruab^{i-1} is $\equiv_I$-equivalent to uab^i, we obtain $\delta'((\overline{S}(q),[z]),u') = (\overline{S}(q),[zu'])$. By Theorem 18.97 we have $\sigma_U(zu'b) = uab^i$. With Theorem 17.6 this gives $zu'b \equiv_I uab^i$. By the definition of δ', we then obtain

$$\delta'((\overline{S}(q),[zu']),b) = (\delta(\overline{S}(q),b),[uab^i]).$$

Thus, for $q \in S$, we find $\delta'((q, [\varepsilon]), z^r uab^i)) = (\delta(q, b), [uab^i])$. Next, we have

$$\delta'((\delta(q, b), [uab^i]), b^{l \cdot p}) = (\delta(q, b^{l+1}), [uab^i]).$$

Now, assume $\delta(q, b^{l+1}) \notin T$. Then, since $uab^i c \not\equiv_I uab^i$ and by the definition of δ', we find

$$\delta'((\delta(q, b^{l+1}), [uab^i]), c) = (\overline{S}(\delta(q, b^{l+1})), [uab^i c]).$$

Since the state $(\overline{S}(\delta(q, b^l)), [uab^i c])$ is mapped to a state in $\{t\} \times P$ by vy^s, by the definition of δ' we must have $uab^i cw \equiv_I uab^i$ for some $w \in \mathrm{Pref}(vy^s)$. By Theorem 18.96 we have $uab^i cw \not\equiv_I uab^i$ for $w \in \mathrm{Pref}(v)$. Thus, there exists $j > 0$ such that $uab^i cvy^j \equiv_I uab^i$ where we choose j to be minimal. This gives $y = b$, since for $y \neq b$ we have $uab^i cvycv \notin I$ since the word has no suffix from U, but $uab^i cv \in I$. Next, by the definition of δ', since $uab^i cw \not\equiv_I uab^i$ for $w \in \mathrm{Pref}(v)$, we have

$$\delta'((\overline{S}(\delta(q, b^{l+1})), [uab^i c]), v) = (p, [uab^i cv]))$$

for some $p \in S$. Then with $y = b$ we find

$$\delta'((\overline{S}(\delta(q, b^{l+1})), [uab^i c]), vy^s) \in Q \times \{[uab^i cvy^s]\}.$$

Thus, the state $(\overline{S}(\delta(q, b^l)), [uab^i c])$ cannot be mapped into $\{t\} \times P$. Hence, we have $\delta(q, b^{l+1}) \in T$ for all $q \in S$.

Conversely, suppose there exists $l \geq 0$ such that $\delta(q, b^l) \in T$ for all $q \in S$. Since $S \not\subseteq T$, we have $l > 0$. Consider the word $uab^{i+(l-1)\cdot p} cv$ from L. Let $q \in Q$ and $[w] \in P$ with $w \in \mathrm{Pref}(uab^i cv)$. By Theorem 18.97, we have $\sigma_U(wuab^i) = uab^i$. With Theorem 17.6 we get $wuab^i \equiv_I uab^i$. If we have $\delta'((q, [w], uab^i) = \delta'((q, [\varepsilon]), wuab^i) = (t, [uab^i])$, then $\delta'((q, [w], uab^i cv) = (t, [uab^i cv])$. Otherwise, similarly as above, we find

$$\delta'((q, [w], uab^i) \in \delta(S, b) \times [uab^i].$$

Next, we have

$$\delta'(\delta(S, b) \times [uab^i], b^{(l-1)\cdot p}) \in \delta(S, b^l) \times [uab^i]).$$

Since $\delta(S, b^l) \subseteq T$, we find $\delta'(\delta(S, b^l) \times [uab^i]), c) \subseteq \{t\} \times \{[uab^i c]\}$. Since $\delta'((t, [uab^i c], v) = (t, [uab^i cv])$, we find that $(q, [w])$ is mapped to $(t, [uab^i cv])$ by $uab^{i+(l-1)\cdot p} cv$. Since we also have $\delta'((t, [w]), uab^{i+(l-1)\cdot p} av) = (t, [uab^i cv])$ for each $w \in \Sigma^*$, we find that $uab^{i+l\cdot p} cv$ maps all states of $\mathcal{A}'$ to $(t, [uab^i av])$. □

18.4 Remarks on Constraint PFDAs

In Section 18.4.1 we show that, in a constraint automaton, we have some degree of freedom on how to distribute the start state and the final states. Later, in some proofs, this allows us to choose those states in a way that fits best in a given situation. In Section 18.4.2 we remark how to reformulate some results from Section 18.2.5 in

terms of PDFAs. Various conditions on constraint PDFAs implying that the problem is equivalent to $\textsc{Sync}[\Sigma^*]$ are collected in Section 18.4.3.

18.4.1 Distributing the Start State and the Final States

By Theorems 18.4 and 18.5 the complexity of the constrained synchronization problem depends on the set of factors of the constraint language. Theorem 18.99 concerns how we can distribute a start state and a set of final states in a trim PDFA without changing the factor set of the accepted language. Various conclusions from Theorem 18.99 on altering the start state and set of final states while preserving the computational complexity are collected in Theorem 18.100. Lastly, in Theorem 18.101, we prove that for trim PDFAs, considering multiple starting states does not change the complexity.

Proposition 18.99 *Suppose $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ is a trim PDFA. Let $p \in P$ and $E \subseteq P$ be such that p_0 is accessible*[10] *in $\mathcal{B}_{p,E}$ and every state in F is coaccessible in $\mathcal{B}_{p,E}$. Then* $\mathrm{Fact}(L(\mathcal{B})) = \mathrm{Fact}(L(\mathcal{B}_{p,E}))$.

Proof. Let $u \in L(\mathcal{B})$. As p_0 is accessible in $\mathcal{B}_{p,E}$, there exists $v \in \Sigma^*$ such that $\mu(p, v) = p_0$. Furthermore, as $\mu(p_0, u) \in F$, and each state in F is coaccessible in $\mathcal{B}_{p,E}$ there exists $w \in \Sigma^*$ such that $\mu(\mu(p_0, u), w) \in E$. Hence $vuw \in L(\mathcal{B}_{p,E})$.

Now, suppose $u \in L(\mathcal{B}_{p,E})$. As $\mathcal{B}$ is trim, there exist $v, w \in \Sigma^*$ such that $\mu(p_0, v) = p$ and $\mu(\mu(p, u), w) \in F$. So $vuw \in L(\mathcal{B})$. □

Next, using Theorems 18.4 and 18.5, we state some consequences of Theorem 18.99 regarding changes to the start state or modifications to the final state set, while preserving the computational complexity of the associated constrained problem.

Corollary 18.100 *Suppose $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ is a trim PDFA.*

1. $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L(\mathcal{B}_{s,F\cup\{q\}})]$ *for every $s \in [p_0]_{\simeq_R}$ and $q \in P$.*[11]
2. *For $q \in P$ coaccessible in $\mathcal{B}_{p_0,F\setminus\{q\}}$ we have* $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L(\mathcal{B}_{p_0,F\setminus\{q\}})]$.
3. $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L(\mathcal{B}_{p_0,P})]$.
4. *Let $E \subseteq P$ be a set such that $E \cap [q]_{\simeq_R} \neq \emptyset$ for each maximal strongly connected component $[q]_{\simeq_R}$. Then* $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L(\mathcal{B}_{p_0,E})]$.

Proof. In the following we only show equality of the factor sets, the equivalence for $\leq_m^{\mathsf{AC}^0}$-reductions follows from Theorem 18.5.

1: By Theorem 18.99 applied to $p = p_0$ and $E = F \cup \{q\}$.

2: By Theorem 18.99 applied to $p = p_0$ and $E = F \setminus \{q\}$.

3: By Theorem 18.99 applied to $p = p_0$ and P where, as $F \subseteq P$, every state in F is coaccessible in $L(\mathcal{B}_{p_0,P})$.

4: By the assumption every state is coaccessible in $B_{p_0,E}$ so the claim follows from Theorem 18.99. □

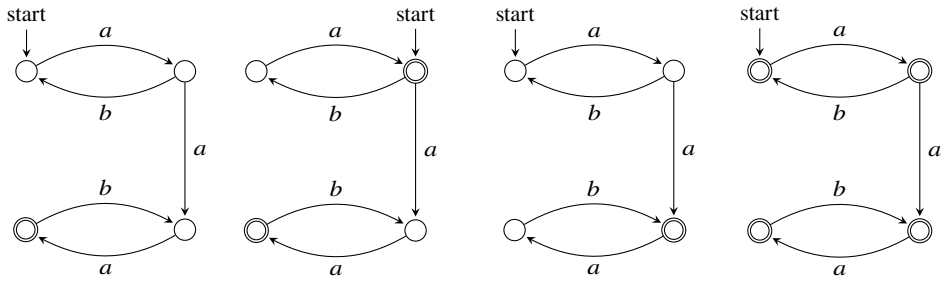


Fig. 18.5: The following distributions of final states (marked with a double circle) and start state all yield the same complexity in the constrained synchronization problem when the accepted languages are used as constraint languages.

To illustrate Theorem 18.100, Figure 18.5 gives different distributions of final states and a start state for a PDFA that all yield the same computational complexity in the constrained synchronization problem when the constraint language is the language accepted by the PDFA.

When we omit the assumption of $\mathcal{B}$ being trim in Theorem 18.100, 1 still holds for every coaccessible $q \in P$, 2 holds as well without modification, 3 holds by taking the set of all coaccessible states instead of P, and in 4 we have to consider only those maximal $[q]_{\simeq_R}$ containing at least one final state.

Lastly, we state that for trim PDFAs, considering multiple starting states does not change the complexity.

Proposition 18.101 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a trim PDFA. Then*

$$\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L(\mathcal{B}) \cup \bigcup_{p \in I} L(\mathcal{B}_{p,F})]$$

for all $I \subseteq P$.

Proof. Let $u \in L(\mathcal{B}_{p,F})$ for some $p \in I$. As $\mathcal{B}$ is trim, there exists $v \in \Sigma^*$ such that $\mu(p_0, v) = p$ and we get $vu \in L(\mathcal{B})$. So $\bigcup_{p \in I} L(\mathcal{B}_{p,F}) \subseteq \mathrm{Suff}(L(\mathcal{B}))$ and by Theorem 18.10 the claim follows. □

18.4.2 Concatenation and Complexity Lower Bounds

The following statement is a reformulation of Theorem 18.27, Item 1 and Theorem 18.31 in terms of constraint PDFAs to obtain complexity lower bounds.

Proposition 18.102 *Let $\mathcal{B} = (\Sigma, P, \mu, p_0, F)$ be a PDFA and set*

[10] As $\mathcal{B}$ is assumed to be trim, $p \precsim_R p_0$, i.e., p_0 being accessible in $\mathcal{B}_{p,E}$, is equivalent ot $p_0 \simeq_R p$.

[11] Note that, as $\mathcal{B}$ is trim, we have $F \neq \emptyset$ and p is coaccessible.

1. $\Gamma_1 = \{x \in \Sigma \mid \mu(p_0, x) \text{ is undefined } \}$ *and*
2. $\Gamma_2 = \{x \in \Sigma \mid \forall p \in P\colon \mu(p, x) \notin F\}$.[12]

If $U_1 \subseteq \Gamma_1^*$, $U_2 \subseteq \Gamma_2^*$ *are regular, then* $\textsc{Sync}[L(\mathcal{B})] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[U_1 L(\mathcal{B}) U_2]$.

Proof. As $\Gamma_1\Sigma^* \cap L(\mathcal{B}) = \emptyset$ and $\Sigma^*\Gamma_2 \cap L(\mathcal{B}) = \emptyset$ the claim follows from Theorem 18.27, Item 1 and Theorem 18.31. □

18.4.3 Equivalence to Unconstrained Synchronization

First, Theorem 18.103 states a rather general condition implying that the factor set of the language accepted by a PDFA equals Σ^*. Then, Theorem 18.104 derives several conditions implying equivalence to $\textsc{Sync}[\Sigma^*]$.

Proposition 18.103 *Let* $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ *be a PDFA. Suppose there exists a set* $S \subseteq P$ *of coaccessible states containing an accessible state* $s \in S$ *and such that* $\mu(p, x) \in S$ *for each* $p \in S$ *and* $x \in \Sigma$. *Then* $\mathrm{Fact}(L(\mathcal{B})) = \Sigma^*$.

Proof. Let $v \in \Sigma^*$. As s is accessible there exists $u \in \Sigma^*$ such that $\mu(p_0, u) = s$. By assumption $\mu(p_0, uv) \in S$ and there exists $w \in \Sigma^*$ such that $\mu(p_0, uvw) \in F$. So, $v \in \mathrm{Fact}(L(\mathcal{B}))$ and we get $\mathrm{Fact}(L(\mathcal{B})) = \Sigma^*$. □

Corollary 18.104 *Let* $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ *be a PDFA. Then* $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\Sigma^*]$ *and so* $\textsc{Sync}[L(\mathcal{B})]$ *is* NL*-complete for* $|\Sigma| \geq 2$ *and* L*-complete for* $|\Sigma| = 1$ *in the following cases:*

1. $\mathcal{B}$ *is trim and there exists a non-empty* $S \subseteq P$ *such that* $\mu(p, x) \in S$ *for all* $p \in S$ *and* $x \in \Sigma$.
2. $\mathcal{B}$ *is complete and every state is coaccessible.*
3. $\mathcal{B}$ *contains an accessible trap state that is a final state, i.e., a state* $p \in F$ *with* $L(\mathcal{B}_{p,\{p\}}) = \Sigma^*$ *and* $\mu(p_0, u) = p$ *for some* $u \in \Sigma^*$.
4. $\mathcal{B}$ *is trim and has a trap state.*

Proof. In the following, we only point out how to obtain a set S such that we can deduce $\mathrm{Fact}(L(\mathcal{B})) = \Sigma^*$ from Theorem 18.103. Then, the claim follows from Theorem 18.5, Theorems 8.13 and 8.20.

1: As $\mathcal{B}$ is trim, S from the statement fulfills the conditions in Theorem 18.103.

2: Consider $S = P$ for Theorem 18.103.

3: Consider the set S containing the accessible final trap state.

4: A coaccessible trap state must be a final state. Thus, since $\mathcal{B}$ is trim, each of its trap states is final and the claim follows from Item 3. □

Corollary 18.105 *Let* $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ *be a trim PDFA. If for each* $u \in \Sigma^*$ *we have* $\mu(p, u) = q$ *for some* $p, q \in P$, *then* $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\Sigma^*]$.

[12] We consider $\mu(p, x) \notin F$ to be true also when $\mu(p, x)$ is undefined; see also the convention from Section 17.1 where for partial functions f, $f(x) \neq y$ included the case that $f(x)$ is undefined.

Proof. Let $u \in \Sigma^*$. Then $\mu(p,u) = q$ for some $p, q \in P$. Since $\mathcal{B}$ is trim, we have $\mu(p_0, v) = p$ and $\mu(q, w) \in F$ for some $v, w \in \Sigma^*$. Thus $u \in \mathrm{Fact}(L(\mathcal{B}))$, which gives $\mathrm{Fact}(L(\mathcal{B})) = \Sigma^*$. So, the claim follows with Theorem 18.5, Theorems 8.13 and 8.20. □

Remark 18.106 For a trim automaton $\mathcal{B}$ with a totally defined transition function, by Theorem 18.104, Item 2 we have $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\Sigma^*]$. Hence, to describe constraint languages giving other complexities than the unconstrained problem, automata with totally defined transitions functions have useless states. So, out of this reason, we opted to use PDFAs to specify constraint languages as a more compact description than automata with a totally defined transition function.

18.5 Discussion

Here, we discuss the papers [215, 298, 301, 316] and how some of the result stated therein relate to the general results stated here.

Much of the general results as complexity-preservation for solid codes, the ideal intersection theorem, sufficient conditions for containment in AC^0, L, or NL, and general PSPACE-hardness conditions were first presented in [316]. The PSPACE-hardness conditions were proved differently in [316] by using the equivalence relation $\equiv_L^{\geq n}$ and a general automaton constructions called *automata unfolding*. The method of proof presented here is more general. We gave proofs using another equivalence relation arising from an abstract construction, which included $\equiv_L^{\geq n}$ as a special case.

Fernau et al. [215] were the first that systematically investigated the constrained synchronization problem on its own (the paper [271] that introduced the constrained synchronization problem only used it as an auxiliary tool). Next, we discuss some general results from [215] and their relation to the results from the present section. For the convenience of the reader, we first restate each result from [215] before discussing it here.[13] All languages in the statements are assumed to be regular.

1. [215, Thm. 15]: Let $L \subseteq L' \subseteq \Sigma^*$. If $L' \subseteq \{ v \in \Sigma^* \mid \exists u, w \in \Sigma^* : uvw \in L \}$, then $\textsc{Sync}[L] \equiv_m^{\log} \textsc{Sync}[L']$.
   This statement was generalized in Theorems 18.4 and 18.5.
2. [215, Thm. 27]: Let $L \subseteq \Gamma^*$ and $\varphi : \Sigma^* \to \Gamma^*$ be a morphism such that $\varphi(\varphi^{-1}(L)) = L$. Then $\textsc{Sync}[L] \leq_m^{\log} \textsc{Sync}[\varphi^{-1}(L)]$.
   Theorem 18.17 of the present work is a slight (but proper) generalization, i.e., Theorem 18.17 implies [215, Thm. 27] but not vice versa. For suppose $L \subseteq \Gamma^*$ is given and a morphism $\varphi : \Sigma^* \to \Gamma^*$ such that $\varphi(\varphi^{-1}(L)) = L$. Set $U = \varphi^{-1}(L)$. Theorem 18.17 then yields

$$\textsc{Sync}[\varphi(U)] \leq_m^{\log} \textsc{Sync}[U].$$

[13] In [215] the results were stated with $\leq_m^{\log}$-reductions but the proofs of the cited statements actually give $\leq_m^{\mathsf{AC}^0}$-reductions.

So, as $\varphi(U) = L$, it is $\text{Sync}[L] \leq_m^{\log} \text{Sync}[\varphi^{-1}(L)]$. However, Theorem 18.17 is more general than [215, Thm. 27] (even when stated for $\leq_m^{\mathsf{AC}^0}$-reductions). For $L \subseteq \Sigma^*$ and a morphism $\varphi\colon \Sigma^* \to \Gamma^*$, in general, we only have $\varphi(\varphi^{-1}(L)) \subseteq L$, but not equality. We can apply [215, Thm. 27] to $\varphi(L)$ as $\varphi(\varphi^{-1}(\varphi(L))) = \varphi(L)$, but this just gives $\text{Sync}[\varphi(L)] \leq_m^{\log} \text{Sync}[\varphi^{-1}(\varphi(L))]$.

3. [215, Thm. 31]: (We have adjusted this statement slightly to conform to our notations and conventions.) Let $\mathcal{B} = (P^{\mathcal{B}}, \Sigma^{\mathcal{B}}, \mu^{\mathcal{B}}, p_0^{\mathcal{B}}, F)$ and $C = (P^C, \Sigma^C, \mu^C, p_0^C, \emptyset)$ be PDFAs with $P^{\mathcal{B}} \cap P^C = \emptyset$. For[14] $p_x \in P^C$ let $\nu \subseteq \{p_x\} \times (\Sigma^{\mathcal{B}} \cup \Sigma^C) \times \{p_0^{\mathcal{B}}\}$ be such that the following three conditions are satisfied:
   a. $(p_x, x, p) \in \mu^C$ for some $x \in \Sigma^C$ and $p \in P^C$ implies $(p_x, x, p_0^{\mathcal{B}}) \notin \nu$,[15]
   b. the automaton $C' = (P^C \cup \{p_0^{\mathcal{B}}\}, \Sigma^{\mathcal{B}} \cup \Sigma^C, \mu^C \cup \nu \cup \{p_0^{\mathcal{B}}\} \times (\Sigma^{\mathcal{B}} \cup \Sigma^C) \times \{p_0^{\mathcal{B}}\})$ is carefully synchronizing, and
   c. there exists a synchronizing[16] word $v = v_1 \dots v_n$ [where $v_i \in \Sigma, i \in \{1, \dots, n\}$] for C' such that $v_1 \dots v_{n-1} \in L(C_{p_0^C, \{p_x\}})$.

   Then $\text{Sync}[L(\mathcal{B})] \leq_m^{\log} \text{Sync}[L(\mathcal{B}')]$ for the PDFA[17] $\mathcal{B}' = (P^{\mathcal{B}} \cup P^C, \Sigma^{\mathcal{B}} \cup \Sigma^C, \mu^{\mathcal{B}} \cup \mu^C \cup \nu, p_0^C, F)$.
   This statement is covered by Theorem 18.28, which can be seen by considering the language accepted by $\mathcal{B}'$. Define

$$\overline{C} = (P^C \cup \{p_0^{\mathcal{B}}\}, \Sigma^{\mathcal{B}} \cup \Sigma^C, \mu^C \cup \nu, p_0^C, \{p_0^{\mathcal{B}}\}).$$

   Then $L(\mathcal{B}') = L(\overline{C})L(\mathcal{B})$ as every accepting word has to pass through the transitions in ν (they form a choke point). As $\overline{C}$ has a unique final state with no outgoing transitions, by Theorem 17.30, Item 1 it accepts a prefix code. Then Theorem 18.28 yields $\text{Sync}[L(\mathcal{B})] \leq_m^{\log} \text{Sync}[L(\overline{C})L(\mathcal{B})]$.

Theorem 18.9 about the union of constraint languages was proven for the special case of P in [215, Lem. 13]. The formulation for multiple complexity classes appeared in [299, Lem. 7] (but without mentioning AC^0, NC^1, L and NL).

Hoffmann [301] showed the Ideal Intersection Theorem (Theorem 18.12), the complexity reduction by morphic images (Theorem 18.17), and Theorem 18.4. Furthermore, in [301, Thm. 4.5] also appeared a general PSPACE-hardness condition (called the uC-Theorem). However, the proof (available in the arXiv version [295]) contained a mistake (fixed in the extended version [316]). The PSPACE-hardness conditions presented in Section 18.3.6 extend the condition from [301]. Our conditions also work with infinite prefix codes, whereas the PSPACE-hardness condition from [301] only holds for finite prefix codes.

---

[14] Here, a transition function ν is identified with the set of tuples $\{(p, x, q) \mid \nu(p, x) = q\}$.

[15] In the original formulation this assumption is missing and instead it is equivalently assumed that $\mathcal{B}'$ is deterministic, but we do not introduce nondeterministic automata here and so reformulated it.

[16] This is the original formulation, not stating whether an exactly synchronizing word or a carefully synchronizing word is meant here (given assumption (a), it probably means carefully synchronizing word).

[17] As often done, functions are viewed here as sets of tuples.

The fact that coding morphisms of solid codes preserve the computational complexity of constraint languages (Theorem 18.19) was first stated in [298, Thm. 17]. However, the connection to solid codes was not made in [298]. Instead, *strongly self-synchronizing codes* were introduced as comma-free codes C satisfying $\mathrm{Pref}(C) \setminus C)C \cap \Sigma^* C \Sigma^+ = \emptyset$. In the present work, we discovered that strongly self-synchronizing codes and solid codes are in fact identical, which follows from Theorems 17.16 and 17.53. For this reason we do not use the term strongly self-synchronizing codes anymore.[18] We note in passing that [298, Remark 2] giving a construction procedure for solid codes misses the condition $\Sigma^*\{c\} \cap X = \emptyset$, Theorem 17.14, Item 14 gives the correct formulation with a proof.

[18] Self-synchronizing code is another name for comma-free code, see Wikipedia [230, 639]. Since originally introduced as self-synchronizing codes satisfying an additional condition, we called this class strongly self-synchronizing codes; without being aware that this class equals the class of solid codes. We note that the term self-synchronizing does not seem to be used consistently in the literature to denote the same class of codes. For instance, in [207] codes admitting a synchronizing word for a canonical decoder (automaton) are called self-synchronizing, which gives an additional reason to avoid the term strongly self-synchronizing codes.

Chapter 19
Classes of Constraint Languages

Here, we collect results for different classes of constraint languages. For some classes we classify the computational complexity of the constrained synchronization problem.

1. In Section 19.1 we show that for infinite languages the problem is L-hard.
2. Section 19.2 investigates sparse languages. We show that the problem is in NP for sparse constraint languages, in AC^0 for the subclass of finite languages, and we state sufficient conditions for containment in L. we conclude with a full classification for the subclass of letter-bounded constraint languages, characterizing precisely when the problem is in AC^0, L-complete or NP-complete.
3. In Section 19.3 we investigate languages that are submonoids of Σ^*. We show that algorithmic results and bounds on shortest synchronizing words from the unconstrained setting can be transferred to this setting. In particular, for these constrained problems the problem is always in NL (characterizing NL-hardness in this case is an open problem). From this result, we also deduce in Section 19.3.3 that constraint languages accepted by strongly connected PDFAs yield constrained synchronization problems in NL.
4. Section 19.4 is concerned with small constraint PDFAs. Section 19.4.1 gives a complexity classification for two-state PDFAs that are not strongly connected. In Section 19.4.2, for three-state binary constraint PDFAs that are not strongly connected we give a full classification into problems in AC^0 and problems that are L-, NL-, NP-, or PSPACE-complete.
5. Lastly, Section 19.5 is concerned with commutative constraint languages. We give a complete classification, stating precisely the conditions for containment in AC^0, L-completeness, NL-completeness, NP-completeness, or PSPACE-completeness.

The complexity classifications for each class we give can all be used to give decision procedures to decide the complexity for a given constraint language from the class.

19.1 Infinite Languages

Theorem 19.1 *For every infinite regular $L \subseteq \Sigma^*$, $\textsc{Sync}[L]$ is L-hard even for $|\Sigma| = 1$.*

Proof. Let $L \subseteq \Sigma^*$ be infinite. Then there exists a letter $a \in \Sigma$ such that $\pi_{\{a\}}(L)$ is infinite. So $\mathrm{Fact}(\pi_{\{a\}}(L)) = a^*$. By Theorem 18.2, Item 2 and Theorem 18.5 we find that $\textsc{Sync}[\pi_{\{a\}}(L)]$ is L-complete. By Theorem 18.17, we have $\textsc{Sync}[\pi_{\{a\}}(L)] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L]$, which gives the claim. □

19.2 Sparse Languages

In Section 19.2.1, we show that for sparse languages, the constrained synchronization problem is always in NP. Then, in Section 19.2.2, we show that for finite languages, which are sparse, the problem is in AC^0. Section 19.2.3 collects sufficient conditions for containment in L. In Section 19.2.4, we conclude with a full classification for the subclass of letter-bounded constraint languages, characterizing precisely when the problem is in AC^0, L-complete or NP-complete. Recall that by Proposition 17.86 the sparse regular languages are precisely the bounded regular languages.

19.2.1 Containment in NP

Theorem 19.2 *For sparse regular $L \subseteq \Sigma^*$, the problem* $\textsc{Sync}[L]$ *is in* NP.

Proof. The sparse regular languages equal the bounded regular languages by Theorem 17.86. Thus, we have $L \subseteq w_1^* w_2^* \cdots w_k^*$ for some $w_1, \ldots, w_k \in \Sigma^*$. By Theorem 17.66, the constraint language L is a finite union of languages of the form $w_1^{i_1}(w_1^{p_1})^* \cdots w_k^{i_k}(w_k^{p_k})^*$ for numbers $i_1, \ldots, i_k, p_1, \ldots, p_k \geq 0$. By Theorem 18.9, we can assume $L = w_1^{i_1}(w_1^{p_1})^* \cdots w_k^{i_k}(w_k^{p_k})^*$. Set $u_r = w_i^{p_r}$ for $r \in \{1, \ldots, k\}$. Now, let $\mathcal{A} = (Q, \Sigma, \delta)$ be an input automaton. If there exists a synchronizing word for $\mathcal{A}$ in L, then for a shortest synchronizing word $w_1^{i_1} u_1^{j_1} \cdots w_k^{i_k} u_k^{j_k}$ in L we have $0 \leq j_r < 2^{|Q|} - 1$ for all $r \in \{1, \ldots, k\}$. For assume $j_r \geq 2^{|Q|} - 1$ for some $r \in \{1, \ldots, k\}$. By Theorem 6.73 applied to $S = \delta(Q, w_1^{i_1} u_1^{j_1} \cdots w_{r-1}^{i_{r-1}} u_{r-1}^{j_{r-1}} w_r^{i_r})$ there exists $0 \leq j_r' < 2^{|Q|} - 1$ such that $\delta(S, u_r^{j_r}) = \delta(S, u_r^{j_r'})$.[1] Then

$$w_1^{i_1} u_1^{j_1} \cdots w_{r-1}^{i_{r-1}} u_{r-1}^{j_{r-1}} w_r^{i_r} u_r^{j_r'} w_{r+1}^{i_{r+1}} u_{r+1}^{j_{r+1}} \cdots w_k^{i_k} u_k^{j_k}$$

is shorter, in L, and synchronizes $\mathcal{A}$.

By using a binary encoding, the indices $0 \leq j_r < 2^{|Q|}$, $r \in \{1, \ldots, k\}$ can be guesses in nondeterministic polynomial time. Having guessed $j_1, \ldots, j_k$ we have to verify that the corresponding word is a synchronizing word for $\mathcal{A}$ in polynomial time. For $r \in \{1, \ldots, k\}$, set $A_r = [u_r]_{\mathcal{A}}$ and $B_r = [w_r^{i_r}]_{\mathcal{A}}$. The product $A_i^{j_r}$ can be computed by a square-and-multiply algorithm in polynomial time. Then, compute $B_1 A_1^{j_1} B_2 A_2^{j_2} \cdots B_k A_k^{j_k}$ in polynomial time and check that the result contains precisely

[1] We set $u_1^{i_1} \cdots u_0^{j_0} = \varepsilon$.

one column consisting entirely of ones and zeros everywhere else, which is equivalent to the fact that the corresponding word synchronizes $\mathcal{A}$. □

In [296, Thm. 2], generalizing an idea from [215, Thm. 19], a proof was given for languages accepted by automata where each cycle only yields powers of a single word. These automata precisely accept the sparse regular languages, see [491, Thm. XII.4.29], and thus giving another proof for Theorem 19.2.

19.2.2 Finite Languages

Here, in Theorem 19.3, we prove that for finite $L \subseteq \Sigma^*$, which are obviously sparse, the problem $\textsc{Sync}[L]$ is in AC^0. In Theorem 19.4 we prove that removing finite sets from a constraint language does not lower the complexity if the resulting language is non-empty.

Theorem 19.3 *Let $L \subseteq \Sigma^*$ be finite. Then $\textsc{Sync}[L]$ is in AC^0 (and in constant time for $L = \emptyset$ or $L = \{\varepsilon\}$).*

Proof. An algorithm for $\textsc{Sync}[\emptyset]$ always returns NO, and for $\textsc{Sync}[\{\varepsilon\}]$ it returns YES if and only if the input automaton has a single state and NO otherwise; both algorithms run in constant time. Next, let $u \in \Sigma^*$ and consider the morphism $\varphi\colon \{a\}^* \to \Sigma^*$ given by $\varphi(a) = u$. By Theorem 18.17, we get $\textsc{Sync}[\{u\}] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[\{a\}]$, where the latter problem is in AC^0 by Theorem 18.2, Item 1. So, $\textsc{Sync}[\{u\}]$ for $u \in \Sigma^*$ is in AC^0. Then, for finite L, the claim follows from Theorem 18.9. □

More precisely, by closer inspection of the proof of Theorem 19.3 (and the proofs of the referenced statements), we obtain that finite sets can be handled inside of $\leq_m^{\mathsf{AC}^0}$-reductions, giving the following statement.

Proposition 19.4 *Let $L \subseteq \Sigma^*$ be regular and $F \subseteq \Sigma^*$ be finite. Then $\textsc{Sync}[L \cup F] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L]$.*

Proof. Looking into the proof of Theorem 19.3, checking whether a word from a finite set synchronizes an input automaton can be incorporated into an $\leq_m^{\mathsf{AC}^0}$-reduction. Specifically, the reduction first checks whether an input automaton has a synchronizing word in F and if it does, passes a YES-instance to $\textsc{Sync}[L]$, or otherwise passes the input to $\textsc{Sync}[L]$. □

Corollary 19.5 *For regular $L \subseteq \Sigma^*$ and finite $F \subseteq \Sigma^*$ we have $\textsc{Sync}[L] \leq_m^{\mathsf{AC}^0} \textsc{Sync}[L \setminus F]$.*

Proof. This follows from Theorem 19.4 since $L = (L \setminus F) \cup (L \cap F)$. □

19.2.3 Cases Contained in L

Here, we identify constraint languages (besides the finite languages) giving a constrained problem in L. Theorem 19.6 shows that Sync$[u^*vw^*]$ is in L, and Theorems 19.7 and 19.8 conclude this for languages in u^*wv^* and u^*Uv^* for finite U, respectively. Intuitively, the problem is in L when the only iterations of a single word are at the beginning or end of a synchronizing word. Thus, for constraint languages in u^*wv^* the constrained synchronization problem is either in AC^0 if the language is finite or the problem is L-complete.

Theorem 19.6 *Let $u, v, w \in \Sigma^*$. Then* Sync$[u^*vw^*]$ *is in* L.

Proof. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an input automaton for Sync$[u^*wv^*]$ with n states. By Theorem 6.73 for each $i \geq 0$ there exists $0 \leq j < n$ such that $\delta(Q, u^i v) = \delta(Q, u^j v)$. Set $S_j = \delta(Q, u^j v)$ for $j \in \{0, 1, \ldots, n-1\}$. By Theorem 6.74 for each $j \in \{0, 1, \ldots, n-1\}$ there exists $r \geq 0$ such that $|\delta(S_j, w^r)| = 1$ if and only if $|\delta(S_j, w^{n-1})| = 1$. So, there exists a synchronizing word for $\mathcal{A}$ in u^*vw^* if and only if there exists $j \in \{0, 1, \ldots, n-1\}$ such that $u^j v w^{n-1}$ synchronizes $\mathcal{A}$. Since the set of synchronizing words is an ideal (Theorem 7.1), this is equivalent to the fact that $u^{n-1} v w^{n-1}$ synchronizes $\mathcal{A}$. Hence we can use SyncWords to solve this problem. A deterministic log-space transducer can produce the output $(\mathcal{A}, u^{n-1} v w^{n-1})$ by copying $\mathcal{A}$ and using a counter to write u^{n-1}, appending v and then writing w^{n-1} (since u, v, and w are fixed they can be appended in constant time). Since, by Theorem 8.9, the problem SyncWords is in L, and, by Theorem 3.75, Item 2, L is closed for $\leq_m^{\log}$-reductions, the claim follows. □

Next, we state two corollaries of the previous result.

Corollary 19.7 *Let $L \subseteq \Sigma^*$ be regular such that $L \subseteq u^*vw^*$ for $u, v, w \in \Sigma^*$. Then* Sync$[L]$ *is in* L.

Proof. By Theorem 17.66 the language L is a finite union of languages of the form $u^i (u^p)^* v w^j (w^r)^*$ where $i, j, p, r \geq 0$. For $p = r = 0$, Sync$[u^i v w^j]$ is in AC^0 by Theorem 19.3 and for $p > 0$ or $r > 0$ we have

$$\mathrm{Fact}(u^i (u^p)^* v w^j (w^r)^*) = \begin{cases} \mathrm{Fact}(u^* v w^*) & \text{if } p, r > 0, \\ \mathrm{Fact}(u^i v w^*) & \text{if } p = 0, r > 0, \\ \mathrm{Fact}(u^* v w^j) & \text{if } p > 0, r = 0. \end{cases}$$

Then, by Theorem 18.5, Theorems 18.9 and 19.6 the claim follows. □

Corollary 19.8 *Let $L \subseteq \Sigma^*$ be regular and $L \subseteq \bigcup_{i=1}^{n} u_i^* v_i w_i^*$ for $u_i, v_i, w_i \in \Sigma^*$ with $i \in \{1, \ldots, n\}$. Then* Sync$[L]$ *is in* L. *In particular, for finite $U \subseteq \Sigma^*$ and $u, v \in \Sigma^*$ such that $L \subseteq u^*Uv^*$,* Sync$[L]$ *is in* L.

Proof. We have

$$L = \bigcup_{i=1}^{n} (L \cap u_i^* v_i w_i^*).$$

For $i \in \{1, \ldots, n\}$, since $L \cap u_i^* v_i w_i^* \subseteq u_i^* v_i w_i^*$ and $L \cap u_i^* v_i w_i^*$ is regular, by Theorem 19.7 the problem $\textsc{Sync}[L \cap u_i^* v_i w_i^*]$ is in L. By Theorem 18.9, it follows that $\textsc{Sync}[L]$ is in L. Furthermore, since $u^*Uv^* = \bigcup_{w \in U} u^* w v^*$, the claim follows for $L \subseteq u^*Uv^*$. □

Corollary 19.9 *Let $A_j \subseteq \{a_j\}^*$ be unary regular languages for $j \in \{1, \ldots, k\}$. If for all $j \in \{1, \ldots, k\}$, A_j infinite implies that $A_i \subseteq \{a_j\}^*$ for all $i < j$ or $A_i \subseteq \{a_j\}^*$ for all $i > j$ (or both), then $\textsc{Sync}[A_1 \cdots A_k]$ is in L.*

Proof. Set $L = A_1 \cdots A_k$. Let $i, j \in \{1, \ldots, k\}$ be the maximal and minimal indices such that $a_1 = \cdots = a_i$ and $a_j = \cdots = a_k$, respectively. If $i \leq j$, then $L \subseteq a_i^* U a_j^*$ where $U = \{\varepsilon\}$ for $i = j$ and $U = A_{i+1} \cdots A_{j-1}$ for $i < j$. The language U is finite since if A_r would be infinite for some $r \in \{i+1, \ldots, j-1\}$ with $i < j$, then either $a_i = \cdots = a_r$ or $a_r = \cdots = a_j$ by assumption, contradicting the choice of i and j. Thus, by Theorem 19.8, $\textsc{Sync}[L]$ is in L. Now, suppose, $j < i$. Then we have $a_i = a_j$, and thus, with $a_1 = \cdots = a_i$ and $a_j = \cdots = a_r$, we find $a_1 = \cdots = a_k$ and so $L \subseteq a_1^*$. Then, by Theorem 19.6, $\textsc{Sync}[L]$, is in L. □

19.2.4 Trichotomy Result for Letter-Bounded Languages

Here, in Theorem 19.10, we show that for letter-bounded regular constraint languages, the constrained synchronization problem is either in AC^0, L-complete, or NP-complete.

Theorem 19.10 (Trichotomy Theorem) *Let $L \subseteq \Sigma^*$ be letter-bounded and regular. Then the problem $\textsc{Sync}[L]$ is*

1. *NP-complete if there exist $a, b, c \in \Sigma$ where $a, c \in \Sigma \setminus \{b\}$ and $u \in \Sigma^* a \Sigma^*$, $v \in \Sigma^* c \Sigma^*$ such that $ub^*v \cap L$ is infinite,*
2. *L-complete if the previous condition is not fulfilled and L is infinite,*
3. *in AC^0 if L is finite.*

Proof. First, assume Item 1 holds. Since L is letter-bounded, $\rho(u)$ is bounded for each $u \in L$ by Theorem 17.69. Since L is regular, it has a finite number n of quotients by Theorem 6.87. Thus pick $u, v \in \Sigma^*$ such that $uab^j cv \in L$ for some $j \geq n$ and $\rho(uab^j cv)$ is maximal with $a, b, c \in \Sigma$ as in the statement. By finiteness, $(uab^r)^{-1}L = (uab^s)^{-1}L$ for some $0 \leq r < s \leq j$. So, $uab^j av = uab^{r+j-r}av \in L$ gives $uab^{s+j-r}av \in L$. Inductively, we obtain $uab^j(b^{s-r})^* cv \subseteq L$. Set $p = s - r$ and $i = j + (|u| + |v| + 1) \cdot p$. Thus $uab^i(b^p)^* cv \subseteq L$ and since $\rho(uab^i cv)$ is maximal we have

$$\Sigma^* uab^i (b^p)^* cv \Sigma^* \cap L \subseteq x^* uab^i (b^p)^* cv y^*$$

for some $x, y \in \Sigma \cup \{\varepsilon\}$ such that $u = \varepsilon$ with $x \in \{a, \varepsilon\}$ or $u \in x\Sigma^*$, or $v = \varepsilon$ with $y \in \{c, \varepsilon\}$ or $v \in \Sigma^* y$. By Theorems 18.12 and 18.98 we find that $\textsc{Sync}[L]$ is NP-hard.

Next, assume Item 1 does not hold. We have $L \subseteq a_1^* \cdots a_k^*$ for some $a_1, \ldots, a_k \in \Sigma$. Since L is regular, by Theorem 17.65, it is a finite union of sets of the form

$$A_1 \cdots A_k$$

with $A_i \subseteq a_i^*$ being regular for $i \in \{1, \ldots, k\}$. Since the condition from Item 1 does not hold, if A_j is infinite for some $j \in \{1, \ldots, k\}$ in a set of the above form, then we must have $A_i \subseteq \{a_j\}^*$ for all $i < j$ or $A_i \subseteq \{a_j\}^*$ for all $i > j$. By Theorem 19.9, we get that $\textsc{Sync}[A_1 \cdots A_k]$ is in L. Thus, Theorem 18.9 yields that $\textsc{Sync}[L]$ is in L. If L is infinite, this problem is also L-hard by Theorem 19.1.

Lastly, if L is finite, then $\textsc{Sync}[L]$ is in AC^0 by Theorem 19.3. □

19.3 Monoid Languages

In Section 19.3.1, we generalize the pair criterion (Theorem 7.18) to constraint languages that are monoids. Then, in Section 19.3.2, we derive algorithmic consequences for testing constrained synchronizability and for the computation of synchronizing words of at most cubic length, as well as sufficient conditions for containment in NL.

19.3.1 A Pair Criterion

Theorem 19.11 generalizes the pair criterion (Theorem 7.18) to the case of monoid constraint languages. Further, in Theorem 19.13, utilizing the pair automaton, we give a quadratic upper bound for shortest words in any constraint language, even those that are not monoids, mapping two states to a common state (we also give time and space bounds for computing such a word).

Theorem 19.11 *Suppose $L \subseteq \Sigma^*$ is a monoid language. An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ has a synchronizing word in L if and only if for all $p, q \in Q$ there exists a word $u \in L$ such that $\delta(p, u) = \delta(q, u)$.*

Proof. Set $n = |Q|$. A synchronizing word in L maps all pairs of states to the same state. Conversely, assume all pairs of states can be mapped to a single state by a word in L. Then, set $Q_1 = Q$ and, for $i \in \{1, \ldots, n-1\}$, $Q_{i+1} = \delta(Q_i, u_i)$ where, if $|Q_i| > 1$, let $u_i \in L$ be such that $\delta(p, u_i) = \delta(q, u_i)$ for two distinct $p, q \in Q_i$, and $u_i = \varepsilon$ otherwise. Since $|Q_{i+1}| < |Q_i|$ for $|Q_i| > 1$ and $|Q_1| = n$, there exists $j \in \{1, \ldots, n-1\}$ such that $|Q_j| = 1$. Set $u = u_1 \cdots u_j$. We have $u \in \mathrm{Syn}(\mathcal{A})$. Since L is a monoid we get $u \in L$. □

Remark 19.12 We opted to present a direct proof for Theorem 19.11 above. Another proof along the lines as in the unconstrained case presented in Section 7.1.5 is possible by (1) first showing that a trap-state automaton has a synchronizing word in a monoid language L if and only if there exists for each state a word in L mapping this state to the trap state and then (2) using that an arbitrary automaton has the same set of synchronizing words as the pair automaton in which the singleton sets are merged into a trap state.

Proposition 19.13 *Suppose $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ is a PDFA. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton and $p, q \in Q$. In time $O(|P|n^2 + |\Sigma||P|n^2)$ and working space $O(|\Sigma||P|n^2)$ we can test whether there exists $u \in L(\mathcal{B})$ such that $\delta(p, u) = \delta(q, u)$ and, if this is the case, compute a shortest u, where we have*

$$|u| < \binom{n}{2} \cdot |P| + |P|.$$

Proof. Consider the product automaton $(\mathcal{A}^{\leq 2}/{\sim_S}) \times \mathcal{B}$ where $S = \{\{q\} \mid q \in Q\}$. A shortest word in $u \in L(\mathcal{B})$ mapping two states $p, q \in Q$ to a single state corresponds to a path from $(\{p, q\}, p_0)$ to a state (S, r) with $r \in F$. Since $(\mathcal{A}^{\leq 2}/{\sim_S}) \times \mathcal{B}$ has $(\binom{n}{2} + 1) \cdot |P|$ states, we obtain $|u| < \binom{n}{2} \cdot |P| + |P|$. Now, $(\mathcal{A}^{\leq 2}/{\sim_S}) \times \mathcal{B}$ with $|P| \cdot (\binom{n}{2} + 1)$ states and at most $|\Sigma||P|(\binom{n}{2} + 1)$ transitions can be computed in time and space $O(|\Sigma||P|n^2)$. Then, existence of u, and its computation in case it exists, can be done by a breadth-first search [163] in time $O(|P|n^2 + |\Sigma||P|n^2)$ and space $O(|P|n^2)$. □

19.3.2 Computing Synchronizing Words and Containment in NL

Theorem 19.14 gives a cubic bound on shortest synchronizing words in constraint languages that are monoids and time and space upper bound to compute such a word. The algorithm is based on the method proofs used for Theorems 19.11 and 19.13. Then, as a consequence of Theorems 19.11 and 19.13, in Theorem 19.15 we conclude that for monoid constraint languages, the constrained synchronization problem is in NL.

Theorem 19.14 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA and $L = L(\mathcal{B})$ be a monoid. Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is an n-state automaton. In time $O(|\Sigma||P|n^2)$ we can test whether $\mathcal{A}$ has a synchronizing word in L. Testing whether $\mathcal{A}$ has a synchronizing word and, if this is the case, computing one of length at most $(n-1) \cdot (\binom{n}{2} \cdot |P| + |P| - 1)$ can be done in time $O(|\Sigma||P|n^3)$ and working space $O(|\Sigma||P|n^2)$ (not counting the space the output needs).*

Proof. To test whether the automaton $\mathcal{A}$ has a synchronizing word in $L(\mathcal{B})$, we construct $(\mathcal{A}^{\leq 2}/{\sim_S}) \times \mathcal{B}$ and check whether, from every state (p, q, p_0) with $p, q \in Q$, a state in $S \times F$ is reachable. This can be done using a breadth-first search that runs "backward" starting from the states in $S \times F$. The algorithm runs in time $O(|\Sigma||P|n^2)$ and is correct by Theorem 19.11.

Next, to give an algorithms that also computes a synchronizing word, first consider the proof of Theorem 19.11. We can compute the sets Q_i using a loop that iterates at most $n-1$ times. By Theorem 19.13, in each iteration, for $|Q_i| > 1$, we can test whether a word u_i with $|\delta(Q_i, u_i)| < |Q_i|$ exists in time $O(|P|n^2 + |\Sigma||P|n^2)$ and working space $O(|\Sigma||P|n^2)$, and, moreover, compute a shortest u_i where

$$|u_i| \leq \binom{n}{2} \cdot |P| + |P| - 1.$$

If there exists no such word in some iteration, then, by Theorem 19.11, there exists no synchronizing word in L. Otherwise, concatenating the u_i gives a synchronizing word in L of length at most $(n-1) \cdot (\binom{n}{2} \cdot |P| + |P| - 1)$. □

The algorithm given in the proof of Theorem 19.14 is an adaption of a similar algorithm for the unconstrained case [198, 548, 613].

Theorem 19.15 *Let $L \subseteq \Sigma^*$ be a regular language that is a monoid.*[2] *Then* Sync$[L]$ *is in* NL.

Proof. Suppose $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ is a PDFA with $L = L(\mathcal{B})$. Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $n = |Q|$. By Theorems 19.11 and 19.13, the problem can be decided in NL by cycling through all pairs of states $p, q \in Q$ and nondeterministically guess letter-by-letter $u \in \Sigma^*$ with $|u| \leq |P| \cdot (\binom{n}{2} + 1)$, $u \in L$, and $\delta(p, u) = \delta(q, u)$. If for all state pairs an appropriate word is found, the automaton admits a synchronizing word in L, if not, we reject. □

Combining Theorem 19.15 and 17.75, we get the next corollary.

Corollary 19.16 *For $\mathcal{B} = (P, \Sigma, \mu, p_0, \{p_0\})$,* Sync$[L(\mathcal{B})]$ *is in* NL.

Next, we derive a sufficient condition on the concatenation with monoid languages resulting in problems in NL.

Corollary 19.17 (UVW-Condition) *Let $U, V, W \subseteq \Sigma^*$ be such that $U \subseteq$ Suff (V), $W \subseteq$ Pref(V) and V is a regular language that is a monoid. Then* Sync$[UVW] \equiv_m^{\mathsf{AC}^0}$ Sync$[V]$ *and* Sync$[UVW]$ *is in* NL.

Proof. As $V = V^*$, the assumptions give Fact$(UVW) =$ Fact(V). Then Corollary 18.5 and Theorem 19.15 yield the claim. □

The property that V is a monoid is essential in Theorem 19.17. For example, as listed in Theorem 18.2, Sync$[ab^*] \in$ L, but Sync$[ab^*a]$ is NP-complete.

Lastly, by Theorem 17.84, a sparse language that is a monoid is either an infinite subset of u^* for some $u \in \Sigma^*$ or equals $\{\varepsilon\}$. The language $\{\varepsilon\}$ is the only finite monoid language (and so, for this language an input semiautoamton has a synchronizing word if and only if it has a single state). Thus, the constrained synchronization problem with such a constraint language is either solvable in constant time or L-complete by Theorems 19.1 and 19.7.

19.3.3 Strongly Connected Constraint Automata

Theorem 19.18 yields with Theorem 17.75 that the the computational complexities arising from strongly connected constraint PDFAs equal those arising from constraint

[2] We avoided to call it a regular monoid, as the notion of regularity is also used in semigroup theory with a different meaning (see, for example, [325]).

languages of the form C^* (which are monoids) for $C = \emptyset$ or a prefix code C that is a regular language. (Recall that these languages equal those of the form $L(\mathcal{B}_{p_0,\{p_0\}})$ by Theorem 17.75.) Theorem 19.19 shows that the constrained synchronization problem is always in NL for strongly connected constraint automata. As a special case, in Theorem 19.20 we classify the complexity for one-state constraint automata.

Observe that for a strongly connected PDFA $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$, $L(\mathcal{B})$ is finite if and only if $L(\mathcal{B}) = \emptyset$ (equivalently $F = \emptyset$) or $L(\mathcal{B}) = \{\varepsilon\}$ (equivalently $P = \{p_0\}$ and we have no outgoing transitions at p_0).

Theorem 19.18 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a strongly connected PDFA with $F \neq \emptyset$. Then* $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[C^*]$ *where C is some prefix code $C \subseteq \Sigma^+$ that is regular or we have $C = \emptyset$.*

Proof. A strongly connected PDFA is trim and every state is accessible and coaccessible. Thus, by Theorems 18.5 and 18.99, we find[3]

$$\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[L(\mathcal{B}_{p_0,\{p_0\}})].$$

Let $C \subseteq \Sigma^+$ be the set of shortest non-empty words $u \in \Sigma^+$ such that $\mu(p_0, u) = p_0$. Then $C = \emptyset$ (if p_0 has no outgoing transitions) or C is a prefix code. We then have $L(\mathcal{B}_{p_0,\{p_0\}}) = C^*$.

For a direct proof, first note that we have $L(\mathcal{B}_{p_0,\{p_0\}}) \subseteq \mathrm{Pref}(L(\mathcal{B}))$ since for $u \in \Sigma^*$ with $\mu(p_0, u) = p_0$ we have $\mu(p_0, uv) \in F$ for some $v \in \Sigma^*$. We also have $L(\mathcal{B}) \subseteq \mathrm{Pref}(L(\mathcal{B}_{p_0,\{p_0\}}))$ since for $u \in L(\mathcal{B})$ we have $\mu(p_0, uv) = p_0$ for some $v \in \Sigma^*$. This gives $\mathrm{Pref}(L(\mathcal{B})) = \mathrm{Pref}(L(\mathcal{B}_{p_0,\{p_0\}}))$. Then proceed as above. □

Note that C^* is sparse if and only if $|C| \leq 1$ for a prefix code $C \subseteq \Sigma^+$ by Theorems 17.20 and 17.83.

Corollary 19.19 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a strongly connected PDFA. Then the problem* $\textsc{Sync}[L(\mathcal{B})]$ *is in* NL*, and we have $L(\mathcal{B}) = \{\varepsilon\}$ (and so* $\textsc{Sync}[L(\mathcal{B})]$ *is solvable in constant time) or $L(\mathcal{B})$ is infinite and thus* $\textsc{Sync}[L(\mathcal{B})]$ *is* L*-complete if $L(\mathcal{B})$ is also sparse.*

Proof. By Theorem 19.18 we have $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[C^*]$ for $C = \emptyset$ or some prefix code $C \subseteq \Sigma^+$. We have $C = \emptyset$ precisely if $L(\mathcal{B}) = \{\varepsilon\}$. So, suppose $C \neq \emptyset$ in the following. By Theorem 19.15 (or Theorem 19.16 since C^* is acceptable by a PDFA with a single final state equal to its start state), we find that $\textsc{Sync}[L(\mathcal{B})]$ is in NL. For $C \neq \emptyset$, from the method of proof of Theorem 19.18 and construction of C, we find that $\textsc{Sync}[L(\mathcal{B})]$ is infinite (we also find that it is sparse if $C = \{u\}$ for some $u \in \Sigma^*$). □

Showing NL-hardness for general regular monoid languages of exponential growth is an *open problem*, even for those of the form C^* for a prefix code C that is regular and contains at least two words.

[3] For language of the form $L(\mathcal{B}_{p_0,\{p_0\}})$ by discarding useless states, i.e., assming $\mathcal{B}$ to be trim, we can assume that $\mathcal{B}$ is strongly connected.

Corollary 19.20 *Let $\mathcal{B} = (\{p_0\}, \Sigma, \mu, p_0, \{p_0\})$ be a one-state PDFA and $\Gamma = \{a \in \Sigma \mid \mu(p_0, a) = p_0\}$. Then $L(\mathcal{B}) = \Gamma^*$ and $\textsc{Sync}[L(\mathcal{B})]$ has the following complexities:*

$$\begin{cases} \textit{constant time} & \textit{if } \Gamma = \emptyset, \\ \mathsf{L}\textit{-complete} & \textit{if } |\Gamma| = 1, \\ \mathsf{NL}\textit{-complete} & \textit{if } |\Gamma| \geq 2. \end{cases}$$

Proof. By Theorem 17.88, we can assume $\Sigma = \Gamma$. By Theorem 19.19 the claim follows for $|\Gamma| \leq 1$. For $|\Gamma| \geq 2$, the claim follows from Theorem 18.2, Item 3. □

19.4 Small Constraint Automata

Here, we investigate constraint PDFAs $\mathcal{B} = (P, \Sigma, \mu, 1, F)$ where the state set is $P = \{1, 2, \ldots, n\}$ for $n \in \{2, 3\}$. Let $\Sigma_{i,j} = \{x \in \Sigma \mid \mu(i, x) = j\}$ w.r.t. $\mathcal{B}$ for $i, j \in \{1, 2, \ldots, n\}$. Note that, by determinism of $\mathcal{B}$, we have $\Sigma_{i,j} \cap \Sigma_{i,k} = \emptyset$ for $j \neq k$ where $i, j, k \in \{1, \ldots, n\}$.

First, in Section 19.4.1, we investigate the case $P = \{1, 2\}$ for an arbitrary alphabet and then, in Section 19.4.2, the case $P = \{1, 2, 3\}$ for $\Sigma = \{a, b\}$.

By choice of notation, 1 is the start state. We assume that $\mathcal{B}$ is trim. The case that $\mathcal{B}$ is strongly connected (including the case $P = \{1\}$, see Theorem 19.20) was already handled in Section 19.3.3 for general automata. So, we also assume that $\mathcal{B}$ is not strongly connected in the following.

19.4.1 Two-State Constraint PDFAs

Theorem 19.21 *Suppose $P = \{1, 2\}$, $\mathcal{B}$ is trim with start state 1 and not strongly connected. Then we have $\Sigma_{1,2} \neq \emptyset$ and $\Sigma_{2,1} = \emptyset$. In this case, by trimness we have $2 \in F$ and by Theorems 18.5 and 18.99 we can assume $F = \{2\}$ since 1 is coaccessible for this final state set. Under these assumptions, $L(\mathcal{B}) = \Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^*$ and $\textsc{Sync}[L(\mathcal{B})]$ has the following complexities:*

$$\begin{cases} \mathsf{PSPACE}\textit{-complete} & \textit{if } (\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset \textit{ and } \max\{|\Sigma_{1,1}|, |\Sigma_{2,2}|\} \geq 2, \\ \mathsf{NL}\textit{-complete} & \textit{if } \Sigma_{1,1} \cup \Sigma_{1,2} \subseteq \Sigma_{2,2} \textit{ and } |\Sigma_{2,2}| \geq 2, \\ \mathsf{L}\textit{-complete} & \textit{if } \max\{|\Sigma_{1,1}|, |\Sigma_{2,2}|\} = 1, \\ \textit{in } \mathsf{AC}^0 & \textit{if } \Sigma_{1,1} = \Sigma_{2,2} = \emptyset. \end{cases}$$

Proof. Under the assumptions from the statement, clearly we have $L(\mathcal{B}) = \Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^*$. First, suppose

$$(\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset \text{ and } \max\{|\Sigma_{1,1}|, |\Sigma_{2,2}|\} \geq 2.$$

Assume $|\Sigma_{1,1}| \geq 2$. Then we can pick distinct $a, b \in \Sigma_{1,1}$ and $c \in \Sigma_{1,2}$ such that $|\{a, b, c\} \cap \Sigma_{2,2}| \leq 2$. We have $\pi_{\{a,b,c\}}(\Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^*) = (a + b)^* c \Gamma^*$ for some $\Gamma \subseteq \{a, b, c\}$ with $|\Gamma| \leq 2$. By Theorem 18.2, Item 5 and Theorem 18.17 we find that $\textsc{Sync}[L(\mathcal{B})]$ is PSPACE-hard. Next, suppose $|\Sigma_{1,1}| \leq 1$ (so that $|\Sigma_{2,2}| \geq 2$). Set

$$L = \begin{cases} \Sigma_{1,2} \Sigma_{2,2}^* & \text{if } \Sigma_{1,2} \subseteq \Sigma_{2,2}, \\ \Sigma_{2,2}^* & \text{otherwise.} \end{cases}$$

By assumption, we can pick $a \in \Sigma$ such that $a \in \Sigma_{1,1} \setminus \Sigma_{2,2}$ if $\Sigma_{1,2} \subseteq \Sigma_{2,2}$ or $a \in \Sigma_{1,2} \setminus \Sigma_{2,2}$ otherwise. For each $v \in \mathrm{Pref}(\Sigma_{2,2}) \setminus \Sigma_{2,2} = \{\varepsilon\}$ the word va is weakly auxiliary state avoiding w.r.t. $\Sigma_{2,2}$. We can pick two distinct $v_1, v_2 \in L$ and we have $\{v_1, v_2\}^* \subseteq L \subseteq \Sigma_{2,2}^*$. Note that $a \in \Sigma_{1,1}$ implies $\Sigma_{1,1} = \{a\}$ by assumption. By Theorem 18.86, we find that $\textsc{Sync}[a^* aL]$ is PSPACE-hard for $\Sigma_{1,1} = \{a\}$ and that $\textsc{Sync}[aL]$ is PSPACE-hard otherwise. In the latter case, we have $\Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^* \cap \Sigma^* a \Sigma^* = \Sigma_{1,1}^* a \Sigma_{2,2}^*$ where $L = \Sigma_{2,2}^*$. In this case, since $|\Sigma_{1,1}| \leq 1$ and $a \notin \Sigma_{1,1}$ the problem $\textsc{Sync}[\Sigma_{1,1}^* aL]$ is PSPACE-hard by Theorem 18.27, Item 2 and so $\textsc{Sync}[L(\mathcal{B})]$ is PSPACE-hard by Theorem 18.12. Containment in PSPACE follows from Theorem 18.1.

Next, suppose $\Sigma_{1,1} \cup \Sigma_{1,2} \subseteq \Sigma_{2,2}$ with $|\Sigma_{2,2}| \geq 2$. Then $\mathrm{Fact}(L(\mathcal{B})) = \mathrm{Fact}(\Sigma_{2,2}^*)$. By Theorem 18.5, $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\Sigma_{2,2}^*]$, and the latter problem is NL-complete by Theorem 19.20. Now, suppose $\max\{|\Sigma_{1,1}|, |\Sigma_{2,2}|\} = 1$. Then, by Theorem 19.1 and Theorem 19.6, the problem is L-complete. Finally, for $\Sigma_{1,1} = \Sigma_{2,2} = \emptyset$, $L(\mathcal{B})$ is finite and containment in AC^0 follows from Theorem 19.3. □

Corollary 19.22 *For a two-state constraint automaton $\mathcal{B}$ over a binary alphabet,* $\textsc{Sync}[L(\mathcal{B})]$ *is in* NL.

Proof. We can assume that $\mathcal{B}$ is trim with start state 1. Then we have $\Sigma_{1,2} \neq \emptyset$. So, as $\Sigma_{1,1} \cap \Sigma_{1,2} = \emptyset$, we have $|\Sigma_{1,1}| \leq 1$. Then, for $|\Sigma_{2,2}| \leq 1$, by Theorem 19.21 the problem is in L. For $|\Sigma_{2,2}| = 2$, we have $\Sigma_{1,1} \cup \Sigma_{1,2} \subseteq \Sigma_{2,2}$ and the problem is in NL by Theorem 19.21. □

19.4.2 Three-State Constraint PDFAs Over a Binary Alphabet

Here, we prove the complexity classification for $P = \{1, 2, 3\}$ with $\Sigma = \{a, b\}$. Broadly, we structure the proof by distinguishing the different cases for the strongly connected components of $\mathcal{B}$.

Theorem 19.23 *Suppose $P = \{1, 2, 3\}$, $\mathcal{B}$ is not strongly connected and $\Sigma = \{a, b\}$. Let $\mathcal{B}$ be trim with start state* 1. *Then we have $\Sigma_{1,2} \cup \Sigma_{1,3} \neq \emptyset$ (and so $|\Sigma_{1,1}| \leq 1$ as $\mathcal{B}$ is deterministic). If $L(\mathcal{B})$ is finite, then* $\textsc{Sync}[L(\mathcal{B})]$ *is in* AC^0 *(Theorem 19.3). So, we assume $L(\mathcal{B})$ is infinite in the following.*

1. *Suppose $\{1\}, \{2\}, \{3\}$ are the strongly connected components. Then we must have $\Sigma_{2,3} = \emptyset$ or $\Sigma_{3,2} = \emptyset$. Without loss of generality, assume $\Sigma_{3,2} = \emptyset$. Then the following complexities arise (recall that $\mathcal{B}$ is trim):*

$\max\{\lvert\Sigma_{2,2}\rvert, \lvert\Sigma_{3,3}\rvert\} = 2$	NL-*complete*
$\max\{\lvert\Sigma_{2,2}\rvert, \lvert\Sigma_{3,3}\rvert\} \leq 1$ $\wedge \lvert\Sigma_{2,2}\rvert = 1 \wedge \Sigma_{2,3} \neq \emptyset \wedge (\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset$	NP-*complete*
$\max\{\lvert\Sigma_{2,2}\rvert, \lvert\Sigma_{3,3}\rvert\} \leq 1$ $\wedge(\Sigma_{2,2} = \emptyset \vee \Sigma_{2,3} = \emptyset \vee (\Sigma_{1,1} \cup \Sigma_{1,2}) \subseteq \Sigma_{2,2})$	L-*complete*

2. *Suppose* $\{1, 2\}$ *is a strongly connected component, or equivalently* $\Sigma_{1,2}$ *and* $\Sigma_{2,1}$ *are both non-empty. Then the following complexities arise:*

$\Sigma_{3,3} = \{a, b\}$	NL-*complete*
$\lvert\Sigma_{3,3}\rvert \leq 1$, $\Sigma_{1,1} = \Sigma_{2,2} = \emptyset$, $\lvert\Sigma_{1,2}\rvert = \lvert\Sigma_{2,1}\rvert = 1$	L-*complete*
All other cases (for the stated assumptions)	PSPACE-*complete*

3. *Suppose* $\{2, 3\}$ *is a strongly connected component, or equivalently* $\Sigma_{2,3}$ *and* $\Sigma_{3,2}$ *are both non-empty. Then the following complexities arise:*

$\lvert\Sigma_{2,3}\rvert + \lvert\Sigma_{2,2}\rvert + \lvert\Sigma_{3,2}\rvert + \lvert\Sigma_{3,3}\rvert = 2$	L-*complete*
$\lvert\Sigma_{2,3}\rvert + \lvert\Sigma_{2,2}\rvert + \lvert\Sigma_{3,2}\rvert + \lvert\Sigma_{3,3}\rvert = 4$	NL-*complete*
$\lvert\Sigma_{2,3}\rvert + \lvert\Sigma_{2,2}\rvert + \lvert\Sigma_{3,2}\rvert + \lvert\Sigma_{3,3}\rvert = 3$	*All possible constraint automata, up to symmetry, and their complexities are listed in Table 19.1 and 19.2.*

Proof. For $\max\{\lvert\Sigma_{2,2}\rvert, \lvert\Sigma_{3,3}\rvert\} = 2$, $\mathcal{B}$ has a trap state. Then, by Theorem 18.104, Item 4, we get that $\textsc{Sync}[L(\mathcal{B})]$ is NL-complete. So, for the remaining proof, we assume $\max\{\lvert\Sigma_{2,2}\rvert, \lvert\Sigma_{3,3}\rvert\} \leq 1$. Also, we assume that $L(\mathcal{B})$ is infinite, which gives L-hardness by Theorem 19.1, since the cases when $L(\mathcal{B})$ is finite are easy to identify, given by trim PDFAs without any loops.

$\{1\}$, $\{2\}$, $\{3\}$ are the Strongly Connected Components. Without loss of generality, suppose $\Sigma_{3,2} = \emptyset$. As $\mathcal{B}$ is trim, the states 2 and 3 are reachable from 1, i.e., $\Sigma_{1,2} \neq \emptyset \wedge (\Sigma_{1,3} \neq \emptyset \vee \Sigma_{2,3} \neq \emptyset)$, which implies $\lvert\Sigma_{1,1}\rvert \leq 1$. Then, by Theorem 18.100 we can assume $F = \{2, 3\}$, so that

$$L(\mathcal{B}) = \Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^* \Sigma_{2,3} \Sigma_{3,3}^* \cup \Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^* \cup \Sigma_{1,1}^* \Sigma_{1,3} \Sigma_{3,3}^*. \tag{19.1}$$

Hence $L(\mathcal{B}) \subseteq \Sigma_{1,1}^* \Sigma_{1,2}^* \Sigma_{1,3}^* \Sigma_{2,2}^* \Sigma_{2,3}^* \Sigma_{3,3}^*$. As $\max\{\lvert\Sigma_{2,2}\rvert, \lvert\Sigma_{3,3}\rvert\} \leq 1$ the language $L(\mathcal{B})$ is letter-bounded. Now, suppose $(\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset$ and there exist $b, c \in \Sigma$ such that $b \in \Sigma_{2,2}$ and $c \in \Sigma_{2,3}$. Then $ub^*c \cap L(\mathcal{B})$ is infinite for some $u \in \Sigma_{1,1}\Sigma_{1,2}$ with $u \notin b^*$. By Theorem 19.10 the problem $\textsc{Sync}[L(\mathcal{B})]$ is NP-complete. Otherwise, we have $\mathrm{Fact}(L(\mathcal{B})) = \mathrm{Fact}(L)$ where (for non-mutually exclusive cases)

$$L = \begin{cases} \Sigma_{2,2}^* \Sigma_{2,3} \Sigma_{3,3}^* \cup \Sigma_{2,2}^* \cup \Sigma_{1,1}^* \Sigma_{1,3} \Sigma_{3,3}^* & \text{if } \Sigma_{1,1} \cup \Sigma_{1,2} \subseteq \Sigma_{2,2}, \\ \Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,3} \Sigma_{3,3}^* \cup \Sigma_{1,1}^* \Sigma_{1,2} \cup \Sigma_{1,1}^* \Sigma_{1,3} \Sigma_{3,3}^* & \text{if } \Sigma_{2,2} = \emptyset, \\ \Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^* \cup \Sigma_{1,1}^* \Sigma_{1,2} \cup \Sigma_{1,1}^* \Sigma_{1,3} \Sigma_{3,3}^* & \text{if } \Sigma_{2,3} = \emptyset. \end{cases}$$

As $\max\{\lvert\Sigma_{1,1}\rvert, \lvert\Sigma_{2,2}\rvert, \lvert\Sigma_{3,3}\rvert\} \leq 1$, for the listed cases L equals a finite union of languages of the form $u^* v w^*$ for $u, v, w \in \Sigma^*$. By Corollary 18.5, Theorems 18.9 and 19.6 it follows that $\textsc{Sync}[L(\mathcal{B})]$ is in L.

$\{1,2\}$ **is a Strongly Connected Component.** In this case, since $\{1,2\}$ contains the start state and $\mathcal{B}$ is trim, we must have $3 \in F$ and that 3 is reachable from every other state. Thus, by Theorem 18.100, Item 2 we can assume $F = \{3\}$. Without loss of generality, suppose $\Sigma_{1,3} = \{a\}$ and $\Sigma_{1,2} = \{b\}$, as for $\Sigma_{1,3} = \emptyset$ it is $\Sigma_{2,3} \neq \emptyset$ and, by Theorem 18.99, we can consider 2 as the start state. Then $\Sigma_{1,1} = \emptyset$ and

$$L(\mathcal{B}) = \begin{cases} (by)^*(a + b\Sigma_{2,3})\Sigma_{3,3}^* & \text{if } \Sigma_{2,1} = \{y\} \text{ and } \Sigma_{2,2} = \emptyset, \\ (bb + ba)^* a\Sigma_{3,3}^* & \text{if } \Sigma_{2,1} = \{a,b\}, \\ (ba^*b)^* a\Sigma_{3,3}^* & \text{if } \Sigma_{2,2} = \{a\} \text{ and } \Sigma_{2,1} = \{b\}, \\ (bb^*a)^* a\Sigma_{3,3}^* & \text{if } \Sigma_{2,2} = \{b\} \text{ and } \Sigma_{2,1} = \{a\}. \end{cases}$$

As $|\Sigma_{3,3}| \leq 1$ by assumption, with Theorem 19.8 we find that $\textsc{Sync}[L(\mathcal{B})]$ is in L. For the remaining cases, as $\mathrm{Fact}((ba^*b)a\Sigma_{3,3}^*) = \mathrm{Fact}((a + bb)ba\Sigma_{3,3}^*)$ and $\mathrm{Fact}((bb^*a)a\Sigma_{3,3}^*) = \mathrm{Fact}((b+ab)aa\Sigma_{3,3}^*)$, PSPACE-completeness follows from Theorem 18.93, 8,9 & 10.

$\{2,3\}$ **is a Strongly Connected Component.** Since $\mathcal{B}$ is trim with start state 1, we have $F \cap \{2,3\} \neq \emptyset$. Thus, by Items 1 and 2 of Theorem 18.100 we can suppose $F = \{3\}$. Now, first, let $|\Sigma_{2,3}| + |\Sigma_{2,2}| + |\Sigma_{3,2}| + |\Sigma_{3,3}| = 2$. Then $\Sigma_{1,1} = \Sigma_{2,2} = \emptyset$ and $\Sigma_{2,3} = \{x\}, \Sigma_{3,2} = \{y\}$ for some $x, y \in \Sigma$, which gives $L(\mathcal{B}) = \Sigma_{1,1}^*\Sigma_{1,2}(xy)^* \cup \Sigma_{1,1}^*\Sigma_{1,3}(yx)^* = \Sigma_{1,1}^*(\Sigma_{1,2} \cup \Sigma_{1,3})(yx)^*$. As $|\Sigma_{1,1}| \leq 1$, by Theorem 19.8, $\textsc{Sync}[L(\mathcal{B})]$ is in L. Since the $L(\mathcal{B})$ is infinite, the problem is L-hard by Theorem 19.1. Next, suppose $|\Sigma_{2,3}| + |\Sigma_{2,2}| + |\Sigma_{3,2}| + |\Sigma_{3,3}| = 4$ and set $S = \{2,3\}$. Then $\mu(p,x)$ is defined with $\mu(p,x) \in S$ for each $p \in S$ and $x \in \Sigma$. As $\mathcal{B}$ is trim, by Theorem 18.104, Item 1, $\textsc{Sync}[L(\mathcal{B})]$ is NL-complete. Lastly, suppose $|\Sigma_{2,3}| + |\Sigma_{2,2}| = |\Sigma_{3,2}| + |\Sigma_{3,3}| = 3$, or equivalently that precisely three transitions are defined between the states in $\{2,3\}$. Then, all possible combinations and the resulting complexities where, without loss of generality, we assume $a \in \Sigma_{1,2}$ are listed in Tables 19.1 and 19.2. Table 19.1 gives the proofs.

Table 19.1: Set $L_i^F = L(\mathcal{B}_{i,F})$. Complexities for $|\Sigma_{2,2}| + |\Sigma_{2,3}| + |\Sigma_{3,3}| + |\Sigma_{3,2}| = 3$ (see Thm. 19.23). PSPACE-completeness follows from Thm. 18.12, Prop. 18.93 and Cor. 18.27 and membership in NL from Cor. 19.17 and Thm. 18.9 (the arguments to apply these statements are indicated in the third column). Up to symmetry, we assume $a \in \Sigma_{1,2}$. By giving the subalphabets $\Sigma_{i,j}$, the first column lists all (up to symmetry) automata $\mathcal{B}_{|\{2,3\}}$, and the second column lists all possibilities to enter the strongly connected component $\{2,3\}$ from 1. The combinations give (up to symmetry) all possibilities (see also Table 19.2).

$\Sigma_{2,2}, \Sigma_{2,3}, \Sigma_{3,3}, \Sigma_{3,2}$	$\Sigma_{1,1}, \Sigma_{1,2}, \Sigma_{1,3}$	Argument	Complexity
$\{a\}, \{b\}, \emptyset, \{b\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq L_2^{\{2\}}$	NL
	$\{b\}, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \mathrm{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\emptyset, \{a,b\}, \emptyset$	$L_1^{\{2\}} \subseteq \mathrm{Suff}\left(L_2^{\{2\}}\right)$	NL

Continued on next page

$\Sigma_{2,2}, \Sigma_{2,3}, \Sigma_{3,3}, \Sigma_{3,2}$	$\Sigma_{1,1}, \Sigma_{1,2}, \Sigma_{1,3}$	Argument	Complexity
	$\emptyset, \{a\}, \{b\}$	$a \in \text{Suff}\left(L_2^{\{2,3\}}\right), b \in \text{Suff}\left(L_3^{\{2,3\}}\right)$	NL
$\{a\}, \{b\}, \emptyset, \{a\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq L_2^{\{2\}}$	NL
	$\{b\}, \{a\}, \emptyset$	$\Sigma^* bba\Sigma^* \cap L_1^{\{2\}} = b^* bba(a + ba)^*$	PSPACE-c
	$\emptyset, \{a, b\}, \emptyset$	$\Sigma^* bba\Sigma^* \cap L_1^{\{2\}} = bba(a + ba)^*$	PSPACE-c
	$\emptyset, \{a\}, \{b\}$	$a \in \text{Suff}\left(L_2^{\{2,3\}}\right), b \in \text{Suff}\left(L_3^{\{2,3\}}\right)$	NL
$\{b\}, \{a\}, \emptyset, \{b\}$	$\emptyset, \{a\}, \emptyset$	$\Sigma^* aab\Sigma^* \cap L_1^{\{2\}} = aab(b + ab)^*$	PSPACE-c
	$\{b\}, \{a\}, \emptyset$	$\Sigma^* aab\Sigma^* \cap L_1^{\{2\}} = b^* aab(b + ab)^*$	PSPACE-c
	$\emptyset, \{a, b\}, \emptyset$	$\Sigma^* aab\Sigma^* \cap L_1^{\{2\}} = aab(b + ab)^*$	PSPACE-c
	$\emptyset, \{a\}, \{b\}$	$\Sigma^* aab\Sigma^* \cap L_1^{\{2\}} = aab(b + ab)^*$	PSPACE-c
$\{b\}, \{a\}, \emptyset, \{a\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \text{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\{b\}, \{a\}, \emptyset$	$\Sigma^* bab\Sigma^* \cap L_1^{\{2\}} = b^* bab(b + aa)^*$	PSPACE-c
	$\emptyset, \{a, b\}, \emptyset$	$L_1^{\{2\}} \subseteq \text{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\emptyset, \{a\}, \{b\}$	$\Sigma^* bab\Sigma^* \cap L_1^{\{2\}} = bab(b + aa)^*$	PSPACE-c
$\emptyset, \{a, b\}, \emptyset, \{a\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \text{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\{b\}, \{a\}, \emptyset$	$\Sigma^* bba\Sigma^* \cap L_1^{\{2\}} = b^* bba(aa + ba)^*$	PSPACE-c
	$\emptyset, \{a, b\}, \emptyset$	$\Sigma^* bba\Sigma^* \cap L_1^{\{2\}} = bba(aa + ba)^*$	PSPACE-c
	$\emptyset, \{a\}, \{b\}$	$a \in \text{Suff}\left(L_2^{\{2,3\}}\right), b \in \text{Suff}\left(L_3^{\{2,3\}}\right)$	NL
$\emptyset, \{a, b\}, \emptyset, \{b\}$	$\emptyset, \{a\}, \emptyset$	$\Sigma^* aab\Sigma^* \cap L_1^{\{2\}} = aab(bb + ab)^*$	PSPACE-c
	$\{b\}, \{a\}, \emptyset$	$\Sigma^* aab\Sigma^* \cap L_1^{\{2\}} = b^* aab(bb + ab)^*$	PSPACE-c
	$\emptyset, \{a, b\}, \emptyset$	$\Sigma^* aab\Sigma^* \cap L_1^{\{2\}} = aab(bb + ab)^*$	PSPACE-c
	$\emptyset, \{a\}, \{b\}$	$\Sigma^* aab\Sigma^* \cap L_1^{\{2\}} = aab(bb + ab)^*$	PSPACE-c
$\emptyset, \{a\}, \{a\}, \{b\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{3\}} \subseteq \text{Suff}\left(L_3^{\{3\}}\right)$	NL
	$\{b\}, \{a\}, \emptyset$	$\Sigma^* bba\Sigma^* \cap L_1^{\{2\}} = b^* bba(aa^* b)^*$	PSPACE-c
	$\emptyset, \{a, b\}, \emptyset$	$L_1^{\{3\}} \subseteq \text{Suff}\left(L_3^{\{3\}}\right)$	NL
	$\emptyset, \{a\}, \{b\}$	$\Sigma^* bba\Sigma^* \cap L_1^{\{2\}}) = bba(aa^* b)^*$	PSPACE-c
$\emptyset, \{a\}, \{b\}, \{a\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \text{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\{b\}, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \text{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\emptyset, \{a, b\}, \emptyset$	$\Sigma^* bab\Sigma^* \cap L_1^{\{3\}} = bab(b + aa)^*$	PSPACE-c
	$\emptyset, \{a\}, \{b\}$	$L_1^{\{3\}} \subseteq \text{Suff}\left(L_3^{\{3\}}\right)$	NL
$\emptyset, \{b\}, \{a\}, \{b\}$	$\emptyset, \{a\}, \emptyset$	$\Sigma^* aba\Sigma^* \cap L_1^{\{3\}} = aba(a + bb)^*$	PSPACE-c
	$\{b\}, \{a\}, \emptyset$	$\Sigma^* aba\Sigma^* \cap L_1^{\{3\}} = b^* aba(a + bb)^*$	PSPACE-c
	$\emptyset, \{a, b\}, \emptyset$	$\Sigma^* aba\Sigma^* \cap L_1^{\{3\}} = aba(a + bb)^*$	PSPACE-c
	$\emptyset, \{a\}, \{b\}$	$\Sigma^* aba\Sigma^* \cap L_1^{\{3\}} = aba(a + bb)^*$	PSPACE-c
$\emptyset, \{b\}, \{b\}, \{a\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \text{Suff}\left(L_2^{\{2\}}\right)$	NL

Continued on next page

$\Sigma_{2,2}, \Sigma_{2,3}, \Sigma_{3,3}, \Sigma_{3,2}$	$\Sigma_{1,1}, \Sigma_{1,2}, \Sigma_{1,3}$	Argument	Complexity
	$\{b\}, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \mathrm{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\emptyset, \{a, b\}, \emptyset$	$L_1^{\{3\}} \subseteq \mathrm{Suff}\left(L_3^{\{3\}}\right)$	NL
	$\emptyset, \{a\}, \{b\}$	$L_1^{\{3\}} \subseteq \mathrm{Suff}\left(L_3^{\{3\}}\right)$	NL
$\emptyset, \{a\}, \emptyset, \{a, b\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \mathrm{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\{b\}, \{a\}, \emptyset$	$\Sigma^* baab\Sigma^* \cap L_1^{\{2\}} = b^* baab(aa + ab)^*$	PSPACE-c
	$\emptyset, \{a, b\}, \emptyset$	$L_1^{\{2\}} \subseteq \mathrm{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\emptyset, \{a\}, \{b\}$	$\Sigma^* bba\Sigma^* \cap L_1^{\{3\}} = bba(aa + ba)^*$	PSPACE-c
$\emptyset, \{b\}, \emptyset, \{a, b\}$	$\emptyset, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \mathrm{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\{b\}, \{a\}, \emptyset$	$L_1^{\{2\}} \subseteq \mathrm{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\emptyset, \{a, b\}, \emptyset$	$L_1^{\{2\}} \subseteq \mathrm{Suff}\left(L_2^{\{2\}}\right)$	NL
	$\emptyset, \{a\}, \{b\}$	$L_1^{\{3\}} \subseteq \mathrm{Suff}\left(L_3^{\{3\}}\right)$	NL

Table 19.2 gives graphical depictions of the resulting constraint automata for the cases from Table 19.1. □

With Theorem 19.21, we need at least three states to realize NP-complete constrained synchronization problems. Over a binary alphabet, by Theorems 19.22 and 19.23, three states are the least number of states to realize PSPACE-complete constrained synchronization problems.

19.5 Commutative Languages

Here, we investigate $\textsc{Sync}[L]$ in case the constraint language $L \subseteq \Sigma^*$ is a commutative regular language. In this case, we prove that the problem is in AC^0 or complete for L, NL, NP, or PSPACE (Theorem 19.31).

First, in Section 19.5.1 we introduce the cropped Parikh map and state basic properties. Section 19.5.2 states conditions to simplify a commutative regular constraint language. Finally, in Section 19.5.3, we conclude with the complexity classification.

Here $\mathbb{N}_0^\Sigma$ and $[n]^\Sigma$ are equipped with the component-wise order induced by the usual order on $\mathbb{N}_0$, see Theorem 3.28. Recall that $\psi\colon \Sigma^* \to \mathbb{N}_0^\Sigma$ denotes the Parikh map and $+_n$ an operation on $[n]^\Sigma$ defined in Example 3.19. Here, elements from $\mathbb{N}_0^\Sigma$ are written in boldface.

Remark 19.24 Note that for commutative $U, V \subseteq \Sigma^*$ we have $U \subseteq \mathrm{Fact}(V)$ if and only if, for each $u \in U$, we have $\psi(u) \leq \psi(v)$ for some $v \in V$. Thus $\mathrm{Fact}(U) = \Sigma^*$ if and only if for each $\mathbf{v} \in \mathbb{N}_0^\Sigma$ we have $\mathbf{v} \leq \psi(u)$ for some $u \in U$.

19.5.1 The Cropped Parikh Map

Here, we introduce the cropped Parikh map. Theorem 19.25 collects several properties of it. Then, in Theorem 19.26, we state a formula for the intersection of inverse images of sets incomparable elements with ideal languages.

Let $n \geq 0$. The *n-cropped Parikh map* $\psi_n \colon \Sigma^* \to [n]^\Sigma$ is given, for $u \in \Sigma^*$, by

$$\psi_n(u)(a) = \min\{|u|_a, n-1\}.$$

Proposition 19.25 *The following holds:*

1. $\forall u, v \in \Sigma^* \colon \psi_n(u) = \psi_n(v) \Leftrightarrow \forall a \in \Sigma \colon |u|_a = |v|_a \vee (|u|_a \geq n-1 \wedge |v|_a \geq n-1)$.
2. *For $u, v \in \Sigma^*$ we have that $\psi(u) = \psi(v)$ implies $\psi_n(u) = \psi_n(v)$.*
3. *$\psi^{-1}(\psi(L)) \subseteq \psi_n^{-1}(\psi_n(L))$ for every $L \subseteq \Sigma^*$.*
4. *$\psi_n^{-1}(F)$ is a commutative and regular language for every $F \subseteq [n]^\Sigma$.*
5. *ψ_n is a monoid morphism from $(\Sigma^*, \cdot)$ onto $([n]^\Sigma, +_n)$.*
6. *For $u, v \in \Sigma^*$ with $u \in \Sigma^* v_1 \Sigma^* \cdots \Sigma^* v_m \Sigma^*$ where $v = v_1 \cdots v_m$ with $v_1, \ldots, v_m \in \Sigma$ we have $\psi_n(u) \leq \psi_n(v)$. Thus also for $u \in \mathrm{Pref}(\{v\})$ or $u \in \mathrm{Suff}(\{v\})$ or $u \in \mathrm{Fact}(\{v\})$ we have $\psi_n(u) \leq \psi_n(v)$.*
7. *Let $u, u', v, w \in \Sigma^*$, $\mathbf{v} \in \mathbb{N}_0^\Sigma$ and $\psi_n(u) = \psi_n(u')$. Then*

$$\psi_n(u) = \psi_n(vu'w) \Leftrightarrow v, w \in \{a \in \Sigma \mid \psi_n(u)(a) = n-1\}^* \tag{19.2}$$

$$\psi_n^{-1}(\mathbf{v}) = \Gamma^* \psi_n^{-1}(\mathbf{v}) \Gamma^* \tag{19.3}$$

where $\Gamma = \{a \in \Sigma \mid \mathbf{v}(a) = n-1\}$.

Proof. Item 1, 5, 6, and 7 are clear. Item 1 implies Item 2.

3: Suppose $u \in \psi^{-1}(\psi(L))$. Then $\psi(u) = \psi(v)$ for some $v \in L$. By Item 2 we find $\psi_n(u) = \psi_n(v)$. So $u \in \psi_n^{-1}(\psi_n(L))$.

4: As $\psi_n(u_1u_2u_3u_4) = \psi_n(u_1u_3u_2u_4)$ for all $u_1, u_2, u_3, u_4 \in \Sigma^*$, the language $\psi_n^{-1}(F)$ is commutative. Define $\mathcal{A} = ([n]^\Sigma, \Sigma, \delta, (0)_{a\in\Sigma}, F)$ where $\delta(q, a)$ equals $\mathbf{v}$ with $\mathbf{v}(a) = \min\{q(a) + 1, n-1\}$ and $\mathbf{v}(b) = q(b)$ for $b \neq a$ and $a, b \in \Sigma$. Then $L(\mathcal{A}) = \psi_n^{-1}(F)$.[4] □

Note that for $N \subseteq [n]^\Sigma$ we have $\mathrm{Fact}(\psi_n^{-1}(N)) = \Sigma^*$ if and only if $(n, \ldots, n) \in N$.

Proposition 19.26 *Let $N \subseteq [n]^\Sigma$ be a set of incomparable elements where $n \geq 0$, $\mathbf{v} \in N$, $\Gamma = \{a \in \Sigma \mid \mathbf{v}(a) = n-1\}$ and $L \subseteq \psi_n^{-1}(\mathbf{v})$. Then $\psi_n^{-1}(N) \cap \Sigma^* L \Sigma^* = \Gamma^* L \Gamma^*$.*

Proof. For $u \in L$ and $v, w \in \Gamma^*$, by Equation (19.2) it is $\psi_n(vuw) = \psi_n(u)$. As $\psi_n(u) \in N$, we get $\Gamma^* L \Gamma^* \subseteq \psi_n^{-1}(N) \cap \Sigma^* L \Sigma^*$. Conversely, suppose $\psi_n(vuw) \in N$ for $u \in L$ and $v, w \in \Sigma^*$. We have $\psi_n(u) \leq \psi_n(vuw)$. As the vectors in N are incomparable, this gives $\psi_n(u) = \psi_n(vuw)$. By Equation (19.2), $v, w \in \Gamma^*$. □

Example 19.27 The assumption of incomparability of the elements in N is necessary. For example, let $\Sigma = \{a_1, a_2\}$ and $n \geq 3$. We identify $\mathbf{v} \in \mathbb{N}_0^\Sigma$ with the tuple

[4] In fact, the finite monoid $([n]^\Sigma, +_n)$ recognizes $\varphi_n^{-1}(F)$ as defined in [489].

$(\mathbf{v}(a_1), \mathbf{v}(a_2))$ and set $N = \{(n,n),(n,2)\}$. Then $a_1^n a_2^n \in \psi_{n+1}^{-1}(N) \cap \Sigma^* a_1^n a_2^2 \Sigma^*$ but $a_1^n a_2^n \notin a_1^* a_1^n a_2^2 a_1^*$. So $\psi_{n+1}^{-1}(N) \cap \Sigma^* a_1^n a_2^2 \Sigma^* \neq a_1^* a_1^n a_2^2 a_1^*$ for $a_1^n a_2^2 \in \psi_{n+1}^{-1}((n,2))$.

19.5.2 Simplifying the Constraint Language

Here, we show two statements that imply that a commutative regular constraint language can be replaced by a simpler commutative constraint language. By Theorem 19.28 considers inverse images of the cropped Parikh map and Theorem 19.29 implies that we can replace a commutative constraint language by some inverse image of the cropped Parikh map.

Proposition 19.28 *Let $N \subseteq [n]^\Sigma$ for some $n \geq 0$ and $M = \{\mathbf{v} \in N \mid \mathbf{v}$ is maximal in $N\}$. Then*

$$\textsc{Sync}[\psi_n^{-1}(N)] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\psi_n^{-1}(M)].$$

Proof. The claim follows from Theorem 18.5 since $\mathrm{Fact}(\psi_n^{-1}(M)) = \psi_n^{-1}(N)$. □

Proposition 19.29 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA and $n > |P|$. Then for a complete and deterministic semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$ we have*

$$\mathrm{Syn}(\mathcal{A}) \cap \psi_n^{-1}(\psi_n(L(\mathcal{B}))) \neq \emptyset \Leftrightarrow \mathrm{Syn}(\mathcal{A}) \cap \psi^{-1}(\psi(L(\mathcal{B}))) \neq \emptyset.$$

Proof. First, suppose $u \in \mathrm{Syn}(\mathcal{A}) \cap \psi_n^{-1}(\psi_n(L(\mathcal{B})))$. Let $\Gamma \subseteq \Sigma$ be a maximal set such that $\psi_n(u) = \psi_n(v)$ for some $v \in L(\mathcal{B})$ with

$$|v|_a \geq |u|_a$$

for all $a \in \Gamma$. Assume there exists $b \in \Sigma \setminus \Gamma$. So, $|v|_b < |u|_b$. As $\psi_n(u) = \psi_n(v)$, we have $|v|_b, |u|_b \geq n-1$. Hence, we can write $v = v_1 \cdots v_{n-1}$ with $|v_i|_b > 0$ for $i \in \{1, \ldots, n-1\}$. As $n > |P|$, by finiteness $\mu(p_0, v_1 \cdots v_i) = \mu(p_0, v_1 \cdots v_j)$ for some $1 \leq i < j < n$. Then $v' = v_1 \cdots v_{i-1}(v_i \cdots v_j)^{|u|_b} v_{j+1} \cdots v_n \in L(\mathcal{B})$, $\psi_n(u) = \psi_n(v')$ and $|v'|_a \geq |u|_a$ for all $a \in \Gamma \cup \{b\}$, contradicting the maximal choice of Γ. So, we must have $\Gamma = \Sigma$. Now, let $\Sigma = \{a_1, \ldots, a_k\}$ and set $w = a_1^{|v|_{a_1} - |u|_{a_1}} \cdots a_1^{|v|_{a_k} - |u|_{a_k}} u$. Then $\psi(w) = \psi(v)$ so that $w \in \psi^{-1}(\psi(L(\mathcal{B})))$. By Theorem 7.1 we have $w \in \mathrm{Syn}(\mathcal{A})$. Conversely, Theorem 19.25, Item 3 yields the other implication. □

19.5.3 Complexity Classification

Here, we give a complete complexity classification for commutative regular constraint languages.

First, in Theorem 19.30, we classify the complexity for regular constraint languages of the form $\textsc{Sync}[\psi_n^{-1}(\mathbf{v})]$ for some $\mathbf{v} \in \mathbb{N}_0^\Sigma$. Then, in Theorem 19.31, we give a complete overview of the computational complexity for different types of commutative

constraint languages. The formulation of the classification uses the set $\max \psi_{n+1}(L(\mathcal{B}))$ of incomparable elements for an n-state constraint PDFAs.

Lemma 19.30 *Let $\mathbf{v} \in [n]^\Sigma$. Then for the problem* Sync$[\psi_n^{-1}(\mathbf{v})]$ *the following complexities are possible:*

1. *in* AC^0 *if $\mathbf{v}(a) < n-1$ for all $a \in \Sigma$,*
2. L*-complete if there exists $a \in \Sigma$ such that $\mathbf{v}(a) = n-1$ and $\mathbf{v}(b) > 0$ for some $b \in \Sigma \setminus \{a\}$ implies $\mathbf{v}(b) = 1$ and $\mathbf{v}(c) = 0$ for all $c \in \Sigma \setminus \{a, b\}$.*
3. NL*-complete if there exist distinct $a, b \in \Sigma$ such that $\mathbf{v}(a) = \mathbf{v}(b) = n-1$ and for all $a \in \Sigma$ it is $\mathbf{v}(a) \in \{0, n-1\}$,*
4. NP*-complete if $\sum_{\substack{a \in \Sigma \\ \mathbf{v}(a) < n-1}} \mathbf{v}(a) \geq 2$ and $\mathbf{v}(b) = n-1$ for precisely one $b \in \Sigma$.*
5. PSPACE*-complete if there exist distinct $a, b, c \in \Sigma$ such that $0 < \mathbf{v}(a) < n-1$ and $\mathbf{v}(b) = \mathbf{v}(c) = n-1$.*

Proof. 1: This is equivalent to finiteness of $\psi_n^{-1}(\mathbf{v})$ and so containment in AC^0 follows from Theorem 19.3.

2 Suppose $\mathbf{v}(a) = n-1$. Then $\psi_n^{-1}(\mathbf{v})$ is infinite and L-hardness follows from Theorem 19.1. If $\mathbf{v}(b) = 0$ for all $b \in \Sigma \setminus \{a\}$, then $\psi_n^{-1}(\mathbf{v}) = a^{n-1}a^* \subseteq a^*$. Otherwise, suppose $\mathbf{v}(b) = 1$ and $\mathbf{v}(c) = 0$ for all $c \in \Sigma \setminus \{a, b\}$. Then $\psi_n^{-1}(\mathbf{v}) = a^*ba^{n-1}a^* \cup a^*aba^{n-2}a^* \cup a^*a^2ba^{n-3} \cup \ldots \cup a^*a^{n-1}ba^* \subseteq a^*b^*a^*$ (note that we have $\mathrm{Fact}(\psi_n^{-1}(\mathbf{v}))) = a^*ba^*$). So, in both cases, containment in L follows from Theorem 19.7.

3 Set $\Gamma = \{a \in \Sigma \mid \mathbf{v}(a) = n-1\}$. Then $\psi_n^{-1}(\mathbf{v}) = \{u \in \Gamma^* \mid \forall a \in \Gamma\colon |u|_a \geq n-1\}$. So $\mathrm{Fact}(\psi_n^{-1}(\mathbf{v})) = \Gamma^*$. As $|\Gamma| \geq 2$, Theorems 18.4 and 19.20 give the claim.

4 First, if there exists $a \in \Sigma$ with $2 \leq \mathbf{v}(a) \leq n-1$, then consider

$$\pi_{\{a,b\}}(\psi_n^{-1}(\mathbf{v})) \cap \{a,b\}^*ab^{\mathbf{v}(b)}b^*a^{\mathbf{v}(a)-1}\{a,b\}^* = b^*ab^{\mathbf{v}(b)}b^*a^{\mathbf{v}(a)-1}b^*.$$

Otherwise, there exist distinct $a, c \in \Sigma$ with $\mathbf{v}(a) = \mathbf{v}(b) = 1$. Then consider

$$\pi_{\{a,b,c\}}(\psi_n^{-1}(\mathbf{v})) \cap \{a,b,c\}^*ab^*c\{a,b,c\}^* = b^*ab^{\mathbf{v}(b)}b^*cb^*.$$

By Theorems 18.12, 18.17, and 19.10, we find that Sync$[\psi_n^{-1}(\mathbf{v})]$ is NP-hard. Next, for $u \in \psi_n^{-1}(\mathbf{v})$ and $a \in \Sigma \setminus \{b\}$ we have $|u|_a < n-1$. By Theorem 17.80, Item 3, $\psi_n^{-1}(\mathbf{v})$ is sparse. So, by Theorem 19.2, Sync$[\psi_n^{-1}(\mathbf{v})]$ is in NP.

5 In this case we have

$$\pi_{\{a,b,c\}}(\psi_n^{-1}(\mathbf{v})) \cap \{a,b,c\}^*a^{\mathbf{v}(a)}b^{n-1}c^{n-1}\{a,b,c\}^* = \{b,c\}^*a^{\mathbf{v}(a)}b^{n-1}c^{n-1}\{b,c\}^*.$$

By Theorem 18.92 the language on the right hand side yields a PSPACE-hard constrained problem. By Theorems 18.12 and 18.17, Sync$[\psi_n^{-1}(\mathbf{v})]$ is PSPACE-hard. Theorem 18.1 yields that the problem is in PSPACE. □

Theorem 19.31 *Let $\mathcal{B}$ be an n-state PDFA recognizing a commutative language. Denote by N the set of maximal vectors in $\psi_{n+1}(L(\mathcal{B}))$.*

1. *If $L(\mathcal{B})$ is finite (equivalently $\mathbf{v} < (n, \ldots, n)$ for all $\mathbf{v} \in N$), then* Sync$[L(\mathcal{B})]$ *is in* AC^0.

2. *If $L(\mathcal{B})$ is infinite (or equivalently there exist $\mathbf{v} \in N$ and $a \in \Sigma$ such that $\mathbf{v}(a) = n$) and if $\mathbf{v}(a) = n$ and $\mathbf{v}(b) > 0$ for some $a, b \in \Sigma$ imply $\mathbf{v}(b) = 1$ and $\mathbf{v}(c) = 0$ for all $c \in \Sigma \setminus \{a, b\}$, then $\textsc{Sync}[L(\mathcal{B})]$ is L-complete.*
3. *If for every $\mathbf{v} \in N$ we have $\mathbf{v}(a) \in \{0, n\}$ for all $a \in \Sigma$ and there exists $\mathbf{v} \in N$ and distinct $a, b \in \Sigma$ such that $\mathbf{v}(a) = \mathbf{v}(b) = n$, then $\textsc{Sync}[L(\mathcal{B})]$ is NL-complete.*
4. *If there exists $\mathbf{v} \in N$ with $\sum_{\mathbf{v}(a)<n-1} \mathbf{v}(a) \geq 2$ and $\mathbf{v}(b) = n$ for some $b \in \Sigma$, then $\textsc{Sync}[L(\mathcal{B})]$ is NP-hard. If additionally, for every $\mathbf{u} \in N$ we have that $|\{a \in \Sigma \mid \mathbf{u}(a) = n\}| \geq 2$ implies $\mathbf{u}(a) \in \{0, n\}$ for all $a \in \Sigma$, then $\textsc{Sync}[L(\mathcal{B})]$ is NP-complete.*
5. *Suppose there exist $a, b, c \in \Sigma$ and $\mathbf{v} \in N$ such that $0 < \mathbf{v}(a) < n$ and $\mathbf{v}(b) = \mathbf{v}(c) = n$. Then $\textsc{Sync}[L(\mathcal{B})]$ is PSPACE-complete.*

Proof. If $L(\mathcal{B})$ is finite, containment in AC^0 follows from Theorem 19.3. Next, suppose $L(\mathcal{B})$ is infinite. As $L(\mathcal{B})$ is commutative, we have $L(\mathcal{B}) = \psi^{-1}(\psi(L(\mathcal{B})))$. Proposition 19.29 then gives

$$\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\psi_n^{-1}(\psi_n(L(\mathcal{B})))].$$

Proposition 19.28 yields $\textsc{Sync}[L(\mathcal{B})] \equiv_m^{\mathsf{AC}^0} \textsc{Sync}[\psi_{n+1}^{-1}(N)]$ with $N = \max \psi_n(L(\mathcal{B}))$.

Now, for each case, 2, 3, 4, or 5, for $\mathbf{v} \in N$, by Theorem 19.30, $\textsc{Sync}[\psi_n^{-1}(\mathbf{v})]$ is in L, NL, NP, or PSPACE, respectively. Then, by Theorem 18.9, as

$$\psi_n^{-1}(N) = \bigcup_{\mathbf{v} \in N} \psi_n^{-1}(\mathbf{v}),$$

we obtain containment in the respective complexity class.

Next, each case 2, 3, 4, or 5, claims existence of a single $\mathbf{v} \in N$ such that we obtain L-, NL-, NP- or PSPACE-hardness, respectively, of $\textsc{Sync}[\psi_n^{-1}(\mathbf{v})]$ from Lemma 19.30. The elements of N are pairwise incomparable. So, by Theorem 19.26 we have $\psi_n^{-1}(N) \cap \Sigma^* \psi_n^{-1}(\mathbf{v}) \Sigma^* = \psi_n^{-1}(\mathbf{v})$. Then, by the ideal intersection theorem (Theorem 18.12), L-, NL-, NP- and PSPACE-hardness follows for $\textsc{Sync}[\psi_n^{-1}(N)]$ in each case 2, 3, 4, or 5, respectively. □

Remark 19.32 The property that the elements in N are pairwise incomparable is crucial in the formulation of Theorem 19.31. Consider $N = \{(n, n), (n, 2)\}$ from Example 19.27 where $\Sigma = \{a_1, a_2\}$. Then N fulfills the condition from Theorem 19.31, Item 4, but $\mathrm{Fact}(\psi_n^{-1}(N)) = \{a_1, a_2\}^*$. So, by Theorems 18.5 and 19.20, $\textsc{Sync}[\psi_n^{-1}(N)]$ is NL-complete.

Example 19.33 1. $\textsc{Sync}[a^*aba^* \cup a^*baa^*]$ is L-complete.
2. $\textsc{Sync}[\{u \in \{a, b\}^* : |u|_a \geq n, |u|_b \geq m\}]$ for some $n, m \geq 0$ is NL-complete.
3. $\textsc{Sync}[\{u \in \{a, b\}^* : |u|_a \leq n, |u|_b \geq m\}]$ for some $n \geq 2$ and $m \geq 0$ is NP-complete.
4. $\textsc{Sync}[\{u \in \{a, b, c\}^* : |u|_a \leq n, |u|_b, |u|_c \geq m\}]$ for some $n \geq 1$ and $m \geq 0$ is PSPACE-complete for distinct a, b, c.

19.6 Discussion

Sparse Languages.

Hoffmann [296] (inspired by a result from [215, Thm. 19]) called a PDFA $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ *polycyclic* if, for each $p \in P$, we have $L(\mathcal{B}_{p,\{p\}}) \subseteq u_p^*$ for some $u_p \in \Sigma^*$, and proved the following:

1. constraint languages accepted by polycyclic automata give constrained problems in NP (generalizing a statement from [215, Thm. 19]),
2. polycyclic automata are equivalently characterized by the property that every strongly connected components forms a single cycle,
3. languages accepted by polycyclic PDFAs are closed for union, concatenation and quotients,
4. nondeterministic automata[5] with the same definitional condition accept the same class of languages.

Gawrychowski et al. [240, Lem. 2] proved that nondeterministic automata fulfilling the definition condition for polycyclic automata accept precisely the sparse regular languages.[6] So, polycyclic automata characterize sparse regular languages (this also follows from our method of proof used for Theorem 17.85). The characterization of sparse languages by PDFAs whose strongly connected components form single cycles was stated by Pin in [491, Sec. XII.4.2] in terms of forbidden patterns. It also appears in [235, Lem. 5.4].

Also an NP-hardness condition was given in [296, Prop. 8] that was generalized in Theorem 18.94. With this condition it is easy to find languages accepted by polycyclic constraint PDFAs that give NP-hard constrained synchronization problems.

Hoffmann [298] gave a classification for letter-bounded constraint languages into NP-complete and polynomial-time solvable constrained problems. The classification of the present work is a refinement. Also, the method of proof is different from [298] (for the interested reader the original proofs are contained in the arXiv version to which a link is provided in the bibliographic entry [298], but actually the present proofs are much more concise). A classification for all sparse languages is an *open problem*.

> Open Problem: Classify the computational complexity of SYNC[L] for sparse regular languages $L \subseteq \Sigma^*$.

Monoid Languages and Strongly Connected Constraint Automata.

The following results carry over from the unconstrained setting to monoid constraint languages: the pair condition (Theorem 19.11); a cubic bound on the length of the shortest synchronizing words in the constraint language (Theorem 19.13); a polynomial-time

[5] See [322] for a definition of nondeterministic automata.

[6] The paper by Gawrychowski et al. [240] appeared earlier than [296], but the author of [296] was unaware of the connection to sparse languages when writing his paper.

algorithm for computing a synchronizing word of at most cubic length (Theorem 19.14); and membership in NL (Theorem 19.15). No condition is known to identify NL-hardness for either monoid languages or strongly connected constraint PDFAs; we therefore pose this as an *open problem*.

> Open Problem: Identify the NL-hard constrained synchronization problem instances for constraint languages that are monoids or are given by a strongly connected constraint automaton. Furthermore, does a monoid contraint language of exponential growth gives an NL-hard constrained problems?

An automaton $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ is *returning* if for every state $p \in P$ there exists $u \in \Sigma^*$ such that $\mu(p, u) = p_0$. For returning automata $\mathcal{B}$, in [215, Lem. 16 & Thm. 17] the pair condition (Theorem 19.11) was established and polynomial-time solvability of $\textsc{Sync}[L(\mathcal{B})]$ derived. An accessible automaton is returning if and only if it is strongly connected. Hence this result is covered by Theorem 19.19 of the present work. In fact, it shows that this is a consequence of the fact that $\textsc{Sync}[L]$ is in NL for regular languages L that are monoids.

The languages $(a + b)^*$ and $c(a + b)^*$ from Theorem 18.2 show that concatenating constraint languages might increase the computational complexity of the constrained synchronization problem w.r.t. the resulting language considerably. Section 18.2.5 gave conditions for which the complexity never drops when concatenating languages, which we used to transfer hardness results from known languages. On the other hand, the UVW-Condition (Corollary 19.17) involved the concatenation of three languages and yielded constrained problems in NL. For example, it implies that $\textsc{Sync}[\Sigma^* L]$ is in NL for any regular language $L \subseteq \Sigma^*$, or more generally $\textsc{Sync}[IL]$ for any regular ideal $I \subseteq \Sigma^*$ and regular $L \subseteq \Sigma^*$. A weaker form was stated in [301, Thm. 4.1], only giving containment in P and stating the result for PDFAs whose initial state equals their unique final state, a subclass of regular languages that are monoids (see Theorem 17.75).

Small Constraint Automata.

Fernau et al. [215] gave the following classification for small constraint automata with up to two states over an at most ternary alphabet.

Theorem 19.34 (Fernau et al. [215]) *Let $\mathcal{B} = (\Sigma, P, \mu, p_0, F)$ be a PDFA. If $|P| \leq 1$ or $|P| = 2$ and $|\Sigma| \leq 2$, then $\textsc{Sync}[L(\mathcal{B})] \in$ P. For $|P| = 2$ with $|\Sigma| = 3$, up to symmetry by renaming of the letters, $\textsc{Sync}[L(\mathcal{B})]$ is* PSPACE*-complete precisely in the following cases for $L(\mathcal{B})$:*

$a(b + c)^*$	$(a + b + c)(a + b)^*$	$(a + b)(a + c)^*$	$(a + b)^* c$
$(a + b)^* ca^*$	$(a + b)^* c(a + b)^*$	$(a + b)^* cc^*$	$a^* b(a + c)^*$
$a^*(b + c)(a + b)^*$	$a^* b(b + c)^*$	$(a + b)^* c(b + c)^*$	$a^*(b + c)(b + c)^*$

and polynomial time solvable in all other cases.

Theorem 19.21 extended this result in the 2-state case to arbitrary alphabets and refined the complexity of the polynomial time cases into the three classes AC^0, L-

complete, and NL-complete for non-strongly connected constraint PDFAs. The extension to arbitrary alphabets for 2-state constraint PDFAs was first given in [301] (by only distinguishing between PSPACE-completeness and P). The classification presented here was given in [316], the extended journal version of [301]. In [215] the following two constrained problems and their complexities given by 3-state binary PDFAs were presented: $\textsc{Sync}[ab^*a]$ (NP-complete) and $\textsc{Sync}[b(aa + ba)^*]$ (PSPACE-complete). So, the smallest automata giving NP-complete problems have three states and are over a binary alphabet. We completed the picture by giving computational complexities for all 3-state binary PDFAs (up to renaming of symbols and states). We gave a full classification for non-strongly-connected PDFAs into problems that are in AC^0, L-, NL-, NP-, or PSPACE-complete. This classification was also given in [316]. As a special case of the open problem posed for strongly connected automata before, we also do not know when the problem for 3-state strongly connected binary automata is NL-hard.

> Open Problem (special case of the open problem for general strongly connected automata): Identify the NL-hard constrained synchronization problem instances for 3-state strongly connected binary constraint PDFAs.

Commutative Languages.

A coarser classification for commutative regular constraint languages distinguishing between PSPACE-complete, NP-complete or polynomial-time solvable constrained problems was given in [299]. Here, the classification was refined by subdividing the case when the constrained problem is in P into the complexities NL-complete, L-complete and containment in AC^0. Furthermore, using the general results from Chapter 18 and the cropped Parikh map, the proof was much simplified and shortened. This simplified proof was also presented in [316].

Deciding the Complexity.

It is readily seen that the classification results for letter-bounded and commutative constraint languages can be used to decide in polynomial time the computational complexity of $\textsc{Sync}[L]$ when $L \subseteq \Sigma^*$ is given as input (for example, via accepting PDFAs).

Theorem 19.35 *Let $\mathcal{B}$ be a PDFA.*

1. *If $L(\mathcal{B})$ is letter-bounded, then we can decide in polynomial time the complexity of* $\textsc{Sync}[L(\mathcal{B})]$*, i.e., whether it is in* AC^0*,* L*-complete or* NP*-complete.*
2. *If $L(\mathcal{B})$ is commutative, then we can decide in polynomial time the complexity of* $\textsc{Sync}[L(\mathcal{B})]$*, i.e., whether it is in* AC^0*,* L*-complete,* NL*-complete,* NP*-complete or* PSPACE*-complete.*

Also, for small constraint automata, the complexity up to the fidelity as stated in the present work can be decided in polynomial time by trimming and comparing an input PDFA with finitely many alternatives.

A Conjecture for the General Case.

In [215, Conclusions and Prospects] it was asked whether there exist constrained problems complete for levels in the polynomial time hierarchy other than NP or PSPACE. However, we do not believe so and in light of the present results we formulate the following conjecture.

> Conjecture: For every $L \subseteq \Sigma^*$ the problem Sync$[L]$ is either in AC^0 or complete for one of the following classes: L, NL, NP, PSPACE.

In [301, Sec. 6] it was also suspected that only constrained problems in P, or complete for PSPACE or NP are possible.

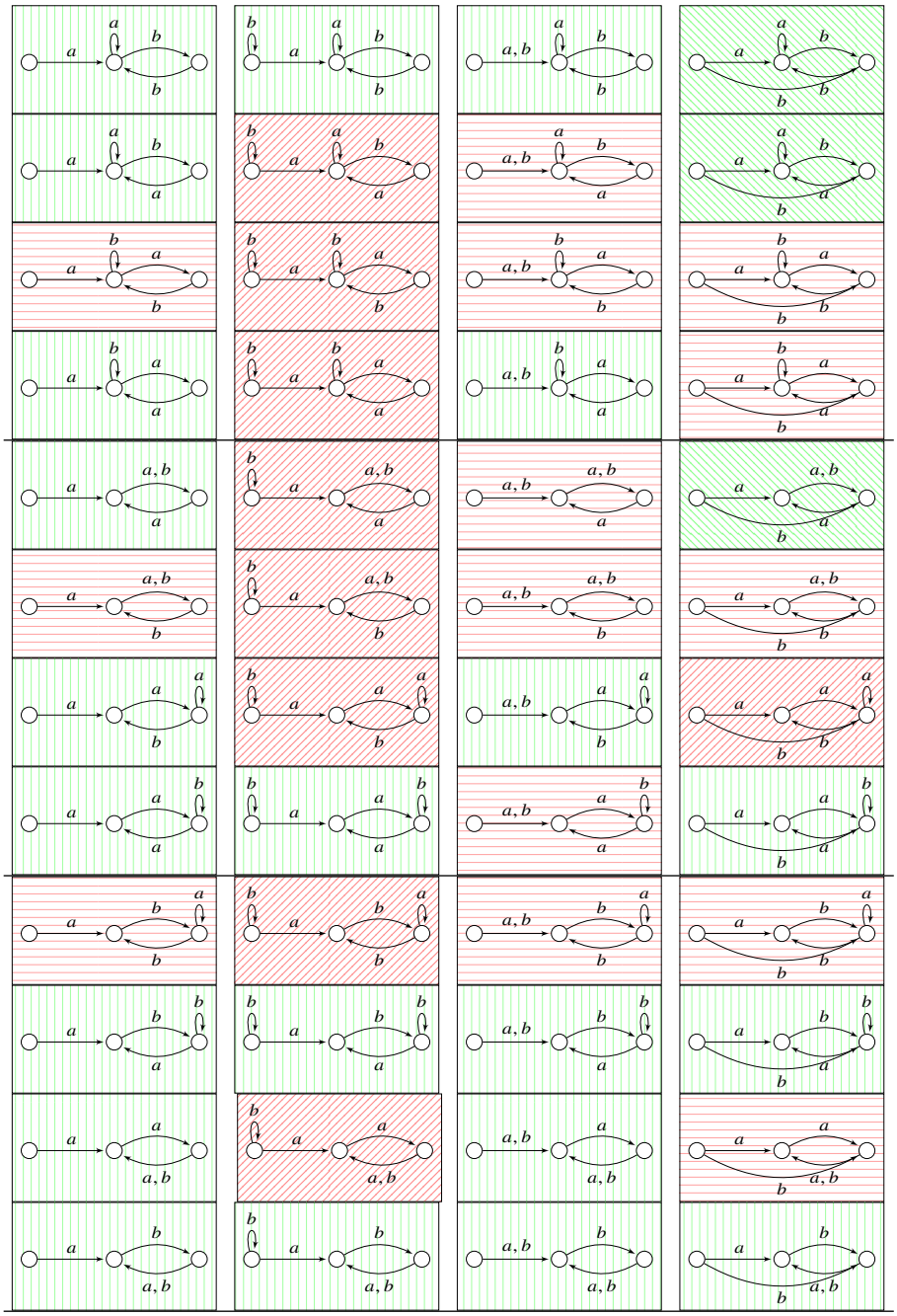
Since every alphabet can be encoded by a solid code over a binary alphabet (as, for example, those given in Theorem 17.14) which preserves the complexity by Theorem 18.19, confirmation of the conjecture for binary automata implies it for all automata.

In [215, Rem. 30] it was remarked that the following construction from [82, pp. 220-221] can be used to lift hardness results: for $\mathcal{A} = (Q, \{a_1, \ldots, a_k\}, \delta)$ let $\mathcal{A} = (Q \times \{1, \ldots, k\}, \{a, b\}, \delta')$ with $\delta'((q, i), a) = (q, \min\{i + 1, k\})$ and $\delta'((q, i), b) = (\delta(q, a_i), 1)$. Set $\varphi(a_i) = a^{i-1}b$ for $i \in \{1, \ldots, k\}$. Then $u_1 \cdots u_m$ with $u_1, \ldots, u_n \in \{a_1, \ldots, a_k\}$ synchronizes $\mathcal{A}$ if and only if $b\varphi(u_1) \cdots \varphi(u_m)$ synchronizes $\mathcal{A}'$. However, since the letter b has to be read first this must be allowed by the binary constrained language restricting the constrained synchronization problem for $\mathcal{A}'$ to obtain a reduction.

Road-Coloring and Generalizations.

Checking, for a given automaton and word, whether the word synchronizes the automaton is L-complete by Theorem 8.7. In [624] the authors classified the computational complexity of the so-called *road-coloring variant of synchronization*: given a digraph of constant outdegree and a word w, whether the digraph admits a labeling of its edges with letters such that a synchronizing automaton with synchronizing word w results. Depending on the class of admissible input words, the problem is either polynomial-time solvable or NP-complete. Now this suggests the following generalization of Sync$[L]$: for a fixed constraint language, given a digraph of constant outdegree, does it admit a labeling of its edges such that a synchronizing automaton results having a synchronizing word in the constraint language. In [624] it has been proved that this problem is NP-complete even for the constraint language $\{abb\}$. When we admit the constraint language to be part of the input, for $L = \Sigma^n$, where n is an input parameter, this problem has been investigated in [515, 518]. The generalization for arbitrary languages as input has been mentioned in [518, 624]. Other generalizations where the constraint language is part of the input and belongs to some class of admissible constraint languages for our constrained problem or the road-coloring variant are conceivable, but we have not pursued to investigate these variants here.

Table 19.2: 3-state PDFAs with at least one undefined transition among the states, numbered 1, 2, 3 from left to right, in the strongly connected component $\{2, 3\}$. The cases are the same as in Table 19.1. The start state is 1. Final states are not marked, as by Theorem 18.100 any non-empty subset of $\{2, 3\}$ gives the same complexity.



Chapter 20
Classes of Input Automata

Here, we investigate the constrained synchronization problem when the input is restricted to the following classes of automata:

1. monotonic automata (Section 20.1),
2. partially ordered automata (Section 20.2),
3. ρ-automata with a trap state (Section 20.3),
4. commutative and unary automata (Section 20.4),
5. solvable ρ-automata (Section 20.5),
6. binary SI-automata (Section 20.6).

Table 20.1: Overview of complexities for different classes of input automata. For general automata and automata with a trap state PSPACE-completeness follows from the results and reductions in Chapter 18. The remaining classes are the content of the present section. For a more refined statement w.r.t. different constraint languages see the respective statements and sections. PSPACE-hardness in the first two cases holds even for $|\Sigma| = 2$ according to Theorem 19.23.

Automaton Class	**Complexity**	**Statement**
General	PSPACE-complete	Th. 18.1 & 18.2 & 19.23
With Trap State	PSPACE-complete	Th. 18.1 & 18.2
Monotonic	NL	Th. 20.6
ρ-Automata	PSPACE-complete	strongly conn. is ρ-aut.
Strongly Connected	PSPACE-complete	Th. 18.1 & 18.2
ρ-Aut. with Trap State	NP-complete	Th. 20.18
Solvable ρ-Aut.	NP-complete	Th. 20.16 & 20.22
Partially Ordered	NP-complete	Th. 20.16
Commutative	NL	Th. 20.19
Unary	L-complete	Th. 19.1
Binary SI-Automata	NL for m-state constr. PDFAs with $m \leq 3$	Th. 20.37

For an overview of the results see Table 20.1 (the table also includes some conclusions that follow readily from previous results and reductions).

Remark 20.1 Theorem 17.88 noted that Sync[L] w.r.t. any Σ with $L \subseteq \Sigma^*$ gives the same complexity for $\leq_m^{\mathsf{AC}^0}$-reductions. When restricting the input to subclasses of automata, this only holds for subclasses closed for the operations used in the reductions from Remark 17.88, namely adding self-loops or restricting input automata to some subalphabet. The subclasses considered in the present work, except strongly connected automata, are closed for these operations. In case of strongly connected automata, we additionally assume the input automata are strongly connected w.r.t. to the smallest alphabet Σ such that $L \subseteq \Sigma^*$ i.e., the alphabet $\Sigma = \{a \in \Sigma \mid \Sigma^* a \Sigma^* \cap L \neq \emptyset\}$.

Remark 20.2 An automaton $\dashv$ is a YES-instance for Sync[L] if and only if $\mathrm{Syn}(\mathcal{A}) \cap L \neq \emptyset$. Since we have $\mathrm{Syn}(\mathcal{A}) = \mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right)$ where $F = \{\{q\} \mid q \in Q\}$ by Theorem 7.16, and the pair automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ has a trap state and can be computed in polynomial time. Thus, up to polynomial time, Sync[L] is equivalent to constrained synchronization problem with constraint language L restricted to automata with a trap state as input. Thus all our NP- and PSPACE-hardness results also hold for trap-state input automata up to polynomial-time many-one reductions.

Remark 20.3 When restricting the type of input automata care has to be taken when using some of the general results from Chapter 18 since some reductions alter the input automaton, which might result in automata that are not contained in some class of automata under consideration. We note that general results solely based on Theorems 18.4 and 18.5, i.e., the equality of factor sets of constraint automata, work for every class of automata since they do not alter the input automaton at all.

20.1 Monotonic Automata

Here, we investigate the constrained synchronization problem for monotonic input automata. When not said otherwise, by $\leq$ we denote the linear order given on the states of some monotonic automaton.

Theorem 20.4 first states a characterization of synchronizing monotonic automata. Then, in Theorem 20.5 we give a quadratic upper bound for a synchronizing word in a given constraint language. Theorem 20.6 states that the constrained synchronization problem for monotonic input automata is in NL, even when we have no access to the linear order on the states.

Lemma 20.4 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a monotonic automaton where $s \in Q$ is the minimum in the linear order on the states and $t \in Q$ the maximum. Then $u \in \Sigma^*$ synchronizes $\mathcal{A}$ if and only if $\delta(s, u) = \delta(t, u)$.*

Proof. If $u \in \Sigma^*$ synchronizes $\mathcal{A}$, then in particular $\delta(s, u) = \delta(t, u)$. Conversely, for every $q \in Q$ we have $s \leq q \leq t$ and so $\delta(s, u) \leq \delta(q, u) \leq \delta(t, u)$ for $u \in \Sigma^*$. Hence, if $\delta(s, u) = \delta(t, u)$, then we get $\delta(Q, u) = \{\delta(t, u)\}$. □

The following proof uses the pair automaton (defined in Section 6.5.6).

Proposition 20.5 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA. If $\mathcal{A} = (Q, \Sigma, \delta)$ is an n-state monotonic automaton with a synchronizing word in $L(\mathcal{B})$, then a shortest word in* $\mathrm{Syn}(\mathcal{A}) \cap L(\mathcal{B})$ *has length at most* $\left(\binom{n}{2} + 1\right) \cdot |P| - 1$.

Proof. We can assume $s \neq t$. Consider the pair automaton $C = \mathcal{A}^{\leq 2}/{\sim_S}$ where $S = \{\{q\} \mid q \in Q\}$. Let $s \in Q$ be the minimum and $t \in Q$ be the maximum state. By Theorem 20.4 we have $\mathrm{Syn}(\mathcal{A}) = L(C_{\{s,t\},\{S\}})$. By Equation (6.22) the language $L(C_{\{s,t\},\{S\}}) \cap L(\mathcal{B})$ is accepted by the product PDFA of C and $\mathcal{B}$ with start state $(\{s,t\}, p_0)$ and final state set $\{S\} \times F$, which has $\left(\binom{n}{2} + 1\right) \cdot |P|$ states in total. So, a shortest path from from $(\{s,t\}, p_0)$ to a state (S, p) with $p \in F$ has length at most $\left(\binom{n}{2} + 1\right) \cdot |P| - 1$. □

Theorem 20.6 *Let $L \subseteq \Sigma^*$ be fixed regular language. The problem to decide whether a given monotonic input automaton $\mathcal{A} = (Q, \Sigma, \delta)$ has a synchronizing word in L is in* NL.

Proof. Set $n = |Q|$. We can assume $n \geq 2$. Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA with $L = L(\mathcal{B})$. Loop through distinct pairs of states $p, q \in Q$ and nondeterministically guess a word $u \in \Sigma^*$ of length at most $\left(\binom{n}{2} + 1\right) \cdot |P| - 1$ letter-by-letter with $\mu(p_0, u) \in F$ and $|\delta(\{p,q\}, u)| = 1$. This procedure can run in log-space by using counters (or storage space) with a binary encoding for the states and the length of the word. Then $\mathcal{A}$ has a synchronizing word in $L(\mathcal{B})$ if and only if a word $u \in L(\mathcal{B})$ as above can be found for each pair of states $p, q \in Q$. For if there exists a synchronizing word in $L(\mathcal{B})$, then by Theorem 20.5 it has length at most $\left(\binom{n}{2} + 1\right) \cdot |P| - 1$. Hence, in each iteration for two states, by nondeterminism, a word in $L(\mathcal{B})$ is guessed that maps both states to a single state. Conversely, if each iteration for two states finds a word in $L(\mathcal{B})$ mapping both to a single state, then in particular for the minimum state $s \in Q$ and the maximum state $t \in Q$ we find a word $u \in L(\mathcal{B})$ such that $\delta(s, u) = \delta(t, u)$. So, by Theorem 20.4, the word $u \in L(\mathcal{B})$ synchronizes $\mathcal{A}$. □

However, we do not know whether the above problem for monotonic automata is NL-hard.

20.2 Partially Ordered Automata

Here, we investigate the complexity of the constrained synchronization problem for partially ordered input automata and when the constraint language is given by a PDFA with up to two states. First, in Section 20.2.1, we state a quadratic upper bound for shortest synchronizing words of partially ordered automata in a given regular constraint language and conclude containment in NP of the corresponding decision problem. Section 20.2.2 states a technical lemma to simplify monoid constraint languages w.r.t. partially ordered input automata. In Section 20.2.3 we collect three particular cases of constraint languages giving problems in AC^0, that are L-complete, or NP-hard. Using these lemmas,

in Section 20.2.4, we give a complexity classification for two-state constraint PDFAs and partially ordered input automata.

20.2.1 A Bound and Containment in NP

In Theorem 20.7, we state a quadratic bound on shortest synchronizing words in a given constraint language. Then, in Theorem 20.8, we conclude that the constrained synchronization problem is in NP for partially ordered input automata.

Proposition 20.7 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton with n states and $\mathcal{B} = (\Sigma, P, \mu, p_0, F)$ be a fixed PDFA. Then, a shortest synchronizing word for $\mathcal{A}$ in $L(\mathcal{B})$ has length at most $|P| \cdot \binom{n}{2}$.*

Proof. Let $w = x_1 \cdots x_m \in L(\mathcal{B})$ with $x_i \in \Sigma$ and $S_i = \delta(Q, x_1 \cdots x_i)$ for $i \in \{1, \ldots, m\}$ and $S_0 = Q$. Suppose $m \geq |P|$ and $S_{i+|P|} = S_i$ for some $i \in \{0, 1, \ldots, n\}$. Since $\mathcal{A}$ is partially ordered, no word can induce a non-trivial permutation of a subset of states. So, for the word $u = x_{i+1} \cdots x_{i+|P|}$, for each $q \in S_j$, we have $\delta(q, u) = q$, and then $\delta(q, x_j) = q$ for $j \in \{i+1, \ldots, i+|P|\}$. Since u has length $|P|$, it induces a loop in the constraint automaton $\mathcal{B}$. So, we can replace some factor of w by a shorter word resulting in a new word $v \in \Sigma^*$ with $|v| < |w|$ such that $S_{i+|P|} = \delta(Q, x_1 \cdots x_i v x_{i+|P|+1} \cdots x_n)$ and $x_1 \cdots x_i v x_{i+|P|+1} \cdots x_n \in L(\mathcal{B})$.

Now, let $q_1, \ldots, q_n$ be a topological sorting of the states of $\mathcal{A}$ w.r.t. $\preceq_R$. Suppose $w \in L(\mathcal{B})$ is a shortest synchronizing word for $\mathcal{A}$ in $L(\mathcal{B})$. Factorize w into blocks of length $|P|$ and a remaining block of length at most $|P|$, i.e., write $w = w_1 \cdots w_m$ with $|w_i| = |P|$ for $i \in \{1, \ldots, m-1\}$ and $|w_m| \leq |P|$. By the arguments from the previous paragraph we have $\delta(Q, w_1 \cdots w_{i-1}) \neq \delta(Q, w_1 \cdots w_i)$ for $i \in \{1, \ldots, m\}$ (set $w_1 \cdots w_0 = \varepsilon$). Thus we can define a function $f \colon \{1, \ldots, m\} \to Q \times Q$ by

$$f(i) = (q_j, \delta(q_j, w_1 \cdots w_i))$$

where $j \in \{1, \ldots, m\}$ is some number such that $\delta(q_j, w_1 \cdots w_{i-1}) \neq \delta(q_j, w_1 \cdots w_i)$. Since $\mathcal{A}$ is partially ordered and $\delta(q_j, w_0 \cdots w_{i-1}) \preceq_R \delta(q_j, w_0 \cdots w_i)$, we find that f is injective. We also have

$$\delta(q_j, w_1 \cdots w_i) \in \{q_{j+1}, q_{j+2}, \ldots, q_n\},$$

which yields that the image of f is contained in the set $\bigcup_{j=1}^{n} \{q_j\} \times \{q_{j+1}, \ldots, q_n\}$ of size $(n-1) + (n-2) + \cdots + 1 = \binom{n}{2}$. This implies $m \leq \binom{n}{2}$ and so $|w| \leq |P| \cdot \binom{n}{2}$. □

By Proposition 20.7 we can conclude that for partially ordered input automata, the constrained synchronization problem is always in NP.

Theorem 20.8 *For partially ordered input automata and an arbitrary constraint automaton, the constrained synchronization problem is in* NP.

Proof. By Proposition 20.7 we can guess a shortest synchronizing word in $L(\mathcal{B})$ of polynomial length. Verifying that such a word is indeed synchronizing can be done in polynomial time as SyncWords is in L by Theorem 8.9. □

Lastly, we also give a bound for shortest words in a constraint language mapping one subset into another subset that, as a special case, also yields a quadratic bound for shortest synchronizing words.

Proposition 20.9 *Suppose $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ is a PDFA. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton and $S, T \subseteq Q$. Then for a shortest word $u \in L(\mathcal{B})$ such that $\delta(S, u) \subseteq T$ we have $|u| < |P||S|(|Q| - 1) + |P|$.*

Proof. We can suppose S and T are non-empty. Suppose $u \in L(\mathcal{B})$ is a shortest word such that $\delta(S, u) \subseteq T$. Write $u = u_1 \cdots u_m u_{m+1}$ with $|u_1| = \cdots = |u_m| = |P|$ and $|u_{m+1}| < |P|$. Assume there exists $i \in \{1, \ldots, m\}$ such that $\delta(s, u_1 \cdots u_{i-1}) \simeq_R \delta(s, u_1 \cdots u_{i-1} u_i)$ for all $s \in S$.[1] Since $\mathcal{A}$ is partially ordered, we find $\delta(s, u_1 \cdots u_{i-1}) = \delta(s, u_1 \cdots u_{i-1} v)$ for each prefix v of u_i and each $s \in S$. Since $|u_i| = |P|$, some state of $\mathcal{B}$ is repeated when reading u_i and thus a non-empty factor of u_i can be removed to obtain a shorter word $w \in \Sigma^*$ with $\mu(u_1 \cdots u_{i-1} u_i) = \mu(u_1 \cdots u_{i-1} w)$ and $\delta(s, u_1 \cdots u_{i-1} w) = \delta(s, u_1 \cdots u_{i-1} u_i)$ for each $s \in S$. Then $u_1 u_{i-1} w u_{i+1} \cdots u_m u_{m+1} \in L(\mathcal{B})$ and $\delta(S, u_1 \cdots u_{i-1} w u_{i+1} \cdots u_m) \subseteq T$, contradicting the minimal choice of u. So, for each $i \in \{1, \ldots, m\}$ there exists $s \in S$ such that $\delta(s, u_1 \cdots u_{i-1} u_i) \not\simeq_R \delta(s, u_1 \cdots u_{i-1})$. Now, every $s \in S$ can be mapped to a different state at most $|Q| - 1$ times until reaching a maximal state. Hence, for $s \in S$ and the maximal set of indices $1 \leq j_1 < \cdots < j_r \leq m$ such that $\delta(s, u_1 \cdots u_{j_l - 1}) \neq \delta(s, u_1 \cdots u_{j_l})$ for $l \in \{1, \ldots, r\}$ we have $r \leq |Q| - 1$. Since each $i \in \{1, \ldots, m\}$ appears as such an index for some state from S, we find $m \leq |S|(|Q| - 1)$ and thus $|u| < |P||S|(|Q| - 1) + |P|$. □

20.2.2 A Reduction Lemma

Here, in Theorem 20.10, we prove that for partially ordered input automata constraint languages that are monoids can be replaced by languages of the form Γ^* for $\Gamma \subseteq \Sigma$.

Lemma 20.10 *Suppose $L \subseteq \Sigma^*$ is a language that is a monoid and $\Gamma = \{a \in \Sigma \mid L \cap \Sigma^* a \Sigma^* \neq \emptyset\}$. If $\mathcal{A} = (Q, \Sigma, \delta)$ is a partially ordered automaton, then $\mathcal{A}$ has a synchronizing word in L if and only if $\mathcal{A}$ has a synchronizing word in Γ^*.*

Proof. First, suppose $w \in \Gamma^*$ synchronizes $\mathcal{A}$. Let $q_1, \ldots, q_n$ be a topological sorting of the states. Then $\delta(Q, w) = \{q_n\}$ and for each state q_i with $i \in \{1, \ldots, n-1\}$ there exists a letter $a_i \in \Gamma$ such that

$$w \in \Sigma^* a_i \Sigma^* \text{ and } \delta(q_i, a_i) \neq q_i.$$

[1] We set $u_1 \cdots u_0 = \varepsilon$.

By choice of Γ, for each $a_i \in \Gamma$ there exists a word $u_i \in L \cap \Sigma^* a_i \Sigma^*$. Then $\delta(q_i, u_i) \neq q_i$ and so $\delta(q_i, u_i) \in \{q_{i+1}, \ldots, q_n\}$. Hence $\delta(Q, u_1 \cdots u_i) \subseteq \{q_{i+1}, \ldots, q_n\}$ for $i \in \{1, \ldots, n-1\}$ and the word $u = u_1 \cdots u_{n-1}$ synchronizes $\mathcal{A}$. As L is a monoid, we have $u \in L$.

Conversely, we have $L \subseteq \Gamma^*$ so that every synchronizing word in L is also a synchronizing word in Γ^*. □

Remark 20.11 In the proof of Theorem 20.10, each letter is replaced by words containing this letter. This is done for each state, and we cannot simply replace letters by words containing those letters and obtain a synchronizing words in a partially ordered automaton, but must take the states moved by these letters into account. For example, consider the partially ordered automaton in Figure 20.1. It has ab as a synchronizing word, but $cacb$ is not a synchronizing word. Here, a moves the states q_1 and q_3 and b moves the state q_2. Considering the states q_1, q_2, q_3 in that order, we obtain the synchronizing word $cacbca$ from the method of proof of Theorem 20.10.

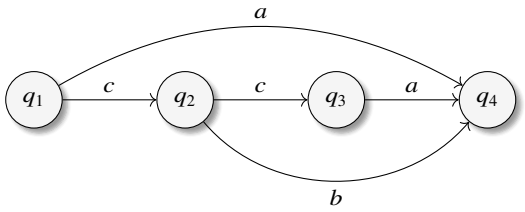


Fig. 20.1: Partially ordered automaton with synchronizing word ab, but $cacb$ is not a synchronizing word, see Theorem 20.11.

Remark 20.12 Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state partially ordered automaton and $\Gamma \subseteq \Sigma$ with $w \in \bigcap_{a \in \Gamma} \Sigma^* a \Sigma^*$. Then for $u \in \Gamma^*$ and $v \in \Sigma^*$ we have the following equivalences:

$$\begin{aligned}
\delta(Q, uv) = \{q\} &\Leftrightarrow \delta(Q, u) \subseteq \delta^{-1}(q, v) \\
&\Leftrightarrow \delta^{-1}(q, v) \text{ contains all trap states of } \mathcal{A}_{|\Gamma} && \text{(Cor. 7.54)} \\
&\Leftrightarrow \delta(Q, w^{n-1}) \subseteq \delta^{-1}(q, v) && \text{(Prop. 7.56)} \\
&\Leftrightarrow \delta(Q, w^{n-1}v) = \{q\}.
\end{aligned}$$

This implies that for $L \subseteq \Sigma^*$, $\mathcal{A}$ has a synchronizing word in $\Gamma^* L$ if and only if it has one in $w^{n-1}L$.

20.2.3 Three Special Cases

Here, we give three lemmas stating sufficient conditions for containment in AC^0 (Theorem 20.13), L-completeness (Theorem 20.14), and for NP-hardness (Theorem 20.15).

We use these lemmas in our classification for two-state constraint PDFAs in Section 20.2.4.

Lemma 20.13 *Let $\Sigma_1, \Sigma_2 \subseteq \Sigma$. Then, for a partially ordered automaton the problem to decide whether it has a synchronizing word in $\Sigma_1^*\Sigma_2$ is in AC^0.*

Proof. By Theorem 18.9 and as $\Sigma_1^*\Sigma_2 = \bigcup_{a\in\Sigma_2} \Sigma_1^* a$ we can assume $\Sigma_2 = \{a\}$. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered input automaton. By Theorem 7.58, $\mathcal{A}$ has a synchronizing word in $\Sigma_1^* a$ if and only if there exists a word in $\Sigma_1^* a$ mapping all states to a unique trap state $t \in Q$. This is equivalent to the existence of a word $u \in \Sigma_1^*$ such that $\delta(Q, u) \subseteq S$ where $S = \delta^{-1}(\{t\}, a)$. We assume some numbering $Q = \{1, \ldots, n\}$ of the states and that $\mathcal{A}$ is given by a set of transition matrices $(m_{ij}^{(a)})_{i,j\in Q}$ for each letter $a \in \Sigma$. Let $UniqueTrap_j = \left(\bigwedge_{a\in\Sigma} m_{jj}^{(a)}\right) \wedge \left(\bigwedge_{r\in Q\setminus\{j\}} \bigvee_{a\in\Sigma} \neg m_{rr}^{(a)}\right)$ be an abbreviation for the formula that checks whether j is a unique trap state. Having a trap state $t \in Q$ in $\mathcal{A}$, a state $i \in Q$ is in S if and only if the following formula is true:

$$\bigvee_{j\in Q} \left[UniqueTrap_j \wedge m_{ij}^{(a)}\right].$$

So, we can check whether $\mathcal{A}$ has a unique trap state and determine the states in S by an AC^0 circuit. Then, by feeding the input $(\mathcal{A}_{|\Sigma_1}, S)$ to SyncInto, we obtain an $\leq_m^{\mathsf{AC}^0}$-reduction to the problem SyncInto on partially ordered automata, which is in AC^0 by Theorem 8.28. □

Lemma 20.14 *Let $\Sigma_1, \Sigma_2 \subseteq \Sigma$ and $b \in \Sigma$ with $(\Sigma_1 \cup \Sigma_2) \setminus \{b\} \neq \emptyset$. Then the problem to test whether a given partially ordered automaton has a synchronizing word in $\Sigma_1^*\Sigma_2 b^*$ is L-complete.*

Proof. Containment in L: By Theorem 18.9 and as $\Sigma_1^*\Sigma_2 b^* = \bigcup_{a\in\Sigma_2} \Sigma_1^* a b^*$ we can assume $\Sigma_2 = \{a\}$. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton with n states. By Equation (7.5) and Theorem 7.1 a word uab^i with $i \geq 0$ and $u \in \Sigma_1^*$ synchronizes $\mathcal{A}$ if and only if uab^n synchronizes $\mathcal{A}$. By Theorem 7.58, the synchronizing state is a unique trap state. So, $\mathcal{A}$ has a synchronizing word in $\Sigma_1^* a b^*$ if and only if there exists a unique trap state $t \in Q$ and a word $u \in \Sigma_1^*$ such that $\delta(Q, u) \subseteq S$ where $S = \delta^{-1}(\{t\}, ab^n)$. By iterating through the states we can check whether $\mathcal{A}$ has a unique trap state $t \in Q$. Then, by iterating through the states $q \in Q$ again and computing, by using a counter and a memory cell, $\delta(q, ab^n)$ we can compute the set S. Then, solve SyncInto with input $(\mathcal{A}_{|\Sigma_1}, S)$, which is in AC^0 by Theorem 8.28 as $\mathcal{A}_{|\Sigma_1}$ is partially ordered.

L-hardness: We give an $\leq_m^{\mathsf{AC}^0}$-reduction from 1-DAG, which is L-complete by Theorem 3.76. Suppose (G, s, t) is a 1-DAG-ST-instance with $G = (V, E)$. We can suppose s has no incoming transitions and t has no outgoing transitions. Pick $u \in \Sigma_1^*\Sigma_2$ with $|u|_a > 0$ for some $a \in (\Sigma_1 \cup \Sigma_2) \setminus \{b\}$.

Construction: Let $\mathcal{A} = (V, \Sigma, \delta)$ with

$$\delta(p, a) = \begin{cases} s & \text{if } p \neq t, \\ t & \text{if } p = t. \end{cases} \qquad \delta(p, b) = \begin{cases} q & \exists (p, q) \in E, \\ p & \text{otherwise,} \end{cases}$$

and set $\delta(q, x) = q$ for all $x \in \Sigma \setminus \{a, b\}$.

Properties:

1. For each $q \in Q$ we have $\delta(q, u) = \delta(q, a)$.
   This follows since $|u|_b = 0$, only the letters a and b move any state, and $\delta(q, a) = \delta(q, aa)$ for each $q \in Q$.
2. For $v \in \Sigma^*$ and $|\delta(Q, v)| \geq 2$ we have $\delta(Q, va) = \{s, t\}$.
   As t is a trap state, $t \in \delta(Q, v)$. For $q \in \delta(Q, v)$ with $q \neq t$ we have $\delta(q, a) = \{s\}$. This yields the claim.
3. There exists a word $v \in \Sigma^*$ with $\delta(\{s, t\}, v) = \{t\}$ if and only if there exists a path of length n from s to t in G and in this case $\delta(\{s, t\}, b^n) = \{t\}$.
   If there exists a path of length n from s to t in G, then $\delta(\{s, t\}, b^n) = \{t\}$. Conversely, suppose $v \in \Sigma^*$ is a shortest word with $\delta(\{s, t\}, v) = \{t\}$. By Property 2 we can assume $|v|_a = 0$. As the other letters only induce self-loop, we have $v \in b^*$. Now, by following the transitions along b we obtain a path in G from s to t, i.e., the states $s, \delta(s, b), \delta(s, bb), \ldots, \delta(s, b^{|v|-1}), t$ give a path in G.

Verification: Suppose there exists a path of length n from s to t in G. By Properties 1, 2, and 3, $\delta(Q, ub^n) = \{t\}$. Conversely, suppose $v \in \Sigma_1^* \Sigma_2 b^*$ synchronizes $\mathcal{A}$. Then $\delta(Q, v) = \{t\}$. In particular $\delta(\{s, t\}, v) = \{t\}$. Thus, by Property 3, there exists a path from s to t in G. □

Lemma 20.15 *Let $\Sigma_1, \Sigma_2, \Sigma_3 \subseteq \Sigma$ be such that $\Sigma_1 \cap \Sigma_2 = \emptyset$ and there exist distinct letters $b, c \in \Sigma_3$ and $a \in (\Sigma_1 \cup \Sigma_2) \setminus \Sigma_3$. Then the problem to decide whether a given partially ordered automaton has a synchronizing word in $\Sigma_1^* \Sigma_2 \Sigma_3^*$ is* NP*-hard.*

Proof. We give a reduction from the NP-complete problem SAT (Theorem 3.76) that is a modification of a reduction from Eppstein [198] and Rystsov [528] used to show NP-hardness of SyncShort. A similar modification was used in [537, Thm. 4] to show that SyncFrom is NP-hard for partially ordered input automata.

Let (X, C) be a SAT-instance with variables $X = \{x_1, \ldots, x_n\}$ and clauses $C = \{c_1, \ldots, c_m\}$.

Construction: We construct a partially ordered automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q = \{q_{i,j} \mid 1 \leq i \leq m, 0 \leq j \leq n+1\} \cup \{q_f\}$ where q_f is a trap state and $\delta \colon Q \times \Sigma \to Q$ is given, for $i \in \{1, \ldots, m\}$, $j \in \{0, 1, \ldots, n+1\}$ and $x \in \Sigma$, by

$$\delta(q_{i,j}, x) = \begin{cases} q_{i,j+1} & \text{if } j \in \{1, \ldots, n\} \wedge ((x = b \wedge \neg x_j \notin c_i) \vee (x = c \wedge x_j \notin c_i)), \\ q_f & \text{if } j \in \{1, \ldots, n\} \wedge ((x = b \wedge \neg x_j \in c_i) \vee (x = c \wedge x_j \in c_i)), \\ q_{i,1} & \text{if } j = 0 \wedge x \in \Sigma_2, \\ q_f & \text{if } j \in \{1, 2, \ldots, n+1\} \wedge x = a, \\ q_{i,j} & \text{otherwise,} \end{cases}$$

and $\delta(q_f, x) = x$. See Figure 20.2 for an example.

Properties: Properties 1, 2, 3 and 4 follow readily. For the remaining two properties we give the proofs.

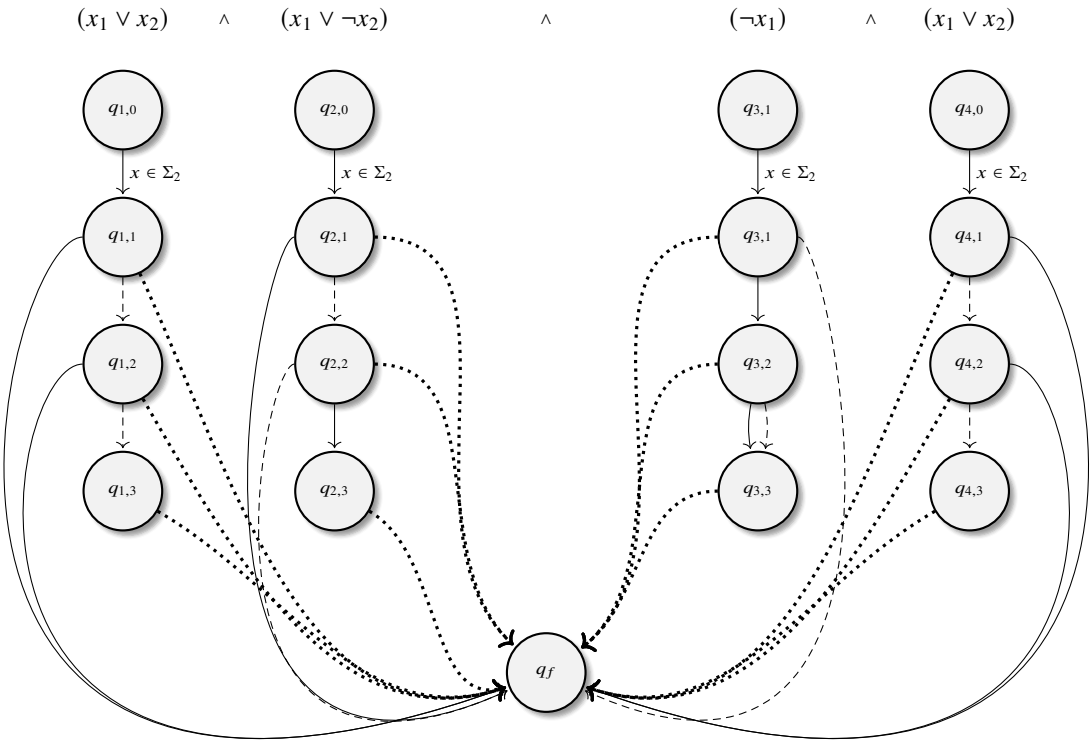


Fig. 20.2: Reduction from the proof of Theorem 20.16 for the formula $(x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1) \wedge (x_1 \vee x_2)$. Transitions labeled by a, b, and c are drawn with dotted, dashed, and solid lines, respectively. Self-loops omitted.

1. $\delta(q_{i,0}, x) \neq q_{i,0}$ if and only if $x \in \Sigma_2$.
2. For $u \in (\Sigma \setminus \{a\})^*$ and $i \in \{1, \dots, m\}$ we have $\delta(q_{i,1}, u) = \delta(q_{i,1}, \pi_{\{b,c\}}(u))$.
3. Let $i \in \{1, \dots, m\}$ and $j, j' \in \{0, 1, \dots, n+1\}$. If $\delta(q_{i,j}, u) = q_{i,j'}$ for some $u \in \Sigma^*$, then $j \leq j'$.
4. Let $i \in \{1, \dots, m\}$, $v_j \in \{b, c\}$ for $j \in \{1, \dots, n\}$ and $w \in \Sigma^*$. Then

$$\begin{aligned}
&\delta(q_{i,1}, v_1 \cdots v_n w) = q_f \\
&\Leftrightarrow \exists j \in \{1, \dots, n\} \colon (v_j = b \wedge x_j \in c_i) \vee (v_j = c \wedge \neg x_j \in c_i) \\
&\Leftrightarrow \text{the assignment } x_1, \dots, x_n \text{ satisfies the clause } c_i,
\end{aligned}$$

where

$$x_j = \begin{cases} 0 & \text{if } v_j = b, \\ 1 & \text{if } v_j = c \end{cases} \tag{20.1}$$

for $j \in \{1, \dots, n\}$.

5. Let $i \in \{1, \ldots, m\}$ and $v \in \{b, c\}^*$. Then $\delta(q_{i,1}, v) = q_f$ implies the existence of a satisfying assignment for the clause c_i.
   Let $w \in \mathrm{Pref}(vb^*) \cap \Sigma^n$. Then $\delta(q_{i,1}, w) = q_f$, which, for $|v| > n$, follows as otherwise the word v would end up in $q_{i,n+1}$, and, for $|v| \leq n$, as q_f is a trap state. Property 4 then yields the claim.
6. $\delta(Q, u) = \{q_f\}$ for $u \in \Sigma^*$ if and only if $u \in \Sigma^* a \Sigma^* \cap (\Sigma \setminus \Sigma_2)^* \Sigma_2 w$ for some $w \in \Sigma^*$ with $\delta(q_{i,1}, w) = q_f$ for all $i \in \{1, \ldots, m\}$.
   First, assume $u \in \Sigma^* a \Sigma^* \cap (\Sigma \setminus \Sigma_2)^* \Sigma_2 w$ with $\delta(q_{i,1}, w) = q_f$ for all $i \in \{1, \ldots, m\}$. Set $S = \{q_{i,j} \mid i \in \{1, \ldots, m\}, j \in \{1, \ldots, n+1\}\}$. By Property 3, $\delta(S, u) \subseteq S$. As $\delta(S, a) = \{q_f\}$ and $u \in \Sigma^* a \Sigma^*$, we find $\delta(S, u) = \{q_f\}$. As precisely letters from Σ_2 map $q_{i,0}$ to $q_{i,1}$ and all other letters fix $q_{i,0}$ we get $\delta(q_{i,0}, u) = \delta(q_{i,1}, w) = q_f$ for all $i \in \{1, \ldots, m\}$. Hence, $\delta(Q, u) = \{q_f\}$. Conversely, suppose $\delta(Q, u) = \{q_f\}$. As the states $q_{i,n+1}$ for $i \in \{1, \ldots, m\}$ must be mapped to q_f as well, we have $u \in \Sigma^* a \Sigma^*$. By Property 1 we get $u = vxw$ for some $v \in (\Sigma \setminus \Sigma_2)^*$, $x \in \Sigma_2$ and $w \in \Sigma^*$. Then $\delta(q_{i,0}, vx) = q_{i,1}$ for all $i \in \{1, \ldots, m\}$. So, $\delta(q_{i,1}, w) = q_f$ for $i \in \{1, \ldots, m\}$.

Verification: Let $v \in \Sigma_1^*$, $x \in \Sigma_2$ and $w \in \Sigma_3^*$ and set $S_1 = \{q_{i,1} \mid i \in \{1, \ldots, m\}\}$. Then we have the following equivalences:

$$
\begin{aligned}
&|\delta(Q, vxw)| = 1 \\
&\Leftrightarrow \delta(Q, vxw) = \{q_f\} && (q_f \text{ is a trap state}) \\
&\Leftrightarrow \delta(S_1, w) = \{q_f\} \wedge vxw \in \Sigma^* a \Sigma^* && (\text{Property 6 \& } v \in (\Sigma \setminus \Sigma_2)^*, x \in \Sigma_2) \\
&\Leftrightarrow \delta(S_1, w) = \{q_f\} \wedge vx \in \Sigma^* a \Sigma^* && (a \notin \Sigma_3) \\
&\Leftrightarrow \delta(S_1, \pi_{\{b,c\}}(w)) = \{q_f\} \wedge vx \in \Sigma^* a \Sigma^*. && (\text{Property 2 \& } a \notin \Sigma_3)
\end{aligned}
$$

By Property 5, $\delta(S_1, \pi_{\{b,c\}}(w)) = \{q_f\}$ for some $w \in \Sigma_3^*$ if and only if there exists a satisfying assignment for each clause c_i, $i \in \{1, \ldots, m\}$. So, $|\delta(Q, vxw)| = 1$ with $v \in \Sigma_1^*$, $x \in \Sigma_2$, and $w \in \Sigma_3^*$ implies that the given SAT-instance is solvable, i.e., a YES-instance. Conversely, suppose there exists a satisfying assignment for each clause. Then, by Property 4, $\delta(S_1, w) = \{q_f\}$ for some $w \in \{b, c\}^* \subseteq \Sigma_3^*$. As $a \in \Sigma_1 \cup \Sigma_2$ we find $v \in \Sigma_1^*$ and $x \in \Sigma_2$ such that $vx \in \Sigma^* a \Sigma^*$. Then $|\delta(Q, vxw)| = 1$ with $vxw \in \Sigma_1^* \Sigma_2 \Sigma_3^*$. □

20.2.4 Classification for Two-State Constraint PDFAs

Here, in Theorem 20.16, we classify the complexity for constraint automata with two states. We find that some constraint languages giving PSPACE-hard problems in the general case, as for example $(a + b)^* c a^*$ or $(a + b)^* c c^*$, yield L-complete problems or, like $(a + b)^* c$, problems in AC^0 when restricted to partially ordered input automata. But we also find problems that are PSPACE-hard in the general case where the complexity only drops to NP-hardness, as for example for the constraint language $a(b + c)^*$. With the assumption from Theorem 17.88, for one-state automata the constraint language is either empty or equals Σ^*. So, in this case, by Theorem 8.29, for partially ordered input automata the constrained synchronization problem is in AC^0.

For a two-state constraint PDFA, we use the notational conventions as introduced in Section 19.4 and the assumption that the constraint PDFA is trim.

Theorem 20.16 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a two-state constraint automaton that is trim with $P = \{1, 2\}$, $p_0 = 1$, $F = \{2\}$, and $\Sigma_{i,j} = \{x \in \Sigma \mid \mu(i, x) = j\}$ for $i, j \in P$. Then the problem to decide whether a given partially ordered automaton $\mathcal{A}$ has a synchronizing word in $L(\mathcal{B})$ is*

1. NP*-complete for $\Sigma_{2,1} = \emptyset$, $(\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset$ and $|\Sigma_{2,2}| \geq 2$,*
2. L*-complete for $\Sigma_{2,1} = \emptyset$, $(\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset$ and $|\Sigma_{2,2}| = 1$,*
3. *in AC^0 in the remaining cases, which are covered by the following conditions:*
   a. *$\mathcal{B}$ is strongly connected (equivalently $\Sigma_{2,1} \neq \emptyset$), or*
   b. *$\Sigma_{2,2} = \emptyset$, or*
   c. *$\Sigma_{1,1} \cup \Sigma_{1,2} \subseteq \Sigma_{2,2}$.*

Proof. As $\mathcal{B}$ is trim we have $\Sigma_{1,2} \neq \emptyset$. First, suppose $\Sigma_{2,1} \neq \emptyset$. Then $\mathcal{B}$ is strongly connected. So $\mathrm{Fact}(L(\mathcal{B})) = \mathrm{Fact}(L(\mathcal{B}_{p_0,\{p_0\}}))$. By Theorem 17.75, $L(\mathcal{B}_{p_0,\{p_0\}})$ is a monoid. By Theorems 18.4 and 20.10, the problem is equivalent to finding a synchronizing word in Γ^* for some $\Gamma \subseteq \Sigma$, which is in AC^0 by Theorem 8.29 as we can assume $\Gamma = \Sigma$ (see Section 17.8). Next, suppose $\Sigma_{2,1} = \emptyset$. Then $L(\mathcal{B}) = \Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^*$. For $\Sigma_{2,2} = \emptyset$, the problem is in AC^0 by Theorem 20.13. If $\Sigma_{1,1} \cup \Sigma_{1,2} \subseteq \Sigma_{2,2}$ (and $\Sigma_{2,1} = \emptyset$), then $\mathrm{Fact}(L(\mathcal{B})) = \mathrm{Fact}(\Sigma_{2,2}^*)$ and containment in AC^0 follows from Theorem 18.4 and Theorem 8.29 by considering $\mathcal{A}_{|\Sigma_{2,2}}$. Now, suppose $(\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset$ and $\Sigma_{2,2} \neq \emptyset$. For $|\Sigma_{2,2}| = 1$, L-completeness follows from Theorem 20.14. For $|\Sigma_{2,2}| \geq 2$, NP-hardness follows from Theorem 20.15. By Theorem 20.8 we can decide in NP whether $\mathcal{A}$ has a synchronizing word in $L(\mathcal{B}) = \Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^*$. □

Comparing Theorem 20.16 and Theorem 19.21, $(\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset$ with $|\Sigma_{1,1}| \geq 2$ and $|\Sigma_{2,2}| \leq 1$ implies intractability in the general case, whereas for partially ordered input automata we obtain a tractable problem. However, for $(\Sigma_{1,1} \cup \Sigma_{1,2}) \setminus \Sigma_{2,2} \neq \emptyset$ and $|\Sigma_{2,2}| \geq 2$, the constrained synchronization problem is intractable in the general case and for partially ordered input automata, where we have PSPACE-completeness in the former case and NP-completeness in the latter case.

20.3 ρ-Automata With a Trap State

First, in Theorem 20.17, we give a quadratic bound for shortest synchronizing words of ρ-automata with a trap state in a given constraint language. Then, in Theorem 20.18, we prove that the constrained synchronization problem for ρ-automata with a trap state is in NP, and there exist constraint languages giving NP-hard problems.

Proposition 20.17 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA. If $\mathcal{A} = (Q, \Sigma, \delta)$ is a ρ-automaton with a trap state having a synchronizing word in $L(\mathcal{B})$, then there exists a synchronizing word $u \in L(\mathcal{B})$ with $|u| \leq |P| \cdot \binom{|Q|}{2}$.*

Proof. By Theorems 7.64 and 7.65 the automaton $\mathcal{A}/\simeq_R$ has the same set of synchronizing words and is partially ordered. So, the bound follows from Theorem 20.7. □

The assumption of a trap state is essential in Theorem 20.17. For example, the Černý family (Figure 1.7) consists of ρ-automata with shortest synchronizing words of length $(n-1)^2$.

Theorem 20.18 *The constrained synchronization problem for ρ-automata with a trap state as input is in* NP. *Furthermore, there exist constraint languages for which this problem is* NP*-complete.*

Proof. By Theorem 20.17 the length of a shortest synchronizing word is polynomially bounded in the size of the input automaton. So, it can be guessed in polynomial time and checked in polynomial time as SyncWords is in L (Theorem 8.9). The construction in the proof of Lemma 20.15 gave a partially ordered automaton, which is also a ρ-automaton by Theorem 7.63, with a trap state. This shows NP-hardness for the same constraint languages as stated in Lemma 20.15. □

The constrained synchronization problem from Theorem 18.2, Item 6 is PSPACE-complete even for strongly connected automata, which are ρ-automata.

20.4 Commutative and Unary Automata

Here, we show that for commutative n-state input automata and an arbitrary regular constraint language, the constrained synchronization problem is in NL (Theorem 20.19) and solvable in time $O(|\Sigma|^2 n^{|\Sigma|+2})$ (Theorem 20.20). For unary automata (which are commutative), by Theorems 19.1 and 19.3, the problem is L-complete when the constraint language is infinite and in AC^0 otherwise.

Theorem 20.19 *Fix a regular language $L \subseteq \Sigma^*$. Then, for a commutative input n-state automaton $\mathcal{A} = (Q, \Sigma, \delta)$, the problem whether $\mathcal{A}$ admits a synchronizing word in L is in* NL.

Proof. Let $\Sigma = \{a_1, \ldots, a_k\}$ and $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA such that $L = L(\mathcal{B})$. We give an algorithm that uses a memory cell to store a state from $\mathcal{B}$ initialized with p_0 and k counters $c_1, \ldots, c_k$. The algorithm repeats at most $|P| \cdot \binom{n}{2}$ times the following steps to compute, in the i-th step, a letter $x_i \in \Sigma$:

1. Guess the letter $x_i \in \Sigma$, update the state p stored in the memory cell with $\mu(p, x_i)$ and add one to the counter c_j where $x_i = a_j$, $j \in \{1, \ldots, k\}$.
2. If $\mu(p, x_i) \in F$, then iterate through the states $q \in Q$, compute the state $\delta(q, a_1^{c_1} \cdots a_k^{c_k})$, and check whether this always results in the same state. If this is the case, then $a_1^{c_1} \cdots a_k^{c_k}$ synchronizes $\mathcal{A}$. We have $|x_1 \cdots x_i|_{a_j} = c_j$ for each $j \in \{1, \ldots, k\}$ and, since $\mathcal{A}$ is commutative and so its set of synchronizing words is commutative by Theorem 7.90, Item 1, $a_1^{c_1} \cdots a_k^{c_k}$ synchronizes $\mathcal{A}$ if and only if $x_1 \cdots x_i$ synchronizes $\mathcal{A}$. So, for $\mu(p, x_i) \in F$ and when $a_1^{c_1} \cdots a_k^{c_k}$ synchronizes

$\mathcal{A}$, we can abort the procedure and return YES. Otherwise, we continue with the next step or, for $i > |P| \cdot \binom{n}{2}$, abort the procedure and return NO.

Part 2 of the above algorithm can be realized by using a memory cell to compute $\delta(q, a_1^{c_1} \cdots a_k^{c_k})$ on-the-fly, an additional memory cell to store a resulting state $\delta(q, a_1^{c_1} \cdots a_k^{c_k})$ for comparison between different passes, a counter to iterate through the states $q \in Q$, a counter to iterate through $c_1, \ldots, c_k$ and a counter to iterate from zero to c_j for each $j \in \{1, \ldots, k\}$. By using a binary encoding all counters can be stored using logarithmic space.

By Theorem 7.63, a commutative automaton is a ρ-automaton. So, by Theorem 20.17, if there exists a synchronizing word in L, then there exists one of length at most $|P| \cdot \binom{n}{2}$. Therefore, by nondeterminism, the above algorithm finds it, which shows its correctness. □

Theorem 20.20 *Fix a regular language $L \subseteq \Sigma^*$. Then, for a commutative input automaton $\mathcal{A}$ with n states, the problem whether $\mathcal{A}$ admits a synchronizing word in L is solvable in time $O(|\Sigma|^2 n^{|\Sigma|+2})$.*

Proof. By Proposition 12.5 we can compute in time $O(n^{|\Sigma|+2})$ an automaton C of size $n^{|\Sigma|}$ recognizing the set of synchronizing words of $\mathcal{A}$. Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be an automaton such that $L = L(\mathcal{B})$. By Equation (6.22), the product automaton of C and $\mathcal{B}$ accepts $L(C) \cap L$. and can be computed in time $O(|P||\Sigma|^2 n^{|\Sigma|})$. Then, checking that the resulting automaton for the intersection accepts a non-empty language can also be done in time $O(|P||\Sigma|^2 n^{|\Sigma|})$ where $O(|P||\Sigma|^2 n^{|\Sigma|}) = O(|\Sigma|^2 n^{|\Sigma|})$ since $\mathcal{B}$ is fixed, as we simply have to check whether there exists a path from the start state to a final state which can be done by a breadth-first search algorithm [163]. □

The degree of the polynomial measuring the running time depends on the alphabet size. But note that a fixed constraint automaton also fixes the alphabet, hence this parameter is not allowed to vary in the input semiautomata.

20.5 Solvable ρ-Automata

Every synchronizing solvable automaton has a trap state (Theorem 7.112). So, by Theorem 20.17, we get the following result.

Proposition 20.21 *Let $\mathcal{B} = (P, \Sigma, \mu, p_0, F)$ be a PDFA. If $\mathcal{A} = (Q, \Sigma, \delta)$ is a solvable ρ-automaton having a synchronizing word in $L(\mathcal{B})$, then there exists a synchronizing word $u \in L(\mathcal{B})$ with $|u| \leq |P| \cdot \binom{|Q|}{2}$.*

Proof. If $\mathcal{A}$ is synchronizing, it has a trap state by Theorem 7.112. So, it is a synchronizing ρ-automaton with a trap state. Hence, the result follows from Theorem 20.17. □

As a synchronizing solvable automaton has a trap state, with the same argument as for ρ-automata with a trap state (see Theorem 20.18), we can conclude that for solvable ρ-automata as input the constrained synchronization problem is in NP. Furthermore, as

partially ordered automata are solvable ρ-automata, by the results from Section 20.2.4, there exist constraint languages giving NP-complete problems.

Theorem 20.22 *The constrained synchronization problem for solvable ρ-automata as input is in* NP. *Furthermore, there exist constraint languages for which this problem is* NP*-complete.*

20.6 Automata with Simple Idempotents

Here, we investigate the constrained synchronization problem for binary SI-automata as input. Section 20.6.1 collects results for constraint languages that are monoids. Section 20.6.2 collects results for bounded constraint languages. Lastly, Section 20.6.3 shows that for constraint PDFAs with up to three states and binary SI-automata as input the constrained synchronization problem is in NL.

In the remainder of this section we denote by $\mathcal{A}_t$ the synchronizing binary SI-automaton from Figure 7.4c with a trap state and by $\mathcal{A}_c$ the other automaton from Figure 7.4d where b permutes the states, a is a simple idempotent and $s, t \in Q$ and $0 < d < n$ are the parameters as in Theorem 7.31 w.r.t. $\mathcal{A}_t$ and $\mathcal{A}_c$ for $n \geq 4$.

20.6.1 Monoid Constraint Languages

Here, we collect various results on monoid languages. For example, Theorem 20.26 characterizes those words $u \in b^*ab^*$ with precisely one a such that u^* contains a synchronizing word for $\mathcal{A}_c$.

Proposition 20.23 *Suppose $L \subseteq ((ba^*)^{m-1}b)^*$ for a divisor m of n (resp. $n-1$) with $m > 1$. Then there exists a proper subset of states of size n/m (resp. $(n-1)/m$) permuted by each word from L in $\mathcal{A}_c$ (resp. $\mathcal{A}_t$).*

Proof. Recall that $s \in Q$ denotes the unique state with $\delta(s, a) \neq s$. Set $S = \{\delta(s, b^{mi}) \mid i \geq 0\}$. Then $|S| = n/m$ (resp. $(n-1)/m$) in $\mathcal{A}_c$ (resp. $\mathcal{A}_t$). We have $s \notin \delta(S, b^i)$ for all $1 \leq i \leq m-1$. Thus $\delta(q, a) = q$ for all $q \in \delta(S, b^i)$ with $1 \leq i \leq m-1$. This gives $\delta(S, u) = \delta(S, u^{|u|_b}) = S$ for all $u \in L$. □

Corollary 20.24 *For $\mathcal{A}_c$ and n even (resp. $\mathcal{A}_t$ and n odd) there exists a proper subset of states of size $n/2$ (resp. $(n-1)/2$) permuted by each word in $(ba^*b)^*$.*

The next statement generalizes Theorem 7.32, Item 1.

Proposition 20.25 *Let $u \in \{a, b\}^*$ with $|u|_a > 0$ and $|u|_b > 0$ and suppose $|u|_b$ and $n-1$ are coprime. Then* $\mathrm{Syn}(\mathcal{A}_t) \cap u^* \neq \emptyset$.

Proof. As $|u|_b$ and $n-1$ are coprime, there exists $i \geq 1$ such that $i \cdot |u|_b \equiv 1 \pmod{n-1}$ and so $b^{i\cdot|u|_b}$ and b induce the same permutation on Q. Hence, for each $q \in Q$, we have $\delta(q, u^i) \in \{t, \delta(q, b)\}$. As $|u|_a > 0$ there exists $q \in Q \setminus \{t\}$ such that $\delta(q, u) = t$. So, we obtain that for each $p \in Q \setminus \{t\}$ there exists $0 \leq j < n-1$ such that $\delta(p, u^j) \in \{t, q\}$, which gives $\delta(p, u^{j+1}) = t$. Hence $\delta(p, u^{n-1}) = t$ for all $p \in Q$ and so $u^{n-1} \in \mathrm{Syn}(\mathcal{A}_t)$. □

Proposition 20.26 *Let $u \in b^*ab^*$. Then $\mathrm{Syn}(\mathcal{A}_c) \cap u^* \neq \emptyset$ if and only if $d + |u|_b \equiv 0 \pmod{n}$.*

Proof. We have $\mathrm{Fact}(u^*) = \mathrm{Fact}((ab^{|u|_b})^*)$. So, by Theorem 18.4, we can assume $u = av$ for some $v \in b^*$. Let $m = \gcd(|u|_b, n)$. Then v induces m cycles of size n/m on the states. As a only moves the state s, for $m > 1$, we get $\mathrm{Syn}(\mathcal{A}_c) \cap u^* = \emptyset$. Also, since $d + |u|_b \equiv 0 \pmod{n}$ implies $d + |u|_b \equiv 0 \pmod{m}$ and so $d \equiv 0 \pmod{m}$, for $m > 1$, we have $d + |u|_b \not\equiv 0 \pmod{n}$. So, the claim follows when it holds for $m = 1$, which we assume in the following. Then $d \equiv j \cdot |u|_b \pmod{n}$ for some $0 < j < n$. So, a induces $[s \mapsto \delta(s, v^j)]$ and v induces $(s\ \delta(s, v)\ \ldots\ \delta(s, v^{n-1}))$ as transformations on the states. By Equation (4.19), the largest set permuted by av is $\{\delta(s, v^{j+1}), \delta(s, v^{j+2}), \ldots, \delta(s, v^n)\}$. Hence, $\mathrm{Syn}(\mathcal{A}_c) \cap u^* \neq \emptyset$ if and only if $\delta(s, v^{j+1}) = \delta(s, v^n) = s$, which is equivalent to $(j+1)|u|_b \equiv 0 \pmod{n}$. As $(j+1)|u|_b \equiv d + |u|_b \pmod{n}$, $\mathrm{Syn}(\mathcal{A}_c) \cap u^* \neq \emptyset$ if and only if $d + |u|_b \equiv 0 \pmod{n}$. □

The condition $d + |u|_b \equiv 0 \pmod{n}$ implies that $|u|_b$ and n are coprime. For certain words u, this is, in fact, a necessary condition. For example, the word bab^{m-1} for a divisor $m > 1$ of n cannot synchronize $\mathcal{A}_c$ by Theorem 20.23.

Corollary 20.27 *Let $u \in b^*ab^*$ and $i \geq 1$ coprime to n. Then $(u + b^i)^* \cap \mathrm{Syn}(\mathcal{A}_c) \neq \emptyset$.*

Proof. By coprimality of i and n there exists $j \geq 0$ such that $|u|_b + j \cdot i + d \equiv 0 \pmod{n}$. Then set $v = ub^{j\cdot i}$ and apply Proposition 20.26. □

Corollary 20.28 *Let $\mathcal{A}_t = (Q, \{a, b\}, \delta)$ be an n-state SI-automaton with $n \geq 4$, simple idempotent letter a and permutational letter b. Then the following are equivalent.*

1. $\mathrm{Syn}(\mathcal{A}) \cap (a + bb)^* \neq \emptyset$,
2. $\mathrm{Syn}(\mathcal{A}) \cap (ba^*b)^* \neq \emptyset$,
3. *$\mathcal{A} = \mathcal{A}_t$ and n is even or $\mathcal{A} = \mathcal{A}_c$ and n is odd.*

Proof. $1 \Leftrightarrow 2$: As $\mathrm{Fact}((a + bb)^*) = \mathrm{Fact}((ba^*b)^*)$, this follows from Theorem 18.4.

$3 \Rightarrow 2$: For $\mathcal{A} = \mathcal{A}_t$ and n even, by Theorem 20.25, we have $\mathrm{Syn}(\mathcal{A}_t) \cap (ba^*b)^* \neq \emptyset$. For $\mathcal{A} = \mathcal{A}_c$ and n odd, by Theorem 20.27, we have $\mathrm{Syn}(\mathcal{A}_c) \cap (a + bb)^* \neq \emptyset$ and the claim follows as 1 and 2 are equivalent.

$2 \Rightarrow 3$: By Theorem 7.31, we have $\mathcal{A} = \mathcal{A}_t$ or $\mathcal{A} = \mathcal{A}_c$. By Theorem 20.24 we find that in the former case n must be even, and in the latter case n must be odd. □

Lastly, we collect various constraint languages which give a problem equivalent to the unconstrained problem for a specific choice of a simple idempotent letter and a permutational letter.

Lemma 20.29 *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be an n-state SI-automaton with simple idempotent letter a and permutational letter b. Then the following are equivalent:*

1. $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$,
2. $\mathrm{Syn}(\mathcal{A}) \cap (bb^*a)^* \neq \emptyset$,
3. $\mathrm{Syn}(\mathcal{A}) \cap (ab^*a)^* \neq \emptyset$,
4. $\mathrm{Syn}(\mathcal{A}) \cap (b + aa)^* \neq \emptyset$,
5. $\mathrm{Syn}(\mathcal{A}) \cap (b + ab)^* \neq \emptyset$,
6. $\mathrm{Syn}(\mathcal{A}) \cap (bb + ab)^* \neq \emptyset$,
7. $\mathrm{Syn}(\mathcal{A}) \cap (bb + ba)^* \neq \emptyset$.

Proof. $1 \Leftrightarrow 2 \Leftrightarrow 3 \Leftrightarrow 4 \Leftrightarrow 5$: As a is a simple idempotent we can replace, for any $i > 0$, each a by a^i. This gives the following equivalences:

$$\mathrm{Syn}(\mathcal{A}) \neq \emptyset \Leftrightarrow (bb^*a)^* \cap \mathrm{Syn}(\mathcal{A}) \neq \emptyset \Leftrightarrow (ab^*a)^* \cap \mathrm{Syn}(\mathcal{A}) \neq \emptyset$$
$$\Leftrightarrow (b + aa)^* \cap \mathrm{Syn}(\mathcal{A}) \neq \emptyset \Leftrightarrow (b + ab)^* \cap \mathrm{Syn}(\mathcal{A}) \neq \emptyset.$$

$1 \Leftrightarrow 6$: Suppose $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$. By Theorem 7.31 we have that $\mathcal{A}$ equals $\mathcal{A}_t$ or $\mathcal{A}_c$. Set

$$w = \begin{cases} (ab)^{n-1} & \text{if } \mathcal{A} = \mathcal{A}_t, \\ (ab^{n-d})^{n-2}ab & \text{if } \mathcal{A} = \mathcal{A}_c \text{ and } n \text{ is even}, \\ (a(b(b^{n+1})^{n-d-1})^{n-2}ab & \text{if } \mathcal{A} = \mathcal{A}_c \text{ and } n \text{ is odd}. \end{cases}$$

For odd n and $\mathcal{A} = \mathcal{A}_c$, $b^{n+1} \in (bb)^*$ induces the same transformation as b on the states. Then, by Theorem 7.32, in each case, we have $w \in \mathrm{Syn}(\mathcal{A}) \cap (bb + ab)^*$. This yields that $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$ if and only if $\mathrm{Syn}(\mathcal{A}) \cap (bb + ab)^* \neq \emptyset$.

$6 \Leftrightarrow 7$: As $\mathrm{Fact}((bb+ab)^*) = \mathrm{Fact}((bb+ba)^*)$, this follows from Theorem 18.4. □

20.6.2 Bounded Constraint Languages

Here, Theorem 20.30 states a lower bounds in terms of $\rho\colon \Sigma^* \to \mathbb{N}_0$ for the size of subsets reachable from a given set (recall that $\rho\colon \Sigma^* \to \mathbb{N}_0$ from Section 17.7 measures the number of symbol switches). Then, in Theorem 20.31, we show that we can decide in constant time whether a given SI-automaton has a synchronizing word in a fixed letter-bounded language. Lastly, in Theorem 20.32, we show that deciding whether a given SI-automaton has a synchronizing word in a regular language in $u_1^* \cdots u_k^*$ with $\rho(u_j) \leq 1$ for each $j \in \{1, \ldots, k\}$ is in L.

Proposition 20.30 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton, $S \subseteq Q$ and $u \in \Sigma^*$. Then $|S| - \rho(u) - 1 \leq |\delta(S, u)|$. In particular, $|\delta(S, u)| = 1$ implies $\rho(u) \geq |S| - 2$.*

Proof. First, let $i \geq 0$ be maximal such that $u = vx^i$ for some $v \in \Sigma^*$ and $x \in \Sigma$. Since x is a permutational letter or a simple idempotent, we have

$$|\delta(S, v)| - 1 \leq |\delta(S, u)|. \tag{20.2}$$

Next, to prove the statement, we do induction on $\rho(u)$. For $\rho(u) = 0$, we have $v = \varepsilon$ and Equation (20.2) gives $|S| - 1 \leq |\delta(S, u)|$. Next, suppose $\rho(u) > 0$. Then v ends with a letter different from x, so $\rho(u) = \rho(v) + 1$. Now, we have the following:

$$\begin{aligned} |S| - \rho(u) - 1 &= |S| - \rho(v) - 1 - 1 && (\rho(u) = \rho(v) + 1) \\ &\leq |\delta(S, v)| - 1 && \text{(induction hypothesis)} \\ &\leq |\delta(S, u)|. && \text{(Eqn. (20.2))} \end{aligned}$$

Thus, by induction, we obtain the claim. □

Corollary 20.31 *Let $L \subseteq \Sigma^*$ be a fixed letter-bounded language.*

1. *There exist only finitely many SI-automata $\mathcal{A}$ with $\mathrm{Syn}(\mathcal{A}) \cap L \neq \emptyset$.*
2. *For a given SI-automaton we can decide in constant time whether it has a synchronizing word in L.*

Proof. 1 By Theorem 17.69 we have $\rho(u) \leq N$ for some $N \geq 0$ and all $u \in L$. Then, Theorem 20.30 gives that for an SI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$ we have $|\delta(Q, u)| \geq |Q| - N - 1$ so that for a synchronizing word $u \in \Sigma^*$ we find $N + 2 \geq |Q|$.

2 By 1, to test whether a given SI-automaton has a synchronizing word in L we only have to compare it against finitely many automata, which can be done in constant time. □

Proposition 20.32 *Let $L \subseteq u_1^* \cdots u_k^*$ be regular with $u_1, \ldots, u_k \in a^*b^* \cup b^*a^*$. Then deciding whether an SI-automaton $\mathcal{A} = (Q, \{a, b\}, \delta)$ has a synchronizing word in L is in L.*

Proof. Let n be the number of states of $\mathcal{A}$. We can assume $n \geq 1$ and that $\mathcal{A}$ has a simple idempotent letter and a permutational letter. Since L is regular, it has a finite number m of Myhill-Nerode equivalence classes by Theorem 6.86.

Let $u_1^{m_1} \cdots u_k^{m_k} \in L$ be a shortest synchronizing word for $\mathcal{A}$. Assume we have $m_j \geq nm$ for some $j \in \{1, \ldots, k\}$. By finiteness, we have

$$u_1^{m_1} \cdots u_{j-1}^{m_{j-1}} u_j^{i_1} \equiv_L u_1^{m_1} \cdots u_{j-1}^{m_{j-1}} u_j^{i_1 + p_1}$$

for some $0 \leq j_1 < m$ and $0 < p_1 < m$. Now, u_j either is a permutational word, a simple idempotent, or the composition of a permutational word with a simple idempotent. With Theorem 4.13 we have $\delta(q, u_j^{i_2}) = \delta(q, u_j^{i_2 + p_2})$ for all $q \in Q$ for some $0 \leq i_2 < n$ and $0 < p_2 < n$. Since

$$nm - p_1 p_2 = (n - p_2)m + p_2(m - p_1) = n(m - p_1) - (n - p_2)p_1$$

we have $m_j - p_1 p_2 \geq nm - p_1 p_2 \geq \max\{j_1, j_2\}$. Thus, we find

$$u_1^{m_1} \cdots u_j^{m_j - p_1 p_2} \cdots u_k^{m_k} \equiv_L u_1^{m_1} \cdots u_j^{m_j} \cdots u_k^{m_k}$$

and $\delta(q, u_j^{m_j - p_1 p_2}) = \delta(q, u_j^{m_j})$ for all $q \in Q$. This yields that $u_1^{m_1} \cdots u_j^{m_j - p_1 p_2} \cdots u_k^{m_k} \in L$ and that this word synchronizes $\mathcal{A}$, which contradicts the minimal choice of the original word. Thus, we have $m_j < nm$ for all $j \in \{1, \ldots, k\}$.

Hence, there exists a synchronizing word in L if and only if there exists one of the form $u_1^{m_1} \cdots u_k^{m_k}$ in L with $0 \leq m_1, \ldots, m_k < nm$, and so we only have to search among these words. By utilizing k counters each using $\log nm$ space, we iterate through all tuples $(m_1, \ldots, m_k)$ representing the words $u_1^{m_1} \cdots u_k^{m_k}$ with $0 \leq m_1, \ldots, m_k < nm$. In each iteration, we test whether $u_1^{m_1} \cdots u_k^{m_k}$ synchronizes $\mathcal{A}$ and is in L. The word $u_1^{m_1} \cdots u_k^{m_k}$ can be tested to be synchronizing for $\mathcal{A}$ in deterministic log-space by iterating through $Q = \{q_1, \ldots, q_n\}$, compute $\delta(q_i, u_1^{m_1} \cdots u_k^{m_k})$ on-the-fly and test, for $i > 1$, whether q_{i-1} equals q_i. This only requires two additional memory cells and three counters for the state q_i, $i \in \{1, \ldots, n\}$, for the index $j \in \{1, \ldots, k\}$ and for u_j^i with $0 \leq i \leq m_j$. Checking whether $u_1^{m_1} \cdots u_k^{m_k} \in L$ with input $(m_1, \ldots, m_k)$ can be done by simulating an automaton accepting L from the start state while using counters as before. □

20.6.3 Binary SI-Automata and Small Constraint Automata

Here, first, in Theorem 20.33 we show L-hardness of deciding whether a given binary SI-automaton has a synchronizing word in a certain constraint language. Recall that by Theorem 13.10 the unconstrained synchronization problem is L-complete on binary SI-automaton as input. In Theorem 20.35 we show that for binary 3-state constraint PDFAs $\mathcal{B}$ with two strongly connected components, deciding whether a given SI-automaton has a synchronizing word in $L(\mathcal{B})$ is in L. Lastly, Theorem 20.37 states that the constrained synchronization problem for binary constraint PDFAs with at most three states and SI-automata as input is in NL.

Proposition 20.33 *Let $L \subseteq \{a, b\}^*$ be such that $vu^*w \subseteq L$ for some $v, w \in \Sigma^*$ and $u \in \Sigma^* a \Sigma^* \cap \Sigma^* b \Sigma^*$. Then it is L-hard w.r.t. $\leq_m^{\mathsf{AC}^0}$-reductions to test whether a given input SI-automaton has a synchronizing word in L.*

Proof. We give a reduction from SingleCycle, which is L-complete by Theorem 3.76. Let $f\colon [m] \to [m]$ be a permutation. By modifying f by adding a new point n and mapping $m - 1$ to m and m to $f(m - 1)$, we obtain a permutation on $[m + 1]$ that cyclically permutes its domain if and only if f cyclically permutes $[m]$. So, assume that m and $|u|_b$ are coprime. Then, let $\mathcal{A} = (Q, \{a, b\}, \delta)$ with $Q = [m] \cup \{t\}$ where a induces a simple idempotent mapping 0 to t, $\delta(t, b) = t$ and $\delta(q, b) = f(q)$ for $q \in [m]$.

For the input 1^m, since coprimality can be tested in L for unary inputs by Theorem 3.79, a Turing machine can determine how many points must be added to the input f so that the total number of points is coprime to $|u|_b$, and can compute an AC^0 circuit realizing this modification of f and computing $\mathcal{A}$. Hence, we obtain an $\leq_m^{\mathsf{AC}^0}$-reduction.

Now, f cyclically permutes $[m]$ if and only if $\mathcal{A}$ is isomorphic to the automaton from Figure 7.4c. So, if f cyclically permutes $[m]$, by Theorem 20.25, as $|Q| - 1$ and $|u|_b$ are coprime we have $u^i \in \mathrm{Syn}(\mathcal{A})$ for some $i \geq 1$. Then, $vu^i w \in \mathrm{Syn}(\mathcal{A})$. Conversely, if $L \cap \mathrm{Syn}(\mathcal{A}) \neq \emptyset$, then, as $\mathcal{A}$ has a trap state, it must be isomorphic to the automaton from Figure 7.4c. So, f cyclically permutes $[m]$. Hence, we have a valid reduction. □

Lemma 20.34 *Let $\mathcal{B} = (\{1,2,3\}, \{a,b\}, \mu, 1, \{3\})$ be a trim 3-state constraint automaton with two strongly connected components. Then $L(\mathcal{B})$ is letter-bounded or there exist $v, u, w \in \Sigma^*$ with $vu^*w \subseteq L(\mathcal{B})$ and $|u|_a, |u|_b > 0$.*

Proof. Let $p \in \{1,2,3\}$ be a state in the strongly connected component of size two and q be the single state in the other strongly connected component. By Theorem 17.75 we have $L(\mathcal{B}_{p,\{p\}}) = C_1^*$ and $L(\mathcal{B}_{q,\{q\}}) = C_2^*$ where C_1 is a prefix code (the case $C_1 = \emptyset$ is not possible since the strongly connected component has size two) and $C_2 = \emptyset$ or it is also a prefix code. Now, $L(\mathcal{B})$ is letter-bounded if and only if $C_1 \subseteq x^*$ and $C_2 \subseteq y^*$ for $x, y \in \Sigma$. Otherwise, C_1^* or C_2^* contains a word u with $|u|_a, |u|_b > 0$. Without loss of generality, suppose $u \in C_1^*$. We have $u^* \subseteq C_1^*$ and since $\mathcal{B}$ is trim, we have $\mu(1, v) = p$ and $\mu(p, w) = 3$ for some $v, w \in \{a,b\}^*$. This gives $vu^*w \subseteq L(\mathcal{B})$. □

Lemma 20.35 *Let $\mathcal{B} = (\{1,2,3\}, \{a,b\}, \mu, 1, \{3\})$ be a trim 3-state constraint automaton with two strongly connected components. Then the problem to decide whether a given SI-automaton over $\{a,b\}$ has a synchronizing word in $L(\mathcal{B})$ is in L.*

Proof. Let $\mathcal{A} = (Q, \{a,b\}, \delta)$ be an n-state input SI-automaton. As finitely many cases can be hard-coded into a decision algorithm, we can assume $n \geq 24$. Then, by Theorem 7.31, a necessary condition for $\mathcal{A}$ having a synchronizing word in L is that it equals, up to renaming of the letters, $\mathcal{A}_c$ or $\mathcal{A}_t$. By Theorem 13.8 we can test in deterministic log-space whether this is the case and compute the respective parameters. So, we can assume $\mathcal{A}$ equals $\mathcal{A}_c$ or $\mathcal{A}_t$, where a is a simple idempotent and b the permutational letter. Further, assume $\mathrm{Fact}(L(\mathcal{B})) \neq \{a,b\}^*$, as otherwise, by Theorem 18.4, when $\mathcal{A}$ equals $\mathcal{A}_c$ or $\mathcal{A}_t$ it is $\mathrm{Syn}(\mathcal{A}) \cap L(\mathcal{B}) \neq \emptyset$. Next, set $\Sigma_{i,j} = \{x \in \Sigma \mid \mu(i,x) = j\}$ for $i, j \in \{1,2,3\}$. Let $S \subseteq \{1,2,3\}$ be the strongly connected component with two states. Then

$$L(\mathcal{B}) = \begin{cases} (\Sigma_{1,1}^* \Sigma_{1,2} \Sigma_{2,2}^* \Sigma_{2,3} \cup \Sigma_{1,1}^* \Sigma_{1,3}) L(\mathcal{B}_{3,\{3\}}) & \text{if } S = \{2,3\}, \\ L(\mathcal{B}_{1,\{1\}})(\Sigma_{1,3} \Sigma_{3,3}^* \cup \Sigma_{1,2} \Sigma_{2,2}^* \Sigma_{2,3} \Sigma_{3,3}^*) & \text{if } S = \{1,2\}. \end{cases} \tag{20.3}$$

So, $L(\mathcal{B}) = ULV$ for letter-bounded $U, V \subseteq \Sigma^*$ with $\rho(u) \leq 4$ for $u \in U \cup V$ and L equals $L(\mathcal{B}_{3,\{3\}})$ for $S = \{2,3\}$ or $L(\mathcal{B}_{1,\{1\}})$ for $S = \{1,2\}$. As $\mathrm{Fact}(L(\mathcal{B})) \neq \{a,b\}^*$, we have $|\Sigma_{i,i}| \leq 1$ for all $i \in \{1,2,3\}$ and only two or three transitions are defined among the states in S. This yields that there are only 13 possible cases for L, which we handle next. Observe that, as $L \subseteq \mathrm{Fact}(L(\mathcal{B}))$, by Theorem 18.3, $\mathrm{Syn}(\mathcal{A}) \cap L \neq \emptyset$ implies $\mathrm{Syn}(\mathcal{A}) \cap L(\mathcal{B}) \neq \emptyset$.

$(ab)^*, (ba)^*, (a + ba)^*, (aa^*b)^*, (ab + aa)^*$: As a is a simple idempotent, we can replace $(a + ba)^*$, $(aa^*b)^*$ or $(ab + aa)^*$ by the bounded languages $(a + \varepsilon)(ba)^*$, $(ab)^*$ or $(ab)^*(a + \varepsilon)$, respectively. So, for each language, we can assume that we have $L(\mathcal{B}) = ULV \subseteq u_1^* \cdots u_k^*$ for some words $u_i \in \Sigma^{\leq 2}$ with $i \in \{1, \ldots, k\}$. Then, by Proposition 20.32, we find that the decision problem is in L, giving the claim.

$(bb^*a)^*, (ab^*a)^*, (b + aa)^*, (b + ab)^*, (bb + ab)^*, (bb + ba)^*$: By Theorem 20.29, as it lists these languages, $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$ is equivalent to $\mathrm{Syn}(\mathcal{A}) \cap L \neq \emptyset$. As $\mathrm{Syn}(\mathcal{A}) \cap L \neq \emptyset$ implies $\mathrm{Syn}(\mathcal{A}) \cap L(\mathcal{B}) \neq \emptyset$, we find that $\mathrm{Syn}(\mathcal{A}) \cap L(\mathcal{B}) \neq \emptyset$ is equivalent to $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$, which gives the claim.

$(ba^*b)^*$, $(a + bb)^*$: First, assume $\mathcal{A} = \mathcal{A}_c$ and n is even (resp. $\mathcal{A} = \mathcal{A}_t$ and n is odd). Let $u_1 \in U$, $u_2 \in L$ and $u_3 \in V$. By Theorem 20.24, there exists $S \subseteq Q$ with $|S| \geq n/2 \geq 12$ (resp. $|S| \geq (n-1)/2 \geq 12$) such that $\delta(S, u_2) = S$. As $\rho(u_1), \rho(u_3) \leq 4$, by Theorem 20.30, we get

$$|\delta(Q, u_1)| \geq n - 5 \quad \text{and} \quad |\delta(Q, u_1u_2u_3)| \geq |\delta(Q, u_1u_2)| - 5.$$

The former equation yields $|\delta(Q, u_1) \cap S| \geq 7$. As u_2 permutes the states in S we have $|\delta(Q, u_1u_2) \cap S| = |\delta(Q, u_1) \cap S|$. Then the latter equation yields $|\delta(Q, u_1u_2u_3)| \geq |\delta(Q, u_1u_2) \cap S| - 5 \geq 2$. So, $\mathrm{Syn}(\mathcal{A}) \cap L(\mathcal{B}) = \emptyset$. Otherwise, if $\mathcal{A} = \mathcal{A}_c$ and n is odd (resp. $\mathcal{A} = \mathcal{A}_t$ and n even), then, by Theorem 20.28, we have $\mathrm{Syn}(\mathcal{A}) \cap L \neq \emptyset$. As written above, this implies $\mathrm{Syn}(\mathcal{A}) \cap L(\mathcal{B}) \neq \emptyset$. Now, whether n is odd or even can be decided in deterministic log-space. □

Proposition 20.36 *Let $\mathcal{B} = (P, \{a, b\}, \mu, p_0, F)$ be a constraint automaton with $|P| \leq 3$ that is not strongly connected. For an SI-automaton $\mathcal{A} = (Q, \{a, b\}, \delta)$ the problem to decide whether it has a synchronizing word is decidable in constant time if $L(\mathcal{B})$ is letter-bounded and it is* L*-complete otherwise.*

Proof. For $|P| \leq 2$ or if $\mathcal{B}$ has precisely three strongly connected components and three states, then it is easy to see that either $L(\mathcal{B})$ is letter-bounded or $\mathrm{Fact}(L(\mathcal{B})) = \{a, b\}^*$ (in fact, in the latter case, we have $L(\mathcal{B}_{p,\{p\}}) = \{a, b\}^*$ for some $p \in P$ that can assumed to be final, which gives $\mathrm{Fact}(L(\mathcal{B})) = \{a, b\}^*$, or for all $p \in P$ we have $L(\mathcal{B}_{p,\{p\}}) \subseteq x^*$ for some $x \in \Sigma$, and so the accepted language is letter-bounded by Theorem 17.72, Item 2). In the latter case, by Theorem 18.4 the problem is the same as deciding whether the automaton is synchronizing, and so L-complete by Theorem 13.10. In the former case, the problem is solvable in constant time by Theorem 20.31, Item 2.

Lastly, if $|P| = 3$ and $\mathcal{B}$ has two strongly connected components, then we can assume $F = \{3\}$ by Theorems 18.5 and 18.99 and since $\mathcal{B}$ is trim. Then Theorems 20.31 and 20.33 to 20.35 give the claim. □

Theorem 20.36 shows that for several constraint languages that yield PSPACE-complete problems in general, for SI-automata as input the complexity drops to containment in L-completeness.

Theorem 20.37 *Let $\mathcal{B} = (P, \{a, b\}, \mu, p_0, F)$ be a constraint automaton with $|P| \leq 3$. For an n-state SI-automaton $\mathcal{A} = (Q, \{a, b\}, \delta)$ we can decide in* NL *whether it has a synchronizing word in $L(\mathcal{B})$.*

Proof. If $\mathcal{B}$ is strongly connected, then the claim follows from Theorem 19.19. Otherwise, it follows from Theorem 20.36. □

20.7 Discussion

Literature.

Polynomial-time solvability of the problem of constrained synchronization for commutative input automata was stated in [299]. Here, we improved that result to containment in NL. Partially ordered input automata were investigated in [300] and a complexity classification for two-state constraint PDFAs over a ternary alphabet was given, which we extended in the present work to an arbitrary alphabet and refined it by differentiating the polynomial-time solvable cases into AC^0 and L-completeness. Monotonic, solvable and SI-automata were firstly investigated in [305]. However, the paper [305] contained the following two mistakes that were fixed in the present work:

1. It claimed the bound $|P| \cdot n$ for n-state monotonic automata, but the argument was wrong and here we were only able to obtain the bound $|P| \cdot \binom{n}{2}$ (Proposition 20.5). However, we do not know whether this bound is tight, and the linear bound might be correct after all, as it is known that a shortest synchronizing word (without a constraint) has length at most $n - 1$ for monotonic n-state automata [23].
2. The proof of the bound stated for solvable automata involving various parameters (one related to the cycle lengths) contained a mistake, were a synchronizing word was commutatively reordered without considering whether this is permissible w.r.t. the constraint language.[2] In the present work in Theorem 20.21 we obtained a better bound for the restricted class of solvable ρ-automata, but have no general bound. The claim in [300] that the constrained synchronization problem is in NP for solvable input automata used this erroneous result, and so it is no longer certain whether this is true. Here, we have shown containment in NP for the proper subclass of solvable ρ-automata.

Open Problems.

We present various open problems suggested by the obtained results.

1. For monotonic input automata it was shown (Theorem 20.6) that the constrained synchronization problem is solvable in NL. However, it is an *open problem* whether the problem is NL-hard. Also, as discussed above, can the quadratic bound from Proposition 20.5 be improved to a linear bound?
2. We have shown containment in NP for solvable ρ-automata. We do not know whether the problem remains in NP for arbitrary solvable automata as input. Furthermore, we do not know a bound on shortest synchronizing words in a given constraint language for general solvable automata (compare also the discussion above with respect to the bound claimed in [305]).
3. For commutative input automata, the problem was shown to be in NL (Theorem 20.19). We do not know whether there exist constraint languages giving NL-hard constrained problems.

[2] Quite a blatant mistake in hindsight, but the overall argument was quite involved.

4. A partially ordered automaton with at least three states is not sync-maximal by Theorem 10.7, since a synchronizing partially ordered automaton has a trap state by Theorem 7.112. Polynomial time constructability of an automaton for the set of synchronizing words implies that the constrained synchronization problem is in P. As we have found constraint languages giving NP-hard constrained synchronization problems for partially ordered input automata, it follows that automata for the sets of synchronizing words of partially ordered automata are not constructible in polynomial time. This suggests to broaden the investigation of the set of synchronizing words further by not only investigating the property of sync-maximality, but under what circumstances a minimal automaton for the set of synchronizing words has size superpolynomial in the size of the original automaton (see also the results from Section 15.2 and the discussion in Section 15.3).
5. Lastly, the study of SI-automata as input is still in its beginnings. Here, we only considered certain monoid languages, certain bounded languages, and languages accepted by binary constraint automata with at most three states as constraints. We always obtained that the corresponding constrained problem is in NL. We do not know if this is the case for any constraint language, or whether there exist constraint languages giving PSPACE-hard (or NP-hard) problems for SI-automata as input. As there exist sync-maximal SI-automata (for example, the Černý family, see Figure 1.7 and Theorem 10.4), the set of synchronizing words might be "complex" enough to admit hard instances of the constrained synchronization problem. However, the NP-hardness reductions for the problem to find shortest synchronizing words in SI-automata from [436, 216] all used an unbounded alphabet, but a fixed constraint languages essentially fixes the alphabet as well. Lastly, we do not know whether there exist constraint languages giving an NL-hard constrained synchronization problem for SI-automata as input.

References

[1] Roy L. Adler, L. Wayne Goodwyn, and Benjamin Weiss. "Equivalence of Topological Markov Shifts". In: *Israel Journal of Mathematics* 27.1 (1977), pp. 49–63. DOI: 10.1007/BF02761605 (cit. on p. 10).

[2] Manindra Agrawal, Eric Allender, and Steven Rudich. "Reductions in Circuit Complexity: An Isomorphism Theorem and a Gap Theorem". In: *Journal of Computer and System Sciences* 57.2 (1998), pp. 127–143. DOI: 10.1006/JCSS.1998.1583 (cit. on p. 86).

[3] Miklós Ajtai. "First-Order Definability on Finite Structures". In: *Annals of Pure and Applied Logic* 45.3 (1989), pp. 211–225. DOI: 10.1016/0168-0072(89)90036-5 (cit. on p. 89).

[4] Jorge L. Ramírez Alfonsín. *The Diophantine Frobenius Problem*. Oxford University Press, Dec. 2005. ISBN: 9780198568209. DOI: 10.1093/acprof:oso/9780198568209.001.0001 (cit. on p. 446).

[5] Eric Allender. "P-uniform Circuit Complexity". In: *Journal of the ACM* 36.4 (1989), pp. 912–928. DOI: 10.1145/76359.76370 (cit. on p. 88).

[6] Eric Allender. "Some Pointed Questions Concerning Asymptotic Lower Bounds, and News from the Isomorphism Front". In: *Current Trends in Theoretical Computer Science, Entering the 21th Century*. Ed. by Gheorghe Paun and Grzegorz Rozenberg and Arto Salomaa. World Scientific, 2001, pp. 25–41 (cit. on p. 85).

[7] Eric Allender. "The Division Breakthroughs". In: *Bulletin EATCS* 74 (2001). available online: https://people.cs.rutgers.edu/~allender/papers/division.eatcs.pdf, pp. 61–77 (cit. on pp. 74, 88).

[8] Eric Allender and Vivek Gore. "Rudimentary Reductions Revisited". In: *Information Processing Letters* 40.2 (1991), pp. 89–95. DOI: 10.1016/0020-0190(91)90015-A (cit. on p. 85).

[9] Eric Allender, Michael C. Loui, and Kenneth W. Regan. "Reducibility and Completeness". In: *Algorithms and Theory of Computation Handbook*. Ed. by Mikhail J. Atallah. Chapman & Hall/CRC Applied Algorithms and Data Structures series. available online: https://people.cs.rutgers.edu/~allender/papers/ALRch34.pdf. CRC Press, 1999. DOI: 10.1201/9781420049503-c29 (cit. on p. 86).

[10] Eric Allender, Michael C. Loui, and Kenneth W. Regan. "Reducibility and Completeness". In: *Algorithms and Theory of Computation Handbook: General Concepts and Techniques*. 2nd ed. Chapman & Hall/CRC, 2010, p. 23. ISBN: 9781584888222. eprint: https://dl.acm.org/doi/10.5555/1882757.1882780 (cit. on p. 81).

[11] Jorge Almeida. *Finite Semigroups and Universal Algebra*. Singapore: World Scientific, 1995. ISBN: 978-981-02-1895-9. DOI: 10.1142/2481 (cit. on p. 82).

[12] Jorge Almeida and Angela Cardoso. "A Sequence of Weakly Monotonic Automata with Increasing Level". In: *International Journal of Algebra* 7.2 (2013), pp. 91–100. DOI: 10.12988/ija.2013.13010 (cit. on p. 223).

[13] Jorge Almeida, Zoltán Ésik, and Jean-Éric Pin. "Commutative Positive Varieties of Languages". In: *Acta Cybernetica* 23.1 (2017), pp. 91–111. DOI: 10.14232/actacyb.23.1.2017.7 (cit. on p. 223).

[14] Jorge Almeida and Emanuele Rodaro. "Semisimple Synchronizing Automata and the Wedderburn-Artin Theory". In: *International Journal of Foundations of Computer Science* 27.2 (2016), pp. 127–146. DOI: 10.1142/S0129054116400037 (cit. on pp. 14, 302).

[15] Jorge Almeida and Benjamin Steinberg. "Matrix Mortality and the Cerný-Pin Conjecture". In: *Developments in Language Theory, 13th International Conference, DLT 2009, Stuttgart, Germany, June 30 - July 3, 2009. Proceedings*. Ed. by Volker Diekert and Dirk Nowotka. Vol. 5583. Lecture Notes in Computer Science. Springer, 2009, pp. 67–80. DOI: 10.1007/978-3-642-02737-6_5 (cit. on pp. 14, 224).

[16] Jorge Almeida et al. "Representation Theory of Finite Semigroups, Semigroup Radicals and Formal Language Theory". In: *Transactions of the American Mathematical Society* 361.3 (2009), pp. 1429–1461. DOI: 10.1090/S0002-9947-08-04712-0 (cit. on p. 14).

[17] Lucas V.R. Alves and Patrícia N. Pena. “Synchronism Recovery of Discrete Event Systems”. In: *IFAC-PapersOnLine* 53.2 (2020). 21th IFAC World Congress, pp. 10474–10479. ISSN: 2405-8963. DOI: 10.1016/j.ifacol.2020.12.2791 (cit. on p. 416).

[18] D. S. Ananichev. “The annulation threshold for partially monotonic automata”. In: *Russian Mathematics (Iz. VUZ)* 54.1 (2010). Translated from Izv. Vyssh. Uchebn. Zaved. Mat., no. 1, pp. 3–13, 2010, pp. 1–9. DOI: 10.3103/S1066369X10010019 (cit. on p. 13).

[19] Dimitry S. Ananichev. “The Mortality Threshold for Partially Monotonic Automata”. In: *Developments in Language Theory, 9th International Conference, DLT 2005, Palermo, Italy, July 4-8, 2005, Proceedings*. Ed. by Clelia de Felice and Antonio Restivo. Vol. 3572. Lecture Notes in Computer Science. Springer, 2005, pp. 112–121. DOI: 10.1007/11505877_10 (cit. on p. 223).

[20] Dimitry S. Ananichev, Vladimir V. Gusev, and Mikhail V. Volkov. “Slowly Synchronizing Automata and Digraphs”. In: *Mathematical Foundations of Computer Science 2010, 35th International Symposium, MFCS 2010, Brno, Czech Republic, August 23-27, 2010. Proceedings*. Ed. by Petr Hlinený and Antonín Kucera. Vol. 6281. Lecture Notes in Computer Science. Springer, 2010, pp. 55–65. DOI: 10.1007/978-3-642-15155-2_7 (cit. on pp. 397, 406).

[21] Dimitry S. Ananichev, Vladimir V. Gusev, and Mikhail V. Volkov. “Primitive Digraphs with Large Exponents and Slowly Synchronizing Automata”. In: *Journal of Mathematical Sciences* 192.3 (2013), pp. 263–278. DOI: 10.1007/s10958-013-1392-8 (cit. on pp. 12, 182, 334, 397, 402–404, 406).

[22] Dimitry S. Ananichev and Mikhail V. Volkov. “Some Results on Černý Type Problems for Transformations Semigroups”. In: *Semigroups and Languages, Proceedings of the Workshop, Lisboa, Portugal, 27 – 29 November 2002*. Ed. by Isabel M Araújo et al. World Scientific, 2004, pp. 23–42. DOI: 10.1142/9789812702616_0002 (cit. on pp. 8, 10, 13, 14, 167, 223).

[23] Dimitry S. Ananichev and Mikhail V. Volkov. “Synchronizing Monotonic Automata”. In: *Theoretical Computer Science* 327.3 (2004), pp. 225–239. DOI: 10.1016/j.tcs.2004.03.068 (cit. on pp. 10, 13, 192, 223, 250, 547).

[24] Dimitry S. Ananichev and Mikhail V. Volkov. “Synchronizing Generalized Monotonic Automata”. In: *Theoretical Computer Science* 330.1 (2005), pp. 3–13. DOI: 10.1016/j.tcs.2004.09.006 (cit. on pp. 13, 223).

[25] Dimitry S. Ananichev, Mikhail V. Volkov, and Yu. I. Zaks. “Synchronizing Automata with a Letter of Deficiency 2”. In: *Theoretical Computer Science* 376.1-2 (2007), pp. 30–41. DOI: 10.1016/j.tcs.2007.01.010 (cit. on pp. 403, 404, 406).

[26] Dana Angluin and Leslie G. Valiant. “Fast Probabilistic Algorithms for Hamiltonian Circuits and Matchings”. In: *Journal of Computer and System Sciences* 18.2 (1979), pp. 155–193. DOI: 10.1016/0022-0000(79)90045-X (cit. on p. 85).

[27] P. N. Anh, G. F. Birkenmeier, and L. van Wyk. “Peirce Decompositions, Idempotents and Rings”. In: *Journal of Algebra* 564 (2021), pp. 247–275. DOI: 10.1016/j.jalgebra.2020.08.003 (cit. on p. 84).

[28] João Araújo, Jorge Andre, and Peter J. Cameron. "The Classification of Partition Homogeneous Groups with Applications to Semigroup Theory". In: *Journal of Algebra* 452 (2016), pp. 288–310. DOI: 10.1016/j.jalgebra.2015.12.025 (cit. on p. 395).

[29] João Araújo, Wolfram Bentz, and Peter J. Cameron. "Groups Synchronizing a Transformation of Non-Uniform Kernel". In: *Theoretical Computer Science* 498 (2013), pp. 1–9. DOI: 10.1016/j.tcs.2013.06.016 (cit. on pp. 83, 125, 390).

[30] João Araújo, Wolfram Bentz, and Peter J. Cameron. "Orbits of Primitive k-Homogenous Groups on (n-k)-Partitions with Applications to Semigroups". In: *Transactions of the American Mathematical Society* 371 (2019), pp. 105–136. DOI: 10.1090/tran/7274 (cit. on p. 395).

[31] João Araújo, Wolfram Bentz, and Peter J. Cameron. "Primitive Permutation Groups and Strongly Factorizable Transformation Semigroups". In: *Journal of Algebra* 565 (2021), pp. 513–530. ISSN: 0021-8693. DOI: 10.1016/j.jalgebra.2020.05.023 (cit. on pp. 358, 395).

[32] João Araújo, Wolfram Bentz, and Peter J. Cameron. "The Existential Transversal Property: A Generalization of Homogeneity and its Impact on Semigroups". In: *Transactions of the American Mathematical Society* 374 (2021), pp. 1155–1195. DOI: 10.1090/tran/8285 (cit. on p. 395).

[33] João Araújo and Peter J. Cameron. "Permutation Groups and Transformation Semigroups: Results and Problems". In: *Proceedings of Groups St Andrews 2013*. Ed. by C. M. Campbell et al. Vol. 422. London Math. Soc. Lecture Notes. Cambridge University Press, 2013, pp. 128–141. DOI: 10.1017/CBO9781316227343.006 (cit. on p. 395).

[34] João Araújo and Peter J. Cameron. "Primitive Groups Synchronize Non-Uniform Maps of Extreme Ranks". In: *Journal of Combinatorial Theory, Series B* 106 (2014), pp. 98–114. ISSN: 0095-8956. DOI: 10.1016/j.jctb.2014.01.006 (cit. on pp. 36, 110, 126, 357–359, 381, 395).

[35] João Araújo and Peter J. Cameron. "Two Generalizations of Homogeneity in Groups with Applications to Regular Semigroups". In: *Transactions of the American Mathematical Society* 368 (2016), pp. 1159–1188. DOI: 10.1090/tran/6368 (cit. on p. 395).

[36] João Araújo, Peter J. Cameron, and Benjamin Steinberg. "Between Primitive and 2-Transitive: Synchronization and its Friends". In: *EMS Surveys in Mathematical Sciences* 4.2 (2017), pp. 101–184. DOI: 10.4171/EMSS/4-2-1 (cit. on pp. 3, 5, 14, 115, 123, 127, 357, 358, 366, 390, 395).

[37] João Araújo and Janusz Konieczny. "Centralizers in the Full Transformation Semigroup". In: *Semigroup Forum* 86.1 (2013), pp. 1–31. DOI: 10.1007/s00233-012-9424-0 (cit. on p. 171).

[38] João Araújo et al. "Primitive Groups, Graph Endomorphisms and Synchronization". In: *Proceedings of the London Mathematical Society* 113.6 (2016), pp. 829–867. DOI: 10.1112/plms/pdw040 (cit. on pp. 390, 395).

[39] Michael A. Arbib. *Algebraic Theory of Machines, Languages and Semigroups*. Academic Press, 1968 (cit. on p. 170).

[40] Fredrick Arnold and Benjamin Steinberg. "Synchronizing Groups and Automata". In: *Theoretical Computer Science* 359.1-3 (2006), pp. 101–110. DOI: 10.1016/j.tcs.2006.02.003 (cit. on pp. 14, 364, 365, 390, 394).

[41] Sanjeev Arora and Boaz Barak. *Computational Complexity - A Modern Approach*. Cambridge University Press, 2009. ISBN: 978-0-521-42426-4. DOI: 10.1017/CBO9780511804090 (cit. on pp. 65, 85, 86, 249).

[42] Emmanuel Arrighi et al. "On the Complexity of Intersection Non-Emptiness for Star-Free Language Classes". In: *41st IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2021, December 15-17, 2021, Virtual Conference*. Ed. by Mikolaj Bojanczyk and Chandra Chekuri. Vol. 213. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, 34:1–34:15. DOI: 10.4230/LIPIcs.FSTTCS.2021.34 (cit. on pp. 231, 253, 256).

[43] Emmanuel Arrighi et al. "Synchronization and Diversity of Solutions". In: *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*. Ed. by Brian Williams, Yiling Chen, and Jennifer Neville. AAAI Press, 2023, pp. 11516–11524. DOI: 10.1609/AAAI.V37I10.26361 (cit. on pp. 252, 418).

[44] Emil Artin. "Zur Theorie der Hyperkomplexen Zahlen". In: *Abhandlungen Math. Seminar Univ. Hamburg* 5 (1928). Reprinted in *Collected Papers of Emil Artin* (1965), pp. 251–260. DOI: 10.1007/BF02952526 (cit. on p. 84).

[45] Vikraman Arvind and T. C. Vijayaraghavan. "Abelian Permutation Group Problems and Logspace Counting Classes". In: *19th Annual IEEE Conference on Computational Complexity (CCC 2004), 21-24 June 2004, Amherst, MA, USA*. IEEE Computer Society, 2004, pp. 204–214. DOI: 10.1109/CCC.2004.1313844 (cit. on pp. 255, 256).

[46] Michael Aschbacher and Leonard Scott. "Maximal Subgroups of Finite Groups". In: *Journal of Algebra* 92.1 (1985), pp. 44–80. ISSN: 0021-8693. DOI: 10.1016/0021-8693(85)90145-0 (cit. on p. 396).

[47] William R. Ashby. *An Introduction to Cybernetics*. Chapman & Hall, London, 1956. DOI: 10.5962/bhl.title.5851 (cit. on p. 7).

[48] László Babai. "Automorphism Groups, Isomorphism, Reconstruction". In: *Handbook of Combinatorics (Vol. 2)*. Cambridge, MA, USA: MIT Press, 1996, pp. 1447–1540. ISBN: 0262071711 (cit. on pp. 127, 130).

[49] László Babai, Eugene M. Luks, and Ákos Seress. "Permutation Groups in NC". In: *Proceedings of the 19th Annual ACM Symposium on Theory of Computing, 1987, New York, New York, USA*. Ed. by Alfred V. Aho. ACM, 1987, pp. 409–420. DOI: 10.1145/28395.28439 (cit. on p. 256).

[50] László Babai et al. "Multiplicative Equations over Commuting Matrices". In: *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, 28-30 January 1996, Atlanta, Georgia, USA*. Ed. by Éva Tardos. ACM/SIAM, 1996, pp. 498–507 (cit. on p. 247).

[51] István Babcsányi. "Automata with Finite Congruence Lattices". In: *Acta Cybernetica* 18.1 (2007), pp. 155–165. eprint: https://cyber.bibl.u-

szeged.hu/index.php/actcybern/article/view/3710 (cit. on pp. 153, 169).

[52] István Babcsányi. "The Černý Conjecture for Automata with Blocking States". In: *Automata, Formal Languages, and Related Topics - Dedicated to Ferenc Gécseg on the occasion of his 70th birthday*. Ed. by Zoltán Ésik and Zoltán Fülöp. available online: https://math.bme.hu/~babcs/cikkek/cerny.pdf. Institute of Informatics, University of Szeged, Hungary, 2009, pp. 17–25 (cit. on pp. 13, 14, 178, 181).

[53] Paul Bachmann. *Die Analytische Zahlentheorie*. Zahlentheorie, pt. 2. Leipzig: Teubner, 1894 (cit. on p. 64).

[54] Balázs Imreh and Magnus Steinby. "Some Remarks on Directable Automata". In: *Acta Cybernetica* 12.1 (1995), pp. 23–35. eprint: https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/3441 (cit. on p. 14).

[55] Zoltán Ésik and Balázs Imreh. "Subdirectly Irreducible Commutative Automata". In: *Acta Cybernetica* 5.3 (1981), pp. 251–260. eprint: https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/3217 (cit. on p. 222).

[56] José L. Balcázar, Josep Díaz, and Joaquim Gabarró. *Structural Complexity II*. Vol. 22. EATCS Monographs on Theoretical Computer Science. Springer, 1990. ISBN: 978-3-642-75359-6. DOI: 10.1007/978-3-642-75357-2 (cit. on pp. 72, 85, 87–89).

[57] José L. Balcázar, Josep Díaz, and Joaquim Gabarró. *Structural Complexity I, Second Edition*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 1995. ISBN: 978-3-642-79237-3. DOI: 10.1007/978-3-642-79235-9 (cit. on pp. 72, 85, 86, 89).

[58] Bruce H. Barnes. "Groups of Automorphisms and Sets of Equivalence Classes of Input for Automata". In: *Journal of the ACM* 12.4 (1965), pp. 561–565. DOI: 10.1145/321296.321306 (cit. on p. 168).

[59] Bruce H. Barnes. "On the Group of Automorphisms of Strongly Connected Automata". In: *Mathematical Systems Theory* 4.4 (1970), pp. 289–294. DOI: 10.1007/BF01695770 (cit. on p. 168).

[60] David A. Mix Barrington. "Bounded-Width Polynomial-Size Branching Programs Recognize Exactly Those Languages in NC^1". In: *Journal of Computer and System Sciences* 38.1 (1989), pp. 150–164. DOI: 10.1016/0022-0000(89)90037-8 (cit. on p. 89).

[61] David A. Mix Barrington. "Extensions of an Idea of McNaughton". In: *Mathematical Systems Theory* 23.3 (1990), pp. 147–164. DOI: 10.1007/BF02090772 (cit. on pp. 88, 89).

[62] David A. Mix Barrington, Neil Immerman, and Howard Straubing. "On Uniformity within NC^1". In: *Journal of Computer and System Sciences* 41.3 (1990), pp. 274–306. DOI: 10.1016/0022-0000(90)90022-D (cit. on p. 88).

[63] David A. Mix Barrington and Denis Thérien. "Finite Monoids and the Fine Structure of NC^1". In: *Journal of the ACM* 35.4 (1988), pp. 941–952. DOI: 10.1145/48014.63138 (cit. on p. 89).

[64] David A. Mix Barrington et al. “Regular Languages in NC^1”. In: *Journal of Computer and System Sciences* 44.3 (1992), pp. 478–499. DOI: 10.1016/0022-0000(92)90014-A (cit. on p. 89).

[65] Julien Baste et al. “Diversity of Solutions: An Exploration Through the Lens of Fixed-Parameter Tractability Theory”. In: *Artificial Intelligence* 303 (2022), p. 103644. DOI: 10.1016/j.artint.2021.103644 (cit. on p. 252).

[66] Zamir Bavel. “Structure and Transition-Preserving Functions of Finite Automata”. In: *Journal of the ACM* 15.1 (1968), pp. 135–158. DOI: 10.1145/321439.321448 (cit. on p. 168).

[67] Zamir Bavel. “On the Number of Automorphisms of a Singly Generated Automaton”. In: *Communications of the ACM* 13.9 (1970), pp. 574–575. DOI: 10.1145/362736.362767 (cit. on p. 168).

[68] Rudolf Bayer. “Automorphism Groups and Quotients of Strongly Connected Automata and Monadic Algebras”. In: *7th Annual Symposium on Switching and Automata Theory, Berkeley, California, USA, October 23-25, 1966*. IEEE Computer Society, 1966, pp. 282–297. DOI: 10.1109/SWAT.1966.5 (cit. on p. 169).

[69] Rudolf Bayer. “On Endomorphisms and Congruences of Automata”. In: *8th Annual Symposium on Switching and Automata Theory, Austin, Texas, USA, October 18-20, 1967*. IEEE Computer Society, 1967, pp. 314–321. DOI: 10.1109/FOCS.1967.22 (cit. on p. 169).

[70] Marie-Pierre Béal, Mikhail V. Berlinkov, and Dominique Perrin. “A Quadratic Upper Bound on the Size of a Synchronizing Word in One-Cluster Automata”. In: *International Journal of Foundations of Computer Science* 22.2 (2011), pp. 277–288. DOI: 10.1142/S0129054111008039 (cit. on pp. 14, 452).

[71] Marie-Pierre Béal and Dominique Perrin. “Synchronised Automata”. In: *Combinatorics, Words and Symbolic Dynamics*. Ed. by Valérie Berthé and Michel Rigo. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2016, pp. 213–240. DOI: 10.1017/CBO9781139924733.008 (cit. on p. 3).

[72] Paul Beame, Stephen A. Cook, and H. James Hoover. “Log Depth Circuits for Division and Related Problems”. In: *SIAM Journal on Computing* 15.4 (1986), pp. 994–1003. DOI: 10.1137/0215070 (cit. on p. 88).

[73] Martin Beaudry. “Membership Testing in Commutative Transformation Semigroups”. In: *Information and Computation* 79.1 (1988), pp. 84–93. DOI: 10.1016/0890-5401(88)90018-1 (cit. on pp. 223, 243, 244, 256).

[74] Martin Beaudry. “Membership Testing in Transformation Monoids”. PhD thesis. McGill University, Montreal, Quebec, 1988 (cit. on pp. 231, 255).

[75] Martin Beaudry. “Characterization of Idempotent Transformation Monoids”. In: *Information Processing Letters* 31.3 (1989), pp. 163–166. DOI: 10.1016/0020-0190(89)90226-3 (cit. on pp. 110, 130).

[76] Martin Beaudry. “Membership Testing in Threshold One Transformation Monoids”. In: *Information and Computation* 113.1 (1994), pp. 1–25. DOI: 10.1006/inco.1994.1062 (cit. on p. 256).

[77] Martin Beaudry, Pierre McKenzie, and Denis Thérien. "The Membership Problem in Aperiodic Transformation Monoids". In: *Journal of the ACM* 39.3 (1992), pp. 599–616. DOI: 10.1145/146637.146661 (cit. on p. 255).

[78] Jordan R. Bell and Brett Stevens. "A Survey of Known Results and Research Areas for n-Queens". In: *Discrete Mathematics* 309.1 (2009), pp. 1–31. DOI: 10.1016/j.disc.2007.12.043 (cit. on p. 251).

[79] Mikhail V. Berlinkov. "Approximating the Minimum Length of Synchronizing Words Is Hard". In: *Computer Science - Theory and Applications, 5th International Computer Science Symposium in Russia, CSR 2010, Kazan, Russia, June 16-20, 2010. Proceedings*. Ed. by Farid M. Ablayev and Ernst W. Mayr. Vol. 6072. Lecture Notes in Computer Science. Springer, 2010, pp. 37–47 (cit. on p. 250).

[80] Mikhail V. Berlinkov. "On a Conjecture by Carpi and D'Alessandro". In: *International Journal of Foundations of Computer Science* 22.7 (2011), pp. 1565–1576. DOI: 10.1142/S012905411100888X (cit. on pp. 307, 324).

[81] Mikhail V. Berlinkov. "Synchronizing Quasi-Eulerian and Quasi-One-Cluster Automata". In: *International Journal of Foundations of Computer Science* 24.6 (2013), pp. 729–746. DOI: 10.1142/S0129054113400157 (cit. on p. 14).

[82] Mikhail V. Berlinkov. "Approximating the Minimum Length of Synchronizing Words Is Hard". In: *Theory of Computing Systems* 54.2 (2014), pp. 211–223. DOI: 10.1007/s00224-013-9511-y (cit. on pp. 250, 525).

[83] Mikhail V. Berlinkov. "On Two Algorithmic Problems about Synchronizing Automata - (Short Paper)". In: *Developments in Language Theory - 18th International Conference, DLT 2014, Ekaterinburg, Russia, August 26-29, 2014. Proceedings*. Ed. by Arseny M. Shur and Mikhail V. Volkov. Vol. 8633. Lecture Notes in Computer Science. Springer, 2014, pp. 61–67. DOI: 10.1007/978-3-319-09698-8_6 (cit. on p. 250).

[84] Mikhail V. Berlinkov, Robert Ferens, and Marek Szykuła. "Preimage Problems for Deterministic Finite Automata". In: *Journal of Computer and System Sciences* 115 (2021), pp. 214–234. DOI: 10.1016/j.jcss.2020.08.002 (cit. on pp. 231, 232, 252).

[85] Mikhail V. Berlinkov and Marek Szykuła. "Algebraic Synchronization Criterion and Computing Reset Words". In: *Information Sciences* 369 (2016), pp. 718–730. DOI: 10.1016/j.ins.2016.07.049 (cit. on p. 14).

[86] Leonard Berman and Juris Hartmanis. "On Isomorphisms and Density of NP and Other Complete Sets". In: *SIAM Journal on Computing* 6.2 (1977), pp. 305–322. DOI: 10.1137/0206023 (cit. on pp. 86, 453).

[87] Jean Berstel, Dominique Perrin, and Christophe Reutenauer. *Codes and Automata*. Vol. 129. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2009. ISBN: 978-0-521-88831-8. DOI: 10.1017/CBO9781139195768 (cit. on pp. 3, 5, 427, 428, 445, 446, 452).

[88] Jean Berstel and Christophe Reutenauer. *Rational Series and Their Languages*. Vol. 12. EATCS Monographs on Theoretical Computer Science. Springer, 1988. ISBN: 978-3-540-15399-6. DOI: 10.1007/978-3-642-82811-4 (cit. on p. 84).

[89] Alberto Bertoni, Massimiliano Goldwurm, and Nicoletta Sabadini. "Computing the Counting Function of Context-Free Languages". In: *STACS 87, 4th*

Annual Symposium on Theoretical Aspects of Computer Science, Passau, Germany, February 19-21, 1987, Proceedings. Ed. by Franz-Josef Brandenburg, Guy Vidal-Naquet, and Martin Wirsing. Vol. 247. Lecture Notes in Computer Science. Springer, 1987, pp. 169–179. DOI: 10.1007/BFb0039604 (cit. on p. 453).

[90] Norman Biggs. *Algebraic Graph Theory*. 2nd ed. Cambridge Mathematical Library. Cambridge University Press, 1993. ISBN: 978-0-521-45897-9. DOI: 10.1017/CBO9780511608704 (cit. on p. 130).

[91] Bryan J. Birch et al. "On the Orbit-Sizes of Permutation Groups Containing Elements Separating Finite Subsets". In: *Bulletin of the Australian Mathematical Society* 14.1 (2009), pp. 7–10. DOI: 10.1017/S0004972700024813 (cit. on p. 115).

[92] Marek Biskup. "Error Resilience in Compressed Data – Selected Topics". available online: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=7d6a911d8ccc01f64e466f240efd6eebad5dfd7b. PhD thesis. University of Warsaw, Institute of Informatics, 2008 (cit. on p. 452).

[93] Simon R. Blackburn. "Non-Overlapping Codes". In: *IEEE Transactions on Information Theory* 61.9 (2015), pp. 4890–4894. DOI: 10.1109/TIT.2015.2456634. arXiv: 1303.1026 [cs.IT] (cit. on p. 426).

[94] Vincent D. Blondel and Natacha Portier. "The Presence of a Zero in an Integer Linear Recurrent Sequence is NP-Hard to Decide". In: *Linear Algebra and its Applications* 351-352 (2002), pp. 91–98. DOI: 10.1016/S0024-3795(01)00466-9 (cit. on pp. 413, 414).

[95] Michael Blondin. "Complexité Raffinée du Probléme d'Intersection d'Automates". MA thesis. Université de Montréal, 2012. eprint: https://papyrus.bib.umontreal.ca/xmlui/handle/1866/8440 (cit. on p. 256).

[96] Michael Blondin, Andreas Krebs, and Pierre McKenzie. "The Complexity of Intersecting Finite Automata Having Few Final States". In: *Computational Complexity* 25.4 (2016), pp. 775–814. DOI: 10.1007/s00037-014-0089-9 (cit. on pp. 226, 252, 255, 256).

[97] Stojan Bogdanović et al. "Directable Automata and Their Generalizations: A Survey". In: *Novi Sad Journal of Mathematics* 29.2 (1999). available online: http://eudml.org/doc/226950 or https://sites.dmi.uns.ac.rs//nsjom/issue.html?year=1999 (cit. on pp. 3, 167).

[98] Udi Boker, Karoliina Lehtinen, and Salomon Sickert. "On the Translation of Automata to Linear Temporal Logic". In: *Foundations of Software Science and Computation Structures - 25th International Conference, FOSSACS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings*. Ed. by Patricia Bouyer and Lutz Schröder. Vol. 13242. Lecture Notes in Computer Science. Springer, 2022, pp. 140–160. DOI: 10.1007/978-3-030-99253-8_8 (cit. on p. 171).

[99] Eugenija A. Bondar and Mikhail V. Volkov. "Completely Reachable Automata". In: *Descriptional Complexity of Formal Systems - 18th IFIP WG 1.2 International Conference, DCFS 2016, Bucharest, Romania, July 5-8, 2016. Proceedings*. Ed. by Cezar Câmpeanu, Florin Manea, and Jeffrey O. Shallit. Vol. 9777.

Lecture Notes in Computer Science. Springer, 2016, pp. 1–17. ISBN: 978-3-319-41113-2. DOI: 10.1007/978-3-319-41114-9_1 (cit. on pp. 226, 231, 239, 252, 265, 266, 291, 300, 304, 323, 352).

[100] Eugenija A. Bondar and Mikhail V. Volkov. "A Characterization of Completely Reachable Automata". In: *Developments in Language Theory - 22nd International Conference, DLT 2018, Tokyo, Japan, September 10-14, 2018, Proceedings*. Ed. by Mizuho Hoshi and Shinnosuke Seki. Vol. 11088. Lecture Notes in Computer Science. Springer, 2018, pp. 145–155. ISBN: 978-3-319-98653-1. DOI: 10.1007/978-3-319-98654-8_12 (cit. on pp. 291, 303–305, 323).

[101] Evgeniya A. Bondar, David Casas, and Mikhail V. Volkov. "Completely Reachable Automata: an Interplay Between Automata, Graphs, and Trees". In: *International Journal of Foundations of Computer Science* 34.6 (2023), pp. 655–690. DOI: 10.1142/S0129054123450053 (cit. on pp. 265, 266, 291, 303–305, 323).

[102] Allan Borodin. "On Relating Time and Space to Size and Depth". In: *SIAM Journal on Computing* 6.4 (1977), pp. 733–744. DOI: 10.1137/0206054 (cit. on pp. 66, 68, 72, 87, 88).

[103] John N. Bray et al. "The Hall–Paige Conjecture, and Synchronization for Affine and Diagonal Groups". In: *Journal of Algebra* 545 (2020), pp. 27–42. DOI: 10.1016/j.jalgebra.2019.02.025 (cit. on p. 395).

[104] Martin R. Bridson and Robert H. Gilman. "Context-Free Languages of Sub-exponential Growth". In: *Journal of Computer and System Sciences* 64.2 (2002), pp. 308–310. DOI: 10.1006/jcss.2001.1804 (cit. on p. 453).

[105] Jens Bruchertseifer and Henning Fernau. "Synchronizing Series-Parallel Deterministic Finite Automata with Loops and Related Problems". In: *RAIRO Theoretical Informatics and Applications* 55.7 (2021), pp. 1–24. DOI: 10.1051/ita/2021005 (cit. on pp. 223, 249–251, 253).

[106] Janusz A. Brzozowski. "Derivatives of Regular Expressions". In: *Journal of the ACM* 11.4 (1964), pp. 481–494. DOI: 10.1145/321239.321249 (cit. on pp. 57, 163).

[107] Janusz A. Brzozowski. "In Search of Most Complex Regular Languages". In: *International Journal of Foundations of Computer Science* 24.6 (2013), pp. 691–708. DOI: 10.1142/S0129054113400133 (cit. on pp. 95, 391).

[108] Janusz A. Brzozowski and Sylvie Davies. "Most Complex Deterministic Union-Free Regular Languages". In: *Descriptional Complexity of Formal Systems - 20th IFIP WG 1.02 International Conference, DCFS 2018, Halifax, NS, Canada, July 25-27, 2018, Proceedings*. Ed. by Stavros Konstantinidis and Giovanni Pighizzini. Vol. 10952. Lecture Notes in Computer Science. Springer, 2018, pp. 37–48. DOI: 10.1007/978-3-319-94631-3_4 (cit. on p. 391).

[109] Janusz A. Brzozowski and Sylvie Davies. "Most Complex Non-Returning Regular Languages". In: *International Journal of Foundations of Computer Science* 30.6n7 (2019), pp. 921–957. DOI: 10.1142/S0129054119400239 (cit. on p. 391).

[110] Janusz A. Brzozowski, Sylvie Davies, and Bo Yang Victor Liu. "Most Complex Regular Ideal Languages". In: *Discrete Mathematics & Theoretical Computer Science* 18.3 (2016). DOI: 10.46298/dmtcs.1343 (cit. on pp. 391–393).

[111] Janusz A. Brzozowski and Faith E. Fich. “Languages of $\mathcal{R}$-Trivial Monoids”. In: *Journal of Computer and System Sciences* 20.1 (1980), pp. 32–49. DOI: 10.1016/0022-0000(80)90003-3 (cit. on pp. 82, 172, 223).

[112] Janusz A. Brzozowski and Bo Liu. “Quotient Complexity of Star-Free Languages”. In: *International Journal of Foundations of Computer Science* 23.6 (2012), pp. 1261–1276. DOI: 10.1142/S0129054112400515 (cit. on p. 327).

[113] Janusz A. Brzozowski and Imre Simon. “Characterizations of Locally Testable Events”. In: *Discrete Mathematics* 4.3 (1973), pp. 243–271. DOI: 10.1016/S0012-365X(73)80005-6 (cit. on p. 172).

[114] Janusz A. Brzozowski, Marek Szykuła, and Yuli Ye. “Syntactic Complexity of Regular Ideals”. In: *Theory of Computing Systems* 62.5 (2018), pp. 1175–1202. DOI: 10.1007/s00224-017-9803-8 (cit. on p. 393).

[115] Arthur W. Burks and Hao Wang. “The Logic of Automata - Part I”. In: *Journal of the ACM* 4.2 (1957), pp. 193–218. DOI: 10.1145/320868.320880 (cit. on p. 170).

[116] Arthur W. Burks and Hao Wang. “The Logic of Automata - Part II”. In: *Journal of the ACM* 4.3 (1957), pp. 279–297. DOI: 10.1145/320881.320885 (cit. on pp. 169, 170).

[117] Tim Burness. *Topics in Permutation Group Theory*. https://seis.bristol.ac.uk/~tb13602/docs/permgroups_14.pdf. Young Algebraists’ Conference, Lausanne. June 2014 (cit. on p. 129).

[118] William Burnside. “On the Representation of a Group of Finite Order as a Permutation Group, and on the Composition of Permutation Groups”. In: *Proceedings of the London Mathematical Society* s1-34.1 (1901), pp. 159–168. DOI: 10.1112/plms/s1-34.1.159 (cit. on p. 124).

[119] William Burnside. *Theory of Groups of Finite Order*. Cambridge: The University Press, 1911. ISBN: 9781139237253. DOI: 10.1017/CBO9781139237253 (cit. on pp. 129, 396).

[120] G. Butler. *Fundamental Algorithms for Permutation Groups*. Vol. 559. Lecture Notes in Computer Science. Springer, 1991. ISBN: 978-3-540-53983-7. DOI: 10.1007/3-540-54233-5 (cit. on p. 130).

[121] Albert R. Meyer and C. Thompson. “Remarks on Algebraic Decomposition of Automata”. In: *Mathematical Systems Theory* 3.2 (1969), pp. 110–118. DOI: 10.1007/BF01746516 (cit. on pp. 170, 172).

[122] Justin Cai and Rafael M. Frongillo. “Computational Complexity of Problems for Deterministic Presentations of Sofic Shifts”. In: *Theoretical Computer Science* 936 (2022), pp. 92–115. DOI: 10.1016/J.TCS.2022.09.017 (cit. on p. 418).

[123] Liming Cai, Jianer Chen, and Rodney G. Downey and Michael R. Fellows. “On the Parameterized Complexity of Short Computation and Factorization”. In: *Archive for Mathematical Logic* 36.4-5 (1997), pp. 321–337. DOI: 10.1007/s001530050069 (cit. on p. 256).

[124] Peter Cameron. *The symmetric group, 12*. https://cameroncounts.wordpress.com/2011/04/21/the-symmetric-group-12/. [Online; accessed 2026-01-04]. 2011 (cit. on p. 396).

[125] Peter J. Cameron. "Finite Permutation Groups and Finite Simple Groups". In: *Bulletin of the London Mathematical Society* 13.1 (1981), pp. 1–22. DOI: 10.1112/blms/13.1.1 (cit. on p. 396).

[126] Peter J. Cameron. *Permutation Groups*. London Mathematical Society Student Texts. Cambridge University Press, 1999. DOI: 10.1017/CBO9780511623677 (cit. on pp. 82, 112, 117, 123, 124, 127, 129, 168, 365, 369, 379, 395, 396).

[127] Peter J. Cameron. "Automorphisms of graphs". In: *Topics in Algebraic Graph Theory*. Ed. by Lowell W. Beineke and Robin J. Wilson. Vol. 110. Encyclopedia of Mathematics and Its Applications. Cambridge University Press, 2004, pp. 137–155. ISBN: 978-0-521-80197-3 (cit. on p. 130).

[128] Peter J. Cameron et al. "Lengths of Words in Transformation Semigroups Generated by Digraphs". In: *Journal of Algebraic Combinatorics* 45.1 (2017), pp. 149–170. DOI: 10.1007/s10801-016-0703-9 (cit. on pp. 95, 323).

[129] Pascal Caron et al. "New Tools for State Complexity". In: *Discrete Mathematics & Theoretical Computer Science* 22.1 (2020). DOI: 10.23638/DMTCS-22-1-9 (cit. on p. 394).

[130] Arturo Carpi and Flavio D'Alessandro. "Strongly Transitive Automata and the Cerný Conjecture". In: *Acta Informatica* 46.8 (2009), pp. 591–607. DOI: 10.1007/S00236-009-0106-7 (cit. on pp. 181, 324).

[131] Arturo Carpi and Flavio D'Alessandro. "On Incomplete and Synchronizing Finite Sets". In: *Theoretical Computer Science* 664 (2017), pp. 67–77. DOI: 10.1016/J.TCS.2015.08.042 (cit. on pp. 324, 452).

[132] Arturo Carpi and Flavio D'Alessandro. "Locally Strongly Transitive Automata in the Černý Conjecture and Related Problems". In: *Journal of Automata, Languages and Combinatorics* 24.2-4 (2019), pp. 165–184. DOI: 10.25596/JALC-2019-165 (cit. on p. 324).

[133] Arturo Carpi et al. "Relationships Between Bounded Languages, Counter Machines, Finite-Index Grammars, Ambiguity, and Commutative Regularity". In: *Theoretical Computer Science* 862 (2021), pp. 97–118. DOI: 10.1016/J.TCS.2020.10.006 (cit. on p. 454).

[134] David Casas and Mikhail V. Volkov. "Binary Completely Reachable Automata". In: *LATIN 2022: Theoretical Informatics - 15th Latin American Symposium, Guanajuato, Mexico, November 7-11, 2022, Proceedings*. Ed. by Armando Castañeda and Francisco Rodríguez-Henríquez. Vol. 13568. Lecture Notes in Computer Science. Springer, 2022, pp. 345–358. DOI: 10.1007/978-3-031-20624-5_21 (cit. on pp. 32, 291, 323, 331, 342, 343, 349).

[135] David Casas and Mikhail V. Volkov. "Don's Conjecture for Binary Completely Reachable Automata: An Approach and its Limitations". In: *Journal of Automata, Languages and Combinatorics* 29.2–4 (2024), pp. 159–175. DOI: https://arxiv.org/abs/2311.00077. arXiv: 2310.12345 [cs.FL] (cit. on pp. 14, 347).

[136] Christos G. Cassandras and Stéphane Lafortune. *Introduction to Discrete Event Systems, Second Edition*. Springer, 2008. ISBN: 978-0-387-33332-8. DOI: 10.1007/978-0-387-68612-7 (cit. on p. 416).

[137] Arthur Cayley. "On the Theory of Groups, as Depending on the Symbolic Equation $\theta^n = 1$ – Part I". In: *The London, Edinburgh, and Dublin Philosophical*

Magazine and Journal of Science 7.42 (1854), pp. 40–47. DOI: 10.1080/14786445408647421 (cit. on pp. 45, 129).

[138] Arthur Cayley. "On the Theory of Groups, as Depending on the Symbolic Equation $\theta^n = 1$ – Part II". In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 7.47 (1854), pp. 408–409. DOI: 10.1080/14786445408651852 (cit. on p. 129).

[139] Arthur Cayley. "On the Theory of Groups, as Depending on the Symbolic Equation $\theta^n = 1$ – Part III". In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 18.117 (1859), pp. 34–37. DOI: 10.1080/14786445908642716 (cit. on p. 129).

[140] Ján Černý. "Poznámka k Homogénnym Experimentom s Konečnými Automatmi". In: *Matematicko-fyzikálny časopis* 14.3 (1964). (Translation: A Note on Homogeneous Experiments with Finite Automata. *Journal of Automata, Languages and Combinatorics* 24 (2019) 2-4, 123–132), pp. 208–216. DOI: 10.25596/jalc-2019-123 (cit. on pp. 5, 10–13, 182, 225, 232, 402, 406).

[141] Ján Cerný, Alica Pirická, and Blanka Rosenauerová. "On Directable Automata". In: *Kybernetika* 7.4 (1971), pp. 289–298. eprint: http://www.kybernetika.cz/content/1971/4/289 (cit. on pp. 12, 13).

[142] Ashok K. Chandra, Steven Fortune, and Richard J. Lipton. "Unbounded Fan-In Circuits and Associative Functions". In: *Journal of Computer and System Sciences* 30.1 (1985), pp. 222–234. DOI: 10.1016/0022âĂŚ0000(85)90014âĂŚ2 (cit. on p. 89).

[143] Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. "Alternation". In: *Journal of the ACM* 28.1 (1981), pp. 114–133. DOI: 10.1145/322234.322243 (cit. on p. 86).

[144] Ashok K. Chandra, Larry J. Stockmeyer, and Uzi Vishkin. "Constant Depth Reducibility". In: *SIAM Journal on Computing* 13.2 (1984), pp. 423–439. DOI: 10.1137/0213028 (cit. on pp. 80, 86).

[145] Yeow Meng Chee et al. "Cross-Bifix-Free Codes Within a Constant Factor of Optimality". In: *IEEE Transactions on Information Theory* 59.7 (2013), pp. 4668–4674. DOI: 10.1109/TIT.2013.2252571. arXiv: 1209.0236 [cs.IT] (cit. on p. 426).

[146] Yui-Bin Chen and Doug Ierardi. "The Complexity of Oblivious Plans for Orienting and Distinguishing Polygonal Parts". In: *Algorithmica* 14.5 (1995), pp. 367–397. DOI: 10.1007/BF01192046 (cit. on p. 9).

[147] Andrew Chiu, George I. Davida, and Bruce E. Litow. "Division in Logspace-Uniform NC^1". In: *RAIRO Theoretical Informatics and Applications* 35.3 (2001), pp. 259–275. DOI: 10.1051/ita:2001119 (cit. on p. 74).

[148] Noam Chomsky. "Three Models for the Description of Language". In: *IRE Transactions on Information Theory* 2.3 (1956), pp. 113–124. DOI: 10.1109/TIT.1956.1056813 (cit. on pp. 169, 170).

[149] Noam Chomsky. "Context-Free Grammars and Pushdown Storage". In: *MIT Research Laboratory Electronics Quarterly Progress Report* 65 (1962), pp. 187–194 (cit. on p. 84).

[150] Noam Chomsky and George A. Miller. “Finite State Languages”. In: *Information and Computation* 1.2 (1958), pp. 91–112. DOI: 10.1016/S0019-9958(58)90082-2 (cit. on p. 169).

[151] Berk Çirisci et al. “Synchronizing Heuristics for Weakly Connected Automata with Various Topologies”. In: *Model-Driven Engineering and Software Development - 6th International Conference, MODELSWARD 2018, Funchal, Madeira, Portugal, January 22-24, 2018, Revised Selected Papers*. Ed. by Slimane Hammoudi, Luís Ferreira Pires, and Bran Selic. Vol. 991. Communications in Computer and Information Science. Springer, 2018, pp. 475–493. DOI: 10.1007/978-3-030-11030-7_21 (cit. on p. 254).

[152] Alfred H. Clifford and Gordon Preston. *The Algebraic Theory of Semigroups, Volume I*. Mathematical Surveys 7. Providence, Rhode Island: American Mathematical Society, 1961 (cit. on p. 84).

[153] Alfred H. Clifford and Gordon Preston. *The Algebraic Theory of Semigroups, Volume II*. Mathematical surveys and monographs Bd. 2. American Mathematical Society, 1967. ISBN: 9780821802724 (cit. on p. 84).

[154] Peter Clote and Evangelos Kranakis. *Boolean Functions and Computation Models*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2002. ISBN: 978-3-642-08217-7. DOI: 10.1007/978-3-662-04943-3 (cit. on p. 89).

[155] Joëlle Cohen, Dominique Perrin, and Jean-Eric Pin. “On the Expressive Power of Temporal Logic”. In: *Journal of Computer and System Sciences* 46.3 (1993), pp. 271–294. DOI: 10.1016/0022-0000(93)90005-H (cit. on pp. 171, 172).

[156] Stephen A. Cook. “The Complexity of Theorem-Proving Procedures”. In: *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing, May 3-5, 1971, Shaker Heights, Ohio, USA*. Ed. by Michael A. Harrison, Ranan B. Banerji, and Jeffrey D. Ullman. ACM, 1971, pp. 151–158. DOI: 10.1145/800157.805047 (cit. on pp. 70, 73, 85).

[157] Stephen A. Cook. “Deterministic CFL’s Are Accepted Simultaneously in Polynomial Time and Log Squared Space”. In: *Proceedings of the 11h Annual ACM Symposium on Theory of Computing, April 30 - May 2, 1979, Atlanta, Georgia, USA*. Ed. by Michael J. Fischer et al. ACM, 1979, pp. 338–345. DOI: 10.1145/800135.804426 (cit. on pp. 67, 87, 88).

[158] Stephen A. Cook. “Towards a Complexity Theory of Synchronous Parallel Computation”. In: *L’Enseignement Matheématique* 27.1–2 (1981), pp. 99–124. DOI: 10.5169/seals-51742 (cit. on p. 73).

[159] Stephen A. Cook. “The Classification of Problems which have Fast Parallel Algorithms”. In: *Fundamentals of Computation Theory, Proceedings of the 1983 International FCT-Conference, Borgholm, Sweden, August 21-27, 1983*. Ed. by Marek Karpinski. Vol. 158. Lecture Notes in Computer Science. Springer, 1983, pp. 78–93. DOI: 10.1007/3-540-12689-9_95 (cit. on pp. 67, 88).

[160] Stephen A. Cook. “A Taxonomy of Problems with Fast Parallel Algorithms”. In: *Information and Computation* 64.1-3 (1985), pp. 2–21. DOI: 10.1016/S0019-9958(85)80041-3 (cit. on pp. 67, 71, 86–88).

[161] Stephen A. Cook and Pierre McKenzie. “Problems Complete for Deterministic Logarithmic Space”. In: *Journal of Algorithms* 8.3 (1987), pp. 385–394. DOI: 10.1016/0196-6774(87)90018-6 (cit. on pp. 70, 73).

[162] Stephen A. Cook and Robert A. Reckhow. "Time Bounded Random Access Machines". In: *Journal of Computer and System Sciences* 7.4 (1973), pp. 354–375. DOI: 10.1016/S0022-0000(73)80029-7 (cit. on pp. 72, 85).

[163] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009. ISBN: 978-0-262-03384-8 (cit. on pp. 64, 184, 185, 328, 509, 539).

[164] Edwin Hamel-De le Court. "An Algebraic Theory for State Complexity. (Une théorie Algébrique pour la compléxité en Etats)". PhD thesis. Normandy University, Caen, France, 2020. eprint: https://tel.archives-ouvertes.fr/tel-03329819 (cit. on p. 394).

[165] Flavio D'Alessandro, Benedetto Intrigila, and Stefano Varricchio. "On the Structure of the Counting Function of Sparse Context-Free Languages". In: *Theoretical Computer Science* 356.1-2 (2006), pp. 104–117. DOI: 10.1016/j.tcs.2006.01.037 (cit. on p. 453).

[166] Emma D'Aniello and Martina Maiuriello. "A Survey on Composition Operators on Some Function Spaces". In: *Aequationes Mathematicae* 95 (2021), pp. 677–697. DOI: 10.1007/s00010-020-00768-5 (cit. on p. 82).

[167] Elias Dahlhaus. "Reduction to NP-complete problems by interpretations". In: *Logic and Machines: Decision Problems and Complexity, Proceedings of the Symposium "Rekursive Kombinatorik" held from May 23-28, 1983 at the Institut für Mathematische Logik und Grundlagenforschung der Universität Münster/Westfalen*. Ed. by Egon Börger, Gisbert Hasenjaeger, and Dieter Rödding. Vol. 171. Lecture Notes in Computer Science. Springer, 1983, pp. 357–365. DOI: 10.1007/3-540-13331-3_51 (cit. on p. 85).

[168] Samir Datta et al. "Counting Classes and the Fine Structure Between NC^1 and L". In: *Theoretical Computer Science* 417 (2012), pp. 36–49. DOI: 10.1016/j.tcs.2011.05.050 (cit. on p. 89).

[169] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. 2nd ed. Cambridge: Cambridge University Press, 2002. ISBN: 0521784514 (cit. on p. 84).

[170] Elena Biagi and Davide Cenzato, Zsuzsanna Lipták, and Giuseppe Romana. "On the Number of Equal-Letter Runs of the Bijective Burrows-Wheeler Transform". In: *Theoretical Computer Science* 1027 (2025), p. 115004. DOI: 10.1016/J.TCS.2024.115004 (cit. on p. 454).

[171] Sylvie Davies. "Primitivity, Uniform Minimality, and State Complexity of Boolean Operations". In: *Theory of Computing Systems* 62.8 (2018), pp. 1952–2005. DOI: 10.1007/s00224-018-9859-0 (cit. on pp. 164, 169, 191, 360, 391).

[172] Sylvie Davies. "Algebraic Approaches to State Complexity of Regular Operations". PhD thesis. University of Waterloo, Ontario, Canada, 2019. eprint: https://hdl.handle.net/10012/15203 (cit. on pp. 164, 169, 360, 391).

[173] Christian Dax and Felix Klaedtke. "Alternation Elimination by Complementation (Extended Abstract)". In: *Logic for Programming, Artificial Intelligence, and Reasoning, 15th International Conference, LPAR 2008, Doha, Qatar, November 22-27, 2008. Proceedings*. Ed. by Iliano Cervesato, Helmut Veith,

and Andrei Voronkov. Vol. 5330. Lecture Notes in Computer Science. Springer, 2008, pp. 214–229. DOI: 10.1007/978-3-540-89439-1_16 (cit. on p. 223).

[174] R.G. Deshmukh and G.N. Hawat. "An Algorithm to Determine Shortest Length Distinguishing, Homing, and Synchronizing Sequences for Sequential Machines". In: *Conference Record Southcon, Orlando, FL, USA, 39–31 March 1994*. Ed. by Alfred E. Dunlop. IEEE, 1994, pp. 496–501. DOI: 10.1109/SOUTHC.1994.498154 (cit. on p. 254).

[175] Volker Diekert, Paul Gastin, and Manfred Kufleitner. "A Survey on Small Fragments of First-Order Logic over Finite Words". In: *International Journal of Foundations of Computer Science* 19.3 (2008), pp. 513–548. DOI: 10.1142/S0129054108005802 (cit. on p. 82).

[176] Volker Diekert, Manfred Kufleitner, and Benjamin Steinberg. "The Krohn-Rhodes Theorem and Local Divisors". In: *Fundamenta Informaticae* 116.1-4 (2012), pp. 65–77. DOI: 10.3233/FI-2012-669 (cit. on p. 170).

[177] Reinhard Diestel. *Graph Theory*. 5th ed. Vol. 173. Graduate Texts in Mathematics. Springer, 2017. ISBN: 978-3-662-53621-6. DOI: 10.1007/978-3-662-53622-3 (cit. on p. 84).

[178] Gabriel A. Dirac. "Extensions of Menger's Theorem". In: *Journal of the London Mathematical Society* s1-38.1 (1963), pp. 148–154. DOI: 10.1112/jlms/s1-38.1.148 (cit. on p. 61).

[179] John D. Dixon and Brian Mortimer. *Permutation Groups*. Graduate Texts in Mathematics. Graduate Studies in Mathematics. Springer New York, 1996. ISBN: 9780387945996 (cit. on pp. 82, 116, 117, 129, 168, 170, 172, 365, 369, 379, 395, 396).

[180] Akshunna S. Dogra and William T. Redman. "Optimizing Neural Networks via Koopman Operator Theory". In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Poster session; 13 pages. 2020, pp. 2087–2097 (cit. on p. 82).

[181] Pál Dömösi and Chrystopher L. Nehaniv. *Algebraic Theory of Automata Networks - An Introduction*. Vol. 11. SIAM monographs on discrete mathematics and applications. SIAM, 2005. ISBN: 978-0-89871-569-9 (cit. on p. 170).

[182] Henk Don. "The Černý Conjecture and 1-Contracting Automata". In: *Electronic Journal of Combinatorics* 23.P3.12 (3 2016), P3.12. DOI: 10.37236/5616 (cit. on pp. 14, 338, 347).

[183] Henk Don and Hans Zantema. "Counting Symbol Switches in Synchronizing Automata". In: *Journal of Automata, Languages and Combinatorics* 24.2-4 (2019), pp. 253–286. DOI: 10.25596/JALC-2019-253 (cit. on p. 454).

[184] Henk Don, Hans Zantema, and Michiel de Bondt. "Slowly Synchronizing Automata with Fixed Alphabet Size". In: *Information and Computation* 279 (2021), p. 104614. DOI: 10.1016/j.ic.2020.104614 (cit. on pp. 397, 398, 406).

[185] Christopher F. Freiling and Douglas S. Jungreis, François Théberge, and Kenneth Zeger. "Almost all Complete Binary Prefix Codes Have a Self-Synchronizing String". In: *IEEE Transactions on Information Theory* 49.9 (2003), pp. 2219–2225. DOI: 10.1109/TIT.2003.815803 (cit. on p. 452).

[186] Louis Dubuc. “Les Automates Circulaires Biaisés Vérifient la Conjecture de Černý”. In: *RAIRO Theoretical Informatics and Applications* 30.6 (1996), pp. 495–505. DOI: 10.1051/ITA/1996300604951 (cit. on p. 13).

[187] Louis Dubuc. “Sur Les Automates Circulaires et la Conjecture de Černý”. In: *RAIRO Theoretical Informatics and Applications* 32.1-3 (1998), pp. 21–34. DOI: 10.1051/ITA/1998321-300211 (cit. on pp. 13, 338, 409).

[188] James East. “Defining Relations for Idempotent Generators in Finite Full Transformation Semigroups”. In: *Semigroup Forum* 86.3 (2013), pp. 451–485. DOI: 10.1007/s00233-012-9447-6 (cit. on p. 95).

[189] James East, Maximilien Gadouleau, and James D. Mitchell. “Structural Aspects of Semigroups Based on Digraphs”. en. In: *Algebraic Combinatorics* 2.5 (2019), pp. 711–733. DOI: 10.5802/alco.56 (cit. on p. 323).

[190] Attila Egri-Nagy. “Algebraic Hierarchical Decomposition of Finite State Automata : A Computational Approach”. PhD thesis. University of Hertfordshire, UK, 2005. eprint: https://hdl.handle.net/2299/14267 (cit. on p. 170).

[191] Attila Egri-Nagy, James D. Mitchell, and Chrystopher L. Nehaniv. “Algorithms for the Efficient Calculation of the Holonomy Decomposition”. In: *Automata and Formal Languages, 13th International Conference, AFL 2011, Debrecen, Hungary, August 17-22, 2011, Proceedings*. Ed. by Pál Dömösi and Szabolcs Iván. 2011, pp. 389–390 (cit. on p. 170).

[192] Attila Egri-Nagy and Chrystopher L. Nehaniv. “Algebraic Hierarchical Decomposition of Finite State Automata: Comparison of Implementations for Krohn-Rhodes Theory”. In: *Implementation and Application of Automata, 9th International Conference, CIAA 2004, Kingston, Canada, July 22-24, 2004, Revised Selected Papers*. Ed. by Michael Domaratzki and Alexander Okhotin and Kai Salomaa and Sheng Yu. Vol. 3317. Lecture Notes in Computer Science. Springer, 2004, pp. 315–316. DOI: 10.1007/978-3-540-30500-2_32 (cit. on p. 170).

[193] Attila Egri-Nagy and Chrystopher L. Nehaniv. “Cycle Structure in Automata and the Holonomy Decomposition”. In: *Acta Cybernetica* 17.2 (2006), pp. 199–211. eprint: https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/3663 (cit. on p. 170).

[194] Samuel Eilenberg. *Automata, Languages, and Machines, Volume A*. Orlando, FL, USA: Academic Press, Inc., 1974. ISBN: 978-0-12-234001-7 (cit. on pp. 4, 40, 58, 82–84, 159, 170, 431, 445).

[195] Samuel Eilenberg. *Automata, Languages, and Machines, Volume B*. Orlando, FL, USA: Academic Press, Inc., 1976. ISBN: 978-0-12-234002-4 (cit. on pp. 58, 82, 84, 170, 222).

[196] Samuel Eilenberg and Marcel P. Schützenberger. “On Pseudovarieties”. In: *Advances in Mathematics* 19.3 (1976), pp. 413–418. ISSN: 0001-8708. DOI: 10.1016/0001-8708(76)90029-3 (cit. on p. 170).

[197] Achille Frigeri and Emanuele Rodaro. “Reset Sink Complexity of Regular Ideals”. In: *Journal of Automata, Languages and Combinatorics* 30.1-3 (2025), pp. 77–94. DOI: 10.25596/JALC-2025-077 (cit. on pp. 16, 292, 302).

[198] David Eppstein. “Reset Sequences for Monotonic Automata”. In: *SIAM Journal on Computing* 19.3 (1990), pp. 500–510. DOI: 10.1137/0219033 (cit. on pp. 5, 10, 12, 13, 184, 185, 223, 226, 232, 249, 250, 254, 255, 510, 534).

[199] David Eppstein and Jean-Claude Falmagne. “Algorithms for Media”. In: *Discrete Applied Mathematics* 156.3 (2008), pp. 291–307. DOI: 10.1016/j.dam.2007.05.035 (cit. on p. 14).

[200] Michael A. Erdmann and Matthew T. Mason. “An Exploration of Sensorless Manipulation”. In: *IEEE J. Robotics Autom.* 4.4 (1988), pp. 369–379. DOI: 10.1109/56.800 (cit. on p. 9).

[201] Zoltán Ésik. “A Proof of the Krohn-Rhodes Decomposition Theorem”. In: *Theoretical Computer Science* 234.1-2 (2000), pp. 287–300. DOI: 10.1016/S0304-3975(99)00315-1 (cit. on p. 170).

[202] Zoltán Ésik and Balázs Imreh. “Remarks on Finite Commutative Automata”. In: *Acta Cybernetica* 5.2 (1981), pp. 143–146. eprint: https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/3211 (cit. on p. 222).

[203] Kousha Etessami. “Counting Quantifiers, Successor Relations, and Logarithmic Space”. In: *Journal of Computer and System Sciences* 54.3 (1997), pp. 400–411. DOI: 10.1006/JCSS.1997.1485 (cit. on pp. 71, 73).

[204] Shimon Even and Oded Goldreich. “The Minimum-Length Generator Sequence Problem is NP-Hard”. In: *Journal of Algorithms* 2.3 (1981), pp. 311–313. DOI: 10.1016/0196-6774(81)90029-8 (cit. on p. 256).

[205] Robert Ferens and Marek Szykuła. “Completely Reachable Automata: A Polynomial Algorithm and Quadratic Upper Bounds”. In: *50th International Colloquium on Automata, Languages, and Programming, ICALP 2023, July 10-14, 2023, Paderborn, Germany*. Ed. by Kousha Etessami and Uriel Feige and Gabriele Puppis. Vol. 261. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 59:1–59:17. DOI: 10.4230/LIPIcs.ICALP.2023.59 (cit. on pp. 14, 331, 349, 352, 355).

[206] Robert Ferens and Marek Szykuła. “Recognizing Completely Reachable Automata in Quadratic Time”. In: *ACM Transactions on Algorithms* 22.2 (2026), pp. 1–36. DOI: 10.1145/3798283 (cit. on p. 347).

[207] Thomas J. Ferguson and J. H. Rabinowitz. “Self-Synchronizing Huffman Codes”. In: *IEEE Transactions on Information Theory* 30.4 (1984), pp. 687–693. DOI: 10.1109/TIT.1984.1056931 (cit. on p. 502).

[208] Henning Fernau. *Diskrete Strukturen – Grundlagen und Anwendungen*. Vorlesungsskript. 2022 (cit. on p. 83).

[209] Henning Fernau and Jens Bruchertseifer. “Synchronizing Words and Monoid Factorization, Yielding a new Parameterized Complexity Class?” In: *Mathematical Structures in Computer Science* 32.2 (2022), pp. 189–215. DOI: 10.1017/S0960129522000184 (cit. on pp. 250, 251, 253, 255, 256).

[210] Henning Fernau, Carolina Haase, and Stefan Hoffmann. “The Synchronization Game on Subclasses of Automata”. In: *11th International Conference on Fun with Algorithms, FUN 2022, May 30 to June 3, 2022, Island of Favignana, Sicily, Italy*. Ed. by Pierre Fraigniaud and Yushi Uno. Vol. 226. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, 14:1–14:17. DOI: 10.4230/LIPIcs.FUN.2022.14 (cit. on pp. 208, 232, 249, 255).

[211] Henning Fernau, Pinar Heggernes, and Yngve Villanger. "A Multi-Parameter Analysis of Hard Problems on Deterministic Finite Automata". In: *Journal of Computer and System Sciences* 81.4 (2015), pp. 747–765. DOI: 10.1016/j.jcss.2014.12.027 (cit. on pp. 232, 249, 253, 255, 256).

[212] Henning Fernau and Stefan Hoffmann. "Extensions to Minimal Synchronizing Words". In: *Journal of Automata, Languages and Combinatorics* 24.2–4 (2019), pp. 287–307. DOI: 10.25596/jalc-2019-287 (cit. on pp. 14, 205, 207, 209, 222, 250, 255).

[213] Henning Fernau, Stefan Hoffmann, and Michael Wehar. "Finite Automata Intersection Non-Emptiness: Parameterized Complexity Revisited". In: *CoRR* abs/2108.05244 (2021). arXiv: 2108.05244 (cit. on pp. 223, 231, 253, 255, 256).

[214] Henning Fernau and Andreas Krebs. "Problems on Finite Automata and the Exponential Time Hypothesis". In: *Algorithms* 10.1 (2017), pp. 1–25. DOI: 10.3390/a10010024 (cit. on pp. 231, 256).

[215] Henning Fernau et al. "Computational Complexity of Synchronization under Regular Constraints". In: *44th International Symposium on Mathematical Foundations of Computer Science, MFCS 2019, August 26-30, 2019, Aachen, Germany*. Ed. by Peter Rossmanith, Pinar Heggernes, and Joost-Pieter Katoen. Vol. 138. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, 63:1–63:14. ISBN: 978-3-95977-117-7. DOI: 10.4230/LIPIcs.MFCS.2019.63 (cit. on pp. 15, 19, 20, 62, 415, 426, 451, 456, 500, 501, 505, 522–525).

[216] Henning Fernau et al. "Winning Strategies for the Synchronization Game on Subclasses of Finite Automata". In: *Proceedings 14th International Workshop on Non-Classical Models of Automata and Applications (NCMA 2024), NCMA 2024, Göttingen, Germany, 12-13 August 2024*. Ed. by Florin Manea and Giovanni Pighizzini. Vol. 407. EPTCS. 2024, pp. 77–85. DOI: 10.4204/EPTCS.407.6 (cit. on pp. 224, 548).

[217] Davide Ferri. "Split Lemma and First Isomorphism Theorem for Groupoids". In: *arXiv preprint* (2025). arXiv: 2509.13973 [math.GR] (cit. on p. 82).

[218] Arthur C. Fleck. *Structure Preserving Properties of a Certain Class of Functions on Automata*. Tech. rep. 16. available online: http://homepage.cs.uiowa.edu/~fleck/TR16.pdf. Computer Laboratory, Michigan State University, 1961, pp. 469–496 (cit. on p. 167).

[219] Arthur C. Fleck. "Isomorphism Groups of Automata". In: *Journal of the ACM* 9.4 (1962), pp. 469–476. DOI: 10.1145/321138.321144 (cit. on pp. 167, 168, 222).

[220] Arthur C. Fleck. "On the Automorphism Group of an Automaton". In: *Journal of the ACM* 12.4 (1965), pp. 566–569. DOI: 10.1145/321296.321307 (cit. on p. 168).

[221] Lukas Fleischer. "Algorithms and Complexity Results for Finite Semigroups". PhD thesis. Universität Stuttgart, 2019. DOI: 10.18419/opus-10339 (cit. on p. 256).

[222] Lukas Fleischer. "The Intersection Problem for Finite Semigroups". In: *International Journal of Foundations of Computer Science* 31.6 (2020), pp. 827–842. DOI: 10.1142/S0129054120410075 (cit. on p. 256).

[223] Lukas Fleischer and Manfred Kufleitner. “The Intersection Problem for Finite Monoids”. In: *35th Symposium on Theoretical Aspects of Computer Science, STACS 2018, February 28 to March 3, 2018, Caen, France*. Ed. by Rolf Niedermeier and Brigitte Vallée. Vol. 96. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018, 30:1–30:14. DOI: 10.4230/LIPIcs.STACS.2018.30 (cit. on p. 256).

[224] Jörg Flum and Martin Grohe. *Parameterized Complexity Theory*. Text in Theoretical Computer Science. Springer, 2006. DOI: 10.1007/3-540-29953-X (cit. on p. 249).

[225] Steven Fortune and James Wyllie. “Parallelism in Random Access Machines”. In: *Proceedings of the 10th Annual ACM Symposium on Theory of Computing, May 1-3, 1978, San Diego, California, USA*. Ed. by Richard J. Lipton et al. ACM, 1978, pp. 114–118. DOI: 10.1145/800133.804339 (cit. on p. 89).

[226] Ronald M. Foster. “Geometrical Circuits of Electrical Networks”. In: *Transactions of the American Institute of Electrical Engineers* 51.2 (1932), pp. 309–317. DOI: 10.1109/T-AIEE.1932.5056068 (cit. on p. 130).

[227] Peter Frankl. “An Extremal Problem for two Families of Sets”. In: *European Journal of Combinatorics* 3 (1982), pp. 125–127. DOI: 10.1016/S0195-6698(82)80025-5 (cit. on p. 13).

[228] Achille Frigeri and Emanuele Rodaro. “Missing Factors of Ideals and Synchronizing Automata”. In: *Journal of Automata, Languages and Combinatorics* 24.2-4 (2019), pp. 309–320. DOI: 10.25596/jalc-2019-309 (cit. on pp. 14, 16, 302).

[229] Andrea Frosini et al. “Logarithmic Equal-Letter Runs for BWT of Purely Morphic Words”. In: *Developments in Language Theory - 26th International Conference, DLT 2022, Tampa, FL, USA, May 9-13, 2022, Proceedings*. Ed. by Volker Diekert and Mikhail V. Volkov. Vol. 13257. Lecture Notes in Computer Science. Springer, 2022, pp. 139–151. DOI: 10.1007/978-3-031-05578-2_11 (cit. on p. 454).

[230] Yuichiro Fujiwara and Vladimir D. Tonchev. “High-rate self-synchronizing codes”. In: *IEEE Transactions on Information Theory* 59.4 (2013), pp. 2328–2335. DOI: 10.1109/TIT.2012.2234501 (cit. on p. 502).

[231] Merrick L. Furst, John E. Hopcroft, and Eugene M. Luks. “Polynomial-Time Algorithms for Permutation Groups”. In: *21st Annual Symposium on Foundations of Computer Science, Syracuse, New York, USA, 13-15 October 1980*. IEEE Computer Society, 1980, pp. 36–41. DOI: 10.1109/SFCS.1980.34 (cit. on p. 256).

[232] Merrick L. Furst, James B. Saxe, and Michael Sipser. “Parity, Circuits, and the Polynomial-Time Hierarchy”. In: *Mathematical Systems Theory* 17.1 (1984), pp. 13–27. DOI: 10.1007/BF01744431 (cit. on pp. 86, 89).

[233] Harold N. Gabow and Robert E. Tarjan. “A Linear-Time Algorithm for Finding a Minimum Spanning Pseudoforest”. In: *Information Processing Letters* 27.5 (1988), pp. 259–263. ISSN: 0020-0190. DOI: 10.1016/0020-0190(88)90089-0 (cit. on p. 62).

[234] Zvi Galil. “Hierarchies of Complete Problems”. In: *Acta Informatica* 6 (1976), pp. 77–88. DOI: 10.1007/BF00263744 (cit. on pp. 231, 254, 255).

[235] Moses Ganardi et al. “Automata Theory on Sliding Windows”. In: *35th Symposium on Theoretical Aspects of Computer Science, STACS 2018, February 28 to March 3, 2018, Caen, France*. Ed. by Rolf Niedermeier and Brigitte Vallée. Vol. 96. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018, 31:1–31:14. DOI: 10.4230/LIPIcs.STACS.2018.31 (cit. on p. 522).

[236] Olexandr Ganyushkin and Volodymyr Mazorchuk. *Classical Finite Transformation Semigroups: An Introduction*. Algebra and Applications. London: Springer, 2009. ISBN: 978-1-84800-280-7. DOI: 10.1007/978-1-84800-281-4 (cit. on pp. 45, 84, 110).

[237] Yuan Gao et al. “A Survey on Operational State Complexity”. In: *Journal of Automata, Languages and Combinatorics* 21.4 (2017), pp. 251–310. DOI: 10.25596/jalc-2016-251 (cit. on pp. 16, 390).

[238] Michael Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness (Series of Books in the Mathematical Sciences)*. First Edition. W. H. Freeman, 1979. ISBN: 0716710455 (cit. on pp. 70, 71).

[239] Pawel Gawrychowski and Damian Straszak. “Strong Inapproximability of the Shortest Reset Word”. In: *Mathematical Foundations of Computer Science 2015 - 40th International Symposium, MFCS 2015, Milan, Italy, August 24-28, 2015, Proceedings, Part I*. Ed. by Giuseppe F. Italiano, Giovanni Pighizzini, and Donald Sannella. Vol. 9234. Lecture Notes in Computer Science. Springer, 2015, pp. 243–255. DOI: 10.1007/978-3-662-48057-1_19 (cit. on p. 250).

[240] Pawel Gawrychowski et al. “Finding the Growth Rate of a Regular or Context-Free Language in Polynomial Time”. In: *International Journal of Foundations of Computer Science* 21.4 (2010), pp. 597–618. DOI: 10.1142/S0129054110007441 (cit. on pp. 453, 522).

[241] Marcus Gelderie. “Classifying Regular Languages via Cascade Products of Automata”. Diploma Thesis. Rheinisch-Westf¨alische Technische Hochschule Aachen, Lehrstuhl für Informatik VII, available online: https://old.automata.rwth-aachen.de/download/papers/gelderie/ge11.pdf, 2010 (cit. on p. 222).

[242] Marcus Gelderie. “Classifying Regular Languages via Cascade Products of Automata”. In: *Language and Automata Theory and Applications - 5th International Conference, LATA 2011, Tarragona, Spain, May 26-31, 2011. Proceedings*. Ed. by Adrian-Horia Dediu, Shunsuke Inenaga, and Carlos Martín-Vide. Vol. 6638. Lecture Notes in Computer Science. Springer, 2011, pp. 286–297. DOI: 10.1007/978-3-642-21254-3_22 (cit. on p. 222).

[243] Michael Gerbush and Brent Heeringa. “Approximating Minimum Reset Sequences”. In: *Implementation and Application of Automata - 15th International Conference, CIAA 2010, Winnipeg, MB, Canada, August 12-15, 2010. Revised Selected Papers*. Ed. by Michael Domaratzki and Kai Salomaa. Vol. 6482. Lecture Notes in Computer Science. Springer, 2010, pp. 154–162. DOI: 10.1007/978-3-642-18098-9_17 (cit. on p. 250).

[244] Edgar N. Gilbert. “Synchronization of Binary Messages”. In: *IRE Transactions on Information Theory* 6.4 (1960), pp. 470–477. DOI: 10.1109/TIT.1960.1057587 (cit. on p. 426).

[245] Seymour Ginsburg and Edwin H. Spanier. "Bounded ALGOL-Like Languages". In: *Transactions of the American Mathematical Society* 113.2 (1964), pp. 333–368. DOI: 10.1090/S0002-9947-1964-0181500-1 (cit. on pp. 428, 441, 448, 453).

[246] Seymour Ginsburg and Edwin H. Spanier. "Bounded Regular Sets". In: *Proceedings of the American Mathematical Society* 17.5 (1966), pp. 1043–1049. ISSN: 00029939, 10886826 (cit. on pp. 442, 453).

[247] Abraham Ginzburg. *Algebraic Theory of Automata*. ACM monograph series. New York: Academic Press, 1968. ISBN: 0122850505 (cit. on pp. 82, 170).

[248] Victor M. Glushkov. "The Abstract Theory of Automata". In: *Russian Mathematical Surveys* 16 (5 1961), pp. 1–53. DOI: 10.1070/RM1961v016n05ABEH004112 (cit. on p. 171).

[249] Chris Godsil and Gordon F. Royle. *Algebraic Graph Theory*. Graduate Texts in Mathematics Book 207. Springer, 2001. ISBN: 978-0-387-95241-3. DOI: 10.1007/978-1-4613-0163-9 (cit. on pp. 126–128).

[250] Wolf Göhring. "Minimal Initializing Word: A Contribution to Černý's Conjecture". In: *Journal of Automata, Languages and Combinatorics* 2.4 (1997), pp. 209–226. DOI: 10.25596/jalc-1997-209 (cit. on p. 395).

[251] Christian Golaszewski and Peter J. Ramadge. "The Complexity of Some Reachability Problems for a System on a Finite Group". In: *Systems & Control Letters* 12.5 (1989), pp. 431–435. ISSN: 0167-6911. DOI: https://doi.org/10.1016/0167-6911(89)90079-0 (cit. on p. 252).

[252] Kenneth Y. Goldberg. "Orienting Polygonal Parts Without Sensors". In: *Algorithmica* 10.2-4 (1993), pp. 210–225. DOI: 10.1007/BF01891840 (cit. on pp. 5, 9).

[253] Solomon Golomb, Basil Gordon, and Lloyd R. Welch. "Comma-Free Codes". In: *Canadian Journal of Mathematics* 10 (1958), pp. 202–209. DOI: 10.4153/CJM-1958-023-9 (cit. on p. 427).

[254] Gracinda M. S. Gomes and John M. Howie. "On the Ranks of Certain Finite Semigroups of Transformations". In: *Mathematical Proceedings of the Cambridge Philosophical Society* 101.3 (1987), pp. 395–403. DOI: 10.1017/S0305004100066780 (cit. on p. 110).

[255] François Gonze and Raphaël M. Jungers. "Hardly Reachable Subsets and Completely Reachable Automata with 1-Deficient Words". In: *Journal of Automata, Languages and Combinatorics* 24.2-4 (2019), pp. 321–342. DOI: 10.25596/jalc-2019-321 (cit. on pp. 61, 323, 347).

[256] François Gonze et al. "On the Interplay Between Černý and Babai's Conjectures". In: *International Journal of Foundations of Computer Science* 30.1 (2019), pp. 93–114. DOI: 10.1142/S0129054119400057 (cit. on pp. 14, 167, 323).

[257] Pavel Goralčík and Václav Koubek. "Rank Problems for Composite Transformations". In: *International Journal of Algebra and Computation* 5.3 (1995), pp. 309–316. DOI: 10.1142/S0218196795000185 (cit. on pp. 70, 91, 232, 249, 254, 255).

[258] Frank Göring. "Short Proof of Menger's Theorem". In: *Discrete Mathematics* 219.1-3 (2000), pp. 295–296. DOI: 10.1016/S0012-365X(00)00088-1 (cit. on p. 61).

[259] Mariusz Grech and Andrzej Kisielewicz. "The Černý Conjecture for Automata Respecting Intervals of a Directed Graph". In: *Discrete Mathematics & Theoretical Computer Science* 15.3 (2013), pp. 61–72. DOI: 10.46298/dmtcs.619 (cit. on p. 14).

[260] Mariusz Grech and Andrzej Kisielewicz. "Synchronizing Sequences for Road Colored Digraphs". In: *Discrete Applied Mathematics* 285 (2020), pp. 128–140. DOI: 10.1016/j.dam.2020.04.029 (cit. on p. 14).

[261] James A. Green. "On the Structure of Semigroups". In: *Annals of Mathematics (second series).* 54.1 (1951), pp. 163–172. DOI: 10.2307/1969317 (cit. on p. 172).

[262] Raymond Greenlaw, H. James Hoover, and Walter L. Ruzzo. *Limits to Parallel Computation: P-Completeness Theory*. available online: https://homes.cs.washington.edu/~ruzzo/papers/limits.pdf. USA: Oxford University Press, Inc., 1995. ISBN: 0195085914 (cit. on p. 89).

[263] Rostislav I. Grigorchuk and A. Machí. "An Example of an Indexed Language of Intermediate Growth". In: *Theoretical Computer Science* 215.1-2 (1999), pp. 325–327. DOI: 10.1016/S0304-3975(98)00161-3 (cit. on p. 453).

[264] David D. Grossman and Mike W. Blasgen. "Orienting Mechanical Parts by Computer-Controlled Manipulator". In: *IEEE Transactions on Systems, Man, and Cybernetics* 5.5 (1975), pp. 561–565. DOI: 10.1109/TSMC.1975.5408381 (cit. on p. 9).

[265] Lucian Ilie and Grzegorz Rozenberg and Arto Salomaa. "A Characterization of Poly-Slender Context-Free Languages". In: *RAIRO Theoretical Informatics and Applications* 34.1 (2000), pp. 77–86. DOI: 10.1051/ITA:2000100 (cit. on p. 453).

[266] Jerzy W. Grzymala-Busse. "On the Endomorphisms of Finite Automata". In: *Mathematical Systems Theory* 4.4 (1970), pp. 373–384. DOI: 10.1007/BF01704080 (cit. on p. 168).

[267] Jerzy W. Grzymala-Busse. "Operation-Preserving Functions and Autonomous Factors of Finite Automata". In: *Journal of Computer and System Sciences* 5.5 (1971), pp. 465–474. DOI: 10.1016/S0022-0000(71)80010-7 (cit. on p. 168).

[268] Jerzy W. Grzymala-Busse. "On the Set of All Automata with the Same Monoid of Endomorphisms". In: *Mathematical Foundations of Computer Science 1975, 4th Symposium, Mariánské Lázne, Czechoslovakia, September 1-5, 1975, Proceedings*. Ed. by Jirí Becvár. Vol. 32. Lecture Notes in Computer Science. Springer, 1975, pp. 246–251. DOI: 10.1007/3-540-07389-2_203 (cit. on p. 168).

[269] Sylvain Guillemot. "Parameterized Complexity and Approximability of the Longest Compatible Sequence Problem". In: *Discrete Optimization* 8.1 (2011), pp. 50–60. DOI: 10.1016/j.disopt.2010.08.003 (cit. on pp. 252, 253).

[270] Yuri Gurevich and Saharon Shelah. "Nearly Linear Time". In: *Logic at Botik '89, Symposium on Logical Foundations of Computer Science, Pereslav-Zalessky,*

USSR, July 3-8, 1989, Proceedings. Ed. by Albert R. Meyer and Michael A. Taitslin. Vol. 363. Lecture Notes in Computer Science. Springer, 1989, pp. 108–118. DOI: 10.1007/3-540-51237-3_10 (cit. on p. 85).

[271] Vladimir V. Gusev. "Synchronizing Automata of Bounded Rank". In: *Implementation and Application of Automata - 17th International Conference, CIAA 2012, Porto, Portugal, July 17-20, 2012. Proceedings*. Ed. by Nelma Moreira and Rogério Reis. Vol. 7381. Lecture Notes in Computer Science. Springer, 2012, pp. 171–179. DOI: 10.1007/978-3-642-31606-7_15 (cit. on pp. 15, 413, 414, 500).

[272] Vladimir V. Gusev and Elena V. Pribavkina. "On Non-complete Sets and Restivo's Conjecture". In: *Developments in Language Theory - 15th International Conference, DLT 2011, Milan, Italy, July 19-22, 2011. Proceedings*. Ed. by Giancarlo Mauri and Alberto Leporati. Vol. 6795. Lecture Notes in Computer Science. Springer, 2011, pp. 239–250. DOI: 10.1007/978-3-642-22321-1_21 (cit. on p. 452).

[273] Yahya O. Hamidoune. "Sur Les Atomes d'un Graphe Orienté". In: *Comptes Rendus de l'Académie des Sciences, Série A* 284 (1977), pp. 1253–1256 (cit. on p. 127).

[274] Yahya O. Hamidoune. "An Application of Connectivity Theory in Graphs to Factorizations of Elements in Groups". In: *European Journal of Combinatorics* 2.4 (1981), pp. 349–355. DOI: 10.1016/S0195-6698(81)80042-X (cit. on p. 127).

[275] Yahya O. Hamidoune. "Quelques Problèmes de Connexité dans les Graphes Orientés". In: *Journal of Combinatorial Theory, Series B* 30.1 (1981), pp. 1–10. DOI: 10.1016/0095-8956(81)90085-X (cit. on p. 127).

[276] Yo-Sub Han and Kai Salomaa. "Overlap-Free Languages and Solid Codes". In: *International Journal of Foundations of Computer Science* 22.5 (2011), pp. 1197–1209. DOI: 10.1142/S0129054111008647 (cit. on p. 452).

[277] Yo-Sub Han and Derick Wood. "Overlap-Free Regular Languages". In: *Computing and Combinatorics, 12th Annual International Conference, COCOON 2006, Taipei, Taiwan, August 15-18, 2006, Proceedings*. Ed. by Danny Z. Chen and D. T. Lee. Vol. 4112. Lecture Notes in Computer Science. Springer, 2006, pp. 469–478. DOI: 10.1007/11809678_49 (cit. on p. 452).

[278] Frank Harary. *Graph Theory*. Reading, Massachusetts: Addison-Wesley, 1969. ISBN: 978-0201001797 (cit. on pp. 84, 130).

[279] Juris Hartmanis. "Symbolic Analysis of a Decomposition of Information Processing Machines". In: *Information and Control* 3.2 (1960), pp. 154–178. DOI: 10.1016/S0019-9958(60)90744-0 (cit. on p. 169).

[280] Juris Hartmanis. "Computational Complexity of Random Access Stored Program Machines". In: *Mathematical Systems Theory* 5.3 (1971), pp. 232–245. DOI: 10.1007/BF01694180 (cit. on p. 85).

[281] Juris Hartmanis and Leonard Berman. "On Tape Bounds for Single Letter Alphabet Language Processing". In: *Theoretical Computer Science* 3.2 (1976), pp. 213–224. DOI: 10.1016/0304-3975(76)90024-4 (cit. on p. 240).

[282] Juris Hartmanis, Neil Immerman, and Stephen R. Mahaney. "One-Way Log-Tape Reductions". In: *19th Annual Symposium on Foundations of Computer*

Science, Ann Arbor, Michigan, USA, 16-18 October 1978. IEEE Computer Society, 1978, pp. 65–72. DOI: 10.1109/SFCS.1978.31 (cit. on pp. 73, 85).

[283] Juris Hartmanis and Stephen R. Mahaney. "An Essay About Research on Sparse NP Complete Sets". In: *Mathematical Foundations of Computer Science 1980 (MFCS'80), Proceedings of the 9th Symposium, Rydzyna, Poland, September 1-5, 1980*. Ed. by Piotr Dembinski. Vol. 88. Lecture Notes in Computer Science. Springer, 1980, pp. 40–57. DOI: 10.1007/BFb0022494 (cit. on p. 453).

[284] Juris Hartmanis and Richard E. Stearns. "On the Computational Complexity of Algorithms". In: *Transactions of the American Mathematical Society* 117 (1965), pp. 285–306. DOI: 10.2307/1994215 (cit. on pp. 85, 87).

[285] Juris Hartmanis and Richard E. Stearns. *Algebraic Structure Theory of Sequential Machines*. Prentice-Hall Series in Automatic Computation. Englewood Cliffs, NJ: Prentice-Hall, 1966 (cit. on p. 170).

[286] Yong He et al. "Extremal Synchronizing Circular Automata". In: *Information and Computation* 281 (2021), p. 104817. DOI: 10.1016/j.ic.2021.104817 (cit. on p. 399).

[287] Daniel Heath et al. "Symmetric and Alternating Groups Generated by a Full Cycle and Another Element". In: *The American Mathematical Monthly* 116.5 (2009), pp. 447–451. ISSN: 00029890, 19300972. DOI: 10.1080/00029890.2009.11920958 (cit. on pp. 113, 388).

[288] Emmanuel Hebrard et al. "Finding Diverse and Similar Solutions in Constraint Programming". In: *Proceedings, The Twentieth National Conference on Artificial Intelligence and the Seventeenth Innovative Applications of Artificial Intelligence Conference, July 9-13, 2005, Pittsburgh, Pennsylvania, USA*. Ed. by Manuela M. Veloso and Subbarao Kambhampati. AAAI Press / The MIT Press, 2005, pp. 372–377 (cit. on p. 252).

[289] William Hesse, Eric Allender, and David A. Mix Barrington. "Uniform Constant-Depth Threshold Circuits for Division and Iterated Multiplication". In: *Journal of Computer and System Sciences* 65.4 (2002), pp. 695–716. DOI: 10.1016/S0022-0000(02)00025-9 (cit. on p. 74).

[290] William Hesse, Eric Allender, and David A. Mix Barrington. "Corrigendum to "Uniform Constant-Depth Threshold Circuits for Division and Iterated Multiplication" [J. Comput. System Sci. 65(4) (2002) 695-716]". In: *Journal of Computer and System Sciences* 80.2 (2014), pp. 496–497. DOI: 10.1016/j.jcss.2013.09.002 (cit. on p. 74).

[291] Peter M. Higgins. *Techniques of Semigroup Theory*. Oxford science publications. Oxford University Press, 1992. ISBN: 9780198535775 (cit. on p. 84).

[292] Donald G. Higman. "Intersection Matrices for Finite Permutation Groups". In: *Journal of Algebra* 6.1 (1967), pp. 22–42. ISSN: 0021-8693. DOI: 10.1016/0021-8693(67)90011-7 (cit. on p. 358).

[293] Stefan Hoffmann. "Commutative Regular Languages - Properties and State Complexity". In: *Algebraic Informatics - 8th International Conference, CAI 2019, Niš, Serbia, June 30 - July 4, 2019, Proceedings*. Ed. by Miroslav Ciric, Manfred Droste, and Jean-Éric Pin. Vol. 11545. Lecture Notes in Computer Science. Springer, 2019, pp. 151–163. DOI: 10.1007/978-3-030-21363-3_13 (cit. on p. 223).

[294] Stefan Hoffmann. "Computational Complexity of Synchronization Under Regular Commutative Constraints". In: *Computing and Combinatorics - 26th International Conference, COCOON 2020, Atlanta, GA, USA, August 29-31, 2020, Proceedings*. Ed. by Donghyun Kim and R. N. Uma and Zhipeng Cai and Dong Hoon Lee. Vol. 12273. Lecture Notes in Computer Science. Springer, 2020, pp. 460–471. DOI: 10.1007/978-3-030-58150-3_37 (cit. on pp. 19–21).

[295] Stefan Hoffmann. "Ideal Separation and General Theorems for Constrained Synchronization and their Application to Small Constraint Automata". In: *CoRR* abs/2005.05907 (2020). arXiv: 2005.05907 (cit. on p. 501).

[296] Stefan Hoffmann. "On A Class of Constrained Synchronization Problems in NP". In: *Proceedings of the 21st Italian Conference on Theoretical Computer Science, Ischia, Italy, September 14-16, 2020*. Ed. by Gennaro Cordasco and Luisa Gargano and Adele A. Rescigno. Vol. 2756. CEUR Workshop Proceedings. available online: http://ceur-ws.org/Vol-2756/paper_19.pdf. CEUR-WS.org, 2020, pp. 145–157 (cit. on pp. 19, 20, 494, 505, 522).

[297] Stefan Hoffmann. "Completely Reachable Automata, Primitive Groups and the State Complexity of the Set of Synchronizing Words". In: *Language and Automata Theory and Applications - 15th International Conference, LATA 2021, Milan, Italy, March 1-5, 2021, Proceedings*. Ed. by Alberto Leporati et al. Vol. 12638. Lecture Notes in Computer Science. Springer, 2021, pp. 305–317. DOI: 10.1007/978-3-030-68195-1_24 (cit. on pp. 18, 19, 21, 22, 292).

[298] Stefan Hoffmann. "Computational Complexity of Synchronization Under Sparse Regular Constraints". In: *Fundamentals of Computation Theory - 23rd International Symposium, FCT 2021, Athens, Greece, September 12-15, 2021, Proceedings*. Ed. by Evripidis Bampis and Aris Pagourtzis. Vol. 12867. Lecture Notes in Computer Science. Springer, 2021, pp. 272–286. DOI: 10.1007/978-3-030-86593-1_19. arXiv: 2108.00081 (cit. on pp. 19–21, 426, 440, 500, 502, 522).

[299] Stefan Hoffmann. "Constrained Synchronization and Commutativity". In: *Theoretical Computer Science* 890 (2021), pp. 147–170. DOI: 10.1016/j.tcs.2021.08.030 (cit. on pp. 18–21, 232, 233, 235, 326, 501, 524, 547).

[300] Stefan Hoffmann. "Constrained Synchronization and Subset Synchronization Problems for Weakly Acyclic Automata". In: *Developments in Language Theory - 25th International Conference, DLT 2021, Porto, Portugal, August 16-20, 2021, Proceedings*. Ed. by Nelma Moreira and Rogério Reis. Vol. 12811. Lecture Notes in Computer Science. Springer, 2021, pp. 204–216. DOI: 10.1007/978-3-030-81508-0_17 (cit. on pp. 21, 23, 325, 547).

[301] Stefan Hoffmann. "Ideal Separation and General Theorems for Constrained Synchronization and Their Application to Small Constraint Automata". In: *Computing and Combinatorics - 27th International Conference, COCOON 2021, Tainan, Taiwan, October 24-26, 2021, Proceedings*. Ed. by Chi-Yeh Chen et al. Vol. 13025. Lecture Notes in Computer Science. Springer, 2021, pp. 176–188. DOI: 10.1007/978-3-030-89543-3_15 (cit. on pp. 19–22, 500, 501, 523–525).

[302] Stefan Hoffmann. "Regularity Conditions for Iterated Shuffle on Commutative Regular Languages". In: *Implementation and Application of Automata - 25th*

International Conference, CIAA 2021, Virtual Event, July 19-22, 2021, Proceedings. Ed. by Sebastian Maneth. Vol. 12803. Lecture Notes in Computer Science. Springer, 2021, pp. 27–38. DOI: 10.1007/978-3-030-79121-6_3 (cit. on p. 223).

[303] Stefan Hoffmann. "State Complexity of the Set of Synchronizing Words for Circular Automata and Automata over Binary Alphabets". In: *Language and Automata Theory and Applications - 15th International Conference, LATA 2021, Milan, Italy, March 1-5, 2021, Proceedings*. Ed. by Alberto Leporati et al. Vol. 12638. Lecture Notes in Computer Science. Springer, 2021, pp. 318–330. DOI: 10.1007/978-3-030-68195-1_25 (cit. on pp. 18, 19, 21, 22, 291, 292, 316).

[304] Stefan Hoffmann. "Sync-Maximal Permutation Groups Equal Primitive Permutation Groups". In: *Descriptional Complexity of Formal Systems - 23rd IFIP WG 1.02 International Conference, DCFS 2021, Virtual Event, September 5, 2021, Proceedings*. Ed. by Yo-Sub Han and Sang-Ki Ko. Vol. 13037. Lecture Notes in Computer Science. Springer, 2021, pp. 38–50. DOI: 10.1007/978-3-030-93489-7_4 (cit. on pp. 18, 19, 21, 292).

[305] Stefan Hoffmann. "Constrained Synchronization for Monotonic and Solvable Automata and Automata with Simple Idempotents". In: *Implementation and Application of Automata - 26th International Conference, CIAA 2022, Rouen, France, June 28 - July 1, 2022, Proceedings*. Ed. by Pascal Caron and Ludovic Mignot. Vol. 13266. Lecture Notes in Computer Science. Springer, 2022, pp. 225–237. DOI: 10.1007/978-3-031-07469-1_18 (cit. on pp. 19, 20, 22, 23, 547).

[306] Stefan Hoffmann. "Reset Complexity and Completely Reachable Automata with Simple Idempotents". In: *Descriptional Complexity of Formal Systems - 24rd IFIP WG 1.02 International Conference, DCFS 2022, August 29 - 31, 2022, Debrecen, Hungary, Proceedings*. Ed. by Yo-Sub Han and György Vaszil. Vol. 13439. Lecture Notes in Computer Science. Springer, 2022, pp. 85–99. DOI: 10.1007/978-3-031-13257-5_7 (cit. on pp. 18, 19, 22, 23, 292, 347).

[307] Stefan Hoffmann. "Binary and Circular Automata Having Maximal State Complexity for the Set of Synchronizing Words". In: *Information and Computation* 295 (2023). Special Issue: Selected papers of the 15th International Conference on Language and Automata Theory and Applications, LATA 2021, p. 105076. ISSN: 0890-5401. DOI: 10.1016/j.ic.2023.105076 (cit. on pp. 18, 19, 22, 291, 329, 339, 406).

[308] Stefan Hoffmann. "Completely Distinguishable Automata and the Set of Synchronizing Words". In: *Developments in Language Theory - 27th International Conference, DLT 2023, Umeå, Sweden, June 12-16, 2023, Proceedings*. Ed. by Frank Drewes and Mikhail Volkov. Vol. 13911. Lecture Notes in Computer Science. Springer, 2023, pp. 128–142. DOI: 10.1007/978-3-031-33264-7_11 (cit. on pp. 18, 19, 22, 291, 292, 355).

[309] Stefan Hoffmann. "New Characterizations of Primitive Permutation Groups with Applications to Synchronizing Automata". In: *Information and Computation* 295 (2023). Special Issue: Selected papers of the 15th International Conference on Language and Automata Theory and Applications, LATA 2021,

p. 105086. ISSN: 0890-5401. DOI: 10.1016/j.ic.2023.105086 (cit. on pp. 18, 19, 22, 292, 361–363, 366, 377).

[310] Stefan Hoffmann. "Regularity Conditions for Iterated Shuffle on Commutative Regular Languages". In: *International Journal of Foundations of Computer Science* 34.08 (2023). Special Issue: International Conference on Implementation and Application of Automata (CIAA 2021), pp. 923–957. DOI: 10.1142/S0129054123430037 (cit. on p. 223).

[311] Stefan Hoffmann. "State Complexity of Permutation and the Language Inclusion Problem up to Parikh Equivalence on Alphabetical Pattern Constraints and Partially Ordered NFAs". In: *International Journal of Foundations of Computer Science* 34.8 (2023), pp. 959–986. DOI: 10.1142/S0129054123430025 (cit. on p. 222).

[312] Stefan Hoffmann. "State Complexity Bounds for Projection, Shuffle, Up- and Downward Closure and Interior on Commutative Regular Languages". In: *Information and Computation* 301 (2024), p. 105204. DOI: 10.1016/J.IC.2024.105204 (cit. on pp. 222, 223, 327).

[313] Stefan Hoffmann. "Subset Mapping Problems in Solvable Automata". In: *Journal of Automata, Languages and Combinatorics* 29.2-4 (2024), pp. 199–236. DOI: 10.25596/JALC-2024-199 (cit. on pp. 17, 22, 232).

[314] Stefan Hoffmann. "Synchronization of Parikh Automata". In: *Theory of Computing Systems* (2024). Special Issue DLT & WORDS 2023 (accepted, to appear) (cit. on p. 84).

[315] Stefan Hoffmann. "Completely Distinguishable Automata and the Set of Synchronizing Words". In: *Theory of Computing Systems* 70.20 (2 2026). DOI: 10.1007/s00224-024-10204-9 (cit. on pp. 18, 19, 22, 292).

[316] Stefan Hoffmann. "General Theorems for Constrained Synchronization and their Application to Small Constraint Automata". In: *Journal of Combinatorial Optimization* (to appear) (cit. on pp. 19, 20, 22, 463, 500, 501, 524).

[317] W.M.L. Holcombe. *Algebraic Automata Theory*. Cambridge Studies in Advanced Mathematics. Vol. 1. Cambridge University Press, 1982. ISBN: 978-0-521-60492-5 (cit. on p. 170).

[318] Christopher Hollings. "The Early Development of the Algebraic Theory of Semigroups". In: *Archive for History of Exact Sciences* 63.5 (2009), pp. 497–536. DOI: 10.1007/s00407-009-0044-3 (cit. on p. 129).

[319] Derek F. Holt, Bettina Eick, and Eamonn A. O'Brien. *Handbook of Computational Group Theory*. Discrete Mathematics and Its Applications. Chapman and Hall/CRC, 2005. ISBN: 978-1-58488-372-6 (cit. on p. 130).

[320] Markus Holzer and Sebastian Jakobi. "On the Computational Complexity of Problems Related to Distinguishability Sets". In: *Information and Computation* 259.2 (2018), pp. 225–236. DOI: 10.1016/j.ic.2017.09.003 (cit. on pp. 167, 232, 233).

[321] Markus Holzer and Martin Kutrib. "Descriptional and Computational Complexity of Finite Automata - A Survey". In: *Information and Computation* 209.3 (2011), pp. 456–470. DOI: 10.1016/j.ic.2010.11.013 (cit. on pp. 69, 252, 390).

[322] John E. Hopcroft and Jeff D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley Publishing Company, 1979 (cit. on pp. 57, 65, 69, 70, 72, 73, 84–87, 154, 159, 167, 169, 243, 451, 522).

[323] James M. Howie. "The Subsemigroup Generated by the Idempotents of a Full Transformation Semigroup". In: *Journal of the London Mathematical Society* s1-41.1 (1966), pp. 707–716. DOI: 10.1112/jlms/s1-41.1.707 (cit. on p. 110).

[324] James M. Howie. "Products of Idempotents in Certain Semigroups of Transformations". In: *Proceedings of the Edinburgh Mathematical Society* 17.3 (1971), pp. 223–236. DOI: 10.1017/S0013091500026936 (cit. on p. 223).

[325] John M. Howie. *Fundamentals of Semigroup Theory*. LMS monographs. Clarendon Press, 1995. ISBN: 9780198511946 (cit. on pp. 45, 63, 84, 314, 510).

[326] Oscar H. Ibarra. "Reversal-Bounded Multicounter Machines and Their Decision Problems". In: *Journal of the ACM* 25.1 (1978), pp. 116–133. DOI: 10.1145/322047.322058 (cit. on p. 84).

[327] Oscar H. Ibarra and Bala Ravikumar. "On Sparseness, Ambiguity and Other Decision Problems for Acceptors and Transducers". In: *STACS 86, 3rd Annual Symposium on Theoretical Aspects of Computer Science, Orsay, France, January 16-18, 1986, Proceedings*. Ed. by Burkhard Monien and Guy Vidal-Naquet. Vol. 210. Lecture Notes in Computer Science. Springer, 1986, pp. 171–179. DOI: 10.1007/3-540-16078-7_74 (cit. on p. 453).

[328] Oscar H. Ibarra and Bala Ravikumar. "On Bounded Languages and Reversal-Bounded Automata". In: *Information and Computation* 246 (2016), pp. 30–42. DOI: 10.1016/J.IC.2015.11.007 (cit. on p. 454).

[329] Karel Culík II, Juhani Karhumäki, and Jarkko Kari. "A Note on Synchronized Automata and Road Coloring Problem". In: *International Journal of Foundations of Computer Science* 13.3 (2002), pp. 459–471. DOI: 10.1142/S0129054102001217 (cit. on p. 183).

[330] Lucian Ilie. "On a Conjecture about Slender Context-Free Languages". In: *Theoretical Computer Science* 132.2 (1994), pp. 427–434. DOI: 10.1016/0304-3975(94)00042-5 (cit. on p. 453).

[331] Neil Immerman. "First Order Expressibility as a New Complexity Measure". Cornell University, Aug. 1980 (cit. on p. 85).

[332] Neil Immerman. "Languages that Capture Complexity Classes". In: *SIAM Journal on Computing* 16.4 (1987), pp. 760–778. DOI: 10.1137/0216051 (cit. on pp. 86, 88).

[333] Neil Immerman. *Descriptive Complexity*. Graduate Texts in Computer Science. Springer, 1999. ISBN: 978-1-4612-6809-3. DOI: 10.1007/978-1-4612-0539-5 (cit. on pp. 79, 80, 85, 88, 89).

[334] Balázs Imreh. "On Isomorphic Representations of Commutative Automata with Respect to α_1-products". In: *Acta Cybernetica* 5.1 (1980), pp. 21–32. eprint: https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/3202 (cit. on p. 222).

[335] Roberto Incitti. "The Growth Function of Context-Free Languages". In: *Theoretical Computer Science* 255.1-2 (2001), pp. 601–605. DOI: 10.1016/S0304-3975(00)00152-3 (cit. on p. 453).

[336] Linnea Ingmar et al. "Modelling Diversity of Solutions". In: *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 2020, pp. 1528–1535. DOI: 10.1609/AAAI.V34I02.5512 (cit. on p. 252).

[337] Irving M. Isaacs. *Finite Group Theory*. Graduate Studies in Mathematics 92. American Mathematical Society, 2008. ISBN: 9780821843444 (cit. on p. 359).

[338] Irving M. Isaacs and Thilo Zieschang. "Generating Symmetric Groups". In: *The American Mathematical Monthly* 102.8 (1995), pp. 734–739. DOI: 10.1080/00029890.1995.12004650 (cit. on pp. 125, 126, 370).

[339] Masami Ito. *Algebraic Theory of Automata and Languages*. 1st ed. Singapore: World Scientific, 2004, p. 212. DOI: 10.1142/4791 (cit. on pp. 135, 167).

[340] Masami Ito. "Algebraic Structures of Automata". In: *Theoretical Computer Science* 429 (2012), pp. 164–168. DOI: 10.1016/J.TCS.2011.12.035 (cit. on p. 172).

[341] Joseph F. JáJá. *An Introduction to Parallel Algorithms*. Reading, MA: Addison-Wesley Pub. Co., 1992. ISBN: 0-201-54856-9 (cit. on p. 89).

[342] Andrzej Kisielewicz and Jakub Kowalski and Marek Szykuła. "Computing the Shortest Reset Words of Synchronizing Automata". In: *Journal of Combinatorial Optimization* 29.1 (2015), pp. 88–124. DOI: 10.1007/s10878-013-9682-0 (cit. on p. 254).

[343] Mark Jerrum. "The Complexity of Finding Minimum-Length Generator Sequences". In: *Theoretical Computer Science* 36 (1985), pp. 265–289. DOI: 10.1016/0304-3975(85)90047-7 (cit. on p. 256).

[344] Galina Jirásková and Tomás Masopust. "On the State and Computational Complexity of the Reverse of Acyclic Minimal DFAs". In: *Implementation and Application of Automata - 17th International Conference, CIAA 2012, Porto, Portugal, July 17-20, 2012. Proceedings*. Ed. by Nelma Moreira and Rogério Reis. Vol. 7381. Lecture Notes in Computer Science. Springer, 2012, pp. 229–239. DOI: 10.1007/978-3-642-31606-7_20 (cit. on p. 223).

[345] Neil D. Jones. "Space-Bounded Reducibility Among Combinatorial Problems". In: *Journal of Computer and System Sciences* 11.1 (1975), pp. 68–85. DOI: 10.1016/S0022-0000(75)80050-X (cit. on pp. 70, 73, 85, 86, 350, 352).

[346] Camille Jordan. *Traité des Substitutions et des Équations Algébriques*. available online: https://gallica.bnf.fr/ark:/12148/bpt6k290534. Paris: Gauthier-Villars, 1870 (cit. on pp. 129, 359, 395, 396).

[347] Stasys Jukna. *Boolean Function Complexity - Advances and Frontiers*. Vol. 27. Algorithms and combinatorics. Springer, 2012. ISBN: 978-3-642-24507-7. DOI: 10.1007/978-3-642-24508-4 (cit. on p. 89).

[348] Helmut Jürgensen. "Synchronization". In: *Information and Computation* 206.9-10 (2008), pp. 1033–1044. DOI: 10.1016/j.ic.2008.03.005 (cit. on pp. 3, 5, 427, 438, 452).

[349] Helmut Jürgensen and Stavros Konstantinidis. “Codes”. In: *Handbook of Formal Languages, Volume 1: Word, Language, Grammar*. Ed. by Grzegorz Rozenberg and Arto Salomaa. Springer, 1997, pp. 511–607. DOI: 10.1007/978-3-642-59136-5_8 (cit. on pp. 426, 438, 452).

[350] Helmut Jürgensen and Ludwig Staiger. “Automata for Solid Codes”. In: *Theoretical Computer Science* 892 (2021), pp. 25–47. DOI: 10.1016/j.tcs.2021.09.007 (cit. on pp. 420, 424, 426, 427, 437–439, 452).

[351] Helmut Jürgensen and S. S. Yu. “Solid Codes”. In: *Elektronische Informationsverarbeitung und Kybernetik/Journal of Information Processing and Cybernetics (now Journal of Automata, Languages and Combinatorics)* 26.10 (1990), pp. 563–574 (cit. on pp. 438, 439).

[352] Lev Kaloujnine and Marc Krasner. “Produit Complet des Groupes de Permutations et le Problème d’Extension de Groupes : II”. In: *Acta Scientiarum Mathematicarum (Szeged)* 14 (1952), pp. 39–66. eprint: http://acta.bibl.u-szeged.hu/id/eprint/13687 (cit. on p. 170).

[353] Mark Kambites, Pedro V. Silva, and Benjamin Steinberg. “On the Rational Subset Problem for Groups”. In: *Journal of Algebra* 309.2 (2007). Computational Algebra, pp. 622–639. ISSN: 0021-8693. DOI: 10.1016/j.jalgebra.2006.05.020 (cit. on p. 256).

[354] Ravindran Kannan and Richard J. Lipton. “Polynomial-Time Algorithm for the Orbit Problem”. In: *Journal of the ACM* 33.4 (Aug. 1986), pp. 808–821. DOI: 10.1145/6490.6496 (cit. on pp. 413–415).

[355] Sertaç Karahoda, Kamer Kaya, and Hüsnü Yenigün. “Synchronizing Heuristics: Speeding up the Fastest”. In: *Expert Systems with Applications* 94 (2018), pp. 265–275. DOI: 10.1016/J.ESWA.2017.10.054 (cit. on p. 254).

[356] George Karakostas, Richard J. Lipton, and Anastasios Viglas. “On the Complexity of Intersecting Finite State Automata and NL versus NP”. In: *Theoretical Computer Science* 302.1-3 (2003), pp. 257–274. DOI: 10.1016/S0304-3975(02)00830-7 (cit. on pp. 254, 256).

[357] Jarkko Kari. “A Counter Example to a Conjecture Concerning Synchronizing Words in Finite Automata”. In: *EATCS Bulletin* 73 (2001), p. 146 (cit. on p. 398).

[358] Jarkko Kari. “Synchronization and Stability of Finite Automata”. In: *Journal of Universal Computer Science* 8.2 (2002), pp. 270–277. DOI: 10.3217/jucs-008-02-0270 (cit. on p. 183).

[359] Jarkko Kari. “Synchronizing Finite Automata on Eulerian Digraphs”. In: *Theoretical Computer Science* 295 (2003), pp. 223–232. DOI: 10.1016/S0304-3975(02)00405-X (cit. on p. 14).

[360] Jarkko Kari and Mikhail V. Volkov. “Černý’s Conjecture and the Road Colouring Problem”. In: *Handbook of Automata Theory, Volume I*. Ed. by Jean-Éric Pin. European Mathematical Society Publishing House, 2021, pp. 525–565. DOI: 10.4171/automata (cit. on pp. 3, 5, 12, 185, 381, 452).

[361] Richard M. Karp. “Reducibility Among Combinatorial Problems”. In: *Complexity of Computer Computations: Proceedings of a symposium on the Complexity of Computer Computations, held March 20–22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, and sponsored by the Office of Naval Research, Mathematics Program, IBM World Trade Cor-*

poration, and the IBM Research Mathematical Sciences Department. Ed. by Raymond E. Miller, James W. Thatcher, and Jean D. Bohlinger. Boston, MA: Springer US, 1972, pp. 85–103. ISBN: 978-1-4684-2001-2. DOI: 10.1007/978-1-4684-2001-2_9 (cit. on pp. 70, 71, 73, 85).

[362] Takumi Kasai. *Computational Complexity of Multitape Turing Machines and Random Access Machines*. Tech. rep. 13. Kyoto University Research Information Repository (KURENAI), 1977, pp. 469–496. eprint: https://www.jstage.jst.go.jp/article/kyotoms1969/13/2/13_2_469/_pdf (cit. on p. 85).

[363] Takumi Kasai and Shigeki Iwata. "Gradually Intractable Problems and Nondeterministic Log-Space Lower Bounds". In: *Mathematical Systems Theory* 18.2 (1985), pp. 153–170. DOI: 10.1007/BF01699467 (cit. on p. 254).

[364] Jyrki Katajainen, Jan van Leeuwen, and Martti Penttonen. "Fast Simulation of Turing Machines by Random Access Machines". In: *SIAM Journal on Computing* 17.1 (1988), pp. 77–88. DOI: 10.1137/0217005 (cit. on p. 85).

[365] Thomas P. Kezlan and Noah H. Rhee. "A Characterization of the Centralizer of a Permutation". In: *Missouri Journal of Mathematical Sciences* 11.3 (1999), pp. 158–163. DOI: 10.35834/1999/1103158 (cit. on p. 171).

[366] D.J. Kfoury. "Synchronizing Sequences for Probabilistic Automata". In: *Studies in Applied Mathematics* 49.1 (1970), pp. 101–103. DOI: 10.1002/sapm1970491101 (cit. on p. 13).

[367] Mati Kilp, Ulrich Knauer, and Alexander V. Mikhalev. *Monoids, Acts and Categories: With Applications to Wreath Products and Graphs*. Vol. 29. De Gruyter Expositions in Mathematics. Reprint 2011 edition. Berlin / Boston: Walter de Gruyter, 2000. ISBN: 978-3-11-015248-7. DOI: 10.1515/9783110812909 (cit. on pp. 129, 168).

[368] Andrzej Kisielewicz and Marek Szykuła. "Synchronizing Automata with Extremal Properties". In: *Mathematical Foundations of Computer Science 2015 - 40th International Symposium, MFCS 2015, Milan, Italy, August 24-28, 2015, Proceedings, Part I*. Ed. by Giuseppe F. Italiano, Giovanni Pighizzini, and Donald Sannella. Vol. 9234. Lecture Notes in Computer Science. Springer, 2015, pp. 331–343. DOI: 10.1007/978-3-662-48057-1_26 (cit. on pp. 397, 403, 404).

[369] Stephen C. Kleene. "Representation of Events in Nerve Nets and Finite Automata". In: *Automata Studies*. Ed. by Claude Shannon and John McCarthy. Vol. 34. Annals of Mathematics Studies. Princeton, NJ: Princeton University Press, 1956, pp. 3–41. DOI: 10.1515/9781400882618-002 (cit. on pp. 159, 170).

[370] Israel Kleiner. *A History of Abstract Algebra*. First edition. Boston, MA: Birkhäuser Boston, 2007, p. 168. ISBN: 978-0-8176-4684-4. DOI: 10.1007/978-0-8176-4685-1 (cit. on p. 129).

[371] Ondřej Klíma and Libor Polák. "Alternative Automata Characterization of Piecewise Testable Languages". In: *Developments in Language Theory - 17th International Conference, DLT 2013, Marne-la-Vallée, France, June 18-21, 2013. Proceedings*. Ed. by Marie-Pierre Béal and Olivier Carton. Vol. 7907. Lecture Notes in Computer Science. Springer, 2013, pp. 289–300. DOI: 10.1007/978-3-642-38771-5_26 (cit. on pp. 256, 257).

[372] Alexander A. Klyachko, Igor K. Rystsov, and M.A. Spivak. "An Extremal Combinatorial Problem Associated with the Bound on the Length of a Synchronizing Word in an Automaton". In: *Kibernetika* 2 (1987). in Russian; Engl. translation: Cybernetics and Systems Analysis **23** (1987) 165–171, pp. 16–20. DOI: 10.1007/BF01071771 (cit. on p. 13).

[373] Reinhard Knörr. "A Note on Permutation Groups". In: *Communications in Algebra* 35.2 (2007), pp. 613–616. DOI: 10.1080/00927870601074848. eprint: https://doi.org/10.1080/00927870601074848 (cit. on p. 126).

[374] Donald E. Knuth. "Big Omicron and Big Omega and Big Theta". In: *ACM SIGACT News* 8.2 (1976). available online: http://www.phil.uu.nl/datastructuren/07-08/knuth_big_omicron.pdf, pp. 18–24. DOI: 10.1145/1008328.1008329 (cit. on p. 64).

[375] Zvi Kohavi and Niraj K. Jha. *Switching and Finite Automata Theory*. 3rd ed. Cambridge University Press, 2009, pp. 414–416. DOI: 10.1017/CBO9780511816239 (cit. on pp. 3, 13, 76).

[376] Zvi Kohavi and Joseph Winograd. "Bounds on the Length of Synchronizing Sequences and the Order of Information Losslessness". In: *Theory of Machines and Computations*. Ed. by Zvi Kohavi and Azaria Paz. Academic Press, 1971, pp. 197–206. ISBN: 978-0-12-417750-5. DOI: 10.1016/B978-0-12-417750-5.50023-3 (cit. on p. 13).

[377] Zvi Kohavi and Joseph Winograd. "Establishing Certain Bounds Concerning Finite Automata". In: *Journal of Computer and System Sciences* 7.3 (1973), pp. 288–299. DOI: 10.1016/S0022-0000(73)80010-8 (cit. on p. 13).

[378] Vassili N. Kolokoltsov and Victor P. Maslov. *Idempotent Analysis and Its Applications*. Vol. 401. Mathematics and Its Applications. Springer Netherlands, 1997. ISBN: 978-0792345091. DOI: 10.1007/978-94-015-8901-7 (cit. on p. 84).

[379] Konrad Königsberger. *Analysis 1*. Springer-Lehrbuch. Springer Berlin Heidelberg, 2003. ISBN: 9783540403715 (cit. on p. 82).

[380] Stavros Konstantinidis. "Every Regular Bifix Code is a Finite Union of Regular Infix Codes". In: *Journal of Automata, Languages and Combinatorics* 25.1 (2020), pp. 55–59. DOI: 10.25596/JALC-2020-055 (cit. on p. 453).

[381] Stavros Konstantinidis and Joshua Young. "f-Words and Binary Solid Codes". In: *Journal of Automata, Languages and Combinatorics* 15.3/4 (2010), pp. 269–283. DOI: 10.25596/jalc-2010-269 (cit. on p. 452).

[382] Eryk Kopczyński. "Half-Positional Determinacy of Infinite Games". In: *Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10-14, 2006, Proceedings, Part II*. Ed. by Michele Bugliesi, Bart Preneel, and Vladimiro Sassone and Ingo Wegener. Vol. 4052. Lecture Notes in Computer Science. Springer, 2006, pp. 336–347. DOI: 10.1007/11787006_29 (cit. on p. 223).

[383] Eryk Kopzyński. "Half-positional Determinay of Infinite Gamess". available online: https://www.mimuw.edu.pl/~erykk/papers/hpwc.pdf. PhD thesis. Warsaw University, 2008 (cit. on p. 223).

[384] L. G. Kovács. "Maximal Subgroups in Composite Finite Groups". In: *Journal of Algebra* 99.1 (1986), pp. 114–131. ISSN: 0021-8693. DOI: 10.1016/0021-8693(86)90058-X (cit. on p. 396).

[385] L. G. Kovács. "Wreath Decompositions of Finite Permutation Groups". In: *Bulletin of the Australian Mathematical Society* 40.2 (1989), pp. 255–279. DOI: 10.1017/S0004972700004366 (cit. on p. 395).

[386] Jakub Kowalski and Adam Roman. "A New Evolutionary Algorithm for Synchronization". In: *Applications of Evolutionary Computation - 20th European Conference, EvoApplications 2017, Amsterdam, The Netherlands, April 19-21, 2017, Proceedings, Part I*. Ed. by Giovanni Squillero and Kevin Sim. Vol. 10199. Lecture Notes in Computer Science. 2017, pp. 620–635. DOI: 10.1007/978-3-319-55849-3_40 (cit. on p. 254).

[387] Dexter Kozen. "Lower Bounds for Natural Proof Systems". In: *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*. IEEE Computer Society, 1977, pp. 254–266. DOI: 10.1109/SFCS.1977.16 (cit. on pp. 226, 231, 255).

[388] Dexter Kozen. *Automata and Computability*. Undergraduate Texts in Computer Science. Springer, 1997. ISBN: 978-0-387-94907-9 (cit. on pp. 84, 85, 167, 451).

[389] Kenneth Krohn and John Rhodes. "Algebraic Theory of Machines. I. Prime Decomposition Theorem for Finite Semigroups and Machines". In: *Transactions of the American Mathematical Society* 116 (1965), pp. 450–464. DOI: 10.1090/S0002-9947-1965-0188316-1 (cit. on pp. 170, 222).

[390] Rafał Kudłacik, Adam Roman, and Hubert Wagner. "Effective Synchronizing Algorithms". In: *Expert Systems with Applications* 39.14 (2012), pp. 11746–11757. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2012.04.079 (cit. on p. 254).

[391] Werner Kuich and Arto Salomaa. *Semirings, Automata, Languages*. Vol. 5. Monographs in Theoretical Computer Science. An EATCS Series. Berlin / Heidelberg: Springer-Verlag, 1986, pp. v + 374. ISBN: 3-540-13716-5. DOI: 10.1007/978-3-642-69959-7 (cit. on p. 84).

[392] Michał Kukiela. "Reversible and Bijectively Related Posets". In: *Order* 26 (2009), pp. 119–124. DOI: 10.1007/s11083-009-9111-2 (cit. on p. 84).

[393] Hans Kurzweil and Bernd Stellmacher. *The Theory of Finite Groups: An Introduction*. Springer Science & Business Media, 2003. ISBN: 9780387405100. DOI: 10.1007/b97433 (cit. on pp. 41, 82, 83, 112, 117, 149, 395).

[394] Richard E. Ladner, Richard J. Lipton, and Larry J. Stockmeyer. *Alternating Pushdown and Stack Automata*. 1984. DOI: 10.1137/0213010 (cit. on p. 86).

[395] Arthur E. Laemmel. *A General Class of Discrete Codes and Certain of Their Properties*. Tech. rep. R-459-55. PIB-389, Microwave Research Inst., Polytechnic Inst., Brooklyn, NY, 1956 (cit. on p. 452).

[396] Arthur E. Laemmel. *Study on Application of Coding Theory*. Tech. rep. PIBMRI-895.5-63. Dept. Electrophysics, Microwave Research Inst., Polytechnic Inst., Brooklyn, NY 1963A, 1963 (cit. on pp. 5, 452).

[397] Arthur E. Laemmel and B. Rudner. *Study of the Application of Coding Theory*. Tech. rep. PIBEP-69-034. Dept. Electrophysics, Polytechnic Inst., Brooklyn, Farmingdale, NY, 1969 (cit. on p. 452).

[398] Joseph-Louis Lagrange. “Réflexions sur la Résolution Algébrique des Équations”. In: *Oeuvres de Lagrange*. Ed. by Joseph-Alfred Serret. Vol. 3. (Originally published in 1770 (printed 1772) on pages 134–215 and 1771 (printed 1773) on pages 138–253 in two parts in *Nouveaux Mémoires de l’Académie Royale des Sciences et Belles-Lettres de Berlin*). Paris Place: Gauthier-Villars, 1869, pp. 205–421. eprint: http://sites.mathdoc.fr/cgi-bin/oeitem?id=OE_LAGRANGE__3_205_0 (cit. on p. 395).

[399] R. A. Laing and J.B. Wright. *Commutative Machines : Technical Note*. University of Michigan, 1962. eprint: https://hdl.handle.net/2027.42/6130 (cit. on pp. 84, 222).

[400] Gerard Lallement. “On the Prime Decomposition Theorem for Finite Monoids”. In: *Mathematical Systems Theory* 5.1 (1971), pp. 8–12. DOI: 10.1007/BF01691462 (cit. on p. 170).

[401] Gerard Lallement. *Semigroups and Combinatorial Applications*. Pure and Applied Mathematics. John Wiley and Sons Inc, 1979 (cit. on p. 84).

[402] Tsit-Yuen Lam. *A First Course in Noncommutative Rings*. 2nd. Vol. 131. Graduate Texts in Mathematics. New York, NY: Springer-Verlag, 2001. ISBN: 978-0-387-95325-0 (cit. on p. 84).

[403] Edmund Landau. “Über die Maximalordnung der Permutationen gegebenen Grades [On the Maximal Order of Permutations of Given Degree]”. In: *Archiv der Mathematik und Physik* 5.3 (1903), pp. 92–103 (cit. on p. 156).

[404] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. 2 volumes. Reprinted by Chelsea, New York, 1953. Leipzig: Teubner, 1909 (cit. on p. 28).

[405] Serge Lang. *Algebra*. New York, NY: Springer, 2002. ISBN: 978-1-4613-0041-0 (cit. on pp. 82, 84, 158, 171).

[406] Klaus-Jörn Lange and Peter Rossmanith. “The Emptiness Problem for Intersections of Regular Languages”. In: *Mathematical Foundations of Computer Science 1992, 17th International Symposium, MFCS’92, Prague, Czechoslovakia, August 24-28, 1992, Proceedings*. Ed. by Ivan M. Havel and Václav Koubek. Vol. 629. Lecture Notes in Computer Science. Springer, 1992, pp. 346–354. DOI: 10.1007/3-540-55808-X_33 (cit. on pp. 254, 256).

[407] Michel Latteux and Gabriel Thierrin. “On Bounded Context-Free Languages”. In: *Elektronische Informationsverarbeitung und Kybernetik/Journal of Information Processing and Cybernetics (now Journal of Automata, Languages and Combinatorics)* 20.1 (1984), pp. 3–8 (cit. on p. 453).

[408] Alexaner A. Letichevskii. “The Minimization of Finite Automata”. In: *Cybernetics and Systems Analysis* 1.1 (1965), pp. 17–19. DOI: 10.1007/BF01071438 (cit. on p. 172).

[409] Vladimir Levenshtein. “Decoding Automata, Invariant with Respect to the Initial State”. In: *Problemy Kibernetiki* 12.12 (1964). (in Russian), pp. 125–136 (cit. on pp. 419, 426, 438, 439).

[410] Vladimir I. Levenshtein. “Maximum Number of Words in Codes without Overlaps”. In: *Problems of Information Transmission* 6.4 (1970), pp. 355–357 (cit. on p. 426).

[411] Ming Li and Paul M. B. Vitányi. "Kolmogorov Complexity and Its Applications". In: *Handbook of Theoretical Computer Science, Volume A: Algorithms and Complexity*. Ed. by Jan van Leeuwen. Survey on Kolmogorov complexity theory. Amsterdam and Cambridge, MA: Elsevier and MIT Press, 1990, pp. 187–254. ISBN: 0-444-88074-6. DOI: 10.1016/B978-0-444-88071-0.50009-6 (cit. on p. 79).

[412] Ming Li and Paul M. B. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. 4th ed. Texts in Computer Science. Comprehensive textbook on Kolmogorov complexity and its applications. Cham, Switzerland: Springer Cham, 2019. ISBN: 978-3-030-11297-4. DOI: 10.1007/978-3-030-11298-1 (cit. on p. 79).

[413] Martin W. Liebeck, Cheryl E. Praeger, and Jan Saxl. "On the O'Nan-Scott Theorem for Finite Primitive Permutation Groups". In: *Journal of the Australian Mathematical Society* 44.3 (1988), pp. 389–396. DOI: 10.1017/S144678870003216X (cit. on p. 396).

[414] Stephen Leon Lipscomb. "The Structure of the Centralizer of a Permutation". In: *Semigroup Forum* 37.3 (1988), pp. 301–312. DOI: 10.1007/BF02573142 (cit. on p. 171).

[415] Bruce E. Litow. "Parallel Complexity of Integer Coprimality". In: *Electronic Colloquium on Computational Complexity* TR98-009 (1998). eprint: https://eccc.weizmann.ac.il/eccc-reports/1998/TR98-009/index.html (cit. on p. 74).

[416] Chung Laung Liu. *Some Memory Aspects of Finite Automata*. Tech. rep. 411. Research Lab. Electronics, Massachusetts Inst., MIT Res. Lab. Electronics, Cambridge, MA, May 1963, pp. 1–71. eprint: http://hdl.handle.net/1721.1/4414 (cit. on pp. 5, 452).

[417] Haiyan Liu, Yuqi Guo, and K. P. Shum. "Some Combinatorial Properties of Solid Codes". In: *Indian Journal of Pure and Applied Mathematics* 52.3 (2021), pp. 932–944. DOI: 10.1007/s13226-021-00093-w (cit. on p. 439).

[418] Donald Livingstone and Ascher Wagner. "Transitivity of Finite Permutation Groups on Unordered Sets". In: *Mathematische Zeitschrift* 90.5 (Oct. 1965), pp. 393–403. ISSN: 1432-1823. DOI: 10.1007/BF01112361 (cit. on p. 118).

[419] Evgenii S. Ljapin. *Semigroups*. Vol. 3. American Mathematical Society. Translations of Mathematical Monographs. American Mathematical Society, 1963. ISBN: 978-0-8218-1553-3. DOI: 10.1090/mmono/003 (cit. on p. 84).

[420] Markus Lohrey. "The Rational Subset Membership Problem for Groups: a Survey". In: *Groups St Andrews 2013*. Ed. by C. M. Campbell et al. London Mathematical Society Lecture Note Series. Cambridge University Press, 2015, pp. 368–389 (cit. on p. 256).

[421] Markus Lohrey and Andreas Rosowski. "On the Complexity of Diameter and Related Problems in Permutation Groups". In: *50th International Colloquium on Automata, Languages, and Programming, ICALP 2023, July 10-14, 2023, Paderborn, Germany*. Ed. by Kousha Etessami, Uriel Feige, and Gabriele Puppis. Vol. 261. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 134:1–134:18. DOI: 10.4230/LIPIcs.ICALP.2023.134 (cit. on p. 256).

[422] M. Lothaire. *Combinatorics on Words*. 2nd ed. Cambridge Mathematical Library. Cambridge University Press, 1997. ISBN: 978-0-521-59924-5. DOI: 10.1017/CBO9780511566097 (cit. on pp. 84, 428, 445, 453).

[423] Michael C. Loui and David R. Luginbuhl. "The Complexity of On-Line Simulations Between Multidimensional Turing Machines and Random Access Machines". In: *Mathematical Systems Theory* 25.4 (1992), pp. 293–308. DOI: 10.1007/BF01213861 (cit. on p. 85).

[424] Eugene M. Luks. "Parallel Algorithms for Permutation Groups and Graph Isomorphism". In: *27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27-29 October 1986*. IEEE Computer Society, 1986, pp. 292–302. DOI: 10.1109/SFCS.1986.39 (cit. on p. 256).

[425] Eugene M. Luks and Pierre McKenzie. "Parallel Algorithms for Solvable Permutation Groups". In: *Journal of Computer and System Sciences* 37.1 (1988), pp. 39–62. DOI: 10.1016/0022-0000(88)90044-X (cit. on pp. 226, 251).

[426] Roger C. Lyndon and Marcel P. Schützenberger. "The Equation $a^M = b^N c^P$ in a Free Group". In: *Michigan Mathematical Journal* 9.4 (1962), pp. 289–298. DOI: 10.1307/mmj/1028998766 (cit. on p. 428).

[427] Wolfgang Mader. "Über den Zusammenhang Symmetrischer Graphen". In: *Archiv der Mathematik* 21.1 (1970), pp. 331–336. DOI: 10.1007/BF01220924 (cit. on p. 127).

[428] Wolfgang Mader. "Minimale n-fach Kantenzusammenhängende Graphen". In: *Mathematische Annalen* 191.1 (1971), pp. 21–28. DOI: 10.1007/BF01433466 (cit. on p. 127).

[429] Stephen R. Mahaney. "Sparse Complete Sets of NP: Solution of a Conjecture of Berman and Hartmanis". In: *Journal of Computer and System Sciences* 25.2 (1982), pp. 130–143. DOI: 10.1016/0022-0000(82)90002-2 (cit. on p. 453).

[430] Oded Maler. "On the Krohn-Rhodes Cascaded Decomposition Theorem". In: *Time for Verification, Essays in Memory of Amir Pnueli*. Ed. by Zohar Manna and Doron A. Peled. Vol. 6200. Lecture Notes in Computer Science. Springer, 2010, pp. 260–278. DOI: 10.1007/978-3-642-13754-9_12 (cit. on p. 171).

[431] Oded Maler and Amir Pnueli. "Tight Bounds on the Complexity of Cascaded Decomposition of Automata". In: *31st Annual Symposium on Foundations of Computer Science, St. Louis, Missouri, USA, October 22-24, 1990, Volume II*. IEEE Computer Society, 1990, pp. 672–682. DOI: 10.1109/FSCS.1990.89589 (cit. on p. 170).

[432] Oded Maler and Amir Pnueli. "On the Cascaded Decomposition of Automata, its Complexity and its Application to Logic". available online: http://www-verimag.imag.fr/~maler/Papers/decomp.pdf (unpublished manuscript). 1994 (cit. on p. 171).

[433] Vladimir V. Gusev and Marina I. Maslennikova and Elena V. Pribavkina. "Finitely Generated Ideal Languages and Synchronizing Automata". In: *Combinatorics on Words - 9th International Conference, WORDS 2013, Turku, Finland, September 16-20. Proceedings*. Ed. by Juhani Karhumäki and Arto Lepistö and Luca Q. Zamboni. Vol. 8079. Lecture Notes in Computer Science. Springer, 2013, pp. 143–153. DOI: 10.1007/978-3-642-40579-2_16 (cit. on p. 16).

[434] Wim Martens and Frank Neven. “Frontiers of Tractability for Typechecking Simple XML Transformations”. In: *Journal of Computer and System Sciences* 73.3 (2007), pp. 362–390. DOI: 10.1016/j.jcss.2006.10.005 (cit. on pp. 231, 240, 255).

[435] Pavel V. Martugin. “A Series of Slowly Synchronizing Automata with a Zero State over a Small Alphabet”. In: *Information and Computation* 206.9-10 (2008), pp. 1197–1203. DOI: 10.1016/j.ic.2008.03.020 (cit. on pp. 397, 401, 403, 406).

[436] Pavel Martyugin. “Complexity of Problems Concerning Reset Words for Some Partial Cases of Automata”. In: *Acta Cybernetica* 19.2 (2009), pp. 517–536. eprint: https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/3780 (cit. on pp. 184, 187, 223, 231, 232, 246, 249, 250, 253, 255, 328, 354, 548).

[437] Pavel Martyugin. “Complexity of Problems Concerning Reset Words for Cyclic and Eulerian Automata”. In: *Theoretical Computer Science* 450 (2012), pp. 3–9. DOI: 10.1016/j.tcs.2012.04.022 (cit. on p. 249).

[438] Pavel Martyugin. “Computational Complexity of Certain Problems Related to Carefully Synchronizing Words for Partial Automata and Directing Words for Nondeterministic Automata”. In: *Theoretical Computer Science* 54.2 (2014), pp. 293–304. DOI: 10.1007/s00224-013-9516-6 (cit. on pp. 15, 238, 413, 414, 418, 419).

[439] Pavel V. Martyugin. “Synchronization of Automata with One Undefined or Ambiguous Transition”. In: *Implementation and Application of Automata - 17th International Conference, CIAA*. Ed. by Nelma Moreira and Rogério Reis. Vol. 7381. Lecture Notes in Computer Science. Springer, 2012, pp. 278–288. DOI: 10.1007/978-3-642-31606-7_24 (cit. on p. 238).

[440] Marina I. Maslennikova. “Reset Complexity of Ideal Languages”. In: *CoRR* abs/1404.2816 (2014). arXiv: 1404.2816 (cit. on pp. 16, 176, 302, 329, 406).

[441] Marina I. Maslennikova. “Reset Complexity of Ideal Languages Over a Binary Alphabet”. In: *International Journal of Foundations of Computer Science* 30.6-7 (2019), pp. 1177–1196. DOI: 10.1142/S0129054119400343 (cit. on pp. 16, 176, 251, 302, 401, 402, 406).

[442] Marina I. Maslennikova and Emanuele Rodaro. “Trim Strongly Connected Synchronizing Automata and Ideal Languages”. In: *Fundamenta Informaticae* 162.2-3 (2018), pp. 183–203. DOI: 10.3233/FI-2018-1720 (cit. on pp. 16, 302).

[443] Yoshifumi Masunaga, Shoichi Noguchi, and Juro Oizumi. “A Structure Theory of Automata Characterized by Groups”. In: *Journal of Computer and System Sciences* 7.3 (1973), pp. 300–305. DOI: 10.1016/S0022-0000(73)80011-X (cit. on p. 168).

[444] Alexandru Mateescu and Arto Salomaa. “Many-Valued Truth Functions, Cernys’ Conjecture, and Road Coloring”. In: *Current Trends in Theoretical Computer Science, Entering the 21th Century*. Ed. by Gheorghe Paun, Grzegorz Rozenberg, and Arto Salomaa. World Scientific, 2001, pp. 693–707 (cit. on p. 3).

[445] Donald B. McAlister. "Semigroups Generated by a Group and an Idempotent". In: *Communications in Algebra* 26.2 (1998), pp. 243–254. DOI: 10.1080/00927879808826145 (cit. on pp. 118, 366, 395).

[446] Warren S. McCulloch and Walter Pitts. "A Logical Calculus of the Ideas Immanent in Nervous Activity". In: *The Bulletin of Mathematical Biophysics* 5 (1943), pp. 115–133. DOI: 10.1007/BF02478259 (cit. on p. 170).

[447] Pierre McKenzie and Stephen A. Cook. "The Parallel Complexity of Abelian Permutation Group Problems". In: *SIAM Journal on Computing* 16.5 (1987), pp. 880–909. DOI: 10.1137/0216058 (cit. on p. 256).

[448] Robert McNaughton. "The Loop Complexity of Pure-Group Events". In: *Information and Control* 11.1/2 (1967), pp. 167–176. DOI: 10.1016/S0019-9958(67)90481-0 (cit. on p. 222).

[449] Robert McNaughton and Seymour A. Papert. *Counter-Free Automata*. M.I.T. Research Monograph No. 65 65. The MIT Press, 1971. ISBN: 0262130769 (cit. on pp. 42, 170, 192).

[450] Yu. T. Medvedev. "On the Class of Events Representable in a Finite Automaton". In: *Automata Studies (Russian Translation of Automata Studies, Princeton University Press, 34 (1956) with Appendix by Medvedev)* (1956). Ed. by Medvedev. English transl. in E.F. Moore (ed.), *Sequential Machines – Selected Papers*, Addison Wesley, Boston, MA 1964, pp. 385–401 (cit. on pp. 167, 169).

[451] Karl Menger. "Zur Allgemeinen Kurventheorie". In: *Fundamenta Mathematicae* 10.1 (1927). available online: http://eudml.org/doc/211191, pp. 96–115 (cit. on p. 61).

[452] Soheil Mohajer et al. "Parallel Unary Computing Based on Function Derivatives". In: *ACM Transactions on Reconfigurable Technology and Systems* 14.1 (4 2020), pp. 1–25. DOI: 10.1145/3418464 (cit. on p. 76).

[453] Juan A. Montoya and Christian Nolasco. "On the Synchronization of Small Sets of States". In: *Applied Mathematical Sciences* 11.44 (2017), pp. 2151–2173. DOI: 10.12988/ams.2017.77232 (cit. on p. 253).

[454] Juan A. Montoya and Christian Nolasco. "On the Synchronization of Planar Automata". In: *Language and Automata Theory and Applications - 12th International Conference, LATA 2018, Ramat Gan, Israel, April 9-11, 2018, Proceedings*. Ed. by Shmuel Tomi Klein, Carlos Martín-Vide, and Dana Shapira. Vol. 10792. Lecture Notes in Computer Science. Springer, 2018, pp. 93–104. DOI: 10.1007/978-3-319-77313-1_7 (cit. on p. 253).

[455] Ketan Mulmuley. "A Fast Parallel Algorithm to Compute the Rank of a Matrix over an Arbitrary Field". In: *Combinatorica* 7.1 (1987), pp. 101–104. DOI: 10.1007/BF02579205 (cit. on p. 256).

[456] John Myhill. "Finite Automata and the Representation of Events". In: *WADD Technical Report* 57 (1957), pp. 112–137 (cit. on pp. 161, 170).

[457] Balas K. Natarajan. "An Algorithmic Approach to the Automated Design of Parts Orienters". In: *27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27-29 October 1986*. IEEE Computer Society, 1986, pp. 132–142. DOI: 10.1109/SFCS.1986.5 (cit. on pp. 5, 9, 10, 184, 185, 232, 250, 254).

[458] Balas K. Natarajan. “Some Paradigms for the Automated Design of Parts Feeders”. In: *The International Journal of Robotics Research* 8.6 (1989), pp. 98–109. DOI: 10.1177/027836498900800607 (cit. on p. 9).

[459] Anil Nerode. “Linear Automaton Transformations”. In: *Proceedings of the American Mathematical Society* 9.4 (1958), pp. 541–544. DOI: 10.1090/S0002-9939-1958-0135681-9 (cit. on pp. 161, 170).

[460] John von Neumann. “The General and Logical Theory of Automata”. In: *Cerebral Mechanisms in Behavior: The Hixon Symposium*. Ed. by Lloyd A. Jeffress. New York: John Wiley & Sons, 1951, pp. 1–41 (cit. on p. 167).

[461] Peter Neumann, J. J. O’Connor, and E. F. Robertson. *The Abstract Group Concept*. https://mathshistory.st-andrews.ac.uk/HistTopics/Abstract_groups/. MacTutor History of Mathematics Archive, University of St Andrews. 2001 (cit. on p. 129).

[462] Peter G. Neumann. “Self-Synchronizing Sequential Coding with Low Redundancy”. In: *Bell System Technical Journal* 50.3 (1971), pp. 951–981. DOI: 10.1002/j.1538-7305.1971.tb01891.x (cit. on p. 5).

[463] Peter M. Neumann. “Primitive Permutation Groups Containing a Cycle of Prime-Power Length”. In: *Bulletin of the London Mathematical Society* 7.3 (1975), pp. 298–299. DOI: 10.1112/blms/7.3.298 (cit. on p. 359).

[464] Peter M. Neumann. “The Lawlessness of Groups of Finitary Permutations”. In: *Archiv der Mathematik* 26 (1975), pp. 561–566. DOI: 10.1007/BF01229781 (cit. on p. 115).

[465] Peter M. Neumann. “Primitive Permutation Groups and Their Section-Regular Partitions”. In: *The Michigan Mathematical Journal* 58 (2009), pp. 309–322. DOI: 10.1307/mmj/1242071695 (cit. on pp. 364, 365).

[466] Peter M. Neumann. *The Mathematical Writings of Évariste Galois*. Heritage of European Mathematics. European Mathematical Society, 2011. DOI: 10.4171/104 (cit. on pp. 129, 395).

[467] Peter M. Neumann, Gabrielle A. Stoy, and Edward C. Thompson. *Groups and Geometry*. Oxford University Press, 1994. ISBN: 9780198534518. DOI: 10.1093/oso/9780198534525.001.0001 (cit. on pp. 129, 168).

[468] nLab authors. *Composition*. https://ncatlab.org/nlab/show/composition. Revision 30. Oct. 2025 (cit. on pp. 82, 84).

[469] Robert H. Oehmke. “On the Structures of an Automaton and Its Input Semigroup”. In: *Journal of the ACM* 10.4 (1963), pp. 521–525. DOI: 10.1145/321186.321194 (cit. on p. 168).

[470] Mateus de Oliveira Oliveira and Michael Wehar. “Intersection Non-Emptiness and Hardness Within Polynomial Time”. In: *Developments in Language Theory - 22nd International Conference, DLT*. Ed. by Mizuho Hoshi and Shinnosuke Seki. Vol. 11088. Lecture Notes in Computer Science. Springer, 2018, pp. 282–290. DOI: 10.1007/978-3-319-98654-8_23 (cit. on p. 256).

[471] Mateus de Oliveira Oliveira and Michael Wehar. “On the Fine Grained Complexity of Finite Automata Non-Emptiness of Intersection”. In: *Developments in Language Theory - 24th International Conference, DLT 2020, Tampa, FL, USA, May 11-15, 2020, Proceedings*. Ed. by Natasa Jonoska and Dmytro Savchuk.

Vol. 12086. Lecture Notes in Computer Science. Springer, 2020, pp. 69–82. DOI: 10.1007/978-3-030-48516-0_6 (cit. on p. 256).

[472] Jörg Olschewski and Michael Ummels. "The Complexity of Finding Reset Words in Finite Automata". In: *Mathematical Foundations of Computer Science 2010, 35th International Symposium, MFCS 2010, Brno, Czech Republic, August 23-27, 2010. Proceedings*. Ed. by Petr Hlinený and Antonín Kucera. Vol. 6281. Lecture Notes in Computer Science. Springer, 2010, pp. 568–579. DOI: 10.1007/978-3-642-15155-2_50 (cit. on p. 250).

[473] Joël Ouaknine and James Worrell. "Decision Problems for Linear Recurrence Sequences". In: *Reachability Problems - 6th International Workshop, RP 2012, Bordeaux, France, September 17-19, 2012. Proceedings*. Ed. by Alain Finkel, Jérôme Leroux, and Igor Potapov. Vol. 7550. Lecture Notes in Computer Science. Springer, 2012, pp. 21–28. DOI: 10.1007/978-3-642-33512-9_3 (cit. on pp. 413, 414).

[474] Rohit Parikh. "On Context-Free Languages". In: *Journal of the ACM* 13.4 (1966), pp. 570–581. DOI: 10.1145/321356.321364 (cit. on pp. 58, 84).

[475] M. S. Paterson. *Boolean Function Complexity*. London Mathematical Society Lecture Note Series. Cambridge University Press, 1992. ISBN: 978-1316707197. DOI: 10.1017/CBO9780511526633 (cit. on p. 89).

[476] Gheorghe Păun and Arto Salomaa. "Thin and Slender Languages". In: *Discrete Applied Mathematics* 61.3 (1995), pp. 257–270. DOI: 10.1016/0166-218X(94)00014-5 (cit. on p. 453).

[477] I. Peák. "Automaten und Semigruppen. II (in Russian)". In: *Acta Scientiarum Mathematicarum (Szeged)* 26 (1965), pp. 49–54 (cit. on p. 222).

[478] Benjamin Peirce. *Linear Associative Algebra*. New ed., with addenda and notes / Algebra, Universal. Extracted from the American Journal of Mathematics. New York: Van Nostrand, 1882 (cit. on p. 84).

[479] Martti Penttonen. "Relative Cost Random Access Machines". In: *Fundamenta Informaticae* 38 (1,2 1999). DOI: 10.3233/FI-1999-381202 (cit. on p. 85).

[480] Tatjana Petkovic, Miroslav Ciric, and Stojan Bogdanovic. "Unary algebras, Semigroups and Congruences on Free Semigroups". In: *Theoretical Computer Science* 324.1 (2004), pp. 87–105. DOI: 10.1016/J.TCS.2004.03.054 (cit. on p. 167).

[481] Tatjana Petkovic and Magnus Steinby. "Piecewise Directable Automata". In: *Journal of Automata, Languages and Combinatorics* 6.2 (2001), pp. 205–220. DOI: 10.25596/jalc-2001-205 (cit. on p. 14).

[482] Sophie Piccard. *Sur les Bases du Groupe Symétrique et les Couples de Substitutions qui Engendrent un Goupe Régulier*. 1st ed. Librairie Vuibert, Paris, 1946 (cit. on p. 126).

[483] Eugene M. Luks and Pierre McKenzie. "Parallel Algorithms for Solvable Permutation Groups". In: *Journal of Computer and System Sciences* 37.1 (1988), pp. 39–62. DOI: 10.1016/0022-0000(88)90044-X (cit. on p. 256).

[484] Jean-Eric Pin. "Sur Les Mots Synthronisants Dans un Automate Fini". In: *Elektronische Informationsverarbeitung und Kybernetik/Journal of Information Processing and Cybernetics (now Journal of Automata, Languages and Combinatorics)* 14.6 (1978), pp. 297–303 (cit. on p. 13).

[485] Jean-Éric Pin. "Sur la Longueur des Mots de Rang Donné d'un Automate Fini". In: *Comptes Rendus de l'Académie des Sciences*. A 284 (1977), pp. 1233–1235 (cit. on p. 13).

[486] Jean-Éric Pin. "Sur un Cas Particulier de la Conjecture de Cerny". In: *Automata, Languages and Programming, Fifth Colloquium, Udine, Italy, July 17-21, 1978, Proceedings*. Ed. by Giorgio Ausiello and Corrado Böhm. Vol. 62. Lecture Notes in Computer Science. Springer, 1978, pp. 345–352. ISBN: 3-540-08860-1. DOI: 10.1007/3-540-08860-1_25 (cit. on pp. 13, 394).

[487] Jean-Éric Pin. "Le Problème de la Synchronisation et la Conjecture de Černý". In: *Non-Commutative Structures in Algebra and Geometric Combinatorics*. Ed. by Aldo de Luca. Vol. 109. Quaderni de la Ricerca Scientifica. Roma: CNR, 1981, pp. 37–48 (cit. on pp. 13, 181).

[488] Jean-Éric Pin. "On two Combinatorial Problems Arising From Automata Theory". In: *Annals of Discrete Mathematics* 17 (1983), pp. 535–548. DOI: 10.1016/S0304-0208(08)73432-7 (cit. on p. 13).

[489] Jean-Éric Pin. *Varieties Of Formal Languages*. Ed. by R. E. Miller. Plenum Publishing Co., 1986. ISBN: 0306422948 (cit. on pp. 82, 170, 222, 223, 518).

[490] Jean-Éric Pin. "Syntactic Semigroups". In: *Handbook of Formal Languages, Volume 1*. Ed. by Grzegorz Rozenberg and Arto Salomaa. Springer, 1997, pp. 679–746. DOI: 10.1007/978-3-642-59136-5_10 (cit. on pp. 82, 170, 222, 223, 249).

[491] Jean-Éric Pin. *Mathematical Foundations of Automata Theory*. available online: https://www.irif.fr/~jep/PDF/MPRI/MPRI.pdf (preprint of a book, accessed at 16th February 2024). 2022 (cit. on pp. 82–84, 159, 170, 223, 448, 453, 454, 505, 522).

[492] Nicholas Pippenger. "On Simultaneous Resource Bounds (Preliminary Version)". In: *20th Annual Symposium on Foundations of Computer Science, San Juan, Puerto Rico, 29-31 October 1979*. IEEE Computer Society, 1979, pp. 307–311. DOI: 10.1109/SFCS.1979.29 (cit. on p. 87).

[493] Carl Pixley, Seh-Woong Jeong, and Gary D. Hachtel. "Exact Calculation of Synchronizing Sequences Based on Binary Decision Diagrams". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 13.8 (1994), pp. 1024–1034. DOI: 10.1109/43.298038 (cit. on p. 254).

[494] Robert Piziak and P. L. Odell. "Full Rank Factorization of Matrices". In: *Mathematics Magazine* 72.3 (1999), pp. 193–201. DOI: 10.1080/0025570X.1999.11996730 (cit. on p. 413).

[495] Igor T. Podolak, Adam Roman, and Dariusz Jedrzejczyk. "Application of Hierarchical Classifier to Minimal Synchronizing Word Problem". In: *Artificial Intelligence and Soft Computing - 11th International Conference, ICAISC 2012, Zakopane, Poland, April 29-May 3, 2012, Proceedings, Part I*. Ed. by Leszek Rutkowski andn Marcin Korytkowski et al. Vol. 7267. Lecture Notes in Computer Science. Springer, 2012, pp. 421–429. DOI: 10.1007/978-3-642-29347-4_49 (cit. on p. 254).

[496] Igor T. Podolak et al. "A Machine Learning Approach to Synchronization of Automata". In: *Expert Systems with Applications* 97 (2018), pp. 357–371. DOI: 10.1016/j.eswa.2017.12.043 (cit. on pp. 8, 9, 254).

[497] Cheryl E. Praeger. "Finite Primitive Permutation Groups: A Survey". In: *Groups—Canberra 1989*. Ed. by L. G. Kovács. Berlin, Heidelberg: Springer Berlin Heidelberg, 1990, pp. 63–84. ISBN: 978-3-540-46900-1. DOI: 10.1007/BFb0100731 (cit. on p. 396).

[498] Cheryl E. Praeger, Cai-Heng Li, and Alice C. Niemeyer. "Finite Transitive Permutation Groups and Finite Vertex-Transitive Graphs". In: *Graph Symmetry: Algebraic Methods and Applications*. Ed. by G. Hahn and G. Sabidussi. Vol. 497. NATO ASI Series C: Mathematical and Physical Sciences. Dordrecht, Netherlands: Kluwer Academic Publishers, 1997, pp. 277–318. ISBN: 0792346688. DOI: 10.1007/978-94-015-8937-6_7 (cit. on p. 130).

[499] Nicola Prezza. "Compressed Computation for Text Indexing". available online: https://nicolaprezza.github.io/pdfs/CompressedComputationforTextIndexing.pdf. PhD thesis. Universita degli Studi di Udine: Universita degli Studi di Udine, Dipartimento di Scienze Matematiche, Informatiche e Fisiche, July 2016 (cit. on p. 454).

[500] Elena V. Pribavkina. "Slowly synchronizing automata with zero and noncomplete sets". In: *Mathematical Notes* 90.3-4 (2011), pp. 411–417. DOI: 10.1134/S0001434611090094 (cit. on p. 406).

[501] Elena V. Pribavkina and Emanuele Rodaro. "Synchronizing Automata With Finitely Many Minimal Synchronizing Words". In: *Information and Computation* 209.3 (2011), pp. 568–579. DOI: 10.1016/j.ic.2010.11.020 (cit. on pp. 14, 156, 302).

[502] Michael O. Rabin and Dana S. Scott. "Finite Automata and Their Decision Problems". In: *IBM Journal of Research and Development* 3.2 (1959), pp. 114–125. DOI: 10.1147/rd.32.0114 (cit. on pp. 85, 154, 169).

[503] Peter J. Ramadge and Walter M. Wonham. "Supervisory Control of a Class of Discrete Event Processes". In: *SIAM Journal on Control and Optimization* 25.1 (1987), pp. 206–230. DOI: 10.1137/0325013 (cit. on p. 416).

[504] Danny Raz. "Length Considerations in Context-Free Languages". In: *Theoretical Computer Science* 183.1 (1997), pp. 21–32. DOI: 10.1016/S0304-3975(96)00308-8 (cit. on pp. 447, 453).

[505] David Rees. "On Semi-Groups". In: *Mathematical Proceedings of the Cambridge Philosophical Society* 36.4 (1940), pp. 387–400. DOI: 10.1017/S0305004100017436 (cit. on pp. 150, 172).

[506] Rogério Reis and Emanuele Rodaro. "Ideal Regular Languages and Strongly Connected Synchronizing Automata". In: *Theoretical Computer Science* 653 (2016), pp. 97–107. DOI: 10.1016/j.tcs.2016.09.026 (cit. on pp. 16, 302).

[507] Antonio Restivo. "Codes and Automata". In: *Formal Properties of Finite Automata and Applications, LITP Spring School on Theoretical Computer Science, Ramatuelle, France, May 23-27, 1988, Proceedings*. Ed. by Jean-Éric Pin. Vol. 386. Lecture Notes in Computer Science. Springer, 1988, pp. 186–198. DOI: 10.1007/BFb0013121 (cit. on pp. 445, 446).

[508] Antonio Restivo and Roberto Vaglica. "A Graph Theoretic Approach to Automata Minimality". In: *Theoretical Computer Science* 429 (2012), pp. 282–291. DOI: 10.1016/j.tcs.2011.12.049 (cit. on pp. 165, 169, 391).

[509] Antonio Restivo and Roberto Vaglica. "Extremal Minimality Conditions on Automata". In: *Theoretical Computer Science* 440-441 (2012), pp. 73–84. DOI: 10.1016/j.tcs.2012.03.049 (cit. on pp. 165, 169).

[510] June-Kyung Rho, Fabio Somenzi, and Carl Pixley. "Minimum Length Synchronizing Sequences of Finite State Machine". In: *Proceedings of the 30th Design Automation Conference. Dallas, Texas, USA, June 14-18, 1993*. Ed. by Alfred E. Dunlop. ACM Press, 1993, pp. 463–468. DOI: 10.1145/157485.164978 (cit. on p. 254).

[511] John Rhodes and Benjamin Steinberg. *Applications of Automata Theory and Algebra: Via the Mathematical Theory of Complexity to Biology, Physics, Psychology, Philosophy, and Games*. Singapore: World Scientific, 2009. ISBN: 978-9812833754. DOI: 10.1142/7107 (cit. on p. 170).

[512] John Rhodes and Benjamin Steinberg. *The q-Theory of Finite Semigroups*. Springer Monographs in Mathematics. New York: Springer, 2009. ISBN: 978-0-387-09780-0. DOI: 10.1007/b104443 (cit. on p. 170).

[513] Emanuele Rodaro. "Strongly Connected Synchronizing Automata and the Language of Minimal Reset Words". In: *Advances in Applied Mathematics* 99 (2018), pp. 158–173. DOI: 10.1016/j.aam.2018.04.006 (cit. on pp. 16, 302).

[514] Emanuele Rodaro. "A Bound for the Length of the Shortest Reset Words for Semisimple Synchronizing Automata via the Packing Number". In: *Journal of Algebraic Combinatorics* 50.3 (2019), pp. 237–253. DOI: 10.1007/s10801-018-0851-1 (cit. on p. 14).

[515] A. Roman and M. Drewienkowski. "A Complete Solution to the Complexity of Synchronizing Road Coloring for Non-Binary Alphabets". In: *Information and Computation* 242 (2015), pp. 383–393. ISSN: 0890-5401. DOI: https://doi.org/10.1016/j.ic.2015.04.004 (cit. on p. 525).

[516] Adam Roman. "A Note on Černý Conjecture for Automata over 3-Letter Alphabet". In: *Journal of Automata, Languages and Combinatorics* 13.2 (2008), pp. 141–143. DOI: 10.25596/JALC-2008-141 (cit. on p. 398).

[517] Adam Roman. "Synchronizing Finite Automata with Short Reset Words". In: *Applied Mathematics and Computation* 209.1 (2009), pp. 125–136. DOI: 10.1016/j.amc.2008.06.019 (cit. on p. 254).

[518] Adam Roman. "The NP-completeness of the Road Coloring Problem". In: *Information Processing Letters* 111.7 (2011), pp. 342–347. DOI: 10.1016/J.IPL.2010.12.016 (cit. on p. 525).

[519] Adam Roman and Marek Szykuła. "Forward and Backward Synchronizing Algorithms". In: *Expert Systems with Applications* 42.24 (2015), pp. 9512–9527. DOI: 10.1016/j.eswa.2015.07.071 (cit. on p. 254).

[520] O.T. Romanov. "On Invariant Decoding Automata Without Look-Ahead". In: *Probl. Kibern.* 17.17 (1966). (in Russian), pp. 233–236 (cit. on pp. 419, 438, 439).

[521] J. C. Rosales and P. A. García-Sánchez. *Numerical Semigroups*. Vol. 20. Developments in Mathematics. New York: Springer, 2009. ISBN: 978-1-4419-0159-0. DOI: 10.1007/978-1-4419-0160-6 (cit. on p. 446).

[522] Jakub Ruszil. "Synchronizing Automata with Coinciding Cycles". In: *Developments in Language Theory - 27th International Conference, DLT 2023, Umeå, Sweden, June 12-16, 2023, Proceedings*. Ed. by Frank Drewes and Mikhail Volkov. Vol. 13911. Lecture Notes in Computer Science. Springer, 2023, pp. 208–218. DOI: 10.1007/978-3-031-33264-7_17 (cit. on p. 14).

[523] Walter L. Ruzzo. "On Uniform Circuit Complexity". In: *Journal of Computer and System Sciences* 22.3 (1981), pp. 365–383. DOI: 10.1016/0022-0000(81)90038-6 (cit. on pp. 87–89).

[524] Igor Rystsov and Marek Szykula. "Reset Thresholds of Transformation Monoids". In: *Cybernetics and Systems Analysis* 60.2 (2024), pp. 189–197. DOI: 10.1007/s10559-024-00660-z (cit. on pp. 14, 97, 110, 167).

[525] Igor Rystsov and Marek Szykuła. "Primitive Automata that are Synchronizing". In: *arXiv preprint arXiv:2307.01302* (2023). arXiv: 2307.01302 [cs.FL] (cit. on p. 153).

[526] Igor K. Rystsov. "Estimation of the Length of a Kernel Word for a Finite Automaton". In: *Automata*. Saratov Univ. Press, Saratov, 1977, pp. 45–50 (cit. on p. 178).

[527] Igor K. Rystsov. "Asymptotic Estimate of the Length of a Diagnostic Word for a Finite Automaton". In: *Cybernetics and Systems Analysis* 16.2 (1980), pp. 194–198. ISSN: 1573-8337. DOI: 10.1007/BF01069104 (cit. on p. 419).

[528] Igor K. Rystsov. "On Minimizing the Length of Synchronizing Words for Finite Automata". In: *Theory of Designing of Computing Systems*. (in Russian). Institute of Cybernetics of Ukrainian Acad. Sci., 1980, pp. 75–82 (cit. on pp. 226, 232, 249, 255, 534).

[529] Igor K. Rystsov. "Polynomial Complete Problems in Automata Theory". In: *Information Processing Letters* 16.3 (1983), pp. 147–151. DOI: 10.1016/0020-0190(83)90067-4 (cit. on pp. 226, 231, 239, 251, 252).

[530] Igor K. Rystsov. "Rank of a Finite Automaton". In: *Cybernetics and Systems Analysis* 28 (1992). Engl. translation, pp. 323–328. DOI: 10.1007/BF01125412 (cit. on p. 181).

[531] Igor K. Rystsov. "Almost Optimal Bound of Recurrent Word Length for Regular Automata". In: *Kibern. Sist. Anal.* 5 (1995). in Russian; Engl. translation: Cybernetics and Systems Analysis **31** (1995) 669–674, pp. 40–48. DOI: 10.1007/BF02366314 (cit. on pp. 14, 307, 323).

[532] Igor K. Rystsov. "Quasioptimal Bound for the Length of Reset Words for Regular Automata". In: *Acta Cybernetic* 12.2 (1995), pp. 145–152. eprint: https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/3453 (cit. on pp. 14, 323).

[533] Igor K. Rystsov. "Exact Linear Bound for the Length of Reset Words in Commutative Automata". In: *Publicationes Mathematicae, Debrecen* 48.3-4 (1996). https://www.researchgate.net/publication/268245703_Exact_linear_bound_for_the_length_of_reset_words_in_commutative_automata, pp. 405–409 (cit. on pp. 14, 207, 220, 222, 224).

[534] Igor K. Rystsov. "Reset Words for Commutative and Solvable Automata". In: *Theoretical Computer Science* 172.1-2 (1997), pp. 273–279. DOI: 10.1016/

S0304-3975(96)00136-3 (cit. on pp. 13, 14, 168, 171, 172, 177–179, 182, 207, 210, 213, 218).

[535] Igor K. Rystsov. "Estimation of the Length of Reset Words for Automata with Simple Idempotents". In: *Cybernetics and Systems Analysis* 36.3 (2000), pp. 339–344. ISSN: 1573-8337. DOI: 10.1007/BF02732984 (cit. on pp. 14, 106, 186, 222, 307–309, 320, 323, 357, 358).

[536] Igor K. Rystsov. "Cerny's Conjecture for Automata with Simple Idempotents". In: *Cybernetics and Systems Analysis* 58.1 (2022), pp. 1–7. DOI: 10.1007/s10559-022-00428-3 (cit. on pp. 14, 127, 222, 308, 309).

[537] Andrew Ryzhikov. "Synchronization Problems in Automata Without Non-Trivial Cycles". In: *Theoretical Computer Science* 787 (2019), pp. 77–88. DOI: 10.1016/j.tcs.2018.12.026 (cit. on pp. 14, 223, 231, 232, 242, 243, 248, 251, 534).

[538] Andrew Ryzhikov. "Synchronizing Automata and Coding Theory. (Automates Synchronisant et Théorie du Codage)". PhD thesis. University of Paris-Est, France, 2020. eprint: https://tel.archives-ouvertes.fr/tel-03157297 (cit. on pp. 446, 452).

[539] Andrew Ryzhikov and Anton Shemyakov. "Subset Synchronization in Monotonic Automata". In: *Fundamenta Informaticae* 162.2-3 (2018), pp. 205–221. DOI: 10.3233/FI-2018-1721 (cit. on pp. 223, 231).

[540] Gert Sabidussi. "Vertex-Transitive Graphs". In: *Monatshefte für Mathematik* 68 (1964), pp. 426–438. DOI: 10.1007/BF01304186 (cit. on p. 130).

[541] Udhayakumar Sadasivam. "Development of Vibratory Part Feeder for Material Handling in Manufacturing Automation: A Survey". In: *Journal of Automation, Mobile Robotics and Intelligent Systems* 9.4 (2015), pp. 3–10. DOI: 10.14313/JAMRIS_4-2015/27 (cit. on p. 9).

[542] Arto Salomaa. "A Theorem Concerning the Composition of Functions of Several Variables Ranging Over a Finite Set". In: *The Journal of Symbolic Logic* 25.3 (1960), pp. 203–208. ISSN: 00224812. DOI: 10.2307/2964675 (cit. on p. 395).

[543] Arto Salomaa. "On Basic Groups for the Set of Functions Over a Finite Domain". In: *Ann. Acad. Scient. Fennicae, Ser. AI, 338*. complete issue available online: https://www.acadsci.fi/mathematica/1964/year1964.html. 1963. DOI: 10.5186/aasfm.1964.338 (cit. on p. 395).

[544] Arto Salomaa. "Depth of Functional Compositions". In: *Bulletin EATCS* 71 (2000), pp. 143–150 (cit. on p. 394).

[545] Arto Salomaa. "Generation of Constants and Synchronization of Finite Automata". In: *Journal of Universal Computer Science* 8.2 (2002), pp. 332–347. DOI: 10.3217/jucs-008-02-0332 (cit. on pp. 126, 394).

[546] Arto Salomaa. "Composition Sequences for Functions Over a Finite Domain". In: *Theoretical Computer Science* 292.1 (2003), pp. 263–281. DOI: 10.1016/S0304-3975(01)00227-4 (cit. on pp. 12, 13, 232, 249, 394).

[547] Arto Salomaa. "Composition Sequences and Synchronizing Automata". In: *Computation, Physics and Beyond - International Workshop on Theoretical Computer Science, WTCS 2012, Dedicated to Cristian S. Calude on the Occasion of His 60th Birthday, Auckland, New Zealand, February 21-24, 2012,*

Revised Selected and Invited Papers. Ed. by Michael J. Dinneen and Bakhadyr Khoussainov and André Nies. Vol. 7160. Lecture Notes in Computer Science. Springer, 2012, pp. 403–416. DOI: 10.1007/978-3-642-27654-5_31 (cit. on pp. 12, 167, 394).

[548] Sven Sandberg. "Homing and Synchronizing Sequences". In: *Model-Based Testing of Reactive Systems, Advanced Lectures [The volume is the outcome of a research seminar that was held in Schloss Dagstuhl in January 2004]*. Ed. by Manfred Broy et al. Vol. 3472. Lecture Notes in Computer Science. Springer, 2005, pp. 5–33. DOI: 10.1007/11498490_2 (cit. on pp. 3, 5, 169, 185, 510).

[549] N. Ege Saraç et al. "Boosting Expensive Synchronizing Heuristics". In: *Expert Systems with Applications* 167 (2021), p. 114203. DOI: 10.1016/J.ESWA.2020.114203 (cit. on p. 254).

[550] Walter J. Savitch. "Relationships Between Nondeterministic and Deterministic Tape Complexities". In: *Journal of Computer and System Sciences* 4.2 (1970), pp. 177–192. DOI: 10.1016/S0022-0000(70)80006-X (cit. on pp. 72, 238, 350, 352).

[551] Marcel P. Schützenberger. "On an Application of Semi Groups Methods to some Problems in Coding". In: *IRE Transactions on Information Theory* 2.3 (1956), pp. 47–60. DOI: 10.1109/TIT.1956.1056809 (cit. on pp. 169, 428).

[552] Marcel P. Schützenberger. "On Finite Monoids Having Only Trivial Subgroups". In: *Information and Computation* 8.2 (1965), pp. 190–194. DOI: 10.1016/S0019-9958(65)90108-7 (cit. on p. 170).

[553] Marcel P. Schützenberger. "On Synchronizing Prefix Codes". In: *Information and Computation* 11.4 (1967), pp. 396–401. DOI: 10.1016/S0019-9958(67)90620-1 (cit. on p. 452).

[554] Leonard Scott. "Representations in Characteristic p". In: *The Santa Cruz Conference on Finite Groups (Univ. California, Santa Cruz, Calif., 1979)*. Vol. 37. Proceedings of Symposia in Pure Mathematics. American Mathematical Society, 1980, pp. 319–331. DOI: 10.1090/pspum/037/604599 (cit. on p. 395).

[555] Sidi Mohamed Sedjelmaci. "Some Related Functions to Integer GCD and Coprimality". In: *Electronic Notes in Discrete Mathematics* 37 (2011), pp. 135–140. DOI: 10.1016/J.ENDM.2011.05.024 (cit. on p. 74).

[556] Ákos Seress. *Permutation Group Algorithms*. Vol. 152. Cambridge Tracts in Mathematics. Cambridge: Cambridge University Press, 2003. ISBN: 9780521661034. DOI: 10.1017/CBO9780511546549 (cit. on p. 130).

[557] Jean-Pierre Serre. "On a Theorem of Jordan". In: *Bulletin of the American Mathematical Society* 40.4 (2003), pp. 429–440. DOI: 10.1090/S0273-0979-03-00992-3 (cit. on p. 359).

[558] Hanan Shabana. "Exact Synchronization in Partial Deterministic Automata". In: *Journal of Physics: Conference Series* 1352 (Oct. 2019), p. 012047. DOI: 10.1088/1742-6596/1352/1/012047 (cit. on p. 419).

[559] Claude E. Shannon. "A Symbolic Analysis of Relay and Switching Circuits". Not submitted until 1940; available online: http://dspace.mit.edu/handle/1721.1/7582. Master of Science. Cambridge, MA, USA: Department of Electrical Engineering, MIT, Aug. 1937, p. 72 (cit. on p. 87).

[560] Claude E. Shannon. “A Symbolic Analysis of Relay and Switching Circuits”. In: *Transactions of the American Institute of Electrical Engineers* 57.12 (Dec. 1938), pp. 713–723. ISSN: 0096-3860. DOI: 10.1109/T-AIEE.1938.5057767 (cit. on p. 87).

[561] Claude E. Shannon. “A Mathematical Theory of Communication”. In: *The Bell System Technical Journal* 27.3 (July 1948). Reprinted in [577]., pp. 379–423. ISSN: 0005-8580 (cit. on pp. 169, 428).

[562] Claude E. Shannon. “A Mathematical Theory of Communication (continued)”. In: *The Bell System Technical Journal* 27.4 (Oct. 1948). Reprinted in [577], pp. 623–656. ISSN: 0005-8580 (cit. on p. 428).

[563] Claude E. Shannon. “Computers and Automata”. In: *Proceedings of the IRE* 41.10 (1953), pp. 1234–1241. ISSN: 0096-8390. DOI: 10.1109/JRPROC.1953.274273 (cit. on p. 167).

[564] Claude E. Shannon and John McCarthy, eds. *Automata Studies. (AM-34), Volume 34*. Princeton: Princeton University Press, 1956. ISBN: 9781400882618. DOI: 10.1515/9781400882618 (cit. on p. 167).

[565] Tamara Shcherbak. “The Interval Rank of Monotonic Automata”. In: *Implementation and Application of Automata, 10th International Conference, CIAA 2005, Sophia Antipolis, France, June 27-29, 2005, Revised Selected Papers*. Ed. by Jacques Farré, Igor Litovsky, and Sylvain Schmitz. Vol. 3845. Lecture Notes in Computer Science. Springer, 2005, pp. 273–281. DOI: 10.1007/11605157_23 (cit. on p. 223).

[566] John C. Shepherdson and Howard E. Sturgis. “Computability of Recursive Functions”. In: *Journal of the ACM* 10.2 (1963), pp. 217–255. DOI: 10.1145/321160.321170 (cit. on pp. 64, 85).

[567] Lev Naumovich Shevrin. “On the theory of epigroups. I”. In: *Sbornik: Mathematics* 82.2 (1995), pp. 485–512. DOI: 10.1070/SM1995v082n02ABEH003577 (cit. on p. 224).

[568] Yaroslav Shitov. “An Improvement to a Recent Upper Bound for Synchronizing Words of Finite Automata”. In: *Journal of Automata, Languages and Combinatorics* 24.2–4 (2019), pp. 367–373. DOI: 10.25596/jalc-2019-367 (cit. on p. 13).

[569] Arseny M. Shur. “Combinatorial Complexity of Regular Languages”. In: *Computer Science - Theory and Applications, Third International Computer Science Symposium in Russia, CSR 2008, Moscow, Russia, June 7-12, 2008, Proceedings*. Ed. by Edward A. Hirsch et al. Vol. 5010. Lecture Notes in Computer Science. Springer, 2008, pp. 289–301. DOI: 10.1007/978-3-540-79709-8_30 (cit. on p. 454).

[570] H. J. Shyr and S.S. Yu. “Solid Codes and Disjunctive Domains”. In: *Semigroup Forum* 41.1 (1990), pp. 23–41. DOI: 10.1007/BF02573375 (cit. on p. 438).

[571] anonymous SICOMP referee. “Comment on Parallel Complexity of Coprimality”. In: *Electronic Colloquium on Computational Complexity* TR98-009 (2001). eprint: https://eccc.weizmann.ac.il/eccc-reports/1998/TR98-009/index.html (cit. on p. 74).

[572] Imre Simon. “Hierarchies of Events with Dot-Depth One”. PhD thesis. Waterloo, Ontario, Canada: University of Waterloo, 1972 (cit. on p. 256).

[573] Imre Simon. "Piecewise Testable Events". In: *Automata Theory and Formal Languages, 2nd GI Conference, Kaiserslautern, May 20-23, 1975*. Ed. by H. Barkhage. Vol. 33. Lecture Notes in Computer Science. Springer, 1975, pp. 214–222. DOI: 10.1007/3-540-07407-4_23 (cit. on p. 256).

[574] Charles C. Sims. "Computational Methods in the Study of Permutation Groups". In: *Computational Problems in Abstract Algebra*. Ed. by John Leech. Proceedings of a Conference Held at Oxford Under the Auspices of the Science Research Council Atlas Computer Laboratory, 29th August to 2nd September 1967. Pergamon, Oxford, 1970, pp. 169–183. DOI: 10.1016/B978-0-08-012975-4.50020-5 (cit. on p. 256).

[575] Michael Sipser. *Introduction to the Theory of Computation*. 3rd ed. Boston, MA: Cengage Learning, 2012. ISBN: 978-1-133-18779-0 (cit. on pp. 65, 72, 84–86).

[576] Sven Skyum and Leslie G. Valiant. "A Complexity Theory Based on Boolean Algebra". In: *Journal of the ACM* 32.2 (1985), pp. 484–502. DOI: 10.1145/3149.3158 (cit. on p. 86).

[577] David Slepian, ed. *Key Papers in the Development of Information Theory*. IEEE Press selected reprint series. 1109 Spring Street, Suite 300, Silver Spring, MD 20910, USA: IEEE Computer Society Press, 1974, pp. vi + 463. ISBN: 0-87942-027-8, 0-87942-028-6 (paperback) (cit. on p. 595).

[578] Jonathan P. Sorenson. "A Randomized Sublinear Time Parallel GCD Algorithm for the EREW PRAM". In: *Information Processing Letters* 110.5 (2010), pp. 198–201. DOI: 10.1016/J.IPL.2009.12.008 (cit. on p. 74).

[579] Michael Spivak. *Calculus On Manifolds: A Modern Approach To Classical Theorems Of Advanced Calculus*. 1st ed. Mathematics monograph series. CRC Press, 1965. ISBN: 9780813346120. DOI: 10.1201/9780429501906 (cit. on p. 82).

[580] Peter H. Starke. "Eine Bemerkung über Homogene Experimente." In: *Elektronische Informationverarbeitung und Kybernetik (later Journal of Information Processing and Cybernetics)* 2.2 (1966). (Translation: A remark about homogeneous experiments. *Journal of Automata, Languages and Combinatorics* 24 (2019) 2-4, 133 – 237), pp. 61–82. DOI: 10.25596/JALC-2019-133 (cit. on pp. 12, 13).

[581] Benjamin Steinberg. "A Theory of Transformation Monoids: Combinatorics and Representation Theory". In: *The Electronic Journal of Combinatorics* 17.1 (2010), R164. DOI: 10.37236/436 (cit. on p. 153).

[582] Benjamin Steinberg. "Černý's Conjecture and Group Representation Theory". In: *Journal of Algebraic Combinatorics: An International Journal* 31.1 (Feb. 2010), pp. 83–109. ISSN: 0925-9899. DOI: 10.1007/s10801-009-0185-0 (cit. on p. 14).

[583] Benjamin Steinberg. "The Černý Conjecture for One-Cluster Automata with Prime Length Cycle". In: *Theoretical Computer Science* 412.39 (2011), pp. 5487–5491. DOI: 10.1016/j.tcs.2011.06.012 (cit. on p. 14).

[584] Larry J. Stockmeyer and Albert R. Meyer. "Word Problems Requiring Exponential Time (Preliminary Report)". In: *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing, STOC*. ACM. Association for Computing Ma-

chinery, 1973, pp. 1–9. ISBN: 9781450374309. DOI: 10.1145/800125.804029 (cit. on pp. 73, 85, 231, 255).

[585] Larry J. Stockmeyer and Uzi Vishkin. "Simulation of Parallel Random Access Machines by Circuits". In: *SIAM Journal on Computing* 13.2 (1984), pp. 409–422. DOI: 10.1137/0213027 (cit. on p. 87).

[586] Howard Straubing. *Finite Automata, Formal Logic, and Circuit Complexity*. Progress in Theoretical Computer Science. Boston: Birkhäuser Boston, MA, 1994. ISBN: 978-1-4612-6695-2. DOI: 10.1007/978-1-4612-0289-9 (cit. on pp. 67, 89).

[587] Ivan H. Sudborough and A. Zalcberg. "On Families of Languages Defined by Time-Bounded Random Access Machines". In: *SIAM Journal on Computing* 5.2 (1976), pp. 217–230. DOI: 10.1137/0205018 (cit. on p. 85).

[588] Anton Suschkewitsch. "Über die Darstellung der eindeutig nicht umkehrbaren Gruppen mittelst der verallgemeinerten Substitutionen". In: *Matematicheskii Sbornik* 33.4 (1926). https://www.mathnet.ru/eng/sm7472, pp. 371–374 (cit. on p. 45).

[589] Michio Suzuki. *Group Theory I*. Vol. 247. Grundlehren der mathematischen Wissenschaften. Berlin, Heidelberg: Springer-Verlag, 1982. ISBN: 978-3-540-11142-2. DOI: 10.1007/978-3-642-68698-3 (cit. on p. 171).

[590] Andrew Szilard et al. "Characterizing Regular Languages with Polynomial Densities". In: *Mathematical Foundations of Computer Science 1992, 17th International Symposium, MFCS'92, Prague, Czechoslovakia, August 24-28, 1992, Proceedings*. Ed. by Ivan M. Havel and Václav Koubek. Vol. 629. Lecture Notes in Computer Science. Springer, 1992, pp. 494–503. DOI: 10.1007/3-540-55808-X_48 (cit. on p. 453).

[591] Edward Szpilrajn. "Sur l'Extension de l'Ordre Partiel". In: *Fundamenta Mathematicae* 16 (1930), pp. 386–389 (cit. on p. 46).

[592] Marek Szykuła. "Checking Whether an Automaton Is Monotonic Is NP-complete". In: *Implementation and Application of Automata - 20th International Conference, CIAA 2015, Umeå, Sweden, August 18-21, 2015, Proceedings*. Ed. by Frank Drewes. Vol. 9223. Lecture Notes in Computer Science. Springer, 2015, pp. 279–291. DOI: 10.1007/978-3-319-22360-5_23 (cit. on p. 223).

[593] Marek Szykuła. "Improving the Upper Bound on the Length of the Shortest Reset Word". In: *35th Symposium on Theoretical Aspects of Computer Science (STACS 2018)*. Ed. by Rolf Niedermeier and Brigitte Vallée. Vol. 96. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018, 56:1–56:13. ISBN: 978-3-95977-062-0. DOI: 10.4230/LIPIcs.STACS.2018.56 (cit. on p. 13).

[594] Marek Szykuła and Adam Zyzik. "An Improved Algorithm for Finding the Shortest Synchronizing Words". In: *30th Annual European Symposium on Algorithms, ESA 2022, September 5-9, 2022, Berlin/Potsdam, Germany*. Ed. by Shiri Chechik et al. Vol. 244. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, 85:1–85:15. DOI: 10.4230/LIPIcs.ESA.2022.85 (cit. on p. 254).

[595] Marek Szykuła and Adam Zyzik. (personal communication). July 29, 2024 (cit. on pp. 349, 355).

[596] Robert E. Tarjan. "Depth-First Search and Linear Graph Algorithms". In: *SIAM Journal on Computing* 1.2 (1972), pp. 146–160. DOI: 10.1137/0201010 (cit. on pp. 185, 199).

[597] Mustafa Kemal Tas, Kamer Kaya, and Hüsnü Yenigün. "Synchronizing Billion-Scale Automata". In: *Information Sciences* 574 (2021), pp. 162–175. DOI: 10.1016/j.ins.2021.05.072 (cit. on p. 254).

[598] Gabriel Thierrin. "Permutation Automata". In: *Mathematical Systems Theory* 2.1 (1968), pp. 83–90. DOI: 10.1007/BF01691347 (cit. on p. 222).

[599] Gabriel Thierrin. "Simple Automata". In: *Kybernetika* 6.5 (1970), pp. 343–350. eprint: http://www.kybernetika.cz/content/1970/5/343 (cit. on pp. 153, 169).

[600] David Fernando Casas Torres. "Completely Reachable Almost Group Automata". In: *Ural Mathematical Journal* 10.2 (2024), pp. 37–48. DOI: 10.15826/umj.2024.2.004 (cit. on p. 390).

[601] Avraham Trahtman. "Some Aspects of Synchronization of DFA". In: *Journal of Computer Science and Technology* 23.5 (2008), pp. 719–727. DOI: 10.1007/s11390-008-9165-4 (cit. on p. 14).

[602] Avraham N. Trahtman. "Some Results of Implemented Algorithms of Synchronization". In: *10th Journées Montoises d'Informatique Théorique, Liege, Belgium, September 8–11*. available online: https://u.cs.biu.ac.il/~trakhta/apLieg04.pdf. 2004 (cit. on p. 254).

[603] Avraham N. Trahtman. "An Efficient Algorithm Finds Noticeable Trends and Examples Concerning the Černý Conjecture". In: *Mathematical Foundations of Computer Science 2006, 31st International Symposium, MFCS 2006, Stará Lesná, Slovakia, August 28-September 1, 2006, Proceedings*. Ed. by Rastislav Kralovic and Pawel Urzyczyn. Vol. 4162. Lecture Notes in Computer Science. Springer, 2006, pp. 789–800. DOI: 10.1007/11821069_68 (cit. on p. 398).

[604] Avraham N. Trahtman. "The Černý Conjecture for Aperiodic Automata". In: *Discrete Mathematics & Theoretical Computer Science* 9.2 (2007), pp. 3–10. DOI: 10.46298/DMTCS.395 (cit. on p. 14).

[605] Avraham N. Trahtman. "The Road Coloring Problem". In: *Israel Journal of Mathematics* 172 (2009), pp. 51–60. DOI: 10.1007/s11856-009-0062-5 (cit. on pp. 5, 10).

[606] Nicholas F. Travers and James P. Crutchfield. "Exact Synchronization for Finite-State Sources". In: *Journal of Statistical Physics* 145.5 (2011), pp. 1181–1201. DOI: 10.1007/s10955-011-0342-4 (cit. on p. 419).

[607] Vladimir I. Trofimov. "Growth Functions of Some Classes of Languages". In: *Cybernetics and Systems Analysis* 17.6 (1981), pp. 727–731. ISSN: 1573-8337. DOI: 10.1007/BF01069218 (cit. on p. 453).

[608] Alan M. Turing. "On Computable Numbers, with an Application to the Entscheidungsproblem". In: *Proceedings of the London Mathematical Society*. 2nd ser. 42.1 (1936), pp. 230–265. DOI: 10.1112/plms/s2-42.1.230 (cit. on p. 85).

[609] Uraz Cengiz Türker and Hüsnü Yenigün. "Complexities of Some Problems Related to Synchronizing, Non-Synchronizing and Monotonic Automata". In: *International Journal of Foundations of Computer Science* 26.1 (2015), pp. 99–122. DOI: 10.1142/S0129054115500057 (cit. on p. 251).

[610] Mikhail V. Volkov. "Synchronization of Primitive Automata". In: *RAIRO Theoretical Informatics and Applications* 58.3 (2024), pp. 1–7. DOI: https://doi.org/10.1051/ita/2024003 (cit. on pp. 153, 335, 345, 381, 382, 390).

[611] Leslie G. Valiant. "Reducibility by Algebraic Projections". In: *Espace et Temps* 28 (1982), pp. 393–406 (cit. on p. 460).

[612] Dmitry S. Ananichev and Vojtech Vorel. "A new Lower Bound for Reset Threshold of Binary Synchronizing Automata with Sink". In: *Journal of Automata, Languages and Combinatorics* 24.2-4 (2019), pp. 153–164. DOI: 10.25596/JALC-2019-153 (cit. on pp. 401, 406).

[613] Mikhail V. Volkov. "Synchronizing Automata and the Černý Conjecture". In: *Language and Automata Theory and Applications, 2nd Int. Conference, LATA 2008, Tarragona, Spain, March 13-19, 2008. Revised Papers*. Ed. by Carlos Martín-Vide, Friedrich Otto, and Henning Fernau. Vol. 5196. Lecture Notes in Computer Science. Springer, 2008, pp. 11–27. ISBN: 978-3-540-88281-7. DOI: 10.1007/978-3-540-88282-4_4 (cit. on pp. 3, 5, 7, 8, 10, 185, 397, 399, 406, 510).

[614] Mikhail V. Volkov. "Synchronizing Automata Preserving a Chain of Partial Orders". In: *Theoretical Computer Science* 410.37 (2009), pp. 3513–3519. DOI: 10.1016/j.tcs.2009.03.021 (cit. on pp. 14, 181, 182).

[615] Mikhail V. Volkov. "Preface". In: *Journal of Automata, Languages and Combinatorics* 24.2-4 (2019), pp. 119–121. DOI: 10.25596/jalc-2019-119 (cit. on p. 13).

[616] Mikhail V. Volkov. "Slowly Synchronizing Automata with Idempotent Letters of Low Rank". In: *Journal of Automata, Languages and Combinatorics* 24.2-4 (2019), pp. 375–386. DOI: 10.25596/JALC-2019-375 (cit. on pp. 14, 397, 406).

[617] Mikhail V. Volkov. "Synchronization of Finite Automata". In: *Russian Mathematical Surveys* 77.5 (2022), pp. 819–891. DOI: 10.4213/rm10005e (cit. on pp. 3, 5, 8, 167, 232, 233, 452).

[618] Mikhail V. Volkov. *List of Results on the Černý Conjecture and Reset Thresholds for Synchronizing Automata*. 2025. arXiv: 2508.15655 [cs.FL] (cit. on p. 14).

[619] Heribert Vollmer. *Introduction to Circuit Complexity - A Uniform Approach*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 1999. ISBN: 978-3-540-64310-4. DOI: 10.1007/978-3-662-03927-4 (cit. on pp. 66, 72, 73, 87–89).

[620] Vojtěch Vorel. "Subset Synchronization and Careful Synchronization of Binary Finite Automata". In: *International Journal of Foundations of Computer Science* 27.5 (2016), pp. 557–578. DOI: 10.1142/S0129054116500167 (cit. on p. 232).

[621] Vojtěch Vorel. "Complexity of a Problem Concerning Reset Words for Eulerian Binary Automata". In: *Information and Computation* 253 (2017), pp. 497–509. DOI: 10.1016/j.ic.2016.06.013 (cit. on p. 249).

[622] Vojtěch Vorel. "Synchronization, Road Coloring, and Jumps in Finite Automata". Available online: https://www.itspy.cz/wp-content/uploads/2015/11/acmspy2015__1.pdf. Master's thesis. Prague, Czech Republic: Charles University in Prague, 2015 (cit. on p. 406).

[623] Vojtěch Vorel and Adam Roman. “Parameterized Complexity of Synchronization and Road Coloring”. In: *Discrete Mathematics & Theoretical Computer Science* 17.1 (2015), pp. 283–306. DOI: 10.46298/dmtcs.2103 (cit. on p. 253).

[624] Vojtěch Vorel and Adam Roman. “Complexity of Road Coloring with Prescribed Reset Words”. In: *Journal of Computer and System Sciences* 104 (2019), pp. 342–358. DOI: 10.1016/j.jcss.2016.05.009 (cit. on p. 525).

[625] B. L. van der Waerden. *Moderne Algebra. Grundlehren der mathematischen Wissenschaften*. 2nd ed. Vol. 33–34. Grundlehren der mathematischen Wissenschaften. Digital reissue / Springer Book Archive eBook. Springer Berlin Heidelberg, 1937. ISBN: 978-3-662-36434-5. DOI: 10.1007/978-3-662-36434-5 (cit. on p. 129).

[626] Todd Wareham. “The Parameterized Complexity of Intersection and Composition Operations on Sets of Finite-State Automata”. In: *Implementation and Application of Automata, 5th International Conference, CIAA 2000, London, Ontario, Canada, July 24-25, 2000, Revised Papers*. Ed. by Sheng Yu and Andrei Paun. Vol. 2088. Lecture Notes in Computer Science. Springer, 2000, pp. 302–310. DOI: 10.1007/3-540-44674-5_26 (cit. on pp. 253, 256).

[627] Mark E. Watkins. “Connectivity of Transitive Graphs”. In: *Journal of Combinatorial Theory* 8.1 (1970), pp. 23–29. ISSN: 0021-9800. DOI: 10.1016/S0021-9800(70)80005-9 (cit. on p. 127).

[628] Heinrich Weber. *Lehrbuch der Algebra. Zweiter Band*. Volume 2 of Lehrbuch der Algebra. Braunschweig: F. Vieweg und Sohn, 1896 (cit. on p. 129).

[629] Heinrich Martin Weber. “Beweis des Satzes, dass jede eigentlich primitive quadratische Form unendlich viele Primzahlen darzustellen fähig ist”. In: *Mathematische Annalen* 20.3 (1882). Enthält eine frühe axiomatische Definition des Gruppenbegriffs im Kontext algebraischer Strukturen, pp. 301–329. DOI: 10.1007/BF01443599 (cit. on p. 129).

[630] Joseph H. MacLagan Wedderburn. “On Hypercomplex Numbers”. In: *Proceedings of the London Mathematical Society*. 2d Ser. 6.1 (1908), pp. 77–118. DOI: 10.1112/plms/s2âĂŚ6.1.77 (cit. on p. 84).

[631] Gerard P. Weeg. “The Structure of an Automaton and Its Operation-Preserving Transformation Group”. In: *Journal of the ACM* 9.3 (1962), pp. 345–349. DOI: 10.1145/321127.321131 (cit. on pp. 168, 169).

[632] Gerard P. Weeg. “The Automorphism Group of the Direct Product of Strongly Related Automata”. In: *Journal of the ACM* 12.2 (1965), pp. 187–195. DOI: 10.1145/321264.321269 (cit. on p. 168).

[633] Gerard P. Weeg. “Corrigendum: "The Automorphism Group of the Direct Product of Strongly Related Automata"”. In: *Journal of the ACM* 14.2 (1967), p. 421. DOI: 10.1145/321386.321405 (cit. on p. 168).

[634] Ingo Wegener. *The Complexity of Boolean Functions*. USA: John Wiley & Sons, Inc., 1987. ISBN: 0471915556 (cit. on pp. 88, 89).

[635] Michael Wehar. “Hardness Results for Intersection Non-Emptiness”. In: *Automata, Languages, and Programming - 41st International Colloquium, ICALP, Part II*. Ed. by Javier Esparza and Pierre Fraigniaud, Thore Husfeldt, and Elias

Koutsoupias. Vol. 8573. Lecture Notes in Computer Science. Springer, 2014, pp. 354–362. DOI: 10.1007/978-3-662-43951-7_30 (cit. on pp. 254, 256).

[636] Michael Wehar. "On the Complexity of Intersection Non-Emptiness Problems". available online: http://www.michaelwehar.com/documents/mwehar_dissertation.pdf. PhD thesis. University at Buffalo, 2016 (cit. on pp. 86, 254, 256).

[637] Charles E. Wells. "Some Applications of the Wreath Product Construction". In: *The American Mathematical Monthly* 83.5 (1976), pp. 317–338. DOI: 10.1080/00029890.1976.11994114 (cit. on p. 171).

[638] Helmut Wielandt. *Finite Permutation Groups*. Academic paperbacks. Mathematics. Translated from the German by Ronald D. Bercov. Academic Press, 1964. ISBN: 9781483258294 (cit. on pp. 82, 117, 129, 168).

[639] Wikipedia. *Self-synchronizing code — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Self-synchronizing%20code&oldid=1083645813. [Online; accessed 17-Juli-2025]. 2022 (cit. on p. 502).

[640] Walter M. Wonham and Kai Cai. *Supervisory Control of Discrete-Event Systems*. Springer, 2019. ISBN: 978-3-319-77451-0. DOI: 10.1007/978-3-319-77452-7 (cit. on p. 416).

[641] Hans Wussing. *The Genesis of the Abstract Group Concept: A Contribution to the History of the Origin of Abstract Group Theory*. Trans. by Abraham Shenitzer. English edition. Cambridge, MA and London, UK: The MIT Press, 1984, p. 272. ISBN: 978-0-262-23109-1 (cit. on p. 129).

[642] Sheng Yu. "Regular Languages". In: *Handbook of Formal Languages, Volume 1: Word, Language, Grammar*. Ed. by Grzegorz Rozenberg and Arto Salomaa. Springer, 1997, pp. 41–110. DOI: 10.1007/978-3-642-59136-5_2 (cit. on p. 453).

[643] Sheng Yu, Qingyu Zhuang, and Kai Salomaa. "The State Complexities of Some Basic Operations on Regular Languages". In: *Theoretical Computer Science* 125.2 (1994), pp. 315–328. DOI: 10.1016/0304-3975(92)00011-F (cit. on p. 16).

[644] H. Paul Zeiger. "Cascade Synthesis of Finite-State Machines". In: *6th Annual Symposium on Switching Circuit Theory and Logical Design, Ann Arbor, Michigan, USA, October 6-8, 1965*. IEEE Computer Society, 1965, pp. 45–51. DOI: 10.1109/FOCS.1965.4 (cit. on p. 222).

[645] H. Paul Zeiger. "Cascade Synthesis of Finite-State Machines". In: *Information and Control* 10.4 (1967), pp. 419–433. DOI: 10.1016/S0019-9958(67)90228-8 (cit. on pp. 170, 222).

[646] H. Paul Zeiger. "Yet Another Proof of the Cascade Decomposition Theorem for Finite Automata". In: *Mathematical Systems Theory* 1.3 (1967), pp. 225–228. DOI: 10.1007/BF01703821 (cit. on p. 170).

[647] Yinfeng Zhu. "A Quadratic Upper Bound on the Reset Thresholds of Synchronizing Automata Containing a Transitive Permutation Group". In: *44th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2024, Gandhinagar, Gujarat, India, December 16-18, 2024*. Ed. by Siddharth Barman and Slawomir Lasota. Vol. 323. LIPIcs.

Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, 34:1–34:14. DOI: 10.4230/LIPICS.FSTTCS.2024.34 (cit. on p. 14).

[648] Yinfeng Zhu. "Around Don's Conjecture for Binary Completely Reachable Automata". In: *Developments in Language Theory - 28th International Conference, DLT 2024, Göttingen, Germany, August 12-16, 2024, Proceedings*. Ed. by Joel D. Day and Florin Manea. Lecture Notes in Computer Science. Springer, 2024, pp. 282–295. DOI: 10.1007/978-3-031-66159-4_20. arXiv: 2402.19089 [cs.FL] (cit. on p. 347).

[649] Karl-Heinz Zimmermann. "On Krohn-Rhodes Theory for Semiautomata". In: *arXiv preprint* (2020). arXiv: 2010.16235 [cs.FL] (cit. on p. 170).

Index

1-set, 28
A-primitive permutation group, 368
G-orbit, 112
L-uniform NC^1 Turing reducible, *see* L-uniform NC^1 oracle reducible
L-uniform NC^1 oracle reducible, 72
L-uniform families of circuits, 67
M-act, 168
R_1-solvable, 220
R_2-solvable, 220
S-congruence, 45
$\leq_T^{\mathsf{NC}^1}$-reduction, 72
$\leq_m^{\log}$-reduction, 72
$\leq_m^{\mathsf{AC}^0}$-reduction, 72
$\leq_m^{\mathsf{NC}^1}$-reduction, 72
$\leq_m^{\mathsf{P}}$-reduction, 72
ρ-automaton, 198
$f(n)$-space bounded, 65
$f(n)$-time bounded, 65
i-fold composition, 38
k-connected graph, 60
k-homogeneous, 112
k-regular set of words, 323
k-set, 28
k-transitive, 112
m-cycle, *see* cycle
n-cropped Parikh map, 518
n-tuple, *see* tuple
r-decomposition, 413
Sync[L], *see* constrained synchronization problem
k-DAG-ST , 71
k-DAG, 71
3SAT, 71
DAG, 70
Intersection, 226
ReachFrom, 226
Reach, 226
SingleCycle, 70
StronglyConn, 70
SyncFrom, 226
SyncShort, 225
SyncState, 225
SyncWords, 225
Sync, 225
DisjSetTrans, 226
SetTrans, 226
SyncInto, 226
SyncTrap, 225
SAT, 70
Černý automata family, 10
Černý bound, 12
Černý conjecture, 12
IdempotentRank, 254
Membership, 254
OrderBetweenVertices, 70
TransformationRank, 254

abelian group, 43
abstract group, 122

accepted
  language
    by automaton, 159
    by PDFA, 418
  word
    by automaton, 159
    by PDFA, 418
accepts an input
  Turing machine, 65
accessible
  automaton, 159
  PDFA, 418
  state
    in automaton, 159
    in PDFA, 418
acts transitively, 112
acyclic graph, 60
almost sync-maximal, 406
alphabet, 3
alternating group, 112
annihilation law, 41
anti-Leibniz order, 81
antisymmetric relation, 39
Ashby's ghost taming automaton, 7
assignment, 70
associative operation, 40
atom in a graph, 127
augmentative subset, 307
automata with output, 169
automaton, *see* deterministic and complete finite semiautomaton
  as language acceptor, 159
automaton graph, 131
automaton with one-point singular idempotents, 186
automaton with simple idempotents, 186
automorphism
  of graphs, 59
  semiautomaton, 137
automorphism group
  of graphs, 59
auxiliary state avoiding word
  strongly, 472
  weakly, 472
avoiding word, 252
bifix code, 424
bijective, 30
binary automaton, 132
binary language, 57
binary relation, 39
bipartite graph, 377
block
  in automaton, 132
  in permutation group, 111
Bondar-Volkov-Casas graphs, 303
Boolean circuit, 66
Boolean matrix, 44
Boolean semiring, 41
Boolean transition matrix, 44
  automaton, 132
bottom-up method, 307
bounded language, 441

canonical map of $\mathcal{A}$ onto MinAut(L), 161
canonical morphism, 149
canonical projection map, 39
cardinality, 3
careful synchronization of automaton, 419
carefully synchronizing words, 419
Cartesian product, 28
center of group, 43
channel alphabet, 427
circular automaton, 338
clause, 70
coaccessible state
  in automaton, 159
  in PDFA, 418
coarser equivalence relation, 40
code, 424
coding morphism, 427
codomain
  of function, 3
column index, 44
column of matrix, 44
comma-free code, 424
commutative automaton, 200
commutative language, 57
commutative semigroup, 41
commutativity, 41

complete problem (or language), 72
completely distinguishable
  automaton, 279
  transformation semigroup, 288
completely reachable
  automaton, 265
  transformation semigroup, 288
complex product, *see* product on subsets
complexity class, 71
component projection mapping, 28
component-wise addition, 43
component-wise order, 47
component-wise product, 43
composition
  of total functions, 31
  of partial functions, 417
computing a function
  circuit, 66
concatenation product, 3
congruence class, 45
congruent elements, 45
conjugate transformation, 112
conjunction, 27
connected component, 60
connected graph, 60
connectivity, 61
constant function, 30
constrained synchronization problem, 15
constraint language, 15
constraint PDFA, 450
coprime number, 28
coset, *see* right coset
cycle, 93
cycle contained in transformation
  see cycle induced by transformation, 93
cycle in a graph, 60
cycle induced by transformation, 93
cycle notation, 93
cycle point, 38
cyclic automaton, 135
cyclic group, 43
cyclic permutation, *see* cycle
cyclically permuting a set, 93
DAG, *see* directed acyclic graph
deciding a language
  circuit, 66
  computational mechanism, 69
  Turing machine, 65
decision problem, 69
defect
  of a function, 36
  of a word, 132
defect theorem, 453
degree of permutation group, 111
dense language, 453
depth of a circuit, 66
derivative (of a language), *see* language quotient
deterministic and complete finite automaton, *see* deterministic and complete finite semiautomaton
deterministic and complete finite semiautomaton, 4
diagonal action, 122, 123
diagrammatic order, 81
digraph, *see* directed graph
direct product (semigroup), 43
directed acyclic graph, 60
directed graph, 59
discrete event system, 416
disjoint, 28
disjoint cycles, 93
disjoint union, 28
disjunction, 27
distinguishable pair, 262
distinguishable states, 161
distinguishable subsets
  in automaton, 262
  in transformation semigroup, 288
distinguishing
  transformation, 288
  word, 262
domain
  of function, 3
  of permutation group, 111
  of transformation semigroup, 45
duplicate set
  of a function, 30
  of a word, 132

edge of a graph, 59
empty function, 30
empty set, 3
empty word, 3
ending vertex, 59
endofunction, 38
endomorphism
  semiautomaton, 137
endpoints of edge, 59
equal-letter run, 443
equality relation, 40
equivalence, 27
equivalence class, 39
equivalence relation, 39
equivalence relation induced
  by partition, 39
  by quasiorder, 46
exact synchronization of automaton, 419
exactly synchronizing word, 419
excess
  of a subset under a function, 36
excluded set
  of a function, 30
  of a word, 132
existential quantifier, 27
exponential growth language, 447
extending problem, 252
extension method, 307
extension of a function to subsets, 30

factor, 57
factor (semi-)automaton, 149
fan-in, 66
fan-out, 66
final state set
  of a PDFA, 418
  of an automaton, 159
final vertex, 59
finer equivalence relation, 40
finite alphabet, *see* alphabet
finite set, 28
finite state set, *see* state set
fixed point (by transformation), *see* fixpoint
fixpoint
  of transformation, 38
  of transformation semigroup, 45
forced edges in $\Gamma_k(\mathcal{A})$, 304
forest, 60
formal language (over Σ), 3
free monoid (over X^*), 56
free submonoid (of Σ^*), 453
Frobenius coin problem, 446
full symmetric group, 111
full transformation semigroup, 45
function, 3
function composition, *see* composition
function extended to subsets, 30
function induced by word, 132
functional graph, 62

generated
  subgroup, 43
  submonoid, 43
  subsemigroup, 43
  equivalence relation, 40
  lower set, 46
  upper set, 46
generator, 43
greatest common divisor, 28
greatest element, *see* maximum
group, 41
group of units, 42

hard problem (or language), 72
holonomy decomposition, 170
homing sequence, 169
homogeneous relation, 39

ideal language, 57
idempotent element, 41
idempotent letter/word, 132
idempotent transformation, 103
identity element, 41
identity matrix, 44
identity transformation, 30, 38
image of subset
  of transition function, 4
image of subsets
  of a function, 30
implication, 27

imprimitive action, 368
imprimitive permutation group, 112
incoming edge, 59
incomparable elements, 46
incompressible set, 382
indegree of vertex, 59
indistinguishable states, 161
induced partial order, 47
induction proof, 27
infix, *see* factor
infix code, 424
inherited edges in $\Gamma_k(\mathcal{A})$, 304
initial state, *see* start state
initial vertex, 59
initially connected, *see* accessible automaton
injective, 30
input alphabet
  of a PDFA, 418
  of an automaton, 4
input instance, 70
integers, 4
intercode of index one, 427
intersection, 28
interval, 11
intransitive permutation group, 112
inverse element, 41
inverse function, 30
inverse image of a function, 30
inverse operation, 41
inverse transformation, 91
isomorphic automata, 137
isomorphism
  of monoids, 42
  of semigroups, 41
  order isomorphism, 46
  semiautomaton, 137

kernel relation, 39
kernel type of a function, 36
Kleene plus, 56
Kleene star, 56
Klein four-group, 113
Krasner–Kaloujnine universal embedding theorem, 170
Krohn-Rhodes decomposition theory, 170

language, *see* formal language (over Σ)
language quotient, 57
least common multiple, 28
least element, *see* minimum
left (language) quotient, *see* language quotient
left action, 81
left coset, 42
left ideal language, 57
left regular (Cayley) representation, 122
left-distributivity, 41
Leibniz order, 81
length of word, 3
letter, *see* symbol
letter-bounded language, 441
Levenshtein-Romanov mapping, 419
linear extension, *see* topological sorting
linear order, *see* total order
literal, 70
literal automaton, 446
lower set
  quasiorder, 46

many-one equivalent
  L-uniform AC^0, 72
  L-uniform NC^1, 72
  log-space, 72
  polynomial-time, 72
many-one reducible
  L-uniform AC^0, 72
  L-uniform NC^1, 72
  log-space, 72
  polynomial-time, 72
map, *see* function
matrix orbit problem, 414
matrix over semiring, 43
matrix product, 44
maximal element, 46
maximal prefix code, 478
maximal subgroup, 42
maximum, 46

membership problem, 69
minimal automaton, 161
minimal element, 46
minimum, 46
monoid, 41
monoid action, 168
monoid language, 445
monotone function, *see* morphism
monotonic automaton, 191
morphic image, 139
morphism
  of (semi-)automata, 137
  of groups, 42
  of monoids, 42
  of quasiordered sets, 46
  of semigroups, 41
most complex regular languages, 390
moved point (by transformation), 38
Myhill-Nerode relation, 161

natural numbers, 4
negation, 27
node of a graph, *see* vertex of a graph
nondeterministic automata, 522
normal subgroup, 43

O'Nan-Scott theorem, 395
OI-automaton, *see* automaton with one-point singular idempotents
one-hot unary, 76
one-point singular, 97
oracle equivalent
  L-uniform NC^1, 72
oracle reducible
  L-uniform NC^1, 72
orbit-stabilizer property, 117
orbital graph
  directed, 127
  undirected, 127
order of group, 43
oriented digraph, 60
outdegree of graph, 126
outdegree of vertex, 59
outgoing edge, 59
overlap-free code, 424

pair (semi)automaton, 157
pair criterion, 182
Parikh map, 57
Parikh morphism, *see* Parikh map
partial automaton, *see* partial deterministic automaton
partial deterministic automaton, 418
partial function, 417
partial order, 46
partially ordered automaton, 196
partition, 28
path in a graph, 60
PDFA, *see* partial deterministic automaton
  as language acceptor, 418
permutation, 91
permutation automaton, 190
permutation group, 111
permutation-reset automaton, 190
permutational letter/word, 132
permuted subset, 91, 93
pigeonhole principle, 27
Pisot's problem, 414
point contained in cycle, 93
point stabilizer, 112
point w.r.t. a transformation, 38
point-symmetric graph, 130
point-wise product, 43
point-wise representation, 78
polycyclic PDFA, 522
power (semi-)automaton, 154
power set, 28
prefix, 57
prefix code, 424
  maximal, 478
preimage, *see* inverse image
preorder, *see* quasiorder
prime number, 28
prime number theorem, 28
primitive automaton, 132
primitive permutation group, 112
principal ideal, 462
product (semi-)automaton, 154
product in semigroup, 40
product on subsets, 41
projection morphism, 57
proper factor, 57

proper prefix, 57
proper subset, 28
proper suffix, 57

quasiorder, 46
quasipermutation, 91

RAM, *see* random access machine
random access machine, 64
rank
  of a function, 36
  of a word, 132
rank factorization, 413
rational expression, 58
reachability (quasi-)order, *see* reachability order
reachability equivalence, 134
reachability order
  automaton, 134
  PDFA, 418
reachable subset
  in automaton, 262
  in transformation semigroup, 288
real numbers, 4
recognized language, *see* accepted language
Rees congruence, 150
reflexive relation, 39
register, 64
register machine, *see* random access machine
regular constraint language, *see* constraint language
regular element, 314
regular expression, 58
regular language, 159
regular permutation group, 112
relation, 39
reset automaton, 190
reset complexity, 16, 302
reset length, 10
reset word, *see* synchronizing word
resizing problem, 252
restriction of function to subset, 30
restriction on automaton, 148
returning automaton, 523
right action, 81
right coset, 42
right ideal language, 57
right regular (Cayley) representation, 123
right-distributivity, 41
road coloring theorem, 10
root, *see* rooted tree
root point, 38
rooted tree, 427
row index, 44
row of matrix, 44
Rystsov graph, 309

satisfiability problem, *see* SAT
satisfying assignment, 70
saturating a subset, 39
section, 28
section-regular partition, 364
self-loop, 131
semiautomaton, *see* deterministic and complete finite semiautomaton
semiconnected graph, *see* unilatterally connected graph
semigroup, 40
semigroup language, 446
semiregular permutation group, 112
semiring, 41
sequential transducer, 169
set builder notation, 3
set difference, 28
set membership, 3
set of duplicate states
  of a function, 30
set of excluded states
  of a function, 30
set of factors, 57
set of fixed points, 38
set of moved points, 38
set of prefixes, 57
set of suffixes, 57
set of synchronizing words, 4
SI-automaton, *see* automaton with simple idempotents
simple automata, 168
simple idempotent, 94

simple idempotent letter/word, 132
simple subgroup, 43
singleton, *see* 1-set
sink component, 135
sink state, *see* trap state
size of a circuit, 66
Skolem problem, 414
slender language, 453
slowly synchronizing automaton, 397
slowly synchronizing family of automata, 397
socle of a group, 395
solid code, 424
solvable automaton, 210
source alphabet, 427
source vertex, 59
sparse language, 447
stability relation, 183
standardized automaton, 343
start state
  of a PDFA, 418
  of an automaton, 159
starting vertex, 59
state complexity, 164
state diagram, 4
state set
  of a PDFA, 418
  of an automaton, 4
strictly bounded language, 441
strong connectivity $\kappa(G)$, 126
strongly $\frac{3}{2}$-connected digraph, 60
strongly $\{a, b\}$-primitive, 373
strongly k-connected graph, 60
strongly connected
  digraph, 60
  in automaton, 134
strongly connected automaton, 135
strongly connected component
  automaton, 134
  in digraph, 60
strongly regular codes, 438
strongly self-synchronizing code, 501
strongly transitive automaton, 323
strongly trap-connected automaton, 136
subautomaton, 148
subgraph, 59
subgroup, 42
submonoid, 42
subsemigroup, 41
subset, 28
suffix, 57
suffix code, 424
surjective, 30
symbol, 3
symmetric digraph, 60
symmetric group, 111
symmetric relation, 39
sync-maximal automaton, 293
synchronization diversity k, 252
synchronizing
  a non-permutation, 357
  group, 364
synchronizing automaton, 4
synchronizing prefix code, 451
synchronizing state, 4
synchronizing word, 4
system of blocks, 111

tail point, 38
tail transformation, 94
terminal vertex, 59
ternary automaton, 132
ternary language, 57
thermometer unary, 76
thin language, 453
topological order, *see* topological sorting
topological sorting
  of a DAG, 61
  of a partial order, 46
total function, *see* function
total order, 46
totally extending problem, 252
transformation monoid, 45
  of automaton, 133
transformation of a set, 38
transformation semigroup, 45
  of automaton, 133
transition function
  of a PDFA, 418
  of an automaton, 4

transition matrix, *see* Boolean transition matrix
transitive on subset, 112
transitive permutation group, 112
transitive relation, 39
transposition, 93
transversal, *see* section
trap state, 131
trim, 418
trivial block, 111
trivial congruences, 45
trivial equivalence relations, 40
trivial group, 42
trivial partition, 28
trivial section-regular partitions, 364
tuple, 28
Turing machine
  deterministic, 65
  nondeterministic, 65
Turing reducible
  L-uniform NC^1, 72
Turing transducer, 65
twisted wreath product action, 395

unary automaton, 132
unary language, 57
unilaterally connected
  automaton, 134
  digraph, 60
unilaterally connected automaton, 135
union, 28
unique factorization property, *see* code
universal mapping property, 55
universal quantifier, 27
universal relation, 40
upper set
  for quasiorder, 46
  in automaton, 134
  quasiorder, 46

variety of group languages, 222
vertex of a graph, 59
vertex-transitive graph, 126

walk in a graph, 60
weakly k-connected digraph, 60
weakly connected component, 60
well-ordering principle, 27
word, 3
wreath product construction, 368

zero matrix, 44